

The blueprint of the leanVM proof system in leanerVM

A review of its faithfulness to leanVM, its non-vacuity, and its audit surface

Review session of 29 and 30 September 2026

leanerVM main at b435631 (upgraded mid-review to 144c5aa) leanVM pinned at a386121f

Summary

What was reviewed. The blueprint of the leanVM proof system in leanerVM (`docs/roadmap/protocol-blueprint.md`) and the code built from it on `main` at `b435631`: the spine (the abstract instance, the relation, the seams, the composition and the two master theorems), Layer 0, Layer 1 and the public-input phase. The ground truth is leanVM at the pin `a386121f`: the specification's tex, the Rust verifier (the deployed protocol) and the Python verifier. Mid-review the owner upgraded `main` to Lean 4.34.1 with new library pins; every statement here is about `b435631`, and section 2.4 says what the upgrade changes (the proof-system statements are the same; ArkLib's pin moved to what the review had examined as upstream; a numeral in $\mathbb{K}$ now means something else).

How. Thirteen dossiers, each by an agent with its own context, on the ground truth phase by phase, on the code, on the libraries, on the boundary with leanISA, on the documentation and on the literature; every citation of the four ground-truth dossiers checked against the sources by a second agent (1,273 rows covering about 1,360 references: 13 wrong lines, 3 misquoted, 33 not supported, 4 unchecked; no conclusion reversed); about fifty Lean and Python probes, of which the twenty-two that carry a conclusion were re-run at the new pins; a hostile second reading of eleven major findings that do not concern the opening, which confirmed nine and weakened two (section 8.2); one register of 138 findings, 115 negative results and 58 items not verified.

The verdict on the three criteria.

Faithfulness. The blueprint is faithful to leanVM in the bus phase's checks, in the table `sumcheck`'s numbers, in the relation's five clauses and in Layer 1's transcriptions, which the review confirmed line by line. It is not faithful in the opening (it runs a `sumcheck` leanVM does not run, because it gives the stack an evaluation oracle where leanVM commits to an inner-product scheme), in the GKR (a round message of five coefficients where the wire carries four, a layer check with an equality factor the deployed one lacks, one combiner too few, and a generic component that cannot be appended), in the Flock interface (it drops the eighteen limb claims and attributes to the protocol a defect of the Rust prover), in the public-input phase as built (it proves the specification's check, which none of the three executable verifiers runs), and in the compilation, whose theorems are not well formed and whose error lacks three terms.

Non-vacuity. The relation and the seams are inhabited and refuted as they should be, and every extractor computes. But the knowledge-soundness master theorem has content only with a bound on the error, and the spine states none: phases that check nothing, at declared error one, satisfy its hypotheses for every instance. The check on the public-input message is not load-bearing for any theorem, because the verifier pools what it computes rather than what it received. And the master theorems constrain the leanISA instance by nothing until the adaptor is built, so a weakened instance (two public lines instead of three, a trivial Flock predicate, an aliasing layout) is caught by no theorem on `main`.

Auditability. The named knowledge-soundness theorem unfolds to 100 declarations and 354 lines of statements; twelve proposals bring it to about 80 and 290 (chapter 10); the first two change what the theorem and the bus seam say, for the better, and the others change no statement's meaning. The trusted base today is Lean's kernel with its three axioms, four libraries' definitions, four assumed interfaces none of which is inhabited, and a leanISA instance that does not yet exist (chapter 7).

The findings that matter most (the register's numbers in parentheses).

1. The declared error is unconstrained; the soundness theorem says nothing until the spine fixes the error in closed form (R5; machine-checked).
2. The opening phase runs a sumcheck leanVM does not run; an inner-product oracle repairs it and makes WHIR the whole opening (R18; the interface typechecks at both pins).
3. The bus seam admits statements the deployed table sumcheck cannot serve (R2), and the sumcheck's tables are not distinguished from the instance's other tables (R11).
4. The public-input phase proves a check no executable verifier runs (R12), and the check it proves is not load-bearing (R13); pooling the values sent repairs both, and the deployed check's key lemma is proved.
5. The GKR is specified with the wrong wire and the wrong layer check, its per-round error is loose, it draws one combiner too few, and its generic component cannot be appended in a one-oracle phase (R7, R9, R10; the first weakened by the second reading: the blueprint's $5/|E|$ is a valid, loose bound and its "check on the cofactor" is right).
6. The Flock interface drops the limb claims, is circular with the instance, cannot be written over an abstract instance, and files the Rust prover's defect under a soundness error (R14 to R17).
7. The two target theorems are unprovable as sketched: one omits the program's well-formedness and attaches a probability to a Boolean; the other is false without a resource bound (R27 to R29). The adaptor's instance is not what the blueprint says it is, and the boundary blocks have no bridge lemma (R30, R31).
8. The non-interactive error omits grinding, the list-size factor and the hash term; the assumed Fiat-Shamir and Merkle interfaces are false as stated or inhabited by nothing, and are about other constructions than leanVM's one BLAKE2s map (R20 to R26).
9. ArkLib's existential notion of round-by-round knowledge soundness carries no knowledge content (R1, critical as a warning about the library, not a defect of a leanerVM theorem); the spine's named form is adequate with the extractor read.

The status file's finding that the Python verifier omits the caps is false at the pin and must not be reported upstream (R38). Thirteen matters are to be reported to leanVM, consolidated in section 11.4: among them, the public-input check differs between the specification and its three verifiers; the Rust Flock prover fails at one challenge value; several constants and layouts of Flock, and four details of the opening's schedule, exist only in the code.

The owner's question. If the verifier did not check, through some claim, that the top lane of both public words is zero, would the theorems catch it? Not the master theorems: the third public line is data of the instance, and every theorem of the spine and of the phase holds of an instance with two lines. The adaptor's soundness theorem would, since `SatisfiedBy` fixes the memory cells to the public words whose top limb is zero by type, and behind it the constraint-soundness theorem; neither is built. The general rule the review draws is that a check is load-bearing only if the verifier commits to what it checked, and that every check of a phase must come with the theorem that refutes its omission (chapter 14).

What the report proposes. Ten recommendations in priority order (section 11.1); the proposed change for every finding, as the passage stands and as proposed (appendix A); one owner per kind of fact in the documentation, with the status and the tracker as pointers to the blueprint, names in place of the letter codes and one index for those that stay (chapter 13, appendix D, appendix C); and the mechanisms that keep the Lean verifier tied to the deployed one as the work proceeds and the pin moves (chapter 14). Nothing has been applied: the blueprint, the status and the trackers are unchanged, and the drafts wait for the owner's go-ahead.

Contents

Summary	1
I The map	7
1 Scope, vocabulary and method	8
1.1 What was reviewed, and against what	8
1.2 The three criteria	8
1.3 Vocabulary	9
1.4 Method	10
1.5 What this report is not	10
2 The proof system, as deployed and planned	11
2.1 Two fields, one committed column	11
2.2 The protocol at the pin, phase by phase	11
2.3 What the blueprint makes of it	14
2.4 The reviewed revision and the one after it	14
3 The libraries, from scratch	17
3.1 VCVio: oracle computations, and what a probability means	17
3.2 CompPoly: tables, polynomials and the two fields	22
3.3 Clean: circuits, tables, buses, and why its balance is unsatisfiable over $\mathbb{K}$	30
3.4 Mathlib and Lean itself: what the statements unfold to, and the trusted base	35
3.5 ArkLib: the objects, from scratch	38
3.6 ArkLib: what round-by-round knowledge soundness guarantees	49
3.7 ArkLib: what is proved and admitted, at the old pin and at the new	59
3.8 Layer 0 of the proof system, audited	67
4 Catalogue of the built code	74
4.1 The spine, declaration by declaration	74
4.2 The public-input phase, declaration by declaration	92
4.3 Layer 1, declaration by declaration	97
4.4 The spine: do the statements say what they claim?	105

II	The analysis	111
5	The phases: what each proves, and how	112
5.1	The commit phase	112
5.2	The bus phase	112
5.3	The table sumcheck	113
5.4	The public-input phase	114
5.5	BLAKE2s validity and ring switching	115
5.6	The opening and the compilation	115
6	The proof obligations, as a hierarchy	117
6.1	The shape of the tree	117
6.2	The tree	119
6.3	The error, composed	138
6.4	The two existing outlines, validated	142
6.5	Findings from the tree	146
7	The trusted base	149
7.1	Lean and its three axioms	149
7.2	The libraries' definitions	149
7.3	Theorems the libraries admit, on which the chain will rest	149
7.4	The interfaces the blueprint assumes	150
7.5	The instance for leanISA	150
7.6	Hypotheses of the chain, and who discharges each	150
7.7	Outside Lean	151
7.8	What the blueprint says of its trusted surface	151
III	The audit	152
8	Faithfulness	153
8.1	How to read the tables	153
8.2	The major findings, re-derived	153
8.3	The spine	154
8.4	The setup, the commitment and the bus	154
8.5	The table sumcheck and the public input	165
8.6	BLAKE2s validity and ring switching	178
8.7	The opening and the compilation	191
8.8	The adaptor and the boundary with leanISA	221
8.9	The code as built: the spine, Layer 1, the public-input phase	251
8.10	The libraries and Layer 0	272

9 Non-vacuity	279
9.1 The relation and the seams	279
9.2 The hypotheses of the master theorems	279
9.3 The instance	280
9.4 The error bound	281
9.5 The extractor	281
9.6 The public-input phase	281
9.7 The notion itself	282
9.8 The blueprint’s acceptance tests	282
9.9 What was not established	282
10 Auditability	283
10.1 What is load-bearing	283
10.2 What is not, and whether it leaks	284
10.3 Proposals to reduce the surface	284
10.4 The measured tables	285
10.5 The audit surface, measured	285
IV What to do	293
11 Options, limitations and boundaries	294
11.1 Recommendations, in priority order	294
11.2 Exactly as deployed, or easier to audit?	295
11.3 What cannot be reproduced today	295
11.4 What to report to leanVM	296
11.5 The boundaries	296
12 ArkLib: exists, pending, ours	297
13 The documentation: one source of truth	298
13.1 Which document says what	298
13.2 Contradictions and stale statements	301
13.3 Vocabulary	309
13.4 The letter codes	311
13.5 The proposal	314
13.6 Findings on the documentation	323
14 Keeping the Lean verifier from drifting	332
14.1 Where a weakening can hide	332
14.2 The public-input phase, as the test case	333
14.3 The checks of the deployed verifier, one by one	334
14.4 What keeps it true	348

V	Appendices	350
A	Proposed changes to the blueprint	351
A.1	Changes to the blueprint	351
A.2	Changes to the code	372
A.3	Changes that depend on a decision of the owner	376
B	The probes	379
B.1	The spine, the public-input phase, Layer 1	379
B.2	The opening and the compilation	431
B.3	The boundary with leanISA	440
B.4	Re-runs at the new pins	454
C	Index of the letter codes	456
D	Proposed texts for the documents and the tracker	466
D.1	The blueprint’s section “How work is tracked”	466
D.2	The blueprint’s table “The holes”, its heading “The build”, and the two sentences that send the reader to the hole comment	468
D.3	The status file’s head and “At a glance”, for <code>main</code> at <code>144c5aa</code>	471
D.4	The body of issue 12, and where the hole comment goes	474
D.5	The other issues and pull requests, as far as the proof system’s documentation goes	480
E	The brief, reviewed	483
E.1	What the brief settles well	483
E.2	Choices the brief leaves open, and the choice made	483
E.3	What happened that the brief did not foresee	484
E.4	Scope the brief does not name, which the review covered	484
E.5	What a future run of the brief would gain	484
F	The register of findings	485
F.1	The register of findings	486
F.2	Negative results	508
F.3	What was not verified	516
F.4	Counts, disagreements and merged rows	519
	References	522

Part I

The map

Chapter 1

Scope, vocabulary and method

1.1 What was reviewed, and against what

The object of the review is the *blueprint* of the leanVM proof system in leanerVM, the document `docs/roadmap/protocol-blueprint.md`, together with the code already built from it. leanerVM was reviewed on its branch `main` at commit `b435631` (local and remote agree). Table 1.1 lists every revision a statement of this report can be about. A statement about a library is about its *pin*, the revision leanerVM builds against, unless it says “upstream”.

What	Revision	Where it was read	Role
leanerVM	b435631 (main)	the working directory	the object of the review
leanVM specification	a386121f	doc/leanvm/body/*.tex	ground truth: the intended protocol
leanVM Rust verifier	a386121f	crates/	ground truth: the deployed protocol
leanVM Python verifier	a386121f	python-verifier/verifier.py	ground truth: second implementation
ArkLib	dca90385	.lake/packages/ArkLib	the framework of the statements
VCVio	f9dc47d9	.lake/packages/VCVio	probabilistic oracle computations
CompPoly	3468b38c	.lake/packages/CompPoly	fields and computable polynomials
Clean	93c9d1ef	.lake/packages/Clean	the circuit language of the arithmetization
Mathlib, Lean	v4.33.1	.lake/packages/mathlib	mathematics, and the kernel

Table 1.1: The revisions this report speaks about.

Three sources define leanVM at the pin, and they do not always agree. This report uses two words for them and keeps them apart:

- the **specification** is the tex source at the pin: what leanVM is meant to be;
- the **deployed** protocol is what the Rust verifier at the pin accepts: what would run on Ethereum. The Python verifier is a second implementation of it.

A disagreement between them is a finding in its own right (chapter 8), and every theorem discussed is labelled with the verifier it describes. A PDF of the specification circulates (`leanVM-b-2.pdf`); it was built from an earlier revision, differs from the pinned text, and is never cited here.

1.2 The three criteria

Faithfulness. The blueprint models leanVM’s proof system interaction by interaction: every prover message, every verifier challenge and every verifier check, in leanVM’s order. A divergence is classified as one of: *an error of the blueprint*, to be fixed; *a deliberate deviation*, to be kept with its reason recorded and with the theorem that relates it to the deployed verifier named and owed; or *a deviation forced by an upstream library*, recorded with its workaround and the condition under which it is retired.

Non-vacuity. A definition is inhabited by the object leanVM intends and refuted by the near misses; a hypothesis can be met, and by the real protocol, not only by a trivial one; an error bound means

something at leanVM’s parameters; an extractor computes. A statement that typechecks and a proof that compiles are evidence of neither.

Auditability. The declarations a reader has to read and trust in order to know what the master theorems say are as few and as short as they can be.

Faithfulness and non-vacuity of the relation, the seams and the master theorems come first; the size of the audit surface comes second and can be improved iteratively once the statements are right.

1.3 Vocabulary

The repository’s documents use several names for overlapping things. This report uses one name for each, as follows; chapter 13 proposes that the documents do the same.

Term	Meaning
blueprint	<code>docs/roadmap/protocol-blueprint.md</code> , the specification of what is wanted for the proof system. It titles itself “Roadmap”; the two are one document.
status	<code>docs/roadmap/protocol-status.md</code> .
tracker	issue #12 of the repository, with the long comment that holds the specifications of the holes (the <i>hole comment</i>).
oracle protocol	the interactive protocol in the ideal model where the committed polynomial is an oracle the verifier may query and challenges are uniformly random. In the literature: a polynomial interactive oracle proof.
phase	one step of the oracle protocol: commit, bus, table sumcheck, public input, Flock, opening.
spine	the part of the blueprint, and of the code, that fixes what the phases share: the abstract instance <code>M3Instance</code> , the relation <code>M3Holds</code> , the seam relations, the interfaces a phase has to meet, the composition and the two master theorems.
seam	the relation that holds between two consecutive phases: the output relation of one and, by definition, the input relation of the next.
hole	a hole the spine leaves open (a phase, a generic component, the adaptor, the compilation).
layer	the blueprint’s numbering of the same work (Layers 0 to 13).
adaptor	the part, not yet built, that instantiates the abstract instance at leanISA and carries the proof system’s relation to leanISA’s constraint relation <code>SatisfiedBy</code> .
compilation	what turns the oracle protocol into a non-interactive proof: the polynomial commitment (WHIR over Merkle trees) in place of the oracle, and Fiat–Shamir in place of the verifier’s coins.
master theorems	<code>piop_perfectCompleteness</code> and <code>piop_rbrKnowledgeSoundness</code> , and the theorems the blueprint builds on them, not yet proved: <code>verify_knowledgeSound</code> for the compiled verifier, and <code>baseVerifier_exttractsExecution</code> with <code>baseProver_complete</code> , the proof system’s target theorem.
pin	the revision of a dependency recorded in <code>upstreams.json</code> .
load-bearing	said of a declaration that a master theorem’s statement unfolds to, so that a reader must read it to know what the theorem says.
Category A, Category B	the blueprint’s two kinds of content: A is what the verifier accepts and why it is sound, written from the specification and then diffed against the code; B is transcribed data (layouts, encodings, constants, parameters), copied from its source and wrong if it deviates.
Layer N , acceptance test N , decision N	the blueprint’s own numbering of its sections, its tests and its decisions; the report keeps these numbers, since they are the document’s structure and not codes for other things.

Letter codes. The repository’s documents label holes, ledger entries, findings and decisions with a letter and a number. This report names things in words. Where a code helps to find the thing in the repository it is given once, in parentheses, after the name. Appendix C is the index of every code met.

The report has one code family of its own, the register’s row numbers R1 to R140 (appendix F), used only to point at a finding beside its name.

1.4 Method

The review was run by one orchestrating session and independent sub-agents, each with its own context and a self-contained task, all bound by the limits of the session: no edit to a tracked file, nothing posted to an issue tracker, no checkout moved.

1. **Ground truth first.** For each part of the protocol a sub-agent read the specification, the Rust and the Python verifier at the pin and wrote down the transcript (who sends what, in which order, which challenge follows which message), every check of the verifier with its rejection path, and the errors claimed. Only then was the blueprint set against it.
2. **The code as evidence.** The built code (the spine, Layers 0 and 1, the public-input phase) was catalogued declaration by declaration from the source, classified as load-bearing, interface, helper or test fixture, and read adversarially against its docstrings and the blueprint.
3. **Probes.** Claims of non-vacuity were tested by Lean files that build against `main`: inhabitants, near misses, and mutations of a verifier that show where a proof stops when a check is removed. The probes are scratch work and are reproduced in appendix B.
4. **Validation of findings and of their absence.** Every finding carries its evidence: the line of the pinned source, the Lean declaration, and the probe or counterexample that reproduces it. Where a part was found to have no issue, the report says what was checked. Every citation of the four ground-truth dossiers was checked against the sources by a second agent, and eleven of the thirty-seven major findings (those not concerning the opening, whose dossier landed last) were re-derived by a hostile second reader before this report was written; the others rest on their dossier’s evidence and the citation checks.

Severity. **CRITICAL:** a theorem or definition is wrong, vacuous or unfaithful in a way that would let an unsound verifier be “proved”. **MAJOR:** the blueprint specifies something leanVM does not do, or omits something it does, or a statement cannot be proved or stated as written. **MINOR:** imprecision, stale text, avoidable audit surface. **NOTE:** worth recording.

1.5 What this report is not

It is the record of a review, not a further source of truth about the proof system. Whatever it concludes should change is written as a proposed change to the blueprint (appendix A), with the status and the tracker following, so that the blueprint remains the one place that says what is wanted. Nothing proposed here has been applied.

Chapter 2

The proof system, as deployed and as planned

This chapter is the map. It says what leanVM’s proof system does at the pin, interaction by interaction, where the blueprint puts each part, and where the code that exists sits. The detailed tables of the transcript, of the verifier’s checks and of the disagreements between the sources, one set per phase, are in chapter 8; this chapter gives the shape.

2.1 Two fields, one committed column

Everything the prover commits to is a table of values in the field

The fields $\mathbb{K}$ and $\mathbb{E}$

$\mathbb{K} = \text{GF}(2^{64})$ is the field of the columns: every cell of every table of the arithmetization is an element of $\mathbb{K}$, represented as 64 bits. $\mathbb{E} = \text{GF}(2^{192})$ is the field of the challenges: $\mathbb{E} = \mathbb{K}[y]/(y^3 + y + 1)$, so an element of $\mathbb{E}$ is three *limbs* in $\mathbb{K}$, and $\mathbb{K}$ sits inside $\mathbb{E}$ as the elements whose upper two limbs are zero. Both have characteristic 2: $x + x = 0$, so subtraction is addition and $-1 = 1$. In Lean, CompPoly’s **BF64** and **BF64.Ext3**; the review confirmed that their defining polynomials, bit encodings and limb order are those of the Rust and of the specification (chapter 3).

The prover’s whole witness is one column: every table of the arithmetization (the six opcode tables, the three memory limbs, the two finalize counts, and the packed bit witness of the BLAKE2s circuit) is stacked into a single table q of 2^μ cells of $\mathbb{K}$, with $15 \leq \mu \leq 28$. The verifier never sees q ; it holds a commitment to it and, at the very end, asks the commitment scheme to certify weighted sums $\sum_w W(w) \tilde{q}(w)$ over the cube, where $\tilde{q}$ is the multilinear extension of q and W is a weight the verifier can evaluate itself. An evaluation claim $\tilde{q}(r) = v$ at a point $r \in \mathbb{E}^\mu$ is the case $W = \text{eq}(r, \cdot)$.

2.2 The protocol at the pin, phase by phase

Figure 2.2 shows the six phases the blueprint distinguishes, with what each phase adds to the *claim pool*: the list of evaluation claims on columns of q that the opening discharges at the end. The counts below are those of the deployed verifier at the pin (`crates/lean_vm/src/cpu/mod.rs:711-779` is the top-level flow); each is cited in chapter 8.

Setup (before any phase). The Fiat–Shamir chain is seeded with a digest of the program and of the Flock circuit’s identifier, then with the two public words. The prover announces eight sizes: the memory log-size κ_{mem} , the six tables’ log-heights $\tau_0, \dots, \tau_5$, and the rate exponent ρ of the commitment scheme. The verifier checks that each is canonically encoded (upper limbs zero), that each public word has a zero top limb, the caps ($2^{\kappa_{\text{bc}}}$ instructions with $\kappa_{\text{bc}} \leq 32$; $16 \leq \kappa_{\text{mem}} \leq 32$; $\tau_j \leq 32$; $\tau_{\text{BLAKE2S}} \geq 3$;

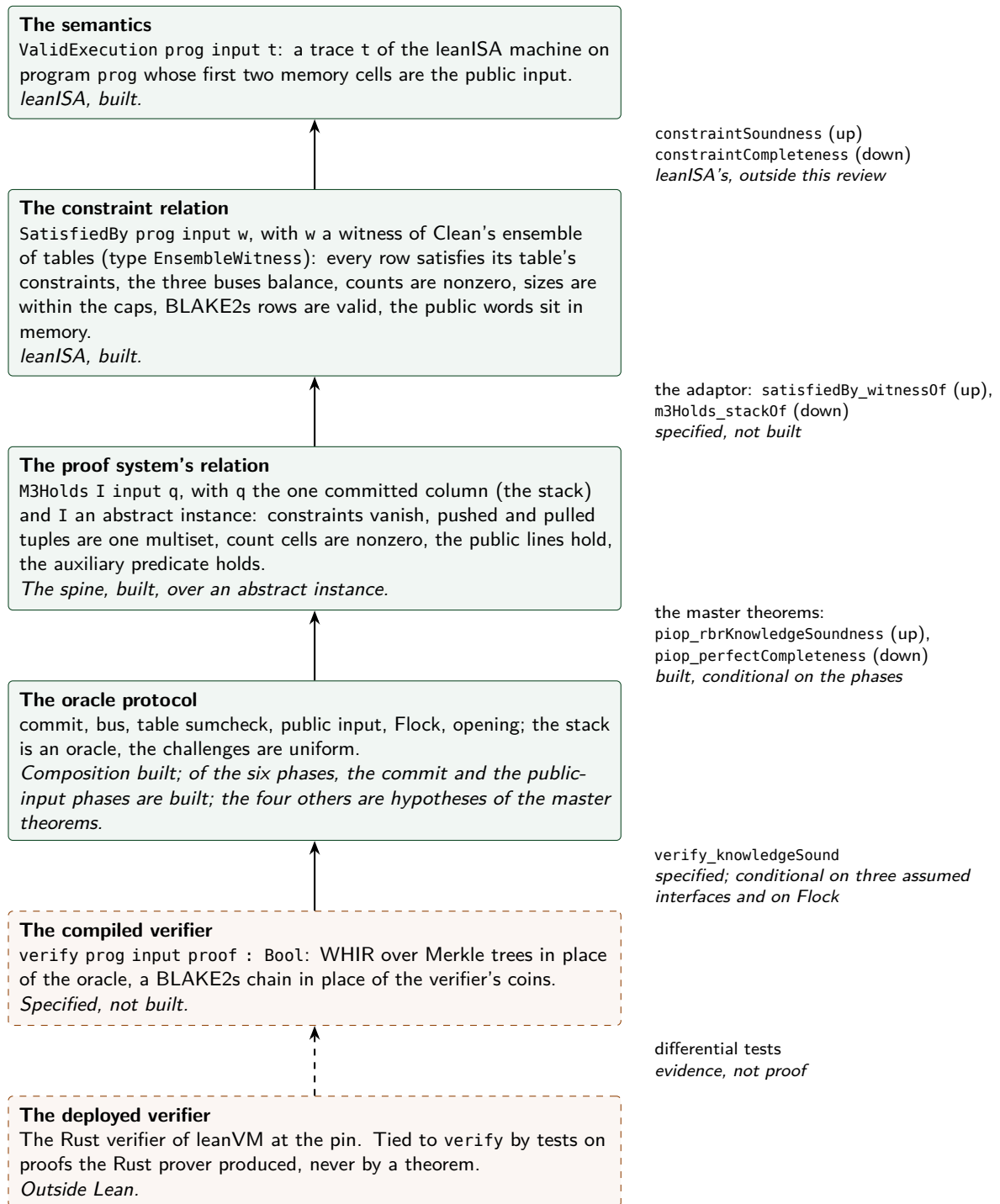


Figure 2.1: The chain from the deployed verifier to the semantics. An arrow pointing up is a soundness direction (what is below implies what is above, up to an error where the step is probabilistic); each step also has a completeness direction, pointing down. Solid boxes are built on `main`; dashed boxes are specified in the blueprint and not built.

$1 \leq \rho \leq 4$), derives every layout, and checks the stacked size $15 \leq \mu \leq 28$. None of this is a phase of the oracle protocol; all of it is a check of the deployed verifier, and chapter 14 counts it.

Commit. The prover sends the commitment to q : a Merkle root, two field elements with zero top limbs. In the oracle protocol this is the one oracle message: from here on the verifier may query $\tilde{q}$ at points of $\mathbb{E}^\mu$ and nothing else.

Bus. The bus argument shows that the tuples every table *pushes* onto the three buses (state, memory, bytecode) are, as a multiset, the tuples that are *pulled*, and that no read count is zero. The verifier draws the fingerprint challenge $\alpha \in \mathbb{E}^4$ and the multiset challenge $\beta \in \mathbb{E}$; each 16-coordinate tuple t becomes the scalar $\beta + \sum_i \text{eq}(\alpha, i) t_i$; the pushed scalars, the pulled scalars and the count cells are each stacked into a tree of $2^{\mu_{\text{bus}}}$ leaves (padded with 1), and the prover sends two roots: R , claimed to be the product of the pushed leaves *and* of the pulled leaves, one scalar for both sides, and R_c , the product of the counts. A batched GKR grand-product argument then walks the three trees from the roots to the leaves, radix 4 (a binary first layer when μ_{bus} is odd): per layer, a combiner λ batches the three trees; a sumcheck of $\mu_{\text{bus}} - \ell$ rounds, each round a cofactor of degree 4 sent as four coefficients; the twelve children; one layer check; two combination challenges. A fresh combiner is drawn after every layer, the last one included, where it is used by nothing. The GKR ends with three claims on the leaf tables at one point ζ assembled from the last layer's challenges. The verifier checks $R_c \neq 0$, computes itself the public part of each leaf claim (the selectors, the constants, the index column, the program's bytecode columns), reads five boundary evaluations the prover sends (the three memory limbs and the memory finalize count at $\zeta_{<\kappa_{\text{mem}}}$, the bytecode finalize count at $\zeta_{<\kappa_{\text{bc}}}$), and derives what the tables owe each side: three values rem_s . Into the pool: five column claims.

Table sumcheck. One sumcheck proves the two constraints of the JUMP table and the tables' share of the three bus sides together. The verifier draws $\xi \in \mathbb{E}$; the constraints take ξ^0, ξ^1 and the three sides ξ^2, ξ^3, ξ^4 , shared by every table; the target is $\sum_s \xi^{2+s} \text{rem}_s$, derived, never sent. The sumcheck runs τ_{max} rounds, the maximum log-height over the six opcode tables, binding the highest variable first, each table joining when its height is reached; a round message is three coefficients of a cubic, the fourth derived from the running claim, so that no round check can fail. The prover then sends one value per column of every opcode table, 104 in all (the eighteen value limbs of the BLAKE2S table included), the verifier recomputes the summand from them and checks it against the running claim. Into the pool: 104 column claims, each at the prefix $\rho_{<\tau_j}$ of the sumcheck's point; the eighteen limb claims are claims on strided slots of the packed bit witness.

Public input. The verifier draws $r \in \mathbb{E}$; the prover sends c_0, c_1 , claimed to be the values of the two low memory limbs at $(r, 0, \dots, 0)$, the line through cells 0 and 1. The specification checks each against the public words' limb, $c_\ell = (1+r)w_{0,\ell} + rw_{1,\ell}$; the three executable verifiers (Rust, Python, the recursion guest) check one combined equation $c_0 + y c_1 = (1+r)w_0 + r w_1$ in $\mathbb{E}$. Into the pool: three claims, on the three memory limbs at $(r, 0, \dots, 0)$, with the values sent and 0 for the top limb, for which nothing is sent.

BLAKE2s validity (Flock) and ring switching. Flock proves that the bits packed into the region q_{flock} of q satisfy the R1CS of the BLAKE2s compression, block by block, with position 512 of every block equal to 1. It commits nothing of its own and reads no claim of the pool: it is a zerocheck with a univariate skip ($k+1$ equality coordinates drawn first, seven more being fixed constants; 64 values; one challenge; $8+k$ rounds of two coefficients; two final values) followed by a lincheck (one batching challenge, 8 rounds, then 64 values $s_0, \dots, s_{63}$), with a single check that can reject, the lincheck's terminal identity. Ring switching then turns the 64 claims on the bit polynomials into one weighted claim on the q_{flock} region of q , with six challenges and no message. Into the pool: one weighted claim.

Opening. The verifier draws $\lambda \in \mathbb{E}$; the ring-switched claim takes λ^0 and the 112 pooled point claims the next powers, each point claim first turned into a weighted claim through the stack's selectors; the batch is one weighted claim $\sum_w W_\lambda(w) \tilde{q}(w) = C_\lambda$, and the commitment scheme (WHIR over Reed–Solomon codes of $\mathbb{K}$ in the novel polynomial basis, with Merkle trees over BLAKE2s) opens it. The specification is explicit: “This one opening discharges every pooled claim; there is no separate reduction sumcheck”

(doc/leanvm/body/08-end-to-end-protocol.tex:99). The verifier finally checks that the proof stream has been consumed entirely.

2.3 What the blueprint makes of it

Figure 2.1 shows the chain of relations and theorems the blueprint builds, from the deployed verifier up to the semantics, with what is built and what is specified.

The blueprint models the protocol as a sequential composition of ArkLib *oracle reductions*, one per phase, in the ideal model where q is an oracle and the challenges are uniform (chapter 3 introduces the vocabulary). It fixes, in the spine, what the phases share: the abstract instance `M3Instance`, the relation `M3Holds`, the seam relations, the interfaces `Phase.Def`, `Phase.Complete` and `Phase.Security`, and the composition with its two master theorems. It then specifies the generic components and each phase (Layers 4 to 10), the adaptor to leanISA (Layers 2 and 3), the compilation (Layers 11 and 12) and the target theorem (Layer 13). Table 2.1 says where each part is.

Part	Blueprint	Code on main at b435631
Fields, sampling, the oracle interface of q	Layer 0	<code>LeanerVM/Protocol/Field.lean</code> , <code>ToArkLib/Oracles.lean</code> ; built
Hypercube tables, stacking, padding, the index and byte-code columns	Layer 1	<code>ToCompPoly/*</code> , <code>Stack.lean</code> , <code>Padding.lean</code> , <code>ClaimWeights.lean</code> , <code>BlockClaims.lean</code> , <code>FixedColumns.lean</code> ; built
The abstract instance, the relation, the seams, the interfaces, the composition, the master theorems	the section “The spine”	<code>Spine/</code> : <code>Instance</code> , <code>Seams</code> , <code>Phase</code> , <code>Compose</code> , <code>Toy</code> ; <code>ToArkLib/</code> : <code>Component</code> , <code>KnowledgeAppend</code> , <code>PassThrough</code> , <code>SendOracle</code> , <code>Refinement</code> , <code>GuardedVerdict</code> , <code>KeepOracles</code> ; built
Clean components as polynomials	Layer 2	not built
The leanISA instance and the adaptor	Layer 3	not built
The sumcheck and the batching component	Layer 4	not built (an open pull request holds the honest round algebra)
Fingerprints, the grand product, GKR	Layer 5	not built
The bus phase	Layer 6	not built
The table sumcheck	Layer 7	not built
The public-input phase	Layer 8	<code>PublicInput.lean</code> ; built, both halves proved
Flock and ring switching	Layer 9	not built; owned by the Flock roadmap (issue #3), which has not started
The claim pool and the opening	Layer 10	not built
WHIR, Merkle trees, the parameters	Layer 11	not built
Fiat-Shamir, the proof object, <code>verify</code>	Layer 12	not built
The target theorem	Layer 13	not built

Table 2.1: Where each part of the proof system is specified and whether it is built.

Of the six phases, two are built: the commit phase, which is the spine’s, and the public-input phase. The four others, the generic components they rest on, the adaptor and the compilation are specified only. The master theorems are therefore theorems about a *bundle* of phases supplied as a hypothesis; what they say, and what they leave to be trusted, is the subject of chapters 5 and 9.

2.4 The reviewed revision and the one after it

The review’s object is main at b435631. While the review was under way, the owner upgraded main to Lean 4.34.1 (144c5aa, pull request #61) and moved every library pin: ArkLib to 7653a901, CompPoly

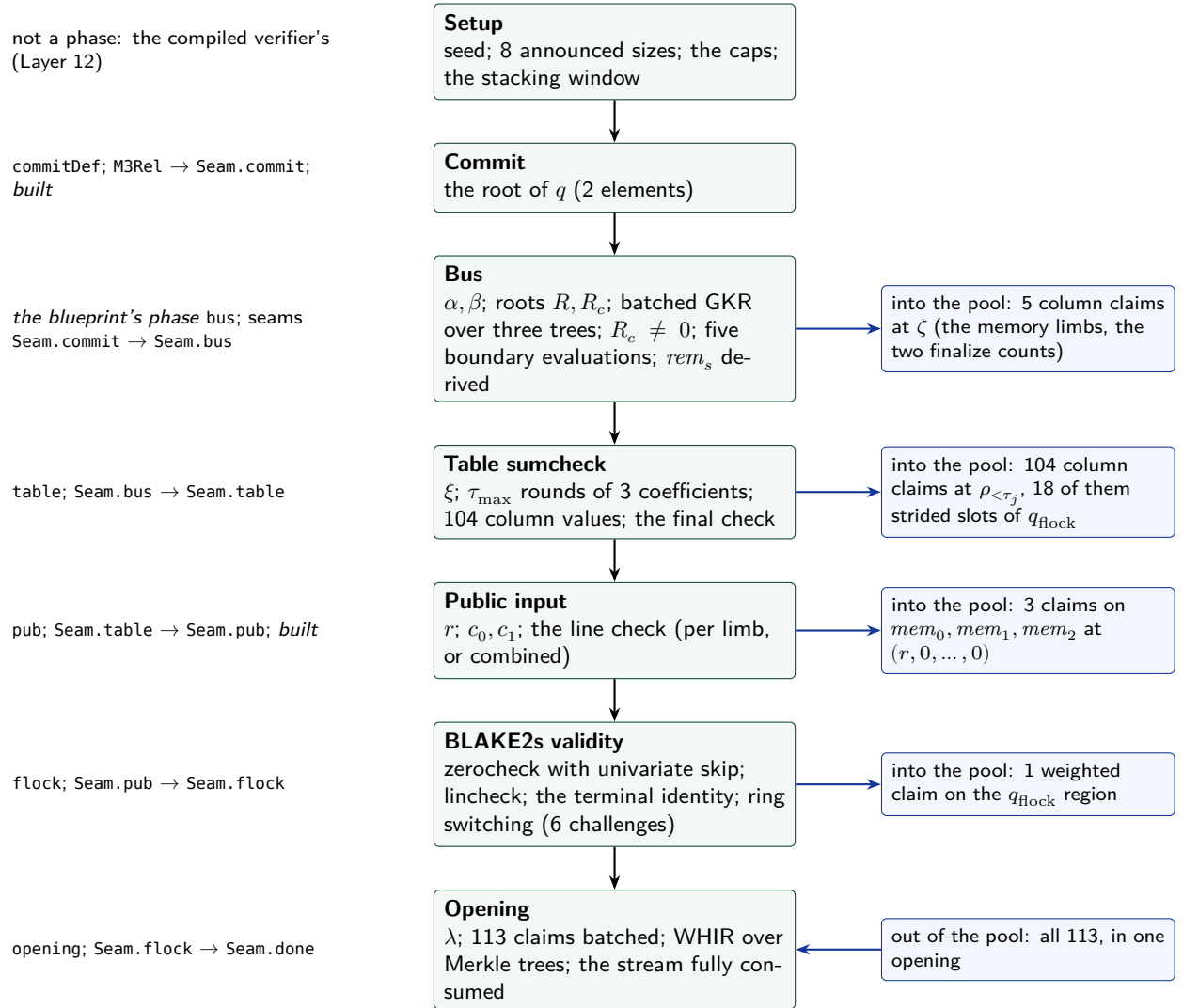


Figure 2.2: leanVM's proof system at the pin, as the deployed verifier runs it (centre), what each phase adds to the claim pool (right), and the blueprint's phase and seams for it (left).

to 572f9973, Clean to 42fe4b26, VCVio to a4232d08. The leanVM pin did not move. The proof-system sources changed in proofs and imports only (77 lines in, 76 out), so every statement this report discusses is the same at both revisions; the two probability bridges of the spine were restated on VCVio’s new notation with the same relations and bounds. Two things did change in substance and are recorded where they matter: ArkLib’s new pin is the revision several dossiers examined as “upstream”, and it lifts none of the admissions the blueprint’s ledger names (chapter 3); and CompPoly’s $\mathbb{K}$ now gives natural-number literals their characteristic-two meaning, $(2 : \mathbb{K}) = 0$, with encoded words written `K.ofBits n`, so that any probe or fixture using a literal other than 0 or 1 in $\mathbb{K}$ means something else at the new pin (appendix B).

Chapter 3

The libraries, from scratch

Every statement of the proof system unfolds to objects of four libraries. This chapter introduces each object before anything is built on it: what it is, what it is for here, its Lean text at the pin, and what a reader trusts in accepting it. The revision is named with each snippet; where the upgrade of `main` to the new pins changed an object, both are given. Readers who know Lean may skim; readers who do not should not skip the sections on `VCVio` and on `ArkLib`'s security notions, since the master theorems are statements in that vocabulary and mean exactly what those definitions say.

3.1 VCVio: oracle computations, and what a probability means

`VCVio` is the library of *oracle computations* on which `ArkLib` states every security notion. Every snippet of this section is from `VCVio` at the old pin `f9dc47d9` (“feat(complexity): add honest oracle-PPT foundations and measure-native OTP (#545)”), read under `.lake/packages/VCVio/`; section 3.1.5 says what the new pin `a4232d08` changes.

3.1.1 Oracles and computations

OracleSpec: the list of oracles a computation may call

An oracle specification names a set of oracles by an index type ι and gives, for each index (each possible query), the type of the answer. It is nothing more than a function to types. $A \rightarrow B$ is the specification of one oracle with queries in A and answers in B ; $[]_o$ is the empty specification (no oracle at all); `unifSpec` is the source of all randomness, the oracle of uniform choices from `Fin (n+1)`.

```
-- An `OracleSpec`  $\iota$  specifies a set of oracles indexed by  $\iota$ .  
Defined as a map from each input to the type of the oracle's output. -/  
def OracleSpec ( $\iota$  : Type u) : Type (max u (v + 1)) :=  
   $\iota \rightarrow$  Type v
```

VCVio/OracleComp/OracleSpec.lean:27-30, VCVio f9dc47d9

```
-- Specifies access to no oracles, using the empty type as the indexing type. -/  
@[reducible] def emptySpec : OracleSpec PEmpty := PEmpty  $\rightarrow$  PEmpty  
notation "[]"  $\Rightarrow$  emptySpec
```

VCVio/OracleComp/OracleSpec.lean:215-217, VCVio f9dc47d9

```
-- Access to oracles for uniformly selecting from  $\text{Fin } (n + 1)$  for arbitrary  $n : \mathbb{N}$ .  
By adding  $1$  to the index we avoid selection from the empty type  $\text{Fin } 0 = \text{empty}$ . -/  
@[inline, reducible] def unifSpec : OracleSpec  $\mathbb{N}$  :=  
  OracleSpec.ofFn fun n  $\Rightarrow$  Fin (n + 1)
```

VCVio/OracleComp/OracleSpec.lean:233-236, VCVio f9dc47d9

(The notation is notation:25 (name := singletonSpec) A:25 " $\rightarrow_0$ " B:26 $\Rightarrow$ OracleSpec.ofFn ($\iota :=$ A) (fun $_ \Rightarrow$ B), OracleSpec.lean:84-85.)

OracleComp and ProbComp: programs that call oracles

OracleComp spec α is a program returning an α that may call the oracles of spec. It is a *free monad*: a syntax tree whose nodes are “query t , then continue with the answer”. It has no meaning by itself; a meaning is supplied by interpreting the queries (section 3.1.2). A *probabilistic* computation, ProbComp, is one whose only oracle is the uniform-choice oracle, with the primitive coin $\$[0..n]$ (def uniformFin ($n : \mathbb{N}$) : ProbComp (Fin ($n + 1$)) := unifSpec.query n, ProbComp.lean:55-58).

```

/-- `OracleComp spec  $\alpha$ ` represents computations with oracle access to oracles in `spec`,
where the final return value has type ` $\alpha$ `, represented as a free monad over the `PFunctor`
corresponding to `spec.` -/
@[reducible]
def OracleComp { $\iota$  : Type u} (spec : OracleSpec.{u, v}  $\iota$ ) :
  Type w  $\rightarrow$  Type (max u v w) :=
  PFunctor.FreeM spec.toPFunctor

```

VCVio/OracleComp/OracleComp.lean:23-29, VCVio f9dc47d9

```

/-- Simplified notation for computations with no oracles besides random inputs.
This specific case can be used with `#eval` to run a random program, see `OracleComp.runIO`.
NOTE: Need to decide if this should be more opaque than `abbrev`, seems like no as of now.. -/
abbrev ProbComp : Type  $\rightarrow$  Type := OracleComp unifSpec

```

VCVio/OracleComp/ProbComp.lean:42-45, VCVio f9dc47d9

Failure. An OracleComp tree has no failure node: it always returns. Failure (a verifier rejecting, a guard not met) is added by Lean’s OptionT monad transformer, OptionT m $\alpha = m$ (Option α), where none means “failed”. ArkLib’s verifiers run in OptionT (OracleComp oSpec). VCVio sends none to the “missing mass” of a sub-distribution (PMF is Mathlib’s type of probability mass functions: a function $\alpha \rightarrow \mathbb{R}_{\geq 0}$ summing to 1):

```

/-- A subprobability mass function is a function ` $\alpha \rightarrow \mathbb{R}_{\geq 0}$ ` such that values have an infinite
sum at most `1` represented by applying an `OptionT` transformer to the `PMF` monad.
The new `failure`/`none` value holds the "missing" mass to reach the total sum of `1`. -/
def SPMF : Type u  $\rightarrow$  Type u := OptionT PMF

```

ToMathlib/ProbabilityTheory/SPMF.lean:55-58, VCVio f9dc47d9

```

/-- Lift a `MonadLiftT m SPMF` instance to `MonadLiftT (OptionT m) SPMF`. Failure in `OptionT`
contributes to the failure mass of the resulting `SPMF`. -/
noncomputable instance instMonadLiftTSPMF (m : Type u  $\rightarrow$  Type v) [Monad m]
  [MonadLiftT m SPMF] [LawfulMonadLiftT m SPMF] :
  MonadLiftT (OptionT m) SPMF where
  monadLift x := OptionT.mapM' (MonadHom.ofLift m SPMF) x

```

VCVio/EvalDist/Instances/OptionT.lean:126-131, VCVio f9dc47d9

QueryImpl and simulateQ: answering the queries

A QueryImpl spec m answers every query of spec by a computation in another monad m; simulateQ impl oa replaces every query node of oa by its answer. This is how ArkLib’s challenger answers “give me challenge i ” by a uniform sample, and how an oracle is implemented from the message it stands for.

```

/-- A monadic handler for the polynomial interface induced by `spec`.

Concretely, this maps every oracle input ` $x$ ` to a computation returning an
answer of type `spec.Range x`. It extends first to `OracleQuery spec` by
applying the continuation, then to `OracleComp spec` by preserving `pure` and
`bind`; see `QueryImpl.mapQuery` and `simulateQ`. -/
@[reducible] def QueryImpl { $\iota$ } (spec : OracleSpec  $\iota$ ) (m : Type u  $\rightarrow$  Type v) :=
  ( $x$  : spec.Domain)  $\rightarrow$  m (spec.Range x)

```

VCVio/OracleComp/SimSemantics/QueryImpl/Basic.lean:30-37, VCVio f9dc47d9

```

/-- Given an implementation of `spec` in the monad `r`, convert an `OracleComp spec α` to a
implementation in `r α` by substituting `impl t` for `query t` throughout. -/
def simulateQ {ι} {spec : OracleSpec ι} {r : Type u → Type _} [Monad r]
  (impl : QueryImpl spec r) {α : Type u} (mx : OracleComp spec α) : r α :=
  PFunc.FreeM.liftM impl mx

```

VCVio/OracleComp/SimSemantics/SimulateQ.lean:27-31, VCVio f9dc47d9

3.1.2 The distribution of a computation, $\text{Pr}[\dots]$, and the support

The distribution of a computation

The distribution of an `OracleComp` is obtained by `simulateQ` with every query answered by a fixed distribution on its answers. For `unifSpec` (hence every `ProbComp`) that distribution is the uniform distribution on `Fin (n+1)`, declared once. The probabilities are then read off that sub-distribution.

```

@[reducible] noncomputable def IsUniformSpec.offFintypeInhabited
  {ι : Type u} (spec : OracleSpec ι)
  [hF : spec.Fintype] [hI : spec.Inhabited] : IsUniformSpec spec where
  toPMF t := PMF.uniformOffFintype (spec.Range t)
  fintype := hF
  inhabited := hI
  toPMF_eq_uniform _ := rfl

noncomputable instance : IsUniformSpec unifSpec := IsUniformSpec.offFintypeInhabited _

```

VCVio/OracleComp/EvalDist.lean:71-80, VCVio f9dc47d9

```

lemma evalSPMF_eq_simulateQ [IsProbabilitySpec spec] (mx : OracleComp spec α) :
  ⌈mx⌋ = simulateQ IsProbabilitySpec.toPMF mx := rfl

```

VCVio/OracleComp/EvalDist.lean:146-147, VCVio f9dc47d9

```

def probOutput [MonadLiftT m SPMF] (mx : m α) (x : α) : ℝ≥0∞ :=
  evalSPMF mx x
...
noncomputable def probEvent [MonadLiftT m SPMF] (mx : m α) (p : α → Prop) : ℝ≥0∞ :=
  (evalSPMF mx).run.toOuterMeasure (some '' {x | p x})

/-- Probability that a computation `mx` will fail to return a value. -/
def probFailure [MonadLiftT m SPMF] (mx : m α) : ℝ≥0∞ :=
  (evalSPMF mx).run none

/-- Probability that a computation returns a particular output. -/
notation "Pr[= " x " | " mx "]" => probOutput mx x

/-- Probability that a computation returns a value satisfying a predicate. -/
macro (name := probEventNotation) "Pr[ " p:term " | " mx:term "]" : term =>
  `(probEvent $mx $p)

/-- Probability that a computation fails to return a value. -/
notation "Pr[⊥ " " | " mx "]" => probFailure mx

```

VCVio/EvalDist/Defs/Basic.lean:114-140, VCVio f9dc47d9

So $\text{Pr}[= x \mid mx]$ is the probability that `mx` returns `x`, $\text{Pr}[p \mid mx]$ the probability that it returns a value satisfying `p` (failure never satisfies an event), $\text{Pr}[\perp \mid mx]$ the probability that it fails; all are extended non-negative reals ($\mathbb{R}_{\geq 0\infty}$). The **support** is the set of values a computation can return, computed syntactically (every answer allowed at every query):

```

/-- The set of possible outputs of running the monadic computation `mx`. -/
def support [MonadLiftT m SetM] {α : Type u} (mx : m α) : Set α :=
  SetM.run (liftM mx)

```

VCVio/EvalDist/Defs/Support.lean:42-44, VCVio f9dc47d9

3.1.3 Uniform sampling, with uniformity as a law

SampleableType and $\t : uniform sampling

A `SampleableType` β instance is a sampler `selectElem : ProbComp  $\beta$` and *two proofs*: every element is reachable, and any two elements have equal probability. $\$^t \beta$ is that sampler. Uniformity is part of the structure: because a `ProbComp` never fails, the probabilities sum to one, and `VCVio` derives the uniform law for every instance. Consequently **no instance of `SampleableType` can be non-uniform**, and two instances on the same finite type have the same distribution (proved in the probe `SamplerDiamond`, `sampler_unique`; section 3.8.2).

```

/-- A `SampleableType  $\beta` instance means that ` $\beta` is a finite inhabited type,
with a computation `selectElem` that selects uniformly at random from the type.
...
class SampleableType ( $\beta$  : Type) where
  selectElem : ProbComp  $\beta$ 
  mem_support_selectElem (x :  $\beta$ ) : x  $\in$  support selectElem
  probOutput_selectElem_eq (x y :  $\beta$ ) : Pr[= x | selectElem] = Pr[= y | selectElem]

/-- Select uniformly from the type ` $\beta` using a type-class provided definition.
NOTE: naming is somewhat strange now that `Fintype` isn't explicitly required. -/
def uniformSample ( $\beta$  : Type) [h : SampleableType  $\beta$ ] : ProbComp  $\beta$  := h.selectElem

notation:90 "$^t "  $\alpha$ :91 => uniformSample  $\alpha$$$$ 
```

VCVio/OracleComp/Constructions/SampleableType.lean:37-53, VCVio f9dc47d9

```

/-- Every element of a uniform sample over a `Fintype` has output probability ` $\text{card}^{-1}$ . -/
@[simp, grind =]
lemma probOutput_uniformSample [Fintype  $\alpha$ ] (x :  $\alpha$ ) :
  Pr[= x |  $\$^t \alpha$ ] = (Fintype.card  $\alpha$  :  $\mathbb{R}_{\geq 0}$ ) $^{-1}$  := by

```

SampleableType.lean:57-66, VCVio f9dc47d9

```

@[simp, grind =]
lemma probEvent_uniformSample [Fintype  $\alpha$ ] (p :  $\alpha \rightarrow \text{Prop}$ ) [DecidablePred p] :
  Pr[ p |  $\$^t \alpha$ ] = (Finset.univ.filter p).card / Fintype.card  $\alpha$  := by

```

SampleableType.lean:224-228, VCVio f9dc47d9

The constructors `leanerVM` uses are the base case, a uniform `Fin n`, which is the primitive coin; the transport along a bijection, which re-proves the laws; vectors, with independent coordinates (laws proved at :317-336); and any `FinEnum` type, that is, a type with a computable enumeration $\alpha \approx \text{Fin } n$ (this instance matters for the diamond on $\mathbb{K}$, section 3.8.2).

```

@[reducible] def SampleableType.Fin (n :  $\mathbb{N}$ ) : SampleableType (Fin (n + 1)) where
  selectElem :=  $\$[0..n]$ 
  mem_support_selectElem := by simp
  probOutput_selectElem_eq := by simp

instance (n :  $\mathbb{N}$ ) [hn : NeZero n] : SampleableType (Fin n) :=
  match n, hn with
  | _ + 1, _ => SampleableType.Fin _

```

SampleableType.lean:232-239, VCVio f9dc47d9

```

/-- A type equivalent to a `SampleableType` is also `SampleableType`. -/
@[reducible] def SampleableType.ofEquiv { $\alpha \beta$  : Type} [SampleableType  $\alpha$ ] (e :  $\alpha \approx \beta$ ) :
  SampleableType  $\beta$  where
  selectElem := e <$> ( $\$^t \alpha$ )
  mem_support_selectElem x := by simp
  probOutput_selectElem_eq x y := by
    rw [probOutput_map_equiv, probOutput_map_equiv]
    exact probOutput_uniformSample_inj  $\alpha$  (e.symm x) (e.symm y)

```

SampleableType.lean:260-267, VCVio f9dc47d9

```

/-- Select a uniform element from `Vector α n` by independently selecting `α` at each index. -/
instance (α : Type) (n : ℕ) [SampleableType α] : SampleableType (Vector α n) where
  selectElem := by induction n with
  | zero => exact pure #v[]
  | succ m ih => exact Vector.push <$> ih <*> ($↑ α)

```

SampleableType.lean:313-316, VCVio f9dc47d9

```

instance FinEnum.SampleableType (α : Type)
  [h : FinEnum α] [Nonempty α] : SampleableType α := by
  have : NeZero (FinEnum.card α) := NeZero.mk FinEnum.card_ne_zero
  exact SampleableType.ofEquiv h.equiv.symm

```

SampleableType.lean:269-273, VCVio f9dc47d9

ArkLib's challenger answers the challenge query of round i by exactly this sampler:

```

def challengeQueryImpl {pSpec : ProtocolSpec n} [∀ i, SampleableType (pSpec.Challenge i)] :
  QueryImpl ([pSpec.Challenge] ⋅ challengeOracleInterface) ProbComp :=
  fun q => $↑ (pSpec.Challenge q.1)

```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:732-734, ArkLib dca90385

3.1.4 What a reader trusts when a theorem is a VCVio probability bound

A statement $\Pr[\text{bad} \mid \text{game}] \leq \varepsilon$ in VCVio unfolds to: interpret the program **game** (a free-monad tree) by answering each uniform-choice query n with Mathlib's `PMF.uniformOfFintype (Fin (n+1))` and each `OptionT` failure by the missing mass, compose these distributions by Mathlib's `PMF bind`, and measure the event. The reader trusts:

1. that the **program game** is the intended experiment (for the proof system, that ArkLib's round-by-round knowledge-soundness game is the right game; that is the subject of section 3.6);
2. the **three definitions** above (`IsUniformSpec` `unifSpec` answers with the uniform distribution; `evalSPMF = simulateQ toPMF`; `probEvent` measures some $\{x \mid p\ x\}$), which are short and standard; and Mathlib's `PMF (bind, uniformOfFintype, toOuterMeasure)`);
3. that each **challenge type's** `SampleableType` instance is the intended sampler. Since any instance is uniform, what remains to trust is only that the challenge *type* is right ($\mathbb{E}$, not a subtype or a smaller field). `Fintype.card E = 2192 (card_E, section 3.8.1)` fixes it.

Nothing else in VCVio is on the trusted path of a statement: its many lemmas are proofs, checked by the kernel. (VCVio at the old pin also has a measure-theoretic semantics $\mathcal{Q}[\dots]$; the $\Pr[\dots]$ notation used by ArkLib and leanerVM is the discrete SPMF one quoted above, `Defs/Basic.lean:110-125`.)

3.1.5 At the new pin a4232d08

At VCVio **a4232d08** the primary semantics is a Mathlib `Measure (ℚ[mx])`, a fold of the free monad with per-query measures, proved to agree with the old mass-function semantics on discrete answer spaces (the agreement is proved in `OracleComp/EvalDist/UniformCompatibility.lean:47` and in `EvalDist/PFuncMeasure.lean:96-106`). `probOutput`, `probEvent` and `probFailure` are deprecated since 2026-09-13 (VCVio/EvalDist/Defs/Basic.lean:90-110 at a4232d08, @[deprecated "VCVio retiring probability API: use $\mathcal{Q}[mx] \{x \mid p\ x\}$ "] in favour of $\Pr\{\text{let } x \leftarrow c\} [P\ x]$, a macro for the measure of the event `{True}` under the computation returning the proposition:

```

macro_rules (kind := prEvent)
-- `doSeqBracketed`
| `(Pr{{items*}}[t]) => `(ℚ[do $items:doSeqItem* return $t:term] {True})
-- `doSeqIndent`
| `(Pr{{items*}}[t]) => `(ℚ[do $items:doSeqItem* return $t:term] {True})

```

VCVio/EvalDist/ProbabilityNotation.lean:65-69, VCVio a4232d08

`SampleableType`'s law is now stated directly as “the output measure is uniform”, and sampler irrelevance is proved upstream (`SampleableType.prEvent_uniformSample_inst_irrel`, `NativeMeasure.lean:280-284`):

```
class SampleableType (β : Type) where
  /-- The canonical sampler of `β`, written `$` β`. -/
  selectElem : ProbComp β
  /-- The canonical sampler denotes the uniform measure on `β`. -/
  evalDist_selectElem_eq_uniform :
    ∀ [MeasurableSpace β] [MeasurableSingletonClass β],
      Ⓞ[selectElem] = uniformOn Set.univ
```

VCVio/OracleComp/Constructions/SampleableType/Basic.lean:51-57, VCVio a4232d08

What a reader trusts changes accordingly: the measure fold (`VCVio/EvalDist/PFuncutorMeasure/Core.lean:226-229`, `instEvalDistSemanticsFreeM`; a bind whose continuation is not almost-everywhere measurable denotes 0, :102, which cannot arise for discrete answer types) plus the compatibility proof. `leanerVM` at 144c5aa restated its two probability bridges in the new notation; for the counting bounds the two statements are the same mathematical statement (section 3.8.5).

3.2 CompPoly: tables, polynomials and the two fields

`CompPoly` is the library of *computable* polynomials: representations that run (`#eval`, compiled code) and are proved equal to `Mathlib`'s non-computable ones. Every snippet of this section is from `CompPoly` at the old pin 3468b38c (“feat(fields): add polynomial-basis $\text{GF}(2^{64})$ and its cubic extension ($\#321$)”), read with `git show 3468b38c:<path>`; section 3.2.6 says what the new pin 572f9973 changes.

3.2.1 A multilinear polynomial as its table of values

CMLPolynomialEval: a table on the Boolean cube

`CMLPolynomialEval R n` is a table of 2^n values in R ; entry i is the value at the cube point whose coordinate j is bit j of i (**bit 0 is variable 0**, little-endian). It stands for the multilinear polynomial with those values on $\{0,1\}^n$, defined through the Lagrange (equality) basis. Being `@[reducible]`, it is a `Vector` to instance search, which is why `leanerVM` wraps it in a structure `Column` (section 3.8.3).

```
/- `CMLPolynomialEval n R` is the type of multilinear polynomials in `n` variables over a ring `R`.
   It is represented by its evaluations over the Boolean hypercube `{0,1}^n`,
   i.e. Lagrange basis coefficients.
   The indexing is little-endian (i.e. the least significant bit is the first bit). -/
@[reducible]
def CMLPolynomialEval (R : Type*) (n : ℕ) := Vector R (2 ^ n) -- coefficient of Lagrange basis
```

CompPoly/Multilinear/Basic.lean:42-48, CompPoly 3468b38c

```
/- Lagrange (hypercube) basis at point `w`.

Returns the length-`2^n` vector `v` such that for any `x ∈ {0,1}^n`, letting
`i = ∑_{j=0}^{n-1} x_j · 2^j` (little-endian indexing), we have
`v[i] = ∏_{j < n} (x_j · w[j] + (1 - x_j) · (1 - w[j]))`.
Equivalently, for `i : Fin (2^n)`,
`v[i] = ∏_{j < n}, (if the j-th bit of i is 1 then w[j] else 1 - w[j])`.
-/
def lagrangeBasis (w : Vector R n) : Vector R (2 ^ n) :=
  Vector.ofFn (fun i => ∏ j : Fin n, if (BitVec.ofFin i).getLsb j then w[j] else 1 - w[j])
```

CompPoly/Multilinear/Basic.lean:401-411, CompPoly 3468b38c

```

/-- Evaluate a `CMLPolynomialEval` at a point -/
def eval (p : CMLPolynomialEval R n) (x : Vector R n) : R :=
  Vector.dotProduct p (lagrangeBasis x)
...
/-- Evaluate a `CMLPolynomialEval` at a point using a ring homomorphism -/
def eval₂ (p : CMLPolynomialEval R n) (f : R →+* S) (x : Vector S n) : S := eval (map f p) x

/-- Evaluate the multilinear equality kernel `eq̃(w, x)` -/
@[inline] def eqTilde (w x : Vector R n) : R :=
  eval (lagrangeBasis w) x

```

CompPoly/Multilinear/Basic.lean:523-544, CompPoly 3468b38c

So $\text{eval } p \ x = \sum_i p[i] \cdot \prod_j (\text{bit}_j(i) ? x_j : 1 - x_j)$, the multilinear extension $\tilde{p}(x)$, and $\text{eqTilde } w \ x = \sum_i \text{eq}(w, i) \text{eq}(x, i) = \prod_j (w_j x_j + (1 - w_j)(1 - x_j))$. The fast evaluator folds one variable at a time (variable 0 first; `evalMleStep`, :467-471: entry j of the folded table is $(1 - x_0) \cdot \text{values}[2j] + x_0 \cdot \text{values}[2j+1]$) and is proved equal to the dot product. `map f p` applies a ring homomorphism $f : R \rightarrow^+ S$ to every entry (:461-463); with $f = \text{algebraMap } K \ E$ it lifts a K -table to an E -table, so `eval₂Mle q (algebraMap K E) r` evaluates the K -multilinear $\tilde{q}$ at a point $r \in E^n$.

```

/-- Evaluate a `CMLPolynomialEval` by recursive multilinear-extension interpolation. -/
@[inline, specialize]
def evalMle (p : CMLPolynomialEval R n) (x : Vector R n) : R :=
  evalMleValues p x
...
/-- Evaluate a `CMLPolynomialEval` through a ring homomorphism using multilinear-extension
interpolation. -/
@[inline, specialize]
def eval₂Mle (p : CMLPolynomialEval R n) (f : R →+* S) (x : Vector S n) : S :=
  evalMle (map f p) x

```

CompPoly/Multilinear/Basic.lean:497-521, CompPoly 3468b38c

```

theorem eval_mle_eq_eval (p : CMLPolynomialEval R n) (x : Vector R n) :
  evalMle p x = eval p x := by
...
theorem eval₂_mle_eq_eval₂ (p : CMLPolynomialEval R n) (f : R →+* S) (x : Vector S n) :
  eval₂Mle p f x = eval₂ p f x := by

```

CompPoly/Multilinear/Basic.lean:585-586 and :638-640, CompPoly 3468b38c

Bridge to Mathlib. At the CompPoly pin there is a conversion `CMLPolynomialEval.toMvPolynomial` (Multilinear/Equiv.lean:337) but **no theorem that `eval` is Mathlib’s evaluation of it**; that bridge is ArkLib’s. The blueprint cites it correctly as ArkLib’s (protocol-blueprint.md:246). For a reader the definition `eval` above is self-explanatory and needs no bridge to be trusted.

```

theorem eval_eq_MvPolynomial_MLE (evals : (Fin n → Fin 2) → R) (x : Fin n → R) :
  eval
    (Vector.ofFn fun i => evals (finFunctionFinEquiv.symm i))
    (Vector.ofFn x) =
    MvPolynomial.eval x (MvPolynomial.MLE evals) := by

```

ArkLib/ToCompPoly/Multilinear/Basic.lean:56, ArkLib dca90385

Bit order against leanVM. The specification fixes “ $\langle u \rangle := \sum_i u_i 2^i \dots$ (bit 0 first)” (doc/leanvm/body/b-polynomial-commitment-scheme.tex:278) and places each table’s selector in the *high* bits of its offset (04-committing-the-witness.tex:10, “ $\text{sel}_i \in \{0, 1\}^{M-\kappa_i}$ for the high bits of that offset”). CompPoly’s little-endian order agrees: the low variables of the stack index the position inside a block.

3.2.2 Computable multivariate polynomials

CMvPolynomial: sparse multivariate polynomials that run

`CMvPolynomial n R` is a finite map from monomials (exponent vectors `Vector ℕ n`) to coefficients, with no zero coefficient stored (`Unlawful n R` is the map, `Lawful` the subtype without zeros). The spine states its constraint and flush polynomials in this type (`LeanerVM/Protocol/Spine/Instance.lean:123-132`, `Seams.lean:77`). `eval2`, `eval` and `totalDegree` are computed by folding over the map; a bijection with Mathlib’s `MvPolynomial` (the standard, non-computable multivariate polynomial type) transfers evaluation and degree.

```

/-- A computable multivariate polynomial in `n` variables with coefficients in `R`. -/
abbrev CMvPolynomial (n : ℕ) (R : Type*) [Zero R] : Type _ := Lawful n R
...
/-- The subtype of polynomials with no zero coefficients. -/
@[implicit_reducible]
def Lawful (n : ℕ) (R : Type*) [Zero R] : Type _ :=
  {p : Unlawful n R // p.isNoZeroCoef}

```

`CompPoly/Multivariate/CMvPolynomial.lean:47-48` and `Multivariate/Lawful.lean:33-36`, `CompPoly 3468b38c`

```

def eval2 {R S : Type*} {n : ℕ} [Semiring R] [CommSemiring S] :
  (R →+* S) → (Fin n → S) → CMvPolynomial n R → S :=
  fun f vs p => ExtTreeMap.foldl (fun s m c => (f c * MonoR.evalMonomial vs m) + s) 0 p.1
...
def eval {R : Type*} {n : ℕ} [CommSemiring R] : (Fin n → R) → CMvPolynomial n R → R :=
  eval2 (RingHom.id _)
...
def totalDegree {R : Type*} {n : ℕ} [Zero R] : CMvPolynomial n R → ℕ :=
  fun p => Finset.sup (List.toFinset (List.map CMvMonomial.toFinsupp (Lawful.mononials p)))
    (fun s => Finsupp.sum s (fun _ e => e))

```

`CMvPolynomial.lean:131-135, :216-218, :229-233`, `CompPoly 3468b38c`

```

def fromCMvPolynomial (p : CMvPolynomial n R) : MvPolynomial (Fin n) R :=
  let support : List (Fin n → ℕ) := p.mononials.map CMvMonomial.toFinsupp
  let toFun (f : Fin n → ℕ) : R := p[CMvMonomial.ofFinsupp f]?.getD 0
  let mem_support_fun {a : Fin n → ℕ} : a ∈ support ↔ toFun a ≠ 0 := by grind
  AddMonoidAlgebra.ofCoeff <| Finsupp.mk support.toFinset toFun (by simp [mem_support_fun])

```

`CompPoly/Multivariate/MvPolyEquiv/Core.lean:35-39`, `CompPoly 3468b38c`

```

lemma eval_equiv {p : CMvPolynomial n R} {vals : Fin n → R} :
  p.eval vals = (fromCMvPolynomial p).eval vals := by
...
lemma totalDegree_equiv {S : Type*} {p : CMvPolynomial n R} [CommSemiring S] :
  p.totalDegree = (fromCMvPolynomial p).totalDegree := by rfl

```

`CompPoly/Multivariate/MvPolyEquiv/Eval.lean:53-61`, `CompPoly 3468b38c`

With `polyEquiv : CMvPolynomial n R = MvPolynomial (Fin n) R` (`Core.lean:134`), a degree bound `C.totalDegree ≤ d` in the spine is Mathlib’s total degree, and Schwartz–Zippel from Mathlib applies through `eval_equiv`. Building polynomials (C, X, ring operations) needs `[BEq R] [LawfulBEq R]` (`CMvPolynomial.lean:55-62`). $\mathbb{K}$ has it (instance search finds `instLawfulBEq` from `DecidableEq`; probe `FieldFidelity`, last lines); $\mathbb{E}$ **has none at the old pin** (the same probe: failed to synthesize `LawfulBEq E`), so the spine cannot build a `CMvPolynomial` over $\mathbb{E}$. The blueprint records this (`protocol-blueprint.md:570`) and keeps every polynomial over $\mathbb{K}$, which matches leanVM (constraints have $\mathbb{K}$ coefficients).

3.2.3 The fields $\mathbb{K} = \text{GF}(2^{64})$ and $\mathbb{E} = \text{GF}(2^{192})$

BF64: the field $\mathbb{K}$

$\mathbb{K} = \text{GF}(2)[x]/(x^{64} + x^4 + x^3 + x + 1)$. CompPoly keeps the specification object and the computable carrier apart and joins them by a proved bijective ring homomorphism. The specification is Mathlib's quotient `AdjoinRoot basePoly`, a field because the modulus is irreducible; the carrier is a 64-bit word whose bit i is the coefficient of x^i , with addition XOR and multiplication the carry-less product reduced modulo the modulus.

```

/-- The modulus `x^64 + x^4 + x^3 + x + 1` over `GF(2)`. Part of the specification only;
it is never evaluated. -/
noncomputable def basePoly : Polynomial (ZMod 2) := X ^ 64 + X ^ 4 + X ^ 3 + X + 1
...
theorem basePoly_irreducible : Irreducible basePoly := by
  refine Polynomial.irreducible_of_rabin (d := 64) ?_ (by norm_num) ?_ ?_
...
noncomputable abbrev BF64Quot : Type := AdjoinRoot basePoly

```

CompPoly/Fields/Binary/BF64/Basic.lean:100-102, :152-153, :199, CompPoly 3468b38c

Irreducibility is proved by Rabin's test (CompPoly's `Polynomial.irreducible_of_rabin`) against certificate chains that the kernel checks by `rfl` (BF64/BaseCertificate.lean, 552 lines); no `native_decide` (axioms of `BF64.basePoly_irreducible`: `propext`, `Classical.choice`, `Quot.sound`, `probe Layer0`).

```

/-- `GF(2^64)` in its computable, machine representation: a 64-bit word whose bit `i` is
the coefficient of `x^i`. -/
abbrev BF64 : Type := BitVec 64

namespace BF64

instance : Zero BF64 := ((0 : BitVec 64))
instance : One BF64 := ((1 : BitVec 64))

/-- Addition in characteristic two is `xor`. -/
instance : Add BF64 := {fun a b => a ^^^ b}
...
/-- Multiplication: the carry-less product, reduced modulo the modulus. -/
instance : Mul BF64 :=
  {fun a b => reduce (carryLessMul (w := 128) a b)}

```

CompPoly/Fields/Binary/BF64/Impl.lean:52-71, CompPoly 3468b38c

The carrier maps to the quotient by `toQuot a = AdjoinRoot.mk basePoly (toPoly a) (:77-79)`, proved a ring homomorphism (`toQuot_add`, :103; `toQuot_mul`, :107), injective (:118) and surjective (:398). Every ring and field law of BF64 is transported through it (`CommRing`, :237-255; `Field`, :442-450); inversion is the Itoh-Tsujii chain, proved correct (`mul_invItohTsujii`, :404), with $0^{-1} = 0$. `DecidableEq K` is `BitVec`'s (`instDecidableEqBitVec`, `probe Layer0`). Cardinality and the instances:

```

/-- The carrier is in bijection with `Fin (2 ^ 64)`, by its underlying representation. -/
def equivFin : BF64 ≃ Fin (2 ^ 64) where
  toFun a := a.toFin
  invFun i := BitVec.ofFin i
  left_inv _ := rfl
  right_inv _ := rfl

instance : Fintype BF64 := Fintype.ofEquiv _ equivFin.symm

/-- `BF64` has `2 ^ 64` elements. -/
theorem card_bf64 : Fintype.card BF64 = 2 ^ 64 := by
  rw [Fintype.card_congr equivFin, Fintype.card_fin]

```

CompPoly/Fields/Binary/BF64/Impl.lean:384-395, CompPoly 3468b38c

Hazard of the eager Fintype BF64 (old pin). The instance is a closed computable term; compiled code initialises it at program start by building `List.finRange (2^64)`, so any executable linking the module is killed for lack of memory. `leanerVM` records this (`docs/roadmap/leanisa-status.md`:753-769).

at **b435631**, the finding “instance : Fintype BF64 is evaluated at executable startup” (P3)): the test driver had to become a library and no leanerVM executable may link the field. It cannot simply be made **noncomputable** at this pin, because **ExtensionParams F** takes **[Fintype F]** and so every **Ext** operation carries it at run time. Consequence for the blueprint: the executable verifier **verify**, differential tests against the Rust, and any native target are **blocked at this pin** until the **CompPoly** fix lands; it has (section 3.2.6).

Hazard of abbrev BF64 := BitVec 64 (old pin). Because **BF64** is reducible, **CompPoly**’s **Add**, **Mul**, **Neg**, **Inv**, **NatCast**, **Fintype**, **CommRing**, **Field** instances are instances **on BitVec 64 itself** in every file that imports the field. The probe **FieldFidelity** (lines 85–100, output lines 2–12) shows it:

```
#eval u + v          -- u v : BitVec 64 := 3, 5  → 6#64  (XOR, not 8)
#eval (u + v == 8, u + v == 6)                → (false, true)
#synth Add (BitVec 64)                        → BF64.instAdd
#synth Add (BitVec 32)                        → BitVec.instAdd
```

probe **FieldFidelity** (probes/lib-others/FieldFidelity.lean), lines 85–100 and output lines 2–12, run at the old pins

and the numeral **(2 : K)** elaborates as **BitVec.instOfNat 64 2** (the bit pattern x), not the cast of the natural number two (which is 0): **#guard ((2 : ℕ) : K) = 0** and **#guard (2 : K) ≠ 0** both pass. No current leanerVM declaration adds or multiplies a **BitVec 64** meaning integers (a **git grep** of **b435631** for **BitVec 64**, **UInt64** and 64-bit literals finds only **BitVec.ofNat 64**, **extractLsb**, **++**, **<<<**, and **UInt64** in **BLAKE2s**, whose operations are core’s), so this is a hazard, not a defect. The companion hazard, core’s **BitVec** simplification procedures firing on $\mathbb{K}$, is the finding “**simp** misreads $\mathbb{K}$ arithmetic” (finding 133).

BF64.Ext3: the field $\mathbb{E}$

$\mathbb{E} = \mathbb{K}[y]/(y^3 + y + 1)$, built by **CompPoly**’s generic extension framework **Ext P**: the carrier is a dense coefficient vector of length **P.d**, little-endian (index i is the coefficient of y^i), so an element of $\mathbb{E}$ is the limb triple $c_0 + c_1y + c_2y^2$. **ext3Params** records the modulus below its implicit leading term (lower **:= #v[1, 1, 0]** are the coefficients of $1, y, y^2$), **ofBase** embeds $\mathbb{K}$ as the y^0 limb, and **gen** is y .

```
noncomputable def ext3Poly : Polynomial BF64 := X ^ 3 + X + 1
...
theorem ext3Poly_irreducible : Irreducible ext3Poly :=
  Polynomial.irreducible_of_degree_le_three_of_not_isRoot
    (by rw [ext3Poly_natDegree]; decide) ext3Poly_no_root
...
def ext3Params : ExtensionParams BF64 where
  d := 3
  two_le := by norm_num
  lower := #v[1, 1, 0]
  q := 2 ^ 64
  card_eq := card_bf64
...
abbrev Ext3 : Type := Ext ext3Params
...
@[simp] theorem card_ext3 : Fintype.card Ext3 = 2 ^ 192 := by
  rw [Ext.card_ext, ext3Params_q, ext3Params_d, ← pow_mul]
```

CompPoly/Fields/Binary/BF64/Ext3.lean:56-57, :125-127, :133-140, :165-168, :198-200, CompPoly 3468b38c

Irreducibility: a root a of $y^3 + y + 1$ has $a^7 = 1$; $\gcd(7, 2^{64} - 1) = 1$ (**decide +kernel**), so $a = 1$, which is not a root (**Ext3.lean:78-122**). **ext3Params_poly** (:148-160) proves that the coefficient vector **#v[1, 1, 0]** denotes $y^3 + y + 1$.

```
--
The carrier of the extension `F[X] / f`: a dense coefficient vector of length `P.d`,
little-endian (index `i` is the coefficient of `X^i`).
-/
def Ext {F : Type*} [Field F] [Fintype F] (P : ExtensionParams F) : Type _ := Vector F P.d
...
@[inline] def ofBase (c : F) : Ext P := ofFn fun i => if (i : ℕ) = 0 then c else 0
...
def gen : Ext P := ofFn fun i => if (i : ℕ) = 1 then 1 else 0
```

CompPoly/Fields/Extension/Defs.lean:132-136, :185, :188, CompPoly 3468b38c

Ext is a plain def (not reducible), so Vector’s instances do not leak onto $\mathbb{E}$. Its multiplication folds products by the reduced monomials $X^k \bmod f$ (Defs.lean:227-233; the compiled code uses the table version mulTbl, swapped in by a proved @[csimp] equation mul_eq_mulTbl, :267-272). The ring laws are transported from AdjoinRoot P.poly through a proved bijection (Extension/Field.lean:73-107, ringEquivQuot); inversion is Fermat, $x^{-1} = x^{q^d-2}$ (Field.lean:127); the Field instance needs [Fact (Irreducible P.poly)] (:159-166), supplied by Ext3.lean:162-163. Fintype (Ext P) is by the coefficient bijection (Field.lean:54-63), card_ext : Fintype.card (Ext P) = P.q ^ P.d. The embedding $\mathbb{K} \rightarrow \mathbb{E}$ is Ext.ofBase, packaged as algebraMap:

```
-- The extension is an `F`-algebra. -/
instance instAlgebra : Algebra F (Ext P) where
  algebraMap := ofBaseRingHom P
  commutes' _ _ := mul_comm _ _
  smul_def' c x :=
    toQuot_injective (by simp only [toQuot_smul, toQuot_mul, ofBaseRingHom_apply, toQuot_ofBase])
@[simp] theorem algebraMap_eq_ofBase (c : F) : algebraMap F (Ext P) c = ofBase c := rfl
```

CompPoly/Fields/Extension/Bridge.lean:363-370, CompPoly 3468b38c

DecidableEq (Ext P) and BEq (Ext P) are Vector’s (Defs.lean:282-284); there is no LawfulBEq (Ext P) at the old pin (section 3.2.2).

3.2.4 Are these leanVM’s fields? Yes, bit for bit

A formalization over another presentation of $\text{GF}(2^{64})$ would say nothing about the deployed verifier’s arithmetic. Table 3.1 checks the fields on four sources. Specification: doc/leanvm/body/02-vm-specification.tex:6-9; Rust: under crates/primitives/src/field/; Python: python-verifier/verifier.py; all at leanVM’s pin a386121f.

Table 3.1: The fields $\mathbb{K}$ and $\mathbb{E}$ in the specification, the Rust, the Python and CompPoly/leanerVM.

	Specification	Rust	Python	CompPoly / leanerVM
$\mathbb{K}$ modulus	$x^{64} + x^4 + x^3 + x + 1$	gf2_64.rs:2 GF(2)[x]/(x ⁶⁴ + x ⁴ + x ³ + x + 1), reduction R64 = 0x1B (gf2_64x3.rs:30)	“K = :39 F2[x]/(x ⁶⁴ + x ⁴ + x ³ + x + 1)”, fold high ^ high<<1 ^ high<<3 ^ high<<4 (:31)	basePoly (BF64/Basic.lean:102)
$\mathbb{K}$ encoding	—	gf2_64.rs:19 “bit i = coefficient of x ⁱ ”, F64(pub u64)	“bit i = K(value: int), 64-bit (:39-46)	BitVec 64, bit i = coefficient of x ⁱ (Impl.lean:52-54)
$\mathbb{K}$ addition	—	self.0 ^ rhs.0 (gf2_64.rs:75)	self.value ^ rhs.value (:67)	a ^^ b (Impl.lean:62)
$\mathbb{E}$ modulus	$\mathbb{K}[y]/(y^3 + y + 1)$	gf2_64x3.rs:3 K[y]/(y ³ + y + 1)	“F192 = :92 “K[y]/(y ³ + y + 1)”	ext3Poly, lower := #v[1, 1, 0]
$\mathbb{E}$ limbs	—	F192 { c0, c1, c2 }, “c0 + c1·y + c2·y ² ” (gf2_64x3.rs:5,34-37)	E(c0, c1, c2), bytes <30 (:94-107)	Vector K 3, index i = coefficient of y ⁱ
y	—	F192::Y = (0, 1, 0) (gf2_64x3.rs:44)	Y = E(0, 1) (:183)	Ext.gen = limbs (0, 1, 0)
$\mathbb{K} \hookrightarrow \mathbb{E}$	—	limb c0	E(lifted) = (k, 0, 0) (:111-115)	ofBase = (c, 0, 0)
generator	“a generator of order 2 ⁶⁴ − 1” (:12)	F64::G = F64(2) (gf2_64.rs:28)	GEN = E(2) (:182)	g : K := 0x2 (Generator.lean:259), order proved

Beyond the matching definitions, the probe FieldFidelity ran the Rust’s own reference vectors through leanerVM’s compiled $\mathbb{K}$ and $\mathbb{E}$ arithmetic at the old pins, and every #guard passed (the three products of gf2_64.rs:271-275, the four products and squares of gf2_64x3.rs:991-1016, $y \cdot y \cdot y = y + 1$, $x^{63} \cdot x = 0x1B$, $0^{-1} = 0$) while a mutated product and a mutated limb were rejected (appendix B). Python’s E.__mul__

(:145-152) folds p_0+p_3 , $p_1+p_3+p_4$, p_2+p_4 , which is exactly leanerVM's `mul_limbs` (LeanerVM/Parameters/Field.lean:206-210).

Conclusion. leanerVM's $\mathbb{K}$ and $\mathbb{E}$ are the specification's fields *with the Rust's and Python's encodings*: the identification is the identity on 64-bit words and on limb triples. (leanVM also uses $\text{GF}(2^8)$ with the AES modulus and an embedding $\varphi_8 : \text{GF}(2^8) \rightarrow \mathbb{K}$ inside Flock, `crates/primitives/src/field/gf2_8.rs`, `phi8_tower.rs:15-24`, `c-flock-protocol.tex:35`; CompPoly has no such field at the old pin and the blueprint says so, `protocol-blueprint.md:281`.)

3.2.5 leanerVM's field modules (at b435631)

$\mathbb{K}$ and $\mathbb{E}$ are *abbreviations* of CompPoly's types, so leanerVM adds no field of its own; y is CompPoly's generator; `ofK` is `algebraMap K E` (probe Layer0: example `(a : K) : ofK a = algebraMap K E a := rfl` is accepted). The file's other declarations are limb lemmas (`limb_add`, `mul_limbs`, `isInK_iff`, ...), all proved.

```

/-- `K = GF(2^64)`, CompPoly's `BF64`: a `BitVec 64` with bit `i` the coefficient of `x^i`. -/
abbrev K : Type := BF64

/-- `E = K[y]/(y^3 + y + 1)`, CompPoly's `BF64.Ext3`: a `Vector K 3` in limb order
`c0 + c1·y + c2·y^2`. -/
abbrev E : Type := BF64.Ext3

/-- The adjoined root `y` of `y^3 + y + 1`. -/
def y : E := BF64.ext3Gen

/-- The defining relation of `E`: `y^3 = y + 1`. -/
theorem y_pow_three : y ^ 3 = y + 1 := BF64.ext3Gen_pow_three
...
/-- The embedding `K ↪ E` as the `y^0` limb; it is `algebraMap K E` by `rfl`. -/
abbrev ofK (a : K) : E := Ext.ofBase a
...
/-- The word `c0 + c1·y + c2·y^2`, as the literal limb vector. -/
def E.ofLimbs (c0 c1 c2 : K) : E := Ext.ofVector #v[c0, c1, c2]

```

LeanerVM/Parameters/Field.lean:70-92, leanerVM b435631

LeanerVM/Parameters/Generator.lean:258-311: $g : K := 0x2$, and `orderOf g` : `orderOf g = 264 - 1` by Lagrange plus seven `decide` +kernel checks $g^{(2^{64}-1)/p} \neq 1$ over the prime factors $3 \cdot 5 \cdot 17 \cdot 257 \cdot 641 \cdot 65537 \cdot 6700417$. The generator is certified, not assumed.

LeanerVM/Parameters/CleanField.lean:360-372 supplies Clean's field interface for $\mathbb{K}$, so Clean's $\mathbb{K}$ has CompPoly's field structure (one field structure, by `rfl`). Clean's `val/fromNat` read $\mathbb{K}$ as the integers $0, \dots, 2^{64} - 1$, which is *not* a ring homomorphism in characteristic two; the file's docstring records that Clean's core never consumes them.

```

instance instFiniteFieldK : FiniteField K where
  val := BitVec.toNat
  fromNat n := BitVec.ofNat 64 n
  size := 2 ^ 64
  ...
theorem instFiniteFieldK_toField : (instFiniteFieldK.toField : Field K) = BF64.instField := rfl

```

LeanerVM/Parameters/CleanField.lean:360-372, leanerVM b435631

3.2.6 At the new pin 572f9973

CompPoly moved by 64 commits, from 3468b38c to 572f9973. What changes for the proof system:

- **BF64 is a structure** (#329, “isolate the BF64 presentation”, e5f87c8): `structure BF64` where `toBitVec : BitVec 64` deriving `DecidableEq`, `BEq` (CompPoly/Fields/Binary/BF64/Impl.lean:57-61 at 572f9973), with `ofBitVec`. CompPoly's field instances no longer leak onto `BitVec 64`; core's `BitVec` simplification procedures no longer see $\mathbb{K}$; and `FinEnum (BitVec 64)` no longer gives a sampler on $\mathbb{K}$ (section 3.8.2).

- **Numerals change meaning.** There is no `OfNat BF64` instance; a numeral $n \geq 2$ elaborates through Mathlib’s `NatCast` numeral instance, and the cast is by parity (`Impl.lean:223-227` at `572f9973`). So $(2 : \mathbb{K}) = 0$, $(3 : \mathbb{K}) = 1$, $(0x1B : \mathbb{K}) = 1$: the characteristic-two meaning. Bit patterns are written `K.ofBits n`, which leanerVM defines at `144c5aa` (`LeanerVM/Parameters/Field.lean`).

```

/-- Cast a natural number by its parity in characteristic two. -/
@[inline] def natCast (n : ℕ) : BF64 := if n % 2 = 0 then 0 else 1

instance : NatCast BF64 := {natCast}

```

`CompPoly/Fields/Binary/BF64/Impl.lean:223-227, CompPoly 572f9973`

```

def K.ofBits (n : ℕ) : K := BF64.ofBitVec (BitVec.ofNat 64 n)

```

`LeanerVM/Parameters/Field.lean, leanerVM 144c5aa`

Effect on probes and fixtures. Every numeral ≥ 2 in $\mathbb{K}$ written at the old pin *compiles at the new pin with another value*. The probe `FieldFidelity` at the new pin would read `(0x01090913877ed8ed : K) * 0x66ab35ac2768468f = 0x50c4519dc383744a` as $1 \cdot 1 = 0$ (the parities of the three words) and fail; its `#guard (2 : K) ≠ 0` would fail, `#guard ((2 : ℕ) : K) = 0` would pass, `g = (2 : K)` would fail (`g` is `K.ofBits 0x2` at `144c5aa`). Its conclusions are about the old pin and stand there; at the new pin the field and its encoding are the same (`ofBitVec` of the same words), so re-running it with `K.ofBits` on every literal would be the check (unverified: no run is possible now). The probe `NumeralsHazard` is moot at the new pin. Pull request #61 rewrote leanerVM’s fixtures with `K.ofBits` (for example `tests/LeanerVMTests/Protocol/Field.lean` at `144c5aa`: `{#v[1, K.ofBits 2, K.ofBits 3, K.ofBits 4]}`) and pins the new meaning (`tests/LeanerVMTests/Parameters/Field.lean:50-51` at `144c5aa`: `#guard (2 : K) = 0, #guard K.ofBits 2 ≠ (2 : K)`). A scan of `144c5aa` for `ofK n`, $(n : \mathbb{K})$, $(n : \mathbb{E})$, `: K := n`, `E.ofLimbs n` with $n \geq 2$ or a hex literal finds only those two intended lines and `E.ofLimbs 0x0000000000000000 ... (= 0)` in `tests/LeanerVMTests/Semantics/Blake2s.lean:197`; numerals inside vectors and other shapes were not scanned (unverified).

The other changes.

- #330, “separate extension arithmetic from finite certificates” (`cf340b2`): `ExtensionParams F` no longer takes `[Fintype F]`; its `q` is uncertified data (“Raw arithmetic does not certify this value. Field laws require a separate proof that `Nat.card F = q`”, `CompPoly/Fields/Extension/Arithmetic.lean:57-61` at `572f9973`), supplied for `Ext3` by instance `: Fact (Nat.card BF64 = ext3Params.q) := {nat_card_bf64} (Ext3.lean:143-144)`.
- #331, “avoid eager BF64 enumeration at native startup” (`290c351`): no `Fintype BF64`; there is instance `: Finite BF64` and theorem `card_bf64 [Fintype BF64] : Fintype.card BF64 = 2 ^ 64 (Impl.lean:459-468)`, likewise `card_ext3 [Fintype Ext3] (Ext3.lean:206-208)`. leanerVM supplies, at `144c5aa`, noncomputable instance `: Fintype K := Fintype.ofFinite K` (“A proof-only enumeration of $\mathbb{K}$; executable code uses its explicit bit coordinates”), from which `CompPoly`’s `Ext.instFintype [Fintype F] (Extension/Cardinality.lean:52)` gives `Fintype E`. **So Fintype K is now proof-only:** `Fintype.card K` in statements is unchanged in meaning (a subsingleton class), no executable can evaluate it, and the startup out-of-memory failure of the eager instance is gone. The blocker on executables (leanISA status finding P3) is lifted by the new pin; the blueprint’s executable `verify` and the differential tests are unblocked on this count (other blockers not assessed).
- leanerVM also adds, at `144c5aa`, its own `instDecidableEqK` (decided on `toBitVec`) beside `CompPoly`’s derived `DecidableEq BF64`: two instances, equal as propositions (`Decidable` is a subsingleton), chosen for kernel reduction per its docstring. Harmless; one more trusted-surface line.
- `LawfulBEq (Ext P)` now exists (`Extension/Arithmetic.lean:380`, given `LawfulBEq F`), so `CMvPolynomial n E` gets `C`, `X` and ring operations at the new pin; the blueprint’s “no `LawfulBEq E` at the pin” (`protocol-blueprint.md:570`) is stale after #61 (not re-checked by instance search: no run).
- New fields: the AES polynomial field $\text{GF}(2^8)$, modulus $X^8 + X^4 + X^3 + X + 1$ (`CompPoly/Fields/Binary/Aes/Basic.lean`, #356), and its embedding into GHASH $\text{GF}(2^{128})$ (#357). leanerVM’s Flock

uses the same $\text{GF}(2^8)$ and an embedding $\varphi_8 : \text{GF}(2^8) \rightarrow \mathbb{K}$; CompPoly has the source field now but no embedding into BF_{64} , so the blueprint’s gap statement (`protocol-blueprint.md:281`, “no $\text{GF}(2^8) \hookrightarrow E$ embedding”) remains true but should say the field itself has landed.

- Serialization (#373–#376: byte codecs for every field carrier, “exact fiber counts and the bias bound of the challenge decoder”): relevant to the compiled verifier’s transcript (Layer 12) and to modelling `sample()` -> F192 from a hash output; not assessed further.

leanerVM’s CompPoly-bound modules and upstream. The four modules `Multilinear.lean`, `BitProductTable.lean`, `Stacking.lean` and `AmbientStacking.lean` under `LeanerVM/Protocol/ToCompPoly/` were checked by name and by content keyword (`sumCube`, `hypercube sums`, `hadamard/pointwise products`, `evalMle_eq_sum`, `evalMle_split/append`, `bitProductTable/powersTable`, `stacking`) in CompPoly 3468b38c and 572f9973 (`CompPoly/Multilinear/`) and ArkLib dca90385 and 7653a901 (`ArkLib/ToCompPoly/`): **no counterpart** of any of their 80-odd declarations. What CompPoly added between the pins in `Multilinear/Basic.lean` is linearity (`eval_add`, `eval_zero`, `eval_smul` for both representations, `map_eval`, `evalWithProducts`), which the local modules do not restate. The local modules remain CompPoly candidates. One name clash is inferred, not run: CompPoly 572f9973 now declares `CompPoly.CMLPolynomialEval.eval_zero` (`CompPoly/Multilinear/Basic.lean:719`), and ArkLib 7653a901 declares the same name in `ArkLib/ToCompPoly/Multilinear/Basic.lean:35-45`; ArkLib builds against its own CompPoly pin df591bb8, which lacks it, and leanerVM’s root pin 572f9973 wins the resolution. So a leanerVM file importing `ArkLib.ToCompPoly.Multilinear.Basic` would fail with a duplicate declaration. leanerVM imports none of these modules today, but the blueprint names that module’s `eval_eq_MvPolynomial_MLE` as *the* bridge from tables to Mathlib multinears (`protocol-blueprint.md:246`); see finding 135.

3.3 Clean: circuits, tables, buses, and why its balance is unsatisfiable over $\mathbb{K}$

Clean is a library for writing zkVM circuits (AIR tables and the buses between them) with proofs that the constraints imply a specification. leanISA (the arithmetization) is written in it; the proof system consumes its *tables* through the spine’s adaptor, not its soundness statement. Every snippet of this section is from Clean at the old pin 93c9d1ef, read with `git show 93c9d1ef:<path>` under `.lake/packages/Clean`. At this pin Clean is **not** a Lean module library, which is why `LeanerVM/Parameters/CleanField.lean` is a plain file (section 3.2.5). Section 3.3.8 says what the new pin 42fe4b26 changes.

3.3.1 The circuit language

FiniteField, Expression, Environment, Operation, Circuit

Every Clean circuit is generic over a field with Clean’s `FiniteField` interface: a Mathlib field together with a reading `val` into the natural numbers, its inverse `fromNat`, and a `size`. An `Expression` is a polynomial expression over the variables of a row. An `Environment` assigns the variables (`get`) and names the side tables a specification may mention (`data`, of type `ProverData F = String → (n : ℕ) → Array (Vector F n)`, `Expression.lean:20-21`). A circuit emits `Operations`: a fresh witness variable, a polynomial asserted to be zero, a lookup, a bus interaction, or a nested circuit; a `Circuit` is a state monad producing operations from a variable offset.

```
class FiniteField (F : Type) extends Field F where
  /-- Canonical embedding of field elements into natural numbers. -/
  val : F → ℕ
  /-- Inverse of `val`: interpret a natural number (below the field size) as a field
  element. For prime fields this is `Nat.cast`; for binary fields it interprets the
  binary digits as polynomial coefficients. Note that `Nat.cast` would be wrong there:
  it reduces via the characteristic, collapsing `GF(2^n)` to `{0, 1}`. -/
  fromNat : ℕ → F
  ...
  size : ℕ
```

Clean/Utils/FiniteField.lean:23-45, Clean 93c9d1ef

```
inductive Expression (F : Type) where
| var : Variable F -> Expression F
| const : F -> Expression F
| add : Expression F -> Expression F -> Expression F
| mul : Expression F -> Expression F -> Expression F
```

Clean/Circuit/Expression.lean:12-16, Clean 93c9d1ef

```
structure Environment (F : Type) where
/-- Assignment of a circuit's variables to field elements -/
get : ℕ → F
...
data : ProverData F
```

Clean/Circuit/Expression.lean:46-52, Clean 93c9d1ef

```
inductive Operation (F : Type) [FiniteField F] where
| witness : (m : ℕ) → (compute : WitgenIR F m) → Operation F
| assert : Expression F → Operation F
| lookup : Lookup F → Operation F
| interact : AbstractInteraction F → Operation F
| subcircuit : {n : ℕ} → Subcircuit F n → Operation F
...
def Operations (F : Type) [FiniteField F] := List (Operation F)
```

Clean/Circuit/Operations.lean:315-320 and :355, Clean 93c9d1ef

```
def Circuit (F : Type) [FiniteField F] (α : Type) := ℕ → α × List (Operation F)
```

Clean/Circuit/Basic.lean:29, Clean 93c9d1ef

3.3.2 Components, tables, ensembles

Component, Table, Ensemble, EnsembleWitness

A **Component** is one circuit checked on every row (**GeneralFormalCircuit** bundles the circuit with its assumptions, specification and soundness proof). A **Table** is a component with its concrete rows. An **Ensemble** is a list of components and the channels (buses) between them; its witness, an **EnsembleWitness**, is one table per component, the shared data and the public input.

```
structure Component (F : Type) [FiniteField F] where
{Input : TypeMap} {Output : TypeMap}
[provableInput : ProvableType Input] [provableOutput : ProvableType Output]
circuit : GeneralFormalCircuit F Input Output
```

Clean/Air/FlatComponent.lean:7-15, Clean 93c9d1ef

```
structure Table (F : Type) [FiniteField F] where
component : Component F
width : ℕ
table : List (Array F)
data : ProverData F
uniform_width : ∀ row ∈ table, row.size = width
```

Clean/Air/FlatComponent.lean:150-156, Clean 93c9d1ef


```

structure Ensemble (F : Type) [FiniteField F] (PublicIO : TypeMap) [ProvableType PublicIO] where
  tables : List (Component F)
  channels : List (RawChannel F)
  -- TODO: the verifier shouldn't be treated as a "circuit", and possibly shouldn't even be on here
  verifier : GeneralFormalCircuit F PublicIO unit := .empty F PublicIO
  ...
structure EnsembleWitness (ens : Ensemble F PublicIO) where
  tables : List (Table F)
  data : ProverData F
  publicInput : PublicIO F
  same_length : ens.tables.length = tables.length
  same_circuits : ∀ i (hi : i < ens.tables.length), ens.tables[i] = tables[i].component
  same_data : ∀ table ∈ tables, table.data = data

```

Clean/Air/FlatEnsemble.lean:11-25, Clean 93c9d1ef

Why EnsembleWitness lives in Type 1. A Table stores its Component, which stores its row types Input Output : TypeMap, and TypeMap := Type → Type (Clean/Circuit/CircuitType.lean:10) is itself a Type 1. A structure with a field in Type 1 is in Type 1 (probe CleanBalance, output lines 1–4: Air.Flat.Component : (F : Type) → [FiniteField F] → Type 1, likewise Table, Ensemble, EnsembleWitness). ArkLib's statements and witnesses are in Type, so an EnsembleWitness cannot be the proof system's witness; the blueprint draws exactly this consequence (protocol-blueprint.md:367-369, :582-586) and makes the stack q : Column μ, a Type, the witness, with a pointwise transport along the adaptor.

3.3.3 Channels and interactions

RawChannel, Channel.toRaw, Interaction

A raw channel is a name, a message arity, and two propositions indexed by the multiplicity: what a receiver may assume (**Guarantees**) and what a sender must prove (**Requirements**). The typed channel reads the *direction from the sign* of the multiplicity: multiplicity -1 is a pull, any other non-zero multiplicity a push. An interaction is a channel, a multiplicity and a message: symbolic in a circuit (**AbstractInteraction**), evaluated on a row (**Interaction**).

```

structure RawChannel (F : Type) where
  name : String
  arity : ℕ
  Guarantees (mult : F) (message : Vector F arity) (data : ProverData F) : Prop
  Requirements (mult : F) (message : Vector F arity) (data : ProverData F) : Prop

```

Clean/Circuit/Channel.lean:15-25, Clean 93c9d1ef

```

def toRaw (channel : Channel F Message) : RawChannel F where
  name := channel.name
  arity := size Message
  Guarantees mult message data :=
    mult = -1 → channel.Guarantees (fromElements message) data
  Requirements mult message data :=
    mult ≠ -1 →
    mult ≠ 0 →
    channel.Guarantees (fromElements message) data

```

Channel.lean:35-45, Clean 93c9d1ef

```

structure AbstractInteraction (F : Type) where
  channel : RawChannel F
  mult : Expression F
  msg : Vector (Expression F) channel.arity
  assumeGuarantees : Bool
  ...
structure Interaction (F : Type) where
  channel : RawChannel F
  mult : F
  msg : Array F
  same_size : msg.size = channel.arity
  assumeGuarantees : Bool

```

Channel.lean:101-105 and :305-310, Clean 93c9d1ef

3.3.4 Clean's ensemble statement and its soundness

balanceOf, BalancedInteractions, Ensemble.Statement

The balance of a message on a list of interactions is the sum, *in the field*, of the multiplicities of the interactions carrying it. A list is balanced when every message has balance zero and a side condition bounds the number of interactions by the characteristic. The ensemble statement for a public input says that some witness has that public input, satisfies every table's constraints, and balances every channel; Soundness says that the assumptions and the statement imply the specification.

```
--
Balance of one element of an interaction list.
This is just multiplicity of the element when viewing the list as a multiset.
-/
def balanceOf (interactions : List (Interaction F)) (msg : Array F) : F :=
  interactions.filter (·.msg = msg) |>.map (·.mult) |>.sum

--
Channel balance: for any message, the sum of multiplicities is 0.
We also require a side condition that ensures the interaction count does not overflow.
-/
def BalancedInteractions (interactions : List (Interaction F)) : Prop :=
  (interactions.length < ringChar F ∨ ringChar F = 0) ∧
  ∀ msg : Array F, balanceOf interactions msg = 0
```

Clean/Air/Balance.lean:13-26, Clean 93c9d1ef

```
abbrev BalancedChannel [DecidableEq F] {ens : Ensemble F PublicIO} (witness : EnsembleWitness ens)
  (channel : RawChannel F) : Prop :=
  BalancedInteractions (witness.allTablesWitness.interactionsWith channel)
...
def Statement (ens : Ensemble F PublicIO) (publicInput : PublicIO F) : Prop :=
  ∃ witness : EnsembleWitness ens,
    witness.publicInput = publicInput ∧
    witness.Constraints ∧
    witness.BalancedChannels

-- Soundness: assumptions plus the raw statement imply the spec. -/
def Soundness (ens : Ensemble F PublicIO) (Assumptions Spec : PublicIO F → Prop) : Prop :=
  ∀ publicInput, Assumptions publicInput → ens.Statement publicInput → Spec publicInput
```

Clean/Air/FlatEnsemble.lean:334-337, :361-370, Clean 93c9d1ef

The theorem that assembles per-table soundness into ensemble soundness has a proof that is plain logic; the bus argument is inside `TableSoundness` and the channel lemmas of `Balance.lean` (for example `exists_push_of_pull`, :155-157: every pull has a matching push, derived from `balance` and the side condition).

```
theorem soundness_of_tableSoundness_and_specConsistency (ens : Ensemble F PublicIO)
  (Assumptions Spec : PublicIO F → Prop) :
  ens.TableSoundness →
  ens.AssumptionsConsistency Assumptions →
  ens.SpecConsistency Spec →
  ens.Soundness Assumptions Spec := by
```

Clean/Air/FlatEnsemble.lean:419-424, Clean 93c9d1ef

3.3.5 Why the balance argument is vacuous over a field of characteristic two

The balance is a LogUp-style statement: multiplicities are **summed in the field**. That is sound over a large prime field because a sum of at most $p - 1$ terms ± 1 vanishes only if the counts match; the side condition `length < ringChar F` is that bound. Over $\mathbb{K}$, `ringChar K = 2`:

1. **Without the side condition the sum is unsound.** A message pushed twice (multiplicity 1, twice) and never pulled has `balanceOf = 1 + 1 = 0`. The probe `CleanBalance` proves both statements below (axioms: the standard three and `[propxt]` respectively):

```
def pushedTwice : List (Interaction K) := [push 7, push 7]
theorem pushedTwice_balance (msg : Array K) : balanceOf pushedTwice msg = 0
theorem pushedTwice_not_perm :
  ¬ (pushedTwice.map (·.msg)).Perm ([] : List (Array K))
```

probe `CleanBalance` (probes/lib-others/CleanBalance.lean), run at the old pins; appendix B

In addition $-1 = 1$ in $\mathbb{K}$ (example : `(-1 : K) = 1 := by decide`), so `Channel.toRaw` grants the pull guarantee to every multiplicity-1 push and makes every `Requirements` vacuous (leanerVM issue #16, table row 1, with kernel-checked examples in `tests/LeanerVMTests/Arithmetization/Channels.lean`).

2. **With the side condition, the relation holds of no real bus.** `length < 2` admits at most one interaction per channel, and a balanced singleton has multiplicity 0. The probe `CleanBalance` proves all of the following on the standard axioms:

```
theorem pushedTwice_not_balanced : ¬ BalancedInteractions pushedTwice
theorem length_le_one_of_balanced {l : List (Interaction K)} (h : BalancedInteractions l) :
  l.length ≤ 1
theorem mult_eq_zero_of_balanced_singleton {i : Interaction K}
  (h : BalancedInteractions [i]) : i.mult = 0
theorem honest_pair_not_balanced : ¬ BalancedInteractions [push 7, pull 7]
theorem statement_forces_at_most_one_interaction {PublicIO : TypeMap} [ProvableType PublicIO]
  (ens : Ensemble K PublicIO) (w : EnsembleWitness ens) (h : w.BalancedChannels)
  (c : RawChannel K) (hc : c ∈ ens.channels) :
  (w.allTablesWitness.interactionsWith c).length ≤ 1
```

probe `CleanBalance` (probes/lib-others/CleanBalance.lean), run at the old pins; appendix B

So over $\mathbb{K}$ `Ensemble.Statement` is false for every ensemble whose honest witness puts two interactions on a channel, including the smallest honest bus (one push, one matching pull): Clean’s **Soundness** is then *vacuously* true and its completeness false. That is the precise sense of “vacuous”: the countermodel of item 1 is excluded, but only by excluding every real witness too.

The blueprint’s relation-ladder row (`protocol-blueprint.md:359`) says “Over $\mathbb{K}$ the side condition fails for every real ensemble and a tuple pushed twice and never pulled balances”. The first half is right; the second is true of the field sum, not of `BalancedInteractions`, which rejects that list (`pushedTwice_not_balanced`): the finding finding 125.

3.3.6 State of the tracking items

Read with gh on 2026-09-30.

Table 3.2: The issues and pull requests on Clean’s bus balance and fixed columns.

Item	State	What it is
leanerVM #16	open (updated 2026-09-15)	“leanISA: Clean’s bus argument is vacuous over $\mathbb{K}$, PR Clean for direction tags and multiset balance”: the request; tabulates <code>Channel.toRaw</code> , <code>emit</code> , <code>BalancedInteractions</code> , and the <code>[Fact (ringChar F ≠ 2)]</code> lemmas as degenerate over $\mathbb{K}$.
leanerVM #20	open (updated 2026-09-24)	the Clean-side roadmap: opt-in <code>DirectedChannel</code> (direction as a tag in the message), <code>BalanceModel.multiset</code> (permutation, counted in $\mathbb{N}$), <code>Ensemble.StatementWith model</code> ; Layers 0–4 on Clean #464, Layer 5 (leanerVM adoption) open.
leanerVM #23	open (updated 2026-09-17)	fold the three named fixed-column hypotheses of <code>SatisfiedBy</code> into Clean fixed columns and proof-committed data once Clean #446 lands.
Clean #446	open, draft (updated 2026-08-16)	“Add fixed columns and sound prover data to Flat AIR”.
Clean #452	open issue (not a pull request)	“Model verifier-checked interaction capacity in Flat AIR soundness”.

Item	State	What it is
Clean #464	open, ready for review (updated 2026-09-24)	“Bus balance over binary fields: opt-in directed channels and explicit balance models”; legacy statements unchanged; includes <code>legacy_rejects_every_run</code> over $\mathbb{F}_2$.
Clean #466	open (updated 2026-09-24)	“interpret expressions as bounded-degree polynomials” (the spine’s hole “Clean components as polynomials”, <code>protocol-blueprint.md:604</code>).
Clean #474	open (updated 2026-09-29)	“preserve balance proof across Lean 4.34 normalization”: the new pin 42fe4b26 is this branch merged with Clean main; it ports proofs only. At 42fe4b26 <code>balanceOf</code> and <code>BalancedInteractions</code> are unchanged (Clean/Air/Balance.lean:21-29, identical text), so the vacuity stands at the new pin.

3.3.7 What leanISA does instead, and when the workaround retires

leanISA never consumes `Ensemble.Statement`. Its relation `SatisfiedBy` states balance as a permutation of message lists, counted in $\mathbb{N}$, with direction given by *which channel of a pair* an interaction is on:

```

/-- Pushed and pulled messages of a channel pair form the same multiset (specification §5.1):
a permutation of message lists, counted in `N`, never a field sum (acceptance test 14). -/
def BalancedPair (w : EnsembleWitness (leanISAEnsemble prog)) (pull push : RawChannel K) :
  Prop :=
  (messagesOn w push).Perm (messagesOn w pull)

```

LeanerVM/Arithmetization/Statement.lean:227-231, leanerVM b435631

`BalancedPair` is used three times in `SatisfiedBy` (`Statement.lean:320-324`: `state_balanced`, `mem_balanced`, `bytecode_balanced`). Every interaction has multiplicity 1, so the message multiset is the interaction multiset (`Statement.lean:57-64`; `Channels.lean:58-72` for the channel pairs). `List.Perm` (core: $l_1 \sim l_2$, one list is a reordering of the other) is exact multiset equality with no characteristic condition.

Retirement condition. `BalancedPair` and the channel-pair encoding of direction (`Direction`, `channelDir`) are deleted when (1) Clean #464 is merged into Clean main, (2) leanerVM’s pin reaches a Clean revision containing it, and (3) leanerVM’s channels are rebuilt as `DirectedChannels` and `SatisfiedBy` is stated through `Ensemble.StatementWith BalanceModel.multiset` (leanerVM #20, Layer 5, “a separate pull request”; `Channels.lean:71-72`). Until then the workaround is **a deviation forced by an upstream library**, and its correctness is by inspection of a four-line definition. Neither #446 nor #452 is required for the balance repair (issue #20: “#446 (independent)”).

3.3.8 At the new pin 42fe4b26

Clean is a module library (#460), so `LeanerVM/Parameters/CleanField.lean` became a module at 144c5aa and the “Clean bridge is plain” convention (leanISA status finding C8) retires; the balance proofs were ported (#474); `balanceOf` and `BalancedInteractions` are unchanged, and #464 (directed channels) is not in the pin (a `git grep DirectedChannel` of 42fe4b26 finds nothing), so the vacuity over $\mathbb{K}$ and leanISA’s workaround stand as described above.

3.4 Mathlib and Lean itself: what the statements unfold to, and the trusted base

3.4.1 Mathlib notions in the proof system’s statements

Table 3.3 lists the Mathlib (and core) notions counted in the statements, not the proofs, of `LeanerVM/Protocol/` at b435631 and in the library objects of the previous sections. Definitions are quoted from

Mathlib at the old pin v4.33.1 (0df444a3, read with `git show 0df444a3:<path>`) and core at Lean v4.33.1.

Table 3.3: Mathlib and core notions the proof system’s statements unfold to.

Notion	What it is	Definition (old pin)	Where the proof system meets it
$\mathbb{R}_{\geq 0}, \mathbb{R}_{\geq 0^\infty}$	non-negative reals, and with ∞	<code>def NNReal := { r : ℝ // 0 ≤ r } (Mathlib/Data/NNReal/Defs.lean:58);</code> <code>def ENNReal := WithTop ℝ_{≥0} (Mathlib/Data/ENNReal/Basic.lean:101)</code>	error bounds: <code>piopError : ... → ℝ_{≥0} (Spine/Compose.lean:149);</code> every <code>Pr[...]</code> is an $\mathbb{R}_{\geq 0^\infty}$
PMF, <code>uniformOf Fintype</code>	probability mass function; the uniform one	<code>def PMF (α) := { f : α → ℝ_{≥0} // HasSum f 1 } (Probability/ProbabilityMassFunction/Basic.lean:46-47);</code> <code>uniformOfFintype α := uniformOfFinset Finset.univ ... (Probability/Distributions/Uniform.lean:285-286)</code>	the semantics of every coin (section 3.1.2)
Set α	a predicate on α	<code>core/Mathlib Set α := α → Prop</code>	ArkLib relations: <code>M3Rel I : Set ((I.Stmt × ...) × Column I.μ) (Spine/Instance.lean:230),</code> <code>Seam.of (Seams.lean:166),</code> language sets in <code>Spine/Phase.lean</code>
<code>Fintype α</code> , <code>Fintype.card</code>	a finite type with its list of elements; its size	<code>class Fintype (α) where elems : Finset α; complete : ∀ x, x ∈ elems (Data/Fintype/Defs.lean:57-61);</code> <code>def card (α) [Fintype α] : ℕ := (@univ α _).card (Data/Fintype/Card.lean:39-41)</code>	the size of the challenge space, <code>card_E</code> (section 3.8.1); the counting bounds (section 3.8.5); <code>PublicInput.lean</code>
<code>Finset.sum</code> ($\sum$), <code>Finset.prod</code>	finite sums and products	<code>Mathlib Finset.sum s f</code> (fold over the underlying multiset)	stacking and claim weights: <code>Seams.lean:113</code> ($\sum i : \text{Fin } (2^\mu), W.\text{onCube.get } i * \text{ofK } (q.\text{values.get } i)$), <code>Stack</code> , <code>Padding</code> , <code>ClaimWeights</code> , <code>FixedColumns</code>
<code>List.Perm</code> ($\sim$)	one list is a reordering of the other (multiset equality)	core inductive <code>Perm : List α → List α → Prop</code> with <code>nil</code> , <code>cons</code> , <code>swap</code> , <code>trans</code> (<code>Init/Data/List/Basic.lean:1858-1874</code> , Lean v4.33.1)	bus balance: <code>def Balanced (q) : Prop := (I.tuples q .push).Perm (I.tuples q .pull) (Spine/Instance.lean:201),</code> leanISA’s <code>BalancedPair</code> (section 3.3.7)
<code>MvPolynomial σ R</code>	Mathlib’s multivariate polynomials (non-computable, finitely supported coefficient maps)	<code>abbrev MvPolynomial (σ R) := AddMonoidAlgebra R (σ →₀ ℕ) (Algebra/MvPolynomial/Basic.lean:82-83)</code>	reached through <code>fromCMvPolynomial</code> (section 3.2.2) for <code>totalDegree</code> and Schwartz–Zippel
<code>Polynomial</code> , <code>AdjoinRoot</code> , <code>Irreducible</code>	univariate polynomials, the quotient $R[X]/(f)$, irreducibility	<code>def AdjoinRoot (f : R[X]) := Polynomial R / (span {f}) : Ideal R[X] (RingTheory/AdjoinRoot.lean:62)</code>	the <i>definitions</i> of $\mathbb{K}$ and $\mathbb{E}$ (section 3.2.3), through the bijections <code>toQuot</code>

A reader who knows these notions from mathematics can take them at face value: each is a short definition whose meaning is the textbook one, and Mathlib’s theory about them is proved.

3.4.2 The trusted base of any Lean development

The kernel and the three standard axioms

The **kernel** is Lean’s type checker for its dependent type theory. Every declaration, however produced (by hand, by tactics, by automation), is re-checked by it; tactics are untrusted. **Three standard axioms** are assumed: **propext** (logically equivalent propositions are equal), **Classical.choice** (choice; gives excluded middle) and **Quot.sound** (elements related by a relation have equal classes in its quotient; gives function extensionality). They are the axioms of Mathlib and are consistent relative to standard set theory.

(The probe `NumeralsHazard` shows the kernel rejecting two ill-typed proof terms that `simp` produced: finding 133.)

What would enlarge it. `sorryAx` (an admitted proof: it proves anything); `native_decide`, which adds the axiom `Lean.ofReduceBool` and so trusts the compiler, the interpreter and every `@[implemented_by]/@[extern]` replacement of a definition by native code; any user axiom.

How closed claims are checked, and what each way trusts.

decide The elaborator evaluates the `Decidable` instance, and the kernel re-checks the resulting term `of_decide_eq_true` (`Eq.refl true`) by its own reduction. Trusted: the kernel.

decide +kernel Skips the elaborator’s evaluation and asks the kernel directly (the form `leanerVM` uses for `ℕ` powers, `Parameters/Generator.lean:281-299`). Trusted: the kernel.

#guard e Evaluates `e` by the **compiler or interpreter** and fails the build if it is not `true`. It proves nothing to the kernel and trusts compiled code, including `@[csimp]` replacements (`CompPoly`’s `mul_eq_mulTbl` swaps `Ext` multiplication for a table version; that equation is itself proved, `Extension/Defs.lean:267`) and `core`’s native `Nat/BitVec` code. `leanerVM` uses `#guard` only in tests, as executable evidence.

native_decide Decides the claim by compiled code and records the axiom `Lean.ofReduceBool`: it trusts the compiler, the interpreter and every native replacement of a definition. Banned in `leanerVM` (section 3.4.3).

rfl example ... := rfl is kernel definitional equality. Trusted: the kernel.

3.4.3 What `leanerVM`’s validation enforces (at `b435631`)

- `scripts/audit-lean.sh:8-21`: a lexical ban on `axiom`, `sorry`, `admit`, `unsafe`, `native_decide` in `LeanerVM`, `LeanerVM.lean` and tests, plus `check-lean-options.py` (no local override of linter or implicit-variable options).
- `scripts/audit-axioms.lean:17-29`: for **every** constant whose name starts with `LeanerVM` or `LeanerVMTests`, `collectAxioms` (recursive, through all imported libraries) must return only `propext`, `Classical.choice`, `Quot.sound`.

```
for (name, _) in env.constants.toList do
  if (!LeanerVM).isPrefixOf name || (!LeanerVMTests).isPrefixOf name then
    count := count + 1
    for ax in ← collectAxioms name do
      unless #[\`propext, \`Classical.choice, \`Quot.sound].contains ax do
        unexpected := unexpected.push (name, ax)
```

`scripts/audit-axioms.lean:17-29, leanerVM b435631`

- `scripts/test-axiom-audit.py:28-45` re-runs the audit with a planted axiom `AuditNegative.hidden : False` used by one production and one test theorem, and requires the audit to reject both (a negative control). CI runs the same audit through `lean-action` (`axiom-audit-root: LeanerVM, .github/workflows/ci.yml:40-43`) and the script (`:44-45`); `docs/ci.md:8-9` marks both workflows required.
- `scripts/validate.sh`: the above plus the import DAG, layer and documentation checks, `lake build`, `lake test`, and the test root with warnings as errors.

Consequence. A leanerVM theorem cannot silently rest on one of ArkLib’s admitted theorems (ArkLib’s `sorryAx` would appear in `collectAxioms`), nor on `native_decide` anywhere in its dependency cone. What remains trusted is the kernel, the three axioms, and the *meaning of the definitions* in the statements (sections 3.1.4, 3.2, 3.3 and 3.4.1). `#guard` evidence in tests is compiled-code evidence, never proof. (The audit covers the two project namespaces; an instance declared outside them would escape it. None of Layer 0’s is: `LeanerVM.Protocol.instSampleableTypeK` and the others are in the namespace, per the probe `Layer0`’s output.)

At 144c5aa the pins are Mathlib and Lean v4.34.1; `docs/dependencies.md` at that revision describes the upgrade.

3.5 ArkLib: the objects, from scratch

ArkLib is the library of interactive oracle reductions in which the blueprint states the proof system: a protocol is a schedule of prover messages and verifier challenges, a reduction is a prover and a verifier for it, and the security notions are predicates on verifiers. This section introduces its objects in dependency order. **Every snippet is from ArkLib at the old pin dca90385**, read with `git show` at that commit.

At the new pin 7653a901. The objects below exist with the same shapes (checked by name): `OracleReduction`, `OracleVerifier.toVerifier`, `KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCaseWith`, `GuardedForm`, `OracleReduction.append`, the guarded completeness theorems and `Verifier.GuardedForm.ofEmpty`. Every security definition is restated in VCVio’s new probability notation (section 3.1.5): the diff of `Security/RoundByRound.lean` between the pins is the notation and the lemma names (`probEvent_* → prEvent_*`, `OptionT.prEvent_mk_*`); the mathematical content of `KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCaseWith` and the two implications between them is unchanged. ArkLib now calls this whole framework “legacy” (section 3.7.5).

Reading conventions. A `Type` is a set of values; `Fin n` is $\{0, \dots, n-1\}$; a `structure` is a record; a `class` is a record found automatically by type; `Prop` is a proposition; `OracleComp spec α (VCVio)` is a computation that may ask questions to the oracles named by `spec` and returns an α ; `ProbComp` is `OracleComp` over one uniform-sampling oracle; `OptionT m α` adds a failure outcome to $m \alpha$; `Pr[p | c]` is the probability that the outcome of `c` satisfies `p` (failure is not an outcome; VCVio at f9dc47d9); `StateT σ m` threads a state of type σ . Section 3.1 introduces VCVio’s objects.

3.5.1 Protocols and transcripts

Direction: who speaks in a round

A round is either a prover message (`P_to_V`) or a verifier challenge (`V_to_P`).

```
inductive Direction where
| P_to_V -- Message
| V_to_P -- Challenge
deriving DecidableEq, Inhabited, Repr
```

ArkLib/OracleReduction/Prelude.lean:66-69, ArkLib dca90385

ProtocolSpec n: the schedule of an n -round protocol

For each of the n rounds, who speaks and the type of what is sent.

```
@[ext]
structure ProtocolSpec (n : ℕ) where
/-- The direction of each message in the protocol. -/
dir : Fin n → Direction
/-- The type of each message in the protocol. -/
«Type» : Fin n → Type
deriving Inhabited
```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:30-36, ArkLib dca90385

Message and challenge indices, messages, challenges

MessageIdx and ChallengeIdx are the rounds of each direction, as subsets of Fin n; Message and Challenge are their types.

```

/-- Subtype of `Fin n` for the indices corresponding to messages in a protocol specification -/
@[reducible, simp]
def MessageIdx (pSpec : ProtocolSpec n) :=
  {i : Fin n // pSpec.dir i = Direction.P_to_V}

/-- Subtype of `Fin n` for the indices corresponding to challenges in a protocol specification -/
@[reducible, simp]
def ChallengeIdx (pSpec : ProtocolSpec n) :=
  {i : Fin n // pSpec.dir i = Direction.V_to_P}

```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:50-58, ArkLib dca90385

```

@[reducible, inline, specialize, simp]
def Message (pSpec : ProtocolSpec n) (i : MessageIdx pSpec) := pSpec.«Type» i.val

/-- Unbundled version of `Message`, which supplies the proof separately from the index. -/
@[reducible, inline, specialize, simp]
def Message' (pSpec : ProtocolSpec n) (i : Fin n) (_ : pSpec.dir i = .P_to_V) := pSpec.«Type» i

/-- The type of the `i`-th challenge in a protocol specification -/
@[reducible, inline, specialize, simp]
def Challenge (pSpec : ProtocolSpec n) (i : ChallengeIdx pSpec) := pSpec.«Type» i.val

```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:68-77, ArkLib dca90385

FullTranscript, Transcript k: all entries, and a prefix

FullTranscript is the list of all n entries; Transcript k is its prefix of length k (the full transcript is Transcript (Fin.last n) definitionally).

```

@[reducible, inline, specialize]
def FullTranscript (pSpec : ProtocolSpec n) := (i : Fin n) → pSpec.«Type» i

```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:104-105, ArkLib dca90385

```

@[reducible, inline, specialize]
def Transcript (k : Fin (n + 1)) (pSpec : ProtocolSpec n) : Type :=
  (pSpec[:k.val]).FullTranscript

```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:261-263, ArkLib dca90385

Concatenation of schedules

$\text{pSpec}_1 \text{ ++ } \text{pSpec}_2$ runs the first schedule then the second; seqCompose iterates it over a family.

```

/-- Appending two `ProtocolSpec`s -/
abbrev append (pSpec : ProtocolSpec m) (pSpec' : ProtocolSpec n) : ProtocolSpec (m + n) :=
  (Fin.vappend pSpec.dir pSpec'.dir, Fin.vappend pSpec.Type pSpec'.Type)

@[inherit_doc]
infixl : 65 " ++ " => ProtocolSpec.append

```

ArkLib/OracleReduction/ProtocolSpec/SeqCompose.lean:38-43, ArkLib dca90385

```

@[inline]
def seqCompose {m : ℕ} {n : Fin m → ℕ} (pSpec : ∀ i, ProtocolSpec (n i)) :
  ProtocolSpec (Fin.vsum n) where
  dir := Fin.vflatten (fun i => (pSpec i).dir)
  «Type» := Fin.vflatten (fun i => (pSpec i).Type)

```

ArkLib/OracleReduction/ProtocolSpec/SeqCompose.lean:358-362, ArkLib dca90385

3.5.2 Oracles

OracleInterface M: how a value of type M is queried

A query type and an answer, given as an oracle context over `ReaderM M` (the value is read, the answer computed). `Response q` and `answer m q` are derived. `leanerVM`'s instance for a column has queries in $\mathbb{E}^n$ and answers the multilinear extension (section 3.8.3).

```
@[ext]
class OracleInterface (Message : Type u) where
  Query : Type v
  toOC : OracleContext Query (ReaderM Message)
```

ArkLib/OracleReduction/OracleInterface.lean:54-57, ArkLib dca90385

```
def Response {Message : Type*} [O : OracleInterface Message]
  (q : O.Query) : Type _ :=
  O.toOC.spec q

def spec {Message : Type*} [O : OracleInterface Message] :
  OracleSpec O.Query :=
  O.toOC.spec

@[implicit_reducible]
def answer {Message : Type*} [O : OracleInterface Message]
  (m : Message) (q : O.Query) : O.Response q :=
  (O.toOC.impl q).run m
```

ArkLib/OracleReduction/OracleInterface.lean:67-78, ArkLib dca90385

3.5.3 Provers, verifiers, reductions

Prover: the honest prover as a state machine

A state type per round; input from statement and witness; `sendMessage` and `receiveChallenge` per round (computations over the shared oracle `oSpec`); output, the final statement and witness. Provers of the soundness games are the same structure (with any `WitIn`, `WitOut`).

```
@[ext]
structure ProverState (n : ℕ) where
  PrvState : Fin (n + 1) → Type
```

ArkLib/OracleReduction/Basic.lean:132-134, ArkLib dca90385

```
@[ext]
structure ProverRound {ι : Type} (oSpec : OracleSpec ι) {n : ℕ} (pSpec : ProtocolSpec n)
  extends ProverState n where
  /-- Send a message and update the prover's state -/
  sendMessage (i : MessageIdx pSpec) :
    PrvState i.1.castSucc → OracleComp oSpec (pSpec.Message i × PrvState i.1.succ)
  /-- Receive a challenge and update the prover's state -/
  receiveChallenge (i : ChallengeIdx pSpec) :
    PrvState i.1.castSucc → OracleComp oSpec (pSpec.Challenge i → PrvState i.1.succ)
```

ArkLib/OracleReduction/Basic.lean:154-162, ArkLib dca90385

```
@[ext]
structure Prover {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn WitIn StmtOut WitOut : Type)
  {n : ℕ} (pSpec : ProtocolSpec n) extends
    ProverState n,
    ProverInput StmtIn WitIn (PrvState 0),
    ProverRound oSpec pSpec,
    ProverOutput oSpec (StmtOut × WitOut) (PrvState (Fin.last n))
```

ArkLib/OracleReduction/Basic.lean:222-229, ArkLib dca90385

Verifier: a verdict on a full transcript

A function of the statement and the full transcript, returning the output statement or failing (rejecting).

```
@[ext]
structure Verifier {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn StmtOut : Type) {n : ℕ} (pSpec : ProtocolSpec n) where
  verify : StmtIn → FullTranscript pSpec → OptionT (OracleComp oSpec) StmtOut
```

ArkLib/OracleReduction/Basic.lean:247-250, ArkLib dca90385

OracleProver

A prover whose input statement also carries the contents of the input oracles and whose output carries the output oracles.

```
@[reducible, inline]
def OracleProver {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn : Type) {ιsi : Type} (OStmtIn : ιsi → Type) (WitIn : Type)
  (StmtOut : Type) {ιso : Type} (OStmtOut : ιso → Type) (WitOut : Type)
  {n : ℕ} (pSpec : ProtocolSpec n) :=
  Prover oSpec (StmtIn × (∀ i, OStmtIn i)) WitIn (StmtOut × (∀ i, OStmtOut i)) WitOut pSpec
```

ArkLib/OracleReduction/Basic.lean:255-260, ArkLib dca90385

OracleVerifier: a verifier that only queries

`verify` sees the statement and the challenges and may *query* the input oracles and the prover's messages (it never sees a message whole). `outputOracle` says which oracles it hands on: either an embedding naming, for each output oracle, an input oracle or a message it *is* (`OracleOutputEmbedding`), or a simulation computing a virtual output oracle query by query (`OracleOutputSimulation`).

```
@[ext]
structure OracleVerifier {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn : Type) {ιsi : Type} (OStmtIn : ιsi → Type)
  (StmtOut : Type) {ιso : Type} (OStmtOut : ιso → Type)
  {n : ℕ} (pSpec : ProtocolSpec n)
  [Osi : ∀ i, OracleInterface (OStmtIn i)]
  [Om : ∀ i, OracleInterface (pSpec.Message i)]
  [Oso : ∀ i, OracleInterface (OStmtOut i)]
  where

  /-- The core verification logic. Takes the input statement `stmtIn` and all verifier challenges
  `challenges` (which are determined outside this function, typically by sampling for
  public-coin protocols). Returns the output statement `StmtOut` within an `OracleComp` that has
  access to external oracles `oSpec`, input statement oracles `OStmtIn`, and prover message
  oracles `pSpec.Message`. -/
  verify : StmtIn → pSpec.Challenges →
    OptionT (OracleComp (oSpec + ([OStmtIn]₀ + [pSpec.Message]₀))) StmtOut

  /-- The output family has exactly one semantic representation: either the
  legacy embedded-source form or a virtual query implementation. -/
  outputOracle :
    OracleOutputEmbedding OStmtIn pSpec.Message OStmtOut @
    OracleOutputSimulation oSpec OStmtIn OStmtOut pSpec
```

ArkLib/OracleReduction/Basic.lean:322-344, ArkLib dca90385

```
structure OracleOutputEmbedding {ιsi ιso : Type}
  (OStmtIn : ιsi → Type) {ιm : Type} (Message : ιm → Type)
  (OStmtOut : ιso → Type)
  [Osi : ∀ i, OracleInterface (OStmtIn i)]
  [Om : ∀ i, OracleInterface (Message i)]
  [Oso : ∀ i, OracleInterface (OStmtOut i)] where
  embed : ιso ↪ ιsi @ ιm
  hEq : ∀ i, OStmtOut i = match embed i with
```

```

| Sum.inl j => OStmtIn j
| Sum.inr j => Message j
outputInterface_heq : ∀ i, match embed i with
| Sum.inl j => HEq (Oso i) (Osi j)
| Sum.inr j => HEq (Oso i) (Om j)

```

ArkLib/OracleReduction/Basic.lean:295-307, ArkLib dca90385

toVerifier: the plain verifier of an oracle verifier

It answers the oracle verifier's queries from the actual oracle contents and messages of the transcript, and materialises the output oracles. Every security notion of an oracle verifier is the notion of its `toVerifier` (section 3.6.6).

```

def toVerifier : Verifier oSpec (StmtIn × ∀ i, OStmtIn i) (StmtOut × (∀ i, OStmtOut i)) pSpec where
  verify := fun (stmt, oStmt) transcript => OptionT.mk <|
    Option.map (fun stmtOut =>
      (stmtOut, verifier.materializeOutput
        transcript.challenges oStmt transcript.messages)) <$>
    simulateQ (OracleInterface.simOracle2 oSpec oStmt transcript.messages)
      (verifier.verify stmt transcript.challenges).run

```

ArkLib/OracleReduction/Basic.lean:490-496, ArkLib dca90385

Reduction, OracleReduction, OracleProof

A reduction is a prover and a verifier; an oracle reduction is an oracle prover and an oracle verifier; an oracle proof is an oracle reduction whose output is a Boolean, with no output oracle and no output witness.

```

@[ext]
structure Reduction {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn WitIn StmtOut WitOut : Type) {n : ℕ} (pSpec : ProtocolSpec n) where
  prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec
  verifier : Verifier oSpec StmtIn StmtOut pSpec

```

ArkLib/OracleReduction/Basic.lean:624-628, ArkLib dca90385

```

@[ext]
structure OracleReduction {ι : Type} (oSpec : OracleSpec ι)
  (StmtIn : Type) {ιsi : Type} (OStmtIn : ιsi → Type) (WitIn : Type)
  (StmtOut : Type) {ιso : Type} (OStmtOut : ιso → Type) (WitOut : Type)
  {n : ℕ} (pSpec : ProtocolSpec n)
  [Osi : ∀ i, OracleInterface (OStmtIn i)] [Om : ∀ i, OracleInterface (pSpec.Message i)]
  [Oso : ∀ i, OracleInterface (OStmtOut i)]
  where
  prover : OracleProver oSpec StmtIn OStmtIn WitIn StmtOut OStmtOut WitOut pSpec
  verifier : OracleVerifier oSpec StmtIn OStmtIn StmtOut OStmtOut pSpec

```

ArkLib/OracleReduction/Basic.lean:632-641, ArkLib dca90385

```

@[reducible] def OracleProof {ι : Type} (oSpec : OracleSpec ι)
  (Statement : Type) {ιsi : Type} (OStatement : ιsi → Type) (Witness : Type)
  {n : ℕ} (pSpec : ProtocolSpec n)
  [Osi : ∀ i, OracleInterface (OStatement i)]
  [Om : ∀ i, OracleInterface (pSpec.Message i)] :=
  @OracleReduction ι oSpec Statement ιsi OStatement Witness Bool Empty
  (fun _ : Empty => Unit) Unit n pSpec Osi Om (fun i => nomatch i)

```

ArkLib/OracleReduction/Basic.lean:669-675, ArkLib dca90385

3.5.4 Running a protocol

A run

`Prover.runToRound` folds the rounds: at a challenge round the challenge is obtained by a query to the challenge oracle `[pSpec.Challenge]₀` (one oracle per challenge round, trivial query) and handed to `receiveChallenge`; at a message round `sendMessage` runs. `Reduction.run` runs the prover, then the verifier on the transcript; the verifier's failure makes the run fail.

```
def processRound (j : Fin n)
  (prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec)
  (currentResult : OracleComp (oSpec + [pSpec.Challenge]₀))
  (pSpec.Transcript j.castSucc × prover.PrvState j.castSucc) :
  OracleComp (oSpec + [pSpec.Challenge]₀)
  (pSpec.Transcript j.succ × prover.PrvState j.succ) := do
let (transcript, state) ← currentResult
match hDir : pSpec.dir j with
| .V_to_P => do
  let challenge ← pSpec.getChallenge (j, hDir)
  letI newState := (← prover.receiveChallenge (j, hDir) state) challenge
  return (transcript.concat challenge, newState)
| .P_to_V => do
  let (msg, newState) ← prover.sendMessage (j, hDir) state
  return (transcript.concat msg, newState)
```

ArkLib/OracleReduction/Execution.lean:96-110, ArkLib dca90385

```
def Reduction.run (stmt : StmtIn) (wit : WitIn)
  (reduction : Reduction oSpec StmtIn WitIn StmtOut WitOut pSpec) :
  OptionT (OracleComp (oSpec + [pSpec.Challenge]₀))
  ((FullTranscript pSpec × StmtOut × WitOut) × StmtOut) := do
-- `ctxOut` contains both the output statement and witness after running the prover
let proverResult ← reduction.prover.run stmt wit
let stmtOut ← liftM (reduction.verifier.run stmt proverResult.1).run
return (proverResult, ← stmtOut.getM)
```

ArkLib/OracleReduction/Execution.lean:209-216, ArkLib dca90385

In the security definitions the challenge oracle is implemented by uniform sampling (`challengeQueryImpl`) and the shared oracle `oSpec` by `impl : QueryImpl oSpec (StateT σ ProbComp)` from an initial state drawn by `init : ProbComp σ`. leanerVM's shared oracle is empty (`[]₀`), so `init` and `impl` are inert and the master theorems quantify over them.

```
def challengeQueryImpl {pSpec : ProtocolSpec n} [∀ i, SampleableType (pSpec.Challenge i)] :
  QueryImpl ([pSpec.Challenge]₀ 'challengeOracleInterface) ProbComp :=
  fun q => $† (pSpec.Challenge q.1)
```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:732-734, ArkLib dca90385

3.5.5 Security notions

Completeness is in section 3.6.8; round-by-round knowledge soundness, the notion of the master theorem, is in section 3.6. The others:

soundness

For every prover and every statement outside the input language, the run ends in the output language with probability at most the error; failure counts for the verifier.

```
def soundness (langIn : Set StmtIn) (langOut : Set StmtOut)
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (soundnessError : ℝ≥0) : Prop :=
  ∀ WitIn WitOut : Type,
  ∀ witIn : WitIn,
  ∀ prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec,
  ∀ stmtIn ∉ langIn,
  let pImpl : QueryImpl (oSpec + [pSpec.Challenge]₀) (StateT σ ProbComp) :=
```

```

impl.addLift challengeQueryImpl
letI reduction := Reduction.mk prover verifier
Pr[fun (_, stmtOut) => stmtOut ∈ langOut | OptionT.mk do
  (simulateQ pImpl (reduction.run stmtIn witIn).run).run' (← init)] ≤ soundnessError

```

ArkLib/OracleReduction/Security/Basic.lean:272-283, ArkLib dca90385

Extractor.Straightline, knowledgeSoundness

A straight-line extractor gets the statement, the output witness, the transcript and both query logs and returns an input witness (or fails). Knowledge soundness bounds the probability that the run's output is in the output relation while the extracted input witness is not in the input relation (an extractor that fails counts against it, :329-342).

```

def Straightline :=
  StmtIn → -- input statement
  WitOut → -- output witness
  FullTranscript pSpec → -- reduction transcript
  QueryLog oSpec → -- prover's query log
  QueryLog oSpec → -- verifier's query log
  OptionT (OracleComp oSpec) WitIn -- input witness

```

ArkLib/OracleReduction/Security/Basic.lean:248-254, ArkLib dca90385

```

def knowledgeSoundness (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec) (knowledgeError : ℝ≥0) : Prop :=
  ∃ extractor : Extractor.Straightline oSpec StmtIn WitIn WitOut pSpec,
  ∀ stmtIn : StmtIn,
  ∀ witIn : WitIn,
  ∀ prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec,
  let pImpl : QueryImpl (oSpec + [pSpec.Challenge]) (StateT σ ProbComp) :=
    impl.addLift challengeQueryImpl
  let exec := do
    let ((transcript, (_, witOut)), stmtOut), proveQueryLog, verifyQueryLog
    ← (Reduction.mk prover verifier).runWithLog stmtIn witIn
    let extractedWitIn? ←
      liftM (extractor stmtIn witOut transcript proveQueryLog.fst verifyQueryLog).run
    return (stmtIn, extractedWitIn?, stmtOut, witOut)
  Pr[fun (stmtIn, extractedWitIn?, stmtOut, witOut) =>
    (∀ extractedWitIn ∈ extractedWitIn?, (stmtIn, extractedWitIn) ∈ relIn) ∧
    (stmtOut, witOut) ∈ relOut
    | OptionT.mk do (simulateQ pImpl exec.run).run' (← init)] ≤ knowledgeError

```

ArkLib/OracleReduction/Security/Basic.lean:344-361, ArkLib dca90385

Verifier.StateFunction, rbrSoundness

Round-by-round soundness (section 3.6.3): a predicate on partial transcripts, true at the start exactly on the language, preserved by prover messages when false, false at the end unless the verifier can output a statement of the output language, and made true by a challenge with small probability. `rbrSoundnessWorstCase` (Security/RoundByRound.lean:517-527) is its per-prefix form.

```

structure StateFunction
  (langIn : Set StmtIn) (langOut : Set StmtOut)
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  where
  toFun : (m : Fin (n + 1)) → StmtIn → Transcript m pSpec → Prop
  /-- For all input statement not in the language, the state function is false for that statement
  and the empty transcript -/
  toFun_empty : ∀ stmt, stmt ∈ langIn → toFun 0 stmt default
  /-- If the state function is false for a partial transcript, and the next message is from the
  prover to the verifier, then the state function is also false for the new partial transcript
  regardless of the message -/
  toFun_next : ∀ m, pSpec.dir m = .P_to_V →
    ∀ stmt tr, → toFun m.castSucc stmt tr →
    ∀ msg, → toFun m.succ stmt (tr.concat msg)
  /-- If the state function is false for a full transcript, the verifier will not output a statement
  in the output language -/
  toFun_full : ∀ stmt tr, → toFun (.last n) stmt tr →
    Pr[(· ∈ langOut) | OptionT.mk do (simulateQ impl (verifier.run stmt tr)).run' (← init)] = 0

```

ArkLib/OracleReduction/Security/RoundByRound.lean:135-152, ArkLib dca90385

```

def rbrSoundness (langIn : Set StmtIn) (langOut : Set StmtOut)
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (rbrSoundnessError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∃ stateFunction : verifier.StateFunction init impl langIn langOut,
  ∀ stmtIn ∉ langIn,
  ∀ WitIn WitOut : Type,
  ∀ witIn : WitIn,
  ∀ prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec,
  ∀ i : pSpec.ChallengeIdx,
  Pr[fun (transcript, challenge) =>
    ¬ stateFunction i.1.castSucc stmtIn transcript ∧
    stateFunction i.1.succ stmtIn (transcript.concat challenge)
  | do
    (simulateQ (impl.addLift challengeQueryImpl : QueryImpl _ (StateT o ProbComp))
      (do
        let (transcript, _) ← prover.runToRound i.1.castSucc stmtIn witIn
        let challenge ← liftComp (pSpec.getChallenge i) _
        return (transcript, challenge))).run' (← init)] ≤
    rbrSoundnessError i

```

ArkLib/OracleReduction/Security/RoundByRound.lean:347-365, ArkLib dca90385

KnowledgeStateFunction, Extractor.RoundByRound and the round-by-round knowledge notions

Defined and discussed in section 3.6.2. The existential worst-case form quantifies the intermediate witness types, the extractor and the knowledge state function existentially:

```

def rbrKnowledgeSoundnessWorstCase (relIn : Set (StmtIn × WitIn))
  (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (rbrKnowledgeError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∃ WitMid : Fin (n + 1) → Type,
  ∃ extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid,
  ∃ kSF : verifier.KnowledgeStateFunction init impl relIn relOut extractor,
  ∀ stmtIn : StmtIn,
  ∀ i : pSpec.ChallengeIdx,
  ∀ transcript : Transcript i.1.castSucc pSpec,
  Pr[fun challenge =>
    ∃ witMid,
    ¬ kSF i.1.castSucc stmtIn transcript
      (extractor.extractMid i.1 stmtIn (transcript.concat challenge) witMid) ∧
    kSF i.1.succ stmtIn (transcript.concat challenge) witMid
  | $t (pSpec.Challenge i)] ≤ rbrKnowledgeError i

```

ArkLib/OracleReduction/Security/RoundByRound.lean:534-549, ArkLib dca90385

Verifier.GuardedForm, Prover.OutputIsPure

A guarded verifier equals “if check then pure (out ...) else fail”, with check and out as data. A prover’s output is pure when it is a function of its final state with no oracle query.

```

def IsGuardedWith (V : Verifier oSpec StmtIn StmtOut pSpec)
  (check : StmtIn → FullTranscript pSpec → Bool)
  (out : StmtIn → FullTranscript pSpec → StmtOut) : Prop :=
  ∀ stmt tr, V.verify stmt tr = if check stmt tr then pure (out stmt tr) else failure

```

ArkLib/OracleReduction/Security/CoordinateWiseSpecialSoundness/Guarded.lean:86-89, ArkLib dca90385

```

structure GuardedForm (V : Verifier oSpec StmtIn StmtOut pSpec) where
  /-- The runtime guard. -/
  check : StmtIn → FullTranscript pSpec → Bool
  /-- The verdict where the guard passes. -/
  out : StmtIn → FullTranscript pSpec → StmtOut
  /-- The verifier is guarded with exactly these. -/
  verify_eq : V.IsGuardedWith check out

```

ArkLib/OracleReduction/Security/CoordinateWiseSpecialSoundness/Guarded.lean:112-118, ArkLib dca90385

```

/-- The prover's output is a deterministic function of its final private state, with no
oracle queries. This condition does not constrain the message-sending steps. -/
class Prover.OutputIsPure (P : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec) where
  output_is_pure :  $\exists$  output :  $\_ \rightarrow \_$ ,  $\forall$  st, P.output st = pure (output st)

```

ArkLib/OracleReduction/Basic.lean:989-992, ArkLib dca90385

3.5.6 Composition

append of provers, verifiers and reductions

`Prover.append` (Append/Basic.lean:58-148) runs the first prover and feeds its output to the second prover's input at the seam; `Verifier.append` runs the first verifier on the first half of the transcript and the second on the second half at the first's output; `OracleVerifier.append` (:619-629) routes the second verifier's queries through the first's output-oracle simulation; `OracleReduction.append` pairs them; `seqCompose` (General.lean:255-268) iterates.

```

def Verifier.append (V1 : Verifier oSpec Stmt1 Stmt2 pSpec1)
  (V2 : Verifier oSpec Stmt2 Stmt3 pSpec2) :
  Verifier oSpec Stmt1 Stmt3 (pSpec1 ++p pSpec2) where
  verify := fun stmt transcript => do
    return  $\leftarrow$  V2.verify ( $\leftarrow$  V1.verify stmt transcript.fst) transcript.snd

/-- Compose the component provers and verifiers of two reductions. -/
def Reduction.append (R1 : Reduction oSpec Stmt1 Wit1 Stmt2 Wit2 pSpec1)
  (R2 : Reduction oSpec Stmt2 Wit2 Stmt3 Wit3 pSpec2) :
  Reduction oSpec Stmt1 Wit1 Stmt3 Wit3 (pSpec1 ++p pSpec2) where
  prover := Prover.append R1.prover R2.prover
  verifier := Verifier.append R1.verifier R2.verifier

```

ArkLib/OracleReduction/Composition/Sequential/Append/Basic.lean:236-247, ArkLib dca90385

```

def OracleReduction.append (R1 : OracleReduction oSpec Stmt1 OStmt1 Wit1 Stmt2 OStmt2 Wit2 pSpec1)
  (R2 : OracleReduction oSpec Stmt2 OStmt2 Wit2 Stmt3 OStmt3 Wit3 pSpec2) :
  OracleReduction oSpec Stmt1 OStmt1 Wit1 Stmt3 OStmt3 Wit3 (pSpec1 ++p pSpec2) where
  prover := Prover.append R1.prover R2.prover
  verifier := OracleVerifier.append R1.verifier R2.verifier

```

ArkLib/OracleReduction/Composition/Sequential/Append/Basic.lean:709-713, ArkLib dca90385

Extractor.RoundByRound.append, Verifier.StateFunction.append

The appended extractor runs the second extractor on the second half at the statement `verify s tr1` that a *given* verdict function assigns, then the first on the first half; at the seam it hands the second extractor's round-0 witness to the first's `extractOut` (Append/StateFunction.lean:75-139). The appended (language) state function is disjunctive past the seam (:292, docstring :17-19). The knowledge version is leanerVM's port (section 3.7.4).

```

/-- Compose round-by-round extractors using a deterministic intermediate-statement function.
The composed witness family retains the first extractor's final witness at the boundary. -/
def RoundByRound.append
  {WitMid1 : Fin (m + 1)  $\rightarrow$  Type} {WitMid2 : Fin (n + 1)  $\rightarrow$  Type}
  (E1 : Extractor.RoundByRound oSpec Stmt1 Wit1 Wit2 pSpec1 WitMid1)
  (E2 : Extractor.RoundByRound oSpec Stmt2 Wit2 Wit3 pSpec2 WitMid2)
  (verify : Stmt1  $\rightarrow$  pSpec1.FullTranscript  $\rightarrow$  Stmt2) :
  Extractor.RoundByRound oSpec Stmt1 Wit1 Wit3 (pSpec1 ++p pSpec2)
  (Fin.append (m := m + 1) WitMid1 (Fin.tail WitMid2)  $\circ$  Fin.cast (by omega)) where

```

ArkLib/OracleReduction/Composition/Sequential/Append/StateFunction.lean:73-81, ArkLib dca90385

The composition of completeness (`append_perfectCompleteness_of_guarded_verifiers`, `append_completeness_of_guarded_verifiers`, `seqCompose_*`) is in section 3.6.8; all are proved at both pins.

3.5.7 Fiat–Shamir and commitments

Reduction.fiatShamir, Verifier.fiatShamir, the challenge oracle

The “slow” Fiat–Shamir transformation: the challenge of round i is the answer of an oracle whose query is the statement together with all messages before round i (`challengeOracleInterfaceSR`); the transformed verifier receives all messages in one round, derives the challenges by querying that oracle and runs the original verifier.

```
@[reducible, inline, specialize]
def challengeOracleInterfaceSR (StmtIn : Type) (pSpec : ProtocolSpec n) :
  ∀ i, OracleInterface (pSpec.Challenge i) := fun i =>
  { Query := StmtIn × pSpec.MessagesUpTo i.1.castSucc
    toOC.spec := fun _ => pSpec.Challenge i
    toOC.impl := fun _ => read }
```

ArkLib/OracleReduction/ProtocolSpec/Basic.lean:829-834, ArkLib dca90385

```
/-- The (slow) Fiat-Shamir transformation for the verifier. -/
def Verifier.fiatShamir (V : Verifier oSpec StmtIn StmtOut pSpec) :
  NonInteractiveVerifier (∀ i, pSpec.Message i) (oSpec + fsChallengeOracle StmtIn pSpec)
  StmtIn StmtOut where
  verify := fun stmtIn proof => do
    let messages : pSpec.Messages := proof 0
    let transcript ← (messages.deriveTranscriptFS (oSpec := oSpec) stmtIn)
    Option.getM (← (V.verify stmtIn transcript).run)

/-- The Fiat-Shamir transformation for an (interactive) reduction, which consists of applying the
Fiat-Shamir transformation to both the prover and the verifier. -/
def Reduction.fiatShamir (R : Reduction oSpec StmtIn WitIn StmtOut WitOut pSpec) :
  NonInteractiveReduction (∀ i, pSpec.Message i) (oSpec + fsChallengeOracle StmtIn pSpec)
  StmtIn WitIn StmtOut WitOut where
  prover := R.prover.fiatShamir
  verifier := R.verifier.fiatShamir
```

ArkLib/OracleReduction/FiatShamir/Basic.lean:129-144, ArkLib dca90385

Commitment.Scheme, binding, extractability

Keys, `commit : ComKey → Data → OracleComp (Commitment × Decommitment)`, and an opening *proof* for the statement “commitment, query, response” with witness “data, decommitment”. **binding** bounds any adversary’s probability of opening one commitment to two different responses at the same query; **extractability** is a placeholder (its body is `False`, at both pins: `Commitments/Functional/Basic.lean:250-255` at `dca90385`, `:249-254` at `7653a901`).

```
/-- Key generation for a commitment scheme, producing a committer key and a verifier key. -/
structure KeyGen where
  keygen : OracleComp oSpec (ComKey × VerifKey)

/-- The commitment algorithm, parameterized by the committer key and the data to commit. -/
structure Commit where
  commit : ComKey → Data → OracleComp oSpec (Commitment × Decommitment)

variable [0 : OracleInterface Data] {n : ℕ} (pSpec : ProtocolSpec n)

/-- The opening protocol used to prove a claimed oracle response for committed data. -/
structure Opening where
  opening : (ComKey × VerifKey) →
    Proof oSpec (Commitment × (q : 0.Query) × 0.Response q) (Data × Decommitment) pSpec

/-- A commitment scheme with key generation, commitment, and opening algorithms. -/
structure Scheme extends
  KeyGen oSpec ComKey VerifKey,
  Commit oSpec Data Commitment Decommitment ComKey,
  Opening oSpec Data Commitment Decommitment ComKey VerifKey pSpec
```

ArkLib/Commitments/Functional/Basic.lean:51-70, ArkLib dca90385


```
def binding (bindingError : ℝ≥0) : Prop :=
  ∀ AuxState : Type,
  ∀ adversary : BindingAdversary oSpec Data Commitment AuxState pSpec ComKey,
  bindingExperiment init impl scheme AuxState adversary ≤ bindingError
```

ArkLib/Commitments/Functional/Basic.lean:220-223, ArkLib dca90385

3.5.8 Generic components and the sumcheck specification

Generic components

DoNothing: the identity reduction (`ProofSystem/Component/DoNothing.lean:33-66`). **ReduceClaim**: zero rounds, maps the statement and the witness (`ReduceClaim.lean:50-66`; oracle version :282-323). **CheckClaim**: zero rounds; the plain verifier guards a decidable predicate (`CheckClaim.lean:72-73`); the oracle verifier is a pass-through (:228-231). **RandomQuery**: one challenge, a query; the verifier outputs the query and both oracles; the output relation says the two oracles agree there (`RandomQuery.lean:45-57, :94-111`). **SendSingleWitness**: one prover message, the witness, which becomes an oracle (`SendWitness.lean:316-343`; completeness admitted, :440-443).

```
-- The verifier for the `ReduceClaim` reduction. -/
def verifier : Verifier oSpec StmtIn StmtOut !p[] where
  verify := fun stmt _ => pure (mapStmt stmt)

-- The reduction for the `ReduceClaim` reduction. -/
def reduction : Reduction oSpec StmtIn WitIn StmtOut WitOut !p[] where
```

ArkLib/ProofSystem/Component/ReduceClaim.lean:61-66, ArkLib dca90385

```
@[inline, specialize]
def verifier : Verifier oSpec Statement Statement !p[] where
  verify := fun stmt _ => do guard (pred stmt); return stmt
```

ArkLib/ProofSystem/Component/CheckClaim.lean:71-73, ArkLib dca90385

```
@[reducible, simp]
def relIn : Set ((StmtIn × ∀ i, OStmtIn OStatement i) × WitIn) :=
  { {(), oracles}, () | oracles 0 = oracles 1 }

--
The output relation states that if the verifier's single query was `q`, then
`a` and `b` agree on that `q`, i.e. `answer a q = answer b q`.
-/
@[reducible, simp]
def relOut : Set ((StmtOut OStatement × ∀ i, OStmtOut OStatement i) × WitOut) :=
  { {q, oStmt}, () | answer (oStmt 0) q = answer (oStmt 1) q }

@[reducible]
def pSpec : ProtocolSpec 1 := (!v[.V_to_P], !v[Query OStatement])
```

ArkLib/ProofSystem/Component/RandomQuery.lean:44-57, ArkLib dca90385

The sumcheck specification

A round's statement is the running target and the challenges so far; the oracle statement is a multivariate polynomial of individual degree at most **deg** in n variables; the round relation says the sum of the polynomial over the remaining cube (D^{n-i}) at the challenges equals the target; a round is “prover sends a univariate of degree at most **deg**, verifier sends a field element”; the whole protocol is the `seqCompose` of n rounds. Both the completeness and the round-by-round knowledge soundness of the specification are admitted at both pins (section 3.7.3). **SumcheckDomain** (`Domain.lean:55-59`) generalises the evaluation domain to a per-coordinate family.

```

structure StatementRound (i : Fin (n + 1)) where
  -- The target value for sum-check
  target : R
  -- The challenges sent from the verifier to the prover from previous rounds
  challenges : Fin i → R

abbrev OutputStatement := StatementRound R _ (.last n)

/-- Oracle statement for sum-check, which is a multivariate polynomial over `n` variables of
    individual degree at most `deg`, equipped with the poly evaluation oracle interface. -/
@[reducible]
def OracleStatement : Unit → Type := fun _ => R[≤ deg][X Fin n]

/-- The sum-check relation for the `i`-th round, for `i ≤ n` -/
def relationRound (i : Fin (n + 1)) :
  Set (((StatementRound R n i) × (∀ i, OracleStatement R n deg i)) × Unit) :=

```

ArkLib/ProofSystem/Sumcheck/Spec/SingleRound.lean:130-145, ArkLib dca90385

```

namespace SingleRound

/-- The protocol specification for a single round of sum-check.
Has the form `(!v[.P_to_V, .V_to_P], !v[R[≤ deg][X], R])` -/
@[reducible]
def pSpec : ProtocolSpec 2 :=

```

ArkLib/ProofSystem/Sumcheck/Spec/SingleRound.lean:149-154, ArkLib dca90385

```

def pSpec : ProtocolSpec (Fin.vsum (fun _ : Fin n => 2)) :=
  ProtocolSpec.seqCompose (fun _ => SingleRound.pSpec R deg)
  -- n * 2
  -- fun i => if i % 2 = 0 then (.P_to_V, R[≤ d][X]) else (.V_to_P, R)

```

ArkLib/ProofSystem/Sumcheck/Spec/General.lean:129-133, ArkLib dca90385

```

/-- Perfect completeness for the (full) sum-check protocol -/
theorem reduction_perfectCompleteness :
  (reduction R deg D n oSpec).perfectCompleteness init impl
    (relationRound R n deg D 0) (relationRound R n deg D (.last n)) :=
  Reduction.seqCompose_perfectCompleteness_of_guarded_verifiers
    (fun i => StatementRound R n i × ∀ j, OracleStatement R n deg j)
    (fun _ => Unit) init impl (relationRound R n deg D)
    (SingleRound.reduction R n deg D oSpec)
    (fun _ => inferInstance) (SingleRound.verifierGuardedForm R n deg D oSpec)
    (fun i s => SingleRound.reduction_perfectCompleteness (init := pure s) i)

/-- Round-by-round knowledge soundness with error `deg / |R|` per challenge for the (full)
    sum-check protocol -/
theorem oracleVerifier_rbrKnowledgeSoundness [Fintype R] :
  (oracleVerifier R deg D n oSpec).rbrKnowledgeSoundness init impl
    (relationRound R n deg D 0) (relationRound R n deg D (.last n))
    (fun _ => (deg : ℝ≥0) / (Fintype.card R)) :=
  OracleVerifier.seqCompose_rbrKnowledgeSoundness
    (rel := relationRound R n deg D)

```

ArkLib/ProofSystem/Sumcheck/Spec/General.lean:209-228, ArkLib dca90385

3.6 ArkLib: what round-by-round knowledge soundness guarantees

What the soundness master theorem means

`piop_rbrKnowledgeSoundness` says: *for every public input and every transcript, if the verifier accepts then the stack sent as the first message satisfies `M3Holds`, unless some challenge of the transcript lies in the bad set of its prefix, and every bad set has probability at most `piopError` of its round.* The probability is over one uniform challenge at a time, at every prefix (the worst-case form); no prover appears. The step from this to “an interactive prover convinces the verifier without a good first message with probability at most $\sum \text{piopError}$ ” is a union bound that `ArkLib` admits at both pins (`rbrKnowledgeSoundness_implies_knowledgeSoundness`); the step to Fiat–Shamir security is stated nowhere in `ArkLib`. At the oracle-protocol level this is soundness with respect to the committed oracle; knowledge (extracting the stack from a commitment) belongs to the compiled protocol, for which `ArkLib` offers no theorem (`BCSTransform` commented out, `Commitment.extractability` a placeholder whose body is `False`).

This section derives that reading. Revisions: `ArkLib` at the old pin `dca90385` for every snippet, `leanerVM` at `b435631`; every probe ran at the old pin and none could be re-run after the upgrade (appendix B).

3.6.1 The statement under review

`leanerVM`’s soundness master theorem is an instance of one `ArkLib` predicate, `Verifier.rbrKnowledgeSoundnessWorstCaseWith`, applied to the verifier of the composed oracle protocol after it has been turned into a plain verifier (`toVerifier`, section 3.6.6):

```
-- **Round-by-round knowledge soundness** at `piopError P`, for the extractor
`piopExtractor P S` (the commit phase's, then the phases') and its knowledge state function,
composed from the phases' own by a proved composition: each challenge can turn the state from
false to true with probability at most its error. The extractor's stack satisfies `M3Holds` (see
the module docstring for the plain reading and what it waits on). -/
theorem piop_rbrKnowledgeSoundness (P : Phases I) (S : P.Security) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl []σ (StateT σ ProbComp)) :
  (leanVmVerifier P).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (M3Rel I)
  (Seam.done I) S.toDef.witMid (piopExtractor P S) (S.toDef.kSF init impl)
  (piopError P) :=
  S.toDef.rbr init impl
```

`LeanerVM/Protocol/Spine/Compose.lean:167-177`, `leanerVM b435631`

Its arguments are: the input relation `M3Rel I` (pairs of a public input and a stack `q`), the output relation `Seam.done I` (everything), the family of intermediate witness types `S.toDef.witMid`, the extractor `piopExtractor P S`, the knowledge state function `S.toDef.kSF init impl`, and the error per challenge `piopError P`. The shared oracle is empty (`[]σ`).

3.6.2 The definition, in mathematical language

Setting. A protocol has n rounds. Each round is either a prover message of a type M_k or a verifier challenge of a finite type C_k . A *prefix* $\tau \in \text{Tr}_k$ is the list of the first k entries (messages and challenges); Tr_n is the set of full transcripts. Statements are $s \in S$, input witnesses W_{in} , output statements T , output witnesses W_{out} . The relations are $R_{\text{in}} \subseteq S \times W_{\text{in}}$ and $R_{\text{out}} \subseteq T \times W_{\text{out}}$. The verifier is a function $V : S \times \text{Tr}_n \rightarrow (\text{computation with failure}) T$; *accepting* means returning some $t \in T$, and *rejecting* means failing.

The round-by-round extractor (`Extractor.RoundByRound`)

A family of intermediate witness types $W_0, \dots, W_n$ with $W_0 = W_{\text{in}}$, and functions

$$\text{extractOut} : S \times \text{Tr}_n \times W_{\text{out}} \rightarrow W_n, \quad \text{extractMid}_k : S \times \text{Tr}_{k+1} \times W_{k+1} \rightarrow W_k \quad (k < n),$$

from the output witness to the last intermediate witness, and one round backwards. It has no law of its own. It is given no query log, no access to the shared oracle and no randomness, and nothing bounds its running time.

```

structure RoundByRound
  (oSpec : OracleSpec 1) (StmtIn WitIn WitOut : Type) {n : ℕ} (pSpec : ProtocolSpec n)
  (WitMid : Fin (n + 1) → Type) where
  /-- The first intermediate witness type is equal to the input witness type -/
  eqIn : WitMid 0 = WitIn
  /-- Extract intermediate witness for round `m` from intermediate witness for round `m+1`,
  using the transcript up to round `m+1` -/
  extractMid : (m : Fin n) → StmtIn → Transcript m.succ pSpec → WitMid m.succ → WitMid m.castSucc
  /-- Construct the intermediate witness for the final round from the output witness -/
  extractOut : StmtIn → FullTranscript pSpec → WitOut → WitMid (.last n)

```

ArkLib/OracleReduction/Security/RoundByRound.lean:77-86, ArkLib dca90385

The knowledge state function (Verifier.KnowledgeStateFunction)

A family of predicates $K_k \subseteq S \times \text{Tr}_k \times W_k$, $k = 0, \dots, n$, with three laws.

1. *Start* (toFun_empty): for all s and $w \in W_0$,

$$(s, w) \in R_{\text{in}} \iff K_0(s, \emptyset, w).$$

2. *Prover rounds* (toFun_next): for every round k that is a prover message, all $s, \tau \in \text{Tr}_k$, messages $m \in M_k$ and $w \in W_{k+1}$,

$$K_{k+1}(s, \tau \| m, w) \implies K_k(s, \tau, \text{extractMid}_k(s, \tau \| m, w)).$$

No prover message can make the state true unless the witness extracted one round back was already good.

3. *End* (toFun_full): for all s , full τ and w_{out} ,

$$\Pr[V(s, \tau) \text{ returns some } t \text{ with } (t, w_{\text{out}}) \in R_{\text{out}}] > 0 \implies K_n(s, \tau, \text{extractOut}(s, \tau, w_{\text{out}})).$$

The probability is over the initial state of the shared oracle (drawn by `init`) and the oracle's answers (`impl`); for a verifier that makes no query to a shared oracle it is 0 or 1.

```

structure KnowledgeStateFunction
  (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  {WitMid : Fin (n + 1) → Type}
  (extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid)
  where
  /-- The knowledge state function: takes in round index, input statement, transcript up to that
  round, and intermediate witness of that round, and returns True/False. -/
  toFun : (m : Fin (n + 1)) → StmtIn → Transcript m pSpec → WitMid m → Prop
  /-- The input statement and witness are in the input relation if and only if the state function is
  true for the empty transcript and the input witness -/
  toFun_empty : ∀ stmtIn witMid,
    (stmtIn, cast extractor.eqIn witMid) ∈ relIn ↔ toFun 0 stmtIn default witMid
  /-- If the state function is true for a partial transcript extended with a prover message, then
  the state function is also true for the original partial transcript with the extracted
  intermediate witness -/
  toFun_next : ∀ m, pSpec.dir m = .P_to_V →
    ∀ stmtIn tr msg witMid, toFun m.succ stmtIn (tr.concat msg) witMid →
      toFun m.castSucc stmtIn tr (extractor.extractMid m stmtIn (tr.concat msg) witMid)
  /-- If the verifier can output a statement `stmtOut` that is in the output relation with some
  output witness `witOut`, then the state function is true for the full transcript and the
  extracted last middle witness. -/
  toFun_full : ∀ stmtIn tr witOut,
    Pr[fun stmtOut => (stmtOut, witOut) ∈ relOut
      | OptionT.mk do (simulateQ impl (verifier.run stmtIn tr)).run' (← init)] > 0 →
      toFun (.last n) stmtIn tr (extractor.extractOut stmtIn tr witOut)

```

ArkLib/OracleReduction/Security/RoundByRound.lean:164-189, ArkLib dca90385

The bound (Verifier.rbrKnowledgeSoundnessWorstCaseWith)

For a statement s , a verifier round k and a prefix $\tau \in \text{Tr}_k$, call *bad set* the set of challenges

$$\text{Bad}_k(s, \tau) = \{ c \in C_k : \exists w \in W_{k+1}, K_{k+1}(s, \tau \| c, w) \wedge \neg K_k(s, \tau, \text{extractMid}_k(s, \tau \| c, w)) \}.$$

The predicate says: for **every** statement s , **every** verifier round k and **every** prefix $\tau \in \text{Tr}_k$,

$$\Pr_{c \leftarrow C_k \text{ uniform}} [c \in \text{Bad}_k(s, \tau)] \leq \varepsilon_k.$$

```

/-- Worst-case-per-prefix RBR knowledge soundness for one exact
intermediate-witness family, extractor, and knowledge-state function. -/
def rbrKnowledgeSoundnessWorstCaseWith
  (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (WitMid : Fin (n + 1) → Type)
  (extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid)
  (kSF : verifier.KnowledgeStateFunction init impl relIn relOut extractor)
  (rbrKnowledgeError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∀ stmtIn : StmtIn,
  ∀ i : pSpec.ChallengeIdx,
  ∀ transcript : Transcript i.1.castSucc pSpec,
  Pr[fun challenge =>
    ∃ witMid,
      ¬ kSF i.1.castSucc stmtIn transcript
        (extractor.extractMid i.1 stmtIn (transcript.concat challenge) witMid) ∧
        kSF i.1.succ stmtIn (transcript.concat challenge) witMid
    | $ (pSpec.Challenge i)] ≤ rbrKnowledgeError i

```

ArkLib/OracleReduction/Security/RoundByRound.lean:551-568, ArkLib dca90385

What is quantified over, and what is not.

Object	Quantified how
Statements s	universally, including the statements that have a valid witness
Verifier rounds k	universally; each has its own error ε_k
Prefixes τ	universally: every list of prover messages and earlier challenges, whether or not any prover could reach it with noticeable probability
Intermediate witnesses w	existentially inside the event: the worst one counts
The challenge c	the only random variable: uniform on C_k (the <code>SampleableType</code> instance; its two laws, full support and equal probabilities, force the uniform distribution on a finite type, <code>VCVio/OracleComp/Constructions/SampleableType.lean:44-47</code>)
Provers	not at all : no prover, no prover state and no input witness appears
The shared oracle	only inside law 3 (can the verifier accept). The state function and the extractor cannot depend on the oracle's state or on a query log
WitMid, the extractor, the state function	named arguments of the predicate (“With”); in <code>rbrKnowledgeSoundnessWorstCase</code> they are existentially quantified

What “worst case” adds. ArkLib’s older predicate `rbrKnowledgeSoundness` bounds the same event in a game: a prover is run against the challenger up to round k , which produces the prefix, then the challenge is drawn, and the bound is on the probability over the whole game (so it is an average over the earlier challenges). It quantifies over provers, and the prefix is a random variable:

```

def rbrKnowledgeSoundness (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (rbrKnowledgeError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∃ WitMid : Fin (n + 1) → Type,
  ∃ extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid,
  ∃ kSF : verifier.KnowledgeStateFunction init impl relIn relOut extractor,
  ∀ stmtIn : StmtIn,
  ∀ witIn : WitIn,
  ∀ prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec,

```

```

∀ i : pSpec.ChallengeIdx,
  Pr[fun (transcript, challenge, _proveQueryLog) =>
    ∃ witMid,
      ¬ KSF i.1.castSucc stmtIn transcript
        (extractor.extractMid i.1 stmtIn (transcript.concat challenge) witMid) ∧
        KSF i.1.succ stmtIn (transcript.concat challenge) witMid
  ] do
    (simulateQ (impl.addLift challengeQueryImpl : QueryImpl _ (StateT σ ProbComp))
      (do
        let ((transcript, _), proveQueryLog) ← prover.runWithLogToRound i.1.castSucc stmtIn witIn
        let challenge ← liftComp (pSpec.getChallenge i) _
        return (transcript, challenge, proveQueryLog))).run' (← init)] ≤
  rbrKnowledgeError i

```

ArkLib/OracleReduction/Security/RoundByRound.lean:416-437, ArkLib dca90385

The worst-case form implies the averaged one with the same errors (proved: `Verifier.rbrKnowledgeSoundnessWorstCaseWith_implies_rbrKnowledgeSoundnessWith`, `Security/RoundByRound.lean:625`, three standard axioms, probe `AxiomsArkLibBuilt`). The converse fails: from the averaged bound at round k one gets, for one fixed value of the earlier challenges, only ε_k multiplied by the inverse probability of those challenges. The difference matters exactly where it should: a Fiat–Shamir (state-restoration) prover chooses among many prefixes, so it needs the bound at every prefix, which only the worst-case form gives. The worst-case form is the one of the literature (section 3.6.3), and the one leanerVM states.

The plain reading, proved by probe. Run the extractor backwards along a full transcript τ :

$$w_n = \text{extractOut}(s, \tau, w_{\text{out}}), \quad w_k = \text{extractMid}_k(s, \tau_{\leq k}, w_{k+1}), \quad \text{Ext}(s, \tau, w_{\text{out}}) = w_0 \in W_{\text{in}}.$$

Then, for any verifier, extractor and knowledge state function, with **no** hypothesis on the errors: for all s, τ, w_{out} ,

$$\begin{aligned} \Pr[V(s, \tau) \text{ returns } t, (t, w_{\text{out}}) \in R_{\text{out}}] &> 0 \\ \implies (s, \text{Ext}(s, \tau, w_{\text{out}})) &\in R_{\text{in}} \vee \exists k \text{ a verifier round, } \tau_k \in \text{Bad}_k(s, \tau_{<k}). \end{aligned}$$

This is `Probe.accept_imp_extract_or_bad` of the probe `PlainReading` (appendix B), proved in 40 lines by induction on the round, on the three standard axioms. With the bound of the predicate, each bad set has probability at most ε_k . So the predicate reduces knowledge soundness to: “no challenge of the accepted transcript fell in the bad set of its prefix”. What remains to get a statement about a prover is a union bound over the verifier rounds (interactive prover: $\sum_k \varepsilon_k$) or over the prover’s queries (Fiat–Shamir prover with Q queries: about $Q \cdot \max_k \varepsilon_k$). That last step is what ArkLib admits (section 3.6.4); it is a statement about ArkLib’s execution semantics, not about the protocol.

Instantiated to leanVM. S is the public input (there is no input oracle), W_{in} is the stack q , the first message is the stack, W_{out} is trivial and R_{out} is everything. The commit phase’s extractor returns the first message whatever intermediate witness it is handed (`Component.sendExtractor`, `LeanerVM/Protocol/ToArkLib/SendOracle.lean:134-138`), so $\text{Ext}(s, \tau, ()) = \tau_0$. The master theorem therefore reads as in the box at the head of this section. At the level of the oracle protocol this is a soundness statement about the committed oracle. The extraction of the stack from a commitment is not in it: it belongs to the compiled protocol (WHIR, Merkle trees, the random oracle), which the blueprint places in Layers 11 and 12 behind assumed interfaces.

3.6.3 Comparison with the literature

Round-by-round soundness. Block, Garreta, Katz, Thaler, Tiwari and Zając (*Fiat–Shamir Security of FRI and Related SNARKs*, ePrint 2023/1071, Definition 3.12, read from the paper’s PDF; it says it follows [CMS19], which follows Canetti et al. 2019) define it with a “doomed set” D of partial and complete transcripts (a state function taking the value *doomed*):

1. If $x \notin L$ then $(x, \emptyset) \in D$.
2. For any complete transcript τ , if $(x, \tau) \in D$ then $V(x, \tau) = \text{reject}$.
3. If (x, τ) is an $(i-1)$ -round partial transcript with $(x, \tau) \in D$, then for every potential prover next message m , $\Pr_{c \leftarrow C_i}[(x, \tau, m, c) \notin D] \leq \varepsilon(i)$.

ArkLib’s `Verifier.StateFunction` (`Security/RoundByRound.lean:135-152`) is this notion for a *reduction* (the final condition is “the verifier does not output a statement of the output language”) with two differences of shape. Condition 3 is split into a law for prover messages (`toFun_next`: a doomed prefix stays doomed after any prover message) and the challenge bound, which lets ArkLib schedule messages and challenges in any order rather than in alternation; the two forms are equivalent for alternating protocols (extend the doomed set to a prefix ending in a prover message by the value at the prefix before it). Condition 1 is an equivalence, `stmt ∈ langIn ↔ toFun 0 stmt default`; the extra direction is harmless (a state function may always be defined so) and is what sequential composition uses. Canetti et al.’s original definition (as the dossier’s author recalls it; not re-read) is the same three conditions with a function `State` in place of the set `D`.

Round-by-round knowledge soundness. The same paper, Definition 3.13 (verbatim):

A (public-coin) holographic IOP Π for an indexed relation R has round-by-round knowledge with error ε_k if there exists a polynomial-time extractor Ext and for all i there exists a (not necessarily efficiently computable) “doomed set” $D(i)$ of partial and complete transcripts such that:

- $(x, \emptyset) \in D(i)$ for all possible input x , regardless of whether $x \in L$ or not.
- For any possible input x and any complete transcript (x, τ) , if $(x, \tau) \in D(i)$ then $V(x, \tau) = \text{reject}$.
- If $i \in [\mu]$ and (x, τ) is an $(i-1)$ -round partial transcript such that $(x, \tau) \in D(i)$, then for every potential prover next message m , it holds that, if $\Pr_{c \leftarrow C_i}[(x, \tau, m, c) \notin D(i)] > \varepsilon_k(i)$, then $\text{Ext}(i, x, \tau, m)$ outputs a valid witness for x .

Chiesa and Yoge’s book gives (as the dossier’s author recalls it) the same notion with a state function: whenever a prefix is doomed and the next challenge leaves the doomed set with probability above the error, the extractor, given the prefix up to and including the prover’s last message, outputs a witness. ArkLib’s own docstring says that its knowledge state function “deliberately differs from ABF26 Definition A.5” (Arnon, Boneh, Fenzi, *Open Problems in List Decoding and Correlated Agreement*, 2026, per ArkLib’s bibliography; not read for the dossier), so ArkLib models a witness-carrying state function of that manuscript, not Definition 3.13 directly.

Where ArkLib’s definition differs, and whether it weakens the guarantee.

1. *The extractor’s resources.* Definition 3.13 asks for a polynomial-time extractor. ArkLib’s `Extractor.RoundByRound` is any Lean function: classical choice is allowed (section 3.6.7, probe `Extractors`), and nothing bounds running time. In the existential forms (`rbrKnowledgeSoundness`, `rbrKnowledgeSoundnessWorstCase`, and leanerVM’s `piop_rbrKnowledgeSoundness_exists`) this removes the knowledge content: the probe proves “knowledge soundness at error 0” for an arbitrary relation from a verifier that merely decides the language, with an extractor that chooses a witness. **This is a real weakening of the notion as a definition.** It is repaired only by the named form together with a reading of the named extractor (what leanerVM does: `piopExtractor` returns the first message). ArkLib’s own conversion `Extractor.RoundByRoundOneShot.toRoundByRoundOfRel` (`Security/RoundByRound.lean:102-123`) uses `h.choose`, and its docstring says why: “`extractMid` is a mathematical function rather than an algorithm with a tracked running time”.
2. *What the extractor sees and when.* Definition 3.13 extracts from the prefix (τ, m) at the round where the prover becomes likely to leave the doomed set. ArkLib extracts backwards from the full transcript and the output witness. For an IOP whose witness is its first oracle message (leanVM), both read that message, and ArkLib’s chain is the identity on it. For the Fiat–Shamir reading, ArkLib’s form is at least as usable: the extractor is run once, on the final accepting transcript; the bound on each prefix’s bad set gives the union bound over the prover’s queries. This is the dossier’s analysis; ArkLib proves no Fiat–Shamir theorem (section 3.6.4).
3. *The state is on (transcript, intermediate witness).* Definition 3.13’s doomed set is on transcripts alone and the empty transcript is always doomed; ArkLib’s state at the empty transcript is “the witness is valid” and the bad event has an existential over intermediate witnesses. When all intermediate witnesses are trivial (every leanVM phase after the commit has `witMid := fun _ => Unit`), ArkLib’s knowledge state function is a transcript predicate and the notion is round-by-round *soundness* of the verifier with respect to the seams, which is exactly what a phase over an already committed oracle can offer.
4. *Errors.* ArkLib’s error `pSpec.ChallengeIdx → ℝ≥0` may depend on the protocol (in leanVM on the instance `I`, which carries the sizes) but not on the statement; Definition 3.13’s $\varepsilon_k(i)$ depends on the index. Equivalent for leanVM, whose sizes index the protocol family.

5. *Acceptance*. ArkLib’s law 3 triggers on any positive probability of a related output, so a doomed state must reject with probability 1, as in Definition 3.13’s second item.
6. *No prover appears* in the worst-case form; Definition 3.13 has none either (“for every potential prover next message”). The averaged `rbrKnowledgeSoundness` does quantify over provers and is the weaker notion.

State-restoration knowledge soundness and straight-line extraction. State-restoration soundness (Ben-Sasson, Chiesa, Spooner 2016; Chiesa–Yogev) lets the prover rewind the verifier to any earlier state at most Q times; round-by-round soundness implies it with error about $Q \cdot \varepsilon$ (Theorem 3.15 of the paper above, quoted in section 3.6.4). ArkLib defines the state-restoration games (`Security/StateRestoration.lean:133-154` at the pin) and states no theorem from round-by-round to state restoration (the statements are commented out, `Security/Implications.lean:230-254`); the theorems from state restoration to plain soundness have `sorry` inside their statements (`:269, :284`). Straight-line extraction in the random oracle model (the extractor reads the prover’s oracle queries and never rewinds) is the shape of ArkLib’s `Extractor.Straightline` (`Security/Basic.lean:248-254`, which takes the prover’s and the verifier’s query logs), but no ArkLib theorem produces such an extractor from a round-by-round one at the pin, and the round-by-round extractor itself receives no query log.

3.6.4 Which standard consequences are proved and which are admitted

Status by `#print axioms` (probe `AxiomsArkLibBuilt`) for the modules in `leanerVM`’s build; by the source and ArkLib’s committed axiom baseline (section 3.7.3) for the others (`Security/Implications.lean` is not in `leanerVM`’s build). Lines are those of `dca90385`.

Table 3.5: The standard implications between ArkLib’s notions, at the two pins.

Implication	Where	At <code>dca90385</code>	At <code>7653a901</code>
worst-case rbr knowledge $\Rightarrow$ averaged rbr knowledge (<code>rbrKnowledgeSoundnessWorstCase_implies_rbrKnowledgeSoundness, ...With_implies_...With</code>)	<code>Security/ RoundByRound.lean:606, 625</code>	proved	proved
worst-case rbr soundness $\Rightarrow$ averaged rbr soundness rbr knowledge $\Rightarrow$ rbr soundness (<code>rbrKnowledgeSoundness_implies_rbrSoundness</code>)	<code>Security/ Implications.lean:87</code>	proved proved (not in the baseline; the proof is in the file)	proved proved
one-shot rbr knowledge $\Rightarrow$ rbr knowledge	<code>Security/ RoundByRound.lean:657</code>	proved, with a classical-choice extractor	proved
rbr knowledge $\Rightarrow$ plain knowledge soundness (<code>rbrKnowledgeSoundness_implies_knowledgeSoundness, error $\sum \varepsilon$</code>)	<code>Security/ Implications.lean:223-228</code>	admitted (by <code>sorry</code>)	admitted
rbr soundness $\Rightarrow$ soundness (<code>rbrSoundness_implies_soundness</code>)	<code>:79-83</code>	admitted	admitted
knowledge soundness $\Rightarrow$ soundness	<code>:49-63</code>	admitted	admitted
rbr $\Rightarrow$ state restoration	<code>:230-254</code>	commented out	commented out
state restoration $\Rightarrow$ plain	<code>:259-313</code>	admitted, <code>sorry</code> in the statements	same
Fiat–Shamir completeness (<code>fiatShamir_completeness</code>)	<code>FiatShamir/ Basic.lean:163-173</code>	admitted , and stated with the challenge oracle fixed to the constant function <code>default</code>	admitted
Fiat–Shamir (knowledge) soundness	—	not stated (“TODO”, <code>:175</code>)	not stated
BCS transform	<code>BCS/Basic.lean</code>	commented out	commented out

So a reader of `piop_rbrKnowledgeSoundness` trusts the *definition* (section 3.6.2) and, for the plain reading, the union bound. The transcript-level half of that reading is proved by the probe `PlainReading` (`accept_imp_extract_or_bad`); the probabilistic half (a union bound over the rounds of an interactive execution, or over the queries of a Fiat–Shamir prover) is the admitted `rbrKnowledgeSoundness_implies_`

knowledgeSoundness and the absent Fiat-Shamir theorem. For the expected form of the latter, the paper above states (Theorem 3.15, verbatim): “if (P, V) is a public-coin IOP for R with ... round-by-round knowledge error $\varepsilon_{\text{rbr-}k}(x)$... then $\text{BCS}^H(P, V)$ is a non-interactive random oracle proof system for R with ... adaptive knowledge error $\varepsilon_{\text{fs-}k}(x, Q, \kappa) = Q \cdot \varepsilon_{\text{rbr-}k}(x) + 3(Q^2 + 1)/2^\kappa$ against Q -query adversaries”. The blueprint’s FiatShamirSecurity interface names this shape (“error $Q \cdot \max_i \varepsilon_i$ ”, docs/roadmap/protocol-blueprint.md:1224); nothing in ArkLib at either pin states it.

3.6.5 Non-vacuity of the notion

The probe NonVacuity (appendix B) takes the relation “the statement is a bit and only true has a witness” ($\text{relIn} := \{p \mid p.1 = \text{true}\}$, $\text{relOut} := \text{Set.univ}$), for any shared oracle, any init and any impl.

- **No round, accept everything.** `acceptAll0` (`verify := fun _ => pure ()`) has *no* knowledge state function at all, for any extractor (`acceptAll0_no_stateFunction`): law 3 forces the state at false, law 1 then puts false in the language.
- **One challenge, accept everything.** `acceptAll1_not_sound_at_zero`: for *every* extractor and *every* knowledge state function, the verifier is not worst-case round-by-round knowledge sound at error 0. The attempt fails exactly where it should: at the statement false, the empty prefix and the challenge true, the state after the challenge is forced true by acceptance and the state before is forced false by the relation, so the bad set is everything and its probability is 1, not 0.
- **The error carries the content.** `acceptAll1_sound_at_one`: the same verifier *is* knowledge sound at error 1 (state “valid witness” at round 0, True afterwards). Every verifier with a challenge satisfies the predicate at error 1. So a theorem of the form `rbrKnowledgeSoundnessWorstCaseWith ... ε` says nothing until ε is read; in leanerVM the error `piopError P := P.toDef.err` is *data supplied by each phase’s definition* (`Component.Def.err`), not derived from anything, and a phase may charge 1 to a challenge and discharge its rbr obligation trivially. The blueprint’s planned `piopError_le ( $\sum i, \text{piopError } s_i \leq 2^{40/|E|} + \text{flockError}$ , docs/roadmap/protocol-blueprint.md:1130)` is therefore the load-bearing companion of the master theorem, and it does not exist yet.
- **Positive control.** `checkBit` (`verify := fun s _ => if s then pure () else failure`) is knowledge sound at error 0 with the trivial extractor (`checkBit_sound_at_zero`): the intended object inhabits the definition.
- **Constant state functions.** For any knowledge state function and any statement with no witness, the state at round 0 is false for every witness (`stateFunction_false_at`); for a statement with a valid witness it is true (`stateFunction_true_at`). So a knowledge state function is never constantly true nor constantly false as soon as the relation has a statement with a witness and one without.
- **Trivial output relation.** `full_iff_of_univ`: for a guarded verifier and `relOut = Set.univ`, the last law of a knowledge state function is equivalent to “if the check passes on (s, tr) then the state at the full transcript holds at the extracted witness”. With the spine’s `Seam.done I (fun _ _ => True, LeanerVM/Protocol/Spine/Seams.lean:191)` and its output statement `Unit`, “accept” means “the verifier returns rather than fails”, and law 3 says exactly “accepted $\Rightarrow$ state true”. A design consequence: since the final output statement is `Unit`, the last phase can only reject by failing; a verifier that returned a Boolean verdict would count false as an acceptance under `Set.univ`.

3.6.6 The oracle verifier is judged through toVerifier

Every security predicate ArkLib gives an `OracleVerifier` (`OracleVerifier.rbrSoundness`, `rbrKnowledgeSoundness`, `rbrKnowledgeSoundnessWith`, `Security/RoundByRound.lean:729-758`) is the predicate of its plain image `toVerifier` (section 3.5.3), and leanerVM states its master theorem on `(leanVmVerifier P).toVerifier`. `simOracle2` answers each query of the oracle verifier from the actual oracle statements and the actual messages of the transcript, and the output oracles are materialised. So the plain verifier’s verdict on a transcript is the oracle verifier’s verdict when its queries are answered faithfully; the state function and the extractor are functions of the whole transcript, including the full contents of every oracle message (the stack q , 2^μ field elements). This is the standard reading of IOP soundness, and nothing is lost for soundness at the oracle-protocol level. Three things are outside the statement:

- the verifier’s query complexity and locality (an `OracleVerifier` may query every point;

`OracleVerifier.numQueries` is a sorry definition, `Basic.lean:505-507`); they matter for the cost of the compiled protocol, not for its soundness;

- the *type* of the oracle message: a prover in the security game can only send an element of `Column I.μ` (a table of 2^μ values in $\mathbb{K}$), that is, a genuine multilinear over the base field of the announced size. Whether a prover of the compiled protocol can commit to something else is the commitment scheme’s problem (extractability or list binding of WHIR, Layer 11), not this theorem’s;
- the composition `OracleVerifier.append` (`Append/Basic.lean:619-629`) routes the second verifier’s queries through the first’s output-oracle simulation; `append_toVerifier` (:658, proved) says its `toVerifier` is the plain `Verifier.append` of the two `toVerifiers`, which is what lets leanerVM compose state functions of plain verifiers.

3.6.7 Must an extractor compute?

A classical-choice extractor is accepted by the type. ArkLib forces nothing: `Extractor.RoundByRound` is a structure of Lean functions with no computability requirement, and ArkLib itself defines a noncomputable one with `h.choose` (`Extractor.RoundByRoundOneShot.toRoundByRoundOfRel`, `Security/RoundByRound.lean:118-123`). The probe `Extractors` shows the consequence: `chooser R` (`extractOut` chooses a witness of $(s, \cdot) \in R$ if one exists, classically) is accepted as an extractor, and `chooser_sound` proves `rbrKnowledgeSoundnessWorstCaseWith ... (chooser R) ...` (`fun _ => 0`) for the verifier that decides the language of `R`, for *every* relation `R` with that language. Knowledge has been proved of nothing.

The existential form is soundness only. In the existential forms (`rbrKnowledgeSoundness`, `rbrKnowledgeSoundnessWorstCase`, leanerVM’s `piop_rbrKnowledgeSoundness_exists`) the extractor is chosen by the prover of the theorem, so such a theorem proves soundness, not knowledge (finding 118).

The named form is meaningful with the extractor read. What tells a computable extractor:

- `#print axioms` does not tell: `piopExtractor`, `commitExtractor` and `PublicInput.extractor` all report `Classical.choice` (probe `AxiomsLeanerVM`), through the proofs and instances inside their types, as does nearly every declaration built on Mathlib.
- What tells is the compiler: a `def` without the `noncomputable` keyword is accepted only if Lean can generate code for it, and Lean refuses a definition using classical choice in a value position (“failed to compile definition, consider marking it as ‘noncomputable’ because it depends on ‘Classical.propDecidable’”, probe `ExtractorsExpectedFailure`, line 12). The refusal is transitive: a compiled definition cannot call a noncomputable one.
- `#eval` is the positive test: it runs the code.

Applied to leanerVM (probe `Extractors`): `commitExtractor I`, `PublicInput.extractor I`, `piopExtractor P S`, (`commitSecurity I`).`extractor`, (`publicInputSecurity I`).`extractor` and an appended extractor are each the body of a compiled `def`; the commit extractor evaluated on the toy instance returns the honest stack (`true`, twice), and the public-input extractor returns `()`. The definitions are moreover one line each: “read the message” (`SendOracle.lean:137-138`) and “return `()`” (`PublicInput.lean:320-321`), so their efficiency is read off directly, which matters because Lean-computable is not polynomial-time: for a finite witness type and a decidable relation, exhaustive search is a computable extractor for any sound verifier.

Two limits of the check:

1. `piopExtractor P S` is computable *as a function of* the bundle `S : P.Security`, whose `extractor` field any future phase fills. The check must be repeated on each phase’s `Security` value (`publicInputSecurity` passes today). The type does not enforce it.
2. `Component.Def` carries the real-valued error `err : ... → ℝ≥0`, so every phase definition with a non-zero error is noncomputable (`publicInputPhase`, `LeanerVM/Protocol/PublicInput.lean:374-381`, “non-computable because of the error alone”). Lean then refuses to compile anything that takes the phase’s definition as an argument: the probe `ExtractorsExpectedFailure` shows (`publicInputPhase I`).`red.verifier` and `extractorOf (publicInputPhase I) (publicInputSecurity I)` rejected, although the verifier and the extractor are compiled when named directly. `leanVmVerifier P :=`

`P.toDef.red.verifier` and `piopExtractor P S` take `P` explicitly, so on the real bundle of phases neither will be a compiled definition. This does not affect the theorems; it affects the blueprint’s promises that the extractor and the honest prover “compute” and the planned code-generation probes (finding 122).

3.6.8 Perfect completeness, and completeness with an error

Completeness (`Reduction.completeness`, `perfectCompleteness`)

For every $(stmtIn, witIn)$ in the input relation, run the honest prover and the verifier (`Reduction.run`, `Execution.lean:209-216`: the prover runs, then the verifier on the transcript; challenges answered uniformly by `challengeQueryImpl`, shared oracle by `impl` from a state drawn by `init`); the probability that the run does not fail, the verifier’s output statement with the prover’s output witness is in the output relation, **and** the prover’s output statement equals the verifier’s, is at least $1 - \varepsilon$. Perfect completeness is $\varepsilon = 0$, so probability 1 (`perfectCompleteness_eq_prob_one`, :165). Failure (the verifier rejecting) counts against completeness. `OracleReduction.perfectCompleteness` (:469) is the same for `toReduction`.

```
def completeness (relIn : Set (StmtIn × WitIn))
  (relOut : Set (StmtOut × WitOut))
  (reduction : Reduction oSpec StmtIn WitIn StmtOut WitOut pSpec)
  (completenessError : ℝ ≥ 0) : Prop :=
  ∀ stmtIn : StmtIn,
  ∀ witIn : WitIn,
  (stmtIn, witIn) ∈ relIn →
  let pImpl : QueryImpl (oSpec + [pSpec.Challenge]) (StateT σ ProbComp) :=
    QueryImpl.addLift impl challengeQueryImpl
  Pr[fun (_, (prvStmtOut, witOut)), stmtOut =>
    ((stmtOut, witOut) ∈ relOut ∧ prvStmtOut = stmtOut) | OptionT.mk do
      (simulateQ pImpl (reduction.run stmtIn witIn).run).run' (← init)] ≥ 1 - completenessError

/-- A reduction satisfies perfect completeness if it satisfies completeness with error `0`. -/
def perfectCompleteness (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (reduction : Reduction oSpec StmtIn WitIn StmtOut WitOut pSpec) : Prop :=
  completeness init impl relIn relOut reduction 0
```

ArkLib/OracleReduction/Security/Basic.lean:89-105, ArkLib dca90385

What the composition theorem needs. `leanerVM`’s `Component.Complete.append` (`Component.lean:162-169`) applies the following theorem of ArkLib, `OracleReduction.append_perfectCompleteness_of_guarded_verifiers`, which is proved (probe `AxiomsArkLibBuilt`):

```
/-- Oracle reductions with guarded ordinary verifiers compose perfectly when prover execution
factors at the seam and the suffix is perfectly complete from every shared state. -/
theorem append_perfectCompleteness_of_guarded_verifiers
  (R1 : OracleReduction oSpec Stmt1 OStmt1 Wit1 Stmt2 OStmt2 Wit2 pSpec1)
  (R2 : OracleReduction oSpec Stmt2 OStmt2 Wit2 Stmt3 OStmt3 Wit3 pSpec2)
  (V1 : R1.toReduction.verifier.GuardedForm) (V2 : R2.toReduction.verifier.GuardedForm)
  (hSeam : ∀ hn : 0 < n,
    R1.prover.OutputIsPure v pSpec2.dir (0, hn) = .P_to_V)
  (h1 : R1.perfectCompleteness init impl rel1 rel2)
  (h2 : ∀ s : σ, R2.perfectCompleteness (pure s) impl rel2 rel3) :
  (R1.append R2).perfectCompleteness init impl rel1 rel3 := by
  change (R1.append R2).completeness init impl rel1 rel3 0
  simpa only [add_zero] using
    append_completeness_of_guarded_verifiers R1 R2 V1 V2 hSeam h1 h2
```

ArkLib/OracleReduction/Composition/Sequential/OracleCompleteness.lean:54-67, ArkLib dca90385

Its hypotheses: guarded forms of both plain verifiers (so that a run is “the prover’s run, then a deterministic check and verdict”, `GuardedCompleteness.lean:44-48`); the seam condition `hSeam` (if the second protocol has a round: the first prover’s output is a pure function of its state, `Prover.OutputIsPure`, or the second protocol starts with a prover message), which is what makes the appended prover’s execution factor

into the two provers' executions (`Prover.SimulatedAppendFactorization`, `Append/Simulation.lean:131-153`); completeness of the first component from `init`; and completeness of the second **from every deterministic state pure s** of the shared oracle, because the second component starts in whatever state the first left. `leanerVM` asks every component for completeness from every `init` (`Complete.complete`, `Component.lean:84-85`), which gives both. Over the empty shared oracle the guarded forms and the pure outputs are automatic (`Verifier.GuardedForm.ofEmpty`, `Prover.instOutputIsPureEmpty`; section 3.7.4).

Completeness with an error. `ArkLib` at the old pin composes completeness **with errors**, and the errors add: `Reduction.append_completeness_of_guarded_verifiers` (`GuardedCompleteness.lean:161-173`, error $\varepsilon_1 + \varepsilon_2$), `OracleReduction.append_completeness_of_guarded_verifiers` (`OracleCompleteness.lean:39-52`), and the n -ary `seqCompose_completeness_of_guarded_verifiers` (`GuardedNary.lean:45-59`, `OracleCompleteness.lean:77-99`, error $\sum_i \varepsilon_i$); all proved (probe `AxiomsArkLibBuilt`). The perfect versions are their $\varepsilon = 0$ corollaries. So a phase complete only up to an error can be composed in `ArkLib` today.

`leanerVM`'s spine cannot take one: `Component.Complete.complete` is `perfectCompleteness` (`Component.lean:84-85`), `Complete.append` uses the perfect theorem, and the blueprint's `FlockInterface` (`docs/roadmap/protocol-blueprint.md:1083`) demands `perfectCompleteness : reduction.perfectCompleteness ...`. The blueprint's list of wrong readings, item 20 (:1323-1325), says: "Perfect completeness needs no exceptional-challenge clause in Layers 4-8 and 10: no inverse of a challenge is taken. Flock's $(1 + r_{eq})^{-1}$ at $r_{eq} = 1$ is #3's, inside `flockError`. Witness: `piop_perfectCompleteness` has no side condition on challenges." The pinned Rust Flock prover does take that inverse (`crates/flock/src/zerocheck.rs:117`, let `g0 = (claim + r_eq * g1) * (F192::ONE + r_eq).inv()`), which is undefined at $r_{eq} = 1$ in characteristic 2 (the Rust `inv` returns 0 there, `crates/primitives/src/field/gf2_64x3.rs:144-153`, so the honest prover sends a wrong g_0). Whether the honest Flock prover then fails is for the Flock review; if it does, the Flock phase has completeness error $1/|E|$ per such challenge, not 0, and a completeness failure cannot be charged to `flockError`, which is a soundness error. Then `FlockInterface`'s `perfectCompleteness` field is unsatisfiable, `Phases.Complete` cannot be built for the real protocol, and `piop_perfectCompleteness` is vacuous for it. The fix is available at the pin: give `Component.Complete` an error field and use the additive theorems (finding 119).

3.7 ArkLib: what is proved and admitted, at the old pin and at the new

3.7.1 The axioms of `leanerVM`'s theorems

The probe `AxiomsLeanerVM` ran `#print axioms` on the proof system's master theorems, its two built phases, the composition and the port, at `b435631` with `ArkLib` at `dca90385` (appendix B). None depends on `sorryAx`.

Table 3.6: The axioms of `leanerVM`'s proof-system declarations, at `b435631` (probe `AxiomsLeanerVM`; names under `LeanerVM.Protocol`).

Declaration	Kind	Axioms
<code>piop_perfectCompleteness</code>	theorem	the standard three
<code>piop_rbrKnowledgeSoundness</code>	theorem	the standard three
<code>piop_rbrKnowledgeSoundness_exists</code>	theorem	the standard three
<code>commitSecurity</code>	def	the standard three
<code>commitComplete</code>	def	the standard three
<code>publicInputSecurity</code>	def	the standard three
<code>publicInputComplete</code>	def	the standard three
<code>Component.Security.append</code>	def	the standard three
<code>Component.Complete.append</code>	def	the standard three
<code>Verifier.KnowledgeStateFunction.appendGuarded</code>	def	the standard three
<code>Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first</code>	theorem	the standard three
<code>piopExtractor</code>	def	the standard three

Declaration	Kind	Axioms
<code>commitExtractor</code>	<code>abbrev</code>	the standard three
<code>PublicInput.extractor</code>	<code>def</code>	the standard three
<code>Component.sendExtractor</code>	<code>def</code>	<code>propext</code> , <code>Quot.sound</code>
<code>Phases.Security.toDef</code>	<code>def</code>	the standard three
<code>Phases.Complete.toDef</code>	<code>def</code>	the standard three

(“The standard three” are `propext`, `Classical.choice`, `Quot.sound`. `Component.Security.append`, `Component.Complete.append`, `commitSecurity`, `publicInputSecurity` and `appendGuarded` are definitions (structures bundling data and proofs), not theorems; the kinds were read from the source at [b435631](#).) This does *not* say that the master theorems are unconditional: they take `P : Phases I`, `C : P.Complete`, `S : P.Security` as arguments, and only the commit and public-input phases exist. It says that nothing admitted in `ArkLib` is consumed. The validation enforces it transitively (section 3.4.3).

3.7.2 The blueprint’s `ArkLib` table, row by row

The blueprint lists the `ArkLib` objects it relies on at `docs/roadmap/protocol-blueprint.md:232-251`. “Exists” means the name resolves at the pin at the place given; “proved” means no `sorryAx`, by probe where the module is in leanerVM’s build (P), else by `ArkLib`’s axiom baseline (B, section 3.7.3) and by reading the proof (S). Line references are those of [dca90385](#), under `.lake/packages/Arklib/ArkLib/`.

Table 3.7: The blueprint’s `ArkLib` table checked at [dca90385](#).

Blueprint row	At the pin	Proof status
ProtocolSpec n, MessageIdx, FullTranscript, (ProtocolSpec/Basic.lean)	Direction, ChallengeIdx, Transcript exist; Direction is in OracleReduction/Prelude.lean:66-69, not ProtocolSpec/Basic.lean	definitions
OracleInterface with Query, Response, answer (OracleInterface.lean:53-73)	exist; the class (:54-57) has fields Query and toOC : OracleContext Query (ReaderM Message); Response (:67) and answer (:75-78) are derived; answer is at :76, outside 53–73	definitions
Prover, Verifier, OracleVerifier, Reduction, OracleReduction, OracleProof (Basic.lean:222-669)	exist (:222-229, :247-250, :322-344, :624-628, :632-641, :669-675); cor- rect	definitions; OracleVerifier.numQueries (:505-507) is a sorry definition, unused by leanerVM
Reduction.completeness, perfectCompleteness, OracleReduction. perfectCompleteness (Security/ Basic.lean:89-103, 460-469)	exist (:89, :103, :460, :469); cor- rect	definitions (P)
Verifier.knowledgeSoundness, Extractor.Straightline (Security/ Basic.lean:248-359)	exist (:248-254, :344-361); correct	definitions (P)
Verifier.KnowledgeStateFunction, Extractor.RoundByRound, rbrKnowledgeSoundness, rbrKnowledgeSoundnessWorstCase, rbrKnowledgeSoundnessWorstCase_ implies_rbrKnowledgeSoundness (RoundByRound.lean:77-190, 416, 534, 606), ...WorstCaseWith (:553)	exist (:164, :77, :416, :534, :606, :553); correct	the implication is proved (P)
Verifier.rbrKnowledgeSoundness_ implies_rbrSoundness (Implications.lean:85)	exists (:87, docstring at 85); correct	proved (B, S)
OracleReduction.append, OracleReduction.seqCompose, ProtocolSpec.seqCompose (Append/ Basic.lean:709, General.lean:255)	exist (:709, :255, ProtocolSpec/ SeqCompose.lean:359); correct	definitions (P)

Blueprint row	At the pin	Proof status
Reduction.seqCompose_ perfectCompleteness_of_pure, OracleReduction.append_ perfectCompleteness_of_pure_ verifiers, seqCompose_completeness_ of_guarded_verifiers	exist (Completeness.lean:66, Append/Completeness.lean:259, GuardedNary.lean:45 and OracleCompleteness.lean:77); correct	proved (P for the last two; B for the first, whose module is not in leanerVM’s build). The theorem the spine actually uses, OracleReduction. append_perfectCompleteness_of_guarded_ verifiers (OracleCompleteness.lean:56), is not in the table; proved (P)
Verifier.append_ rbrSoundnessWorstCase_ of_pure_first, Verifier. StateFunction.append, Extractor. RoundByRound.append (Append/ RoundByRound.lean:37, Append/ StateFunction.lean:292, 75) Component.ReduceClaim, CheckClaim, RandomQuery, DoNothing (ProofSystem/ Component/), “all proved”	exist; correct the namespaces are ReduceClaim, CheckClaim, RandomQuery, DoNothing (no Component. prefix)	proved (P) no sorry in the four files and none of their names in the baseline (B, S). But CheckClaim’s oracle verifier is a pass- through that checks nothing at run time (CheckClaim.lean:19-24: “the verifier is a pure pass-through ... does <i>not</i> run any check at runtime; the checked predicate ... is instead carried by the output rela- tion”); only the plain-reduction variant guards. Not consumed by leanerVM at b435631
Sumcheck.Spec.reduction, StatementRound, relationRound, Sumcheck.Domain (Spec/ General.lean:171, SingleRound.lean:130-144, Domain.lean) MvPolynomial.MLE, eqPolynomial, eqTilde, eqTilde_append, MLE_eq_zero_iff, MLEEquivFin (Data/ MvPolynomial/Multilinear.lean) MvPolynomial.schwartz_zippel_ counting, prob_eval_zero_le_div (SchwartzZippelCounting.lean)	exist (General.lean:172; SingleRound.lean:130-145), one line off; the domain structure is SumcheckDomain in the root namespace (Domain.lean:55), not Sumcheck.Domain exist (:151, :88, :95, :116, :252, :421), namespace MvPolynomial; correct the first is schwartz_zippel_counting in the root namespace (:30); MvPolynomial.schwartz_zippel_ counting is an unknown constant (probe). prob_eval_zero_le_div (:131) is stated with Mathlib’s PMF.uniformOffFintype, not with VCVio’s $\Pr[\cdot \mid \$^t \alpha]$ that every ArkLib error bound uses; a bridge is needed (leanerVM’s ToVCVio/ UniformSample.lean supplies one for a subsingleton bad set)	definitions. Not consumed by leanerVM at b435631 proved (B: none in the baseline; no sorry in the file) proved (P); the name is wrong
Reduction.fiatShamir, Verifier.fiatShamir, fsChallengeOracle (FiatShamir/ Basic.lean:114-138)	Verifier.fiatShamir :130, Reduction.fiatShamir :140; fsChallengeOracle is an alias in ProtocolSpec/Basic.lean:854; partly off	definitions; the one theorem of the file is admitted
Commitment.Scheme, binding, perfectCorrectness_of_opening_ perfectCompleteness (Commitments/ Functional/Basic.lean) ToyProblem.Codegen SampleableType, uniformSample, \$^t (VCVio SampleableType.lean:44)	exist (:67, :220, :128); correct file exists, no sorry exist (:44-47, :51, :53); correct	proved (B, S). Commitment.extractability (:250-255) is a placeholder whose body is False under its quantifiers: it is never satisfiable — the class laws force uniformity (sec- tion 3.1.3)

3.7.3 ArkLib’s axiom baseline, and what is admitted at each pin

The baseline. scripts/axiom_baseline.json at the old pin (dated 2026-09-08, last changed by ArkLib #887) is the output of ArkLib’s axiomsweep tool (scripts/AxiomSweep.lean:8-60): it walks the compiled environment of the whole library and records every declaration whose statement or proof depends on sorryAx (“the same information as #print axioms, for the whole library at once”). It is an allow-list: ArkLib’s CI runs lake exe axiomsweep --check (.github/workflows/ci.yml:162-163), which fails when a tainted declaration is missing from the file and stays green

when a listed one has since been proved. The build check run of the pin commit concluded success. Hence a declaration absent from the baseline is `sorryAx-free` at the pin; one present may or may not be. The baseline lists 291 declarations ("sorry") and no non-standard axiom ("nonstandard": []). Counted `sorry` in code (non-comment lines) at the old pin: `Security/Implications.lean` 9, `Append/Security.lean` 4, `General.lean` 2, `Basic.lean` 1 (the `numQueries` definition), `FiatShamir/Basic.lean` 1, `LiftContext/Reduction.lean` 10, `Sumcheck/Spec/SingleRound.lean` 13, `Component/SendWitness.lean` 1, `CapacityBounds.lean` 3; none in `Security/Basic.lean`, `RoundByRound.lean`, `RbrGame.lean`, `Execution.lean`, `OracleInterface.lean`, `ProtocolSpec/*.lean`, `Append/Basic.lean`, `Append/StateFunction.lean`, `Append/RoundByRound.lean`, `Append/Completeness.lean`, `Append/Execution.lean`, `GuardedCompleteness.lean`, `GuardedNary.lean`, `OracleCompleteness.lean`, `CWSS/Guarded.lean`, `Commitments/Functional/Basic.lean`, `Component/ReduceClaim.lean`, `Component/CheckClaim.lean`, `Component/RandomQuery.lean`, `Component/DoNothing.lean`, `Sumcheck/Spec/General.lean`, `Domain.lean`, `Multilinear.lean`, `SchwartzZippelCounting.lean`. Whole library at the old pin: 192 code occurrences of `sorry` in 77 files.

At the new pin. The baseline shrinks from 291 to 286 entries between the pins, but among the declarations of the composition layer, the implications, the lifting, Fiat-Shamir and the sum-check specification **none was removed and none added** (the two baselines were compared by name over the prefixes `Verifier.`, `OracleVerifier.`, `Reduction.`, `OracleReduction.`, `Sumcheck.`, `fiatShamir`, `SendSingleWitness`, `NoInteraction`, `Extractor.`, `CodingTheory.rs_mca`). Code-line `sorry` counts at the new pin: `Security/Implications.lean` 9 (unchanged), `Append/Security.lean` 4 (unchanged), `General.lean` 2, `FiatShamir/Basic.lean` 1, `LiftContext/Reduction.lean` 10, `Sumcheck/Spec/SingleRound.lean` 13, which the dossier calls one fewer than the old pin although it also gives 13 for the old pin (a discrepancy in the dossier's counts, not resolved here); the leaves `verifier_rbrKnowledgeSoundness` and the lens instances remain. Table 3.8 sets the blueprint's ledger of admissions (`protocol-blueprint.md:257-267`) against both pins.

Table 3.8: What ArkLib admits, at the old pin and at the new, with the blueprint's ledger rows checked.

Admission (ledger row)	Declarations	At dca90385	At 7653a901
the sumcheck specification (A1)	<code>Sumcheck.Spec.SingleRound.verifier_rbrKnowledgeSoundness</code> and the lens instances (<code>SingleRound.lean:1091-1094</code> , <code>:1010-1080</code>); also its perfect completeness, <code>Sumcheck.Spec.SingleRound.reduction_perfectCompleteness (:1085-1088, lensComplete := by simp; sorry, through the admitted <code>Reduction.liftContext_perfectCompleteness</code>) and hence <code>Sumcheck.Spec.reduction_perfectCompleteness (General.lean:211)</code>, which the ledger omits</code>	admitted	admitted
the composition of soundness notions (A2, ArkLib issue #676)	<code>Verifier.append_knowledgeSoundness</code> , <code>append_rbrKnowledgeSoundness</code> , <code>seqCompose_rbrKnowledgeSoundness</code> (probe: <code>sorryAx</code>); also <code>append_soundness</code> , <code>append_rbrSoundness</code> , <code>seqCompose_soundness</code> , <code>seqCompose_knowledgeSoundness</code> , <code>seqCompose_rbrSoundness</code> and the <code>OracleVerifier.*</code> forms	admitted	admitted (<code>Append/Security.lean: the same four sorry</code>)
round-by-round $\Rightarrow$ plain (A3)	<code>rbrKnowledgeSoundness_implies_knowledgeSoundness</code> , <code>rbrSoundness_implies_soundness</code> ; also <code>knowledgeSoundness_implies_soundness</code>	admitted	admitted
context lifting (A4)	every <code>liftContext_*</code> security theorem; the definitions <code>Verifier.StateFunction.liftContext</code> and <code>Extractor.RoundByRound.liftContext</code> are <code>sorry</code> too	admitted	admitted
Fiat-Shamir and BCS (A5)	<code>fiatShamir_completeness</code> (stated for the constant challenge oracle, <code>FiatShamir/Basic.lean:163-173</code>); <code>BCSTransform</code> commented out (#627); <code>DuplexSponge</code> security; no Fiat-Shamir <i>soundness</i> statement exists	admitted; soundness absent	the same
the Johnson-range bound (A8)	<code>rs_mcaError_le_in_johnson_range (CapacityBounds.lean:203, in the baseline)</code> , an external admit	admitted	admitted

Admission (ledger row)	Declarations	At dca90385	At 7653a901
ring switching (A9)	the packing leaves sorry (RingSwitching.* in the baseline)	admitted	admitted
the query count	OracleVerifier.numQueries (Basic.lean:505-507), a sorry definition	admitted	admitted
commitment extractability	Commitment.extractability: a placeholder whose body is False (Commitments/Functional/Basic.lean:250-255; :249-254 at 7653a901)	never satisfiable	never satisfiable

So, for the blueprint’s ledger, the knowledge-soundness composition, the round-by-round to plain implications, context lifting, Fiat–Shamir completeness and the sumcheck round-by-round knowledge soundness and completeness are exactly as admitted at the new pin as at the old.

3.7.4 The ported composition theorem

Provenance, verified. `git show ca7a2577:ArkLib/OracleReduction/Composition/Sequential/Append/Knowledge.lean` in the sibling ArkLib checkout (after `git fetch origin`; the commit is “style(ring-switching): align names and streamline documentation”, 2026-09-09, head of pull request #615, “feat(ring-switching): add reusable packing proofs and protocol integrations”, author alexanderlhicks) gives a 632-line file, saved as `probes/lib-arklib/Knowledge_pr615_ca7a2577.lean.txt`. A diff against the local 587-line `LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean` shows exactly these differences (the whole diff output was read):

1. Header: the local file adds its own title lines and keeps the ArkLib copyright, license and “Authors: ArkLib Contributors” lines.
2. The pull request’s file is a classic Lean file (`import ...`); the local one is a module with `public import` of the same two modules and an `@[expose] public section`. (The pull request predates ArkLib’s move to the module system, which is the pin’s own commit, `dca90385 feat(module-system): migrate to modules (#897)`; this is one reason the pull request is in conflict with `main`.)
3. The module docstring is rewritten (the pull request’s says the same things: guarded first verifier, exact-object worst-case theorem, import it explicitly to avoid a cycle).
4. Everything is wrapped in `namespace LeanerVM.Protocol` (so the declarations are `LeanerVM.Protocol.Verifier.KnowledgeAppend.*`, `LeanerVM.Protocol.Verifier.KnowledgeStateFunction.appendGuarded`, `LeanerVM.Protocol.Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first`).
5. The two lemmas `witness_left` and `witness_right` (six and nine lines of proof in the pull request) are replaced by references to ArkLib’s own `Extractor.wit_mid_append_left` and `wit_mid_append_right` (`Append/StateFunction.lean:51-71` at the pin), which are the same statements.
6. Docstrings are added to fourteen declarations that had none.
7. Two names are qualified (`Verifier.guarded_accepting_of_mem`, `Verifier.append_run_guardedLeft`), as the namespace requires.
8. The four wrappers of the pull request are omitted: the averaged exact-object form (`append_rbrKnowledgeSoundnessWith_of_worstCase_of_guarded_first`), the existential worst-case form (`append_rbrKnowledgeSoundnessWorstCase_of_guarded_first`), the existential averaged form (`append_rbrKnowledgeSoundness_of_worstCase_of_guarded_first`) and the `OracleVerifier` form.

No proof step of the two main declarations differs. The blueprint’s description (“verbatim port ... except that its two lemmas on the witness type reuse the pinned ArkLib’s proofs; the wrappers into the existential and averaged forms are left out”, file docstring :31-37) is accurate. Both declarations depend on the three standard axioms (table 3.6).

What the two declarations say. `Verifier.KnowledgeStateFunction.appendGuarded G K1 K2` (`KnowledgeAppend.lean:474-488`) builds, from a guarded form `G` of the first verifier and knowledge state functions `K1` (for V_1 , relations $R_1 \rightarrow R_2$) and `K2` (for V_2 , $R_2 \rightarrow R_3$), a knowledge state function of V_1 . `append V2` for $R_1 \rightarrow R_3$ and for ArkLib’s appended extractor `E1.append E2 G.out`. Its state (`state, :182-196`) is: up to and including the seam, `K1` on the prefix; after the seam, “the first check passes on the first half and `K2` on the second half, at the statement the first verifier outputs (`G.out s tr1`)”.


```

extractors and knowledge states. The first round of the right component is included. -/
theorem append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first (G : V1.GuardedForm)
  (K1 : V1.KnowledgeStateFunction init impl R1 R2 E1)
  (K2 : V2.KnowledgeStateFunction init impl R2 R3 E2)
  {ε1 : pSpec1.ChallengeIdx → ℝ≥0} {ε2 : pSpec2.ChallengeIdx → ℝ≥0}
  (h1 : V1.rbrKnowledgeSoundnessWorstCaseWith init impl R1 R2 W1 E1 K1 ε1)
  (h2 : V2.rbrKnowledgeSoundnessWorstCaseWith init impl R2 R3 W2 E2 K2 ε2) :
  (V1.append V2).rbrKnowledgeSoundnessWorstCaseWith init impl R1 R3
    (Witness W1 W2) (E1.append E2 G.out) (KnowledgeStateFunction.appendGuarded G K1 K2)
    (Sum.elim ε1 ε2 ∘ ChallengeIdx.sumEquiv.symm) := by
  classical

```

LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean:508-518, leanerVM b435631

In words: if V_1 is worst-case round-by-round knowledge sound for (E_1, K_1) at errors ε_1 and V_2 for (E_2, K_2) at errors ε_2 , then $V_1.append V_2$ is, for the appended extractor and state function, at the error that assigns to each challenge **the error of the component it belongs to** ($Sum.elim \varepsilon_1 \varepsilon_2 \circ ChallengeIdx.sumEquiv.symm$). No additive term appears: a bad transition of the composite at a challenge of the first component is a bad transition of K_1 at the same prefix (`state_left`, `extractMid_left`); at a challenge of the second component, including its first round where the seam extraction happens (`backward_right`, `extractMid_seam`, `left_related`), it is a bad transition of K_2 at the prefix's second half and the statement $G.out \leq tr_1$. Since h_2 is quantified over all statements, it applies to that statement whatever tr_1 is. This is the right error for round-by-round notions (each challenge keeps its own bound); the sum $\sum \varepsilon$ only appears when passing to plain knowledge soundness (section 3.6.4).

Hypotheses. $G : V_1.GuardedForm$ (the first verifier is a Boolean check followed by a deterministic verdict, `CWSS/Guarded.lean:112-118`) is the only hypothesis beyond the two components' soundness; the second verifier is arbitrary; either component may have no round; the shared oracle is arbitrary but the state functions and errors are for the same `init`, `impl`. The guarded form is *data*: the composed extractor and state use $G.out$, the first verifier's verdict function, to compute the statement handed to the second component. On a rejected first half $G.out$ is whatever the form says (`ArkLib's ofEmpty` uses a fallback); it is used only under $G.check = true$ in the proofs, so this is harmless.

Is it a real restriction for leanVM's phases? No. Over the empty shared oracle every verifier is guarded: `ArkLib proves Verifier.GuardedForm.ofEmpty (Composition/Sequential/NoAmbient.lean:46-55` at the pin: `check := (V.verify stmt tr).run.runEmpty.isSome, out := ...getD (fallback stmt))`, together with `Prover.instOutputIsPureEmpty (:39-43)`. `leanerVM's` phases live over `[]`. (`Component.Def.red : OracleReduction []`), and `leanerVM` supplies the guarded forms by hand (`PublicInput.guarded, sendVerifierPure.toGuardedForm`), which is more readable but not necessary. It would be a restriction for a verifier that queries a shared oracle (a random oracle): such a verifier is not guarded, and the compiled protocol's verifier (Merkle paths hashed through the oracle) is of that kind; the blueprint composes only at the oracle-protocol level, where none is.

The upstream, read on 2026-09-29 (read-only gh). Pull request #615 is **open**, not a draft, mergeable: `CONFLICTING`, created 2026-07-07, last updated 2026-09-08, head `ca7a2577`. Its `Knowledge.lean` is not on `ArkLib's` main (`Composition/Sequential/Append/` there has the same eleven files as the pin, no `Knowledge.lean`); `Append/Security.lean` still admits `append_soundness`, `append_knowledgeSoundness`, `append_rbrSoundness`, `append_rbrKnowledgeSoundness` (four sorry). Issue #676 ("Sequential-composition and context-lifting layer is stubbed, making composed security theorems vacuous") is **open**, its body refreshed 2026-09-26. Its current scope says (verbatim):

This remains an open **legacy OracleReduction** issue. ... Generic legacy append soundness, knowledge soundness, and round-by-round security still contain admissions in `Composition/Sequential/Append/Security.lean`; derived wrappers inherit them. ... `Security/Implications.lean` still have proof gaps. ... Native `Sumcheck` (#1214), shared strategy execution (#1216), and native composition (#1218) are separate proved results. They do not repair the axiom dependencies of legacy protocol clients. Use #1 for the current interaction framework and #1222–#1229 for the next composition work. Keep this issue open until its remaining legacy claims are proved with correct assumptions or its consumers migrate with verified correspondence. Do not assume an unrestricted stateful composition statement is provable merely because its components are secure from the same fresh initialization.

The last sentence is a warning that the admitted `append_rbrKnowledgeSoundness` (stateful shared oracle, second component proved from a fresh `init`) may be unprovable as stated; leanerVM’s port avoids the problem by requiring both components at the same `init` and `impl` and, for completeness, the second from every deterministic state.

Consequence for the blueprint’s plan. The blueprint plans that “the port is deleted, and its two names in `Component.lean` replaced by `ArkLib`’s, when the pin moves past #615” (:260, :1143-1144, :1429). There is no evidence that #615 will merge in its current form (conflicting, pre-module, on a framework `ArkLib` now calls legacy, section 3.7.5). The condition should be stated as “when a revision of `ArkLib` contains a proved guarded-first append for `rbrKnowledgeSoundnessWorstCaseWith`”, whichever pull request brings it, and the plan should allow that this never happens on the legacy framework (finding 120).

3.7.5 What the new pin 7653a901 changes

The upgrade (144c5aa, “chore(deps): upgrade to Lean 4.34.1”) moves `ArkLib` from dca90385 to 7653a901, 347 commits later (the revision the dossier had read as `origin/main`), with VCVio a4232d08, `CompPoly` 572f9973 and Lean v4.34.1. What follows was read in `.lake/packages/ArkLib` at 7653a901; the probes’ outputs are about the old pin and were not re-run.

The admitted theorems of the old pin are still admitted. See section 3.7.3 and table 3.8. `OracleVerifier.numQueries` is still sorry (`Basic.lean:505-507`), `Commitment.extractability` still has `False` for its body (`Commitments/Functional/Basic.lean:249-254`), `fiatShamir_completeness` is still stated for the constant challenge oracle (`FiatShamir/Basic.lean:163-173`).

OracleReduction is now the “legacy” framework, and the typed framework has no knowledge notion. `ArkLib`’s guide already called `ArkLib/OracleReduction/` “legacy IOR abstractions and security theory” and `ArkLib/Interaction/` “typed interactions and dependent reduction foundations” at the old pin (`AGENTS.md`, unchanged at 7653a901). Between the pins `ArkLib/Interaction/` grew from 7 to 28 files (+7280 lines) and `ArkLib/ProofSystem/Sumcheck/Interaction/` was created (22 files, +3963 lines): the design documents (`docs/design/05-roadmap.md`, dated 2026-09-26) say the composition milestones “C1–C8 are merged” and “the immediate goal is to complete native Sumcheck: computable messages and honest execution, round-by-round security, and precisely stated extraction guarantees”. At 7653a901:

- the typed framework proves *plain* soundness and completeness of composed interactions and of native sumcheck (`Interaction/CompositionSoundness.lean: run_appendFlat_soundness*`; `Sumcheck/Interaction/ProtocolSoundness.lean:201 execute_soundness`, `ProtocolCompleteness.lean:110 execute_perfectCompleteness`), with no sorry under `ArkLib/Interaction/` or `Sumcheck/Interaction/` per its status page;
- it defines **no** knowledge soundness, no round-by-round notion and no extractor: a search for `knowledgeSound`, `rbrKnowledge`, `RoundByRound` and `extractor` in those two trees finds nothing;
- issue #676 (refreshed 2026-09-26) directs composition work to the typed framework and keeps the legacy admissions open “until its remaining legacy claims are proved with correct assumptions or its consumers migrate with verified correspondence”.

Consequences. (1) The spine’s vocabulary (`OracleReduction`, `OracleVerifier.toVerifier`, `KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCaseWith`, `GuardedForm`, `OracleReduction.append`, the guarded completeness theorems, `Verifier.GuardedForm.ofEmpty`) exists at the new pin with the same shapes (checked by name); leanerVM’s port compiled against it with proof-only edits. (2) Nothing in the typed framework can replace the master theorems today: it has no notion of knowledge soundness. (3) The blueprint’s expectation that upstream will prove the legacy admissions (its ledger’s “Action” column, `docs/roadmap/protocol-blueprint.md:257-267`; “the port is deleted ... when the pin moves past #615”) runs against upstream’s stated direction, which is migration rather than repair (finding 120). The status file’s line “the spine stays on `OracleReduction`, whose security definitions the new executor does not yet carry” (`protocol-status.md:201`) is the accurate description and should become the blueprint’s.

The probability API. VCVio at a4232d08 introduces the notation $\text{Pr}\{\text{let } x \leftarrow mx\}[p \mid x]$ (`prEvent`, `VCVio/EvalDist/ProbabilityNotation.lean:27`) and keeps `probEvent` with the lemma $\text{Pr}\{\text{let } x \leftarrow mx\}[p \mid x] = \text{Pr}[p \mid mx]$ (`VCVio/EvalDist/Defs/Basic.lean:1102`), so the two spellings denote the same number. ArkLib at 7653a901 restates every security definition in the new notation (the diff of `Security/RoundByRound.lean` between the pins is the notation and the lemma names `probEvent_*` $\rightarrow$ `prEvent_*`, `OptionT.prEvent_mk_*`; the mathematical content of `KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCase(With)` and the two implications is unchanged). `leanerVM 144c5aa` follows: `git diff b435631 144c5aa -- LeanerVM/Protocol` is 77 lines in, 76 out, confined to proofs and notation (`GuardedVerdict.lean`, `UniformSample.lean`, `KnowledgeAppend.lean`: $\text{Pr}[P \mid c] \rightarrow \text{Pr}\{\text{let } x \leftarrow c\}[P \mid x]$, `if_pos/dif_pos` $\rightarrow$ `ite_eq_left/dite_eq_left`, `probEvent_mono` $\rightarrow$ `prEvent_mono`; `PublicInput.lean` 10 lines, `Field.lean` 15 lines for `CompPoly`’s new $\mathbb{K}$). `docs/dependencies.md` at 144c5aa says the bridges’ “relations, extraction conditions, and error bounds are retained” and asks to “review these statement changes alongside the kernel audit, rather than relying on successful elaboration alone”. The statements of the two theorems of `KnowledgeAppend.lean` differ from b435631 only in the spelling of the three probabilities they mention (`state_full`’s hypothesis and the two `calc` blocks); the dossier read the diff and confirms no other change to a statement. The probes are written in the old notation; at the new pin they would need the same mechanical rewriting before they run (unverified: no Lean run is possible in the checkout).

The local port is still needed. `Composition/Sequential/Append/` at 7653a901 has the same eleven files as at dca90385 and no `Knowledge.lean`; pull request #615 is open and conflicting. The local `KnowledgeAppend.lean` remains the only proved guarded-first knowledge append, and `Component.Security.append` depends on it at both revisions.

The other named pull requests and issues (state on 2026-09-29, gh api, read-only). None is merged.

Table 3.9: ArkLib pull requests and issues the blueprint names or depends on.

Number	Kind	Title	State	Created / updated
#615	PR	feat(ring-switching): add reusable packing proofs and protocol integrations	open, conflicting	2026-07-07 / 2026-09-08
#676	issue	Sequential-composition and context-lifting layer is stubbed, making composed security theorems vacuous	open	2026-08-02 / 2026-09-26
#627	issue	Design: the BCS transform as a change of oracle implementation (<code>QueryImpl</code>), with a staged opening plan	open	2026-07-10 / 2026-07-11
#818	PR	feat(gkr): formalize the GKR protocol with perfect completeness	open	2026-08-31 / 2026-09-04
#848	PR	feat(dsfs): prove Theorems 6.1 and 6.2	open	2026-09-03 / 2026-09-17
#469	PR	feat(dsfs): statements of Section 5	open	2026-04-19 / 2026-09-17
#383	PR	feat: completeness and <code>rbrKnowledgeSoundness</code> of FRI-Binius protocols	open	2026-03-03 / 2026-09-26
#992	PR	feat(fri): prove end-to-end soundness of the computable IOP	open	2026-09-22 / 2026-09-22
#1128	PR	feat(sumcheck): expose honest round polynomial identities	open	2026-09-24 / 2026-09-24
#1129	PR	test(sumcheck): cover collision, degree and challenge-order controls	open	2026-09-24 / 2026-09-24
#900	issue	feat(data): formalize extension-ring block readout	open	2026-09-18 / 2026-09-24
#901	issue	feat(data): formalize multiset product injectivity and collision bounds	open	2026-09-18 / 2026-09-24
#926	PR (draft)	feat(lift-context): close the structural obligations in <code>LiftContext/Reduction.lean</code>	open	2026-09-21 / 2026-09-21

The status file’s dates (“open since 2026-09-08” for #615, “since 2026-09-04” for #818) are last-update dates, not opening dates (finding 124).

3.8 Layer 0 of the proof system, audited

Layer 0 is the proof system’s interface to the libraries: the committed column, the challenge samplers, the size of the challenge space, the oracle interface of the stack, and two counting bounds. It lives in `LeanerVM/Protocol/Field.lean`, `Basic.lean`, `ToArkLib/Oracles.lean` and `ToVCVio/UniformSample.lean`, with tests in `tests/LeanerVMTests/Protocol/Field.lean`. Everything below is at `b435631`; what the upgrade to `144c5aa` changes is said where it matters. The audit’s five results:

1. **The samplers are uniform, by proof** (section 3.8.2). The instance diamond on $\mathbb{K}$ at the old pins is definitionally one term; the $\mathbb{K}$ sampler has no consumer other than the $\mathbb{E}$ sampler.
2. **The column oracle answers the specification’s evaluation:** query $r \in \mathbb{E}^\mu$, answer $\tilde{q}(r) = \sum_i \text{ofK}(q_i) \text{eq}(r, i)$, in the specification’s little-endian cube order (section 3.8.3).
3. **card_E is correct** and robust to the choice of instance (section 3.8.4).
4. **The counting bounds are correct, tight and load-bearing**, and their restatement on VCVio’s new notation is the same statement (section 3.8.5).
5. **The code conforms to the blueprint’s Layer 0** except for four points: the explicit `n` of `evalOracle_answer`, the unlisted scalar interfaces, an internal inconsistency of the blueprint about the $\mathbb{K}$ sampler, and prose that is stale after the upgrade (section 3.8.6).

3.8.1 Catalogue

ArkLib’s `OracleInterface` (section 3.5) is what makes a message an *oracle*: a query type, and an implementation that answers a query by reading the message. Its default instance, `instDefault`, answers the query `()` with the whole message. (`OracleContext` `ι m` is VCVio’s pair of an `OracleSpec` `ι` and a `QueryImpl` of it in `m`, `VCVio/OracleComp/OracleContext.lean:25-27` at `f9dc47d9`.) ArkLib also registers `instVector : OracleInterface (Vector  $\alpha$  n)` answering *positions* (`OracleInterface.lean:333-337`), and instance `(i : Fin  $\theta$ ) : OracleInterface i.elim0 (:119)`.

```
class OracleInterface (Message : Type u) where
  Query : Type v
  toOC : OracleContext Query (ReaderM Message)
  ...
def answer {Message : Type*} [O : OracleInterface Message]
  (m : Message) (q : O.Query) : O.Response q :=
  (O.toOC.impl q).run m
  ...
@[reducible]
def instDefault {Message : Type u} : OracleInterface Message where
  Query := Unit
  toOC.spec := fun _ => Message
  toOC.impl _ := read
```

ArkLib/OracleReduction/OracleInterface.lean:54-57, :75-78, :88-96, ArkLib dca90385

Table 3.10: The declarations of Layer 0, at `b435631`. Paths are under `LeanerVM/Protocol/`.

Declaration	File:line	Kind	Plain meaning; what it rests on
Column	Field.lean:67-69	structure	a committed column: a table of 2^n values in $\mathbb{K}$; a <i>structure</i> so that ArkLib’s position-query <code>instVector</code> does not also apply. Rests on <code>CompPoly’s CMLPolynomialEval</code>
finEquivK	Field.lean:74-78	def	$\mathbb{K}$ is the 2^{64} bit patterns. Core <code>BitVec</code>
instSampleableTypeK	Field.lean:81-83	instance	uniform $\mathbb{K}$: a uniform <code>Fin (2⁶⁴)</code> read as bits. VCVio’s <code>ofEquiv</code> and <code>SampleableType (Fin n)</code>
limbsEquiv	Field.lean:86-90	def	$\mathbb{E}$ is its three limbs. <code>CompPoly’s Ext</code>

Declaration	File:line	Kind	Plain meaning; what it rests on
instSampleableTypeE	Field.lean:93	instance	uniform $\mathbb{E}$: three independent uniform limbs. VCVio's <code>Vector</code> instance, <code>instSampleableTypeK</code>
card_E	Field.lean:96	theorem	the challenge space has 2^{192} elements. <code>CompPoly</code> 's <code>card_ext3</code>
evalOracle	Field.lean:102-106	instance	a column as an oracle: query a point $r \in \mathbb{E}^n$, get $\tilde{q}(r)$. <code>ArkLib</code> 's <code>OracleInterface</code> , <code>CompPoly</code> 's <code>eval_Mle</code>
evalOracle_answer	Field.lean:109-110	theorem	the unfolding, for rewriting
instOracleInterfaceE	Field.lean:115	instance	a scalar the prover sends: query <code>()</code> , answer the scalar. <code>ArkLib</code> 's <code>instDefault</code>
instOracleInterfaceListE	Field.lean:118	instance	a list the prover sends: query <code>()</code> , answer the list. <code>ArkLib</code> 's <code>instDefault</code>
NoOracle	ToArkLib/Oracles.lean:26	abbrev	the empty family of oracles (matches <code>ArkLib</code> 's <code>Fin 0</code> instance)
OneOracle	ToArkLib/Oracles.lean:29	abbrev	exactly one oracle, of type <code>M</code>
noOracle_eq	ToArkLib/Oracles.lean:32	theorem	the empty family has one inhabitant
probEvent_uniformSample_le_of_card_le	ToVCVio/ UniformSample.lean:35-40	theorem	an event with at most k witnesses has probability at most $k/ \alpha $. <code>VCVio</code> 's <code>probEvent_uniformSample</code>
probEvent_uniformSample_le_of_subsingleton	ToVCVio/ UniformSample.lean:44-51	theorem	an event with at most one witness has probability at most $1/ \alpha $. The previous bound
Basic.lean	Basic.lean	—	an empty public section in namespace <code>LeanerVM.Protocol</code> (18 lines): nothing

The declarations themselves (the counting bounds are in section 3.8.5):

```

/-- A column of height `2^n`: the values of a `K`-multilinear on the cube, low bit first. A
structure rather than an abbreviation so that its evaluation oracle below does not overlap
ArkLib's position-query interface on `Vector`, which Layer 11's codeword oracles use. -/
structure Column (n : ℕ) where
  /-- The value table, CompPoly's hypercube representation. -/
  values : CMLPolynomialEval K n

/-! ## Sampling -/

/-- `K` as the `2^64` bit patterns, the `BitVec` view of `Fin (2^64)`. -/
def finEquivK : Fin (2 ^ 64) ≈ K where
  toFun := BitVec.ofFin
  invFun := BitVec.toFin
  left_inv _ := rfl
  right_inv _ := rfl

/-- Uniform sampling of `K`: a uniform `Fin (2^64)` read as a bit pattern. -/
instance instSampleableTypeK : SampleableType K :=
  haveI : NeZero (2 ^ 64) := (by norm_num)
  SampleableType.ofEquiv finEquivK

/-- `E` as its three limbs, CompPoly's carrier view. -/
def limbsEquiv : Vector K 3 ≈ E where
  toFun := fun v ↦ CompPoly.Extension.Ext.ofVector (P := BF64.ext3Params) v
  invFun := fun x ↦ CompPoly.Extension.Ext.coeffs (P := BF64.ext3Params) x
  left_inv _ := rfl
  right_inv _ := rfl

/-- Uniform sampling of `E`: three independent uniform limbs. -/
instance instSampleableTypeE : SampleableType E := SampleableType.ofEquiv limbsEquiv

/-- `|E| = 2^192`: the size of the challenge space in every error bound. -/
theorem card_E : Fintype.card E = 2 ^ 192 := BF64.card_ext3

/-! ## The evaluation oracle -/

/-- A column as an oracle: a query is a point `r ∈ E^n`, the answer is the multilinear extension

```

```

`q(r)`, lifted from `K` to `E`. -/
instance evalOracle (n : ℕ) : OracleInterface (Column n) where
  Query := Vector E n
  toOC :=
    { spec := (Vector E n) → E
      impl := fun r => do return CMLPolynomialEval.evalMle (← read).values (algebraMap K E) r }

/-- The oracle answers the lifted multilinear extension. -/
theorem evalOracle_answer (n : ℕ) (q : Column n) (r : Vector E n) :
  OracleInterface.answer q r = CMLPolynomialEval.evalMle q.values (algebraMap K E) r := rfl

/-! ## Scalar messages -/

/-- A scalar the prover sends is queried trivially: the answer is the scalar. -/
instance instOracleInterfaceE : OracleInterface E := OracleInterface.instDefault

/-- A list of scalars the prover sends is queried trivially: the answer is the list. -/
instance instOracleInterfaceListE : OracleInterface (List E) := OracleInterface.instDefault

```

LeanerVM/Protocol/Field.lean:64-118, leanerVM b435631

```

/-- No oracle: the empty family. -/
abbrev NoOracle : Fin 0 → Type := fun i => i.elim0

/-- Exactly one oracle, of type `M`. -/
abbrev OneOracle (M : Type) : Fin 1 → Type := fun _ => M

/-- All functions out of `Fin 0` are equal, so the empty oracle family has one inhabitant. -/
theorem noOracle_eq (o o' : ∀ i, NoOracle i) : o = o' := funext fun i => i.elim0

```

LeanerVM/Protocol/ToArkLib/Oracles.lean:25-32, leanerVM b435631

Axioms. Every declaration above depends on at most `propext`, `Classical.choice`, `Quot.sound` (`noOracle_eq`: `Quot.sound` only; `finEquivK`: `propext` only; `probe Layer0`, section 3).

Tests. `tests/LeanerVMTests/Protocol/Field.lean` (at `b435631`): a two-variable column (`#v[1, 2, 3, 4]`) (:27); four `#guards` that the oracle returns the table entry on the cube (:33-36); one off the cube, `answer col #v[y, ofK 0] = ofK 1 + ofK 3 * y` (:39, where at this pin 2 and 3 are bit patterns, so $1 + y \cdot (1 \oplus 2)$); agreement with `evalMle` of the lifted table at (y, y^2) (:41-42); a mutated column answers differently (:44); `sampleK`, `sampleE` as compiled definitions (:49, :52: they fail to compile if a sampler has no compiler IR); `#synth OracleInterface E` and `(List E)` (:59-60); `example : Fintype.card E = 2 ^ 192 := card_E` (:64).

3.8.2 The samplers: uniform, proved, and the diamond on $\mathbb{K}$

Uniformity is proved, by construction. A `SampleableType` instance cannot be non-uniform (section 3.1.3: the class carries `probOutput_selectElem_eq` and `mem_support_selectElem`, and `VCVio` derives $\Pr[= x \mid \$^t \alpha] = |\alpha|^{-1}$ for every instance). Both instances are `SampleableType.ofEquiv` of a `MathlibEquiv` ($\alpha \approx \beta$ bundles both inverse laws, so it is a bijection; here both laws are `rfl`), and `ofEquiv` improves the two laws (`SampleableType.lean:260-267` at `f9dc47d9`). The chain for $\mathbb{E}$ is: the primitive coin `#[0..2^64-1]` (`unifSpec.query`, uniform by `IsUniformSpec unifSpec`) $\rightarrow$ `finEquivK` $\rightarrow$ three independent draws (`VCVio`'s `Vector` instance) $\rightarrow$ `limbsEquiv`. The probe `Layer0` (section 4) has all of the following accepted:

```

example : Function.Bijective finEquivK := finEquivK.bijective
example : Function.Bijective limbsEquiv := limbsEquiv.bijective
example (x : K) : Pr[= x | $^t K] = (Fintype.card K : ℝ≥0∞)-1 := probOutput_uniformSample K x
example (x : E) : Pr[= x | $^t E] = (Fintype.card E : ℝ≥0∞)-1 := probOutput_uniformSample E x
example (x : E) : Pr[= x | $^t E] = ((2 ^ 192 : ℕ) : ℝ≥0∞)-1 := by
  rw [probOutput_uniformSample, card_E]
example : Pr[1 | $^t E] = 0 := probFailure_uniformSample E
example : support ($^t E) = Set.univ := support_uniformSample E
example : ($^t E) = limbsEquiv <$> ($^t (Vector K 3)) := rfl
example : ($^t K) = finEquivK <$> ($^t (Fin (2 ^ 64))) := rfl

```

probe Layer0 (probes/lib-others/Layer0.lean), section 4, run at the old pins

Instance search (probe Layer0, section 1): `SampleableType K → instSampleableTypeK`, `SampleableType E → instSampleableTypeE`, `SampleableType (Vector K 3) → instSampleableTypeVector K 3`, `SampleableType (Fin (2 ^ 64)) → instSampleableTypeFinOfNeZeroNat (2 ^ 64)`.

The diamond on $\mathbb{K}$ (old pins). Because $\mathbb{K}$ is `BitVec 64` to instance search and Mathlib has `FinEnum (BitVec n)`, VCVio’s `FinEnum.SampleableType` already supplies a sampler on $\mathbb{K}$ *without* leanerVM’s instance. The probe `SamplerDiamond` (which imports `LeanerVM.Parameters.Field` and VCVio only) shows it:

```
#synth SampleableType K      → FinEnum.SampleableType K
#synth SampleableType (BitVec 64) → FinEnum.SampleableType (BitVec 64)
#synth FinEnum K             → FinEnum.instBitVec 64
#synth SampleableType E      → error: failed to synthesize SampleableType E
```

probe `SamplerDiamond` (probes/lib-others/SamplerDiamond.lean), run at the old pins; the last line is an intended error

With `LeanerVM.Protocol.Field` imported, leanerVM’s instance is found (declared later, same priority), and the two are **the same term up to unfolding**; the probe `SamplerDiamond2` has both of these accepted (exit 0):

```
example :
  (instSampleableTypeK).selectElem = (FinEnum.SampleableType K).selectElem := rfl
example : (instSampleableTypeK : SampleableType K) = FinEnum.SampleableType K := rfl
```

probe `SamplerDiamond2` (probes/lib-others/SamplerDiamond2.lean), run at the old pins

In any case any two samplers on a finite type give every element the same probability (probe `SamplerDiamond`, accepted on the standard axioms):

```
theorem sampler_unique {α : Type} [Fintype α] (s t : SampleableType α) (x : α) :
  Pr[= x | @uniformSample α s] = Pr[= x | @uniformSample α t] := by
  rw [@probOutput_uniformSample α s, @probOutput_uniformSample α t]
```

probe `SamplerDiamond` (probes/lib-others/SamplerDiamond.lean), run at the old pins

So the diamond is harmless: no statement can depend on which instance is chosen. On $\mathbb{E}$ there is exactly one instance in scope. **Both samplers are uniform, proved; no distribution question remains.**

SampleableType K has no consumer. No declaration outside `Protocol/Field.lean` mentions it (a `git grep` of b435631 over `LeanerVM`, tests and docs finds only the blueprint’s Layer 0 sketch), and leanVM samples every challenge in $\mathbb{E}$ (crates/fiat_shamir/src/lib.rs:95-99, `pub fn sample(&mut self) -> F192`; the module docstring `crates/primitives/src/field/mod.rs:3-6`: “challenges, sumcheck/GKR values, and transcript scalars are E-valued”). The blueprint’s own dependency row says “Layer 0 supplies the E instance” (`protocol-blueprint.md:251`). At the old pins the instance is moreover a definitional duplicate of the library one. It is still needed as a *building block* of `instSampleableTypeE`, through VCVio’s `Vector` instance; at the old pin the library `FinEnum` instance would serve. Finding: finding 126.

At 144c5aa (new pins). $\mathbb{K}$ is a structure (section 3.2.6), so `FinEnum (BitVec 64)` no longer applies to it and the diamond disappears; `finEquivK` becomes `toFun := fun i ↦ BF64.ofBitVec (BitVec.ofFin i)`, `invFun := fun x ↦ x.toBitVec.toFin` (git diff b435631 144c5aa -- `LeanerVM/Protocol/Field.lean`); `instSampleableTypeK` is then the only $\mathbb{K}$ sampler (by reading; not run). VCVio a4232d08 states uniformity directly as a class law and proves sampler irrelevance upstream, the statement of the probe’s `sampler_unique` (section 3.1.5).

3.8.3 The oracle interface of the stack

- **Query:** Vector E n , a point $r \in \mathbb{E}^n$. **Answer:** $\mathbb{E}$ (spec := (Vector E n) $\rightarrow_o E$). **Implementation:** read the column and return `eval2Mle q.values (algebraMap K E) r`, which is `evalMle (map (algebraMap K E) q) r =  $\sum_i \text{ofK}(q_i) \cdot \prod_j (\text{bit}_j(i) ? r_j : 1 - r_j)$` , the multilinear extension $\tilde{q}(r)$ of the $\mathbb{K}$ -table lifted to $\mathbb{E}$ (section 3.2.1). The probe `Layer0`, section 5, has these accepted:

```
example (n : ℕ) : OracleInterface.Query (Column n) = Vector E n := rfl
example (n : ℕ) (r : Vector E n) : OracleInterface.Response (Message := Column n) r = E := rfl
example (n : ℕ) (q : Column n) (r : Vector E n) :
  OracleInterface.answer q r =
    CMLPolynomialEval.eval (CMLPolynomialEval.map (algebraMap K E) q.values) r := by
  rw [evalOracle_answer, CMLPolynomialEval.eval2_mle_eq_eval2]; rfl
```

probe `Layer0` (probes/lib-others/Layer0.lean), section 5, run at the old pins

- This is leanVM’s oracle: “we commit to a $\mathbb{K}$ -valued multilinear, and opening claims are in $\mathbb{E}$ ” (doc/leanvm/body/b-polynomial-commitment-scheme.tex:4).
- **Name:** the code calls it `evalOracle`, as the blueprint does (protocol-blueprint.md:638); its full name is `LeanerVM.Protocol.evalOracle` (probe `Layer0`: `#synth OracleInterface (Column 3)  $\Rightarrow$  evalOracle 3`).
- **Why a structure.** `CMLPolynomialEval K n` is `@[reducible] Vector K (2^n)`, and `ArkLib` has `instVector : OracleInterface (Vector  $\alpha$   $n$ )` with `Query := Fin n` (positions). Were `Column` an abbreviation, a column could be queried at positions. The structure prevents it; the justification is correct at `dca90385` (`OracleInterface.lean:333-337`).
- **Other instances.** `OracleInterface E` and `OracleInterface (List E)` are `ArkLib`’s `instDefault`: `query Unit`, `answer the whole message` (probe `Layer0`: `OracleInterface.Query E = Unit := rfl`, `OracleInterface.answer x () = x := rfl`, the same for `List E`). They are the “the verifier reads what the prover sent” interface for scalars and coefficient lists; `ArkLib` registers no default instance for any type, at `dca90385` or at `7653a901` (a `git grep` for `instDefault` instances finds only two local ones in `ArkLib/ProofSystem/RingSwitching/Packing/Spec.lean:105-106` at `7653a901`), so there is no ambiguity yet. Finding: finding 128.

No issue found with the oracle interface: the query space, the answer and the lift are those of the specification, and the mutated-column test makes the answer depend on the column.

3.8.4 card_E

`theorem card_E : Fintype.card E = 2 ^ 192 := BF64.card_ext3` (`Protocol/Field.lean:96`). The `Fintype E` instance is `CompPoly`’s `Ext.instFintype` (probe `Layer0`: `#synth Fintype E  $\Rightarrow$  Extension.Ext.instFintype`), and `card_ext3` is $(2^{64})^3$ by the coefficient bijection (section 3.2.3). `Fintype.card` does not depend on the instance (all `Fintype  $\alpha$` instances are equal: `Fintype` is a sub-singleton), so the statement is robust. Correct. At `144c5aa` the proof is unchanged in text; `Fintype K` is leanerVM’s proof-only `Fintype.ofFinite K` and `card_ext3 [Fintype Ext3]` accepts any instance (section 3.2.6).

3.8.5 The counting bounds

```
variable { $\alpha$  : Type} [SampleableType  $\alpha$ ] [Fintype  $\alpha$ ]

/-- An event with at most `k` witnesses has probability at most `k/| $\alpha$ ` under a uniform sample. -/
theorem probEvent_uniformSample_le_of_card_le (p :  $\alpha \rightarrow \text{Prop}$ ) [DecidablePred p] {k : ℕ}
  (h : (Finset.univ.filter p).card  $\leq k$ ) :
  Pr[p | $t  $\alpha$ ]  $\leq ((k / \text{Fintype.card } \alpha : \mathbb{R}_{\geq 0}) : \mathbb{R}_{\geq 0\infty}) := by
  ...
/-- An event any two of whose witnesses are equal has probability at most `1/| $\alpha$ ` under a uniform sample. -/
theorem probEvent_uniformSample_le_of_subsingleton (p :  $\alpha \rightarrow \text{Prop}$ )
  (h :  $\forall a b, p a \rightarrow p b \rightarrow a = b$ ) :
  Pr[p | $t  $\alpha$ ]  $\leq ((1 / \text{Fintype.card } \alpha : \mathbb{R}_{\geq 0}) : \mathbb{R}_{\geq 0\infty}) := by$$ 
```


LeanerVM/Protocol/ToVCVio/UniformSample.lean:31-51 (proofs elided), leanerVM b435631

Correct. The left side is $|\text{filter } p|/|\alpha|$ exactly (VCVio `probEvent_uniformSample`, section 3.1.3); the right side is the real $k/|\alpha|$ (division in $\mathbb{R}_{\geq 0}$, then coerced; $|\alpha| \neq 0$ because a `SampleableType` is nonempty). **Not vacuous, and the hypotheses carry the weight.** The probe `CountingBounds` has all of the following accepted, on the standard axioms (the only messages are unused-variable warnings): the single-witness bound is attained, the two-witness event exceeds it, and the bound with $k = 2$ is attained.

```
theorem one_witness : Pr[fun x : Fin 4 ↦ x = 2 | $^t (Fin 4)] = 1 / 4
theorem one_witness_bound :
  Pr[fun x : Fin 4 ↦ x = 2 | $^t (Fin 4)] ≤ ((1 / Fintype.card (Fin 4) : ℝ≥0) : ℝ≥0∞)
theorem rhs_eq : ((1 / Fintype.card (Fin 4) : ℝ≥0) : ℝ≥0∞) = 1 / 4 -- attained
theorem two_witnesses : Pr[fun x : Fin 4 ↦ x = 1 ∨ x = 2 | $^t (Fin 4)] = 1 / 2
theorem two_witnesses_exceed :
  ¬ Pr[fun x : Fin 4 ↦ x = 1 ∨ x = 2 | $^t (Fin 4)] ≤ 1 / 4 -- hypothesis needed
theorem two_witnesses_bound :
  Pr[fun x : Fin 4 ↦ x = 1 ∨ x = 2 | $^t (Fin 4)]
    ≤ (((2 : ℕ) / Fintype.card (Fin 4) : ℝ≥0) : ℝ≥0∞) -- k = 2, attained
theorem one_bad_challenge (c : E) :
  Pr[fun x : E ↦ x = c | $^t E] = ((2 ^ 192 : ℕ) : ℝ≥0∞)-1
theorem one_bad_challenge_bound (c : E) :
  Pr[fun x : E ↦ x = c | $^t E] ≤ ((1 / Fintype.card E : ℝ≥0) : ℝ≥0∞)
```

probe `CountingBounds` (`probes/lib-others/CountingBounds.lean`), run at the old pins

Consumers at b435631: `..._of_subsingleton` in the public-input phase (`PublicInput.lean:362`); `..._of_card_le` only through it.

At 144c5aa. Both statements were restated on VCVio’s new probability API (`git diff b435631 144c5aa -- LeanerVM/Protocol/ToVCVio/UniformSample.lean`): only the left side changes, $\text{Pr}[p \mid \$^t \alpha]$ becomes $\text{Pr}\{\text{let sample} \leftarrow \$^t \alpha\}[p \text{ sample}]$; binders, hypotheses and right sides are unchanged. At VCVio a4232d08, $\text{Pr}\{\dots\}[\dots]$ is a macro for the measure of the event `{True}` under the computation returning the proposition (section 3.1.5). **The two statements are the same mathematical statement.** At each pin the left side equals the same closed form by a library theorem: at f9dc47d9, `probEvent_uniformSample` : $\text{Pr}[p \mid \$^t \alpha] = (\text{Finset.univ.filter } p).card / \text{Fintype.card } \alpha$ (`SampleableType.lean:224-226`); at a4232d08, `SampleableType.prEvent_uniformSample` : $\text{Pr}\{\text{let } x \leftarrow \$^t \alpha\}[p \ x] = (\text{Finset.univ.filter } p).card / (\text{Fintype.card } \alpha : \text{ENNReal})$ (`Constructions/SampleableType/NativeMeasure.lean:41-45`). And VCVio a4232d08 proves its measure semantics agrees with the old mass-function semantics on discrete answer spaces. So the restated bounds bound the same number by the same number (verified by reading; not re-run at the new pin).

Upstream at the new pin. At VCVio f9dc47d9 there is no duplicate: VCVio has the equalities `probEvent_uniformSample` and `probOutput_uniformSample`, and the two local bounds are their two-line corollaries. At VCVio a4232d08, `..._of_card_le` is, up to a coercion, the $\Leftarrow$ direction of the following theorem (section variables $\{\alpha : \text{Type}\} [\text{SampleableType } \alpha] [\text{Fintype } \alpha] (p : \alpha \rightarrow \text{Prop}) [\text{DecidablePred } p], :117$); the local right side $((k / \text{Fintype.card } \alpha : \mathbb{R}_{\geq 0}) : \mathbb{R}_{\geq 0\infty})$ equals $k / (\text{Fintype.card } \alpha : \mathbb{R}_{\geq 0\infty})$ by `ENNReal.coe_div`.

```
theorem prEvent_uniformSample_le_div_iff {c : ℕ} :
  Pr[let x ← $^t α][p x] ≤ c / (Fintype.card α : ℝ≥0∞) ↔
  (Finset.univ.filter p).card ≤ c
```

VCVio/OracleComp/Constructions/SampleableType/NativeMeasure.lean:126-130, VCVio a4232d08

For single-witness events VCVio has `prEvent_uniformSample_eq_singleton` : $\text{Pr}\{\text{let } x \leftarrow \$^t \alpha\}[x = a] = (\text{Fintype.card } \alpha : \mathbb{R}_{\geq 0\infty})^{-1}$ (:110-111), but no “at most one witness” form (a search for `subsingleton` and `card_le_one` in `SampleableType/` and `EvalDist/ProbabilityBounds.lean` finds none). The probe’s `sampler_unique` is upstream as `prEvent_uniformSample_inst_irrel` (:280-284). So at the new pin `..._of_card_le` is redundant and can be replaced by the upstream equivalence; `..._of_subsingleton` is kept, or upstreamed (finding 136).

3.8.6 Conformance with the blueprint’s Layer 0

The blueprint’s Layer 0 is `protocol-blueprint.md:625-650` at `b435631`.

Table 3.11: The blueprint’s Layer 0 against the code at `b435631`.

Blueprint	Code	Verdict
<code>instance : SampleableType K -- a uniform Fin (2^64) read as a bit pattern</code>	<code>instSampleableTypeK := SampleableType.ofEquiv finEquivK</code>	agrees; unused (section 3.8.2)
<code>instance : SampleableType E -- three independent limbs through Ext.ofVector; never enumerates</code>	<code>instSampleableTypeE := SampleableType.ofEquiv limbsEquiv</code>	agrees
<code>theorem card_E : Fintype.card E = 2 ^ 192</code>	identical	agrees
<code>instance evalOracle (n : ℕ) : OracleInterface (Column n) where Query := Vector E n ... answer := evalMle q (algebraMap K E)</code>	identical	agrees
<code>theorem evalOracle_answer (q : Column n) (r : Vector E n) : ...</code>	<code>theorem evalOracle_answer (n : ℕ) (q : Column n) (r : Vector E n) : ...</code>	mismatch: <code>n</code> explicit in the code, implicit (auto-bound) in the sketch; callers must write <code>evalOracle_answer n q r</code> although <code>n</code> is determined by <code>q</code> (finding 127)
“SampleableType.ofEquiv on the carrier views, as ArkLib’s own KoalaBear.Ext6.sampleableType, and a test probes that it has compiler IR”	<code>yes (sampleK, sampleE)</code>	agrees
Tests: “#guards that the oracle answers evalMle on a two-variable table, on and off the cube, with a mutated column; example : Fintype.card E = 2 ^ 192 := card_E”	<code>yes (section 3.8.1)</code>	agrees
—	<code>instOracleInterfaceE, instOracleInterfaceListE Protocol/Field.lean</code>	not in the Layer 0 sketch, nor in the interface list <code>protocol-blueprint.md:1400</code> (“ <code>instSampleableTypeE evalOracle card_E</code> ”), which also omits <code>instSampleableTypeK</code> and <code>Column</code> (the latter is at :666) (finding 128)
<code>protocol-blueprint.md:251</code> : “Layer 0 supplies the E instance”	Layer 0 supplies $\mathbb{K}$ and $\mathbb{E}$	internal inconsistency of the blueprint (finding 126)
“Add ArkLib at dca90385 ... the root pin 3468b38c wins” (:629-631; :633-635 at 144c5aa), “The sampler is the point at which a wrong Fintype would enumerate 2^{64} or 2^{192} elements (finding P3)” (:645-646; :649-650 at 144c5aa)	at 144c5aa the pins moved and Fintype K is proof-only	stale after #61 (the disclaimer added at 144c5aa:protocol-blueprint.md:52-54 covers “dependency API tables and signatures”, not this prose) (finding 129)

`NoOracle` and `OneOracle` are specified in the spine section (`protocol-blueprint.md:472, :1377`), and `ToVCVio/UniformSample.lean`’s bound is cited at :1046; both agree with the code.

Chapter 4

Catalogue of the built code

Every public declaration of the proof system on `main` at `b435631`, grouped by file: the structures with their fields, the definitions with their signatures (and their bodies where the body is what a reader has to check), the theorem statements, the instances and the concrete inhabitants. Each entry says what it means in plain language, what it depends on, and its class: **load-bearing** (a master theorem’s statement unfolds to it), **interface** (a phase author or the adaptor must meet or consume it), **helper** (used inside proofs only) or **test fixture**. Snippets are copied from the source at the revision reviewed. The blueprint’s per-layer sketches are compared with this catalogue in section 8.9; the audit surface it implies is measured in chapter 10.

4.1 The spine, declaration by declaration

Every public declaration of the modules in scope, in file order, with its class: **load-bearing** (the statement of a master theorem unfolds to it; “named form” marks those reached only through `piop_rbrKnowledgeSoundness`’s named extractor and state function, section B), **interface** (a phase author must meet or use it), **helper** (used only inside proofs; those a master statement unfolds to are marked), **test fixture**. Snippets are extracted from `git show b435631:<file>` by `probes/code-spine/catalogue.py`: structures and inductives in full, definitions with their data body and without proof fields, theorems and instances as their statement. Helpers that are pure proof lemmas (`passThrough_materializeOutput`, `passThroughVerifier_toVerifier_run`, `passThroughPure`, `passThroughProver`, `passThroughVerifier`, `passThroughExtractor`, `passThroughStateFunction`, `passThrough_rbr`, `sendSpec`, `sendProver`, `sendEmbedding`, `sendVerifier`, `sendOracle_outputPure`, `send_materializeOutput`, `sendVerifier_toVerifier_run`, `sendVerifierPure`, `sendOracle_complete`, `sendWitMid`, `sendOracle_rbr`, `Refinement.id`, `Refinement.comp`, `noOracle_eq`, and the seventeen lemmas of `KnowledgeAppend` other than its two named declarations) are named here and not listed one by one; the send-oracle ones are counted in section B where the named statement reaches them.

Library objects (all at the pins `ArkLib dca90385`, `CompPoly 3468b38c`, `VCVio f9dc47d9`) are introduced where first used in sections C and D; the catalogue names them in the “depends on” column.

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

Order of this section: the five spine modules (`Instance`, `Seams`, `Phase`, `Compose`, `Toy`), then the generic modules under `ToArkLib/`, then `Field.lean` where the spine unfolds to it. Every load-bearing and interface declaration is shown with its snippet, its source at `b435631`, and its meaning and dependencies as the dossier states them; the helpers and test fixtures of each file are listed in a table at the end of the file’s subsection. The class is given at the right of each entry.

4.1.1 LeanerVM/Protocol/Spine/Instance.lean

Side

[load-bearing]

```
inductive Side
| push
| pull
deriving DecidableEq, Repr
```

LeanerVM/Protocol/Spine/Instance.lean:51-54 at b435631

Which side of the bus a tuple is on: provided (**push**) or consumed (**pull**). *Depends on:* —.

Shape

[load-bearing]

```
structure Shape where
  ntab : ℕ
  τ : Fin ntab → ℕ
  width : Fin ntab → ℕ
```

LeanerVM/Protocol/Spine/Instance.lean:57-63 at b435631

The tables: how many, each one's log-height (the announced sizes) and width. *Depends on:* —.

Shape.ColumnId

[load-bearing]

```
abbrev Shape.ColumnId (S : Shape) : Type :=  $\Sigma$  j : Fin S.ntab, Fin (S.width j)
```

LeanerVM/Protocol/Spine/Instance.lean:66-66 at b435631

A column: a table and a column index in it. *Depends on:* **Shape**.

Layout

[load-bearing]

```
structure Layout (μ : ℕ) (ι : Type) (κ : ι → ℕ) where
  read : Column μ → (c : ι) → Column (κ c)
  extend : (c : ι) → Vector E (κ c) → Vector E μ
  read_eval : ∀ (q : Column μ) (c : ι) (z : Vector E (κ c)),
    CMLPolynomialEval.evalMle (read q c).values (algebraMap K E) z =
      CMLPolynomialEval.evalMle q.values (algebraMap K E) (extend c z)
```

LeanerVM/Protocol/Spine/Instance.lean:73-81 at b435631

How a column is read off the stack (**read**) and how a point of the column lifts to a point of the stack (**extend**), with the one law: the column's extension at z is the stack's at **extend c z**. A reading law: it does not say the columns are disjoint slices (C.2, D.4). *Depends on:* **Column**, **CompPoly** **evalMle**.

Coord

[load-bearing]

```
inductive Coord (S : Shape) (κ : ℕ)
| const (c : K)
| known (col : Column κ)
| committed (c : S.ColumnId) (h : S.τ c.1 = κ)
```

LeanerVM/Protocol/Spine/Instance.lean:86-89 at b435631

One coordinate of a boundary tuple: a constant, a column both parties know (index, bytecode), or a committed column of the block's height. *Depends on:* **Shape**, **Column**.

BoundaryBlock

[load-bearing]

```
structure BoundaryBlock (S : Shape) where
  κ : ℕ
  side : Side
  coords : Vector (Coord S κ) 16
```

LeanerVM/Protocol/Spine/Instance.lean:92-98 at b435631

2^k boundary tuples on one side of the bus, given coordinate by coordinate (sixteen, separator first).
Depends on: Coord, Side.

PublicLine

[load-bearing]

```
structure PublicLine (S : Shape) where
  col : S.ColumnId
  cell0 : K
  cell1 : K
  sent : Bool
  pos : 0 < S.τ col.1
```

LeanerVM/Protocol/Spine/Instance.lean:102-114 at b435631

A column whose cells 0 and 1 the public statement fixes; *sent* says whether the proof carries its value (transcript only, not the relation); *pos* says the column has a cell 1. *Depends on:* Shape.

M3Instance

[load-bearing]

```
structure M3Instance extends Shape where
  Stmt : Type
  constraints : (j : Fin ntab) → List (CMvPolynomial (width j) K)
  flushes : (j : Fin ntab) → List (Side × Vector (CMvPolynomial (width j) K) 16)
  d : ℕ
  constraints_degree : ∀ j, ∀ C ∈ constraints j, C.totalDegree ≤ d
  flushes_degree : ∀ j, ∀ f ∈ flushes j, ∀ k, (f.2.get k).totalDegree ≤ d
  counts : (j : Fin ntab) → List (Fin (width j))
  boundary : List (BoundaryBlock toShape)
  μ : ℕ
  layout : Layout μ toShape.ColumnId (fun c ↦ τ c.1)
  publicLines : Stmt → List (PublicLine toShape)
  aux : Column μ → Prop
  decAux : DecidablePred aux
```

LeanerVM/Protocol/Spine/Instance.lean:119-148 at b435631

Everything the verifier reads about an arithmetization: the statement type, per table its constraints and flushes (polynomials over K of the row), a degree bound d with the two proofs, the count columns, the boundary blocks, the stack height μ and layout, the public lines of a statement, and the auxiliary predicate *aux* with its decision procedure. Trusted data (section F). *Depends on:* Shape, Layout, BoundaryBlock, PublicLine, CompPoly CMvPolynomial, totalDegree.

M3Instance.κ

[load-bearing]

```
abbrev κ (c : I.ColumnId) : ℕ := I.τ c.1
```

LeanerVM/Protocol/Spine/Instance.lean:157-157 at b435631

Log-height of a column: its table's. *Depends on:* M3Instance.

M3Instance.column

[load-bearing]

```
def column (q : Column I.μ) (c : I.ColumnId) : Column (I.κ c) := I.layout.read q c
```

LeanerVM/Protocol/Spine/Instance.lean:160-160 at b435631

One column of the stack, through the layout. *Depends on:* Layout.read.

M3Instance.row

[load-bearing]

```
def row (q : Column I.μ) (j : Fin I.ntab) (x : Fin (2 ^ I.τ j)) : Fin (I.width j) → K :=
  fun i ↦ (I.column q (j, i)).values.get x
```

LeanerVM/Protocol/Spine/Instance.lean:163-164 at b435631

Row x of table j : the function giving each column's cell x . *Depends on:* column.

M3Instance.coordCell

[load-bearing]

```
def coordCell (q : Column I.μ) {κ : ℕ} : Coord I.toShape κ → Fin (2 ^ κ) → K
| .const c, _ => c
| .known col, x => col.values.get x
| .committed c h, x => (I.column q c).values.get (Fin.cast (congrArg (2 ^ ·) h.symm) x)
```

LeanerVM/Protocol/Spine/Instance.lean:167-170 at b435631

The value of a boundary coordinate at row x of its block. *Depends on:* Coord, column.

M3Instance.flushTuples

[load-bearing]

```
def flushTuples (q : Column I.μ) (s : Side) : List (Vector K 16) :=
  (List.finRange I.ntab).flatMap fun j =>
    ((I.flushes j).filter fun f => decide (f.1 = s)).flatMap fun f =>
      (List.finRange (2 ^ I.τ j)).map fun x => f.2.map fun P => P.eval (I.row q j x)
```

LeanerVM/Protocol/Spine/Instance.lean:174-177 at b435631

The tuples of one side flushed by the tables: for every table, every flush of that side, every row, the sixteen coordinate polynomials evaluated on the row. *Depends on:* row, CompPoly CMvPolynomial.eval.

M3Instance.boundaryTuples

[load-bearing]

```
def boundaryTuples (q : Column I.μ) (s : Side) : List (Vector K 16) :=
  (I.boundary.filter fun b => decide (b.side = s)).flatMap fun b =>
    (List.finRange (2 ^ b.κ)).map fun x => b.coords.map fun co => I.coordCell q co x
```

LeanerVM/Protocol/Spine/Instance.lean:180-182 at b435631

The tuples of one side on the boundary blocks: for every block of that side, every row. *Depends on:* coordCell.

M3Instance.tuples

[load-bearing]

```
def tuples (q : Column I.μ) (s : Side) : List (Vector K 16) :=
  I.flushTuples q s ++ I.boundaryTuples q s
```

LeanerVM/Protocol/Spine/Instance.lean:185-186 at b435631

Every tuple of one side: the tables' then the boundary's. *Depends on:* flushTuples, boundaryTuples.

M3Instance.ConstraintsVanish

[load-bearing]

```
def ConstraintsVanish (q : Column I.μ) : Prop :=
  ∀ j, ∀ C ∈ I.constraints j, ∀ x, C.eval (I.row q j x) = 0
```

LeanerVM/Protocol/Spine/Instance.lean:197-198 at b435631

Every constraint of every table is zero on every row (the zerocheck target). *Depends on:* row.

M3Instance.Balanced

[load-bearing]

```
def Balanced (q : Column I.μ) : Prop := (I.tuples q .push).Perm (I.tuples q .pull)
```

LeanerVM/Protocol/Spine/Instance.lean:201-201 at b435631

The pushed tuples are a permutation of the pulled tuples: the same multiset, counted in $\mathbb{N}$. *Depends on:* tuples, Mathlib List.Perm.

M3Instance.CountsNonzero

[load-bearing]

```
def CountsNonzero (q : Column I.μ) : Prop :=
  ∀ j, ∀ i ∈ I.counts j, ∀ x, (I.column q {j, i}).values.get x ≠ 0
```

LeanerVM/Protocol/Spine/Instance.lean:204-205 at b435631

Every cell of every count column is nonzero (the count product). *Depends on:* column.

M3Instance.PublicLinesHold

[load-bearing]

```
def PublicLinesHold (input : I.Stmt) (q : Column I.μ) : Prop :=
  ∀ l ∈ I.publicLines input,
    (I.column q l.col).values.get {0, Nat.two_pow_pos _} = l.cell0 ∧
    (I.column q l.col).values.get {1, Nat.one_lt_two_pow l.pos.ne'} = l.cell1
```

LeanerVM/Protocol/Spine/Instance.lean:208-211 at b435631

Cells 0 and 1 of every line the statement names hold the line's values. *Depends on:* column, PublicLine.

M3Holds

[load-bearing]

```
def M3Holds (I : M3Instance) (input : I.Stmt) (q : Column I.μ) : Prop :=
  I.ConstraintsVanish q ∧ I.Balanced q ∧ I.CountsNonzero q ∧ I.PublicLinesHold input q ∧ I.aux q
```

LeanerVM/Protocol/Spine/Instance.lean:219-220 at b435631

The relation: the five clauses. What the verifier establishes about the committed stack. *Depends on:* the five clauses.

TheOracle

[load-bearing]

```
abbrev TheOracle (I : M3Instance) : Fin 1 → Type := OneOracle (Column I.μ)
```

LeanerVM/Protocol/Spine/Instance.lean:226-226 at b435631

The one oracle from the commit on: the stack. *Depends on:* OneOracle, Column.

M3Rel

[load-bearing]

```
def M3Rel (I : M3Instance) : Set ((I.Stmt × (∀ i, NoOracle i)) × Column I.μ) :=
  {p | M3Holds I p.1.1 p.2}
```

LeanerVM/Protocol/Spine/Instance.lean:230-231 at b435631

M3Holds as ArkLib states relations: statement = public input with no oracle, witness = the stack. *Depends on:* M3Holds, NoOracle.

4.1.2 LeanerVM/Protocol/Spine/Seams.lean

ColumnClaim

[load-bearing]

```
structure ColumnClaim (I : M3Instance) where
  col : I.ColumnId
  point : Vector E (I.κ col)
  value : E
```

LeanerVM/Protocol/Spine/Seams.lean:57-63 at b435631

One column's extension at a point of E^κ equals a value. *Depends on:* M3Instance.

ColumnClaim.Holds

[load-bearing]

```
def ColumnClaim.Holds {I : M3Instance} (q : Column I.μ) (c : ColumnClaim I) : Prop :=
  CMLPolynomialEval.evalMle (I.column q c.col).values (algebraMap K E) c.point = c.value
```

LeanerVM/Protocol/Spine/Seams.lean:66-67 at b435631

The claim is true of the stack, reading the column through the layout. *Depends on:* column, evalMle.

VirtualTerm

[load-bearing]

```
structure VirtualTerm (I : M3Instance) where
  weight : E
  j : Fin I.ntab
  poly : CMvPolynomial (I.width j) K
  point : Vector E (I.τ j)
```

LeanerVM/Protocol/Spine/Seams.lean:71-79 at b435631

One term of a linear claim: a weight in E, a table, a polynomial of its row over K, and the point the virtual table (the polynomial on every row) is extended to. *Depends on:* M3Instance.

VirtualTerm.table

[load-bearing]

```
def VirtualTerm.table {I : M3Instance} (q : Column I.μ) (t : VirtualTerm I) :
  CMLPolynomialEval E (I.τ t.j) :=
  Vector.ofFn fun x => ofK (t.poly.eval (I.row q t.j x))
```

LeanerVM/Protocol/Spine/Seams.lean:82-84 at b435631

The virtual table of a term: its polynomial evaluated on every row of its table, lifted to E. *Depends on:* row.

VirtualTerm.eval

[load-bearing]

```
def VirtualTerm.eval {I : M3Instance} (q : Column I.μ) (t : VirtualTerm I) : E :=
  t.weight * CMLPolynomialEval.evalMle (t.table q) t.point
```

LeanerVM/Protocol/Spine/Seams.lean:87-88 at b435631

The term's value: weight times the virtual table's extension at the point. *Depends on:* table, CompPoly evalMle.

LinearClaim

[load-bearing]

```
structure LinearClaim (I : M3Instance) where
  terms : List (VirtualTerm I)
  value : E
```

LeanerVM/Protocol/Spine/Seams.lean:91-95 at b435631

A weighted sum of virtual-table extensions equals a value (a zerocheck claim is one term of weight 1 and value 0; a bus form is a list of terms). *Depends on:* VirtualTerm.

LinearClaim.Holds

[load-bearing]

```
def LinearClaim.Holds {I : M3Instance} (q : Column I.μ) (c : LinearClaim I) : Prop :=
  (c.terms.map fun t => t.eval q).sum = c.value
```

LeanerVM/Protocol/Spine/Seams.lean:98-99 at b435631

The sum of the terms' values is the claimed value. *Depends on:* VirtualTerm.eval.

Weight

[load-bearing]


```
structure Weight (μ : ℕ) where
  onCube : CMLPolynomialEval E μ
  mle : Vector E μ → E
  mle_eq : ∀ r, mle r = CMLPolynomialEval.evalMle onCube r
```

LeanerVM/Protocol/Spine/Seams.lean:103-109 at b435631

A weight on the stack (specification Definition 3.13): its values on the cube, an evaluator for its extension, and their agreement. *Depends on:* CompPoly evalMle.

Weight.pair

[load-bearing]

```
def Weight.pair {μ : ℕ} (W : Weight μ) (q : Column μ) : E :=
  ∑ i : Fin (2 ^ μ), W.onCube.get i * ofK (q.values.get i)
```

LeanerVM/Protocol/Spine/Seams.lean:112-113 at b435631

$\sum_x W(x) \cdot q(x)$ over the cube. *Depends on:* Weight, Column.

WeightedClaim

[load-bearing]

```
structure WeightedClaim (I : M3Instance) where
  weight : Weight I.μ
  value : E
```

LeanerVM/Protocol/Spine/Seams.lean:116-120 at b435631

The pairing of a weight with the stack equals a value. *Depends on:* Weight.

WeightedClaim.Holds

[load-bearing]

```
def WeightedClaim.Holds {I : M3Instance} (q : Column I.μ) (c : WeightedClaim I) : Prop :=
  c.weight.pair q = c.value
```

LeanerVM/Protocol/Spine/Seams.lean:123-124 at b435631

True of the stack when the pairing is the value. *Depends on:* Weight.pair.

BusOut

[load-bearing]

```
structure BusOut (I : M3Instance) where
  linear : List (LinearClaim I)
  columns : List (ColumnClaim I)
```

LeanerVM/Protocol/Spine/Seams.lean:130-134 at b435631

What the bus phase hands on: linear claims and column claims. *Depends on:* LinearClaim, ColumnClaim.

TableOut

[load-bearing]

```
structure TableOut (I : M3Instance) where
  columns : List (ColumnClaim I)
```

LeanerVM/Protocol/Spine/Seams.lean:137-139 at b435631

What the table sumcheck hands on: column claims. *Depends on:* ColumnClaim.

PubOut

[load-bearing]

```
structure PubOut (I : M3Instance) where
  columns : List (ColumnClaim I)
```

LeanerVM/Protocol/Spine/Seams.lean:142-144 at b435631

What the public-input phase hands on: column claims (the same type as `TableOut`). *Depends on:* `ColumnClaim`.

FlockOut [load-bearing]

```
structure FlockOut (I : M3Instance) where
  columns : List (ColumnClaim I)
  weighted : List (WeightedClaim I)
```

LeanerVM/Protocol/Spine/Seams.lean:147-151 at b435631

What the Flock phase hands on: column claims and weighted claims. *Depends on:* `ColumnClaim`, `WeightedClaim`.

theStack [load-bearing]

```
abbrev theStack {I : M3Instance} (o : ∀ i, TheOracle I i) : Column I.μ := o 0
```

LeanerVM/Protocol/Spine/Seams.lean:159-159 at b435631

The stack behind the one oracle. *Depends on:* `TheOracle`.

Seam.of [load-bearing]

```
def of {S : Type} (P : S → Column I.μ → Prop) : Set ((S × ∀ i, TheOracle I i) × Unit) :=
  {p | P p.1.1 (theStack p.1.2)}
```

LeanerVM/Protocol/Spine/Seams.lean:166-167 at b435631

A seam from a predicate on the statement and the stack: the set of `((statement, oracles), ())` where it holds of the one oracle. *Depends on:* `theStack`.

Seam.commit [load-bearing]

```
def commit := of I (M3Holds I)
```

LeanerVM/Protocol/Spine/Seams.lean:171-171 at b435631

After the commit: `M3Holds` of the oracle itself. *Depends on:* `of`, `M3Holds`.

Seam.bus [load-bearing]

```
def bus := of I fun (s : I.Stmt × BusOut I) q ↦ (∀ c ∈ s.2.linear, c.Holds q) ∧
  (∀ c ∈ s.2.linear, ∀ t ∈ c.terms, t.poly.totalDegree ≤ I.d) ∧
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q
```

LeanerVM/Protocol/Spine/Seams.lean:175-177 at b435631

After the bus phase: every linear claim holds, every term is within the degree bound, every column claim holds, the lines hold, `aux` holds. *Depends on:* `of`, the claims, `PublicLinesHold`.

Seam.table [load-bearing]

```
def table := of I fun (s : I.Stmt × TableOut I) q ↦
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q
```

LeanerVM/Protocol/Spine/Seams.lean:180-181 at b435631

After the table sumcheck: column claims, lines, `aux`. *Depends on:* `of`.

Seam.pub [load-bearing]

```
def pub := of I fun (s : I.Stmt × PubOut I) q ↦ (∀ c ∈ s.2.columns, c.Holds q) ∧ I.aux q
```

LeanerVM/Protocol/Spine/Seams.lean:184-184 at b435631

After the public-input phase: column claims and aux. *Depends on:* of.

Seam.flock

[load-bearing]

```
def flock := of I fun (s : I.Stmt × FlockOut I) q ↦
  (∀ c ∈ s.2.columns, c.Holds q) ∧ ∀ c ∈ s.2.weighted, c.Holds q
```

LeanerVM/Protocol/Spine/Seams.lean:187-188 at b435631

After the Flock phase: column claims and weighted claims. *Depends on:* of.

Seam.done

[load-bearing]

```
def done := of I fun (_ : Unit) _ ↦ True
```

LeanerVM/Protocol/Spine/Seams.lean:191-191 at b435631

After the opening: nothing (the whole set). *Depends on:* of.

4.1.3 LeanerVM/Protocol/Spine/Phase.lean

Phase.Def

[interface]

```
abbrev Def (StmtIn StmtOut : Type) : Type 1 :=
  Component.Def StmtIn (TheOracle I) Unit StmtOut (TheOracle I) Unit
```

LeanerVM/Protocol/Spine/Phase.lean:37-38 at b435631

A phase: a component from a statement to a statement over the stack, with trivial witnesses. *Depends on:* Component.Def, TheOracle.

Phase.Complete

[interface]

```
abbrev Complete {StmtIn StmtOut : Type} (D : Def I StmtIn StmtOut)
  (relIn : Set ((StmtIn × ∀ i, TheOracle I i) × Unit))
  (relOut : Set ((StmtOut × ∀ i, TheOracle I i) × Unit)) : Type :=
  Component.Complete D relIn relOut
```

LeanerVM/Protocol/Spine/Phase.lean:41-44 at b435631

Its completeness half against two seams. *Depends on:* Component.Complete.

Phase.Security

[interface]

```
abbrev Security {StmtIn StmtOut : Type} (D : Def I StmtIn StmtOut)
  (relIn : Set ((StmtIn × ∀ i, TheOracle I i) × Unit))
  (relOut : Set ((StmtOut × ∀ i, TheOracle I i) × Unit)) : Type 1 :=
  Component.Security D relIn relOut
```

LeanerVM/Protocol/Spine/Phase.lean:47-50 at b435631

Its security half against two seams. *Depends on:* Component.Security.

Phase.passThrough

[interface]

```
abbrev passThrough (f : StmtIn → StmtOut) : Def I StmtIn StmtOut :=
  Component.passThrough (TheOracle I) f
```

LeanerVM/Protocol/Spine/Phase.lean:55-56 at b435631

The pass-through phase. *Depends on:* `Component.passThrough`.

Phase.passThroughComplete

[interface]

```
def passThroughComplete (f : StmtIn → StmtOut)
  {relIn : Set ((StmtIn × ∀ i, TheOracle I i) × Unit)}
  {relOut : Set ((StmtOut × ∀ i, TheOracle I i) × Unit)}
  (h : ∀ s o, ((s, o), ()) ∈ relIn → ((f s, o), ()) ∈ relOut) :
  Complete I (passThrough I f) relIn relOut :=
  Component.passThroughComplete (TheOracle I) f fun s o _ hin ↦ h s o hin
```

LeanerVM/Protocol/Spine/Phase.lean:59-64 at b435631

Its completeness when the map carries one seam into the other. *Depends on:* `Component.passThroughComplete`.

Phase.passThroughSecurity

[interface]

```
def passThroughSecurity (f : StmtIn → StmtOut)
  {relIn : Set ((StmtIn × ∀ i, TheOracle I i) × Unit)}
  {relOut : Set ((StmtOut × ∀ i, TheOracle I i) × Unit)}
  (hc : ∀ s o, ((s, o), ()) ∈ relIn → ((f s, o), ()) ∈ relOut)
  (h : ∀ s o, ((f s, o), ()) ∈ relOut → ((s, o), ()) ∈ relIn) :
  Security I (passThrough I f) relIn relOut :=
  Component.passThroughSecurity (TheOracle I) f (fun s o _ ↦ h s o) fun s o _ ↦ hc s o
```

LeanerVM/Protocol/Spine/Phase.lean:68-74 at b435631

Its security when the map also reflects the second seam into the first (D.3 (a): impossible at the bus seam). *Depends on:* `Component.passThroughSecurity`.

4.1.4 LeanerVM/Protocol/Spine/Compose.lean

commitSpec

[load-bearing]

```
| abbrev commitSpec : ProtocolSpec 1 := Component.sendSpec (Column I.μ)
```

LeanerVM/Protocol/Spine/Compose.lean:51-51 at b435631

The commit phase's schedule: one prover message, the stack. *Depends on:* `sendSpec`.

commitDef

[load-bearing]

```
| abbrev commitDef : Component.Def I.Stmt NoOracle (Column I.μ) I.Stmt (TheOracle I) Unit :=
  Component.sendOracle I.Stmt (Column I.μ)
```

LeanerVM/Protocol/Spine/Compose.lean:55-56 at b435631

The commit phase: the send-oracle component at the stack. *Depends on:* `sendOracle`.

commitExtractor

[load-bearing (named form)]

```
| abbrev commitExtractor := Component.sendExtractor (S := I.Stmt) (M := Column I.μ)
```

LeanerVM/Protocol/Spine/Compose.lean:59-59 at b435631

Its extractor: read the stack off the message. *Depends on:* `sendExtractor`.

commitComplete

[load-bearing (named form)]

```
| def commitComplete : Component.Complete (commitDef I) (M3Rel I) (Seam.commit I) :=
  Component.sendOracleComplete (M3Rel I)
```

LeanerVM/Protocol/Spine/Compose.lean:63-64 at b435631

Its completeness from M3Rel to Seam.commit. *Depends on:* sendOracleComplete.

commitSecurity

[load-bearing (named form)]

```
def commitSecurity : Component.Security (commitDef I) (M3Rel I) (Seam.commit I) :=
  Component.sendOracleSecurity (M3Rel I)
```

LeanerVM/Protocol/Spine/Compose.lean:67-68 at b435631

Its security at error zero. *Depends on:* sendOracleSecurity.

Phases

[load-bearing]

```
structure Phases where
  bus : Phase.Def I I.Stmt (I.Stmt × BusOut I)
  table : Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I)
  pub : Phase.Def I (I.Stmt × TableOut I) (I.Stmt × PubOut I)
  flock : Phase.Def I (I.Stmt × PubOut I) (I.Stmt × FlockOut I)
  opening : Phase.Def I (I.Stmt × FlockOut I) Unit
```

LeanerVM/Protocol/Spine/Compose.lean:73-84 at b435631

The five phases after the commit, by their statement types. *Depends on:* Phase.Def, the four outputs.

Phases.toDef

[load-bearing]

```
def Phases.toDef (P : Phases I) :
  Component.Def I.Stmt NoOracle (Column I.μ) Unit (TheOracle I) Unit :=
  (((((commitDef I).append P.bus).append P.table).append P.pub).append P.flock).append P.opening
```

LeanerVM/Protocol/Spine/Compose.lean:89-91 at b435631

The whole protocol as one component: the commit, then five appends. *Depends on:* commitDef, Def.append.

Phases.Complete

[load-bearing]

```
structure Phases.Complete (P : Phases I) where
  bus : Phase.Complete I P.bus (Seam.commit I) (Seam.bus I)
  table : Phase.Complete I P.table (Seam.bus I) (Seam.table I)
  pub : Phase.Complete I P.pub (Seam.table I) (Seam.pub I)
  flock : Phase.Complete I P.flock (Seam.pub I) (Seam.flock I)
  opening : Phase.Complete I P.opening (Seam.flock I) (Seam.done I)
```

LeanerVM/Protocol/Spine/Compose.lean:94-104 at b435631

The five completeness halves against the seams. *Depends on:* Phase.Complete, the seams.

Phases.Security

[load-bearing]

```
structure Phases.Security (P : Phases I) where
  bus : Phase.Security I P.bus (Seam.commit I) (Seam.bus I)
  table : Phase.Security I P.table (Seam.bus I) (Seam.table I)
  pub : Phase.Security I P.pub (Seam.table I) (Seam.pub I)
  flock : Phase.Security I P.flock (Seam.pub I) (Seam.flock I)
  opening : Phase.Security I P.opening (Seam.flock I) (Seam.done I)
```

LeanerVM/Protocol/Spine/Compose.lean:113-123 at b435631

The five security halves against the seams. *Depends on:* Phase.Security, the seams.

Phases.Security.toDef

[load-bearing (named form)]

```
def Phases.Security.toDef {P : Phases I} (S : P.Security) :
  Component.Security P.toDef (M3Rel I) (Seam.done I) :=
  (((commitSecurity I).append S.bus).append S.table).append S.pub).append
  S.flock).append
  S.opening
```

LeanerVM/Protocol/Spine/Compose.lean:126-129 at b435631

The composed security: the commit's, then five appends. *Depends on:* `commitSecurity`, `Security.append`.

leanVmPiop

[load-bearing]

```
def leanVmPiop (P : Phases I) :
  OracleReduction [] I.Stmt NoOracle (Column I.μ) Unit (TheOracle I) Unit P.toDef.pSpec :=
  P.toDef.red
```

LeanerVM/Protocol/Spine/Compose.lean:134-136 at b435631

The oracle protocol: the composed reduction. *Depends on:* `Phases.toDef`.

leanVmVerifier

[load-bearing]

```
def leanVmVerifier (P : Phases I) :
  OracleVerifier [] I.Stmt NoOracle Unit (TheOracle I) P.toDef.pSpec :=
  P.toDef.red.verifier
```

LeanerVM/Protocol/Spine/Compose.lean:139-141 at b435631

Its verifier. *Depends on:* `Phases.toDef`.

leanVmProver

[interface]

```
def leanVmProver (P : Phases I) :
  OracleProver [] I.Stmt NoOracle (Column I.μ) Unit (TheOracle I) Unit P.toDef.pSpec :=
  P.toDef.red.prover
```

LeanerVM/Protocol/Spine/Compose.lean:144-146 at b435631

Its honest prover (not in either theorem's statement; `leanVmPiop` carries it). *Depends on:* `Phases.toDef`.

piopError

[load-bearing]

```
def piopError (P : Phases I) : P.toDef.pSpec.ChallengeIdx → ℝ≥0 := P.toDef.err
```

LeanerVM/Protocol/Spine/Compose.lean:149-149 at b435631

Its error per challenge: the phases' declarations side by side (section F). *Depends on:* `Def.append`.

piopExtractor

[load-bearing (named form)]

```
def piopExtractor (P : Phases I) (S : P.Security) :
  Extractor.RoundByRound (OracleSpec.emptySpec.{0, 0}) (I.Stmt × ∀ i, NoOracle i)
  (Column I.μ) Unit P.toDef.pSpec S.toDef.witMid :=
  S.toDef.extractor
```

LeanerVM/Protocol/Spine/Compose.lean:153-156 at b435631

Its extractor: the composed one (D.5). *Depends on:* `Phases.Security.toDef`.

piop_perfectCompleteness

[master theorem]

```
theorem piop_perfectCompleteness (P : Phases I) (C : P.Complete) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  (leanVmPiop P).perfectCompleteness init impl (M3Rel I) (Seam.done I) :=
```

LeanerVM/Protocol/Spine/Compose.lean:162-165 at b435631

Given every phase's completeness, the honest prover convinces the verifier with probability one on every (input, q) with M3Holds I input q. *Depends on:* leanVmPiop, M3Rel, Seam.done, ArkLib perfectCompleteness.

piop_rbrKnowledgeSoundness

[master theorem]

```
theorem piop_rbrKnowledgeSoundness (P : Phases I) (S : P.Security) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  (leanVmVerifier P).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (M3Rel I)
  (Seam.done I) S.toDef.witMid (piopExtractor P S) (S.toDef.kSF init impl)
  (piopError P) :=
```

LeanerVM/Protocol/Spine/Compose.lean:172-177 at b435631

Given every phase's security, the verifier is round-by-round knowledge sound at the composed error, for the composed extractor and state function. *Depends on:* leanVmVerifier, piopExtractor, piopError, ArkLib rbrKnowledgeSoundnessWorstCaseWith.

piop_rbrKnowledgeSoundness_exists

[master theorem (existential form)]

```
theorem piop_rbrKnowledgeSoundness_exists (P : Phases I) (S : P.Security) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  (leanVmVerifier P).toVerifier.rbrKnowledgeSoundnessWorstCase init impl (M3Rel I)
  (Seam.done I) (piopError P) :=
```

LeanerVM/Protocol/Spine/Compose.lean:181-185 at b435631

The same with the extractor and state function forgotten. *Depends on:* piop_rbrKnowledgeSoundness.

Helpers and test fixtures of Compose.lean.

Declaration	Lines	Class	Meaning
Phases.Complete.toDef	107-110	helper	The composed completeness.

4.1.5 LeanerVM/Protocol/Spine/Toy.lean

Helpers and test fixtures of Toy.lean.

Declaration	Lines	Class	Meaning
shape	36-36	test fixture	One table of height two and width three.
slice	42-44	test fixture	Column i is the slice at cells 2i, 2i+1.
extend	47-48	test fixture	The selector of column i: (z, i mod 2, i div 2).
read_eval	52-57	test fixture	The law for the three slices.
layout	60-63	test fixture	The toy's layout.
constraint	66-66	test fixture	Column 2 is Boolean: $X_2^2 - X_2$.
flush	77-78	test fixture	The one flush: (X_0 , 0, ...), pushed.
boundary	89-92	test fixture	The one boundary block: pulls ([1, 1], 0, ...).
toy	95-111	test fixture	The toy instance: d = 2, column 1 a count column, the public line on column 2 with cells (input, 0), aux := True.
honest	114-114	test fixture	The honest stack [1, 1, 1, 1, 1, 0, 0, 0].

4.1.6 LeanerVM/Protocol/ToArkLib/Oracles.lean

NoOracle

[load-bearing]

```
abbrev NoOracle : Fin 0 → Type := fun i ↦ i.elim0
```

LeanerVM/Protocol/ToArkLib/Oracles.lean:26-26 at b435631

The oracle family with no oracle (before the commit). *Depends on:* —.

OneOracle

[load-bearing]

```
| abbrev OneOracle (M : Type) : Fin 1 → Type := fun _ ↦ M
```

LeanerVM/Protocol/ToArkLib/Oracles.lean:29-29 at b435631

The oracle family with exactly one oracle of type M (the stack, from the commit on). *Depends on:* —.

Helpers and test fixtures of Oracles.lean.

Declaration	Lines	Class	Meaning
noOracle_eq	32-32	helper	Any two empty oracle families are equal.

4.1.7 LeanerVM/Protocol/ToArkLib/Component.lean

Component.Def

[interface]

```
structure Def (StmntIn : Type) {i1 : Type} (OStmntIn : i1 → Type) (WitIn : Type)
  (StmntOut : Type) {i0 : Type} (OStmntOut : i0 → Type) (WitOut : Type)
  [V i, OracleInterface (OStmntIn i)] [V i, OracleInterface (OStmntOut i)] where
  n : ℕ
  pSpec : ProtocolSpec n
  [msgOracle : V i, OracleInterface (pSpec.Message i)]
  [chalSample : V i, SampleableType (pSpec.Challenge i)]
  red : OracleReduction []. StmntIn OStmntIn WitIn StmntOut OStmntOut WitOut pSpec
  err : pSpec.ChallengeIdx → ℝ≥0
```

LeanerVM/Protocol/ToArkLib/Component.lean:52-66 at b435631

A component: its number of rounds, its schedule (who speaks, what type), the instances that every message can be queried and every challenge sampled, the honest prover with the verifier (an ArkLib `OracleReduction` over the empty shared oracle), and the knowledge error it declares per challenge. *Depends on:* ArkLib `ProtocolSpec`, `OracleReduction`, `OracleInterface`, `VCVio` `SampleableType`.

Component.Complete

[interface]

```
structure Complete (D : Def StmntIn OStmntIn WitIn StmntOut OStmntOut WitOut)
  (relIn : Set ((StmntIn × V i, OStmntIn i) × WitIn))
  (relOut : Set ((StmntOut × V i, OStmntOut i) × WitOut)) where
  outputPure : D.red.prover.OutputIsPure
  guarded : D.red.toReduction.verifier.GuardedForm
  complete : V {σ : Type} (init : ProbComp σ) (impl : QueryImpl []. (StateT σ ProbComp)),
    D.red.perfectCompleteness init impl relIn relOut
```

LeanerVM/Protocol/ToArkLib/Component.lean:76-85 at b435631

The completeness half a phase must supply: the prover's output is pure (redundant, D.7), the verifier is a check followed by a verdict (data), and perfect completeness from any shared state. *Depends on:* Def, ArkLib `OutputIsPure`, `GuardedForm`, `perfectCompleteness`.

Component.Security

[interface]

```
structure Security (D : Def StmntIn OStmntIn WitIn StmntOut OStmntOut WitOut)
  (relIn : Set ((StmntIn × V i, OStmntIn i) × WitIn))
  (relOut : Set ((StmntOut × V i, OStmntOut i) × WitOut))
  extends Complete D relIn relOut where
  witMid : Fin (D.n + 1) → Type
  extractor : Extractor.RoundByRound (OracleSpec.emptySpec.{0, 0}) (StmntIn × V i, OStmntIn i)
    WitIn WitOut D.pSpec witMid
```



```

kSF : ∀ {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)),
  D.red.verifier.toVerifier.KnowledgeStateFunction init impl relIn relOut extractor
rbr : ∀ {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)),
  D.red.verifier.toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl relIn relOut witMid
extractor (kSF init impl) D.err

```

LeanerVM/Protocol/ToArkLib/Component.lean:90-106 at b435631

The security half: completeness, the intermediate witness types, a round-by-round extractor, its knowledge state function from any shared state, and worst-case round-by-round knowledge soundness for them at the declared error. *Depends on:* Complete, ArkLib Extractor.RoundByRound, KnowledgeStateFunction, rbrKnowledgeSoundnessWorstCaseWith.

Component.Def.append

[load-bearing]

```

def Def.append (D₁ : Def Stmt₁ OStmt₁ Wit₁ Stmt₂ OStmt₂ Wit₂)
  (D₂ : Def Stmt₂ OStmt₂ Wit₂ Stmt₃ OStmt₃ Wit₃) : Def Stmt₁ OStmt₁ Wit₁ Stmt₃ OStmt₃ Wit₃ where
  n := D₁.n + D₂.n
  pSpec := D₁.pSpec ++p D₂.pSpec
  red := D₁.red.append D₂.red
  err := Sum.elim D₁.err D₂.err (ChallengeIdx.sumEquiv.symm

```

LeanerVM/Protocol/ToArkLib/Component.lean:143-148 at b435631

Two components in sequence: rounds add, schedules concatenate, reductions append (ArkLib), each challenge keeps its component's error. *Depends on:* ArkLib OracleReduction.append, ChallengeIdx.sumEquiv.

Helpers and test fixtures of Component.lean.

Declaration	Lines	Class	Meaning
Component.stateFunctionOfEq	110-120	helper (in the named statement)	Transport a knowledge state function along an equality of verifiers (needed because ArkLib's appended oracle verifier is only propositionally the appended ordinary verifier).
rbrKnowledgeSoundnessWorstCaseWith_of_eq	124-135	helper	The same transport for the bound.
Component.guardedAppend	156-159	helper (in the named statement)	The guarded form of an appended verifier from those of its parts (ArkLib's GuardedForm.append, cast along the append equation).
Component.Complete.append	162-169	helper (in the named statement)	Completeness composes: purity, guarded form, and ArkLib's guarded-append completeness theorem.
Component.Security.append	174-184	helper (in the named statement)	Security composes: extractors appended through the first verdict (ArkLib), state functions by the ported appendGuarded, the bound by the ported theorem.

4.1.8 LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean

Helpers and test fixtures of KnowledgeAppend.lean.

Declaration	Lines	Class	Meaning
Verifier.KnowledgeStateFunction.appendGuarded	474-488	helper (in the named statement)	The knowledge state function of two appended verifiers, the first guarded: the first's state up to the seam, then the first check together with the second's state on the first verdict (ported from ArkLib #615).

Declaration	Lines	Class	Meaning
<code>Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first</code>	510-582	helper	Round-by-round knowledge soundness composes across an append whose first verifier is guarded, each challenge keeping its error (ported from ArkLib #615; admitted upstream at the pin).

4.1.9 LeanerVM/Protocol/ToArkLib/PassThrough.lean

Component.passThrough

[interface]

```
def passThrough (f : StmtIn → StmtOut) : Def StmtIn 0Stmt W StmtOut 0Stmt W where
  n := 0
  pSpec := !p[]
  red := (passThroughProver 0Stmt f, passThroughVerifier 0Stmt f)
  err := fun i ↦ Fin.elim0 i.1
```

LeanerVM/Protocol/ToArkLib/PassThrough.lean:53-57 at b435631

The component with no round that maps the statement and keeps the oracles and the witness: the shape of bookkeeping steps. *Depends on:* `passThroughProver`, `passThroughVerifier`, `keep0oracles`.

Component.passThroughComplete

[interface]

```
def passThroughComplete (f : StmtIn → StmtOut)
  {relIn : Set ((StmtIn × ∀ i, 0Stmt i) × W)} {relOut : Set ((StmtOut × ∀ i, 0Stmt i) × W)}
  (h : ∀ s o w, ((s, o), w) ∈ relIn → ((f s, o), w) ∈ relOut) :
  Complete (passThrough 0Stmt f) relIn relOut where
  outputPure := (λ_, fun _ ↦ rfl)
  guarded := (passThroughPure 0Stmt f).toGuardedForm
```

LeanerVM/Protocol/ToArkLib/PassThrough.lean:81-95 at b435631

Its completeness, whenever the map carries the input relation into the output relation. *Depends on:* `passThrough`, `GuardedVerdict`.

Component.passThroughSecurity

[interface]

```
def passThroughSecurity (hc : ∀ s o w, ((s, o), w) ∈ relIn → ((f s, o), w) ∈ relOut) :
  Security (passThrough 0Stmt f) relIn relOut where
  toComplete := passThroughComplete 0Stmt f hc
  witMid := fun _ ↦ W
  extractor := passThroughExtractor 0Stmt
  kSF := passThroughStateFunction 0Stmt f h
```

LeanerVM/Protocol/ToArkLib/PassThrough.lean:132-138 at b435631

Its security at error zero, with the extractor that keeps the witness, whenever the map also reflects the output relation into the input relation. *Depends on:* `passThroughExtractor`, `passThroughStateFunction`, `passThrough_rbr`.

4.1.10 LeanerVM/Protocol/ToArkLib/SendOracle.lean

Component.sendOracle

[load-bearing]

```
def sendOracle : Def S NoOracle M S (OneOracle M) Unit where
  n := 1
  pSpec := sendSpec M
  red := (sendProver S M, sendVerifier S M)
  err := fun _ ↦ 0
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:61-65 at b435631

The component whose one message becomes the one oracle: the prover sends the witness, the verifier keeps the statement and exposes the message; no challenge, error zero. *Depends on:* sendSpec, sendProver, sendVerifier, sendEmbedding.

Component.sendOracle_relOut

[load-bearing (named form)]

```
def sendOracle_relOut (rel : Set ((S × ∀ i, NoOracle i) × M)) :
  Set ((S × ∀ i, OneOracle M i) × Unit) :=
  {p | ((p.1.1, fun i ↦ i.elim0), p.1.2 0) ∈ rel}
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:71-73 at b435631

The input relation read on the oracle instead of the witness (definitionally Seam.commit at M3Rel). *Depends on:* —.

Component.sendOracleComplete

[load-bearing (named form)]

```
def sendOracleComplete (rel : Set ((S × ∀ i, NoOracle i) × M)) :
  Complete (sendOracle S M) rel (sendOracle_relOut rel) where
  outputPure := sendOracle_outputPure
  guarded := (sendVerifierPure (S := S) (M := M)).toGuardedForm
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:121-125 at b435631

Its completeness half. *Depends on:* sendVerifierPure, sendOracle_complete.

Component.sendExtractor

[load-bearing (named form)]

```
def sendExtractor : Extractor.RoundByRound (OracleSpec.emptySpec.{0, 0}) (S × ∀ i, NoOracle i) M
  Unit (sendSpec M) (sendWitMid M) where
  extractMid := fun m _ tr _ ↦ tr (0, Nat.succ_pos m.val)
  extractOut := fun _ tr _ ↦ tr 0
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:134-138 at b435631

Its extractor: read the message, at every round. *Depends on:* ArkLib Extractor.RoundByRound.

Component.sendStateFunction

[load-bearing (named form)]

```
def sendStateFunction :
  (sendVerifier S M).toVerifier.KnowledgeStateFunction init impl rel (sendOracle_relOut rel)
  (sendExtractor (S := S) (M := M)) where
  toFun := fun m stmt tr w ↦
    if h : m.val = 0 then (stmt, w) ∈ rel else (stmt, tr (0, Nat.pos_of_ne_zero h)) ∈ rel
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:145-161 at b435631

Its knowledge state function: before the message, the relation of the candidate witness; after it, the relation of the message. *Depends on:* sendExtractor, GuardedVerdict.

Component.sendOracleSecurity

[load-bearing (named form)]

```
def sendOracleSecurity : Security (sendOracle S M) rel (sendOracle_relOut rel) where
  toComplete := sendOracleComplete rel
  witMid := sendWitMid M
  extractor := sendExtractor
  kSF := sendStateFunction rel
```

LeanerVM/Protocol/ToArkLib/SendOracle.lean:172-177 at b435631

Its security half at error zero. *Depends on:* sendOracleComplete, sendExtractor, sendStateFunction, sendOracle_rbr.

4.1.11 LeanerVM/Protocol/ToArkLib/Refinement.lean

Refinement

[interface (the adaptor)]

```
structure Refinement {Stmt : Type} {W1 W2 : Type _} (R : Set (Stmt × W1))
  (S : Set (Stmt × W2)) where
  map : Stmt → W1 → W2
  map_valid : ∀ x w, (x, w) ∈ R → (x, map x w) ∈ S
```

LeanerVM/Protocol/ToArkLib/Refinement.lean:31-36 at b435631

A map on witnesses between two relations over the same statements, sending valid witnesses to valid witnesses; the two witness types may live in different universes (D.7). *Depends on:* —.

Refinement.map_option_valid

[interface (the adaptor)]

```
theorem map_option_valid {R : Set (Stmt × W1)} {S : Set (Stmt × W2)} (f : Refinement R S)
  (x : Stmt) (w? : Option W1) (h : ∀ w ∈ w?, (x, w) ∈ R) :
  ∀ w' ∈ w?.map (f.map x), (x, w') ∈ S :=
```

LeanerVM/Protocol/ToArkLib/Refinement.lean:55-60 at b435631

On an extractor's output slot: if every witness the slot may hold is valid for R, every witness of the mapped slot is valid for S. Pointwise; the probabilistic form is D.7. *Depends on:* **Refinement**.

Helpers and test fixtures of Refinement.lean.

Declaration	Lines	Class	Meaning
Extractor.Straightline.map	68-71	helper (no consumer)	Post-compose ArkLib's straight-line extractor with a witness map (same universe).

4.1.12 LeanerVM/Protocol/ToArkLib/GuardedVerdict.lean

Helpers and test fixtures of GuardedVerdict.lean.

Declaration	Lines	Class	Meaning
Verifier.GuardedForm.of_probEvent_pos	40-69	helper	If a guarded verifier can output a statement satisfying P, its check passes and its verdict satisfies P: the last obligation of every knowledge state function.
Reduction.mem_support_run_of_guarded	73-112	helper	Every outcome of a run with a guarded verifier is a prover run with the verdict where the check passes, a rejection otherwise: how perfect completeness is proved.

4.1.13 LeanerVM/Protocol/ToArkLib/KeepOracles.lean

Helpers and test fixtures of KeepOracles.lean.

Declaration	Lines	Class	Meaning
keepOracles	34-37	helper	The output-oracle description “the output oracles are the input oracles”.
OracleVerifier.materializeOutput_of_keepOracles	42-50	helper	Such a verifier outputs the oracles it was given.

4.1.14 LeanerVM/Protocol/Field.lean

Column

[load-bearing]

```
structure Column (n : ℕ) where
  values : CMLPolynomialEval K n
```

LeanerVM/Protocol/Field.lean:67-69 at b435631

A table of 2^n values of K , indexed by the cube $\{0,1\}^n$ low bit first: what is committed. Wrapped in a structure so that its oracle interface is the evaluation one, not ArkLib's position query on **Vector**. *Depends on:* **CompPoly CMLPolynomialEval**.

evalOracle

[load-bearing]

```
instance evalOracle (n : ℕ) : OracleInterface (Column n) where
  Query := Vector E n
  toOC :=
    { spec := (Vector E n) → E
      impl := fun r => do return CMLPolynomialEval.eval₂Mle (← read).values (algebraMap K E) r }
```

LeanerVM/Protocol/Field.lean:102-106 at b435631

How the verifier queries a committed column: a query is a point $r \in E^n$, the answer is the multilinear extension $\tilde{q}(r)$ with the entries lifted from K to E . Every seam is read through it. *Depends on:* **Column**, **CompPoly eval₂Mle**, **ArkLib OracleInterface**.

instSampleableTypeE

[load-bearing]

```
instance instSampleableTypeE : SampleableType E := SampleableType.ofEquiv limbsEquiv
```

LeanerVM/Protocol/Field.lean:93-93 at b435631

Uniform sampling of a challenge in E as three independent uniform limbs (VCVio needs a sampler for every challenge type). *Depends on:* **limbsEquiv**, **VCVio SampleableType**.

4.2 The public-input phase, declaration by declaration

LeanerVM/Protocol/PublicInput.lean at b435631, 400 lines, a module; namespace **LeanerVM.Protocol**, @[expose] public section, inner namespace **PublicInput**. Classes: **load-bearing** (in the statement of a top-level result, or read by later work), **interface** (what other layers name), **helper** (proof support). The future executable verifier (Layer 12) reads, per the blueprint (:1049-1051), **pSpec**, **check**, **pooled**, and through them **expectedValues**, **lineValue**, **linePoint**: marked “read by Layer 12”.

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier **dossiers/code-pubininput.md**, whose codes the headings keep in parentheses: A (catalogue) is section 4.2; B (faithfulness), E (the flag **PublicLine.sent**) and F (statement checks) is section 8.9.1.2; C (mutation probes) and D (the top limb) is ?? B.1.1.3; G (findings) is section 8.9.2.2.

Each load-bearing and interface declaration is shown with the text the dossier copied (for a definition its signature and, where the dossier gives it, its body; for a theorem its statement), its line at **b435631**, its meaning and what it depends on. The helpers follow in one table.

4.2.1 LeanerVM/Protocol/PublicInput.lean

linePoint

[load-bearing, read by Layer 12]

```
def linePoint {n : ℕ} (hn : 0 < n) (r : E) : Vector E n := Vector.cast (by omega : 1 + (n - 1) = n) (#v[r] ++
  ↪ Vector.replicate (n - 1) (0 : E))
```

LeanerVM/Protocol/PublicInput.lean, line 65, at b435631

The point $(r, \theta, \dots, \theta)$. *Depends on:* E.

eval₂Mle_linePoint

[load-bearing (gives linePoint its meaning)]

```
theorem eval2Mle_linePoint {n : ℕ} (hn : 0 < n) (q : CMLPolynomialEval K n) (r : E) :
  ↪ CMLPolynomialEval.eval2Mle q (algebraMap K E) (linePoint hn r) = (1 - r) * ofK (q.get (0, Nat.two_pow_pos
  ↪ n)) + r * ofK (q.get (1, Nat.one_lt_two_pow hn.ne'))
```

LeanerVM/Protocol/PublicInput.lean, line 71, at b435631

The extension at that point is the line through cells 0 and 1: the bit order is proved (coordinate 0 = low bit). *Depends on:* CompPoly eval₂Mle, Layer 1's evalMle_append_boolVec, boolVec_zero.

pSpec

[interface, read by Layer 12]

```
def pSpec : ProtocolSpec 2 := (!v[.V_to_P, .P_to_V], !v[E, List E])
```

LeanerVM/Protocol/PublicInput.lean, line 99, at b435631

The schedule: a challenge in E, then a message List E. *Depends on:* ArkLib ProtocolSpec.

The two instances

[interface]

```
instance : ∀ i, OracleInterface (pSpec.Message i)
instance : ∀ i, SampleableType (pSpec.Challenge i)
```

LeanerVM/Protocol/PublicInput.lean, line 101, 105, at b435631

The message is read whole; the challenge is uniform in E (Protocol/Field.lean). *Depends on:* ArkLib, VCVio.

error

[load-bearing]

```
noncomputable def error : ℝ≥0 := 1 / Fintype.card E
```

LeanerVM/Protocol/PublicInput.lean, line 111, at b435631

The knowledge error. *Depends on:* Fintype E.

lineValue

[load-bearing, read by Layer 12]

```
def lineValue (r : E) (l : PublicLine I.toShape) : E := (1 + r) * ofK l.cell0 + r * ofK l.cell1
```

LeanerVM/Protocol/PublicInput.lean, line 119, at b435631

The line through the statement's two cells at r. *Depends on:* PublicLine.

lineClaim

[load-bearing]

```
def lineClaim (r : E) (l : PublicLine I.toShape) : ColumnClaim I := (l.col, linePoint l.pos r, lineValue I r l)
```

LeanerVM/Protocol/PublicInput.lean, line 123, at b435631

The claim “column l.col at $(r, \theta, \dots)$ equals the line's value”. *Depends on:* ColumnClaim.

pooled

[load-bearing, read by Layer 12]

```
def pooled (s : I.Stmt × TableOut I) (r : E) : I.Stmt × PubOut I := (s.1, (s.2.columns ++ (I.publicLines
  ↪ s.1).map (lineClaim I r)))
```

LeanerVM/Protocol/PublicInput.lean, line 127, at b435631

The verifier's output: the claims received, then one per line. *Depends on:* TableOut, PubOut.

expectedValues [load-bearing, read by Layer 12]

```
def expectedValues (input : I.Stmt) (r : E) : List E := ((I.publicLines input).filter (·.sent)).map (lineValue ↦ I r)
```

LeanerVM/Protocol/PublicInput.lean, line 132, at b435631

The honest message. *Depends on:* PublicLine.sent.

check [interface only: no theorem of the phase depends on it (C.4), read by Layer 12]

```
def check (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool := decide (cs = expectedValues I s.1 r)
```

LeanerVM/Protocol/PublicInput.lean, line 136, at b435631

The check: the message is the expected values. *Depends on:* nothing of the phase.

prover [load-bearing]

```
def prover : OracleProver [] (I.Stmt × TableOut I) (TheOracle I) Unit (I.Stmt × PubOut I) (TheOracle I) Unit ↦ pSpec
```

LeanerVM/Protocol/PublicInput.lean, line 208, at b435631

The honest prover: receives r, sends expectedValues, outputs pooled. *Depends on:* ArkLib OracleProver.

verifier [load-bearing]

```
def verifier : OracleVerifier [] (I.Stmt × TableOut I) (TheOracle I) (I.Stmt × PubOut I) (TheOracle I) pSpec
```

LeanerVM/Protocol/PublicInput.lean, line 229, at b435631

Reads the message, checks, pools or rejects; keeps the stack as the oracle. *Depends on:* check, pooled, keepOracles.

verifier_verify [load-bearing (both proofs start here)]

```
theorem verifier_verify ... : (verifier I).toVerifier.verify (s, o) tr = if check I s (tr 0) (tr 1) then pure ↦ (pooled I s (tr 0), o) else failure
```

LeanerVM/Protocol/PublicInput.lean, line 253, at b435631

The verdict, as an ordinary verifier. *Depends on:* materializeOutput_of_keepOracles, simulateQ_optionT_bind_run (VCVio).

guarded [load-bearing (ArkLib's composition needs it)]

```
def guarded : (verifier I).toVerifier.GuardedForm
```

LeanerVM/Protocol/PublicInput.lean, line 273, at b435631

The verifier as check + verdict data. *Depends on:* verifier_verify.

complete [load-bearing]

```
theorem complete ... : (OracleReduction.mk (prover I) (verifier I)).perfectCompleteness init impl (Seam.table I) ↦ (Seam.pub I)
```

LeanerVM/Protocol/PublicInput.lean, line 300, at b435631

Perfect completeness. *Depends on:* `mem_support_run_of_guarded`, `pooled_mem_pub`.

extractor [load-bearing]

```
def extractor : Extractor.RoundByRound (OracleSpec.emptySpec.{0, 0}) ((I.Stmt × TableOut I) × ∀ i, TheOracle I
  ↪ i) Unit Unit pSpec (fun _ ↪ Unit)
```

LeanerVM/Protocol/PublicInput.lean, line 317, at b435631

The trivial extractor. *Depends on:* `ArkLib`.

stateFunction [load-bearing]

```
def stateFunction : (verifier I).toVerifier.KnowledgeStateFunction init impl (Seam.table I) (Seam.pub I)
  ↪ (extractor I)
```

LeanerVM/Protocol/PublicInput.lean, line 327, at b435631

The three-round invariant. *Depends on:* `of_probEvent_pos`.

rbr [load-bearing]

```
theorem rbr : (verifier I).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (Seam.table I) (Seam.pub I)
  ↪ (fun _ ↪ Unit) (extractor I) (stateFunction I init impl) (fun _ ↪ error)
```

LeanerVM/Protocol/PublicInput.lean, line 350, at b435631

Round-by-round knowledge soundness at $1/|E|$. *Depends on:* `probEvent_uniformSample_le_of_subsingl`
`eton`, `bad_challenge_unique`.

publicInputPhase [interface]

```
noncomputable def publicInputPhase : Phase.Def I (I.Stmt × TableOut I) (I.Stmt × PubOut I)
```

LeanerVM/Protocol/PublicInput.lean, line 376, at b435631

The phase. *Depends on:* All above.

publicInputComplete [interface]

```
def publicInputComplete : Phase.Complete I (publicInputPhase I) (Seam.table I) (Seam.pub I)
```

LeanerVM/Protocol/PublicInput.lean, line 384, at b435631

The completeness bundle. *Depends on:* `guarded`, `complete`.

publicInputSecurity [interface]

```
def publicInputSecurity : Phase.Security I (publicInputPhase I) (Seam.table I) (Seam.pub I)
```

LeanerVM/Protocol/PublicInput.lean, line 391, at b435631

The security bundle. *Depends on:* `extractor`, `stateFunction`, `rbr`.

Helpers of `PublicInput.lean`.

Declaration	Line	Class	Meaning
<code>lineClaim_holds_iff</code>	141 (private)	helper	A line's claim holds iff the stack's line at <code>r</code> is the statement's. (<i>depends on:</i> <code>evalzMle_linePoint</code>)

Declaration	Line	Class	Meaning
line_challenge_unique	152 (private)	helper (the whole of the soundness argument)	Two lines through cells in K that differ agree at one r at most. (<i>depends on:</i> ofK_injective, char 2)
pooled_mem_pub	174 (private)	helper (completeness)	From the table seam, the pool is in the public seam at every r . (<i>depends on:</i> the two above)
bad_challenge_unique	186 (private)	helper (soundness)	Outside the table seam, at most one r puts the pool in the public seam. (<i>depends on:</i> the three above)
def queryValues : OracleCom p [pSpec.Message]₀ (List E)	223	helper	The verifier’s read of the message. (<i>depends on:</i> ArkLib oracle query)
simulateQ_queryValues	239 (private)	helper	Reading the message returns the transcript’s entry. (<i>depends on:</i> VCVio simulateQ)
prover_run_support	282 (private)	helper	Every prover run sends expectedValues and outputs pooled. (<i>depends on:</i> ArkLib processRound lemmas)

4.2.2 The generic modules the phase uses

The three generic modules (all at **b435631**; the brief’s section 8 says the first and third were restated at **144c5aa** on VCVio’s new probability notation):

Verifier.GuardedForm.of_probEvent_pos [load-bearing (used by three components)]

```
theorem Verifier.GuardedForm.of_probEvent_pos {V : Verifier oSpec StmtIn StmtOut pSpec} (G : V.GuardedForm) ...
  ⇐ (h : Pr[P | OptionT.mk do (simulateQ impl (V.run stmt tr)).run' (← init)] > 0) : G.check stmt tr = true ∧ P
  ⇐ (G.out stmt tr)
```

LeanerVM/Protocol/ToArkLib/GuardedVerdict.lean, line 40, at **b435631**

If a guarded verifier can output a statement with property P , its check passes and its verdict has P : the last obligation of a knowledge state function.

Reduction.mem_support_run_of_guarded [load-bearing]

```
theorem Reduction.mem_support_run_of_guarded (reduction : Reduction ...) (G : reduction.verifier.GuardedForm)
  ⇐ (stmt) (wit) {y} (hy : y ∈ support (reduction.run stmt wit).run) : ∃ pr ∈ support (reduction.prover.run
  ⇐ stmt wit), y = if G.check stmt pr.1 then some (pr, G.out stmt pr.1) else none
```

LeanerVM/Protocol/ToArkLib/GuardedVerdict.lean, line 73, at **b435631**

Every outcome of a run is a prover run plus the verdict or a rejection: completeness reduces to the prover’s runs.

keepOracles [interface]

```
def keepOracles : OracleOutputEmbedding 0Stmt pSpec.Message 0Stmt
```

LeanerVM/Protocol/ToArkLib/KeepOracles.lean, line 34, at **b435631**

“the output oracles are the input oracles”.

probEvent_uniformSample_le_of_subsingleton [load-bearing (the bound of rbr)]

```
theorem probEvent_uniformSample_le_of_subsingleton (p : α → Prop) (h : ∀ a b, p a → p b → a = b) : Pr[p | $ᵗ α]
  ⇐ ≤ ((1 / Fintype.card α : ℝ≥0) : ℝ≥0∞)
```

LeanerVM/Protocol/ToVCVio/UniformSample.lean, line 44, at **b435631**

An event any two of whose witnesses are equal has probability at most $1/|\alpha|$.

Helpers among the generic modules.

Declaration	File and line	Class	Meaning
<code>theorem OracleVerifier.materializeOutput_of_keepOracles ... (h : V.outputOracle = .i nl (keepOracles 0stmt pSpec)) ... : V.materializeOutput challenges o messages = o</code>	<code>ToArkLib/KeepOracle.lean:42</code>	helper	Such a verifier outputs the oracles it was given.
<code>theorem probEvent_uniformSample_le_of_card_le (p : $\alpha \rightarrow \text{Prop}$) [DecidablePred p] {k : $\mathbb{N}$} (h : (Finset.univ.filter p).card $\leq k$) : $\text{Pr}[p \mid \\$^t \alpha] \leq ((k / \text{Fintype.card } \alpha) : \mathbb{R}_{\geq 0}) : \mathbb{R}_{\geq 0\infty}$</code>	<code>ToVCVio/UniformSample.lean:35</code>	helper	An event with at most k witnesses has probability at most $k/ \alpha $ under a uniform sample.

The two `ToVCVio` bounds and the two `ToArkLib` verdict lemmas are generic in the sense the repository asks of `To*` folders (any oracle spec, any schedule, any finite type); checked by reading their variable blocks (`GuardedVerdict.lean:35-36`, `UniformSample.lean:31`).

4.3 Layer 1, declaration by declaration

4.3.1 The objects Layer 1 is built on

CompPoly's hypercube table (`CompPoly` is the Verified-zkEVM library of computable polynomials; revision 3468b38c, `CompPoly/Multilinear/Basic.lean`). A multilinear polynomial in n variables is stored by its 2^n values on the Boolean cube, as a vector; index i is the cube point whose coordinate k is bit k of i (low bit first):

```
-- CompPoly/Multilinear/Basic.lean:45-47 at 3468b38c
The indexing is **little-endian** (i.e. the least significant bit is the first bit). -/
@[reducible]
def CMLPolynomialEval (R : Type*) (n :  $\mathbb{N}$ ) := Vector R (2 ^ n) -- coefficient of Lagrange basis
```

`lagrangeBasis w` is the vector of the equality kernel $\text{eq}(w, x) = \prod_k (x_k w_k + (1-x_k)(1-w_k))$ over the cube points x ; `evalMle t x` is the multilinear extension of the table t evaluated at a point $x \in R^n$ (by folding one variable at a time, low variable first); `eval2Mle t  $\phi$  x` first maps the entries through a ring homomorphism $\phi : R \rightarrow^+ S$ (for leanVM, the embedding `algebraMap K E` of $K = \text{GF}(2^{64})$ into $E = \text{GF}(2^{192})$), then evaluates at $x \in S^n$:

```
-- CompPoly/Multilinear/Basic.lean:410-411, 499-500, 520-521 at 3468b38c
def lagrangeBasis (w : Vector R n) : Vector R (2 ^ n) :=
  Vector.ofFn (fun i =>  $\prod j : \text{Fin } n, \text{if } (\text{BitVec.ofFin } i).\text{getLsb } j \text{ then } w[j] \text{ else } 1 - w[j])$ 
def evalMle (p : CMLPolynomialEval R n) (x : Vector R n) : R :=
  evalMleValues p x
def eval2Mle (p : CMLPolynomialEval R n) (f :  $R \rightarrow^+ S$ ) (x : Vector S n) : S :=
  evalMle (map f p) x
```

The spine's objects Layer 1 plugs into (`LeanerVM/Protocol/Field.lean`, `Spine/Instance.lean`, `Spine/Seams.lean` at b435631). `Column n` wraps a K -table of n variables; its oracle answers a query $r \in E^n$ with `eval2Mle q (algebraMap K E) r` (`Field.lean:102-110`, `evalOracle_answer` is `rfl`). A `Layout  $\mu$   $\iota$   $\kappa$` is a *reading law*: how to read column c (height $2^\kappa c$) off a stack of 2^μ cells, how to lift a point of the column to a point of the stack, and the law that the two agree:

```
-- LeanerVM/Protocol/Spine/Instance.lean:73-81
structure Layout ( $\mu : \mathbb{N}$ ) ( $\iota : \text{Type}$ ) ( $\kappa : \iota \rightarrow \mathbb{N}$ ) where
  /-- Read column `c` off the stack. -/
  read :  $\text{Column } \mu \rightarrow (c : \iota) \rightarrow \text{Column } (\kappa c)$ 
  /-- Lift a point of column `c` to the point of the stack that reads the same value. -/
  extend :  $(c : \iota) \rightarrow \text{Vector } E (\kappa c) \rightarrow \text{Vector } E \mu$ 
  /-- The selector law: reading then extending is extending then reading. -/
  read_eval :  $\forall (q : \text{Column } \mu) (c : \iota) (z : \text{Vector } E (\kappa c)),$ 
    CMLPolynomialEval.eval2Mle (read q c).values (algebraMap K E) z =
      CMLPolynomialEval.eval2Mle q.values (algebraMap K E) (extend c z)
```

An `M3Instance` carries one such `layout` for all its columns (`Instance.lean:140`), and the relation `M3Holds` reads every value through it. A `Weight` μ is a weight on the stack given by its cube values `onCube` and an evaluator `mle` with the proof `mle_eq` that it computes the extension of `onCube`; `Weight.pair W q =  $\sum_i W.onCube[i] \cdot q[i]$` ; a `ColumnClaim` is (column, point, value) and holds when the column’s extension at the point is the value; a `WeightedClaim` is (weight, value) and holds when the pairing is the value (`Seams.lean:57-67, 103-124`).

Mathlib vocabulary. Antitone f means $a \leq b \rightarrow f\ b \leq f\ a$ (non-increasing). Vector $R\ n$ is an array of length n ; $z ++ s$ concatenates, so z fills the low coordinates. `Fin m` is $\{0, \dots, m-1\}$.

4.3.2 Classes, and how this catalogue is laid out

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-layer1.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.3; E (audit surface) is section 10.5.3; F (conformance) is section 8.9.1.3; G (findings) is section 8.9.2.3; I (probes) is ?? B.1.1.5; B (faithfulness), C (statements), D (the wall and the To-folders) and H (the upgrade) is not reproduced in these fragments; they are in the dossier.

Class legend. **L** load-bearing: a definition the instance `leanIsaInstance` (Layer 3) or the executable verifier (Layer 12) will unfold, so it becomes trusted surface when they are built (no master theorem, seam or built instance unfolds any Layer 1 declaration today; see E). **I** interface: consumed by name by a later phase, the adaptor, or listed in the blueprint’s interface list. **H** helper: a proof step. **T** test fixture (none: fixtures live under `tests/`). Counts are public declarations (the two `private` lemmas are not counted).

For each module: the load-bearing definitions as the dossier copied them, with their meaning in words; then a table of the load-bearing and interface declarations; then a table of the helpers. The dossier gives a snippet for the load-bearing definitions only; the interface declarations are listed by name, line and meaning.

4.3.3 LeanerVM/Protocol/ToCompPoly/Multilinear.lean

(generic: no protocol vocabulary; 43 public.)

Load-bearing definitions, copied:

```
-- LeanerVM/Protocol/ToCompPoly/Multilinear.lean:144-149
def cubeIndex {k m : ℕ} (i : Fin (2 ^ k)) (j : Fin (2 ^ m)) : Fin (2 ^ (k + m)) :=
  (i.val + 2 ^ k * j.val, by ...) -- (proof of the bound elided here, 4 lines)
-- :170-171
def cubeSplit (k m : ℕ) : Fin (2 ^ k) × Fin (2 ^ m) = Fin (2 ^ (k + m)) :=
  (Equiv.prodComm _ _).trans (finProdFinEquiv.trans (finCongr (by rw [pow_add, mul_comm])))
-- :235-236
def boolVec {m : ℕ} (j : Fin (2 ^ m)) : Vector R m :=
  Vector.ofFn fun b => if j.val.testBit b then 1 else 0
-- :307-309
def slice {k m : ℕ} (t : CMLPolynomialEval R (k + m)) (j : Fin (2 ^ m)) :
  CMLPolynomialEval R k :=
  Vector.ofFn fun i => t[cubeIndex i j]
```

LeanerVM/Protocol/ToCompPoly/Multilinear.lean, at b435631; further ranges in the comments

In words: `cubeIndex i j` is the index with low bits i and high bits j ; `cubeSplit` is the same map as an equivalence (`cubeSplit_apply` proves it equals `cubeIndex`); `boolVec j` is the cube point of index j (coordinate b is bit b); `slice t j` is the subtable of t whose high m bits are j (an aligned block).

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
<code>sumCube t</code>	67	interface	$\sum_i t[i]$
<code>hadamard s t</code>	70	interface (listed)	pointwise product
<code>evalMle_eq_sumCube_hadamard</code>	91	interface (listed)	same, as <code>sumCube</code> (hadamard ...)
<code>cubeIndex</code>	144	load-bearing	above

Declaration	Line	Class	Meaning
<code>cubeSplit, cubeSplit_apply</code> <code>lowVec, highVec</code>	170, 173 185, 189	load-bearing, helper interface	above low k / high m coordinates of a point (inside <code>lowPoint</code> , <code>highPoint</code>)
<code>boolVec</code> <code>onesIndex m</code> <code>boolVec_zero</code>	235 239 243	load-bearing interface interface	above the index $2^m - 1$ (used by <code>padHigh</code>) <code>boolVec 0 = (0, ..., 0)</code> (used by <code>PublicInput</code>)
<code>slice</code> <code>evalMle_append_boolVec</code>	307 342	load-bearing interface (listed; used by <code>PublicInput</code>)	above a Boolean high part selects a slice
<code>placeSlice t j</code> <code>evalMle_placeSlice</code>	351 371	interface (listed) interface (listed)	t on the slice j , zero elsewhere (<code>padHigh</code> , <code>windowTable</code>) extension of a placed table = (<code>lagrangeBasis s</code>)[j]. $\tilde{t}$ (z)

Helpers.

Declaration	Line	Class	Meaning
<code>hadamard_getElem</code> <code>lagrangeBasis_getElem_nat</code> <code>evalMle_eq_sum</code>	73 78 85	helper helper helper	entry of the product <code>lagrangeBasis_getElem</code> with a $\mathbb{N}$ index <code>evalMle t x = $\sum_i t[i] \cdot (\text{lagrangeBasis } x)[i]$</code>
<code>evalMle_cast, evalMle_cast</code>	97, 103	helper	evaluation invariant under a cast of n (used by <code>PublicInput</code>)
<code>sumCube_lagrangeBasis</code> <code>evalMle_replicate</code> <code>cubeIndex_val, cubeIndex_div, cubeIndex_mod, testBit_cubeIndex</code> <code>sum_cube_split</code> <code>lowVec_append, highVec_append, lowVec_append_highVec</code>	125 131 151-164 179 193-208	helper helper helper helper helper	partition of unity a constant table extends to the constant arithmetic of <code>cubeIndex</code> a cube sum is a double sum, high outside split and join are inverse
<code>lagrangeBasis_cubeIndex</code> <code>boolVec_onesIndex</code> <code>fin_eq_iff_testBit</code> <code>lagrangeBasis_boolVec</code> <code>lagrangeBasis_onesIndex, lagrangeBasis_zero_index</code> <code>evalMle_boolVec</code>	220 250 257 269 285, 292 298	helper helper helper helper helper	the basis factors across the index split <code>boolVec (2^m-1) = (1, ..., 1)</code> indices equal iff low bits equal basis at a cube point is an indicator basis at the two corners
<code>slice_getElem, cubeIndex_lt, slice_getElem_nat</code> <code>evalMle_split</code>	312-323 331	helper helper	entries of a slice <code>evalMle t (z ++ s) = $\sum_j (\text{lagrangeBasis } s)[j] \cdot \text{evalMle (slice } t \text{ } j) \text{ } z$</code>
<code>slice_placeSlice</code> <code>sumCube_placeSlice</code>	357 381	helper helper	slicing a placed table placing keeps the cube sum

4.3.4 LeanerVM/Protocol/ToCompPoly/BitProductTable.lean

(generic; 13 public.)

```
-- LeanerVM/Protocol/ToCompPoly/BitProductTable.lean:127-128
def powersTable (a : R) (n : ℕ) : CMLPolynomialEval R n :=
  Vector.ofFn fun i => a ^ i.val
```

LeanerVM/Protocol/ToCompPoly/BitProductTable.lean, at b435631; further ranges in the comments

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
<code>bitProductTable f</code> <code>evalMle_bitProductTable</code>	44 58	interface (listed) interface (listed)	entry i is $\prod_k f \ k$ (bit k of i) extension = $\prod_k ((1 - x_k) f \ k \text{ false} + x_k f \ k \text{ true})$

Declaration	Line	Class	Meaning
<code>powersTable a n</code>	127	load-bearing (inside <code>idxColumn</code>)	entry i is a^i
<code>evalMle_powersTable</code>	144	interface (listed)	extension = $\prod_k ((1 - x_k) + x_k a^{\{2^k\}})$
<code>evalMle_lagrangeBasis</code>	156	interface (field <code>mle_eq</code> of <code>eqWeight</code>)	the extension of <code>lagrangeBasis w</code> at x is $\prod_k ((1-x_k)(1-w_k) + x_k w_k)$

Helpers.

Declaration	Line	Class	Meaning
<code>bitProductTable_getElem, map_bitProductTable</code>	47, 51	helper	entries; commutes with a ring map
<code>evalMle_bitProductTable</code>	86	helper	same at a point of another ring
<code>lagrangeBasis_eq_bitProductTable</code>	93	helper	the Lagrange basis is such a table
<code>pow_eq_prod_testBit</code>	100	helper	$a^i = \prod_k (a^{2^k})^{\text{bit } k \text{ of } i}$
<code>powersTable_eq_bitProductTable, map_powersTable</code>	130, 137	helper	
<code>evalMle_powersTable</code>	149	helper (proof of <code>idxColumn_eval</code>)	same at a point of another ring

4.3.5 LeanerVM/Protocol/ToCompPoly/Stacking.lean

(generic; 38 public.)

```
-- LeanerVM/Protocol/ToCompPoly/Stacking.lean:62-68
structure Blocks where
  /-- The number of blocks. -/
  n : ℕ
  /-- The number of variables of each block; its height is `2 ^ size b`. -/
  size : Fin n → ℕ
  /-- Largest first. -/
  descending : Antitone size
-- :80-87
def offsetNat (k : ℕ) : ℕ :=
  ∑ c ∈ Finset.range k, if h : c < B.n then 2 ^ B.size (c, h) else 0
def offset (b : Fin B.n) : ℕ := B.offsetNat b.val
def total : ℕ := B.offsetNat B.n
-- :159-163
def stackAt (t : B.Tables R) (μ : ℕ) (pad : R) : CMLPolynomialEval R μ :=
  Vector.ofFn fun x ↦
    if B.total ≤ x.val then pad
    else ∑ b : Fin B.n,
      if B.InWindow b x.val then ((t b)[x.val - B.offset b]?).getD 0 else 0
-- :213-214 (bound proof elided, 6 lines)
def selector {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (b : Fin B.n) : Fin (2 ^ (μ - B.size b)) :=
  (B.offset b / 2 ^ B.size b, by ...)
-- :227-230
def extendPoint {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (b : Fin B.n) (z : Vector R (B.size b)) :
  Vector R μ :=
  Vector.cast (Nat.add_sub_cancel' (B.size_le hμ b))
  (z ++ (boolVec (B.selector hμ b) : Vector R (μ - B.size b)))
-- :264-267
def unstack {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (q : CMLPolynomialEval R μ)
  (b : Fin B.n) : CMLPolynomialEval R (B.size b) :=
  slice (Vector.cast (congrArg (2 ^ ·) (Nat.add_sub_cancel' (B.size_le hμ b)).symm) q)
  (B.selector hμ b)
```

LeanerVM/Protocol/ToCompPoly/Stacking.lean, at b435631; further ranges in the comments

In words: a `Blocks` is a list of sizes, non-increasing; block b starts at `offset b`, the sum of the heights before it; `total` is the sum of all heights; `stackAt` lays given tables end to end and fills the rest with `pad`; `selector b = offset b / 2size b`; `extendPoint b z = (z, bits of selector b)`; `unstack q b` reads block b 's window off any table q .

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
Blocks	62	load-bearing	above
Blocks.Tables R	75	interface (listed)	one table per block
offsetNat, offset, total	80, 84, 87	load-bearing	above
pow_size_dvd_offset	109	interface (sketch)	alignment: $2^{\text{size } b} \mid \text{offset } b$
stackAt	159	interface (listed; the honest prover's stack)	above
selector, selector_val	213, 222	load-bearing, helper	above
extendPoint	227	load-bearing	above
lowPoint, highPoint	233, 238	interface	the low size b / high μ - size b coordinates of a stack point
unstack	264	load-bearing	above
unstack_getElem	271	interface (the cell meaning of the reader)	$(\text{unstack } q \ b)[i] = q[i + \text{offset } b]$
unstack_stackAt	280	interface	reading the stack returns the table
unstack_eval ₂	330	interface (listed)	$\text{eval}_2 \text{Mle } q \ \emptyset (\text{extendPoint } b \ z) = \text{eval}_2 \text{Mle } (\text{unstack } q \ b) \ \emptyset \ z$, every q
stack_eval, stack_eval ₂	339, 345	interface (listed)	the same on the stack, any pad

Helpers.

Declaration	Line	Class	Meaning
offsetNat_succ, offsetNat_mono	89, 93	helper	
offset_add_pow_le_offset, offset_add_pow_le_total	97, 103	helper	windows disjoint, in order, inside the total
InWindow b x	118	helper	$\text{offset } b \leq x < \text{offset } b + 2^{\text{size } b}$
inWindow_unique, exists_inWindow, inWindow_iff_div	121, 129, 147	helper	one window per index; no gaps below total; window = quotient
stackAt_getElem_of_inWindow, stackAt_getElem_of_total_le	165, 176	helper	the cells of the stack
map_stackAt, stackAt_eq_of_total_eq	182, 197	helper	commutes with ring maps; a full stack ignores the pad
size_le	206	helper	every block fits in μ variables
cast_lowPoint_append_highPoint, lowPoint_extendPoint, highPoint_extendPoint	244-255	helper	
unstack_map, unstack_eq_of_window_eq	292, 304	helper	
unstack_eval	319	helper	selection identity, one ring
unstack_eval ₂ _eq_sumCube	352	helper (only BlockClaims uses it)	the identity as a cube sum

4.3.6 LeanerVM/Protocol/ToCompPoly/AmbientStacking.lean

(generic; 10 public.)

```
-- LeanerVM/Protocol/ToCompPoly/AmbientStacking.lean:69-70
def selectorWeight {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (b : Fin B.n) (z : Vector R μ) : R :=
  (lagrangeBasis (B.highPoint hμ b z))[B.selector hμ b]
```

LeanerVM/Protocol/ToCompPoly/AmbientStacking.lean, at b435631; further ranges in the comments

`selectorWeight b z = eq(sel_b, z_hi)`, the weight of block b at a stack point.

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
selectorWeight	69	interface (in stack_eval_ambient; L once a verifier computes <code>eq(sel_b, ζ_hi)</code> with it)	above
stack_eval_ambient	134	interface (listed)	the stack at any point (below, B.3)

Helpers.

Declaration	Line	Class	Meaning
selectorWeight_extendPoint_self	73	helper	weight 1 at a lifted point of the block
windowTable, windowTable_getElem, ev	80, 86,	helper	a table placed in one window
aLMle_windowTable	102		
stackAt_decomposition	111	helper	stack = pad + Σ windows of (block – pad)
stack_eval_ambient_zero	159	helper	pad 0
sum_selectorWeight_of_total_eq	166	helper	a full layout's weights sum to 1
stack_eval_ambient	178	helper (used by Stack.lean)	at a point of another ring

4.3.7 LeanerVM/Protocol/Stack.lean

(leanVM's: specialises to K, E, the spine's Column and Layout; 9 public.)

```
-- LeanerVM/Protocol/Stack.lean:52-59
def Layout.comap {μ : ℕ} {ι ι' : Type} {κ : ι → ℕ} {κ' : ι' → ℕ} (L : Layout μ ι κ)
  (f : ι' → ι) (h : ∀ c, κ (f c) = κ' c) : Layout μ ι' κ' where
  read := fun q c ↦ (Vector.cast (congrArg (2 ^ ·) (h c)) (L.read q (f c))).values)
  extend := fun c z ↦ L.extend (f c) (Vector.cast (h c).symm z)
  read_eval := fun q c z ↦ by ... (3 lines)
-- :70, :73-74
def stackColumn (t : B.Tables K) (μ : ℕ) : Column μ := (B.stackAt t μ 0)
def readColumn (hμ : B.total ≤ 2 ^ μ) (q : Column μ) (b : Fin B.n) : Column (B.size b) :=
  (B.unstack hμ q.values b)
-- :97-100
def layout (hμ : B.total ≤ 2 ^ μ) : Layout μ (Fin B.n) B.size where
  read := B.readColumn hμ
  extend := B.extendPoint hμ
  read_eval := B.readColumn_eval hμ
```

LeanerVM/Protocol/Stack.lean, at b435631; further ranges in the comments

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
Layout.comap L f h	52	load-bearing	column c of the new layout is column f c of L
Blocks.stackColumn	70	interface (listed; stack0f)	the witness stack, pad 0
Blocks.readColumn	73	load-bearing	block b of a column
Blocks.readColumn_eval	78	interface (listed)	the reading law, every q
Blocks.readColumn_stackColumn	85	interface	reading the honest stack returns the block
Blocks.layout	97	load-bearing	the aligned blocks as a Layout
Blocks.stackColumn_eval_ambient	104	interface (not listed)	the zero-padded stack at any point
Blocks.stack_eval_ambient_one	118	interface (listed; the leaf decomposition of Layer 6)	the one-padded stack at any point, char. 2 form

Helpers.

Declaration	Line	Class	Meaning
Blocks.stackColumn_eval	90	helper	the honest stack at a lifted point

4.3.8 LeanerVM/Protocol/Padding.lean

(leanVM's: the protocol's choice of slice, over any ring; 6 public.)

```
-- LeanerVM/Protocol/Padding.lean:43-44, 60-61
def prodVars (m : ℕ) : CMLPolynomialEval R m :=
  Vector.ofFn fun i ↦ if i = onesIndex m then 1 else 0
def padHigh {k : ℕ} (t : CMLPolynomialEval R k) (m : ℕ) : CMLPolynomialEval R (k + m) :=
  placeSlice t (onesIndex m)
```

LeanerVM/Protocol/Padding.lean, at b435631; further ranges in the comments

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
prodVars m	43	interface (listed)	the table of $x_0 \dots x_{m-1}$
sumCube_prodVars	47	interface (listed; acceptance test 7)	$\sum x_0 \dots x_{m-1} = 1$
padHigh t m	60	interface (listed)	t lifted by the product of m new high variables
sumCube_padHigh	64	interface (listed)	keeps the cube sum (completeness of the table sumcheck)
evalMle_padHigh	70	interface (not listed ; the verifier's final check needs it)	extension = $\tilde{t}(z) \cdot \prod_b s_b$

Helpers.

Declaration	Line	Class	Meaning
evalMle_prodVars	51	helper	its extension is $\prod s_b$

4.3.9 LeanerVM/Protocol/ClaimWeights.lean

(leanVM's: over an abstract instance; 4 public.)

```
-- LeanerVM/Protocol/ClaimWeights.lean:45-48
def eqWeight {μ : ℕ} (p : Vector E μ) : Weight μ where
  onCube := lagrangeBasis p
  mle := fun r ↦ ∏ k : Fin μ, ((1 - r[k]) * (1 - p[k]) + r[k] * p[k])
  mle_eq := fun r ↦ (evalMle_lagrangeBasis p r).symm
```

LeanerVM/Protocol/ClaimWeights.lean, at b435631; further ranges in the comments

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
eqWeight p	45	load-bearing (the opening phase's weights; the verifier runs mle)	the weight $\text{eq}(p, \cdot)$
ColumnClaim.holds_iff_weighted	58	interface (listed; the opening phase)	a column claim is the weighted claim with weight $\text{eq}(\text{extend } c \ z, \cdot)$

Helpers.

Declaration	Line	Class	Meaning
Weight.pair_eq_sumCube	34	helper	the pairing as sumCube (hadamard ...)
eqWeight_pair	51	helper	pairing $\text{eq}(p, \cdot)$ with q is $\tilde{q}(p)$

4.3.10 LeanerVM/Protocol/BlockClaims.lean

(generic content, placed in the leanVM half; 8 public; no consumer.)

Helpers.

Declaration	Line	Class	Meaning
BlockClaim B S	54	helper (unused)	(block, point, value)
ambientPoint, weight	69, 73	helper (unused)	the lifted point; lagrangeBasis of it
IsValid ϕ hμ q	78	helper (unused)	eval ₂ Mle (unstack q block) ϕ point = value
pairing_eq, isValid_iff_pairing	83, 90	helper (unused)	unstack_eval ₂ _eq_sumCube restated
isValid_iff_of_map_window_eq, isValid	101, 113	helper (unused)	locality: only the window matters
id_iff_of_window_eq			

git grep -w BlockClaim b435631 -- LeanerVM tests finds BlockClaims.lean and its test only.

4.3.11 LeanerVM/Protocol/FixedColumns.lean

(leanVM's; imports the arithmetization; 12 public.)

```
-- LeanerVM/Protocol/FixedColumns.lean:48, 56-57
def idxColumn (κ : ℕ) : Column κ := (powersTable g κ)
def idxColumnEval {κ : ℕ} (z : Vector E κ) : E :=
  [ j : Fin κ, ((1 - z[j]) + z[j] * algebraMap K E (g ^ (2 ^ j.val))) ]
-- :79-82
def bytecodeColumn (prog : Program) : Column (prog.logSize + 4) :=
  (Vector.ofFn fun i ↦
    let p := (cubeSplit prog.logSize 4).symm i
    (encodeSlots (prog.code p.1))[p.2])
-- :108-111
def bytecodeColumnEval (prog : Program) (z : Vector E prog.logSize) (w : Vector E 4) : E :=
  [ s : Fin 16, (lagrangeBasis w)[s] *
    [ i : Fin (2 ^ prog.logSize),
      algebraMap K E ((encodeSlots (prog.code i))[s]) * (lagrangeBasis z)[i] ] ]
-- LeanerVM/Arithmetization/Bytecode.lean:94-96 (what bytecodeColumn puts in a cell)
def encodeSlots (i : Instr) : Vector K 16 :=
  let e := entry i
  #v[0, 0, 0, e[0], e[1], e[2], e[3], e[4], e[5], e[6], e[7], 0, 0, 0, 0, 0]
```

LeanerVM/Protocol/FixedColumns.lean, at b435631; further ranges in the comments

Load-bearing and interface declarations.

Declaration	Line	Class	Meaning
idxColumn κ	48	load-bearing (a Coord.known column of the instance)	cell i holds g^i
idxColumnEval	56	load-bearing (the verifier)	$[_j ((1 - z_j) + z_j g^{2^j})]$
idxColumn_eval	60	interface (listed)	the oracle's answer is idxColumnEval
idxColumnEval_eq	66	interface (listed)	the specification's form $[(1 + z_k(1 + g^{2^k}))]$
bytecodeColumn prog	79	load-bearing (a Coord.known column; the verifier)	cell $i + 2^κ \cdot s$ holds slot s of instruction i
bytecodeColumn_answer_boolVec	97	interface (listed)	at the cube point (bits of i, bits of s) the oracle answers slot s of instruction i
bytecodeColumnEval	108	load-bearing (the verifier)	$\sum_s \text{eq}(w, s) \sum_i \text{slot}_s(i) \text{eq}(z, i)$
bytecodeColumn_eval	114	interface (listed)	the oracle's answer at $z ++ w$ is bytecodeColumnEval prog z w

Helpers.

Declaration	Line	Class	Meaning
<code>idxColumn_get</code>	51	helper	the cells
<code>bytecodeSlotColumn prog s</code>	75	helper	slot <code>s</code> of every instruction
<code>bytecodeColumn_slot</code>	85	helper (restates the definition; section C)	
<code>slice_bytecodeColumn</code>	90	helper	slice <code>s</code> is <code>bytecodeSlotColumn s</code>

Tally. 143 public declarations: load-bearing 20 (`cubeIndex`, `cubeSplit`, `boolVec`, `slice`, `powersTable`, `Blocks`, `offsetNat`, `offset`, `total`, `selector`, `extendPoint`, `unstack`, `Layout.comap`, `readColumn`, `layout`, `eqWeight`, `idxColumn`, `idxColumnEval`, `bytecodeColumn`, `bytecodeColumnEval`); interface 41; helper 82 (of which 8 in `BlockClaims.lean`, unused). Per module (L / I / H): `Multilinear` 4 / 10 / 29, `BitProductTable` 1 / 4 / 8, `Stacking` 7 / 10 / 21, `AmbientStacking` 0 / 2 / 8, `Stack` 3 / 5 / 1, `Padding` 0 / 5 / 1, `ClaimWeights` 1 / 1 / 2, `BlockClaims` 0 / 0 / 8, `FixedColumns` 4 / 4 / 4. Generic modules: the four under `ToCompPoly/` (104 declarations); `BlockClaims.lean` is generic in content too (any `Blocks`, any rings) but sits in the leanVM half.

4.4 The spine: do the statements say what they claim?

An adversarial reading of each load-bearing statement of the spine at **b435631**: what it says, whether it says what its docstring and the blueprint say, and what it leaves to the instance or to the phases.

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

4.4.1 M3Holds, clause by clause, against the pinned specification (C.1)

```
-- LeanerVM/Protocol/Spine/Instance.lean:196-220
def ConstraintsVanish (q : Column I.μ) : Prop :=
  ∀ j, ∀ C ∈ I.constraints j, ∀ x, C.eval (I.row q j x) = 0
def Balanced (q : Column I.μ) : Prop := (I.tuples q .push).Perm (I.tuples q .pull)
def CountsNonzero (q : Column I.μ) : Prop :=
  ∀ j, ∀ i ∈ I.counts j, ∀ x, (I.column q {j, i}).values.get x ≠ 0
def PublicLinesHold (input : I.Smt) (q : Column I.μ) : Prop :=
  ∀ l ∈ I.publicLines input,
    (I.column q l.col).values.get (0, Nat.two_pow_pos _) = l.cell0 ∧
    (I.column q l.col).values.get (1, Nat.one_lt_two_pow l.pos.ne') = l.cell1
def M3Holds (I : M3Instance) (input : I.Smt) (q : Column I.μ) : Prop :=
  I.ConstraintsVanish q ∧ I.Balanced q ∧ I.CountsNonzero q ∧ I.PublicLinesHold input q ∧ I.aux q
```

`List.Perm` (Mathlib) is the relation “one list is a rearrangement of the other”: the same elements with the same multiplicities, in any order. `CMvPolynomial.eval` (`CompPoly Multivariate/CMvPolynomial.lean:217`) evaluates a computable multivariate polynomial at a row, a function from column index to K .

Clause	What it says	Specification at a386121f	Agreement
ConstraintsVanish	every constraint polynomial of every table evaluates to 0 on every row $x \in \{0,1\}^{\tau_j}$ of that table, the row read off the stack	05-arithmetization.tex:10 : “each a polynomial over K of degree at most d ... required to vanish on every row in $\{0,1\}^{\tau_j}$ ”	yes. The degree is not in the clause; it is the instance’s field d with two proofs (<code>Instance.lean:128-132</code>).
Balanced	the list of every pushed tuple is a permutation of the list of every pulled tuple	:14 : “The bus balances when its pushed tuples and its pulled tuples form the same multiset”; :47 (Lemma 5.2): “ $\prod P = \prod Q$ if and only if $P = Q$ ”	yes: equality of multisets of K^{16} -tuples, counted in $\mathbb{N}$. The deployed check compares products of fingerprints (:30-33); the spec’s Theorem 5.1 (:37) makes that a $4 \cdot 2^\mu / E $ approximation of this clause, which is the bus phase’s error to pay.

Clause	What it says	Specification at a386121f	Agreement
— the tuples	tuples $q\ s = \text{flushTuples } q\ s \ ++ \ \text{boundaryTuples } q\ s$: for every table j , every flush of side s , every row $x < 2^{\tau_j}$, the 16 coordinate polynomials evaluated on the row; then for every boundary block of side s , every row $x < 2^\kappa$, its 16 coordinates (Instance. lean:174-186)	:12: “Table T_j has n_j flushes ... each is either a push or a pull. Each one of the m coordinates in each tuple is a fixed polynomial of degree at most d in the table’s columns. A few boundary tuples are pushed (resp. pulled) by default”; :101: “A block of a 2^κ -row table holds 2^κ leaves; the row- z leaf is ... $c_i(z)$ coordinate i of the row’s tuple”; :111: boundary entries “a constant, the index column, a public component, or one committed column”	yes. Every row flushes, including the prover’s fill rows (08-end-to-end-protocol.tex:35-42, which is why fill blocks are closed walks). Co-ord.const / known / committed is :111’s list, with the index column and a public component both known. Probed: the side filter, the row ranges, multiplicity (D.1).
CountsNonzero	every cell of every column listed in counts j , for every table, is nonzero	06-bus-interactions.tex:78: “All counts are nonzero exactly when their product is. The prover therefore sends that product ... by GKR over the stacking of every count column”; :74: “Nothing checks the finalize counts”	yes, provided the instance lists exactly the tables’ count columns and not the finalize counts (layout.rs:412-414 per the blueprint’s convention <i>Count tree</i>). The spine cannot check that; it is leanISAInstance’s.
PublicLinesHold	for every line the statement names, cell 0 and cell 1 of its column are the line’s two values	08-end-to-end-protocol.tex:29: “The public input is the first two memory cells $mem[g^0]$, $mem[g^1]$, two 192-bit words (with top limb 0)”; the check :31	yes as a relation: with the three leanISA lines of Layer 3 (mem_0 , mem_1 with the words’ limbs, mem_2 with 0, 0) it says the two cells are the two words with top limb zero. The relation is stronger than the deployed check (a line, not two cells), which is the phase’s error to pay (:33, 1/ E).
aux	whatever the instance says	:87-92 “BLAKE2s validity”	not checkable at the spine: see below.

Against the verifier’s accept list (08-end-to-end-protocol.tex:100: “the caps, the one bus root for both sides, the nonzero count root, every sumcheck’s rounds and final value, the public-input line, flock’s reduction, and the PCS opening”): the caps are outside by decision (the blueprint :556-559, the adaptor takes admissibility as a hypothesis); the bus root is **Balanced**; the count root is **CountsNonzero**; the sumchecks establish **ConstraintsVanish** through the seams’ claims; the line is **PublicLinesHold**; Flock is **aux**; the opening is the ideal oracle. This agrees with the earlier review’s table (docs/reviews/protocol-spine.md:273-279), re-checked here against the tex at the pin.

M3Rel (Instance.lean:230-231) is **M3Holds** in ArkLib’s shape: the statement is the public input paired with the empty oracle family, the witness is the stack. Nothing is lost.

The aux field. $\text{aux} : \text{Column } \mu \rightarrow \text{Prop}$ with $\text{decAux} : \text{DecidablePred}$ aux is data of the instance. The spine constrains it in no way: an instance with $\text{aux} := \text{fun } _ \mapsto \text{True}$ proves knowledge of a stack whose BLAKE2s rows may be anything, and one with $\text{aux} := \text{fun } _ \mapsto \text{False}$ has an empty relation. So the meaning of **M3Holds** I is exactly as trustworthy as the instance I , and for the leanISA instance the field’s meaning is fixed by the adaptor’s theorem **satisfiedBy_witnessOf** (**M3Holds** $\rightarrow$ **SatisfiedBy**, not built), which will have to derive **Blake2sRowsValid** from aux . That is the right place: the spine cannot know Flock. What the spine could do, and does not, is keep the field out of the relation it names “the relation the verifier establishes” and put it where it is consumed. This is a design remark, not a defect (section F, *the instance is data the theorems trust*). decAux is used by nothing load-bearing: it makes **M3Holds** decidable for the tests’ **#guards** (section B, proposal 8).

4.4.2 Layout (C.2)

```
-- LeanerVM/Protocol/Spine/Instance.lean:73-81
structure Layout (μ : ℕ) (ι : Type) (κ : ι → ℕ) where
  read : Column μ → (c : ι) → Column (κ c)
  extend : (c : ι) → Vector E (κ c) → Vector E μ
  read_eval : ∀ (q : Column μ) (c : ι) (z : Vector E (κ c)),
    CMLPolynomialEval.eval₂Mle (read q c).values (algebraMap K E) z =
      CMLPolynomialEval.eval₂Mle q.values (algebraMap K E) (extend c z)
```

$\text{CMLPolynomialEval.eval}_2\text{Mle } t\ f\ z$ (CompPoly) is the multilinear extension of the value table t at the point z , the table’s entries first mapped by the ring homomorphism f (here the inclusion $K \rightarrow E$, algebraMap

K E). The law says: reading column `c` and extending it to `z` gives the stack’s extension at `extend c z`, for every stack, column and point.

What a mistaken or dishonest instance can do with it is in D.4, by probe: it can alias columns (`toyAlias`, a `Layout`, whose relation is empty) and it cannot read anything but the stack (the law fails at the all-zero and all-one stacks unless `read` depends on `q`). The stronger statement, that the law **determines** `read` from `extend`, is on paper: for a cube point `x` of the column, `read_eval` at `z = x` gives `(read q c)[x]`, the extension of a table at a cube point being the entry, as `q̃ (extend c x)`. So an instance’s only layout data is `extend`, and `read` with its law could be replaced by the definition `read q c := (Vector.ofFn fun x ↦ (q̃ (extend c (bits x))).limb 0)` together with the one proof that this `read` obeys the law for the instance’s `extend`, which holds exactly when every `extend c x` at a cube point reads a `K`-valued combination of cells. The blueprint’s sentence “It is a reading law, not a stacking law: it does not say the columns are disjoint slices of `q`” (`Instance.lean:71-72`, blueprint :447-450) is right and understates: it does not say the columns are slices at all (an `extend` with a non-Boolean selector reads a `K`-linear combination of slices).

Who rules aliasing out, and by which theorem: nothing in the spine; for `leanISA`, the completeness-side adaptor theorems `m3Holds_stackOf` and `witnessOf_stackOf` (blueprint :817-819), neither built. Until then the only instance whose relation is known to be inhabited is the toy.

4.4.3 The seams, as input and output relations (C.3)

Each `Seam.x I` is `Seam.of I P = {p | P p.1.1 (theStack p.1.2)}` (`Seams.lean:166-167`): a set of `((statement, oracle family), ())` with the predicate `read` on the one oracle. The witness is `Unit` from the commit phase on; the stack is the oracle.

- **Seam.commit** = `M3Holds` of the oracle. Output of the commit phase (`sendOracle_relOut (M3Rel I)` is definitionally this set, which is why `commitSecurity` typechecks at `Compose.lean:67-68`), input of the bus phase. As the bus phase’s input relation it is the whole of `leanVM`’s arithmetic claim; the bus phase’s knowledge soundness must reflect `Seam.bus` back into it: every constraint vanishing on the cube (from zerocheck claims at `ζ`, the escape charged to its own challenges), balance (from the grand product), counts (from the count root). The pass-through cannot (D.3 (a), machine-checked).
- **Seam.bus** (:175-177): every linear claim holds, every term has total degree $\leq I.d$, every column claim holds, the public lines hold, `aux` holds. As the **output** of the bus phase it is right that the lines and `aux` are carried untouched. As the **input** of the table phase it is quantified over by `ArkLib`’s knowledge soundness for every statement (`rbrKnowledgeSoundnessWorstCaseWith, ∀ stmtIn`, quoted in section D). Two consequences the blueprint does not draw:
 1. The degree conjunct is a property of the statement alone. For a statement with a term of degree $> I.d$ the seam is empty whatever the stack, so `toFun_empty` makes the table phase’s knowledge state false at the start; if the verifier could accept such a statement with a true final check, the state would be true at the end (`toFun_full`) and some challenge would have to pay for the flip. Probe D.2 shows a **true** cubic claim on the honest toy stack; the sibling dossier `gt-table-pub.md` (section 6, E.1 (a)) exhibits a statement on which every round polynomial is identically zero and a degree-blind verifier accepts with probability 1. So the table phase’s verifier must compute `totalDegree` of every term it receives and reject above `I.d`: a guard `leanVM`’s verifier does not have, dead in the composition, live in the phase’s theorem. Our reading agrees with the sibling’s.
 2. The same applies to everything else `Seam.bus` leaves free and the deployed table sumcheck fixes: the number of linear claims, the points of the terms, the tables they sit on. The sibling dossier establishes this by probe and proposes a `BusOut` in the shape of the Rust’s `BusVerify` (its section 8, first finding). We agree, and add only that the degree conjunct is the smallest instance of the phenomenon and that the type-level fix (a subtype of polynomials within the bound, or forms indexed by the instance’s own constraint and flush lists) removes the guard and the conjunct together. For **completeness** the conjunct does what the blueprint says (:571-574): the table phase’s completeness is owed on statements within the bound only. For the **bus phase’s knowledge soundness** it costs nothing: the bus verifier builds its terms from `I.constraints` and `I.flushes`, bounded by `constraints_degree` and `flushes_degree`.
- **Seam.table** (:180-181): column claims, lines, `aux`. Right for the deployed order (`08-end-to-end-pr`

`otocol.tex:73-85`: table sumcheck, then public input), which is why the lines survive this seam and are consumed at the next.

- **Seam.pub** (:184): column claims and `aux`. The statement is no longer mentioned (D.2: the wrong public statement is inside this seam); that is right, the lines have become claims.
- **Seam.flock** (:187-188): column claims and weighted claims. `aux` is consumed here. Whether the Flock phase consumes column claims (the eighteen limbs) or passes them all on is the Flock dossier’s question; the seam admits both.
- **Seam.done** (:191): True. Knowledge soundness with a trivial output relation is ArkLib’s notion of a *proof*: `toFun_full`’s hypothesis becomes “the verifier can output something”, that is, accepts (a rejecting run is `failure` in the `OptionT` layer and contributes no mass to the event, `Security/Basic.lean:329-342`). So `piop_rbrKnowledgeSoundness` reads: on every accepting full transcript the composed state is true; the state is `M3Holds` of the message after the commit round; a prover message never turns it true; a challenge does with probability at most its declared error. The plain reading, “the committed stack satisfies `M3Holds` except with probability the sum of the errors”, is ArkLib’s `rbrKnowledgeSoundness_implies_knowledgeSoundness` (`Security/Implications.lean:223-228`), sorry at the pin, and existential in the extractor.

One remark on the oracle the seams are read through. Every seam reads the stack through `evalOracle` (`Field.lean:102-106`): a query is a point of E^μ and the answer is $\tilde{q}$ there. The specification’s PCS interface is Definition 3.13 (`03-proving-primitives.tex:112-118`), a *weighted* claim $\langle W, g \rangle = c$ for any MLE-friendly weight, of which an evaluation is the case $W = \text{eq}(r, \cdot)$ (:117); and §8.5 says of the opening “This one opening discharges every pooled claim; there is no separate reduction sumcheck” (`08-end-to-end-protocol.tex:99`). With an evaluation oracle, a weighted claim (`Seam.flock`’s `WeightedClaim`) can only be checked by a sumcheck ending in one evaluation query, which is the opening phase the blueprint’s Layer 10 adds (:1112-1113). An oracle answering weighted queries (query `Weight  $\mu$` , answer `Weight.pair`) would make the opening phase one challenge and one query, as the specification has it. This is the spine’s choice (convention *The oracle*, :317) and is examined by the opening dossier; it is recorded here because `evalOracle` is load-bearing for both master theorems.

4.4.4 What the master theorems are, and what remains (C.4)

```
-- LeanerVM/Protocol/Spine/Compose.lean:162-177
theorem piop_perfectCompleteness (P : Phases I) (C : P.Complete) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl [] σ (StateT σ ProbComp)) :
  (leanVmPiop P).perfectCompleteness init impl (M3Rel I) (Seam.done I) :=
  C.toDef.complete init impl
theorem piop_rbrKnowledgeSoundness (P : Phases I) (S : P.Security) {σ : Type}
  (init : ProbComp σ) (impl : QueryImpl [] σ (StateT σ ProbComp)) :
  (leanVmVerifier P).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (M3Rel I)
  (Seam.done I) S.toDef.witMid (piopExtractor P S) (S.toDef.kSF init impl)
  (piopError P) :=
  S.toDef.rbr init impl
```

Both are one-line proofs: the composed component’s own field. They are composition theorems: for every instance `I`, every bundle `P` of five phases with the right statement types, and every proof of the five phases’ completeness (resp. security) against the seams, the composed protocol is complete (resp. round-by-round knowledge sound at the composed error). They say nothing about leanVM. What remains, in order:

1. An instance: `leanIsaInstance prog s` (blueprint Layer 3). Category B data (layouts, separators, count columns, lines), trusted as a transcription and checked by the fixture.
2. Five phase definitions with leanVM’s schedules and checks (Layers 6, 7, 8, 9, 10). Built: the public-input phase.
3. Their `Complete` and `Security` against the spine’s seams. Built: the commit phase’s (spine) and the public-input phase’s. Not provable as the seams stand for the deployed table sumcheck (C.3, sibling dossier).
4. A bound on `piopError P` (C.5).
5. The adaptor’s four theorems, for non-vacuity (`m3Holds_stackOf`) and for soundness (`satisfiedBy_witnessOf`).

6. The plain reading of round-by-round knowledge soundness (ArkLib, admitted), in a form that names the extractor if acceptance test 24 is to survive it (C.7).
7. Compilation (Layers 11 to 13).

The headline (:11-17) writes `piop_rbrKnowledgeSoundness (I : M3Instance) : for every prover, leanVmVerifier accepts → except with probability piopError I, the extractor's column q satisfies M3Holds I input q`. On main: there is no `piopError I` (the error is `piopError P`, a function per challenge with no bound); “except with probability” is the plain form, admitted upstream; “the extractor’s column q” is no definition (D.5); and the theorem takes `P` and `S`, absent from the headline. The sentence “states the two theorems that make it a proof system for the constraint relation” overstates what is on main (finding *the headline describes theorems that do not exist*).

4.4.5 piopError (C.5)

`piopError P := P.toDef.err (Compose.lean:149)`, and `Def.append` assigns each challenge the error its component declared (`Component.lean:148`, `err := Sum.elim D1.err D2.err ◦ ChallengeIdx.sumEquiv.symm`). No statement of the spine bounds it, sums it, or ties it to any parameter of leanVM; the phases declare it, and a phase may declare `1` (D.3 (b), machine-checked). The blueprint’s `piopError_le` (`h s : s.Admissible prog`) : $\Sigma i, \text{piopError } s \ i \leq 2^{40} / |E| + \text{flockError}$ (:1130) is stated over a `piopError s` the spine does not have, and “the interactive error is its sum” (:318) is a sentence, not a theorem. Finding *the declared error is unconstrained*, with the proposal, in section F.

4.4.6 The quantification over σ , `init`, `impl` (C.6)

`Component.Complete.complete` and `Security.kSF`, `Security.rbr` are stated for every $\{\sigma : \text{Type}\}$ (in `it : ProbComp  $\sigma$`) (`impl : QueryImpl [] ◦ (StateT  $\sigma$  ProbComp)`) (`Component.lean:84-85`, 101-106). `QueryImpl [] ◦ m` (`VCVio OracleComp/SimSemantics/QueryImpl/Basic.lean:36`) is a function from the queries of the specification to computations; `[]` is `PEmpty → PEmpty` (`VCVio OracleSpec.lean:216`), so there is exactly one `impl`, never called. `ProbComp  $\sigma$` is a probabilistic computation with no failure, so `init` always yields a state. The quantification is therefore without content: for a deterministic verifier the event in `toFun_full` has probability 1 or 0 whatever σ , `init`, `impl` (this is what `Verifier.GuardedForm.of_probEvent_pos`, `GuardedVerdict.lean:40-45`, extracts). It is **forced by the upstream library**: ArkLib’s `perfectCompleteness` and `KnowledgeStateFunction` take `init impl`, and its composition theorem needs completeness of the second component “from every shared state” (`OracleCompleteness.lean:63`, `h2 :  $\forall s : \sigma, R_2.\text{perfectCompleteness (pure } s) \text{ impl rel}_2 \text{ rel}_3$`), which is why the fields are universally quantified rather than instantiated. Cost: the three binders appear in eleven load-bearing declarations. No workaround short of a leanerVM-side wrapper (section B, proposal 9); retired when ArkLib offers an ambient-free form.

4.4.7 Refinement and what T4 needs (C.7)

```
-- LeanerVM/Protocol/ToArkLib/Refinement.lean:31-36, 55-57
structure Refinement {Stmt : Type} {W1 W2 : Type} (R : Set (Stmt × W1))
  (S : Set (Stmt × W2)) where
  map : Stmt → W1 → W2
  map_valid : ∀ x w, (x, w) ∈ R → (x, map x w) ∈ S
  theorem map_option_valid {R : Set (Stmt × W1)} {S : Set (Stmt × W2)} (f : Refinement R S)
    (x : Stmt) (w? : Option W1) (h : ∀ w ∈ w?, (x, w) ∈ R) :
    ∀ w' ∈ w?.map (f.map x), (x, w') ∈ S
```

What is transported: validity of one witness slot (an `Option`, the shape of ArkLib’s straight-line extractor’s output). Nothing probabilistic is proved in the repository, as the blueprint says (:582-586). “Knowledge transports at the same error” (:364-366) is therefore justified only by the pointwise step. The probabilistic step is, however, three lines and needs no prover conversion: probe D.7 (`knowledge_transport`) proves it for ArkLib’s `knowledgeSoundnessWith`, by leaving the game alone and applying the map inside its event; the target witness type may be in any universe (`Refinement`’s two universes are independent, checked). T4 (`baseVerifier_exttractsExecution`, :1247-1248) needs exactly this shape: a bound on the event “the verifier accepts and the mapped extracted witness has no execution”, in the game of `verify_`

`knowledgeSound`, which is not an `ArkLib knowledgeSoundness` of a relation on `EnsembleWitness` (that would need `WitIn : Type`) but a statement about one event of one game. The blueprint’s “not proved ... and has no consumer yet” should say instead that it is proved in the event form (once the lemma is moved into the repository) and that Layer 13 is its consumer. What T4 does still wait on is upstream: the plain form of the round-by-round theorem (`Implications.lean:223-228`, sorry at the pin), whose conclusion is `knowledgeSoundness`, existential in the extractor, so that the named extractor `piopExtractor` would be lost at that step unless a `With` version is proved.

Part II

The analysis

Chapter 5

The phases: what each proves, and how

This chapter gives, phase by phase, the design as the blueprint has it, the design as leanVM has it, and the intuition behind the two proofs each phase owes: perfect completeness (an honest prover always passes) and round-by-round knowledge soundness (an invariant on partial transcripts that a cheating prover escapes only when a fresh challenge lands badly, with a bound per challenge). Where the blueprint falls short of leanVM the finding is named and detailed in chapter 8; where a definition cannot be met or says less than it should, in chapter 9. The vocabulary of the ideal model (oracle reduction, knowledge state function, extractor) is introduced in chapter 3; the reader who has that chapter in mind can read this one on its own.

5.1 The commit phase

The prover sends the stack q as the one oracle message; the verifier keeps the statement and exposes the message as the oracle. There is no challenge and no check. The phase is the spine's own and both its halves are proved (`commitComplete`, `commitSecurity`). Its knowledge state function says, before the message, that the candidate witness satisfies `M3Holds` and, after it, that the *message* does; its extractor reads the message. Error zero is right: there is no challenge to charge. In the deployed protocol the message is a Merkle root; what a root must provide for the compiled protocol to inherit the oracle protocol's guarantee is the subject of section 5.6.

5.2 The bus phase

What is proved. That the multiset of tuples the tables push equals the multiset they pull, on each of the three buses, and that every read count is nonzero. With the boundary blocks (the initial and final machine state, the memory and bytecode seeds and finalizations) counted in, balance is the whole of the memory and bytecode consistency of the execution.

How. A fingerprint turns a 16-tuple t into the scalar $\beta + \sum_{i<16} \text{eq}(\alpha, i) t_i$ for random $\alpha \in \mathbb{E}^4$, $\beta \in \mathbb{E}$; two multisets of at most $2^{\mu_{\text{bus}}}$ tuples with different contents have different products of fingerprints except with probability $4 \cdot 2^{\mu_{\text{bus}}} / |\mathbb{E}|$ (the specification's Theorem 5.1; each factor has total degree 4 in (α, β)). The product itself is proved by a grand-product argument: the leaves are the fingerprints (padded with 1), each layer of the tree holds pairwise products, and a GKR walk from the root descends layer by layer, reducing a claim on a layer to a claim on the layer below by a sumcheck; the three trees (push, pull, count) share every challenge and are batched by the powers of a combiner λ . The prover states a single root R for the push and pull trees, so their equality needs no check, and a root R_c for the count tree, which the verifier requires to be nonzero. The walk ends with three claims on the leaf tables at one point ζ , which the leaf decomposition (the specification's equation (2) of §5.4) splits into a public part the verifier

computes, five boundary evaluations the prover sends, and the tables' share, which the table sumcheck settles.

Completeness. From **M3Holds**: the constraints vanish, so the zerocheck claims hold at every point; balance is a permutation, so the two products are one field element and the honest prover has a single R to send; every count is nonzero, so $R_c \neq 0$; the GKR's honest messages pass every layer check (no inverse of a challenge is taken); the leaf decomposition holds of the honest stack with the derived rem_s . This is perfect.

Knowledge soundness. One state function for the whole phase, a conjunction: before (α, β) , **M3Holds** of the oracle; after them, “the two products at (α, β) are equal and R_c is the true count product and every constraint vanishes”; from then on the GKR's own invariant (“the current layer claims are the true values”) together with the zerocheck conjunct “every constraint's extension, restricted to the coordinates of ζ drawn so far, is identically zero”. A challenge flips a false state to true with probability at most: $4 \cdot 2^{\mu_{\text{bus}}} / |\mathbb{E}|$ for (α, β) ; $2/|\mathbb{E}|$ for a combiner (the batching polynomial has degree 2); $4/|\mathbb{E}|$ for a round challenge (the cofactor has degree 4); $1/|\mathbb{E}|$ for a combination challenge; and 0 for the last, unused combiner. The zerocheck conjunct adds nothing: on a coordinate of ζ it flips with probability at most $1/|\mathbb{E}|$, once for all constraints (a nonzero multilinear restricted at one more variable becomes zero for at most one value of that variable), and a conjunction escapes with the larger of the two probabilities, not their sum. The blueprint's three different accounts of this charge, and its GKR with the wrong wire and a loose error, are the findings of section 8.4.

Where the blueprint falls short. The GKR layer sumcheck it specifies sends five coefficients where the deployed one sends four (the *normalized* cofactor of degree 4) and checks a layer with an equality factor the deployed check lacks; its per-round error $5/|\mathbb{E}|$ is a valid bound, loose by one; it draws one combiner fewer than the deployed verifier; its generic GKR takes the leaf tables as oracles, which the spine's one-oracle phases cannot supply without ArkLib's admitted context lifting; and the zerocheck conjunct, which changes on the GKR's own challenges, cannot be framed around a GKR that knows nothing of the constraints. The admissible sizes it equates with leanISA's caps leave out the stacking window and the rate window that the deployed verifier checks. None of this touches the spine as built: the real bus phase's output is expressible as a **BusOut**, and its completeness and knowledge soundness against the spine's two seams can hold (section 8.4).

5.3 The table sumcheck

What is proved. That every constraint of every opcode table vanishes on every row (for leanVM at the pin, the two identities of the JUMP table; the other tables have none), and that each table's bus forms sum, over its rows, to the share rem_s the bus phase derived for it. The second is what ties the committed columns to the bus trees: the trees were built by the prover, and only this sumcheck says that they were built from the columns.

How. One sumcheck over τ_{max} variables, the highest bound first, with each table joining at the round where its height is reached and padded below it by the product of the variables it lacks (so that its sum is unchanged). The summand batches the constraints and the three bus forms by the powers of ξ , with $\text{eq}(\zeta_{<\tau_j}, \cdot)$ as the weight of table j : the constraint terms contribute zero to the target and the bus terms contribute $\sum_s \xi^{2+s} rem_s$, which the verifier computes, so no target is ever sent. A round message is a cubic (the equality factor times a degree-2 term) of which three coefficients travel; the verifier derives the fourth from the running claim. At the end the prover sends every column's value at the sumcheck's point and the verifier recomputes the summand.

Completeness. Immediate from the input seam: the zerocheck claims are true, the bus forms sum to their values, the honest round polynomials pass, the honest column values reproduce the summand. No inverse of a challenge is taken.

Knowledge soundness. Before ξ : every claim is true. After ξ : the batched claim $\sum_c \xi^c \delta_c = 0$ where δ_c is claim c 's error; a nonzero polynomial of degree at most $k-1$ in ξ has at most $k-1$ roots, so the error on ξ is $(k-1)/|\mathbb{E}|$, which is $(B+2)/|\mathbb{E}|$ for B constraints and three sides. Each round: if the running claim is false, the prover's cubic differs from the true one and they agree at most at three points, $3/|\mathbb{E}|$. After the last message: the verifier accepts and every column claim holds, so the recomputed summand is the true summand at the point and the running claim was true. The escape from “the zerocheck claims hold at ζ ” to “the constraints vanish on the cube” belongs to the bus phase, where ζ was drawn (above); it is not charged here.

Where the blueprint falls short. Not in the numbers, which are right and tight, but in the seam it consumes. `Seam.bus` as built admits statements that the deployed table sumcheck cannot serve: any number of linear claims (so the error on ξ is unbounded over the seam), terms of one table at unrelated points, terms on any table of the instance, and a degree clause on the statement alone, which forces the verifier to check degrees, a check leanVM does not make. Completeness and knowledge soundness are owed on all of them. The blueprint also leaves open which tables the sumcheck ranges over: its leanISA instance makes the six shared columns tables of the instance, and read literally its table sumcheck then has at least sixteen rounds where leanVM has as few as three (section 8.5). And its oracle protocol sends four coefficients and checks the round identity, where the deployed verifier reads three and derives the fourth; the lemma relating the two is not stated.

5.4 The public-input phase

What is proved. That cells 0 and 1 of the three memory limbs hold the two public words: for the two low limbs, the words' limbs; for the top limb, zero.

How. The verifier draws $r \in \mathbb{E}$ and considers the line through cells 0 and 1 of a column: $\tilde{c}(r, 0, \dots, 0) = (1-r)c(0) + rc(1)$, where $-1 = 1$ in characteristic 2. The prover sends the values c_0, c_1 it claims for the two low limbs on that line. The specification checks each against the line through the public words' limbs; the deployed verifiers check the single equation $c_0 + yc_1 = (1+r)w_0 + rw_1$ in $\mathbb{E}$, which the two per-limb equations imply and which does not imply them. Three claims are then pooled: the two low limbs at the values sent and the top limb at 0, for which nothing is sent.

Completeness. If the lines hold, the honest values pass either check and every pooled claim is a true evaluation. Perfect.

Knowledge soundness. A wrong stack (a cell of a limb differing from the public word's) makes the pooled claim on that limb a nonzero polynomial of degree one in r , true at one challenge at most: error $1/|\mathbb{E}|$, whatever the number of lines and for either check. For the deployed check the argument runs through the independence of $1, y, y^2$ over $\mathbb{K}$: if the three pooled claims hold of a $\mathbb{K}$ -valued memory and the combined equation holds at two distinct challenges, the cells are the limbs of the public words and the top limbs are zero. This is proved in Lean by the review (appendix B).

Where the phase as built falls short. It follows the specification, so its theorem is about a verifier that none of the three executable verifiers runs. And it pools the values it *computes* from the statement rather than the values *sent*: on the accepting path they agree, but the consequence is that the check on the prover's message is not load-bearing for any theorem of the phase: a verifier that reads the message and ignores it has the same completeness and the same knowledge soundness at the same error. The requirement that a missing check leave a theorem unprovable is therefore not met by this phase as designed, and cannot be; pooling the values sent, as the deployed verifiers do, restores it (section 8.5, chapter 14). The top-limb claim exists because the leanISA instance lists a third public line; nothing in the phase or the spine forces that line, and what catches its absence is the adaptor's theorem, not built (chapter 14).

5.5 BLAKE2s validity and ring switching

What is proved. That the bits packed into the region q_{flock} of the stack satisfy, block by block, the R1CS of the BLAKE2s compression with the constant position of every block equal to 1; the eighteen value limbs each BLAKE2S table row carries are strided slots of that region and are opened with every other point claim.

How. Flock is a zerocheck over the R1CS's rows, with the first six variables skipped univariately (the prover sends the summand on a coset of 64 points), followed by a lincheck that reduces the three matrix products to one evaluation of the bit witness at a point, with a single terminal identity the verifier checks. Ring switching then turns the 64 resulting claims on the bit polynomials Q_i (bit i of every word) at one point into a weighted claim on the packed column: for a random $\mathbb{F}_2$ -linear map Φ built from six challenges, $\sum_u \Phi(\text{eq}(r, u)) q_{\text{flock}}(u) = \sum_i x^i \Phi(s_i)$, whose weight the verifier evaluates in closed form. Neither step commits anything: the only oracle is q .

Completeness. Perfect, for the honest prover that sends the coefficients of the true round polynomials: the verifier derives the untransmitted coefficient by a multiplication and takes no inverse. The *deployed* Rust prover is not that prover: it derives $G(0)$ through $(1 + r_{\text{eq}})^{-1}$, whose value at $r_{\text{eq}} = 1$ is 0 by the field's convention, and at that challenge emits a proof its own verifier rejects. The blueprint's acceptance test 20 attributes this inverse to the protocol and files it under a soundness error; both are wrong (section 8.6).

Knowledge soundness. The specification's Annex C accounts $(4k_{\text{batch}} + 163)/|\mathbb{E}|$ for the reduction, which the review re-derived term by term from the Rust verifier's parameters, and Annex A accounts below $2^{32}/|\mathbb{E}|$ for ring switching (a nonzero polynomial of total degree below 2^{32} in the six challenges). Both are errors for a fixed committed polynomial; after compilation the list-decoding regime multiplies them by the list size (section 5.6).

Where the blueprint falls short. Its Layer 9 makes the eighteen limb claims an *input* of Flock and one weighted claim its only output, dropping the limb claims; with leanVM's verifier, which reads no claim, that interface has knowledge error one. The spine's slot, which keeps the column claims and adds the weighted one, is right. The slot map of the limbs is placed inside the Flock interface while the instance's layout needs it. And a Flock phase cannot be written over an abstract instance at all, since the instance offers it only an opaque predicate **aux**; the review proposes a small structure that names the region, the number of compressions and what **aux** says of them (section 8.6). Today, with the only built instance having **aux** trivially true, a Flock phase that checks nothing is knowledge sound at error zero (appendix B): the master theorems constrain the Flock verifier exactly as much as the instance's predicate does.

5.6 The opening and the compilation

What is proved. That every pooled claim holds of the committed column: the 112 point claims (each an evaluation of a column of q at a point, turned into a weighted sum over the stack through the stack's selectors) and the one ring-switched weighted claim.

How, as deployed. The verifier draws λ ; the ring-switched claim takes λ^0 and the pool the next powers; the batch is one weighted claim $\sum_w W_\lambda(w) \tilde{q}(w) = C_\lambda$. The next scalar the verifier reads is the first round polynomial of WHIR's *own* sumcheck on that claim: the μ reducing rounds are WHIR's, interleaved with its level roots, out-of-domain samples, grinding, queries and per-level batchings, and the protocol closes on one equation without ever evaluating $\tilde{q}$ at a point. There is no separate reduction sumcheck, as the specification says. Four details of the deployed schedule are not in the specification's Protocol B.6: the introductory round polynomials of the per-level claims (the out-of-domain claim's before the level's batching challenge, the query batch's after it and depending on it), seventeen bits of grinding at every level, the level-0 lane relay layout with its rotated terminal point, and the omission of the last round message.

How, in the blueprint. Layer 10 runs a sumcheck reducing the batched weighted claim to an evaluation and then queries the oracle once; Layer 11 then runs WHIR “in place of the evaluation oracle”. Composed, that is a transcript with 2μ sumcheck rounds, an extra scalar and a second batching challenge, which no Rust proof has: the refinement theorem of Layer 12 cannot hold as planned. The cause is the oracle interface of Layer 0, an evaluation oracle; the specification’s commitment scheme is an inner-product scheme (its Definition 3.13), whose idealization is an oracle that answers *weighted sums*. With that interface the opening phase is “ λ , then one weighted query”, with error $(J-1)/|\mathbb{E}|$ on λ and nothing else, and WHIR replaces the whole opening phase, its level-0 batching being that λ . The review’s probe shows that the interface typechecks against ArkLib at both pins, that the evaluation queries every built theorem uses are its special case, and that nothing built queries the oracle, so nothing built breaks (section 8.7).

Completeness. Of the oracle protocol with the inner-product interface: perfect. After compilation: *not* perfect. The honest prover’s grind is an unbounded search for a nonce, so the honest **prove** is a partial or fuelled function, and the blueprint’s “perfect completeness survives the transform without any assumption” does not hold as written; what survives is completeness of **prove** whenever it returns.

Knowledge soundness. Of the oracle protocol: the batching error on λ . Of the compiled verifier: the commitment is a Merkle root of a $\mathbb{K}$ -valued word, not of a codeword, close to at most L_0 codewords of the code over $\mathbb{E}$; the compilation must show that some member of that list satisfies every pooled claim and charge L_0 times each front-end error on the challenges before the opening. Every such codeword is $\mathbb{K}$ -valued (agreement beyond the code’s dimension on a $\mathbb{K}$ -valued word kills the y, y^2 components), a lemma the specification does not state and the extracted witness’s type needs; the review gives a two-line argument for it, not yet a Lean proof. The non-interactive error then needs the hash-collision term, the list-size factor and an account of grinding, which the blueprint’s sketch of `niError` lacks; and the interfaces it assumes for Fiat–Shamir and for the Merkle compilation are, as written, false as universals or stated for other constructions than leanVM’s single BLAKE2s map, and inhabited by nothing at either pin of ArkLib (section 8.7, chapter 7).

The numbers. Under the stacking window the blueprint’s phase errors sum to about $2^{-159.7}$, ring switching’s $2^{32}/|\mathbb{E}|$ dominating, so its interactive bound holds; without the window the bus depth reaches 38 and it is false. The deployed parameters claim 128 bits round-by-round in the Johnson regime, provable given the coding-theory theorem the blueprint names: the query rounds close 111 bits plus 17 bits of grinding at every level, the fold terms sit at 128 to 146 bits, and the list size L_0 runs from a few hundred at $\mu = 15$ to about 2^{20} at deep levels. The review’s port of the Rust verifier’s parameter search reproduces the Python verifier’s query table for all 56 configurations, and the citation check re-ran it over all 56 sizes (section 8.7).

Where the blueprint falls short. In the shape of the oracle (an evaluation oracle where leanVM commits to an inner-product scheme), in the transcript of the opening (a sumcheck that is not there), in the form of the non-interactive theorems (no probability space, no adversary, soundness rather than knowledge), in the interfaces it assumes (false or empty as stated), in the error it sketches (three terms missing), and in the wording of the Fiat–Shamir chain (the chaining value is hashed as part of the message, not fed as the compression’s chaining-value input, which the blueprint’s “compress with the state as chaining value” invites one to get wrong).

Chapter 6

The proof obligations, as a hierarchy

The tree of everything that must be proved for the proof system’s target theorem to hold, from the root down to the library facts the leaves rest on, with the status of every node at **b435631** and the nodes the review found wrong, missing or unprovable. The two existing outlines, the obligation map of `docs/architecture.md` and the blueprint’s table of holes, are validated against it at the end. The tree was drawn by a review agent from the blueprint, the dossiers and the sources; its Markdown form is `dossiers/obligations.md` on the review branch. Two of its nodes keep wordings later corrected: the random oracle of the compilation must also stand for the multi-block leaf hash (section 8.7), and the GKR round error the blueprint charges is a valid, loose bound (section 8.2).

6.1 The shape of the tree

Sources and revisions. The tree was drawn by the review’s task `obligations` on 2026-09-30. Object: `leanerVM main` at **b435631** (every Lean citation at that revision, `git show b435631:<path>`); ground truth `leanVM` at **a386121f** (read with `git -C /home/scaraven/Documents/leanEthereum/leanVM show a386121f:<path>`; the checkout has moved, brief §8); libraries at the old pins (`ArkLib dca90385`, `CompPoly 3468b38c`, `VCVio f9dc47d9`, `Clean 93c9d1ef`) unless the new pin (`7653a901`, `572f9973`, `a4232d08`, `42fe4b26`) is named. No Lean was run for the dossier: every status below rests on the sibling dossiers’ recorded probes (named at each node) or on reading the sources.

Inputs, read in the order the task gave: `docs/architecture.md` and `docs/roadmap/protocol-blueprint.md` at **b435631**; the dossiers `code-spine`, `lib-arklib`, `gt-bus`, `gt-table-pub`, `gt-flock-ring`, `code-pubinput`, `boundary-adaptor`, `literature`, `code-layer1`, `gt-opening-compile` (complete when read: its summary is headed “Conclusions” with five items and its sections A to G are written), `lib-others`, and the `register` staging (its row identifiers are quoted in parentheses only where they locate a finding); the Lean sources of `LeanerVM/Protocol/` at **b435631** for every name.

Abbreviations for pointers: `bp:NNN` is a line of the blueprint at **b435631**; `arch:NNN` a line of `docs/architecture.md` at **b435631**; a dossier is named by its task name and section.

What was examined. The two existing outlines (`docs/architecture.md`’s T4 and its obligation map; the blueprint’s holes table and Layers 0 to 13) and, through the review’s dossiers, what `leanVM` at the pin actually runs, what `ArkLib` and the other libraries prove and admit, and what is built at **b435631**. From these the complete hierarchy of proof obligations was written (section 6.2): the tree whose root is the pair of Layer 13 theorems, whose next level is the compiled verifier’s theorems, then the master theorems of the oracle protocol, then the six phases’ completeness and knowledge-soundness obligations, then the generic components each is built from and the helper statements each needs, down to the library facts; beside it the adaptor’s obligations (2.6) and the table of hypotheses discharged by verifier checks or by conditions on public data (2.7). Section 6.3 composes the errors; section 6.4 validates the two outlines against the tree; section 6.5 states what the tree reveals that no sibling dossier says.

The shape of the tree. 155 numbered nodes in seven tiers (2.0 the root, 2.1 the compiled verifier, 2.2 the master theorems and the spine, 2.3 the phases, 2.5 the generic components and the layers below the phases, 2.6 the adaptor and `leanISA`, 2.7 the hypotheses table), with up to four numbered levels inside a

tier (22 nodes at the first level, 80 at the second, 53 at the third and fourth; 152 carry a status, the other three are a group header and two pointers). Seven arrows carry the error from the phases to the deployed claim (section 6.3.3, table 6.5): the spine’s per-challenge composition (built), the list-binding compilation (not stated), Theorem B.7 (specified), round-by-round to state-restoration (commented out upstream), the Merkle interface with its hash term (empty as stated), the chain lemma (not stated), the grinding model (no theorem exists), and the family composition over the prover’s sizes (not stated).

The status vocabulary of the tree (one tag per node), with the number of nodes whose *first* tag it is:

Table 6.1: The status tags of the tree and the count of nodes by first tag (a node may carry two).

Tag	Meaning at b435631	Nodes
[status: built and proved]	stated and proved in leanerVM; #print axioms gives the kernel’s three standard axioms only (as recorded by the named dossier’s probe)	21
[status: built, definition]	a definition in leanerVM, load-bearing, with no proof obligation of its own (its content is what later theorems prove things about)	10
[status: library, proved at the old pin]	proved in ArkLib, VCVio, CompPoly, Clean or Mathlib at the old pin (and, where the dossier checked, at the new one)	6
[status: library, proved at the new pin only]	landed upstream between the pins	0
[status: admitted upstream]	stated in a library and sorry at both pins	2
[status: specified only]	stated in the blueprint (a sketch), not built	66
[status: not specified]	needed by the composition and stated nowhere in the blueprint	25
[status: unprovable as written]	the review found the blueprint’s statement cannot be proved (or stated) as it stands	10
[status: wrong as written]	the review found the blueprint’s description false of leanVM or of the code	2
[status: trusted, assumed interface]	a structure whose fields are theorems another roadmap or library owes; every theorem that uses it takes it as an argument	2
[status: trusted, hypothesis]	a hypothesis of a theorem on the chain that nothing on the chain discharges	4
[status: trusted, transcribed data]	Category B data (layouts, constants, schedules) whose only check is a fixture	4
<i>total</i>	nodes with a status (the other three are a group header and two pointers)	152

Nodes on which the review found the blueprint’s statement wrong, missing or unprovable: 57 (wrong 22, missing 26, unprovable 9). Every built theorem (the spine, the public-input phase, Layer 1, the ported append, the counting bounds) rests on the kernel’s three standard axioms; no admitted library theorem is in its closure (lib-arklib B.1; code-spine P1; probes-rerun 1.0 at the new pin). A node marked [review: ...] is one on which the review found the blueprint’s statement wrong, missing or unprovable; the tag says which. In the tree (section 6.2) such a node carries a leading [!] mark.

The trusted leaves (what a reader of the planned chain must accept, today and after every hole is filled as planned; the nodes are 2.1.1.3, 2.1.2.1, 2.1.2.2, 2.1.2.4.1, 2.1.2.4.11, 2.1.2.6, 2.1.2.7, 2.1.5, 2.2.2.6, 2.2.3.2, 2.3.3.5, 2.3.4 (whole), 2.3.4.3.2, 2.3.4.6, 2.5.6.4, 2.5.7.4, 2.6.1, 2.6.2.5, 2.6.2.6, 2.6.3, and the four unbuilt phases’ bundles):

1. *Assumed interfaces*: FiatShamirSecurity (false as stated, 2.1.2.1), BcsSecurity (empty as stated, 2.1.2.2), McaJohnson (2.1.2.4.1), the Flock phase’s definition, completeness and security (FlockInterface, 2.3.4), the witness generator FlockWitnessGen with its lemma (2.6.3), and, until every hole is filled, the Complete and Security fields of the bus, table, Flock and opening phases (2.3).
2. *Admitted upstream theorems the plan consumes*: [BCHKS25] Theorem 4.6 (rs_mcaError_le_in_johnson_range, a preprint with a sketched proof); the round-by-round to plain implication, if the plain corollary is stated (2.1.2.7). The sumcheck’s admitted completeness and knowledge soundness (2.5.1.2, 2.5.1.3) and ArkLib’s admitted append (2.2.2.1) are replaced by local proofs in the plan and are not leaves.
3. *Hypotheses nothing discharges*: the random-oracle heuristic for BLAKE2s’s compression and its collision resistance (2.1.5); WellFormedBytecode prog (absent from Layer 13’s statements; 2.7); the resource bound on the trace and the constructed witness that baseProver_complete needs (2.7); the

success of the honest grind (2.1.4); the carrier of “the R1CS with the constant position holds $\Rightarrow$ the limb slots compress” (2.6.2.5); the F_2 -independence of Flock’s 128 fixed equality weights (2.3.4.3.2, true by a probe, in no theorem); the oracle-freeness of the front phases (2.1.2.3.3, finding 2); a bound on `piopError` (2.2.2.5); `s.Admissible prog` at the adaptor (2.6.2.6, discharged by the verifier’s setup check once `Admissible` has the two windows).

4. *Transcribed data whose only check is a fixture*: the leanISA instance (layouts, separators, count columns, boundary blocks, lines, the tie order of equal-size blocks, the leaf stacks’ order; 2.6.1, 2.5.6.4, 2.3.1.3.8), the WHIR parameters and Merkle formats (2.1.2.4.11, 2.1.2.6, 2.1.1.3), Flock’s circuit and constants including `R1CS_DIGEST` (2.3.4.6), the scalar oracle interfaces (2.5.7.4), and the fact that the phases’ definitions are leanVM’s (2.2.2.6: no theorem of the oracle protocol says so; `verify_iff_compiled` with its fixture and the review of each `Def` do).

The error, composed (section 6.3). Per challenge, the review finds the blueprint’s charges right and tight for $\xi ((B+2)/|E|)$, the table rounds $(3/|E|)$, $r (1/|E|)$, $\lambda ((J-1)/|E|)$ and $(\alpha, \beta) (4 \cdot 2^{\{\mu_{\text{bus}}\}}/|E|)$, within a factor 2; wrong for the GKR rounds $(4/|E|)$, not $5/|E|$, the last combiner and the binary layer’s challenge; charged three incompatible ways for the recycled zerocheck point, which adds nothing; and describing rounds that do not exist for the opening (WHIR’s own rounds carry $2L_i/|E| + 2^{\{\ell_i - j\}}a_i/|E|$). The interactive error of the oracle protocol is about $2^{\{-159.7\}}$ under the stacking window, dominated by ring switching’s $2^{\{32\}}/|E|$; the blueprint’s `piopError_le` is false at the per-log caps and true under the window. After Fiat–Shamir the governing quantity is the largest per-challenge error of the compiled protocol, set by WHIR’s query rounds ($2^{\{-111\}}$ before 17 bits of grinding) and folds ($2^{\{-128.2\}}$), not by anything in the oracle protocol; the blueprint’s `niError = Q \cdot \max_i \epsilon_i` lacks the list-size factor `L_0` on every front challenge, the hash-collision term, a model of grinding (without which it certifies about 111 bits), the family composition over the announced sizes and the program-hash term.

Findings by severity (section 6.5): *major* — twenty-one obligations of the composition have no hole, layer or owner (finding 1); the compilation needs every phase before the opening to make no oracle query, and nothing states or enforces it (finding 2). *Minor* — the per-challenge error has no carrier for the grinding bits (finding 3). *Note* — the smallest set of unbuilt nodes that makes the master theorems about leanVM (finding 4).

Validation of the outlines (section 6.4): `docs/architecture.md`’s T4 has the right decomposition and the right headings, gets the shape of the base theorem wrong (a probability attached to a closed Boolean, the defect the blueprint transcribes), drops `WellFormedBytecode` at T4, and lacks the list-binding compilation, the grinding model and the family composition; its completeness dual (T2 with the honest prover, resource conditions recorded) is right and the blueprint diverges from it. The holes table covers what it names, with the corrections of section 6.4.2 per row, and omits the twenty-one units of finding 1.

Not done, and what would verify it. No Lean was run: every status rests on the sibling dossiers’ recorded probes or on reading; the counts are by the first status tag of each node (a node may carry two). The per-challenge errors were taken from the dossiers that worked them (gt-bus D.3, gt-table-pub D, gt-flock-ring 5.2, gt-opening-compile F) and checked for consistency between them, not re-derived. The value of `L_0` at production parameters is the opening dossier’s reproduction of the Rust’s search (110 to 396 at level 0), not a run of the Rust’s `validate()`.

6.2 The tree

How to read it. The root is what the proof system must deliver to the rest of the verification: the two theorems of the blueprint’s Layer 13, which are the proof-system half of target T4 (`arch:261-275`). Each level below is what the level above consumes. A node has: a name (and the Lean name the blueprint or the code gives it), what it states for a zkVM engineer, which node(s) it feeds, its status tag, and where the review’s dossiers speak about it. The tree is written for the proof system as the blueprint plans it, with the review’s corrections marked in place; the adaptor (Layer 3) and leanISA’s theorems are a separate branch (2.6), and the hypotheses discharged by verifier checks or by conditions on public data are collected in a table at the end (2.7).

One convention: “the front” names the oracle protocol’s phases before the opening, `commit » bus » table » pub » flock`; “the pool” is the list of claims those phases leave for the opening.

Each node is shown as the dossier writes it: its number and name in bold; in parentheses the Lean name and the lines that state it; what it states for a zkVM engineer; the nodes it feeds; its status tag; and where

the review’s dossiers speak about it (*See*). A leading **[!]** marks a node on which the review found the blueprint’s statement wrong, missing or unprovable; its **[review: ...]** tag says which. The numbering is the dossier’s, kept so that the text can refer to a node by its number; the dossier has no tier 2.4.

6.2.1 The root (2.0): what the proof system delivers (Layer 13, the proof-system half of T4)

[!] **2.0.1 Base extraction** (`baseVerifier_extractsExecution`, bp:1247-1248; arch:264-269). *States*: an adversary that makes the executable verifier `verify prog input proof` accept yields, except with probability `niError`, a stack whose rebuilt witness satisfies `leanISA`’s relation, hence an execution of the program on the public input. *Feeds*: T4, T7 through T5 and T6 (recursion consumes the extracted witness). *Status*: **[status: unprovable as written]** **[review: wrong]** — as sketched it has no probability space (its hypothesis is a closed Boolean), concludes an existential (soundness, not knowledge), names no adversary or query budget, and drops `WellFormedBytecode prog`, which `constraintSoundness` needs; the well-formed statement is a bound on the event “`verify^H` accepts and the extracted stack has no execution” over a random oracle `H`, for a `Q`-query adversary, under `WellFormedBytecode`. *See*: `gt-opening-compile E.1`; boundary-adaptor findings 3 and 16, E.6; docs-debt (DD1). *Consumes*: 2.1.2 (knowledge soundness of `verify`), 2.6.2 (the adaptor’s soundness bridge), 2.6.6 (`constraintSoundness`), 2.1.5 (the random-oracle heuristic, non-formal).

[!] **2.0.2 Base completeness** (`baseProver_complete`, bp:1249-1250; arch:273-275). *States*: from a valid execution (with fill blocks in the program), the honest Lean prover’s proof is accepted by `verify`. *Feeds*: T4’s completeness half, T8. *Status*: **[status: unprovable as written]** **[review: unprovable]** — false without a resource hypothesis (a valid execution with `κmem = 32` has a stack above 2^{28} , which the verifier rejects), and “the witness of `t`” needs a witness generator (T2, out of scope) or a classical choice; the honest prover’s grinding search makes `prove` partial, so the equation must be `prove ... = some proof → verify ... = true`. *See*: boundary-adaptor finding 4, C (rows “resource bound”, “constructed witness”); `gt-opening-compile E.5`; docs-debt (DD19). *Consumes*: 2.6.7 (`constraintCompleteness`), 2.6.3 (`m3Holds_stackOf`), 2.2.1 (`piop_perfectCompleteness`), 2.1.4 (completeness through the compilation), 2.1.6 (the prover’s grind).

6.2.2 The compiled verifier (2.1; Layers 11 and 12)

[!] **2.1.1 Refinement of the executable verifier** (`verify_iff_compiled`, bp:1218-1221). *States*: `verify prog input proof = true` exactly when, for some admissible sizes `s` read from the proof, the Fiat–Shamir compilation of the Merkle-compiled oracle verifier of `leanISAInstance prog s` accepts the decoded messages under the BLAKE2s chain, and every grinding check passes. *Feeds*: 2.0.1, 2.0.2 (it is what ties `verify` to every theorem below). *Status*: **[status: unprovable as written]** **[review: unprovable]** — the sketch applies `ArkLib’s Verifier.fiatShamir` to an oracle verifier whose messages are the stack and the codewords in the clear, has no `decode`, no Merkle compilation, and no grinding conjunct; statable in the corrected form of `gt-opening-compile E.4`. *See*: `gt-opening-compile E.4`; lib-arklib G.4. Sub-obligations:

[!] **2.1.1.1 The Merkle compilation of the IOPP** (`bcsCompile`, no blueprint name). *States*: the interactive verifier whose messages are roots, scalars, nonces and openings, obtained from the codeword-oracle verifier by replacing each oracle message by a root and each query by an opening check. *Status*: **[status: not specified]** **[review: missing]** — `ArkLib’s BCSTransform` is commented out at both pins. *See*: `gt-opening-compile E.4` item 1; lib-arklib D.3.

2.1.1.2 The Fiat–Shamir chain as a challenge oracle (`FsState.seed/observe/sample/grind`, bp:1200-1205). *States*: the chain is a function of statement and absorbed messages (in the wire encoding: three of four round coefficients, roots as two scalars, openings not absorbed), so it is a `QueryImpl` for `ArkLib’s fsChallengeOracle`. *Status*: **[status: specified only]** (Category B). *See*: `gt-opening-compile C.1`, E.4 item 2; probe H.2 there.

2.1.1.3 The proof object and its decoding (`Proof, RoundPoly.decode`, bp:1207-1212). *States*: the stream order of `cpu/mod.rs:711-779`, the dropped coefficient recovered from the running claim, the Merkle format (pruned wire form or raw form: unspecified). *Status*: **[status: specified only]** **[status: trusted, transcribed data]**. *See*: `gt-opening-compile C.2`, C.3; `gt-table-pub` finding “the round message”.

2.1.1.4 The setup checks (`Sizes.Admissible` and the canonical-encoding checks). *States*: verify rejects sizes outside the caps, the stacking window $15 \leq \mu \leq 28$ and the rate window $1 \leq \rho \leq 4$, non-canonical limbs, an unconsumed stream. *Status*: `[status: unprovable as written]` for `Admissible` (it lacks the two windows; `Sizes.ofWitness` cannot produce `logInvRate`), `[status: not specified]` for the encoding checks. *See*: gt-bus B, G4, G9; gt-opening-compile D.1, D.2; boundary-adaptor finding 5.

2.1.1.5 Native settlement of the fixed columns (`settleFixedClaims`, bp:1235-1236). *States*: verify evaluates the bytecode multilinear and the Flock matrices itself (`idxColumnEval`, bytecode `ColumnEval` of Layer 1; the circuit walk of Flock). *Status*: `[status: specified only]`; the Flock walk has no owner. *See*: gt-flock-ring 5.5; code-layer1 A.9.

[!] **2.1.2 Knowledge soundness of the executable verifier** (`verify_knowledgeSound` (`fs`) (`bcs`) (`mca`) (`flock`), bp:1226-1227). *States*: against a Q-query adversary in the random-oracle model, `verify^H` accepts a proof whose extracted stack fails `M3Holds` with probability at most `niError s` Q. *Feeds*: 2.0.1. *Status*: `[status: unprovable as written]` `[review: unprovable]` — its three interfaces cannot be inhabited by any `ArkLib` theorem at either pin, and even inhabited they do not compose to the conclusion: four steps are missing (2.1.2.3 to 2.1.2.6). *See*: gt-opening-compile E.2, E.3, G; lib-arklib G.4; literature A.3.

[!] **2.1.2.1 Fiat–Shamir security interface** (`FiatShamirSecurity`, bp:1224). *States*, as the blueprint has it: round-by-round knowledge soundness of the IOPP gives knowledge soundness of its Fiat–Shamir compilation in the random-oracle model with error $Q \cdot \max_i \epsilon_i$. *Status*: `[status: trusted, assumed interface]` `[review: wrong]` — as a universal claim over IOPPs it is false (an IOPP’s messages are oracles; the compiled error carries a hash term; leanVM grinds); to be true and strong enough it must be stated for leanVM’s construction: a state-restoration-sound interactive argument, the `fs/lib.rs` chain with `H` random, named ground rounds with bits `b_i`, error $(t + k) \cdot \max_i 2^{\{-b_i\}} \epsilon_i + 3.5 t^2 / 2^{256}$; the proof-of-work factor is an assumption no published theorem covers. *See*: literature A.3, A.4 (two findings); gt-opening-compile E.2; lib-arklib D.3, G.4. Its own leaves:

[!] **2.1.2.1.1 Round-by-round to state-restoration knowledge soundness**. *States*: the per-round errors bound a state-restoration adversary’s success by $(t + k) \cdot \max_i \epsilon_i$. *Status*: `[status: not specified]` `[review: missing]` — commented out in `ArkLib` with the wrong error (`Implications.lean:230-254`); not in the blueprint’s interface list. *See*: gt-opening-compile E.2 (last bullets); lib-arklib D.3.

[!] **2.1.2.1.2 The hash chain is as good as the ideal per-round challenge oracle**. *States*: the ratcheted BLAKE2s chain with its four tags and fixed-width blocks loses at most a collision term against the oracle keyed by the whole prefix; the encoding is injective. *Status*: `[status: not specified]` `[review: missing]`. *See*: literature E.6 (LT10), G.3 (transcript-encoding advisories); gt-opening-compile E.2 (c).

[!] **2.1.2.1.3 The grinding model**. *States*: a query to `H` at a ground round costs $2^{\{b_i\}}$ queries in expectation, so the effective error of that round is $2^{\{-b_i\}} \epsilon_i$. *Status*: `[status: not specified]` `[review: missing]`; no theorem in the literature for multi-round IOPs. *See*: literature A.3 item 3, G.5 (LT13); gt-opening-compile D.3.

[!] **2.1.2.1.4 The hash-collision term**. *States*: $3.5 t^2 / 2^{256}$ (or $3(Q^2 + 1) / 2^{256}$) for the chain and the trees. *Status*: `[status: not specified]` `[review: missing]`. *See*: literature A.3 item 2 (LT3).

2.1.2.1.5 Program binding through its hash. *States*: two programs with the same `bytecode_hash` share every challenge; the statement binds the program up to a BLAKE2s collision, a term of `niError`. *Status*: `[status: not specified]`. *See*: gt-opening-compile E.1.

[!] **2.1.2.1.6 The family composition over the prover’s sizes**. *States*: verify dispatches on eight announced sizes; a bound per admissible `s` transfers to the union only with an argument on how the adversary’s queries split among members, and `niError` must be uniform in `s` or take the maximum. *Status*: `[status: not specified]` `[review: missing]`. *See*: boundary-adaptor B.3, finding 6; gt-opening-compile E.1.

[!] **2.1.2.2 Merkle (BCS) security interface** (`BcsSecurity`, bp:1225). *States*, as the blueprint has it: “Merkle-compiled oracles: extraction from collision resistance / ROM”. *Status*: `[status: trusted, assumed interface]` `[review: wrong]` — names a property with no error and no object; to be true and strong enough: the Merkle compilation of `pcs/merkle.rs`, `fs/merkle.rs` (leaf = BLAKE2s of the row image, node = BLAKE2s of two children, height and leaf width fixed from public data) preserves

state-restoration knowledge soundness in the random-oracle model with the straight-line extractor reading every committed leaf off the query log, up to $3(Q^2+1)/2^{256}$. *See:* gt-opening-compile E.2, D.4 (checks M1 to M7); lib-arklib G.4. Its leaves:

2.1.2.2.1 Binding of the root to the opened rows (check M6). *States:* the recomputed root equals the committed one, so the opened rows are the committed ones; without it a total break. *Status:* [status: specified only] (Layer 11’s merkleVerify). *See:* gt-opening-compile D.4.

[!] 2.1.2.2.2 Role separation of the one BLAKE2s map (chain block, Merkle node, Merkle leaf, grinding). *States:* no cross-role input is exploitable; replaces domain separation, which the trees lack (fixed height and leaf width do the work, check M7). *Status:* [status: not specified] [review: missing]. *See:* literature E.6 (LT10); gt-opening-compile D.4 (M7), B.4.

2.1.2.2.3 Straight-line leaf extraction from the query log. *States:* every leaf the prover can open was hashed, so it is in the log except with the collision or preimage term. *Status:* [status: not specified]. *See:* gt-opening-compile E.1.

2.1.2.2.4 VCVio’s Merkle-tree library (CryptoFoundations/MerkleTree/, nineteen sorry-free modules at f9dc47d9). *States:* a generic Merkle tree with its collision-resistance argument. *Status:* [status: library, proved at the old pin]; fit with leanVM’s trees unverified. *See:* docs-debt (DD15).

[!] 2.1.2.3 The list-binding compilation (no blueprint name; “Layer 12 turns list binding into extraction of the one q ”, bp:1190; acceptance test 11). *States:* let Front' be the front with the commit message replaced by an interleaved K -word w (the level-0 codeword with position queries) and $C := \text{Front}' \gg \text{whirOpen}$; then C is worst-case round-by-round knowledge sound for $M3\text{Rel}$ with the extractor “list-decode w , return the first member satisfying $M3\text{Holds}$ ”, with per-challenge errors $L_0 \cdot \epsilon_i$ at the front’s challenges and Theorem B.7’s at WHIR’s. *Feeds:* 2.1.2; it is where the list-size factor enters niError . *Status:* [status: not specified] [review: missing] — the blueprint assigns the step to `verify_knowledgeSound`, whose sketched hypotheses cannot prove it. *See:* gt-opening-compile E.3, G; literature A.3 item 4 (LT2); gt-flock-ring 8.13. Its leaves:

[!] 2.1.2.3.1 Every member of the list is K -valued and of the right size. *States:* a codeword of the code over E within radius $\gamma_0 < 1 - \sqrt{\rho_0}$ of a K -valued word is K -valued (its y and y^2 coordinate projections vanish on more than $2^{\kappa_0} - 1$ points), of μ variables after the level-0 relayout, with zero rows where w has them. *Status:* [status: not specified] [review: missing]; a two-line argument in the dossier, not in the specification. *See:* gt-opening-compile G.

2.1.2.3.2 State functions indexed by a finite set compose by a union bound. *States:* a knowledge state function “for every $g \in \Lambda(w)$, the front’s state at (g, τ) is false” escapes with probability at most $|\Lambda(w)|$ times the front’s worst-case bound; the worst-case form of the spine is what makes this a per-transcript union. *Status:* [status: not specified]; ArkLib has no such lemma. *See:* gt-opening-compile E.3.

[!] 2.1.2.3.3 The front’s verifier makes no oracle query. *States:* every phase before the opening reads the stack only through its extractor, never through a query, so Front' ’s verifier is Front ’s (a cast of `OracleReduction`, ArkLib #383’s `Cast.lean`). *Status:* [status: not specified] [review: missing] — true of the two built phases; nothing states or enforces it for the others (finding 2). *See:* gt-opening-compile E.3.

2.1.2.3.4 The seam step. *States:* the front’s `toFun_full` in the contrapositive: state false at the front’s last transcript implies some pooled claim is false of g , which is $\notin R_{\text{open}}$, Theorem B.7’s start invariant. *Status:* [status: not specified]. *See:* gt-opening-compile E.3.

2.1.2.3.5 The compiled extractor computes. *States:* enumeration of the finite code is a definition (Lean has no cost model); efficiency in the literature’s sense is Guruswami–Sudan, unnamed. *Status:* [status: specified only] (acceptance test 24 says “computable”). *See:* literature A.4 (“computable is not efficient”, LT4), G.2.

2.1.2.4 WHIR’s round-by-round soundness (`whirOpen_rbrSoundness` (mca : McaJohnson), bp:1175; Annex B Theorem B.7, b-polynomial-commitment-scheme.tex:139-152). *States:* for the relation R_{open} (“some codeword within γ_0 of the committed word satisfies every weighted claim”), round-by-round soundness with the per-message errors of Annex B’s table; soundness for a language, no extractor named. *Feeds:* 2.1.2.3. *Status:* [status: specified only]; a Lean proof would be the first machine-checked proof of this variant. *See:* gt-opening-compile B.5, B.6; literature C.3. Its leaves:

2.1.2.4.1 Mutual correlated agreement up to the Johnson bound (McaJohnson, bp:1176; Ark-

- Lib `rs_mcaError_le_in_johnson_range`, `CapacityBounds.lean:203-219`). *States*: [BCHKS25] Theorem 4.6 as printed (with $m = \max(\lceil \sqrt{\rho}/(1-\sqrt{\rho}-6) \rceil, 3)$ and the reduced rate), for the affine line, converted to the fold in characteristic two by Annex B’s footnote. *Status*: [status: **admitted upstream**] [status: **trusted, assumed interface**] — a preprint theorem with a sketched proof; the only coding-theoretic input. *See*: literature C.2, C.4 (LT7, LT8); gt-opening-compile E.2; lib-arklib B.3.
- 2.1.2.4.2 The Johnson bound** (Annex B B:391-414). *States*: the list within radius $\gamma_i < 1 - \sqrt{\rho_i} - \eta_i$ has at most $L_i = 1/(2\eta_i\sqrt{\rho_i})$ members. *Status*: [status: specified only] (proved in the annex, not in Lean). *See*: gt-opening-compile B.5, F.2.
- 2.1.2.4.3 Folding preserves lists** (Lemma B.10, B:189-198, with the $2^{\ell-1}$ row union). *States*: the fold of a word close to a list is close to the folded list, except with the MCA error times the row union. *Status*: [status: specified only]. *See*: gt-opening-compile B.5.
- 2.1.2.4.4 Out-of-domain separation** (Lemma B.11, B:200-206; check W2). *States*: after the OOD sample the new oracle’s list has at most one member consistent with the transcript, except with $C(L_i, 2) \cdot \mu_i/|E|$; without OOD binding the query phase would pay a union over the list. *Status*: [status: specified only]. *See*: gt-opening-compile D.5 (W2), F.2.
- 2.1.2.4.5 Per-level batching** (check W4). *States*: the level’s claims (OOD first, queries after, running claim at λ_i^{θ}) settle in one identity, error $(J_i - 1)L_i/|E|$. *Status*: [status: specified only]. *See*: gt-opening-compile D.5 (W4).
- 2.1.2.4.6 The fold rounds**. *States*: each of the ℓ_i sumcheck rounds of level i costs $2L_i/|E| + 2^{\ell_i-j} \epsilon_i$ (the running claim is a cofactor with c_1 derived, as in the table sumcheck). *Status*: [status: specified only]; the blueprint’s Layer 10 “ $2/|E|$ per round” is the tail’s value, wrong for these. *See*: gt-opening-compile B.6.
- 2.1.2.4.7 The query rounds** (check G1 for the nonce, W3 for row consistency). *States*: t_i positions sampled after a 17-bit proof of work; error $(1 - \gamma_i)^{t_i} \approx 2^{\{-111\}}$ before grinding. *Status*: [status: specified only]; the grinding is outside every theorem (2.1.2.1.3). *See*: gt-opening-compile D.3, D.5, F.2; literature G.5.
- 2.1.2.4.8 The terminal check** (W5) and the final plaintext level. *States*: one equation weight $\cdot \tilde{f}_{\text{final}}(\rho_{\text{tail}}) = t_r$ closes the chain; without it nothing is checked about q . *Status*: [status: specified only]. *See*: gt-opening-compile D.5.
- [!] 2.1.2.4.9 The column weight of the encoder** (`encode_column_weight`, bp:1170; Lemma B.14). *States*: position x of the codeword is an inner product of the message with a weight whose extension the verifier computes in closed form. *Status*: [status: specified only] [review: **wrong**] — as an existential it is true of any linear map and says nothing about the novel basis; the needed object is the definition `columnWeight x` with its closed-form `mle` and the equation `encode κ R f x = (columnWeight x).pair f`. *See*: gt-opening-compile B.6.
- [!] 2.1.2.4.10 The encoder** (`encode`, `novelBasis`, bp:1168-1169; CompPoly’s AdditiveNTT). *States*: Reed–Solomon encoding in the novel polynomial basis on the additive domain in K , over K at level 0 and over E at every later level. *Status*: [status: specified only] [review: **wrong**] — `encode` over K alone serves level 0 only. *See*: gt-opening-compile B.6; lib-others B (CompPoly’s NTT exists, generic over a basis).
- 2.1.2.4.11 The parameters** (`ladder`, `ladder_queries_eq`, the six constants, bp:1155-1165). *States*: fold factors, rates, query counts and grinding bits per level as `whir_config.rs` derives them; `(ladder 15 1).map (·.queries) = [223, 55]`. *Status*: [status: **trusted, transcribed data**]; the Python table is reproduced by a scratch port of the Rust’s search. *See*: gt-opening-compile F.2, B.3 (Level lacks η).
- 2.1.2.5 Perfect completeness of WHIR** (`whirOpen_perfectCompleteness`, bp:1174). *States*: the honest prover’s folds and openings pass every level check. *Feeds*: 2.1.4. *Status*: [status: specified only]. *See*: gt-opening-compile B.6.
- 2.1.2.6 Merkle definitions** (`merkleRoot`, `merkleVerify`, `blake2sBytes`, bp:1179-1182). *States*: BLAKE2s-256 of leaf images and of two-child nodes, the raw path check, the RFC 7693 byte hasher over leanISA’s `compress`. *Status*: [status: specified only] [status: **trusted, transcribed data**]. *See*: gt-opening-compile B.4, B.6.
- 2.1.2.7 Plain knowledge soundness from the round-by-round form** (ArkLib `rbrKnowledge`

`Soundness_implies_knowledgeSoundness`, `Implications.lean:223-228`). *States*: an interactive prover convinces the verifier without a good first message with probability at most $\Sigma \epsilon_i$. *Feeds*: the blueprint’s plain corollary (`bp:261`); not needed if 2.1.2.1 consumes the round-by-round form directly. *Status*: [\[status: admitted upstream\]](#); its transcript-level half is proved by `lib-arklib`’s probe `PlainReading` in forty lines; its conclusion forgets the named extractor. *See*: `lib-arklib` D.1, D.3, G.3; code-spine C.7.

[!] **2.1.3 The interactive-oracle form of the compiled protocol** (`leanVmIopp`, `bp:1214-1215`). *States*: `commit' » bus » table » pub » flock » whirOpen`, where `commit'` sends the level-0 codeword and `whirOpen`’s level-0 batching is the opening phase’s λ . *Status*: [\[status: specified only\]](#) [\[review: wrong\]](#) — the blueprint says “WHIR in place of the evaluation oracle” after a μ -round opening sumcheck; see 2.3.5.1. *See*: `gt-opening-compile` A.4, A.5.

[!] **2.1.4 Completeness survives the compilation** (`bp:1256-1257`: “without any assumption”). *States*: an honest proof accepted by the interactive verifier on the chain’s challenges is accepted by `verify` (a statement about two definitions), and the grinding search succeeds. *Feeds*: 2.0.2. *Status*: [\[status: wrong as written\]](#) [\[review: wrong\]](#) — the grind is an unbounded search; in the random-oracle model it fails with probability $(1 - 2^{-17})^{2^{64}}$, and with BLAKE2s fixed its success is a conjecture; `prove` must be partial or fuelled. *See*: `gt-opening-compile` E.5.

2.1.5 The random-oracle heuristic. *States*: BLAKE2s’s compression function is the random oracle H of 2.1.2; the theorem about `verify` itself is the instantiation of the definitions only. *Status*: [\[status: trusted, hypothesis\]](#) — cannot be a Lean hypothesis (false for any fixed function) nor an axiom; must be named in Layer 13 and in `docs/architecture.md`; classical (no quantum random-oracle model). *See*: `gt-opening-compile` E.1; boundary-adaptor C (row “BLAKE2s as a random oracle”), finding 3; literature B.4 (LT6), G.5 (LT14).

2.1.6 The honest prover (`prove`, `leanVmProver`, `bp:1145`, `bp:1250`). *States*: each phase’s prover is a computable function of the witness, the challenges so far and the layer tables; the compiled prover adds encoding, Merkle trees and the grind. *Feeds*: 2.0.2. *Status*: [\[status: specified only\]](#) (the spine’s `leanVmProver` is built as the composed prover of whatever bundle is supplied); the phase-level `Defs` bundle a real-number error, so on the real protocol neither the verifier nor the extractor compiles (`lib-arklib` G.5). *See*: `lib-arklib` G.5 (AK5); code-spine D.5.

6.2.3 The master theorems of the oracle protocol (2.2; the spine)

Both are one-line proofs of a composed component’s own field, stated over an abstract instance $I : M3Instance$ and a bundle $P : Phases\ I$ of five phases with their proofs; they say nothing about `leanVM` until an instance and `leanVM`’s phases are supplied (code-spine C.4).

2.2.1 Perfect completeness of the oracle protocol (`piop_perfectCompleteness` (P) ($C : P$. Complete), `Spine/Compose.lean:162-170`). *States*: for every (input, q) with `M3Holds` I input q , the honest prover and the composed verifier accept with probability one and end in `Seam.done` (which is `True`). *Feeds*: 2.0.2 through 2.1.4. *Status*: [\[status: built and proved\]](#) (probe `PlAxioms` of code-spine; conditional on `Phases.Complete`). *See*: code-spine C.4, C.6; `lib-arklib` E.

2.2.1.1 ArkLib’s completeness composition (`OracleReduction.append_perfectCompleteness_of_guarded_verifiers`, `Composition/Sequential/OracleCompleteness.lean:54-67`). *States*: two perfectly complete reductions with guarded verifiers, the second complete from every shared-oracle state, compose perfectly; the additive-error form exists too. *Status*: [\[status: library, proved at the old pin\]](#) (probe `AxiomsArkLibBuilt`). *See*: `lib-arklib` E.2, E.3.

2.2.1.2 The component-level append (`Component.Complete.append`, `Component.lean:162-169`). *Status*: [\[status: built and proved\]](#). *See*: code-spine A (catalogue).

2.2.1.3 The side conditions guarded and outputPure (`Component.Complete`, `Component.lean:76-85`). *States*: the verifier is a check followed by a pure verdict; the prover’s output makes no oracle query. *Status*: [\[status: built, definition\]](#); both derivable over the empty shared oracle (`ArkLib’s` `GuardedForm.ofEmpty`, `Prover.instOutputIsPureEmpty`), so `outputPure` is redundant. *See*: code-spine finding “outputPure is redundant” (CS5); `lib-arklib` G.7.

[!] **2.2.1.4 Completeness carries no error** (`Component.Complete.complete` is perfect Completeness). *States*: every phase must be perfectly complete; a phase complete up to

an error has no place. *Status:* [status: built, definition] [review: missing] — ArkLib composes additive completeness errors at both pins; whether Flock needs one is settled by gt-flock-ring (it does not, if the Lean prover sends the true coefficients). *See:* lib-arklib G.2 (AK2); gt-flock-ring 4, 8.4, 8.12.

2.2.2 Round-by-round knowledge soundness of the oracle protocol (piop_rbrKnowledgeSoundness (P) (S : P.Security), Spine/Compose.lean:172-179). *Status:* ArkLib’s worst-case round-by-round knowledge soundness of the composed verifier’s toVerifier, from M3Rel I to Seam.done I, for the named extractor piopExtractor P S and the composed knowledge state function, at the per-challenge error piopError P: for every statement, every prefix and every challenge index, the probability over one uniform challenge that the state goes from false to true is at most the error of that challenge. *Feeds:* 2.1.2.3 (through the list-binding compilation), 2.1.2.7. *Status:* [status: built and proved] (probes P1Axioms, AxiomsLeanerVM; conditional on Phases.Security). *See:* code-spine C.4, D.3; lib-arklib D.1, D.2 (adequacy of the notion).

2.2.2.1 The knowledge-soundness append for a guarded first verifier (Verifier.append_rbr KnowledgeSoundnessWorstCaseWith_of_guarded_first, ToArkLib/KnowledgeAppend.lean:510; the port of ArkLib #615). *Status:* two worst-case round-by-round knowledge-sound verifiers, the first guarded, append to one with the appended extractor and each challenge keeping its own error. *Status:* [status: built and proved] (diffed line by line against #615’s file). *See:* lib-arklib C.1, C.2; ArkLib’s own append_rbrKnowledgeSoundness is [status: admitted upstream] at both pins (ledger A2), and #615 is open and in conflict on a framework upstream calls legacy (lib-arklib G.3).

2.2.2.2 The appended knowledge state function (Verifier.KnowledgeStateFunction.append Guarded, KnowledgeAppend.lean:474). *Status:* the first component’s state until its verdict, then the second’s, with the seam round handled (backward_right, extractMid_seam). *Status:* [status: built and proved]. *See:* lib-arklib C.2, G.9.

2.2.2.3 ArkLib’s extractor append (Extractor.RoundByRound.append, Append/StateFunction.lean:75). *Status:* [status: library, proved at the old pin]. *See:* lib-arklib B.2.

[!] **2.2.2.4 The composed extractor** (piopExtractor P S, Compose.lean:153; commitExtractor, Component.sendExtractor, SendOracle.lean:134). *Status:* the commit phase’s extractor reads the stack off the first message; the phases’ extractors (all with witness Unit) are appended through the verdicts; the chain is computable (isNoncomputable = false). *Status:* [status: built, definition] [review: missing] — no definition “the extracted stack” exists: with a phase that has a round, extractOut has type Unit and the stack is extractMid at round 0, so acceptance test 24’s rfl witness is ill-typed (probe P3cExtractor). *See:* code-spine D.5, finding “the extracted stack is not a definition” (CS4); lib-arklib D.6.

[!] **2.2.2.5 A bound on the composed error** (piopError P := P.toDef.err, Compose.lean:149; the blueprint’s piopError_le, bp:1130). *Status, as the blueprint has it:* $\sum_i \text{piopError } s_i \leq 2^{40}/|E| + \text{flockError}$ for admissible sizes. *Status:* [status: unprovable as written] [review: unprovable] — the spine’s piopError takes a bundle P, not sizes s, and nothing bounds it: five phases that draw a challenge, check nothing and declare error 1 inhabit Phases.Security toy and both master theorems hold of them (probe P4Junk); the numeric bound is false at the per-log caps ($\mu_{\text{bus}} = 38$ makes the (α, β) term alone $2^{40}/|E|$) and true with the stacking window, with a margin of about 2^8 . *See:* code-spine C.5, D.3 (b), finding “the declared error is unconstrained” (CS1); gt-bus D.4, G4; gt-opening-compile F.1.

2.2.2.6 The phases’ definitions are leanVM’s. *Status:* nothing in the oracle protocol’s theorems pins the phases to leanVM: a zero-round phase whose verifier reads the whole stack through the oracle and decides the seam itself fills every slot at error zero. What pins them is 2.1.1 with its fixture and review of each Def. *Status:* [status: trusted, transcribed data] (as a fact about the design). *See:* code-spine D.3 (d); gt-table-pub E.4 item 6 (TP13).

2.2.3 The relation and the seams (built definitions the theorems are about).

2.2.3.1 The relation on the stack (M3Holds I input q, Spine/Instance.lean:219-220; M3Rel). *Status:* every constraint polynomial of every table vanishes on that table’s rows read out of q; the pushed and pulled 16-tuples (flush rows and boundary blocks of each side) are the same multiset (List.Perm, counted in $\mathbb{N}$); every count cell is nonzero; every public line’s cells 0 and 1 hold the statement’s values; the instance’s auxiliary predicate holds. *Status:* [status: built, definition]; says what the specification’s accept list says, clause by clause, and each clause is refuted alone on the toy (probe P2Relation). *See:* code-spine C.1, D.1; gt-bus E.5; boundary-adaptor A.2.

2.2.3.2 The abstract instance (`M3Instance`, `Instance.lean:119-148`; `Layout`, `Coord`, `BoundaryBlock`, `PublicLine`). *States*: the tables’ widths and log-heights, the constraint and flush polynomials with a degree bound d and its two proofs, the count columns, the boundary blocks, the stack layout with its one reading law, the public lines, the auxiliary predicate. *Status*: [\[status: built, definition\]](#) [\[status: trusted, transcribed data\]](#) — an instance is data the theorems trust: a layout may alias columns and empty the relation (probe `P2Relation`, `toyAlias`); only the toy’s relation is known inhabited on `main`. *See*: code-spine C.2, D.4, finding “the instance is data the theorems trust” (CS8); code-layer1 G.10.

[!] 2.2.3.3 The seams (`Seam.commit`, `Seam.bus`, `Seam.table`, `Seam.pub`, `Seam.flock`, `Seam.done`, `Spine/Seams.lean:171-191`). *States*: the output relation of each phase is the input relation of the next by definition: `M3Holds` of the oracle; then the linear and column claims hold, every term’s degree is within d , the public lines and `aux` hold; then column claims, lines, `aux`; then column claims and `aux`; then column and weighted claims; then `True`. *Status*: [\[status: built, definition\]](#); each seam inhabited and refuted per conjunct (probe `P3aSeams`). [\[review: wrong\]](#) on `Seam.bus`: it admits statements the deployed table sumcheck cannot serve (any number of claims, unrelated points, any table, a degree clause on the statement alone), so guards leanVM lacks are forced into the table phase (2.3.2.6). *See*: code-spine C.3, D.2, finding “the degree conjunct” (CS2); `gt-table-pub` E.1, finding “the bus seam admits statements” (TP1); `gt-bus` E.4, G10.

2.2.3.4 The claim types (`ColumnClaim`, `VirtualTerm`, `LinearClaim`, `Weight`, `WeightedClaim`, each with `.Holds`, `Seams.lean:57-124`). *States*: a column claim is an evaluation of a column at a point through the layout; a linear claim is a weighted sum of extensions of K -polynomials of one table’s rows; a weighted claim is $\sum_w W(w) \cdot q(w) = c$ with the weight’s cube values and its evaluator agreeing. *Status*: [\[status: built, definition\]](#); `Weight.onCube` carries 2^μ values no deployed verifier builds (`gt-flock-ring` 8.16). *See*: code-spine A; `gt-opening-compile` A.6.

2.2.4 The commit phase (`commitDef`, `commitComplete`, `commitSecurity`, `Compose.lean:51-71`; `Component.sendOracle`, `SendOracle.lean`). *States*: one prover message, the stack itself, becomes the oracle; the output relation is `M3Holds` of the oracle; perfectly complete; knowledge sound at error zero with the extractor that returns the message. *Feeds*: 2.2.1, 2.2.2 (first component of the bundle). *Status*: [\[status: built and proved\]](#) (probe `P3bCommit`). `ArkLib`’s own `SendSingleWitness.oracleReduction_completeness` is admitted; the local version is justified by the named form. *See*: code-spine D.6; `lib-arklib` G.8.

2.2.5 Transport of knowledge along the adaptor (`Refinement`, `Refinement.map_option_valid`, `ToArkLib/Refinement.lean:31-57`; `Extractor.Straightline.map`). *States*: validity of one extracted witness slot for `M3Rel` gives validity of its image under `witnessOf` for `SatisfiedBy` (pointwise). *Feeds*: 2.0.1. *Status*: [\[status: built and proved\]](#); the probabilistic transport, which the blueprint calls unproved and consumer-less, is three lines in the event form (probe `P6Surface`, `knowledge_transport`) and `Layer 13` is its consumer; the lemma has the polarity of the good event where the game needs the bad one (one line to flip); `Extractor.Straightline.map` cannot serve (its target witness must be in `Type`, and `EnsembleWitness` is in `Type 1`). *See*: code-spine C.7, D.7, finding “knowledge transport is three lines” (CS9); `boundary-adaptor` E.6, finding 13.

[!] 2.2.6 The adequacy of the notion (`ArkLib`’s `rbrKnowledgeSoundnessWorstCaseWith`, `Security/RoundByRound.lean:553-569`; `KnowledgeStateFunction`, :164-189). *States*: what the definition guarantees (`lib-arklib` D.1) matches the literature’s round-by-round knowledge soundness in its worst-case-per-prefix form, with one difference: the extractor may be any function, classical choice included, so the existential forms (`rbrKnowledgeSoundness`, `rbrKnowledgeSoundnessWorstCase`, hence `piop_rbrKnowledgeSoundness_exists`) prove soundness only. *Status*: [\[status: library, proved at the old pin\]](#) as a definition; [\[review: wrong\]](#) for every blueprint interface stated in the existential form (`Layer 9`’s `FlockInterface.rbrKnowledgeSoundness`, `Layer 10`’s sketch). *See*: `lib-arklib` D.2, D.4, D.6, G.1 (AK1); literature A.3 (last paragraph).

6.2.4 The phases (2.3): the fields of `Phases.Complete` and `Phases.Security`

Each phase owes three things at its two seams: a definition (`Phase.Def`: schedule, prover, verifier, per-challenge error), perfect completeness (`Phase.Complete`), and round-by-round knowledge soundness with a named extractor and state function (`Phase.Security`). Until every hole is filled, the two bundles are assumed interfaces (`bp:1419-1426`): today [\[status: trusted, assumed interface\]](#) for the bus, table, `Flock` and

opening phases.

6.2.4.1 The bus phase (2.3.1; Layer 6; `Seam.commit` → `Seam.bus`)

[!] **2.3.1.1 Definition** (`busPhase`, bp:955-963). *States*: the verifier draws $(\alpha, \beta) \in E^5$; the prover sends the two roots (R, R_c) (one root for both bus sides); a combiner λ after the roots and after every layer (the last drawn and never used); the batched radix-4 GKR over the three fingerprinted product trees (push, pull, count), odd first layer if μ_{bus} is odd, ending at one point $\zeta = (u_0, u_1, \chi_0, \dots)$ drawn entirely in the last layer; the check $R_c \neq 0$; the five boundary evaluations; the output is $B + 3$ linear claims (one per constraint at $\zeta_{\{<\tau_j\}}$ with value 0; the push, pull and count forms with the derived `rem_s`) and five column claims, everything expressible as a `BusOut` `I`. *Status*: [status: specified only] [review: wrong] — the sketch has `StmtIn := Unit`, a `BusOut` with ζ , `rem`, `pool`, α , β the spine does not have, an undefined `LeafLayout`, one combiner per layer, and a degree-5 round message where the deployed GKR sends the degree-4 cofactor with c_0 derived and no round check. *See*: gt-bus A.3, A.4, E.1, G1, G2, G8, G11; register (GB1, GB2, GB11).

2.3.1.2 Perfect completeness (`busPhase_perfectCompleteness`, bp:971). *States*: from `M3Holds`: the zerocheck claims hold at every point; the two products are one element; $R_c \neq 0$ since every count is nonzero; every layer check passes (no inverse of a challenge); the bus forms hold with the derived `rem_s` by the leaf decomposition; the degree conjunct of `Seam.bus` by `constraints_degree` and `flushes_degree`, provided $1 \leq d$ whenever there is a count column. *Status*: [status: specified only]; provable on paper. *See*: gt-bus E.2, G10.

2.3.1.3 Knowledge soundness (`busPhase_rbrKnowledgeSoundness ... (busError s)`, bp:972). *States*: one knowledge state function for the whole phase (gt-bus D.3's table): after (α, β) “the two products of q 's leaves are equal and every count is nonzero, the constraints vanish, lines and `aux` hold”; after the roots “ R is both products and R_c the count product, nonzero”; along the GKR the running claims are the true values; on the last layer's challenges the zerocheck restriction $Z(T)$ is carried coordinate by coordinate; after the boundary evaluations every output claim holds. *Status*: [status: specified only]; provable on paper, but not as the blueprint plans it (2.3.1.3.6). *See*: gt-bus D.3, E.2. Its leaves:

2.3.1.3.1 Balance from the product equality (`sideProduct_collision`, Theorem 5.1; `sideProduct_poly_eq_iff`, Lemma 5.2). *States*: unequal multisets of at most 2^μ 16-tuples give unequal polynomials in $K[A_0..A_3, X]$ (unique factorization: the factors $X - \pi_A(t)$ are monic linear, $t \mapsto \pi_A(t)$ injective), so they collide at (α, β) with probability at most $4 \cdot 2^\mu / |E|$ (total degree 4 in (α, β) ; Schwartz–Zippel). The specification leaves Lemma 5.2's proof `TODO`. *Status*: [status: specified only] (Layer 5, hole G4). *See*: gt-bus D.1, E.2; blueprint acceptance test 1; 2.5.3.

2.3.1.3.2 Nonzero counts from $R_c \neq 0$ (check B13). *States*: exact, no probability: the true count product is nonzero iff every count cell is (padding leaves are 1). *Status*: [status: specified only]. *See*: gt-bus B (B13), E.2; acceptance test 3.

2.3.1.3.3 One root, not two (check B12). *States*: structural: one stream scalar read into both slots; with two roots and no comparison any unbalanced stack passes. *Status*: [status: specified only]. *See*: gt-bus B (B12), G13; acceptance test 4.

[!] **2.3.1.3.4 The GKR's errors** (`gkrError`, bp:941-942; 2.5.4). *States*: a combiner costs $(n_{\text{side}} - 1)/|E| = 2/|E|$ (a nonzero quadratic in λ), the last one 0; a layer sumcheck round $4/|E|$ (two distinct polynomials of degree at most 4 agree at 4 points; the normalized cofactor is what the verifier holds); a combination pair $2/|E|$ ($1/|E|$ per coordinate), the binary layer's single challenge $1/|E|$; a prover message costs nothing given the layer check. *Status*: [status: specified only] [review: wrong] — the blueprint charges $5/|E|$ per round (the error of a different verifier that keeps the equality factors), omits the last combiner and the binary layer's challenge. *See*: gt-bus D.3, D.4, G1, G2, G12.

[!] **2.3.1.3.5 The recycled zerocheck point** (acceptance test 6; row *Seams* bp:333; bp:1001-1003). *States*: with the state a conjunction $G \wedge Z$, the escape “some constraint is violated on the cube but every $\check{C}_{\{j,i\}}(\zeta_{\{<\tau_j\}}) = 0$ ” costs at most $1/|E|$ per coordinate of ζ once for all constraints (a multilinear restriction becomes zero for at most one value of the next variable), and the error of a coordinate is the maximum of the GKR's and $1/|E|$, that is the GKR's: the recycled point adds nothing to `busError`. *Status*: [status: specified only] [review: wrong] — charged in the blueprint in three incompatible ways (per coordinate per constraint; $\tau_{\text{max}}/|E|$ per constraint to the (α, β) and GKR challenges, though (α, β) precede ζ ; nothing in `busError`). *See*: gt-table-pub D.3; gt-bus D.3, G6 (GB6, TP5).

[!] **2.3.1.3.6 The GKR as a separate component cannot be appended** (bp:930-937). *States:* Layer 5’s gkr has the leaf tables as its oracles while a phase’s one oracle is the stack; carrying the stack along needs ArkLib’s `liftContext` theorems, admitted at both pins, and an appended state function cannot carry the zerocheck clause that changes with the GKR’s challenges. *Status:* [status: unprovable as written] [review: unprovable] — restate the GKR over a context with leaves and riders, or prove the phase with one state function. *See:* gt-bus G3 (GB3); lib-arklib B.3 (ledger A4).

2.3.1.3.7 The leaf decomposition (`leaf_decomposition`, bp:969-970; Layer 1’s `Blocks.stack_eval_ambient_one`, `Stack.lean`:118). *States:* the extension of a one-padded leaf stack at ζ is $\Sigma_b \text{ eq}(\text{sel}_b, \zeta_{hi}) \cdot (\beta - \Sigma_i \text{ eq}(\alpha, i) \cdot \tilde{c}_{\{b,i\}}(\zeta_{lo})) + (1 + \Sigma_b \text{ eq}(\text{sel}_b, \zeta_{hi}))$, equation (2) of §5.4 in characteristic two. *Status:* [status: built and proved] for the Layer 1 identity (probe `ValuesProbe`); the phase-level statement over the leaf layout [status: specified only]. *See:* code-layer1 B.3, A.5; gt-bus A.5.

[!] **2.3.1.3.8 The leaf stacks’ layout** (the three unowned blocks first; ties by column index). *States:* the order of blocks of equal size decides offsets, selectors and the verdict; `M3Instance.tuples` lists flush tuples first, the opposite of the deployed leaf order. *Status:* [status: not specified] [review: missing]. *See:* gt-bus G7 (GB7); code-layer1 G.3.

[!] **2.3.1.3.9 Two side conditions on the instance** ($1 \leq d$ with a count column; every table with a constraint has a flush or count on the bus). *Status:* [status: not specified] [review: missing]; leanVM’s instance meets both. *See:* gt-bus G10 (GB10).

[!] **2.3.1.3.10 The order of the roots, the boundary evaluations and the linear claims.** *Status:* [status: not specified] [review: missing]. *See:* gt-bus G8 (GB8).

6.2.4.2 The table sumcheck phase (2.3.2; Layer 7; `Seam.bus` $\rightarrow$ `Seam.table`)

[!] **2.3.2.1 Definition** (`tableSumcheck`, `tableSummand`, bp:986-990). *States:* the verifier draws ξ ; constraint i of table t takes $\xi^{\{o_t+i\}}$, the three bus sides $\xi^{\{B+s\}}$ shared by every table; the target $\Sigma_s \xi^{\{B+s\}} \cdot \text{rem}_s$ is derived, never sent; $\tau_{\max}$ rounds (the maximum over the six opcode tables), highest variable first, each with three coefficients c_0, c_2, c_3 on the wire and c_1 derived from the running claim; table t joins at round $\tau_{\max} - \tau_t$ with the weight $\prod_{\{m < \tau_t\}} (1 + \zeta_m + r_m) \cdot \prod_{\{m \geq \tau_t\}} r_m$ (back-loaded padding); the final message is one value per column of every opcode table (104 for leanVM, the eighteen BLAKE2S limbs included); the final check reproduces the running claim; the claims pooled are the 104 column claims after the five received. *Status:* [status: specified only] [review: wrong] — the sketch’s types predate the spine (`StmtIn := BusOut` with `rem`, `StmtOut := BusOut  $\times$  ColumnClaims`); $\tau_{\max}$ and the final message range over every table of the instance, so with Layer 3’s column-only tables the schedule has at least 16 rounds and 110 values where leanVM has $\tau_{\max}$ (as few as 3) and 104; the oracle protocol sends four coefficients and checks the round where the wire carries three and no check (a transport lemma is owed and unstated). *See:* gt-table-pub A.1, C.2, findings “the table sumcheck’s tables are not distinguished” (TP2), “the round message” (TP4), “Layer 7’s sketch” (TP6); gt-bus G16.

2.3.2.2 The target identity (`tableSummand_target`, bp:991). *States:* the cube sum of the back-loaded, eq-weighted summand equals $\Sigma_s \xi^{\{B+s\}} \cdot \text{rem}_s$ (the zerocheck claims contribute 0). *Status:* [status: specified only]; its two Layer 1 ingredients are built: `sumCube_padHigh` (`Padding.lean`:64, the lift by $\prod_{\{k \geq \tau_j\}} X_k$ keeps the sum) and `evalMle_padHigh` (:70, the extension the verifier’s final weight uses). *See:* code-layer1 A.6, B.4, G.9 (acceptance test 7 names the wrong lemma); acceptance tests 7, 9.

[!] **2.3.2.3 Perfect completeness** (`tableSumcheck_perfectCompleteness`, bp:993). *States:* on a statement in `Seam.bus` and a stack satisfying it, the honest round polynomials pass and the final values are the true column values. *Status:* [status: unprovable as written] [review: unprovable] — against `Seam.bus` as built the deployed sumcheck is not complete (terms of one table at unrelated points need one `eq` factor per term; a term on a table taller than $\tau_{\max}$ cannot be folded); provable only for a generalized phase with the guards of 2.3.2.6, or after the seam carries the deployed shape. *See:* gt-table-pub E.1 (a) to (d), E.4.

2.3.2.4 Knowledge soundness (`tableSumcheck_rbrKnowledgeSoundness`, bp:994; state function of gt-table-pub D.2). *States:* before ξ every input claim’s discrepancy is zero; after ξ the batching polynomial $\Sigma_c \delta_c X^c$ vanishes at ξ (error $(k-1)/|E| = (B+2)/|E| = 4/|E|$, tight); after each round the running claim is the true partial sum ($3/|E|$ per round, tight: two cubics agree at three points); after the final

message every column claim holds (no error on a prover message). Back at the bus seam: a false column claim, false lines or `aux` are handed on (error 0). *Status*: [status: specified only]; the two charged terms are correct, tight and on the right challenges. *See*: `gt-table-pub` D.2, E.3.

2.3.2.5 The generic sumcheck it rests on (2.5.1). *Status*: [status: specified only] locally; ArkLib’s single-round bound [status: admitted upstream].

[!] **2.3.2.6 The guards the seam forces** (degree of every term at most $I \cdot d$; at most $B + 3$ claims; one point; the tables the sumcheck opens). *States*: ArkLib’s round-by-round notion quantifies over every input statement (`toFun_empty` is an iff), so a verifier must reject a statement outside the seam that its checks would otherwise accept: without the degree guard a true cubic claim passes the all-zero run; without the count guard $\text{err } \xi \geq (k-1)/|E|$ for every k . *Status*: [status: not specified] [review: missing] — the deployed verifiers have no such checks; smallest fix: the shape in the type (`BusOut` as the Rust’s `BusVerify`: one point, forms per side and table with the degree in the type, three totals) or a subtype for the polynomial. *See*: `gt-table-pub` E.1, finding “the bus seam admits statements” (TP1); `code-spine` C.3, finding “the degree conjunct” (CS2).

[!] **2.3.2.7 The eighteen limb claims as column claims.** *States*: a limb claim at z is the claim on `q_flock` at (`slot bits`, z , `selector`); it rests on the layout’s reading law (strided low-index selection), which no built layer supplies. *Status*: [status: not specified] [review: missing] (the strided reader, 2.5.6.3). *See*: `gt-table-pub` E.2; `code-layer1` G.1; `gt-flock-ring` 5.4, 8.2.

6.2.4.3 The public-input phase (2.3.3; Layer 8; `Seam.table` $\rightarrow$ `Seam.pub`; built)

[!] **2.3.3.1 Definition** (`publicInputPhase`, `PublicInput.lean`:376-381; `pSpec` :99, `prover` :208, `verifier` :229, `check` :136, `pooled` :127, `expectedValues` :132, `lineValue` :119, `linePoint` :65). *States*: one challenge $r \in E$; the prover sends, as one `List E`, the values at r of the lines whose value is sent; the verifier checks the message equals the expected values $(1 + r) \cdot \text{cell0} + r \cdot \text{cell1}$ and rejects otherwise; it pools one claim per line, on the line’s column at $(r, 0, \dots, 0)$, with the value it computes, after the received claims. *Status*: [status: built, definition]; noncomputable for its real error only. Faithful to §8.2’s per-limb check; [review: wrong] in one respect: the three executable verifiers (Rust, Python, the recursion guest) check one combined equation $c_0 + y \cdot c_1 = w_0 + r \cdot (w_0 + w_1)$ and pool the scalars sent, and the blueprint assigns their soundness to Layer 12, which can neither state nor detect it. *See*: `code-pubininput` B.1, A, G.2 (PI2); `gt-table-pub` F, finding “the public-input phase proves a check no executable verifier runs” (TP3).

2.3.3.2 Perfect completeness (`complete`, :300; `publicInputComplete`, :384). *States*: from the table seam, for every r , the honest values pass the check and the pool is in the public seam. *Status*: [status: built and proved] (`probe Probe0Baseline`, re-run at the new pin: three standard axioms). *See*: `code-pubininput` F; `probes-rerun` 1.0.

[!] **2.3.3.3 Knowledge soundness** (`rbr`, :350; `stateFunction`, :327; `extractor`, :317; `publicInputSecurity`, :391). *States*: worst-case round-by-round knowledge soundness at $1/|E|$ on the one challenge: outside the table seam, at most one r puts the pool in the public seam (`bad_challenge_unique`); the bound is attained (`badLine`). *Status*: [status: built and proved]. [review: wrong] on what it protects: the check on the message is not load-bearing for any theorem of the phase, because `pooled` reads the statement, not the message; a verifier that ignores the message has the same `Phase.Security` (`probe ProbeAnyCheck`, `securityG`). The requirement “a missing check leaves knowledge soundness unprovable” fails for this check; pooling the values sent, as §8.2 and the deployed verifiers do, restores it. *See*: `code-pubininput` C.3, C.4, F, G.1 (PI1).

2.3.3.3.1 The line identity (`evalMle_linePoint`, :71). *States*: the extension at $(r, 0, \dots, 0)$ is $(1 - r) \cdot q(0) + r \cdot q(1)$; the bit order (coordinate 0 is the low bit) is proved, from Layer 1’s `evalMle_append_boolVec` at slice zero. *Status*: [status: built and proved]. *See*: `code-pubininput` A, negative results.

2.3.3.3.2 Two distinct lines through k -cells meet at most once (`line_challenge_unique`, :152, `private`; `bad_challenge_unique`, :186). *States*: cells differing by $(a, c) \neq 0$ are hit when $(1 + r) \cdot a + r \cdot c = 0$, one r at most (`ofK` injective, characteristic two). *Status*: [status: built and proved].

2.3.3.3.3 The uniform sampler’s law (`probEvent_uniformSample_le_of_subsingleton`, `ToVCVio/UniformSample.lean`:44; `_of_card_le`, :35). *States*: an event with at most k witnesses has probability at most $k/|\alpha|$ under $\$^t \alpha$; uniformity is a law of `VCVio`’s `SampleableType`, so no instance can

be non-uniform. *Status:* [status: built and proved] (probe CountingBounds); redundant with VCvio’s prEvent_uniformSample_le_div_iff at the new pin. *See:* lib-others E.4, G.10; 2.5.7.2.

2.3.3.3.4 The guarded-verdict lemmas (Verifier.GuardedForm.of_probEvent_pos, ToArkLib/GuardedVerdict.lean:40; Reduction.mem_support_run_of_guarded, :73). *States:* if a guarded verifier can output a statement with a property, its check passed and its verdict has it (the last law of a knowledge state function); every outcome of a run is a prover run plus the verdict. *Status:* [status: built and proved]; restated on VCvio’s new probability API at 144c5aa with the same bounds (brief §8). *See:* code-pubinput A.

2.3.3.3.5 The verdict as an ordinary verifier (verifier_verify, :253; guarded, :273; keepOracles, materializeOutput_of_keepOracles, ToArkLib/KeepOracles.lean). *Status:* [status: built and proved].

[!] **2.3.3.4 The deployed verifier’s phase** (deployedPhase, deployedSecurity; no blueprint name). *States:* the combined check on the two words with the values sent pooled is knowledge sound at $1/|E|$: the words’ equation with true claims at two distinct challenges forces the cells and zero top limbs (accepts_two_challenges). *Status:* [status: not specified] [review: missing]; key lemma proved in a probe (ProbWordsLemma), the full Phase.Security on paper. *See:* code-pubinput C.13, G.2.

2.3.3.5 The top limb (the third public line mem_2 with cells (0, 0), sent = false). *States:* the claim on mem_2 is pooled with value 0 although no scalar is sent; every theorem of the phase and the spine holds for an instance with two lines; what a missing third line breaks is the adaptor’s satisfiedBy_witnessOf at SatisfiedBy.word0_eq, whose right-hand side has a zero top limb by the type of PublicInput. *Status:* [status: specified only] (Layer 3 lists the line; PublicLine.sent is pinned by prose only). *See:* code-pubinput D, G.3, G.4; boundary-adaptor D; acceptance test 10 (names the wrong witness).

6.2.4.4 The Flock phase (2.3.4; Layer 9; Seam.pub → Seam.flock; nothing built)

Today nothing named FlockInterface, FlockWitnessGen, limbColumns or flockError exists in Lean; the only instance built has aux := True, and on it a Flock phase that checks nothing is knowledge sound at error zero (probe NoCheckFlock). Issue #3 records “Flock F0 has not started”. Everything below is [status: trusted, assumed interface] for a reader of the master theorems today (gt-flock-ring 7.1).

[!] **2.3.4.1 Definition** (the blueprint’s FlockInterface, bp:1075-1086). *States, as deployed:* fifteen steps (gt-flock-ring 2.2): seven fixed equality coordinates, k_batch + 1 sampled ones, the univariate skip (64 values on the coset, z_skip, the interpolant with 64 assumed zeros), n_flock = 8 + k_batch zerocheck rounds with c_0 derived, v_a, v_b with v_c derived, α_lc, eight lincheck rounds with c_1 derived, the 64 slice values s_i, the one check (the lincheck terminal identity with the α³ constant-position term), six ring-switching challenges, the derived target $T = \sum_i x^i \Phi(s_i)$; the phase reads no claim of the pool and adds one weighted claim on the q_flock region. *Status:* [status: specified only] [review: wrong] — Layer 9’s interface takes the eighteen limb claims as input and emits one claim, which with leanVM’s verifier has knowledge-soundness error 1 (the limb claims stay in the pool and are opened); the spine’s slot (FlockOut with columns and weighted) is right. The circuit, wire positions, slot map, g_0, φ_8 and the floor exist only in the Rust and Python (Category B, no owner). *See:* gt-flock-ring 2, 5.1, 5.5, 8.1, 8.7 (FR1, FR7); gt-table-pub finding “Layer 9’s sketch makes the eighteen limb claims an input of Flock” (TP7).

[!] **2.3.4.2 Perfect completeness** (FlockInterface.perfectCompleteness, bp:1083). *States:* the specification’s prover (true coefficients) is accepted at every challenge; no verifier inverts a challenge-dependent value. *Status:* [status: specified only] [review: wrong] on its obstacle: acceptance test 20 files a completeness failure under flockError, a soundness error, and attributes to the protocol the inverse $(1 + r_{eq})^{-1}$ only the Rust prover takes (at r_eq = 1 that prover emits a proof its verifier rejects: an implementation defect, probability at most $(k_batch + 1)/|E|$). *See:* gt-flock-ring 4, 8.4, 8.12 (FR4, FR12); lib-arklib G.2 (AK2, the disagreement is settled gt-flock-ring’s way).

[!] **2.3.4.3 Knowledge soundness** (FlockInterface.rbrKnowledgeSoundness ... flockError, flockError_le, bp:1084-1085). *States:* if aux q fails, the weighted claim is false except with the error; $\sum \text{flockError} \leq (4 \cdot k_batch + 163)/|E| + 2^{32}/|E|$, right for a fixed committed polynomial and term by term the Rust’s parameters (gt-flock-ring 5.2). *Status:* [status: specified only] [review: wrong] in form: stated in the existential rbrKnowledgeSoundnessWorstCase (soundness only, 2.2.6) and bundled with completeness (Security extends Complete). *See:* gt-flock-ring 5.2, 8.5 (FR5); lib-arklib G.1. Its leaves:

2.3.4.3.1 The partially fixed zerocheck (Annex C.7; 03:95-100). *States*: $k_batch + 1$ sampled coordinates at $1/|E|$; the univariate skip $127/|E|$ on z_skip (two polynomials of degree below 128); $2/|E|$ per quadratic round; $1/|E|$ more on each of the last k_batch rounds for the constant position. *Status*: [status: specified only].

2.3.4.3.2 The F_2 -independence of the 128 fixed equality weights. *States*: the hypothesis of the partially fixed zerocheck holds for the pinned constants. *Status*: [status: trusted, hypothesis]; verified by a Python probe (rank 128), asserted by a Rust unit test, stated nowhere in the blueprint. *See*: gt-flock-ring 5.2, 9.3, 7.1 (row 7).

2.3.4.3.3 The lincheck (c-flock:112-133). *States*: $3/|E|$ on α_lc (degree three), $2/|E|$ per round for eight rounds; the terminal identity is the phase’s only check, and without it any q_flock passes. *Status*: [status: specified only]. *See*: gt-flock-ring 3.

[!] **2.3.4.3.4 The constant position** ($z(512, t) = 1$; the α^3 term). *States*: without it the all-zero block satisfies the homogeneous R1CS and “the limb slots compress” is false. *Status*: [status: specified only] [review: missing] — absent from the blueprint’s and the docstring’s wording of *aux*. *See*: gt-flock-ring 5.3, 8.6 (FR6).

2.3.4.3.5 Ring switching (Annex A; a-ring:104-128). *States*: the 64 slice claims at one point reduce to one weighted claim on the packed polynomial through a random F_2 -linear map built from six challenges; no message, no check; error below $2^{32}/|E|$ (a nonzero polynomial of total degree $2^{31} + 2^{15} + 2^7 + 2^3 + 2 + 1$ in $f_0..f_5$); the weight’s extension has the closed form $\sum_k c_k \prod_n (1 + r_n^{2^k} + r'_n)$. *Status*: [status: specified only]; needs “ x has 64 distinct conjugates” and “the c_k are distinct monomials” (Annex A, tested in the Rust); ArkLib’s RingSwitching/Packing is a different protocol (Diamond–Posen), admitted at both pins, and no profile turns it into leanVM’s. *See*: gt-flock-ring 6, 8.9 (FR9); lib-arklib B.3 (ledger A9).

[!] **2.3.4.3.6 The auxiliary predicate** (*I.aux*; for leanISA “Flock’s R1CS with the constant position holds of the bits packed into q_flock ”). *States*: decidable; well defined only from the Rust (the circuit, wire positions and slot map are not in the specification). *Status*: [status: specified only] [review: unprovable] as placed: Layer 9 is over an abstract *I* whose *aux* is opaque and names no column, count or relation, so the Flock phase cannot be written there (a *FlockRegion I* supplied by Layer 3 is needed). *See*: gt-flock-ring 5.3, 8.3 (FR3); code-spine finding “the instance is data the theorems trust” (CS8).

2.3.4.4 The weighted claim’s weight is MLE-friendly (*Weight.mle_eq* for the ring-switched weight). *Status*: [status: specified only], owed by #3. *See*: gt-flock-ring 8.16.

[!] **2.3.4.5 The limb columns’ slot map** (*FlockInterface.limbColumns*, bp:1077). *States*: eighteen numbers (*hash_flock.rs*:87-115) and a generic low-index selection identity. *Status*: [status: unprovable as written] [review: unprovable] — placed inside an interface that takes the instance as parameter while the instance’s layout needs it. *See*: gt-flock-ring 8.2 (FR2); boundary-adaptor finding 7; code-layer1 G.1.

2.3.4.6 The constants (ϕ_8 , $0xf7$, $0x53$, $0xb5$, g_0 , *R1CS_DIGEST*, the floor $\tau_{BLAKE2S} \geq 3$). *Status*: [status: trusted, transcribed data]; *R1CS_DIGEST* cannot be recomputed at the pin and has no owner. *See*: gt-flock-ring 5.5, 8.14 (FR14).

6.2.4.5 The opening phase (2.3.5; Layer 10; *Seam.flock* → *Seam.done*)

[!] **2.3.5.1 Definition** (*openingPhase*, bp:1112-1113). *States*, as the blueprint has it: the verifier draws λ ; a μ -round sumcheck on $W_\lambda \cdot q$; a final evaluation query of $\tilde{q}$. *As deployed*: after λ (drawn after the six ring-switching challenges, every claim value already bound; the ring-switched claim takes $\lambda^{\wedge 0}$, the pooled claims the next powers in pool order; $J = 113$) the next message is WHIR’s first round polynomial; the sumcheck rounds that reduce the weighted claim are WHIR’s own, interleaved with its levels, and the protocol ends on one equation, never on an evaluation of $\tilde{q}$ at a point. *Status*: [status: wrong as written] [review: wrong] — a *verify* built as written has μ extra round polynomials, one extra challenge and one extra scalar and rejects every Rust proof; *verify_iff_compiled* is then false. The fix that keeps the spine’s statements: give the stack the inner-product oracle interface (a query is a *Weight* μ , the answer $\sum_w W(w) \cdot q(w)$, the idealization of Definition 3.13); the opening phase is then “ λ , then one weighted query”, a one-challenge component, and WHIR realizes that one query (typechecked against ArkLib at the pin, probe *InnerProductOracle*). *See*: gt-opening-compile A.1 to A.5 (recommendation (ii)); brief §7.

2.3.5.2 Perfect completeness. *States:* by linearity of `Weight.pair`, the batched claim holds of q when every pooled claim does. *Status:* [status: specified only].

2.3.5.3 Knowledge soundness (`openingPhase_rbrKnowledgeSoundness`, bp:1114). *States:* before λ “some pooled claim is false of q ”; after λ “the batched claim is false of q ”, error $(J - 1)/|E|$ on λ (a nonzero polynomial of degree below J); the one query makes the verdict. *Status:* [status: specified only]; the blueprint’s “ $2/|E|$ per round” is the tail’s value of Theorem B.7 and belongs to no round of this phase. *See:* gt-opening-compile A.5 (table, Layer 10 row), B.6.

2.3.5.3.1 A column claim is a weighted claim (`ColumnClaim.holds_iff_weighted`, `ClaimWeights.lean`:58; `eqWeight`, `eqWeight_pair`, `Weight.pair_eq_sumCube`). *States:* a claim on column c at z is the weighted claim with weight `eq(extend c z, ·)`; the strided limb claims need the strided extension (2.5.6.3). *Status:* [status: built and proved] (probe `ValuesProbe`). *See:* code-layer1 A.7, B.7.

2.3.5.3.2 The pool’s order and the assignment of the powers of λ . *Status:* [status: specified only] (convention *Claim pool order*, bp:324); agrees with the three sources. *See:* gt-opening-compile A.2, A.6; gt-flock-ring negative results.

2.3.5.3.3 The batching component (2.5.2).

2.3.5.4 Seam.done is True. *States:* the last phase leaves nothing to check; the opening’s knowledge soundness therefore says “if the verifier accepts, every pooled claim holds of q except with the error”. *Status:* [status: built, definition] (`Seam.done = Set.univ`, probe of code-spine E.2).

6.2.5 The generic components and the layers below the phases (2.5)

6.2.5.1 Sumcheck for eq-weighted virtual polynomials (2.5.1; Layer 4, holes G1 and G2)

[!] **2.5.1.1 Definition** (`Virtual`, `sumcheck`, bp:880-893). *States:* n rounds, each a round polynomial of degree d sent as $d + 1$ coefficients and a challenge, then the claimed table values at the point; the output relation says each value is the table’s extension at the point and the formula reproduces the running claim. *Status:* [status: specified only] [review: missing] — two variants are needed and only the plain one is defined: the normalized (Gruen) variant, whose round message is the cofactor of degree d with `c_0` derived through the equality factor and no equality factor in the final check, is the one the GKR runs; the table sumcheck sends three of four coefficients with `c_1` derived. *See:* gt-bus G1 (GB1); gt-table-pub A.1 (T5, T6).

2.5.1.2 Perfect completeness (`sumcheck_perfectCompleteness`, bp:894). *Status:* [status: specified only]; `ArkLib`’s `Sumcheck.Spec.SingleRound.reduction_perfectCompleteness` is [status: admitted upstream] (through the admitted `liftContext_perfectCompleteness`), which the blueprint’s ledger row A1 omits. *See:* lib-arklib B.3 (row A1).

2.5.1.3 Round-by-round knowledge soundness (`sumcheck_rbrKnowledgeSoundness ... (fun _ => d/|F|)`, bp:895-896). *States:* if the running claim is false before a round, the prover’s message differs from the true round polynomial and the two agree at most at d points. *Status:* [status: specified only]; `ArkLib`’s `Sumcheck.Spec.SingleRound.verifier_rbrKnowledgeSoundness` and its lens instances [status: admitted upstream] at both pins (ledger A1). *See:* lib-arklib B.3, F.1.

2.5.1.3.1 Two distinct polynomials of degree at most d agree at most at d points (`Mathlib.Polynomial.card_roots'`, `Polynomial.eq_of_degree_le_of_eval_finset_eq`). *Status:* [status: library, proved at the old pin]. *See:* bp:275-277.

2.5.1.3.2 The uniform sampler’s counting bound (2.3.3.3.3).

[!] **2.5.1.3.3 The round identity as derivation** (the wire drops one coefficient; the verifier derives it from the running claim). *States:* the dropped-coefficient encoding is inverted by the running claim, so a protocol that sends all coefficients and checks the identity accepts the same transcripts as one that derives; Fiat–Shamir absorbs different data, so a transport lemma is owed. *Status:* [status: not specified] [review: missing]. *See:* gt-table-pub finding “the round message” (TP4); gt-bus B (B15).

2.5.1.4 The eq-weighted, back-loaded variant (bp:898-904). *States:* tables of different heights lifted by $\prod_{k \geq \tau_j} X_k$ (`sumCube_prodVars`, `sumCube_padHigh`), the verifier’s running weight $\prod$ of the challenges a table sat out, `eq( $\zeta_{\tau_j}$ , ·)` as an explicit factor evaluated by the verifier. *Status:* the Layer 1 identities [status: built and proved] (`Padding.lean`:47, 64, 70); the variant [status: specified only]. *See:* code-layer1 A.6, B.4.

6.2.5.2 Batching by the powers of one challenge (2.5.2; Layer 4, hole G3)

2.5.2.1 Definition and knowledge soundness (batchClaims, batchClaims_rbrKnowledgeSoundness ... (k - 1)/|F|, bp:907-911). *States*: the invariant “some claim is false”; escape needs the batching polynomial, of degree below k, to vanish at the challenge. *Feeds*: 2.3.2.4 (ξ), 2.3.5.3 (λ), the GKR’s combiners, WHIR’s per-level batching. *Status*: [status: specified only]; ArkLib #615’s gammaPowers is a pattern. *See*: gt-table-pub D.2 (the argument, worked).

6.2.5.3 Fingerprints and the grand product (2.5.3; Layer 5, hole G4)

2.5.3.1 The fingerprint and the side product (fingerprint, sideProduct, bp:923-924). *States*: $\pi_{\alpha}(t) = \sum_{i < l6} \text{eq}(\alpha, \text{bits } i) \cdot t_i$ with $\alpha \in E^4$; the product over the multiset of $\beta - \pi_{\alpha}(t)$; padding leaves are 1. *Status*: [status: specified only] [status: built, definition] for the bit order (bitProductTable, lagrangeBasis).

2.5.3.2 The product polynomial determines the multiset (sideProduct_poly_eq_iff, Lemma 5.2, bp:926-928). *States*: in $K[A_0, \dots, A_3, X]$ the products of the monic linear factors $X - \pi_A(t)$ agree iff the multisets do (unique factorization; monic factors are equal, not merely associate; $t \mapsto \pi_A(t)$ injective). *Status*: [status: specified only]; Mathlib’s UniqueFactorizationMonoid on MvPolynomial [status: library, proved at the old pin]; the specification’s proof is TODO. *See*: gt-bus E.2; bp:278-279.

2.5.3.3 The collision bound (sideProduct_collision, Theorem 5.1, bp:929-931). *States*: unequal multisets of size at most 2^μ collide at (α, β) with probability at most $4 \cdot 2^\mu / |E|$ (total degree $4N$, $N \leq 2^\mu$). *Status*: [status: specified only]; Schwartz–Zippel [status: library, proved at the old pin] (ArkLib schwartz_zippel_counting in the root namespace, prob_eval_zero_le_div a PMF statement: the blueprint’s names are wrong). *See*: lib-arklib G.6 (AK6); acceptance test 1.

6.2.5.4 The batched radix-4 GKR (2.5.4; Layer 5, holes G5 and G6)

[!] 2.5.4.1 Definition (ProductTree, gkr_nside μ , gkrError, bp:933-944). *States*: from the roots to the leaf claims: radix 4 from the root down, one radix-2 layer first if μ is odd; a combiner after the roots and after every layer (the last unused); per layer $k = \mu - \text{layer}$ normalized sumcheck rounds of degree 4, twelve children, the layer check $\text{claim} = \sum_s \lambda^s \prod \text{children}[s]$ with no equality factor, two combination challenges; the three trees share every challenge and end at one ζ . *Status*: [status: specified only] [review: wrong] (degree 5, one combiner per layer, no binary-layer error; 2.3.1.3.4). *See*: gt-bus A.4, D.4, G1, G2, G12.

2.5.4.2 Perfect completeness (gkr_perfectCompleteness, bp:942). *Status*: [status: specified only]; an honest GKR prover written from the specification is accepted by the pinned Python verifier (gt-bus probe H.1).

2.5.4.3 Round-by-round knowledge soundness (gkr_rbrKnowledgeSoundness ... (gkrError_nside μ), bp:943-944). *States*: per challenge as in 2.3.1.3.4; the layer check turns a false running claim into a false child. *Status*: [status: specified only]; the specification states no error for the GKR (gt-bus G17). Its leaves: the sumcheck of 2.5.1 in its normalized variant; the batching of 2.5.2 for the combiners; the bilinear interpolation of the four children ($1/|E|$ per coordinate).

6.2.5.5 The ring-switching and WHIR components (2.5.5)

Ring switching is 2.3.4.3.5 (inside the Flock phase, #3’s); WHIR is 2.1.2.4 (Layer 11, the compilation). Neither is a phase of the oracle protocol; under recommendation (ii) the opening phase is the batching component alone.

6.2.5.6 Layer 1 (2.5.6): hypercube tables, stacking, padding, claim weights, the fixed columns (built)

Every theorem below is [status: built and proved] (seventeen declarations checked with #print axioms by code-layer1, three standard axioms; the sketch matches the code declaration by declaration). Today an auditor of the master theorems reads none of them; seventeen definitions become trusted when the adaptor is built, about eight more with the executable verifier (code-layer1 E).

2.5.6.1 The generic identities (ToCompPoly/; over any commutative ring). *Status:* [status: built and proved] for each of the following.

- evalMle_eq_sumCube_hadamard, sumCube_lagrangeBasis (partition of unity), evalMle_replicate, evalMle_lagrangeBasis (the eq kernel; duplicates CompPoly’s eqTilde_eq_prod) — Multilinear. lean:91, 125, 131, BitProductTable.lean:156.
- evalMle_append_boolVec (Multilinear.lean:342): the extension at (z, bits of j) is the extension of slice j at z (high-index selection; the low-index, strided selection is absent: 2.5.6.3). evalMle_placeSlice, sumCube_placeSlice (:371, 381).
- evalMle_bitProductTable, evalMle_powersTable (BitProductTable.lean:58, 144): a table that factors over the index bits evaluates as a product.
- Blocks.pow_size_dvd_offset (alignment from the order), Blocks.stack_eval (the one selector fact: $\tilde{q}(z, \text{sel}_b) = \tilde{P}_b(z)$ for the honest stack), Blocks.unstack_eval₂ (the same for a table the prover chose), Blocks.stack_eval_ambient (equation (2) of §5.4 for any padding) — Stacking. lean:109, 339, 330, AmbientStacking.lean:134.

2.5.6.2 The leanVM half ([status: built and proved] for each). Blocks.readColumn_eval, Blocks.layout (the aligned blocks as the spine’s Layout), Layout.comap (renaming; accepts a non-injective renaming, so a layout built with it can alias), Blocks.stack_eval_ambient_one (padding 1 over E, the leaf decomposition), sumCube_padHigh, evalMle_padHigh, eqWeight, eqWeight_pair, ColumnClaim.holds_iff_weighted, idxColumn_eval, idxColumnEval_eq (the index column g^i , coordinate k carrying g^{2^k} , §6.5), bytecodeColumn_answer_boolVec, bytecodeColumn_eval (slot s of instruction z at (z, s), cell for cell the Rust encoder’s table on a sixteen-instruction program). *See:* code-layer1 A.5 to A.9, B.1 to B.7.

[!] 2.5.6.3 The strided reader of the eighteen BLAKE2S limb columns (sliceLow, evalMle_boolVec_append, Blocks.stridedLayout, Layout.pieces; no blueprint name). *States:* a limb column is read from q_flock at the low eight coordinates frozen to the slot’s bits; the selection lemma is twenty lines (probe StridedProbe, proved). *Status:* [status: not specified] [review: missing] — exists nowhere, assigned to no layer; leanIsaInstance cannot be built without it. *See:* code-layer1 G.1 (L1-1); gt-flock-ring 8.2.

[!] 2.5.6.4 The order of equal-size blocks. *States:* the six shared columns first, then every table’s columns in table order (witness.rs:67-79, cpu/layout.rs:13-49); Blocks is the sorted sizes only, and leanVM’s order and “tables first” give different offsets with the same Blocks. *Status:* [status: trusted, transcribed data] [review: missing] — pinned by no statement or test before the compiled verifier. *See:* code-layer1 G.3 (L1-3), B.2 (the 92 offsets of the pinned Python layout reproduced by Blocks.offset).

6.2.5.7 Layer 0 (2.5.7): the fields, the samplers, the oracle interface (built)

2.5.7.1 The fields and their cardinality (K, E, card_E : Fintype.card E = 2^{192} , Field.lean:96; CompPoly’s card_ext3). *States:* $K = \text{GF}(2)[x]/(x^{64}+x^4+x^3+x+1)$ as 64-bit words, $E = K[y]/(y^3+y+1)$ as limb triples, bit for bit leanVM’s (probe FieldFidelity: eleven Rust reference products reproduced). *Status:* [status: library, proved at the old pin] (irreducibility by a Rabin certificate, no native_decide); at the new pin BF64 is a structure and $(2 : K) = 0$. *See:* lib-others B, G.8.

2.5.7.2 Uniform challenges (instSampleableTypeE, Field.lean:93; VCVio’s SampleableType). *States:* uniformity is a law of the class, so the E sampler (three limbs through a bijection) is uniform by proof; $\Pr[= x \mid \$^t E] = 2^{-192}$ (probe Layer0); the K sampler has no consumer. *Status:* [status: built and proved]. *See:* lib-others E.1, G.2.

[!] 2.5.7.3 The evaluation oracle (evalOracle, evalOracle_answer, Field.lean:102-113). *States:* a query is a point of E^n , the answer $\tilde{q}(r) = \sum_i \text{ofK}(q_i) \cdot \text{eq}(r, i)$ in the specification’s little-endian order. *Status:* [status: built and proved]; [review: wrong] as the interface of the stack: the compilation needs the inner-product interface (2.3.5.1), of which evaluation is the case $W = \text{eq}(r, \cdot)$. *See:* lib-others E.2; gt-opening-compile A.5.

2.5.7.4 The scalar oracle interfaces (instOracleInterfaceE, instOracleInterfaceListE, Field.lean:115-118). *States:* a scalar or list message is read whole. *Status:* [status: built, definition] [status: trusted, transcribed data], unlisted in the blueprint’s interface list. *See:* lib-others G.4 (LO4).

6.2.5.8 Layer 2 (2.5.8): Clean components as polynomials (hole I1; not built)

[!] **2.5.8.1 Expressions as polynomials** (`Expression.toMvPolynomial`, `eval_toMvPolynomial`, `degreeBound`, `totalDegree_le_degreeBound`, bp:759-765). *States*: a Clean expression evaluates as the polynomial with $\text{var } i \mapsto X \ i$; the syntactic degree bounds the total degree. *Status*: [status: specified only] [review: unprovable] as placed: Layer 2 produces Mathlib `MvPolynomials` while the instance holds `CompPoly CMvPolynomials` and the conversion is noncomputable at the pin, which would make `M3Holds (leanIsaInstance ...)` undecidable by evaluation and `verify` noncomputable; a direct translation is twelve computable lines (probe `PolyBridge`). *See*: boundary-adaptor E.5, finding 8.

2.5.8.2 The polynomial view of a component (`Component.toM3`, `Ensemble.toM3`, `toM3_constraints_iff`, `toM3_flushes_eq`, `vars_lt_width`, bp:767-780). *States*: the constraints of a row vanish iff Clean's `ConstraintsHold` does; the multiset of evaluated flush tuples is the interactions mapped to 16-tuples, with the separator and direction given per channel. *Feeds*: 2.6.2 (the constraint and balance clauses). *Status*: [status: specified only]; Clean has no polynomial, degree, height or padding notion. *See*: boundary-adaptor A.4; blueprint bp:781-786.

[!] **2.5.8.3 The count columns' derivation.** *States*: which cells are the pulls' count coordinates (`count_columns()` per table, `layout.rs`:412-414), which `Component.toM3` cannot derive from the separator and direction alone. *Status*: [status: not specified] [review: missing]. *See*: boundary-adaptor finding 11.

6.2.6 The adaptor and what it needs of leanISA and of Flock (2.6; Layer 3, hole I2; not built)

The only place the proof system meets leanISA. Nothing of it exists in Lean at b435631 (`M3Holds`, `M3Rel` and `Refinement` are built; `leanIsaInstance`, `stackOf`, `witnessOf` and the four theorems are not).

[!] **2.6.1 The leanISA instance** (`leanIsaInstance prog s`, `Sizes`, `Sizes.Admissible`, `leanIsaInstance_degree`, `leanIsaInstance_flush_degree`, `leanIsaInstance_fits`, bp:794-808). *States*: the polynomial view of the eight tables with leanISA's separators and directions, $d := 2$, three public lines on `mem_0`, `mem_1`, `mem_2`, the six shared columns as constraint-free tables, the boundary blocks with the index and bytecode columns as known coordinates, the stack layout of `witness.rs`, `cpu/layout.rs`, `leaf.rs`, the strided limbs, the auxiliary predicate. *Feeds*: everything above through I. *Status*: [status: specified only] [status: trusted, transcribed data]; [review: wrong] on four points: it is not `Ensemble.toM3` of the eight tables (the memory and bytecode seed/finalize blocks and the state boundary must be boundary blocks with known program data, or the program would be committed and no adaptor theorem provable); `leanIsaInstance_fits` mentions a field `Layout` does not have (the fit belongs to `Blocks`, before the instance); its layout and `aux` depend on a `FlockInterface I` that takes the instance as parameter; it must be reducible for the tests' `Decidable` search. *See*: boundary-adaptor A.5, E.4, findings 1, 7, 12; code-layer1 G.2; gt-flock-ring 8.2, 8.3.

2.6.2 The soundness bridge (`satisfiedBy_witnessOf (hs : s.Admissible prog) (h : M3Holds (leanIsaInstance prog s) input q) : SatisfiedBy prog input (witnessOf prog s q)`, bp:815-816). *States*: the witness rebuilt from a stack satisfying `M3Holds` satisfies leanISA's thirteen-conjunct relation; carries knowledge from `M3Holds` to `SatisfiedBy` at the same error (pointwise, 2.2.5). *Feeds*: 2.0.1. *Status*: [status: specified only]; can work as sketched for the reason of 2.6.1. *See*: boundary-adaptor A.4, A.5, E.1. Conject by conjunct:

2.6.2.1 Clean's constraints from `ConstraintsVanish` through `toM3_constraints_iff` (2.5.8.2). *Status*: [status: specified only].

[!] **2.6.2.2 The three BalancedPairs (counted in $\mathbb{N}$)** from `Balanced` (a `List.Perm` of 16-tuples) through `toM3_flushes_eq` and a bridge for the boundary blocks against the two Clean blocks and the verifier component. *Status*: [status: not specified] [review: missing] for the boundary bridge; leanISA's `BalancedPair` replaces Clean's field-summed balance, unsatisfiable over K . *See*: boundary-adaptor finding 2; lib-others C.5.

2.6.2.3 CountsNonzero (both relations quantify over the same cells). *Status*: [status: specified only]; the count columns' derivation is 2.5.8.3.

2.6.2.4 The public words (`word0_eq`, `word1_eq`) from `PublicLinesHold` on the three lines; the top limb is zero by the type of `PublicInput` and needs the third line (2.3.3.5); $\kappa < 64$ for `MemImage.read_`

gpow from admissibility. *Status*: [status: specified only]. *See*: boundary-adaptor D; code-pubinput D.

[1] **2.6.2.5 Blake2sRowsValid** from `aux` through `#3`'s lemma “the R1CS with the constant position holds of `q_flock` $\Rightarrow$ the eighteen limb slots compress” (the functional correctness of a circuit with 14,720 AND gates: the forward walk, two adders, leanISA's compress). *Status*: [status: trusted, hypothesis] [review: missing] — nothing carries it: the sketched signature has no argument for it, `FlockInterface` has no such field, and `aux` needs `#3`'s R1CS to be defined. *See*: boundary-adaptor C (row “the R1CS holds $\Rightarrow$ the limb slots compress”), finding 7; gt-flock-ring 5.3, 7.1 (row 2).

2.6.2.6 Caps from `s.Admissible prog`: a hypothesis, since `M3Holds` leaves the caps out and `imageOf` truncates above `maxLogMem`. *Status*: [status: trusted, hypothesis], discharged by the verifier's setup check (2.7). `admissible_iff_caps` [status: unprovable as written] (a free `w`; false in either reading; `Sizes.ofWitness` cannot produce the rate). *See*: boundary-adaptor E.3, finding 5; gt-bus G4.

2.6.2.7 The three fixed-column facts and decodability (from the instance's known columns and the type of `Program`). *Status*: [status: specified only]; “statement binding” needs the injectivity of `prog` $\mapsto$ `bytecodeColumn prog`, unstated. *See*: boundary-adaptor B.2, F.

2.6.3 The completeness bridge (`m3Holds_stackOf (h : SatisfiedBy prog input w) (hs : Sizes.ofWitness w = some s) : M3Holds (leanIsaInstance prog s) input (stackOf gen w hs)`, bp:817-818). *States*: the stack of a satisfying witness (with the `Flock` wires generated from the limbs) satisfies `M3Holds`; the non-vacuity of the whole protocol for leanISA. *Feeds*: 2.0.2. *Status*: [status: specified only]; needs `Sizes.ofWitness` (ill-defined while `Sizes` holds `logInvRate`) and `FlockWitnessGen` with its lemma “the generated wires satisfy the R1CS when the limbs compress” [status: trusted, assumed interface] (`#3`'s). *See*: boundary-adaptor A.6, C; gt-flock-ring 7.1 (row 3).

2.6.4 The round trip (`witnessOf_stackOf ... = w`, bp:819). *States*: reading the stack back gives the witness on the committed fields. *Status*: [status: unprovable as written] — not an equality any `EnsembleWitness` satisfies (`ProverData` is a function, `Table.width` is free); serves nothing on the chain (a non-vacuity test). *See*: boundary-adaptor E.2, finding 10, F.

2.6.5 witnessOf itself (bp:812-814). *States*: rebuilds the tables from the columns, the limbs from their slots, the interactions from the components, the image from the memory columns, the program from `prog`; total and computable. *Status*: [status: specified only]; has no input argument. *See*: boundary-adaptor finding 11.

2.6.6 leanISA's constraint soundness (`constraintSoundness (hwf : WellFormedBytecode prog) (h : SatisfiedBy prog input w) :  $\exists t$ , AssignmentRepresents w t  $\wedge$  ValidExecution prog input t`, leanisa-blueprint.md:1156-1177; T1-S). *States*: every satisfying assignment of a well-formed program represents a valid execution. *Feeds*: 2.0.1. *Status*: [status: specified only] — a block comment, blocked on Clean's N-counted balance (issue #16, Clean #452/#464). *See*: boundary-adaptor C; lib-others C.5 to C.7.

2.6.7 leanISA's constraint completeness (`constraintCompleteness (hwf) (h : ValidExecution prog input t) :  $\exists w$ , SatisfiedBy prog input w  $\wedge$  AssignmentRepresents w t`; T1-C). *Feeds*: 2.0.2. *Status*: [status: specified only]; needs a resource hypothesis of its own (out of scope). *See*: boundary-adaptor note 18.

2.6.8 The witness generator (T2, `witnessGen_correct`, arch:230-236). *States*: the executable witness generation produces a satisfying, representing assignment. *Feeds*: 2.0.2 as docs/architecture.md states it. *Status*: [status: not specified] in the blueprint (out of scope, bp:147-148); T4's completeness composes T1-C instead. *See*: boundary-adaptor F; docs-debt (DD19).

6.2.7 The hypotheses (2.7): what a verifier check or a condition on public data discharges

Every hypothesis on the two chains, with what discharges it (from boundary-adaptor C, extended by the opening and `Flock` dossiers). Kinds: a condition on public data (P), a property the verifier checks (V), a theorem another roadmap or library owes (O), nothing (N).

Table 6.2: The hypotheses on the two chains and what discharges them. Kinds: a condition on public data (P), a property the verifier checks (V), a theorem another roadmap or library owes (O), nothing (N).

Hypothesis	Appears in	Kind	Discharged by	Status
s.Admissible prog (caps, τ , $\text{BLAKE2S} \geq 3$, $15 \leq \mu \leq 28$, $1 \leq \rho \leq 4$)	2.6.2, 2.1.1	V	read_public (cpu/mod.rs: 130-178), reproduced by verify; supplied by verify_iff_compiled's $\exists s$	consistent once Admissible carries the two windows (2.1.1.4)
the top limb of each public word is zero	2.6.2.4	V in the Rust (cpu/mod.rs:141-143); by type in Lean	the parser of PublicInput; defence in depth through the third line	consistent; the parsing obligation undocumented (boundary-adaptor finding 15)
the bytecode is decodable, of power-of-two length $\leq 2^{32}$	2.6.2.7	V in the Rust; by type in Lean	the loader of Program	consistent
WellFormedBytecode prog (sentinel not a JUMP; fill blocks)	2.6.6, 2.6.7	P (decidable; no verifier checks it; the compiler emits both)	the guest owner (T3)	N in the blueprint: baseVerifier_ extractsExecution has no such hypothesis, baseProver_complete has HasFillBlocks only
Sizes.ofWitness w = some s (power-of-two heights)	2.6.3	O (leanISA: Caps. heights, seed_rows, bytecode_rows)	SatisfiedBy	provable; ill-defined while Sizes holds logInvRate
a resource bound on the trace (every table $\leq 2^{32}$ rows; the stack $\leq 2^{28}$)	2.0.2, 2.6.7	N	nothing: ValidExecution bounds nothing but $\kappa \leq 32$	N: baseProver_complete is false without it
a witness constructed from the trace	2.0.2	O (T2, out of scope)	constraintCompleteness gives $\exists w$ only	N as written
the honest grind succeeds	2.0.2, 2.1.4	N (a search)	running the prover	N: prove must be partial
FlockWitnessGen and its lemma	2.6.3	O (#3)	an explicit argument	assumed interface
“the R1CS with the constant position $\Rightarrow$ the limb slots compress”	2.6.2.5	O (#3)	nothing names a carrier	N until a carrier is named
the F_2-independence of the 128 fixed equality weights	2.3.4.3.2	P (a rank computation on constants)	a Rust unit test; a Python probe	not in any theorem
Phases.Complete, Phases.Security of leanISAInstance prog s	2.2.1, 2.2.2	O (holes P1 to P8; the public-input phase built)	the phases	assumed interface until every hole is filled
a bound on piopError P	2.2.2.5	O (this roadmap)	nothing in the spine	N today
every front phase's verifier makes no oracle query	2.1.2.3.3	O (this roadmap)	nothing; the spine's types allow queries	N (finding 2)
FiatShamirSecurity, BcsSecurity, McaJohnson	2.1.2	O (ArkLib; the literature)	explicit arguments	assumed interfaces; the first two as stated are false or empty
the list-binding compilation, with L_0	2.1.2.3	O (this roadmap)	nothing	N
BLAKE2s's compression function is a random oracle; BLAKE2s-256 is collision resistant	2.1.5	N (a heuristic)	nothing	N: must be written down as the assumption it is
the grinding check at every query round (17 bits)	2.1.2.1.3, 2.1.2.4.7	V (fs/lib.rs:165-174)	verify	no planned theorem or mutation makes it load-bearing (literature LT13)
the canonical encodings (upper limbs, root halves) and full consumption of the stream	2.1.1.4	V	verify	none protects soundness; verify without them accepts streams the Rust rejects
R1CS_DIGEST in the seed	2.3.4.6	V (a constant compared)	a Rust-produced proof accepted by verify	no theorem; not recomputable at the pin
the round-by-round $\Rightarrow$ plain implication	2.1.2.7	O (ArkLib, admitted)	“stated once the implication lands”	consistent; forgets the named extractor
the count columns are exactly the pulls' count coordinates	2.6.2.3	O (Layer 2/3, Category B)	a transcription of count_columns()	unspecified

(continued)

Hypothesis	Appears in	Kind	Discharged by	Status
the order of equal-size blocks; the leaf stacks' order	2.5.6.4, 2.3.1.3.8	O (Category B)	nothing before the fixture	unspecified

6.3 The error, composed

6.3.1 The interactive error of the oracle protocol, challenge by challenge

Every entry of table 6.3 is a multiple of $1/|\mathbb{E}|$, $|\mathbb{E}| = 2^{192}$. “Blueprint” is what the blueprint charges at **b435631**; “tight” is what the review finds to be the exact worst-case escape probability of that challenge for the deployed verifier (the sibling dossier named in the last column does the argument). Counts are at the largest admissible instance under the stacking window ($\mu_{\text{bus}} \leq 28$, $\tau_{\text{max}} \leq 24$, $B = 2$, $k_{\text{batch}} = \tau_{\text{BLAKE2S}} \leq 24$, $J = 113$).

Table 6.3: The interactive error of the oracle protocol, challenge by challenge, in multiples of $1/|\mathbb{E}|$, $|\mathbb{E}| = 2^{192}$.

Phase	Challenge	Count	Blueprint charges	Review finds tight	Note	See
commit	none	0	0	0	the extractor reads the message	code-spine D.6
bus	$(\alpha, \beta) \in \mathbb{E}^5$	1	$4 \cdot 2^{\mu_{\text{bus}}}$	$4N$, $N \leq 2^{\mu_{\text{bus}}}$ (within a factor 2)	Theorem 5.1 through Lemma 5.2	gt-bus D.3, D.4
bus	combiner λ	$\lceil \mu_{\text{bus}}/2 \rceil + 1$	$n_{\text{side}} - 1 = 2$ each, one per layer	2 each; the last one 0	the blueprint has no last combiner	gt-bus G2
bus	layer sumcheck round χ_t	$\sum_{\text{layers}} k \approx \mu_{\text{bus}}^2/4$	5	4 (degree-4 cofactor, c_0 derived)	5 is another verifier's	gt-bus G1, D.3
bus	combination pair (u_0, u_1)	$\lfloor \mu_{\text{bus}}/2 \rfloor$	2 per pair	2 per pair (1 each); binary layer 1	the binary layer's challenge is unassigned	gt-bus G12
bus	the recycled zerocheck point $(\zeta, \text{drawn in the last layer})$	0 extra	three incompatible charges (1 per coordinate per constraint; τ_{max} per constraint to (α, β) and the GKR; nothing)	adds nothing: $\max(\varepsilon_{\text{GKR}}, 1) = \varepsilon_{\text{GKR}}$ per coordinate; τ_{max} once if separated	one conjunct of one state function	gt-table-pub D.3; gt-bus G6
bus	prover messages (roots, children, boundary evaluations)	—	0	0		
bus, sum			$4 \cdot 2^{\mu_{\text{bus}}} + 5\mu_{\text{bus}}^2/4 - \mu_{\text{bus}}/2 + 2$ (plus the zerocheck charge)	$4 \cdot 2^{\mu_{\text{bus}}} + \mu_{\text{bus}}^2$	at $\mu_{\text{bus}} = 28$: about 2^{30}	gt-bus D.3
table	ξ	1	$B + 2 = 4$	$k - 1 = 4$ for $k = B + 3$ claims (tight)	unbounded over <code>Seam.bus</code> without a guard on the number of claims	gt-table-pub D.2, E.1 (b)
table	round r_m	$\tau_{\text{max}} \leq 24$	3	3 (tight)		gt-table-pub D.2
table	final message	—	0	0		
table, sum			$4 + 3\tau_{\text{max}} \leq 76$	the same		
public input	r	1	1	1 (tight, attained)	built and proved; the deployed combined check also 1	code-pubinput F; gt-table-pub D.4
Flock	sampled equality coordinates $r_7, \dots$	$k_{\text{batch}} + 1$	inside ($4k_{\text{batch}} + 163$)	1 each		gt-flock-ring 5.2
Flock	z_{skip}	1	inside	127	two polynomials of degree below 128	
Flock	zerocheck round χ_i	$n_{\text{flock}} = 8 + k_{\text{batch}}$	inside	2 each, plus 1 more on the last k_{batch} (the constant position)		
Flock	α_{lc}	1	inside	3		
Flock	lincheck round	8	inside	2 each		
Flock	ring switching $f_0, \dots, f_5$	6	2^{32} (as one term)	$2^{31}, 2^{15}, 2^7, 8, 2, 1$	acceptance test 23 leaves this term out (2^{-160} dominates its 2^{-183} by 2^{23})	gt-flock-ring 5.2, 8.8

(continued)

Phase	Challenge	Count	Blueprint charges	Review finds tight	Note	See
Flock, sum			$4k_{\text{batch}} + 163 + 2^{32}$	$4k_{\text{batch}} + 163 + 2^{31} + 2^{15} + 2^7 + 11 < 2^{32}$	right for a fixed committed polynomial	gt-flock-ring 5.2
opening	λ	1	$J - 1 = 112$	112 (tight)		gt-opening-compile A.5
opening	“sumcheck $W_\lambda \cdot q$ rounds”	μ in the blueprint, 0 deployed	2 per round	no such rounds: they are WHIR’s and carry $2L_i + 2^{\ell_i - j}a_i$	the blueprint’s transcript is not leanVM’s	gt-opening-compile A.3, A.4
oracle protocol, sum			piopError_le: $\leq 2^{40} + \text{flockError}$	about $1.25 \cdot 2^{32}$, that is $2^{-159.7}$, dominated by ring switching	true under the stacking window (margin about 2^8); false at the per-log caps ($\mu_{\text{bus}} = 38$ gives 2^{40} on (α, β) alone)	gt-opening-compile F.1; gt-bus D.4

What $\Sigma \text{piopError}$ measures: the interactive error of the ideal-oracle protocol, of no consequence for the deployed 128-bit claim, which the query rounds of WHIR set.

6.3.2 The compiled protocol’s per-challenge errors

Table 6.4: The compiled protocol’s per-challenge errors (Theorem B.7, per level i).

Challenge	Error	At the deployed parameters ($\mu = 28$, $\rho = 1/2$)	See
level batching λ_i	$(J_i - 1)L_i/ \mathbb{E} $	level 0: $J_0 = 113$, $L_0 \leq 110$ to 396 over the sizes	gt-opening-compile F.2
fold round j of level i	$2L_i/ \mathbb{E} + 2^{\ell_i-j} \cdot a_i/ \mathbb{E} $ (a_i from MCA, Theorem 4.6, with the $2^{\ell-1}$ row union)	128.2 to 146 bits	
out-of-domain sample	$\binom{L_i}{2} \cdot \mu_i/ \mathbb{E} $	155 to 180 bits	
query positions (after 17 bits of proof of work)	$(1 - \gamma_i)^{t_i}$	111 bits, plus 17 by grinding	
final batching	$t_{r-1}/ \mathbb{E} $		
tail	$2/ \mathbb{E} $		
every front challenge, after compilation	$\mathbf{L}_0 \cdot \boldsymbol{\varepsilon}_i$ (the list-binding compilation, 2.1.2.3)	$2^{-160+8.6}$ at worst; benign, structurally load-bearing	gt-opening-compile E.3, G; literature A.3

The Rust’s own accounting multiplies only the ring-switching degree by the list size (`whir_config.rs:540-567`); the bus’s $4 \cdot 2^{\mu_{\text{bus}}}$ is not unioned (still clears 128 bits at these parameters).

6.3.3 The non-interactive error

As the blueprint writes it (`bp:1224`, `bp:1234-1235`):

$$niError = Q \cdot \max_i \varepsilon_i$$

“plus the grinding-adjusted WHIR terms”.

What the review finds it must be, per random-oracle query, for a t -query adversary against `verify^H` with k rounds:

$$\begin{aligned}
 niError(t) = (t + k) \cdot \max \Big(& L_0 \cdot \max_{\text{front}} \varepsilon_i, \max_{\text{fold}} \frac{2L_i + 2^{\ell_i-j} a_i}{|\mathbb{E}|}, \max_{\text{query}} 2^{-b_i} (1 - \gamma_i)^{t_i}, \\
 & \text{the OOD, batching, final and tail terms} \Big) \\
 & + \frac{3.5 t^2}{2^{256}} \quad \text{(the hash chain and the Merkle trees)} \\
 & + \text{the program-hash collision term,}
 \end{aligned}$$

uniform in the announced sizes s or maximized over them, with the multiplier $(t + k)$ of [CY24] Theorems 31.2.1/31.3.1 (or Q of [BGKTTZ23] Theorem 3.15), the list factor L_0 on every challenge of the oracle protocol (they precede the point where the claims pin one list member), the grinding bits $b_i = 17$ at every query round as a per-round multiplicative factor (an assumption no published theorem covers), and the hash term that caps a 256-bit digest at the edge of 128 bits of average-case security.

The terms the blueprint’s formula lacks:

- the list-size factor L_0 (literature LT2; gt-flock-ring FR13);
- the hash-collision term (LT3);
- a model of grinding (LT1: without it the formula certifies about 111 bits where leanVM claims 128);
- the family composition over the prover’s sizes (boundary-adaptor finding 6);
- the program-hash collision (gt-opening-compile E.1).

The maximum is set by the query rounds ($2^{-111} \cdot 2^{-17} = 2^{-128}$) and the folds ($2^{-128.2}$), not by anything in the oracle protocol; the theorem is classical (no quantum random-oracle model, LT14).

Which theorems carry which term. `piop_rbrKnowledgeSoundness` carries the front’s ε_i per challenge (2.2.2); the list-binding compilation multiplies them by L_0 and adds Theorem B.7’s (2.1.2.3, 2.1.2.4); the round-by-round to state-restoration step gives the $(t + k)$ multiplier (2.1.2.1.1); the Merkle interface adds the hash term (2.1.2.2); the chain lemma adds the encoding collision (2.1.2.1.2); the grinding model turns b_i into 2^{-b_i} (2.1.2.1.3); the family composition handles s (2.1.2.1.6). Of these seven steps, one is built (the first), one is admitted upstream in a weaker form (the third), and five are stated nowhere. Table 6.5 sets the same steps against the arrows of the dossier’s summary (section 6.1).

Table 6.5: The arrows that carry the error from the phases to the deployed claim, as the dossier’s summary lists them, with the term each carries.

Arrow	Term it carries	Node	State (the summary’s word)
the spine’s per-challenge composition (<code>piop_rbrKnowledgeSoundness</code>)	the front’s ε_i per challenge	2.2.2	built
the list-binding compilation Theorem B.7	multiplies the front’s ε_i by L_0	2.1.2.3	not stated
	adds WHIR’s per-challenge errors (table 6.4)	2.1.2.4	specified
round-by-round to state-restoration	the multiplier $(t + k)$	2.1.2.1.1	commented out upstream
the Merkle interface	the hash term	2.1.2.2	empty as stated
the chain lemma	the encoding collision	2.1.2.1.2	not stated
the grinding model	turns b_i into 2^{-b_i}	2.1.2.1.3	no theorem exists
the family composition over the prover’s sizes	handles the sizes s	2.1.2.1.6	not stated

6.4 The two existing outlines, validated

6.4.1 The obligation map of `docs/architecture.md` (at `b435631`)

The map has three parts that touch the proof system: the target ladder’s T4 (`arch:261-275`), the coverage diagram’s S4 rows (“Prover ... S4: prover implementation correspondence”, “Verifier ... S4: soundness extracts constraints”, `arch:96-131`) and the “Proof system” list of the layer ownership (the “Verification coverage” list at `arch:446-462`, checked by boundary-adaptor F). Set against the tree:

Table 6.6: The obligation map of `docs/architecture.md` set against the tree.

The map says	In the tree	Verdict
T4 first proves “BaseVerifier.Accepts ... $\rightarrow$ except with probability <code>baseError</code> , $\exists$ assignment, <code>Constraints.SatisfiedBy ...</code> ”	2.0.1, 2.1.2, 2.6.2	has the right decomposition (verifier, extraction into the arithmetization’s relation, then T1-S); gets wrong the shape: a closed Boolean with a probability attached (the same defect the blueprint transcribes; <code>gt-opening-compile E.1</code>)
“compose it with T1-S”	2.6.6	lacks the hypothesis <code>WellFormedBytecode</code> of T1-S in the composed statement (<code>arch:224-228</code> states it for T1 and drops it at T4)
“must expose the component soundness/knowledge-soundness bounds, Fiat-Shamir or random-oracle model, commitment and hash assumptions, transcript/serialization agreement, and executable-verifier refinement”	2.2.2.5, 2.1.2.1, 2.1.2.2, 2.1.1.2, 2.1.1	has every heading; lacks the list-binding compilation (2.1.2.3), the grinding model (2.1.2.1.3), the family composition over sizes (2.1.2.1.6), and it files the random-oracle instantiation under “must expose” rather than under “assumed” (2.1.5)
“Its completeness dual composes T2 with the honest prover and records any failure/resource conditions”	2.0.2, 2.6.8	right , and the blueprint diverges from it: Layer 13 composes T1-C (an existence theorem) instead of T2 and records no resource condition (boundary-adaptor finding 4; <code>docs-debt DD19</code>)
“T4 — base proof extraction/completeness: proof-system components, T1, and T2” (<code>arch:66</code>)		as above

(continued)

The map says	In the tree	Verdict
“Relate the executable verifier to the protocol specification, including serialization, transcript order, domain separation, challenge derivation, statement binding, and rejection behavior” (arch: 453-455)	2.1.1, 2.1.1.2 to 2.1.1.4	has the headings; “domain separation” is absent from leanVM’s trees (role separation replaces it, 2.1.2.2.2); “statement binding” needs the injectivity of <code>prog ↦ bytecodeColumn prog</code> and the program-hash collision term, both unstated
“Prove completeness of the abstract prover and test or verify completeness of the executable prover” (arch: 456-457)	2.2.1, 2.1.6	has ; the executable (Rust) prover’s completeness is evidence, not a theorem, as the map allows; the Rust Flock prover is in fact not perfectly complete (2.3.4.2)
the coverage diagram: “Verifier ... S4: soundness extracts constraints”	2.1.2	has the right level (constraints, not executions); the diagram has no row for the commitment’s list binding or for the challenge derivation, which the tree shows are where the compiled error’s structure lives
the extracted witness “should contain ordered pc/fp steps and the full memory image” (arch: 332-335, for T6)	2.0.1	the blueprint’s <code>baseVerifier_extractExecution</code> keeps only $\exists t, \text{ValidExecution}$, dropping <code>AssignmentRepresents (witnessOf q) t</code> , which the chain has and recursion consumes (boundary-adaptor note 16)
T1–T8 refine the six obligations of arXiv:2607.23752 (CC-S, CC-C, PS, VC, WC, PC)		not attributed in the file; no T-theorem covers the deployed (Rust, Python) verifier: VC for it is differential testing only (literature E.5, LT11)

What the map lacks that the tree has: every obligation of node 2.1.2’s leaves (the seven steps of section 6.3.3); the per-phase and per-component obligations (it names components without listing them, which is by design); the hypotheses table (table 6.2). What the map has that the tree does not need: nothing (every item feeds a node). What it gets wrong: the shape of the base theorem (a probability over nothing) and the omission of `WellFormedBytecode`.

6.4.2 The blueprint’s holes table (bp:594-617)

Twenty-two units in twenty rows. Each row against the tree: what it has, what it lacks, what it gets wrong. In the first column the hole is named first and its code follows in parentheses.

Table 6.7: The blueprint’s holes table (bp:594-617) set against the tree: what each row has, lacks and gets wrong.

Hole	Tree nodes	Has	Lacks	Gets wrong
the spine (S)	2.2	the composition, the seams, the commit phase, the extractor chain, the transport lemma; all built and proved	a bound on <code>piopError</code> (2.2.2.5); “the extracted stack” as a definition (2.2.2.4); a completeness error field (2.2.1.4); the oracle-freeness requirement (2.1.2.3.3)	<code>Seam.bus</code> ’s shape (2.2.3.3); the evaluation interface of the stack (2.5.7.3, 2.3.5.1)
sumcheck Def, completeness; Security (G1, G2)	2.5.1	the plain variant with $d/ F $ per round	the normalized variant the GKR runs (2.5.1.1); the transport lemma for the dropped coefficient (2.5.1.3.3); that <code>ArkLib</code> ’s completeness is admitted too	nothing else
batching (G3)	2.5.2	$(k - 1)/ F $	—	— (the row is right; it is also the whole opening phase under recommendation (ii))
fingerprint, Lemma 5.2, the collision bound (G4)	2.5.3	the three statements	—	“Lemma 5.1” (bp:601) is Lemma 5.2 (gt-bus G14); the Schwartz–Zippel names (lib-arklib G.6)
GKR (G5, G6)	2.5.4, 2.3.1.3.6	the shape	the last combiner, the binary layer’s challenge, the normalized round	degree 5 and $5/ E $; and the GKR cannot be appended in the bus phase with the zerocheck clause (gt-bus G3)
Layer 1 (L1)	2.5.6	every listed name, built and proved	the strided reader (2.5.6.3); the tie order (2.5.6.4); the two verifier evaluators and <code>evalMLe_padHigh</code> in the interface list (code-layer1 G.6)	acceptance tests 7, 14, 15 name wrong or vacuous witnesses (code-layer1 G.4, G.5, G.9)
Clean components as polynomials (I1)	2.5.8	the bridge theorems	the count columns’ derivation (2.5.8.3); the boundary-block bridge (2.6.2.2)	Mathlib polynomials where the instance holds <code>CompPoly</code> ’s, noncomputable at the pin (2.5.8.1)
the adaptor (I2)	2.6	the four theorems and the two maps	the carrier of “ $R1CS \Rightarrow$ the limb slots compress” (2.6.2.5); <code>witnessOf</code> ’s input; the reducibility of the instance	<code>Ensemble.toM3</code> of the eight tables (2.6.1); <code>leanIsaInstance_fits</code> ; <code>admissible_iff_caps</code> ; <code>witnessOf_stackOf</code> as an equality (2.6.4); the circular <code>FlockInterface</code> dependency
bus (P1, P2)	2.3.1	<code>busPhase</code> , <code>leaf_decomposition</code> , <code>busError</code> ; the right (α, β) term; $R_c \neq 0$; one root	the leaf stacks’ order, the orders of roots/evaluations/claims, the two side conditions (2.3.1.3.8 to 2.3.1.3.10)	the sketch’s types; the GKR’s degree and errors; the zerocheck charge (three ways); “the invariant after (α, β) is the multisets differ” (it must be “the products differ”, gt-bus G11)
table sumcheck (P3, P4)	2.3.2	<code>tableSummand</code> , <code>tableSummand_target</code> ; the right ξ and round errors	the guards the seam forces (2.3.2.6); which tables the sumcheck opens (2.3.2.1); the wire’s three-coefficient message	the sketch’s types; the rounds and final message ranging over every table of the instance; <code>tableSumcheck_relOut_implies_constraints</code> is a placeholder
public input (P5)	2.3.3	both halves, built and proved	the deployed verifier’s phase (2.3.3.4)	the check is not load-bearing (2.3.3.3); the debt assigned to Layer 12
Flock at the flock seam (P6)	2.3.4	the spine’s slot (<code>FlockOut</code> , <code>Seam.flock</code>), which is right; <code>flockError_le</code> ’s numbers	the constant position in <code>aux</code> ; a <code>FlockRegion</code> in the instance; the F_2 -independence hypothesis; the Category B constants’ owner; the <code>With</code> form	<code>FlockInterface</code> consumes the limb claims (error 1 against leanVM’s verifier); <code>limbColumns</code> inside the interface; acceptance test 20’s inverse; test 23’s number
claim pool and opening (P7, P8)	2.3.5	<code>Weight</code> , <code>WeightedClaim</code> , $(J - 1)/ E $ on λ	—	a μ -round sumcheck and an evaluation query leanVM does not run; “ $2/ E $ per round”

(continued)

Hole	Tree nodes	Has	Lacks	Gets wrong
the knowledge append (C1)	2.2.2.1, 2.2.2.2	done, proved	—	“deleted at the pin bump”: #615 is open, conflicting, on a framework upstream calls legacy (lib-arklib G.3)
WHIR (K1)	2.1.2.4	whirOpen, whirOpen_rbrSoundness, McaJohnson, encode	the η_i in ladder; the level-0 relayout, the intro messages, the omitted last message; encode over E; the K-valuedness lemma (2.1.2.3.1)	encode_column_weight as an existential; the annex’s numbering; whirOpen “batches a second time” after the opening sumcheck
Merkle, bytes, parameters (K2)	2.1.2.6, 2.1.2.4.11	the definitions	the leaf-image order, the two leaf widths, the pruned-versus-raw path format; VCVio’s Merkle library as a possible base (docs-debt DD15)	—
transcript, Proof, verify, verify_iff_compiled, the FS and BCS interfaces (K3)	2.1.1, 2.1.2.1, 2.1.2.2	the definitions; the fixture	the Merkle compilation (2.1.1.1); the grinding conjunct; the canonical-encoding checks; the chain lemma; the state-restoration step; the hash term; the grinding model; the list-binding compilation; the six mutations do not cover the grinding nonce, a truncated path, a size above the caps	verify_iff_compiled with ArkLib’s Verifier. fiatShamir on the oracle verifier; FiatShamirSecurity and BcsSecurity as stated; “perfect completeness survives the transform”
T4 (K4)	2.0	the two statements’ intent	WellFormedBytecode; a query budget and a random oracle; the resource hypothesis; the witness generator; the extracted witness in the conclusion	the shape of both theorems (2.0.1, 2.0.2)

Obligations the tree needs that no hole owns (consolidated; each is recorded by a sibling dossier individually, the list as such is finding 1): the list-binding compilation with its three lemmas (2.1.2.3); round-by-round to state-restoration (2.1.2.1.1); the chain lemma (2.1.2.1.2); the grinding model (2.1.2.1.3); the hash term (2.1.2.1.4); the program-hash term (2.1.2.1.5); the family composition over sizes (2.1.2.1.6); the Merkle compilation of the IOPP (2.1.1.1); the role-separation lemma (2.1.2.2.2); the strided reader (2.5.6.3); the boundary-block bridge (2.6.2.2); the carrier of “R1CS $\Rightarrow$ compress” (2.6.2.5); the deployed public-input phase (2.3.3.4); the normalized sumcheck variant (2.5.1.1); the transport lemma for the dropped coefficient (2.5.1.3.3); the guards or the shape at the bus seam (2.3.2.6); the leaf stacks’ order and the instance’s two side conditions (2.3.1.3.8, 2.3.1.3.9); the count columns’ derivation (2.5.8.3); the oracle-freeness of the front (2.1.2.3.3); the bound on `piopError` (2.2.2.5); the Flock walk in `settleFixedClaims` (2.1.1.5).

The same twenty-one obligations, one per row, with the hole to which the proposed change of finding 1 assigns each (table 6.7 gives each hole’s row):

Table 6.8: The twenty-one obligations of the composition that no hole owns.

Obligation	Node	Proposed hole
1 the list-binding compilation with its three lemmas	2.1.2.3	a new hole, “list-binding compilation (between WHIR and the transcript hole)
2 round-by-round to state-restoration	2.1.2.1.1	the transcript hole (K3)
3 the chain lemma	2.1.2.1.2	the transcript hole (K3)
4 the grinding model	2.1.2.1.3	the transcript hole (K3)
5 the hash term	2.1.2.1.4	the transcript hole (K3)
6 the program-hash term	2.1.2.1.5	the transcript hole (K3)
7 the family composition over sizes	2.1.2.1.6	the transcript hole (K3)
8 the Merkle compilation of the IOPP	2.1.1.1	the transcript hole (K3)
9 the role-separation lemma	2.1.2.2.2	<i>none named in the proposed change</i>
10 the strided reader	2.5.6.3	the Layer 1 hole (L1)
11 the boundary-block bridge	2.6.2.2	the hole of Clean components as polynomials (I1)
12 the carrier of “R1CS $\Rightarrow$ compress”	2.6.2.5	the adaptor (I2)
13 the deployed public-input phase	2.3.3.4	the public-input hole (P5)
14 the normalized sumcheck variant	2.5.1.1	the sumcheck hole (G1)
15 the transport lemma for the dropped coefficient	2.5.1.3.3	the sumcheck hole (G1)
16 the guards or the shape at the bus seam	2.3.2.6	<i>none named in the proposed change</i>
17 the leaf stacks’ order and the instance’s two side conditions	2.3.1.3.8, 2.3.1.3.9	<i>none named in the proposed change</i>
18 the count columns’ derivation	2.5.8.3	the hole of Clean components as polynomials (I1)
19 the oracle-freeness of the front	2.1.2.3.3	the new hole, “list-binding compilation”
20 the bound on <code>piopError</code>	2.2.2.5	the spine (S)
21 the Flock walk in <code>settleFixedClaims</code>	2.1.1.5	<i>none named in the proposed change</i>

The proposed change of finding 1 also gives the transcript hole (K3) the canonical-encoding checks, the WHIR hole (K1) the K-valuedness lemma and `encode` over E, and the Layer 1 hole (L1) the tie order (2.5.6.4), none of which is among the twenty-one; it names no hole for rows 9, 16, 17 and 21.

6.5 Findings from the tree

Only what the tree reveals and no sibling dossier states as such; each finding cites the nodes it rests on. The severities are the brief’s.

Finding 1 (major). Twenty-one obligations of the composition have no hole, no layer and no owner

Evidence. The closing list of section 6.4.2 against the holes table (bp:594-617) and the layers (bp:619-1259): each item is needed by an arrow of the chain in section 6.2 and appears in no hole’s “Produces” column and in no layer’s sketch. Sibling dossiers record most of them one at a time (gt-opening-compile E.2, E.3; literature A.4; code-layer1 G.1; boundary-adaptor findings 2, 6, 7; gt-bus G7, G10; gt-table-pub TP1, TP4; code-spine CS1); none states that the plan’s unit of work leaves them unassigned.

Classification. An error of the blueprint (the plan is incomplete, not wrong where it is stated).

Proposed change. Add to the holes table one row per item, or fold them into the nearest hole’s “Produces” column: the transcript hole (K3) gains the Merkle compilation, the chain lemma, the state-restoration step, the grinding model, the hash and program-hash terms, the family composition, the canonical-encoding checks; the WHIR hole (K1) gains the K-valuedness lemma and `encode` over `E`; a new hole “list-binding compilation” (between the WHIR hole and the transcript hole) gains 2.1.2.3 with its union-bound lemma and the oracle-freeness requirement; the Layer 1 hole (L1) gains the strided reader and the tie order; the hole of Clean components as polynomials (I1) gains the boundary-block bridge and the count columns; the adaptor (I2) gains the carrier of “R1CS $\Rightarrow$ compress”; the public-input hole (P5) gains the deployed phase; the sumcheck hole (G1) gains the normalized variant and the dropped-coefficient transport; the spine (S) gains the bound on `piopError`.

Reason. A reader of the holes table today believes that filling twenty-two units yields `baseVerifier_extractsExecution`; it yields the two master theorems and a `verify` with no theorem tying them together.

Finding 2 (major). The compilation needs every phase before the opening to make no oracle query, and nothing states or enforces it

Evidence. The list-binding compilation (2.1.2.3, gt-opening-compile E.3) replaces the commit message by a codeword and keeps the front’s verifier unchanged; that is possible only if the front’s verifier never queries the stack, since a query answered by $\tilde{q}$ has no counterpart when the oracle is a word close to a list. The two built phases do not query (the commit verifier reads nothing; the public-input verifier queries only its own message, `PublicInput.lean:222-231`). The spine’s `Phase.Def` allows a verifier to query the stack at any point any number of times (ArkLib’s `OracleVerifier`; neither ArkLib nor the spine counts queries, `OracleVerifier.numQueries` being `sorry`), and code-spine D.3 (d) and gt-table-pub E.4 item 6 show that a phase reading the whole stack fills every slot. So a bundle `Phases I` may satisfy both master theorems and be uncompileable. Under the blueprint’s evaluation interface the opening phase itself queries once (the final evaluation), so the requirement is “every phase but the last”; under recommendation (ii) it is “every phase but the opening’s one weighted query, which WHIR realizes”. No sentence of the blueprint, the tracker or the spine’s docstrings states it.

Classification. An error of the blueprint (an omission in the design of the seams).

Proposed change. Either a field of `Phases` (or a hypothesis of the compilation theorem) “the verifier of each front phase makes no query to `TheOracle I`”, stated as a property of the `OracleVerifier` (its `verify` computation is in the image of the query-free embedding), with the two built phases as its first inhabitants; or a stronger type for the front phases, an `OracleReduction` whose verifier is a plain `Verifier` over `[]`, lifted by `keepOracles`, so that a query is a typing error. Add to the conventions row *The oracle* (bp:317): “No phase before the opening queries it; the opening queries it once (under (ii)); the compilation depends on this.”

Reason. Without it the sentence “WHIR realizes the oracle” has no theorem behind it for an arbitrary bundle, and a phase written in the ideal model with a convenient query would pass every planned check and break the compilation.

Finding 3 (minor). The per-challenge error has no carrier for the grinding bits, and the compiled protocol’s schedule must carry them

Evidence. `Component.Def.err : pSpec.ChallengeIdx  $\rightarrow$   $\mathbb{R}_{\geq 0}$` (`Component.lean:66`) is what `piopError` sums and what the Fiat–Shamir interface consumes (bp:1224: “error $Q \cdot \max_i \epsilon_i$ ”); leanVM’s 128-bit claim needs, per challenge, both ϵ_i and the grinding bits b_i (2.1.2.1.3; literature A.3 item 3). The ground rounds are WHIR’s query rounds only, which live in the compiled protocol (2.1.2.4.7), not in the oracle protocol, so the spine’s `err` is the right type for the front; but `leanVmIopp`’s definition (2.1.3) must expose b_i per challenge and the Fiat–Shamir interface must take it. The blueprint’s `niError` prose (“grinding-adjusted WHIR terms”) names no such datum.

Classification. An error of the blueprint (an omission in the data of Layer 12).

Proposed change. Give the compiled reduction’s definition a second function `grind : pSpec.ChallengeIdx  $\rightarrow$   $\mathbb{N}$` (zero everywhere but at the query rounds, 17 there), make `FiatShamirSecurity` take it, and define `niError` from `err` and `grind`.

Reason. The statement of the interface must be able to say which challenge is ground.

Finding 4 (note). The two master theorems are conditional on bundles no instance but the toy can inhabit today, and the tree shows the smallest set of nodes that makes them about leanVM

Evidence. Reading the tree bottom-up from `piop_perfectCompleteness` and `piop_rbrKnowledgeSoundness` (2.2.1, 2.2.2), the nodes that must exist for the theorems to be about leanVM’s verifier, and are not built, are: the instance (2.6.1, with 2.5.6.3, 2.5.8), the four unbuilt phases’ `Defs` (2.3.1.1, 2.3.2.1, 2.3.4.1, 2.3.5.1), their `Complete` and `Security` (eight fields), the bound 2.2.2.5, the non-vacuity theorem 2.6.3, and the refinement 2.1.1 with its fixture. That is the whole of Layers 3, 6, 7, 9, 10 and 12’s `verify_iff_compiled`; nothing smaller pins the phases (2.2.2.6). Every other node of the tree is above (the compilation’s security) or below (the generic components, whose theorems the phases’ proofs consume).

Classification. Not a divergence; a reading aid for the report’s chapter.

Proposed change. None to the blueprint beyond finding 1; the status file should list these nodes as “what makes the master theorems about leanVM” separately from the holes.

Negative results.

- Every node of the blueprint’s Layers 0 to 13 and of its holes table has a place in the tree; no blueprint obligation was found redundant (every layer feeds `verify` or a theorem about it; agrees with boundary-adaptor F). Three named declarations serve nothing on the chain (`witnessOf_stackOf`, `Extractor.Straightline.map`, `piop_rbrKnowledgeSoundness_exists`): non-vacuity tests or upstream candidates, as boundary-adaptor F says.
- The per-challenge errors of the built phase ($1/|E|$) and of the two sketched phases whose arguments the sibling dossiers worked (table: $(B+2)/|E|$ on ξ , $3/|E|$ per round; opening: $(J-1)/|E|$ on λ) are correct and tight, on the right challenges (gt-table-pub D; code-pubinput F; gt-opening-compile A.5). The bus phase’s (α, β) term is correct within a factor 2.
- The composition of errors through the spine is per challenge with no additive term (lib-arklib G.9, `KnowledgeAppend.lean` read in full by that dossier); section 6.3.1 adds nothing at the seams.
- The hypotheses table (table 6.2) reproduces boundary-adaptor C’s rows and adds six (the grinding check, the canonical encodings, `R1CS_DIGEST`, the `F_2`-independence of the fixed weights, the oracle-freeness of the front, the bound on `piopError`); none of boundary-adaptor’s rows was found wrong.

Chapter 7

The trusted base

What a reader has to accept without a proof in this repository, in order to believe the master theorems and the theorems the blueprint builds on them. The accounting is in layers, from the kernel outward; each entry says what is trusted, why, and whether the blueprint names it. Chapter 10 measures the part of it that is leanerVM’s own definitions.

7.1 Lean and its three axioms

Every theorem of the proof system, the port of the composition theorem included, closes on `propext`, `Classical.choice` and `Quot.sound` and on nothing else, at both pins (appendix B); the repository’s validation rejects `sorryAx` across its two namespaces with a planted-axiom control, and forbids `native_decide`, whose trust would be the compiler’s rather than the kernel’s. A `#guard` trusts the compiled evaluator; a `decide` the kernel’s reduction (chapter 3). The kernel and the elaborator are trusted as every Lean development trusts them.

7.2 The libraries’ definitions

The statements unfold to definitions of four libraries, which a reader must accept as meaning what they should: ArkLib’s protocol specifications, oracle reductions, run semantics, completeness, knowledge state functions, extractors and round-by-round knowledge soundness; VCVio’s oracle computations as distributions, its probabilities and its uniform sampler; and CompPoly’s tables, polynomials and the two fields. Clean’s objects enter through `SatisfiedBy`, on the other side of the adaptor. Chapter 3 introduces each and records what the review checked of it: the fields are leanVM’s bit for bit, the samplers are uniform by the class’s law, the oracle interface of the stack answers the extension in the specification’s bit order, and ArkLib’s notion of knowledge soundness is non-vacuous. Two caveats stay: ArkLib’s definition of a knowledge state function deliberately differs from the literature’s (its own docstring says so), and its existential notion of round-by-round knowledge soundness is soundness, since the extractor may be a classical choice; the spine’s named form is knowledge with the extractor read.

7.3 Theorems the libraries admit, on which the chain will rest

None of leanerVM’s theorems depends on an admitted theorem today; the chain the blueprint plans does, at both pins:

- the implication from round-by-round to plain knowledge soundness, which gives the master theorem its prover-level reading (a bound on the success of a cheating prover rather than a property of transcripts); its conclusion is existential in the extractor;
- the security of the Fiat–Shamir and Merkle (BCS) compilations, which ArkLib does not state (its BCS transform is commented out, its commitment extractability has `False` for a body, its Fiat–Shamir completeness is admitted and stated for a constant challenge oracle);

- the coding-theory theorem behind WHIR’s soundness, mutual correlated agreement up to the Johnson bound, which ArkLib admits and which in the literature is a single-version preprint with a sketched proof;
- ring switching, whose leaves ArkLib admits, and which is in any case another protocol than leanVM’s.

The blueprint’s ledger names the first and the second and the third; it describes the fourth as a missing profile, which it is not (section 8.6). The new pin lifts none of them.

7.4 The interfaces the blueprint assumes

Four structures are taken as arguments of the non-interactive theorems and one of the adaptor: the Flock phase with its security and its error, the Fiat–Shamir and BCS security statements, the coding-theory theorem, and the Flock witness generator. Each is meant to be a structure whose fields are theorems another roadmap or an upstream library owes. What the review found: none exists in Lean; the Flock interface as sketched has knowledge error one against leanVM’s verifier and cannot be stated over an abstract instance; the Fiat–Shamir and BCS interfaces are stated for constructions that are not leanVM’s single BLAKE2s map, and the Fiat–Shamir one, as “the query bound times the largest round error”, cannot certify leanVM’s 128 bits, which the query rounds reach only through grinding (section 8.7). A structure taken as a hypothesis is trusted exactly when it is inhabited by a theorem; until then a theorem that takes it says nothing, and the repository’s own rule against uninhabited load-bearing types applies.

7.5 The instance for leanISA

The oracle protocol is stated over an abstract instance; leanVM enters through the leanISA instance, which is data: the tables’ shapes, the constraint polynomials and flush tuples read off Clean’s components, the count columns, the boundary blocks with their known columns (the index column and the program’s bytecode columns), the layout of the stack with its tie-breaking order, the three public lines, the Flock region and its predicate. Two theorems are meant to pin that data to the arithmetization (`satisfiedBy_witnessOf` and `m3Holds_stackOf`) and one fixture to the deployed layout; none exists. What the review established about the data’s shape (chapter 8): the instance is not `Ensemble.toM3` of the eight tables, and the adaptor’s soundness direction works only because the seed, finalization and state blocks are boundary blocks with known columns built from the same program the witness is rebuilt from; the layout has no reader for the eighteen limb columns; the admissibility predicate must contain the stacking and rate windows; the polynomial bridge of Layer 2 is noncomputable at the pin. Until the adaptor is built, the instance is trusted in full, and no theorem on `main` says that its relation is inhabited.

7.6 Hypotheses of the chain, and who discharges each

Hypothesis	Kind	Discharged by
<code>Phases.Complete</code> , <code>Phases.Security</code> a bound on <code>piopError</code>	the phases’ proofs	four of five phases unbuilt; the built ones prove theirs
<code>s.Admissible prog</code> (sizes within the caps and the windows)	missing from the spine a property of announced public data	nothing; to be fixed in the spine the compiled verifier’s setup checks; today, nothing
<code>WellFormedBytecode prog</code> (the sentinel is not a jump; fill blocks)	a decidable property of the public program	leanISA’s constraint-soundness theorem takes it; the blueprint’s target theorem omits it, and nobody is told to check it
the public words’ top limbs are zero	a property of the public input	by the type of <code>PublicInput</code> ; in the deployed verifier, a rejection at setup that the blueprint does not mention

Hypothesis	Kind	Discharged by
resource conditions on the execution (a memory that fits in 2^{28} committed cells, heights within the caps)	a property of the trace	nothing; the blueprint’s completeness target is false without them
<code>HasFillBlocks prog</code>	a property of the program	leanISA’s constraint-completeness theorem takes it
the Flock witness generator and its lemma	another roadmap’s	the Flock roadmap, which has not started
the Fiat–Shamir, BCS and coding-theory interfaces	upstream theorems	ArkLib, which states none of them today
BLAKE2s as a random oracle	a heuristic, not a theorem	nothing; the blueprint does not state it as a hypothesis
the Rust verifier is <code>verify</code>	an implementation correspondence	a fixture of Rust-produced proofs with mutations; evidence, not proof

7.7 Outside Lean

Three things no theorem of the repository can reach. The random-oracle heuristic: every non-interactive guarantee is a statement about a verifier with a random oracle, and the deployed verifier hashes with BLAKE2s; the known attacks on this gap do not reach leanVM’s structure, and the guarantee is classical (chapter 11). The implementation: the Rust verifier verifier is what Ethereum would run, and it is tied to the Lean verifier by tests only; the review found one check on which they already differ in strength. The hardware, the compiler and the toolchain, as for every Lean development.

7.8 What the blueprint says of its trusted surface

Its convention *Trusted surface* asks that every trusted definition fit on one screen, cite its source line, and appear in its interface list; its interface list closes with “everything not listed is a proof, a helper, or a test”. The review found fourteen load-bearing declarations of the spine not listed, 198 of the proof system’s 333 public declarations absent from it, and several listed names that do not exist (chapter 8); its “assumed interfaces a reviewer must know” are the right four, with the qualifications above; and its headline’s account of what is proved without hypotheses (“everything about the oracle protocol”) overstates what is on `main` by the four unbuilt phases, the missing error bound and the instance.

Part III

The audit

Chapter 8

Faithfulness

For each phase of the oracle protocol and for the compilation, this chapter sets the blueprint side by side with the specification and with the Rust and Python verifiers at the pin: the transcript (who sends what, in which order, which challenge is drawn after which message), the checks the verifier makes (in chapter 14, where they serve the check-by-check enumeration), the disagreements between the three sources, and the findings, each classified as an error of the blueprint, a deliberate deviation with the theorem it owes, or a deviation an upstream library forces. The tables and the findings were written by the review’s agents from the sources, then checked citation by citation by independent agents; the corrections are marked in place. leanVM’s revision is `a386121f` throughout, read from git after the checkout moved (appendix E).

8.1 How to read the tables

A transcript row is one step of the deployed verifier: a prover message with its size, a verifier challenge, a computation or a check, with the three sources’ lines. A disagreement is a place where the specification, the Rust and the Python do not say the same thing; where the two verifiers agree and the specification is silent or differs, the blueprint follows the verifiers and the report says so. A finding names what the blueprint gets wrong or leaves out, with its evidence and the change proposed.

8.2 The major findings, re-derived

A second agent, given only the sources and told to refute, re-derived eleven of the major findings (those not concerning the opening, whose dossier landed last); its dossier is `dossiers/verify-majors.md` on the review branch. Nine were confirmed as stated; two were weakened in their wording, not in their class.

Finding	Verdict
The knowledge theorem has content only with a bound on the error; none is stated (R5)	confirmed
The bus seam admits statements the deployed table sumcheck cannot serve; the real bus phase can fill its slot (R2, R3)	both confirmed, no tension: completeness is universal over the input relation, so the seam’s surplus burdens only the table phase
The public-input check is not load-bearing for any theorem (R13)	confirmed
The three executable verifiers check one combined equation; the specification checks per limb (R12)	confirmed; the counterexample challenge checked by hand

Finding	Verdict
The GKR round is the normalized cofactor of degree 4; a combiner is drawn after every layer, the last unused (R7, R9)	weakened: the blueprint’s $5/ \mathbb{E} $ is a valid, loose bound and its “check on the cofactor” is right; wrong are the wire (five coefficients where four travel), the layer check (an equality factor the deployed one lacks), the absence of the normalized variant, and the extra combiner
The generic GKR cannot be appended without admitted lifting, and its state function cannot carry the zerocheck conjunct (R10) The four Flock findings (R14 to R17)	confirmed; the lifting theorems are still admitted at the new pin all confirmed; “the verifiers take no inverse” corrected to “no inverse of a challenge-dependent value”
The target theorems are unprovable as sketched; the instance is not <code>Ensemble.toM3</code> of the eight tables (R27 to R30)	confirmed; the <code>Ensemble.toM3</code> phrase is stale against a decision the blueprint already took, and the real gap is the missing bridge lemma
Admissibility cannot equal leanISA’s caps (R33) Layer 1 has no reader for the limb columns; the fit theorem names a missing field (R35)	confirmed weakened: both facts hold, but “assigned to no layer” overstates: the blueprint assigns the strided reader to Layer 3 (with the slot map placed circularly), and the generic identity is a short derivation from Layer 1’s lemmas
The status’s finding on the Python verifier’s caps is false (R38)	confirmed

8.3 The spine

The spine is not a phase, but the seams and the error are where the phases’ faithfulness is decided, and three of the review’s findings on it belong here: the bus seam admits statements the deployed table sumcheck cannot serve (section 8.5); the error of the composed protocol is the phases’ own declaration, so that the knowledge-soundness theorem constrains nothing until a bound is stated (section 9.2); and the blueprint states the master theorems, the phases’ statement types and the opening’s objects two or three times differently, with the code implementing the spine’s version every time (section 8.9). The findings on the spine’s code are collected in section 8.9.

8.4 The setup, the commitment and the bus

8.4.1 The setup, the commitment and the bus phase: the transcript as deployed

Short forms in this section. Specification files are under `doc/leanvm/body/`: 03: is 03-proving-primitives.tex, 05: is 05-arithmetization.tex, 06: is 06-bus-interactions.tex, 08: is 08-end-to-end-protocol.tex. Rust files are under `crates/lean_vm/src/`: `mod.rs` is `cpu/mod.rs`, `layout.rs` is `cpu/layout.rs`, and `leaf.rs`, `gkr.rs`, `witness.rs`, `tables.rs`, `pcs.rs`, `constraints.rs` are at that root; `fs/` is `crates/flat_shamir/src/`, and `primitives/src/` is `crates/primitives/src/`. `py:` is `python-verifier/verifier.py`. The number after the colon is the line, at leanVM’s pin `a386121f`.

8.4.1.1 Notation and sizes

- $\mathbb{K} = \text{GF}(2^{64})$, $\mathbb{E} = \text{GF}(2^{192})$; every stream element is one element of $\mathbb{E}$ (24 bytes).

- Announced: κ_{mem} (memory log-size), $\tau_0 \dots \tau_5$ (log-heights of XOR, MUL, SET, Deref, JUMP, BLAKE2S), ρ (the rate exponent `log_inv_rate`). Public: the program, of $2^{\kappa_{\text{bc}}}$ instructions, and the public input, two words.
- μ_{stack} : log-size of the committed stack. μ (written μ_{bus} where confusion is possible): depth of the three leaf trees of the bus, $\mu = \lceil \log_2 N_{\text{push}} \rceil$ with $N_{\text{push}} = 1 + 2^{\kappa_{\text{mem}}} + 2^{\kappa_{\text{bc}}} + \sum_j n_{fl_j} \cdot 2^{\tau_j}$, where $n_{fl} = (5, 5, 3, 5, 5, 11)$ is the number of flush pairs of each table (`tables.rs:483-495, 548-561, 633-643, 731-751, 873-908`). For leanVM $17 \leq \mu \leq 28$ (the lower bound from $\kappa_{\text{mem}} \geq 16$; the upper from $\mu_{\text{stack}} \leq 28$, dossier `gt-bus.md`, section D.4).
- The count side has $ncnt = (4, 4, 2, 4, 4, 10)$ blocks per table, 28 in all (`tables.rs:479-482, 544-547, 629-632, 718-721, 869-872`).

8.4.1.2 Setup and commitment

$P \rightarrow V$ is a prover message (a read of the stream, bound into the Fiat–Shamir state as it is read, `fs/transcript.rs:283-287`), V a verifier computation or check, $V \rightarrow P$ a challenge (one squeeze of the chain per element of $\mathbb{E}$, `fs/lib.rs:95-99`).

Table 8.2: Setup and commitment, as deployed.

#	Step	Specification	Rust	Python
S0	V: seed the chain with <code>compress(iv, public input), iv = BLAKE2s("leanvm" len R1CS_DIGEST bytecode_hash)</code>	08:55 “seeded by the public input, the bytecode and Flock’s BLAKE2s R1CS matrices”; §8.4 is TODO (08:44-46)	<code>mod.rs:82-93, mod.rs:712; fs/lib.rs:69-73</code>	<code>py:1366-1369, py:347-351</code>
S1	$P \rightarrow V$: 8 elements: $\kappa_{\text{mem}}, \tau_0 \dots \tau_5, \rho$, each a small integer in the low limb	08:55 “The prover sends the memory log-size κ_{mem} and the six table log-heights τ_j ” (no ρ)	prover <code>mod.rs:118-124</code> ; verifier <code>mod.rs:144-149</code>	<code>py:1372-1376</code>
S2	V: each announced element has its two upper limbs zero	absent	<code>mod.rs:133-135</code> if <code>word.c1 != 0 word.c2 != 0 { return Err(CpuError::PublicInput) }</code>	<code>py:1373</code>
S3	V: each public word has its third limb zero	08:29 “two 192-bit words (with top limb 0)”	<code>mod.rs:141-143</code>	by type: the input is a 32-byte Digest (<code>py:204-219</code>)
S4	V: the caps (the checks B4 to B9 of table 14.1)	06:80, 06:86, 06:90	<code>mod.rs:157-170</code>	<code>py:857-864, py:1377</code>
S5	V: derive the layouts (the stack, the three leaf stacks)	08:55 “derives the layout of every stack (PCS and GKR leaves)”	<code>mod.rs:171, layout.rs:328-427, witness.rs:67-102</code>	<code>py:856-895, py:506-511</code>
S6	V: the stacking window $15 \leq \mu_{\text{stack}} \leq 28$	absent	<code>mod.rs:174-176; pcs.rs:49, 51</code>	<code>py:1379; py:907-908</code>
C1	$P \rightarrow V$: the commitment, a Merkle root as 2 elements (two 128-bit halves)	08:60-61 “sends its commitment”	prover <code>pcs.rs:118-119, mod.rs:560-562</code> ; verifier <code>mod.rs:714, pcs.rs:136-138, fs/transcript.rs:97-99</code>	<code>py:1382</code>
C2	V: both halves of the root have a zero third limb	absent	<code>fs/merkle.rs:26-29 (NonCanonicalEncoding)</code>	<code>py:223</code>

No challenge is drawn before the root is read (`pcs.rs:93-96` “bind its root into the transcript, before any challenge is sampled”; the first `sample` is `leaf.rs:876`).

8.4.1.3 The bus phase

Table 8.3: The bus phase, as deployed.

#	Step	Specification	Rust	Python
U0	V: structural assertions on the leaf layouts (panics, not rejections)	absent	<code>leaf.rs:875, leaf.rs:123-146</code>	absent
U1	$V \rightarrow P$: $\alpha_0, \alpha_1, \alpha_2, \alpha_3$ then β : five squeezes, no message between them	08:67; 05:22, 05:28	<code>leaf.rs:876, leaf.rs:882</code>	<code>py:542, py:544</code>

#	Step	Specification	Rust	Python
U2	$P \rightarrow V$: 2 elements, the bus root R then the count root R_c	08:68 “Sends the count root R_c and one bus root R ” (prose order reversed) 05:91	prover gkr.rs:272-276; verifier gkr.rs:364-367	py:434, py:437
U3	$V \rightarrow P$: the first combiner λ	05:91	gkr.rs:369	py:435
U4	The batched GKR (section 8.4.1.4): messages, challenges and one check per layer	05:61-95, 08:69	gkr.rs:373-427	py:439-455
U5	V : $R_c \neq 0$ (evaluated after the GKR)	06:78, 08:69, 08:100	leaf.rs:884-889 (Error::ZeroCount)	py:546
U6	$P \rightarrow V$: 5 elements, the evaluations of $\text{mem}_0, \text{mem}_1, \text{mem}_2, \text{cntfin_mem}$ at $\zeta_{<\kappa_{\text{mem}}}$ and of cntfin_bc at $\zeta_{<\kappa_{\text{bc}}}$, in that order	08:70 “For their committed columns the prover sends one evaluation each”; 05:111	verifier leaf.rs:909-920, leaf.rs:428-439, leaf.rs:548-550; prover leaf.rs:500-529	py:552-557
U7	V : computes, per side, the public part of the leaf claim (selectors, constants, index column, bytecode column, the five evaluations, the padding) and derives what the tables owe, rem_s ; builds the tables’ bus forms. No check.	08:70; 05:105-123	leaf.rs:389-461, leaf.rs:921-924	py:559-590

After the last of these steps (U7) the table sumcheck starts with the challenge ξ (mod.rs:726, py:1389). Nothing of the bus phase is sent after the boundary evaluations (U6).

Element counts (checked by the dossier’s probe, dossier **gt-bus.md**, section H.1, against the pinned Python function, for $\mu = 0 \dots 12$; the citation check re-ran the probe, recounted every message from the code, and agrees):

Message	Elements of $\mathbb{E}$	Challenge	Squeezes
announced sizes	8	α, β	5
commitment root	2	GKR, μ even	$\mu^2/4 + \mu + 1$
roots	2	GKR, μ odd	$3 + ((\mu - 1)/2)^2 + 3(\mu - 1)/2$
GKR	$\mu^2 + 4\mu$, plus 1 if μ is odd		
boundary evaluations	5		
total, seed to end of bus	$\mu^2 + 4\mu + 17$ (+1 if μ is odd)		

8.4.1.4 The GKR in full detail

Both verifiers run the same loop (gkr.rs:358-430, py:433-457). State: the list **values** of three claimed values (initially (R, R, R_c)), the point **point** (initially empty), the combiner λ , the level **layer** (initially μ ; the root is level μ , the leaves level 0).

Radix and parity. While $\text{layer} > 0$: if **layer** is odd the step is one binary level, else two levels at once (gkr.rs:377, py:442). **layer** is odd only at the first iteration, when μ is odd (gkr.rs:378 `debug_assert_eq!(round_count, 0, "only the root-most layer may be binary")`). Specification: 05:85.

One iteration, radix 4 (**layer** even, $k = \mu - \text{layer}$ rounds).

Table 8.4: One radix-4 iteration of the GKR verifier.

Step	What happens	Rust	Python
1	V : $\text{claim} := \text{values}[0] + \lambda \cdot \text{values}[1] + \lambda^2 \cdot \text{values}[2]$. The three trees, in the order push, pull, count, are batched by the powers 1, λ , λ^2 (05:91-94).	gkr.rs:376, poly_eval_low degree first, primitives/src/ multilinear.rs:245-247	py:443

Step	What happens	Rust	Python
2	k sumcheck rounds. Round t ($t = 0 \dots k - 1$) uses the coordinate $point[t]$ as its equality factor.		
2, message	$P \rightarrow V$: 4 elements , the coefficients c_1, c_2, c_3, c_4 of a polynomial h of degree 4.	gkr.rs:319-326; read at gkr.rs:399-401 next_round_poly(5, claim, Some(equality_point))	py:445 with count = 2**step + 1 = 5
2, derivation	V : derives $c_0 := claim + point[t] \cdot (c_1 + c_2 + c_3 + c_4)$. This is the round identity $(1 + r) \cdot h(0) + r \cdot h(1) = claim$ solved for c_0 . There is no round check to fail : it holds by construction.	fs/transcript.rs:297-302 “c0 + r.(c1 + ... + cd) = claim, and every ci above c0 was read”	py:411-413
2, challenge	$V \rightarrow P$: χ_t ; $claim := h(\chi_t)$.	gkr.rs:402-404	py:424-426
3	$P \rightarrow V$: 12 elements , the four children of each tree at the point, tree by tree.	gkr.rs:406-409	py:447
4	V check : $claim = \sum_s \lambda^s \cdot \prod_{c < 4} children[s][c]$, with no equality factor .	gkr.rs:410-413 (LayerMismatch)	py:448-449
5	$V \rightarrow P$: u_0 then u_1 . $values[s] :=$ the bilinear interpolation of the four children, u_0 on the low child bit and u_1 on the high one.	gkr.rs:414-415; gkr.rs:416-422	py:451; py:452
6	$V \rightarrow P$: a new combiner λ .	gkr.rs:423	py:453
7	$point := (u_0, u_1, \chi_0, \dots, \chi_{k-1})$; $layer := layer - 2$.	gkr.rs:424-425	py:454

h is the **cofactor**: the honest $h_t(Y) = \sum_x eq(point_{>t}, x) \cdot \sum_s \lambda^s \cdot \prod_{a,b} \tilde{V}_{layer-2}^s(a, b, \chi_{<t}, Y, x)$, without the factor $eq(point[t], Y)$ and without the factors $eq(point[u], \chi_u)$ of the earlier rounds. The running claim is the value of the cofactor, never multiplied by an equality factor (the “normalized” form of gkr.rs:4-5 “Its normalized eq-trick sumcheck has degree four”).

One iteration, binary ($layer = \mu$ odd, no round): $P \rightarrow V$ 6 elements, two children per tree; check $claim = \sum_s \lambda^s \cdot left_s \cdot right_s$; $V \rightarrow P$ u ; $values[s] := (1 + u) \cdot left_s + u \cdot right_s$; $V \rightarrow P$ a new λ ; $point := (u)$ (gkr.rs:377-395, py:442-455 with step = 1).

The last combiner is never used. Step 6 runs in every iteration, the last included, where **layer** then becomes 0 and the loop ends. So the number of combiners is the number of layers plus one, and the last squeeze changes the state of the chain before ξ is drawn and enters no computation (gkr.rs:423 then gkr.rs:429; py:453 then py:457). The probe confirms it: at $\mu = 4$ the GKR draws 9 challenges and ζ is draws 6, 7, 4, 5; draw 8 is unused.

How ζ is assembled. ζ is the **point** after the last iteration ($layer = 2$, $k = \mu - 2$): $\zeta = (u_0, u_1, \chi_0, \dots, \chi_{\mu-3})$. Every coordinate of ζ is a challenge of the **last** layer; no challenge of an earlier layer is a coordinate of ζ . In time order the last layer draws $\chi_0, \dots, \chi_{\mu-3}$ (which become $\zeta_2, \dots, \zeta_{\mu-1}$), then $u_0 = \zeta_0$, then $u_1 = \zeta_1$, then the unused combiner. Coordinate 0 is the lowest bit of the leaf index: the Rust test `radix_four_roundtrip_at_even_and_odd_depths` (gkr.rs:479-508) asserts `proved.values[lane] == mle_eval_e(&leaves[lane], &proved.point)` with `mle_eval_e` folding `point[0]` over adjacent pairs (gkr.rs:436-447), and the probe checks the same against the pinned Python. For $\mu = 1$, $\zeta = (u)$; for $\mu = 0$ there is no layer and ζ is empty.

The probe’s table (dossier `gt-bus.md`, section H.1): ζ_k is the draw of that number, counting the GKR’s draws from 0.

μ	$\zeta_0, \zeta_1, \zeta_2, \dots$	draws
2	1, 2	4
3	4, 5, 3	7
4	6, 7, 4, 5	9
5	10, 11, 7, 8, 9	13
8	22, 23, 16, ..., 21	25

How the three trees share challenges. Entirely: one point, one sequence of χ , one pair (u_0, u_1) per layer; the three are told apart only by the power of λ . The count tree is shallower; it is padded with leaves 1 to the depth of the bus trees (`leaf.rs:879-881` `count_lay.mu = push_lay.mu`; `py:540`, `py:578-579`, where the selector of a count block is extended by zero bits to the length of ζ).

8.4.1.5 The leaf stacks and the decomposition

Blocks, in index order (`layout.rs:349-415`; `py:537-540`, `py:866-876`). Push side: the state boundary $\kappa = 0$, (sep_{ST}, g^0, g^0) ; the memory seed $\kappa = \kappa_{mem}$, $(sep_{MEM}, idx, 1, mem_0, mem_1, mem_2)$; the bytecode seed $\kappa = \kappa_{bc}$, $(sep_{BC}, idx, 1, P_3, \dots, P_{10})$; then for each table in order each of its pushes, $\kappa = \tau_j$. Pull side: the state boundary $(sep_{ST}, g^{2^{\kappa_{bc}}-1}, g^0)$ (`layout.rs:335`, `355-358`); the memory finalization with `cntfin_mem` in place of 1; the bytecode finalization with `cntfin_bc`; then the tables' pulls. Count side: for each table in order, one block per count column, a single coordinate (`layout.rs:412-414`). **The three blocks no table owns come first.**

Placement. Largest first at aligned offsets, ties by index (`witness.rs:67-79` `order.sort_by(|&a, &b| kappas[b]...cmp(...).then(a.cmp(&b)))`, called by `leaf.rs:149-156`; `py:305-311`). $sel_b = offset_b \gg \kappa_b$, its bits low first (`leaf.rs:407-411`; `py:299-302`). The depth is $\lceil \log_2 \Sigma_b 2^{\kappa_b} \rceil$ with **no floor** (`leaf.rs:153`), unlike the witness stack (`witness.rs:97`).

Leaves. Row z of block b holds $\beta + \Sigma_i eq(\alpha, i) \cdot c_{b,i}(z)$ (`leaf.rs:226-260`; characteristic 2), the weights $eq(\alpha, \text{bits of } i)$, bit k of i against α_k (`leaf.rs:89-98`). The count side runs at $\alpha = 0$, $\beta = 0$, so its leaf is the count cell (`leaf.rs:606-622`; `py:573`). Padding leaves are 1 (`leaf.rs:222-224`, `gkr.rs:61`, `114`).

Decomposition (the last computation of the phase, U7). With $\zeta_{lo} = \zeta_{<\kappa_b}$, $\zeta_{hi} = \zeta_{\geq\kappa_b}$, $w_b = eq(sel_b, \zeta_{hi})$:

- a block no table owns contributes $w_b \cdot (\beta + \Sigma_i eq(\alpha, i) \cdot v_i)$ where v_i is the constant, the index column's extension $\prod_k (1 + \zeta_k(1 + g^{2^k}))$ (`primitives/src/field/mod.rs:105-113`, `py:271-278`; `06:97-100`), the extension of the public bytecode column, or the value the prover sent for a committed column (`leaf.rs:440-457`). A committed column already valued at the same point is not sent again (`leaf.rs:466-471`), which is why the boundary evaluations (U6) are 5 elements and not 8;
- a block of table j contributes nothing to the check: it is accumulated into the form of $(side, j)$, coefficient $w_b \cdot eq(\alpha, i)$ on coordinate i and $w_b \cdot \beta$ on the constant (`leaf.rs:416-423`, `leaf.rs:367-382`; `py:582-587`);
- the padding contributes $1 + \Sigma_b w_b$ over all the blocks of the side (`leaf.rs:459-460`; `py:589`; `05:107`);
- $rem_s := \tilde{V}_0^s(\zeta) + (\text{public part} + \text{padding})$ (`leaf.rs:921-924` “What the tables owe this side: DERIVED, never read”; `py:590`).

The Rust evaluates the eight bytecode columns one by one (`leaf.rs:453`); the Python evaluates the stacked bytecode table once at $(\zeta_{<\kappa_{bc}}, \alpha)$ (`py:566`), as `08:70` writes it. The two are equal when the stacked table is zero outside slots 3 to 10 (table 8.5, item 6).

8.4.2 The setup, the commitment and the bus phase: disagreements between the three sources

Short forms in this section. Specification files are under `doc/leanvm/body/`: `05:` is `05-arithmeticization.tex`, `06:` is `06-bus-interactions.tex`, `08:` is `08-end-to-end-protocol.tex`. Rust files are under `crates/lean_vm/src/`: `mod.rs` is `cpu/mod.rs`, `layout.rs` is `cpu/layout.rs`, and `leaf.rs`, `constraints.rs` are at that root; `fs/` is `crates/fiat_shamir/src/`. `py:` is `python-verifier/verifier.py`. The number after the colon is the line, at leanVM's pin `a386121f`.

The three sources agree on the transcript of the setup, the commitment and the bus phase, up to omissions of the specification (it does not mention the announced rate, the BLAKE2S floor, the stacking window, the canonical encodings, the last unused combiner) and the prose order of the two roots. The Rust and the Python verifier accept the same transcripts in these phases; no divergence between them was found. The

specification is silent on seven points the verifiers fix, and its prose names the two roots in the other order. The blueprint follows the Rust on each, rightly (finding 19).

Table 8.5: Disagreements between the specification, the Rust and the Python in the setup, the commitment and the bus phase.

#	Subject	Disagreement
1	The announced rate	The announced rate ρ is sent with the sizes (<code>mod.rs:123, 149</code> ; <code>py:1376</code>); <code>08:55</code> names seven announced values.
2	The order of the roots	<code>08:68</code> names R_c first; the stream has R first.
3	The seed	<code>08:55</code> has “Flock’s BLAKE2s R1CS matrices”; the Rust hashes one constant naming the circuit (<code>mod.rs:66-93</code>). (The status file’s finding on the seed, F14.)
4	Checks of the verifiers the specification does not mention	The upper limbs of the announced sizes (B2), the BLAKE2S floor (B7), the rate window (B8), the stacking window (B9), the zero third limb of the root’s halves (B10), the length of ζ (B16), the consumption of the stream (B17): the checks of table 14.1.
5	The combiner after the last layer	The verifiers draw one combiner more than layers (finding 6); <code>05:91</code> has one per layer.
6	The program in the Python verifier	The Python verifier takes the program as the stacked table of $16 \cdot 2^{\kappa_{bc}}$ words and evaluates it whole (<code>py:566, 857</code>); it does not check that slots 0 to 2 and 11 to 15 are zero. The Rust builds the eight columns from the instructions (<code>layout.rs:229-309</code>). The two agree on every table that is a program’s; the Python’s statement space is larger. Not a soundness matter: the table is the public statement.
7	The fingerprint bound, the count columns, the assertions	The Rust’s bound for the fingerprint is $5 \cdot 2^\mu$ (<code>leaf.rs:110</code>), the specification’s $4 \cdot 2^\mu$ (<code>05:37</code>); and “every count column” (<code>06:78</code>) against the tables’ read counts. The Rust’s assertions on the layout (B11) have no counterpart in the Python.
8	The table sumcheck’s round message, in the Rust’s own comment (added by the citation check)	The comment at <code>constraints.rs:265-266</code> says “ $h(0)$ is derived from the running claim rather than transmitted”, but the call at <code>:267, next_round_poly(4, claim, None)</code> , transmits $c_0 = h(0)$ and derives c_1 (<code>fs/transcript.rs:291, 299</code>). The comment is stale; the code and the Python agree with each other.

8.4.3 The setup, the commitment and the bus phase: findings

Short forms in this section. Specification files are under `doc/leanvm/body/`: `05:` is `05-arithmetization.tex`, `08:` is `08-end-to-end-protocol.tex`. Rust files are under `crates/lean_vm/src/`: `mod.rs` is `cpu/mod.rs`, `layout.rs` is `cpu/layout.rs`, and `leaf.rs`, `gkr.rs`, `constraints.rs` are at that root; `fs/` is `crates/flat_shamir/src/`. `py:` is `python-verifier/verifier.py`. These are at leanVM’s pin `a386121f`. `bp:` is the blueprint `docs/roadmap/protocol-blueprint.md` and `st:` the status `docs/roadmap/protocol-status.md`, both at leanerVM `b435631` (the merge of the Lean 4.34.1 upgrade moved blueprint lines after its line 51 by four). `Phase.lean`, `Seams.lean`, `Instance.lean` are under `LeanerVM/Protocol/Spine/`; `Component.lean` and `KnowledgeAppend.lean` under `LeanerVM/Protocol/ToArkLib/`; `Statement.lean` is `LeanerVM/Arithmetization/Statement.lean`; all at `b435631`. ArkLib is cited at its old pin `dca90385`. The number after the colon is the line.

Finding 5 (major). The GKR layer sumcheck is the normalized one: the blueprint gives it the wrong degree, the wrong error and no definition

Classification. An error of the blueprint.

Evidence. The deployed round message is the cofactor of degree 4, four elements on the wire, c_0 derived through the equality factor; the running claim is the cofactor’s value; the layer’s final check has no equality factor (section 8.4.1.4; `gkr.rs:399-404, 410-413`; `fs/transcript.rs:297-302`; `py:411-413, 445, 449`). The probe’s honest prover, written to that reading, is accepted by the pinned Python verifier (dossier `gt-bus.md`, section H.1; the citation check re-ran the probe and obtained the same output). The table sumcheck is the other variant: three of the cubic’s four coefficients are sent, c_1 derived, and the verifier multiplies the equality factors into its final check (`constraints.rs:267 next_round_poly(4, claim, None)`, `:271-274`; `py:608-614`). (The citation check corrected the dossier’s “the whole cubic is sent, c_1 derived”, which contradicts itself: the wire carries c_0, c_2, c_3 .)

The blueprint:

- **bp:321**: “Layer sumcheck messages have degree 5 ($\text{eq} \times \text{four multilinear}$); the round check is on the cofactor (Gruen).”
- **bp:315**: “A round polynomial travels as its coefficient list, low degree first, of length $d + 1$; the oracle protocol sends all of them. Dropping one is the *encoding* of Layer 12.”
- **bp:898-901**: the eq -weighted variant has “ $\text{eq}(\zeta_{<\tau_j}, \cdot)$ ” as an explicit factor whose evaluation at r the verifier computes itself (so d counts the cofactor plus one).”
- **bp:941-942**: “ $5/|\mathbb{E}|$ to each sumcheck round”. The tracker, in its hole for the generic GKR (G5): “degree-5 round polynomials”.

Read literally, a message of degree 5 has six coefficients and five on the wire, where the Rust has four: a verifier built so rejects every Rust proof. Read with Layer 4’s variant, the verifier multiplies the equality factors in, which is a different verifier from the deployed one (it accepts when $\chi_t = 1 + \text{point}[t]$, whatever the children). Layer 12’s `RoundPoly.decode (d) (claim) (eq? : Option E)` (**bp:1212**) knows both encodings; Layers 4 and 5 define only one protocol.

Proposed change.

Proposed change: the GKR row of the conventions (bp:321)

As it stands. “Layer sumcheck messages have degree 5 ($\text{eq} \times \text{four multilinear}$); the round check is on the cofactor (Gruen).”

As proposed. “The layer sumcheck is normalized: the prover’s message in round t is the cofactor h_t , of degree 4 (the four children’s lines multiplied, summed against the equality kernel of the coordinates not yet bound), five coefficients; the verifier checks $(1 + r_t) \cdot h(0) + r_t \cdot h(1)$ against the running claim, r_t the coordinate of the layer’s point; the new running claim is $h(\chi_t)$; the layer’s final check compares the running claim with $\sum_s \lambda^s \prod \text{children}$, with no equality factor (**gkr.rs:399-413**). The table sumcheck is not normalized (**constraints.rs:267-274**).”

Reason. Faithfulness of the wire format and of the verdict; the error is then tight.

Also:

- Layer 4: define the two variants by name, the plain one (error $d/|F|$, d the degree of the polynomial sent, the verifier evaluating the equality factors at the end) and the normalized one (error $d_c/|F|$, d_c the degree of the cofactor).
- **bp:941-942** and the tracker: “ $4/|\mathbb{E}|$ to each sumcheck round”.

Finding 6 (major). One combiner more than layers: the last is drawn and never used

Severity. Major, by the brief’s definition: the blueprint omits a challenge the verifiers draw; the fix is one sentence.

Classification. An error of the blueprint.

Evidence. **gkr.rs:369** (before the loop) and **gkr.rs:391, 423** (at the end of every iteration, the last included); **py:435, 453**. The dossier’s probe (section H.1): the GKR draws $\mu^2/4 + \mu + 1$ challenges for even μ , and ζ never contains the last. The specification has “at every layer a combiner” (**05:91**). The blueprint: “A fresh combiner λ per layer” (**bp:321**), and Layer 5 “a fresh λ per layer ... exactly as pinned” (**bp:940-941**).

In the oracle model a challenge nobody uses is harmless. After Fiat–Shamir it is a squeeze that changes the chain’s state (**fs/lib.rs:95-99**), so ξ and everything after depend on it.

Proposed change.

Proposed change: the GKR row of the conventions (bp:321)

As it stands. “A fresh combiner λ per layer”.

As proposed. “A combiner λ is drawn after the roots and after every layer, the last layer included; that last one is used by nothing and its error is 0 (**gkr.rs:369, 423**).”

Add the message count of the bus phase (the element counts after table 8.3) to acceptance test 17.

Finding 7 (major). The generic GKR of Layer 5 cannot be appended in the bus phase, and cannot carry the zerocheck clause

Classification. An error of the blueprint; the first half is also a deviation an upstream library forces.

Evidence. Layer 5’s component (bp:930-937):

```
def gkr (nside  $\mu$  :  $\mathbb{N}$ ) : OracleReduction [] :=
  (StmntIn := Fin nside  $\rightarrow$  E) -- the claimed roots
  (OStmntIn := fun _ : Fin nside  $\rightarrow$  ETable  $\mu$ ) Unit -- the leaf tables
  (StmntOut := (Fin  $\mu$   $\rightarrow$  E)  $\times$  (Fin nside  $\rightarrow$  E)) --  $\zeta$  and the leaf claims  $\tilde{V}_0^*(\zeta)$ 
  (OStmntOut := fun _ : Fin nside  $\rightarrow$  ETable  $\mu$ ) Unit ...
```

docs/roadmap/protocol-blueprint.md:930-937 at b435631

Its oracles are the leaf tables. A phase’s one oracle is the stack (Phase.lean:37-38), and two components are appended only when the output oracles of the first are the input oracles of the second (Component.lean:143-149, Def.append). The leaf tables are functions of the stack and of (α, β) ; carrying a reduction along such a map is ArkLib’s context lifting, whose completeness and knowledge-soundness theorems are admitted at the pin:

```
-- .lake/packages/ArkLib/ArkLib/OracleReduction/LiftContext/Reduction.lean:355-365, 542-560
theorem liftContext_completeness ... := by ... sorry
theorem liftContext_rbr_knowledgeSoundness ... := by ... sorry
```

ArkLib at dca90385, ArkLib/OracleReduction/LiftContext/Reduction.lean:355-365, 542-560 (the dossier’s elided excerpt)

The blueprint’s ledger row on context lifting (A4) says of them “Not consumed: statements are shaped so that no lens is needed” (bp:262). The holes table has the bus phase consume the generic GKR holes (G5, G6) (bp:606), and the tracker writes busPhase I (G : Gkr.Def).

Second half. The bus phase’s state function must turn ConstraintsVanish into the claims at ζ while the coordinates of ζ are drawn, which is inside the GKR’s last layer (dossier gt-bus.md, section D.3). The appended state function (Verifier.KnowledgeStateFunction.appendGuarded, KnowledgeAppend.lean:474, leanerVM’s port of ArkLib pull request #615) is made of the components’ own; a clause that is constant can be framed around a component, a clause that changes with the component’s challenges cannot. So the security of a GKR that knows nothing of the constraints does not give the bus phase’s. (The citation check corrected the attribution: the dossier called it ArkLib’s appended state function, but at the pin dca90385 ArkLib has no such declaration; the substance holds, the definition is the components’ own state functions spliced at the seam, KnowledgeAppend.lean:21-26, 474-488.)

Proposed change. Restate Layer 5’s component over a context, with riders:

```
-- proposed shape (Layer 5); S, OStmnt arbitrary
def gkr (nside  $\mu$  :  $\mathbb{N}$ ) (leaves : S  $\rightarrow$  ( $\forall$  i, OStmnt i)  $\rightarrow$  Fin nside  $\rightarrow$  ETable  $\mu$ )
  (riders : S  $\rightarrow$  ( $\forall$  i, OStmnt i)  $\rightarrow$  List ( $\Sigma$   $\tau$ , ETable  $\tau$ )) :
  Component.Def (S  $\times$  (Fin nside  $\rightarrow$  E)) OStmnt Unit
  (S  $\times$  Vector E  $\mu$   $\times$  (Fin nside  $\rightarrow$  E)) OStmnt Unit
-- relIn : the roots are the products of `leaves s o`, and every rider table is zero
-- relOut: the leaf claims hold at  $\zeta$ , and every rider’s extension vanishes at  $\zeta_{\{<\tau\}}$ 
-- err : 2/|E| per combiner (0 for the last), 4/|E| per round, 1/|E| per combination challenge
```

and say in Layer 6 that the bus phase is $[(\alpha, \beta); \text{roots}] \gg \text{gkr} \gg [\text{boundary evaluations}]$ with `leaves` the three leaf tables of the stack and `riders` the constraint tables. Replace bp:262’s “no lens is needed” by the reason this shape needs none. The workaround is retired when ArkLib proves the lifting theorems **and** offers a way to run a rider on a component’s challenges; the first alone does not give the second half.

Finding 8 (major). The admissibility predicate: the stacking window and the rate window

Classification. An error of the blueprint (three of its statements cannot hold together).

Evidence. The verifiers reject $\mu_{\text{stack}} \notin [15, 28]$ and $\rho \notin [1, 4]$ (the rate window and the stacking window, B8 and B9 of table 14.1). The blueprint:

- bp:800-801: “def Sizes.Admissible (prog : Program) (s : Sizes) : Prop -- the caps; decidable” and “theorem admissible_iff_caps : s.Admissible prog $\leftrightarrow$ (Caps w $\wedge$ Sizes.ofWitness w = some s)”.

- leanISA’s Caps has neither window: “(7) the WHIR rate (:167) and (8) the stacked size $\mu \in [\text{MIN_MU}, \text{MAX_MU}]$ (:174-176) are parameters of the commitment, left to the protocol layer” (Statement.lean:79-81; the structure is at :250-263).
- bp:1218-1220: `verify_iff_compiled` quantifies “ $\exists s, s.\text{Admissible prog} \wedge \dots$ ”.
- bp:1130: `piopError_le` (hs : s.Admissible prog) : $\sum i, \text{piopError } s \ i \leq 2^{40}/|E| + \text{flockError}$.

If `Admissible` is the caps alone, `verify_iff_compiled` is false (the Rust rejects sizes that are admissible) and `piopError_le` is false at the caps (dossier `gt-bus.md`, section D.4). If it contains the windows, `admissible_iff_caps` is false. Besides, `Sizes.ofWitness` w cannot produce `logInvRate`, which no witness determines, and `Caps` w has the conjunct `well_shaped`, which is no property of sizes.

Proposed change. bp:800-801, as proposed:

```
def Sizes.Admissible (prog : Program) (s : Sizes) : Prop -- decidable; read_public, mod.rs:157-176
  --  $\kappa_{\text{mem}} \in [16, 32] \wedge \forall j, \tau_j \leq 32 \wedge 3 \leq \tau_5 \wedge \log\text{InvRate} \in [1, 4] \wedge \mu_{\text{stack}} \text{ prog } s \leq 28$ 
  theorem caps_of_admissible (hs : s.Admissible prog) (hw : Sizes.heightsOf w = s.heights)
    (hd : WellShapedData w.data) : Caps w
```

with `Sizes.ofWitness` returning the heights only. Layer 3’s tests: add $\mu_{\text{stack}} = 29$ and $\log\text{InvRate} = 5$ rejected. State in `piopError_le`’s comment that the bound uses $\mu_{\text{bus}} \leq 28$, a lemma of Layer 3 ($N_{\text{push}} \leq \sum_{\text{stack}} 2^{\kappa}$).

Finding 9 (major). The status file’s finding on the Python verifier’s caps is false

Severity. Major, of the status file.

Classification. An error of the status.

Evidence. The status file’s finding on the Python caps (F9), st:316-318: “the Python verifier omits the caps $\log_mem \in [16, 32]$, $\tau_j \leq 32$, the bytecode power-of-two bound and $\tau_{\text{BLAKE2S}} \geq 3$ (verifier.py:1372-1379 versus `cpu/mod.rs:158-170`): a divergence between the two verifiers, soundness-relevant, to report upstream.” st:164: “Finding F9 ... is still to be reported to leanVM.” At the pin, py:1378 calls `build_layout`, which begins:

```
# python-verifier/verifier.py:856-864 at a386121f
def build_layout(bytecode: Sequence[K], log_memory: int, table_log_heights: Sequence[int]) -> Layout:
    log_bytecode = log2_strict(len(bytecode)) - BUS_BITS
    require(
        16 <= log_memory <= 32
        and all(0 <= log_height <= 32 for log_height in table_log_heights)
        and table_log_heights[OP_BLAKE2S] >= 3
        and 0 <= log_bytecode <= 32,
        "invalid announced table sizes",
    )
```

`python-verifier/verifier.py:856-864` at `a386121f` `log2_strict` requires a power of two (py:252-254). The check is present at the pin; it was assembled in the commits `8a6e7750`, `348ca455` and `5751a5c7`, all ancestors of `a386121f`. (The citation check corrected the dossier, which said the check was introduced by commit `14fbca8f`: that commit’s parent already has the same `require`, and `14fbca8f` only renamed the constant `BLAKE2S.opcode` to `OP_BLAKE2S`.) The Rust itself says so: “The other two verifiers reject it here too (`python-verifier`, `guests/aggregate.py`)” (`mod.rs:164-165`; the citation check corrected the line from :165). The status read lines 1372 to 1379 and not the function they call.

Proposed change. Delete the finding (F9) from st:316-318 and the line st:164; nothing is to be reported upstream. Correct the memory of the earlier reviews that cite it.

Finding 10 (minor). The zerocheck escape is charged in four different ways, none tight, one to the wrong challenge

Severity. Minor: every version is an upper bound, so none makes a false theorem; the definition of `busError` is nevertheless undetermined.

Classification. An error of the blueprint.

Evidence. bp:333 (row *Seams*): “charged in the bus phase, coordinate by coordinate as ζ is drawn, $1/|\mathbb{E}|$ per coordinate per constraint”. bp:974: “busError is $4 \cdot 2^{\mu_{\text{bus}}}/|\mathbb{E}|$ on the (α, β) message plus gkrError 3 μ_{bus} ”, no zerocheck term. bp:1001-1003: “charged to the (α, β) and GKR challenges of Layer 6, where ζ is drawn, as an extra $\tau_{\text{max}}/|\mathbb{E}|$ per constraint”. The tracker, in its hole for the bus phase’s security (P2): the second plus the first. (α, β) is drawn before the roots are sent; no coordinate of ζ exists then (section 8.4.1.4).

Proposed change. One definition, in Layer 6, replacing the three passages: “busError: $4 \cdot 2^{\mu_{\text{bus}}}/|\mathbb{E}|$ on (α, β) ; on the GKR’s challenges, gkrError. The zerocheck escape adds nothing: on each coordinate of ζ (the last layer’s round challenges and its two combination challenges) the state is the conjunction of the GKR’s and of ‘every constraint’s extension, partially evaluated, is zero’, and a conjunction escapes with the larger of the two probabilities, the second being $1/|\mathbb{E}|$ whatever the number of constraints.” Row *Seams*: drop “per constraint”; Layer 7: drop “the (α, β) and”.

Finding 11 (minor). The order of the leaf stacks is nowhere stated, and the spine’s tuples has the opposite order

Classification. An error of the blueprint (an omission of a transcribed convention).

Evidence. The deployed order puts the three blocks no table owns first, then the tables’ (section 8.4.1.5; layout.rs:354-410; py:507, “the blocks no table owns, stacked first”, py:499). Ties between equal sizes go by that index, so whenever a table’s height equals κ_{mem} or κ_{bc} the order decides the offsets, the selectors, and so the verdict. The blueprint states the rule for the witness stack only (bp:322); for the leaves it cites files (bp:196). The spine lists the table tuples first:

```
-- LeanerVM/Protocol/Spine/Instance.lean:185-186
def tuples (q : Column I.μ) (s : Side) : List (Vector K 16) :=
  I.flushTuples q s ++ I.boundaryTuples q s
```

LeanerVM/Protocol/Spine/Instance.lean:185-186 at b435631

which is immaterial to **Balanced** (a permutation) and a trap for whoever derives the leaf layout from it. This is the same kind of finding as the status file’s finding on the tie order of the witness stack (F17).

Proposed change. A new row of the pinned conventions: “Leaf stacks | Per side, the blocks in index order are the boundary blocks of that side in the order of **I.boundary** (state, memory, bytecode for leanVM), then for each table in order its flushes of that side in order; the count side has, for each table in order, one block per count column in the order of **I.counts**. Stacked largest first at aligned offsets, ties by that index, no floor on the depth (layout.rs:354-415, leaf.rs:149-156). The depth of the three trees is the push side’s.” And a guard in Layer 6’s tests on sizes with $\tau_j = \kappa_{\text{mem}}$.

Finding 12 (minor). Orders the blueprint leaves open: the two roots, the boundary evaluations, the linear claims

Classification. An error of the blueprint (omissions of transcribed conventions).

Evidence and proposed change. Each a clause to add to Layer 6:

- the roots message is (R, R_c) , the bus root first (gkr.rs:274-275, py:434), against the specification’s prose order;
- the boundary evaluations are sent in the order in which the sides (push, pull), their blocks and their coordinates first name a committed column, a column already valued at the same point being skipped (leaf.rs:428-439, 466-471); for leanVM mem_0, mem_1, mem_2, cntfin_mem, cntfin_bc;
- BusOut.linear lists the zerocheck claims table by table, then push, pull, count, which is the order the table phase gives its powers of ξ (bp:323; dossier gt-bus.md, section E.1);
- $\zeta = (u_0, u_1, \chi_0, \dots, \chi_{\mu-3})$ of the last layer (section 8.4.1.4).

Finding 13 (minor). The canonical-encoding checks and the consumption check are absent from the blueprint

Classification. An error of the blueprint (omission).

Evidence. The checks of the upper limbs of the announced sizes (B2), of the third limb of the root’s

halves (B10) and of the consumption of the stream (B17), in table 14.1. No passage of the blueprint concerns canonical encodings or the consumption of the stream: a search for “canonical” finds only `bp:1294`, the public word of acceptance test 10; “upper limb” finds nothing; “consumed” occurs only in the sense of “used by” (for example `bp:129, 146, 262`). (The citation check corrected the wording: the dossier said that the search for the three words finds only the public word of acceptance test 10.) The status lists the Rust’s rejections as “four predicates plus truncations” (`st:319-320`), without `NonCanonicalEncoding` and `NotFullyConsumed`. None protects soundness; without them `verify` accepts streams the Rust rejects, and `verify` is specified as the verifier “the Rust prover’s proofs are checked against”. Layer 12’s six mutations (`bp:1238-1239`) exercise none.

Proposed change. Layer 12: list the three checks among `verify`’s, with their source lines; add three mutations (a nonzero upper limb in an announced size, a nonzero third limb in a half of the root, one element appended to the stream).

Finding 14 (minor). Two side conditions a bus phase over an abstract instance needs

Classification. An error of the blueprint (the spine’s instance admits instances no bus phase of the deployed shape serves; leanVM’s is not one).

Evidence. (The dossier’s probe on the spine, section H.2, parts 1 and 2. The citation check could not re-run it, Lean not running in the checkout since the upgrade; the probe’s source is identical to the dossier’s reproduction.)

1. With $d = 0$ and a count column, `M3Holds` is inhabited and the count form’s one polynomial, X_c , has total degree 1: a statement carrying it is outside `Seam.bus`, whose second conjunct is `t.poly.totalDegree ≤ I.d` (`Seams.lean:176`). Completeness of a bus phase that emits the count form fails on that instance.
2. A table with a constraint and no flush, no count column and no boundary block of its height is not on the bus: the trees have depth 0, ζ has no coordinate, and the table’s zerocheck point needs τ_j . The Rust guards the inequality at the start of the table sumcheck (`constraints.rs:250-253`).

Proposed change. Either two fields of `M3Instance`, `one_le_d : 1 ≤ d` and `constrained_on_bus : ∀ j, constraints j ≠ [] →  $\tau_j ≤ \mu_{bus}$` , or two hypotheses of the bus phase, stated in Layer 6; and acceptance test 28 to name the first.

Finding 15 (minor). The sketch of Layer 6 and the tracker’s bus section are stale, and the invariant they name is not one

Classification. An error of the blueprint (text the spine superseded).

Evidence. `bp:954-979` has `StmtIn := Unit`, a `BusOut` with fields ζ , `rem`, `pool`, α , β , a `relOut` with “count root $\neq 0$ ” and without the zerocheck claims, the public lines and `aux`, and a `LeafLayout` `prog s` that nothing defines; the spine has `I.Stmt`, `BusOut` with `linear` and `columns`, and `Seam.bus`. The tracker’s hole for the bus phase (P1) has `Phase.Def I Unit (BusOut I) (busSpec I) (M3Holds I) (Seam.bus I)` and cites “specification (5.4)”, which does not exist (`st:303`). `bp:975-977`: “The knowledge state function’s invariant after (α, β) is ‘the pushed and pulled multisets of q differ, or some count is zero, or a boundary claim is false’”. After (α, β) the first clause must be “the two products at (α, β) differ”: if the multisets differ and the products collide, the prover is honest from there on and no later challenge can be charged. And no boundary claim exists before the last message.

Proposed change. Replace the code block of Layer 6 by the slot’s types (dossier `gt-bus.md`, section C) and the schedule `V_to_P ( $\alpha, \beta$ ) ; P_to_V (R, R_c) ; the GKR’s ; P_to_V` the boundary evaluations; replace the sentence on the invariant by the table of the dossier’s section D.3.

Finding 16 (minor). The binary layer’s combination challenge has no error assigned

Classification. An error of the blueprint (omission).

Evidence. `gkrError` (`bp:941-942`) names combiners, rounds and pairs; for odd μ the first layer has one combination challenge (`gkr.rs:387`), whose error is $1/|\mathbb{E}|$.

Proposed change. “ $1/|\mathbb{E}|$ to each combination challenge (two per radix-4 layer, one for the binary layer)”.

Finding 17 (minor). The status file’s finding on the one bus root misreads the pinned specification

Classification. An error of the status.

Evidence. The status file’s finding on the specification’s listing of the bus roots (S11), `st:289-290`: “§8.5 lists the bus roots as ‘the count root R_c and one bus root R ’ but does not say the push and pull roots are one scalar”. It does: “one R for both sides is the balance check” (`08:69`) and “the one bus root for both sides” (`08:100`). The status file’s finding “one root” (F3) is an agreement of the three sources, not a divergence.

Proposed change. Delete the finding on the listing of the bus roots (S11); reword the finding “one root” (F3) as a convention.

Finding 18 (minor). Citation drift

Evidence. `bp:197` cites `leaf.rs:664-668` for the count blocks: those lines are the signature of `prove_balance`; the count side is `leaf.rs:606-622`. `bp:601` has “Lemma 5.1” for the multiset lemma, which is Lemma 5.2 (Theorem 5.1 is the product check, `05:36-48`), as Layer 5 itself writes. The tracker’s hole for the bus phase (P1) has “specification (5.4)”.

Finding 19 (note). Disagreements between the three sources (seven, none a divergence of the two verifiers)

The blueprint follows the Rust on each, rightly. The seven are items 1 to 7 of table 8.5: the announced rate; the order of the roots; the seed (the status file’s finding on the seed, F14); the checks of the verifiers the specification does not mention; the combiner after the last layer (finding 6); the program as a stacked table in the Python verifier; the fingerprint bound, the count columns and the Rust’s layout assertions. (Item 8 of the table was added by the citation check.)

Finding 20 (note). The table phase must know which tables it opens

For the dossier on the table sumcheck. The deployed sumcheck has $\tau_{\max} = \max$ over the six tables (`constraints.rs:250`, `py:607`) and opens every column of those six and no other (`mod.rs:344-351`). In the instance the shared columns are tables too, of log-height $\kappa_{\text{mem}} \geq 16$; a table phase taking its number of rounds from every table of **I** has a different transcript. The criterion “has a constraint, a flush or a count column” is derivable and must be written down.

Finding 21 (note). The specification gives no round-by-round analysis of the bus phase

(Dossier `gt-bus.md`, section D.1.) `gkrError` and the zerocheck charge are the blueprint’s own analysis; the specification cannot be cited for them, and the Rust asserts only a coarse sum.

The review’s verdict on the GKR round error. Two dossiers charged a radix-4 round differently: $4/|\mathbb{E}|$ (the bus dossier, from the deployed cofactor of degree 4) and $5/|\mathbb{E}|$ (the table-sumcheck dossier, taking the blueprint’s number when it argued that the zerocheck conjunct adds nothing). The orchestrator read the verifier loop (`crates/lean_vm/src/gkr.rs:399-413`): the round message is five coefficients of which four travel, the first derived through the equality factor from the running claim, and the layer’s final check has no equality factor. A wrong running claim makes the prover’s cofactor differ from the true one, both of degree 4, so the next claim is right at four challenges at most: $4/|\mathbb{E}|$ is the deployed error, the blueprint’s $5/|\mathbb{E}|$ is a loose bound that also describes a different, non-normalized protocol, and the second dossier’s argument stands with $4/|\mathbb{E}|$ in place of $5/|\mathbb{E}|$.

8.5 The table sumcheck and the public input

8.5.1 The table sumcheck and the public input: the transcript as deployed

Short forms used in this section. The number after the colon is the line, at leanVM’s pin `a386121f`; a bare `:N` is a line of the file last named.

03:, 05:, 07:, 08: doc/leanvm/body/03-proving-primitives.tex, 05-arithmetization.tex,
 07-instruction-tables.tex, 08-end-to-end-protocol.tex
 Rust files under crates/lean_vm/src/, except the following
 transcript.rs crates/flat_shamir/src/transcript.rs
 stack_open.rs crates/pcs/src/stack_open.rs
 primitives/src/... crates/primitives/src/...
 py: python-verifier/verifier.py
 aggregate.py crates/rec_aggregation/guests/aggregate.py, the recursion guest,
 a third executable verifier

8.5.1.1 Notation

$\mathbb{K} = \text{GF}(2^{64})$, $\mathbb{E} = \text{GF}(2^{192}) = \mathbb{K}[y]/(y^3 + y + 1)$. Six opcode tables $t = 0..5$ in the order XOR, MUL_NATIVE, SET_CONSTANT, DEREf, JUMP, BLAKE2S, of log-heights τ_t and widths $n_t = 15, 15, 8, 15, 14, 37$ (sum **104**). $\tau_{\max} = \max_t \tau_t$ (at least 3: the BLAKE2S table has $\tau_5 \geq 3$). $B = \Sigma_t(\text{number of constraints of } t) = 2$ (JUMP's two; every other table has none). Three *sides* $s = 0, 1, 2$: push, pull, count. $\zeta \in \mathbb{E}^{\mu_{\text{bus}}}$ is the terminal point of the bus phase's GKR. totals_s (the specification's rem_s) is what the tables owe side s . B_t^s is table t 's bus form for side s , a polynomial of degree at most 2 in the columns of t with coefficients in $\mathbb{E}$.

Widths, from tables.rs: arith::N = 15 (:455), set::N = 8 (:537), deref::N = 15 (:608), jump::N = 14 (:711), blake2st::N = 37 (:862); Python TABLE_WIDTHS (py:823-834, 852); specification §7 column lists (07:12, 30, 58, 78, 99, 122), which give the same counts (BLAKE2S: 19 stacked columns plus the 18 limbs).

8.5.1.2 The table sumcheck

Table 8.6: The table sumcheck, step by step, in the three sources.

#	Step	Specification	Rust	Python
T0	What precedes: the bus phase's last prover message is the five boundary evaluations ($\text{mem}_0, \text{mem}_1, \text{mem}_2, \text{cntfin_mem}$ at $\zeta_{<\kappa_{\text{mem}}}$; cntfin_bc at $\zeta_{<\kappa_{\text{bc}}}$)	08:70 “For their committed columns the prover sends one evaluation each”	leaf.rs:909-920 (decompose_verify), order fixed by cpu/layout.rs:361-395	py:552-557
T1	Challenge $\xi \leftarrow \mathbb{E}$ (one element), drawn after the boundary evaluations (T0)	08:75 “Samples and sends the batch challenge $\xi \in \mathbb{E}$ ”; 05:145	cpu/mod.rs:726 let $\text{zc_xi} = \text{vs.sample}()$; (prover :590)	py:1389
T2	Powers. Constraint i (0-based) of table t takes ξ^{o_t+i} , o_t the number of constraints of the tables before t ; side s takes ξ^{B+s} , shared by every table. For leanVM: $\xi^0 \rightarrow b + v_{\text{cond}} \cdot w$, $\xi^1 \rightarrow v_{\text{cond}} \cdot (b + 1)$, $\xi^2, \xi^3, \xi^4 \rightarrow$ push, pull, count	05:145; 08:75 “a disjoint range per table; the last ν_{side} weight the bus forms, one per side and shared by every table”	constraints.rs:74-82 (xi_offsets), :254-255, 281; cpu/mod.rs:413-422 (xi_form_base, xi_form_pows); tables.rs:70-74	py:1390-1392, :621-623, :798-800
T3	Target, derived, never sent: $T = \Sigma_s \xi^{B+s} \cdot \text{totals}_s$	05:151-155 “the verifier computing the right side itself”; 08:76 “against the target the verifier derived from the three leaf claims”	cpu/mod.rs:735	py:1393
T4	Number of rounds $\tau_{\max}$, the maximum over the six opcode tables	05:145, 155; 08:76	constraints.rs:250 airs.iter().map(a a.tau).max()	py:607 max(table_log_heights)
T5	Round $j = 0.. \tau_{\max} - 1$ binds variable $m = \tau_{\max} - 1 - j$ (highest first). Prover message: three elements of $\mathbb{E}$, the coefficients c_0, c_2, c_3 of the cubic h_j , in that order	05:155 “Rounds bind $X_{\tau_{\max}-1}$ first”; 08:76 “the round check pins one of the four coefficients (§3), so a round costs three $\mathbb{E}$ -elements” (which coefficient: not said)	prover constraints.rs:164, 191-194 (add_round_poly(&h, false)); encoding transcript.rs:63-71 (fixed = 1)	py:409 “The transcript contains c_0, c_2, c_3 ”

#	Step	Specification	Rust	Python
T6	Verifier computation: $c_1 := \text{claim} + c_2 + c_3$, so that $h_j(0) + h_j(1) = \text{claim}$ holds by construction. No rejection path	03:84 states it as a check, 03:110 as the reason one coefficient is not sent	transcript.rs:289-309; constraints.rs:267	py:409-410
T7	Challenge $r_m \leftarrow \mathbb{E}$ after the round message; $\text{claim} := h_j(r_m)$	03:84	constraints.rs:268-270	py:424-426
T8	Joining and padding. Table t is active in the round binding m iff $\tau_t > m$, that is from round $\tau_{\max} - \tau_t$ on. Verifier's weight: $w_t := \prod_{m < \tau_t} (1 + \zeta_m + r_m) \cdot \prod_{\tau_t \leq m < \tau_{\max}} r_m$	05:146-150, 155	constraints.rs:271-274	py:609-614
T9	Final prover message: for $t = 0.5$ in order, n_t elements: the value of every column of t , the eighteen BLAKE2S limbs included, at $r_{<\tau_t} = (r_0, \dots, r_{\tau_t-1})$. 104 elements, no challenge in between	08:77 “The prover sends one value per column of every table”	constraints.rs:224-238 (prover), :279-280 (verifier); cpu/mod.rs:378 n_cols: table.n_committed_columns()	py:619-620
T10	Final check: $\sum_t w_t \cdot (\sum_i \xi^{o_t+i} \cdot C_{t,i}(e_t) + \sum_s \xi^{B+s} \cdot B_t^s(e_t)) = \text{claim}$, else reject	08:77 “checks that they reproduce the sumcheck’s final value”	constraints.rs:277-290 (Error::FinalMismatch); summand cpu/mod.rs:380-383	py:616-625
T11	Claims pooled: for t , then for local column c : column $\text{base}_t + c$, point $r_{<\tau_t}$, value $e_{t,c}$	08:77 “the column values then enter the claim pool, T_j ’s at $\rho_{<\tau_j}$ ”	cpu/mod.rs:428-441 (constraint_claims)	py:624
T12	The eighteen limbs: their claims are in the pool at their place (BLAKE2S local columns 9 to 26) and are <i>located</i> on q_{flock} : low 8 coordinates frozen to the slot’s bits, then the point, then q_{flock} ’s selector	08:77 “live in q_{flock} , not the stack, so their claims route there”; 07:120-122	cpu/mod.rs:447-453, 790-814 (SlotClaim::Strided); tables.rs:394-413; hash_flock.rs:96-115	py:847-850, 886-894; :295-297

Phase	Elements of $\mathbb{E}$ sent by the prover	Challenges drawn by the verifier
table sumcheck	$3 \cdot \tau_{\max} + 104$	$1 + \tau_{\max}$
public input	2 (step P2)	1 (step P1)

Table 8.7: Counts of the two phases.

Verbatim, the Rust verifier’s loop and final check (constraints.rs:261-291):

```

let mut claim = target;
let mut chi = vec![F192::ZERO; n];
for j in 0..n {
    let m = n - 1 - j;
    // `h(0)` is derived from the running claim rather than transmitted, so
    // the round-consistency check it used to enable holds by construction.
    let h = vs.next_round_poly(4, claim, None).map_err(|_| Error::Truncated)?;
    let rk = vs.sample();
    chi[m] = rk;
    claim = poly_eval(&h, rk);
    let eq_k = F192::ONE + zeta[m] + rk;
    for (t, air) in airs.iter().enumerate() {
        weights[t] *= if air.tau > m { eq_k } else { rk };
    }
}

let mut acc = F192::ZERO;
let mut claims = Vec::with_capacity(airs.len());
for (t, air) in airs.iter().enumerate() {
    let evals = vs.next_scalars(air.n_cols).map_err(|_| Error::Truncated)?;
    let w = &pows[offsets[t]..offsets[t] + air.n_constraints];
    acc += weights[t] * (air.eval)(w, &evals);
    claims.push(Claims {
        chi: chi[..air.tau].to_vec(),
        evals,
    });
}

```



```

    if acc != claim {
        return Err(Error::FinalMismatch);
    }
    Ok(claims)

```

crates/lean_vm/src/constraints.rs:261-291

and the decoding (transcript.rs:289-302): with `eq = None`, `fixed = 1`, the three transmitted coefficients are read into indices 0, 2, 3 and `coeffs[fixed] = claim + sum_from(2)`.

8.5.1.3 The public input

Table 8.8: The public-input phase, step by step, in the three sources.

#	Step	Specification	Rust	Python
P0	Precondition: each public word has top limb 0	08:29 “two 192-bit words (with top limb 0)”; no check stated	type [F192; 2]; read_public rejects half.c2 != 0 with CpuError::PublicInput (cpu/mod.rs:141-143)	by type: Digest is 32 bytes, halves() builds E(w0, w1) (py:204-219)
P1	Challenge $r \leftarrow \mathbb{E}$, after the 104 values of the final message (T9)	08:83	cpu/mod.rs:745 (prover :610)	py:1398
P2	Prover message: two elements c_0, c_1	08:29, 84	cpu/mod.rs:746-749 (prover :611-618)	py:1399
P3	Check	per limb: $c_\ell = (1 + r) \cdot \text{mem}[g^0]_\ell + r \cdot \text{mem}[g^1]_\ell$, $\ell \in \{0, 1\}$ (08:29-32)	combined: $c_0 + y \cdot c_1 = \text{interp}(\text{pi}_0, \text{pi}_1, r)$ (cpu/mod.rs:752-755)	combined, the same (py:1400)
P4	Claims pooled: $\text{mem}_0, \text{mem}_1, \text{mem}_2$ at $(r, 0, \dots, 0)$ with values $c_0, c_1, 0$; no scalar for the third	08:84 “pools them, with 0 for the top limb”	cpu/mod.rs:746 (array initialised to zero, two entries read), :674-682 (bind_pi_claim)	py:1399, 1401-1402

8.5.1.4 The claim pool, in order

Table 8.9: The order of the claim pool.

Rank	Claims	Number	Source
first power of λ	the ring-switched claim on q_{flock}	1	08:98; stack_open.rs:518-519; py:1413
then	bus: $\text{mem}_0, \text{mem}_1, \text{mem}_2, \text{cntfin_mem}$ at $\zeta_{<\kappa_{\text{mem}}}$, cntfin_bc at $\zeta_{<\kappa_{\text{bc}}}$	5	cpu/mod.rs:663; py:555-557, 1395
then	table sumcheck: the column claims (T11), of which 18 strided on q_{flock}	104	cpu/mod.rs:664; py:1395
then	public input: the pooled claims (P4)	3	cpu/mod.rs:665; py:1402

`finish_claims` (cpu/mod.rs:656-667) is the one place the order is fixed in the Rust. The pool has 112 point claims and the batch 113 powers of λ .

8.5.1.5 The public-input check, verbatim in the four sources

Specification (08:29-33):

```

The public input is the first two memory cells  $\text{mem}[\text{gen}^{\{0\}}], \text{mem}[\text{gen}^{\{1\}}]$ , two 192-bit words (with top
 $\hookrightarrow$  limb  $\$0$ ) both parties know. The verifier samples  $r_m \in \mathbb{E}$ , the prover sends  $\$c_0, \$c_1$  claiming
 $\hookrightarrow \$c_\ell = \text{mle}(\text{mem}_\ell)(r_m, 0, \dots, 0)$ , and the verifier checks
\l
    c_\ell \equiv \text{qqeq}; (1+r_m) \cdot \text{mem}[\text{gen}^{\{0\}}]_\ell + r_m \cdot \text{mem}[\text{gen}^{\{1\}}]_\ell \pmod{\ell} \in \{0, 1\}.
\l
Both sides are degree one in  $r_m$ , so a limb disagreeing with the public words survives with probability at
 $\hookrightarrow$  most  $\$1/|\mathbb{E}|$  (Lemma-\ref{lem:sz}). The claims  $\$c_0, \$c_1$  are discharged to the PCS.

```

doc/leanvm/body/08-end-to-end-protocol.tex:29-33

Rust (cpu/mod.rs:745-756):

```

let r_pi = vs.sample();
let mut pi_limbs = [F192::ZERO; 3];
for v in &mut pi_limbs[..2] {
    *v = vs.next_scalar().map_err(CpuError::Transcript)?;
}
// The two claimed evaluations must sit on the public-input line, the top
// limb's being zero ($sec:e2e-pi).
let want = primitives::multilinear::interp(l.pi[0], l.pi[1], r_pi);
if pi_limbs[0] + F192::Y * pi_limbs[1] != want {
    return Err(CpuError::PublicInput);
}
let slots = finish_claims(&l, bus.claims, &table_claims, r_pi, pi_limbs);

```

crates/lean_vm/src/cpu/mod.rs:745-756

with $\text{interp}(lo, hi, t) = lo + t * (lo + hi)$ (primitives/src/multilinear.rs:39-41) and $Y = (0, 1, 0)$ (primitives/src/field/gf2_64x3.rs:44).

Python (py:1398-1402):

```

public_challenge = transcript.sample()
public_limbs = (*transcript.next_scalars(2), ZERO)
require(poly_eval(public_limbs, Y) == multilinear_eval(public_input.halves(), [public_challenge]), "public
↳ input check failed")
public_point = (public_challenge, *[ZERO] * (layout.placements[MEMORY_0].variables - 1))
claims.extend(ColumnClaim(column, public_point, value) for column, value in zip((MEMORY_0, MEMORY_1,
↳ MEMORY_2), public_limbs))

```

python-verifier/verifier.py:1398-1402

Recursion guest (aggregate.py:1672-1688):

```

# ---- public-input binding claim: MEM as ONE logical E-column ----
# The VM's bind_pi_claim makes a SINGLE E-claim at [rm, 0..]:
# MEM(rm) = interp(pi_0, pi_1, rm) = pi_0 + rm*(pi_0 + pi_1)
# over the E-valued public input (no lane splitting, no Frobenius). Both
# public words have a zero top limb, so that limb's evaluation is zero at
# every rm; only the two low ones ride the stream and must reassemble it:
# MEM = v_lo + Y*v_hi (doc sec:e2e-pi).
fs, rm = squeeze(fs)
mem = pi_0 + rm * (pi_0 + pi_1)
fs, mem_lo, cursor = fs_next(fs, cursor)
fs, mem_hi, cursor = fs_next(fs, cursor)
assert mem == mem_lo + mem_hi * Y_TOWER
claim_pool[GEN ** claim_idx] = mem_lo
claim_idx += 1
claim_pool[GEN ** claim_idx] = mem_hi
claim_idx += 1
claim_pool[GEN ** claim_idx] = 0

```

crates/rec_aggregation/guests/aggregate.py:1672-1688

Which transcripts each accepts. Write $w_i = w_{i,0} + y \cdot w_{i,1}$ for the public words and $L_\ell(r) = (1+r) \cdot w_{0,\ell} + r \cdot w_{1,\ell}$, elements of $\mathbb{E}$. Then $\text{interp}(w_0, w_1, r) = L_0(r) + y \cdot L_1(r)$.

	Accepts (r, c_0, c_1) iff	Pairs per r
Specification	$c_0 = L_0(r)$ and $c_1 = L_1(r)$	1
Rust, Python, guest	$c_0 + y \cdot c_1 = L_0(r) + y \cdot L_1(r)$	$ \mathbb{E} = 2^{192}$: the pairs $(L_0(r) + y \cdot \delta, L_1(r) + \delta)$, $\delta \in \mathbb{E}$

Table 8.10: The transcripts the per-limb check and the combined check accept.

The first implies the second. Not conversely: c_0, c_1 are elements of $\mathbb{E}$, so the independence of $1, y$ over $\mathbb{K}$ says nothing of them.

With the pooled claims true of a committed memory. Let the cells differ from the public words by a_ℓ (cell 0) and c_ℓ (cell 1), in $\mathbb{K}$. The true evaluations pass

- the per-limb check iff $(1+r) \cdot a_\ell + r \cdot c_\ell = 0$ for $\ell = 0$ and $\ell = 1$;
- the combined check iff $(1+r) \cdot A + r \cdot C = 0$, $A = a_0 + y \cdot a_1$, $C = c_0 + y \cdot c_1$.

If the memory is wrong, $(A, C) \neq (0, 0)$ because $1, y$ are independent over $\mathbb{K}$, and $A + r \cdot (A + C) = 0$ has at most one solution ($A + C = 0$ forces $A = 0$, then $C = 0$). So the combined check has knowledge error $1/|\mathbb{E}|$ too. The bad challenges differ: with $a_0 = 1, c_0 = 0, a_1 = 1, c_1 = 1 + g$ the per-limb check has none (limb 0 needs $r = 1$, limb 1 needs $r = g^{-1}$) and the combined check has $r = (1 + y)/(1 + y \cdot g)$. The status’s example (zero words, limbs $[0, 1]$ and $[1, 0]$, $r = y/(1 + y)$) is checked by `tests/LeanerVMTests/Protocol/PublicInput.lean:182-210` (leanerVM **b435631**); the dossier’s author re-derived it by hand.

Is the Rust’s public input two words of $\mathbb{E}$ or of 128 bits? By type two words of $\mathbb{E}$ (`[F192; 2]`, `Layout.pi`, `cpu/layout.rs:83`). By what the verifier accepts, two words of 128 bits: `read_public` rejects a nonzero top limb, and the transcript is seeded with four lanes.

Which is the deployed protocol. The combined check. Three executable verifiers at the pin run it, the recursion guest among them, and the guest’s comment states the intent (“MEM as ONE logical E-column”). The specification’s §8.2 is the outlier. An honest Rust proof passes both: the Rust prover sends $L_0(r), L_1(r)$ (`cpu/mod.rs:611-613`).

8.5.2 The table sumcheck and the public input: disagreements between the three sources

Short forms used in this section. The number after the colon is the line, at leanVM’s pin **a386121f**; a bare `:N` is a line of the file last named.

```
03:, 05:, 07:, 08:      doc/leanvm/body/03-proving-primitives.tex, 05-arithmetization.tex,
                        07-instruction-tables.tex, 08-end-to-end-protocol.tex
Rust files              under crates/lean_vm/src/, except the following
transcript.rs          crates/flat_shamir/src/transcript.rs
py:                    python-verifier/verifier.py
leanisa-blueprint.md    docs/roadmap/leanisa-blueprint.md, leanerVM at b435631
```

The step numbers refer to tables 8.6 and 8.8.

Table 8.11: Disagreements between the specification, the Rust and the Python on the table sumcheck and the public input.

Subject	Disagreement	Which the blueprint follows
The public-input check (P3)	The specification checks two equations; the Rust, the Python and the recursion guest check one (section 8.5.1.5)	the specification
The round consistency equation (T6)	The specification lists “every sumcheck’s rounds” among the checks (08:100); in both verifiers it holds by construction and cannot reject	neither: its oracle protocol sends four coefficients and checks (dossier gt-table-pub.md , section 4, item 7)
Which coefficient is omitted (T5)	The specification does not say; both verifiers omit the linear coefficient c_1	the verifiers (acceptance test 8)
The batching error	The specification charges $(\nu_{\text{side}} + B)/ \mathbb{E} $ (05:155); its own Corollary 3.9 (cor:idtest , 03:67) gives $(\nu_{\text{side}} + B - 1)/ \mathbb{E} $. The code states none. (The citation check corrected the number of the corollary, which the dossier gave as 3.7.)	neither: $(B + 2)/ \mathbb{E} $, the tight value

Subject	Disagreement	Which the blueprint follows
Order of JUMP's columns	The specification lists w, b before the counts (07:99); both verifiers put them last (tables.rs:689-711; py:826). The listing is not presented as an order	the verifiers, through leanISA (leanisa-blueprint.md:612-613: “Row is the table’s column list in the Rust’s order”; the citation check corrected the lines, which the dossier gave as 611-612)
Guard $\text{zeta.len()} < n$	Rust only (constraints.rs:251-253). Unreachable: every table has at least three push blocks, so $\mu_{\text{bus}} \geq \tau_{\text{max}} + 1$	not modelled; nothing to model
Comments in the Rust	constraints.rs:24 says the round polynomial “is sent WHOLE, at four nodes”; :265-266 says “ $h(0)$ is derived”. The code sends three <i>coefficients</i> and derives c_1 ; $h(0) = c_0$ is transmitted (transcript.rs:63-71). The citation check found two more stale comments: constraints.rs:26 says the verifier checks “ $h(0) + h(1) = \text{claim}$, then interpolate at the challenge” (there is neither a check nor an interpolation: coefficients are read and one is derived); :257-260 says “Each round arrives as the round polynomial itself at nd ” (it arrives as three coefficients, transcript.rs:289-302)	the code; but the status repeats “four nodes” (finding 29)

No other disagreement found. Checked and agreeing: the moment of every challenge; the power assignment; the three sides and their order (leaf.rs:610-622; py:570-574); the variable order and the weights; the order of the 104 values (column lists compared entry by entry, tables.rs:436-863 against py:823-834); the slot of each limb (hash_flock.rs:85-115 against py:847-850: cv 0 to 3, out 4 to 7, m 10 to 17, metadata 18, 19); the pool order.

8.5.3 The table sumcheck and the public input: findings

Short forms used in this section. The number after the colon is the line; leanVM files are at the pin a386121f, leanerVM files at b435631; a bare :N is a line of the file last named.

bp: docs/roadmap/protocol-blueprint.md (leanerVM)
 Seams.lean, Compose.lean LeanerVM/Protocol/Spine/Seams.lean, .../Spine/Compose.lean (leanerVM)
 PublicInput.lean LeanerVM/Protocol/PublicInput.lean (leanerVM)
 03:, 05:, 07:, 08: doc/leanvm/body/03-proving-primitives.tex, 05-arithmetization.tex, 07-instruction-tables.tex, 08-end-to-end-protocol.tex
 Rust files under crates/lean_vm/src/, except the following
 transcript.rs crates/fiat_shamir/src/transcript.rs
 primitives/src/... crates/primitives/src/...
 py: python-verifier/verifier.py
 aggregate.py crates/rec_aggregation/guests/aggregate.py, the recursion guest

Blueprint lines are at b435631. (The citation check noted that at the merged HEAD, 8d3ea7d, every blueprint line cited here is 4 higher: the blueprint gained a four-line paragraph after line 51 and a rewritten conventions row *Module system*.) The step numbers (T0 to T12, P0 to P4) refer to tables 8.6 and 8.8.

Finding 22 (major). The bus seam admits statements the deployed table sumcheck cannot serve

Evidence. The dossier’s section 6 (E.1), its probes (section 9) and ArkLib’s definition of round-by-round knowledge soundness (rbrKnowledgeSoundnessWorstCaseWith, RoundByRound.lean:553-568, ArkLib dca90385), which quantifies over every input statement. The seam as built:

```
def bus := of I fun (s : I.Stmt × BusOut I) q ↦ (∀ c ∈ s.2.linear, c.Holds q) ∧
  (∀ c ∈ s.2.linear, ∀ t ∈ c.terms, t.poly.totalDegree ≤ I.d) ∧
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q
```

LeanerVM/Protocol/Spine/Seams.lean:175-177 (b435631)

The consequence the dossier uses: if for some statement outside the input seam a prover makes the verifier accept with an output inside the output seam with probability p , then $\Sigma_i \text{err } i \geq p$. The

deployed verifier takes one point ζ , one form per side and table, three values, and reads the constraints off the instance. **Seam.bus** admits four things it does not handle.

- (a) *A clause on the statement alone, which the verifier must then check.* The degree clause does not mention q . Probe: on the toy, the stack **q0** with columns $[1, 1], [1, 1], [0, 0]$ satisfies **M3Holds toy 0**; the statement with the single claim “the term X_2^3 at the point 0 sums to 0” is true of **q0**, its public line holds, and it is outside **Seam.bus toy** by the degree clause alone. On that statement the summand is identically zero: a verifier that does not inspect degrees accepts the all-zero run with true column claims, with probability 1. So a table verifier must compute **totalDegree** of every term it is handed and reject above **I.d**. The deployed verifiers have no such check and no such input. In the composed protocol the guard never fires.
- (b) *Any number of linear claims: the error on ξ is unbounded over the seam.* Take k claims, each one term with the constant polynomial 1 (degree 0), weight 1: true sum 1. Give claim c the value $1 + \delta_c$ with $\sum \delta_c X^c = \prod_{i < k-1} (X + a_i)$ for distinct $a_i \in \mathbb{E}$. The statement is outside the seam, and at each of the $k - 1$ roots the batched target is the true batched sum, after which the honest prover wins. So $\text{err } \xi \geq (k - 1)/|\mathbb{E}|$ for every k the verifier accepts. With $\text{err } \xi = (B + 2)/|\mathbb{E}|$ the verifier must reject more than $B + 3$ claims. The probe exhibits a list of 1000 true claims on the toy, which has one constraint and one flush.
- (c) *Terms of one table at unrelated points.* The probe’s claim **twoPoints** has two terms of table 0, at the points 0 and 1, and is inside the seam (each clause evaluated). The deployed verifier computes one factor $\prod (1 + \zeta_m + r_m)$ per table. Completeness on **twoPoints** needs one factor per term.
- (d) *Terms on any table of the instance.* With Layer 3’s instance (finding 23), a term may sit on the memory’s table, of log-height $\kappa_{\text{mem}} \geq 16$. A sumcheck of τ_{max} rounds, τ_{max} the maximum over the opcode tables, cannot fold it when $\tau_{\text{max}} < \kappa_{\text{mem}}$.

(The citation check could not re-run the two Lean probes: no Lean may be run in the merged checkout. It found the probe sources reproduced in the dossier byte-identical to **probes/gt-table-pub/SeamBusShape.lean** and **SeamBusMember.lean**; they use only the numerals 0 and 1 in $\mathbb{K}$, so the change of meaning of numerals at the new CompPoly pin does not affect them; whether they still elaborate on the new pins is unknown.)

Classification. An error of the blueprint. The seam is more general than any phase with leanVM’s transcript and leanVM’s checks can serve. **Phase.Complete** and **Phase.Security** for the deployed verifier against **Seam.bus** as built cannot be proved: they need guards (degree, number of claims, one point per table, tables of the sumcheck) that the deployed verifier has not.

Proposed change.

Proposed change: the bus seam takes leanVM's shape

As it stands. (bp:481, bp:487-489, bp:560-563, bp:565-574; Seams.lean:71-95, 130-134, 175-177) BusOut I is linear : List (LinearClaim I), columns; “The seams carry *claims*, not challenges ... which is what lets the table sumcheck consume any list of linear claims without knowing the bus.”

As proposed. The statement has leanVM's shape, the Rust's BusVerify (finding 27).

```

/-- A polynomial of the row of table `j` within the degree bound. -/
abbrev M3Instance.RowPoly (I) (j) := {p : CMvPolynomial (I.width j) K // p.totalDegree ≤ I.d}
/-- A bus form of table `j`: a combination of row polynomials with weights in `E`. -/
abbrev M3Instance.Form (I) (j) := List (E × I.RowPoly j)
structure BusOut (I : M3Instance) where
  point   : Vector E I.τmax -- ζ_{<τ_max}
  forms   : Fin 3 → (j : I.SumcheckTable) → I.Form j -- push, pull, count
  totals  : Fin 3 → E
  columns : List (ColumnClaim I)

```

Seam.bus: every constraint of every sumcheck table has extension zero at $point_{<\tau_j}$; for every side, the forms' extensions at $point_{<\tau_j}$ sum to **totals**; every column claim holds; the public lines; **aux**. The prose becomes: “The bus seam carries the point the bus phase ended at, since the table sumcheck runs at it (§5.5); α, β and the leaf layout stay inside the bus phase, which hands on the forms it built.”

Reason. The phase is then leanVM's, with error $(B + 2)/|E|$ for B read off I , and its verifier has no check leanVM has not. The degree bound moves from a clause to a type.

Alternative, smaller: keep the types and add to **Seam.bus** the clauses “at most $B + 3$ linear claims”, “the terms of a table share their point and the points are prefixes of one vector”, “terms sit on sumcheck tables”; state in Layer 7 that the verifier checks them and the degree, and in Layer 12 that the four guards are dead in the composition.

Finding 23 (major). The table sumcheck's tables are not distinguished from the instance's other tables

Evidence. Blueprint bp:836-839, “The stack columns that belong to no opcode table (`mem_0`, `mem_1`, `mem_2`, `cntfin_mem`, `cntfin_bc`, `q_flock` ...) are tables of the instance with no constraints and no flushes”; Layer 7 bp:987-989, $\tau_{\max}$ and $\sum_j width_j$ over the instance; ground truth: the number of rounds (T4) and the final message (T9). With $\kappa_{\text{mem}} \geq 16$ every leanISA instance would have at least 16 rounds and 110 final values; leanVM has $\tau_{\max}$ rounds (as few as 3) and 104. **Unverified:** `leanIsaInstance` is not built; this is a reading of the blueprint's text.

Classification. An error of the blueprint (an omission).

Proposed change.

Proposed change: say which tables the sumcheck ranges over

As it stands. (row *Table sumcheck*, bp:323) “Variables bound highest first; table j joins at round $\tau_{\max} - \tau_j$; ...”

As proposed. Prepend “The sumcheck ranges over the tables that have a constraint, a flush or a count column (`I.SumcheckTable`; for leanISA the six opcode tables, never the tables of the shared columns); $\tau_{\max}$ is the largest log-height among them and the final message has one value per column of each, in table order then column order.” Add `SumcheckTable` (a decidable predicate on `Fin I.ntab`, derived from `constraints`, `flushes`, `counts`) to the spine, and an acceptance test: on an instance with a taller table of no constraint and no flush, the number of rounds is the smaller height.

Reason. The schedule is Category B; a longer one rejects every Rust proof.

Finding 24 (major). The public-input phase proves a check no executable verifier runs, and the debt is assigned to a layer that cannot pay it

Evidence. The ground truth of the public-input check (section 8.5.1.5). New to the record: the recursion guest has the combined check too (`aggregate.py:1683`).

Classification. A deliberate deviation (decision 15; the status file’s finding on the combined public-input check (F18)). It owes a theorem: round-by-round knowledge soundness of the verifier that checks the combined equation and pools the scalars sent. The blueprint says “Layer 12 owes it” (`bp:1062`). Layer 12 is the compilation. The combined check is a different *oracle verifier*; `verify_knowledgeSound` is derived from the oracle protocol `verify` compiles; so either `verify` has the per-limb check and is not the deployed verifier (it rejects proofs the Rust accepts), or it has the combined check and the oracle protocol must have it, with its `Phase.Security`.

Proposed change.

Proposed change: a second public-input phase for the deployed check

As it stands. (`bp:1060-1062`) “So the pinned verifiers accept transcripts this verifier rejects, their knowledge soundness at $1/|\mathbb{E}|$ is a lemma of its own, and Layer 12 owes it.”

As proposed. “So the pinned verifiers accept transcripts this verifier rejects. Layer 8 has a second phase for them, `publicInputPhaseWords`: the same schedule; the check $\Sigma_\ell y^\ell \cdot c_\ell = \Sigma_\ell y^\ell \cdot \text{lineValue}_\ell(r)$ over the lines whose value is sent, at most three; the claims pooled at the values *sent*. Its `Phase.Security` from `Seam.table` to `Seam.pub` has error $1/|\mathbb{E}|$: the state after the challenge is ‘the true evaluations pass the check and every pooled claim holds’, and a wrong memory gives $A + r \cdot (A + C) = 0$ with $(A, C) \neq (0, 0)$ by the independence of $1, y, y^2$ over $\mathbb{K}$. The leanISA protocol and `verify` use this phase; the per-limb phase is kept as the specification’s.”

Reason. Three verifiers against one paragraph; the master theorem must be about the verifier that is deployed. The discrepancy is also a finding against the specification, to report to leanVM when reporting is allowed.

Finding 25 (minor). The round message: four coefficients and a check in the oracle protocol, three and none on the wire

Evidence. The round message and the derivation of c_1 (T5, T6); blueprint `bp:315`.

Classification. A deliberate deviation. It owes a lemma the blueprint does not state. “Inverted by the running claim” holds only on messages that pass the round check; and ArkLib’s `Verifier.fiatShamir` of a protocol with four-coefficient messages derives the challenge from four coefficients, where leanVM’s chain absorbs three (`transcript.rs:303-307`). So `verify_iff_compiled` as sketched (`bp:1218-1221`) needs a challenge oracle defined on the *encoded* message, and `FiatShamirSecurity` must be stated for that oracle.

Proposed change. One of:

- make the wire’s message the oracle protocol’s (three coefficients, the verifier derives the fourth, no round check): the compilation is then literal and one check leaves the audit;
- or keep row *Sumcheck messages* and add: “Layer 4 proves the transport: if a verifier V reading messages m is round-by-round knowledge sound and dec is injective from wire messages onto the messages passing V ’s round check, commuting with the challenges, then $V \circ dec$ is, at the same error, with the state function $state \circ dec$.”

Finding 26 (minor). The zerocheck escape is over-charged, and charged in three different ways

Evidence. What is stated (dossier `gt-table-pub.md`, section 5, D.1). The specification gives no error for the recycled point: `05:132` says it is sound “as long as it results from uniform sampling occurring after the table columns are committed”. The blueprint charges it in three ways: row *Seams* (`bp:333`): “ $1/|\mathbb{E}|$ per coordinate per constraint”, in the bus phase; Layer 7 (`bp:1001-1003`): “charged to the (α, β) and GKR challenges of Layer 6 ... as an extra $\tau_{\max}/|\mathbb{E}|$ per constraint”; Layer 6 (`bp:973-975`): `busError` “is $4 \cdot 2^{\mu_{\text{bus}}}/|\mathbb{E}|$ on the (α, β) message plus `gkrError 3  $\mu_{\text{bus}}$` ”, no such term. Acceptance test 6 (`bp:1283`): “The state function of Layer 7 charges the zerocheck to Layer 6’s challenges”. The

tracker’s hole comment for the bus phase repeats “ $1/|E|$ per coordinate per constraint”. The recycled point, settled (dossier section 5, D.3). The last GKR layer is radix 4 ($\mu_{\text{bus}} \geq 16$); $\zeta = (\text{low}, \text{high}, x_0, \dots, x_{\mu_{\text{bus}}-3})$, so ζ_0, ζ_1 are the last coordinates of ζ drawn and $\zeta_{2+i} = x_i$. (The citation check corrected the dossier’s “are drawn last”: after *(low, high)* the verifier draws one more combiner, unused after the last layer, `gkr.rs:423, py:453`; it is not a coordinate of ζ , so the argument is unaffected.) Neither α nor β is a coordinate of ζ . If the zerocheck state Z (every constraint extension, with the coordinates of ζ drawn so far substituted, is identically zero) is false, some $\tilde{C}_{t,i}$ has a nonzero restriction; Z becomes true only if that one becomes zero: probability at most $1/|E|$ on x_i for $i + 2 < \tau_{\text{max}}$, zero on x_i for $i + 2 \geq \tau_{\text{max}}$ (no table has that coordinate), and at most $2/|E|$ on the pair *(low, high)*. **In total $\tau_{\text{max}}/|E|$, once for all constraints:** no union over the constraints is taken. In the bus phase’s state function, a conjunction $G \wedge Z$, the error of a challenge is $\max(\varepsilon_G, \varepsilon_Z)$, not the sum; on the rounds of the last layer $\varepsilon_G = 5/|E| \geq 1/|E|$, on the last pair $\varepsilon_G = 2/|E| \geq 2/|E|$. **The recycled point adds nothing to busError**, per challenge or in the sum, when the state function is the conjunction. Verdict on the blueprint: an upper bound, so not wrong; loose by the factor B and by the addition; attributed to (α, β) , which do not carry it; stated three ways. Under the spine it can only be the bus phase’s: a phase’s state function charges that phase’s challenges, so acceptance test 6’s “the state function of Layer 7” cannot be meant literally.

Proposed change.

Proposed change: one charge, in the bus phase’s state function

As it stands. (bp:333) “... is charged in the bus phase, coordinate by coordinate as ζ is drawn, $1/|E|$ per coordinate per constraint ...”

As proposed. “... is carried by the bus phase’s state function as one more conjunct, ‘every constraint extension, restricted to the coordinates of ζ drawn so far, is identically zero’, which a coordinate makes true with probability at most $1/|E|$, whatever the number of constraints. The coordinates of ζ are the challenges of the last GKR layer (its rounds, then its combination pair), whose own errors are at least that, so busError has no term for it.” Layer 7 (bp:1000-1003): replace “it is charged to the (α, β) and GKR challenges of Layer 6, where ζ is drawn, as an extra $\tau_{\text{max}}/|E|$ per constraint” by “it is the bus phase’s (Layer 6), and costs $\tau_{\text{max}}/|E|$ at most, once”. Acceptance test 6 (bp:1283): “The state function of Layer 6 carries the zerocheck.”

Finding 27 (minor). Layer 7’s sketch and the tracker’s signature predate the spine

Evidence. Layer 7’s sketch against the spine as built (dossier section 4, C.2):

	Layer 7 (bp:986-995) with Layer 6’s BusOut (bp:960-964)	The spine as built
Input statement	$\text{BusOut} = \zeta : \text{Fin } \mu_{\text{bus}} \rightarrow E, \text{rem} : \text{Fin } 3 \rightarrow E, \text{pool} : \text{List Claim}, \alpha, \beta$	$\text{I.Stmt} \times \text{BusOut I}, \text{BusOut I} = \text{linear}, \text{columns}$
The public input	absent	threaded through every seam
The point ζ	a field	absent; one point per term
The bus forms	recomputed by the table verifier from α, β, ζ and the leaf layout	data: terms with weights in E
The zerocheck claims	implicit (the instance’s constraints at ζ)	data: one linear claim each, if the bus phase emits them
The three values	rem	the value of three linear claims
Output	$\text{BusOut} \times \text{ColumnClaims}$ (keeps $\zeta, \text{rem}, \alpha, \beta$)	$\text{I.Stmt} \times \text{TableOut I}$ (column claims only)
<code>tableSumcheck_relOut_implies_constraints</code>	$(\forall j \in C, \tilde{C}_j(\zeta_{\{<\tau_j\}}) = 0 \rightarrow \dots)$ -- see below	no counterpart; the statement is a placeholder

The tracker’s hole comment has a third signature,

```
tableSumcheck I (S : Sumcheck.Def) : Phase.Def I (BusOut I) (TableOut I) (tableSpec I) (Seam.bus I)
  ↪ (Seam.table I)
```

with `tableSummand I  $\zeta \xi \alpha \beta \text{rem}$` : the spine has no `tableSpec` and `Phase.Def` takes no relation. `tableSumcheck_relOut_implies_constraints` (bp:991-992) has no statement.

Which to keep. Neither as it stands. Layer 7’s needs α, β and the selectors of the leaf stack, which the abstract instance does not carry. The spine’s is too general (finding 22). leanVM’s own seam is

between the two, the Rust's BusVerify (leaf.rs:849-860):

```
pub struct BusVerify {
  pub claims: Vec<ColumnClaim>,
  pub bytecode_claims: Vec<BytecodeClaim>,
  pub count_root: F192,
  /// The GKR point  $\zeta$ , reused as the table sumcheck's eq point.
  pub point: Vec<F192>,
  /// `forms[side][table]`, for the zerocheck to settle.
  pub forms: [Vec<BusForm>; 3],
  /// Per side, what the tables' blocks owe its leaf claim:  $\tilde{V}_0(\zeta)$  less the
  /// framework blocks' decomposition. Derived here, pinned by the batch's target.
  pub totals: [F192; 3],
}
```

crates/lean_vm/src/leaf.rs:849-860

one point, the forms as data per side and table, one value per side, the column claims. Keep from the spine: the public input threaded, the output reduced to column claims, α, β and the leaf layout private to the bus phase. Take from leanVM: the point in the statement and the forms indexed by side and table (the proposed change of finding 22).

Proposed change. Rewrite the sketch on the spine's names, `def tableSumcheck (I) : Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I), tableSumcheckComplete, tableSumcheckSecurity, with tableSummand I (s : BusOut I) ( $\xi$  : E); delete the placeholder (what it meant is the bus phase's zerocheck conjunct).`

Finding 28 (minor). Layer 9's sketch makes the eighteen limb claims an input of Flock

Evidence. (For whoever reviews Layer 9.) Dossier section 6, E.2; blueprint bp:1078-1081, reduction ... (StmtIn := ColumnClaims) ... (StmtOut := WeightedClaim), "relIn ... the eighteen column claims are true of limbColumns, and aux". In leanVM Flock's reduction takes no claim and the limb claims are opened with the others: the Rust's Flock verifier takes no claim (cpu/mod.rs:765, `verify_reduction(n_blocks, &mut vs)`), and nothing between the table sumcheck and the opening treats these claims apart (cpu/mod.rs:788-789, "No downstream special-casing: it folds into the one opening like every other point claim."). The spine's Flock slot is right: it passes the column claims on.

Finding 29 (minor). Two status findings are wrong at the pin

Evidence and proposed change.

- *The round message.* The status file's finding on the table round message (F6): "the table round polynomial is a cubic sent whole, four nodes, three wire scalars (constraints.rs:187-194, 267)". The wire carries coefficients, not values at nodes (constraints.rs:191-194; transcript.rs:56-71: "Send one sumcheck round polynomial, as its COEFFICIENTS, constant first"). The phrase copies a stale comment (constraints.rs:24; finding 32). Proposed: "a cubic, three of its four coefficients on the wire (c_0, c_2, c_3)".
- *The caps in the Python verifier.* The status file's finding on the Python caps (F9): "the Python verifier omits the caps $\log_mem \in [16, 32]$, $\tau_j \leq 32$, the bytecode power-of-two bound and $\tau_{\text{BLAKE2S}} \geq 3$ ". At the pin it has them (py:857-864):

```
log_bytecode = log2_strict(len(bytecode)) - BUS_BITS
require(
  16 <= log_memory <= 32
  and all(0 <= log_height <= 32 for log_height in table_log_heights)
  and table_log_heights[OP_BLAKE2S] >= 3
  and 0 <= log_bytecode <= 32,
  "invalid announced table sizes",
)
```

python-verifier/verifier.py:857-864

called from `verify_execution` at :1378; `log2_strict` requires a power of two (:252-254). Outside the two phases of this dossier; recorded because a later part enumerates checks. Proposed: retire that finding (F9).

Finding 30 (minor). The tracker’s text for the public-input phase is stale

Evidence. The hole comment says of the phase “the prover sends nothing” and “the two scalars are Layer 12’s encoding to read and reconcile”; the phase merged with the prover sending the values (decision 15). Text to draft only; nothing was posted.

Finding 31 (note). The specification’s batching error is loose by one; it gives no error for the recycled point; it does not say which coefficient is dropped

Evidence. What is stated (dossier section 5, D.1):

- *Batching by ξ .* The specification charges $(\nu_{\text{side}} + B)/|\mathbb{E}|$ (05:155, citing Corollary 3.9, cor:idtest, 03:67); the blueprint $(B + 2)/|\mathbb{E}|$ on ξ (bp:994). The tight value is $(k - 1)/|\mathbb{E}| = (B + 2)/|\mathbb{E}|$ for the $k = B + 3$ claims of the deployed input (dossier section 5, D.2): the specification’s $(\nu_{\text{side}} + B)$ is an upper bound one too large. (The citation check corrected the number of the corollary, which the dossier gave as 3.7.)
- *Each round.* $3/|\mathbb{E}|, 3 \cdot \tau_{\text{max}}$ in all (05:155, citing Fact 3.10, fact:sumcheck, 03:79); the blueprint $3/|\mathbb{E}|$ per round (bp:994). (The citation check corrected the number of the fact, which the dossier gave as 3.8, the number of the Schwartz–Zippel lemma.)
- *The recycled point.* None in the specification: 05:132 says it is sound “as long as it results from uniform sampling occurring after the table columns are committed” (the blueprint’s three charges: finding 26).
- *Public input.* $1/|\mathbb{E}|$ (08:33); the blueprint $1/|\mathbb{E}|$ on the one challenge (bp:1027).
- The specification has no round-by-round statement for these phases: its only one is Theorem B.7 (thm:rbr), for the opening (doc/leanvm/body/b-polynomial-commitment-scheme.tex:139-152), with the remark that the round-by-round error “is the maximum of these entries over all rounds”. (The citation check corrected the number of the theorem, which the dossier gave as B.2.)
- *Which coefficient is dropped.* The specification does not say which coefficient is omitted; §3’s “a zerocheck round on a degree- d cofactor therefore costs d field elements” (03:110) does not apply to the table sumcheck, which sends the cubic (08:76), because the waiting tables’ line $Y \cdot u$ is not a multiple of the eq factor (constraints.rs:20-26).

Finding 32 (note). Stale comments in the Rust on the round message

Evidence. constraints.rs:24 says the round polynomial “is sent WHOLE, at four nodes”; :265-266 says “ $h(0)$ is derived from the running claim rather than transmitted”. The code sends three *coefficients* and derives c_1 ; $h(0) = c_0$ is transmitted (transcript.rs:63-71). The blueprint follows the code; but the status repeats “four nodes” (finding 29). The citation check found two more stale comments in constraints.rs: :26 says the verifier checks “ $h(0) + h(1) = \text{claim}$, then interpolate at the challenge” (there is neither a check nor an interpolation: coefficients are read and one is derived); :257-260 says “Each round arrives as the round polynomial itself at nd ” (it arrives as three coefficients, transcript.rs:289-302).

Finding 33 (note). The spine’s slot does not pin the protocol: a phase that reads the whole oracle fills it

Evidence. Dossier section 6, E.4, item 6: *Faithfulness itself*. In the ideal oracle model a verifier may query q anywhere. A “phase” with no message, whose verifier reads q on the whole cube and decides every claim, fills the slot with both proofs and error zero. The master theorems hold for it. Nothing in the spine’s types separates it from leanVM’s sumcheck: that rests on the phase’s definition being reviewed against the transcript (section 8.5.1), and on Layer 12’s `verify_iff_compiled` with its fixture.

Finding 34 (note). Other notes: the column order inside a table, and perfect completeness

Evidence.

- *The order of the columns inside a table* fixes the order of the 104 values, the ties of the stack and the powers of λ . The protocol blueprint does not cite it; the leanISA blueprint does (docs/roadmap/leanisa-blueprint.md:612-613; the citation check corrected the lines, which

the dossier gave as 611-612). Whether leanISA’s tables have the Rust’s order, the two witnessed columns w, b of JUMP last, was not checked.

- *Perfect completeness without exceptional challenges* (acceptance test 20) holds of both phases: the verifier’s weights are products of $1 + \zeta_m + r_m$ and r_m , the derivation of c_1 is a sum; the only inverse in the prover is of the constant $g + g^2$ (primitives/src/multilinear.rs:233-241).

8.6 BLAKE2s validity and ring switching

8.6.1 The BLAKE2s validity phase (Flock) and ring switching: the transcript as deployed

Short forms in this section; the number after the colon is the line, at leanVM’s pin a386121f.

- Specification, under doc/leanvm/body/: c-flock: is c-flock-protocol.tex, a-ring: a-ring-switching.tex, 03: 03-proving-primitives.tex, 04: 04-committing-the-witness.tex, 07: 07-instruction-tables.tex, 08: 08-end-to-end-protocol.tex.
- Rust, under crates/flock/src/: hash.rs, zerocheck.rs, lincheck.rs, reduction_tests.rs, flock/src/lib.rs; under crates/flock/src/zerocheck/: univariate_skip_optimized.rs, zerocheck/multilinear.rs.
- Rust, under crates/lean_vm/src/: cpu/mod.rs, cpu/layout.rs, hash_flock.rs, tables.rs; under crates/pcs/src/: pack.rs, stack_open.rs, ring_switch.rs, tensor_algebra.rs; transcript.rs is crates/flat_shamir/src/transcript.rs.
- Python: verifier.py is python-verifier/verifier.py.

Notation. k_{batch} is the log of the number of BLAKE2s compressions (the log-height τ of the BLAKE2S table), $k_{\text{bool}} = 14 + k_{\text{batch}}$ the number of variables of the bit witness z , $n_{\text{flock}} = k_{\text{bool}} - 6 = 8 + k_{\text{batch}}$ the number of variables of the packed witness q_{flock} .

8.6.1.1 What is committed

Nothing in this phase. The bit witness z was committed in the commit phase, packed 64 bits to a $\mathbb{K}$ element, as the region q_{flock} of the one stack q .

Table 8.12: What is committed for the Flock phase.

	Specification	Rust	Python
z is packed into q_{flock} , a region of the stack	c-flock:30 “The resulting multilinear q_{flock} ... is committed as one region of the stack”; 08:60	hash_flock.rs:3-5 “committed as a column in leanVM’s ONE stacked F64 witness ..., with no separate flock commitment”; cpu/layout.rs:25 QFLOCK = 5	verifier.py:633, :879-880
Packing: bit i of word u is $z(i, u)$, coefficient of x^i	a-ring:13-16 $q_{\text{flock}}(u) = \sum_i Q_i(u)x^i$	pack.rs:1-2 “least significant bit first”; hash_flock.rs:187-197	(verifier only)
No other oracle, no other commitment	c-flock:264 “the PCS proves the claims opened on q_{flock} ”	flock/src/lib.rs:6-13; cpu/mod.rs:564-571	verifier.py:1404-1413

The vectors $a = Az$ and $b = Bz$ are never committed and never opened: only z is (c-flock:30; in the Rust the only commitment is pcs::commit(&mut ps, &w.q, ...), cpu/mod.rs:560-562, and a, b are not columns of the layout). (The citation check corrected the support of this sentence: the dossier cited c-flock:18, which only defines $a = Az$, $b = Bz$, $c = Cz = z$, and hash.rs:592-593, “No c buffer. Since $C = I$, $c == z$ ”, which supports only $c = z$.)

8.6.1.2 The messages, in order

$P \rightarrow$ is a prover message, $V \rightarrow$ a verifier challenge, V : a verifier computation. All values are in $\mathbb{E}$. Counts are per proof.

Table 8.13: The Flock phase and ring switching, message by message, as deployed.

#	Step	Specification	Rust (verifier unless said)	Python
1	V : seven fixed coordinates $r_0..r_6 = \varphi_8(0xf7), \varphi_8(0x53), \varphi_8(0xb5), g_0^{2^j}/(1 + g_0^{2^j}), j < 4$	c-flock:54 (value of g_0 not given: “where $g_0 \in \mathbb{E}$ is public”)	zerocheck.rs:51-58; univariate_skip_optimized.rs:65, 79-101; g_0 at :104-106	verifier.py:1095-1100
2	$V \rightarrow n_{\text{flock}} - 7 = k_{\text{batch}} + 1$ coordinates $r_7..$, before any Flock message	c-flock:55, :173	zerocheck.rs:362 equality_tail(m, n vs.sample_vec(n))	verifier.py:1139
3	$P \rightarrow P$ on the coset $H' = \varphi_8(64..127)$: 64 values	c-flock:62 “The prover sends $P _{H'}$, 64 values”	zerocheck.rs:365 (prover :185-187)	verifier.py:1142
4	$V \rightarrow z_{\text{skip}}$	c-flock:64	zerocheck.rs:366	verifier.py:1143
5	V : v_P , the value at z_{skip} of the interpolant of the 64 received values and 64 assumed zeros on H ; no check	c-flock:62-67	zerocheck.rs:378; zerocheck/multilinear.rs:129-138	verifier.py:1144
6	n_{flock} rounds. $P \rightarrow$ two coefficients c_1, c_2 of the quadratic cofactor G_i ; V : $c_0 := \text{claim} + r_i \cdot (c_1 + c_2)$; $V \rightarrow \chi_i$; V : $\text{claim} := G_i(\chi_i)$. Low variable first. No check	c-flock:69 (“with Gruen’s eq-factor optimization”); 03:110	zerocheck.rs:397-405; transcript.rs:289-309 (prover zerocheck.rs:116-123)	verifier.py:1147, :406-413, :420-427
7	$P \rightarrow v_a, v_b$; V : $v_c := R_{zc} + v_a \cdot v_b$ (never sent, never checked)	c-flock:69-73 “is what <i>defines</i> v_c : both sides solve it rather than transmit it”	zerocheck.rs:421-423	verifier.py:1148-1149
8	$V \rightarrow \alpha_{lc}$; V : target $v_a + \alpha v_b + \alpha^2 v_c + \alpha^3$	c-flock:112-118	lincheck.rs:1179, :1199-1203	verifier.py:1155, :1162
9	8 rounds. $P \rightarrow$ two coefficients c_0, c_2 of the quadratic; V : $c_1 := \text{claim} + c_2$; $V \rightarrow$ challenge; high variable first, so the point χ'_{in} is the challenges reversed	c-flock:122	lincheck.rs:1205-1211, :1218-1219 (prover :1085-1107)	verifier.py:1163, :1169
10	$P \rightarrow s_0..s_{63}$, claimed $s_i = \tilde{z}(i, \chi'_{in}, \chi_{out})$	c-flock:124	lincheck.rs:1214 (prover :1113-1115)	verifier.py:1168
11	V : check the terminal identity $R_{lc} = e_{\text{row}}^T(A_0 + \alpha B_0)w_{\text{col}} + \alpha^2 \cdot \text{eq}(\chi_{in}, \chi'_{in}) \cdot \Sigma_i \varpi_i(z_{\text{skip}})s_i + \alpha^3 \cdot \text{eq}(\chi'_{in}, 8) \cdot s_0$; reject otherwise	c-flock:126-133, :150	lincheck.rs:1233-1256, error ConsistencyFailed, surfaced as CpuError::Blake2s at cpu/mod.rs:765; the matrix term is hash.rs:361-389 (bilinear_walk)	verifier.py:1170-1176 require(terminal == r_lc, ...); matrix term :1180-1301
12	$V \rightarrow$ six challenges $f_0..f_5$ (ring switching)	a-ring:104; 08:91	stack_open.rs:513; ring_switch.rs:160-162	verifier.py:1343
13	V : target $T = \Sigma_i x^i \Phi(s_i)$; the weight is $W(u) = \Phi(\text{eq}(r, u))$ on the q_{flock} region, $r = (\chi'_{in}, \chi_{out})$. No message, no check	a-ring:44-46, :104-110	stack_open.rs:521-524; ring_switch.rs:555-557 (through the transposed columns, tensor_algebra.rs:42-71, and build_coordinate_weights, ring_switch.rs:133-144)	verifier.py:1345-1347

#	Step	Specification	Rust (verifier unless said)	Python
14	(opening) $V \rightarrow \lambda$; the ring-switched claim takes λ^0 , the pooled claims the next powers	08:98	<code>stack_open.rs:518-527</code>	<code>verifier.py:1361, :1413</code>
15	(opening, last step) V : evaluates $\widetilde{W}$ at the final point r'	a-ring:150-154: $\Sigma_k c_k \prod_n (1 + r_n^{2^k} + r'_n)$	<code>stack_open.rs:530-546;</code> <code>ring_switch.rs:597-617</code> (<code>eval_rs_eq</code> , the tensor-algebra algorithm of Diamond–Posen)	<code>verifier.py:1324-1335,</code> <code>:1412</code> (the closed form)

Counts, checked against `reduction_tests.rs:166-168` and by the probe of the dossier’s section 9.3 (the citation check re-derived both from the code):

What	Count
Scalars the reduction (steps 3 to 10) puts on the stream	$64 + 2 \cdot n_{\text{flock}} + 2 + 16 + 64 = 162 + 2 \cdot k_{\text{batch}}$ (168 at $k_{\text{batch}} = 3$)
Challenges drawn by steps 2 to 12	$(k_{\text{batch}} + 1) + 1 + n_{\text{flock}} + 1 + 8 + 6 = 2 \cdot k_{\text{batch}} + 25$

Steps 12 and 13 are “BLAKE2s validity, step 2” in the specification (08:91) and the first lines of the opening function in the Rust (`open_batch_mixed_whir_stacked`, `verify_opening_batch_mixed_whir_stacked`). The transcript position is the same.

8.6.1.3 Values derived, never transmitted

v_P (step 5); c_0 of each of the n_{flock} zerocheck rounds (step 6); v_c (step 7); the lincheck target (step 8); c_1 of each of the 8 lincheck rounds (step 9); T (step 13). The s_i are sent once, by the lincheck, and reused by ring switching (`stack_open.rs:113-116` “this layer reads nothing off the stream”).

8.6.1.4 The eighteen limb claims

They are produced by the table sumcheck, routed to the opening, and never seen by Flock.

Table 8.14: The eighteen limb claims, from the table sumcheck to the opening.

	Specification	Rust	Python
The table sumcheck sends one value per column of every table, the eighteen limb columns included	08:77 “The prover sends one value per column of every table”	<code>cpu/mod.rs:428-441</code> (for <code>c</code> in <code>0..table.n_committed_columns()</code> , 37 for BLAKE2S, <code>tables.rs:866-868</code> , :862)	<code>verifier.py:620</code> (<code>table.width</code>), :827-834
A limb claim at z becomes a claim on q_{flock} with the low 8 coordinates frozen to the slot	08:77 “their claims route there”; 07:122. The selector is not described (§4.1 has aligned blocks only, 04:10-18)	<code>cpu/mod.rs:790-814</code> , which builds a <code>pcs::SlotClaim::Strided</code> (quoted below the table); <code>stack_open.rs:84-97</code> , :305-323; <code>hash_flock.rs:223</code> ($= 14 - 6 = 8$)	<code>verifier.py:884-894</code> <code>Placement(kappa, offsets[QFLOCK] + limbs[column], QFLOCK_SLOT_BITS)</code> ; :295-302
Slot of each limb	not given	<code>hash_flock.rs:87-115</code> : cv 0..3, out 4..7, message 10..17, metadata 18, 19	<code>verifier.py:847-850</code> (same)
The Flock verifier takes no claim	<code>c-flock:170-179</code> (the summary has no input claim)	<code>cpu/mod.rs:765</code> <code>verify_reduction(n_blocks, &mut vs)</code> ; <code>hash_flock.rs:282-284</code>	<code>verifier.py:1405</code> , called with the size only (quoted below the table); the signature <code>verify_flock(log_n: int, transcript: Transcript)</code> is at :1304
The limb claims enter the opening batch with every other pooled claim	08:97-99	<code>cpu/mod.rs:756</code> , 768 (slots), <code>stack_open.rs:525-527</code>	<code>verifier.py:1395</code> , 1413

The Rust’s strided claim, and the Python’s call of the Flock verifier:

```
return pcs::SlotClaim::Strided {
    offset: l.placements[QFLOCK].offset,
```

```

    slot,
    stride_log: crate::hash_flock::SLOT_STRIDE_LOG,
    point: c.point.clone(),
    value: c.value,
};

```

crates/lean_vm/src/cpu/mod.rs:799-805

```
flock_point, flock_s = verify_flock(BLAKE2S_RICS_LOG_SIZE + layout.table_log_heights[0P_BLAKE2S], transcript)
```

python-verifier/verifier.py:1405

(The citation check corrected two quotations of the dossier: `cpu/mod.rs:790-814` was quoted in abbreviated form as `SlotClaim::Strided { offset, slot, stride_log: SLOT_STRIDE_LOG, point, value }`, and `verifier.py:1405` as `verify_flock(log_n, transcript)`, which is the signature at :1304.)

8.6.1.5 What leaves the phase

One family of 64 slice claims at one point, turned by ring switching into **one weighted claim** $\langle W, q \rangle = T$ whose weight is supported on the q_{flock} region (lifted to the stack by the region's selector: `stack_open.rs:41-48`, `verifier.py:1412`), plus the pool unchanged.

8.6.2 The BLAKE2s validity phase (Flock) and ring switching: disagreements between the three sources

Short forms in this section; the number after the colon is the line, at leanVM's pin a386121f.

- Specification, under `doc/leanvm/body/`: `c-flock`: is `c-flock-protocol.tex`, `a-ring`: `a-ring-switching.tex`.
- Rust, under `crates/flock/src/`: `hash.rs`, `lincheck.rs`, `gf2.rs`; under `crates/pcs/src/`: `ring_switch.rs`; under `crates/primitives/src/field/`: `gf2_64x3.rs`, `gf2_64.rs`; `transcript.rs` is `crates/flat_shamir/src/transcript.rs`.
- Python: `verifier.py` is `python-verifier/verifier.py`.

None on a message, a challenge, an order or a formula. The differences are all of one kind: **the specification is silent where the two verifiers agree**. The table lists what was compared in particular, and, in its last row, the one disagreement between the two verifiers that the citation check found; it is not on the verifier path.

Table 8.15: The Flock phase and ring switching: the three sources compared.

Subject	Where	Verdict
The order of binding of the round coefficients	<code>transcript.rs:293-307</code> reads then binds in index order; <code>verifier.py:409-413</code> binds as it reads	the same sequence
The index of the constant position ($512 = 0 + 8 \cdot 64$, so $s_0 \cdot \text{eq}(\chi'_{\text{in}}, 8)$)	<code>c-flock:131</code> ; <code>lincheck.rs:1240</code> with <code>hash.rs:146</code> ; <code>verifier.py:1174</code> with :642	agree
The row and column weight layouts	<code>lincheck.rs:832-837</code> , <code>1234</code> ; <code>verifier.py:1159</code> , <code>1170</code>	agree
The two circuit walks, line by line	<code>hash.rs:295-359</code> with <code>gf2.rs:109-154</code> against <code>verifier.py:1180-1295</code>	agree; offsets 0, 61, 92, 153 in both
The ring switching stages and coefficients	<code>a-ring:104-117</code> ; <code>ring_switch.rs:85</code> , <code>147-156</code> ; <code>verifier.py:1314-1321</code> , <code>1345</code>	agree
The two forms of the target	$\Sigma_w \Phi(b_w) \cdot t_w$ in the Rust; $\Sigma_i x^i \Phi(s_i)$ in the specification and the Python	equal by $\mathbb{F}_2$ -linearity, and the Rust tests it, <code>ring_switch.rs:683-705</code>

Subject	Where	Verdict
The constants, the circuit, the slot map, two caps	the specification is silent; the two verifiers agree	omissions of the specification, not contradictions; collected in finding 41
The inverse of zero (row added by the citation check)	Rust returns 0 (gf2_64x3.rs:137 “ZERO.inv() == ZERO”, gf2_64.rs:42); Python raises (verifier.py:168-170, “division by zero in GF(2 ¹⁹²)”)	a disagreement, harmless on the verifier path (only constants and guarded roots are inverted); it matters for anyone modelling the prover’s send_round with the Python field: the dossier’s probe 9.2 rightly uses $x^{2^{192}-2}$, not $E.inv$

8.6.3 The BLAKE2s validity phase (Flock) and ring switching: findings

Short forms in this section; the number after the colon is the line. leanVM is read at its pin a386121f, leanerVM at main b435631.

- leanerVM: bp: is the blueprint docs/roadmap/protocol-blueprint.md, st: the status docs/roadmap/protocol-status.md; Lean files are under LeanerVM/Protocol/.
- Specification, under doc/leanvm/body/: c-flock: is c-flock-protocol.tex, a-ring: a-ring-switching.tex, 03: 03-proving-primitives.tex, 06: 06-bus-interactions.tex, 07: 07-instruction-tables.tex, 08: 08-end-to-end-protocol.tex.
- Rust, under crates/flock/src/: hash.rs, gf2.rs, zerocheck.rs, lincheck.rs; under crates/flock/src/zerocheck/: univariate_skip_optimized.rs; under crates/lean_vm/src/: cpu/mod.rs, cpu/layout.rs, hash_flock.rs, tables.rs; under crates/pcs/src/: stack_open.rs, ring_switch.rs, whir_config.rs; under crates/primitives/src/field/: gf2_8.rs, phi8_tower.rs, gf2_64x3.rs; primitives/src/multilinear.rs is crates/primitives/src/multilinear.rs; transcript.rs is crates/flat_shamir/src/transcript.rs.
- Python: verifier.py is python-verifier/verifier.py.

No finding is critical: no Lean theorem about Flock exists yet that could be wrong or vacuous. The four major findings are all in text that the Flock roadmap will build from.

Finding 35 (major). The Flock interface of Layer 9 consumes the limb claims and emits one claim; leanVM’s Flock reads no claim and the limb claims are opened

Evidence. Table 8.14, and the dossier’s section 5.1. The deployed Flock verifier reads no claim of the pool: the eighteen limb claims stay in the pool and are opened as evaluation claims on q_{flock} (strided). Layer 9’s FlockInterface, with StmtIn := ColumnClaims (the eighteen claims) and StmtOut := WeightedClaim, drops them. Its knowledge soundness is not provable: take q with a valid q_{flock} and limb claims with wrong values; relIn fails, the deployed verifier never reads the claims, the honest transcript is accepted and relOut holds: error 1. A reduction of Layer 9’s shape could be made sound only by folding the eighteen claims into the weighted claim with a fresh challenge, which leanVM does not do.

Classification. An error of the blueprint, to be fixed; the spine is faithful, Layer 9 and the tracker’s section on the Flock phase (P6) are not.

Proposed change.

Proposed change: Layer 9: the Flock phase at the spine's seams

As it stands. (bp:1075-1092) The structure

```
structure FlockInterface (I : M3Instance) where
  reduction : OracleReduction [] (StmtIn := ColumnClaims) (OStmtIn := fun _ : Unit → Column μ) Unit
    (StmtOut := WeightedClaim) (OStmtOut := fun _ : Unit → Column μ) Unit pSpecFlock
  limbColumns : Column μ → Fin 18 → Column (s.τ 5) -- the limbs: strided slots of q_flock
  relIn : Set _ -- the eighteen column claims are true of `limbColumns`, and aux (Flock's RICS)
  relOut : Set _ -- the weighted claim holds for q
```

docs/roadmap/protocol-blueprint.md:1077-1082 at b435631

and “relIn carries the strong auxiliary predicate of Layer 3 ... limbColumns is the strided reader of Layer 3”. (The citation check noted that the dossier cites this block as bp:1075-1086; it reproduces lines 1077 to 1082.)

As proposed. A sketch in the blueprint's style, not compiled:

```
-- What the Flock roadmap supplies: the phase at the spine's seams, for an instance that says
where the packed witness is (`FlockRegion`, Layer 3). -/
structure FlockPhase (I : M3Instance) (R : FlockRegion I) where
  phase : Phase.Def I (I.Stmt × PubOut I) (I.Stmt × FlockOut I)
  security : Phase.Security I phase (Seam.pub I) (Seam.flock I) -- completeness included
  err_le : ∑ i, phase.err i ≤ (4 * R.kBatch + 163) / |E| + 2 ^ 32 / |E|
```

with the text: “The verifier reads no claim of the pool. It hands the column claims on unchanged, the eighteen limb claims among them (cpu/mod.rs:790-814: they are opened as evaluation claims on q_{flock}), and adds one weighted claim, the ring-switched one. The schedule is: $k_{\text{batch}} + 1$ challenges; 64 values; one challenge; $8 + k_{\text{batch}}$ rounds of two values and one challenge; two values; one challenge; 8 rounds of two values and one challenge; 64 values; six challenges.”

Reason. Soundness of the sketch as written fails with error 1.

Finding 36 (major). The slot map of the limb columns is placed inside the Flock interface, so the instance and the interface each need the other

Evidence. Cannot be stated as written. bp:846-848 “leanIsaInstance.layout combines the two, the slot map being #3's FlockInterface.limbColumns (Layer 9)”, and bp:1077 structure FlockInterface (I : M3Instance). The instance's layout field needs a value of a structure that takes the instance as its parameter. The slot map is eighteen numbers (hash_flock.rs:96-115) and a generic identity on multilinear tables.

Classification. An error of the blueprint.

Proposed change.

- blake2sLimbSlot : Fin 18 → Fin 256 in LeanerVM/Parameters/, transcribed from its source (the blueprint's Category B: hash_flock.rs:87-115, cross-checked with verifier.py:847-850);
- the low-index selection identity next to evalMle_append_boolVec in ToCompPoly/Multilinear.lean, generic (a table of $m + k$ variables evaluated at boolVec $j \mapsto z$ is its low-index slice evaluated at z);
- Layer 3 builds the strided reader from the two;
- limbColumns is deleted from Layer 9 and from the tracker's sections on the adaptor (I2) and on the Flock phase (P6).

The boundary paragraph with leanISA (bp:1439-1441) becomes “reconciled by the instance's layout”.

Finding 37 (major). The Flock phase cannot be written over an abstract instance: the instance gives it only an opaque predicate

Evidence. Cannot be stated as written.

```
-- What the Flock phase proves of the committed region it owns, beyond the polynomial
checks: the statement its honest prover can convince the verifier of, so it names the
committed witness (Flock's RICS on `q_flock`), never only a consequence of it. -/
aux : Column μ → Prop
```

```

/-- The auxiliary predicate is decidable, so that the relation is. -/
decAux : DecidablePred aux

```

LeanerVM/Protocol/Spine/Instance.lean:143-148 at b435631

Layer 9 is “module, over I ” (bp:1072), and the convention *The wall* (bp:331) lists the modules allowed to know leanISA; the Flock phase is not among them. But a phase that knows only $I.aux : \text{Column } \mu \rightarrow \text{Prop}$ cannot name the column, the number of compressions or the relation it is to prove; the sketch itself reaches for $s.\tau 5$, a leanISA size. For an abstract I the only phases one can write are those of the probe (the dossier’s section 9.1: a Flock phase that checks nothing is knowledge sound when aux is trivial).

Classification. An error of the blueprint (a design gap the spine left open).

Proposed change. A sketch: Layer 3 supplies, and Layer 9 takes,

```

/-- Where the packed witness of the BLAKE2s circuit sits, and what `aux` says of it. -/
structure FlockRegion (I : M3Instance) where
  col : I.ColumnId -- the column q_flock
  kBatch : ℕ -- log2 of the number of compressions
  height : I.k col = 8 + kBatch -- 2^8 packed words per compression, low coordinates
  aux_iff : ∀ q, I.aux q ↔ Flock.Holds kBatch (I.column q col) -- R1CS and constant position

```

`Flock.Holds` is the Flock roadmap’s predicate on a packed column and mentions no instance. The toy keeps $aux := \text{True}$ and has no `FlockRegion`. The wall stands: the phase imports nothing from the arithmetization.

Finding 38 (major). Acceptance test 20 files a completeness failure under a soundness error, and attributes to the protocol an inverse that only the Rust prover takes

Evidence. The dossier’s section 4. No verifier takes the inverse of a value that depends on a challenge: both derive the untransmitted coefficient of a round polynomial by a multiplication (transcript.rs:297-302; verifier.py:411-413). (The citation check corrected the dossier’s sentence “The verifiers take no inverse”, which is literally false: both verifiers invert constants, for example `univariate_skip_optimized.rs:96-99`, `primitives/src/multilinear.rs:185`, `verifier.py:1099` and `:1110`.) The Rust prover takes one, to avoid computing $G(0)$:

```

fn send_round(ps: &mut impl Transmitter, claim: F192, r_eq: F192, g1: F192, g_inf: F192, chis: &mut
  ↪ Vec<F192>) -> F192 {
  let g0 = (claim + r_eq * g1) * (F192::ONE + r_eq).inv();
  ps.add_round_poly(&[g0, g0 + g1 + g_inf, g_inf], true);
  let chi = ps.sample();
  chis.push(chi);
  // G(X) = G(0)·(1+X) + G(1)·X + G(inf)·X·(1+X).
  g0 + chi * (g0 + g1 + (F192::ONE + chi) * g_inf)
}

```

crates/flock/src/zerocheck.rs:116-123

and the inverse of zero is zero, by contract (gf2_64x3.rs:137). At $r_{eq} = 1$ the true claim is $(1 + 1) \cdot G(0) + 1 \cdot G(1) = G(1)$: it carries no information on $G(0)$. The prover computes $g_0 = 0$, sends $c_1 = G(1) + G(\infty)$ (wrong unless $G(0) = 0$) and the right c_2 ; the verifier derives $c_0 = 0$, and the error propagates into the derived v_c . The verifier rejects, at the lincheck terminal identity; no panic on either side. The challenges concerned are the $k_{batch} + 1$ sampled equality coordinates only, with probability at most $(k_{batch} + 1)/|\mathbb{E}| \leq 33/2^{192}$, times the chance that $G(0) \neq 0$. It is not the protocol’s: a prover that sends the true c_1, c_2 is accepted at $r_{eq} = 1$, and the specification never mentions an inverse (c-flock:69, 03:110). Derived by reading and confirmed on a model of the two formulas over the pinned Python field (the dossier’s probe 9.2); not executed in Rust. The spine has no place for a completeness error at all (Component.Security extends Complete, which is perfect completeness).

Classification. An error of the blueprint.

Proposed change.

Proposed change: acceptance test 20

As it stands. (bp:1323-1325) “20. **Perfect completeness needs no exceptional-challenge clause** in Layers 4–8 and 10: no inverse of a challenge is taken. Flock’s $(1+r_{eq})^{-1}$ at $r_{eq} = 1$ is #3’s, inside flockError. Witness: piop_perfectCompleteness has no side condition on challenges.”

As proposed. “20. **Perfect completeness needs no exceptional-challenge clause, in any phase.** No verifier takes the inverse of a challenge: the untransmitted coefficient of a round polynomial is $claim + r \cdot (c_1 + \dots + c_d)$ (transcript.rs:296-302). Every honest prover, Flock’s included, sends the coefficients of the true round polynomial. The pinned Rust prover of Flock does not: it derives $G(0) = (claim + r \cdot G(1)) \cdot (1+r)^{-1}$ (zerocheck.rs:116-118), and at $r = 1$ emits a proof its verifier rejects. That is a completeness defect of the implementation, of probability at most $(k_{batch} + 1)/|\mathbb{E}|$; it is recorded in docs/leanvm-target.md and is no part of flockError, a knowledge-soundness error. Witness: Phase.Complete is perfect completeness; the Flock phase’s test runs the round at $r = 1$ with $G(0) \neq 0$, accepted for the true coefficients and rejected for the derived ones.”

Reason. The inverse is taken by the Rust prover alone; the protocol of the specification, with a prover that sends the true coefficients, is perfectly complete. Filing its failure under flockError, a knowledge-soundness error, is a category mistake (the dossier’s conclusion 3).

The same sentence must reach issue #3, whose text plans “exceptional-challenge completeness conditions” (draft for the tracker only; nothing was posted).

Finding 39 (minor). Knowledge soundness is bundled with perfect completeness

Evidence. An avoidable coupling; it is what turns finding 38 into a threat to the soundness theorem. ToArkLib/Component.lean:90-93 (Security ... extends Complete); the composition uses only the guard of the first verifier:

```
rbr := fun init impl => rbrKnowledgeSoundnessWorstCaseWith_of_eq _
  (Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first S1.guarded
    (S1.kSF init impl) (S2.kSF init impl) (S1.rbr init impl) (S2.rbr init impl))
```

LeanerVM/Protocol/ToArkLib/Component.lean:182-184 at b435631

Classification. A deliberate deviation of the spine from ArkLib’s separate statements, whose reason (one structure per hole) does not need the coupling.

Proposed change. A structure Component.Guarded with outputPure and guarded; Complete and Security both extend it; piop_rbrKnowledgeSoundness then holds for a protocol whose completeness is unproved or imperfect. (The citation check adds: the composed Security also builds toComplete := S1.toComplete.append S2.toComplete, ToArkLib/Component.lean:176, so the proposed split must also give Security.append a completeness-free path; the excerpt above shows only the rbr field.)

Proposed change: convention Holes

As it stands. (bp:332) “X.Security (perfect completeness, and round-by-round knowledge soundness ...)”

As proposed. “X.Security (the guard, and round-by-round knowledge soundness ...)”

Reason. piop_rbrKnowledgeSoundness then holds for a protocol whose completeness is unproved or imperfect.

Finding 40 (minor). The auxiliary predicate must include the constant-one position

Evidence. Imprecise wording on a load-bearing definition. c-flock:23-28 “We also enforce the position 512 of every block to be 1 (preventing the all-zero witness)”; hash.rs:1172-1183; against bp:850-851 “aux q says that Flock’s R1CS holds of the bits packed into the q_flock region”, st:239-240, Spine/Instance.lean:24-25, 143-145. Read literally, aux holds of the zero region and the lemma “aux $\Rightarrow$ the limb slots compress” is false. (The citation check adds that the blueprint’s citation “Annex C.1” does point at the subsection that contains both conditions, so the omission is of wording,

not of reference.)

Classification. An error of the blueprint (of wording).

Proposed change.

Proposed change: the auxiliary predicate, everywhere the phrase occurs

As it stands. “aux q says that Flock’s R1CS holds of the bits packed into the q_{flock} region”

As proposed. “aux q says that, for every compression, the bits packed into the q_{flock} region satisfy Flock’s R1CS and hold 1 at the constant position 512 (Annex C.1, equations (flock:eq:r1cs) and (flock:eq:const-one))”. Add to the acceptance tests: “The constant position is part of the predicate. The zero block satisfies the R1CS and is no compression. Witness: aux fails on the zero region.”

Reason. Read literally, aux holds of the zero region and the lemma “aux $\Rightarrow$ the limb slots compress” is false.

Finding 41 (minor). Flock is listed as written from the specification; its circuit, layout and constants exist only in the Rust and the Python

Evidence. bp:192 “Flock | A, owned by #3 | specification Annex C; crates/flock/src/”. What the specification does not contain, with the place that fixes it:

Missing from the specification	Fixed by
The BLAKE2s circuit and the position of each wire	hash.rs:40-64, 139-162, 295-359; gf2.rs:23-31, 109-154; verifier.py:1180-1295
The slot of each limb	hash_flock.rs:87-115; verifier.py:847-850
The selector of a limb claim (a low-index selection; §4.1 has aligned blocks only)	stack_open.rs:84-97; verifier.py:288-302
g_0 (“is public”, c-flock:54)	univariate_skip_optimized.rs:104-106; verifier.py:1095
φ_8 : which of the eight embeddings, and the modulus of $\mathbb{F}_{2^8}$ (c-flock:35)	gf2_8.rs:13; phi8_tower.rs:15-24; verifier.py:1092
$k_{\text{batch}} = \tau_{\text{BLAKE2S}}$	cpu/mod.rs:721, 763; verifier.py:1405
The floor $\tau_{\text{BLAKE2S}} \geq 3$ (06:80 has the caps only)	cpu/mod.rs:162-166; hash.rs:283-286; verifier.py:861
The wording “live in q_{flock} , not the stack” (08:77): q_{flock} is a region of the stack	cpu/layout.rs:17-25

Classification. A deviation forced by the source (the specification is incomplete); the blueprint follows the Rust, rightly, and should say so.

Proposed change.

Proposed change: the row *Flock* of the table of artifacts

As it stands. “Flock | A, owned by #3 | specification Annex C; crates/flock/src/”

As proposed. “Flock: protocol and errors | A, owned by #3 | specification Annex A, Annex C” and a second row “Flock: the BLAKE2s circuit, wire positions, limb slots, φ_8 , the fixed coordinates, R1CS_DIGEST | B | crates/flock/src/{hash,gf2}.rs, zerocheck/univariate_skip_optimized.rs:65, 104-106, primitives/src/field/phi8_tower.rs:15-24, lean_vm/src/hash_flock.rs:87-115; verifier.py:847-850, 1092-1100, 1180-1295”.

Reason. The specification is incomplete; the blueprint follows the Rust, rightly, and should say so.

Each omission is a line for docs/leanvm-target.md and for a report to leanVM (drafted here only).

Finding 42 (minor). The Flock error quoted in acceptance test 23 leaves out ring switching

Evidence. bp:1331-1332: “ $\Sigma \text{piopError} < 2^{-150}$ plus Flock’s $< 2^{-183}$ ”. 2^{-183} is $(4 \cdot k_{\text{batch}} + 163)/|E|$ alone (c-flock:282); flockError_le (bp:1085) adds $2^{32}/|E| = 2^{-160}$, which dominates by a factor 2^{23} .

Proposed change.

Proposed change: acceptance test 23

As it stands. “plus Flock’s $< 2^{-183}$ ”

As proposed. “plus the Flock phase’s, below $2^{-183} + 2^{-160}$, the second term being ring switching’s”

Reason. flockError_le adds $2^{32}/|E| = 2^{-160}$, which dominates 2^{-183} by a factor 2^{23} .

The convention *Errors* wants the closed form next to the theorem: give flockError per challenge as in the table of the dossier’s section 5.2.

Finding 43 (minor). The ledger row on ring switching suggests a missing profile; ArkLib’s construction is another protocol

Evidence. bp:267: “A9 ring switching packing leaves | RingSwitching/Packing leaves are sorry, and no $\text{GF}(2) \rightarrow \text{GF}(2^{64})$ profile”. The dossier’s section 6, as its summary puts it: ArkLib’s RingSwitching/Packing is a different protocol (Diamond–Posen: a carrier message, a batching vector, a relocation sumcheck, error $\kappa/|L|$), with completeness and soundness both admitted at the pin and on origin/main; no “profile” turns it into leanVM’s. With every leaf proved and a profile for $\mathbb{F}_2 \subset \mathbb{K}$, ArkLib’s reduction would still send a carrier element, run a relocation sumcheck, have the point in $\mathbb{K}$ and the error $\kappa/|L|$. (The citation check notes that since the merge of 144c5aa the ArkLib pin is 7653a901, not dca90385; the 14 sorry of RingSwitching/Packing/ are the same at both, so “admitted at the pin and on origin/main” holds at the old and at the new pin.)

Classification. An error of the blueprint (of description).

Proposed change.

Proposed change: the ledger row on ring switching

As it stands. “A9 ring switching packing leaves | RingSwitching/Packing leaves are sorry, and no $\text{GF}(2) \rightarrow \text{GF}(2^{64})$ profile | Layer 9 | Owned by #3 (F5); consumed through the Flock interface.”

As proposed. “A9 ring switching | ArkLib’s Packing is Diamond–Posen’s interactive reduction (carrier message, batching vector, relocation sumcheck), 14 sorry at the pin and on main; leanVM’s is rectangular ($\mathbb{K} \otimes \mathbb{E}$), batches by a linear map from six challenges and hands a weighted claim to the commitment scheme | Layer 9 | New work of #3 (its gate F5); at most the tensor algebra is shared”.

Reason. With every leaf proved and a profile for $\mathbb{F}_2 \subset \mathbb{K}$, ArkLib’s reduction would still send a carrier element, run a relocation sumcheck, have the point in $\mathbb{K}$ and the error $\kappa/|L|$.

Finding 44 (minor). The status file’s finding that the Python verifier omits the caps is false at the pin

Evidence. Stale text; it asks for a report upstream that would be wrong. st:316-319: “F9 the Python verifier omits the caps $\log_{\text{mem}} \in [16, 32]$, $\tau_j \leq 32$, the bytecode power-of-two bound and $\tau_{\text{BLAKE2S}} \geq 3$ (verifier.py:1372-1379 ...)”. At the pin:

```
log_bytecode = log2_strict(len(bytecode)) - BUS_BITS
require(
  16 <= log_memory <= 32
  and all(0 <= log_height <= 32 for log_height in table_log_heights)
  and table_log_heights[OP_BLAKE2S] >= 3
  and 0 <= log_bytecode <= 32,
  "invalid announced table sizes",
)
```


python-verifier/verifier.py:857-864

and `log2_strict` requires a power of two (`verifier.py:252-254`). The check is present at the pin; it was assembled in `8a6e7750`, `348ca455` and `5751a5c7`, all ancestors of `a386121f`. (The citation check of the bus dossier corrected the provenance: the dossier says the lines were introduced by `14fbca8f`, which only renamed `BLAKE2S.opcode` to `OP_BLAKE2S`; this dossier’s citation check had accepted `14fbca8f` from `git log -S` on the renamed string. The conclusion stands.)

Proposed change. Delete the finding from the status. (It touches this dossier through the floor on `τBLAKE2S`; the other caps belong to another dossier.)

Finding 45 (minor). The executable verifier needs the Flock phase’s definition, which no roadmap has started

Evidence. Planning. Every proof carries a Flock transcript: the floor is 8 compressions (`hash.rs:283-286`), so the fixture of Layer 12, “a proof dumped from the pinned Rust prover ... accepted by `verify`” (`bp:1237-1238`), cannot pass before a Lean Flock verifier exists, with the circuit walk and the constants. The hole for the transcript and `verify` (K3) lists “S (schedules, phase Defs)” among what it consumes (`bp:614`); the Flock Def is “the inhabitant is #3’s” (`bp:609`), and #3’s first gate has not started.

Proposed change. Split the hole of the Flock phase (P6) as every other phase is split. The definition with its completeness (the schedule of finding 35, the verifier, the circuit walk transcribed, the constants in `Parameters`, an honest run on a toy circuit) is a hole that can be taken now and tested against the Python verifier’s walk; the security stays #3’s. State in the hole for the transcript and `verify` (K3) that its fixture waits on it.

Finding 46 (note). The pinned Rust prover of Flock is not perfectly complete

Evidence. A finding against the source, for `docs/leanvm-target.md`. The dossier’s section 4.1 and its probe 9.2 (see the evidence of finding 38).

Classification. The Lean honest prover **deliberately deviates** from the Rust prover; the reason is that the specification’s protocol is perfectly complete and the Rust’s shortcut is not; the theorem it owes is `baseProver_complete` for the Lean `prove` (not for the Rust), and the fixture “a Rust-produced proof is accepted” is unaffected, the verifiers being the same.

Finding 47 (note). The error for a fixed committed polynomial is not the error after compilation

Evidence. For the dossier on the compiled verifier. The commitment is list binding only (`b-polynomial-commitment-scheme.tex:4`), so `c-flock:264` “The commitment determines q_{flock} ” holds up to a list. The Rust’s parameter analysis multiplies the ring switching degree by the list size: `whir_config.rs:540-541` “A degree- d identity test unioned over a Johnson list of size L fails with probability at most $dL/|F|$ ”, and `:564-567`. Annex A’s 2^{-160} and the blueprint’s `flockError` have no such factor, which is right in the ideal oracle model; `niError` (`bp:1234-1235`) must have it, for every challenge of every phase. Issue #3 names the obligation (“Joint candidate lists”).

Finding 48 (note). The Fiat–Shamir seed binds a constant that cannot be recomputed at the pin

Evidence. Extends the status file’s finding on the seed (F14). `08:55` says the seed covers “Flock’s BLAKE2s RICS matrices”. The Rust binds `RICS_DIGEST`, of which `hash.rs:259-275` says: “baked as an opaque constant ... That recipe needed the materialized matrices, which this module no longer builds ... To recompute it, check out the last commit that still had `build_matrices`”. The Python copies the bytes (`verifier.py:629`). The constant is transcribed from its source (the blueprint’s Category B) and has no owner in the blueprint; no theorem depends on its value.

Finding 49 (note). Stale comments in the Rust on what binds the counter and the flags

Evidence. `cpu/mod.rs:568-570` says “Message, chaining-value, and output words bind through the memory bus; counter and flags bind through bytecode”, and `cpu/mod.rs:619-620` says “bytecode binds the counter and flags”. (The citation check corrected the quotation: the dossier attributed the words “bytecode binds the counter and flags” to both places; they are at `:619-620` only.) The table

reads the metadata cell from memory (`tables.rs:904-907` “The metadata rides the memory bus like every other operand”), as the specification says (`07:120`, `135`). No effect on the protocol; the Lean should cite the table.

The citation check found two more stale comments of the same kind: `zerocheck.rs:389-390` says “The prover sends $(G(1), G(\infty))$ ”, but the stream carries the coefficients $c_1 = G(0) + G(1) + G(\infty)$ and $c_2 = G(\infty)$ (`zerocheck.rs:118` with `transcript.rs:63-71`); `hash_flock.rs:8-9` speaks of “the reduction’s two tower-field claims”, while one ring-switched claim is built (`cpu/mod.rs:767`, `hash.rs:801-808`).

Finding 50 (note). A weighted claim carries the 2^μ cube values of its weight

Evidence. For the dossier on the opening. `Weight` (`Spine/Seams.lean:103-109`) has the field `onCube : CMLPolynomialEval E μ`, a vector of 2^μ elements of $\mathbb{E}$. The Flock verifier’s output statement contains one. As a specification this is fine; as a function that runs it builds a table of 2^μ values (μ up to 28) that no deployed verifier builds (`ring_switch.rs:63-66`). The field `mle_eq`, for the ring switching weight, is the boxed identity of `a-ring:150-154`, owed by the Flock roadmap.

8.7 The opening and the compilation

8.7.1 The opening and the compilation: the transcript as deployed

Short forms in this section. Specification files are under `doc/leanvm/body/`: `03:` is `03-proving-primitives.tex`, `04:` is `04-committing-the-witness.tex`, `08:` is `08-end-to-end-protocol.tex`, `A:` is `a-ring-switching.tex` and `B:` is `b-polynomial-commitment-scheme.tex`. Rust files: `whir.rs`, `whir_config.rs`, `whir_induce.rs`, `whir_ntt_ext.rs` and `stack_open.rs` are under `crates/pcs/src/`, and `pcs/merkle.rs` is `crates/pcs/src/merkle.rs`; `fs/` is `crates/flat_shamir/src/`; `mod.rs` is `crates/lean_vm/src/cpu/mod.rs`, and `pcs.rs`, `witness.rs`, `leaf.rs` are under `crates/lean_vm/src/`; `primitives/src/` is `crates/primitives/src/`. `py:` is `python-verifier/verifier.py`. These are at leanVM’s pin `a386121f`. `bp:` is the blueprint `docs/roadmap/protocol-blueprint.md` at leanerVM `b435631` (the checkout `144c5aa` shifts its lines after line 168 by four); Lean files named without a directory are under `LeanerVM/Protocol/` at `b435631`; `ArkLib` is cited at its old pin `dca90385` unless a revision is named. A bare line number (`:1012`) is in the file the column heading or the sentence names. The number after the colon is the line.

The leanVM working tree was moved off the pin on 2026-09-30 at 09:42, not by this review (its `reflog: pull: Fast-forward`, then `checkout: moving from main to feat/drop-prover-public-input-challenge`; HEAD `248da071`). Every line cited here was read on 2026-09-29 while HEAD was `a386121f`, and the ones quoted were re-verified on 2026-09-30 with `git show a386121f:<path>`.

$P \rightarrow V$ is a prover message (a read of the stream, bound into the Fiat–Shamir state as it is read), V a verifier computation or check, $V \rightarrow P$ a challenge (one squeeze of the chain per element of $\mathbb{E}$).

8.7.1.1 From a pooled claim to a weighted claim on the stack

A pooled claim is $\tilde{P}_i(\zeta) = c$ on one column; through the stacking selectors it is $\sum_w W(w) \cdot \tilde{q}(w) = c$ with $W(w) = \text{eq}((\zeta, \text{sel}_i), w)$ (`04:14-18`; `08:97`: “Each pooled claim is one evaluation of a column at a point, its value already supplied; via the stacking selectors (§4.1) it becomes a weighted sum $\sum_w W_j(w) \tilde{q}(w) = c_j$ over the stacked witness. (The ring-switched claim arrives already in this form, its weight supported on the q_{flock} region.)”).

Table 8.16: From a pooled claim to a weighted claim, as deployed.

Step	Specification	Rust	Python
A column claim becomes a slot claim	08:97; 04:14-18	mod.rs:790-814 slot_claims: SlotClaim::Point { offset, low_point, value } for a committed column, SlotClaim::Strided { offset, slot, stride_log, point, value } for a BLAKE2S value limb (“virtual”, routed to q_{flock})	py:520-522 ColumnClaim.on_stack: point = layout.placements[column].stack_point(self.point, layout.stack_log); Placement.stack_point (py:295-297) inserts the selector bits around the point, the low bits for a strided limb
The weight of a point claim	04:18 $W(w) = \text{eq}((\zeta, \text{sel}_i), w)$	stack_open.rs:294-303 stack_claim_eq_at: eq(low_point, x[..n]) · $\prod_k (\text{sel}_k ? x_k : 1 + x_k)$ with sel = offset >> n; stack_open.rs:305-323 the strided form eq(slot bits, x[..stride]) · eq(point, x[stride..block]) · eq(sel, x[block..])	py:522 lambda x: eq_eval(point, x) on the full stack point
The ring-switched claim’s weight	08:91, A:46 $W(u) = \Phi(\text{eq}(r, u))$ on q_{flock} , lifted by its selector	stack_open.rs:45-48 b(x) = eq(sel, x_hi) · $\sum_i \lambda^i \cdot \text{MLE}(\text{rs_eq_ind_i})(x_{\text{lo}}) + \sum_j \lambda^{(n_{\text{rs}}+j)} \cdot \text{eq}(\text{claim}_j, x)$; verifier stack_open.rs:530-546	py:1411-1412 ringswitch = (lambda x: qflock.eq_above(x) * ringswitch_weight(x[: qflock.variables])), ringswitch_target)

The verifier never materializes a weight: it evaluates the batched weight’s extension once, at WHIR’s terminal point (stack_open.rs:529-548 “Evaluate the lifted weight once, at the terminal sumcheck point”; py:1362, py:1079). That is what “MLE-friendly” (03:113) buys.

8.7.1.2 The batching challenge λ and the assignment of its powers

Table 8.17: The batching challenge and its powers, as deployed.

Step	Specification	Rust	Python
When λ is drawn	08:98 “drawn after every claim value is bound”	stack_open.rs:397-400: after the six ring-switching map challenges (stack_open.rs:394), let lambdas = powers(ps.sample(), ring.claims.len() + point_claims.len()); with the comment “Nothing is observed first: every claim value reached the caller through a binding stream read”; verifier stack_open.rs:518	py:1361 scales = powers(transcript.sample(), len(claims)), after ring_switch drew its six (py:1343)
Which claim takes which power	08:98 “Claim j takes the weight λ^{j-1} , the ring-switched claim first and the pooled column claims after”	stack_open.rs:401 let (lambdas_rs, lambdas_pd) = lambdas.split_at(ring.claims.len()); the ring-switched claims take $\lambda^0, \dots$, the point claims the rest in finish_claims order (mod.rs:656-667: bus, table, public input)	py:1413 [ringswitch, *(c.on_stack(layout) for c in claims)] with claims = [*bus.claims, *table_sumcheck_claims, *public] (py:1395, 1402)
Number of claims	J (08:98)	1 + 112 = 113 (one ring-switched claim, mod.rs:767; the pool as the dossier on the table sumcheck, gt-table-pub.md, section A.3, gives it)	same
The batched target	08:99 $C_\lambda = \sum_j \lambda^{j-1} c_j$	stack_open.rs:521-527 (verifier), :413-415, 257, 285 (prover)	py:1362 dot(scales, values)
The batched weight	08:99 $W_\lambda = \sum_j \lambda^{j-1} W_j$, “the verifier evaluating W_λ itself”	stack_open.rs:531-546 eval_b_at	py:1362 lambda point: dot(scales, [weight(point) for weight in weights])

The three sources agree on every row (the dossier on Flock and ring switching, gt-flock-ring.md, section 6, gives the ring switching; the dossier on the table sumcheck, gt-table-pub.md, section A.3, the pool). powers starts at λ^0 in both implementations (primitives/src/field/mod.rs:51-61; py:186-188), matching λ^{j-1} for $j = 1$. (The citation check corrected the dossier’s whir_induce.rs:99-107, which is power_weights, the per-query weights inside a level; both start at λ^0 .)

8.7.1.3 What runs after λ : the question settled

Specification. 08:99: “Fold the J claims into $W_\lambda = \sum_j \lambda^{j-1} W_j$ and $C_\lambda = \sum_j \lambda^{j-1} c_j$, and run the inner-product PCS opening on $\sum_w W_\lambda(w) \tilde{q}(w) = C_\lambda$ (Annex B), the verifier evaluating W_λ itself. This one opening discharges every pooled claim; there is no separate reduction sumcheck.” Annex B’s **Open** takes $J_0 \geq 1$ weighted claims and its level-0 step 1 is “(batch) V sends $\lambda \leftarrow \mathbb{E}$; the batched claim is $(W, \sigma_0) := (\sum_t \lambda^{t-1} W_t, \sum_t \lambda^{t-1} c_t)$ ” (B:108–110): §8.5’s λ is the level-0 batching challenge of Annex B’s opening protocol (Protocol B.6 at the pin; the dossier’s “Protocol B.1” here is the blueprint’s number, as the citation check noted, finding 63), with $J_0 = J$. Its step 2 is the level’s sumcheck, ℓ_0 rounds (B:111).

Rust. `stack_open.rs:452-461` hands `(stack, b_stack, target)` to `recursive_prover_with_basis` (“One WHIR over the full stack against the combined claim”). Its first act:

```
let (mut sc_prover, start_msg) =
  sumcheck_span.in_scope(|| SumcheckProver::new(witness, b_initial, target, lane_block));
send_msg(ps, start_msg, target);
```

`crates/pcs/src/whir.rs:1324-1326` at `a386121f`

`send_msg` transmits a round polynomial (`whir.rs:509-511`). Then, `whir.rs:1328-1334`, six times: sample r_j , fold, send the next round polynomial. Only after that is the first recursive root sent (`whir.rs:1358` `ps.add_root(&wtms_1.root());`). The verifier mirrors it: `whir.rs:2087-2090` `let mut t_r = target;` `let Some(mut running_quad) = rcv_quad(vs, t_r),` `:2100` `replay_fold_rounds(vs, initial_k, ...),` `:2104` `vs.next_root();`

Python. `py:1362` calls `verify_whir(transcript, stack_log, log_inv_rate, dot(scales, values), root, ...)`, whose first read is `py:1012` `running_quad = transcript.sumcheck_round_poly(3, target)`, then `py:1017-1024` the fold rounds, then `py:1034` the next root.

So. In the deployed protocol there is **no sumcheck between λ and WHIR**; the first message after λ is WHIR’s first round polynomial. The sumcheck rounds that reduce the weighted claim are **WHIR’s own**: $\mu = 6 + \sum_i k_i + n_{\text{res}}$ rounds in all (`whir_config.rs:748-751` “`initial_k + \sum k (L1+) + yr_log_n = log_n`”), interleaved with the level roots, the out-of-domain samples, the grinding nonces, the query positions and the per-level batching challenges. The running claim is not $\sum W_\lambda \cdot q$ throughout: at every level the out-of-domain claim and the query-consistency claim are glued into it with powers of that level’s λ_i (`whir.rs:1131-1154` `glue_pending`, verifier `:1681-1701` `batch_level_claims`; `py:1058-1063`), and the protocol closes on one equation $\text{weight} \cdot \tilde{f}_{\text{final}}(\rho_{\text{tail}}) = t_r$ (`whir.rs:2308`; `py:1082-1083`), where weight is $\tilde{W}_\lambda$ at the rotated terminal point plus every glued weight (`whir.rs:2264-2307`). There is never an evaluation of $\tilde{q}$ at a point; q is folded, and the folds are committed.

8.7.1.4 What the blueprint specifies instead

- Layer 10 (`bp:1112-1114`): `openingPhase (μ) (J) ... (pSpec := V_to_P : E ; sumcheck (W $_\lambda$ · q) rounds ; the final evaluation query), error “($(J - 1)/|\mathbb{E}|$ on λ , $2/|\mathbb{E}|$ per round”. The section is titled “the claim pool, the opening sumcheck, and the oracle protocol” (bp:1101), and the scope says “the claim pool and the opening sumcheck, as ArkLib reductions” (bp:127).`
- Layer 11 (`bp:1173`): `whirOpen (params) : OracleReduction ... (StmtIn := Fin J → WeightedClaim  $\mu$ ) ...`, “stated exactly as Protocol B.1: batch, ℓ_i sumcheck rounds, commit, ...” (`bp:1185`).
- Layer 12 (`bp:1214-1215`): “The compiled oracle protocol: WHIR in place of the evaluation oracle. `def leanVmIopp ... (0StmtIn := codeword oracles)`”.
- The convention *The oracle* (`bp:317`): “OracleInterface query Vector E μ_{stack} and answer `eval2Mle q`”; Layer 0 (`bp:637-642`) and the built instance (`LeanerVM/Protocol/Field.lean:102-106` at `b435631`).

Read together, the compiled verifier of the blueprint runs λ , a μ -round sumcheck on $W_\lambda \cdot q$ (two coefficients a round), receives a value $v = \tilde{q}(\rho)$, checks $\tilde{W}_\lambda(\rho) \cdot v$ against the running claim, then runs `whirOpen` on

the one claim $(\text{eq}(\rho, \cdot), v)$, which, “stated exactly as Protocol B.1”, starts with its own level-0 batching challenge ($J_0 = 1$) and its own μ -round sumcheck. That transcript has one extra challenge, μ extra round polynomials and one extra scalar, and is not leanVM’s: a `verify` that is “the Fiat–Shamir compilation of that oracle verifier” (bp:27-29) cannot accept a Rust proof, and `verify_iff_compiled` (bp:1218-1221) is false for every proof the pinned prover emits.

This is the divergence between the pinned §8.5 (08:94-101) and the blueprint’s Layer 10 that the review set out to classify. It is **an error of the blueprint**, introduced by the choice of the evaluation interface at Layer 0: with an evaluation oracle, the only way to discharge a weighted claim in the ideal model is to reduce it to an evaluation, and the extra sumcheck follows (finding 51).

8.7.1.5 The alternatives, and the recommendation on the oracle interface

(i) Keep the evaluation oracle and the extra sumcheck; owe a theorem relating the two protocols. The ideal-model theorems stay as they are, but `verify` can no longer be the Fiat–Shamir compilation of the oracle verifier (section 8.7.1.4). The theorem owed would be: “the compiled protocol *with* the extra sumcheck and WHIR on an evaluation claim is secure $\Rightarrow$ the deployed protocol is secure”. No such implication exists between two protocols with different transcripts; each must be proved on its own, so the security of `verify` would rest on a WHIR theorem for weighted claims anyway (Theorem B.7 is stated for J_0 weighted claims), and the opening sumcheck of Layer 10 would be proved and never used. Rejected.

(ii) The inner-product oracle interface. A query is a weight $W : \text{Weight } \mu$, the answer is $\Sigma_w W(w) \cdot q(w)$: the idealization of Definition 3.13 (03:112-118: “An *inner-product commitment scheme* commits to g and later proves any such claim ... An evaluation claim $\tilde{g}(r) = c$ is the case $W = \text{eq}(r, \cdot)$ ”). The opening phase is then “ λ , then one weighted query at W_λ compared with C_λ ”, a one-challenge component with error $(J - 1)/|E|$ and no sumcheck; WHIR realizes the one query. Checked at the pins (probe InnerProductOracle.lean, ?? B.2.1.1, exit 0 on 2026-09-29 against ArkLib dca90385):

- ArkLib’s `OracleInterface` is a class with a query type and an answer function (ArkLib/OracleReduction/OracleInterface.lean:55-57 at dca90385: `class OracleInterface (Message : Type u) where Query : Type v; toOC : OracleContext Query (ReaderM Message)); Weight  $\mu$  is a structure in Type (Spine/Seams.lean:103-109 at b435631), so Query := Weight n is accepted and the answer is W.pair q (Seams.lean:112-113).`
- The evaluation query is the special case `eqWeight p`, by the built lemma `eqWeight_pair` (ClaimWeights.lean:51-54 at b435631): the current `evalOracle_answer` becomes a corollary.
- ArkLib’s functional commitment scheme over this interface has for opening statement (`commitment, weight, value`) (ArkLib/Commitments/Functional/Basic.lean:62-64 at dca90385, copied in the probe): it is an inner-product commitment scheme, which is what Layer 11 must instantiate (“Commitment.Scheme ... The commitment interface WHIR’s commit and open instantiate”, bp:249).
- The opening verifier “read λ , query at W_λ , compare with C_λ ” is an `OracleVerifier` over `ProtocolSpec 1` with `keepOracles` (the probe’s `openVerifier`).

Consequences, layer by layer:

Table 8.18: The inner-product oracle interface: what changes, layer by layer.

Where	Change
Layer 0 (Field.lean)	replace <code>evalOracle</code> by the inner-product instance; <code>Weight</code> (and <code>Weight.pair</code>) move below <code>Field.lean</code> (today they are in <code>Spine/Seams.lean</code> , above it); <code>evalOracle_answer</code> becomes <code>answer q (eqWeight p) = evalMle ...</code> . The only consumers of <code>OracleInterface.answer</code> on a <code>Column</code> at b435631 are three theorems on verifier-computed columns (FixedColumns.lean:59-61, 97-99, 113-115) and the tests (tests/LeanerVMTTests/Protocol/Field.lean:30-44); the former are restated with <code>evalMle</code> , since the index and bytecode columns are never oracles. (The citation check found this list incomplete: tests/LeanerVMTTests/Protocol/FixedColumns.lean:32, 49, 76, 115 also use <code>OracleInterface.answer</code> on columns, and the docstring of <code>LeanerVM/Protocol/Stack.lean:18</code> refers to it; the recommendation is unaffected in kind, the list of files to change is longer.) The convention <i>The oracle</i> (bp:317) and Layer 0 (bp:637-642) change accordingly

Where	Change
The spine	nothing in the types: <code>TheOracle I</code> , <code>Seam.flock</code> , <code>Seam.done</code> , <code>Phases.opening : Phase.Def I (I.Stmt × FlockOut I) Unit</code> stay. No built phase queries the oracle (the public-input verifier queries only its own message, <code>PublicInput.lean:222-231</code> at <code>b435631</code>), so no built proof changes. (That last step is the dossier’s inference: every type that mentions <code>[TheOracle I]</code> , for example <code>PublicInput.lean:242, 246, 261, 263</code> , elaborates against whichever <code>OracleInterface</code> instance is in scope, so the citation check marks it as needing a build to confirm: unverified.)
Layer 10	the opening phase is the batching component alone (the batching hole, <code>G3, bp:600</code>): <code>schedule V_to_P : E</code> , <code>verifier = batch</code> , <code>query</code> , <code>compare</code> ; error $(J - 1)/ E $; knowledge state function “some pooled claim is false of q ” before λ , “the batched claim is false of q ” after, with the query making the verdict; completeness by linearity of <code>Weight.pair</code> . “ $2/ E $ per round” leaves Layer 10 (those rounds are WHIR’s and carry $2L_i/ E + 2^{\ell_i-j}\varepsilon_i$, Theorem B.7). The title and <code>bp:127</code> lose “the opening sumcheck”
Layer 11	<code>whirOpen</code> takes the J weighted claims and its level-0 batching challenge is the opening phase’s λ : the compilation replaces the whole opening phase by <code>whirOpen</code> , not the query alone. <code>encode</code> must be over E as well as K (section 8.7.1.11)
Layer 12	<code>leanVmIopp = commit' » bus » table » pub » flock » whirOpen</code> , where <code>commit'</code> sends the level-0 codeword (an interleaved K -valued word with position queries) and the front <code>bus ... flock</code> is the spine’s; then the Merkle compilation, then Fiat–Shamir with grinding (section 8.7.3.1)
Layer 13	unchanged in shape; its hypotheses change with section 8.7.3.1

(iii) **End the oracle protocol at the pool.** Drop the opening phase from the oracle protocol: the composed reduction ends with output statement `I.Stmt × FlockOut I` and output relation `Seam.flock`, and the compilation appends `whirOpen` there. Same transcript as (ii), same compile theorem; it changes the master theorems’ shape (`Seam.done` becomes `Seam.flock`, five phases instead of six) and leaves the stack’s oracle interface unused, which misleads (an interface that is never queried says nothing). Acceptable; (ii) is preferred because it keeps the spine’s statements and makes the ideal object the specification names.

Recommendation: (ii). It is the specification’s own object (`03:112-118`; `B:4` “we commit to a K -valued multilinear, and opening claims are in E ”), it matches `ArkLib’s Commitment.Scheme` without adaptation, it makes `verify_iff_compiled` statable with leanVM’s transcript, and it removes a μ -round sumcheck from the trusted surface of Layer 10.

Negative results. Checked and agreeing across the three sources: the moment of λ (after the six ring-switching challenges, before any WHIR message); the power of each claim; $J = 113$; the batched target and weight; that the verifier evaluates $\widetilde{W}_\lambda$ exactly once. Checked: the spine’s `WeightedClaim.Holds (Seams.lean:123-124)` is $\Sigma_w W(w) \cdot q(w) = c$ over the cube with q lifted by `ofK`, the specification’s $\langle W, g \rangle = c$ (`03:115`); and `Weight.mle_eq` ties the verifier’s evaluator to the cube values, which is what “MLE-friendly” needs. No finding on the pool’s translation into weighted claims.

8.7.1.6 WHIR level by level

Notation: μ the stack’s log-size; $k_0 = 6$ the initial fold (`whir_config.rs:67`), $k_i = 4$ after (`:68`), the residual $n_{\text{res}} \leq 5$ (`:86`); r the number of recursive levels (`level_steps`); t_i the query count of level i ; z an out-of-domain (OOD) point. The specification’s column is Protocol B.6 (`B:100-123`); the Rust column gives the prover and the (succinct) verifier, both in `whir.rs` unless another file is named; the Python column is `py:1006-1087`.

Table 8.19: WHIR’s message schedule per level, as deployed.

#	Message	Specification, Protocol B.6	Rust prover / verifier	Python
0	$V \rightarrow P \lambda$ (level-0 batching), $J_0 = 113$ claims	step 1 at $i = 0$ (B:110)	outside WHIR: stack_open.rs:400 / :518 (section 8.7.1.2)	py:1361
1	$P \rightarrow V h_1$: round polynomial of $\Sigma_x q(x) \cdot W_\lambda(x) = C_\lambda$ (2 scalars: c_0, c_2 ; c_1 derived)	step 2, $j = 1$ (B:111)	:1324-1326 / :2087-2090	py:1012
2	$6 \times (V \rightarrow P r_j; P \rightarrow V \text{ next round polynomial})$, the lane fold on the stack’s top 6 coordinates	step 2, $j = 1.. \ell_0$ (B:111-112); the row index is the stack’s most significant ℓ_0 coordinates, $f(u, x) := q(x, u)$ (B:380)	:1328-1334 / :2100 (replay_fold_rounds, :1641-1655); the point is rotated by 6 at the end, :2297-2306	py:1017-1024; rotation py:1078-1079
3	$P \rightarrow V$ root of $f^{(1)}$ (2 scalars), the fold $f^{(1)} = \tilde{q}(r_1..r_6, \cdot)$, $\mathbb{E}$ -valued, interleaved 2^4 lanes	step 3(a) (B:115)	:1340-1358 / :2104	py:1034
4	OOD: $V \rightarrow P z \in \mathbb{F}^{\mu-6}$ ($\mu - 6$ squeezes); $P \rightarrow V y$ (1 scalar); $P \rightarrow V$ intro round polynomial for the claim ($\text{eq}(z, \cdot), y$) (2 scalars)	step 3(b) (B:116): z, y ; no intro message	send_ood, :1201-1208 / replay_ood, :1670-1675; count_ood_samples[1] = 1 (whir_config.rs:677-684, test :1032-1034)	py:1035-1037
5	$P \rightarrow V$ grinding nonce (1 raw scalar); V checks 17 bits	absent	:1366 / :2117 (grind_check, fs/transcript.rs:314-323); QUERY_GRINDING_BITS = 17 (whir_config.rs:60)	py:1039
6	$V \rightarrow P t_0$ positions in $[0, 2^{\mu-6+\rho})$, by chunking squeezes	step 3(c): “ t_i i.i.d. columns” (B:117)	sample_queries_ordered, :1175-1193: $\lfloor 192/d \rfloor$ d -bit chunks per squeeze, duplicates kept / :2122	py:943-951, py:1041
7	$V \rightarrow P \lambda_0$, the level’s batching challenge	step 1 at $i = 1$ (B:110), after step 3(d)’s claims are formed	:1373 / :2123	py:1044
8	$P \rightarrow V$ Merkle openings of level 0 at the positions (rows of n_{lanes} words and one octopus), not absorbed	step 3(c): “it reads $C^{(i)}[\cdot, x_q]$ ” (B:117)	hint_merkle, :1380 / recv_level_rows, :2125-2135 (fs/transcript.rs:263-279)	py:1049-1050 (raw paths)
9	$P \rightarrow V$ intro round polynomial for the query batch (2 scalars), with claim enforced_sum_0 = $\Sigma_q \lambda_0^q \cdot \langle \text{row}_q, \text{eq}(r_{\text{lane}}, \cdot) \rangle$ computed by V from the openings	step 3(d): the t_i consistency claims (W_{x_q}, c_q) with $c_q = C_{\text{fold}}[x_q]$ (B:117-118); no intro message , the batch’s h_1 is sent after λ	:1405-1406 / :2140-2144 (induce_sumcheck_enforced_sum, whirl_induce.rs:203-218)	py:1051, py:1057
10	V glues: running quad += $\lambda_0 \cdot \text{quad_ood} + \lambda_0^2 \cdot \text{quad_query}$, claim t_r += $\lambda_0 \cdot y + \lambda_0^2 \cdot \text{enforced_sum}$	step 1 at level 1: $(W, \sigma_0) := (\Sigma \lambda^{t-1} W_t, \dots)$ with the order residual, OOD, queries (B:118, B:110)	glue_pending, :1131-1154 / batch_level_claims, :1681-1701	py:1058-1063
11	level $i \geq 1$: $4 \times (V \rightarrow P r; P \rightarrow V \text{ round polynomial})$; then as rows 3 to 10 with t_i , the leaf $3 \cdot 2^4$ words	steps 2–3 (B:111-119)	:1420-1425, 1509-1560 / :2191-2194, 2311-2384	py:1017-1063 (loop)
12	last level: after its 4 folds, $P \rightarrow V f_{\text{final}}$ ($2^{n_{\text{res}}}$ scalars in the clear); grind; $V \rightarrow P t_{\text{last}}$ positions; $V \rightarrow P \lambda_{\text{last}}$; openings; intro; glue; then $n_{\text{res}} \times (V \rightarrow P r; P \rightarrow V \text{ round polynomial except after the last } r)$	step 4 (B:120): f_{final} , queries, λ, ν_L rounds, “ V closes by evaluating $\tilde{f}_{\text{final}}$ ”; no intro; nothing said about the last message	:1431-1476 / :2197-2262	py:1031-1032, py:1065-1074
13	terminal check $\text{weight} \cdot \tilde{f}_{\text{final}}(\rho_{\text{tail}}) = t_r$ with $\text{weight} = \tilde{W}_\lambda(\rho \text{ rotated}) + \Sigma_{\text{levels}} \beta_i \cdot (\text{induced weight of the level’s queries at } \rho) + \Sigma_{\text{ood}} \beta \cdot \text{eq}(z, \rho)$	the closing check of step 4 (B:120), the consistency claims being $\langle W_{x_q}, f_{\text{final}} \rangle$ (B:120, Lemma lem:colweight)	:2264-2308 / induce_sumcheck_evaluate_at_residual, whirl_induce.rs:228-293	py:1075-1083, _induced_weight, py:971-989

The two verifiers agree with each other message for message (checked by walking `recursive_verifier_with_basis_succinct` and `verify_whir` side by side; the Python is a transcription of the Rust). Against Protocol B.6 the deployed schedule differs in four ways: the per-claim intro messages, the grinding, the lane relayout, and the omitted last round message; they are items 6 to 9 of table 8.24.

Also: the OOD count is one per level ≥ 1 and zero at level 0 (`whir_config.rs:677-684`, `:829-841`; `whir.rs:1261-1265` “L0 takes no OOD sample”), as Annex B (B:105 “No OOD sample is taken”) and the blueprint (`bp:1186-1187`) say.

Element counts (from table 8.19, section 8.7.1.7 and section 8.7.1.14).

Message or challenge	Elements of $\mathbb{E}$
round polynomial of a fold round	2 scalars, c_0 and c_2 (c_1 derived); its challenge r is 1 squeeze
root of a level	2 scalars
OOD point after the first root	$\mu - 6$ squeezes
OOD answer y	1 scalar, one OOD sample per level ≥ 1 , none at level 0
intro round polynomial (OOD claim; query batch)	2 scalars each
grinding nonce	1 raw scalar at every level
query positions	$\lfloor 192/d \rfloor$ chunks of d bits per squeeze, duplicates kept
level batching challenge λ_i	1 squeeze
Merkle openings	none on the stream: a side channel, not absorbed
f_{final}	$2^{n_{\text{res}}}$ scalars
round message after the last r	omitted

The ladder of folds is $[6, 4, 4, \dots]$ until at most 5 variables remain: 2 levels and a residual of 5 at $\mu = 15$, 6 levels and a residual of 2 at $\mu = 28$ (section 8.7.1.7). The query counts per level are those of `py:910` (at $\mu = 15$ and rate $1/2$: 223, then 55).

8.7.1.7 The code

- **Field and domain.** Reed–Solomon over $\mathbb{E}$ with the evaluation domain an $\mathbb{F}_2$ -subspace of $\mathbb{K}$ (B:54, B:90); the level-0 word is $\mathbb{K}$ -valued because the message is (B:306; the codeword buffer is `ArenaVec<F64>`, `whir.rs:293`, `:364`); deeper levels are $\mathbb{E}$ -valued (`ArenaVec<F192>`, `:397`) encoded with $\mathbb{K}$ twiddles by the mixed product (`whir.rs:17-19`, `whir_ntt_ext.rs`).
- **Basis.** The novel polynomial basis of Lin–Chung–Han on the standard basis $e_c = x^c$ of $\mathbb{K}$ (B:278, B:301-306): `whir_induce.rs:19-21` “Standard basis only (`v_i = x^i = F64(1 << i)`)”, `eval_sk_at_vks` (:32-52) and `normalized_sks_at` (:56-67, $\hat{W}_i(x) = s_i(x)/s_i(v_i)$), the specification’s $\hat{W}_i := W_i/W_i(e_i)$, B:299); the Python `_subspace_roots` (`py:962-968`). A position $q \in [0, 2^{\kappa+\rho})$ is the domain point `F64(q)` (`whir_induce.rs:163`, `py:983`), the specification’s $x_v = \sum_c v_c e_c$ (B:357). The message is read directly as novel-basis coefficients (B:306; the encoder is the additive NTT, `ntt::AdditiveNttF64::standard`, `whir.rs:372-373`).
- **The column weight** the verifier evaluates: $\prod_k (1 + p_k \cdot (1 + \hat{W}_k(q)))$ (`whir_induce.rs:220-224`, `py:971-977`), which is Lemma `lem:colweight`’s $\prod((1 + r_i) + r_i \hat{W}_i(x))$ (B:312) in characteristic two. Agrees.
- **Rates.** Level 0: $\rho_0 = 2^{-\rho}$ with $\rho \in [1, 4]$ chosen by the prover and announced (`whir_config.rs:41-55`, `mod.rs:123`, `149`, `167`); then the rate exponent grows by $k_0 - 3 = 3$ after the initial fold and by $k_i - 1 = 3$ after each later fold (`whir_config.rs:73-78`, `298-311`; `py:929`). Block lengths $2^{\kappa_i + \rho_i}$, all $\leq 2^{28}$, within $\kappa + \rho \leq 64$ (B:90).
- **The ladder.** Folds $[6, 4, 4, \dots]$ until at most 5 variables remain (`derive_ladder`, `whir_config.rs:260-296`; `py:924-932`): 2 levels and a residual of 5 at $\mu = 15$, 6 levels and a residual of 2 at $\mu = 28$.

8.7.1.8 The parameters

SECURITY_BITS = 128, LOG_INV_RATE_0 = 1, MIN/MAX_LOG_INV_RATE = 1/4, QUERY_GRINDING_BITS = 17, INITIAL_FOLDING_FACTOR = 6, SUBSEQUENT_FOLDING_FACTOR = 4, RS_DOMAIN_INITIAL_REDUCTION_FACTOR = 3, RS_DOMAIN_SUBSEQUENT_REDUCTION_FACTOR = 1, RESIDUAL_MAX_LOG = 5 (whir_config.rs:38-86); MIN_MU = 15, MAX_MU = 28 (pcs.rs:49-51); the Python’s copies py:900-908. The query counts per level come from the search of whir_config.rs:639-707 (section 8.7.3.2) and are tabulated in py:910 for every (ρ, μ) ; the scratch port reproduces the table (?? B.2.1.3).

The blueprint’s parameter block (bp:1157-1165) lists the six constants and ladder ... -- fold, rate exponent, queries per level with ladder_queries_eq against py:910: right, with two gaps. (a) The analysis parameter η_i (or m_i) per level is not a field of Level, yet whirError params (bp:1175) is a function of $\gamma_i = 1 - \sqrt{\rho_i} - \eta_i$ and $L_i = 1/(2\eta_i\sqrt{\rho_i})$; a Lean whirError needs the η_i as data (any admissible value proves the theorem; the Rust’s are the floating-point optimum, whir_config.rs:624-633), and acceptance of “128 bits” needs the specific ones. (b) The OOD count (1 from level 1) and the grinding bits per level are parameters too (finding 62).

8.7.1.9 The Merkle trees and the encodings

- **Tree.** Binary, 2^k leaves, flat layout (pcs/merkle.rs:11-18); leaf hash = BLAKE2s-256 of the leaf bytes (fs/merkle.rs:38-41), node = BLAKE2s-256 of the 64 bytes *left||right* (:44-51); **no domain separation** between leaves and nodes. Safe because the verifier fixes the height (open, fs/merkle.rs:174: height = num_leaves.trailing_zeros(), num_leaves a function of the announced sizes) and the leaf width (row_words, leaf_words, :139-148); a node’s 64-byte preimage can only be presented as a leaf of a tree whose height the verifier does not accept. (The citation check gives the more direct reason, the length: a leaf is BLAKE2s-256 of 512 bytes at level 0 or 384 bytes deeper, eight or six chained compressions, the committer relying on the chaining value after the leading zero blocks, pcs/merkle.rs:147-155; a node is one 64-byte compression.)
- **Leaf images.** Level 0: 2^6 words of 8 bytes (512 bytes) per position, lane-**descending**, the absent lanes a zero prefix supplied by the verifier (whir.rs:331-345, pcs/merkle.rs:144-192, fs/merkle.rs:53-58); the words are little-endian u64s (fs/merkle.rs:60-67). Deeper levels: 2^4 elements of $\mathbb{E}$ as $3 \cdot 2^4 = 48$ words, 384 bytes (whir.rs:461-467, ext_row_words, :1211-1213; py:1049, _ext_row, py:938-940). The comment whir.rs:393-395 says “one Merkle leaf of num_interleaved * 16 bytes”; the code hashes num_interleaved * size_of::() = 24 * num_interleaved bytes (:463); the comment is stale (finding 70).
- **Digests on the stream.** A 32-byte digest travels as two scalars, each two little-endian words in the low limbs and a zero third limb (hash_to_scalars, fs/merkle.rs:14-20; py:216-219); reading rejects a nonzero third limb (scalars_to_hash, :26-36; py:222-224).
- **Wire format.** One phase per level: the rows at the *distinct, sorted* positions and one “octopus” of siblings (PrunedMerklePaths, fs/merkle.rs:78-92, prune, :101-136); the verifier rebuilds every path, refuses a missing or surplus sibling, a wrong row count or width, an out-of-range position, a root mismatch (open, :163-235). The raw form (RawMerklePath, :246-264: full leaf image plus full sibling path, one per query, duplicates repeated) is what an accepting Rust verifier *emits* (into_raw_proof, fs/transcript.rs:198-203; mod.rs:702-704, 777) and what the Python and the recursion guest consume (py:392-404; the test harness crates/lean_vm/tests/verifiers/python_verifier.rs:50-80 writes it out).

8.7.1.10 What Annex B’s soundness theorem says, and on what it rests

Theorem B.7 (B:139-152, thm:rbr): the scheme is L_0 -list binding (Definition B.4, B:70-72) and its opening has **round-by-round soundness** (Definition B.5, B:74-84) for the relation R_{open} (B:133-136), with the per-message errors of the table B:141-150. The model is the interactive oracle proof: the oracles $C^{(i)}$ are read at queried columns, the challenges are uniform, the prover unbounded; the extractor is not named (soundness for a language). Inputs: the Johnson bound, proved in the annex (B:391-414); mutual correlated agreement up to the Johnson bound, Theorem B.9 (B:176-185), quoted from [BCHKS25] Theorem 4.6 and external; Lemma B.10 (B:189-198) turns it into “folding preserves lists” with the $2^{\ell-1}$ union of rows; Lemma B.11 (B:200-206) is the out-of-domain separation. Fiat–Shamir is one sentence (B:84).

8.7.1.11 The blueprint’s Layer 11 set against it

Table 8.20: The blueprint’s Layer 11 (bp:1150-1193) against the deployed WHIR.

Blueprint	Ground truth	Judgement
<pre> whirOpen (params) : OracleReduction ... (StmtIn := Fin J → WeightedClaim μ) (0StmtIn := codeword oracles), “stated exactly as Protocol B.1: batch, ℓ_i sumcheck rounds, commit, one out-of-domain sample from level 1 on, t_i queries, and the final plaintext level” </pre>	Protocol B.6 (numbered on the section counter, theorems.tex:4, 12) with the four deployed differences of section 8.7.1.6	the input type is right (J weighted claims; under (ii) its level-0 batch is the opening phase’s λ, section 8.7.1.5). The relayout, the intro messages and the omitted last message must be pinned as transcribed content (the blueprint’s Category B) for verify to accept Rust proofs; grinding is Layer 12’s. “Protocol B.1”, “Theorem B.2”, “Definition B.1” (bp:1188; the citation check corrected the dossier’s bp:1187), “Lemma B.7” (bp:1170), “Annex B.3” (bp:278), “(Annex B.4)” (bp:1172) are not the pinned numbers (Protocol B.6, Theorem B.7, Definition B.4, Lemma B.14, §B.5, §B.3); cite labels right in kind; <code>whirError</code> needs the η_i (section 8.7.1.8); the errors are $(J_i - 1)L_i/ E$, $2L_i/ E + 2^{\ell_i - j}\varepsilon_i$, $C(L_i, 2)\mu_i/ E$, $(1 - \gamma_i)^{t_i}$, t_{r-1}/ E, $2/ E$; Layer 10’s “$2/ E$ per round” is the tail’s value, wrong for the fold rounds (section 8.7.1.4)
<pre> whirOpen_rbrSoundness (mca : McaJohnson) : ... (whirError params) -- Theorem B.2, per message; “Its soundness is <i>round-by-round</i> <i>soundness for the list relation</i> relopen, not knowledge soundness” encode (κ R : ℕ) : Column κ → (Fin (2^(κ+R)) → K) -- additive NTT on the K basis </pre>	Theorem B.7 as read in section 8.7.1.10 the encoder is $\text{Enc} : \mathbb{E}^{\text{cube}} \rightarrow \mathbb{E}^{\text{dom}}$, $\mathbb{K}$-valued on $\mathbb{K}$-valued messages (B:54, 306); every level past 0 encodes an $\mathbb{E}$-valued message (whir.rs:419-485)	too narrow: encode over $\mathbb{K}$ serves level 0 only; the deeper levels need the same encoder over $\mathbb{E}$ (or over any $\mathbb{K}$-algebra), and the $\mathbb{K}$-valuedness of level 0 is a lemma about it (finding 58)
<pre> theorem encode_column_weight (x) : ∃ W_x : Weight κ, ∀ f, encode κ R f x = ⟨W_x, f⟩ -- Lemma B.7 </pre>	Lemma B.14 (B:309-315): $W_x(u) = \prod_i \hat{W}_i(x)^{u_i}$ and $\tilde{W}_x(r) = \prod_i ((1 + r_i) + r_i \hat{W}_i(x))$, which the verifier computes (whir_induce.rs:220-224)	near-vacuous as stated: every linear map $\mathbb{K}^{2^\kappa} \rightarrow \mathbb{K}$ is an inner product with some cube table, and <code>Weight</code> is inhabited by <code>(row, evalMle row, rfl)</code>, so the existential is true of any linear encode and says nothing about the novel basis. What Layer 11 needs is the <i>definition</i> <code>columnWeight x : Weight κ</code> with the closed-form <code>mle</code> and the theorem <code>encode κ R f x = (columnWeight x).pair f</code>, since <code>verify</code> evaluates that closed form (finding 59)
<pre> merkleRoot : List (Vector K leafWords) → Digest, merkleVerify : Digest → ℕ → Vector K leafWords → Path → Bool, blake2sBytes </pre>	section 8.7.1.9	<code>merkleVerify</code> is the raw path (<code>RawMerklePath::root</code>, <code>fs/merkle.rs:252-264</code>), not the wire’s pruned octopus; which one <code>Proof.merkle : List MerklePaths</code> (bp:1209-1211) holds decides which proofs verify accepts (section 8.7.1.13). The leaf image order (lane-descending, zero prefix) and the two leaf widths are not stated
<pre> Tests: encode against CompPoly’s additive NTT at κ = 3; merkleVerify on four leaves; honest whirOpen at μ = 4, rate 1/2; the parameter tables against py:910 </pre>		the honest run at $\mu = 4$ cannot use the production ladder (<code>derive_ladder</code> needs $\mu > 6$ and two levels, <code>whir_config.rs:266-267</code>, 291-293; the Rust’s tests fall back to <code>default_config</code>, :196-224; the fallback selector is <code>test_configs_for</code>, :232-244, as the citation check adds); say which shape the toy uses

8.7.1.12 Fiat-Shamir: the chain

§8.4 of the specification is **TODO** (08:44-46); §8.5’s “Setup” says only “The Fiat-Shamir initial state (§8.4) is seeded by the public input, the bytecode and Flock’s BLAKE2s R1CS matrices” (08:55). **The Rust is the only source**; the Python is a transcription of it and is checked against it below (every row agrees unless marked).

Table 8.21: The Fiat–Shamir chain, as deployed.

Element	Rust (fs/lib.rs unless named)	Python
The primitive compress(a, b) on two 256-bit halves	:18-24: BLAKE2s(a b) over 64 bytes, the words little-endian, that is the RFC compression $F(\text{PARAM_IV}, a b, t = 64, f_0 = 1)$ from the unkeyed parameter state (primitives/src/hash.rs:83-87, 209-224); the chain state is the <i>first half of the message</i> , never a BLAKE2s chaining value	py:341-343 compress(left, right) = blake2s(16 bytes of the 8 words)
Tags, lane 3 of the right half Seed	:36-39: observe 1, squeeze 2, grinding base 3, grinding nonce 4; the seeding block has no tag, “its position is its tag” (:31-35) $cv_0 = \text{compress}(iv, \text{public_input})$ (:69-73); $iv = \text{BLAKE2s}(\text{"leanvm"} \parallel \text{len}(\text{R1CS_DIGEST}) \text{ as u64 LE } \parallel \text{R1CS_DIGEST} \parallel \text{bytecode_hash})$ as four words (mod.rs:82-93, :712); the public input as its four low words, the third limbs dropped (mod.rs:99-106)	py:335-338 py:1366-1369, py:349
Observe $x \in \mathbb{E}$ Squeeze	$cv \leftarrow \text{compress}(cv, (x.c0, x.c1, x.c2, 1))$ (:85-87) $out \leftarrow \text{compress}(cv, (0, 0, 0, 2))$; $cv \leftarrow out$; challenge = (out_0, out_1, out_2) (:95-99): the challenge is 192 of the 256 state bits	py:353-354 py:356-358
Grinding base Nonce bound	compress(cv, (0, 0, 0, 3)), not written to the state (:108-110) $cv \leftarrow \text{compress}(cv, (\text{nonce}.c0, \text{nonce}.c1, \text{nonce}.c2, 4))$ (:118-120), on both sides, also when the check fails (:165-174; the status file’s finding F13)	py:380 py:382
Grinding predicate	the low bits bits of compress(base, (nonce, 4))[0] are zero; bits = 0 accepts only nonce = 0 (:47-51, :165-174)	py:381
The honest grind	the smallest u64 nonce, unbounded search (:126-155)	not the verifier’s

The blueprint’s row *Fiat–Shamir* (bp:325) states the tags, the seed and “one squeeze yields one $\mathbb{E}$ challenge (three low words)” correctly. It does not say that the step is BLAKE2s-256($cv||block$) with cv in the *message* and the RFC chaining value fixed at the parameter state; Layer 12’s `FsState.observe ... -- lane 3 = 1` (bp:1204) and the leanISA row “compress (Layer 1): the compression the transcript, Merkle hashing and the Flock circuit share” (bp:216) invite $F(cv, x||tag, \dots)$ with cv as chaining value, which is a different function. It is transcribed content (the blueprint’s Category B); the transcription must say `hash64 (cv ++ x ++ tag) = compress iv64 (cv ++ x ++ tag) 64 true false` in the terms of leanISA’s `compress` (LeanerVM/Semantics/Blake2s.lean:105 at b435631) (finding 64). The citation check qualifies both halves. bp:216 also says “the byte hasher ... is the RFC 7693 wrapper over `compress` with the last-block flag only”, which the dossier does not mention, so “invite” is an inference; and the proposed transcription is imprecise: leanISA’s `compress` takes ($h : \text{Vector UInt32 } 8$) ($m : \text{Vector UInt32 } 16$) ($t : \text{UInt64}$) ($f0 f1 : \text{UInt32}$), the flags being `0xFFFFFFFF` when set, so `true false` does not typecheck, and `iv64` must be the parameter-folded IV (`PARAM_IV, primitives/src/hash.rs:64-81`), not the bare IV. The source’s own field comment calls cv “The 256-bit chaining value: a Merkle–Damgård hash of the transcript so far” (fs/lib.rs:58), in the Merkle–Damgård sense, which the check names as the likely origin of the ambiguity.

8.7.1.13 The proof object and the stream order

`Proof<M = PrunedMerklePaths> { stream: Vec<F192>, merkle: Vec<M> }` (fs/transcript.rs:9-12); `RawProof = Proof<RawMerklePath>` (:19) is what the Rust verifier emits after accepting (:198-203) and what the Python reads (py:322-330: a byte file of 24-byte scalars and a byte file of leaf words followed by sibling digests, no lengths). The blueprint’s `Proof` (bp:1209-1211: `stream : List E, merkle : List MerklePaths`) cites transcript.rs:9-19, which defines both; `merkleVerify ... Path` (bp:1182) is the raw shape; the fixture “a proof dumped from the pinned Rust prover” (bp:1237-1238): the prover emits the pruned form (`ps.into_proof()`, :138-143), the verifier the raw one. Which one `verify` takes is unspecified, and they are not equivalent as sets of accepted byte strings:

- the raw form repeats a row for a duplicated position and carries every sibling, the pruned form dedups and shares (fs/merkle.rs:69-76, 101-136);
- at level 0 the pruned form carries n_{lanes} words per row and the verifier supplies the zero prefix (fs/merkle.rs:53-58; whir.rs:2130-2131), while the raw form carries the full 64-word image (py:1049: `lanes if level == 0 else 3 * lanes`) and **the Python does not check that the prefix is zero**: run standalone on an adversarial raw proof it accepts a commitment whose absent lanes are nonzero (soundness-neutral, check M2 of table 14.5; a divergence in accepted sets nonetheless). The citation check agrees by reading, not by running: the Python never checks that a word is zero and has no notion of n_{lanes} , so such a proof passes its Merkle check; that the rest of its verification then passes is the dossier’s argument, plausible and unverified by execution.

A choice must be made and named: the pruned wire format is “what the Rust prover emits”; the raw one is what the recursion guest verifies (finding 60).

The stream, in order (`mod.rs:711-769`; the phases’ internals are in the sibling dossiers and in sections 8.4.1, 8.5.1 and 8.6.1):

Table 8.22: The proof stream, as deployed.

#	Scalars read ($P \rightarrow V$, each absorbed as read)	Squeezes ($V \rightarrow P$)	Source
1	seed	—	<code>mod.rs:712</code>
2	8: $\kappa_{\text{mem}}, \tau_0 \dots \tau_5, \rho$	—	<code>mod.rs:144-149</code> ; <code>py:1372</code>
3	2: the commitment root	—	<code>mod.rs:714</code> , <code>pcs.rs:136-138</code> ; <code>py:1382</code>
4	bus: 2 roots; per GKR layer the round polynomials and children	α (4), β , per layer the combiners and the two combination challenges; then 5 boundary evaluations read	the bus dossier, <code>gt-bus.md</code> , sections A.3–A.4 (section 8.4.1.3)
5	table sumcheck: τ_{max} round polynomials (3 scalars each), then 104 column values	ξ , then τ_{max} challenges	<code>gt-table-pub.md</code> , section A.1
6	2: c_0, c_1	r_{pi} first	<code>mod.rs:745-749</code> ; <code>py:1398-1399</code>
7	Flock: zerocheck and lincheck messages	their challenges	<code>gt-flock-ring.md</code> , section 2.2
8	—	6 ring-switching challenges	<code>stack_open.rs:394</code> / <code>:513</code> ; <code>py:1343</code>
9	—	λ	<code>stack_open.rs:400</code> / <code>:518</code> ; <code>py:1361</code>
10	WHIR: as rows 1 to 13 of table 8.19; Merkle phases on the side channel, one per level	as table 8.19	<code>whir.rs:2087-2308</code> ; <code>py:1006-1087</code>
11	<code>vs.finish()</code> : the stream and the phase list fully consumed	—	<code>mod.rs:769</code> , <code>fs/transcript.rs:213-219</code> ; <code>py:1414</code>

What is absorbed before each challenge is therefore: everything read from the stream before it (including the grinding nonces, with tag 4) and every earlier squeeze (which ratchets the state), and **nothing else**: the Merkle openings are never absorbed (`fs/transcript.rs:223-224`, the doc comment of `hint_merkle` at `:225-227`; the citation check corrected the dossier’s `:225-228`), the derived coefficient of a round polynomial is never absorbed (`:56-62` “Binding it would add nothing”), the table sumcheck’s target is never sent (the table-sumcheck dossier, `gt-table-pub.md`), the program enters through its hash and the public input through its four low words at the seed. A challenge is a function of (statement, sizes, all scalars so far, the count of squeezes so far).

8.7.1.14 The encoding of round polynomials

`add_round_poly(coeffs, eq)` sends every coefficient but one, “constant first” (`fs/transcript.rs:56-71`): with `eq = false` it drops c_1 , with `eq = true` it drops c_0 . `next_round_poly(n, claim, eq)` (`:289-309`) reads the others in index order, derives the missing one ($c_1 = \text{claim} + \sum_{i \geq 2} c_i$ for a plain round, $h(0) + h(1) = \text{claim}$ in characteristic two; $c_0 = \text{claim} + r \cdot \sum_{i \geq 1} c_i$ for a round whose equality factor r the verifier holds) and binds the read ones. The Python `sumcheck_round_poly` (`py:406-413`) is the same. WHIR’s rounds are plain and quadratic: two scalars (c_0, c_2) per round (`whir.rs:509-522`; `py:1012`). The blueprint’s row *Sumcheck messages* (`bp:315`) and acceptance test 8 (`bp:1287-1289`: “decode reconstructs c_1 ”) state this for the table sumcheck; `RoundPoly.decode(d)(claim)(eq? : Option E)` (`bp:1212`) has the right signature. Consequence for the oracle protocol: the round identity is not a check in the deployed verifier (check B15 of table 14.1; the table-sumcheck dossier’s “three and none on the wire”, finding 25); the oracle protocol’s explicit check is what `decode` makes true by construction.

8.7.1.15 The blueprint’s Layer 12 set against it

Table 8.23: The blueprint’s Layer 12 against the deployed chain and proof object.

Blueprint	Judgement
FsState with seed, observe, sample, grind (bp:1202-1206)	right shape; grind (bits) (nonce) : FsState → Bool × FsState must bind the nonce whether or not the check passes (fs/lib.rs:165-174), and compress must be the 64-byte hash (section 8.7.1.12)
Proof (bp:1209-1211)	which Merkle format (section 8.7.1.13)
RoundPoly.decode (bp:1212)	right
verify ... phase by phase in the stream order of the conventions (bp:1230)	the conventions’ rows give neither the order inside WHIR (table 8.19) nor the Merkle side channel
verify_iff_compiled (bp:1218-1221)	section 8.7.3.1
Tests: six mutations, “a flipped stream scalar in each phase, a wrong Merkle sibling” (bp:1238-1239)	add: a nonce that fails the grind; a nonzero third limb in a root; a nonzero upper limb in a size; an extra trailing scalar; an extra sibling; a level-0 row of the wrong width (the bus dossier asks for three of these, finding 13)

8.7.2 The opening and the compilation: disagreements between the sources

Short forms in this section. Specification files are under `doc/leanvm/body/`: `08`: is `08-end-to-end-protocol.tex`, `B`: is `b-polynomial-commitment-scheme.tex`. Rust files: `whir.rs`, `whir_config.rs` are under `crates/pcs/src/`; `fs/` is `crates/flat_shamir/src/`; `mod.rs` is `crates/lean_vm/src/cpu/mod.rs`. `py`: is `python-verifier/verifier.py`. These are at leanVM’s pin `a386121f`. `bp`: is the blueprint at leanerVM `b435631`. The number after the colon is the line.

The three sources agree on the pool, the batching challenge, the power of each claim and the batched target and weight (table 8.17); there is no sumcheck between λ and WHIR in any of them (section 8.7.1.3). The two verifiers agree with each other on WHIR message for message, the Python being a transcription of the Rust (section 8.7.1.6), and on the chain row by row (table 8.21); their one divergence is in the proof format they read (item 4). The specification is silent on the Fiat–Shamir chain, the rate and several checks; and the deployed WHIR schedule differs from the specification’s Protocol B.6 in four ways (items 6 to 9), none of which the blueprint records (finding 61).

Table 8.24: Disagreements between the specification, the Rust and the Python in the opening and the compilation, and the deployed WHIR schedule against Protocol B.6.

#	Subject	Disagreement	The blueprint
<i>Between the three sources</i>			
1	The Fiat–Shamir chain and the proof object	§8.4 of the specification is T0D0 (08:44-46); §8.5’s “Setup” says only that the initial state is “seeded by the public input, the bytecode and Flock’s BLAKE2s R1CS matrices” (08:55). The Rust is the only source; the Python transcribes it and agrees on every row of table 8.21.	states the tags, the seed and the squeeze correctly (bp:325); invites the other reading of the step (finding 64)
2	The rate	The specification names no rate (08:55); both verifiers read the announced rate and reject it outside [1,4] (mod.rs:167, py:1377; S-rate of table 14.5).	Sizes.Admissible lacks the rate window (finding 8)
3	The closing list of the specification (08:100)	“the caps” omits the rate window and the stacking window; “every sumcheck’s rounds” is not a check in either verifier (W1); “the PCS opening” stands for M1 to M7 and W2 to W5, and grinding (G1, G2) is absent from Annex B altogether. The canonical encodings (B2, B3, B10 of table 14.1; R2), full consumption (R4) and grinding are in both verifiers and not in the specification.	verify inherits these omissions unless Layer 12 lists the checks (section 14.3.4)

#	Subject	Disagreement	The blueprint
4	The proof format, and the Python’s level-0 leaves	The Rust prover emits the pruned wire form; the Rust verifier emits the raw form after accepting; the Python reads the raw form (<code>fs/transcript.rs:9-19</code> , 138-143, 198-203; <code>py:322-330</code>). In the raw form the level-0 leaf carries the full 64-word image and the Python does not check that the absent lanes are zero (<code>py:1049</code>): run standalone on an adversarial raw proof it accepts a commitment whose absent lanes are nonzero (the citation check agrees by reading; the Python was not run on such a proof), while the Rust verifier supplies the zero prefix itself (<code>fs/merkle.rs:53-58</code>): the Python’s accepted set is strictly larger than the Rust’s on this point. Soundness-neutral (M2 of table 14.5); a divergence in accepted sets nonetheless.	unspecified: <code>merkleVerify</code> has the raw shape, the fixture is the prover’s pruned form (finding 60)
5	A stale comment in the Rust	<code>whir.rs:393-395</code> says “one Merkle leaf of <code>num_interleaved * 16</code> bytes”; the code hashes <code>24 · num_interleaved</code> bytes (<code>whir.rs:463</code>). The citation check found more at the pin: <code>whir.rs:1364-1365</code> says level 0 grinds “0 bits in the production profile” (and <code>whir.rs:1771-1772</code> in the test-only dense verifier), while every level grinds 17 bits (<code>whir_config.rs:922</code> , 955, 988, asserted at <code>whir_config.rs:1027</code> and <code>whir.rs:2485</code>); <code>whir.rs:1239-1241</code> and 1636-1637 mention fold grinds, and the pinned code has none.	— (for <code>docs/leanvm-target.md</code> , finding 70)
<i>The deployed WHIR schedule against the specification’s Protocol B.6 (B:100-123)</i>			
6	Per-claim intro messages before the batching challenge	The specification sends, after λ_i , one round polynomial h_1 of the batched claim (B:111). The Rust sends a round polynomial <i>per claim</i> (the residual’s, which is the last fold message; the OOD claim’s, <code>whir.rs:1206</code> , before λ_i is drawn; the query batch’s, <code>whir.rs:1406</code> , after it), and both sides form the batched h_1 by linearity (<code>RoundQuad::fold</code> , <code>whir.rs:548-554</code>). Equivalent claims, more prover messages. Soundness-neutral; transcript-relevant. (The citation check corrected the dossier, which had all the intro messages “before or after” λ_i and “two verifier challenges in a row (λ_i then r_1) with no prover message between them”: the query batch’s intro follows λ_i , prover <code>whir.rs:1373</code> then 1405-1406, verifier 2123 then 2142, <code>py:1044</code> then 1057, and its claim depends on λ_i , whose powers weight the queries, <code>whir.rs:1374</code> ; the challenges that follow one another with no prover message between are the query positions and λ_i , <code>whir.rs:1370</code> then 1373, 2122 then 2123, <code>py:1041</code> then 1044.)	not recorded; <code>whirOpen</code> is “stated exactly as Protocol B.1” (<code>bp:1185</code>)
7	Grinding	17 bits at every level, including the last, ground after the level’s root (or f_{final}) and before its positions (<code>whir_config.rs:57-60</code> , 335-336); <code>bits = 0</code> would still read a canonical zero nonce (<code>fs/lib.rs:165-174</code>). Not in Annex B (the status file records it, its finding S12).	not recorded in Layer 11; Layer 12’s <code>grind</code> (table 8.23)
8	The lane relay layout	Level 0 folds the stack’s <i>top</i> six coordinates (the lane index), the later levels the low coordinates of the current fold; the verifier rotates the terminal point left by 6 before evaluating $\widetilde{W}_\lambda$ (<code>whir.rs:2297-2306</code> , <code>py:1079</code>). Annex B states it only in “Optimizations” (B:380).	nowhere mentioned; a <code>verify</code> without it evaluates $\widetilde{W}_\lambda$ at the wrong point and rejects every Rust proof
9	The last round message	Omitted (<code>whir.rs:1471-1475</code> , :2256-2261; <code>py:1073-1074</code>): the verifier’s last running claim is checked directly against $\widetilde{f}_{\text{final}}$, so the message that would carry it is redundant. The specification says nothing about the last message (B:120).	not recorded

The OOD count agrees: one per level ≥ 1 and none at level 0 (whir_config.rs:677-684, :829-841; whir.rs:1261-1265 “L0 takes no OOD sample”), as Annex B (B:105 “No OOD sample is taken”) and the blueprint (bp:1186-1187) say.

8.7.3 The compilation: the non-interactive theorems, the numbers and the hidden assumption

Short forms in this section. Specification files are under doc/leanvm/body/: 03: is 03-proving-primitives.tex, 08: is 08-end-to-end-protocol.tex, A: is a-ring-switching.tex, B: is b-polynomial-commitment-scheme.tex. Rust files: whir.rs, whir_config.rs are under crates/pcs/src/; fs/ is crates/fiat_shamir/src/; mod.rs is crates/lean_vm/src/cpu/mod.rs, and pcs.rs, leaf.rs are under crates/lean_vm/src/; zerocheck.rs is crates/flock/src/zerocheck.rs. py: is python-verifier/verifier.py. These are at leanVM’s pin a386121f. bp: is the blueprint at leanerVM b435631; architecture.md is docs/architecture.md there; Compose.lean, Seams.lean and Instance.lean are under LeanerVM/Protocol/Spine/ at b435631. ArkLib at its old pin dca90385 unless a revision is named: Basic.lean, FiatShamir/Basic.lean, BCS/Basic.lean, ProtocolSpec/Basic.lean, Security/Implications.lean and Security/RoundByRound.lean are under ArkLib/OracleReduction/; Commitments/Functional/Basic.lean is under ArkLib/; CapacityBounds.lean is ArkLib/Data/CodingTheory/ProximityGap/CapacityBounds.lean. The number after the colon is the line. Other dossiers are cited by file and section: lib-arklib.md (ArkLib), literature.md (the literature), gt-bus.md, gt-table-pub.md, gt-flock-ring.md (the other phases).

8.7.3.1 Are the blueprint’s non-interactive theorems well formed?

The four sketches (bp:1218-1227, bp:1247-1250 at b435631):

```
theorem verify_iff_compiled (prog input proof) :
  verify prog input proof = true ↔
    ∃ s, s.Admissible prog ∧ (Verifier.fiatShamir (leanVmIopp prog s input flock).verifier ...)
      accepts (decode s proof) under the BLAKE2s challenge oracle
structure FiatShamirSecurity where ... -- rbr knowledge soundness of the IOPP ⇒ knowledge soundness of its FS
  ↳ compilation in the ROM, error  $Q \cdot \max_i \epsilon_i$ 
structure BcsSecurity where ... -- Merkle-compiled oracles: extraction from collision resistance / ROM
theorem verify_knowledgeSound (fs : FiatShamirSecurity) (bcs : BcsSecurity) (mca : McaJohnson) (flock) :
  ... knowledge soundness of `verify` with error  $\text{niError } s \ Q$ 
theorem baseVerifier_extractExecution (fs bcs mca flock) (h : verify prog input proof = true) :
  except with probability  $\text{niError}$ ,  $\exists t$ , ValidExecution prog input t
theorem baseProver_complete (hfill : HasFillBlocks prog) (h : ValidExecution prog input t) :
  verify prog input (prove prog input (witness of t)) = true
```

docs/roadmap/protocol-blueprint.md:1218-1227, 1247-1250 at b435631

They are read here as statements to be written in Lean. Two library objects are introduced first.

Random oracle; the random-oracle model

A **random oracle** is a function drawn uniformly from all functions of a fixed domain and range, to which every party has query access only. A theorem “in the random-oracle model” (ROM) is a probability statement over that draw, against adversaries that make at most Q queries.

ArkLib’s Fiat–Shamir transform (Verifier.fiatShamir)

Verifier.fiatShamir (FiatShamir/Basic.lean:129-136 at dca90385, verbatim in the probe of ?? B.2.1.2) turns an interactive Verifier into a NonInteractiveVerifier whose one message is the tuple of all prover messages, and which obtains each challenge by querying a *challenge oracle* fsChallengeOracle StmtIn pSpec (ProtocolSpec/Basic.lean:849-854) at the pair (statement, messages so far) (challengeOracleInterfaceSR, :830-834: “Query := StmtIn × pSpec.MessagesUpTo i.1.castSucc”). That oracle is an OracleSpec; any deterministic function of the same signature is a QueryImpl for it, so a concrete hash chain can be plugged in by simulateQ (the probe’s runWithChain).

baseVerifier_extractExecution, and what a well-formed statement looks like

As sketched, it has no probability space. `verify prog input proof = true` is a decidable proposition about three fixed values: `verify` is “a total, computable function of `(prog, input, proof)` to `Bool`” (bp:326) and `BLAKE2s` is a fixed function. “Except with probability `niError`” then quantifies over nothing: for fixed `(prog, input, proof)` the conclusion $\exists t, \text{ValidExecution } \text{prog } \text{input } t$ is either true or false, and no randomness makes the hypothesis probabilistic. The same defect is in the source it transcribes, the target theorem T4 in `architecture.md:264-269` (“`BaseVerifier.Accepts version proof program publicInput → except with probability baseError, ∃ assignment, ...`”), whose next sentence asks for what the sketch omits: “The theorem must expose the component soundness/knowledge-soundness bounds, Fiat–Shamir or random-oracle model, commitment and hash assumptions” (`architecture.md:271-273`).

As sketched, it is soundness, not knowledge soundness. The conclusion is an existential over traces. Knowledge soundness names an extractor that outputs the witness; the blueprint’s own doctrine (bp:96-103, acceptance test 24) insists on a named, computable extractor for the oracle protocol, and then discards it at the last step. Since `ValidExecution` is not what an extractor can output (bp:96-99 says so), the right shape is: the extractor outputs a stack `q`; `witnessOf q` satisfies `SatisfiedBy` except with the error; `constraintSoundness` (the target theorem T1) then gives the trace. The knowledge statement is about the first step; the existential is its corollary.

What the randomness is over, and what the adversary is. In the ROM the verifier is verify^H , `verify` with the compression function $H : \{0,1\}^{512} \rightarrow \{0,1\}^{256}$ (the primitive of `fs/lib.rs:18-24`, “`f(a, b) = BLAKE2s(a||b)`”, used for the chain, the grinding and every Merkle node and leaf, `fs/merkle.rs:40-51`) left as an oracle. (The citation check corrected two points. The quoted text is the doc comment at `fs/lib.rs:9`; `compress` is at `:18-24`. And the one compression does not hash the leaves: a node is one 64-byte `BLAKE2s-256`, but a leaf is `BLAKE2s-256` of the whole leaf image, 512 bytes at level 0 and 384 bytes deeper, eight or six compressions chained through `BLAKE2s`’s own chaining value and counter (`fs/merkle.rs:38-42`, `61-67`; `primitives/src/hash.rs:209-219`). A random oracle for the single 64-byte compression does not model the leaf hash; a ROM statement needs either the iterated construction or a second oracle for leaves.) The adversary A^H is a Q -query oracle algorithm that outputs `(prog, input, proof)` (adaptively: leanVM’s statement includes the program, whose hash seeds the chain, `mod.rs:82-93`, and the sizes are the prover’s, `mod.rs:118-124`). The probability is over H (and A ’s coins). A well-formed statement:

```
theorem verify_knowledgeSound (Q : ℕ) (A : Adversary H Q) :
  Pr_H [ let (prog, input, proof, log) ← A^H ;          -- log: A's queries to H and the answers
          verify^H prog input proof = true ∧
          ¬ SatisfiedBy prog input (witnessOf (extract prog input proof log)) ]
    ≤ niError Q
theorem baseVerifier_extractExecution (Q) (A) :
  Pr_H [ ... verify^H prog input proof = true ∧ ∃ t, ValidExecution prog input t ] ≤ niError Q
```

the dossier’s proposed statements (`dossiers/gt-opening-compile.md`, section E.1)

with `extract` a **straight-line** extractor: it reads the Merkle leaves of the level-0 commitment off the query log (every leaf the prover can open was hashed, so it is in the log except with the collision or preimage term), list-decodes the word to its $\leq L_0$ nearby codewords (section 8.7.3.3), and returns the member that satisfies `M3Holds`, or a default. The second theorem follows from the first by `satisfiedBy_witnessOf` and `constraintSoundness` pointwise. Two further features of the deployed object a faithful statement carries:

- **The sizes are the adversary’s.** `verify` dispatches on the eight announced sizes (`mod.rs:144-149`), so it is the union over admissible s of the members of a protocol family; a bound proved per member (per s) transfers to the union only by an argument about how A ’s Q queries split among members (they do, since a query to the chain seeded and then fed the sizes belongs to one s); `niError` must be uniform in s or take the maximum. Acceptance test 21 (bp:1326-1328) records the dispatch and not this obligation.
- **The program enters the chain by its hash.** `iv = BLAKE2s("leanvm" || len || R1CS_DIGEST || bytecode_hash)` (`mod.rs:82-93`): two programs with the same `bytecode_hash` share every challenge, so the statement binds the program only up to a `BLAKE2s` collision, a term of `niError`.

The random-oracle heuristic. `verify` is a function; `verifyH` is a family. The only theorem provable is about `verifyH` with H random; the sentence “BLAKE2s’s compression function is that H ” is the **random-oracle heuristic**, a non-formal step to be named as such in Layer 13 and in `architecture.md` (which already has the words “Fiat-Shamir or random-oracle model” but files them under “must expose”, not under “assumed”). It cannot be a Lean hypothesis (it is false for any fixed function, since an adversary may hard-wire it) and it cannot be an axiom (`AGENTS.md`); the theorem about `verify` itself, with BLAKE2s fixed, is the instantiation $H := \text{blake2sCompress}$ of the definitions only (`verify = verifyblake2sCompress`, by `rfl`), never of the security theorem. The same holds of the Merkle side (collision resistance of BLAKE2s-256 is the second thing the heuristic buys).

The three assumed interfaces: what each must state to be true and strong enough

The ground on ArkLib, at both pins. The ArkLib dossier (`lib-arklib.md`, section G.4) established it; it was re-verified here at `dca90385` and `7653a901`. `fiatShamir_completeness` is admitted and stated for the constant challenge oracle (`FiatShamir/Basic.lean:163-173`); soundness after Fiat-Shamir is a `TODO (:175)`; round-by-round $\Rightarrow$ state-restoration is commented out (`Security/Implications.lean:230-254`, and the commented statement has the wrong error, $\sum i$, `rbrError i`, where a state-restoration bound must depend on the query budget); state-restoration $\Rightarrow$ plain knowledge soundness is admitted (`:305-313`); `BCSTransform` is commented out (`BCS/Basic.lean:59-63, 79-82`); `Commitment.extractability` has `False` for its body (`Commitments/Functional/Basic.lean:250-255`). (The citation check adds the lines at the new pin `7653a901`: `Security/Implications.lean:200-224` and `:275-283`, `Commitments/Functional/Basic.lean:249-254`; the body there is $\exists \text{_extractor}, \forall \text{AuxState } \text{_adversary } \text{_prover}, \text{False}$, with the intended probability statement commented out below it; and the commented round-by-round theorems are about the protocol with added salts, `addSalt`, which the dossier does not say.) The pull requests the tracker names: `#469` (open since 2026-04-19, “statements of Section 5” of [CO25], the duplex-sponge Fiat-Shamir, no security proof) and `#848` (open, stacked on `#469`, “prove Theorems 6.1 and 6.2”); **#627 is an issue, not a pull request** (a design for the BCS transform, 2026-07-10), whose staged plan says: “Full ROM/state-restoration soundness inheritance is deliberately **out of scope** and will be fenced as a named open statement”.

McaJohnson. What it must state: [BCHKS25] Theorem 4.6 as Annex B quotes it (`B:176-185`, `thm:mca-johnson`): for the Reed-Solomon code of block length n and reduced rate $\rho = (k-1)/n$, every radius $\delta < 1 - \sqrt{\rho}$, every two-row word C , the probability over $s \leftarrow \mathbb{E}$ that C_s agrees with the code on a set of $(1-\delta)n$ columns on which C does not agree with the interleaved code is at most $a/|\mathbb{E}|$, a the explicit expression in $m = \max(\lceil \sqrt{\rho}/(1 - \sqrt{\rho} - \delta) \rceil, 3)$. ArkLib states exactly this, admitted, at both pins:

```
-- ArkLib/Data/CodingTheory/ProximityGap/CapacityBounds.lean:203-219 (dca90385; 7653a901 has the
-- same statement)
theorem rs_mcaError_le_in_johnson_range
  (domain :  $\iota \hookrightarrow F$ ) (k :  $\mathbb{N}$ ) ( $\delta : \mathbb{R} \geq 0$ )
  (_hk :  $1 < k$ ) (_h $\delta$ _pos :  $0 < \delta$ )
  (_h $\delta$  :
    ( $\delta : \mathbb{R}$ ) <
    1 - (((k - 1 :  $\mathbb{N}$ ) :  $\mathbb{R}$ ) / Fintype.card  $\iota$ ) ^ ((1 :  $\mathbb{R}$ ) / 2))) :
mcaError (AffineLineGenerator F) (ReedSolomon.code domain k) ( $\delta : \mathbb{R}$ )  $\leq$ 
  ENNReal.ofReal
    (let n :  $\mathbb{R} :=$  Fintype.card  $\iota$ 
     let  $\rho : \mathbb{R} :=$  (k - 1 :  $\mathbb{N}$ ) / n
     let m :  $\mathbb{R} :=$  max [( $\rho$  ^ ((1 :  $\mathbb{R}$ ) / 2)) /
      (1 -  $\rho$  ^ ((1 :  $\mathbb{R}$ ) / 2) -  $\delta$ )] 3
     ((2 * (m + 1/2) ^ 5 + 3 * (m + 1/2) *  $\delta$  *  $\rho$ )
      / (3 *  $\rho$  ^ ((3 :  $\mathbb{R}$ ) / 2)) * n
      + (m + 1/2) /  $\rho$  ^ ((1 :  $\mathbb{R}$ ) / 2))
     / (Fintype.card F :  $\mathbb{R}$ )) := by
sorry -- ABF26-T4.12; external admit [BCHKS25 Thm 4.6].
```

ArkLib at `dca90385` (and `7653a901`), `ArkLib/Data/CodingTheory/ProximityGap/CapacityBounds.lean:203-219`

Instantiated at $F := E$ and `domain` the additive subspace of $\mathbb{K} \subset \mathbb{E}$ (`B:90, 302`), for the affine line $u_0 + s \cdot u_1$, which Annex B’s footnote (`B:158`) converts to the fold $(1-s)u_0 + s \cdot u_1$ in characteristic two. It is a statement of mathematics, not about protocols, so quantifying it universally (over ι , F , `domain`, k , δ)

is right and true if the theorem is; its consumer is Lemma B.10 (`lem:fold-list`; the citation check corrected the dossier’s “Lemma B.8”), through the $2^{\ell-1}$ union of rows. Strong enough: yes, it is the only coding-theoretic input Theorem B.7 uses (`B:140-152`). (The citation check qualifies: the Johnson bound, Theorem B.3, proved at `B:391-414`, and Lemma B.11 are also inputs; Theorem B.9 is the only *external* one.) Its witness obligation is the preprint’s theorem (the literature dossier, `literature.md`, section C.3, notes the two readings of the constant m ; the printed one, the larger error, is what both Annex B and ArkLib state). Well formed.

FiatShamirSecurity. As the comment states it (“rbr knowledge soundness of the IOPP $\Rightarrow$ knowledge soundness of its FS compilation in the ROM, error $Q \cdot \max_i \varepsilon_i$ ”) it is a universally quantified claim over IOPPs and is **false**:

- (a) an IOPP’s messages are oracles; its Fiat–Shamir compilation hashes commitments, not oracles, and the compiled error carries a hash term (`literature.md`, section A.3, item 2: $3.5 \cdot t^2/2^\lambda$ in [CY24] Theorem 25.2.1, $3(Q^2 + 1)/2^\kappa$ in [BGKTTZ23] Theorem 3.15);
- (b) leanVM’s compilation grinds 17 bits at every query round, which no theorem in the literature covers (section A.3, item 3) and which changes the error from $Q \cdot \max_i \varepsilon_i$ to $(t + k) \cdot \max_i 2^{-b_i} \varepsilon_i$ at best;
- (c) the hash-chain construction (`fs/lib.rs:85-99`, a state ratcheted by H) is neither ArkLib’s per-round oracle keyed by the whole prefix nor [CO25]’s permutation sponge, so a theorem about either applies only through a further “chain is as good as the ideal oracle” lemma (collision term $Q^2/2^{256}$).

What ArkLib is about to offer (#848, `SingleSalt.lean:973-993` at its head `1c5c5bd7`, `single_salt_fiat_shamir_knowledge_soundness`) takes **state-restoration** knowledge soundness of a plain Verifier with coins, for the challenge oracle `fsChallengeOracle (StmtIn  $\times$  Salt) pSpec` implemented by `srChallengeQueryImpl'`, and concludes adaptive NARG knowledge soundness with the same error; it has no BCS side, no grinding, and needs round-by-round $\Rightarrow$ state-restoration, which is not stated anywhere. (The citation check corrected the dossier’s “implemented by lazy uniform sampling”: `srChallengeQueryImpl'` answers from a function that `fsInit` produced ahead of time and never updates it, `ProtocolSpec/Basic.lean:1094-1098` at `1c5c5bd7`, ArkLib’s docstring saying “the whole function is sampled ahead of time”; the distribution is whatever `fsInit`, a hypothesis, supplies.) To be true and strong enough the interface must be stated for **this** construction: (1) a public-coin interactive argument V^H (the Merkle-compiled protocol, section 8.7.3.1) that is state-restoration knowledge sound against Q -query provers in the ROM with error $\varepsilon_{\text{sr}}(Q)$, (2) the hash chain of `fs/lib.rs` with H random, (3) the named ground rounds with their bits b_i ; and conclude adaptive knowledge soundness of the chain-compiled NARG with error $\varepsilon_{\text{sr}}(Q) + c \cdot Q^2/2^{256}$; and the grinding must be inside ε_{sr} as “a query to H at a ground round costs 2^{b_i} queries in expectation”. None of this exists in Lean; the honest interface names the literature and the gap (`literature.md`, section A.3: “a new assumption to name”).

BcsSecurity. As the comment states it (“Merkle-compiled oracles: extraction from collision resistance / ROM”) it names a property with no error and no object. At the pin nothing can inhabit it, and #627’s plan excludes the ROM inheritance it would need. To be true and strong enough: for the IOP C of section 8.7.3.1 (oracle messages: the level-0 $\mathbb{K}$ -word and the deeper $\mathbb{E}$ -words, all with position queries), the Merkle compilation of `crates/pcs/src/merkle.rs` and `fs/merkle.rs` (leaf = BLAKE2s of the row image, node = BLAKE2s of the two children, fixed height and fixed leaf width; the leaf hash being multi-block, the ROM must model it by a second oracle or through the iterated construction, as the citation check notes, section 8.7.3.1) in the ROM preserves state-restoration knowledge soundness, with the straight-line extractor that reads every committed leaf off the query log, up to $3(Q^2 + 1)/2^{256}$ ([BGKTTZ23] Theorem 3.15’s hash term; [CY24] Theorem 25.2.1). Its hypothesis must include that the verifier fixes the leaf width and the tree height from public data (checks M2 to M7 of table 14.5): that is what makes leaf/node confusion impossible without domain separation.

Two interfaces the blueprint does not list and that the theorem needs.

- **Round-by-round $\Rightarrow$ state-restoration** knowledge soundness, with error $Q_i \cdot \varepsilon_i$ summed over rounds or $Q \cdot \max_i \varepsilon_i$ ([CY24] Theorem 31.2.1/31.3.1: $(t + k) \cdot \max_i \varepsilon_i$; `literature.md`, section A.3, item 1); commented out in ArkLib (`Implications.lean:230-254`) with the wrong error.
- **The list-binding compilation** of section 8.7.3.1, which is not an upstream theorem at all.

From WHIR’s list soundness to extraction of one q

What Theorem B.7 gives. Round-by-round **soundness** (Definition B.5 at the pin, the citation check correcting the dossier’s “Definition B.2”; B:74-84, an invariant on partial transcripts with an escape probability per verifier message) for the relation $R_{\text{open}} = \{(C^{(0)}, \{(W_t, c_t)\}) : \exists U \in L_{\gamma_0}(C^{(0)}), \langle W_t, g_U \rangle = c_t \forall t\}$ (B:133-136), where $L_{\gamma_0}(C^{(0)})$ is the set of interleaved codewords within radius γ_0 of the committed word, of size at most $L_0 = 1/(2\eta_0\sqrt{\rho_0})$ (B:137). Its start invariant is “every $U \in L_{\gamma_0}(C^{(0)})$ violates at least one of the J_0 claims” (B:218). No extractor is named: it is soundness for a language, in the IOP model (the oracles $C^{(i)}$ are read at queried columns only; the prover is unbounded).

What the master theorem gives. `piop_rbrKnowledgeSoundness` (Compose.lean:172-177 at b435631) is ArkLib’s worst-case round-by-round knowledge soundness for `piopExtractor`, whose first step reads the stack q off the commit message (`commitExtractor`, Compose.lean:58-59), with a knowledge state function whose `toFun_full` property says (Security/RoundByRound.lean:186-189 at dca90385): if the verifier can output a statement in `relOut` with witness `witOut`, the state is true at the full transcript. Everything is relative to **one** q .

The step “Layer 12 turns list binding into extraction of the one q ” (bp:1190, acceptance test 11, bp:1296-1299) is a theorem about the composed IOP, and it can be stated. Let `Front` be the spine’s prefix `commit » bus » table » pub » flock`, whose verifier never queries the oracle (true of the built phases; a requirement on the others under the inner-product interface of section 8.7.1.5), with worst-case round-by-round knowledge soundness from `M3Rel` to `Seam.flock` at errors ε_i (the composition of the first four fields of `Phases.Security`). Let `Front'` be `Front` with the commit message’s type replaced by an interleaved $\mathbb{K}$ -word w (the level-0 codeword, with position queries) and the statement carried unchanged; since the front’s verifier never reads the oracle, `Front'`’s verifier is `Front`’s. Let $C := \text{Front}' \gg \text{whirOpen}$ (the level-0 batching of `whirOpen` being the opening phase’s λ). Then:

List-binding compilation (the theorem the dossier proposes)

C is worst-case round-by-round knowledge sound for `M3Rel` with extractor $\mathcal{E}(w, \tau) :=$ the first $g \in \Lambda(w)$ with `M3Holds input g` (else a default), where $\Lambda(w) = \{g_U : U \in L_{\gamma_0}(w)\}$, and per-challenge errors $L_0 \cdot \varepsilon_i$ at the front’s challenges and Theorem B.7’s at WHIR’s. Its knowledge state function is: after w , “no $g \in \Lambda(w)$ satisfies `M3Holds`”; inside the front, “for every $g \in \Lambda(w)$, the front’s state at (g, τ) is false” (the front’s state function evaluated with g in place of the commit message); at the seam this reads “for every $g \in \Lambda(w)$ some pooled claim is false of g ”, which is $\notin R_{\text{open}}$ and Theorem B.7’s start invariant; then Theorem B.7’s invariant.

Proof shape. The front’s `toFun_next` and `toFun_full` transfer pointwise (each g is a transcript of `Front`); the escape probability of a front challenge is bounded by the union over $\Lambda(w)$ of the front’s worst-case bound, which is per transcript and therefore per g (`rbrKnowledgeSoundnessWorstCaseWith`, Security/RoundByRound.lean:553-568: a bound for every statement, every round, every partial transcript); the seam step needs `toFun_full` of the front’s state function in the contrapositive: state false at the front’s last transcript $\Rightarrow$ the verifier’s output claims are not all true of g . That is exactly why the spine’s *worst-case* form is the right one (the averaged `rbrKnowledgeSoundness` would not union-bound). What it needs and does not have: Theorem B.7 in Lean (the hole for it, K1); the lemma that every member of $\Lambda(w)$ is $\mathbb{K}$ -valued and of the right size (section 8.7.3.3); a generic “state functions indexed by a finite set” lemma (union bound; ArkLib has none); and the identification of the front’s oracle-free verifier with `Front'`’s (a cast of `OracleReduction`, which ArkLib #383’s `Cast.lean` is about). The extractor is computable as a definition (enumerate the codewords of the finite code; Lean has no cost model, bp:102-103), and efficient in the literature’s sense through Guruswami–Sudan, which Lean need not implement.

This theorem is where the factor L_0 enters `niError` (section 8.7.3.2), and it is the missing fourth ingredient of `verify_knowledgeSound`: neither `FiatShamirSecurity`, `BcsSecurity` nor `McaJohnson` contains it, and the sentence “`verify_knowledgeSound` extracts the member” (bp:1299) assigns it to a theorem whose sketched hypotheses cannot prove it.

verify_iff_compiled “Unconditional: it is about two definitions” (bp:1231-1232) is right in kind and wrong in the definitions named.

1. **Verifier.fiatShamir takes a Verifier, not an OracleVerifier**, and an oracle verifier becomes one only through `OracleVerifier.toVerifier` (`Basic.lean:490-496` at `dca90385`; the probe of ?? B.2.1.2 prints its type), whose statement is `StmtIn × (∀ i, 0StmtIn i)` and whose transcript holds the oracle messages **in the clear**: (`leanVmIopp ...`).`verifier.toVerifier` has the level-0 word ($2^{\mu-6} \cdot 2^6$ elements of $\mathbb{K}$) and every deeper codeword as messages, and `fsChallengeOracle` would hash them whole. No `decode : Proof → messages` exists: a proof holds roots and opened rows (`fs/transcript.rs:9-12`), from which no codeword is recoverable. The right-hand side must be the Fiat–Shamir compilation of the **Merkle-compiled** interactive verifier, whose messages are the roots, the scalars, the grinding nonces and the openings; `ArkLib` has no such compilation (`BCSTransform` commented out), so Layer 12 must define it (`bcsCompile`) before it can state the theorem.
2. **The challenge oracle can be the chain.** The probe of ?? B.2.1.2 shows the mechanism: a function `chain : (i) → StmtIn × MessagesUpTo i → Challenge i` is a `QueryImpl (fsChallengeOracle ...)` `Id`, and `simulateQ` runs the compiled verifier under it to an `Option StmtOut`. `leanVM`’s chain is such a function once the statement is (`input`) with (`prog, s`) as parameters of the family (the chain’s seed depends on `prog` through `iv, mod.rs:82-93`, and its first eight absorbs are the sizes, `mod.rs:118-124`) and the messages are absorbed in the wire encoding (one coefficient of each round polynomial dropped, `fs/transcript.rs:63-71, :289-309`; a root as two scalars, `fs/merkle.rs:14-20`; the Merkle openings **not absorbed**, `fs/transcript.rs:223-224` “Not absorbed: its binding is the Merkle structure itself”, the citation check correcting the dossier’s :225-228). A challenge that is several squeezes ($\alpha \in \mathbb{E}^4$, the query positions by chunking, `whir.rs:1175-1193`) is one `Challenge i` of a product type, or several rounds.
3. **The grinding checks are not in the right-hand side.** The nonce at each query round is a prover message (absorbed with tag 4, `fs/lib.rs:118-120`), and the verifier’s check `pow_bits_ok(pow_base(cv), nonce, 17)` (`fs/lib.rs:47-51, 165-174`) reads the **chain state** `cv`, which an `ArkLib Verifier` does not see: its `verify` takes the statement and the full transcript (`Basic.lean:248-250`). Either the interactive verifier recomputes the chain from the transcript (definable, since the chain is a function of statement and messages; it then depends on H and is no longer an IOP verifier), or `verify_iff_compiled`’s right-hand side is a conjunction “the compiled verifier accepts $\wedge$ every grinding check passes”, the second conjunct being about the chain. The sketch has neither.
4. **∃ s, s.Admissible prog** is right in shape (the family is indexed by the sizes; the proof carries them as its first eight scalars) provided **Admissible** is the verifier’s whole predicate, the stacking window and the rate window included (finding 8).
5. **The equation is a definition, not a theorem, if verify is defined as the right-hand side.** The blueprint wants `verify` “written from §8.5 before `cpu/mod.rs:711-779` is opened” (bp:1216) and the theorem to “pin the shape” (bp:1277, :1302, :1316, :1328). That is only a check if the two sides are written independently; then the theorem is provable by unfolding (*rfl*-like), and its value is the review of the two definitions.

Statable form (with the inner-product interface and the corrections above):

```
theorem verify_iff_compiled (prog input proof) :
  verify prog input proof = true ↔
    ∃ s, s.Admissible prog ∧ ∃ msgs, decode prog s proof = some msgs ∧
      runWithChain (bcsCompile (leanVmIopp prog s)).verifier (blake2sChain prog s) input msgs
        = some () ∧ grindingChecks prog s msgs = true
```

the dossier’s proposed statement (`dossiers/gt-opening-compile.md`, section E.4)

Completeness “Perfect completeness survives the transform without any assumption” (bp:1256-1257) is false for the deployed transform, for two reasons of different weight.

Grinding. The honest prover’s grind is a search: `fs/lib.rs:126-155 grind_pow` loops over nonces 0,1,2,... “until” `pow_bits_ok (loop { ... break n } at :132-138`, and the parallel form at :139-151), with no bound. For a fixed state `cv` there is no theorem that a `u64` nonce (or any nonce) with

`compress(pow_base(cv), nonce || 4)[0] ≡ 0 mod 217` exists; in the ROM the search fails with probability $(1 - 2^{-17})^{2^{64}}$, not zero; with BLAKE2s fixed the statement “for every reachable `cv` a nonce exists” is a conjecture about BLAKE2s. So `prove prog input w` in Lean is either partial (`Option Proof`, returning `none` when a fuelled search fails) or takes the nonces as an argument, and `baseProver_complete` is `prove ... = some proof → verify ... proof = true` (or “for every choice of nonces that passes”). “Without any assumption” must go; the assumption is the success of the search, and it is harmless only because it is checked by running the prover.

Phases complete up to an error. `Phases.Complete` demands `perfectCompleteness` of every phase (`Compose.lean:94-104`; `Component.Complete.complete` is `perfectCompleteness`, `lib-arklib.md`, section G.2). If any phase were complete only up to an error, `Phases.Complete` would be uninhabitable and `baseProver_complete`, an equation, unprovable. The blueprint’s acceptance test 20 (`bp:1323-1325`) says Flock has an exceptional challenge; the dossier on Flock settles it (`gt-flock-ring.md`, section 4; finding 38): the *protocol* is perfectly complete (no verifier takes an inverse; the derived coefficient $c_0 = \text{claim} + r \cdot (c_1 + \dots)$ is right at every r , `fs/transcript.rs:297-302`), and the exceptional point belongs to the pinned Rust prover’s shortcut (`zerocheck.rs:117`). So the Lean honest prover can be perfectly complete phase by phase, and the composition (`Reduction.seqCompose_perfectCompleteness_of_pure`, `bp:242`) applies; the ArkLib dossier’s proposal of an error field in `Component.Complete` (`lib-arklib.md`, section G.2) remains the right insurance, and with it “survives the transform” becomes “with the sum of the phases’ errors, plus the grind”.

What does survive. With the chain deterministic, an honest proof that the interactive verifier accepts on the chain’s challenges is accepted by `verify`: that is a statement about two definitions (the interactive run with challenges supplied by the chain equals the compiled run), true by unfolding, and the grinding is the only randomness left in the honest prover.

8.7.3.2 The numbers

The blueprint’s `piopError_le` and acceptance test 23 `piopError_le (hs : s.Admissible prog) : Σ i, piopError s i ≤ 240/|E| + flockError (bp:1130)`; test 23: “at the caps, $\Sigma \text{piopError} < 2^{-150}$ plus Flock’s $< 2^{-183}$ ” (`bp:1331-1332`). Evaluated here with the blueprint’s own per-phase charges, at the largest admissible instance. The sizes are bounded by the stacking window, $\mu_{\text{stack}} \leq 28$ (`mod.rs:174-176`, `pcs.rs:51`): every column has at most 2^{28} entries, so $\kappa_{\text{mem}} \leq 26$, $\tau_j \leq 24$, and the bus depth is $\mu_{\text{bus}} \leq 28$ (the Python layout code run over a covering grid of sizes finds 28 as the maximum, at $\kappa_{\text{mem}} = 16$, $\tau = (0, 0, 23, 0, 23, 3)$, $\kappa_{\text{pc}} = 26$, `scratch bus_depth.py`, ?? B.2.1.3; the bus dossier, `gt-bus.md`, section D.4, proves $\mu_{\text{bus}} \leq 28$ by counting). The table sumcheck has $B = 2$ constraints in all (`py:836-843`, the citation check correcting the dossier’s `py:836-851`: only JUMP has constraints, `n_constraints = 2`; `xi_powers = powers(xi, n_constraints + 3)`, `py:1391`), $\tau_{\text{max}} \leq 24$, and $J = 113$.

Table 8.25: The blueprint’s phase errors at the largest admissible instance.

Phase, charge (blueprint)	Where	Value at the maximum, as a multiple of $1/ E $
bus, $4 \cdot 2^{\mu_{\text{bus}}}$ on (α, β)	<code>bp:974</code>	2^{30}
bus, GKR: $(n_{\text{side}} - 1)$ per combiner, 5 per round, 2 per pair	<code>bp:941-942</code>	$\approx 14 \text{ layers} \times 14 \approx 2^{7.6}$
zerocheck escape, τ_{max} per constraint	<code>bp:1001-1003</code>	48
table, $(B + 2)$ on ξ , 3 per round	<code>bp:994</code>	$4 + 72$
public input, 1	<code>bp:1027</code>	1
Flock, $(4 \cdot k_{\text{batch}} + 163) + 2^{32}$	<code>bp:1085</code>	$\approx 2^{32}$
opening (Layer 10 as sketched), $(J - 1)$ on λ , 2 per round	<code>bp:1114</code>	$112 + 56$
sum		$\approx 1.25 \cdot 2^{32} \approx 2^{32.3}$, that is $\approx 2^{-159.7}$

The citation check recounts the GKR row: with radix-4 layers of 0, 2, ..., 26 variables the layer sumchecks have $\sum_{j < 14} 2j = 182$ rounds, so $5 \cdot 182 + 2 \cdot 14 + 2 \cdot 14 \approx 966 \approx 2^{9.9}$, not ≈ 196 (its own reading of `bp:940-942`, not checked against a GKR implementation). Immaterial: the sum stays $\approx 1.25 \cdot 2^{32}$.

So under the window `piopError_le` holds with a margin of 2^8 (the citation check: that compares the whole sum with 2^{40} ; the bound’s non-Flock part, $\approx 2^{30}$, sits 2^{10} under 2^{40}), and the interactive error of the oracle protocol is about 2^{-160} , dominated by ring switching’s $2^{32}/|\mathbb{E}|$ (which test 23 leaves out of “Flock’s $< 2^{-183}$ ”: finding 42). At the per-log caps without the window ($\kappa_{\text{mem}} = \tau_j = 32$), $\mu_{\text{bus}} = 38$ (scratch run, `stack_log 41 bus depth 38`) and the (α, β) term alone is $4 \cdot 2^{38} = 2^{40}$, so the bound is false there, as the bus dossier found (`gt-bus.md`, section D.4; finding 8). The statement is only true if `Admissible` contains the window.

What it measures. $\Sigma \text{piopError}$ is the *interactive* error of the *oracle* protocol (the sum over challenges, `bp:318`). It is not the quantity that governs the deployed security: after Fiat–Shamir the governing quantity is the largest per-challenge error (`B:84`, `whir_config.rs:25-27`), and the deployed challenges with the largest errors are WHIR’s, which the oracle protocol does not contain. 2^{-150} is therefore a fact about a sub-protocol, of no consequence for the 128-bit claim.

What the deployed parameters claim, and in which regime `SECURITY_BITS = 128` (`whir_config.rs:38`; `crates/lean_vm/src/lib.rs:69`) is a **round-by-round** target: “every verifier-challenge transition must have conditional failure probability at most $2^{-\text{SECURITY_BITS}}$ ” (`whir_config.rs:36-37`), “the Fiat–Shamir error per random-oracle query is the MAX of the entries, not their sum” (`:25-27`), “not a claim that the sum of all interactive failure probabilities is bounded” (`:404-407`). The regime is **Johnson list decoding with an explicit slack**, provable given [BCHKS25] Theorem 4.6 (`:314-337`: “That analysis is always the Johnson radius with explicit slack η ... WITH out-of-domain binding”; `analysis_version "bchks25_thm_4_6_exact_reduced_rate_row_union_optimized_eta"`, `:961`); no capacity conjecture, no unique-decoding fallback in production (`udr_queries` is `#[cfg(test)]`, `:175-180`).

The per-level parameters are the output of a floating-point search (`optimize_johnson_level`, `:639-707`) over the theorem parameter $m \geq 3$, minimizing the query count subject to four per-level bounds; the Python tabulates the resulting query counts (`py:910`). A scratch port of the search (`whir_params.py`, ?? B.2.1.3) **reproduces the Python table exactly** for all $\mu \in [15, 28]$ and $\rho \in \{1, \dots, 4\}$ (the ladder geometry, the rates and the query counts), so the blueprint’s `ladder_queries_eq : (ladder 15 1).map (·.queries) = [223, 55]` (`bp:1165`) is the right pin and the Rust and the Python agree. What the search yields, at three sizes:

Table 8.26: The deployed WHIR parameters at three sizes: the analysis parameters, the list bound and the security bits of each term.

μ, ρ	level	fold	rate	m	η	γ	$L \leq 1/(2\eta\sqrt{\rho})$	queries	OOD	fold (MCA)	bits	query bits (+17 grind)	OOD bits	list-unioned	algebraic bits
15, 1/2	0	6	1/2	395	0.00179	0.292	396	223	0		132.9	111.0	179.5		152.4
	1	4	1/16	306	0.00080	0.753	2527	55	1		133.2	111.0	167.2		149.7
28, 1/2	0	6	1/2	110	0.00643	0.286	110	228	0		129.2	111.0	180.4		154.2
	1	4	1/16	81	0.00309	0.747	648	56	1		129.8	111.0	169.9		151.7
	2	4	1/128	46	0.00192	0.910	2944	32	1		130.4	111.0	165.8		149.5
	3	4	1/1024	8	0.00390	0.965	4100	23	1		139.2	111.1	165.2		149.0
	4	4	2^{-13}	4	0.00274	0.986	16644	18	1		140.2	111.4	161.6		147.0
	5	4	2^{-16}	5	0.00068	0.996	218453	14	1		134.7	111.2	154.9		143.3
28, 1/16	0	6	1/16	27	0.00926	0.741	216	57	0		131.7	111.0	179.4		153.3

The residual is 5 variables at $\mu = 15$ and 2 at $\mu = 28$; “bits” is $-\log_2$ of the bound; the columns are `whir_config.rs’s paper_johnson_log_a` (:484-490, Theorem 4.6’s a times the row union $2^{\ell-1}$ of Lemma B.10, the citation check correcting the dossier’s “Lemma B.8”), `paper_per_query_bits × queries` (:494-498), `paper_ood_bits` (:600-608) and `johnson_algebraic_bits` (:556-568).

Reading:

1. the **query rounds** are the weakest: $(1 - \gamma)^t \approx 2^{-111}$ at every level, by design (:57-60: “the query count only needs to close the remaining `SECURITY_BITS - 17` bits”), the other 17 bits being the proof of work ground before the positions are sampled;
2. the **fold challenges** sit at 128.2 to 146 bits in the sample printed (the mutual-correlated-agreement term, with the $2^{\ell-1}$ row union). (The citation check: over all 56 supported sizes they run from 128.19, at $\mu = 27$, $\rho = 1/8$, level 0, to 148.68, at $\mu = 17$, $\rho = 1/4$, level 2. And it corrected the dossier’s “the search stops at the first m that clears 128, :664-665”: the loop at :662-666 breaks at the first m whose mutual-correlated-agreement term *fails* to clear 128, and among the m tried it keeps the one with the fewest queries, :697, which is the largest m still clearing 128; that is why the fold bits sit just above 128.)
3. the Johnson list bound is $L_0 \leq 396$ **at level 0** ($2^{8.6}$, at $\mu = 15$) down to 110 at $\mu = 28$, at rate $1/2$, and grows to $2^{17.7}$ at the deepest levels. (The citation check: over all rates L_0 reaches 648, at $\mu = 20$, $\rho = 1/16$, that is $2^{9.34}$; so L_0 runs from 110 to 648 over the supported sizes.)
4. the algebraic checks “unioned over the list” (:537-555: the batch polynomial’s degree and the ring switching’s $2^{31} + \dots$, times L_i) sit at 143 to 154 bits (the citation check: 141.0 to 154.2 over all sizes, the dossier’s own output having 141.00 and 141.42 at $\mu = 28$, level 5); this is the **only** place the Rust multiplies an oracle-protocol error by the list size. The bus’s $4 \cdot 2^{\mu_{\text{bus}}}$ is not unioned (`leaf.rs:108-118`, `soundness_degree_bound` has no L): with $L_0 \leq 2^{8.6}$ and $\mu_{\text{bus}} \leq 28$ it would still clear 128 ($192 - 30 - 8.6 = 153$), so the omission is not a vulnerability at these parameters, but it is an omission of the same kind the blueprint makes (finding 69). (The citation check: the Rust’s bound is $(N_{\text{TUPLE_BITS}} + 1) \cdot 2^\mu + 8(\mu + 1)^2 = 5 \cdot 2^\mu + \dots$, the “ $4 \cdot 2^{\mu_{\text{bus}}}$ ” being the blueprint’s term; with $L_0 \leq 648$ and $5 \cdot 2^{28}$, $192 - 30.3 - 9.3 \approx 152$, and the conclusion stands.)

niError against this `niError` “is the round-by-round maximum times the query bound plus the grinding-adjusted WHIR terms” (`bp:1234-1235`). Set against section 8.7.3.2 and the literature dossier (`literature.md`, section A.3):

- the “round-by-round maximum” of the oracle protocol is $\approx 2^{-160}$ (section 8.7.3.2); of the compiled protocol it is 2^{-111} at every query round before grinding and $2^{-128.2}$ at the worst fold. “Plus the grinding-adjusted WHIR terms” acknowledges that the maximum is elsewhere but gives no formula; the phrase “grinding-adjusted” has no theorem behind it (section A.3, item 3), and the adjustment is multiplicative on a per-round error ($2^{-17} \cdot 2^{-111}$), not additive;
- the factor L_0 on every oracle-protocol challenge (sections 8.7.3.1 and 8.7.3.3) is absent; numerically benign ($2^{-160+8.6}$; with the citation check’s $L_0 \leq 648$, $2^{-160+9.3}$), structurally load-bearing;
- the hash term ($Q^2/2^{256}$ -type) is absent; numerically it is what caps the 128 bits at a 256-bit digest (section A.3, item 2);
- the prover’s choice of sizes (section 8.7.3.1) is absent.

Does the blueprint’s bound mean something at the deployed parameters? $\Sigma \text{piopError} < 2^{-150}$ is a true statement about the ideal-oracle protocol at admissible sizes and says nothing about `verify`’s security level, which is set by the query rounds at 111 bits of information-theoretic soundness plus 17 bits of work, in the Johnson regime, conditional on Theorem 4.6. A `niError` that could carry leanVM’s claim would read, per random-oracle query,

$$\max \left(L_0 \cdot \max_{\text{front}} \varepsilon_i, \max_{\text{fold}} (2L_i/|\mathbb{E}| + 2^{\ell_i-j} a_i/|\mathbb{E}|), \max_{\text{query}} 2^{-b}(1 - \gamma_i)^{t_i}, \dots \right)$$

times $(Q + \text{rounds})$, plus the hash term, and it would be a statement about the ROM construction of section 8.7.3.1, not about the oracle protocol.

8.7.3.3 The oracle model’s hidden assumption

In the oracle protocol the prover’s first message is $q : \text{Column } \mu$, a table of 2^μ values of $\mathbb{K}$, and the verifier reads it only through the interface (evaluation queries today, weighted queries under the inner-product interface of section 8.7.1.5); the extractor reads it whole (`commitExtractor`). The compiled protocol replaces q by a Merkle root. For the compiled protocol to inherit the oracle protocol’s security, the commitment must provide:

1. **binding to a $\mathbb{K}$ -valued table of size 2^μ** , the object `M3Holds` is about (`M3Rel I` is on `Column I.μ`, `Instance.lean:230` at `b435631`; the bus’s fingerprints, the counts, the packing of q_{flock} all use that the entries are in $\mathbb{K}$);
2. **weighted openings over $\mathbb{E}$** : the pooled claims have weights and values in $\mathbb{E}$ (`03:113`);
3. an **extractor** that recovers the table from what the adversary did, straight-line.

What the deployed WHIR provides:

- **A $\mathbb{K}$ -valued word, not a codeword.** The root commits (up to collisions) to a leaf image per position: 2^6 words of 8 bytes at level 0 (`whir.rs:331-345`, `crates/pcs/src/merkle.rs:144-192`), and every 8-byte pattern is an element of $\mathbb{K}$ ($\mathbb{K} = \mathbb{F}_2[x]/(x^{64} + \dots)$, `A:8; F64(u64)`), so the committed object is an interleaved word $w \in \mathbb{K}^{2^6 \times n_0}$ by encoding, with no check that any row is a codeword: **proximity, not membership** (`B:4` “our PCS is only list binding”; `B:70-72` Definition B.4, the citation check correcting the dossier’s “Definition B.1”). The verifier supplies the zero prefix of the absent lanes (`fs/merkle.rs:53-58`, `whir.rs:2125-2132` with `row_words = n_lanes`), so w ’s rows past n_{lanes} are zero in the Rust’s proof format (section 8.7.1.13 for the Python’s).
- **A list, over $\mathbb{E}$.** Theorem B.7 binds the prover to $\Lambda(w) = \{g_U : U \in L_{\gamma_0}(w)\}$, the interleaved codewords of $\text{RS}[\mathbb{E}, \text{dom}_0, 2^{\kappa_0}]$ within radius γ_0 (`B:137`), of size at most L_0 (110 to 648 over the supported sizes; the citation check corrected the dossier’s “110 to 396”, the range at rate 1/2 only, section 8.7.3.2). A priori these are $\mathbb{E}$ -valued multilinear; the relation needs $\mathbb{K}$ -valued ones. **They are $\mathbb{K}$ -valued**, by the lemma below (the dossier’s argument; the specification does not say it). So every member of $\Lambda(w)$ is $\mathbb{K}$ -valued, of μ variables after the relayout $f(u, x) = q(x, u)$ (`B:380`), with zero rows where w has them. This lemma belongs to Layer 11 or 12 and is a hypothesis of the list-binding compilation (section 8.7.3.1); without it the extracted witness has the wrong type.
- **Openings over $\mathbb{E}$ of weighted claims:** provided, that is Theorem B.7’s relation.
- **Extraction:** from the query log (the leaves) and list decoding, as in section 8.7.3.1; in the Johnson regime the list is small and Guruswami–Sudan finds it; in Lean, enumeration suffices.

Lemma: every codeword close to a $\mathbb{K}$ -valued word is $\mathbb{K}$ -valued (the dossier’s two-line argument)

Write $\mathbb{E} = \mathbb{K} \oplus \mathbb{K} \cdot y \oplus \mathbb{K} \cdot y^2$ and a row $U_u = U_u^{(0)} + y \cdot U_u^{(1)} + y^2 \cdot U_u^{(2)}$. The encoder’s generator matrix is $\mathbb{K}$ -valued (the novel basis evaluated at points of $\text{dom} \subset \mathbb{K}$, `B:302-306`, “a $\mathbb{K}$ -valued message gives a $\mathbb{K}$ -valued word”), so each coordinate projection $U_u^{(j)}$ is a codeword of $\text{RS}[\mathbb{K}, \text{dom}_0, 2^{\kappa_0}]$. On the agreement set A , $|A| \geq (1 - \gamma_0)n_0 > \rho_0 n_0 = 2^{\kappa_0}$ (since $\gamma_0 < 1 - \sqrt{\rho_0} \leq 1 - \rho_0$), $U_u = w_u \in \mathbb{K}$, so $U_u^{(1)}$ and $U_u^{(2)}$ vanish on more than $2^{\kappa_0} - 1$ points and are zero; $U = U^{(0)}$. The citation check re-derived the argument and finds it sound as written.

The assumption, stated. The master theorems assume the verifier’s oracle *is* one table q . The commitment gives a set $\Lambda(w)$ of at most L_0 tables, fixed when the root is sent, among which the prover may choose after seeing challenges. The security of the compiled protocol is the master theorem applied to every member at once, with a union bound: the per-challenge errors of every phase before the opening are multiplied by L_0 , the extractor is “list-decode, then select the member that satisfies `M3Holds`”, and the opening’s job is to show that some member satisfies every pooled claim (Theorem B.7’s relation R_{open} , `B:133-136`; acceptance test 11, `bp:1296-1299`, has the right words). The Rust’s own accounting states the same in one place (“the opening’s own evaluation claim sits at a post-commit random point, so at most one list member matches it except with $L \cdot \mu / |F|$ ”, `whir_config.rs:593-599`) and Flock in another (`literature.md`, section A.3, item 4, [BRW26] Remark 11). Neither the specification (§8.4 is `TODO`, `08:44-46`), nor the blueprint’s `verify_knowledgeSound`, nor any of its three interfaces contains it; the sentence “Layer 12 turns list binding into extraction of the one q that satisfies every pooled claim” (`bp:1190`) names the step without the theorem (finding 55).

8.7.4 The opening and the compilation: findings

Short forms in this section. Specification files are under `doc/leanvm/body/`: `08:` is `08-end-to-end-protocol.tex`, `B:` is `b-polynomial-commitment-scheme.tex`; `theorems.tex` is `doc/leanvm/preamble/theorems.tex`. Rust files: `whir.rs`, `whir_config.rs`, `whir_induce.rs` and `stack_open.rs` are under `crates/pcs/src/`; `fs/` is `crates/flat_shamir/src/`; `mod.rs` is `crates/lean_vm/src/cpu/mod.rs`, and `leaf.rs` is under `crates/lean_vm/src/`; `primitives/src/` is `crates/primitives/src/`. `py:` is `python-verifier/verifier.py`. These are at leanVM’s pin `a386121f`. `bp:` is the blueprint `docs/roadmap/protocol-blueprint.md` and `st:` the status `docs/roadmap/protocol-status.md`, both at leanerVM `b435631` (the checkout `144c5aa` shifts the blueprint’s lines after line 168 by four); `architecture.md` is `docs/architecture.md` there; `Compose.lean` is under `LeanerVM/Protocol/Spine/` at `b435631`. `ArkLib` at its old pin `dca90385`: `Basic.lean`, `FiatShamir/Basic.lean`, `BCS/Basic.lean`, `Security/Implications.lean` under `ArkLib/OracleReduction/`, `Commitments/Functional/Basic.lean` under `ArkLib/`. The number after the colon is the line. The checks named (M2, M6, ...) are those of table 14.5.

No finding makes a stated leanerVM theorem wrong today, since none of Layers 10 to 13 is built: there is no critical finding. Eight are major, nine minor, three notes.

Finding 51 (major). The opening phase runs a sumcheck leanVM does not run

Evidence. Sections 8.7.1.3 and 8.7.1.4 (`08:99`; `stack_open.rs:452-461`; `whir.rs:1324-1334`; `py:1362, 1012`).

Classification. An error of the blueprint, caused by the evaluation interface of Layer 0.

Proposed change.

Proposed change: Layer 10 (bp:1112-1114), with the convention *The oracle* and Layers 0, 11 and 12

As it stands.

```
def openingPhase (μ) (J : ℕ) : OracleReduction [] (StmtIn := Fin J → WeightedClaim μ) ... (StmtOut :=
  ⇔ Unit) ...
  (pSpec := V_to_P : E ; sumcheck (W_λ · q) rounds ; the final evaluation query)
  theorem openingPhase_rbrKnowledgeSoundness : ... ((J - 1)/|E| on λ, 2/|E| per round)
```

As proposed.

```
def openingPhase (I) : Phase.Def I (I.Stmt × FlockOut I) Unit
  (pSpec := V_to_P : E) -- λ; the verifier queries the stack once, at W_λ, and compares with C_λ
  theorem openingPhase_rbrKnowledgeSoundness : ... ((J - 1)/|E| on λ) -- the batching component G3
```

with, in the convention *The oracle* (bp:317), “`OracleInterface query Vector E μ_stack` and answer `eval₂Mle q`” replaced by “`OracleInterface query Weight μ_stack` (specification Definition 3.13) and answer $\sum_w W(w) \cdot q(w)$; an evaluation is the query `eqWeight r`”; in Layer 0 (bp:637-642) the instance accordingly, `Weight` moved below it; in Layer 11 the sentence “its level-0 batching challenge is the opening phase’s λ : the compilation replaces the opening phase by `whirOpen`”; in Layer 12 (bp:1214) “WHIR in place of the evaluation oracle” → “WHIR in place of the opening phase, over the level-0 word as the commit message”; the title of Layer 10 and bp:127 lose “the opening sumcheck”.

Reason. The deployed transcript after λ is WHIR’s, whose sumcheck is interleaved with its commitments and closes on one equation; the sketched transcript has μ extra rounds and an extra scalar, so `verify_iff_compiled` cannot hold for Rust proofs. The alternative (iii) of section 8.7.1.5 is acceptable; (i) is not.

Finding 52 (major). `verify_iff_compiled` names definitions that do not describe the compiled verifier

Evidence. Section 8.7.3.1 (the probe of ?? B.2.1.2; `Basic.lean:490-496`, `FiatShamir/Basic.lean:129-136`, `BCS/Basic.lean:59-63` at `dca90385`; `fs/transcript.rs:9-12, 223-224`, the citation check correcting the dossier’s `:225-228`; `fs/lib.rs:165-174`).

Classification. An error of the blueprint; the absence of a BCS transform in `ArkLib` is a deviation

forced upstream (issue #627 has no implementation and excludes ROM soundness).

Proposed change.

Proposed change: `verify_iff_compiled` (bp:1218-1221, 1231-1232)

As it stands. The theorem with `Verifier.fiatShamir (leanVmIopp ...).verifier` and “decode s proof”, “unconditional: it is about two definitions”.

As proposed. The statement of section 8.7.3.1,

```
theorem verify_iff_compiled (prog input proof) :
  verify prog input proof = true ↔
    ∃ s, s.Admissible prog ∧ ∃ msgs, decode prog s proof = some msgs ∧
      runWithChain (bcsCompile (leanVmIopp prog s)).verifier (blake2sChain prog s) input msgs
        = some () ∧ grindingChecks prog s msgs = true
```

with `bcsCompile` (a leanerVM definition, Layer 12: oracle messages replaced by roots, the openings as messages of the last round or as an appended message, the verifier’s queries answered from authenticated rows), `blake2sChain prog s` (the chain as a `QueryImpl`), and `grindingChecks` conjoined; and the sentence “unconditional, about two definitions, **and** a check only because `verify` is written from §8.5 before the right-hand side is unfolded”.

Finding 53 (major). `baseVerifier_extractsExecution` and `verify_knowledgeSound` are not probability statements

Evidence. Section 8.7.3.1 (bp:1247-1248, bp:326; `architecture.md`:264-273). The citation check adds that the Merkle leaf hash is multi-block BLAKE2s-256, not the one 64-byte compression, so the random oracle of the proposed statements must cover it separately or through the iterated construction.

Classification. An error of the blueprint (and of `docs/architecture.md`’s target theorem T4, which it transcribes).

Proposed change.

Proposed change: `baseVerifier_extractsExecution` (bp:1247-1248), `verify_knowledgeSound`, and T4 in `architecture.md`:264-269

As it stands.

```
theorem baseVerifier_extractsExecution (fs bcs mca flock) (h : verify prog input proof = true) :
  except with probability niError, ∃ t, ValidExecution prog input t
```

As proposed. The two statements of section 8.7.3.1: a random oracle H for the compression function, a Q -query adversary A^H outputting $(\text{prog}, \text{input}, \text{proof})$ with its query log, the straight-line extractor `extract` (leaves off the log, list decoding, selection), the event “ verify^H accepts and `witnessOf (extract ...)` fails `SatisfiedBy`” of probability $\leq \text{niError } Q$, and the existential corollary; the sentence “the random-oracle heuristic, that BLAKE2s’s compression function instantiates H , is the one non-formal step, named here and in `architecture.md`”; and in `architecture.md`:264-269 the same repair of T4.

Reason. For fixed $(\text{prog}, \text{input}, \text{proof})$ the hypothesis is decidable and the conclusion is a fixed proposition; the only randomness is the oracle’s, and the only knowledge statement is about an extractor.

Finding 54 (major). The Fiat–Shamir and BCS interfaces cannot be inhabited, and one is false as stated

Evidence. Section 8.7.3.1 (FiatShamir/Basic.lean:163-175, Security/Implications.lean:230-254, 305-313, BCS/Basic.lean, Commitments/Functional/Basic.lean:250-255 at dca90385, the same text at 7653a901 at Security/Implications.lean:200-224, 275-283 and Commitments/Functional/Basic.lean:249-254 as the citation check found; pull request #848’s `SingleSalt.lean`:973-993; issue #627’s plan; `literature.md`, section A.3). The citation check adds, for the proposed `BcsSecurity`: the Merkle leaves are BLAKE2s-256 of 512 or 384 bytes, eight or six

chained compressions, so a random oracle for the one 64-byte compression does not cover them; the interface must model the leaf hash by a second oracle or through the iterated construction.

Classification. An error of the blueprint (the universal claim of the comment is false; #627 is not a pull request); the missing upstream theorems are a deviation forced by ArkLib, whose workaround is to name the literature and the construction.

Proposed change.

Proposed change: the assumed interfaces (bp:1223-1225), verify_knowledgeSound, the ledger row (A5, bp:263) and the holes table (bp:614)

As it stands.

```

/-- Assumed interfaces, each an ArkLib theorem to come (ledger A5). -/
structure FiatShamirSecurity where ... -- rbr knowledge soundness of the IOPP  $\Rightarrow$  knowledge soundness
   $\hookrightarrow$  of its FS compilation in the ROM, error  $Q \cdot \max_i \varepsilon_i$ 
structure BcsSecurity where ... -- Merkle-compiled oracles: extraction from collision
   $\hookrightarrow$  resistance / ROM

```

As proposed.

```

/-- Assumed interfaces. No theorem of ArkLib states them at either pin (its Fiat-Shamir completeness
 $\hookrightarrow$  is admitted for a constant oracle, its BCS transform is commented out, its round-by-round  $\Rightarrow$ 
 $\hookrightarrow$  state-restoration implication is commented out); their witness obligation is the literature
 $\hookrightarrow$  (BCS16, CY24 Thm 25.2.1/31.2.1, BGKTTZ23 Thm 3.15) for the parts it covers, and a new statement
 $\hookrightarrow$  for grinding. -/
structure RbrToStateRestoration where ... -- worst-case rbr knowledge soundness  $\Rightarrow$  state-restoration
   $\hookrightarrow$  knowledge soundness at  $(Q + \text{rounds}) \cdot \max_i \varepsilon_i$ 
structure BcsSecurity where ... -- the IOP of E.3, Merkle-compiled with H random, keeps
   $\hookrightarrow$  state-restoration knowledge soundness up to  $3(Q^2 + 1)/2^{256}$ , with the straight-line extractor
   $\hookrightarrow$  reading leaves off the query log; hypothesis: leaf width and height fixed by the verifier
structure ChainFiatShamirSecurity where ... -- the hash chain of fs/lib.rs with H random, with
   $\hookrightarrow$  proof-of-work rounds of  $b_i$  bits, turns state-restoration knowledge soundness  $\varepsilon_{sr}(Q)$  into
   $\hookrightarrow$  adaptive knowledge soundness  $\varepsilon_{sr}(Q) + c \cdot Q^2/2^{256}$ , a query at a ground round costing  $2^{b_i}$ 
   $\hookrightarrow$  queries of H

```

(“the IOP of E.3” is the composed IOP of section 8.7.3.1), and verify_knowledgeSound (rs : RbrToStateRestoration) (bcs) (fs : ChainFiatShamirSecurity) (mca : McaJohnson) (lc : ListCompile) (flock). In the ledger row (A5, bp:263) and the holes table (bp:614): “#848 and #469 (single-salt and duplex-sponge Fiat-Shamir from state-restoration soundness, ideal per-round oracle, no proof of work); #627 is a design issue for the BCS transform whose plan excludes ROM soundness”.

Reason. An interface quantified over all IOPPs with error $Q \cdot \max \varepsilon_i$ is false (hash term, oracle messages, grinding), and a hypothesis that is false makes the theorem vacuous.

Finding 55 (major). The list-binding compilation is a theorem the blueprint names and does not state

Evidence. Sections 8.7.3.1 and 8.7.3.3 (B:4, 70-72, 133-137; whirl_config.rs:593-599; bp:1190, 1296-1299; finding 47; literature.md, section A.3, item 4, and section G.2).

Classification. An error of the blueprint (omission).

Proposed change.

Proposed change: Layer 12 (bp:1190) and niError

As it stands. “Layer 12 turns list binding into extraction of the one q that satisfies every pooled claim.”

As proposed. In Layer 12, the theorem of section 8.7.3.1 (the list-binding compilation) with its extractor (list-decode, select), its state function, its errors ($L_0 \cdot \varepsilon_i$ at the front’s challenges, Theorem B.7’s at WHIR’s), and its two lemmas: every member of $\Lambda(w)$ is $\mathbb{K}$ -valued and of the right size (section 8.7.3.3); the front’s verifier makes no oracle query. In **niError**, the factor L_0 on every oracle-protocol challenge.

Reason. Neither **FiatShamirSecurity**, **BcsSecurity** nor **McaJohnson** mentions the list; without the theorem the master theorems (about one q) do not reach **verify**.

Finding 56 (major). Perfect completeness does not survive the transform

Evidence. Section 8.7.3.1 (fs/lib.rs:126-155; Compose.lean:94-104; findings 38 and 46; lib-arklib.md, section G.2).

Classification. An error of the blueprint.

Proposed change.

Proposed change: the proof sketch of the target theorem (bp:1255-1257)

As it stands. “the second composes **constraintCompleteness**, **m3Holds_stackOf**, **piop_perfectCompleteness** and the determinism of the Fiat-Shamir chain (perfect completeness survives the transform without any assumption).”

As proposed. “the second composes **constraintCompleteness**, **m3Holds_stackOf**, **piop_perfectCompleteness** and the determinism of the chain: an honest interactive proof on the chain’s challenges is the compiled proof (an equation between definitions). The grinding is a search (fs/lib.rs:126-155); **prove** takes fuel and returns **Option Proof**, and **baseProver_complete** is **prove ... = some proof → verify ... proof = true**.”

Reason. No theorem gives the honest prover a nonce.

Finding 57 (major). niError cannot carry leanVM’s 128-bit claim

Evidence. Section 8.7.3.2 (whir_config.rs:25-27, 36-38, 57-60, 537-568, 593-599; leaf.rs:108-118; the scratch port of ?? B.2.1.3; literature.md, section A.3).

Classification. An error of the blueprint.

Proposed change.

Proposed change: niError (bp:1234-1235) and acceptance test 23

As it stands. “**niError** is the round-by-round maximum times the query bound plus the grinding-adjusted WHIR terms.”

As proposed. “**niError** Q is $(Q + \text{rounds})$ times the largest per-challenge error of the compiled protocol ($L_0 \cdot \varepsilon_i$ at each oracle-protocol challenge, Theorem B.7’s fold, OOD and batching terms, and $2^{-17} \cdot (1 - \gamma_i)^{t_i}$ at each ground query round) plus the hash term $c \cdot Q^2 / 2^{256}$; at the deployed parameters the maximum is the query rounds’ 2^{-128} and the fold rounds’ $2^{-128.2}$, in the Johnson regime conditional on Theorem 4.6, for every admissible (μ, ρ) (the query counts of **py:910**).” And in acceptance test 23: “ $\Sigma \text{piopError} < 2^{-150}$ is the interactive error of the oracle protocol at admissible sizes (below 2^{-159} with the window); it is not the security level of **verify**, which the WHIR query rounds set at 128 bits including 17 bits of grinding.”

Reason. Section 8.7.3.2.

Finding 58 (major). Layer 11's `whirOpen` batches a second time, and its `encode` serves level 0 only

Evidence. Table 8.20, section 8.7.1.4 (B:108-110; `whir.rs`:419-485; bp:1169, 1173, 1185).

Classification. An error of the blueprint.

Proposed change.

Proposed change: Layer 11 (bp:1169, 1173, 1185)

As it stands. `whirOpen` “stated exactly as Protocol B.1: batch, ...”, after the opening phase’s own batching; `encode` $(\kappa R : \mathbb{N}) : \text{Column } \kappa \rightarrow (\text{Fin } (2^{(\kappa+R)}) \rightarrow K) \text{ -- additive NTT on the } K \text{ basis.}$

As proposed. `whirOpen`’s level-0 batch is the opening phase’s λ (finding 51); `encode` $(\kappa R) : (\text{Fin } (2^\kappa) \rightarrow E) \rightarrow (\text{Fin } (2^{(\kappa+R)}) \rightarrow E)$ with the lemma that a $\mathbb{K}$ -valued message gives a $\mathbb{K}$ -valued word (B:306), level 0 being that case.

Finding 59 (minor). `encode_column_weight` is an existence statement every linear map satisfies

Evidence. Table 8.20 (B:309-315; `whir_induce.rs`:220-224).

Classification. An error of the blueprint (non-vacuity).

Proposed change.

Proposed change: Layer 11 (bp:1170)

As it stands. theorem `encode_column_weight` $(x) : \exists W_x : \text{Weight } \kappa, \forall f, \text{encode } \kappa R f x = (W_x, f) \text{ -- Lemma B.7}$

As proposed. def `columnWeight` $(\kappa R) (x) : \text{Weight } \kappa$ with `mle` $r = \prod i, ((1 + r i) + r i * \tilde{W}_i x)$ and theorem `encode_columnWeight` $(x) (f) : \text{encode } \kappa R f x = (\text{columnWeight } \kappa R x).pair f \text{ -- Lemma lem:colweight.}$

Reason. `Weight` is inhabited by any cube row with `evalMle` as evaluator, so the existential says only that `encode` is linear; the verifier evaluates the closed form.

Finding 60 (minor). The proof object's Merkle format is unspecified

Evidence. Sections 8.7.1.9 and 8.7.1.13, check M2 (`fs/transcript.rs`:9-19, 198-203; `fs/merkle.rs`:78-92, 246-264; `py`:392-404, 1049; the test harness `crates/lean_vm/tests/verifiers/python_verifier.rs`:50-80).

Classification. An error of the blueprint (omission).

Proposed change.

Proposed change: Layer 12, and docs/leanvm-target.md

As proposed. In Layer 12: “Proof.merkle is the pruned wire form `PrunedMerklePaths` (rows at the distinct sorted positions, one octopus), which the Rust prover emits; `merkleVerify` checks a phase, not a path; the raw form the Python and the guest read is derived from an accepting run and is not `verify`’s input. At level 0 the row holds the n_{lanes} committed lanes and the verifier supplies the zero prefix.” Also to `docs/leanvm-target.md`: the Python, run standalone, accepts level-0 leaves with nonzero absent lanes (soundness-neutral; established by reading, not by running, as the citation check notes).

Finding 61 (minor). Four transcript facts of WHIR are not in the blueprint

Evidence. Table 8.19; items 6 to 9 of table 8.24.

Classification. An error of the blueprint (omission), of transcribed content (the blueprint’s Category B).

Proposed change. In Layer 11 or the conventions:

- (a) the lane relay layout $f(u, x) = q(x, u)$ at level 0 and the rotation of the terminal point (`whir.rs:2297-2306`);
- (b) per-claim intro round polynomials for the OOD claim, sent before the level's λ_i , and for the query batch, sent after it, the batched polynomial formed by linearity (the citation check corrected the dossier's "both sent before λ_i ": the query batch's intro follows λ_i and depends on it, `whir.rs:1373` then `1405-1406`, `py:1044` then `1057`);
- (c) the last round message omitted;
- (d) the grinding nonce read before the positions at every level, 17 bits.

Finding 62 (minor). Level lacks the analysis parameter; the OOD count and the grinding bits are parameters

Evidence. Section 8.7.1.8 (`whir_config.rs:624-707`).

Proposed change.

Proposed change: the parameter block of Layer 11 (`bp:1157-1165`)

As proposed. structure Level where (fold rateExp queries oodSamples grindingBits : $\mathbb{N}$) (η : $\mathbb{Q}$) with whirError a function of η ; the deployed η_i (or m_i) recorded per (μ, ρ) as transcribed content (the blueprint's Category B).

Finding 63 (minor). The blueprint's Annex B numbers are not the pinned counter's

Evidence. `theorems.tex:4, 12` (the environment `protocol` shares the theorem counter); section 8.7.1.10 and table 8.20.

Proposed change. "Protocol B.1" $\rightarrow$ Protocol B.6; "Theorem B.2" $\rightarrow$ Theorem B.7; "Definition B.1" (list binding) $\rightarrow$ Definition B.4; "Lemma B.7" (column weights) $\rightarrow$ Lemma B.14; "Annex B.3" (`bp:278`, the novel basis) $\rightarrow$ §B.5; "(Annex B.4)" (`bp:1172`, the protocol) $\rightarrow$ §B.3. Cite labels (`thm:rbr`, `def:listbinding`, `lem:colweight`, `def:enc`) instead.

Finding 64 (minor). The chain's step is the 64-byte hash, not the compression with the state as chaining value

Evidence. Section 8.7.1.12 (`fs/lib.rs:18-24`; `primitives/src/hash.rs:83-87, 209-224`; `bp:216, 325, 1204`). The citation check agrees that the chain state fills the first eight message words of BLAKE2s's F , whose h input is always `PARAM_IV`, and adds two qualifications: `bp:216` also calls the byte hasher "the RFC 7693 wrapper over `compress` with the last-block flag only", so the blueprint's reading is an inference; and the Lean rendering the dossier proposes in section 8.7.1.12 must use leanISA's signature, `UInt32` flags and the parameter-folded IV (`primitives/src/hash.rs:64-81`).

Proposed change.

Proposed change: the *Fiat–Shamir* row of the conventions (`bp:325`)

As proposed. "every step is BLAKE2s-256($cv \parallel block$) over 64 bytes, that is the RFC compression $F(paramIV, cv \parallel block, t = 64, f_0 = 1)$ with the chain state in the message; `compress` of leanISA Layer 1 is F ".

Reason. The two readings are different functions; transcribed content (the blueprint's Category B).

Finding 65 (minor). `piopError_le`'s bound depends on the window; acceptance test 23 omits ring switching

Evidence. Section 8.7.3.2 (agrees with finding 8 and finding 42).

Proposed change. `piopError_le` states $\mu_{\text{bus}} \leq 28$ (from `Admissible`'s window) in its comment; test 23 reads " $\Sigma piopError < 2^{-159}$ at admissible sizes; ring switching's 2^{-160} dominates".

Finding 66 (minor). The status file’s finding on the Python verifier’s caps is false at the pin

Agrees with finding 9 and finding 44.

Evidence. `py:856-864` at `a386121f` checks $16 \leq \log_memory \leq 32$, $\tau_j \leq 32$, $\tau_{\text{BLAKE2S}} \geq 3$, $0 \leq \log_bytecode \leq 32$ with `log2_strict`. The status file’s finding (F9), “the Python verifier omits four caps ... to report upstream” (`st:316-319`, 164), would be a false report.

Finding 67 (minor). Layer 12’s mutation tests exercise too little

Evidence. Table 8.23, section 14.3.4.

Proposed change. Add the grinding nonce, a root’s third limb, a size’s upper limb, a trailing scalar, a surplus sibling, a level-0 row of the wrong width, a duplicated position.

Finding 68 (note). The upper per-log caps are implied by the stacking window

(Table 14.5, row S-implied.) A Lean Admissible with the window needs them only to match the Rust’s rejection set, not for any theorem.

Finding 69 (note). The Rust’s list-size accounting

(Section 8.7.3.2.) Only the algebraic checks are unioned over the Johnson list (`whir_config.rs:537-568`); the bus’s $4 \cdot 2^{\mu_{\text{bus}}}$ is not (`leaf.rs:108-118`), with 23 bits of room at these sizes. For `docs/leanvm-target.md`. (The citation check: the Rust’s own bound is $5 \cdot 2^{\mu} + 8(\mu + 1)^2$, `leaf.rs:110`, the $4 \cdot 2^{\mu}$ being the blueprint’s; with $L_0 \leq 2^{9.34}$ the room is ≈ 23.7 bits by the Rust’s count, so “23 bits” is consistent.)

Finding 70 (note). A stale comment in the source on the leaf size

`whir.rs:393-395` “one Merkle leaf of `num_interleaved * 16` bytes”; the code hashes `24 · num_interleaved (:463)`. For `docs/leanvm-target.md`. The citation check found more stale comments at the pin: `whir.rs:1364-1365` (and 1771-1772) say level 0 grinds “0 bits in the production profile”, while every level grinds 17 bits; `whir.rs:1239-1241` and 1636-1637 mention fold grinds the pinned code does not perform.

Other notes. The leanVM working tree was moved off the pin on 2026-09-30 at 09:42 (reflog), not by this review; the pinned objects are intact and every citation was re-verified with `git show`. Both Lean probes elaborate at the old and the new pins (appendix B.2.1); nothing in this part rests on a probe that did not run. (The citation check found the new-pin log; no log of the old-pin run of 2026-09-29 is on disk, so that run could not be checked.)

Negative results (what was checked and agrees). The moment and powers of λ across the three sources (table 8.17); the Python’s `verify_whir` against the Rust’s succinct verifier message by message (table 8.19); the Python’s `derive_config` against the Rust’s `configs_for_rate` for all 56 (ρ, μ) (?? B.2.1.3); the column weight’s closed form against Lemma `lem:colweight` (section 8.7.1.7); the digest encoding, the tags, the seed, the squeeze and the grinding predicate between Rust and Python (table 8.21); the round-polynomial encoding (section 8.7.1.14); the blueprint’s six WHIR constants and its `ladder_queries_eq` pin (section 8.7.1.8); its *Fiat-Shamir* row’s tags, seed and squeeze (section 8.7.1.12); its *Claim pool order* row (table 8.17); `WeightedClaim.Holds` against Definition 3.13 (section 8.7.1.5); `McaJohnson` against ArkLib’s admitted statement and Annex B (section 8.7.3.1).

8.8 The adaptor and the boundary with leanISA

The two relations side by side, the chain from the program and the public input to the semantics, the hypotheses and who discharges each, and the adaptor’s sketched statements read as statements to be written in Lean; then the findings.

8.8.1 The adaptor: the two relations side by side

Short forms in this section. `leanerVM` is at `b435631`; `bp:` is the blueprint `docs/roadmap/protocol-blueprint.md` there. Lean files are named by their last component: `Statement.lean`, `Boundary.lean`, `Channels.lean`, `Bytecode.lean` are under `LeanerVM/Arithmetization/`; `Memory.lean`, `Execution.lean` under `LeanerVM/Semantics/`; `Spine/Instance.lean`, `Spine/Toy.lean`, `FixedColumns.lean`, `Stack.lean`, `ToArkLib/Refinement.lean` under `LeanerVM/Protocol/`; all at `b435631`. The arithmetization, semantics and parameters modules cited are byte-identical between `b435631` and `144c5aa`, and `Spine/Instance.lean` differs by one import line. `Clean` is cited at its old pin `93c9d1ef` (`FlatEnsemble.lean`, `FlatComponent.lean` under `Clean/Air/`, `Operations.lean`, `Expression.lean` under `Clean/Circuit/`), `ArkLib` at `dca90385`, `CompPoly` at `3468b38c`. Rust files are under `crates/lean_vm/src/` (`layout.rs` is `cpu/layout.rs`), at `leanVM`'s pin `a386121f`, read with `git show a386121f:<path>`. The number after a colon is the line.

The proof system proves knowledge of a stack satisfying `M3Holds I input q`, a relation on one committed column (`Spine/Instance.lean:219-220`); the arithmetization consumes `SatisfiedBy prog input w`, a relation on a `Clean EnsembleWitness` (`Statement.lean:311-340`). The *adaptor* between them is sketched in the blueprint's Layers 2 and 3 (`bp:757-867`) and is not built. This section sets the two relations side by side, clause by clause, and asks what the adaptor's two theorems need.

8.8.1.1 The objects

Every object below is quoted from the pinned sources.

- **Clean** (`93c9d1ef`) is the circuit library the arithmetization is written in. A `Component F` is one AIR table's row circuit: an input row type, and a `GeneralFormalCircuit` with its constraints, its bus interactions and its specification. An `Ensemble F PublicIO` is a list of components, a list of bus channels and one distinguished "verifier" component read once on the public input (`FlatEnsemble.lean:11-17`). An `EnsembleWitness ens` is the prover's data for it: one `Table` (a list of raw rows, `Array F`) per component, the string-keyed `ProverData` (a function `String → (n : ℕ) → Array (Vector F n)`, `Expression.lean:21-22`), and the public input (`FlatEnsemble.lean:19-25`). It lives in `Type 1` because a `Component` bundles type-level fields (probe `Shapes`, output lines 1–4). `w.Constraints` is "every component's asserted expressions vanish on every row and every lookup is contained" (`Operations.lean:687-688`; bus guarantees are *not* part of it, probe `Shapes`, output lines 154–159), and `w.interactions` is the concatenation of every table's emitted bus interactions, the verifier's one row first (`FlatEnsemble.lean:225-226`).
- **The spine** (`LeanerVM/Protocol/Spine/Instance.lean`) is the proof system's abstract view of an arithmetization. An `M3Instance (:119-148)` is: a `Shape` (a number of tables, each with a log-height τ and a width); a statement type `Stmt`; per table, a list of constraint polynomials and a list of flush tuples (a `Side`, push or pull, and sixteen coordinate polynomials, the domain separator first); a degree bound `d` with its two proofs; the count columns; the boundary blocks; the log-height μ of the one committed column and a `Layout` reading every column off it; the public lines; and the Flock predicate `aux`. A `Coord` of a boundary block (`:86-89`) is a constant, a `known` column (data both parties hold: the index column, the program) or a `committed` column of a table. `M3Holds I input q (:219-220)` is the conjunction of five clauses on the column `q`.
- **ArkLib** (`dca90385`) is the proof-system library. Its knowledge-soundness game `Verifier.knowledge SoundnessWith (Security/Basic.lean:299-323)` fixes a statement, runs a malicious prover against the verifier, applies a named straight-line extractor to the transcript, and bounds the probability that the verifier accepts *and* no witness in the extractor's `Option` slot is valid. Statement and witness types are in `Type`. A `Refinement R S (ToArkLib/Refinement.lean:31-36)` is a map on witnesses sending every witness valid for `R` to one valid for `S`; its two witness universes are independent (probe `Shapes`, output lines 10–13).
- **CompPoly** (`3468b38c`) supplies the computable polynomials: `CMvPolynomial n K` (sparse maps from monomials to coefficients, with `eval` and `totalDegree`) and the hypercube tables `CMLPolynomial Eval`. Mathlib's `MvPolynomial` is the classical object proofs are easiest in; `fromCMvPolynomial` maps the computable one to it (`MvPolyEquiv/Core.lean:35`), and `toCMvPolynomial` maps back and is `noncomputable (:41)`.

8.8.1.2 The relation leanISA consumes: **SatisfiedBy**

```

structure SatisfiedBy (prog : Program) (input : PublicInput)
  (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop where
  public_input_eq : w.publicInput = PublicIO.ofInput input
  constraints : w.Constraints
  state_balanced : BalancedPair w StatePull.toRaw StatePush.toRaw
  mem_balanced : BalancedPair w MemPull.toRaw MemPush.toRaw
  bytecode_balanced : BalancedPair w BytecodePull.toRaw BytecodePush.toRaw
  counts_nonzero : CountsNonzero w
  caps : Caps w
  index_columns : IndexColumnsAreRowIndices w
  seed_rows : SeedRowsAreTheImage w
  bytecode_rows : BytecodeRowsAreTheProgram prog w
  blake2s_valid : Blake2sRowsValid w
  word0_eq : (imageOf w.data).2.read (gpow 0) = some input.word0
  word1_eq : (imageOf w.data).2.read (gpow 1) = some input.word1

```

LeanerVM/Arithmetization/Statement.lean:311-340 at b435631, docstrings omitted (probe Shapes prints it, output lines 29–55)

The thirteen conjuncts, as the definitions below unfold them:

- **public_input_eq**: the witness’s public input is the four lanes of **input**; no component reads them (Boundary.lean:120-123).
- **constraints**: Clean’s **w.Constraints**, for every table including the verifier’s one-row table ($\forall e \in \text{ops.constraints}, \text{env } e = 0 \wedge (\forall l \in \text{ops.lookups}, l.\text{Contains env})$) (FlatEnsemble.lean:216, FlatComponent.lean:184-186, Operations.lean:687-688). leanISA emits no lookup (a grep of LeanerVM/Arithmetization/Tables/ finds the word in prose only) and only **jumpTable** asserts (Tables/Jump.lean:197-198, the two residuals on its two local witness cells **w, b**, :195-196).
- **state_balanced**, **mem_balanced**, **bytecode_balanced**: for each channel pair, the messages sent on the channel of that name, multiplicities ignored, form a permutation of each other, counted in $\mathbb{N}$ (BalancedPair).
- **counts_nonzero**: message index 1 of every pull on the memory and bytecode channels, in the six opcode tables, is nonzero (the count product).
- **caps**: three of **read_public**’s eight checks, plus a shape of the data.
- **index_columns**, **seed_rows**, **bytecode_rows**: the three facts Clean cannot express. The memory block’s index column is **gpow i**; the memory block’s rows are the image the prover data names; the bytecode block’s rows are **bytecodeRowOf prog i**.
- **blake2s_valid**: Flock’s conjunct; every BLAKE2S row satisfies the compression relation.
- **word0_eq**, **word1_eq**: the *whole* E-word of the image at index **gpow 0** (resp. **gpow 1**) is **input.word0** (resp. **input.word1**), top limb included.

One level down, in the same file:

```

-- :174-181 the ensemble: eight components, six channels, the program-bearing verifier
def leanIsaEnsemble (prog : Program) : Ensemble K PublicIO where
  tables := [{xorTable}, {mulTable}, {setTable}, {derefTable}, {jumpTable}, {blake2sTable},
    {memTable}, {bytecodeTable}]
  channels := [StatePull.toRaw, StatePush.toRaw, MemPull.toRaw, MemPush.toRaw,
    BytecodePull.toRaw, BytecodePush.toRaw]
  verifier := leanIsaVerifier prog
-- :207-209 messages by channel NAME, multiplicities ignored
def messagesOn (w : EnsembleWitness (leanIsaEnsemble prog)) (c : RawChannel K) :
  List (Array K) :=
  (w.interactions.filter (·.channel.name = c.name)).map (·.msg)
-- :229-231 balance: a permutation of message lists, counted in  $\mathbb{N}$ 
def BalancedPair (w : EnsembleWitness (leanIsaEnsemble prog)) (pull push : RawChannel K) :
  Prop :=
  (messagesOn w push).Perm (messagesOn w pull)
-- :240-243 the count product: message index 1 of every pull of the six tables
def CountsNonzero (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop :=
   $\forall t \in w.tables.take\ 6, \forall i \in t.interactions,$ 
   $(i.channel.name = MemPull.name \vee i.channel.name = BytecodePull.name) \rightarrow$ 
   $\forall h : 1 < i.msg.size, i.msg[1] \neq 0$ 
-- :250-263 three of read_public's eight checks, plus a shape of the data
structure Caps (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop where

```



```

minLogMem_le : minLogMem ≤ (imageOf w.data).1
le_maxLogMem : (imageOf w.data).1 ≤ maxLogMem
heights : ∀ t ∈ w.tables, ∃ τ ≤ maxLogRows, t.table.length = 2 ^ τ
blake2s_height : 2 ^ minLogRowsBlake2s ≤ (blake2sRows w).length
well_shaped : WellShapedData w.data
-- :272-273, :279-283, :290-293 the three facts Clean cannot express
def IndexColumnsAreRowIndices (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop :=
  ∀ i (hi : i < (memBlockRows w).length), (memRowAt w (memBlockRows w)[i]).idx = gpow i
def SeedRowsAreTheImage (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop :=
  ∃ idx cntFin : Fin (2 ^ (imageOf w.data).1) → K,
    memBlockRows w = List.ofFn fun i ↦
      (toElements ((idx i, cntFin i, #v[((imageOf w.data).2 i).limb 0,
        ((imageOf w.data).2 i).limb 1, ((imageOf w.data).2 i).limb 2]) : MemRow K)).toArray
def BytecodeRowsAreTheProgram (prog : Program) (w : EnsembleWitness (leanIsaEnsemble prog)) :
  Prop :=
  ∃ cntFin : Fin (2 ^ prog.logSize) → K,
    bytecodeBlockRows w = List.ofFn fun i ↦ (toElements (bytecodeRowOf prog i (cntFin i))).toArray
-- :301-302 Flock's conjunct
def Blake2sRowsValid (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop :=
  ∀ row ∈ blake2sRows w, Blake2sRelation (blake2sRowAt w row)

```

LeanerVM/Arithmetization/Statement.lean at b435631, lines as marked; docstrings and the ensemble's proof field omitted; the comments are the review's

and, in Channels.lean, the image read off the prover data:

```

def imageOf (data : ProverData K) : (κ : ℕ) × MemImage κ :=
  (min (Nat.log 2 (memRows data).size) maxLogMem,
   fun i ↦ ((memRows data)[(i : ℕ)]?).map fun v ↦ E.ofLimbs v[0] v[1] v[2]).getD 0)

```

LeanerVM/Arithmetization/Channels.lean:188-190 at b435631

In Boundary.lean are the three block components and their honest rows: memTable (:157-160, push (idx, 1, m), pull (idx, cntFin, m)), bytecodeTable (:240-243, push (idx, 1, opcode, op), pull (idx, cntFin, opcode, op)), leanIsaVerifier prog (:311-314, push (1, 1), pull (const prog.finalPc, 1)), memRowOf (:184-185) and bytecodeRowOf (:274-276: {gpow i, cntFin, e[0], #v[e[1], ..., e[7]]} with e := entry (prog.code i)).

8.8.1.3 The relation the proof system proves: M3Holds

```

-- :197-198
def ConstraintsVanish (q : Column I.μ) : Prop :=
  ∀ j, ∀ C ∈ I.constraints j, ∀ x, C.eval (I.row q j x) = 0
-- :201
def Balanced (q : Column I.μ) : Prop := (I.tuples q .push).Perm (I.tuples q .pull)
-- :204-205
def CountsNonzero (q : Column I.μ) : Prop :=
  ∀ j, ∀ i ∈ I.counts j, ∀ x, (I.column q (j, i)).values.get x ≠ 0
-- :208-211
def PublicLinesHold (input : I.Smt) (q : Column I.μ) : Prop :=
  ∀ l ∈ I.publicLines input,
    (I.column q l.col).values.get {0, Nat.two_pow_pos _} = l.cell0 ∧
    (I.column q l.col).values.get {1, Nat.one_lt_two_pow l.pos.ne'} = l.cell1
-- :219-220
def M3Holds (I : M3Instance) (input : I.Smt) (q : Column I.μ) : Prop :=
  I.ConstraintsVanish q ∧ I.Balanced q ∧ I.CountsNonzero q ∧ I.PublicLinesHold input q ∧ I.aux q

```

LeanerVM/Protocol/Spine/Instance.lean at b435631, lines as marked, docstrings omitted (probe Shapes prints M3Holds, output lines 100–134)

The five clauses are: every constraint polynomial vanishes on every row of its table; the pushed tuples are a permutation of the pulled tuples; every cell of every count column is nonzero; cells 0 and 1 of every column a public line names hold the line's two values; and the instance's predicate aux holds. The tuples of a side are the tables' flushes evaluated on every row, then the boundary blocks' tuples, coordinate by coordinate:

```

def coordCell (q : Column I.μ) {κ : ℕ} : Coord I.toShape κ → Fin (2 ^ κ) → K
| .const c, _ => c

```

```

| .known col, x => col.values.get x
| .committed c h, x => (I.column q c).values.get (Fin.cast (congrArg (2 ^ ·) h.symm) x)
def tuples (q : Column I.μ) (s : Side) : List (Vector K 16) :=
  I.flushTuples q s ++ I.boundaryTuples q s

```

LeanerVM/Protocol/Spine/Instance.lean:167-186 at b435631, docstrings and the definitions of flushTuples and boundaryTuples omitted

A **known** coordinate is read from data the instance holds, a **const** coordinate is a constant, and only a **committed** coordinate is read from **q**.

8.8.1.4 The leanISA instance the blueprint implies

Read from Layer 3 (bp:794-867), the status file’s decision that “the fixed columns are **Coord.known** data” (decision 8, `protocol-status.md`:220-223) and the ground truth (`layout.rs`:352-395; the bus dossier’s table of the boundary blocks, section 8.4.1.5):

- **Tables 0–5:** the six opcode components through `Component.toM3` (Layer 2): width `component.width` (JUMP’s includes its two local witness cells), $\tau_j := s.\tau_j$, the constraint polynomials, the flush tuples with separator $g^0/g^1/g^2$ first and side from `channelDir`, the count columns.
- **Column groups:** the six shared committed columns, “tables of the instance with no constraints and no flushes” (bp:836-839). They cannot be one table: their heights differ (`mem_0`, `mem_1`, `mem_2`, `cntfin_mem` at $2^{\log \text{Mem}}$; `cntfin_bc` at $2^{\text{prog.logSize}}$, `layout.rs`:150; `q_flock` at $2^{(\tau_5 + 8)}$, `layout.rs`:155 with `K_LOG = 14`, `LOG_PACKING = 6`). So at least three tables with empty constraints, flushes and counts, which the spine’s `Shape` (`Spine/Instance.lean`:57-63) and `M3Instance.counts` allow.
- **Six boundary blocks:** `Coord.known` (`idxColumn κ`) for the index, `Coord.known` (`bytecodeSlot Column prog s`) for the eight program columns (`FixedColumns.lean`:75-76), `Coord.committed` for the limbs and the finalize counts, `Coord.const` for the separators, the seed count 1 and the state boundary ($\kappa = 0$). The state pull’s counter is `const prog.finalPc`, as `Boundary.lean`:314 and `layout.rs`:357.
- **Three public lines** on `mem_0`, `mem_1`, `mem_2` (bp:834-836): cells (`lanes 0`, `lanes 2`), (`lanes 1`, `lanes 3`), (`0`, `0`), the first two sent.
- **aux:** Flock’s R1CS on `q_flock` (the strong reading, decision 12), `decAux` its check.

8.8.1.5 Conjunct by conjunct

Table 8.27 takes $w := \text{witnessOf prog s q}$ and $I := \text{leanIsaInstance prog s}$, with the hypotheses $hs : s.\text{Admissible prog}$ and $h : M3\text{Holds } I \text{ input q}$, and says for each conjunct of `SatisfiedBy` what delivers it in `satisfiedBy_witnessOf`, through which lemma, and whether that works. The earlier review’s table (`docs/reviews/protocol-spine.md`:331-342) was redone from the definitions.

Table 8.27: The conjuncts of `SatisfiedBy` against the clauses of `M3Holds`, in the soundness direction `satisfiedBy_witnessOf`.

Conjunct	Delivered by	Through	Verdict
<code>public_input_eq</code>	clause 4 (<code>PublicLinesHold</code>), lines 1–2, if <code>witnessOf</code> reads the lanes off cells 0, 1 of <code>mem_0</code> , <code>mem_1</code> ; or by construction if the map takes input (the <code>Refinement.map</code> has it, the blueprint’s <code>witnessOf prog s q</code> does not)	<code>limb_ofLimbs</code>	works either way; harmless as a built part, since no component reads the lanes (<code>Boundary.lean:120-123</code>)
<code>constraints</code>	clause 1 (<code>ConstraintsVanish</code>) for tables 0–5; vacuous for tables 6, 7 and the verifier (no assert)	Layer 2’s <code>toM3_constraints_iff</code> , whose right side must be Clean’s <code>Operations.ConstraintsHold</code> (the two-conjunct form, <code>Operations.lean:687</code>), not the flat <code>constraintsHold_iff_forall_mem</code> of <code>Operations.lean:168-182</code> the blueprint cites (<code>bp:292</code>), which includes the bus guarantees	works, provided <code>witnessOf</code> writes the JUMP witness cells read from <code>q</code> into the rows; citation to fix
<code>state_balanced</code> , <code>mem_balanced</code> , <code>bytecode_balanced</code>	clause 2 (<code>Balanced</code>), one permutation of sixteen-tuples, filtered by coordinate 0 (the separator) and mapped back to messages	<code>List.Perm.filter</code> , injectivity of <code>busTuple</code> on arrays of one size; Layer 2’s <code>toM3_flushes_eq</code> for tables 0–5; a lemma the blueprint does not have for the memory block, the bytecode block and the verifier: their messages in <code>w</code> equal the instance’s boundary tuples (section 8.8.1.6)	works only with the missing lemma and only because the boundary’s known data are <code>prog</code> ’s (section 8.8.1.6)
<code>counts_nonzero</code>	clause 3 (<code>CountsNonzero</code>)	per table, the pull’s count coordinate is <code>var c</code> for <code>c ∈ I.counts j</code> (<code>memRead</code> , <code>bytecodeRead</code> , <code>Channels.lean:244-254</code> : the count is a column in every table)	works if <code>Component.toM3</code> computes count as “coordinate 2 of every pull on a lookup channel, when it is a variable”; the blueprint gives <code>toM3</code> no way to know which channels are lookups (<code>bp:773-774</code> ; finding 81)
<code>caps</code>	nothing in <code>M3Holds</code> ; <code>hs</code> and the construction: tables of exactly 2^{τ} rows, “mem” data of $2^{\log \text{Mem}}$ rows	<code>Admissible</code> must contain $16 \leq \log \text{Mem} \leq 32$, $\tau_j \leq 32$, $3 \leq \tau_5$; table 7’s height is $2^{\text{prog.logSize}} \leq 2^{32}$ by <code>Program.logSize_le</code> and <code>maxLogBytecode = maxLogRows</code>	sound: the verifier checks the sizes (<code>cpu/mod.rs:157-176</code>); <code>WellShapedData</code> is a shape of the built data, no check
<code>index_columns</code>	construction: <code>idx := gpow i</code>	matched by <code>Coord.known (idxColumn logMem)</code> in both memory blocks (<code>idxColumn_get</code> , <code>FixedColumns.lean:50-53</code>)	sound (section 8.8.1.6)
<code>seed_rows</code>	construction: “mem” table and block rows both from <code>q</code> ’s <code>mem_0</code> , <code>mem_1</code> , <code>mem_2</code> , <code>cntFin</code> from <code>cntfin_mem</code>	<code>imageOf</code> returns <code>logMem</code> only if <code>logMem ≤ 32</code> (the min in <code>Channels.lean:189</code>): needs <code>hs</code>	sound
<code>bytecode_rows</code>	construction: <code>bytecodeRowOf prog i (cntfin_bc[i])</code>	matched by <code>Coord.known (bytecodeSlotColumn prog k)</code> , <code>k = 3..10</code> , <code>encodeSlots = entry at slots 3–10</code> (<code>Bytecode.lean:94-96</code>)	sound (section 8.8.1.6); the blueprint’s <code>bytecodeColumn</code> of <code>FixedColumns.lean</code> is the sixteen-slot table, whose eight slices <code>bytecodeSlotColumn</code> are the block’s columns (<code>slice_bytecodeColumn</code> , <code>FixedColumns.lean:90-92</code>)
<code>blake2s_valid</code>	clause 5 (aux)	the Flock roadmap’s lemma “the R1CS holds of the bits packed into <code>q_flock</code> $\Rightarrow$ the eighteen limb slots compress” (decision 12); the limbs of the row are read strided off <code>q_flock</code>	works only with the lemma, which has no carrier in any sketched signature (finding 77)
<code>word0_eq</code> , <code>word1_eq</code>	clause 4, all three lines	<code>MemImage.read_gpow</code> ($\kappa < 64$ from <code>hs</code>), <code>ofLimbs_limb</code> ; the third line supplies <code>limb 2 = 0</code>	works; the third line is load-bearing (section 8.8.2.5)

Two conjuncts hold trivially of the built witness (`index_columns`, `bytecode_rows`) and one half-built (`seed_rows`); this is where the trap of the built parts lives. Every conjunct has a source; none is delivered by nothing once the caps hypothesis, the boundary lemma, the count derivation and the Flock consequence lemma are in place.

8.8.1.6 The trap of the built parts

What `witness0f` builds and what it reads. From `q` through `I.layout`: the rows of tables 0–5 (the BLAKE2S limbs strided off `q.flock`), the memory limbs and the two finalize counts. From `prog`: the eight entry cells of every bytecode-block row (`bytecodeRow0f prog i`) and the sentinel constant of the verifier component. From arithmetic: the index column `gpow i` of the memory block. From nothing: the “mem” table of the data is a copy of the limbs. So `w` is a **mixed witness**, and `SatisfiedBy`’s `bytecode_balanced` and `mem_balanced` are statements about lists that contain the built rows: `messages0n w BytecodePush` includes, for every slot `i`, the built message (`gi, 1, entry (prog.code i)`) (`bytecodeTable.main`, `Boundary.lean:240-243`, on the row `bytecodeRow0f prog i _`), and `messages0n w BytecodePull` the read messages of tables 0–5, taken from `q`.

What `M3Holds` establishes. `I.Balanced q` is a permutation between two lists of sixteen-tuples, both read from `q` *except* the coordinates the instance holds as `Coord.const` and `Coord.known` (`coordCell`, section 8.8.1.3). Whether the built rows’ messages are the instance’s boundary tuples is therefore decided by what `leanIsaInstance prog s` puts in `boundary`:

- If the bytecode blocks are boundary blocks with `Coord.known (bytecodeSlotColumn prog k)` and `Coord.known (idxColumn prog.logSize)` (decision 8; the shape leanVM has, `layout.rs:383-395`, `Coord::Public`, `Coord::Index`, and which the verifier evaluates itself, `leaf.rs:444, 453`), then for every slot `i` the instance’s pushed tuple is (`g2, gi, 1, e[0], ..., e[7], 0, ...`) with `e = encode Slots (prog.code i)` at slots 3–10 = `entry (prog.code i)` (`Bytecode.lean:94-96`), exactly the bus tuple of the built row’s message. The same for the memory blocks (`known idxColumn`, committed limbs and counts read from `q`, which `witness0f` also reads) and for the state boundary (`const prog.finalPc` against `Boundary.lean:314`). Then `I.Balanced q`, filtered by separator, *is* the three balances of the mixed witness, and the proof of `satisfiedBy_witness0f` goes through.
- If instead the two blocks were tables of the instance with flushes, obtained by `Component.toM3 memTable` and `Component.toM3 bytecodeTable` as the blueprint’s phrase “Ensemble.toM3 of the eight tables with leanISA’s separators and directions” (`bp:803-805`) says, then their `idx`, `opcode` and `op` columns would be columns of `q` (the row’s cells, `BytecodeRow`, `Boundary.lean:211-220`), `I.Balanced q` would speak of whatever the prover committed there, and `witness0f`, which must build the block’s rows from `prog` (`bytecode_rows` demands it), would produce a witness whose `bytecode_balanced` does not follow from `I.Balanced q`. The theorem `satisfiedBy_witness0f` would be unprovable; worse, the *protocol* would be a different one from leanVM’s (a stack with committed index and program columns, `μ` and the layout changed), so `verify_iff_compiled` and the Rust fixture would fail too.

The model. Probe `KnownColumn` builds the spine’s toy instance in both shapes: with the block’s column `known` and equal to the “program”, the honest stack satisfies `M3Holds` and fails `Balanced` for another program, so the relation is about the program in the instance; with the block’s column `committed`, a stack that has nothing to do with the program satisfies `M3Holds`, while the witness an adaptor would rebuild from it, the block built from the program, does not balance. It passed at the old pins and, rewritten with `K.ofBits 2` for the numeral 2 (which means 0 at the new `CompPoly` pin), as `KnownColumnNew` at the new pins (appendix B.3.2).

Where “the accepted proof concerns the intended program” is decided. Nowhere in the oracle protocol, whose theorems hold for every `I`; in the adaptor, by the *statement* of `satisfiedBy_witness0f`, which names `leanIsaInstance prog s` and `SatisfiedBy prog` with one `prog`, so that its proof can only close if the instance’s known coordinates are the program’s and the verifier’s sentinel is `prog.finalPc`; and in Layer 12’s `verify prog input proof`, which must evaluate those same known columns (`settleFixedClaims`) and seed the transcript with the program (`fs_seed`, `cpu/mod.rs:82-93`), tied to

the oracle verifier of `leanIsaInstance prog s` by `verify_iff_compiled`. The three are consistent when they name the same `prog`. What is missing is the lemma that lets the adaptor’s proof close: Layer 2’s `toM3_flushes_eq` is about a table’s flushes and cannot say anything about a boundary block. A **third bridge lemma** is needed: for the two Clean block components and the verifier component, the messages of the block’s rows (`Boundary.lean:157-160`, `240-243`, `311-314`) equal, as sixteen-tuples, the instance’s boundary tuples, given the three fixed-column facts of the witness (findings 71 and 72).

A remark on `SatisfiedBy` that helps. Its `word0_eq`, `word1_eq`, `caps.minLogMem_le`, `caps.le_maxLogMem` and `seed_rows` are all stated on `imageOf w.data`, the “mem” table of the prover data, not on the block’s rows; and `MemPull.Guarantees` (`Channels.lean:214-216`) too. `witnessOf` must therefore write the same limbs into the data and into the memory block, and `seed_rows` is what ties them; Clean pull request #446 would derive the data from the block’s columns and delete the conjunct (`Statement.lean:87-100`). Nothing in the chain reads the data from anywhere else.

8.8.1.7 The completeness direction

`stackOf gen w hs` : Column $I.\mu$ stacks the committed columns of `w` at the aligned offsets (Layer 1’s `Blocks.stackColumn`, `Stack.lean:70`), with `q_flock` computed by the Flock witness generator from the BLAKE2S rows’ limbs. Table 8.28 goes clause by clause, from `h : SatisfiedBy prog input w` and `hs : Sizes.ofWitness w = some s`.

Table 8.28: The clauses of `M3Holds` in the completeness direction
`m3Holds_stackOf`.

Clause	From	What else is needed
<code>ConstraintsVanish</code>	<code>h.constraints</code> , tables 0–5	<code>toM3_constraints_iff</code> on rows read back from the stack: <code>Blocks.readColumn_stackColumn</code> (<code>Stack.lean:85-87</code>) returns the block, so the rows read back are the rows stacked, provided every row has exactly <code>component.width</code> cells or <code>stackOf</code> reads <code>row[j]!.getD 0</code> as <code>Environment.fromArray</code> does; the BLAKE2S limbs read back from the strided slots equal the rows’ limbs, which is the generator’s “keeping the limbs in their slots”
<code>Balanced</code>	<code>h.state_balanced</code> , <code>h.mem_balanced</code> , <code>h.bytecode_balanced</code>	the missing boundary lemma in the other direction: the messages of <code>w</code> ’s tables 6, 7 and of the verifier row are the instance’s boundary tuples, which needs <code>h.index_columns</code> (the rows’ <code>idx</code> cells are <code>g^i</code> , the stack does not hold them), <code>h.seed_rows</code> (the block’s limbs are the data’s, which the stack holds) and <code>h.bytecode_rows</code> (the rows’ entry cells are <code>prog</code> ’s, not stacked); then the three permutations concatenate into one, since every flush tuple carries exactly one of the three separators
<code>CountsNonzero</code>	<code>h.counts_nonzero</code>	<code>I.counts j</code> $\subseteq$ the columns that appear as a pull’s count coordinate (the converse inclusion the soundness direction needs; so <code>counts j</code> must be exactly that set)
<code>PublicLinesHold</code>	<code>h.word0_eq</code> , <code>h.word1_eq</code> , <code>h.seed_rows</code> , <code>h.caps</code>	<code>MemImage.read_gpow</code> at $\kappa < 64$, <code>limb_ofLimbs</code> ; the third line’s <code>(0, 0)</code> from the top limb of <code>input.word0</code> , <code>input.word1</code> , which is <code>0</code> by definition (<code>Memory.lean:107-110</code>)
<code>aux</code>	<code>h.blake2s_valid</code>	FlockWitnessGen’s lemma: the wires the generator computes from limbs that compress satisfy the R1CS (decision 12)

What else `m3Holds_stackOf` needs: nothing on the caps beyond `Sizes.ofWitness w = some s`, which `h.caps.heights` (every height a power of two at most 2^{32}) supplies for the eight tables, together with `h.seed_rows` (table 6 has $2^{(\text{imageOf } w.data).1}$ rows) and `h.bytecode_rows` (table 7 has $2^{\text{prog.logSize}}$); and `leanIsaInstance`’s layout fit, which is a `Blocks` fact and must precede the instance (the Layer 1 dossier’s finding that `leanIsaInstance_fits` cannot be stated as written, `code-layer1.md`, G.2). What `verify`’s acceptance needs beyond it is not in `SatisfiedBy` at all: the stacking window $\mu \in [15, 28]$ and the rate window (section 8.8.2.4 and finding 74).

8.8.2 The adaptor: the program, the public input and the hypotheses along the chain

Short forms in this section. `bp`: is the blueprint `docs/roadmap/protocol-blueprint.md` at leanerVM b435631; `architecture.md`, `leanvm-target.md` and `leanisa-blueprint.md` are under `docs/` (the last under `docs/roadmap/`) at the same revision. Lean files are named by their last component, as in section 8.8.1: `Statement.lean`, `Boundary.lean`, `Channels.lean`, `Bytecode.lean` under `LeanerVM/Arithmetization/`; `Memory.lean`, `Execution.lean`, `Instruction.lean`, `Step.lean` under `LeanerVM/Semantics/`; `Spine/Instance.lean`, `Spine/Compose.lean`, `FixedColumns.lean` under `LeanerVM/Protocol/`; all at b435631. Rust files are under `crates/lean_vm/src/` (`layout.rs` is `cpu/layout.rs`), except `lean_compiler/src/lib.rs` and `filler.rs` under `crates/lean_compiler/src/`; `verifier.py` is `python-verifier/verifier.py`; all at leanVM's pin a386121f, read with `git show`. ArkLib is cited at its old pin dca90385 (`Security/Basic.lean` and `ProtocolSpec/Basic.lean` under `ArkLib/OracleReduction/`). The target theorems of `architecture.md` are named in words, with their code in parentheses: `constraint soundness` and `completeness` of the arithmetization (`T1-S`, `T1-C`; `constraintSoundness`, `constraintCompleteness`), witness generation (`T2`), guest correctness (`T3`), the base theorem of the proof system (`T4`), recursion (`T6`), end-to-end soundness (`T7`).

8.8.2.1 How the program and the public input enter each link

Table 8.29: The program and the public input along the chain, from the deployed verifier to the execution.

Link	The program	The public input
The deployed verifier (<code>cpu/mod.rs:711-779</code>)	seeds the transcript with <code>fs_seed(program)</code> , the BLAKE2s of "leanvm" len R1CS_DIGEST <code>bytecode_hash</code> (<code>cpu/mod.rs:82-93</code> , 712), where <code>bytecode_hash</code> is the hash of the stacked sixteen-slot table (<code>cpu/mod.rs:73-76</code> , <code>layout.rs:317-326</code>); builds the layout from <code>prog.prog</code> (<code>read_public</code> , <code>cpu/mod.rs:171</code>): the final counter $g^{(len-1)}$ as a constant of the state pull (<code>layout.rs:335</code> , 357) and the eight public columns of the two bytecode blocks (<code>layout.rs:343</code> , 385-395), which the verifier evaluates itself at ζ_{lo} (<code>leaf.rs:453</code>) and never opens; the bytecode length must be a power of two at most 2^{32} (<code>cpu/mod.rs:157-159</code>). The Python takes the raw stacked table (<code>verifier.py:1365-1368</code>)	seeds the transcript as four 64-bit words (<code>digest_words</code> , <code>cpu/mod.rs:99-106</code> , 712), after rejecting a nonzero third limb (<code>cpu/mod.rs:141-143</code>); enters the layout as <code>l.pi</code> (<code>layout.rs:83</code> , 424), read by the public-input check alone (<code>cpu/mod.rs:752-755</code>); binds nothing else (the boundary tuples do not depend on it: section 8.4.1.5). The Python's Digest is 32 bytes by construction (<code>verifier.py:201-224</code>)
The spine	the instance <code>I : M3Instance</code> is a parameter of every phase and theorem; for <code>leanISA</code> it is <code>leanISAInstance</code> <code>prog s</code> , whose boundary holds the program as <code>Coord</code> . known data (the eight <code>bytecodeSlotColumn</code> <code>prog k</code> , <code>FixedColumns.lean:75-76</code> , and <code>const prog.finalPc</code>) and whose <code>Stmt</code> is <code>PublicInput</code> . Nothing in <code>M3Holds</code> reads a program from <code>q</code>	input : <code>I.Stmt</code> is the statement of the oracle protocol (<code>M3Rel</code> , <code>Spine/Instance.lean:230-231</code>); it is read by one clause, <code>PublicLinesHold</code> input <code>q</code> , through <code>I</code> . <code>publicLines</code> input (three lines)
The adaptor (sketch)	<code>witnessOf prog s q</code> builds the bytecode block's rows and the verifier's sentinel from <code>prog</code> ; <code>satisfiedBy_witnessOf</code> names the same <code>prog</code> in <code>leanISAInstance</code> <code>prog s</code> , <code>witnessOf prog s q</code> and <code>SatisfiedBy prog input</code>	absent from <code>witnessOf</code> 's arguments (<code>bp:813-814</code>); recovered from <code>q</code> through the lines, or supplied by the refinement's <code>map : Stmt → W₁ → W₂</code>
<code>SatisfiedBy prog input w</code>	<code>bytecode_rows</code> (the block's rows are <code>bytecodeRowOf prog i</code>), <code>leanISAVerifier</code> <code>prog</code> pulling <code>const prog.finalPc</code> (<code>Boundary.lean:311-314</code>), and nothing else: the six tables and the channels are program-free (decision 14, <code>Channels.lean:88-105</code>)	<code>public_input_eq</code> (the lanes, read by no component) and <code>word0_eq</code> , <code>word1_eq</code> on the image the data names
<code>ValidExecution prog input t</code>	run <code>prog t.image t.steps</code> <code>Regs.initial = some (Regs.final prog)</code> , <code>Program.fetch</code> at every step (<code>Execution.lean:86-90</code> , 114-115)	<code>HasPublicBoundary</code> input <code>t: 16 ≤ κ ≤ 32</code> and the two words at g^0 , g^1 (<code>Execution.lean:108-110</code>)

8.8.2.2 A different program or input?

Within one instantiation, no: `prog` and `input` are the same variables in `verify prog input proof`, `verify_iff_compiled`, `leanISAInstance prog s`, `satisfiedBy_witnessOf`, `SatisfiedBy prog input` and

`ValidExecution prog input`. Every theorem of the chain is universally quantified over both outside any probability, and the built code confirms the shape: `piop_rbrKnowledgeSoundness (P : Phases I) (S : P.Security)` is stated for a fixed `I` (`Spine/Compose.lean:172-177`), and `ArkLib`’s game fixes `stmtIn` before the prover runs (`Security/Basic.lean:299-323`: $\forall \text{stmtIn} : \text{StmtIn}, \forall \text{witIn}, \forall \text{prover}, \text{Pr}[\dots] \leq \epsilon$). Three observations on the edges of that answer:

1. **The program is bound by the transcript only through its stacked table.** The Lean side holds `prog : Program` (typed instructions, `Instruction.lean:87-93`) and `FsState.seed prog input` must hash `bytecodeColumn prog` (`FixedColumns.lean:78-82`), as the Rust hashes `bytecode_table` (`cpu/mod.rs:73-76, 89`). That `prog  $\mapsto$  (prog.logSize, bytecodeColumn prog)` is injective follows from `entry_injective` (`Bytecode.lean:240-245`) and `bytecodeColumn_slot` (`FixedColumns.lean:84-87`); it is not stated anywhere and Layer 12 will need it for “statement binding” (`architecture.md:453-454`). Unverified: no probe was written for the lemma.
2. **Program and PublicInput are narrower than the deployed surfaces.** The Rust verifier takes `Vec<Op>` with u32 operands; Lean’s operands are any `K` (a superset); the Python takes any `K`-array (`verifier.py:1365`), including undecodable entries, of which Lean has none (the faithfulness review’s row 4, `docs/reviews/leanvm-faithfulness-review.md:72`). A public input with a nonzero third limb is rejected by the Rust (`cpu/mod.rs:141-143`) and by the Python’s Digest (`verifier.py:223`), and cannot be written in Lean (`PublicInput` is four lanes of `K`, `Memory.lean:101-104`). So two of `read_public`’s eight checks (leanISA’s numbering, `Statement.lean:71-73`) are discharged “by type” on the Lean side, which means: by whoever parses bytes into `Program` and `PublicInput`. The blueprint’s *Verifier shape* convention (`bp:326`) says structure checks the Rust `assert!`s are “theorems about the layout, not branches of `verify`”; it does not say that these two are parsing obligations of the fixture harness and, later, of end-to-end soundness’s (T7) `PublicInput.encode` (finding 85).
3. **The sizes are the prover’s**, and this is where the family of instances is quantified. `verify_iff_compiled` (`bp:1218-1221`) has $\exists s, s.\text{Admissible } \text{prog} \wedge \dots$; the prover writes `log_mem`, six `$\tau_j$` and `log_inv_rate` onto the stream (`announce_public`, `cpu/mod.rs:118-124`) and the verifier reads and checks them before anything else (`read_public`, `cpu/mod.rs:144-176`; the Python, `verifier.py:1372-1379`). Section 8.8.2.3 takes this up.

8.8.2.3 Is $\exists s$ sound, and who checks?

For the conclusion, yes. `satisfiedBy_witnessOf` is stated for every admissible `s`, and its conclusion `SatisfiedBy prog input (witnessOf prog s q)` does not mention `s`; so whatever sizes the prover announces, an accepting transcript whose extracted stack satisfies `M3Holds (leanIsaInstance prog s) input q` yields a satisfying witness and, through `constraintSoundness`, an execution. Admissibility is checked by the verifier: the Rust rejects `log_mem  $\notin$  [16, 32]`, `$\tau_j > 32$` , `$\tau_5 < 3$` , an invalid rate, and `$\mu \notin [15, 28]$` (`cpu/mod.rs:157-176`); the Python checks the rate and the `$\mu$` window (`verifier.py:1377-1379`) and the rest through its layout (the bus dossier re-examined the status file’s claim that it omits four caps, finding 9). The hypothesis `hs : s.Admissible prog` of the adaptor is therefore discharged by a verifier check, as the blueprint says (`bp:824-831`).

For the probability, the blueprint does not say how. `verify_knowledgeSound` is sketched with error `niError s Q` (`bp:1226-1227`), a function of `s`; the oracle protocol is “a family indexed by them” (acceptance test 21, *Sizes are parameters*, `bp:1326-1328`); and the per-instance theorems (`piop_rbrKnowledgeSoundness` for one `I`, `FiatShamirSecurity` for one compiled protocol) bound the event for one `s`. A prover against `verify` chooses `s` inside its proof, after its random-oracle queries. Two ways to compose exist and they differ by about 2^{34} :

- a union over the admissible sizes: $\text{Pr}[\text{verify accepts} \wedge \text{bad}] \leq \sum_s \text{Pr}[\text{accepts with } s \wedge \text{bad}_s] \leq \sum_s \epsilon_s$, with $|\{\text{admissible } s\}| \leq 11 \cdot 28^5 \cdot 25 \cdot 4 \approx 2^{34}$ (`log_mem` ≤ 26 and `$\tau_j$` ≤ 28 once `$\mu \leq 28$` is imposed; `$\tau_5 \geq 3$` ; four rates), which the numeric target of acceptance test 23 ($< 2^{-150}$) absorbs for the oracle protocol but the 128-bit WHIR budget (`SECURITY_BITS`, `leaf.rs:143-145`) does not;
- the standard argument, $Q \cdot \max_s \epsilon_s$: each random-oracle query commits to one `s` because the sizes are absorbed into the chain before any challenge (`announce_public`; acceptance test 12, *Absorb before squeeze*). This needs the sizes to be part of the statement the Fiat–Shamir transform hashes, that is, one protocol whose first prover message is `s`, on which the rest of the schedule depends.

ArkLib’s `ProtocolSpec n` has a fixed `n` and fixed message types (`ProtocolSpec/Basic.lean`), so that protocol cannot be written at the pin; the family is the only shape available, and `FiatShamirSecurity` must then be stated for the family (the challenge oracle implemented by one BLAKE2s chain shared across `s`, with `s` in every query), not for one instance.

Finding 76 asks the blueprint to say which, and to make `niError` independent of the prover’s choice (a maximum over admissible `s`, or the union). Either way the *conclusion* of the base theorem (T4) is unaffected; the *number* is. The count of admissible size vectors is a hand estimate from the caps and the stacking window; the exact count depends on the layout’s μ , which Layer 3 defines.

A remark for end-to-end soundness (T7). ArkLib’s game fixes the statement before the prover runs. For end-to-end soundness (`architecture.md:339-369`) the public input is `PublicInput.encode (FinalProof.metadata proof) expectedStatement`, part of which the prover chooses; that needs adaptive soundness in the statement, which the deployed transcript provides (the input seeds the chain) and which no theorem of the blueprint states (finding 87).

8.8.2.4 Hypotheses, and where each is discharged

The chain in the soundness direction:

1. `verify prog input proof = true`;
2. by `verify_iff_compiled`: $\exists s, s.\text{Admissible prog} \wedge$ the Fiat–Shamir-compiled oracle verifier of `leanIsaInstance prog s` accepts;
3. by `verify_knowledgeSound`, in the random-oracle model: the extracted stack `q` satisfies `M3Holds (leanIsaInstance prog s) input q` except with probability `niError`;
4. by `satisfiedBy_witnessOf`: `SatisfiedBy prog input (witnessOf prog s q)`;
5. by `constraintSoundness`: $\exists t, \text{AssignmentRepresents (witnessOf prog s q) } t \wedge \text{Valid Execution prog input } t$.

In the completeness direction:

1. `ValidExecution prog input t`;
2. by `constraintCompleteness`: $\exists w, \text{SatisfiedBy prog input } w \wedge \text{AssignmentRepresents } w t$;
3. by `Sizes.ofWitness` and `stackOf gen`: `M3Holds (leanIsaInstance prog s) input (stackOf gen w hs)`;
4. by `piop_perfectCompleteness` and the compilation: `verify prog input (prove ...) = true`.

Table 8.30 lists every hypothesis on the chain, of four kinds: a condition on *public data* anyone can check; a property the *verifier checks*; a theorem another roadmap or library *owes*; and one that *nothing* discharges.

Table 8.30: The hypotheses of the chain, and who discharges each.

Hypothesis	Where it appears	Kind	Discharged by	Status
s.Admissible prog	satisfiedBy_witnessOf (bp: 815), verify_iff_compiled (bp: 1220)	verifier check	read_public (cpu/mod.rs:157-176), reproduced by verify; supplied by verify_iff_compiled's $\exists$ s	consistent, once Admissible contains the μ and rate windows (finding 8)
Sizes.ofWitness w = some s	stackOf, m3Holds_stackOf, witnessOf_stackOf (bp:811-819)	owed by leanISA	SatisfiedBy.caps.heights, seed_rows, bytecode_rows give power-of-two heights	provable from SatisfiedBy; ill-defined while Sizes holds logInvRate (section 8.8.3.3)
FlockWitnessGen (the generator and its lemma “generated wires satisfy the R1CS when the limbs compress”)	stackOf (bp:811), m3Holds_stackOf	owed by the Flock roadmap (issue #3)	an explicit argument until the Flock roadmap supplies it (bp:857-861)	named; no statement of its type in the blueprint
“the R1CS holds of q_flock $\Rightarrow$ the eighteen limb slots compress”	satisfiedBy_witnessOf (bp: 855-857)	owed by the Flock roadmap	nothing carries it: satisfiedBy_witnessOf's sketched signature has no argument for it, FlockInterface (I) has no such field (bp:1077-1085), and leanIsaInstance's aux needs the Flock roadmap's R1CS to be <i>defined</i>	undischarged until a carrier is named (finding 77)
WellFormedBytecode prog (sentinelSafe, used by soundness; hasFillBlocks, used by completeness)	constraintSoundness, constraintCompleteness (leanisa-blueprint.md:1156-1166)	public data (decidable on the program: SentinelSafe has an instance, Execution. lean:77-78; HasFillBlocks is undefined yet)	no verifier checks it (leanVM's does not, leanvm-target.md:57-62); the compiler emits both (lean_compiler/src/lib.rs:162, filler.rs); guest correctness (T3) owes it	undischarged in the blueprint: baseVerifier_extractsExecution (bp: 1247) has no such hypothesis and baseProver_complete (bp:1249) has HasFillBlocks only (finding 73; the documentation dossier, docs-debt.md B.6)
constraintSoundness, constraintCompleteness themselves	Layer 13 (bp:1253-1256)	owed by leanISA (its Layer 10)	not stated in Lean; Layer 9's four statements are block comments (Statement.lean:442-480), blocked on Clean's N-counted balance (issue #16, Clean #452 and #464)	planned
a resource bound on the trace (every table at most 2^{32} rows)	constraintCompleteness as sketched has none	nothing (leanISA's)	nothing: ValidExecution bounds nothing but $\kappa \leq 32$, and a run of 2^{40} distinct (pc, fp) states needs 2^{40} rows of one table, which Caps.heights forbids	undischarged (finding 88, out of scope; it reaches the base theorem through baseProver_complete)
the stacking window $\mu \in [15, 28]$ and the rate window	verify's acceptance in baseProver_complete	verifier check (cpu/mod.rs:174-176, pcs.rs:49-51)	nothing on the completeness side: SatisfiedBy.caps has neither (Statement.lean:79-81); a valid execution at $\kappa = 32$ has $4 \cdot 2^{32} > 2^{28}$ committed cells	undischarged : baseProver_complete is false without it (finding 74)
a witness <i>constructed</i> from t (“prove prog input (witness of t)”, bp:1250)	baseProver_complete	owed by witness generation (T2), out of scope (bp:147-148)	constraintCompleteness gives $\exists$ w only	undischarged as written (finding 74)
Phases.Complete, Phases.Security for leanIsaInstance prog s	the two master theorems (Spine/Compose.lean:162-177)	owed by the phases (holes P1 to P8; the public-input phase, P5, built)	the phases	assumed interface until every hole is filled (bp:1419-1426)

Hypothesis	Where it appears	Kind	Discharged by	Status
FiatShamirSecurity, BcsSecurity, McaJohnson	verify_knowledgeSound (bp:1226)	owed by ArkLib (ledger rows <i>Fiat-Shamir and BCS, mutual correlated agreement</i>)	explicit arguments	assumed interfaces
the Flock phase's Security (FlockInterface)	verify_knowledgeSound's flock argument	owed by the Flock roadmap	explicit argument	assumed interface; circular with the instance (finding 77)
BLAKE2s as a random oracle	the step from verify_knowledgeSound (random-oracle model) to the concrete verify	nothing (a heuristic)	nothing; the blueprint's baseVerifier_ extractsExecution names verify and a probability in one sentence (bp:1247-1248)	undischarged : must be stated as the hypothesis it is (finding 73)
the round-by-round-to-plain implication	the plain corollary (bp:261)	owed by ArkLib (admitted at the pin)	"stated once the implication lands upstream"	consistent; the spine dossier (code-spine.md, C.7) notes it forgets the named extractor
the top limb of the public words is zero	read_public (cpu/mod.rs:141-143)	verifier check in the Rust; by type in Lean (Memory.lean:107-110)	the parser of PublicInput	consistent; undocumented as a parsing obligation (finding 85)
the bytecode is decodable and of power-of-two length at most 2^{32}	read_public (cpu/mod.rs:157-159); the Python accepts any array	verifier check in the Rust; by type in Lean (Program, Instruction.lean:87-93)	the loader of Program	consistent; coverage gap recorded (leanvm-faithfulness-review.md:72)
$\kappa < 64$ for MemImage.read_gpow	inside satisfiedBy_witness0f's word0_eq	verifier check	from s.Admissible prog ($\log \text{Mem} \leq 32$)	consistent
the count columns are exactly the pulls' count coordinates	counts_nonzero both ways	owed by Layers 2 and 3	a transcription from the Rust (the blueprint's category B) of count_columns() per table (layout.rs:412-414), which Component.toM3 cannot derive from (sep, dir) alone	unspecified (finding 81)

Violations of the boundary. Of the rule “nothing is assumed on one side of the boundary which the other side does not prove”, the rows left undischarged that are not heuristics are four: `WellFormedBytecode` (the proof system assumes nothing of it and the arithmetization requires it), the resource bounds and the constructed witness of completeness (the proof system’s completeness theorem requires what the arithmetization’s does not give), and the Flock consequence lemma (the adaptor consumes a theorem no interface carries). The random-oracle heuristic is a fifth, inherent, and must simply be written down.

8.8.2.5 The top limb of the public input, along the whole chain

The public-input dossier (`code-pubinput.md`, section D) traced the limb through the phase, the spine and the deployed verifiers; this walk adds the two ends of the chain and the anchors. Its facts used here: the oracle protocol’s theorems hold for every instance, so a `leanISA` instance with two lines is as good an `M3Instance` as one with three (its D.1); the deployed verifiers pool the third claim at 0 with no scalar (`cpu/mod.rs:746, 756, bind_pi_claim cpu/mod.rs:674-682; verifier.py:1399-1401; its D.3`).

1. **The type.** `PublicInput` is four lanes of `K`, and the two words are built with a literal zero top limb:

```
structure PublicInput where
  lanes : Fin 4 → K
  deriving DecidableEq
def PublicInput.word0 (p : PublicInput) : E := E.ofLimbs (p.lanes 0) (p.lanes 1) 0
def PublicInput.word1 (p : PublicInput) : E := E.ofLimbs (p.lanes 2) (p.lanes 3) 0
```

`LeanerVM/Semantics/Memory.lean:101-110` at `b435631`, docstrings omitted (probe `Shapes` prints them, output lines 69–78) A public input with a nonzero top limb cannot be written; the probe proves `p.word0.limb 2 = 0` and `p.word1.limb 2 = 0` for every `p` by `simp` (`Shapes.lean:57-61`, exit 0). The specification agrees, “two 192-bit words (with top limb 0)” (`doc/leanvm/body/08-end-to-end-protocol.tex:29`), the Rust rejects a third limb (`cpu/mod.rs:141-143`) and the Python’s `Digest` has none (`verifier.py:223`).

2. **What `SatisfiedBy` says of cells 0 and 1.** `word0_eq : (imageOf w.data).2.read (gpow 0) = some input.word0` and `word1_eq (Statement.lean:338-340)`: the *whole* E-word of the image at index 0 is `input.word0`, top limb included; and `public_input_eq (Statement.lean:315)`, which no component reads.
3. **What `ValidExecution` says.** `HasPublicBoundary` input `t` has the same two equations on `t.image` (`Execution.lean:108-110`), whole words again.
4. **What the instance says.** Three lines (`bp:834-836`): `mem_0` with cells (`word0.limb 0, word1.limb 0`), `mem_1` with (`word0.limb 1, word1.limb 1`), `mem_2` with (`0, 0`). `PublicLinesHold` on them is exactly the six limb equalities, from which `E.ofLimbs mem_0[i] mem_1[i] mem_2[i] = input.word_i` follows (probe `Shapes.lean:71-76`).
5. **If the instance omitted the third line.** `M3Holds` would say nothing of `mem_2[0]`, `mem_2[1]`; the master theorems hold unchanged; the theorem left without a proof is `satisfiedBy_witnessOf`, for every possible `witnessOf`, not only the natural one:
 - the natural `witnessOf` writes the image’s word 0 as `E.ofLimbs mem_0[0] mem_1[0] mem_2[0]`; with `mem_2[0] ≠ 0` it is not `input.word0`, whatever input (probe `Shapes.lean:63-69`: `E.ofLimbs a b c ≠ p.word0` when `c ≠ 0`, by comparing limb 2);
 - a `witnessOf` that zeroed the limb when building the data and the block would break `mem_balanced` for any program that reads cell 0 (the reading row’s pulled tuple carries `mem_2[0]` from `q`, the built seed carries 0);
 - and no `witnessOf` at all can serve when no satisfying witness exists: take the program whose first instruction is `SET_CONSTANT [g^0, y^2]` (`Instr.setConstant 1 (E.ofLimbs 0 0 1)`; at `fp = 1` the operand `g^0` names cell 0) followed by a jump to the sentinel and the fill blocks. Its constraints are satisfiable only with cell 0 = `y^2`, whose low limbs are 0, 0: a stack with `mem_0[0] = mem_1[0] = 0, mem_2[0] = 1` satisfies the constraints, the balances, the counts and the two remaining lines at `input = (![0, 0, l2, l3])`, so it is in the two-line `M3Holds`; but `SatisfiedBy prog input w` demands `word0_eq` with `input.word0 = E.ofLimbs 0 0 0 ≠ y^2`, and `ValidExecution` demands the same of `t.image`, so neither has an inhabitant. (An inference from the definitions: `execute` of `SET_CONSTANT` requires `[o] = k`, `Step.lean`; the two-line stack of that program was not built as a probe; the load-bearing fact, `E.ofLimbs a b c ≠ p.word0` for `c ≠ 0`, is proved by the probe `Shapes`.)

6. **If SatisfiedBy were weakened instead** (its two word conjuncts restricted to limbs 0 and 1), `satisfiedBy_witnessOf` would close with two lines, and the theorem left without a proof would be **constraintSoundness**: it must produce $\exists t, \text{AssignmentRepresents } w \ t \wedge \text{ValidExecution prog input } t, \text{AssignmentRepresents fixes } t.\text{image to imageOf } w.\text{data}$ (`Statement.lean:355-362`, `assignmentRepresents_image`), and `HasPublicBoundary` then demands the whole word (`Execution.lean:110`). For the program above no t exists, so the theorem is false.
7. **If HasPublicBoundary were weakened too**, nothing on the chain would object, and the anchor would be the specification alone; the top limb would then be constrained only where a guest reads it (guest correctness, T3).
8. **The anchor** is therefore the pair `PublicInput.word0, PublicInput.word1` with their literal 0 (`Memory.lean:107-110`), consumed verbatim by `HasPublicBoundary` and by `SatisfiedBy`. The requirement of the owner’s note (“a missing, weak or wrong verifier check must leave knowledge soundness or completeness unprovable”) is met one level down from the oracle protocol, in the adaptor’s theorem, and the acceptance test that claims it (test 10, *Top limb of the public input*, `bp:1293-1295`) should name that theorem, as the public-input dossier also asks (`code-pubinput.md`, G.3). This agrees with that dossier’s section D.

8.8.2.6 The obligation map of docs/architecture.md against the blueprint

Table 8.31 takes the obligation map for the base theorem (T4) (`architecture.md:261-275`, the “Proof system” list `architecture.md:446-462`, the ladder `architecture.md:63-70`) item by item. Line numbers in its first column are of `architecture.md`; the quotations there use the architecture’s own codes.

Table 8.31: The obligation map of docs/architecture.md for the base theorem, against the blueprint and the code at b435631.

Obligation	In the blueprint	In the code	Verdict
the bridge “BaseVerifier.Accepts ... $\rightarrow$ except with probability baseError, $\exists$ assignment, Constraints.SatisfiedBy ...” (:264-269)	verify_knowledgeSound with satisfiedBy_witnessOf (bp:1226-1227, bp:815-816)	neither; M3Holds, M3Rel, Refinement built	consistent in content; the version parameter of the architecture is the pin recorded in prose, not a Lean argument
“then compose it with T1-S” (:270)	Layer 13 (bp:1253-1254)	constraintSoundness not stated	consistent, except that constraint soundness’s hypothesis WellFormedBytecode (architecture.md:224-228) is dropped in the composed statement (finding 73)
expose “the component soundness/knowledge-soundness bounds, Fiat–Shamir or random-oracle model, commitment and hash assumptions, transcript/serialization agreement, and executable-verifier refinement” (:270-273)	niError, FiatShamirSecurity, BcsSecurity, McaJohnson, verify_iff_compiled (bp:1218-1228)	pioError, the two master theorems (Spine/Compose. lean:149-185)	the random-oracle model and the hash assumption are named as interfaces for the <i>compiled</i> protocol but the concrete verify uses BLAKE2s: the heuristic “BLAKE2s is the random oracle” is exposed nowhere (finding 73); the family composition over <i>s</i> is exposed nowhere (finding 76)
“Its completeness dual composes T2 with the honest prover and records any failure/resource conditions” (:273-275)	baseProver_complete composes constraint completeness (T1-C), “witness generation (T2) ... out of scope” (bp:147-148, bp:1255-1256); no resource condition	nothing built	divergent: witness generation is replaced by an existence theorem, which does not yield the witness <i>prove</i> needs, and no resource condition is recorded although two are load-bearing (finding 74; the documentation dossier, docs-debt.md B.6 item 3)
“Relate the executable verifier to the protocol specification, including serialization, transcript order, domain separation, challenge derivation, statement binding, and rejection behavior” (:453-455)	Layer 12: verify_iff_compiled, the stream order, the tags, FsState.seed, total verify	nothing built	consistent; “statement binding” needs the injectivity of <i>prog</i> $\mapsto$ bytecodeColumn <i>prog</i> (section 8.8.2.2, item 1), unstated
“Prove completeness of the abstract prover and test or verify completeness of the executable prover” (:456-457)	pio_perfectCompleteness; prove “a specification that runs” (bp:158); the Rust prover by fixture only	pio_perfectCompleteness (conditional)	consistent; the executable (Rust) prover’s completeness is evidence, not a theorem, as architecture.md:530-533 allows
“T4 — base proof extraction/completeness: proof-system components, T1, and T2” (:66)	inputs: the components and constraint soundness (T1); not witness generation (T2)		as above
leanvm-target.md:123-124: “T4: formal component notions, Fiat–Shamir assumptions, composed error bound, and verifier refinement”; “each [verifier implementation] accepts exactly the specified protocol and statement encoding”	the first is Layers 10–13; the second is the fixture plus verify_iff_compiled		the Rust and Python verifiers do <i>not</i> accept exactly the specified protocol on the public-input check (the status file’s finding on it, F18; the public-input dossier, code-pubinput.md G.2), so “exactly” is owed a lemma the blueprint assigns to Layer 12

The three divergences. The obligation map is not met by the blueprint on three points:

1. the completeness dual composes constraint completeness (T1-C), not witness generation (T2), as `architecture.md:273-275` asks (“composes T2 with the honest prover”);
2. it records no resource conditions, although two are load-bearing (finding 74);
3. the hash assumption, BLAKE2s as a random oracle, is nowhere an explicit hypothesis (finding 73).

Obligations of the map with no counterpart in the blueprint are these three and the statement-binding lemma. Blueprint layers that serve no obligation of the base theorem: none (every layer feeds `verify` or a theorem about it); three named declarations serve nothing on the chain and are tests of non-vacuity or candidates for upstream: `witnessOf_stackOf` (section 8.8.3.2), `Extractor.Straightline.map` (no possible consumer in this repository, section 8.8.3.6), `piop_rbrKnowledgeSoundness_exists`.

The extracted witness. One more point of the architecture the blueprint’s base theorem loses: “the extracted leanVM witness should contain ordered `pc/fp` steps and the full memory image” (`architecture.md:332-335`, for recursion, T6). The chain has it, `constraintSoundness` giving `AssignmentRepresents` (`witnessOf prog s q`) `t` for the extracted `q`; `baseVerifier_extractExecution` keeps only $\exists t$, `ValidExecution prog input t` (`bp:1247-1248`), which is a language membership and needs no extractor at all. Stating the pointwise theorem in the form “for the extractor’s `q`, when it satisfies `M3Holds`, $\exists t$, `AssignmentRepresents` (`witnessOf prog s q`) `t` $\wedge$ `ValidExecution prog input t`” costs nothing and is what recursion consumes (finding 86).

8.8.2.7 What was checked and found consistent

- `M3Holds`’s five clauses against `SatisfiedBy`’s thirteen conjuncts: every conjunct has a source (table 8.27); no conjunct of `SatisfiedBy` is delivered by nothing once the caps hypothesis, the boundary lemma, the count derivation and the Flock consequence lemma are in place.
- `w.Constraints` is asserts and lookups only (`Clean 93c9d1ef`, `Clean/Circuit/Operations.lean:687-688`, probe `Shapes` output lines 154–159); `leanISA` emits no lookup and only `JUMP` asserts; so `toM3_constraints_iff` is provable in principle.
- The degree bound $d = 2$: every flush coordinate of the six tables read (`LeanerVM/Arithmetization/Tables/*.lean`, `main`) is of degree at most two (`JUMP`’s successor $b \cdot v_{pc} + b \cdot (g \cdot pc) + g \cdot pc$, `Jump.lean:199-201`; `DEREF`’s store $fbar \cdot v3 + f_{pc} \cdot (g^2 \cdot pc) + f_{fp} \cdot fp$, `Deref.lean:203-206`; `MUL_NATIVE`’s twelve products, `MulNative.lean:168-173`; the pushes’ $g \cdot count$).
- The state boundary as constants only (`Coord.committed` needs $\tau = \kappa = 0$, impossible for a real column), as `layout.rs:354-358`; the memory and bytecode blocks’ committed coordinates are columns of tables whose τ equals the block’s κ by construction (section 8.8.3.4).
- `Caps` is derivable from `Admissible` plus the construction of `witnessOf` (the row of `caps` in table 8.27), and `Sizes.ofWitness w = some s` from `SatisfiedBy` (section 8.8.1.7).
- The chain names one `prog` and one `input` throughout (section 8.8.2.2); no theorem lets the prover choose either.
- The top limb is anchored (section 8.8.2.5), agreeing with the public-input dossier’s section D.
- `Refinement` accepts a `Type 1` target witness (probe `Transport`); the pointwise composition of the base theorem typechecks (its examples 3–4); `Extractor.Straightline.map` cannot serve (probe `UniverseFail`).
- The axioms of every arithmetization declaration the chain will use (`assumptions_of_blake2s`, `RowsValid`, `memRowOf_bindings`, `bytecodeRowOf_bindings`, `bytecodeRowOf_decodes`, `verifier_pull_eval`, `verifier_push_eval`, `rowAt_toElements`, `decode_eq_some_iff`, `assignmentRepresents_image`, the six tables, the two blocks, the verifier) and of the two master theorems, `bytecodeColumn_eval`, `idxColumn_eval`: the kernel’s three (probe `Shapes`, output lines 168–187); `map_option_valid`: two.

8.8.3 The adaptor: are its sketched statements well formed and provable?

Short forms in this section. `bp`: is the blueprint `docs/roadmap/protocol-blueprint.md` at `leanerVM b435631`; every sketch below is copied from it. Lean files are named as in section 8.8.1, at `b435631`. `Clean` is cited at its old pin `93c9d1ef`, `ArkLib` at `dca90385`

(Security/Basic.lean and Security/RoundByRound.lean under ArkLib/OracleReduction/), CompPoly at 3468b38c (MvPolyEquiv/Core.lean and MvPolyEquiv/Eval.lean under CompPoly/Multivariate/); Rust files are under crates/lean_vm/src/ at leanVM's pin a386121f, and whir_config.rs under crates/pcs/src/. The base theorem of the proof system (T4) and end-to-end soundness (T7) are named as in section 8.8.2.

The Layer 3 sketch (bp:794-820) and the Layer 2 sketch it needs (bp:761-778) are read here as Lean to be written: for each sketched statement, whether it can be written, whether it can be proved, and the restatement the review proposes.

8.8.3.1 witnessOf and satisfiedBy_witnessOf

```
def witnessOf (prog) (s) (q : Column (leanIsaInstance prog s).μ) :
  EnsembleWitness (leanIsaEnsemble prog) -- total and computable
theorem satisfiedBy_witnessOf (hs : s.Admissible prog) -- the caps: a hypothesis
  (h : M3Holds (leanIsaInstance prog s) input q) : SatisfiedBy prog input (witnessOf prog s q)
```

docs/roadmap/protocol-blueprint.md:813-816 at b435631

The definition. “Total and computable” is achievable. `EnsembleWitness` is in `Type 1` (probe `Shapes`, output lines 1–2); a computable definition may inhabit a type in `Type 1` (the ensemble itself is a `def` that compiles), and the fields are constructible: eight `Tables` with `component := (leanIsaEnsemble prog).tables[j]`, rows built by one pass over the columns read through the layout, `data := fun name n ↦ if name = "mem" ∧ n = 3 then ... else #[]`, the public input off cells 0 and 1, and the three proof fields by `rfl/simp`. Total in `s`: for an inadmissible `s` the tables are simply of the announced heights. Two things the sketch omits: `witnessOf` has no input argument (table 8.29, the adaptor’s row), which is fine only if the lanes are read off `q` (then `public_input_eq` needs the first two lines and $0 < \log \text{Mem}$); and the instance must exist first (sections 8.8.3.4 and 8.8.3.5, finding 77).

The theorem. It typechecks as a statement provided `(leanIsaInstance prog s).Stmt` reduces to `PublicInput`. With a `def` instance the elaborator unfolds it (probe `DefInstance`, the example at line 34 elaborates), but instance search does not, so `Decidable (M3Holds (leanIsaInstance prog s) input q)`, which every `#guard` of the Layer 3 tests needs, is found only for an `abbrev` instance (probe `DefInstance`: the two guards on `instA` pass, the two on `inst` fail with “Type mismatch ... expected to have type `Bool`”; the spine’s toy is an `abbrev` for the same reason, `Spine/Toy.lean:94-95`). Provable as sketched: yes, with the additions of section 8.8.1.5 (the boundary lemma, the count derivation, the Flock consequence lemma).

Proposed change: the adaptor’s map and the instance’s reducibility (bp:803, bp:813)

As it stands. `def witnessOf (prog) (s) (q : Column (leanIsaInstance prog s).μ)` and `def leanIsaInstance (prog : Program) (s : Sizes) : M3Instance`.

As proposed. Either add `(input : PublicInput)` to `witnessOf` (the `Refinement.map` has the statement) or say “the lanes are read off cells 0 and 1 of `mem_0`, `mem_1`”. And: “abbrev `leanIsaInstance ...`, reducible, so that `Decidable (M3Holds (leanIsaInstance prog s) input q)` and the instances on `I`. `Stmt` are found by instance search, as the toy is”.

Reason. Findings 81 and 82.

8.8.3.2 stackOf, m3Holds_stackOf, witnessOf_stackOf

```
def stackOf (gen : FlockWitnessGen) (w : EnsembleWitness (leanIsaEnsemble prog))
  (hs : Sizes.ofWitness w = some s) : Column (leanIsaInstance prog s).μ -- gen: #3's
theorem m3Holds_stackOf (h : SatisfiedBy prog input w) (hs : Sizes.ofWitness w = some s) :
  M3Holds (leanIsaInstance prog s) input (stackOf gen w hs)
theorem witnessOf_stackOf (hs) : witnessOf prog s (stackOf gen w hs) = w -- on the committed fields
```

docs/roadmap/protocol-blueprint.md:811-812, 817-819 at b435631

`stackOf` is well formed (a `def` taking a proof to fix the index of its result is ordinary Lean). `m3Holds_stackOf` is provable with the additions of section 8.8.1.7.

`witnessOf_stackOf` is not a Lean statement. As an equality of `EnsembleWitnesses` it is false for almost every `w`: `w.data : String → (n : ℕ) → Array (Vector K n)` is a function and `witnessOf` rebuilds it with the "mem" table only; `Table.width` is a free field (`FlatComponent.lean:151-156`) and `witnessOf` rebuilds rows of exactly the component's width; the memory block's `idx` cells and the bytecode block's entry cells are rebuilt from `gpow` and `prog`, so equality needs `index_columns` and `bytecode_rows` of `w`; and the public input is rebuilt from cells 0 and 1, so it needs `public_input_eq` and the word conjuncts. "On the committed fields" has to be spelled out. Neither composition of the base theorem uses `witnessOf_stackOf` (soundness uses `satisfiedBy_witnessOf`, completeness `m3Holds_stackOf`); its role is acceptance tests 19 and 24 (*The extractor reads the stack, The extractor computes*, bp:1320-1322, 1333-1340), non-vacuity of `witnessOf`. The test `#guard witnessOf (stackOf w) = w` (bp:1339) cannot be written, `EnsembleWitness` having no `DecidableEq`.

Proposed change: the round trip of the adaptor (bp:819, bp:1339)

As it stands. theorem `witnessOf_stackOf (hs) : witnessOf prog s (stackOf gen w hs) = w` -- on the committed fields and the test `#guard witnessOf (stackOf w) = w`.

As proposed. A statement that says what the sketch means and is decidable on a small witness:

```
/-- The cells a component reads: the first `width` cells of a raw row, a missing cell `0`. -/
def rowCells (c : Component K) (row : Array K) : List K :=
  (List.range c.width).map fun j ↦ row[j]!.getD 0
theorem witnessOf_stackOf (h : SatisfiedBy prog input w) (hs : Sizes.ofWitness w = some s) :
  let w' := witnessOf prog s (stackOf gen w hs)
  (∀ j : Fin 8, (tableAt w' j).table.map (rowCells (tableAt w j).component) =
    (tableAt w j).table.map (rowCells (tableAt w j).component)) ∧
    memRows w'.data = memRows w.data ∧ w'.publicInput = w.publicInput
```

and, for bp:1339, "#guard of those three equalities on the one-row witness". Say in bp:822-824 that neither composition of the base theorem uses it.

Reason. `EnsembleWitness` has a function field, a free width and no `DecidableEq` (finding 80).

8.8.3.3 The rate in Sizes

```
structure Sizes where
  logMem : ℕ
  τ : Fin 6 → ℕ
  logInvRate : ℕ
  -- WHIR rate exponent, announced with the sizes
def Sizes.ofWitness (w : EnsembleWitness leanIsaEnsemble) : Option Sizes -- `none` unless every height is 2^τ
def Sizes.Admissible (prog : Program) (s : Sizes) : Prop -- the caps; decidable
theorem admissible_iff_caps : s.Admissible prog ↔ (Caps w ∧ Sizes.ofWitness w = some s)
```

docs/roadmap/protocol-blueprint.md:795-801 at b435631

The Rust announces `log_mem`, six `τ_j` and `log_inv_rate` (`announce_public`, `cpu/mod.rs:118-124`; `read_public`, `cpu/mod.rs:144-149`); the Python reads the same 2 + 6 scalars (`python-verifier/verifier.py:1372-1376`); the specification's Setup names `κ_mem` and the six `τ_j` only (`doc/leanvm/body/08-end-to-end-protocol.tex:55`), a disagreement between the specification and both implementations that the blueprint resolves the Rust's way (its source category B, bp:199). So the rate *is* announced with the sizes. It does not belong in the instance of the oracle protocol: `leanIsaInstance prog s` reads `s.logMem` and `s.τ` (heights, `μ`, layout, lines) and nothing of the rate, which parameterizes WHIR alone (Layer 11); `Sizes.ofWitness (w) : Option Sizes` (bp:799) cannot produce a rate from a witness; and `Sizes.Admissible` would mix a check of the oracle protocol's family index with a check of the commitment. The bus dossier reaches the same point from the caps (finding 8). Besides, `admissible_iff_caps` has a free `w` and, quantified, is false in both directions (`Caps` lacks the `μ` and rate windows, `Statement.lean:79-81`; another `w` has other sizes); `Sizes.ofWitness` lacks `prog`; and `leanIsaInstance_fits` uses `layout.total` (bp:808), a field `Layout` has not (the Layer 1 dossier, `code-layer1.md G.2`).

Proposed change: the sizes, admissibility and the rate (bp:795-808)

As it stands. the sketch above, with `leanIsaInstance_fits : (leanIsaInstance prog s).layout.total ≤ 2 ^ (leanIsaInstance prog s).μ` (bp:808).

As proposed.


```

structure Sizes where (logMem : ℕ) (τ : Fin 6 → ℕ) -- the family index
def Sizes.ofWitness (w : EnsembleWitness (leanIsaEnsemble prog)) : Option Sizes
def leanIsaBlocks (prog) (s) : Blocks ; def leanIsaμ (prog) (s) : ℕ -- witness.rs:67-101
theorem leanIsaBlocks_fits : (leanIsaBlocks prog s).total ≤ 2 ^ leanIsaμ prog s
def Sizes.Admissible (prog) (s) : Prop -- 16 ≤ logMem ≤ 32, τ j ≤ 32, 3 ≤ τ 5, leanIsaμ prog s ≤ 28
def validRate (p : ℕ) : Prop := 1 ≤ p ∧ p ≤ 4 -- whir_config.rs:48-55, Layer 12
theorem caps_of_admissible (hs : s.Admissible prog) (q) : Caps (witnessOf prog s q)
theorem sizes_of_satisfiedBy (h : SatisfiedBy prog input w) : ∃ s, Sizes.ofWitness w = some s

```

The announcement of Layer 12 is $\text{Sizes} \times \text{logInvRate}$.

Reason. The instance depends on the heights alone; the rate is Layer 12's; the two lemmas are what the two directions use (finding 75).

8.8.3.4 The instance's tables, columns and blocks against M3Instance as built

```

def leanIsaInstance (prog : Program) (s : Sizes) : M3Instance -- `Ensemble.toM3` of the eight tables with
-- leanISA's separators and directions; the
-- layout of `witness.rs:67-101`,
-- ↪ `leaf.rs:53-156`
theorem leanIsaInstance_degree (j) (C ∈ (leanIsaInstance prog s).constraints j) : C.totalDegree ≤ 2
theorem leanIsaInstance_flush_degree ... ≤ 2
theorem leanIsaInstance_fits : (leanIsaInstance prog s).layout.total ≤ 2 ^ (leanIsaInstance prog s).μ

```

docs/roadmap/protocol-blueprint.md:803-808 at b435631

and, in prose (bp:836-839): “The stack columns that belong to no opcode table (`mem_0`, `mem_1`, `mem_2`, `cntfin_mem`, `cntfin_bc`, `q_flock`; `cpu/layout.rs:13-26`) are tables of the instance with no constraints and no flushes, so that `Shape.ColumnId` and `Coord.committed` reach them”. Whether the instance the blueprint means is `Ensemble.toM3` of the eight tables is finding 71; read as section 8.8.1.4 reads it, it is expressible, with one consequence:

- the six shared columns as tables of the instance: yes, as at least three tables (section 8.8.1.4), each with `constraints := []`, `flushes := []`, `counts := []`; `Shape.τ` is per table, so the memory columns' height 2^{logMem} sits beside the six tables' heights;
- the finalize counts outside the count tree: `counts jMem = []`, `counts jBc = []`, as `layout.rs:412-414` (the count blocks are the tables' `count_columns()` only);
- the boundary blocks: `Coord.committed (jMem, i) rfl at κ = logMem`, `Coord.committed (jBc, 0) rfl at κ = prog.logSize`, the known and const coordinates of section 8.8.1.4; the $h : S.τ \text{ c.1} = κ$ obligation is met by construction;
- the eighteen limb columns as strided slots of `q_flock`: the `Layout` law admits it (`Spine/Instance.lean:73-81`, a reading law), but no reader exists (the Layer 1 dossier, `code-layer1.md G.1`) and the slot map is placed inside `FlockInterface (I)` (finding 36); finding 77;
- the three lines need `PublicLine.pos : 0 < τ`, that is $0 < s.\text{logMem}$; for a total `leanIsaInstance` the lines must be empty (or the instance junk) when `logMem = 0`, which is harmless because `verify` rejects such `s`, but must be written;
- **the consequence:** the generic table sumcheck (Layer 7) takes its rounds from $τ_{\text{max}} = \max_j τ_j$ over the instance's tables and sends “one value per column of every table” (bp:987-989, 08-end-to-end-protocol.tex:76-77); with the shared columns as tables, a phase over the abstract `I` would run $\max(τ_j, \text{logMem}, \text{prog.logSize}, τ_5 + 8)$ rounds and send scalars for `mem_0`, ..., `q_flock`, a transcript leanVM does not have (`constraints.rs:250`, the six tables; finding 20). The instance needs a criterion the phase can read.

Proposed change: column groups in the spine's conventions (bp:316) and in Layer 7 (bp:987)

As it stands. “tables of the instance with no constraints and no flushes” (bp:836-839), with nothing saying how a phase over the abstract instance treats them.

As proposed. In the spine's conventions (bp:316, or a new row *Column groups*): “A table with no constraint, no flush and no count column is a column group: it takes no part in the table sumcheck (no round, no term, no column value) and its columns receive claims from the boundary blocks, the public lines and the Flock phase only. `M3Instance.active j := (I.constraints j ≠ []) ∨ (I.flushes j`

$\neq []$) v ($I.counts\ j \neq []$) is the criterion, decidable, and Layer 7 ranges over it.” In Layer 7 (bp: 987): τ_{max} and $\Sigma_j\ width_j$ over the active tables.

Reason. A phase over the abstract I that ranges over all of $I.ntab$ has a different transcript from leanVM’s on the leanISA instance, which `verify_iff_compiled` would then fail (finding 79).

8.8.3.5 Mathlib polynomials in Layer 2, CompPoly polynomials in the instance

```
def Expression.toMvPolynomial : Expression F → MvPolynomial ℕ F -- var i ↦ X i
theorem eval_toMvPolynomial (env : Environment F) (e) :
  MvPolynomial.eval env.get e.toMvPolynomial = e.eval env
def Expression.degreeBound : Expression F → ℕ -- syntactic; add max, mul sum
theorem totalDegree_le_degreeBound (e) : e.toMvPolynomial.totalDegree ≤ e.degreeBound

structure M3Table (F) where
  width : ℕ
  constraints : List (MvPolynomial (Fin width) F)
  flushes : List (Direction × Vector (MvPolynomial (Fin width) F) 16) -- separator first
  count : List (Fin width) -- the count columns
def Component.toM3 (c : Component F) (sep : RawChannel F → F) (dir : RawChannel F → Direction) :
  M3Table F
theorem toM3_constraints_iff (t : Table F) (row ∈ t.table) :
  (∀ C ∈ (t.component.toM3 ...).constraints, eval row C = 0) ↔ t.component.operations.ConstraintsHold
  ↪ (t.environment row)
theorem toM3_flushes_eq (t : Table F) : (multiset of evaluated flush tuples) = (multiset of `t.interactions`
  ↪ mapped to 16-tuples)
```

[docs/roadmap/protocol-blueprint.md:762-777](#) at [b435631](#)

Layer 2 produces `Expression.toMvPolynomial : Expression F → MvPolynomial ℕ F` and an `M3Table F` whose constraints are `MvPolynomial (Fin width) F`; `M3Instance` holds `CMvPolynomial (width j) K` (`Spine/Instance.lean:123-125`). No conversion is named. `CompPoly`’s `toCMvPolynomial : MvPolynomial (Fin n) R → CMvPolynomial n R` is noncomputable `def` at the pin [3468b38c](#) (`MvPolyEquiv/Core.lean:41`; the same at the new pin [572f9973](#), read with `git show`), so an instance built through it is noncomputable: `M3Holds (leanISAInstance prog s)` could not be decided by evaluation (the Layer 3 tests, bp:866-867), the phases’ verifiers, which evaluate `I.constraints` at the claimed column values, would not compile, and `verify` (bp:1230-1231, “total and computable”) could not be defined from them. The degree bound travels the other way without trouble: `totalDegree_equiv : p.totalDegree = (fromCMvPolynomial p).totalDegree` (`MvPolyEquiv/Eval.lean:60-61`, `rfl`) and `eval_equiv` (`MvPolyEquiv/Eval.lean:53-57`) let Mathlib prove the bound of a computable polynomial, as the toy does (`Spine/Toy.lean:68-74`).

What Layer 2 must produce is a *direct* computable translation `Expression K → CMvPolynomial n K` (structural recursion on `var`, `const`, `add`, `mul`, `Expression.lean:12-16`, with `CMvPolynomial.X`, `C`, `+`, `*`), and its bridge to Mathlib for the degree lemma. Probe `PolyBridge` writes it in twelve lines, evaluates the JUMP residual `b + v_cond · w` on a row and checks it against Clean’s `Expression.eval`, checks `totalDegree = 2`, and checks that a variable past the width reads 0 on both sides as `Environment.fromArray` reads a missing cell (`Expression.lean:71-73`); it passed at the old pins, and as `PolyBridgeNew`, with `K.ofBits` for the row numerals, at the new pins (appendix B.3.7). The `Direction` of leanISA’s `channelDir` (`Channels.lean:262-272`) and the spine’s `Side` (`Spine/Instance.lean:51-54`) are two enumerations of the same two values; the translation is one match.

Proposed change: the polynomial bridge of Layer 2 (bp:762-767)

As it stands. `def Expression.toMvPolynomial : Expression F → MvPolynomial ℕ F`, with `M3Table`’s fields in `MvPolynomial (Fin width) F` and `Direction`.

As proposed. “`def Expression.toCMvPolynomial (n) : Expression K → CMvPolynomial n K` (`var i < n ↦ X i`, else 0, as `Environment.fromArray` reads a missing cell); `theorem eval_toCMvPolynomial (row) (e) : (e.toCMvPolynomial n).eval (fun i ↦ row[i]?.getD 0) = e.eval (Environment.fromArray row data)`; `theorem fromCMvPolynomial_toCMvPolynomial (e) : fromCMvPolynomial (e.toCMvPolynomial n) = rename ... e.toMvPolynomial`, through which `degreeBound` bounds `totalDegree` (`totalDegree_equiv`)”. `M3Table`’s fields become `CMvPolynomial`; `Direction` becomes `Side` (or the spine adopts leanISA’s `Direction`). Clean #466 (`Expression.toMvPolynomial`) stays the proof-side bridge.

Reason. A deviation forced by an upstream library, with the direct translation as its workaround, retired if CompPoly makes `toCMvPolynomial` computable (finding 78).

8.8.3.6 The composition of the base theorem, arrow by arrow

```
T4 = verify_knowledgeSound ◦ Refinement.map_option_valid satisfiedBy_witnessOf ◦ constraintSoundness
```

docs/roadmap/protocol-blueprint.md:407 at b435631

```
theorem baseVerifier_extractsExecution (fs bcs mca flock) (h : verify prog input proof = true) :
  except with probability niError, ∃ t, ValidExecution prog input t
theorem baseProver_complete (hfill : HasFillBlocks prog) (h : ValidExecution prog input t) :
  verify prog input (prove prog input (witness of t)) = true
```

docs/roadmap/protocol-blueprint.md:1247-1250 at b435631

“The first composes `verify_knowledgeSound`, the adaptor’s pointwise transport `Refinement.map_option_valid` with `satisfiedBy_witnessOf`, and `constraintSoundness`” (bp:1253-1254); the tracker’s section on the base theorem (K4) says `knowledgeSound_of_refinement`, a name that exists nowhere. Arrow by arrow:

- `verify_knowledgeSound (fs bcs mca flock)`: in ArkLib’s shape it bounds, for a fixed `stmtIn = input` and every prover, the probability of the event “the compiled verifier accepts and every `q` in the extracted slot has $(input, q) \notin M3Rel \text{ (leanIsaInstance prog s)}$ ” by `niError` (Security/Basic. lean:299-323, `knowledgeSoundnessWith`; the same event in the round-by-round form, `RoundByRound. lean:553`). The slot is an `Option (Column μ)`, `Column μ : Type`.
- `satisfiedBy_witnessOf`, made a `Refinement (M3Rel I) (SatRel prog)` with `SatRel prog := {p | SatisfiedBy prog p.1.1 p.2}` on witnesses in `Type 1`: `Refinement`’s two universes are independent (probe `Shapes`, output lines 10–13), and the construction typechecks (probe `Transport`, example 1). `Refinement.map_option_valid` has the polarity “every witness in the slot is valid $\Rightarrow$ every mapped witness is valid” (ToArkLib/Refinement.lean:55-57), while the game’s bad event is “no witness in the slot is valid” (an empty slot is bad); the transport of the bad event is $\forall w' \in w?.map f, (x, w') \notin S \rightarrow \forall w \in w?, (x, w) \notin R$, which follows from `map_valid` in one line (probe `Transport`, `bad_of_bad`), and the spine dossier’s probe `knowledge_transport` (code-spine.md) proves the probabilistic statement for ArkLib’s game in the same way. `Extractor.Straightline.map` cannot serve: its `WitIn'` is in `Type` (probe `UniverseFail`: “EnsembleWitness (leanIsaEnsemble prog) has type Type 1 ... but is expected to have type Type”).
- `constraintSoundness (hwf : WellFormedBytecode prog)`: pointwise.

The composition typechecks in principle as an inclusion of events (probe `Transport`, examples 3 and 4): if $\neg \exists t, \text{ValidExecution prog input } t$, then every extracted `q` is outside `M3Rel`, so the game’s bad event is “the verifier accepts”, and $\text{Pr}[\text{accepts}] \leq \text{niError}$. That is the correct form of the first half of the base theorem: **a soundness theorem for the language $\{(\text{prog}, \text{input}) \mid \exists t, \text{ValidExecution prog input } t\}$** (the conclusion is a closed proposition, so “except with probability” attaches to acceptance, not to it), in the random-oracle model, for each admissible `s` or with the family composition of section 8.8.2.3, and under `WellFormedBytecode prog`. It is not, and cannot be, an ArkLib `knowledgeSoundness` of a relation on `EnsembleWitness (WitIn : Type)`, nor a statement about the concrete `verify prog input proof = true` (a closed Boolean), and it does not need the round-by-round implication if `FiatShamirSecurity` consumes the round-by-round form directly.

Proposed change: the base soundness theorem and the composition line (bp:407, bp:1247-1248)

As it stands. the two excerpts above.

As proposed.

```
-- Base soundness, in the random-oracle model: for a well-formed program and an input on
which no execution exists, every prover with at most `Q` oracle queries makes the
Fiat-Shamir-compiled verifier of `leanIsaInstance prog s` accept with probability at most
`niError Q` (the maximum over admissible `s`; finding 6). `verify` is that verifier with the
BLAKE2s chain in place of the oracle (`verify_iff_compiled`), which is a heuristic and not a
theorem. -/
```

```

theorem baseVerifier_sound (fs bcs mca flock) (hwf : WellFormedBytecode prog)
  (hno : ~ ∃ t, ValidExecution prog input t) : ∀ prover, Pr[compiled verifier accepts] ≤ niError Q
/-- Pointwise, for the extractor's stack: what recursion consumes (T6). -/
theorem execution_of_extracted (hwf : WellFormedBytecode prog) (hs : s.Admissible prog)
  (h : M3Holds (leanIsaInstance prog s) input q) :
  ∃ t, AssignmentRepresents (witnessOf prog s q) t ∧ ValidExecution prog input t

```

(“finding 6” in the docstring is finding 76.) The line at bp:407 becomes `T4 = verify_knowledge Sound ◦ Refinement.knowledge_transport satisfiedBy_witnessOf ◦ constraintSoundness`, with `Refinement.knowledge_transport` (the event form of the spine dossier) named as what the base theorem composes, and `map_option_valid` kept as the pointwise lemma of the honest direction; the tracker’s `knowledgeSound_of_refinement` the same.

Reason. Findings 73 and 83.

8.8.4 The adaptor and the boundary with leanISA: findings

Short forms in this section. `bp:` is the blueprint `docs/roadmap/protocol-blueprint.md` at leanerVM b435631; `architecture.md`, `leanvm-target.md` are under `docs/`, `leanisa-blueprint.md` and `protocol-status.md` under `docs/roadmap/`, at the same revision. Lean files are named by their last component: `Statement.lean`, `Boundary.lean`, `Channels.lean` under `LeanerVM/Arithmetization/`; `Execution.lean`, `Memory.lean` under `LeanerVM/Semantics/`; `Spine/Instance.lean`, `Spine/Toy.lean`, `ToArkLib/Refinement.lean` under `LeanerVM/Protocol/`; all at b435631. Clean is cited at its old pin 93c9d1ef, ArkLib at dca90385, CompPoly at 3468b38c (and 572f9973 where said). Rust files are under `crates/lean_vm/src/` (`layout.rs` is `cpu/layout.rs`), `whir_config.rs` under `crates/pcs/src/`, `lean_compiler/src/lib.rs` and `filler.rs` under `crates/lean_compiler/src/`; `verifier.py` is `python-verifier/verifier.py`; the specification’s `08-end-to-end-protocol.tex` is under `doc/leanvm/body/`; all at leanVM’s pin a386121f, read with `git show`. `hole-comment.md` is the review’s copy of the tracker’s hole comment. The target theorems of `architecture.md` are named in words with their code in parentheses: `constraint soundness` and `completeness` (T1-S, T1-C), `witness generation` (T2), `guest correctness` (T3), the base theorem of the proof system (T4), `recursion` (T6), `end-to-end soundness` (T7). The findings rest on sections 8.8.1 to 8.8.3; there are no critical findings, nine major, six minor and three notes.

Finding 71 (major). The instance is not `Ensemble.toM3` of the eight tables

Evidence. The blueprint:

```

def leanIsaInstance (prog : Program) (s : Sizes) : M3Instance -- `Ensemble.toM3` of the eight tables with
                                                                -- leanISA's separators and directions; the
                                                                -- layout of `witness.rs:67-101`,
                                                                ↪ `leaf.rs:53-156`

```

`docs/roadmap/protocol-blueprint.md:803-805` at b435631

bp:77: “which `Ensemble.toM3` derives from any Clean Ensemble”; bp:21: “the polynomial view of any Clean ensemble”. Against it: the status file’s decision that “the fixed columns are `Coord.known` data” (decision 8, `protocol-status.md:220-223`), the spine’s `boundary` field and `Coord.known` (`Spine/Instance.lean:86-98`, 136), leanVM’s three framework blocks per side (`layout.rs:352-395`; `08-end-to-end-protocol.tex:70`, “three blocks per side belong to no table”) whose program columns the verifier evaluates itself (`leaf.rs:453`), and section 8.8.1.6: an instance whose blocks are tables of `q` makes `satisfiedBy_witnessOf` unprovable (probe `KnownColumn`). An `M3Instance` also needs `μ`, `layout`, `publicLines`, `aux`, `Stmt`, none of which a Clean Ensemble has, so no `Ensemble.toM3 : Ensemble → M3Instance` exists; `Ensemble.toM3` is listed as an interface (bp:1369, 1398) and given no signature.

Classification. An error of the blueprint.

Proposed change.

Proposed change: the instance of Layer 3 (bp:803-805)

As it stands. the comment quoted above.

As proposed. “Component.toM3 of the six opcode tables, with leanISA’s separators and directions and their count columns; three column groups for the shared committed columns (mem_0, mem_1, mem_2, cntfin_mem at $2^{\log \text{Mem}}$; cntfin_bc at $2^{\text{prog.logSize}}$; q_flock at $2^{(\tau_5 + 8)}$), tables with no constraint, flush or count; six boundary blocks written from memTable, bytecodeTable and leanIsaVerifier prog with $\text{idx} \mapsto \text{Coord.known}(\text{idxColumn } \kappa)$, the entry cells $\mapsto \text{Coord.known}(\text{bytecodeSlotColumn prog } \kappa)$, the sentinel $\mapsto \text{Coord.const prog.finalPc}$, the limbs and counts $\mapsto \text{Coord.committed}$; the layout of witness.rs:67-101, leaf.rs:53-156; three public lines; aux the Flock predicate of the region”. Delete Ensemble.toM3 from bp:77, 1369, 1398 and from the sentence at bp:21, or define it as the six-table part only.

Finding 72 (major). No bridge lemma for the boundary blocks

Evidence. Layer 2’s two bridges are toM3_constraints_iff and toM3_flushes_eq (bp:775-777), both about a Clean Table read as an instance *table*. The three balances of SatisfiedBy range over *w*. interactions, which include the memory block’s, the bytecode block’s and the verifier’s rows (Clean/Air/FlatEnsemble.lean:225-226 at 93c9d1ef; Statement.lean:207-209); in the instance those are boundaryTuples, not flushTuples (sections 8.8.1.3 and 8.8.1.6). Nothing in Layers 2 or 3 states that the messages of bytecodeRowOf prog i c, memRowOf mem i c and leanIsaVerifier prog’s row are the boundary tuples, coordinate by coordinate.

Classification. An error of the blueprint (omission).

Proposed change.

Proposed change: a third bridge lemma in Layer 3 (bp:810-819)

As it stands. no lemma relates the two block components and the verifier component to the instance’s boundary tuples.

As proposed. Add to Layer 3:

```
theorem boundary_tuples_eq (h1 : IndexColumnsAreRowIndex w) (h2 : SeedRowsAreTheImage w)
  (h3 : BytecodeRowsAreTheProgram prog w) (hs : Sizes.ofWitness w = some s) :
  the multiset of sixteen-tuples of the messages of w's memory block, bytecode block and
  verifier row on each side = (leanIsaInstance prog s).boundaryTuples (stackOf gen w hs) side
```

and its use in both adaptor theorems; add to Layer 2’s tests “the three block components against their boundary blocks on the one-row witness”.

Reason. Without it neither direction of the adaptor can be proved, and it is the one place the program’s known columns meet the program’s rows.

Finding 73 (major). The base soundness theorem omits WellFormedBytecode, and is not a statement about verify

Evidence. The blueprint:

```
theorem baseVerifier_extractsExecution (fs bcs mca flock) (h : verify prog input proof = true) :
  except with probability niError,  $\exists$  t, ValidExecution prog input t
```

docs/roadmap/protocol-blueprint.md:1247-1248 at b435631

which “composes ... constraintSoundness” (bp:1253-1254); but constraintSoundness (hwf : WellFormedBytecode prog) (leanisa-blueprint.md:1163; architecture.md:224-228; the documentation dossier, docs-debt.md B.6). The JUMP-sentinel program has two rows that are steps and balance the state channel with no ValidExecution for the image (tests/LeanerVMTests/Semantics/Execution.lean:415-461, proved), so the theorem is false without the hypothesis. The “except with probability” attaches to `verify prog input proof = true`, a closed Boolean of the concrete BLAKE2s verifier, which has no probability; the theorem must be about the random-oracle verifier of `verify_iff_compiled`, and the step to the concrete `verify` is the random-oracle heuristic, stated nowhere

(section 8.8.2.4). The conclusion mentions no extracted witness, so the statement is a soundness statement for a language (section 8.8.3.6, probe Transport examples 3–4).

Classification. An error of the blueprint.

Proposed change.

Proposed change: the base soundness theorem (bp:1247-1248)

As it stands. the excerpt above.

As proposed.

```

/-- Base soundness, in the random-oracle model: for a well-formed program and an input on
which no execution exists, every prover with at most `Q` oracle queries makes the
Fiat-Shamir-compiled verifier of `leanIsaInstance prog s` accept with probability at most
`niError Q` (the maximum over admissible `s`; finding 6). `verify` is that verifier with the
BLAKE2s chain in place of the oracle (`verify_iff_compiled`), which is a heuristic and not a
theorem. -/
theorem baseVerifier_sound (fs bcs mca flock) (hwf : WellFormedBytecode prog)
  (hno : ¬ ∃ t, ValidExecution prog input t) : ∀ prover, Pr[compiled verifier accepts] ≤ niError Q
/-- Pointwise, for the extractor's stack: what recursion consumes (T6). -/
theorem execution_of_extracted (hwf : WellFormedBytecode prog) (hs : s.Admissible prog)
  (h : M3Holds (leanIsaInstance prog s) input q) :
  ∃ t, AssignmentRepresents (witnessOf prog s q) t ∧ ValidExecution prog input t

```

(“finding 6” in the docstring is finding 76.) And in the prose: who establishes `WellFormedBytecode` (the compiler, `lean_compiler/src/lib.rs:162`, `filler.rs`; a check of guest correctness on the exact guest), and that the random-oracle instantiation is a hypothesis of the deployment, not of any theorem. The same two edits in the tracker’s section on the base theorem (K4, `hole-comment.md:190`).

Finding 74 (major). The base completeness theorem is false without resource hypotheses and needs a witness generator

Evidence. The blueprint:

```

theorem baseProver_complete (hfill : HasFillBlocks prog) (h : ValidExecution prog input t) :
  verify prog input (prove prog input (witness of t)) = true

```

`docs/roadmap/protocol-blueprint.md:1249-1250` at `b435631`

which “composes `constraintCompleteness`, `m3Holds_stackOf`, `piop_perfectCompleteness` and the determinism of the Fiat–Shamir chain” (`bp:1255-1257`).

- `HasPublicBoundary` allows $\kappa = 32$ (`Execution.lean:108-110`); a valid execution at $\kappa = 16$ is one at $\kappa = 32$ with the image extended by zeros (the same run); its four memory columns hold $4 \cdot 2^{32} = 2^{34} > 2^{28}$ cells, and the verifier rejects $\mu > 28$ (`cpu/mod.rs:174-176`, `pcs.rs:51`; `verifier.py:1379`); `AssignmentRepresents` fixes κ (`Statement.lean:358`), so no witness of `t` is provable.
- `constraintCompleteness` gives $\exists w$; “witness of `t`” is a function the blueprint excludes (witness generation, `T2`, out of scope, `bp:147-148`).
- `constraintCompleteness` takes `WellFormedBytecode prog`, not `HasFillBlocks prog` (`leanisa-blueprint.md:1165`).
- The rate is not chosen by `t`.

Classification. An error of the blueprint.

Proposed change.

Proposed change: the base completeness theorem (bp:1249-1250)

As it stands. the excerpt above.

As proposed.

```

/-- Base completeness: a satisfying witness whose sizes are admissible and any valid rate give
a proof `verify` accepts. -/
theorem baseProver_complete (gen : FlockWitnessGen) (hwf : WellFormedBytecode prog)
  (h : SatisfiedBy prog input w) (hs : Sizes.ofWitness w = some s) (hadm : s.Admissible prog)
  (hp : validRate p) : verify prog input (prove gen prog input w p) = true
/-- With leanISA's existence theorem: a valid execution that some admissible-size witness
represents has an accepted proof. -/
theorem baseProver_complete_of_execution (hwf : WellFormedBytecode prog)
  (h : ValidExecution prog input t)
  (hfit :  $\exists w s$ , SatisfiedBy prog input w  $\wedge$  AssignmentRepresents w t  $\wedge$ 
    Sizes.ofWitness w = some s  $\wedge$  s.Admissible prog) :  $\exists$  proof, verify prog input proof = true

```

and in the prose: “The resource condition `architecture.md:273-275` asks for is `hfit`: the stacking window forbids $\kappa > 26$ and long runs; `constraintCompleteness` alone does not give admissible sizes (and, as sketched, has no resource hypothesis of its own, finding 88)”. Ask the leanISA roadmap (*Boundaries*, bp:1433-1441) for a `constraintCompleteness` that returns a witness of *minimal* heights, or accept `hfit` as the base theorem’s hypothesis. The same in the tracker’s section on the base theorem.

Finding 75 (major). `Sizes`, `Sizes.ofWitness` and `admissible_iff_caps` cannot be stated as written

Evidence. The sketch (bp:795-801, quoted in section 8.8.3.3). `logInvRate` is announced (`cpu/mod.rs:118-124`, `verifier.py:1372-1376`; not in the specification’s `Setup`, `08-end-to-end-protocol.tex:55`) but no witness determines it, so `Sizes.ofWitness w : Option Sizes` is ill-defined; `admissible_iff_caps : s.Admissible prog  $\leftrightarrow$  (Caps w  $\wedge$  Sizes.ofWitness w = some s)` has a free `w` and, quantified, is false in both directions (`Caps` lacks the μ and rate windows, `Statement.lean:79-81`; another `w` has other sizes); `Sizes.ofWitness (w : EnsembleWitness leanIsaEnsemble)` lacks `prog` (bp:799); `leanIsaInstance_fits` uses `layout.total` (bp:808), a field `Layout` has not (the Layer 1 dossier, `code-layer1.md G.2`). The bus dossier has the caps side (finding 8).

Classification. An error of the blueprint.

Proposed change.

Proposed change: the sizes (bp:795-808)

As it stands. the sketch of section 8.8.3.3.

As proposed.

```

structure Sizes where (logMem :  $\mathbb{N}$ ) ( $\tau$  :  $\text{Fin } 6 \rightarrow \mathbb{N}$ ) -- the family index
def Sizes.ofWitness (w : EnsembleWitness (leanIsaEnsemble prog)) : Option Sizes
def leanIsaBlocks (prog) (s) : Blocks ; def leanIsa $\mu$  (prog) (s) :  $\mathbb{N}$  -- witness.rs:67-101
theorem leanIsaBlocks_fits : (leanIsaBlocks prog s).total  $\leq 2^{\wedge}$  leanIsa $\mu$  prog s
def Sizes.Admissible (prog) (s) : Prop --  $16 \leq \text{logMem} \leq 32$ ,  $\tau_j \leq 32$ ,  $3 \leq \tau_5$ , leanIsa $\mu$  prog s  $\leq 28$ 
def validRate (p :  $\mathbb{N}$ ) : Prop :=  $1 \leq p \wedge p \leq 4$  -- whir_config.rs:48-55, Layer 12
theorem caps_of_admissible (hs : s.Admissible prog) (q) : Caps (witnessOf prog s q)
theorem sizes_of_satisfiedBy (h : SatisfiedBy prog input w) :  $\exists s$ , Sizes.ofWitness w = some s

```

Reason. The instance depends on the heights alone; the rate is Layer 12’s; the two lemmas are what the two directions use.

Finding 76 (major). The composition over the prover’s announced sizes is unspecified

Evidence. Section 8.8.2.3: `verify_iff_compiled` quantifies $\exists s$ (bp:1218-1221), `verify_knowledge` Sound’s error is `niError s Q` (bp:1226-1227), the per-instance theorems bound one `s`, and `ArkLib’s ProtocolSpec n` cannot express a protocol whose schedule depends on a first message. A union over the admissible sizes costs about 2^{34} ; the $Q \cdot \max \epsilon$ bound needs `FiatShamirSecurity` stated for the family with `s` in the hashed input.

Classification. An error of the blueprint (a missing decision).

Proposed change.

Proposed change: the family of instances (bp:316, bp:1226-1227, bp:1326-1328)

As it stands. the convention *Statements and parameters* (bp:316) says the instance indexes the protocol family and nothing of how the error composes over it; Layer 12 states the error as $\text{niError } s \ Q$.

As proposed. In *Statements and parameters* add: “The sizes are the prover’s. The non-interactive theorem is stated for the family: its error is $\text{niError } Q := \max \text{ over admissible } s \text{ of } \text{niError } s \ Q$ (or the sum, if the interface is per instance), and *FiatShamirSecurity* is the family form, the challenge oracle taking the announced sizes with the statement”. In Layer 12 (bp:1226-1227) replace $\text{niError } s \ Q$ by that. In acceptance test 21 (bp:1326-1328) add “the error of *verify* does not depend on the prover’s choice”.

Finding 77 (major). The instance depends on a Flock interface that depends on the instance

Evidence. bp:846-848: the instance’s layout takes its slot map from `FlockInterface.limbColumns`, and structure `FlockInterface (I : M3Instance)` (bp:1077); aux of `leanIsaInstance` must be Flock’s R1CS (the strong reading, decision 12), which only the Flock roadmap (issue #3) defines; the consequence lemma “R1CS $\Rightarrow$ the limbs compress” that `satisfiedBy_witnessOf` consumes (bp:855-857) has no carrier in any signature (`satisfiedBy_witnessOf`’s arguments, `FlockInterface`’s fields, `FlockWitnessGen`). The Flock dossier proposes the split (findings 36 and 37); the Layer 1 dossier the reader (code-layer1.md G.1).

Classification. An error of the blueprint.

Proposed change. Layer 3 takes, and the Flock roadmap supplies, one instance-free structure:

```
-- What the adaptor needs of Flock, none of it mentioning an instance. -/
structure FlockSpec where
  slot : Fin 18 → Fin 256 -- hash_flock.rs:87-115
  Holds (kBatch : ℕ) : Column (8 + kBatch) → Prop -- the R1CS on the packed column
  decHolds : ∀ k, DecidablePred (Holds k)
  compress_of_holds : Holds k c → ∀ j, Blake2sRelation (limbs read at slot j of c)
  gen : (rows : List (Blake2sRow K)) → Column (8 + kBatch) -- keeps the limbs in their slots
  holds_gen : (∀ r ∈ rows, Blake2sRelation r) → Holds k (gen rows)
```

`leanIsaInstance (F : FlockSpec) prog s` uses `F.slot` in its layout, `F.Holds` as aux; `satisfiedBy_witnessOf` uses `F.compress_of_holds`; `stackOf` uses `F.gen`, `m3Holds_stackOf` uses `F.holds_gen`; `FlockWitnessGen` and `FlockInterface.limbColumns` are deleted; the phase-level `FlockInterface` keeps the reduction and its two proofs and takes the `FlockRegion I` of the Flock dossier (finding 37), whose `aux_iff` is `F.Holds`.

Finding 78 (major). The polynomial bridge of Layer 2 is noncomputable at the pin

Evidence. Section 8.8.3.5: bp:762-771 produce `Mathlib MvPolynomials`; the instance holds `CMvPolynomials (Spine/Instance.lean:123-125)`; `toCMvPolynomial` is noncomputable `def` (`CompPoly 3468b38c, Multivariate/MvPolyEquiv/Core.lean:41`; the same noncomputable `def` at the same line at 572f9973, read with `git show` of the new pin); `M3Holds` is meant to be decided by evaluation and `verify` to be computable (bp:866-867, 1230-1231). Probe `PolyBridge`: the direct translation is computable and agrees with `Expression.eval`.

Classification. A deviation forced by an upstream library, with a workaround: the direct translation; retired if `CompPoly` makes `toCMvPolynomial` computable.

Proposed change.

Proposed change: the polynomial bridge (bp:762-767)**As it stands.**

```

def Expression.toMvPolynomial : Expression F → MvPolynomial N F      -- var i ↦ X i
theorem eval_toMvPolynomial (env : Environment F) (e) :
  MvPolynomial.eval env.get e.toMvPolynomial = e.eval env
def Expression.degreeBound : Expression F → N                      -- syntactic; add max, mul sum
theorem totalDegree_le_degreeBound (e) : e.toMvPolynomial.totalDegree ≤ e.degreeBound

```

As proposed. “def Expression.toCMvPolynomial (n) : Expression K → CMvPolynomial n K (var i < n ↦ X i, else 0, as Environment.fromArray reads a missing cell); theorem eval_toCMvPolynomial (row) (e) : (e.toCMvPolynomial n).eval (fun i ↦ row[i]?.getD 0) = e.eval (Environment.fromArray row data); theorem fromCMvPolynomial_toCMvPolynomial (e) : fromCMvPolynomial (e.toMvPolynomial n) = rename ... e.toMvPolynomial, through which degreeBound bounds totalDegree (totalDegree_equiv)”. M3Table’s fields become CMvPolynomial; Direction becomes Side (or the spine adopts leanISA’s Direction). Clean #466 (Expression.toMvPolynomial) stays the proof-side bridge.

Finding 79 (major). The column-only tables enter the generic table sumcheck’s schedule

Evidence. Section 8.8.3.4; bp:836-839 (the six shared columns are “tables of the instance with no constraints and no flushes”); Layer 7’s $\tau_{\max}$, “one value per column of every table” (bp:987-989; 08-end-to-end-protocol.tex:76-77; constraints.rs:250, the six tables); the bus dossier’s note (finding 20). A phase over the abstract I that ranges over all of I.ntab has a different transcript from leanVM’s on the leanISA instance, which `verify_iff_compiled` would then fail.

Classification. An error of the blueprint (a consequence of the modelling choice, unstated).

Proposed change.

Proposed change: column groups (bp:316, bp:987)

As it stands. nothing says how a phase over the abstract instance treats a table with no constraint, no flush and no count column.

As proposed. In the spine’s conventions (bp:316, or a new row *Column groups*): “A table with no constraint, no flush and no count column is a column group: it takes no part in the table sumcheck (no round, no term, no column value) and its columns receive claims from the boundary blocks, the public lines and the Flock phase only. `M3Instance.active j := (I.constraints j ≠ []) ∨ (I.flushes j ≠ []) ∨ (I.counts j ≠ [])` is the criterion, decidable, and Layer 7 ranges over it.” In Layer 7 (bp:987): $\tau_{\max}$ and $\Sigma_j \text{width}_j$ over the active tables.

Finding 80 (minor). `witnessOf_stackOf` has no Lean statement

Evidence. Section 8.8.3.2; bp:819 (“= w -- on the committed fields”), bp:1339 (“#guard witnessOf (stackOf w) = w”), bp:866-867. EnsembleWitness has a function field and a free width and no DecidableEq.

Classification. An error of the blueprint.

Proposed change.

Proposed change: the round trip (bp:819, bp:1339, bp:822-824)

As it stands. theorem `witnessOf_stackOf (hs) : witnessOf prog s (stackOf gen w hs) = w` - on the committed fields and #guard `witnessOf (stackOf w) = w`.

As proposed. at bp:819, the statement of section 8.8.3.2 (rowCells, the three equalities under SatisfiedBy and Sizes.ofWitness); at bp:1339, “#guard of those three equalities on the one-row witness”. Say in bp:822-824 that neither composition of the base theorem uses it.

Finding 81 (minor). `witnessOf` has no input, and the count columns have no derivation

Evidence. bp:813-814; bp:773-777 (`Component.toM3 (c) (sep) (dir)` with a count field and no rule); layout.rs:412-414; table 8.27, the rows of `public_input_eq` and `counts_nonzero`.

Classification. An error of the blueprint (imprecision).

Proposed change.

Proposed change: the adaptor's map and the count columns (bp:813, bp:773-774)

As it stands. `def witnessOf (prog) (s) (q : Column (leanIsaInstance prog s).μ) and def Component.toM3 (c : Component F) (sep : RawChannel F → F) (dir : RawChannel F → Direction).`

As proposed. At bp:813: either add `(input : PublicInput)` to `witnessOf` (the `Refinement.map` has the statement) or say “the lanes are read off cells 0 and 1 of `mem_0`, `mem_1`”. At bp:773-774: `Component.toM3 (c) (sep) (dir) (lookups : List (RawChannel F))` with “count is coordinate 2 of every pull on a channel in `lookups`, required to be a variable (`vars_lt_width`'s sibling `count_is_var`)”, and the test “the count columns of the six tables are those of `count_columns()` (layout.rs:412-414)”.

Finding 82 (minor). `leanIsaInstance` must be reducible

Evidence. Probe `DefInstance` (appendix B.3.6): the adaptor's statement elaborates on a `def` instance, but no `Decidable` instance is found for `M3Holds (inst ...) input q`, while the `abbrev` passes; the toy is an `abbrev` for that reason (Spine/Toy.lean:94-95); the Layer 3 tests decide `M3Holds` (bp:866-867).

Classification. An error of the blueprint (omission of a constraint on the code).

Proposed change.

Proposed change: the instance's definition (bp:803)

As it stands. `def leanIsaInstance (prog : Program) (s : Sizes) : M3Instance.`

As proposed. “abbrev `leanIsaInstance ...`, reducible, so that `Decidable (M3Holds (leanIsaInstance prog s) input q)` and the instances on `I.Smt` are found by instance search, as the toy is”.

Finding 83 (minor). `Refinement.map_option_valid` has the wrong polarity for ArkLib's game

Evidence. Section 8.8.3.6; ToArkLib/Refinement.lean:55-57; Security/Basic.lean:316 at dca90385 (the bad event is $\forall w \in w?, (x, w) \notin \text{relIn}$); probe `Transport`, `bad_of_bad`. Harmless (a case split on the `Option` converts), and the spine dossier's `knowledge_transport` (code-spine.md) is the probabilistic form.

Classification. Imprecision.

Proposed change.

Proposed change: the transport the base theorem composes (bp:407, bp:552, bp:582-586)

As it stands. `T4 = verify_knowledgeSound ◦ Refinement.map_option_valid satisfiedBy_witnessOf ◦ constraintSoundness` (bp:407).

As proposed. At bp:552 and bp:582-586, name `Refinement.knowledge_transport` (the event form, code-spine.md) as what the base theorem composes, and keep `map_option_valid` as the point-wise lemma of the honest direction; the line at bp:407 becomes “`T4 = verify_knowledgeSound ◦ Refinement.knowledge_transport satisfiedBy_witnessOf ◦ constraintSoundness`”, and the tracker's `knowledgeSound_of_refinement` (hole-comment.md:190) the same.

Finding 84 (minor). Stale names**Evidence.**

- bp:220: `StateMsg` (the state message is `Regs`, `Channels.lean:138-141`; no `StateMsg` exists);
- bp:1280: `leanIsaTables` (no such declaration; the ensemble is `leanIsaEnsemble`);
- bp:799: `EnsembleWitness leanIsaEnsemble` (needs `prog` since the decision that channels and tables are program-free, decision 14);
- bp:808: `layout.total`;
- the tracker’s `knowledgeSound_of_refinement`.

Proposed change. Replace each by the name that exists.

Finding 85 (minor). Two deployed checks are discharged by type, and nobody is told to parse

Evidence. Section 8.8.2.2, item 2: `read_public`’s checks (2) (the public words’ third limb, `cpu/mod.rs:141-143`) and (3) (the bytecode length and, in the Python, decodability) are “by type” in Lean (`Statement.lean:71-75`); `verify : Program → PublicInput → Proof → Bool` (bp:1217) takes the typed values; the fixture (bp:1236-1240) and end-to-end soundness’s (T7) `PublicInput.encode` are the parsers.

Classification. An error of the blueprint (omission).

Proposed change.

Proposed change: the convention *Verifier shape* (bp:326)

As it stands. “Structure checks on public data that the Rust `assert!`s (finding F10) are theorems about the layout, not branches of `verify`.”

As proposed. Add: “`verify` takes a `Program` and a `PublicInput`; the deployed verifier’s rejection of a public word with a nonzero third limb and of a bytecode that is not $2^k \leq 2^{32}$ decodable instructions are the obligations of whoever builds those values from bytes (the fixture’s loader in Layer 12, `PublicInput.encode` in T7), and the differential fixture records a rejected public input as a parse failure, not as `verify = false`”.

Finding 86 (note). The base theorem (T4) drops the extracted witness that recursion (T6) needs

Evidence. Section 8.8.2.6, the last paragraph; `architecture.md:332-335`: “the extracted leanVM witness should contain ordered `pc/fp` steps and the full memory image”. `baseVerifier_extracts Execution` keeps only $\exists t$, `ValidExecution prog input t`.

Proposed change. The second theorem of the proposal of finding 73 (`execution_of_extracted`).

Finding 87 (note). The protocol’s soundness is non-adaptive in the statement

Evidence. Section 8.8.2.3, the last paragraph: `ArkLib`’s games quantify $\forall \text{stmtIn}$ outside the probability (`Security/Basic.lean:299-323`); end-to-end soundness’s (T7) public input has a prover-chosen part (`architecture.md:191-195`, 357-358).

Proposed change. Record in the blueprint’s *Out of scope* that adaptive statement soundness in the random-oracle model (the input seeds the chain, `cpu/mod.rs:712`) is end-to-end soundness’s to state and is not given by the per-statement theorems of this roadmap.

Finding 88 (note). `constraintCompleteness` needs a resource hypothesis of its own (leanISA’s)

Evidence. Section 8.8.2.4: `Caps.heights` bounds every table by 2^{32} rows (`Statement.lean:257`), `ValidExecution` bounds only κ (`Execution.lean:108-115`), and a halting run visits distinct (`pc`, `fp`) states (the step is a function of the state and the fixed image), so a run of more than 2^{32} steps of one opcode has no satisfying witness. The claim that a halting run visits distinct states is an argument from the determinism of the step, not a Lean proof. Out of this review’s scope; it reaches the base theorem through finding 74.

Proposed change. Report to the leanISA roadmap (*Boundaries*, bp:1433-1441).

8.9 The code as built: the spine, Layer 1, the public-input phase

8.9.1 The code against the blueprint

Where the blueprint’s sketches, interface list and acceptance tests match the code as built at **b435631**, and where they do not; and, for the public-input phase, where the Lean verifier matches the specification and the deployed verifiers.

8.9.1.1 The spine

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

All blueprint lines are those of `docs/roadmap/protocol-blueprint.md` at **b435631**; all Lean lines those of `main` at **b435631**.

The Lean sketch (:442-554) against the code, declaration by declaration (E.1) The sketch is a compressed transcription of the code and matches it in every name, field and signature it gives, with the following differences (none changes a statement):

Sketch	Code	Difference
:445 structure Shape where (ntab : ℕ) (τ width : Fin ntab → ℕ)	Instance.lean:5 7-63	none (compressed)
:447-450 Layout ... read_eval : ∀ q c z, eval₂ Mle (read q c) z = eval₂ Mle q (extend c z)	:79-81	the code’s law is on .values with algebraMap K E; same statement
:451 Coord ... known (col : Column κ)	:86-89	none
:453-454 PublicLine ... (sent : Bool) (pos : 0 < S.τ col.1)	:102-114	none
:455-466 M3Instance	:119-148	the sketch omits decAux’s attribute [instance] (:150); otherwise field for field
:467 def M3Instance.column I q c ; def M3Instance.row I q j x ; def M 3Instance.tuples I q (s : Side)	:160, 163, 185	the sketch omits κ, coordCell, flushTuples, boundaryTuples (:15 7, 167, 174, 180), which tuples unfolds to and an auditor must read
:468 Balanced I q (List.Perm)	:201	none
:469-471 M3Holds, deriving instance Decidable	:219-223	none
:472 abbrev TheOracle I : Fin 1 → Type := fun _ ↦ Column I.μ	:226 OneOracle (Column I.μ)	same type
:473 M3Rel	:230-231	none
:476-484 claims and outputs	Seams.lean:57-1 51	Weight’s mle : Vector E μ → E and mle_eq are elided as ...; the . Holds of each claim (:66, 98, 123), VirtualTerm.table, Virtual Term.eval and Weight.pair (:82, 87, 112) are not in the sketch and are load-bearing
:485-490 seams	:166-191	Seam.table/pub/flock/done are summarised in words; the code is as summarised
:493-498 KnowledgeAppend	KnowledgeAppend. lean:474-478, 5 10-518	rbrKS WorstCaseWith is an abbreviation of the sketch for rbrKnow ledgeSoundnessWorstCaseWith; otherwise as stated
:501-513 Component.Def/Complete/Se curity	Component.lean: 52-106	the sketch writes Extractor.RoundByRound [] ...; the code writes OracleSpec.emptySpec.{0, 0} to pin a universe (:96-99), and says so
:514-515 the three appends	:143, 162, 174	the sketch omits guardedAppend and stateFunctionOfEq (:156, 110), which Security.append unfolds to
:517-521 pass-through, send-oracle	PassThrough.lean, SendOracle.lean	the sketch omits sendOracle_relOut, sendVerifierPure, sendStat eFunction, sendWitMid (SendOracle.lean:71, 101, 145, 130), all in the knowledge theorem’s statement (section B)

Sketch	Code	Difference
:523-525 Phase.*	Phase.lean:37-74	none
:528-548 Compose.lean	Compose.lean:51-185	none; the sketch's comment on <code>piopExtractor</code> , "commitExtractor, then the phases", is right of <code>extractMid</code> and wrong of <code>extractOut</code> (D.5)
:551-553 Refinement	Refinement.lean:31-71	none

Names the sketch or the interface list (:1365-1385) give that do not exist under that name or signature on main: none in the spine block of the list (:1366-1385) — every name was found (Side ... Toy.layout), except that the list names `Ensemble.toM3` (Layer 2) (not built, marked) and `Component.Security.{witMid, extractor, kSF, rbr}` (fields, exist). **Public declarations of the reviewed modules that the list does not name:** `M3Instance.κ`, `M3Instance.coordCell`, `flushTuples`, `boundaryTuples`, `M3Instance.decAux` (as an instance), the five `.Holds`, `VirtualTerm.table`, `VirtualTerm.eval`, `Weight.pair`, `Component.stateFunctionOfEq`, `rbrKnowledgeSoundnessWorstCaseWith_of_eq`, `Component.guardedAppend`, `Component.passThroughProver/Verifier/Pure/Extractor/StateFunction`, `passThrough_materializeOutput`, `passThroughVerifier_toVerifier_run`, `passThrough_rbr`, `Component.passThroughComplete`, `Component.sendSpec/sendProver/sendEmbedding/sendVerifier`, `sendOracle_relOut`, `sendOracle_outputPure`, `send_materializeOutput`, `sendVerifier_toVerifier_run`, `sendVerifierPure`, `sendOracle_complete`, `sendOracleComplete`, `sendWitMid`, `sendExtractor`, `sendStateFunction`, `sendOracle_rbr`, `sendOracleSecurity`, `noOracle_eq`, `Refinement.id`, `Refinement.comp`, `Verifier.GuardedForm.of_probEvent_pos`, `Reduction.mem_support_run_of_guarded`, `keepOracles`, `OracleVerifier.materializeOutput_of_keepOracles`, everything in `KnowledgeAppend` but the two named theorems, `Toy.Col`, `Toy.slice`, `Toy.extend`, `Toy.read_eval`, `Toy.constraint`, `Toy.flush`, `Toy.boundary`. The list's sentence "Everything not listed is a proof, a helper, or a test" (:1419) is false of the fourteen of these that section B counts as load-bearing (the `.Holds`, `table`, `eval`, `pair`, `coordCell`, the two tuple lists, `sendOracle_relOut`, `sendVerifierPure`, `sendStateFunction`, `sendWitMid`, `guardedAppend`, `stateFunctionOfEq`).

The blueprint's other statements of the same objects (E.2)

Object	Where	As stated there	The spine as built (the code's version)	Which the code implements
the knowledge theorem	headline :14-16	<code>piop_rbrKnowledgeSoundness (I : M3Instance) : for every prover, leanVmVerifier accepts $\rightarrow$ except with probability <code>piopError I</code>, the extractor's column <code>q</code> satisfies <code>M3Holds I input q</code></code>	<code>(P) (S : P.Security) init impl : (leanVmVerifier P).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (M3Rel I) (Seam.done I) S.toDef.witMid (piopExtractor P S) (S.toDef.kSF init impl) (piopError P)</code> as above	the spine's; the headline's has no <code>P</code> , <code>S</code> , a numeric <code>piopError I</code> , and a plain-form conclusion
the knowledge theorem	Layer 10 :1127-1129	<code>piop_rbrKnowledgeSoundness (hs : s.Admissible prog) : (leanVmVerifier prog s input flock).rbrKnowledgeSoundness init impl (m3Relation prog s) Set.univ (piopError s)</code>	as above	the spine's; Layer 10's is over <code>prog s</code> input flock, has an admissibility hypothesis, the existential and averaged <code>rbrKnowledgeSoundness</code> , the relation <code>m3Relation prog s</code> , <code>Set.univ</code> for <code>Seam.done</code> , and <code>piopError s</code> . The text at :1104-1106 says these are “the spine's, stated over Phases <code>I</code> ”, contradicting the block below it
the completeness theorem	headline :12-13; Layer 10 :1124-1126	<code>(I) : M3Holds I input q $\rightarrow$ the honest prover makes leanVmVerifier accept with probability 1; (hs : s.Admissible prog) : (leanVmPiop prog s input flock).perfectCompleteness init impl (m3Relation prog s) Set.univ</code>	<code>(P) (C : P.Complete) init impl : (leanVmPiop P).perfectCompleteness init impl (M3Rel I) (Seam.done I)</code>	the spine's; Layer 10's admissibility hypothesis is not in the code and is not needed (the sizes are inside <code>I</code>)
leanVmPiop	Layer 10 :1116-1119	<code>leanVmPiop (prog) (s) (input) (flock : FlockInterface prog s) : OracleReduction [] (StmIn := PublicInput) (OStmIn := fun _ : Empty $\mapsto$ Unit) (M3Witness prog s) (StmOut := Unit) (OStmOut := fun _ : Empty $\mapsto$ Unit) Unit (pSpec := ...) := commitPhase $\gg$ busPhase $\gg$ tableSumcheck $\gg$ publicInputPhase $\gg$ flock.reduction $\gg$ openingPhase</code>	<code>leanVmPiop (P : Phases I) : OracleReduction [] I.Stm NoOracle (Column I.μ) Unit (TheOracle I) Unit P.toDef.pSpec := P.toDef.red</code>	the spine's; input is not an argument (it is the statement), the oracle families are <code>Fin 0/Fin 1</code> not <code>Empty</code> , the witness is <code>Column I.μ</code> not <code>M3Witness prog s</code> , the output oracle is the stack not <code>Unit</code> , and the phases are fields of <code>P</code> , not named definitions
piopError	conventions :318; Layer 10 :1122, :1130	“the closed form is a def next to the theorem, and the interactive error is its sum”; <code>piopError (s : Sizes) : (pSpec ...).ChallengeIdx $\rightarrow$ $\mathbb{R}_{\geq 0}$; <code>piopError_le (hs) : Σi, <code>piopError s i $\leq 2^{40} / E + flockError$</code></code></code>	<code>piopError (P : Phases I) := P.toDef.err; no bound</code>	the spine's (finding <i>the declared error is unconstrained</i>)
BusOut	Layer 6 :960-964	structure <code>BusOut</code> where <code>$\zeta : \text{Fin } \mu_{\text{bus}} \rightarrow E$</code> ; <code>rem : $\text{Fin } 3 \rightarrow E$</code> ; <code>pool : List Claim</code> ; <code>$\alpha : \text{Fin } 4 \rightarrow E$</code> ; <code>$\beta : E$</code>	structure <code>BusOut (I)</code> where <code>linear : List (LinearClaim I)</code> ; <code>columns : List (ColumnClaim I)</code> (<code>Seams.lean:130-134</code>) <code>Seam.bus (:175-177)</code>	the spine's; Layer 6's carries the challenges and the three totals, the spine's carries claims (the sibling dossier proposes the shape between the two)
busPhase.reOut	Layer 6 :965-966	<code>{... ($\forall j$, $\Sigma_x \text{eq}(\zeta_{\{<\tau_j\}}$, x) $\cdot B_j^{\wedge}(x) = \text{rem } s$) $\wedge$ every pooled claim holds for $q \wedge (\text{count root} \neq 0)$}</code>		the spine's; Layer 6's has no zerocheck claims, no lines, no <code>aux</code> , a count-root conjunct, and the forms as one equation per side
the table phase's statement types	Layer 7 :988	<code>tableSumcheck (prog) (s) : OracleReduction [] (StmIn := BusOut) ... (StmOut := BusOut $\times$ ColumnClaims)</code>	<code>Phase.Def I (I.Stm $\times$ BusOut I) (I.Stm $\times$ TableOut I) (Compose.lean :78)</code>	the spine's; Layer 7's output keeps <code>BusOut</code>

Object	Where	As stated there	The spine as built (the code's version)	Which the code implements
FlockInterface	Layer 9 :1077-1085	reduction : OracleReduction [] (StmtIn := ColumnClaims) (OStmtIn := fun _ : Unit $\mapsto$ Column μ) Unit (StmtOut := WeightedClaim) ... pSpecFlock ; limbColumns ; relIn ; relOut ; perfectCompleteness ; rbrKnowledgeSoundness : ... rbrKnowledgeSoundnessWorstCase ... flockError ; flockError_le	Phases.flock : Phase.Def I (I.Stmt $\times$ PubOut I) (I.Stmt $\times$ FlockOut I) with Phase.Complete/Phase.Security at Seam.pub/Seam.flock	the spine's; Layer 9's has its own relations, the existential rbrKnowledgeSoundnessWorstCase (no named extractor), a single WeightedClaim output, and an oracle family over Unit
Weight	Layer 10 :1109	structure Weight (μ) where (onCube : ETable μ) (mle : (Fin $\mu \rightarrow E$) $\rightarrow E$) (mle_eq : $\forall r, mle\ r = evalMle\ onCube\ r$)	Weight ($\mu : \mathbb{N}$) ... mle : Vector E $\mu \rightarrow E$ (Seams.lean:103-109)	the spine's: points are Vector E μ (convention <i>Hypercube</i> :312), not Fin $\mu \rightarrow E$
WeightedClaim	Layer 10 :1110; Layer 11 :1173	WeightedClaim (μ) where (W : Weight μ) (c : E); whirlOp en ... (StmtIn := Fin J $\rightarrow$ WeightedClaim μ)	WeightedClaim (I : M3Instance) where weight : Weight I. μ ; value : E (:116-120)	the spine's: indexed by the instance, fields weight/value; Layer 11's WeightedClaim μ does not exist
the opening phase's statement	Layer 10 :1112	openingPhase (μ) (J : $\mathbb{N}$) : OracleReduction [] (StmtIn := Fin J $\rightarrow$ WeightedClaim μ) ... (StmtOut := Unit)	Phases.opening : Phase.Def I (I.Stmt $\times$ FlockOut I) Unit	the spine's; the input carries column claims and weighted claims as lists, and the statement
the extractor	conventions :334; :100-101; :1133-1135; acceptance test 24 :1333-1340	"the commit phase's (which reads the stack off the oracle message) followed by the phases"; "piopExtractor, which reads the stack q off the oracle message, followed by witnessOf"; "the spine's test proves by rfl that piopExtractor returns the committed stack"	piopExtractor P S := S.toDef.extractor, ArkLib's RoundByRound.append nested five times	the spine's; the words are right of extractMid at round 0 and wrong of extractOut (D.5)
the relation ladder's transport	:364-366; :582-586	"along which knowledge transports at the same error (pointwise, Refinement.map_option_valid)"; "the probabilistic transport ... is not proved ... and has no consumer yet"	map_option_valid only	the spine's; the probabilistic form is three lines (D.7)
Seam.done	Layer 10 :1144	"Set.univ as the output relation says the last phase leaves nothing to check"	Seam.done := of I fun (_ : Unit) $\mapsto$ True (:191), equal to Set.univ (probe D.2)	the spine's; same set, different spelling
the hole interfaces	conventions :332	"a phase or generic component is two structures: X.Def (the reduction, its relations and its per-challenge error) and X.Security"	three: Def, Complete, Security extends Complete; the relations are not in Def but arguments of Complete/Security	the spine's
the relation of the first phase	conventions :316	"The relation of the first phase is M3Rel I, that is M3Holds I input q on the stack q"	commitDef I at M3Rel I	agree

Layer 6’s `BusOut`, Layer 7’s types, Layer 9’s `FlockInterface` and Layer 10’s block are the pre-spine sketches; the text around them (:951-952, :983-984, :1103-1106) says the phases are “over `I : M3Instance`” and “the spine’s”, but the blocks were not rewritten. A reader who trusts the blocks builds against types that do not exist. The sibling dossier `gt-table-pub.md` records the same for Layer 7 (its finding *Layer 7’s sketch and the tracker’s signature predate the spine*); we add Layers 6, 9, 10 and the headline.

Proposed change (one for all): replace each pre-spine block by a signature over the spine’s types — `busPhase (I) : Phase.Def I I.Stmt (I.Stmt × BusOut I), busComplete, busSecurity at Seam.commit/Seam.bus; tableSumcheck (I) : Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I); FlockInterface (I) as the pair Phase.Def I (I.Stmt × PubOut I) (I.Stmt × FlockOut I)` with its `Phase.Security` at `Seam.pub/Seam.flock`; `openingPhase (I) : Phase.Def I (I.Stmt × FlockOut I) Unit`; delete Layer 10’s `leanVmPiop ... piopError_le` block and point at the spine, keeping `piopError_le` restated over `I` (section F, first finding).

Acceptance tests 24 to 28 against the code (E.3)

Test	Claim	Status on main
24, the extractor computes	<code>piopExtractor</code> is a definition whose <code>extractOut</code> reads the stack; witness the <code>rfl</code> test	<code>piopExtractor</code> and everything it unfolds to is computable (D.5, <code>isNoncomputable = false</code>); the <code>rfl</code> witness holds for zero-round phases only; a classical phase extractor would go unnoticed and would not matter (D.5)
25, the wall	no module above the adaptor imports <code>LeanerVM.Arithmetization</code>	<code>git grep -n "import LeanerVM.Arithmetization" b435631 -- LeanerVM/Protocol</code> gives two hits: <code>LeanerVM/Protocol/FixedColumns.lean:11</code> (a listed exception, :331) and <code>LeanerVM/Protocol/Basic.lean:3</code> (<code>public import LeanerVM.Arithmetization.Basic</code> , an otherwise empty placeholder module from before the spine, not among the exceptions of convention <i>The wall</i>). The spine’s own five modules and <code>ToArkLib/import nothing from LeanerVM/Arithmetization/</code> . Minor: delete <code>Protocol/Basic.lean</code> or list it.
26, seams are the contract	the compositions typecheck only when the seams agree	true by construction (<code>Phases.Complete.toDef</code> , <code>Phases.Security.toDef</code>); the junk bundle of D.3 (b) also typechecks, so the test says nothing about content
27, the toy is honest	the honest stack passes, each checkable clause fails alone	met by the tests; D.1 adds the multiplicity cases; the fifth clause is untestable on the toy
28, the bus seam bounds the degree	<code>Seam.bus</code> carries <code>totalDegree ≤ I.d</code> , the cubic is rejected	met as a conjunct; its cost is a guard in the table phase (C.3)

8.9.1.2 The public-input phase

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-pubinput.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.2; B (faithfulness), E (the flag `PublicLine.sent`) and F (statement checks) is section 8.9.1.2; C (mutation probes) and D (the top limb) is ?? B.1.1.3; G (findings) is section 8.9.2.2.

The Lean verifier against the specification, the Rust, the Python and the recursion guest (B) All leanVM citations at the pin `a386121f`; Lean at `b435631`.

Item	Specification (08-end-t o-end-protocol.tex)	Rust (crates/lean_vm/src/cpu/mod. rs)	Python (python-verifi er/verifier.py)	Recursion guest (c rates/rec_aggrega tion/guests/aggre gate.py)	Lean (LeanerVM/Protocol/Publi cInput.lean)	Match
challenge	$r_m \in E$, sampled by the verifier (:29; :83 “Sends $\$r_m \backslash \text{in} \backslash E\$$ ”)	let $r_pi = \text{vs.sample}()$; (:745)	public_challenge = tr anscript.sample() (:1398)	fs, rm = squeeze(fs) (:1680)	pSpec := (!v[V_to_P, .P_to_V], !v[E, List E]) (:99): round 0 a challenge in E	yes
prover’s message	c_0, c_1 (:29, :84)	two scalars, $pi_limbs[..2]$ (:747-749)	transcript.next_scala rs(2) (:1399)	mem_lo, mem_hi (:1682-1683)	one message, a List E; its length is fixed by the check to the number of lines with sent (:132-137); two for leanISA’s lines	yes at leanISA’s instance; the type is wider (note in G)
the honest values	“claiming $c_\ell = \text{mem}_\ell(r_m, 0, \dots, 0)$ ” (:29)	computed from the public words: i nterp_k(F64(l.pi[0].c0), F64(l.p i[1].c0), r_pi), limb 1 likewise (:611-613)	(no prover)	(no prover)	expectedValues I st.2.1.1.1 s t.1 (:219): the lines’ values, a function of the statement and the challenge	yes (B.2)
the check	per limb: $c_\ell = (1 + r_m) \cdot \text{mem}[g^0]_\ell + r_m \cdot \text{mem}[g^1]_\ell$ for $\ell \in \{0, 1\}$ (:30-31)	one equation on the words: $pi_limbs[0] + F192::Y * pi_limbs[1] !=$ want with want = interp(l.pi[0], l.pi[1], r_pi), interp(lo, hi, t) = lo + t · (lo + hi) (:752-755; prim itives/src/multilinear.rs:39-42)	poly_eval(public_limb s, Y) == multilinear_ eval(public_input.hal ves(), [public_challe nge]) (:1400)	assert mem == mem _lo + mem_hi * Y_ TOWER with mem = pi_0 + rm * (pi_0 + pi_1) (:1681-1684)	per limb: decide (cs = expect edValues I s.1 r) (:136-137), i.e. $c_\ell = (1 + r) \cdot \text{cell}_0 + r \cdot \text{cell}_1$ for each sent line	specification yes; the three implementa- tions no (B.1)
pooled claims	three, on $\text{mem}_0, \text{mem}_1, \text{mem}_2$ at $(r_m, 0, \dots, 0)$, “with 0 for the top limb” (:84)	three, [MEM_LO, MEM_HI, MEM_TOP] at point[0] = r, values pi_limbs (b ind_pi_claim, :674-682)	three, (MEMORY_0, MEMO RY_1, MEMORY_2) at pub lic_point, values publ ic_limbs (:1401-1402)	three pool entries, the third 0 (:1685-1690)	one per line of I.publicLines, at linePoint l.pos r, value li neValue I r l (:123-128); three for leanISA, the third $(1 + r) \cdot 0 + r \cdot 0 = 0$	yes in number, point and honest value; the pooled value’s provenance differs (B.1)
order in the pool	(not stated for this step; §8.5 Opening gives the ring-switched claim first)	bus claims, table claims, then the three (finish_claims, :656-667), in the order MEM_LO, MEM_HI, MEM_TOP (:680)	claims.extend(...) zip((MEMORY_0, MEMORY_1, M EMORY_2), public_limb s)) after bus and table claims (:1391, :1402)	appended after the table claims (:1685-1690)	s.2.columns ++ (I.publicLines s.1).map (lineClaim I r) (:128): the claims received, then the lines in the instance’s order	yes, given leanISA lists $\text{mem}_0, \text{mem}_1, \text{mem}_2$ in this order (blueprint :834-836, not built)
rejection	“rejecting otherwise” (:29-31 “checks”; :101 accepts if “the public-input line” passed)	return Err(CpuError::PublicInput) (:754)	require(..., “public in put check failed”) (:1400)	assert (:1684)	failure (:233)	yes
error	$1/ E $ “a limb disagreeing with the public words survives with probability at most $1/ E $ ” (:32-33)	(no analysis)	(none)	(none)	error := 1 / Fintype.card E (:111), charged to the one challenge	yes

Which verifier do the theorems describe (B.1) The specification’s, and only it. The three deployed verifiers (the Rust, the Python and the recursion guest all run the combined equation; `verify-gt-table-pub.md` section 7 established the same and it is confirmed above first hand) accept every transcript the Lean verifier accepts, and strictly more: the test at `b435631` (`tests/.../PublicInput.lean:184-210`) pins one transcript accepted by the words’ equation and rejected per limb (`rStar`, `atStar`, `wordsEquation`). The blueprint (`:1053-1062`) says this and assigns the deployed check’s knowledge soundness to Layer 12: “their knowledge soundness at $1/|E|$ is a lemma of its own, and Layer 12 owes it.”

Is Layer 12 the right place, and is “a lemma” the right shape? No, on both counts.

- Layer 12’s theorem is `verify_iff_compiled` (`:1218-1221`): the executable `verify` accepts exactly when the Fiat–Shamir compilation of the oracle verifier accepts. The oracle verifier is the composition of the phases, so its public-input step is *this* verifier. If `verify` is written from the Rust (combined check, values sent pooled), the “iff” is false on the transcripts above. If `verify` is written per limb, `verify_iff_compiled` can hold, but then `verify` is not the deployed verifier, the differential fixture (an honest Rust proof, which passes both checks) cannot tell, and the deployed verifier’s soundness is proved nowhere. In neither case is anything “owed by Layer 12” discharged by a lemma inside it.
- The object that is owed is a `Phase.Security` of a different `Phase.Def`: the deployed verifier reads the same transcript but computes a different verdict and a different pool. Knowledge soundness is a property of the whole reduction against the seams; it is a phase-level object, and the place for it is Layer 8. Its statement, precisely:

```
-- the deployed verifier: the words' equation, the values sent pooled
def deployedVerifier (I) : OracleVerifier [] (I.Stmt × TableOut I) (TheOracle I) (I.Stmt × PubOut I)
  ↪ (TheOracle I) pSpec
  -- verify := do let cs ← queryValues; if checkWords I s r cs then pure (pooledFrom I s r cs) else
  ↪ failure
noncomputable def deployedPhase (I) : Phase.Def I (I.Stmt × TableOut I) (I.Stmt × PubOut I) -- n = 2,
  ↪ err = 1/|E|
def deployedSecurity (I) : Phase.Security I (deployedPhase I) (Seam.table I) (Seam.pub I)
```

with `checkWords` as in C.5 (the words’ equation when exactly two lines are sent, the per-value check otherwise, so that the verifier is total over instances and is the Rust’s on `leanISA`’s) and `pooledFrom` as in C.6. Its completeness is the argument of C.6 and C.5; its knowledge soundness at $1/|E|$ rests on the algebraic lemma proved in C.13 (`accepts_two_challenges`) plus list plumbing.

The alternatives.

- (i) *Model the deployed check in the phase and prove it.* Feasible and cheap: the key lemma is proved (eleven lines), the phase’s other proofs carry over, the estimated total is about 120 lines. Its error is the same $1/|E|$, by the same degree-one argument in E instead of K. It makes the check load-bearing again (C.6: without the check, no `Phase.Security`), and it makes `verify_iff_compiled` provable for a `verify` written from the Rust.
- (ii) *Keep the specification’s check and report upstream.* The specification and the three implementations disagree; the per-limb check is strictly stronger and costs one more field equation; the two are equally sound (the lemma) but not equally strict. This is a leanVM finding in its own right (an ambiguity of the specification rather than a bug: nothing is unsound), to be reported whichever way leanerVM goes. But keeping only the specification’s verifier leaves the deployed one unproved and the refinement theorem unstateable.
- (iii) *Both.* Two `Phase.Defs` over the same schedule, both with `Phase.Security`, and a lemma that the specification’s verifier’s acceptance implies the deployed one’s (the two equations per limb imply the equation on the words; on the values sent, so trivial). The master theorems are instantiated with the deployed phase; the specification’s stays as the documented reference of §8.2.

Recommendation: (iii), with the deployed phase as the one the master theorems and Layer 12 consume, and the divergence reported to leanVM as an ambiguity of §8.2. Reason: the report’s object is deployment on L1; the theorem that matters is about the verifier that runs; the specification’s verifier is one lemma away and documents intent. If only one is kept, keep the deployed one.

The honest prover’s values (B.2) The Lean prover sends `expectedValues` I input `r` (`:219`), the lines’ values $(1 + r) \cdot \text{cell0} + r \cdot \text{cell1}$ computed from the statement. The Rust prover does the same: `interp_k` (`F64(l.pi[0].c0)`, `F64(l.pi[1].c0)`, `r_pi`) with `interp_k(lo, hi, t) = lo + t · (lo + hi)` (`cpu/mod`).

rs:611-613; multilinear.rs:46-48), where `l.pi` is the public input the layout was built from (`layout.rs:83, :471 pi = [exec.mem[0], exec.mem[1]]`). Neither evaluates the committed column. **They agree.** The blueprint says so (:1039-1041).

Does it matter? Not for the theorems: on the accepting path the value is the same as the column’s extension (on an honest stack; the phase’s docstring :40-41). It matters for what the message *is*: a function of public data on both sides, hence redundant for the verifier, hence (C.4) checkable by any check or by none without loss of soundness in the oracle model. The one place it is not redundant is the Fiat–Shamir transcript, where it is absorbed (`ps.add_scalar, :616-618`) and so binds later challenges, though to a value the verifier could derive. The test file’s adversary model, “a prover that sends its true evaluation” (`trueValues, :45-47`), is the right one for the deployed design (values sent pooled) and is what the refutations of C.6 and C.7 use.

The flag `PublicLine.sent` (E) At b435631, Spine/Instance.lean:100-114:

```

/-- A column whose first two cells the public statement fixes (§8.2: the two public words are
the first two memory cells). -/
structure PublicLine (S : Shape) where
  /-- The column. -/
  col : S.ColumnId
  /-- The value cell 0 must hold. -/
  cell0 : K
  /-- The value cell 1 must hold. -/
  cell1 : K
  /-- Whether the proof carries the value claimed for this column on the line through its two
  cells (§8.2: it does for the two low limbs of the memory, and not for the top limb, whose
  value is known to be zero). It fixes the transcript, not the relation. -/
  sent : Bool
  /-- The column has a cell 1. -/
  pos : 0 < S.τ col.1

```

What the theorems say of it. Nothing: `M3Holds` reads `col`, `cell0`, `cell1`, `pos` only (`PublicLinesHold`, D.1), and `publicInputSecurity I` holds for every `I`, so for every assignment of `sent`. Probe C.8 shows the other direction of the same fact: an instance with `sent = false` on its only line is served, and a verifier that drops unsent claims is refuted on it. So `sent` is bound by no theorem; what pins `leanISA`’s choice (`true, true, false`) is (a) the blueprint’s prose (:834-836, :1037-1039) and the status’ decision 15, (b) the definition `leanISAInstance.publicLines` when Layer 3 lands, and (c) Layer 12’s differential fixture, the only thing that would reject a transcript with three or one scalars. Until (b) and (c) exist, the pin is prose.

Does it enlarge what an auditor must trust? By one Boolean per line, whose only effect in the phase as built is on the *transcript’s format* (which values are sent and checked): the pool does not depend on it beyond which claims carry a sent value, and with `pooled` reading the statement, not even that. In the deployed design (values sent pooled) `sent` decides which pooled values are prover-chosen, so it enters the soundness argument (the unsent lines’ claims are at verifier-computed values, and their falsity is a bad challenge on its own). Either way the auditor’s obligation is the same: “`leanISA`’s instance sends exactly the two low limbs”, a transcript fact checked against the Rust (:616-618, :747-749).

Does the specification need the notion? Not as a notion: §8.2 is written for `leanISA`’s three limbs and says “the prover sends `c_0`, `c_1`” and “with 0 for the top limb”. A phase over an abstract instance must say which lines have a scalar; `sent : Bool` is the least it can say.

Simpler designs considered.

- *Every line’s value sent, the third line modelled “as in the specification”.* The specification’s third claim has a constant value and no scalar; sending a scalar for it makes the transcript one scalar longer than `leanVM`’s, so this is not the specification’s protocol. Rejected.
- *A separate list of constant claims* (`publicLines` for the lines with a scalar, and a list “columns whose line is zero” for the rest). Same information, two fields instead of a flag; it hard-codes the constant 0, which is `leanISA`’s accident (cells (0, 0)), not a rule of the protocol. The flag is simpler and more general. Rejected.
- *Provenance in the claim’s type* (a `ColumnClaim` that records whether its value came from the prover). This is the deployed design’s need, and it is met more simply by pooling the values sent (`pooledFrom`) while the claim type stays as it is.

- *Deriving sent from the cells* (send a scalar unless the line’s value is identically zero). A coincidence of leanISA’s data, not a rule; would silently change the transcript for another instance. Rejected.

Verdict: the flag is the right primitive. Its cost is not the flag but the absence of anything that pins it (finding in G).

Statement checks (F) Each declaration read at b435631 against what it should say. The library objects were introduced in C.0.

- `verifier_verify (:253-256)`: as an ordinary verifier, on the transcript `tr`, the verdict is `pure (pooled I s (tr 0), o)` if `check I s (tr 0) (tr 1)` and `failure` otherwise. Reads the challenge at index 0 and the message at index 1 of the transcript, keeps the oracle `o`. Correct, and it is what the two proofs start from through `guarded (:273-276)`.
- `complete (:300-302)`: `perfectCompleteness init impl (Seam.table I) (Seam.pub I)` for every initial state and implementation of the (empty) shared oracle. Direction right (input seam to output seam), no hypothesis beyond the seam, probability one (`perfectCompleteness` is completeness with error 0, `ArkLib Security/Basic.lean:103-105`).
- `rbr (:350-353)`: `rbrKnowledgeSoundnessWorstCaseWith init impl (Seam.table I) (Seam.pub I) (fun _ ↦ Unit) (extractor I) (stateFunction I init impl) (fun _ ↦ error)`. The error function assigns error to every challenge index; there is one (index 0, `pSpec.dir 0 = .V_to_P`), so $1/|E|$ is charged to `r` and nothing to the message round (a message round has no error in this definition). Right challenge.
- *Tightness*. The bound is attained: on the toy at statement 1 with `badLine` (column 2 = [1, 1], outside the table seam), the pooled claim holds at `r = 0` and fails at the sampled `y` (`tests/.../PublicInput.lean:116-118`: `#guard ∀ c ∈ (pooled toy stmt1 0).2.columns, c.Holds badLine` and `#guard ¬ ∀ c ∈ (pooled toy stmt1 y).2.columns, c.Holds badLine`). With at most one bad challenge (`bad_challenge_unique`) and one exhibited, the bad event has probability exactly $1/|E|$ for that stack: the error cannot be lowered. A wrong statement gives the bad challenge `r = 1` (:120-123).
- `stateFunction (:327-346)`: round 0 the table seam; round 1 the public seam of the pool at the challenge; round 2 that and the check. `toFun_empty` is `Iff.rfl` (round 0 is the input relation by definition), `toFun_next` drops the check, `toFun_full` is the guarded verdict lemma. The invariant after the challenge does not mention the message, which is why any message can be accepted (C.3).
- `extractor (:317-321)`: `extractMid, extractOut return ()`. The witness types are `Unit` at every round: the stack is the oracle, and the extraction of the stack is the commit phase’s (`Spine/Compose.lean:58-59`). Correct for this phase; the composition (`piopExtractor`) is what carries knowledge.
- `publicInputPhase (:376-381)`: `n := 2, pSpec, the reduction, err := fun _ ↦ error`. `noncomputable` for the real number only. `publicInputComplete (:384-388)`, `publicInputSecurity (:391-397)`: the two bundles the spine consumes; both quantify over `init impl` inside (the fields `complete`, `kSF`, `rbr` of `Component.Complete/Security`).
- `#print axioms`: not run on the originals after 09:18 (C.9). Run on the verbatim copy `Probe0Baseline.lean` (its diff against the source is the two namespace lines): `publicInputSecurity` and `publicInputComplete` depend on [`propext`, `Classical.choice`, `Quot.sound`], no `sorryAx`: no admitted `ArkLib` theorem is in the closure. The earlier review reported the same for the reviewed names.
- *pooled takes the lines’ values, not the values sent*. In the oracle verifier the pool is computed only on the accepting path, where `cs = expectedValues`, so the two agree there (`pooledFrom_expected`, C.6, proved). Off that path the verifier rejects and pools nothing. For a future executable verifier that reads `pooled` and `check`, they agree as long as it runs *this* check. They differ exactly on the transcripts of B.1 (the words’ equation passes, the per-limb check fails): a `verify` written from the Rust pools the values sent there and accepts, this verifier rejects. So there is a path on which they differ, and it is the set of transcripts on which the two verifiers disagree.

8.9.1.3 Layer 1

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-layer1.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.3; E (audit surface) is

section 10.5.3; F (conformance) is section 8.9.1.3; G (findings) is section 8.9.2.3; I (probes) is ?? B.1.1.5; B (faithfulness), C (statements), D (the wall and the To-folders) and H (the upgrade) is not reproduced in these fragments; they are in the dossier.

The Layer 1 sketch (protocol-blueprint.md:665-731) against the code (F.1) Every declaration of the sketch exists with the sketched type:

Sketch	Code (b435631)	Agreement
Column	Field.lean:67-69	same (Layer 0)
sumCube, hadamard, evalMle_eq_sumCube_hadamard, sumCube_lagrangeBasis, evalMle_replicate, slice, evalMle_append_boolVec, placeSlice, evalMle_placeSlice, sumCube_placeSlice, bitProductTable, evalMle_bitProductTable, powersTable, evalMle_powersTable	Multilinear.lean:67, 70, 91, 125, 131, 307, 342, 351, 371, 381	same
evalMle_lagrangeBasis (w x) : ... = [] k, (...)	BitProductTable.lean:44, 58, 127, 144	same
Blocks, Blocks.Tables, Blocks.stackAt, Blocks.selector, Blocks.extendPoint, Blocks.pow_size_dvd_offset, Blocks.stack_eval, Blocks.unstack, Blocks.unstack_eval ₂	BitProductTable.lean:156-157	the sketch elides the right side ((...))
Blocks.stack_eval_ambient (in mathematical notation)	Stacking.lean:62, 75, 159, 213, 227, 109, 339, 264, 330	same
Blocks.stackColumn, readColumn, readColumn_eval, layout, Layout.comap, stack_eval_ambient_one	AmbientStacking.lean:134-138, through selectorWeight and lowPoint	same meaning
padHigh, sumCube_padHigh, sumCube_prodVars	Stack.lean:70, 73, 78, 97, 52, 118	same (w_b of the sketch is selectorWeight)
eqWeight, ColumnClaim.holds_iff_weighted	Padding.lean:60, 64, 47	same
idxColumn, idxColumn_eval, idxColumnEval_eq, bytecodeColumn, bytecodeColumn_eval	ClaimWeights.lean:45, 58	same
bytecodeColumn_answer_boolVec ... = ofK (encodeSlots (prog. code i))[s]	FixedColumns.lean:48, 60, 66, 79, 114	same
	FixedColumns.lean:97-101, = algebraMap K E (...)	notation: ofK := Ext.ofBase (Parameters/Field.lean:86), equal to algebraMap K E by Extension.Ext.algebraMap_eq_ofBase

In the code, not in the sketch, and appearing in sketched statements or run by a verifier: `Blocks.selectorWeight`, `Blocks.lowPoint`, `Blocks.highPoint`, `Blocks.total`, `Blocks.offset`, `idxColumnEval`, `bytecodeColumnEval`, and `evalMle_padHigh` (the identity the table sumcheck’s final check rests on, B.4). `BlockClaims.lean` is named as a module (:662) and sketches nothing.

The prose (:734-746) agrees with the code: the aligned reader for a table the prover chose is `unstack_eval2`; `stack_eval_ambient` for any pad; back-loaded padding is `placeSlice` at the all-ones index; the index column is the geometric table; the public-input phase’s point $(r, 0, \dots, 0)$ is handled by `evalMle_append_boolVec` at slice zero (as done in `PublicInput.lean:71-89`). The tests paragraph (:748-755) is met item by item (C.5).

Acceptance tests 7, 13, 14, 15 (F.2)

- **7** (:1284-1286): “Lifting table j by $\prod_{k \geq \tau_j} X_k$ sums to the table’s own sum (`sumCube_prodVars`) ... Witness: `tableSummand_target` on heights 2 and 1.” The statement it describes is `sumCube_padHigh`, not `sumCube_prodVars` (which is $\sum x_0 \dots x_{m-1} = 1$); and `tableSummand_target` does not exist (Layer 7 is not built). The existing witness is `tests/.../Padding.lean:44-49` (the copied table sums to 0 in characteristic two).
- **13** (:1303-1305): the named witness exists (`tests/.../FixedColumns.lean:37-47`) and fails on the reversed order. Met.
- **14** (:1306-1307): “Witness: `bytecodeColumn_slot` on a two-instruction program.” The theorem exists but restates the definition and holds of the opposite layout by the same proof (C.3); the witness that pins slot 3 = $(1, 1, 0, 0)$ is `bytecodeColumn_answer_boolVec` and the oracle `#guards` at `tests/.../FixedColumns.lean:82-93`. Stale.
- **15** (:1308-1310): “Blocks sit at offsets that are multiples of their size, largest first; `stack_eval` fails for any other placement. Witness: three blocks of sizes 4, 2, 1 with the small one first.” Two imprecisions: 4, 2, 1 are the *heights* (the sizes, in the code’s sense of variables, are 2, 1, 0); and

“fails for any other placement” is false: sizes 1, 1, 2 in that order sit at offsets 0, 2, 4, all aligned (DuplicatesProbe), and any permutation of equal sizes is aligned too, which is exactly why the tie order needs its own guard (B.2). The witness exists (tests/.../Stack.lean:87-95).

The interface list (:1386-1406) (F.3)

- Every Layer 1 name listed exists: generic Column sumCube hadamard slice placeSlice boolVec evalMle_eq_sumCube_hadamard evalMle_append_boolVec evalMle_placeSlice bitProductTable evalMle_bitProductTable powersTable evalMle_powersTable Blocks Blocks.Tables Blocks.stack At Blocks.selector Blocks.extendPoint Blocks.stack_eval Blocks.stack_eval₂ Blocks.unstack Blocks.unstack_eval₂ Blocks.stack_eval_ambient; leanVM Blocks.stackColumn Blocks.readColumn Blocks.readColumn_eval Blocks.layout Layout.comap Blocks.stack_eval_ambient_one padHigh sumCube_padHigh prodVars sumCube_prodVars eqWeight ColumnClaim.holds_iff_weighted BlockClaim idxColumn idxColumn_eval idxColumnEval_eq bytecodeColumn bytecodeColumn_answer_boolVec bytecodeColumn_eval.
- Mislabelled: Column (over K, Layer 0) and Weight, WeightedClaim (spine objects over E, Seams. lean:103-124, listed a second time under “Spine”, :1370) are listed under “Protocol (generic)” (:1386, :1395).
- Missing, although “Everything not listed is a proof, a helper, or a test” (:1419): idxColumnEval, bytecodeColumnEval (the evaluators the verifier runs; right sides of listed theorems), evalMle_padHigh, Blocks.selectorWeight, Blocks.lowPoint, Blocks.total, Blocks.offset (in listed statements).
- Listed and to be removed if proposal E.1 is taken: BlockClaim.

Elsewhere in the blueprint, touching Layer 1 (F.4)

- Layer 3 (:808): theorem leanIsaInstance_fits : (leanIsaInstance prog s).layout.total ≤ 2 ^ (leanIsaInstance prog s).μ. Layout (Spine/Instance.lean:73-81) has no total; total is a field of Blocks. Moreover Blocks.layout takes the fit as an argument (hμ), so the fit must be proved *before* leanIsaInstance can be defined, not after it. Finding G.2.
- Layer 3 (:841-848): “Layout admits both readers; Blocks expresses only the first, so leanIsaInstance.layout combines the two”. True of the types, but neither the strided selection lemma nor the combinator exists or is sketched in any layer (B.8, G.1).
- Convention *Stacks* (:322) states the tie order and μ_stack ∈ [15, 28]; both agree with the Rust (witness.rs:67-79; pcs.rs:49-51, cpu/mod.rs:174) and the Python (py:305-311, 890, 907-908, 1379). Convention *Hypercube* and eq (:312, 314) agree with B.1 and B.7.
- Convention *The wall* (:331) and acceptance test 25 (:1341-1345) do not list Protocol/Basic.lean (D.1, G.8).

8.9.2 Findings on the built code

Every finding of the three code dossiers, with the severity each dossier gives it, its evidence, its classification where the dossier gives one, and its proposed change. Where the dossier gives the passage as it stands and as proposed, the change is set in a box.

8.9.2.1 Findings on the spine

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

Severities as the brief defines them. Line references to the blueprint are to `docs/roadmap/protocol-blueprint.md` at b435631.

Finding 89 (major). The declared error is unconstrained: the knowledge theorem has content only with a bound on piopError

Severity: major (a statement cannot be stated as written: the headline’s “except with probability piopError I” has no counterpart)

Evidence. piopError P := P.toDef.err (Compose.lean:149); Component.Def.err : pSpec.ChallengeIdx → ℝ≥0 (Component.lean:66), declared by each phase; Def.append keeps each component’s declaration (Component.lean:148). ArkLib’s bound is $\Pr[\dots] \leq \text{err } i$ (RoundByRound.lean:568) with no constraint on err. Probe P4Junk.lean (D.3 (b)): five phases that draw a challenge and check nothing, each declaring error 1, inhabit Phases.Security toy; both master theorems hold of them; piopError_junk : $\forall i, \text{piopError junkPhases } i = 1$. The construction is instance-independent. The blueprint’s only bound, piopError_le (hs : s.Admissible prog) : $\sum i, \text{piopError } s \ i \leq 2^{40} / |E| + \text{flockError} (:1130)$, is over an argument s the spine’s piopError does not take, and the convention row Errors (:318, “the closed form is a def next to the theorem, and the interactive error is its sum”) names no theorem.

Classification. an error of the blueprint (an omission the spine inherited).

Where the bound belongs. Two designs are consistent with the spine; the second is recommended.

- *A field of Phases.Security.* Add to Phases.Security (or to a new structure Phases.Error) a closed form bound : ℝ≥0 and a proof err_le : $\sum i, \text{piopError } P \ i \leq \text{bound}$, and restate the master theorem’s docstring with it. Cost: one field, and each phase’s Security must come with a bound on its own D.err (a lemma per phase, summed by Def.append). It keeps the errors declared by the phases, so a phase author still writes the closed form, and the bound is a hypothesis of the composition, not a consequence.
- *A closed form fixed by the spine.* Make the error a function of the **schedule**, not a declaration: Component.Def.err is removed; Component.Security carries err with its rbr; the spine states, per phase slot, the closed form leanVM’s analysis gives $(4 \cdot 2^{\{\mu_{\text{bus}}\}} / |E|$ on (α, β) and gkrError for the bus, $(B+2)/|E|$ and $3/|E|$ per round for the table sumcheck, $1/|E|$ for the public input, flockError, $(J-1)/|E|$ and $2/|E|$ per round for the opening; blueprint Layers 6 to 10), as a function piopError I : ChallengeIdx → ℝ≥0 of the instance’s sizes, and Phases.Security demands each phase’s rbr at that error. Then piopError I exists as the headline writes it, piopError_le is a theorem of the spine about the instance’s sizes (provable now, by norm_num at the caps once leanIsaInstance’s sizes exist), and a phase that cannot meet its slot’s error has no Security. Cost: the schedules must then also be fixed by the spine (the error is indexed by the challenge), which the blueprint’s convention *Holes* leaves to each phase’s Def; this is the same move the sibling dossier recommends for BusOut’s shape, and Layer 12’s verify needs the schedules fixed anyway.

Proposed change.

Proposed change: The declared error is unconstrained: the knowledge theorem has content only with a bound on piopError

As it stands. (:318) “Errors | ℝ≥0, per verifier message, as ArkLib’s rbrKnowledgeError; the closed form is a def next to the theorem, and the interactive error is its sum.”

As proposed. “Errors | ℝ≥0, per verifier challenge, as ArkLib’s rbrKnowledgeError. The spine fixes the closed form of every phase’s error as a function piopError I of the instance’s sizes, and Phases.Security demands each phase’s round-by-round bound at that error; piop_rbrKnowledgeSoundness is stated at piopError I, and piopError_le bounds its sum at the caps. A phase may not declare its own error: with an unconstrained declaration the knowledge theorem holds of a protocol that checks nothing (declared error 1).”

Also :1122 (def piopError (s : Sizes)) and :1130 to be restated over I; and item 6 of *What the spine fixes* (:431-437) to say that piopError is the spine’s, not the phases’.

Finding 90 (minor). The headline describes theorems that do not exist

Severity: minor (stale and imprecise text at the most-read place)

Evidence. :11-17 against `Compose.lean:162-185` (C.4): no `piopError I`; no “except with probability” (the theorem is round-by-round, the plain form is `ArkLib’s sorry` at `Implications.lean:223-228`); no “the extractor’s column `q`” (no such definition, D.5); the hypotheses `P`, `C`, `S` absent; `Set.univ/Seam.done` unmentioned. “States the two theorems that make it a proof system for the constraint relation” (:8-9): on `main` they make *any* five phases meeting the seams a proof system for `M3Rel I`, for any `I`.

Classification. an error of the blueprint.

Proposed change.

Proposed change: The headline describes theorems that do not exist

As it stands. (:11-17) the two pseudo-signatures.

As proposed.

```

piop_perfectCompleteness (I : M3Instance) (P : Phases I) (C : P.Complete) :
  the honest prover of P convinces its verifier with probability 1 on every (input, q)
  with M3Holds I input q
piop_rbrKnowledgeSoundness (I : M3Instance) (P : Phases I) (S : P.Security) :
  P's verifier is round-by-round knowledge sound for M3Rel I with the extractor
  piopExtractor P S at the error piopError I, so that (ArkLib's round-by-round-to-plain
  implication, admitted at the pin) an accepting prover's committed q satisfies
  M3Holds I input q except with probability  $\Sigma$  piopError I

```

followed by: “Both are composition theorems: they hold of every bundle of phases that meets the seams, and say nothing of `leanVM` until `I` is `leanIsaInstance` and `P` is the five phases of Layers 6 to 10 with their proofs. What is proved on `main` is the spine, the commit phase and the public-input phase.”

Finding 91 (minor). The extracted stack is not a definition, and the rfl witness of acceptance test 24 covers zero-round phases only

Severity: minor (an acceptance test whose witness does not test what it says; avoidable confusion about `piopExtractor`)

Evidence. D.5: `P3cExtractor.lean`. With the repository’s `publicInputPhase` in the bundle, `(piopExtractor P S).extractOut` has type `S.pub.witMid (Fin.last 2) = Unit` and the test’s statement is ill-typed; the stack is `extractMid` at round 0. `ArkLib’s RoundByRound.append` (`Append/StateFunction.lean:129-139`) returns the second extractor’s `extractOut` whenever the second component has a round. The blueprint (:1337-1339): “the spine’s test proves by `rfl` that `piopExtractor` returns the committed stack on the honest transcript, whatever the phases’ extractors”.

Classification. an error of the blueprint.

Proposed change. Code (spine): define `piopExtractedStack (P) (S) : (I.Smt ×  $\forall i$ , NoOracle i) → P.toDef.pSpec.FullTranscript → Column I.μ`, the fold of `extractOut` and the `extractMids` down to round 0 (cast by `eqIn`), and prove `piopExtractedStack P S s tr = tr {0, _}` for every `P`, `S`, `s`, `tr` (the commit phase’s `extractMid` ignores what it is handed, `SendOracle.lean:137`): a theorem for all phases, not a `rfl` on five empty ones. The tests’ `rfl` example is then a corollary.

Blueprint :1333-1340:

Proposed change: The extracted stack is not a definition, and the rfl witness of acceptance test 24 covers zero-round phases only

As it stands. Witness: the spine’s test proves by `rfl` that `piopExtractor` returns the committed stack on the honest transcript, whatever the phases’ extractors

As proposed. Witness: `piopExtractedStack_eq`, for every bundle and every transcript; the composed `extractOut` is the last phase’s and is not the stack

. Same at :100-101 and :1133-1135 (“reads the stack off that message” is true of `extractMid` at round 0).

Finding 92 (major). The degree conjunct of Seam.bus is a guard the table phase's verifier must run

Severity: major (a `Phase.Security` cannot be proved for a verifier that transcribes leanVM's table sumcheck)

Classification. an error of the blueprint; the sibling dossier `gt-table-pub.md` (section 6, E.1 (a); section 8, first finding) establishes it with a probe on which a degree-blind verifier accepts, and proposes the type-level shape. **We agree**, and record only the mechanism in `ArkLib`'s definition (C.3, item 1: $\forall \text{ stmtIn with toFun_empty an iff}$) and the smallest fix if the sibling's `BusOut` is not adopted: make `VirtualTerm.poly` a subtype $\{p : \text{CMvPolynomial } (I.\text{width } j) \text{ K} // p.\text{totalDegree } e \leq I.d\}$ and delete the conjunct; the seam then has four conjuncts, the test `cubicTerm` becomes a typing failure, and acceptance test 28 is met by the type.

Finding 93 (minor). The outputPure field is redundant, and guarded nearly so

Severity: minor (avoidable audit surface)

Evidence. D.7, `outputPure_of_def`: every `Component.Def` over $[]_0$ has a pure prover output, because no query is possible. `ArkLib` at the pin has the instance `Prover.instOutputIsPureEmpty` (`NoAmbient.lean:39-42`), not imported by the spine, and `Verifier.GuardedForm.ofEmpty` (`NoAmbient.lean:46-55`): every verifier over $[]_0$ has a guarded form, given a fallback verdict for the rejecting branch. The field `outputPure` is one of the three of `Component.Complete` (`Component.lean:79-80`); every phase discharges it by $(_, \text{fun } _ \mapsto \text{rfl})$ (`PublicInput.lean:386`, `SendOracle.lean:78`, `PassThrough.lean:85`).

Classification. a deviation forced by an upstream library, retired when the pin includes a module the spine can import.

Proposed change. Code: delete the field; `Complete.append` takes the instance. Keep guarded: the composed extractor names `G.out` (`Component.lean:178`), and a hand-written verdict is what an auditor reads, where `ofEmpty`'s is "run the verifier". Blueprint :505-507 and the sketch accordingly.

Finding 94 (note). The instance is data the theorems trust, and nothing on main says any instance but the toy has an inhabited relation

Evidence. C.1 (aux), C.2 and D.4 (a `Layout` may alias columns; `toyAlias` has an empty relation and is an `M3Instance`); `Layout.read_eval` is the only law an instance obeys. Nothing in the spine relates two instances or bounds what an instance may declare; the master theorems hold of every instance including the empty ones.

Classification. a deliberate deviation (the wall), to be kept, with the theorem it owes named.

Proposed change. Blueprint, *What the spine fixes* item 1 (:412-414): append "An instance is trusted data: the spine proves nothing about it beyond `read_eval`, and the master theorems hold of instances whose relation is empty. For `leanISA` the instance is `Category B` (layouts, separators, count columns, lines) and its non-vacuity is `m3Holds_stackOf` (Layer 3), its faithfulness Layer 12's fixture."

Finding 95 (minor). read is determined by extend

Severity: minor (avoidable audit surface; a field with a law where a definition would do)

Evidence. C.2, on paper (the extension of a table at a cube point is its entry). A probe would be `Layout.read_eq_of_extend`; not run.

Classification. a design remark.

Proposed change. Code: structure `Layout` (μ) (ι) (κ) with `extend` and one law `extend_inK` : $\forall c \times q, (\tilde{q} \text{ (extend } c \text{ (bits } x))) . \text{IsInK}$ (or the equivalent on the `eq` weights); `read` a definition. Every phase reads columns through `I.column` and is unchanged. The toy's `slice` becomes a theorem. Blueprint :447-450 accordingly.

Finding 96 (note). Knowledge transport along the adaptor is three lines in the event form

Evidence. D.7, `knowledge_transport`; blueprint :582-586.

Classification. an error of the blueprint (a claim that something is not proved and has no consumer,

where it is provable now and Layer 13 consumes it).

Proposed change. Blueprint :582-586: “The transport of knowledge along the adaptor is proved in the event form: for the same game, provers and extractor, the probability that the verifier accepts and the extracted witness, mapped by the refinement, is invalid is at most the knowledge error (`Refinement.knowledge_transport`, an `ArkLib` candidate). It needs no prover conversion, since the game’s witness type is unchanged. Layer 13 consumes it. What T4 waits on is the round-by-round-to-plain implication (ledger row *round-by-round implies plain*), whose existential conclusion also forgets the named extractor; a `With` form is to be requested upstream.” Code: move the probe’s theorem into `ToArkLib/Refinement.lean`.

Finding 97 (note). Two statements the earlier review’s disposition recorded as met are met only in part

Evidence. `docs/reviews/protocol-spine.md:36` says the extractor test “proves by `rfl` that `pioExtractor ...` returns the honest stack ... for any `A` and `S`” (true for zero-round phases only, above); `:39` says “No `trivSecurity ...` was added, so the `rfl` test quantifies over `S`” — the reason there is no `trivSecurity` is that none exists (`no_security`, D.3 (a)), which the review’s A6 reads as a property of the toy (“True on the toy”) without a proof. No change to the blueprint; the review file is a handoff.

8.9.2.2 Findings on the public-input phase

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-pubinput.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.2; B (faithfulness), E (the flag `PublicLine.sent`) and F (statement checks) is section 8.9.1.2; C (mutation probes) and D (the top limb) is ?? B.1.1.3; G (findings) is section 8.9.2.2.

Names in words; the earlier documents’ codes in parentheses only where they locate a passage.

Finding 98 (major). The check on the public-input message is not load-bearing for any theorem of the phase (G.1)

Evidence. C.3 (the verifier with no check has `Phase.Security` at the same error, no `sorryAx`), C.4 (for every message function and every check the message passes, `securityG`; instances with no check, with forty-two zeros, with the swapped check and a matching prover), C.5 (the first-value check; the words’ equation). Root cause: `pooled` (`PublicInput.lean:127-128`) reads the statement, not the message; the message enters `check` and nowhere else; the proof of `rbr` (`:350-366`) never mentions `check`. The build the earlier review read `pooled` the values sent, and its item “H1: one output, `pooled`” (`docs/reviews/public-input-phase.md:552-573`) removed the dependence, so the same review’s table “each of five wrong verifiers breaks a stated theorem” (`:104-114`, rows (i) and (v)) stopped being true of the code as merged; the status repeats the claim (`protocol-status.md:440`).

Consequence for the requirement. “A verifier that omits a check leanVM makes, or makes a weaker one, must have no proof of knowledge soundness” fails for this check, and no theorem about this phase as designed can restore it: the verifier without the check is knowledge sound. “A verifier that makes a wrong check must have no proof of completeness” holds only against a fixed honest prover (C.10); with the prover adapted to the wrong check both theorems hold.

Classification. An error of the design (the blueprint’s Layer 8 text describes a phase whose check ties the pool to the statement; the code’s does not), introduced by an audit-surface compression.

Proposed change. Pool the values sent: replace `pooled` by `pooledFrom` (C.6, the definition is in `tools/lib.py`, with `pooledFrom_expected` proved), in `verifier`, `prover.output`, `guarded.out` and the state function’s rounds 1 and 2 (round 1 becomes $\exists cs, check\ s\ r\ cs \wedge pooledFrom\ s\ r\ cs \in Seam.pub$; with the per-limb check `cs` is forced to `expectedValues`, so `bad_challenge_unique` is unchanged). Then the check is load-bearing: its removal is refuted (C.6), its weakening is refuted (C.7), and a wrong check breaks completeness for every prover (C.10, last paragraph). Blueprint :1035-1037

Proposed change: The check on the public-input message is not load-bearing for any theorem of the phase

As it stands. For a line whose value was sent the value pooled is the value sent, since the check has just passed; for the others it is the value the verifier computes.

As proposed. For a line whose value was sent the value pooled is the value sent, as §8.2 and the pinned verifiers pool it; for the others it is the value the verifier computes. The check is what makes the pooled claims the statement's: without it the prover's values are pooled unchecked and the phase has no knowledge-soundness proof (test); a verifier that pools the values it computes would be knowledge sound with no check at all, which is why the pool must read the message.

Status :440: strike “each of five wrong verifiers breaks a stated theorem” or requalify it as a property of the build reviewed, not of the code merged. Tests: add, next to `threeLimbs`, the refutations of C.6 and C.7 as theorems (they are twenty lines each with the two lemmas of C.1, which belong in `ToArkLib/`).

Finding 99 (major). The theorems are about a verifier none of the three executable verifiers runs, and the debt is assigned to a layer that cannot pay it (G.2)

Evidence. B.1: the specification checks per limb (`08-end-to-end-protocol.tex:30-31`); the Rust (`cpu/mod.rs:752-755`), the Python (`verifier.py:1400`) and the recursion guest (`aggregate.py:1681-1684`) check $c_0 + y \cdot c_1 = w_0 + r \cdot (w_0 + w_1)$ and pool the scalars sent; the Lean is the specification's. The blueprint assigns the deployed check's soundness to Layer 12 (`:1053-1062`), whose theorem `verify_iff_compiled` (`:1218-1221`) cannot hold with a `verify` written from the Rust, and cannot detect the difference with a `verify` written per limb (the fixture is an honest proof). The status finding (F18) records the divergence correctly and draws the same wrong assignment (`:330`).

Classification. A disagreement between the specification and its three implementations (a leanVM finding: an ambiguity of §8.2, not an unsoundness: the lemma of C.13 proves the implementations' check sound at $1/|E|$); the blueprint follows the specification, as a deliberate deviation from the deployed verifier, and owes the deployed verifier's theorem in the wrong place and in the wrong shape.

Proposed change. Layer 8 builds the deployed phase as well (B.1: `deployedVerifier`, `deployedPhase`, `deployedSecurity`, with `checkWords` of C.5 and `pooledFrom` of C.6; key lemma `accepts_two_challenges` proved in `ProbewordsLemma.lean`), and the master theorems and Layer 12 consume it; the specification's phase stays with a one-line lemma that its acceptance implies the deployed one's. Blueprint :1061-1062

Proposed change: The theorems are about a verifier none of the three executable verifiers runs, and the debt is assigned to a layer that cannot pay it

As it stands. So the pinned verifiers accept transcripts this verifier rejects, their knowledge soundness at $1/|E|$ is a lemma of its own, and Layer 12 owes it.

As proposed. So the pinned verifiers accept transcripts this verifier rejects. Their verifier is a second `Phase.Def` on the same schedule (`deployedPhase`: the equation on the two words, the values sent pooled), with its own `Phase.Security` at $1/|E|$ (the two low limbs as one line through `E`, `accepts_two_challenges`); `Phases` and Layer 12 take the deployed phase, and the specification's is kept as the reference of §8.2 with the lemma that its acceptance implies the deployed one's.

Report to leanVM: §8.2 states two equations, the three implementations check one; both sound; choose.

Finding 100 (minor). The top limb is checked by the instance's third line and enforced by the adaptor's theorem, which is not built; the acceptance test names the wrong witness (G.3)

Evidence. D.1: every theorem of the phase and the spine is stated over an arbitrary `M3Instance`, so an instance with two lines has them all; `SatisfiedBy.word0_eq` (`Statement.lean:338`) with `PublicInput.word0 = E.ofLimbs input_0 input_1 0` (`Memory.lean:107`) requires the top cell to be zero, and `satisfiedBy_witness0f` (blueprint :815-816) is what would be false without the third line;

constraintSoundness and baseVerifier_extractsExecution follow. D.2: acceptance test 10's witness exists (tests/.../PublicInput.lean:151-180) but shows the pool, not the theorem, and on a toy-shaped instance.

Classification. Not a divergence; an incomplete audit trail in the blueprint.

Proposed change. Acceptance test 10 (:1293-1295)

Proposed change: The top limb is checked by the instance's third line and enforced by the adaptor's theorem, which is not built; the acceptance test names the wrong witness

As it stands. omitting it lets a non-canonical public word pass the line. Witness: the Layer 8 mutation with a nonzero mem_2 at cell 0 and the third claim dropped.

As proposed. omitting it lets a non-canonical public word pass the line. The phase and the spine cannot notice: an instance listing two lines has every one of their theorems. What fails is the adaptor's satisfiedBy_witness0f, at SatisfiedBy.word0_eq, whose right-hand side input.word0 has a zero top limb by type (Memory.lean:107). Witnesses: the Layer 8 test threeLimbs (the third claim rejects the stack badTopLimb; without it the stack passes), and, when Layer 3 lands, a #guard that (leanIsaInstance prog s).publicLines input lists mem_2 with cells (0, 0) and sent = false.

Finding 101 (minor). PublicLine.sent is pinned by prose only (G.4)

Evidence. E: the theorems hold for every assignment; leanISA's (true, true, false) is in the blueprint's prose (:834-836, :1037-1039) and decision 15, with no definition (Layer 3) and no transcript fixture (Layer 12) yet.

Classification. Avoidable audit surface until Layer 3 and 12 land.

Proposed change. In the convention row "Public input" (:335) add: "The choice is pinned by two tests: Layer 3's #guard on leanIsaInstance.publicLines, and Layer 12's differential fixture, which reads exactly two scalars (cpu/mod.rs:747-749)."

Finding 102 (note). The message type is wider than the transcript (G.5)

The message is a List E whose length only the check fixes (:96-99, :136-137); the pinned transcript carries exactly two scalars. Layer 12 must serialize the sent values with no length prefix, which the type does not say (the earlier review's item A4, kept). With the deployed phase (G.2) the check on the length is the words' equation's pattern match, which must also reject a wrong length (C.5's checkWeak does, through its second branch).

Finding 103 (minor). Status text stale (G.6, known)

protocol-status.md at b435631 still calls the phase "draft #60" (:5-6, :101, :119, :146), merged as b435631; and :440 as in G.1. The brief's section 8 says the status did not change at 144c5aa.

Negative results of the public-input dossier

- The schedule, the challenge, the number of scalars, the point (r, 0, ..., 0), the honest values, the three claims and their order, the rejection, and the error agree between the specification, the three implementations and the Lean (table in B); checked line by line as cited there.
- The bit order of linePoint is proved, not assumed (evalMle_linePoint), and separated from the other order by the test on a two-variable table (tests/...:60-66).
- 1 + r versus 1 - r: joined by CharTwo.sub_eq_add (:182), correct in characteristic two.
- The error is charged to the right challenge and is tight (F).
- No admitted ArkLib theorem in the closure of the two bundles (F, on the verbatim copy).
- The knowledge state function's three rounds are the right ones and none is trivial (F; the bad event is inhabited, C.3's rbr' shows the same bound for the unchecked verifier).
- sent is no part of the relation (E; M3Holds does not read it).

8.9.2.3 Findings on Layer 1

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-layer1.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.3; E (audit surface) is section 10.5.3; F (conformance) is section 8.9.1.3; G (findings) is section 8.9.2.3; I (probes) is ?? B.1.1.5; B (faithfulness), C (statements), D (the wall and the To-folders) and H (the upgrade) is not reproduced in these fragments; they are in the dossier.

Finding 104 (major). The strided reader of the eighteen BLAKE2S limb columns exists nowhere and is assigned to no layer (G.1)

Severity: major (the blueprint omits something leanVM does, and `leanIsaInstance` cannot be built without it).

Evidence. leanVM reads the limb columns as strided slots of `q_flock` (`pcs/src/stack_open.rs:84-97`; `py:884-894`, `Placement(..., QFLOCK_SLOT_BITS)` with `low = 8`; `lean_vm/src/hash_flock.rs:87-115`; `gt-flock-ring.md` 2.4, 5.4). Layer 1 has only high-index selection (`slice`, `evalMle_append_boolVec`, `unstack`), and says so (`ToCompPoly/Multilinear.lean:50-51`). `Layout.comap` cannot place a limb column: its height 2^{τ_B} is no block’s (B.8). The blueprint’s Layer 3 (:841-848) says the instance “combines the two” readers but no layer sketches the selection lemma, the strided reader or the combinator; the slot map is placed in Layer 9’s `FlockInterface.limbColumns` (`gt-flock-ring.md` 8.2). `StridedProbe` proves the missing generic lemma in twenty lines from Layer 1’s own lemmas.

Classification. an error of the blueprint (omission).

Proposed change. In the Layer 1 sketch, after `evalMle_append_boolVec`, add:

```
def sliceLow (t : CMLPolynomialEval R (k + m)) (i : Fin (2 ^ k)) : CMLPolynomialEval R m
theorem evalMle_boolVec_append (t) (i) (s) : evalMle t (boolVec i ++ s) = evalMle (sliceLow t i) s
-- Stack.lean
def Blocks.stridedLayout (hμ) (b : Fin B.n) (k : ℕ) (slot : ι → Fin (2 ^ k))
  (h : ∀ c, B.size b = k + κ c) : Layout μ ι κ -- column c = sliceLow of block b at slot c
def Layout.pieces (L1 : Layout μ ι1 κ1) (L2 : Layout μ ι2 κ2) : Layout μ (ι1 ⊕ ι2) (Sum.elim κ1
  ↪ κ2)
```

and in the prose: “leanVM reads the eighteen BLAKE2S limb columns as strided slots of `q_flock`: the low eight coordinates of `q_flock`’s block are frozen to the slot’s bits (`stack_open.rs:84-97`). `evalMle_boolVec_append` is the low-index selection identity; `Blocks.stridedLayout` packages it as a `Layout`, and `Layout.pieces` joins it to the aligned layout; Layer 3 transports the sum type to `ColumnId`.” Tests: the slot’s strided slice against the aligned slice of the same index; a limb claim’s lifted point against `Placement.stack_point` of `py:295-297` on the layout of `offsets.py`. Reason: without it the instance’s layout cannot be written, and the Rust’s `Strided` claims have no Lean counterpart.

Finding 105 (major). `leanIsaInstance_fits` cannot be stated as written, and is needed before the instance, not after (G.2)

Severity: major by the scale’s letter (a statement that cannot be stated as written); the fix is one line.

Evidence. `protocol-blueprint.md:808`: theorem `leanIsaInstance_fits` : (`leanIsaInstance prog s`).`layout.total` ≤ 2^{τ_B} (`leanIsaInstance prog s`).`μ`. `Layout` has the fields `read`, `extend`, `read_eval` only (`Spine/Instance.lean:73-81`); `total` belongs to `Blocks` (`Stacking.lean:87`), and `Blocks.layout` takes the fit as its argument `hμ` (`Stack.lean:97`), so the instance’s `layout` field cannot be filled until the fit is proved.

Classification. an error of the blueprint.

Proposed change.

Proposed change: `leanIsaInstance_fits` cannot be stated as written, and is needed before the instance, not after

As it stands. `theorem leanIsaInstance_fits : (leanIsaInstance prog s).layout.total ≤ 2 ^ (leanIsaInstance prog s).μ`

As proposed. `def leanIsaBlocks (prog) (s) : Blocks (the sorted sizes of the committed columns); def leanIsaμ (prog) (s) : ℕ := max 15 (Nat.clog 2 (leanIsaBlocks prog s).total) (witness.rs:97, py:890); theorem leanIsaBlocks_fits : (leanIsaBlocks prog s).total ≤ 2 ^ leanIsaμ prog s, the hμ of leanIsaInstance's layout`

Reason. the only fit that exists is a `Blocks`'s; the order of definitions must follow.

Finding 106 (minor). The order of equal-size blocks is pinned by no statement, definition or test before the compiled verifier (G.3)

Severity: minor (the blueprint states the order correctly; nothing checks it until Layer 12).

Evidence. `Blocks` is the sorted sizes (`Stacking.lean:62-68`); the column-to-block map is the `f` of `Layout.comap` (`Stack.lean:52`), constrained only to keep heights. `OffsetsProbe`: leanVM's order and the "tables first" order give the same `Blocks` and different offsets per column (`mem_0` at 32 against 64); every Layer 1 theorem holds of both, and so would every Layer 3 theorem (a bijective `f` passes the round trip). The earlier review marked the matter "Met in the documentation ... the instance (I2) supplies it" (`docs/reviews/protocol-layer1.md:34`). Layer 3's text promises a transcription and decided offsets (`:865-866`) but no test on a configuration where a table height equals `κ_mem`. The leaf stacks have the same gap (`gt-bus.md G7`).

Classification. an error of the blueprint (a check deferred with no witness named).

Proposed change. Add to Layer 3's tests (`:866-867`)

Proposed change: The order of equal-size blocks is pinned by no statement, definition or test before the compiled verifier

As it stands. Tests: a witness with one row per table stacked and read back ...; `Sizes.Admissible` rejects `logMem = 15` and `τ_BLAKE2S = 2`

As proposed. and the offsets of every committed column on the two configurations of the Layer 1 review's `OffsetsProbe` (sizes `[4,4,4,4,2,5,4,4]`, and `log_memory = 16`, `τ = (3, 16, 0, 5, 16, 3)`, `κ_bc = 4`, whose 92 offsets are the pinned Python's `build_layout`), where a table's height equals `κ_mem`, so that the tables-first order fails them.

Reason. the order decides every selector, and only this guard can catch a wrong one before a Rust proof does.

Finding 107 (minor). Acceptance test 14 names a tautology as its witness; `bytecodeColumn_slot` and its tests prove nothing about the layout (G.4)

Severity: minor .

Evidence. `C.3: bytecodeColumn_slot` (`FixedColumns.lean:84-87`) unfolds the definition, and the same proof proves the opposite layout's analogue (`ValuesProbe`, `wrongColumn_slot`). Still cited: `protocol-blueprint.md:1306-1307`, `protocol-status.md:51`. Its three test uses (`tests/.../FixedColumns.lean:61-74`) instantiate it. The repository's bytecode-column tests use all-1 operands (`:56-59`), so they cannot see an operand-slot swap; no repository test compares `bytecodeColumn` with leanVM's encoder.

Classification. an error of the blueprint (stale witness), and avoidable audit surface.

Proposed change. Acceptance test 14

Proposed change: Acceptance test 14 names a tautology as its witness; bytecodeColumn_slot and its tests prove nothing about the layout

As it stands. Witness: `bytecodeColumn_slot` on a two-instruction program.

As proposed. Witness: `bytecodeColumn_answer_boolVec`, and the oracle `#guards` of `tests/LeanerVMTests/Protocol/FixedColumns.lean` (slot bits `(1,1,0,0)` answer the opcode, `(0,0,1,1)` answer 0), and a `#guard` of `bytecodeColumn` on a sixteen-instruction program with distinct operands against the table of `leaf.rs:585-604`.

Reason. the witness must fail on the wrong layout, and the operand order needs one independent check.

In the code: delete `bytecodeColumn_slot` and its three examples; add `ValuesProbe`'s `prog16/pyTable` `#guard` (rewritten with `K.ofBits` for the new `CompPoly` pin, H).

Finding 108 (minor). Acceptance test 15 overclaims (G.5)

Severity: minor .

Evidence. `protocol-blueprint.md:1308-1310`: “`stack_eval` fails for any other placement. Witness: three blocks of sizes 4, 2, 1 with the small one first.” Sizes 1, 1, 2 in that order are aligned (`DuplicatesProbe`); any permutation of equal sizes is aligned; “4, 2, 1” are heights, the code’s sizes are 2, 1, 0.

Classification. an error of the blueprint.

Proposed change.

Proposed change: Acceptance test 15 overclaims

As proposed. Selector alignment. Blocks placed largest first sit at offsets that are multiples of their heights (`pow_size_dvd_offset`); with the smallest block first no selector reads it. Largest first is sufficient, not necessary, and says nothing of the order of equal sizes (acceptance test on the tie order, Layer 3). Witness: three blocks of heights 4, 2, 1 with the small one first (`tests/LeanerVMTests/Protocol/Stack.lean`).

Reason. the current wording suggests the descending order pins the placement, which is the reading that hides G.3.

Finding 109 (minor). The interface list omits load-bearing Layer 1 names and mislabels three (G.6)

Severity: minor .

Evidence. F.3: `idxColumnEval`, `bytecodeColumnEval`, `evalMle_padHigh`, `Blocks.selectorWeight`, `Blocks.lowPoint`, `Blocks.total`, `Blocks.offset` are not listed, although :1419 says everything unlisted is a proof, helper or test; `Column`, `Weight`, `WeightedClaim` are listed as generic.

Classification. an error of the blueprint.

Proposed change. In “Protocol (generic)” (:1386-1391) replace `Column` by nothing (it is Layer 0’s, listed under “Protocol (leanVM)” with `evalOracle`) and add `Blocks.total` `Blocks.offset` `Blocks.lowPoint` `Blocks.selectorWeight` (and `sliceLow evalMle_boolVec_append` if G.1 is taken); drop `Weight` `WeightedClaim` from :1395; in “Protocol (leanVM)” (:1403-1406) add `evalMle_padHigh`, `idxColumnEval`, `bytecodeColumnEval`. In the Layer 1 sketch add `evalMle_padHigh` and the two evaluators’ definitions. Reason: these are what a verifier runs or what listed statements mention.

Finding 110 (minor). BlockClaims.lean has no consumer (G.7)

Severity: minor (audit surface).

Evidence. A.8, E.1: `git grep -w BlockClaim b435631 -- LeanerVM tests` finds its module and its test only; `isValid_iff_pairing` restates `unstack_eval_eq_sumCube` (`BlockClaims.lean:31`, “this module only names its arguments”); the opening phase uses `ColumnClaim.holds_iff_weighted`.

Classification. avoidable audit surface.

Proposed change. Delete `LeanerVM/Protocol/BlockClaims.lean` and its test; remove it from the module list of Layer 1 (:662, “`BlockClaims.lean` (claims on aligned blocks)”) and `BlockClaim` from the interface list (:1404). Reason: 8 public declarations, 40 lines, no use.

Finding 111 (minor). `Protocol/Basic.lean` imports the arithmetization and is not a listed exception of the wall (G.8)

Severity: minor .

Evidence. D.1: `LeanerVM/Protocol/Basic.lean:3`, `public import LeanerVM.Arithmetization.Basic`; an empty placeholder imported by `LeanerVM.lean` only. Acceptance test 25 (:1341-1345) says the grep lists the wall’s exceptions only.

Classification. an error of the blueprint (the witness text is false), and dead code.

Proposed change. Delete `LeanerVM/Protocol/Basic.lean` (and its import in `LeanerVM.lean`), or drop its import. Reason: acceptance test 25’s witness becomes true as written.

Finding 112 (minor). Acceptance test 7 names the wrong lemma and a witness that does not exist (G.9)

Severity: minor .

Evidence. F.2: “sums to the table’s own sum (`sumCube_prodVars`)” is `sumCube_padHigh`; `tableSummand_target` is a Layer 7 name, not built.

Classification. an error of the blueprint.

Proposed change.

Proposed change: Acceptance test 7 names the wrong lemma and a witness that does not exist

As proposed. ... sums to the table’s own sum (`sumCube_padHigh`); lifting by nothing (copying) multiplies it by $2^{(\tau_{\max} - \tau_j)}$, which is 0 in characteristic two when $\tau_j < \tau_{\max}$. Witnesses: `tests/LeanerVMTests/Protocol/Padding.lean` (the copied table), and `tableSummand_target` on heights 2 and 1 once Layer 7 lands.

Reason. the witness must exist.

Finding 113 (note). `Layout.comap` accepts a non-injective renaming (G.10)

(C.4, `ValuesProbe`’s `aliased`). Completeness of the adaptor catches aliasing; no change is required. Optional: state `Function.Injective f` for `leanIsaInstance`’s `map` in Layer 3.

Finding 114 (note). Unused or duplicated declarations (G.10)

(`prodVars` and its two lemmas, `unstack_eval`, `stack_eval_ambient_zero`, `stackColumn_eval`, `stackColumn_eval_ambient`, `bytecodeSlotColumn`, `slice_bytecodeColumn`, `evalMle_lagrangeBasis` against `CompPoly`’s `eqTilde_eq_prod`): E, proposals 2 to 5.

Finding 115 (note). `idxColumnEval` is efficient (G.10)

in `CompPoly`’s `K` (binary exponentiation, B.5); no change.

Finding 116 (note). Disagreements between leanVM’s sources (G.10)

, recorded, not new: the specification’s $M = \lceil \log_2 N \rceil$ has no floor or ceiling, the Rust and the Python bound the witness stack to [15, 28] (B.2; the blueprint follows the Rust and the Python); the specification gives no tie rule, both verifiers break ties by column index (B.2; the blueprint follows them); the Rust evaluates the program’s share per column, the Python as one stacked evaluation (B.6, `gt-bus.md` G15; equal on Lean’s column).

Finding 117 (note). The upgrade to 144c5aa (G.10)

leaves every Layer 1 statement and definition unchanged and preserves the meaning of every Layer 1 test (H).

8.10 The libraries and Layer 0

8.10.1 Findings on the libraries and on Layer 0

No finding of this section is a defect of a leanerVM theorem: nothing in the libraries' objects or in Layer 0 lets an unsound verifier be “proved”. The fields are leanVM's bit for bit, the samplers are uniform by proof, the oracle answers the specification's evaluation, the counting bounds are correct and non-vacuous, and the trusted base is the kernel plus the three standard axioms, enforced transitively. The findings concern what ArkLib's notions do and do not guarantee, what the blueprint plans on, and the blueprint's text.

Finding 118 (critical). The existential form of round-by-round knowledge soundness has no knowledge content

Critical as a warning about the definition; not a defect of a leanerVM theorem.

Evidence. `Extractor.RoundByRound` (`Security/RoundByRound.lean:77-86` at `dca90385`) is a structure of unconstrained functions; the probe `Extractors` builds `chooser R`, which selects a witness by `Classical.choice`, and proves `chooser_sound : rbrKnowledgeSoundnessWorstCaseWith ... (chooser R) ... (fun _ => 0)` for the verifier that decides the language of any relation `R`. ArkLib's own `toRoundByRoundOfRel` (`:118-123`) does the same (section 3.6.7).

Classification. A deviation forced by an upstream library; ArkLib has no notion of running time.

Consequence for leanerVM. `piop_rbrKnowledgeSoundness_exists` and any theorem stated with `rbrKnowledgeSoundness` or `rbrKnowledgeSoundnessWorstCase` (the blueprint's Layer 9 `FlockInterface.rbrKnowledgeSoundness : ... rbrKnowledgeSoundnessWorstCase ...`, `protocol-blueprint.md:1084`, and its Layer 10 sketch, `:1127-1129`) are soundness statements, not knowledge statements. The named form the spine uses is adequate only together with a reading of the extractor.

Proposed change. In the blueprint's row for `rbrKnowledgeSoundness` (`:239`) and in the wrong-readings item 24 (`:1333`), add: “ArkLib's extractor type accepts a classical choice (ArkLib's `toRoundByRoundOfRel` is one); a theorem in the existential forms `rbrKnowledgeSoundness` or `rbrKnowledgeSoundnessWorstCase` therefore proves soundness only. Every interface and theorem of this roadmap is stated in the `With` form with a compiled extractor, and the compiled status is checked by a `def` without `noncomputable` at each phase's `Security`.” Change `FlockInterface.rbrKnowledgeSoundness` (`:1084`) to the `With` form with named extractor and state function.

Finding 119 (major). The spine's completeness cannot carry an error, and Flock may need one

Evidence. `Component.Complete.complete` is `perfectCompleteness` (`LeanerVM/Protocol/ToArkLib/Component.lean:84-85`); the blueprint's `FlockInterface` demands `perfectCompleteness` (`protocol-blueprint.md:1083`) and item 20 of the wrong readings (`:1323-1325`) charges the Flock inverse's failure at $r_{eq} = 1$ to `flockError`, a soundness error. The pinned Rust prover takes the inverse (`crates/flock/src/zerocheck.rs:117`), which is 0 at $r_{eq} = 1$ (`crates/primitives/src/field/gf2_64x3.rs:144-153`). ArkLib at both pins composes completeness with additive errors (`Reduction.append_completeness_of_guarded_verifiers`, `GuardedCompleteness.lean:161-173`, and the oracle and n -ary forms), proved (section 3.6.8).

Classification. An error of the blueprint, if the honest Flock prover fails at $r_{eq} = 1$, which the Flock review must confirm; otherwise a note.

Proposed change. Add a field `err : ℝ ≥ 0` to `Component.Complete` (or to `Component.Def`) with `complete : ∀ init impl, D.red.completeness init impl relIn relOut err, compose by OracleReduction.append_completeness_of_guarded_verifiers (err1 + err2)`, state `piop_completeness` with the sum and `piop_perfectCompleteness` as its corollary when every phase's error is 0; give `FlockInterface` a completeness error; and rewrite item 20:

Proposed change: item 20 of the wrong readings (protocol-blueprint.md:1323-1325)

As it stands. “Perfect completeness needs no exceptional-challenge clause in Layers 4–8 and 10: no inverse of a challenge is taken. Flock’s $(1 + r_{eq})^{-1}$ at $r_{eq} = 1$ is #3’s, inside `flockError`. Witness: `piop_perfectCompleteness` has no side condition on challenges.”

As proposed. “No phase of Layers 4–8 and 10 takes an inverse of a challenge; the Flock phase’s completeness error is its own field and is summed.”

Reason. A completeness failure cannot be charged to `flockError`, which is a soundness error; if the Flock phase has a completeness error, `FlockInterface`’s `perfectCompleteness` field is unsatisfiable and `piop_perfectCompleteness` is vacuous for the real protocol.

Finding 120 (major). The blueprint plans on a framework ArkLib calls legacy, and on a pull request that may not merge

Evidence. ArkLib’s `AGENTS.md` at both pins calls `ArkLib/OracleReduction/` “legacy”; issue #676’s refreshed scope directs composition work to the typed framework; `docs/design/05-roadmap.md` at 7653a901; the typed framework has no knowledge soundness, no round-by-round notion and no extractor; pull request #615 is open and conflicting; no relevant admission was lifted between the pins (sections 3.7.4 and 3.7.5). The blueprint’s ledger says of the knowledge-soundness composition (ledger row A2) “the port is deleted ... when the pin moves past #615” (:260, :1143, :1429) and of the round-by-round to plain implications (A3) “the plain corollary is stated once the implication lands upstream” (:261).

Classification. An error of the blueprint’s plan.

Proposed change.

Proposed change: the ledger’s actions for rows A2 and A3 (protocol-blueprint.md:260-261)

As it stands. A2: “the port is deleted ... when the pin moves past #615”. A3: “the plain corollary is stated once the implication lands upstream”.

As proposed. A2: “kept until ArkLib, on whatever framework, proves a guarded-first append for the worst-case named form; if the legacy framework is not repaired, the port stays as a `ToArkLib` candidate”. A3: “the plain corollary is stated from the transcript-level reading (the probe `PlainReading` proves it in 40 lines) and a union bound proved locally, since upstream has no plan to prove `rbrKnowledgeSoundness_implies_knowledgeSoundness`”. Add a ledger row: “typed Interaction framework: no knowledge soundness at 7653a901; migration of the spine is not possible until it has one”.

Reason. Upstream’s stated direction is migration, not repair of the legacy admissions.

Finding 121 (major). Fiat-Shamir and BCS: ArkLib has no statement to serve as the witness obligation

Evidence. The blueprint’s ledger row A5 (:263) plans “Named interfaces `FiatShamirSecurity` and `BcsSecurity` with the upstream theorem as witness obligation”. At both pins: `fiatShamir_completeness (FiatShamir/Basic.lean:163-173)` is admitted **and** initialises the challenge oracle to the constant function `fun {i, _} => default`, so it is not a random-oracle statement and is false in general for a protocol with a non-zero completeness error whose honest prover fails at the default challenge (the dossier’s inference, not machine-checked); Fiat-Shamir soundness is a `TOD0` (:175); the round-by-round to state-restoration statements are commented out (`Implications.lean:230-254`); `BCSTransform` is commented out (`BCS/Basic.lean`); `Commitment.extractability` is `False` under quantifiers (`Commitments/Functional/Basic.lean:250-255`). Also, `Verifier.fiatShamir (FiatShamir/Basic.lean:130-136)` applies to a plain `Verifier` whose messages are whole; on `toVerifier` of the IOPP it would hash entire oracle messages, so the blueprint’s `verify_iff_compiled` sketch, “`Verifier.fiatShamir (leanVmIopp ...).verifier` accepts ... under the BLAKE2s challenge oracle” (:1218-1221), names an ArkLib definition that does not describe leanVM’s compiled verifier (Merkle roots in the transcript, openings in the proof).

Classification. An error of the blueprint.

Proposed change.

Proposed change: ledger row A5 (protocol-blueprint.md:263) and the `verify_iff_compiled` sketch (:1218-1221)

As it stands. “Named interfaces `FiatShamirSecurity` and `BcsSecurity` with the upstream theorem as witness obligation.”

As proposed. State that no ArkLib theorem, at either pin, states Fiat–Shamir or BCS knowledge soundness, that `fiatShamir_completeness` as stated is not the random-oracle model, and that `FiatShamirSecurity` and `BcsSecurity` are assumed interfaces whose witness obligation is the literature (Block et al., ePrint 2023/1071, Theorem 3.15, section 3.6.4) until ArkLib states them. Drop `Verifier.fiatShamir` from `verify_iff_compiled`, or apply it to a verifier of the compiled (Merkle-root) protocol.

Reason. There is no upstream theorem to serve as the witness obligation, and the named definition does not describe the compiled verifier.

Finding 122 (minor). The error bundled in `Component.Def` makes every phase and the composed protocol noncomputable

Minor (auditability).

Evidence. `publicInputPhase` is noncomputable “because of the error alone” (`PublicInput.lean:374-381`); the probe `ExtractorsExpectedFailure` shows that the verifier and the extractor reached through the phase’s definition are rejected by the compiler. `leanVmVerifier P := P.toDef.red.verifier` and `piopExtractor P S` take the bundle `P` as an argument, so on the real protocol neither is compiled; the blueprint promises “the honest prover is computable” (protocol-blueprint.md:1145) and “the extractor computes” (:1333) and plans code-generation probes (section 3.6.7).

Proposed change. Move `err` out of `Component.Def` into the `Security` structure (it is a property of the soundness proof, not of the reduction), or keep the reduction in a separate computable structure that `Def` extends; then `leanVmPiop`, `leanVmVerifier` and `piopExtractor` compile.

Finding 123 (minor). Line references and names in the blueprint’s ArkLib table

Evidence. Probe `AxiomsArkLibBuilt` (two “unknown constant” errors) and the row-by-row check of table 3.7: `MvPolynomial.schwartz_zippel_counting` does not exist (it is `schwartz_zippel_counting`, root namespace); `prob_eval_zero_le_div` is a PMF statement, not a VCVio one; `Sumcheck.Domain` is `SumcheckDomain`; `Direction` is in `Prelude.lean`; `fsChallengeOracle` is in `ProtocolSpec/Basic.lean:854`; `OracleInterface`’s answer is at :76; the theorem the spine uses for completeness, `OracleReduction.append_perfectCompleteness_of_guarded_verifiers`, is not in the table; `CheckClaim`’s oracle variant checks nothing at run time; the sumcheck specification’s perfect completeness is admitted too (ledger row A1).

Classification. An error of the blueprint.

Proposed change. The corrected entries of table 3.7, in the blueprint’s table at protocol-blueprint.md:232-251.

Finding 124 (minor). The status’s dates for ArkLib #615 and #818 are last-update dates

Evidence. The status file says “open since 2026-09-08” for #615 (created 2026-07-07) and “since 2026-09-04” for #818 (created 2026-08-31); read with `gh api`, read-only, on 2026-09-29 (table 3.9).

Classification. An error of the status.

Proposed change. Give the opening dates, or say “last updated”.

Finding 125 (minor). The relation ladder conflates the field sum with Clean’s balance relation

Evidence. protocol-blueprint.md:359 at b435631 (:363 at 144c5aa, same text). The probe `CleanBalance` (section 3.3.5): `pushedTwice_balance` (the field sum of `[push 7, push 7]` is 0) but `pushedTwice_not_balanced` (Clean’s `BalancedInteractions` rejects it, through the side con-

dition), and `length_le_one_of_balanced`, `honest_pair_not_balanced`, `statement_forces_at_most_one_interaction`.

Classification. An error of the blueprint (imprecise wording; the conclusion “unusable here” is right).

Proposed change.

Proposed change: the relation-ladder row on Clean’s balance (protocol-blueprint.md:359)

As it stands. “Over $\mathbb{K}$ the side condition fails for every real ensemble and a tuple pushed twice and never pulled balances (#16).”

As proposed. “Over $\mathbb{K}$ (`ringChar K = 2`) the field sum alone accepts a tuple pushed twice and never pulled ($1 + 1 = 0$), and the side condition `length < ringChar K` that is meant to exclude it admits at most one interaction per channel, so `Ensemble.Statement` holds of no ensemble with a real bus: Clean’s soundness is vacuous and its completeness false over $\mathbb{K}$ (#16).”

Reason. The reader should see that Clean’s relation is unsatisfiable, not unsound.

Finding 126 (minor). `SampleableType K` is unused audit surface, and the blueprint disagrees with itself about it

Evidence. No consumer outside `Protocol/Field.lean`; leanVM samples only $\mathbb{E}$ (`crates/ fiat_shamir/ src/ lib.rs:95-99`); at the old pins it is definitionally the library’s `FinEnum.SampleableType K` (probe `SamplerDiamond2`; section 3.8.2). The blueprint’s Layer 0 sketch declares it (`protocol-blueprint.md:634`) while its dependency table says “Layer 0 supplies the $\mathbb{E}$ instance” (`:251`) and its interface list names only `instSampleableTypeE (:1400)`.

Classification. Avoidable audit surface (not a divergence from leanVM).

Proposed change. Keep the instance only as the building block of the $\mathbb{E}$ sampler (VCVio’s `Vector` instance needs `SampleableType K`; at the new pins no library instance exists) and say so.

Proposed change: the Layer 0 sketch (protocol-blueprint.md:634) and the dependency row (:251)

As it stands. `instance : SampleableType K -- a uniform Fin (2^64) read as a bit pattern`

As proposed. `instance : SampleableType K -- a uniform Fin (2^64) read as a bit pattern; only the building block of the $\mathbb{E}$ sampler: every leanVM challenge is in $\mathbb{E}$ (fiat_shamir/src/lib.rs:95). At :251: “Challenge sampling; Layer 0 supplies the $\mathbb{E}$ instance (built from a K instance)”. The same comment on the instance’s docstring in Protocol/Field.lean:80.`

Reason. The sketch, the dependency row and the interface list should say the same thing.

Finding 127 (minor). `evalOracle_answer` takes n explicitly

Evidence. Code, `Protocol/Field.lean:109`: `theorem evalOracle_answer (n : ℕ) (q : Column n) (r : Vector E n)`; blueprint, `:641`: `theorem evalOracle_answer (q : Column n) (r : Vector E n)`.

Classification. A code/blueprint mismatch, cosmetic.

Proposed change.

Proposed change: `LeanerVM/Protocol/Field.lean:109`

As it stands. `theorem evalOracle_answer (n : ℕ) (q : Column n) (r : Vector E n) :`

As proposed. `theorem evalOracle_answer {n : ℕ} (q : Column n) (r : Vector E n) :`

Reason. n is determined by q . (Alternatively add `(n : ℕ)` to the blueprint; the code change is preferred.)

Finding 128 (minor). The scalar oracle interfaces are trusted surface the blueprint does not list

Evidence. `instOracleInterfaceE` and `instOracleInterfaceListE` (`Protocol/Field.lean:115, :118`) decide what a verifier may learn of a scalar or list the prover sends (the whole message, query ()); they appear in neither the Layer 0 sketch (`protocol-blueprint.md:633-650`) nor the interface list (`:1400`).

Classification. An omission of the blueprint.

Proposed change. Add to the Layer 0 sketch after `evalOracle_answer: instance : OracleInterface E := OracleInterface.instDefault -- a scalar message is read whole` and `instance : OracleInterface (List E) := OracleInterface.instDefault -- likewise a coefficient list`, with the sentence “ArkLib registers no default interface; delete these when it does (the test `#synths` will turn ambiguous).” Add both names and `instSampleableTypeK` to `:1400`.

Finding 129 (minor). Layer 0 prose is stale after the Lean 4.34.1 upgrade

Minor, at 144c5aa.

Evidence. At 144c5aa: `protocol-blueprint.md:633-635` still says “Add ArkLib at dca90385 ... ArkLib requires CompPoly at the v4.33.1 tag; the root pin 3468b38c wins”; `:649-650` “The sampler is the point at which a wrong Fintype would enumerate 2^{64} or 2^{192} elements (finding P3)” while Fintype K is now proof-only and the eager instance gone (section 3.2.6); `:574` “no LawfulBEq E at the pin” while CompPoly 572f9973 has LawfulBEq (Ext P) (Extension/Arithmetic.lean:380); `:285` says CompPoly has no $\text{GF}(2^8) \hookrightarrow E$ embedding, true, but the AES field $\text{GF}(2^8)$ itself has landed (CompPoly/Fields/Binary/Aes/). The disclaimer added at `:52-54` covers “dependency API tables and signatures”, not these sentences.

Classification. Stale text (a deviation forced by an upstream library that has since been lifted).

Proposed change.

Proposed change: four passages of the blueprint at 144c5aa

As it stands. `:633-635`: “Add ArkLib at dca90385 ... the root pin 3468b38c wins”. `:649-650`: “The sampler is the point at which a wrong Fintype would enumerate 2^{64} or 2^{192} elements (finding P3)”. `:574`: “no LawfulBEq E at the pin”. `:285`: no $\text{GF}(2^8) \hookrightarrow E$ embedding.

As proposed. `:633-635`: “ArkLib is the Arklib requirement; current pins in `upstreams.json` and `docs/dependencies.md`.” `:649-650`: “At CompPoly 3468b38c a sampler through Fintype would enumerate the field (leanISA finding P3); since CompPoly #331 Fintype K is proof-only (Fintype.offFinite) and the samplers go through explicit equivalences.” `:574`: re-check LawfulBEq E at the new pin and, if found, drop the restriction to $\mathbb{K}$ -coefficient polynomials or keep it deliberately with a reason. `:285`: “... no embedding $\text{GF}(2^8) \hookrightarrow K$ (CompPoly now has the AES field $\text{GF}(2^8)$, #356, and its embedding into GHASH only)”.

Reason. The pins moved; the prose describes the old ones.

Finding 130 (note). Component.Complete's two side conditions are derivable

Over `[]`, `Verifier.GuardedForm.ofEmpty` and `Prover.instOutputIsPureEmpty` (`Composition/Sequential/NoAmbient.lean:39-55`, both pins) give guarded and outputPure for every component; leanerVM proves them by hand (`PublicInput.guarded`, `sendVerifierPure`). Keeping the hand-written guarded forms is defensible (the check is readable), but the fields could be defaults, shrinking what a phase author must supply.

Finding 131 (note). Locally re-implemented objects that ArkLib has

`Component.passThrough` (`ToArkLib/PassThrough.lean`) is ArkLib’s `ReduceClaim.oracleReduction` with the identity oracle map (`ReduceClaim.lean:282-323`, proved completeness and existential round-by-round knowledge soundness, `:441`); `Component.sendOracle` (`ToArkLib/SendOracle.lean`) is `SendSingleWitness.oracleReduction` (`SendWitness.lean:395-443`, whose completeness is admitted at both pins and whose knowledge soundness is only in the coordinate-wise special-soundness form). The local versions are justified by the With form with a guarded form as data; the doc-

string of `SendOracle.lean:6` says so for `SendSingleWitness`, the one of `PassThrough.lean` does not name `ReduceClaim`. `ToVCVio/UniformSample.lean`'s bound for a subsingleton `bad` set has no `ArkLib` counterpart in `VCVio`'s vocabulary (`ArkLib`'s `prob_eval_zero_le_div` is on PMF).

Finding 132 (note). The eager `Fintype BF64` blocked every executable at the old pins

Note; retired.

Evidence. section 3.2.3; `docs/roadmap/leanisa-status.md:753-769` at `b435631`; `CompPoly #331 (290c351)`, in the new pin.

Classification. A deviation forced by an upstream library (no executable verifier, no differential tests, a library-only test driver); its retirement condition, “`CompPoly` stops threading [`Fintype F`] through `Ext` and `Fintype BF64` is not evaluated at startup”, is **met at 144c5aa** (section 3.2.6). Whether lake test can again be an executable is for the owner to check.

Finding 133 (note). `simp` misreads $\mathbb{K}$ arithmetic (old pins)

Note; retired.

Evidence. The probe `NumeralsHazard` (appendix B): example : $(1 : \mathbb{K}) + 1 = 0 := \text{by simp}$ leaves $\vdash \text{False}$ and the kernel rejects the term (application type mismatch ... $(2\#64 = 0\#64) = \text{False}$); theorem `bad` : $\neg ((1 : \mathbb{K}) + 1 = 0) := \text{by simp}$ and theorem `bad_mul` : $(2 : \mathbb{K}) * 0x8000000000000000 = 0 := \text{by simp}$ are *elaborated* by `simp` (core's `BitVec` simplification procedures read $\mathbb{K}$'s $+$, which is XOR, as addition modulo 2^{64}) and **rejected by the kernel** ((kernel) declaration type mismatch), exit 1. So a false statement about $\mathbb{K}$ was not proved: the kernel is the safety net, and the axiom audit is a second one (after the kernel error Lean adds `bad` as an axiom for error recovery, ‘`bad`’ depends on axioms: [`bad`, `propxt`, `Quot.sound`], which `audit-axioms.lean` would reject). Recorded as `leanISA` status finding E6 at `b435631`. Companion: `CompPoly`'s instances apply to every `BitVec 64` (probe `FieldFidelity`: $(3 : \text{BitVec } 64) + 5 = 6\#64$); no `leanerVM` misuse (section 3.2.3).

Classification. Forced by an upstream library (abbrev `BF64 := BitVec 64`); **retired** at the new pin by `CompPoly #329 (BF64 a structure)`. In its place, numerals in $\mathbb{K}$ now mean their characteristic-two value (finding 134).

Finding 134 (note). Numerals in $\mathbb{K}$ changed meaning at the new pins

Note, for every later probe and fixture.

Evidence. $(2 : \mathbb{K})$ is x at `CompPoly 3468b38c` and 0 at `572f9973`; words are `K.ofBits n` at `144c5aa`. A literal written for the old pin compiles at the new one with another value (section 3.2.6). Pull request #61 converted `leanerVM`'s fixtures; one comment is left behind: `tests/LeanerVMTasks/Protocol/Field.lean:25-26` and `:37-38` at `144c5aa` still say “The column [1, 2, 3, 4]” and “ $1 + y \cdot (1 + 2) = 1 + 3y \dots$ with $-1 = 1$ and $2 = x$ ” while $2 : \mathbb{K}$ is now 0.

Classification. An upstream change adopted; a documentation hazard.

Proposed change.

Proposed change: the comments of `tests/LeanerVMTasks/Protocol/Field.lean:25-26, 37-38` (at `144c5aa`)

As it stands. “The column [1, 2, 3, 4]”; “ $1 + y \cdot (1 + 2) = 1 + 3y \dots$ with $-1 = 1$ and $2 = x$ ”.

As proposed. “The column [1, x , $x + 1$, x^2] (words 1, 2, 3, 4 as `K.ofBits`) ...”; “at $(y, 0)$ the value is $1 + y \cdot (1 + x)$ ”.

Reason. $2 : \mathbb{K}$ is 0 at the new pin. Any future probe of this review run at the new pins must write words as `K.ofBits` (the probe `FieldFidelity` would otherwise fail spuriously).

Finding 135 (note). `ArkLib`'s bridge `eval_eq_MvPolynomial_MLE` is likely unimportable at the new pins

Note; unverified.

Evidence. A duplicate `CompPoly.CMLPolynomialEval.eval_zero` in `CompPoly 572f9973` (`Multilinear/Basic.lean:719`) and `ArkLib 7653a901` (`ToCompPoly/Multilinear/Basic.lean:45`), whose own `CompPoly` pin `df591bb8` lacks it (section 3.2.6). Not run (no Lean in this checkout).

Classification. Forced by upstream version skew.

Proposed change. Before a phase imports `ArkLib.ToCompPoly.Multilinear.Basic`, elaborate a file importing it and `CompPoly.Multilinear.Basic`; if it fails, file the clash upstream (`ArkLib` should drop its `eval_zero` once its `CompPoly` pin has one) and cite the blueprint’s bridge (`protocol-blueprint.md:246`) with that caveat.

Finding 136 (note). The local counting bound is redundant with `VCVio` at the new pin

Evidence. `SampleableType.prEvent_uniformSample_le_div_iff` (`NativeMeasure.lean:127` at `VCVio a4232d08`; section 3.8.5).

Classification. A duplicate of upstream after the pin moved.

Proposed change. Replace `probEvent_uniformSample_le_of_card_le` by the upstream equivalence (plus `ENNReal.coe_div`); keep `..._of_subsingleton` (no upstream form) as the one local lemma, or upstream it, per the blueprint’s generic-code rule (`protocol-blueprint.md:329`).

8.10.1.1 Negative results

Checked, and no issue found:

- The `ArkLib` definitions `KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCaseWith`, `perfectCompleteness`, `GuardedForm`, `OutputIsPure` were read in full at `dca90385`; no way was found to satisfy `rbrKnowledgeSoundnessWorstCaseWith` at error 0 for a verifier that accepts a false statement, other than through the extractor (finding 118) or through an empty output-witness type (which makes the output relation empty and is visible in the statement).
- The port `KnowledgeAppend.lean` was diffed line by line against #615’s file: only the differences listed in section 3.7.4.
- The composed error of the port is per challenge with no additive term, and the seam round is handled (`backward_right`, `extractMid_seam`); read in full, no gap found.
- Every `#print axioms` run reports the three standard axioms only for `leanerVM`’s declarations.
- $\mathbb{K}$, $\mathbb{E}$, γ , `ofK`, $\mathbf{g}$ against the specification, the Rust and the Python: moduli, bit order, limb order, embedding, generator, product formula (table 3.1; the probe `FieldFidelity`: $3 + 8$ reference vectors, the defining relation, inverses, two mutations).
- Irreducibility, field structure, cardinalities: proved in `CompPoly` without `native_decide` (Rabin certificate checked by `rfl`; standard axioms, probe `Layer0`).
- Uniformity of both samplers; the diamond on $\mathbb{K}$ is definitional (section 3.8.2).
- `evalOracle`: query, answer, lift, the reason for the structure (section 3.8.3); `card_E` (section 3.8.4); `NoOracle`, `OneOracle`, `noOracle_eq` (trivially correct; `ArkLib`’s `Fin 0` interface instance matches `NoOracle` at both `ArkLib` pins).
- The counting bounds: correct, tight, hypotheses load-bearing; the same statement before and after the probability-API port (section 3.8.5).
- Axiom enforcement: lexical and transitive, with a negative control (section 3.4.3).
- Clean’s vacuity over $\mathbb{K}$ (section 3.3.5), `leanISA`’s replacement (section 3.3.7), and its unchanged state at Clean `42fe4b26` (section 3.3.8).

Chapter 9

Non-vacuity

A definition is non-vacuous when it is inhabited by the object `leanVM` intends and refuted by the near misses; a theorem, when its hypotheses can be met by the real protocol and not only by a trivial one, and when what it concludes is not already true of everything. This chapter records what the review established by probe, in Lean, against `main` at `b435631`, and what it established on paper. The probes are reproduced in appendix B; the rule of the review was that a conclusion resting on a probe names it.

9.1 The relation and the seams

M3Holds. On the toy instance the honest stack satisfies it, and for each of the four checkable clauses a stack fails that clause alone: the repository’s own tests do this (`tests/LeanerVMTTests/Protocol/Spine.lean`). The review added the cases those tests lack (probe `P2Relation`): a tuple pushed twice and pulled once fails `Balanced`, and a tuple pushed twice and never pulled fails it too, which a balance summed in the field would accept; the side filter and the row ranges of `tuples` were exercised on instances built for the purpose. Clause by clause against the pinned specification, `M3Holds` is the relation the bus, the table sumcheck, the public-input check and Flock together establish (chapter 8); the review found no clause wrong.

The seams. Each seam was inhabited and refuted per conjunct (probe `P3aSeams`); the zerocheck escape, a constraint violated on the cube whose extension vanishes at the point (1), was exhibited on a stack `badConstraint` to show that the bus seam’s claims can hold of a stack outside `M3Holds`, which is what the error of the bus phase pays for. `Seam.done` is `True`, so the last law of a knowledge state function reads “if the verifier accepts then the state is true” (probe `NonVacuity`); this is what an output relation carrying nothing should give.

What the seams admit beyond `leanVM`. `Seam.bus` admits statements the deployed bus phase never emits and the deployed table sumcheck cannot serve: a statement true of an honest stack yet outside the seam by its degree clause alone; a linear claim with two terms of one table at two points; a thousand copies of a true claim (probes `SeamBusShape`, `SeamBusMember`). This is not a vacuity of the seam but a burden on the phase that consumes it, developed in section 8.5.

9.2 The hypotheses of the master theorems

The two master theorems take a bundle of phases with their proofs, `Phases.Complete` and `Phases.Security`. Whether these can be met, and by what, is the question.

They can be met by the real phases. For the bus phase, the table sumcheck and the public-input phase the review worked out, on paper, a knowledge state function and per-challenge errors that meet `Phase.Security` against the spine’s seams (chapter 5); the public-input phase’s is built and proved. For Flock, perfect completeness is attainable by the honest prover of the specification, and the blueprint’s

belief that it is not (its acceptance test 20) is mistaken (section 8.6). For the table sumcheck the answer is “yes, by a generalized phase with guards leanVM does not have”, which is the finding of section 8.5. For the opening, see section 8.7.

They can also be met by protocols that are not leanVM’s. Three constructions, two of them machine-checked:

1. **Phases that check nothing, at declared error one.** A phase that draws one challenge, ignores it, makes no query and outputs a function of its input statement is perfectly complete whenever that function carries one seam into the next, and is round-by-round knowledge sound against *any* two seams at the error it declares for its challenge, namely 1. Five of them inhabit `Phases.Security I` for every instance I ; both master theorems hold of the bundle; the composed error is 1 at each of its five challenges (probe `P4Junk`, standard axioms only). The knowledge-soundness master theorem therefore has content only together with a bound on `piopError`, and the spine states none: `piopError P` is `P.toDef.err`, whatever the phases declare. The blueprint’s `piopError_le` (Layer 10) is stated over sizes the spine’s `piopError` does not take. This is the review’s first major finding on the spine (section 8.3); the proposed repair is a closed-form error fixed by the spine from the instance and demanded of each phase.
2. **Phases that read the whole stack.** In the ideal model the verifier may query $\tilde{q}$ at every point of the cube, and neither `ArkLib` nor the spine counts queries; `M3Holds` is decidable. So a phase with no round whose verifier reads the 2^μ cells and decides the input seam itself is perfectly complete and knowledge sound at error zero. Five of them inhabit `Phases.Security I` for every I with `piopError = 0`. On paper (two dossiers reach it independently); a probe was designed and not run.
3. **A verifier that always rejects** is knowledge sound at error zero and is excluded only because `Security` extends `Complete`: on an instance whose relation is empty (below) the exclusion is void.

Conversely, the five pass-through phases of the repository’s tests, which inhabit `Phases.Complete`, have *no* `Phases.Security`: the type is empty (probe `P5PassThrough`, theorem `no_security`), where the test’s docstring only asserts it.

What this means. The master theorems are composition theorems. They pin the phases to leanVM’s by nothing in their statement; what does is outside the spine and, today, outside the repository: a bound on the error (item 1), the phases’ definitions being leanVM’s, which only the compiled verifier’s refinement theorem and the fixture of a Rust-produced proof can tie down (item 2), and review of each phase’s definition against the specification. The report’s recommendation is to move the first into the spine and to say the other two plainly in the blueprint’s headline (chapter 11).

9.3 The instance

The relation is stated over an abstract instance, and an instance can be wrong in ways the spine does not exclude. A `Layout` that reads every column from cells 0 and 1 satisfies the reading law (probe `P2Relation`, `toyAlias`); on it the three columns are one, `M3Holds` asks for incompatible things, and the relation is empty for every stack and statement (exhaustive on the cells, the general case on paper). The master theorems hold of that instance: completeness is vacuous, and knowledge soundness says that every accepting prover was lucky. The reading law also determines `read` from `extend` (a cube point’s extension is the cell), so `read` is redundant data and the only freedom of an instance is `extend`. A wrong `extend` costs completeness and faithfulness, never the soundness of the Lean chain: the adaptor’s theorem from `M3Holds` to `SatisfiedBy`, if proved of such an instance, still gives the chain to the semantics. Who rules such instances out is the adaptor’s completeness theorems and the transcription of the layout with the compiled verifier’s fixture, none of which is built; until then, no statement on `main` says that the relation of any instance other than the toy is inhabited.

The same holds of the instance’s other fields: `aux` is an arbitrary predicate, and on the only built instance it is `True`, so a `Flock` phase with no message, no challenge and no check is knowledge sound at error zero (probe `NoCheckFlock`); `publicLines` is a list, and an instance with two lines instead of three has every theorem of the spine and of the public-input phase (chapter 14); `Layout.comap`, by which the adaptor is to name the blocks, accepts a renaming that is not one-to-one, so columns can alias.

9.4 The error bound

Beyond item 1 above: the errors the blueprint charges to the bus phase, the table sumcheck and the public-input phase are correct and tight where they are stated, and the review’s round-by-round arguments give the state functions that attain them (chapter 5). Two errors are not tight: the zerocheck escape, which costs $\tau_{\max}/|\mathbb{E}|$ once and nothing in a conjunctive state function, is charged per constraint and placed on the wrong challenges; and the GKR round error is $4/|\mathbb{E}|$, not $5/|\mathbb{E}|$. Whether the sum means something at leanVM’s parameters is settled in section 8.7: the interactive sum does (about $2^{32.3}/|\mathbb{E}|$ under the stacking window, within the blueprint’s $2^{40}/|\mathbb{E}|$), but the non-interactive bound the blueprint sketches leaves out three things that decide whether leanVM reaches the 128 bits it claims.

9.5 The extractor

Every extractor in the repository is a compiled definition: nothing along the chain from `piopExtractor` to the commit phase’s is marked `noncomputable`, and the three of them run (probes `P1Axioms`, `Extractors`). `#print axioms` says nothing about this: it reports the three standard axioms for computable definitions whose proofs use classical lemmas, and the earlier review’s use of it as evidence of computability was a misreading.

Two things are less than the blueprint says. First, “the extractor’s column q satisfies `M3Holds`” names no definition of the repository: the composed extractor’s `extractOut` returns the stack only when every phase has no round; with the repository’s own public-input phase in the bundle, the statement of the test that “the extractor returns the committed stack” is ill-typed, because `extractOut` then lives in that phase’s last intermediate witness type, `Unit` (probe `P3cExtractor`, an expected failure recorded). The stack is what the composed extractor returns at round 0 through its `extractMid` at the commit round, and there it is the transcript’s first message whatever it is handed. No function “the stack extracted from a full transcript” is defined. Second, ArkLib’s extractor type accepts an extractor that chooses a witness by classical choice, and such an extractor is “knowledge sound at error zero” for any relation whose language the verifier decides (probe `Extractors`): ArkLib’s existential notion of round-by-round knowledge soundness is soundness, and the named form the spine uses is knowledge only with the named extractor read. A phase’s `Security` could carry a classical extractor without anything in the repository noticing; it would not matter, because from the commit phase on the witnesses are `Unit` and the stack comes from the commit phase’s extractor alone, which is the spine’s and reads the message. The design “the oracle is the witness” excludes the risk that acceptance test 24 guards against; the test does not.

9.6 The public-input phase

The phase is built and its two theorems are proved; the question is what they exclude. The answer, machine-checked (probe `ProbeAnyCheck`, theorem `securityG`): nothing about the check. For every function the prover may use to compute its message and every Boolean check that the honest message passes, the phase with that check has a `Phase.Security` against the same seams at the same error $1/|\mathbb{E}|$. Instances proved: no check and no message; no check and a message of forty-two zeros; a check with the two cells swapped, with a prover that sends the swapped values. The reason is one line of the definition: `pooled` takes its values from the statement, not from the message, and the proof of knowledge soundness never mentions `check`. What the theorems do exclude, each refuted in Lean with no extractor, state function or error below one able to repair it (theorem `not_rbr`, which denies ArkLib’s existential notion): a verifier that pools the prover’s values without checking them; one that checks only the first value and pools the prover’s values; one that does not pool the claim of a line whose value is not sent. The earlier review’s statement that each of five wrong verifiers breaks a stated theorem was true of the build it read and is false of the code as merged: its own compression (pooling the lines’ values, its own first compression) removed the dependence, and the status file still repeats the claim. The consequence for the requirement that a missing check leave a theorem unprovable is drawn in chapter 14; the repair, pooling the values sent, is proposed in section 8.5.

9.7 The notion itself

ArkLib’s round-by-round knowledge soundness is not trivially satisfiable: a verifier that accepts everything has no knowledge state function for a non-trivial relation with no rounds, and with one challenge it is not knowledge sound at error zero whatever the extractor; every verifier is knowledge sound at error one (probe `NonVacuity`). Its plain reading, that an accepting transcript either extracts a witness or contains a challenge that fell in the bad set of its prefix, is machine-checked at the transcript level (probe `PlainReading`); the step from there to a bound on a prover’s success is ArkLib’s admitted implication from round-by-round to plain knowledge soundness, at both pins (chapter 3).

9.8 The blueprint’s acceptance tests

The blueprint names twenty-eight acceptance tests, each a reading that compiles and is wrong with the witness that rejects it. For each, whether the witness exists on `main` and the review’s verdict. *[table pending]*

9.9 What was not established

The read-everything phase (item 2 of section 9.2) is on paper. The emptiness of the aliased instance’s relation is exhaustive on the cells and on paper in general. Of the mutation probes of the public-input phase, two were never written (the deployed check with the prover’s values pooled, the wrong point) and rest on paper; two (the wrong check with the honest prover unchanged, the extra check) ran at the new pins and refuted completeness as predicted, while their unchanged knowledge-soundness proof failed at the state function like every other mutation’s (appendix B).

Chapter 10

Auditability

The audit surface of a theorem is the set of declarations its statement unfolds to: what a reader must read, and trust to mean what it says, before the theorem tells them anything. This chapter separates those declarations from the helpers, measures them, and proposes what can be removed without changing any statement’s meaning. Faithfulness and non-vacuity come first; the proposals here are for after the statements are right.

10.1 What is load-bearing

For the two master theorems the load-bearing declarations are, transitively: the theorems’ statements (`piop_perfectCompleteness`, `piop_rbrKnowledgeSoundness`); the composition (`leanVmPiop`, `leanVmVerifier`, `piopError`, `piopExtractor`, `Phases`, `Phases.toDef`, `Phases.Complete`, `Phases.Security`, and the commit phase’s four declarations); the component interfaces (`Component.Def`, `Complete`, `Security`, their `append`, the two ported composition declarations); the seams (`Seam.of` and the six seams, the four claim structures with their `Holds`, the four output structures, `theStack`); the instance (`M3Instance` with its fields, `Shape`, `Layout`, `Coord`, `BoundaryBlock`, `PublicLine`, the readers `column`, `row`, `coordCell`, `tuples`, the four clauses, `M3Holds`, `M3Rel`, `TheOracle`); and Layer 0’s oracle interface of the stack with the field and sampling instances. Below them sit the library definitions they unfold to, which chapter 3 introduces: ArkLib’s protocol specifications, oracle reductions, completeness, knowledge state functions, extractors and round-by-round knowledge soundness; VCVio’s computations and probabilities; CompPoly’s tables, polynomials and fields.

Measured at `b435631` (probe `P6Surface`; the tables are in section 10.4): the completeness theorem unfolds to 70 declarations and 217 lines of statements (382 with docstrings); the knowledge-soundness theorem in its existential form to 76 declarations and 248 lines (432); in the named form the spine states, to 100 declarations and 354 lines (612). Layer 1 has 143 public declarations, of which 20 are load-bearing and 41 are interfaces; an auditor of the master theorems reads none of them today, 17 definitions (61 lines) become trusted once the adaptor is built and about 8 more (22 lines) once the executable verifier is. The public-input phase adds its verifier, its check, its pool and its line identity.

Where an auditor starts. In this order: chapter 3 for ArkLib’s notions of completeness and knowledge soundness (nothing else is meaningful before them); the two master theorems in `Spine/Compose.lean` with `Phases`, `Phases.Complete` and `Phases.Security`; the seams in `Spine/Seams.lean`; the instance and the relation in `Spine/Instance.lean`; the component interfaces in `ToArkLib/Component.lean` and the two ported declarations of `ToArkLib/KnowledgeAppend.lean`; the oracle interface and the samplers of `Field.lean`; then, for the one built phase, `PublicInput.verifier`, `check`, `pooled` and the two theorems. Layer 1 is read only when the adaptor is, and the catalogue (chapter 4) gives every declaration in that order with its class.

10.2 What is not, and whether it leaks

A helper lemma cannot enlarge the trusted surface of a theorem it helps to prove: the kernel checks the proof, and what the proof used does not change what the statement says. Two things can leak nonetheless, and were checked. An *axiom* used by a helper propagates to every theorem that uses it: every theorem of the proof system, the port of the composition included, closes on the kernel’s three standard axioms and on nothing else (`#print axioms`, probe `AxiomsLeanerVM`, re-run at the new pins in appendix B), and the repository’s validation rejects `sorryAx` across both namespaces with a planted-axiom control. An *instance* used by a statement (a decidability instance, a sampler, an oracle interface) is part of the statement’s meaning: the sampler instances are uniform by `VCVio`’s class law, the diamond on the sampler of $\mathbb{K}$ resolves to the same term by `rfl`, and the oracle interface of the stack answers the extension at the point in the specification’s bit order (chapter 3). No helper was found to enlarge the trusted surface.

10.3 Proposals to reduce the surface

Each proposal names what is removed, what it costs and whether a statement changes. None changes the meaning of a master theorem; the first two are also faithfulness repairs and are argued in chapter 8.

1. **Fix the error in the spine.** Replace the phases’ declared `err` by a closed form `piopError I` computed from the instance, and demand each phase’s knowledge soundness at it. Removes a field from `Component.Def` and the possibility of item 1 of section 9.2; adds one definition to the spine. Changes the statement of the knowledge-soundness theorem, for the better: it then says something.
2. **Give the bus seam leanVM’s shape.** Replace `BusOut`’s list of linear claims by one point, the forms per side and table with the degree bound in their type, three totals and the column claims. Removes `LinearClaim`, `VirtualTerm`, the degree conjunct of `Seam.bus` and the guards a generalized table sumcheck would need; adds three small types. Changes `Seam.bus` and the two phases around it.
3. **Derive read from extend.** The reading law determines it (section 9.3); `Layout` then has one field of data and one law, and an instance cannot state a `read` that disagrees with its `extend`. Costs one definition and a lemma; changes no statement’s meaning.
4. **Delete outputPure.** It is derivable for every component over the empty shared oracle (probe `P6Surface`, `outputPure_of_def`; `ArkLib` has the instance at the pin). Removes a field of `Component.Complete` and one obligation per phase.
5. **Separate the guard from completeness.** A base structure with the guarded form only, extended by `Complete` and by `Security`, so that knowledge soundness no longer presupposes perfect completeness (section 8.6). Costs one structure; changes no theorem’s meaning; changes what an author of a phase must prove first.
6. **Drop Weight.mle from the seam.** The evaluator the verifier runs is the opening phase’s business, not the seam’s; the seam needs the cube values and the value. Removes a field and a law from `Weight`; the closed form of the ring-switching weight stays owed to whoever builds the opening.
7. **Merge TableOut and PubOut.** Both are a list of column claims; one structure with two uses. Removes one structure; changes the type of the public-input phase’s slot, not its meaning.
8. **Move the error out of Component.Def.** With item 1 the field is gone; even without it, keeping a real number in the definition makes every phase with a challenge `noncomputable`, so that neither the verifier nor the extractor compiles when reached through the phase (probe `ExtractorsExpectedFailure`). The executable verifier of Layer 12 is meant to read the phases’ definitions.
9. **Delete BlockClaims.lean.** It has no consumer, and the earlier review found that no phase can use it over an abstract instance. Removes about a dozen declarations.
10. **Say which tables the sumcheck ranges over,** as a decidable predicate on the instance’s tables derived from its constraints, flushes and counts (section 8.5). Adds one definition; removes a whole class of instances the table sumcheck would otherwise be owed on.
11. **Name the Flock region in the instance** (a column, a log-count, a height fact and what `aux` says of it) instead of the opaque predicate alone (section 8.6). Adds one small structure; makes the Flock phase statable over an abstract instance.
12. **Read the existential theorem first.** `piop_rbrKnowledgeSoundness_exists` unfolds to 24 fewer

declarations than the named form; a reader should start there, then read the named form for the extractor.

With items 2, 4, 6, 7 and 9 applied the review estimates the named knowledge-soundness form at about 80 declarations and 290 lines of statements; the estimate is not a measurement.

10.4 The measured tables

10.5 The audit surface, measured

Three measurements, one per part of the built code: the spine (what the two master theorems unfold to), Layer 1 (what becomes trusted when the adaptor and the executable verifier are built), and the public-input phase. Each is followed by the dossier’s proposals to reduce it: what each removes, what it costs, and whether any statement changes.

10.5.1 The totals

Table 10.1: The audit surface as the dossiers measure it (lines: statement or code lines, counting rules in the subsections below).

What an auditor reads	Declarations	Lines	With docstrings	Note
Spine: <code>piop_perfectCompleteness</code>	70	217	382	declarations the statement unfolds to, transitively
Spine: <code>piop_rbrKnowledgeSoundness_exists</code> (existential form)	76	248	432	adds the <code>Security</code> structures, the verifier, the error
Spine: <code>piop_rbrKnowledgeSoundness</code> (named extractor and state function)	100	354	612	adds the composition of securities, the ported append, the commit phase’s security data
Layer 1: all public declarations	143	388	539	20 load-bearing, 41 interface, 82 helper
Layer 1: load-bearing	20	71	97	read by no master theorem today
Layer 1: once the adaptor is built	17	61	83	definitions that become trusted
Layer 1: once the executable verifier is built	8	22	—	the evaluators it runs

The two dossiers count differently, and their line counts are not to be added: the spine’s dossier counts a definition’s lines from its head to the end of its data body without proof fields, and never a theorem’s proof; the Layer 1 dossier counts a theorem up to `:=` and a definition in full. The public-input phase’s dossier gives no line count; its catalogue’s classes are tallied in the last subsection.

10.5.2 The spine

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

10.5.2.1 Method (B.1)

“Load-bearing” is taken literally: a declaration is counted when the **statement** of a master theorem mentions it or unfolds to it, transitively, down to library objects. Three sets:

- **C**: what `piop_perfectCompleteness` unfolds to (`Compose.lean:162-164`): `Phases`, `Phases.Complete` (so `Phase.Def`, `Phase.Complete`, `Component.Def`, `Component.Complete`), `leanVmPiop` (so `Phases.toDef`, `Def.append`, `commitDef`, the send-oracle reduction), `M3Rel` (so `M3Holds` and everything under it), the six seams (so every claim and its `Holds`), `TheOracle` and the evaluation oracle.

- **Kx**: what `piop_rbrKnowledgeSoundness_exists` adds (:181-184): `Phases.Security`, `Phase.Security`, `Component.Security`, `leanVmVerifier`, `piopError`.
- **K**: what `piop_rbrKnowledgeSoundness` adds by naming its extractor and state function (:172-176, `S.toDef.witMid`, `piopExtractor P S`, `S.toDef.kSF init impl`): the composition of securities (`Security.append`, `Complete.append` for its guarded, `guardedAppend`, `stateFunctionOfEq`), the ported state-function append (`KnowledgeAppend.state`, `left`, `right`, `Witness`, the two witness lemmas, `appendGuarded`) and the commit phase’s security (`commitSecurity`, `commitComplete`, `commitExtractor`, `sendOracleSecurity`, `sendOracleComplete`, `sendVerifierPure`, `sendExtractor`, `sendWitMid`, `sendStateFunction`, `sendOracle_relOut`).

Counting rule: a declaration’s lines from its head to the end of its data body, excluding blank lines, comment-only lines and, for definitions, proof fields (`complete`, `rbr`, `toFun_empty/next/full`, `verify_eq`, ...); “with docstrings” adds the docstring block above it and the field docstrings inside it. Proof bodies of theorems are never counted. The script is `probes/code-spine/surface.py`, which reads every file with `git show b435631`: and finds each declaration by name; its output is reproduced below. Structures’ field docstrings are counted as docstrings, so `M3Instance` is 14 lines of code and 31 with its docstrings.

10.5.2.2 The count (B.2)

The output of the dossier’s counting script (`probes/code-spine/surface.py`), set as tables. Class: *completeness* for what `piop_perfectCompleteness` unfolds to; *existential* for what `piop_rbrKnowledgeSoundness_exists` adds; *named* for what `piop_rbrKnowledgeSoundness` adds by naming its extractor and state function.

Table 10.2: Every declaration the spine’s master theorems unfold to, with its code lines and its lines with docstrings, at `b435631`.

Class	Code	With docs	File	Lines	Declaration
completeness	2	6	Field.lean	67-69	structure Column
completeness	5	6	Field.lean	86-90	def limbsEquiv
completeness	1	2	Field.lean	93-93	instance instSampleableTypeE
completeness	5	7	Field.lean	102-106	instance evalOracle
completeness	1	2	Oracles.lean	26-26	abbrev NoOracle
completeness	1	2	Oracles.lean	29-29	abbrev OneOracle
completeness	4	5	Instance.lean	51-54	inductive Side
completeness	4	8	Instance.lean	57-63	structure Shape
completeness	1	2	Instance.lean	66-66	abbrev Shape.ColumnId
completeness	6	14	Instance.lean	73-81	structure Layout
completeness	4	7	Instance.lean	86-89	inductive Coord
completeness	4	8	Instance.lean	92-98	structure BoundaryBlock
completeness	6	15	Instance.lean	102-114	structure PublicLine
completeness	14	31	Instance.lean	119-148	structure M3Instance
completeness	1	1	Instance.lean	150-150	attribute [instance] M3Instance.decAux
completeness	1	2	Instance.lean	157-157	abbrev k
completeness	1	2	Instance.lean	160-160	def column
completeness	2	3	Instance.lean	163-164	def row
completeness	4	5	Instance.lean	167-170	def coordCell
completeness	4	6	Instance.lean	174-177	def flushTuples
completeness	3	4	Instance.lean	180-182	def boundaryTuples
completeness	2	3	Instance.lean	185-186	def tuples
completeness	2	3	Instance.lean	197-198	def ConstraintsVanish
completeness	1	2	Instance.lean	201-201	def Balanced
completeness	2	3	Instance.lean	204-205	def CountsNonzero
completeness	4	5	Instance.lean	208-211	def PublicLinesHold
completeness	2	4	Instance.lean	219-220	def M3Holds
completeness	1	2	Instance.lean	226-226	abbrev TheOracle
completeness	2	4	Instance.lean	230-231	def M3Rel
completeness	4	8	Seams.lean	57-63	structure ColumnClaim
completeness	2	3	Seams.lean	66-67	def ColumnClaim.Holds
completeness	5	11	Seams.lean	71-79	structure VirtualTerm
completeness	3	4	Seams.lean	82-84	def VirtualTerm.table
completeness	2	3	Seams.lean	87-88	def VirtualTerm.eval
completeness	3	6	Seams.lean	91-95	structure LinearClaim
completeness	2	3	Seams.lean	98-99	def LinearClaim.Holds
completeness	4	9	Seams.lean	103-109	structure Weight
completeness	2	3	Seams.lean	112-113	def Weight.pair

Class	Code	With docs	File	Lines	Declaration
completeness	3	6	Seams.lean	116-120	structure WeightedClaim
completeness	2	3	Seams.lean	123-124	def WeightedClaim.Holds
completeness	3	7	Seams.lean	130-134	structure BusOut
completeness	2	4	Seams.lean	137-139	structure TableOut
completeness	2	4	Seams.lean	142-144	structure PubOut
completeness	3	6	Seams.lean	147-151	structure FlockOut
completeness	1	2	Seams.lean	159-159	abbrev theStack
completeness	2	3	Seams.lean	166-167	def of
completeness	1	3	Seams.lean	171-171	def commit
completeness	3	5	Seams.lean	175-177	def bus
completeness	2	3	Seams.lean	180-181	def table
completeness	1	2	Seams.lean	184-184	def pub
completeness	2	3	Seams.lean	187-188	def flock
completeness	1	2	Seams.lean	191-191	def done
completeness	9	17	Component.lean	52-66	structure Def
completeness	1	1	Component.lean	68-68	attribute [instance] Def.msgOracle
completeness	7	12	Component.lean	76-85	structure Complete
completeness	6	7	Component.lean	143-148	def Def.append
existential	12	20	Component.lean	90-106	structure Security
named	8	13	Component.lean	110-120	def stateFunctionOfEq
named	4	5	Component.lean	156-159	def guardedAppend
named	4	9	Component.lean	162-169	def Complete.append
named	8	14	Component.lean	174-184	def Security.append
named	2	3	KnowledgeAppend.lean	55-56	abbrev Witness
named	3	5	KnowledgeAppend.lean	60-62	theorem witness_left
named	3	5	KnowledgeAppend.lean	66-68	theorem witness_right
named	5	6	KnowledgeAppend.lean	71-75	def left
named	5	6	KnowledgeAppend.lean	78-82	def right
named	15	16	KnowledgeAppend.lean	182-196	def state
named	5	17	KnowledgeAppend.lean	474-488	def appendGuarded
completeness	2	3	SendOracle.lean	38-39	def sendSpec
completeness	6	7	SendOracle.lean	42-47	def sendProver
completeness	2	5	SendOracle.lean	50-53	def sendEmbedding
completeness	3	4	SendOracle.lean	56-58	def sendVerifier
completeness	5	6	SendOracle.lean	61-65	def sendOracle
named	3	5	SendOracle.lean	71-73	def sendOracle_relOut
named	2	4	SendOracle.lean	101-103	def sendVerifierPure
named	4	6	SendOracle.lean	121-125	def sendOracleComplete
named	1	2	SendOracle.lean	130-130	abbrev sendWitMid
named	5	7	SendOracle.lean	134-138	def sendExtractor
named	5	19	SendOracle.lean	145-161	def sendStateFunction
named	5	7	SendOracle.lean	172-177	def sendOracleSecurity
completeness	2	3	Phase.lean	37-38	abbrev Def
completeness	4	5	Phase.lean	41-44	abbrev Complete
existential	4	5	Phase.lean	47-50	abbrev Security
completeness	1	2	Compose.lean	51-51	abbrev commitSpec
completeness	2	4	Compose.lean	55-56	abbrev commitDef
completeness	6	13	Compose.lean	73-84	structure Phases where
completeness	3	4	Compose.lean	89-91	def Phases.toDef
completeness	6	12	Compose.lean	94-104	structure Phases.Complete
completeness	3	4	Compose.lean	134-136	def leanVmPiop
completeness	4	6	Compose.lean	162-165	theorem piop_perfectCompleteness
existential	6	12	Compose.lean	113-123	structure Phases.Security
existential	3	4	Compose.lean	139-141	def leanVmVerifier
existential	1	2	Compose.lean	149-149	def piopError
existential	5	7	Compose.lean	181-185	theorem piop_rbrKnowledgeSoundness_exists
named	1	2	Compose.lean	59-59	abbrev commitExtractor
named	2	4	Compose.lean	63-64	def commitComplete
named	2	3	Compose.lean	67-68	def commitSecurity
named	4	5	Compose.lean	126-129	def Phases.Security.toDef
named	4	6	Compose.lean	153-156	def piopExtractor
named	6	11	Compose.lean	172-177	theorem piop_rbrKnowledgeSoundness

Table 10.3: Code lines per file and per class, and all lines with docstrings.

File	Completeness	Existential adds	Named adds	All, with docstrings
Field.lean	13	0	0	21
Oracles.lean	2	0	0	4

File	Completeness	Existential adds	Named adds	All, with docstrings
Instance.lean	75	0	0	139
Seams.lean	55	0	0	103
Component.lean	23	12	24	98
KnowledgeAppend.lean	0	0	38	58
SendOracle.lean	18	0	25	75
Phase.lean	6	4	0	13
Compose.lean	25	15	19	101

Table 10.4: The three totals of the spine.

Statement	Declarations	Code lines	With docstrings
completeness theorem (C)	70	217	382
knowledge theorem, existential form (C+Kx)	76	248	432
knowledge theorem, With form (C+Kx+K)	100	354	612

In words. To read **what perfect completeness says**, an auditor reads 70 declarations, 217 lines of Lean (382 with their docstrings): 75 in `Instance.lean` (the instance and the relation), 55 in `Seams.lean` (claims and seams), 25 in `Compose.lean`, 23 in `Component.lean`, 18 in `SendOracle.lean` (the commit reduction), 13 in `Field.lean`, 6 in `Phase.lean`, 2 in `Oracles.lean`. **Knowledge soundness in its existential form** adds 6 declarations and 31 lines (the `Security` structures, the verifier, the error). **Knowledge soundness for the named extractor** adds 24 declarations and 106 lines, of which 38 are the ported state-function append and 25 the commit phase’s security data. The whole **With** form is 100 declarations, 354 lines, 612 with docstrings.

The library objects the statements unfold to, by revision `dca90385` (`ArkLib`), `3468b38c` (`CompPoly`), `f9dc47d9` (`VCVio`), not counted line by line:

- `ArkLib`: `ProtocolSpec`, `Direction`, `ChallengeIdx`, `MessageIdx`, `Transcript`, `FullTranscript`, `ProtocolSpec.append` (`++p`) and `ChallengeIdx.sumEquiv`; `OracleInterface`; `Prover`, `OracleProver`, `Verifier`, `OracleVerifier`, `OracleOutputEmbedding`, `OracleVerifier.toVerifier`, `Reduction`, `OracleReduction`, `OracleReduction.append`, `Verifier.append`; `Prover.OutputIsPure`, `Verifier.PureForm`, `Verifier.GuardedForm`, `GuardedForm.append`, `PureForm.toGuardedForm`; `Reduction.completeness`, `perfectCompleteness`, `OracleReduction.perfectCompleteness`; `Extractor.RoundByRound`, `Extractor.RoundByRound.append`, `Verifier.KnowledgeStateFunction`, `rbrKnowledgeSoundnessWorstCase`, `rbrKnowledgeSoundnessWorstCaseWith`; `Verifier.run`, `Prover.run`, `Reduction.run` (through `perfectCompleteness`); `challengeQueryImpl`, `QueryImpl.addLift`.
- `VCVio`: `OracleComp`, `OracleSpec`, `[]` (`emptySpec`), `QueryImpl`, `simulateQ`, `ProbComp`, `OptionT`, `Pr[· | ·]` (`probEvent`), `$t` (`uniformSample`), `SampleableType`, `StateT`.
- `CompPoly`: `CMlPolynomialEval`, `evalMle`, `eval2Mle`, `CMvPolynomial`, `CMvPolynomial.eval`, `totalDegree`, `BF64`, `BF64.Ext3`, `Extension.Ext.ofVector`, `Ext.coeffs`, `algebraMap K E` (through `Mathlib`), `Vector`.
- `Mathlib`: `List.Perm`, `Set`, `Finset.sum` (in `Weight.pair`), `NNReal`, `ENNReal`.

The earlier review counted “306 statement lines ... about 400 lines” for the whole of the spine’s ten files (`docs/reviews/protocol-spine.md:426-448`), every declaration included. The count here is smaller because it keeps to what the two statements unfold to and larger in the **With** form than an auditor of the existential form needs.

10.5.2.3 Proposals to reduce it (B.3)

Each: what is removed, what it costs, whether any statement changes. Lines are code lines of B.2.

1. **State the master knowledge theorem in the existential form and keep the named form as a lemma.** Removes from the reading list the 24 declarations of class `K` (106 lines): the composition of securities, the ported append, the commit phase’s data. Costs: the named extractor leaves the theorem’s statement; acceptance test 24 then rests on the lemma `piop_rbrKnowledgeSoundness` (kept) and on `piopExtractedStack_eq` (section F). Changes no statement; it changes which one is

called the master theorem. Not recommended if the named extractor is what T4 must carry through Fiat–Shamir; recommended as the *reading order* (the existential theorem first, the named one as its refinement).

2. **Delete Component.Complete.outputPure** (2 lines, plus one line in every phase and in `Complete.append`): derivable for every component (D.7, ArkLib’s `Prover.instOutputIsPureEmpty` at the pin). Costs one import (`NoAmbient`) or a four-line restatement. Changes no statement: `Complete` loses a field every instance discharges by `rfl`.
3. **Merge TableOut and PubOut** (4 lines, two structures that are both `columns : List (ColumnClaim I)`; `FlockOut` is `TableOut` plus `weighted`). Replace by one `Pool I` with `columns` and `weighted`, and let each seam say which list is empty where it must be (`Seam.table: weighted = []`). Costs: the statement types of three phases become the same, so a phase in the wrong slot is caught by its seam proof, not by its type; the docstrings’ “column claims, now with the public words’ claims” become a comment on the seam. Changes the statements of `Seam.table`, `Seam.pub` (one conjunct each) and the fields of `Phases`. Recommended only if the sibling dossier’s `BusOut` redesign is adopted, so that the statement types are revisited once.
4. **Derive Layout.read from extend** (C.2; `read` and `read_eval` are 6 of `Layout`’s 6 lines, replaced by one law on `extend`). Costs: a lemma “the extension at a cube point is the entry” (`CompPoly` has it as `eval_mle_eq_eval`, cited in `Field.lean:50`) and, for the toy, `slice` becomes a theorem. Changes the definition of `column` and hence of every clause, definitionally: no statement’s meaning changes.
5. **Fold Phase.Def/Complete/Security** (10 lines): three abbreviations of `Component.*` at `TheOracle I` and `Unit`; `Phase.passThrough*` likewise. A phase author writes `Component.Def I.Stmt (TheOracle I) Unit ...` once. Costs: longer signatures in `Phases` (five fields). Changes no statement. Marginal; the abbreviations are the documented interface and cost 13 lines with docstrings.
6. **Drop Weight.mle and mle_eq** (2 of `Weight`’s 4 lines): the seam never uses `mle` (`WeightedClaim.Holds` is `Weight.pair`, the cube sum). `mle` is the verifier’s evaluator for the opening phase, a property of the *phase*, not of the claim’s truth. Costs: the opening phase must carry the evaluator itself (as a field of its `Def`, or by requiring `Weight` to be MLE-friendly there). Changes the statement of `Seam.flock` by removing a field nobody reads there. Recommended: it also removes the temptation to prove the seam by `mle` rather than by the cube sum.
7. **M3Instance.d with its two proof fields** (3 lines) could be replaced by a subtype of polynomials on constraints and flushes (section F, the degree finding): the two proof fields go, `Seam.bus` loses a conjunct, the table verifier loses a guard. Changes the statements of `M3Instance` and `Seam.bus`, and of `VirtualTerm`.
8. **Drop decAux** from `M3Instance` (2 lines with the attribute): decidability of `aux` is used by nothing load-bearing; the `Decidable` instances for `M3Holds` serve the tests. Costs: the tests declare `instance : DecidablePred toy.aux` locally. Changes the statement of `M3Instance`. Weak: an instance that cannot decide its own `aux` is a smell, and the read-everything phase (D.3 (d)) needs it; keep unless the field list is revisited.
9. **The {σ} init impl binders** (eleven declarations): forced by ArkLib (C.6). A wrapper `Phase.Complete'` at `σ := Unit` would not compose (ArkLib’s `append` needs every `s : σ`). Not removable at this pin.
10. **sendOracle_relOut** (3 lines) is `Seam.commit` under another name, needed because the generic send-oracle component is stated for any relation; `commitComplete`’s type ascription is what makes the two coincide. Keep; note in `Compose.lean` that the coincidence is definitional, so an auditor need not read `sendOracle_relOut`.
11. **sendStateFunction’s data is in the statement** (5 lines) only through the named form (K). If proposal 1 is taken it leaves the surface.

What is **not** proposed: removing `Phases.Complete` in favour of `Phases.Security` (`Security` extends `Complete`, and completeness is landed before security per convention *Holes*); removing `piop_rbrKnowledgeSoundness_exists` (it is the smaller statement).

Net effect of proposals 2, 4, 6 and 7, all of which keep the theorems’ meaning: about 15 code lines and two proof obligations per phase (`outputPure`, the degree of its terms) off the surface, and one guard out of the table phase’s verifier.

10.5.3 Layer 1

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-layer1.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.3; E (audit surface) is section 10.5.3; F (conformance) is section 8.9.1.3; G (findings) is section 8.9.2.3; I (probes) is ?? B.1.1.5; B (faithfulness), C (statements), D (the wall and the To-folders) and H (the upgrade) is not reproduced in these fragments; they are in the dossier.

Method: every `def`, `abbrev`, `structure`, `theorem` outside a `private` modifier in the nine files at `b435631` (all sit in `@[expose] public section`). “`stmt`” counts a theorem’s lines up to `:=` and a definition’s lines in full (a trusted definition is read whole); “`+doc`” adds its docstring. The script (`count2.py`, section I.7) reads the files with `git show b435631:<path>`. Classes as in section A.

Module	L n / stmt / +doc	I n / stmt / +doc	H n / stmt / +doc	total n / stmt / +doc
ToCompPoly/Multilinear.lean	4 / 13 / 18	10 / 23 / 35	29 / 62 / 84	43 / 98 / 137
ToCompPoly/BitProductTable.lean	1 / 2 / 3	4 / 9 / 14	8 / 19 / 22	13 / 30 / 39
ToCompPoly/Stacking.lean	7 / 27 / 38	10 / 29 / 41	21 / 52 / 68	38 / 108 / 147
ToCompPoly/AmbientStacking.lean	0 / 0 / 0	2 / 7 / 11	8 / 28 / 37	10 / 35 / 48
Stack.lean	3 / 14 / 17	5 / 16 / 24	1 / 4 / 5	9 / 34 / 46
Padding.lean	0 / 0 / 0	5 / 10 / 17	1 / 2 / 3	6 / 12 / 20
ClaimWeights.lean	1 / 4 / 6	1 / 3 / 5	2 / 5 / 7	4 / 12 / 18
BlockClaims.lean	0 / 0 / 0	0 / 0 / 0	8 / 30 / 40	8 / 30 / 40
FixedColumns.lean	4 / 11 / 15	4 / 10 / 17	4 / 8 / 12	12 / 29 / 44
total	20 / 71 / 97	41 / 107 / 164	82 / 210 / 278	143 / 388 / 539

What an auditor must read, and when.

- **Today: nothing.** The master theorems, the seams and the phase interfaces are stated over an abstract `I : M3Instance` and import no Layer 1 module (`Spine/Instance.lean` imports `Field`, `ToArkLib/Oracles` and `CompPoly`; `Seams.lean` imports `Instance`). The one built phase that touches Layer 1, the public-input phase, imports `ToCompPoly/Multilinear` privately (`PublicInput.lean:14`, `import`, not `public import`) and uses `boolVec_zero`, `evalMle_append_boolVec`, `evalMle_cast` and `cubeIndex` inside one proof (`evalMle_linePoint`, :71-89); its definitions unfold none of them. Layer 1 is proved code whose statements nobody trusts yet.
- **Once the adaptor is built** (`leanIsaInstance` with `layout := (B.layout hu).comap f h` and the index and bytecode columns as `Coord.known`): seventeen definitions, 61 lines (83 with docstrings): `Blocks`, `offsetNat`, `offset`, `total`, `selector`, `extendPoint`, `unstack`, `slice`, `cubeIndex`, `boolVec`, `cubeSplit`, `Layout.comap`, `readColumn`, `layout`, `powersTable`, `idxColumn`, `bytecodeColumn` (plus `encodeSlots`, `entry`, `dereffFlags` and `Opcode.code` from the arithmetization and parameters), and the strided reader that does not exist yet (B.8).
- **Once the executable verifier is built:** the evaluators it runs, `eqWeight` (its `mle`), `idxColumnEval`, `bytecodeColumnEval`, and, if the verifier computes `eq(sel_b, ζ_hi)` with Layer 1’s function rather than its own, `selectorWeight`, `lowPoint`, `highPoint`, `lowVec`, `highVec`: eight definitions, 22 lines. The theorems relating them to the oracle (`idxColumn_eval`, `bytecodeColumn_eval`, `evalMle_padHigh`, `stack_eval_ambient_one`, `ColumnClaim.holds_iff_weighted`) are proved, so they are read for their statements only by whoever checks that the phases use them as intended.

Proposals to reduce it (each with its saving in public declarations / statement lines):

1. Delete `BlockClaims.lean` and `tests/LeanerVMTests/Protocol/BlockClaims.lean` (8 / 30; 122 + 108 file lines). No module uses `BlockClaim`; the opening phase uses `ColumnClaim.holds_iff_weighted`, over an abstract instance, which the earlier review introduced for that reason; `isValid_iff_pairing` restates `unstack_eval2_eq_sumCube`; the locality lemmas have no consumer. Drop `BlockClaim` from the interface list (`protocol-blueprint.md:1404`). If the leanth credit matters, the derived-source notice can move with `unstack_eval2_eq_sumCube`.
2. Delete `bytecodeColumn_slot` (1 / 2) and its three examples; point acceptance test 14 at `bytecodeColumn_answer_boolVec` and the oracle `#guards` (G.4). `bytecodeSlotColumn` and `slice_bytecodeColumn` (2 / 4) have no consumer either; keep them only if the bus phase takes the per-slot form (`ValuesProbe`’s `bytecodeColumn_answer_slots`, which the earlier review recorded as open).

3. Remove the in-layer duplicates (`DuplicatesProbe`): `unstack_eval` (a special case of `unstack_eval2`), `stack_eval_ambient_zero`, `stackColumn_eval` (no consumer), `stackColumn_eval_ambient` (honest stack only, no consumer): 4 / 14.
4. `prodVars`, `sumCube_prodVars`, `evalMle_prodVars` (3 / 5) prove nothing any other declaration uses; `sumCube_padHigh` is the statement the table `sumcheck` needs. Either delete them and cite `sumCube_padHigh` in acceptance test 7, or keep one as the specification’s sentence.
5. Replace `evalMle_lagrangeBasis` by `CompPoly`’s `eqTilde_eq_prod` (1 / 2), or upstream the reordering; keep `eqWeight.mle` as it is.
6. The 66 helpers of the four `ToCompPoly` files are the upstream pull request’s content, not audit surface; marking the ones used only inside their own file `private` (for example `cubeIndex_div`, `cubeIndex_mod`, `lowVec_append`, `highVec_append`, `slice_placeSlice`, `windowTable*`, `stackAt_decomposition`, `offsetNat_succ`, `offsetNat_mono`) would make the public list of each file read as its interface.

Together 1 to 5 remove about 20 public declarations and 60 statement lines, and no load-bearing definition. The load-bearing core (20 definitions, 71 lines) is already small; the one addition it needs is the strided reader.

10.5.4 The public-input phase

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-pubinput.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.2; B (faithfulness), E (the flag `PublicLine.sent`) and F (statement checks) is section 8.9.1.2; C (mutation probes) and D (the top limb) is ?? B.1.1.3; G (findings) is section 8.9.2.2.

The dossier on the public-input phase measures no line count. Its catalogue (section A) classes the declarations of `LeanerVM/Protocol/PublicInput.lean` (400 lines) as follows; the tally is of the catalogue’s rows, the two instances counted as two.

Table 10.6: The public-input phase’s declarations by class, as its dossier’s catalogue lists them.

Class	Count	Declarations
load-bearing	15	<code>linePoint</code> , <code>evalMle_linePoint</code> , <code>error</code> , <code>lineValue</code> , <code>lineClaim</code> , <code>pooled</code> , <code>expectedValues</code> , <code>prover</code> , <code>verifier</code> , <code>verifier_verify</code> , <code>guarded</code> , <code>complete</code> , <code>extractor</code> , <code>stateFunction</code> , <code>rbr</code>
interface	7	<code>pSpec</code> , its two instances, <code>check</code> (interface only: no theorem of the phase depends on it), <code>publicInputPhase</code> , <code>publicInputComplete</code> , <code>publicInputSecurity</code>
helper	7	<code>queryValues</code> , and six private lemmas: <code>lineClaim_holds_iff</code> , <code>line_challenge_unique</code> , <code>pooled_mem_pub</code> , <code>bad_challenge_unique</code> , <code>simulateQ_queryValues</code> , <code>prover_run_support</code>
generic modules	6	<code>Verifier.GuardedForm.of_probEvent_pos</code> , <code>Reduction.mem_support_run_of_guarded</code> , <code>probEvent_uniformSample_le_of_subsingleton</code> (load-bearing); <code>keepOracles</code> (interface); <code>OracleVerifier.materializeOutput_of_keepOracles</code> , <code>probEvent_uniformSample_le_of_card_le</code> (helper)

The dossier’s remarks on the surface are two. First, on the flag `PublicLine.sent` (section E):

Does it enlarge what an auditor must trust? By one Boolean per line, whose only effect in the phase as built is on the *transcript’s format* (which values are sent and checked): the pool does not depend on it beyond which claims carry a sent value, and with `pooled` reading the statement, not even that. In the deployed design (values sent pooled) `sent` decides which pooled values are prover-chosen, so it enters the soundness argument (the unsent lines’ claims are at verifier-computed values, and their falsity is a bad challenge on its own). Either way the auditor’s obligation is the same: “leanISA’s instance sends exactly the two low limbs”, a transcript fact checked against the Rust (:616-618, :747-749).

Second, the phase as merged is the result of a compression of the audit surface made by the earlier review (its item on a single output, `pooled`), and that compression removed the dependence of every theorem of the phase on the check of the prover’s message; the dossier’s finding on it is in the findings (the check on the public-input message is not load-bearing), with its evidence:

Evidence. C.3 (the verifier with no check has `Phase.Security` at the same error, no `sorryAx`), C.4 (for every message function and every check the message passes, `securityG`; instances with no check, with forty-two zeros, with the swapped check and a matching prover), C.5 (the first-value check; the words' equation). Root cause: `pooled` (`PublicInput.lean:127-128`) reads the statement, not the message; the message enters `check` and nowhere else; the proof of `rbr` (:350-366) never mentions `check`. The build the earlier review read `pooled` the values sent, and its item “H1: one output, `pooled`” (`docs/reviews/public-input-phase.md:552-573`) removed the dependence, so the same review's table “each of five wrong verifiers breaks a stated theorem” (:104-114, rows (i) and (v)) stopped being true of the code as merged; the status repeats the claim (`protocol-status.md:440`).

Part IV

What to do

Chapter 11

Options, limitations and boundaries

For each thing the review found, the options are to fix the blueprint, to keep it with the reason recorded and a theorem owed, or to refactor it; this chapter gives the review’s recommendation with the cost of each, then the general trade-off between specifying leanVM exactly as deployed and specifying a variant that is easier to audit, then what cannot be reproduced today and what it waits on, and finally what the proof system hands over to, and requires of, the rest of the end-to-end verification. The register of findings (appendix F) numbers each finding R1, R2, ...; the appendix of proposed changes (appendix A) gives the text of each change.

11.1 Recommendations, in priority order

1. **Fix the error in the spine (R5).** Define `piopError` I in closed form from the instance’s sizes and demand each phase’s knowledge soundness at it. Cost: one definition and a change of one field; every phase built later pays the bound. Without it the soundness theorem constrains nothing. The alternative, a bound as a field of `Phases.Security`, keeps the phases’ freedom to declare and is weaker; not recommended.
2. **Give the stack an inner-product oracle (R18).** The specification’s commitment is an inner-product scheme; its idealization answers weighted sums. The opening phase becomes “ λ , then one weighted query”, with the deployed transcript, and WHIR replaces the whole opening phase. Cost: Layer 0’s interface, the opening phase and Layers 11 and 12 as sketched; nothing built breaks. The alternative, keeping the evaluation oracle and the extra sumcheck and owing a theorem that relates the two protocols, is a transcript leanVM does not have and a theorem nobody has stated; not recommended. A third, ending the oracle protocol at the Flock seam and treating the opening as part of the compilation, is acceptable but moves the batching error out of the oracle protocol.
3. **Give the bus seam leanVM’s shape and name the sumcheck’s tables (R2, R11).** One point, forms per side and table with the degree bound in the type, three totals, the column claims; a decidable predicate for the tables the sumcheck ranges over. Cost: the seam and the two phases around it; the removal of the guards a generalized table sumcheck would otherwise need. The alternative, keeping the general seam with a generalized verifier and proving the guards dead in the composition, is more code and a verifier that is not leanVM’s; not recommended. The bus phase’s side of the seam is right either way (R3).
4. **Build the public-input phase of the deployed verifiers (R12, R13).** The combined check, with the values sent pooled; its key lemma is proved. Keep the specification’s phase as a reference and report the discrepancy of §8.2 to leanVM as an ambiguity (both are sound). Cost: one phase, on the pattern of the built one. The alternative, keeping only the specification’s phase, leaves the master theorem about a verifier nobody runs and a check that no theorem needs.
5. **Make the Flock phase statable and its interface faithful (R14, R15, R16, R17).** A small structure in the instance naming the region, the log-count and what the predicate says; the slot map of the limbs as parameters, not as a field of the Flock interface; Layer 9’s interface replaced by the spine’s slot; the honest prover the mathematical one; acceptance test 20 rewritten; the Rust prover’s defect at $r_{eq} = 1$ reported to leanVM. Cost: text, one structure, and a request to the Flock roadmap. Whether `Component.Complete` should carry an error (R4) is then a choice: not needed for

faithfulness, useful as insurance; the review recommends separating the guard from completeness so that knowledge soundness no longer presupposes it, and leaving completeness perfect.

6. **Describe the GKR as deployed** (R7, R9, R10). The normalized layer sumcheck of degree 4, four coefficients on the wire, error $4/|\mathbb{E}|$; a combiner after every layer, the last unused; a GKR component over a context with the leaf tables derived from the stack and the zerocheck conjunct riding along, or a monolithic bus phase. Cost: Layers 4 to 6 as sketched. The alternative, ArkLib’s context lifting, is admitted at both pins.
7. **Restate the adaptor** (R30 to R33, R35). The instance built from the tables’ components and from boundary blocks with known columns, never `Ensemble.toM3` of the eight tables; a bridge lemma for the boundary blocks; a computable polynomial bridge; admissibility with the stacking and rate windows and no `logInvRate` in the witness’s sizes; a reducible instance; the strided reader of the limb columns as a Layer 1 lemma and a Layer 3 reader. Cost: Layers 1 to 3 as sketched.
8. **Restate the non-interactive theorems** (R20 to R29). An adversary with a query budget against a random oracle standing for the compression and for the multi-block hash of the Merkle leaves, a straight-line extractor that reads leaves off the query log, list decodes and selects, adversary-chosen sizes inside one protocol family, the program bound by its hash, the well-formedness of the program and the resource bounds as hypotheses, and the random-oracle heuristic as a named non-formal step; the error with the hash term, the list-size factor and grinding; completeness of the honest prover whenever its grind returns. The assumed interfaces restated for leanVM’s constructions (one BLAKE2s map for the chain, the Merkle nodes and the grinding), or replaced by a plan to prove the chain lemma and the role-separation lemma; the list-binding compilation stated as the theorem it is. Cost: Layers 11 to 13 rewritten; the target theorem of `docs/architecture.md` with them.
9. **Decide the framework** (R34). ArkLib now calls `OracleReduction` legacy and its typed successor has no knowledge notion; the composition the spine ports is still admitted upstream and its pull request still open. Stay on the legacy framework with the local port, and record the decision and its retirement condition, or plan the migration. The review recommends staying, since nothing upstream is ready to receive the spine.
10. **Consolidate the documentation** (R36 to R38 and the minor rows). The blueprint as the one specification, its per-layer sketches rewritten on the spine’s types, the hole comment folded into it, the status and the tracker as pointers, one vocabulary, the codes replaced by names or indexed once (chapter 13).

11.2 Exactly as deployed, or easier to audit?

The review’s rule of thumb, from what it found: *the oracle protocol models the checks of the deployed verifier; the specification is the source of the arguments*. Where the two agree, there is no choice. Where they differ, three cases arose. The public-input check: the specification’s per-limb check is easier to reason about and the deployed combined check is what runs; modelling the specification’s produced a theorem about a verifier nobody runs and a check that no theorem needs, so the deployed one should be modelled and the specification amended. The Flock prover’s inverse: a defect of the implementation’s prover, not of the protocol, so the Lean honest prover is the specification’s and the implementation is told. The Python verifier’s acceptance of nonzero absent lanes: soundness-neutral, recorded, not modelled. The general lesson is that an auditable variant is acceptable only where it accepts *no more* than the deployed verifier and its relation to the deployed one is a theorem somebody states; a variant that accepts less (the per-limb check) makes the master theorem silent about the deployed verifier’s extra acceptances, which is where a deployment’s risk is.

Two deliberate deviations the blueprint has are right and should stay with their reasons written: the oracle protocol’s messages are whole objects where the wire carries an encoding (the round polynomial with one coefficient derived), owing the transport lemma the review states; and the relation is polynomial rather than “an execution exists”, with the adaptor as the one place the semantics is met.

11.3 What cannot be reproduced today

- Clean’s bus balance over $\mathbb{K}$ is unsatisfiable, not merely unsound: leanISA’s `BalancedPair` stands in until Clean’s directed, integer-counted balance lands and the pin reaches it (chapter 3).

- ArkLib admits the composition of knowledge soundness (ported locally), the implications from round-by-round to plain knowledge soundness and soundness, the sumcheck’s round-by-round leaf, context lifting, and the coding-theory theorem; it states no Fiat–Shamir or Merkle security; its ring switching is another protocol. None of this moved with the new pin. The chain to the target theorem waits on the implications and on Fiat–Shamir, and the review recommends stating the assumed interfaces for leanVM’s constructions now, so that what is waited for is at least written down.
- Lean’s kernel cannot decide the fields’ arithmetic fast: probes over $\mathbb{K}$ and $\mathbb{E}$ are `#guards` on tiny instances, and a numeral’s meaning in $\mathbb{K}$ changed with the new pin.
- No published theorem covers Fiat–Shamir with proof of work for a multi-round protocol; leanVM’s query rounds reach 128 bits only with it. The blueprint should say what security level its non-interactive theorem can certify and what remains a claim of the parameter analysis.

11.4 What to report to leanVM

The review changes nothing upstream and posts nothing; the matters below are what the owner may report, each with its source lines and its register row. *[list pending]*

11.5 The boundaries

What the proof system requires of leanISA. The relation `SatisfiedBy` as it stands, with its `Caps`, its integer-counted balance, its fixed-column conjuncts and its public-word conjunct whose top limb is zero by type; the constraint-soundness theorem with its hypothesis `WellFormedBytecode`, which the target theorem must carry; a resource condition on valid executions, without which completeness is false; and the two requests the blueprint already makes (power-of-two heights, bus data per channel), both met.

What it requires of the Flock roadmap. A predicate on a packed column (the R1CS with the constant position), a phase at the spine’s seams with perfect completeness for the specification’s honest prover and knowledge soundness at the accounted error, the witness generator with its lemma, the constants and the slot map transcribed from the Rust verifier (they are not in the specification), and the ring-switching weight’s closed form. The roadmap has not started; the compiled verifier’s fixture cannot pass before its definition exists.

What it hands over. To the arithmetization: knowledge of `M3Holds` of the leanISA instance at the proof system’s error, carried by the adaptor to `SatisfiedBy`. To recursion: the compiled verifier with its fixed-claim settlement as the seam, and the extracted witness the recursion target needs, which the target theorem as sketched drops. To Ethereum: a verifier whose every check has a theorem behind it or a named reason why not, which is what chapter 14 sets out to make enforceable.

Chapter 12

ArkLib: what exists, what is pending, what is ours

The blueprint plans to write several generic components locally and to consume others from ArkLib at a later pin. Much of the sumcheck, the composition and the GKR work has moved upstream since the blueprint was written, at revisions later than either pin and in open pull requests. This chapter says, component by component, what ArkLib implements at the new pin (7653a901), what has landed on its `main` since, what its open pull requests would supply and how close they are, what the blueprint plans locally that already exists upstream, and what remains leanerVM's own because nothing upstream is planned for leanVM's specific shapes. It ends with a revised upstream ledger for the blueprint. The survey was made by a review agent from ArkLib's history and pull requests; its Markdown form is `dossiers/arklib-landscape.md` on the review branch. *[The survey is being made; its chapter is inserted here when it lands.]*

Chapter 13

The documentation: one source of truth

The blueprint, the status, the tracker with its hole comment, the review handoffs and the lists of findings overlap, use different terms for the same things, and disagree with the repository. This chapter inventories which document says what, lists the contradictions with evidence, proposes one owner per kind of fact with the others pointing to it, one vocabulary, and the treatment of the letter codes; appendix D holds the resulting texts, ready to apply, and appendix C the index of every code. Nothing here has been applied: the review changes no document and posts nothing.

13.1 Which document says what

Revisions and abbreviations. The object of the review is `main` at `b435631`. `main` has since moved to `144c5aa` (pull request 61, the upgrade to Lean 4.34.1, which moved every library pin; brief, section 8), and the checkout now contains it. Citations `file:line` are of `b435631` (`git show b435631:<path>`) unless a passage says `144c5aa`; the status file is identical at the two revisions, and the blueprint differs by 6 lines (its lines after 50 are shifted by 4 at `144c5aa`). The proposed texts of appendix D describe `144c5aa`, as the orchestrator asked. For the tracker, citations are lines of the raw texts saved under `.claude/reports/blueprint-review/probes/docs-debt/` (`issue-12-body.md` for the body, `issue-12-comment-5833669972.md` for the “hole comment”). “BP”, “ST”, “TB”, “HC”, “OC”, “RS”, “RL”, “RP” in tables abbreviate the blueprint, the status, the tracker body, the hole comment, the other comments, and the handoffs of the reviews of the spine, of Layer 1 and of the public-input phase.

Columns. BP: the blueprint. ST: the status. TB: the tracker body. HC: the hole comment. OC: other comments (four on issue 12, one on issue 3, one each on pull requests 39, 42, 43). RS, RL, RP: the review handoffs of the spine, of Layer 1, of the public-input phase. AR: `architecture.md`. LT: `leanvm-target.md`. PR: pull-request descriptions. II: intention issues. A cell gives line numbers; a dash means absent.

Table 13.1: Which document says what: kinds of fact against places (line numbers; a dash means absent).

Kind of fact	BP	ST	TB	HC	OC	RS	RL	RP	AR	LT	PR	II	Elsewhere
Targets: the theorems wanted	11-31, 543-548, 1124-1130, 1218-1227, 1247-1250	23-25, 81-83	12, 14	16, 176, 190	issue 12 comment of 09-28	49-62	-	85-98	261-275	123	58	-	README.md:19-33
Signatures	442-554 and every layer, 633-1250	41-51, 98, 101	14, 29-34	every section	the comments on 39, 42, 43 and on issue 3	66-249, 493-649, 667-766	28-43	552-626	-	-	58, 59, 60	28, 31, 33, 37	-
Conventions	308-335	214-268	96	-	-	-	-	-	48-57	-	-	-	CONTRIBUTING.md:44-121, AGENTS.md:57-82
Acceptance tests, tests per unit	1261-1358 and the “Tests:” paragraph of each layer	by reference	-	22, 38, 71, 85, 97, 125, 152, 166, 180, 194	-	466-475	91-98	648-659	-	-	-	28 to 37, last paragraph	-
Coverage: what is built	leaks at 260, 339-345, 591, 611, 1105, 1143, 1459	15-83, 85-109	14, 20-40	9, 131-142	issue 12 comment of 09-28	30-47	24-43	64-83	-	112-128, not updated	58, 59, 60	-	README.md:19-33, docs/README.md:31-47, leanth-reuse.md:411-445
Frontier: what can start	1455-1470	134-164	42, 80-85	-	issue 12 comment of 09-28	-	-	-	-	-	-	-	-
Claims: who is doing what	-	92-97	23-28	“Claim” line of each section	three comments of 09-20 and 09-24	-	-	-	-	-	-	28, 31, 33, 37	the label hole
Open pull requests	598-610, column “Existing work”	111-132	44-57	-	same comments	406-422	-	-	-	-	GitHub	-	-
Findings against the leanVM sources	cited at 193, 323-326, 973, 1275, 1289-1291, 1330; stated in full at 322, 1053-1062	279-330	85	66, 80, 120, 176	-	283-322	112-169	239-375	-	57-62, one finding	60	-	leanisa-status.md:634-915
Findings about the libraries and Lean	281-284, 297-298, 569-570	332-432	84-85	-	-	355-381	-	-	-	-	-	-	dependencies.md:52-55, 81-88, development.md:56-57, tests/README.md
Decisions, taken and pending	556-563 and, silently, the conventions	212-277	87-89	13, 24, 67	issue 12 comment of 09-28; the comment on issue 3	34-38	-	71, 568-573	-	-	60	-	leanth-reuse.md:484-489
Upstream ledger	253-267	166-184	59-74	per section	-	-	-	-	-	-	58	31	leanth-reuse.md:447-458, dependencies.md:47-51
Upstream watch	598-614, column “Existing work”	186-210	8, 76-78	40, 111, 154, 168, 182	-	-	-	-	-	-	-	28 to 37	leanth-reuse.md:461-482
Record of reviews	1337	124-132, 438-536	14	3	issue 12 comment of 09-28	whole	whole	whole	-	-	59, 60	-	docs/README.md:31-47
How to work	1455-1515	7-11, 188	18, 91-96	3 and every “Claim” line	-	-	-	-	-	-	-	-	CONTRIBUTING.md, AGENTS.md, development.md, the labels

Kind of fact	BP	ST	TB	HC	OC	RS	RL	RP	AR	LT	PR	II	Elsewhere
Pins and dependency contracts	160-306, 1517-1549	10-11, 161-163, 565-622	3-4, 14	-	-	3-12	3-14	8-19	-	1-24	58, 59, 60	-	dependencies.md, upstreams.json
The list of public names	1360-1429	41-51, 72-76	14	-	issue 12 comment of 09-28	667-714	-	481-525	-	-	58, 59, 60	-	-
Prior work and reuse	170-174, 348-353, 1545-1549	206-210	-	-	-	-	-	-	-	-	-	28 to 37	leanth-reuse.md

Reading the table. Every row but one has at least three filled cells. The three worst cases:

- **Signatures** are written in nine kinds of place. A unit such as the bus phase has four specifications: the hole table (`protocol-blueprint.md:606`), the Layer 6 sketch (`:949-979`), the hole comment (`issue-12-comment-5833669972.md:61-73`) and the checklist line (`issue-12-body.md:30`). Section 13.2 shows that they give three different types for it.
- **The upstream ledger** is written three times, with different columns and different rows. The blueprint’s has nine rows about ArkLib and a column “Action” (`protocol-blueprint.md:257-267`). The status’s and the tracker’s have twelve rows: they drop the context-lifting row and add four about Clean and leanISA (`protocol-status.md:171-184`, `issue-12-body.md:61-74`). The column “Needed by” names layers in the blueprint and holes in the other two.
- **The record of a review** is written four times: the handoff’s disposition table, the status’s survey record (`protocol-status.md:438-536`), the tracker body (`issue-12-body.md:14`), and one line of `docs/README.md:38-47`.

Facts that exist only on the tracker, against the tracker’s own claim to hold nothing of its own: the tests of nine units (`issue-12-comment-5833669972.md:38, 71, 85, 97, 125, 152, 166, 180, 194`), for example “blake2sBytes on the RFC 7693 test vector” (`:166`) and “every pinned coding parameter with an achievability witness” (`:152`); the remark that leanVM’s chain is Merkle-Damgård and not a duplex sponge, “so the transfer theorem, not the sponge, is what applies” (`:182`); the review order of the open pull requests (`issue-12-body.md:48-57`); and what the spine asks of the Flock roadmap, which is a comment on issue 3 (id 5872015596).

13.2 Contradictions and stale statements

Each item quotes both sides. “Today” means main at b435631 and GitHub on 2026-09-29.

13.2.1 The status, after the merge of pull request 60

Table 13.2: The status, after the merge of pull request 60.

#	The status says (<code>docs/roadmap/protocol-status.md</code>)	Today
1	<code>protocol-status.md:3-6</code> : “as of main at f4d858c (Layer 1, PR #59, merged on 2026-09-29) ... together with ... the public-input phase (hole P5) as built on the branch feat/protocol-public-input (pull request #60); this snapshot accompanies that branch”	main is b435631, “feat(protocol): the public-input phase (#12) (#60)”; pull request 60 merged on 2026-09-29 at 15:57 UTC (<code>probes/docs-debt/pr-60.json</code>)
2	<code>:58-60</code> : “The public-input phase (hole P5) is built, on the branch feat/protocol-public-input, pull request #60, a draft”	merged; <code>LeanerVM/Protocol/PublicInput.lean</code> is on main (400 lines)
3	<code>:101</code> : “built, draft #60 ... lands when #60 merges”	landed
4	<code>:119</code> : “#60 (draft) ... mark ready, review and merge”	merged
5	<code>:146-148</code> : “is the draft #60 ... #60 ticks P5 on the dashboard when it merges”	merged, and the box is not ticked (<code>issue-12-body.md:32</code>)
6	<code>:161</code> : “ArkLib main is 246 commits past dca90385”	347 commits on 2026-09-29 (<code>git rev-list --count dca90385..origin/main</code> in the sibling checkout after <code>git fetch origin</code> ; origin/main was 7653a901e). The figure 246 is exact for 66f39b4, the revision the status names at :345
7	<code>:177</code> : “staged in ToCompPoly/{Stacking,AmbientStacking,Claims}.lean (#59)”	there is no <code>ToCompPoly/Claims.lean</code> ; the claim record is <code>LeanerVM/Protocol/BlockClaims.lean</code> , as the same file says at :50 and :415-420
8	<code>:190</code> , table head: “State (checked 2026-09-24)”, in a file whose head says “checked on 2026-09-29” (:4) and whose first row is dated 2026-09-28	mixed dates in one table
9	<code>:192</code> : ArkLib pull request 615 “open since 2026-09-08”; :193: ArkLib 818 “open since 2026-09-04”; :204: Clean 446 “draft since 2026-08-16”	created 2026-07-07, 2026-08-31 and 2026-08-07; the dates given are the dates of last update (<code>probes/docs-debt/upstream/*.json</code>)
10	<code>:183, :203</code> : Clean pull request 464 is “draft, the maintainer’s”	not a draft (<code>isDraft: false</code>), review required; its author is the owner of leanerVM, so “the maintainer’s” is ambiguous
11	<code>:205</code> : VCVio 784 “merged 2026-09-24”	merged 2026-09-23 at 00:41 UTC

#	The status says (<code>docs/roadmap/protocol-status.md</code>)	Today
12	:178: “ArkLib #4 (Merkle)” as the upstream issue of the WHIR and Merkle entry	ArkLib issue 4 was closed as “not planned” on 2026-09-27, its tracking transferred to VCVio issue 571: “The Merkle implementation and its security work are maintained upstream” (<code>probes/docs-debt/upstream/ArkLib-4-comments.txt</code>). See section 13.2.9
13	:281-283: “ F = Rust versus specification, continuing the leanISA numbering where the subject overlaps”	in the leanISA status the letter for Rust against the specification is R, and F is “the roadmap’s or the architecture’s targets” (<code>leanisa-status.md:636-640</code>). Nothing is continued: both files have their own F1 to F10
14	:212, heading “Decisions pending”, over a section of which 58 lines out of 66 (:212-269) are decisions taken	the blueprint’s rule is that a decision taken is “removed from the status file and the dashboard” (<code>protocol-blueprint.md:1506-1507</code>)
15	:179: ArkLib’s coding-theory track, “the #907 slices landing on main”; the hole comment says the same, “landing daily on main” (<code>issue-12-comment-5833669972.md:154</code>)	ArkLib issue 907 was closed as completed on 2026-09-26 (<code>probes/docs-debt/upstream/ArkLib-907.json</code>)

13.2.2 The tracker body, against main and GitHub

Table 13.3: The tracker body, against main and GitHub.

#	The tracker body says (<code>issue-12-body.md</code>)	Today
1	<code>issue-12-body.md:21: “- [] L1 — Layer 1 ... Generic half in draft #18 (rebase onto main pending ...) ... in review; nothing lands before #18 does”</code>	Layer 1 merged as pull request 59 (<code>f4d858c</code>); pull requests 18, 38, 40, 41 are closed
2	<code>:32: “- [] P5 — the public-input phase ...: one challenge, one pooled column claim per line, the prover sends nothing. — open; the smallest phase, a good first hole”</code>	built and merged, and the prover sends a message: <code>Public Input.pSpec</code> is “the challenge in E, then the prover’s values, a List E” (<code>protocol-status.md:101</code> ; <code>blueprint :1013</code>)
3	<code>:44-57, “Open pull requests (2026-09-28)”: eight rows, among them 18 (draft), 40, 38, 41</code>	open today: 39, 42, 43 only
4	<code>:82: “#18 must be rebased and landed before the five pull requests stacked on it can get CI”</code>	18 is closed; its content landed through 59
5	<code>:83: “The status file at main still describes the spine as on its branch and in review; the next pull request that lands a hole rewrites it whole”</code>	the status was rewritten twice since; the sentence is about a state two merges old
6	<code>:40: “- [x] VCVio controls for K3 and P7. #29, #30 — landed upstream”</code>	issues 29 and 30 are still open, although the blueprint says an intention issue “is closed by the pull request that lands the slice” (<code>protocol-blueprint.md:1491</code>)
7	<code>:85: “Findings that bind a definition: F1, F3, F5, F6, F8, F11” and “new are S14 ..., E7 ..., E8 ... and E9”</code>	the status has since added the order of equal-size blocks (F17), which binds the adaptor’s layout, the two public-input checks (F18), which binds the compiled verifier, and nine findings about Lean
8	<code>:89: decisions up to 14</code>	decision 15 (the transcript of the public-input phase) was taken on 2026-09-29 (<code>protocol-status.md:256-268</code>)
9	<code>:6: “This issue holds nothing that is not in them”</code>	see the end of section 13.1
10	<code>:1: “Revision 2 (2026-09-24, <i>The spine</i>) is on main”</code>	the blueprint carries no revision number or date; “revision 2” is defined nowhere
11	rendered body: “ArkLib #1”, “ArkLib #4”, “leanth #16”, and “(sumcheck; #3 there is closed)”	GitHub links them to pull request 1 and issues 4, 16 and 3 of leanerVM: the wrong repository (<code>probes/docs-debt/issue-12-body.html</code> ; the same for “ArkLib #1” once and “ArkLib #4” twice in the hole comment)

The Layer 1 review left exactly these two edits for GitHub: “On GitHub: the L1 line and the open pull request table of the dashboard #12” (`docs/reviews/protocol-layer1.md:211`). Neither was made (items 1 and 3 above): a review handoff whose tracker edits are not applied leaves the tracker wrong while the handoff’s disposition table reads “met”.

13.2.3 The hole comment, against the blueprint and the code

The comment says of itself that the spine’s section “is kept for the record” and that four sections “were rewritten on 2026-09-28” ([issue-12-comment-5833669972.md:3](#)). The other eight were not, and they are the specifications of the units still to be built.

Table 13.4: The hole comment, against the blueprint and the code.

#	The hole comment says (issue-12-comment-5833669972.md)	The blueprint or the code says
1	:66, bus phase: “busPhase I (G : Gkr.Def) : Phase.Def I Unit (BusOut I) (busSpec I) (M3Holds I) (Seam.bus I)”	Phase.Def takes the instance and two statement types (Phase.lean:37); the bus phase is Phase.Def I I.Stmt (I.Stmt × BusOut I) (Compose.lean:76). There is no Gkr.Def and no busSpec; the comment’s own note says so at :9
2	:80, table sumcheck: “tableSumcheck I (S : Sumcheck.Def) : Phase.Def I (BusOut I) (TableOut I) (tableSpec I) (Seam.bus I) (Seam.table I)”	the same: six arguments where there are three, Sumcheck.Def and tableSpec do not exist
3	:120, opening phase: “openingPhase I (B : Batch.Def) (S : Sumcheck.Def) : Phase.Def I (FlockOut I) Unit (openSpec I) (Seam.flock I) (fun _ => True)”, and “Claim.toWeighted (a BlockClaim of #38 ...)”, and “Consumes: ... Pool” (:123)	no Pool (:9 of the same comment); the Layer 1 review established that a phase over an abstract instance cannot form a BlockClaim and that the step is ColumnClaim.holds_iff_weighted (protocol-layer1.md:28, protocol-status.md:127-129)
4	:66: “leaf_decomposition (specification (5.4) ...)”	“equation (5.4)” does not exist; it is equation (2) of section 5.4 (protocol-layer1.md:36, protocol-status.md:303)
5	:32, :148, :162: modules under LeanerVM/Protocol/Generic/	the convention names ToArkLib/, ToCompPoly/, ToVCVio/ (protocol-blueprint.md:329); the blueprint’s Layers 5 and 11 name ToArkLib/GrandProduct.lean and ToArkLib/Whir.lean (:916, :1152)
6	:92-94, public-input phase: “the prover sends nothing”	blueprint :1030-1033: “The prover sends, as one message, the values at r of the lines whose value is sent”; decision 15
7	:190, base theorems: “composed as verify_knowledgeSound ◦ knowledgeSound_of_refinement”	the same comment at :9: “Refinement.map_option_valid replaces knowledgeSound_of_refinement”; blueprint :1253
8	:178: “Consumes: the spine’s six ProtocolSpecs”	the spine fixes one schedule, the commit phase’s; the others “travel with each Def” (protocol-blueprint.md:419-422; the comment’s own note at :9)
9	three ways to claim: “by assigning yourself” (:42, 73, 87, 127, 156, 170, 184, 196), “by assigning yourself and commenting; slices go through [Intention] issues as usual” (:26), “by commenting on this issue” (:59, 100, 113)	the blueprint: “comment on #12 ... No [Hole] or [Intention] issue is opened for it” (:1487-1489); the label hole on GitHub: “claim it by assigning yourself”

The closed [Hole] issue of the public-input phase (issue 50) gave yet another specification: “its Phase.Security with error $2/|E|$ ” ([probes/docs-debt/issue-50-body.md](#)), where the blueprint and the code have $1/|E|$.

13.2.4 The blueprint against itself: the per-layer sketches and the spine

The blueprint announces the difference in one sentence, “where a signature below says (prog) (s), read leanIsaInstance prog s” (protocol-blueprint.md:622), and for Layer 10 says that the master theorems “below are the spine’s” (:1104-1105). The sketches differ by more than that reading.

Table 13.5: The per-layer sketches against the spine.

#	Sketch (docs/roadmap/protocol-blueprint.md)	Spine section of the same document, and the code
1	Layer 6, :956-964: “def busPhase (prog) (s) : OracleReduction [] ◦ (StmtIn := Unit) ... (StmtOut := BusOut)” and “structure BusOut where ζ : Fin μ _{bus} → E ; rem : Fin 3 → E ; pool : List Claim ; α : Fin 4 → E ; β : E”	:481: “structure BusOut I where (linear : List (LinearClaim I)) (columns : List (ColumnClaim I))”, and :560: “The seams carry <i>claims</i> , not challenges”. Code: Seams.lean:130-135, quoted above
2	Layer 7, :988: “def tableSumcheck (prog) (s) : OracleReduction [] ◦ (StmtIn := BusOut) ... (StmtOut := BusOut × ColumnClaims)”	:533: “table : Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I)”

#	Sketch (docs/roadmap/protocol-blueprint.md)	Spine section of the same document, and the code
3	Layer 9, :1075-1085: the docstring says the structure supplies “the Phase.Def and Phase.Security at the flock seam”; its fields are “reduction : OracleReduction [] (StmtIn := ColumnClaims) ... (StmtOut := Weighted Claim)”, “relIn”, “relOut”, “rbrKnowledgeSoundness : reduction.verifier.rbrKnowledgeSoundnessWorstCase ... flockError”; “limbColumns : Column $\mu \rightarrow$ Fin 18 $\rightarrow$ Column (s. τ 5)” with s unbound	:535: “flock : Phase.Def I (I.Stmt $\times$ PubOut I) (I.Stmt $\times$ FlockOut I)”; a phase’s security is the named-extractor form (:332, :512)
4	Layer 10, :1109-1110: “structure Weight (μ) where (on Cube : ETable μ) (mle : (Fin $\mu \rightarrow$ E) $\rightarrow$ E) ...”, “structure WeightedClaim (μ) where (W : Weight μ) (c : E)”	:479-480: “structure Weight μ where (onCube : CM1 PolynomialEval E μ) (mle : Vector E $\mu \rightarrow$ E) ...”, “structure WeightedClaim I where (weight : Weight I. μ) (value : E)”. Code: Seams.lean:103-120
5	Layer 10, :1116-1130: “def leanVmPiop (prog) (s) (input) (flock : FlockInterface prog s) : Oracle Reduction [] (StmtIn := PublicInput) (OStmtIn := fun _ : Empty $\rightarrow$ Unit) (M3Witness prog s) ...”, “theorem piop_rbrKnowledgeSoundness (hs : s.Admissible prog) : (leanVmVerifier prog s input flock).rbrKnowledge Soundness init impl (m3Relation prog s) Set.univ (piop Error s)”	:541-547 and the code quoted above: over (P : Phases I) (S : P.Security), no admissibility hypothesis, rbr KnowledgeSoundnessWorstCaseWith, M3Rel I, Seam.done I, a named extractor
6	Layer 4, :894-895: “theorem sumcheck_rbrKnowledge Soundness : (sumcheck V).verifier.rbrKnowledge SoundnessWorstCase ...”; Layer 5, :936-937, the same form for the grand product	convention <i>Holes</i> , :332: the form “rbrKnowledgeSoundness WorstCaseWith ...”, so the extractor is named, not merely shown to exist”
7	Layers 6, 7, 10 need “Layer 5’s Gkr.Def” (:952), “Layer 4’s Sumcheck.Def” (:984), “Sumcheck.Def and Batch.Def” (:1104); the hole table and the picture name them too (:397, :598-602)	no such shape exists in the spine section or in the code (grep over LeanerVM/ and tests/: no match); a generic component is a Component.Def
8	Layer 2, :768-773: “flushes : List (Direction $\times$ Vector (MvPolynomial (Fin width) F) 16)”, “count : List (Fin width)”	:458, :462: “flushes : (j) $\rightarrow$ List (Side $\times$ Vector (CMv Polynomial (width j) K) 16)”, “counts”. Direction is also ArkLib’s name for the direction of a message, listed as such at :234
9	used and never defined in the document: Claim (:963, :1111), ColumnClaims (:988, :1078), LeafLayout (:955), StackLayout (:1111), M3Witness (:1117), m3Relation (:1126), commitPhase (:1119, :1133)	the spine has ColumnClaim, Layout, M3Rel, commitDef
10	heading :617: “The build: the spine, then fourteen layers”	:620: “The spine (hole S) is built after Layer 0”. The count is right (Layers 0 to 13); the order in the heading is not it does: “done on the spine’s branch” (:611, :1459-1460), “The port is done and closes hole C1” (:1143), “(#13, taken)” (:221), “the [Hole] issues were folded there on 2026-09-25” (:591), “L1 groups the leaves of Layer 1 already in flight” (:591-592), the column “Existing work” with open pull requests (:598-614), “a rule in scripts/ check-layers.sh is still to be written” (:1344-1345)
11	:1474-1475: “This document ... does not record history, status, or who is doing what”	
12	:592-593: “the generic holes carry the ledger letters of the ArkLib work they will become”	the generic units are coded G1 to G6; the ledger’s letters are A and C. The sentence has no reading under which it is true

13.2.5 The blueprint against the repository

The probe `check_interface_names.py` looked up the 194 names of the list “Interfaces supplied to later work” (protocol-blueprint.md:1366-1416): 127 are declared, 3 are structure fields, 64 are absent. All 64 belong to units not yet built, which is expected. The four names the task asked about:

Name	Result
Blocks.stack_eval ₂	exists: LeanerVM/Protocol/ToCompPoly/Stacking.lean:345
Toy.layout	exists: LeanerVM/Protocol/Spine/Toy.lean:60
BlockClaim	exists: LeanerVM/Protocol/BlockClaims.lean:54
Ensemble.toM3, listed under the heading “Spine:” (:1369) and again under “Arithmetization:” (:1398)	absent; it is Layer 2’s, not built, and not the spine’s

What the probes did find:

Table 13.7: The blueprint against the repository.

#	The blueprint says (docs/roadmap/protocol-blueprint.md)	The repository
1	:1419: “Everything not listed is a proof, a helper, or a test”	of 333 non-private declarations under <code>LeanerVM/Protocol/</code> , 198 are not in the list, among them definitions the blueprint itself says later work reads: <code>PublicInput.check</code> , <code>pooled</code> , <code>prover</code> , <code>verifier</code> , <code>pSpec</code> (blueprint :1049-1051: “Layer 12 reads those”), <code>publicInputComplete</code> , <code>publicInputSecurity</code> , and <code>Component.sendSpec</code> , <code>sendExtractor</code> , which <code>commitSpec</code> and <code>commitExtractor</code> abbreviate (<code>probes/docs-debt/unlisted_public.out</code>)
2	:343-344: the spine’s generic modules are “Oracles, Component, KnowledgeAppend, PassThrough, SendOracle, Refinement”	<code>LeanerVM/Protocol/ToArkLib/</code> also holds <code>GuardedVerdict.lean</code> and <code>KeepOracles.lean</code> , which the pass-through and send components now import (<code>protocol-status.md:74-75</code>); <code>KeepOracles</code> appears nowhere in the blueprint
3	:1306-1307, acceptance test 14: “Witness: bytecode Column_slot on a two-instruction program”	the Layer 1 review showed that <code>bytecodeColumn_slot</code> “restates the definition and does not pin the bit order” and that the statement which does is <code>bytecodeColumn_answer_boolVec</code> (<code>protocol-layer1.md:30, 82-89</code>). Both exist (<code>FixedColumns.lean:85, 97</code>); the test with reversed slot bits is at <code>tests/LeanerVMTests/Protocol/FixedColumns.lean:99</code>
4	:220: <code>leanISA</code> supplies “ <code>MemMsg</code> , <code>StateMsg</code> , <code>BytecodeMsg</code> ”	<code>MemMsg</code> and <code>BytecodeMsg</code> exist (<code>Arithmetization/Channels.lean:153, 164</code>); <code>StateMsg</code> does not
5	:1280, acceptance test 5: “distinguished by <code>leanIsaTables</code> flush polynomials”	no <code>leanIsaTables</code> in the code or in any other document
6	:603, :608, :609, :610, column “Produces”: the opening phase produces “ <code>Weight</code> , <code>WeightedClaim</code> , <code>openingPhase</code> ”; the Flock phase produces “ <code>FlockOut</code> , ...”	<code>Weight</code> , <code>WeightedClaim</code> and <code>FlockOut</code> are the spine’s and are on <code>main</code> (<code>Seams.lean:103, 116, 147</code>). A reader, or a script, counting produced names takes the opening phase for three-quarters built (<code>probes/docs-debt/coverage_from_blueprint.out</code>)
7	:1512: “the reviewer runs the <code>leanerVM-review</code> skill’s three passes”	the skill is <code>adversarial-review</code> (<code>.claude/skills/adversarial-review/SKILL.md</code>), and it is what the status and the three handoffs call it (<code>protocol-status.md:438, 479, 506</code>). Moreover <code>.claude/</code> is git-ignored (<code>.gitignore:9</code>), so neither this skill nor <code>lean-spec-authoring</code> , which “sets the rules” at :178, is in the repository a contributor clones
8	:601, and <code>protocol-status.md:95</code> , <code>issue-12-body.md:26</code> : the fingerprint unit is “fingerprint, Lemma 5.1, the collision bound”	the specification has Theorem 5.1 (the collision bound, <code>05-arithmetization.tex:36</code>) and Lemma 5.2 (a product identity is a multiset identity, :40); its environments share one counter per section (<code>doc/leanvm/preamble/theorems.tex:4-6</code>). There is no Lemma 5.1. Layer 5 of the blueprint has it right (:921, 924)
9	:1440: “virtual columns here (finding F3 of the status file)”	in the protocol status that code is “one root for push and pull” (<code>protocol-status.md:309-310</code>). The finding meant is the <code>leanISA</code> status’s: “the BLAKE2S value limbs are virtual columns in Flock’s stack in the Rust; ordinary columns here” (<code>leanisa-status.md</code> , section “Targets”)
10	<code>README.md:32-33</code> : “no proof-system claim has landed”	the spine with its two conditional master theorems, Layer 1 and the public-input phase with <code>publicInputSecurity</code> are on <code>main</code>
11	<code>docs/README.md:38-47</code> : three handoffs “whose findings the branch now meets”	the three branches are merged
12	convention <i>The wall</i> , :331: “imports nothing from <code>LeanerVM/Arithmetization/</code> ; the only exceptions are the fixed columns (<code>FixedColumns.lean</code> , Layer 1), the Clean bridge (Layer 2), the adaptor (Layer 3), the compiled verifier (Layer 12) and T4 (Layer 13)”; acceptance test 25, :1343-1344: the <code>grep</code> “lists the exceptions of convention <i>The wall</i> only”	<code>LeanerVM/Protocol/Basic.lean:3</code> is <code>public import LeanerVM.Arithmetization.Basic</code> . The module is an empty placeholder (a docstring and an empty public section) that nothing under <code>LeanerVM/</code> or <code>tests/</code> imports, so no phase sees the arithmetization through it; but the <code>grep</code> the blueprint names as the witness of the import rule returns it, and the rule as written is false of <code>main</code>

#	The blueprint says (docs/roadmap/protocol-blueprint.md)	The repository
13	convention <i>Generic code</i> , :329: “Comments everywhere are brief and self-contained: they cite the specification, never this roadmap”	LeanerVM/Protocol/Field.lean:18 “Protocol roadmap Layer 0 (docs/roadmap/protocol-blueprint.md)”, :29 “(leanISA status finding P3)”, :31 “every error bound of the roadmap”, :37 “(roadmap convention <i>The oracle</i>)”, :37, 66 “Layer 11”; tests/LeanerVMTests/Protocol/Field.lean:9 “Protocol Layer 0 tests”. Layer 0 predates the rule (merged 2026-09-11 in 51021d9); every later module of LeanerVM/Protocol/ follows it (the same grep over the other 24 files finds nothing)

13.2.6 The blueprint against the leanISA blueprint and architecture.md

Table 13.8: The blueprint against the leanISA blueprint and architecture.md.

#	The proof-system blueprint (docs/roadmap/protocol-blueprint.md)	The other document
1	:1247-1248: “theorem baseVerifier_extractsExecution (fs bcs mca flock) (h : verify prog input proof = true) : except with probability niError, $\exists$ t, ValidExecution prog input t”, which “composes ... constraintSoundness” (:1253-1254)	leanisa-blueprint.md:1163: “theorem constraint Soundness (hwf : WellFormedBytecode prog) (h : SatisfiedBy prog input w) : ...”. architecture.md:224-228: “Both directions are stated for well-formed programs ... a hypothesis of each, since the constraint system enforces neither”. leanvm-target.md:57-62 records why: a bytecode whose sentinel slot holds a JUMP “admits accepted walks that execute the sentinel”. The base theorem as written has no such hypothesis and cannot be obtained by the composition it names
2	:1249-1250: “theorem baseProver_complete (hfill : Has FillBlocks prog) ...”, which “composes constraint Completeness” (:1255); :222 lists “constraintSoundness, HasFillBlocks, constraintCompleteness” as what is consumed	leanisa-blueprint.md:1156-1166: constraintCompleteness takes WellFormedBytecode prog, a structure with two fields, sentinelSafe and hasFillBlocks. HasFillBlocks alone does not give it
3	:147-148: “witness generation from an execution (T2): the honest prover here starts from an EnsembleWitness”; completeness composes leanISA’s existence theorem, the completeness half of arithmetization and ISA equivalence (T1) (:1255)	architecture.md:273-275: the completeness dual of base proof extraction and completeness (T4) “composes T2 with the honest prover”
4	the hole comment repeats items 1 and 2 word for word (issue-12-comment-5833669972.md:190)	-

Item 1 is more than a wording issue and is a finding in section H. Whether the intended theorem takes `WellFormedBytecode prog` as a hypothesis, or the compiled verifier checks it on the public program, is a technical decision for the orchestrator; this dossier only records that the documents disagree and that the statement as written is not provable by the named composition.

13.2.7 How work is tracked: the text against practice and against the owner’s wish

Table 13.9: How work is tracked: the text against practice.

#	Text (a bare line number is a line of docs/roadmap/protocol-blueprint.md)	Practice
1	protocol-blueprint.md:1487-1488: “To claim a hole, or a slice of one, comment on #12”; issue-12-body.md:18, 93 the same	the owner now wants issue bodies updated rather than comments added (the task’s statement). A contributor without write access cannot edit the body: “I lack label/dashboard permissions” (issue-12-comment-5749712916.md, and again in the two later comments). The four live claims are in the bodies of intention issues 28, 31, 33, 37, not in comments on issue 12

#	Text (a bare line number is a line of docs/roadmap/protocol-blueprint.md)	Practice
2	:1489-1491: an intention issue is titled “[Intention]: protocol - <hole>: ...”	all ten intention issues are titled by layer: “[Intention]: protocol - Layer 4: ...”
3	:1466-1470: a pull request’s description names “the hole, the sources and pin, the category of each new definition, the ledger entries it touches, and the T4 obligation it feeds”	the open pull requests 39, 42, 43 name a layer and an intention issue; none names a hole, a category, a ledger entry or the target (probes/docs-debt/pr-39-body.md, pr-42-body.md, pr-43-body.md)
4	:1511: “Pull requests carry awaiting-review when the author is done”	39, 42 and 43 carry no label; their author asked three times for it to be set
5	:1480-1484: issue 12 is “the layer checklist with one of open / claimed / in review / landed per layer”	the checklist is per hole (issue-12-body.md:16-40); the status uses a fifth state, “built, draft” (protocol-status.md:101)
6	:1498-1501: “Durable source discrepancies are also recorded in leanvm-target.md”; protocol-status.md:281: findings are “numbered for citation from pull requests and docs/leanvm-target.md”	leanvm-target.md records one discrepancy, the sentinel (: 57-62), and cites no number. None of the proof system’s twenty-seven findings against the sources is there
7	CONTRIBUTING.md:16-19: “For a large contribution ... open a design issue first”	the blueprint: no issue is opened for a hole (:1488-1489)
8	protocol-status.md:553-554: the reading audit of the stacked pull requests “is recorded on the dashboard’s history”	a fact kept in the edit history of an issue body is not retrievable by a reader of the repository

13.2.8 Decisions

Decisions 1 to 5 were stated as questions in the first version of the status (commit 51021d9). Today the status says only “Decision 1 was taken with #13, and decision 3 with revision 2” (protocol-status.md:214): what decision 1 decided is no longer written anywhere in the current documents, and decisions 6 to 10 exist only as outcomes (:217-224). Two further choices “were made” without a number (:224-227: the seams carry claims only; the schedules travel with the definitions). The blueprint cites “decision 8” and “decision 6” (:557, 560) and defines neither. The leanISA roadmap numbers its own decisions from 1, so “decision 13” is “public lines, not cells” here and “the program is a parameter” in the title of issue 22, and leanth-reuse.md:485, a proof-system document, cites “Decision 14” of leanISA without saying so.

13.2.9 The upstream ledger’s row on Merkle trees

The blueprint’s ledger says of ArkLib at the pin: “A7 WHIR, Merkle, BLAKE2s | absent (only coding-theory lemmas) | Layer 11 | Written here” (protocol-blueprint.md:265), and Layer 11 plans merkleRoot and merkleVerify locally (:1179-1182). The blueprint’s list of what is consumed from VCVio has one row, SampleableType (:251), and the survey record lists the same (protocol-status.md:607-609).

Fact, at the pin f9dc47d9 of VCVio (the revision leanerVM builds against, through ArkLib): the directory .lake/packages/VCVio/VCVio/CryptoFoundations/MerkleTree/ holds nineteen modules (Inductive/{Defs,Completeness,Binding,Extractability,Extractor,QueryBound,Uniqueness}.lean, four under Inductive/Batch/, six under Addressed/, two under Vector/), with no sorry in any of them (grep -rn sorry: 0 lines), for example theorem functional_completeness at Inductive/Completeness.lean:41. And ArkLib’s Merkle tracker, which four documents cite as the upstream of this work, was closed on 2026-09-27 in favour of VCVio issue 571.

Inference, not verified: that this library fits leanVM’s Merkle trees (BLAKE2s, leanVM’s leaf encoding and pruned paths). Verifying it means reading Inductive/Defs.lean against crates/fiat_shamir/src/merkle.rs:14-67; it belongs to the review of Layer 11. What is established here is that the ledger and the survey record do not mention a library that is in the build, and that the upstream destination they name no longer exists.

13.2.10 Cross-references checked, with the negative results

Table 13.10: Cross-references checked.

Check	How	Result
Local Markdown links and anchors, the 24 Markdown files tracked at b435631	check_anchors.py, anchors computed as GitHub does	150 links, 0 broken
The repository’s own link check	python3 scripts/check-docs.py	passes. It checks files only, not anchors, not external links, and matches per line
Citations into leanVM at the pin, blueprint and status	check_leanvm_citations.py	81 distinct citations; every one resolves to a file at a386121f with the cited range inside it and plausible first lines (the full list is probes/docs-debt/leanvm_citations.out). 22 are ambiguous as written, because the file name exists in two or more crates: transcript.rs, witness.rs, lib.rs, filler.rs, and the specification’s file names, which also exist under doc/leanvm/drafts/
Citations into the pinned libraries	check_lib_citations.py	43 declarations; 40 at the cited file and line (within two lines). Three drifted: Direction is in OracleReduction/Prelude.lean: 66, not ProtocolSpec/Basic.lean; fsChallengeOracle is an alias at ProtocolSpec/Basic.lean:854, not in FiatShamir/Basic.lean:114-138; Sumcheck.Domain is ProofSystem/Sumcheck/Domain.lean, which the blueprint’s list reads as under Spec/
References to numbered acceptance tests	every “acceptance test N” and “test N” of the eight documents read against the blueprint’s list	30 references, all to the intended test
Witnesses named by acceptance tests 13, 15, 24, 26, 27, 28	grep in tests/LeanerVMTasks/Protocol/	present: FixedColumns.lean:37-51, Stack.lean:83-90, Spine.lean:158-159, Spine.lean:113, Spine.lean:32-52. Test 14 names the wrong witness (section 13.2.5, item 3)
The spine sketch against the code	PublicLine, M3Instance, BusOut, Weight, WeightedClaim, Phases, the three theorems of Compose. lean read side by side with protocol-blueprint.md:442-554	they agree
The leanISA names the blueprint consumes (:215-222)	looked up in LeanerVM/	30 of 35 exist; constraintSoundness, constraintCompleteness, HasFillBlocks are leanISA Layer 10, not built; StateMsg and leanIsaTables exist nowhere (section 13.2.5)
The import rule of the proof system (acceptance test 25’s own witness)	grep -rn 'import LeanerVM. Arithmetization' LeanerVM/Protocol	two lines: FixedColumns.lean:11, an exception the convention lists, and Basic.lean:3, which it does not (section 13.2.5, item 12)

Not checked: external links other than the hole comment’s; the figure “3298 declarations” of the axiom audit (protocol-status.md:59), which needs a run of validate.sh; the line numbers the handoffs cite into the branches they reviewed, which no longer exist as such.

13.2.11 The pins, after the dependency upgrade (pull request 61, main at 144c5aa)

Pull request 61 (“chore(deps): upgrade to Lean 4.34.1”, merged 2026-09-29 22:31 UTC, commit 144c5aa) moved every pin but leanVM’s: ArkLib dca90385 → 7653a901, CompPoly 3468b38c → 572f9973, Clean 93c9d1ef → 42fe4b26, VCVio f9dc47d9 → a4232d08 (now a direct Lake requirement, lakefile.toml:40-43 at 144c5aa, beside a new direct PolyFun 41d3b21d), Lean and Mathlib v4.33.1 → v4.34.1 (git show 144c5aa:upstreams.json; git show 144c5aa:docs/dependencies.md, lines 5-15 and the section “Lean 4.34 port review”). It added one paragraph to the blueprint and rewrote the blueprint’s convention *Module system*; it did not touch the status. In this table only, blueprint and status lines are those of 144c5aa (the blueprint’s lines after 50 are those of b435631 plus 4).

Table 13.11: The pins, after the dependency upgrade (pull request 61).

#	The document says, at 144c5aa	Today
1	blueprint :52-54 (new): “The dependency API tables and signatures below record the original implementation baseline. [dependencies.md] records current pins and the 4.34 native-probability port, which retains the relations, extraction conditions, and error bounds.”	the only sentence of the blueprint that knows of the upgrade; the passages below still state the old pins as the current ones, and nothing says which of them are history

#	The document says, at 144c5aa	Today
2	blueprint :113 (Scope): “ArkLib as a Lake dependency at dca90385”	7653a901
3	blueprint :167-172: “The pins are in <code>upstreams.json</code> and <code>[dependencies.md]</code> : leanVM a386121f, ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef, VCVio f9dc47d9 (through ArkLib), Lean v4.33.1.”	four of the six pins changed, and VCVio is no longer “through ArkLib”
4	blueprint :257-261: “ Admitted at the pin, and what this roadmap does about it. ” and the column “ArkLib state at dca90385” of the upstream ledger	the ledger describes the old pin. Whether each admitted result is still admitted at 7653a901 is not established by any document of the repository
5	blueprint :633-635 (Layer 0): “Add ArkLib at dca90385 as the ArkLib requirement (its package name). ArkLib requires CompPoly at the v4.33.1 tag; the root pin 3468b38c wins the resolution”	<code>dependencies.md</code> at 144c5aa: “CompPoly’s current main revision overrides ArkLib’s v4.34.0 CompPoly requirement”, and Mathlib is now a root requirement
6	blueprint :1529-1530 (References): ArkLib “at dca90385”	7653a901
7	status :10-11: “The dependency pins are unchanged.”; :161-163: “ The pins have not moved. ArkLib main is 246 commits past dca90385 (finding A18); the next bump is one planned change (Lean 4.34, ...)”; the headings of its findings, “ ArkLib (dca90385)” :332, “ Clean (93c9d1ef)” :355, “ CompPoly (3468b38c)” :360	the planned bump has happened; the findings about the libraries are findings about revisions that are no longer the pins
8	tracker body :4: “ Framework: ... ArkLib at dca90385”; :14: “Pins unchanged: leanVM a386121f, ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef, Lean v4.33.1”; :84: “The pins have not moved.”	the same
9	LeanerVM/Protocol/Field.lean:18-20 at 144c5aa (updated by the upgrade): “the first consumer of ArkLib (pin 7653a901...) and of VCVio through it (a4232d08...)”	the pins are right; “through it” is not, VCVio being a direct requirement

Every other pin statement in the proof system’s documents that is a statement about history (the survey record, the review handoffs, “the port of ArkLib #615 at ca7a2577”) is right as history and should say so where it can be read as current. The proposed texts of appendix D state the new pins once, in the status, and elsewhere point to `upstreams.json`.

13.3 Vocabulary

Counts are whole-word, case-insensitive occurrences (`vocab_counts.py`; BP, ST, TB, HC as in section 13.1). “Keep” follows the brief’s vocabulary where it decides.

Table 13.12: Variants of one term, where each is used, and the term to keep.

One thing	Variants, with counts	Keep	Remark
The document that says what is wanted	“roadmap”: BP 25, ST 16, TB 13; “blueprint”: BP 5, ST 3, TB 3, HC 11, the handoffs 37. The file is <code>protocol-blueprint.md</code> ; its title is “Roadmap: ...”	blueprint	a reader of the tracker meets both words for one file in one sentence (<code>issue-12-body.md:1</code>)
The tracking issue	“dashboard”: BP 7, ST 2; “tracker”: BP 2; “tracking issue”: BP 1, ST 2; “this issue”	tracker	at <code>protocol-blueprint.md:1489</code> “the tracker” means all of GitHub’s issues, not issue 12
A unit of work	“hole”: BP 48, TB 29, HC 21; “layer”: BP 172, HC 40; “unit”: BP 29; “slice”: BP 9; “half”: BP 10; “leaf”: see below	“hole”, defined once (“a unit of work the spine leaves open: a phase, a generic component, the adaptor, or a piece of the compilation”), always with its name; “layer” for the blueprint’s section number	“hole” is not a word a zkVM engineer knows, so its definition goes in the index; a hole has up to four identifiers today (hole code, layer number, the closed issue’s number, the pull request). The earlier draft proposed “unit of work” instead; the second pass keeps “hole”, the brief’s word, the word of the spine’s Lean docstrings (“the hole interfaces”) and of 48 places in the blueprint. In section 13.5.1 and section 13.5.2 below, “unit of work” and “unit” mean a hole

One thing	Variants, with counts	Keep	Remark
The table of upstream gaps	“ledger”: BP 24, ST 10; “upstream ledger”: BP 2; “upstream watch”: BP 3, HC 5; “upstream candidates” (leanth-reuse.md:447)	“upstream ledger” for what is admitted or absent at the pin and what replaces it; “upstream watch” for external work in flight	two different tables; the blueprint’s hole table has a third, the column “Existing work”
The relation between two phases	“seam”: BP 62; “interface”: BP 28; “boundary”: BP 17	seam	“interface” also means an assumed structure (FlockInterface) and the list of public names; “boundary” also means the boundary blocks of the bus and the section “Boundaries” with other roadmaps
One step of the protocol	“phase”: BP 131; “reduction”: BP 17; “component”: BP 60; “stage”: BP 3	phase for the six steps; component for a generic piece	Component is three things: Clean’s table circuit (:291), the spine’s Component.Def (:501), ArkLib’s ProofSystem/Component/ (:244)
The committed object	“stack”: BP 41; “stacked columns”; “the oracle”: BP 51; “committed column/polynomial”; “q”	the stack	“column” is both a table’s column and the stack itself (Column μ)
A coordinate of E over K	“limb”: BP 24; “lane”: BP 5	limb	“lane” has three meanings: a word of the BLAKE2s block (:325), the four K-words of the public input in leanISA (leanisa-blueprint.md:356), and a track of work in the Flock roadmap (:1446). “limb” is also used for a column: “the three memory limbs”
The claims handed to the opening	“claim pool”: BP 5; “pool”, “pooled”: BP 19, RP 33	claim pool	“claim” is also the act of taking a unit of work (“claimed”: BP 7, TB 7)
Done	“landed”: TB 13, ST 9; “built”: BP 8, ST 5; “on main”: ST 10; “merged”: ST 16; “met” (of a finding): the handoffs 39; “done”; “taken” (of a decision)	“on main” for code; “met” for a finding; “taken” for a decision	the tracker defines four states (issue-12-body.md:18); the status uses a fifth, “built, draft”
The import rule	“the wall”: BP 7, ST 4, RL 6	“the import rule of the proof system”, stated once	a metaphor; the rule is one sentence (:331)
The map between the two relations	“adaptor”: BP 29; “adapter”: ST 1 (another thing, a compatibility file); “bridge”: BP 11; “refinement”: BP 13	adaptor for the leanISA one; refinement for the generic notion	“the Clean bridge” is Layer 2, a different unit
The protocol	“oracle protocol”: BP 11; “PIOP” in names (leanVmPiop); “IOR”: BP 1; “IOPP” (leanVmIopp): BP 5	oracle protocol	leanVmIopp names as a proximity proof the compiled protocol “WHIR in place of the evaluation oracle” (:1214-1215)
A bus entry	“flush”: BP 12; “interaction”: ST 10; “tuple”	flush, as the specification	“side” (Side, spine) against “direction” (Direction, Layer 2 sketch, and ArkLib’s message direction)
The public data	“public input”: ST 23; “public statement”: BP 3; “public words”: BP 6; “public line”: BP 4	public input for the two words; public line for the spine’s object	PublicInput is a leanISA type and also a namespace of the proof system (PublicInput.check)
The work item inside a layer	“leaf”, “leaves”: BP 26	-	four meanings: a leaf of the product tree (:319), “the leaves of Layer 1 already in flight” (:591), “the missing leaf” of ArkLib’s sumcheck (:259), “packing leaves” (:267)

Terms a zkVM engineer would not recognize without the blueprint’s definitions: hole, seam, spine, the wall, ledger, frontier, toy (instance), Category A and Category B, M3 (expanded only in the references, :1539), IOR. Of these only “M3” and “IOR” are terms of art elsewhere.

Words with two meanings that matter: **claim** (an evaluation claim; the act of taking work), **instance** (M3Instance; a Lean type-class instance; “an instance of a theorem”), **statement** (the public statement I. Stmt; a theorem’s statement; Clean’s Ensemble.Statement), **Layer N** (the proof system’s and leanISA’s, both from 0; the table at protocol-blueprint.md:215-222 uses the bare form for leanISA’s and “Layer 11 here” for its own in the same row), **component**, **lane**, **leaf**, **Direction**.

13.4 The letter codes

Source of the counts: `extract_codes.py` over the eight documents (blueprint, status, tracker body, hole comment, the four other comments, the three handoffs), and `code_contexts.py`, whose output was read occurrence by occurrence for every code with more than one meaning.

Re-check (this dossier’s second pass). The script’s pattern skips a code written inside backticks or followed by `/` (for example ``L1`` at `protocol-blueprint.md:591`), so every count below is a lower bound. An independent count with a plain whole-word pattern over the same eight texts, for thirteen codes across the families (L1, G4, P5, K2, A4, C1, A18, F17, E16, S16, T4, I2, E7), agrees on 102 of 104 cells; the two differences are corrected below (L1: 6 in the blueprint, not 5; P5: 3 in the public-input handoff, not 2). The “defined at” line of each of those thirteen codes was read at the cited line and is right.

13.4.1 The families

Table 13.13: The families of codes.

Family	Codes	Distinct	Occurrences	Defined in	Letter explained?
Units of work (“holes”)	0, L1, S, I1, I2, G1-G6, P1-P8, C1, K1-K4	24	about 470	blueprint :595-615; hole comment	no. S, G, I, P, C, K, L are never expanded
Layers	Layer 0 to Layer 13	14	“layer” 172 times in the blueprint	blueprint :625-1259	-
Upstream ledger	A1-A9, C1-C4	13	about 95	blueprint :257-267 (the A rows); status :171-184 (the C rows)	no
Findings, ArkLib	A10-A19	10	16	status :332-353	status :281-283
Findings, specification	S6, S9-S16	9	24	status :285-303	yes
Findings, Rust and Python	F1-F18	18	62	status :305-330	yes, wrongly (section 13.2.1, item 13)
Findings, Clean	C5, C6, C10, C11	4	8	status :355-358	yes
Findings, CompPoly	P1, P3-P6	5	8	status :360-363	yes
Findings, Lean environment	E6-E18	13	41	status :365-432	yes
Spine review	A1-A6, B1-B2, I1-I3, C1-C2, H1-H6	19	about 150	protocol-spine.md:32-47	passes A, B, C are named; I and H are not
Layer 1 review	A1-A6, B1-B4 (and B0, B6), D1-D17, C1-C9	38	about 80	protocol-layer1.md:26-43	D is not
Public-input review	B1-B3, C1-C4, A1-A4, H1-H6	17	about 120	public-input-phase.md:66-79	H is not
Flock roadmap’s milestones	F1, F3-F6, F8 cited	6	13	the body of issue 3, :92-101	there, not here
Decisions	1 to 15	15	27 references	status :212-277	-
Acceptance tests	1 to 28	28	30 references	blueprint :1266-1358	-
Target theorems	T1, T2, T4, T5, T6 cited	5	41	architecture.md:203-376	yes
Categories	Category A, Category B	2	22	blueprint :178-184	yes
leanth’s audit codes	FW-1, and in <code>leanth-reuse.md</code> FW-4, DR-101, DR-110, DR-112, DR-113, PS-2, PS-3, F-1, F-3, F-6, F-8	12	1 in the blueprint (: 1337)	the private repository	no
A contributor’s private codes	P17, P05	2	1 each (issue-12-comment-5749874958.md:5; the body of issue 31)	nowhere	no

The machine count over the eight documents: at least 128 distinct letter-number codes and at least 1036

occurrences, of which the status has 266, the blueprint 169, the spine handoff 151, the tracker body 141 (`extract_codes.py`).

Every code, one sentence each. Table C.1 gives every code with its meaning, where it is defined, its uses and the name proposed for it.

13.4.2 Every collision

Inside the proof system’s own documents: 34 codes with two or more meanings. The split of the occurrences by meaning was made by reading each one (`ctx_A.txt`, `ctx_C.txt`, `ctx_I.txt`, `ctx_P.txt`, `ctx_F.txt`).

Table 13.14: Codes with two or more meanings inside the proof system’s own documents.

Code	Meanings, with the number of occurrences of each
A1	ledger entry 10; spine review 11; Layer 1 review 5; public-input review 6
A2	ledger entry 20; spine review 9; Layer 1 review 4; public-input review 6
A3	ledger entry 6; spine review 12; Layer 1 review 3; public-input review 3
A4	ledger entry 2; spine review 8; Layer 1 review 4; public-input review 2
A5	ledger entry 7; spine review 8; Layer 1 review 2
A6	ledger entry 7; spine review 5; Layer 1 review 4
B1, B2	three reviews, three meanings each
B3	Layer 1 review; public-input review
C1	unit of work (the composition) 25; ledger entry (Clean’s expressions) 2; spine review 3; Layer 1 review 5; public-input review 6
C2	ledger entry 2; spine review 2; Layer 1 review 2; public-input review 3
C3, C4	ledger entry 2; Layer 1 review 1; public-input review 2
C5, C6	Clean finding 2; Layer 1 review 1
H1 to H6	spine review; public-input review
I1	unit of work (Clean expressions as polynomials) 21; spine review finding 5
I2	unit of work (the adaptor) 46; spine review finding 3
P1	unit of work 20; CompPoly finding 1
P3	unit of work 15; CompPoly finding 4
P4, P5, P6	unit of work 11, 34, 18; CompPoly finding 1 each
F1	finding (no labels) 7; the Flock roadmap’s milestone “executable algebra” 2
F3	finding (one root) 6; Flock milestone “baseline reduction” 1; a reference that means the leanISA finding of that code 1
F4	finding (derived target) 3; the leanISA finding of that code, so named, 1
F5	finding (shared powers) 4; Flock milestone “packed openings” 2
F6	finding (cubic round polynomial) 3; Flock milestone “concrete PCS” 6
F8	finding (low powers) 4; Flock milestone “concrete hash instances” 1

Within the status file alone the code A1 has four meanings (`protocol-status.md:49`, `:173`, `:254`, `:443`) and the code C1 has four (`:104`, `:145`, `:181`, `:445`); “review finding A1” at `:49` and at `:254` are two different findings of two different reviews. Within the spine handoff alone I1 and I2 are both a unit of work and a finding, once on adjacent rows (`protocol-spine.md:42-44`, `:334-335`).

Against the leanISA status: 11 more codes. The same code, another thing:

Table 13.15: The same code, another thing in the leanISA status.

Code	In the proof-system status	In the leanISA status
S9	the proof of Lemma 5.2 is T0D0	section 7.4 lists the Deref memory flushes in another order than the Rust
F2, F7, F9, F10 (and F1, F3-F6, F8 above)	Rust or Python against the specification	the roadmap’s or the architecture’s targets: for example F9 is “the roadmap’s Layer 7 sketch was unbuildable in two places”
E6	a Lake race on ArkLib’s lint plugin	core’s BitVec simprocs fire on numerals of K

Code	In the proof-system status	In the leanISA status
E7	no LawfulBEq E	circuit_norm strands the Decidable instance of a witness program
E8	no OracleInterface E	lazy unfolding evaluates a table of 2^{16} rows
C10	EnsembleWitness has no generator	Clean’s raw channels and balance are degenerate over K
C11	Ensemble.Statement assumes the non-overflow side condition	RequirementsChannelsLawful
L1	the unit of work “tables and stacking”	core’s DecidableEq for Vector is not exposed (cited as “finding L1” in tests/README.md)
P4, P5 (third meaning)	see above	the derived DecidableEq of extensions; simp lemmas keyed on Ext.coeff

Other documents cite these codes without saying which file: `docs/dependencies.md:55` cites “status finding E6” (the proof system’s), `docs/development.md:56` “finding P3” with the file, `tests/README.md` “finding L1” and “finding E5” without one.

Collisions already in the trackers. The body of intention issue 28 says “Tracks #12, Layer 2/C1”, where C1 is the ledger entry on Clean’s expressions, not the unit of work that C1 names on the checklist of issue 12. Issue 23 says “the bump also has to survive P3, the eager Fintype BF64”, the leanISA finding, while P3 on issue 12 is the table sumcheck’s definition. Issue 4 cites “decision 15” (leanISA’s: fixed columns after Clean pull request 446), while the proof system’s decision 15 is the public-input transcript. `docs/dependencies.md:87` cites “finding C8” without saying which status file. The body of intention issue 31 uses a private code, “P05 depends only on #18’s multilinear API”.

Numbered things that collide across the two roadmaps. Decisions 1 to 15 (section 13.2.8), layers 0 to 13 and 0 to 10, acceptance tests 1 to 28 and 1 to about 20. Inside the proof system’s documents every cross-reference to a leanISA layer or test says “leanISA”, with the exception of the table at `protocol-blueprint.md:215-222` and of `leanth-reuse.md:485`.

Other letters that look like codes. S1 to S4 in `architecture.md:80-83` are the four reference specifications. The single letters L, S, P are classes of declarations in the public-input handoff (`public-input-phase.md:483-484`), and S, M, L are sizes of effort in `leanth-reuse.md:58`. “Category A” and “Category B” are kinds of source; “Pass A”, “Pass B”, “Pass C” are the review’s passes, whose findings take the pass’s letter, which is why a finding of pass A collides with the ledger’s A.

13.4.3 Used and never defined; defined and never used

Used and never defined in any current document of the repository:

Table 13.16: Codes used and never defined in any current document.

Code	Where used
P17	<code>issue-12-comment-5749874958.md:5</code> : “no P17 replacement was implemented”
P05	the body of issue 31: “P05 depends only on #18’s multilinear API”
FW-1	<code>protocol-blueprint.md:1337</code> : “catalog audit FW-1”; the audit is in the private repository
B0, B6	<code>protocol-layer1.md:114-115</code>
D3, D6 and the other single members of D2-D17	named only inside ranges
decisions 1 and 3, as questions	<code>protocol-status.md:214</code> (section 13.2.8)
“revision 2”	<code>issue-12-body.md:1, 10</code> ; <code>protocol-status.md:214</code>
the letters S, G, I, P, C, K, L of the units	everywhere
the Flock roadmap’s F1, F3-F6, F8	defined in the body of issue 3 only

Defined and cited nowhere else (26 codes occur exactly once): A10, A11, A12, A13, A15, A16, A17; S10, S11, S12; F7, F12, F13, F15, F16; E11; C10, C11; the CompPoly findings P4, P5, P6; and five single D codes. A code that is never cited buys nothing over a sentence.

13.5 The proposal

13.5.1 Who owns what

13.5.1.1 The rule

A fact has one home. Every other place links to the home and restates nothing: no summary, no “for convenience” copy, no table repeated with fewer columns. A reader who wants the fact follows the link; a writer who changes the fact changes one place.

13.5.1.2 The owners

Table 13.17: One home per kind of fact.

Kind of fact	Owner	What the other places do
Targets, scope, conventions, the spine, acceptance tests	the blueprint	the status and the tracker link to the section
The specification of each unit of work: what it produces, with signatures; what it needs; its sources; its tests; the upstream work to read first	the blueprint, one section per unit	the hole comment is reduced to a pointer; the checklist line carries a name and a link, no signature
Decisions, taken and open	the blueprint, a section “Decisions”	the status and the tracker hold none
The upstream ledger: what is admitted or absent at the pin, what replaces it here, the upstream number, when the local copy goes	the blueprint, inside “Dependencies and exact contracts”	the status’s and the tracker’s copies are deleted
Coverage: what is on main	the repository; the status prints it, and a script checks the print	the tracker’s checklist is ticked from it; the blueprint says nothing
What can start now	the status, derived from the units’ “needs” and the coverage	the tracker links
Upstream watch: external work in flight, with its adoption condition	the status	the blueprint’s column “Existing work” is deleted; its content becomes the “upstream work to read first” of the unit, without state
Who is doing what	the tracker’s checklist line, linking the claimant’s intention issue or pull request	the status holds no claim
Open pull requests	GitHub	no table anywhere
Discrepancies in the leanVM sources at the pin (specification, Rust, Python)	docs/leanvm-target.md, a new section “Known discrepancies at the pin”, for both roadmaps	the blueprint cites a discrepancy by name where it binds a definition; both status files lose the section
Limits of the pinned libraries	docs/dependencies.md, under each library	the same
Lean pitfalls	tests/README.md, which already holds five of them, linked from docs/development.md	the same
The record of a review	the handoff under docs/reviews/	see section 13.5.1.6
What was read, and when	an archive (section 13.5.1.5)	-
How to work on the proof system	the blueprint’s section “How work is tracked”; the general rules stay in CONTRIBUTING.md and AGENTS.md	the tracker links

No new kind of document is created. Three existing documents each gain one section (`leanvm-target.md`, `dependencies.md`, `tests/README.md`), which is where their subject already lives: `dependencies.md` already

explains two limits of Clean and cites a finding by its code (:52-55, :81-88), and `tests/README.md` already explains five pitfalls.

Why `leanvm-target.md` for the discrepancies. They are facts about the target at the pin, not about either roadmap: they hold until the pin moves, they are shared (the missing section 8.4 is one finding in both status files), and both status files already say they are “numbered for citation from ... `docs/leanvm-target.md`”. One register also ends the collision between the two files’ letters. The alternative, an appendix of the blueprint, keeps two registers, one per roadmap, and is not recommended.

13.5.1.3 What the blueprint contains, and what leaves it

Table of contents proposed:

1. What is proved: the two master theorems, the compiled verifier’s theorem, the two base theorems (T4) (today’s introduction).
2. For zkVM engineers.
3. Scope.
4. Dependencies and exact contracts, ending with the upstream ledger (one table).
5. Pinned conventions.
6. The spine.
7. The units of work: one table with names (appendix D.2), then one section per unit in build order. Each section merges today’s per-layer sketch and the hole comment’s section, and has the same five parts: produces, needs, sources, tests, upstream work to read first.
8. Acceptance tests and nearby false statements.
9. Interfaces supplied to later work.
10. Boundaries with the other roadmaps.
11. Decisions: a table of those taken (number, name, the choice in one line, the convention or test that records it) and a list of those open.
12. How work is tracked (appendix D.1).
13. Index of names and numbers (section 13.5.2.3).
14. References.

What leaves the blueprint: every sentence of status or history listed in section 13.2.4, item 11; the columns “Existing work” and “Issue” of the hole table; the section “Ordering and parallelism”, whose content is the column “needs”; the per-layer sketches that contradict the spine, which are rewritten on the spine’s types when merged into section 7.

What enters it: the tests and upstream notes that exist only in the hole comment (end of section 13.1); the statement of decisions 1 to 10, of which only the outcomes are written today; what the spine asks of the Flock roadmap, which is a comment on issue 3.

13.5.1.4 What the status contains, and what can be generated

Table of contents proposed:

1. Head: two sentences. What the file is; that the coverage table is checked by a script.
2. Coverage: one row per unit, generated.
3. What can start now: generated, or five lines by hand.
4. Upstream watch: the item, the unit, what it would replace, the adoption condition. No state and no date: the state is one click away and is what goes stale.
5. Pointers: to the blueprint’s decisions and ledger, to the three registers, to the reviews.

Coverage from the repository. The repository forbids `sorry`, and the kernel axiom audit rejects `sorryAx`, so a declaration that exists is proved. A unit is therefore built exactly when the declarations that mark it exist. The probe `coverage_from_blueprint.py` does this today from the blueprint’s hole table, with no Lean: it reports the tables-and-stacking unit at 15 of 15 names, the composition at 2 of 2, the public-input phase at 1 of 1, and every unbuilt unit at 0, except where the table lists names that are

not the unit’s own (section 13.2.5, item 6). What it needs from the blueprint is a column “built when declared” holding, for each unit, names that this unit and no other declares. The public-input phase sets the pattern: a phase x is marked by `xPhase` and `xComplete` for its first half and `xSecurity` for its second.

Proposed, for the owner to decide: `scripts/check-protocol-status.py`, run by `scripts/validate.sh` and by the workflow “Documentation integrity”. It reads the units table of the blueprint, looks each marking name up under `LeanerVM/`, and compares the result with the table between two markers in the status; on a difference it fails and prints the table to paste, and with `--write` it rewrites it. A planted difference goes into `scripts/test-policy-checks.py`, as for the other policy scripts. The same script can check two more things this review found by hand: that every name of the interface list that belongs to a built unit is declared, and the import rule of the proof system (no `import LeanerVM.Arithmetization` under `LeanerVM/Protocol/` outside an allow-list), which the Layer 1 review left open.

The head then needs no commit hash. A file cannot name the commit that contains it; today’s head names the parent and is wrong by construction after every merge (section 13.2.1, items 1 to 5).

Upstream state. `scripts/check-upstreams.sh` already needs `gh` and the network and runs weekly in the workflow “Upstream drift”. It can print the state of each watched item next to the pins’ drift. The state then lives in the workflow’s report and is never committed.

13.5.1.5 The survey record

The 190 lines of “Survey record” (`protocol-status.md:434-622`) are a log of what was read. They are history, useful “so the searches are not repeated”, and they are what makes the status long. Two options. Recommended: move them once, unchanged, to `docs/reviews/protocol-survey-record.md`, headed as an archive; the directory is already the repository’s archive of reviews. Alternative: keep them as the last section of the status, headed “Record (history, not a source of truth)”, which keeps a hand-maintained status long.

13.5.1.6 The review handoffs

A handoff is a record of what was found at one commit. It is never a specification.

- A finding that is accepted becomes blueprint text (a convention, a signature, an acceptance test, a decision) in the pull request that meets it. The handoff’s disposition table cites the blueprint’s section, by name.
- The handoff gets one line at its head: the commit it describes, and “an archive; names and paths are the reviewed commit’s; for the current state read the blueprint”.
- The status does not summarise reviews. `docs/README.md` keeps its index of handoffs, one line each, without dispositions.
- Texts a handoff drafts for GitHub (`protocol-spine.md:662-769`) are applied by editing the blueprint, since the specifications they patch move there.

13.5.1.7 The tracker and the hole comment

The body of issue 12: the pointers, the checklist, and two lines on how to claim (appendix D.4). No ledger, no watch list, no table of pull requests, no decisions, no list of names, no “frontier”.

The hole comment: its thirteen sections move to the blueprint, corrected on the way (section 13.2.3). The comment is then edited by its author to one line, “The specifications of the units of work are in the blueprint, section ‘The units of work’”, so that links to it still land somewhere.

The three comments by the other contributor are a record of claims made in September and need nothing. Their content that is still live (four claims) is already in the bodies of intention issues 28, 31, 33, 37.

Claiming without comments. A maintainer, or an agent acting for one, edits the checklist line in the body. A contributor without write access cannot edit the body; they open an intention issue whose body names the unit and the exact declarations taken, which they can edit themselves, and the maintainer links it from the checklist line. Nobody claims by comment. This is what the four live claims already do.

13.5.2 The codes

13.5.2.1 What is replaced by names, and what stays

Table 13.18: What is replaced by names, and what stays.

Family	Proposal
Units of work	names. The letters S, G, I, P, C, K, L go
Ledger entries	names
Findings of reviews	names, in the handoff’s own disposition table; never cited by letter outside the handoff
Findings against the leanVM sources, the libraries, Lean	names, each a heading in its register, so that it has an anchor; the former code is kept once, in the register, to read old pull requests
Layers 0 to 13	stay, as the numbers of the blueprint’s sections, with the name: “the bus phase (Layer 6)”
Acceptance tests 1 to 28	stay, with the name the blueprint already gives each in bold
Decisions 1 to 15	stay, each given a name
Target theorems T1 to T8	stay; owned by <code>architecture.md</code>
Category A, Category B	replaced by the two words they stand for: “written from the specification”, “transcribed from its source”

The rule for what stays: **the name stands beside the number at every use**. “Acceptance test 24” alone is not allowed; “the extractor computes (acceptance test 24)” is. A reference to the other roadmap’s layer, test or decision always says “leanISA”.

If the owner prefers to keep numbers for the findings against the sources, the scheme without collisions is a single counter in `docs/leanvm-target.md`, without letters, across both roadmaps (“source finding 31”), with the former codes in one column. Letters are what collide.

13.5.2.2 The mapping from every code to its name

Units of work. The short form is for branch names, pull-request titles and the checklist.

Table 13.19: The units of work: former code, name, short form.

Former code	Name	Short form
0	field instances (Layer 0)	field-instances
L1	tables and stacking (Layer 1)	tables-stacking
S	the spine	spine
C1	knowledge-soundness composition (the port of ArkLib pull request 615)	knowledge-append
I1	Clean expressions as polynomials (Layer 2)	clean-polynomials
I2	the adaptor (Layer 3)	adaptor
G1	sumcheck: definition and completeness (Layer 4)	sumcheck-def
G2	sumcheck: knowledge soundness (Layer 4)	sumcheck-security
G3	batching by powers (Layer 4)	batching
G4	fingerprint and collision bound (Layer 5)	fingerprint
G5	grand-product GKR: definition and completeness (Layer 5)	gkr-def
G6	grand-product GKR: knowledge soundness (Layer 5)	gkr-security
P1	bus phase: definition and completeness (Layer 6)	bus-def
P2	bus phase: knowledge soundness (Layer 6)	bus-security
P3	table sumcheck phase: definition and completeness (Layer 7)	table-def
P4	table sumcheck phase: knowledge soundness (Layer 7)	table-security
P5	public-input phase (Layer 8)	public-input
P6	Flock phase, the interface to the Flock roadmap (Layer 9)	flock-interface
P7	opening phase: definition and completeness (Layer 10)	opening-def
P8	opening phase: knowledge soundness (Layer 10)	opening-security
K1	WHIR opening (Layer 11)	whir
K2	Merkle trees, byte hasher, WHIR parameters (Layer 11)	merkle-parameters
K3	transcript, proof object, compiled verifier (Layer 12)	compiled-verifier
K4	the base theorems of T4 (Layer 13)	base-theorems

Ledger entries.

Table 13.20: The upstream ledger: former code and name.

Former code	Name
A1	admitted upstream: sumcheck round knowledge soundness
A2	admitted upstream: composition of knowledge soundness
A3	admitted upstream: round-by-round implies plain
A4	admitted upstream: context lifting (not consumed)
A5	admitted or absent upstream: Fiat-Shamir and BCS security
A6	absent upstream: grand product, GKR, batching, stacking
A7	absent upstream: WHIR, Merkle trees, BLAKE2s
A8	admitted upstream: correlated agreement up to Johnson
A9	admitted upstream: ring-switching packing
C1	absent in Clean: expressions as polynomials
C2	asked of leanISA: power-of-two heights, bus data per channel (done)
C3	absent in Clean: balance counted in the naturals
C4	absent in Clean: fixed columns and sound prover data

Findings against the leanVM sources (to docs/leanvm-target.md).

Table 13.21: The findings against the leanVM sources: former code and name.

Former code	Name
S6	Fiat-Shamir section is TODO
S9	proof of the product lemma is TODO
S10	degree of the radix-4 round polynomial not stated
S11	one bus root not stated
S12	grinding per level not in Annex B
S13	retired
S14	the PDF is not the pinned text
S15	no rule for equal-size blocks
S16	equations are (1) to (4)
F1	numeric tags, no labels
F2	Flock's fixed coordinate has no provenance
F3	one root for push and pull
F4	sumcheck target derived, not sent
F5	bus forms share the last three powers
F6	cubic round polynomial, three scalars
F7	one coefficient never sent
F8	ring-switched claims take the low powers
F9	Python verifier omits four caps
F10	rejection set of the Rust verifier
F11	count tree holds the tables' counts only
F12	fill blocks make heights exact
F13	grinding binds the nonce on failure
F14	seed hashes a constant naming the circuit
F15	stack height bound checked separately
F16	security level and no grinding for the bus
F17	equal-size blocks in column-index order
F18	one equation on the words, not one per limb

Findings about the libraries (to docs/dependencies.md) and **about Lean** (to tests/README.md): the one-sentence descriptions of table C.1 serve as names; the former codes A10-A19, C5, C6, C10, C11, P1, P3-P6, E6-E15, E18 are kept in one column of each register. The two “environment” findings that are rules (E16, E17) are already the blueprint’s convention *Generic code* (protocol-blueprint.md:329) and are deleted.

Findings of the three reviews: the names are the third column of the review rows of table C.1. They are used inside the handoffs only.

13.5.2.3 The index, drafted

One table, in the blueprint, section 13. It lists what keeps a number and what had a code.

Table 13.22: The index of names and numbers, drafted for the blueprint.

Name	Number or former code	Kind	In one sentence	Specified or decided at
base proof extraction and completeness	T4	target theorem	an accepted proof has an execution, and an execution has an accepted proof	architecture.md, “Target theorem ladder”
arithmetization and ISA equivalence	T1	target theorem	constraint assignments and executions correspond, for well-formed programs	architecture.md; leanISA blueprint, Layer 10
witness-generator correctness	T2	target theorem	the generated assignment satisfies the constraints; out of scope here	architecture.md
recursive-verifier correctness	T5	target theorem	out of scope here; verify is shaped for it	architecture.md; blueprint, “Boundaries”
recursion extraction	T6	target theorem	open research; out of scope here	architecture.md
field instances	Layer 0; formerly hole 0	unit of work	ArkLib as a dependency; sampling and oracle instances for the fields	blueprint, “The units of work”
tables and stacking	Layer 1; formerly L1	unit of work	hypercube tables, stacking with selectors, the index and bytecode columns	same
the spine	formerly S	unit of work	the instance, the relation, the seams, the phase interfaces, the composition	blueprint, “The spine”
knowledge-soundness composition	formerly hole C1	unit of work	the port of ArkLib pull request 615, deleted when the pin passes it	blueprint, “The spine”
Clean expressions as polynomials	Layer 2; formerly I1	unit of work	constraint and flush polynomials read off a Clean component	blueprint, “The units of work”
the adaptor	Layer 3; formerly I2	unit of work	the leanISA instance and the two maps between the stack and Clean’s witness	same
sumcheck: definition and completeness	Layer 4; formerly G1	unit of work	the sumcheck over eq-weighted virtual polynomials as a <code>Component.Def</code> , in the plain variant and the normalized one of the GKR layers, the linear coefficient derived, not sent (from R36, R53, R7)	same
sumcheck: knowledge soundness	Layer 4; formerly G2	unit of work	its round-by-round knowledge soundness at $d/ E $ a round, in the named form, with the lemma that carries it to the wire message of three coefficients (from R53, R36)	same
batching by powers	Layer 4; formerly G3	unit of work	several claims become one by the powers of one challenge	same
fingerprint and collision bound	Layer 5; formerly G4	unit of work	specification Lemma 5.2 and Theorem 5.1	same
grand-product GKR: definition and completeness	Layer 5; formerly G5	unit of work	the batched grand product of the three trees, over a context whose oracles are the leaf tables, radix 4 with one binary layer for an odd height, a combiner after the roots and after every layer, the last unused (from R10, R7, R9)	same
grand-product GKR: knowledge soundness	Layer 5; formerly G6	unit of work	its round-by-round knowledge soundness at $4/ E $ a round of the normalized layer sumcheck and $1/ E $ a combination challenge, the zerocheck clause riding along (from R7, R9, R10)	same

Name	Number or former code	Kind	In one sentence	Specified or decided at
bus phase: definition and completeness	Layer 6; formerly P1	unit of work	the challenges (α, β) , the two roots, the GKR and the five boundary evaluations, emitting a <code>BusOut I</code> in the shape of the Rust’s <code>BusVerify</code> , the orders of the leaf stacks, roots, evaluations and claims fixed (from R2, R50, R51)	same
bus phase: knowledge soundness	Layer 6; formerly P2	unit of work	one <code>busError</code> , the zerocheck a conjunct of the state function that costs nothing beyond the GKR’s errors, under the two side conditions on the instance (from R47, R43)	same
table sumcheck phase: definition and completeness	Layer 7; formerly P3	unit of work	one sumcheck of $\tau_{\max}$ rounds over the six opcode tables’ constraints and bus forms, three coefficients a round and 104 values at the end, from the bus seam’s <code>BusOut</code> (from R2, R11, R53)	same
table sumcheck phase: knowledge soundness	Layer 7; formerly P4	unit of work	its round-by-round knowledge soundness at $(B+2)/ \mathbb{E} $ on ξ and $3/ \mathbb{E} $ a round, over the tables the instance marks as the sumcheck’s (from R11, R2)	same
public-input phase	Layer 8; formerly P5	unit of work	specification section 8.2: one challenge, the sent values checked per limb, one claim per public line	same
Flock phase	Layer 9; formerly P6	unit of work	the interface the Flock roadmap (issue 3) inhabits	same
opening phase: definition and completeness	Layer 10; formerly P7	unit of work	the batching challenge λ and one weighted query of the stack, with no sumcheck of its own (from R18)	same
opening phase: knowledge soundness	Layer 10; formerly P8	unit of work	its knowledge soundness at $(J-1)/ \mathbb{E} $ on λ , the query answered by the stack’s inner-product oracle (from R18, R19)	same
WHIR opening	Layer 11; formerly K1	unit of work	WHIR realizing the one weighted query, its level-0 batching the opening’s λ , <code>encode</code> over $\mathbb{E}$ at every level with the lemma that a $\mathbb{K}$ -valued message has a $\mathbb{K}$ -valued codeword (from R19, R18, R24)	same
Merkle trees, byte hasher, WHIR parameters	Layer 11; formerly K2	unit of work	Merkle trees with the pruned wire paths, the byte hasher (BLAKE2s-256 of a 64-byte input, not the compression keyed by the chain state) and the parameter ladder; whether the trees are VCVio’s is open (from R63, R66, R77)	same
transcript, proof object, compiled verifier	Layer 12; formerly K3	unit of work	the chain, the proof object and <code>verify</code> , compiled by a leanerVM BCS transform (roots for oracle messages, authenticated rows for queries, grinding and the encoding checks) over the family of admissible sizes, from front phases that never query the stack (from R21, R6, R61, R25, R138)	same
the base theorems	Layer 13; formerly K4	unit of work	both with <code>WellFormedBytecode prog</code> : soundness for a Q -query adversary against the random-oracle verifier, with a straight-line extractor, at <code>niError Q</code> ; completeness from a witness with admissible sizes and a valid rate (from R27, R28, R29)	same
heights are powers of two	decision 1	decision, taken	leanISA’s <code>Caps</code> requires it, so the two roadmaps state one relation	leanISA blueprint, Layer 8; issue 13

Name	Number or former code	Kind	In one sentence	Specified or decided at
statement and parameters	decision 2	decision, taken	the public input is the statement; the instance indexes the protocol family	convention <i>Statements and parameters</i>
where the zerocheck is charged	decision 3	decision, taken	in the bus phase, where the point is drawn; no separate phase	convention <i>Seams</i>
where generic code lives	decision 4	decision, open	the <code>To<Library></code> folders, or an ArkLib branch	blueprint, “Decisions”
an end-to-end run of the honest prover	decision 5	decision, open	depends on the cost of the encoder	blueprint, “Decisions”
the witness is the stack	decision 6	decision, taken	the protocol’s witness is the committed column, in <code>Type 0</code>	“The relation ladder”
phases over an abstract instance	decision 7	decision, taken	every phase is tested on the toy instance	convention <i>The wall</i>
five clauses, caps outside	decision 8	decision, taken	the caps are a hypothesis of the adaptor’s soundness theorem	“What the spine fixes”
balance is a permutation	decision 9	decision, taken	the pushed and pulled tuple lists are permutations of each other	“What the spine fixes”
worst-case round-by-round knowledge soundness	decision 10	decision, taken	per component, composed by the ported theorem	convention <i>Holes</i>
extractors are computable definitions	decision 11	decision, taken	the protocol’s extractor is <code>pioExtractor</code>	convention <i>Extractors</i> ; acceptance test 24
the strong Flock predicate	decision 12	decision, taken	the predicate is Flock’s RICS on the committed region	Layers 3 and 9
public lines, not cells	decision 13	decision, taken	the statement fixes cells 0 and 1 of listed columns	convention <i>Public input</i>
the degree bound belongs to the instance	decision 14	decision, taken	the bus seam bounds every term by it	convention <i>Seams</i> ; acceptance test 28
the public-input transcript	decision 15	decision, taken	the prover sends the values; a line says whether its value is sent	convention <i>Public input</i> ; Layer 8
seams carry claims, never challenges	none today	decision, taken	what a phase emits is fixed by its knowledge soundness	“What the spine fixes”
schedules travel with the definitions	none today	decision, taken	the spine fixes the commit phase’s schedule only	“What the spine fixes”, item 4
fingerprint degree	acceptance test 1	acceptance test	the error carries a factor 4	blueprint, “Acceptance tests”
padding leaves are 1	acceptance test 2	acceptance test	a zero pad zeroes every product	same
the nonzero count root is load-bearing	acceptance test 3	acceptance test	a read with count zero balances	same
one root, not two	acceptance test 4	acceptance test	the push and pull roots are one scalar	same
domain separators	acceptance test 5	acceptance test	coordinate 0 separates the three buses	same
zerocheck point recycling	acceptance test 6	acceptance test	the point is drawn after the commitment	same
back-loaded padding	acceptance test 7	acceptance test	the lift keeps the table’s sum	same
degree three, three scalars	acceptance test 8	acceptance test	the wire carries three coefficients	same
shared bus powers	acceptance test 9	acceptance test	the three bus forms share the last three powers	same
top limb of the public input	acceptance test 10	acceptance test	the third claim is pooled though nothing is sent for it	same
joint list binding	acceptance test 11	acceptance test	one member of the list satisfies all claims	same
absorb before squeeze	acceptance test 12	acceptance test	sizes and root are observed before the first challenge	same
index column bit order	acceptance test 13	acceptance test	bit k is coordinate k	same
bytecode slot bits	acceptance test 14	acceptance test	the opcode is slot 3, low bit first; witness <code>bytecodeColumn_answer_bool Vec</code>	same

Name	Number or former code	Kind	In one sentence	Specified or decided at
selector alignment	acceptance test 15	acceptance test	blocks sit at multiples of their size, largest first	same
variable order	acceptance test 16	acceptance test	the highest variable is bound first	same
radix and parity	acceptance test 17	acceptance test	one radix-2 layer when the height is odd	same
counts in the count tree	acceptance test 18	acceptance test	the tables' count columns only	same
the extractor reads the stack	acceptance test 19	acceptance test	the witness is read off the committed column	same
no exceptional challenge	acceptance test 20	acceptance test	perfect completeness has no side condition	same
sizes are parameters	acceptance test 21	acceptance test	the protocol is a family indexed by the sizes	same
tags, not labels	acceptance test 22	acceptance test	four numeric tags	same
numeric error	acceptance test 23	acceptance test	the bound at the caps	same
the extractor computes	acceptance test 24	acceptance test	an extractor chosen classically proves soundness, not knowledge	same
the import rule holds	acceptance test 25	acceptance test	no phase imports the arithmetization	same
seams are the contract	acceptance test 26	acceptance test	a phase's output relation is the next one's input relation by definition	same
the toy instance is honest	acceptance test 27	acceptance test	each clause fails alone	same
the bus seam bounds the degree	acceptance test 28	acceptance test	a cubic term is rejected	same
admitted upstream: sumcheck round knowledge soundness	formerly ledger A1	ledger entry	the single-round bound is admitted in ArkLib at the pin	blueprint, "Dependencies", the ledger
admitted upstream: composition of knowledge soundness	formerly ledger A2	ledger entry	proved here by the port; the port is deleted at the pin bump	same
(the other eleven ledger entries, as in section 13.5.2.2)		ledger entry		same

The index gains one row per finding of the review against leanVM's own sources, of the same kind as the status findings on the sources mapped above: each is recorded by name in `docs/leanvm-target.md`, keeps no number, and is among the matters the owner may report to leanVM (section 11.4) (from R57, R105 to R109, R111, R112, R116 to R118, R133).

Table 13.23: The index, drafted: the rows for the review's findings against leanVM's sources.

Name	Number or former code	Kind	In one sentence	Specified or decided at
seven disagreements in the setup, the commitment and the bus	none	finding on the sources	the rate announced with the sizes, the roots' order, the seed's constant naming the circuit, checks the specification omits, the last combiner, the Python's unchecked zero slots of the stacked bytecode, the fingerprint bound $5 \cdot 2^\mu$ against $4 \cdot 2^\mu$ (R105)	<code>docs/leanvm-target.md</code>
the seed's circuit digest	none	finding on the sources	the Fiat-Shamir seed binds a constant that cannot be recomputed at the pin (R106)	same
no round-by-round analysis of the bus phase	none	finding on the sources	the specification's one round-by-round theorem is the opening's; the bus phase's errors are the blueprint's own analysis (R107)	same

Name	Number or former code	Kind	In one sentence	Specified or decided at
the batching error of the table sumcheck	none	finding on the sources	the specification charges one more than its own Corollary 3.9 gives, no error for the recycled point, and does not say which coefficient is dropped (R108)	same
stale Rust comments on the round message	none	finding on the sources	the comments say the round polynomial is sent whole at four nodes; three coefficients are sent (R109)	same
Flock’s circuit and constants	none	finding on the sources	the circuit, the limb slots, g_0 , φ_8 , k_{batch} and the floor on τ_{BLAKE2S} are fixed only by the Rust and the Python (R57)	same
the Rust Flock prover’s exceptional point	none	finding on the sources	at $r_{\text{eq}} = 1$ the pinned prover emits a proof its verifier rejects; the specification’s protocol is perfectly complete (R111)	same
stale Rust comments on the counter and the flags	none	finding on the sources	the comments say the bytecode binds them; the table reads the metadata cell from memory, as the specification says (R112)	same
a stale Rust comment on level-0 grinding	none	finding on the sources	it says level 0 grinds no bits; every level grinds 17 in production (R116)	same
the Rust’s list-size accounting	none	finding on the sources	only the algebraic checks are unioned over the Johnson list; the bus’s term is not, with 23 bits of room at these sizes (R117)	same
a stale Rust comment on the Merkle leaf size	none	finding on the sources	it says a leaf is $16 \cdot n$ bytes; the code hashes $24 \cdot n$ (R118)	same
the witness stack’s size and tie rule	none	finding on the sources	the specification’s $M = \lceil \log_2 N \rceil$ has neither the verifiers’ window $[15, 28]$ nor their tie rule by column index (R133)	same

13.6 Findings on the documentation

Twenty findings: 3 major, 13 minor, 4 notes; none critical. None is a divergence from leanVM in the sense of the brief (the documents disagree with each other and with the repository, not with the pinned sources), except where a finding says otherwise; the classification line says so each time. Proposed changes that are long are in appendix D and referenced.

Finding 137 (major). The base theorem of extraction lacks the program hypothesis its own composition needs

Severity. major (a statement cannot be proved as written).

Evidence.

blueprint Layer 13, protocol-blueprint.md:1247-1250: “theorem baseVerifier_extractsExecution (fs bcs mca flock) (h : verify prog input proof = true) : except with probability ni Error, $\exists t$, ValidExecution prog input t” and “theorem baseProver_complete (hfill : Has FillBlocks prog) (h : ValidExecution prog input t) : ...”, said to compose constraint Soundness and constraintCompleteness (:1253-1255); the contract row :222 consumes “constraint Soundness, HasFillBlocks, constraintCompleteness (Layer 10)”. The leanISA blueprint states both with a program hypothesis (leanisa-blueprint.md:1156-1166): “theorem constraintSoundness (hwf : WellFormedBytecode prog) (h : SatisfiedBy prog input w) : ...”, “theorem constraintCompleteness (hwf : WellFormedBytecode prog) ...”, where WellFormedBytecode has two fields, sentinelSafe and hasFillBlocks. architecture.md:224-228: “Both directions are stated for well-formed programs ... a hypothesis of each, since the constraint system enforces neither”. leanvm-target.md:57-62 gives the reason: a bytecode whose sentinel slot holds a JUMP “admits accepted walks that execute the sentinel”. The hole comment repeats both statements (issue-12-comment-5833669972.md:190). So the soundness theorem cannot be obtained by the composition it names,

and the completeness theorem’s `HasFillBlocks` does not give `constraintCompleteness`’s hypothesis.

Classification.

an error of the blueprint, against the leanISA roadmap (the program condition itself is faithful to leanVM, where the compiler guarantees it: `leanvm-target.md:60-62`).

Proposed change. `:1247-1250` become

```
`theorem baseVerifier_extractsExecution (fs bcs mca flock) (hwf : WellFormedBytecode prog) (h : verify
  ⇨ prog input proof = true) : except with probability niError, ∃ t, ValidExecution prog input t`
```

and

```
`theorem baseProver_complete (hwf : WellFormedBytecode prog) (h : ValidExecution prog input t) : verify
  ⇨ prog input (prove prog input (witness of t)) = true`
```

`:222` consumes

```
`constraintSoundness`, `WellFormedBytecode`, `constraintCompleteness` (leanISA Layer 10)
```

Reason: the named composition then typechecks, and the hypothesis is visible in the statement a reader audits.

The review recommends the hypothesis on both theorems, as drafted above: the condition is faithful to leanVM, whose verifiers do not check it and whose compiler emits well-formed programs (`docs/leanvm-target.md:57-62`), and the proved sentinel counterexample (`tests/LeanerVMTests/Semantics/Execution.lean:415-461`) makes the soundness theorem false without it. The alternative, `verify` deciding the condition on the public program and rejecting otherwise, removes the hypothesis from the soundness theorem at the price of a check the deployed verifiers do not make; it is left to the owner (appendix A.3, item 8) (from R27).

Finding 138 (major). The blueprint hands the specification of the holes to a tracker comment that contradicts it

Severity. major (through the delegation, the blueprint specifies a public-input transcript leanVM does not use, and signatures that cannot be stated).

Evidence.

the blueprint sends the reader to the comment three times (`protocol-blueprint.md:339-340` “its specification is a section of the hole comment”; `:590-591` “Its specification is a section of the hole comment on the dashboard #12”; `:1485-1487`). The tracker body says it “holds nothing that is not in them” (`issue-12-body.md:6`). The comment’s own head says only four of its thirteen sections were rewritten after the spine was built (`issue-12-comment-5833669972.md:3`); the other sections still give the pre-spine signatures (section 13.2.3, items 1 to 3, 5, 7, 8), which do not typecheck against `Phase.Def` as built (`LeanerVM/Protocol/Spine/Phase.lean:37`, three arguments). Its section on the public-input phase says “the prover sends nothing” (`:93-94`), while leanVM’s §8.2 says “The verifier samples $r_m \in \mathbb{E}$, the prover sends c_0, c_1 claiming ...” (`doc/leanvm/body/08-end-to-end-protocol.tex:29` at `a386121f`), and the Rust verifier reads the two scalars (“`*v = vs.next_scalar()`”, `crates/lean_vm/src/cpu/mod.rs:746-749`), and the phase on `main` follows it (`PublicInput.pSpec, LeanerVM/Protocol/PublicInput.lean:99`: “`def pSpec : ProtocolSpec 2 := (!v[.V_to_P, .P_to_V], !v[E, List E])`”). The comment also holds tests and one technical remark found nowhere else (appendix D.4, column “only in the comment”).

Classification. an error of the blueprint (its delegated specification of the public-input phase differs from leanVM’s §8.2; the built phase is faithful).

Proposed change. move each section as appendix D.4 says; replace the three pointers as appendix D.2 and appendix D.1 say; edit the comment in place to the pointer of appendix D.4.

Finding 139 (major). The per-layer sketches use types the spine does not have

Severity. major (the statements cannot be stated as written).

Evidence.

section 13.2.4, items 1 to 9: Layer 6’s local structure `BusOut` where $\zeta \dots \text{rem} \dots \text{pool} : \text{List Claim} \dots \alpha \dots \beta$ (:960-964) against the spine’s `BusOut` `I` of two claim lists (:481; `Seams.lean:130-135`); Layer 7’s `tableSumcheck` $\dots (\text{StmtIn} := \text{BusOut}) \dots (\text{StmtOut} := \text{BusOut} \times \text{ColumnClaims})$ (:988) against `Phases.table` (:533); Layer 9’s `FlockInterface` with an `OracleReduction` field from `ColumnClaims` to `WeightedClaim` and an unbound `s` (:1077-1085) against `Phases.flock` (:535); Layer 10’s own `Weight`, `WeightedClaim`, `Claim.toWeighted`, `leanVmPiop` (`prog`) (`s`) (`input`) (`flock`) over `M3Witness` and `m3Relation`, and master theorems with `Set.univ` and `rbrKnowledgeSoundness` (:1109-1130) against the spine’s (:541-547; `Compose.lean:162-177`); `Sumcheck.Def`, `Gkr.Def`, `Batch.Def` (:952, 984, 1104), which exist nowhere; seven names used and never defined (`Claim`, `ColumnClaims`, `LeafLayout`, `StackLayout`, `M3Witness`, `m3Relation`, `commitPhase`). Layer 8 (:1011-1027) is the only phase sketch on the spine’s types.

Classification. an error of the blueprint.

Proposed change. restate Layers 6, 7, 9 and 10 on Layer 8’s pattern:

```
`def busPhase (I : M3Instance) : Phase.Def I I.Stmt (I.Stmt × BusOut I)`
`def busComplete I : Phase.Complete I (busPhase I) (Seam.commit I) (Seam.bus I)`
`def busSecurity I : Phase.Security I (busPhase I) (Seam.commit I) (Seam.bus I)`
```

and likewise `tableSumcheck` (from `Seam.bus` to `Seam.table`), the `Flock` phase (`Seam.pub` to `Seam.flock`) and `openingPhase` (`Seam.flock` to `Seam.done`); delete Layer 6’s `BusOut`, Layer 10’s `Weight`, `WeightedClaim` and its `leanVmPiop` block (the spine’s are in “What the spine fixes”); restate the generic components of Layers 4 and 5 as `Component.Defs` with their `Complete` and `Security`, and their theorems in the `rbrKnowledgeSoundnessWorstCaseWith` form of convention *Holes* (:332). Reason: the spine is built and is the contract.

The schedules, errors and internal relations the restated sections receive are those of the deployed verifiers as chapter 8 transcribes them (the bus phase in table 8.3, with the normalized layer sumcheck of degree 4 and a combiner after the roots and after every layer, the last unused; the table sumcheck and the public-input phase in tables 8.6 and 8.8; `Flock` in table 8.13; the opening in table 8.19), with the errors of chapter 5. Against specification §8.5 (08-end-to-end-protocol.tex:94-101: “there is no separate reduction sumcheck”) the opening phase’s schedule is settled: the batching challenge λ , then one weighted query of the stack, which WHIR realizes; Layer 10’s μ -round sumcheck on $W_\lambda \cdot q$ and its final evaluation query leave the sketch (from R18, R36; the bus phase’s schedule from R7, R9).

Finding 140 (minor). The status and the tracker describe a repository two merges old

Severity. minor (stale text), the most visible one.

Evidence.

section 13.2.1 (the status heads itself as a snapshot of `f4d858c` and calls the public-input phase a draft on a branch; fifteen items), section 13.2.2 (the tracker shows Layer 1 in review behind the closed #18, the public-input phase open with the opposite transcript, four closed pull requests as open, “revision 2” defined nowhere, and renders “ArkLib #1”, “ArkLib #4” and “#3 there” as links to leanerVM items), section 13.2.11 (both say the pins are unchanged; pull request 61 moved four). The Layer 1 review’s two edits of the tracker were never made (`docs/reviews/protocol-layer1.md:211`). `README.md:32-33` says “no proof-system claim has landed” and `docs/README.md:38-47` says the three reviewed branches are still branches.

Classification. not a divergence from leanVM.

Proposed change. appendix D.3 (status), appendix D.4 (tracker body). `README.md:32-33`:

```
The proof system's spine, with its two master theorems conditional on the phases, Layer 1 and the
↪ public-input phase are on `main`; see `docs/roadmap/protocol-status.md`.
```

`docs/README.md`: “whose findings the branch now meets” becomes “whose findings were met before it merged”, in the three entries. For the future: a status whose coverage is checked against the repository (section 13.5.1.4) cannot drift this way unnoticed.

Finding 141 (minor). The blueprint states superseded pins as current

Severity. minor (stale text), with a technical consequence left to the orchestrator.

Evidence.

section 13.2.11, items 1 to 6: after pull request 61 the blueprint says in one new paragraph that its API tables “record the original implementation baseline” (:52-54 at 144c5aa) and elsewhere still says ArkLib is “a Lake dependency at dca90385” (:113), that the pins are ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef, VCVio f9dc47d9 “(through ArkLib)”, Lean v4.33.1 (:167-172), that Layer 0 adds “ArkLib at dca90385” with CompPoly resolved at 3468b38c (:633-635), and heads the upstream ledger “ArkLib state at dca90385” (:261).

Classification. not a divergence from leanVM.

Proposed change. :167-172 becomes

```
| The pins are in `upstreams.json`, with the history of each in [dependencies.md](../dependencies.md);
| ↪ leanVM is pinned at `a386121f`.
```

:113 becomes

```
| ArkLib as a Lake dependency, with ...
```

Layer 0 (:633-635) says what Layer 0 did at the time and that the pins have since moved (“Layer 0 added ArkLib, then at dca90385, ...”). The ledger’s column becomes “ArkLib state at the pin” and is re-read at 7653a901.

At ArkLib 7653a901 every admission the ledger records still holds (table 3.8): the sumcheck’s single-round knowledge soundness (and, which the ledger omits, its perfect completeness); the composition of knowledge soundness (the same four **sorry** in `Append/Security.lean`, pull request 615 open and conflicting, so the local port stays); round-by-round to plain; context lifting; Fiat–Shamir completeness, still stated for the constant challenge oracle, with no soundness statement and the BCS transform commented out; the Johnson-range correlated agreement. The ring-switching packing is still admitted too, but its row is wrong for another reason: ArkLib’s construction is Diamond–Posen’s reduction, another protocol than leanVM’s ring switching (finding 43). The ledger’s column is re-read with these states, and a row is added for ArkLib’s typed framework, which has no knowledge soundness at 7653a901 (from R34, R59, R84).

Finding 142 (minor). Letter codes collide, and some point at the wrong thing

Severity. minor (avoidable reading cost; two misdirected references).

Evidence.

section 13.4: at least 128 codes and 1036 occurrences in the eight texts; 34 codes with two or more meanings inside the proof system’s own documents (the code **C1** has five: a hole, a ledger entry, and a finding of each of the three reviews), 11 more that mean something else in the leanISA status; within the status alone **A1** and **C1** each have four meanings. Two references resolve to the wrong finding: “finding F3 of the status file” (`protocol-blueprint.md:1440`) means the leanISA status’s finding, the protocol status’s being “one root for push and pull” (section 13.2.5, item 9); and the status’s legend says its letter **F** continues the leanISA numbering (`protocol-status.md:281-283`), where leanISA’s letter for the same subject is **R** (section 13.2.1, item 13). The blueprint’s “the generic holes carry the ledger letters of the ArkLib work they will become” (:592-593) is false (section 13.2.4, item 12). The codes have reached the trackers: issue 28 “Layer 2/C1”, issue 23 “P3”, issue 31 “P05” (section 13.4.2).

Classification. not a divergence from leanVM.

Proposed change.

section 13.5.2 (names for holes, ledger entries and findings; numbers kept, with names, for layers, acceptance tests, decisions and target theorems; one index, drafted in section 13.5.2.3). The two misdirected references: :1440 “(finding F3 of the status file)” becomes

```
| (the leanISA status's finding on the BLAKE2S value limbs)
```

`protocol-status.md:282-283` “**F** = Rust versus specification, continuing the leanISA numbering where the subject overlaps” becomes

****F**** = Rust or Python versus the specification (numbered independently of the leanISA status, whose
 ↪ letter for this is ****R****)

until section 13.5.2 retires the letters.

Finding 143 (minor). The same fact is written in three to six places

Severity. minor (drift, and a larger text to audit).

Evidence.

section 13.1: the upstream ledger three times with different rows and columns (`protocol-blueprint.md:257-267`, `protocol-status.md:171-184`, `issue-12-body.md:61-74`; the blueprint's has nine ArkLib rows, the other two drop one and add four); the specification of the bus phase four times with three different types (section 13.1, "Signatures"); the record of a review four times.

Classification. not a divergence from leanVM.

Proposed change. section 13.5.1 (one home per kind of fact), applied by appendix D.

Finding 144 (minor). The import rule of the proof system is false of main as written

Severity. minor.

Evidence.

section 13.2.5, item 12: convention *The wall* (:331) lists the modules allowed to import `Leaner VM/Arithmetization/`, and acceptance test 25 (:1341-1345) takes as its witness a `grep` that "lists the exceptions of convention *The wall* only". The `grep` returns `LeanerVM/Protocol/FixedColumns.lean:11` and `LeanerVM/Protocol/Basic.lean:3` (`public import LeanerVM.Arithmetization.Basic`); `Basic.lean` is the layer's empty root module (a docstring and an empty section), imported by `Leaner VM.lean:19` and by no module. It exposes nothing, so no phase sees the arithmetization through it; the rule's statement and its test are what is wrong. The rule has no script (`scripts/check-layers.sh:30-41` checks the layer DAG only), as the Layer 1 review found.

Classification. not a divergence from leanVM.

Proposed change.

either drop the import from `LeanerVM/Protocol/Basic.lean` (it declares nothing, so it needs none; the policy tests build their own fixture tree, `scripts/test-policy-checks.py:35-45`, and do not read this file), or add "the layer's empty root `Basic.lean`" to the exceptions of :331 and of acceptance test 25. Then write the rule into `scripts/check-layers.sh` with an allow-list, and a planted violation in `scripts/test-policy-checks.py`.

Finding 145 (minor). The review tooling the blueprint prescribes is not in the repository

Severity. minor.

Evidence.

`protocol-blueprint.md:1512`: "the reviewer runs the `leanerVM-review` skill's three passes"; no skill has that name; the skill is `adversarial-review`, as the status and the handoffs call it (`protocol-status.md:438, 479, 506`). Both it and `lean-spec-authoring`, which "sets the rules" (:178), live under `.claude/`, which is git-ignored (`.gitignore:9`; `git ls-files .claude` is empty), so a contributor who clones the repository has neither.

Classification. not a divergence from leanVM.

Proposed change.

the blueprint describes the review by what it does (appendix D.1, last bullet) and :178 says "(the rules are in `CONTRIBUTING.md`)"; or the owner tracks the two skills in the repository and the blueprint names them correctly.

Finding 146 (minor). "How work is tracked" is contradicted by practice

Severity. minor.

Evidence.

section 13.2.7: claims by comment on issue 12 are impossible for a contributor without write access, and the four live claims are in intention issues; intention issues are titled by layer, not by hole; the open pull requests name no hole, category, ledger entry or target; the labels are unused; the status uses a state (“built, draft”) the tracker does not define; `CONTRIBUTING.md:16-19` asks for a design issue first where the blueprint says no issue is opened for a hole; a review fact “is recorded on the dashboard’s history” (`protocol-status.md:553-554`), where no reader of the repository finds it.

Classification. not a divergence from leanVM.

Proposed change. appendix D.1, which also states the owner’s rule that a tracker changes by an edit of its body.

Finding 147 (minor). The blueprint records state and history, against its own rule

Severity. minor.

Evidence.

:1474-1475 “does not record history, status, or who is doing what”, and section 13.2.4, item 11 (seven places: “done on the spine’s branch”, “The port is done and closes hole C1”, “(#13, taken)”, “folded there on 2026-09-25”, “already in flight”, the column “Existing work” with pull requests in flight, “a rule ... is still to be written”).

Classification. not a divergence from leanVM.

Proposed change. delete those phrases and the columns “Existing work” and “Issue” (appendix D.2); the facts are the status’s.

Finding 148 (minor). Decisions are numbered, but most of their content is gone and the numbers collide

Severity. minor.

Evidence.

section 13.2.8: decisions 1 to 5 were questions in the first status (51021d9); today decisions 1 and 3 survive as “was taken” (`protocol-status.md:214`) and decisions 6 to 10 only as outcomes (:217-224); two choices have no number (:224-227); the blueprint cites “decision 8” and “decision 6” (:557, 560) without defining them; the leanISA roadmap numbers its decisions from 1, so “decision 15” means two things (issue 4 against `protocol-status.md:258`).

Classification. not a divergence from leanVM.

Proposed change. a section “Decisions” in the blueprint (section 13.5.1.3, item 11) with one row per decision: number, name, the choice in one line, the convention or test that records it (drafted in section 13.5.2.3); the status keeps none.

Finding 149 (minor). The list of public names is not the public boundary

Severity. minor (auditability: the reviewer’s reading list is incomplete).

Evidence.

:1419 “Everything not listed is a proof, a helper, or a test”. Of 333 non-private declarations under `LeanerVM/Protocol/` at b435631, 198 are not in the list (`unlisted_public.py` on an export of b435631; 197 on the working tree at 144c5aa), among them definitions the blueprint says later work reads: `PublicInput.check`, `pooled`, `prover`, `verifier`, `pSpec` (“Layer 12 reads those”, :1049-1051), `publicInputComplete`, `publicInputSecurity`, and the generic modules `ToArkLib/GuardedVerdict.lean` and `ToArkLib/KeepOracles.lean`, which the spine’s pass-through and send components now import (`protocol-status.md:74-75`) and of which the blueprint names neither the module `KeepOracles` nor its declarations. `Ensemble.toM3` is listed under “Spine:” (:1369) although it is Layer 2’s.

Classification. not a divergence from leanVM.

Proposed change.

add to the list, under “Protocol (leanVM)”, `publicInputComplete` `publicInputSecurity` `PublicInput.pSpec` `PublicInput.check` `PublicInput.pooled` `PublicInput.prover` `PublicInput.verifier` `PublicInput.verifier_verify`, and under “Protocol (generic)”, `Verifier.GuardedForm.of_prob` `Event_pos` `Reduction.mem_support_run_of_guarded` `keepOracles` `OracleVerifier.materialize` `Output_of_keepOracles` `probEvent_uniformSample_le_of_card_le` (names at b435631); name

GuardedVerdict and KeepOracles in the spine’s list of generic modules (:343-344); move Ensemble.toM3 out of “Spine:”. Reason: the list is the reviewer’s reading list (:1362).

Finding 150 (minor). Names and references that point at nothing, or at the wrong thing

Severity. minor.

Evidence.

StateMsg (:220) and leanIsaTables (acceptance test 5, :1280) exist nowhere (section 13.2.5, items 4, 5); “Lemma 5.1” (:601, and the status and tracker) does not exist in the specification, whose Lemma 5.2 and Theorem 5.1 share one counter (05-arithmetization.tex:36, 40; preamble/theorems.tex: 4-5); acceptance test 14 names bytecodeColumn_slot as its witness (:1306-1307), which the Layer 1 review showed restates the definition, the pinning statement being bytecodeColumn_answer_boolVec (FixedColumns.lean:85, 97); the hole table lists the spine’s Weight, WeightedClaim and FlockOut as products of the opening and Flock phases (section 13.2.5, item 6); the status’s ledger names a module ToCompPoly/Claims.lean that does not exist (:177); three citations into ArkLib at the old pin drifted (section 13.2.10).

Classification. not a divergence from leanVM.

Proposed change. :220 drops StateMsg (or names the state channel’s message type as leanISA declares it); :1280

| distinguished by the flush polynomials of the leanISA instance

:601 per appendix D.2; :1306-1307

| Witness: `bytecodeColumn\_answer\_boolVec` on a two-instruction program, against reversed slot bits
 ↳ (`tests/LeanerVMTTests/Protocol/FixedColumns.lean`)

the hole table per appendix D.2; the status per appendix D.3; the three ArkLib citations per section 13.2.10 (to be re-read at the new pin).

Finding 151 (minor). The upstream ledger misses the Merkle trees of the pinned VCVio, and cites a closed issue

Severity. minor (possibly avoidable audit surface).

Evidence.

section 13.2.9: the ledger says of Merkle trees “absent (only coding-theory lemmas) ... Written here” (:265), Layer 11 plans merkleRoot and merkleVerify locally (:1179-1182), and the upstream named for them is ArkLib issue 4, closed on 2026-09-27 in favour of VCVio issue 571. At the VCVio pin f9dc47d9, read before the upgrade, CryptoFoundations/MerkleTree/ held nineteen modules with no sorry.

Classification.

not a divergence from leanVM. Unverified: whether that library fits leanVM’s Merkle trees (BLAKE2s, the leaf encoding, pruned paths, crates/fiat_shamir/src/merkle.rs:14-67); and its state at the new pin a4232d08.

Proposed change. the ledger row and Layer 11 name VCVio’s Merkle-tree library and VCVio issue 571 as the upstream, with the decision whether Layer 11 builds on it.

The review does not make that decision. The library exists at the old VCVio pin f9dc47d9; whether it fits leanVM’s trees (untagged nodes hashed by one 64-byte BLAKE2s call, leaves hashed over several blocks, the pruned paths of the wire format, finding 60) and what it holds at the new pin a4232d08 are unverified. The decision is deferred to the owner, with that check as its first step; until then Layer 11 keeps its local plan and names the library and VCVio issue 571 as the upstream work to read first (from R77; the wire format from R63).

Finding 152 (minor). Discrepancies in the sources are not where both status files say they are

Severity. minor.

Evidence.

`protocol-status.md:281`: findings “numbered for citation from pull requests and `docs/leanvm-target.md`”; the blueprint: “Durable source discrepancies are also recorded in `leanvm-target.md`” (:1500-1501). `leanvm-target.md` records one discrepancy, the sentinel (:57-62), and none of the proof system’s twenty-seven findings against the specification, the Rust and the Python verifier (section 13.2.7, item 6).

Classification. the findings themselves are divergences inside leanVM (between its specification, Rust and Python); this finding is only about where they are recorded.

Proposed change. section 13.5.1.2: one register in `leanvm-target.md`, by name, for both roadmaps.

Finding 153 (note). The Layer 0 module’s comments cite the roadmap, layer numbers and a letter code

Severity. note.

Evidence.

section 13.2.5, item 13: `LeanerVM/Protocol/Field.lean:18, 29, 31, 37, 66` at `b435631` (“Protocol roadmap Layer 0 (`docs/roadmap/protocol-blueprint.md`)”, “(leanISA status finding P3)”, “(roadmap convention *The oracle*)”, “Layer 11”), against the convention “Comments ... cite the specification, never this roadmap” (:329). Still so at `144c5aa`, where the upgrade updated the same docstring’s pins and wrote “and of VCVio through it”, although VCVio is now a direct requirement. Every later module of `LeanerVM/Protocol/` follows the convention.

Classification. not a divergence from leanVM.

Proposed change. rewrite the module docstring’s first paragraph without the roadmap, the layer numbers and the code (“the eager `Fintype BF64` instance” in words), and “of VCVio, a direct requirement”.

Finding 154 (note). A review’s tracker edits are not applied, while its handoff says “met”

Severity. note.

Evidence.

the Layer 1 review left two edits for GitHub, “the L1 line and the open pull request table of the dashboard #12” (`docs/reviews/protocol-layer1.md:211`); neither was made (section 13.2.2, items 1 and 3).

Classification. not a divergence from leanVM.

Proposed change. appendix D.1, bullet “Reviews”: tracker edits a review asks for are made in the pull request that meets it.

Finding 155 (note). The completeness base theorem composes leanISA’s existence theorem, where `architecture.md` says witness generation

Severity. note.

Evidence.

`protocol-blueprint.md:147-148` (“the honest prover here starts from an `EnsembleWitness`”) and :1255 (`baseProver_complete` composes `constraintCompleteness`); `architecture.md:273-275`: the completeness half of base proof extraction and completeness (T4) “composes T2 with the honest prover”.

Classification.

a deliberate scope choice of the blueprint (witness-generator correctness (T2) is out of scope, :147-148), not written down as a deviation from `architecture.md`.

Proposed change. one sentence in Layer 13:

``baseProver_complete`` is existential in the witness (it composes leanISA’s ``constraintCompleteness``); the
 $\hookrightarrow$ computable form through the witness generator (T2) is ``architecture.md``’s, out of scope here.

Finding 156 (note). What was checked and found right

Severity. note (negative results).

Evidence.

section 13.2.10: every local Markdown link and anchor of the 24 Markdown files tracked at **b435631** resolves (150 links); every one of 81 citations into the pinned leanVM sources resolves to an existing range once its crate is known (22 are ambiguous as written: `transcript.rs`, `witness.rs`, `lib.rs`, `filler.rs`, and specification files that also exist under `doc/leanvm/drafts/`); 40 of 43 citations into the pinned libraries are at the cited place (the old pins); all 30 references to numbered acceptance tests point at the intended test; the spine's sketch in the blueprint (:442-554) agrees with the code; the Layer 8 sketch agrees with `PublicInput.lean`.

Proposed change. none; the ambiguous citations would name their crate (`crates/fiat_shamir/src/transcript.rs`).

Chapter 14

Keeping the Lean verifier from drifting away from the deployed one

The requirement, stated by the owner: a verifier that omits a check leanVM makes, or makes a weaker one, must have no proof of knowledge soundness; a verifier that makes a wrong check, or one leanVM does not make, must have no proof of completeness. The theorems, not the tests, are to keep leanerVM faithful. This chapter says where that holds today, where it does not and why, and what would make it hold as the work proceeds and the pin moves.

14.1 Where a weakening can hide

The chain from `verify` to the semantics (figure 2.1) has five places where a missing or wrong check could go unnoticed by the master theorems. For each, the review says whether it occurs today.

The relation weakened in step with the verifier. The relation is `M3Holds` `I` input `q`, and `I` is data: its `publicLines`, its `aux`, its `counts`, its `boundary`, its `constraints` and `flushes`, its `layout`. A verifier that omits the claim on the top memory limb, together with an instance that lists two public lines instead of three, is knowledge sound: every theorem of the spine and of the public-input phase holds of the two-line instance. A Flock verifier that checks nothing, together with an instance whose `aux` is `True`, is knowledge sound at error zero (probe `NoCheckFlock`). An instance whose layout aliases columns has an empty relation and both master theorems (probe `P2Relation`). *This occurs*. It is the price of stating the protocol over an abstract instance: the instance is where leanVM’s relation enters, and the oracle protocol’s theorems cannot check the instance. What catches a weakened instance is the adaptor’s soundness theorem, `satisfiedBy_witnessOf`, whose conclusion `SatisfiedBy` does not weaken with the instance: a two-line instance leaves its public-word conjunct unprovable (`SatisfiedBy.word0\_eq` has the top limb zero by the type of `PublicInput`); a trivial `aux` leaves `Blake2sRowsValid` unprovable; an aliasing layout leaves the completeness theorem `m3Holds_stackOf` unprovable. Behind the adaptor, `constraintSoundness` anchors `SatisfiedBy` to `ValidExecution`. None of these three theorems is built. **Until the adaptor is built, the master theorems constrain the instance by nothing, and a weakening of the instance is caught by no theorem on main.**

A property supplied as a hypothesis that nothing discharges. The master theorems take `Phases.Complete` and `Phases.Security` as hypotheses; four of the five phases are unbuilt, so the hypotheses are undischarged. Worse, `Phases.Security` can be discharged by phases that check nothing at declared error one (probe `P4Junk`): the error is the phases’ own field. *This occurs*, and it is the finding that most changes how the soundness theorem should be read (section 9.2). The repair is to fix the error in the spine, so that a phase can only meet its slot by earning the bound.

A check behind an assumed interface. Flock, ring switching, WHIR’s soundness, the Merkle compilation and Fiat–Shamir are consumed as interfaces. Today none exists in Lean; when they exist, a

check the deployed verifier makes inside one of them (the lincheck’s terminal identity; the Merkle path check; the grinding check) is load-bearing only for the interface’s own theorem, which another roadmap or an upstream library owes. *This occurs by design*, and the report accepts it on two conditions: that each interface is a structure whose fields are theorems with named owners and no inhabitant in the repository until they are proved (the repository’s own rule against uninhabited load-bearing types then flags them), and that the interfaces are stated for leanVM’s constructions, which the Fiat–Shamir and BCS ones today are not (section 8.7).

A property lost between the abstract instance and leanISA’s, or in the adaptor. The instance for leanISA does not exist; the adaptor does not exist. What the review found is that the blueprint’s sketch of them cannot be written as it stands (the admissibility predicate, the strided reader of the limb columns, the Flock predicate, the fit of the layout) and that the sketch’s `witnessOf` builds parts of the witness from the program, which is sound only if balance is then proved of the mixed witness (chapter 8). *Cannot occur yet*, since nothing is there to lose it; the report’s recommendations for Layer 3 are where it must be prevented.

A check present in the oracle verifier and absent from the compiled one, or in Lean and absent from Rust. The compiled verifier does not exist. Between Lean and Rust the review found one check Lean makes in a stricter form than the deployed verifiers (the per-limb public-input check, where they check one combined equation), one that the deployed verifiers make and Lean’s oracle protocol cannot (the proof of work), and three format checks the blueprint does not list (canonical encodings of the sizes and of the root, the stream fully consumed). *This occurs* at the public-input phase today, and would occur at the compilation as planned.

14.2 The public-input phase, as the test case

The owner’s own example was the top lane of the two public words. Following it to its anchor:

1. In the deployed verifier, two things enforce it: the statement-level rejection of a public word with a nonzero top limb at setup (`cpu/mod.rs:141-143`), and the pooled claim that the top memory limb is zero at $(r, 0, \dots, 0)$, for which no scalar is sent.
2. In Lean, the first is enforced by the type: leanISA’s `PublicInput` is four lanes of $\mathbb{K}$, so a nonzero top limb is not writable. The second is the third public line of the leanISA instance, which the blueprint specifies and which nothing on `main` states.
3. The phase pools the third claim because the instance lists a third line. Drop the line and every theorem of the spine and of the phase still holds (section 9.6). The master theorems do not catch it.
4. The adaptor’s theorem `satisfiedBy_witnessOf` would: `SatisfiedBy` requires the memory cells 0 and 1, as three-limb values, to be the public words, whose top limb is zero by type. Without the third line the top limb of the extracted memory is unconstrained and the conjunct is unprovable. Behind it, `constraintSoundness` carries the conjunct to the semantics.
5. So the answer to the owner’s question is: **the master theorems do not catch it; the adaptor’s soundness theorem does, once it exists; the blueprint’s acceptance test 10 is a test on the pool, not a theorem.** The chain is anchored, but its anchor is two unbuilt theorems away from the oracle protocol.

The same phase gave a second, harder lesson. Its check on the prover’s message is not load-bearing for any theorem, because the verifier pools the values it computes rather than the values it received (section 9.6). The requirement cannot be met for that check by any theorem about the phase as designed, since omitting the check yields a sound verifier: a verifier that ignores the message and pools the statement’s values is right to do so. What makes the check load-bearing is the design of the deployed verifiers, which pool the values *sent*: with `pooledFrom` in place of `pooled` the omission and the weakening are refuted by Lean theorems and a wrong check breaks completeness for every prover (probes `Probe2TrustProver`, `Probe3b`, `Probe7a`). The rule this gives is general: *a check is load-bearing only if the verifier commits to what it checked*. A phase that recomputes what the prover sent makes the message, and its check, decorative.

14.3 The checks of the deployed verifier, one by one

For every check the deployed verifier makes at the pin, the tables below give where it is in the three sources, what it protects, the concrete cheating prover or witness a verifier without it would accept, and, where a phase is built, the Lean check and the theorem that makes it necessary. They are the review’s enumeration from the specification and from the rejection paths of the Rust and Python verifiers; the setup’s checks and the format checks are included, since the compiled verifier must make them and the blueprint does not list them.

14.3.1 The setup, the commitment and the bus phase: the checks

Short forms in this section. Specification files are under `doc/leanvm/body/`: `03:` is `03-proving-primitives.tex`, `05:` is `05-arithmetization.tex`, `06:` is `06-bus-interactions.tex`, `08:` is `08-end-to-end-protocol.tex`. Rust files are under `crates/lean_vm/src/`: `mod.rs` is `cpu/mod.rs`, and `leaf.rs`, `gkr.rs`, `witness.rs`, `pcs.rs`, `constraints.rs` are at that root; `fs/` is `crates/flat_shamir/src/`; `flock/src/` is `crates/flock/src/` and `pcs/src/` is `crates/pcs/src/`. `py:` is `python-verifier/verifier.py`. The number after the colon is the line, at leanVM’s pin `a386121f`.

“Attack” is a prover or committed witness that the verifier **without** the check accepts and that lies outside the intended relation. Where the check does not protect soundness this is said: inventing an attack would be padding. The steps named in the table are those of tables 8.2 and 8.3.

Table 14.1: The checks of the setup, the commitment and the bus phase, as deployed.

#	Check	Specification	Rust	Python	Protects	Attack without it
B1	The stream holds every element read (truncation)	absent	mod.rs:132 (Transcript), gkr.rs:359, 381, 401, 408 (Truncated), leaf.rs:549 (Truncated), fs/transcript.rs:184-188	py:363-367	totality of the verifier	none: a verifier that read past the end would fail to run, not accept
B2	Announced sizes: upper limbs zero	absent	mod.rs:133-135	py:1373	one encoding per size (the layout uses the low limb only, the chain binds all three)	none on soundness: the same layout and the same statement; 2^{128} encodings of each proof. It matters to the recursion guest, which does arithmetic on the sizes
B3	Public words: third limb zero	08:29	mod.rs:141-143	by type	the seed binds only the two low limbs of each word (mod.rs:99-106), so two statements would share one transcript	a proof for the words (a, b) replayed for $(a + c \cdot y^2, b)$: the chain is the same. The public-input phase still rejects it except with probability $1/ E $, the memory columns being $\mathbb{K}$ -valued, so this is defence in depth (unverified beyond this argument; the public-input phase is another dossier's)
B4	The program has a power-of-two length, at most 2^{32}	06:90	mod.rs:158-159	py:857, py:862 (log2_strict, 0 =<= log_bytecode =<= 32)	the statement's shape; the read bound	see the check $\tau_j \leq 32$ (B6)
B5	$16 \leq \kappa_{\text{mem}} \leq 32$	06:86	mod.rs:160; mod.rs:51-52	py:859	upper: the read bound; lower: the guest's range checks assume at least 2^{16} cells (mod.rs:45-50)	upper: see the check $\tau_j \leq 32$ (B6). Lower: none on the relation's soundness; a smaller memory makes honest range checks fail
B6	$\tau_j \leq 32$	06:80	mod.rs:161; mod.rs:60	py:860	Lemma 6.3 (06:51-57): fewer than $2^{64} - 1$ reads. (The citation check corrected the number: the dossier had "Lemma 6.2"; Proposition 6.1 and Remark 6.2 share the theorem counter.)	with $2^{64} - 1$ reads of one pair (a, v) that is no entry of the array, counts $g^0, g^1, \dots, g^{2^{64}-2}$: the pulled counts and the pushed counts $g \cdot c$ are the same multiset, every count is nonzero, the bus balances, and the read is wrong. A mathematical counterexample, not a feasible one. Given the stacking window (B9) the three upper caps are implied ($\mu_{\text{stack}} \leq 28$ forces $\tau_j \leq 24$, $\kappa_{\text{mem}} \leq 26$, $\kappa_{\text{bc}} \leq 28$); they are checked first because the layout shifts by them (witness.rs:76). (The citation check found these implied caps true but loose: the tight bounds are $\kappa_{\text{mem}} \leq 25$, $\kappa_{\text{bc}} \leq 27$, $\tau_{\text{BLAKE2S}} \leq 19$, and $\tau_j \leq 24$ is tight for the other tables; nothing depends on the looser figures.)
B7	$\tau_{\text{BLAKE2S}} \geq 3$	absent	mod.rs:162-166	py:861	Flock runs on at least 2^3 instances (flock/src/hash.rs:283-286), so its claim is on 2^{3+8} words, while the block q_{flock} has 2^{75+8} (layout.rs:155)	not constructed. Without the check the ring-switched claim covers a window larger than q_{flock} 's block, overlapping its neighbours in the stack. Unverified: a test of a verifier without the cap at $\tau_5 = 2$ would settle whether a proof is accepted

#	Check	Specification	Rust	Python	Protects	Attack without it
B8	$1 \leq \rho \leq 4$	absent	mod.rs:167; pcs/src/whir_config.rs:44-55	py:1377	the WHIR parameters exist	none here: the opening's parameters are derived from ρ
B9	$15 \leq \mu_{\text{stack}} \leq 28$	absent	mod.rs:174-176	py:1379	the sizes WHIR is configured and tested for (pcs.rs:179-188). The lower bound never fires: μ_{stack} is floored at 15 (witness.rs:97, py:890)	none here: the opening's
B10	The root's halves have a zero third limb	absent	fs/merkle.rs:26-29	py:223	one encoding per root	none on soundness
B11	Layout facts, asserted: $\text{push.mu} = \text{pull.mu}$, $\text{count.mu} \leq \text{push.mu}$, at most 16 coordinates, $\text{soundness_bits}(\mu) \geq 128$	absent	leaf.rs:130-145 (assert!, a panic)	absent	the verifier's own arithmetic	none: they are functions of the announced sizes and hold for every size that passes the caps and the windows (B4 to B9)
B12	One root for both sides	08:68-69, 08:100 "the one bus root for both sides"	structural: one element read into two slots, gkr.rs:364-367	structural, py:434, 437	balance	with two roots and no comparison: any unbalanced stack. Concretely, an honest stack with one cell of <code>mem_0</code> changed at an address that is read: seed and finalization agree with each other, the reads carry the old value
B13	$R_c \neq 0$	06:78, 08:69	leaf.rs:887-889	py:546	every read count nonzero	a read of any (a, v) with count 0: it pulls $(\text{sep}, a, 0, v)$ and pushes $(\text{sep}, a, g \cdot 0, v)$, the same tuple. The bus balances, the honest R_c is 0. The stack has a read of a value no cell holds
B14	The layer check , once per layer ($\lceil \mu/2 \rceil$ checks)	implicit in 05:75, 83	gkr.rs:384-386, gkr.rs:411-413 (LayerMismatch)	py:449	the chain from the roots to the leaf claims	run by the probe (dossier gt-bus.md, section H.1): for an unbalanced bus the prover announces any R , passes the layers above the unchecked one by solving their one equation for one child, and sends the true children at the unchecked layer. The weakened verifier accepts with true leaf claims; the full verifier rejects at that layer
B15	The round identity of the layer sumcheck	03:79-85, 03:110	not a check: c_0 is derived, fs/transcript.rs:297-302	not a check, py:411-413	in a protocol that sends all five coefficients, the link between a round and the running claim	in such a protocol without the check: the prover sends the true cofactor whatever the claim; every later check is then about true values, and any root is accepted
B16	ζ has at least τ_{max} coordinates	absent	constraints.rs:250-253 (start of the table sumcheck)	absent	the zerocheck point exists	none: a function of the announced sizes; every table has a flush, so $\mu > \tau_{\text{max}}$
B17	The stream and the openings are fully consumed (end of the proof)	absent	mod.rs:769; fs/transcript.rs:213-219	py:1414; py:415-417	one encoding per proof	none on soundness

Nothing is checked at the boundary evaluations and the decomposition (U6 and U7): a wrong boundary evaluation is caught by the opening, a wrong rem_s by the table sumcheck’s final check. Rejections of these phases in the Rust are therefore `CpuError::PublicInput`, `CpuError::Transcript`, `Error::Truncated`, `Error::ZeroCount`, `GkrError::Truncated`, `GkrError::LayerMismatch` (`mod.rs:303-313`, `leaf.rs:75-81`, `gkr.rs:17-21`).

14.3.2 The table sumcheck and the public input: the checks

Short forms used in this section. The number after the colon is the line, at leanVM’s pin `a386121f`; a bare `:N` is a line of the file last named.

<code>03:</code> , <code>05:</code> , <code>07:</code> , <code>08:</code>	<code>doc/leanvm/body/03-proving-primitives.tex</code> , <code>05-arithmetization.tex</code> , <code>07-instruction-tables.tex</code> , <code>08-end-to-end-protocol.tex</code>
Rust files	under <code>crates/lean_vm/src/</code> , except the following
<code>transcript.rs</code>	<code>crates/feat_shamir/src/transcript.rs</code>
<code>py:</code>	<code>python-verifier/verifier.py</code>
<code>aggregate.py</code>	<code>crates/rec_aggregation/guests/aggregate.py</code> , the recursion guest

“Deferred” means: the check is a claim entering the pool, enforced by the opening. The last column says what the verifier accepts without the check, although it is outside the relation. The step numbers refer to tables 8.6 and 8.8.

14.3.2.1 The table sumcheck

Table 14.2: The checks of the table sumcheck.

Check	Specification	Rust	Python	Without it, accepted although outside the relation
Round message present (stream long enough)	implicit	constraints.rs:267 → Error::Truncated	py:364	not a soundness check: a format check
Round polynomial of degree at most 3	05:155	structural: the message is three coefficients	structural	with messages of unbounded degree the per-round error $3/ \mathbb{E} $ becomes $D/ \mathbb{E} $; at $D \geq \mathbb{E} - 1$ a prover sends a polynomial agreeing with the true round polynomial everywhere but at 0 and fixes the sum there
Round consistency $h_j(0) + h_j(1) = \text{claim}$	03:84; 08:100	by construction (transcript.rs:296-302)	by construction (py:409-410)	in a protocol sending four coefficients and not checking: any witness. The prover sends arbitrary polynomials and, in the last round, the true last round polynomial along the challenges drawn; the final check passes with the true column values. Example: a JUMP row with $v_{\text{cond}} = 1, b = 0$ (branch not taken on a nonzero condition)
Target derived from the leaf claims (no transmitted target)	05:151-155; 08:76	cpu/mod.rs:728-735	py:1393	with a transmitted target: the prover sends the true sum of F . Nothing then ties the tables' share of the bus to the GKR leaf claims: a committed XOR row whose memory flush is not what the bus trees were built from is accepted. The Rust says so (cpu/mod.rs:733-734: "A transmitted target would be a free value in its own check")
Constraints in the summand (the terms $\xi^{o_{t+i}} \cdot C_{t,i}$, contributing 0 to the target)	05:133-136, 146-150	cpu/mod.rs:380-383; tables.rs:725-730	py:621	a witness violating a JUMP identity: $v_{\text{cond}} = 1, w = 1, b = 0$. The state push $b \cdot v_{\text{pc}} + b \cdot g \cdot pc + g \cdot pc$ then falls through; the bus balances; the execution skipped a jump the program takes
Distinct powers of ξ per constraint	05:145	constraints.rs:74-82	py:621-623	with equal weights, the same row: $C_1 = b + v_{\text{cond}} \cdot w = 1$ and $C_2 = v_{\text{cond}} \cdot (b + 1) = 1$ cancel in characteristic 2
Bus forms in the summand, powers shared across tables	05:138-143	cpu/mod.rs:374, 404-422	py:622	without the forms: the tables' columns are tied to the bus by nothing. With one power per table and side, the target does not factor through totals_s and the per-table shares would have to be sent, free values in their own check
The point is ζ, drawn after the commitment	05:132	cpu/mod.rs:739 (&bus.point); order of pcs::read_commitment (:714) before the bus	py:1382-1385, 1394	with a point fixed before the commitment: commit a column whose constraint extension vanishes at $\zeta_{<\tau}$ and nowhere else on the cube
Final message present	implicit	constraints.rs:280 → Truncated	py:364	format
Final check (T10)	08:77	constraints.rs:288-290 → FinalMismatch	py:625	any witness: the prover sends the true column values and arbitrary rounds
The 104 column claims hold (deferred)	08:77, 97-99	cpu/mod.rs:664, 768	py:1395, 1413	the prover sends column values chosen to pass the final check, unrelated to the commitment
The eighteen limb claims are claims on q_{flock}'s slots (deferred, located by the routing of the limbs, T12)	08:77; 07:120	cpu/mod.rs:798-806	py:886-894	if the limbs were columns of their own: memory words read by BLAKE2S rows unrelated to the words Flock proves the compression of; any digest can be "computed"

14.3.2.2 The public input

Table 14.3: The checks of the public-input phase.

Check	Specification	Rust	Python	Without it, accepted although outside the relation
Top limb of the public words is 0 (P0)	a precondition	<code>cpu/mod.rs:141-143</code> → <code>CpuError::PublicInput</code>	by type	the transcript binds four lanes (<code>digest_words</code> , <code>cpu/mod.rs:99-106</code>), so two statements differing in a top limb would share a transcript (:139-140)
Two scalars present	implicit	<code>cpu/mod.rs:748</code> → <code>CpuError::Transcript</code>	<code>py:364</code>	format
The line (P3)	per limb	combined	combined	with no line check: any cells 0 and 1. The prover sends the true evaluations; every claim holds; the public input is bound to nothing
Claims on mem_0, mem_1 (deferred)	08:33, 84	<code>cpu/mod.rs:674-682</code>	<code>py:1402</code>	the prover sends the line's values whatever the memory holds
Claim $\text{mem}_2(r, 0, \dots, 0) = 0$ (deferred; no scalar)	08:84	<code>cpu/mod.rs:746, 674-682</code>	<code>py:1399, 1402</code>	a memory whose cell g^0 holds $w_0 + y^2 \cdot t$, $t \neq 0$. The two low limbs pass the line. The proof attests an execution on a word that is not the public word

The top limb, source by source. All three pool the claim with value 0 and send nothing for it. The recursion guest does the same (`aggregate.py:1688-1689: claim_pool[GEN ** claim_idx] = 0`). A public word with a nonzero top limb: excluded by the specification’s definition; rejected by the Rust verifier before any message is read (the Rust prover only `debug_assert!`s it, `cpu/mod.rs:552-555`); not expressible in the Python, nor in leanISA, whose `PublicInput` is four lanes of $\mathbb{K}$ (`LeanerVM/Semantics/Memory.lean:101-110`, leanerVM b435631), so the Rust’s rejection has no counterpart to model.

14.3.2.3 Around the two phases

The stream is fully consumed (`cpu/mod.rs:769`, `vs.finish()`; `py:1414`): trailing scalars reject. Every 192-bit pattern is an element of $\mathbb{E}$, so stream scalars have no canonicity check (`py:104-106` requires 24 bytes).

(The citation check made this precise: it holds of the scalars of these two phases, but the stream is not check-free in general. The root’s two scalars are decoded by `scalars_to_hash`, which rejects a non-canonical half (`transcript.rs:95-99`, `Error::NonCanonicalEncoding`), and the announced sizes must have zero high limbs (`cpu/mod.rs:133-135`; `py:1373`).)

14.3.3 The BLAKE2s validity phase (Flock) and ring switching: the checks

Short forms in this section; the number after the colon is the line, at leanVM’s pin a386121f.

- Specification, under `doc/leanvm/body/`: `c-flock`: is `c-flock-protocol.tex`, `a-ring`: `a-ring-switching.tex`, `03`: `03-proving-primitives.tex`, `06`: `06-bus-interactions.tex`, `08`: `08-end-to-end-protocol.tex`.
- Rust, under `crates/flock/src/`: `hash.rs`, `zerocheck.rs`, `lincheck.rs`; under `crates/flock/src/zerocheck/`: `zerocheck/multilinear.rs`, `univariate_skip_optimized.rs`; under `crates/lean_vm/src/`: `cpu/mod.rs`; under `crates/pcs/src/`: `stack_open.rs`; `transcript.rs` is `crates/flat_shamir/src/transcript.rs`.
- Python: `verifier.py` is `python-verifier/verifier.py`.

The phase has one equality check and a set of derivations that replace checks. A Lean verifier that receives what the deployed one derives must check it; each row says what a verifier without it would accept. The step numbers are those of table 8.13.

Table 14.4: The checks of the Flock phase and of ring switching, and the derivations that replace checks.

Check	Specification	Rust	Python	Protects	Accepted without it
Lincheck terminal identity (step 11)	c-flock:126-133; :270 “it is the only thing that checks anything”	lincheck.rs:1252-1256	verifier.py:1176	Everything upstream: v_a, v_b, v_c are the values of a, b, z at the point; the constant position; the link between the s_i and the matrices	Any q_{flock} . The prover commits limb slots with a wrong digest and zeros elsewhere, sends arbitrary messages at steps 3, 6, 7, 9 and the true s_i ; ring switching and the opening accept, since the s_i are true of q
Zeros assumed on H (step 5): the verifier interpolates with 64 zeros, it does not receive them	c-flock:62	zerocheck/ multilinear.rs:133-137	verifier.py:1144	This is where $a \cdot b + c = 0$ is imposed	If the 64 values on H were read from the prover: the prover sends the true P of a witness violating the R1CS; every later step is honest and passes
c_0 derived in each zerocheck round (step 6), equivalently the round check $c_0 + r_i(c_1 + c_2) = \text{claim}$	03:110, c-flock:69	transcript.rs:296-302	verifier.py:411-413	The chain from v_P to R_{zc}	With c_0 free: the prover ignores v_P , runs an honest sumcheck of the true (nonzero) sum, ends at true v_a, v_b, v_c
v_c derived (step 7), equivalently the check $R_{zc} = v_a v_b + v_c$	c-flock:70-73	zerocheck.rs:423	verifier.py:1149	The link between the zerocheck and the lincheck	With v_c sent and unchecked: true v_a, v_b, v_c of a violating witness pass the lincheck; the zerocheck constrains nothing
The α^3 term in the target (step 8) and in the terminal identity (step 11)	c-flock:23-28, :112-118, :131	lincheck.rs:1202, 1240	verifier.py:1162, 1174	$z(512, t) = 1$ for every compression	The all-zero block: every row of the R1CS is homogeneous, so $z_t = 0$ satisfies it (hash.rs:1176-1183 “homogeneous rows accept zero without the pin”). The limb slots are then zero: the statement “the compression of the zero input under the zero chaining value is zero” is accepted
c_1 derived in each lincheck round (step 9), equivalently $c_1 + c_2 = \text{claim}$	03:110	transcript.rs:298-299	verifier.py:409-410	The chain from the target to R_{lc}	With c_1 free: the prover runs the sumcheck of the true inner product of a wrong witness; the target is never compared

Check	Specification	Rust	Python	Protects	Accepted without it
T computed by the verifier from the s_i (step 13)	a-ring:45	stack_open.rs:522-524	verifier.py:1346	The s_i are the slices of the committed q_{flock}	With T sent: the terminal identity is one linear equation in 64 unknowns; the prover picks s satisfying it for any witness and sends the true $\langle W, q \rangle$
The weighted claim enters the batch (step 14)	08:98-99	stack_open.rs:522-524	verifier.py:1413	the same	the same
The limb claims enter the batch as claims on q_{flock}	08:77, 97	cpu/mod.rs:798-806, stack_open.rs:525-527	verifier.py:892, 1413	The limbs the bus used are the words Flock proved	A q_{flock} of valid but unrelated compressions, and limb values chosen to balance the memory bus: a forged digest in memory
The size floor $\tau_{\text{BLAKE2S}} \geq 3$ and the cap ≤ 32	cap only, 06:80; the floor is absent	cpu/mod.rs:161-166	verifier.py:860-861	The layout: q_{flock} has at least 2^3 blocks (hash.rs:283-286), and the limb columns share its instance cube	Nothing unsound is known: the mathematics needs no floor (the zerocheck needs $k_{\text{bool}} \geq 13$, zerocheck.rs:354, always true). A Lean verifier without it accepts size announcements the deployed ones reject. (The citation check adds: the Rust names the floor “the lincheck floor of $n_{\text{blocks_log}} \geq 3$ ($n_{\text{outer}} \geq 8$)”, hash.rs:281-282; the lincheck verifier has no such check, lincheck.rs:1151-1176 comparing lengths only, so the floor is a constraint of the prover’s layout; what in the lincheck prover needs $n_{\text{outer}} \geq 8$ was not checked.)
The stream is consumed	08:100 (implicit)	cpu/mod.rs:769	verifier.py:1414	No trailing data	Malleable proofs
Shape checks inside the reduction	none	lincheck.rs:1151-1176, zerocheck.rs:354-356	none	Internal consistency; they cannot fail on the deployed path (hash.rs:990-1008 passes constants)	nothing

Not a run-time check, and load-bearing: **the seven fixed coordinates have $\mathbb{F}_2$ -independent equality weights** (03:95-100, used at c-flock:278). The Rust asserts it in a unit test (univariate_skip_optimized.rs:920-960); the probe of the dossier’s section 9.3 recomputes it with the Python field: rank 128 (the citation check re-ran the probe and obtained the same output).

14.3.4 The opening and the compilation: the checks

Short forms in this section. Specification files are under doc/leanvm/body/: 06: is 06-bus-interactions.tex, 08: is 08-end-to-end-protocol.tex. Rust files: whir.rs, whir_config.rs and stack_open.rs are under crates/pcs/src/; fs/ is crates/flat_shamir/src/; mod.rs is crates/lean_vm/src/cpu/mod.rs, and pcs.rs, witness.rs are under crates/lean_vm/src/. py: is python-verifier/verifier.py. These are at leanVM’s pin a386121f. bp: is the blueprint at leanerVM b435631. A bare line number (:208) is in the file named before it in the same cell. The number after the colon is the line.

Every check of the opening and of the compiled verifier that is not a check of an earlier phase. The setup and commitment checks are B1 to B10 and B17 of table 14.1; they are listed here only where this part adds something. “Attack” is a prover, committed word or proof that a verifier **without** the check accepts although it should not; where the check does not protect soundness this is said rather than invented. The dossier cites the specification row by row for the setup only; for the other rows the specification’s column gives what its closing list (08:100) says of them, as the text after the table reads it, and a dash where that list says nothing. The messages named are those of table 8.19.

Table 14.5: The checks of the opening and of the compiled verifier, as deployed.

#	Check	Specification	Rust	Python	Protects	Attack without it
<i>Setup: what the sizes buy the opening</i>						
S-rate	$1 \leq \rho \leq 4$ (B8 of table 14.1)	absent (08:55 names no rate)	mod.rs:167 (validate_log_inv_rate, whir_config.rs:48-55)	py:1377	a rate below 1 leaves no redundancy; the parameter search exists only for $[1, 4]$ (whir_config.rs:43-45)	a rate-1 code ($\rho = 0$): every word is a codeword, $\gamma_0 < 1 - \sqrt{1} = 0$ is empty, and the query phase detects nothing: to cheat on the batched claim, the prover chooses $f^{(1)} = g + \Delta$ with $\text{Enc}_0(\Delta)$ a single-position word (rate 1 has weight-1 codewords) and $\langle W', \Delta \rangle$ fixing the residual claim; the t_0 queries miss that position with probability $(1 - 1/n_0)^{t_0} \approx 1$; the deeper levels are honest for $f^{(1)}$. Accepted with probability near 1
S-window	$15 \leq \mu \leq 28$ (B9 of table 14.1)	absent	mod.rs:174-176	py:1379	that a WHIR configuration exists and was validated for the size (pcs.rs:179-188: “every size in the window is configurable at every rate”); the recursion guest’s arms	none known: below 15 the ladder has fewer than two levels and <code>derive_ladder</code> errors (whir_config.rs:291-293), so a verifier without the bound would fail to run rather than accept; above 28 the search would produce a configuration the Python has no table for. The lower bound never fires (μ is floored at 15, witness.rs:97, py:890)
S-implied	the per-log caps $\kappa_{\text{mem}} \leq 32$, $\tau_j \leq 32$, $\kappa_{\text{bc}} \leq 32$	06:80, 86, 90	mod.rs:158-161	py:857-862	(B4 to B6 of table 14.1)	note: implied by the window, since every column’s 2^κ is at most the stack’s $2^\mu \leq 2^{28}$ (witness.rs:67-102, py:305-311); only the lower bounds $\kappa_{\text{mem}} \geq 16$, $\tau_5 \geq 3$ are independent
<i>Reading the stream and the openings</i>						
R1	the stream holds every scalar read	—	fs/transcript.rs:184-188 (ExceededStream)	py:363-367	totality	none: the verifier would fail to run
R2	every root’s two halves have a zero third limb	absent	fs/merkle.rs:26-29 (NonCanonicalEncoding), for the commitment root and every level root (whir.rs:2104, 2311)	py:222-224, py:1034	one encoding per root; the chain binds all three limbs while the tree uses two	none on soundness: 2^{128} streams per root, all with the same tree; a prover re-rolling every later challenge at the cost of one squeeze rather than one commitment, within the query bound’s accounting. Matters to canonicity and to the recursion guest
R3	a Merkle phase exists for every opening	—	fs/transcript.rs:271 (MissingHint)	py:385-390	totality	none
R4	the stream and the phase list are fully consumed	absent	fs/transcript.rs:213-219; mod.rs:769	py:415-417, py:1414	one encoding per proof	none on soundness (B17 of table 14.1)
R5	the residual f_{final} has exactly $2^{n_{\text{res}}}$ scalars	—	by count, whir.rs:2198	py:1032	the closing evaluation is of a table of the right size	none: the count is the verifier’s
<i>Grinding</i>						

#	Check	Specification	Rust	Python	Protects	Attack without it
G1	at every level, before the positions are sampled: the nonce read from the stream satisfies <code>compress(pow_base(cv), (nonce, 4))[0] ≡ 0 (mod 2¹⁷)</code> ; the nonce is bound either way	absent (from Annex B altogether)	<code>whir.rs:2117, 2202, 2325; fs/lib.rs:47-51, 165-174; fs/transcript.rs:314-323</code>	<code>py:377-383, py:1039</code>	17 of the 128 bits at each query round: the queries close $(1 - \gamma_i)^{t_i} \approx 2^{-111}$ (section 8.7.3.2), the grind the rest	a 2¹¹¹-work forgery. Commit to a word at distance $> \gamma_0$ from the code (or fold dishonestly at any level) and re-sample the query positions by changing the unchecked nonce: each nonce gives fresh positions at the cost of one squeeze; after about 2 ¹¹¹ nonces the positions all fall in the agreement set and the level passes. With the check each attempt costs 2 ¹⁷ hashes, restoring 2 ¹²⁸ none on soundness
G2	with <code>bits = 0</code> only the nonce 0 is accepted	absent	<code>fs/lib.rs:167-168</code> (bits = 0 never happens in production, all levels grind 17 bits)	<code>py:381</code>	canonicity of proofs at a zero-bit site	
<i>Merkle openings, per level (fs/merkle.rs:163-235); the Python reads the raw form</i>						
M1	the phase stores one row per distinct position (sorted)	08:100 “the PCS opening”	<code>fs/merkle.rs:139-141</code>	n/a (one row per query, duplicates repeated, <code>py:395-403</code>)	format	none
M2	every stored row has exactly <code>row_words</code> words (level 0: n_{lanes} , from the announced layout; deeper: $3 \cdot 2^4$), and <code>row_words ≤ leaf_words</code>	08:100 “the PCS opening”	<code>fs/merkle.rs:141-148; whir.rs:2130-2131, 2213-2219</code>	the leaf is read as <code>8 · leaf_words</code> bytes (<code>py:396</code>) with <code>leaf_words = 64</code> at level 0 (<code>py:1049</code>): the zero prefix is the prover’s	that the absent lanes of the level-0 image are zero, i.e. that the committed word past the placed columns is zero; the width of every row	none on soundness: a row with nonzero absent lanes is a different word w' , still bound by the root, whose extra lanes lie under no claim’s weight (every claim’s weight is supported inside the placed columns, by construction of the layout, <code>mod.rs:790-814</code> ; the citation check corrected the dossier’s <code>stack_open.rs:359-362, 497-500</code> : the first is the prover’s assertion that every claim lies inside the committed lanes, the second, the verifier’s, bounds claims only by the full cube) and outside every column of the layout, so $\Lambda(w')$ restricted to the layout is the same. The Python’s accepted set is strictly larger than the Rust’s on this point
M3	<code>num_leaves</code> is a power of two, nonzero; the query list is nonempty	08:100 “the PCS opening”	<code>fs/merkle.rs:171-173</code>	by construction	the tree shape	none: verifier data
M4	every position is below <code>num_leaves</code>	08:100 “the PCS opening”	<code>fs/merkle.rs:176-178</code>	<code>py:400</code> (shift)	—	none: positions are masked at sampling (<code>whir.rs:1189, py:950</code>)
M5	the octopus has exactly the siblings the paths need: none missing, none surplus	08:100 “the PCS opening”	<code>fs/merkle.rs:195, 197, 208</code>	by fixed-length parsing and R4	canonical format	none on soundness (a surplus sibling is never hashed)

#	Check	Specification	Rust	Python	Protects	Attack without it
M6	the recomputed root equals the expected root: the commitment root at level 0 (a statement value bound before any challenge, <code>pcs.rs:136-138</code>), the root read from the stream at level $i \geq 1$	08:100 “the PCS opening”	<code>fs/merkle.rs:208</code> (<code>nodes[0].1 != *root</code>)	<code>py:402</code>	binding: the opened rows are the committed ones	total break: open any rows; at level 0 answer the t_0 positions with a row consistent with a dishonest $f^{(1)}$ ($c_q := \text{Enc}_0(f^{(1)}[x_q]$, recomputed after the positions are known), at every level likewise; every intro claim then holds and the terminal check passes for any batched claim
M7	height and leaf width are the verifier’s (<code>height = log₂ num_leaves, leaf_words</code>), never read from the proof	08:100 “the PCS opening”	<code>fs/merkle.rs:174, 139-148; whir.rs:2084, 2213</code>	<code>py:393, 396</code>	that no 64-byte node preimage can be presented as a leaf and vice versa without domain separation (section 8.7.1.9)	if the prover chose the height: build a tree one level deeper than announced and open, as “leaves”, the 64-byte preimages of depth- h nodes; but with fixed <code>leaf_words</code> $\neq 8$ the image length differs and the hash cannot match, so no attack is known even then. Recorded because domain separation is absent and this is what replaces it
<i>WHIR’s per-level checks and the final check</i>						
W1	the round identity $h(0) + h(1) = \text{claim}$ at every round	08:100 “every sumcheck’s rounds”	not a check: c_1 is derived (<code>fs/transcript.rs:297-299</code>)	<code>py:409-410</code>	in a protocol that sends all coefficients: the link between the round and the running claim	in such a protocol without it: send the true h whatever the claim; every later check is about true values and any C_λ is accepted. In the deployed encoding the attack has no message to carry it
W2	the out-of-domain answer y	08:100 “the PCS opening”	not checked when read (<code>whir.rs:1672</code>); glued into the sumcheck with λ_i and settled at W5	<code>py:1036-1037</code>	that the new oracle’s list has at most one member consistent with the transcript (Lemma B.11)	dropping the OOD claim from the glue: the prover commits at level 1 to a word close to two codewords U, U' and answers the queries with whichever fits; Theorem B.7’s query row then pays $(1 - \gamma)^t$ per member (a union over L_1), which the 128-bit accounting does not include (<code>whir_config.rs:329-333</code> : “Plain Johnson without OOD binding would be unsound at these parameters: the query phase would pay a union bound over the interleaved list (19 to 52 bits here)”)
W3	the consistency of the opened rows with the fold: <code>enforced_sum_i</code> $= \sum_q \lambda_i^q \cdot \langle \text{row}_q, \text{eq}(r_{\text{level}}, \cdot) \rangle$ is the claim of the intro polynomial	08:100 “the PCS opening”	computed by V (<code>whir.rs:2140, 2226, 2346</code>) and glued; settled at W5	<code>py:1051, 1057</code>	that $f^{(i+1)}$ is the fold of the committed $C^{(i)}$ on the queried columns (steps 3(c) and 3(d))	with the intro claim taken from the prover instead: commit any $f^{(i+1)}$; all consistency claims are on prover-chosen values; the residual claim is discharged by the dishonest $f^{(i+1)}$; accepted
W4	the level batching: OOD first, queries after, powers $\lambda_i, \lambda_i^2, \dots$ with the running claim at λ_i^0	08:100 “the PCS opening”	<code>whir.rs:1131-1154, :1681-1701</code>	<code>py:1058-1063</code>	that every claim of the level is settled by one polynomial identity (Theorem B.7’s batching row, $(J_i - 1)L_i/ \mathbb{E} $)	a batch with two claims on the same power: the prover balances a false OOD claim against a false query claim $(\lambda \cdot (y - y_{\text{true}}) + \lambda \cdot (s - s_{\text{true}}) = 0)$ with probability 1

#	Check	Specification	Rust	Python	Protects	Attack without it
W5	the terminal check $weight \cdot \tilde{f}_{\text{final}}(\rho_{\text{tail}}) = t_r$, with $weight$ the sum of $\tilde{W}_\lambda$ at ρ rotated, of $\Sigma_i \beta_i \cdot$ (the induced weight of level i 's queries at the tail of ρ from level i 's start) and of $\Sigma \beta \cdot \text{eq}(z, \rho\text{-part})$	08:100 “the PCS opening”	whir.rs:2264-2308	py:1075-1083	everything: it is the one equation the whole sumcheck chain ends on	without it nothing is checked about q : any commitment, any C_λ
W6	the shape guards $n_{\text{current}} \geq k_i$, $\text{ctx.log_msg_cols} \geq \text{yr_log_n}$, $\text{at.len}() = 1$, prev.advance	—	whir.rs:2188-2190, 2266-2268, 2280-2282, 2374-2384	absent	the verifier's own arithmetic on the configuration	none: functions of (μ, ρ) , true for every supported size
W7	$n_{\text{lanes}} \in [1, 2^6]$, $r \geq 1$, vector lengths, $\text{ood_samples}[0] = 0$	—	whir.rs:2064-2072	absent	configuration sanity	none: verifier data
W8	ring switching: $s_{\text{hat_v}}$ present, of length 64	—	stack_open.rs:507-510	n/a (the Python builds the target from Flock's 64 values, py:1346)	statement well-formedness	none (Flock's claim; the dossier on Flock and ring switching, gt-flock-ring.md)

The blueprint’s `Sizes.Admissible` must contain the rate window and the stacking window (S-rate and S-window; finding 8).

The specification’s closing list. `08:100`: “Accepts if every check above passed: the caps, the one bus root for both sides, the nonzero count root, every sumcheck’s rounds and final value, the public-input line, flock’s reduction, and the PCS opening.” Against table 14.5: “the caps” omits the rate window and the stacking window (S-rate, S-window); “every sumcheck’s rounds” is not a check in either verifier (W1); “the PCS opening” stands for M1 to M7, W2 to W5 and, absent from Annex B altogether, G1. Absent from the specification and present in both verifiers: the canonical encodings (B2, B3, B10 of table 14.1; R2), full consumption (R4), grinding (G1, G2). The blueprint’s `verify` inherits the omissions of the specification it is “written from” (`bp:1216`) unless Layer 12 lists the checks of table 14.5; its six mutations (`bp:1238-1239`) exercise M6 and the scalars only (finding 67).

Negative results. Checked, no finding: the query sampling is exactly uniform (d -bit chunks of a uniform 192-bit string, $d \leq 28$, $\lfloor 192/d \rfloor \geq 6$ chunks per squeeze, a valid weaker bound: the citation check finds $d \leq 26$ at the supported sizes, so $\lfloor 192/d \rfloor \geq 7$; `whir.rs:1175-1193`, `py:943-951`); the two verifiers derive n_{lanes} , the block lengths and the leaf widths from the announced sizes (`whir.rs:2061-2066`, `2084`, `2213`; `py:1040`, `1048-1049`); both floor μ at 15 (`witness.rs:97`, `py:890`); the Python’s `derive_config` and the Rust’s `configs_for_rate` give the same ladder for every (μ, ρ) in the window (?? B.2.1.3).

Three checks stand out because no planned theorem makes them load-bearing: the proof of work (grinding), the canonical encodings, and the consumption of the stream. The first is a soundness check in the non-interactive setting only, and no published theorem covers Fiat–Shamir with proof of work for a multi-round protocol; the other two are format checks whose omission lets `verify` accept streams the deployed verifier rejects. All three need a mutation test in the compiled verifier’s fixture (section 14.4).

14.4 What keeps it true

Stated once, so that it cannot drift. The relation, the seams and the error: the first two are the spine’s already; the error is to be (section 10.3). With the error fixed by the spine, a phase that declares less than it earns does not typecheck against its slot.

What each phase delivers before it lands. For each check of the phase’s verifier, a *refutation*: a Lean theorem that the verifier without that check has no round-by-round knowledge soundness against the seams at any error below one, whatever the extractor and the state function; and for each check the deployed verifier does not make, a theorem that the verifier with it is not perfectly complete. The public-input phase’s audit gives the two generic lemmas this needs (`not_rbr` and `not_perfectCompleteness`, appendix B); they belong in `ToArkLib/`. A mutation whose omission cannot be refuted is a check the verifier does not rely on, and the phase’s design should be revised until it does (the rule of section 14.2). These refutations are tests under `tests/`, run by `lake test`, and a phase’s pull request lists them.

What ties the Lean verifier to the deployed one. The compiled verifier’s refinement theorem ties `verify` to the oracle protocol by definition; only a fixture ties either to the Rust verifier. The fixture is a proof produced by the pinned Rust prover, accepted by `verify` under `#guard`, with a mutation per check: each of the checks of section 14.3 gets a mutated proof that the deployed verifier rejects and that `verify` must reject too. The blueprint’s six mutations become as many as there are checks. Reverse tests, transcripts the specification rejects run through the Rust and Python verifiers, are how the per-limb public-input divergence would have been found by a test rather than by a reader; the review recommends them for every check whose statement in the specification differs from the code.

What is examined again when the pin moves. The leanVM pin: every check of section 14.3 against the new verifier’s rejection paths, the message and challenge counts of every phase, the layouts and their tie-breaking orders, the constants of Flock and of Fiat–Shamir, the parameter tables. The library pins: the admitted theorems the ledger names (this upgrade lifted none), the notation of the probability bridges (this upgrade changed it), the meaning of numerals in $\mathbb{K}$ (this upgrade changed it), and every acceptance test that **decides** a numeral.

What the repository’s validation can enforce. Today it enforces the axiom closure, the absence of `sorry`, the layer DAG and the documentation index. It can also enforce: the wall (a rule in `scripts/check-layers.sh` listing the modules allowed to import the arithmetization, which today would flag `Protocol/Basic.lean`); the presence, for every built phase, of a refutation test per check (a script that reads the phase’s checks from a manifest and asserts a test per entry); the fixture and its mutations, once the compiled verifier exists; and, at a pin move, a script that re-derives the message and challenge counts of every phase from its `Def` and compares them with counts dumped from the Rust prover. What it cannot enforce is that a phase’s definition is leanVM’s: that rests on review against the specification, which is why this report’s tables of the transcript exist.

Part V

Appendices

Appendix A

Proposed changes to the blueprint

One entry per change, ordered by where in the blueprint it applies, each pointing to the finding that carries its full text: the passage as it stands, the passage as proposed, and the reason. The changes to the status file, the tracker and the other documents are in appendix D; the changes to the code are in the second table. Nothing here has been applied.

How to read this appendix. Each row of table A.1 is one change to the blueprint (`docs/roadmap/protocol-blueprint.md`), placed where it applies: the rows follow the blueprint’s own order, section by section (the headline and “For zkVM engineers”, the scope, the contracts and the ledger, the pinned conventions row by row, the spine, Layers 0 to 13, the acceptance tests, the interfaces, the boundaries, the ordering, how work is tracked, the references), and a finding that touches passages in several sections has a row in each. The first column names the passage by its section and its lines at leanerVM `b435631` (at `144c5aa` every line after 51 is four higher; the lines were re-read at `b435631` for this table, and where a dossier’s line differs the table gives the right one). The second column says the change in a sentence or two. It is a summary: the text that governs is the one the third column points to, the register row (`R1` to `R140`, appendix F) and the finding box that gives the passage as it stands, as proposed and the reason, or, where no box has been typeset, the section of the dossier under `.claude/reports/blueprint-review/dossiers/`. The fourth column is the classification the dossier gives, in one of four words: an error of the blueprint, a deliberate deviation, a deviation forced by an upstream library, or documentation (stale, missing or misplaced text that is no divergence from leanVM); where the dossier gives none, the column says so. The last column is the severity of the strongest finding behind the row.

Three kinds of change are not in table A.1. Changes to the status, the tracker, the hole comment, `docs/architecture.md`, `docs/leanvm-target.md` and the other documents are in chapter 13 and in the drafts of appendix D (among them the status rows `R38`, `R83`, `R94`, `R95`, `R97` and the attribution of `R100`). Findings against leanVM’s own sources, to be reported to leanVM or recorded in `docs/leanvm-target.md` (`R105`, `R107` to `R109`, `R112`, `R116` to `R118`), change nothing here; `R106` and `R111` appear where they bear on a row. Changes to the code are in the shorter table A.2, with the file and lines at `b435631` in place of the passage. The rows that ask the owner to choose between alternatives are marked [decision]; the alternatives and the review’s recommendation are in appendix A.3. The register’s positions `R3` and `R8` and its rows that propose no change to the blueprint (`R101`, `R104`, `R110`, `R113`, `R115`, `R123`, `R126`, `R133`, `R134`, `R136`, `R140`) appear only where they bear on a row.

A.1 Changes to the blueprint

Table A.1: Proposed changes to the blueprint, in the blueprint’s order (lines at b435631).

Passage	Change	Register row; full text	Classification	Severity
Throughout the blueprint				
Throughout (codes in :260, :592-593, :1440 and passim)	Holes, ledger entries and findings are named in words; layers, acceptance tests, decisions and target theorems keep their numbers with the name beside them; one index of names at the end of the blueprint. The two references that resolve to the wrong finding are rows below.	R85; finding 142; section 13.5.2	documentation	MINOR
Throughout (the ledger, the holes’ specifications, the bus phase’s four statements)	One home per kind of fact: the blueprint holds what is wanted and decided (targets, conventions, the specification of every hole, the acceptance tests, the decisions, the upstream ledger); the status, the tracker and the hole comment link to it and restate nothing.	R86; finding 143; section 13.5.1	documentation	MINOR
The headline and “For zkVM engineers” (:1-103)				
Headline, :8-17	The two pseudo-signatures carry the bundle of phases and its proofs (P, C, S), the named extractor <code>piopExtractor P S</code> and the error <code>piopError I</code> ; a sentence says both are composition theorems, which hold of every bundle meeting the seams and say nothing of leanVM until the instance and the five phases are leanVM’s, and that <code>main</code> proves the spine, the commit phase and the public-input phase.	R45; finding 90	error of the blueprint	MINOR
Headline :21; For zkVM engineers :77	“the polynomial view of any Clean ensemble” and “which <code>Ensemble.toM3</code> derives from any <code>Clean Ensemble</code> ” go (or <code>Ensemble.toM3</code> is defined as the six-table part only): the <code>leanISA</code> instance is not <code>Ensemble.toM3</code> of the eight tables, and no map from an <code>Ensemble</code> to an <code>M3Instance</code> exists (an instance needs μ , a layout, public lines, <code>aux</code> , <code>Stmt</code>).	R30; finding 71	error of the blueprint	MAJOR
For zkVM engineers, :62	Round-by-round <i>knowledge</i> soundness is from [CMS19] Definition 8.5, not “the standard form since [CCHLRR19]” (the reference entry itself is corrected under References).	R98; literature.md §A.4 (fifth finding)	error of the blueprint	MINOR
For zkVM engineers, a paragraph after :94	State the scope of the random-oracle assumption: the non-interactive theorems are in the random-oracle model for the one 64-byte BLAKE2s map the chain, the Merkle trees and the grinding share; they do not cover attacks that use the code of BLAKE2s ([KRS25], [Fen26]); leanVM binds the statement and commits the trace before the first challenge, so the known attacks do not apply, and no theorem says more.	R67; literature.md §B.4	error of the blueprint (omission)	MINOR
For zkVM engineers, :99-101	“reads the stack <code>q</code> off the oracle message” is true of the commit phase’s intermediate extractor at round 0, not of the composed extractor’s output (the last phase’s); the extracted stack becomes a definition, <code>piopExtractedStack</code> , to which the adaptor applies <code>witness0f</code> .	R46; finding 91	error of the blueprint	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Scope (:105-158)				
In scope, :109 (:113 at 144c5aa)	“ArkLib as a Lake dependency at dca90385” becomes “ArkLib as a Lake dependency, with ...”: the pin moved at 144c5aa, and pins are stated only in <code>upstreams.json</code> and <code>docs/dependencies.md</code> .	R84; finding 141	documentation (stale at 144c5aa)	MINOR
In scope, :126-128	“the claim pool and the opening sumcheck” loses “the opening sumcheck”: with the inner-product oracle the opening phase is the batching challenge and one weighted query (specification §8.5, “there is no separate reduction sumcheck”). [decision]	R18; finding 51 and section 8.7.1.5	error of the blueprint (caused by Layer 0’s evaluation interface)	MAJOR
Out of scope, :153-155	Say that [BCHKS25] Theorem 4.6, the one coding-theory input, is a preprint theorem whose proof is sketched (§4.3; the full argument for the quadratic bound in [Hab25]), peer review pending as of 2026-09.	R114; <code>literature.md</code> §C.4 (first finding)	documentation (a fact about the assumption)	NOTE
Out of scope, after :157	Add: security against quantum adversaries (the quantum random-oracle model) is out of scope; the classical bound does not transfer (the loss is quadratic in the query count).	R68; <code>literature.md</code> §G.5 (second finding)	error of the blueprint (omission)	MINOR
Out of scope, a new item	Record that adaptive statement soundness in the random-oracle model (the input seeds the chain) is the deployment theorem’s (T7) to state, and is not given by this roadmap’s per-statement theorems, whose games quantify over the input statement outside the probability.	R103; finding 87	none given	NOTE
Dependencies and exact contracts (:160-306): the pins, the sources, the tables, the ledger				
Introduction, :164-168 (:167-172 at 144c5aa)	The list of pins (ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef, VCVio f9dc47d9 “through ArkLib”, Lean v4.33.1) becomes “The pins are in <code>upstreams.json</code> , with the history of each in <code>dependencies.md</code> ; leanVM is pinned at a386121f.”	R84; finding 141	documentation (stale at 144c5aa)	MINOR
The leanVM specification and implementation, :178	“the authoring skill <code>lean-spec-authoring</code> sets the rules” becomes “(the rules are in <code>CONTRIBUTING.md</code>)”, or the owner tracks the skill in the repository: it lives under the git-ignored <code>.claude/</code> .	R87; finding 145	documentation	MINOR
Table of artifacts, row Flock, :192	Two rows: “Flock: protocol and errors A, owned by #3 specification Annex A, Annex C” and “Flock: the BLAKE2s circuit, wire positions, limb slots, φ_8 , the fixed coordinates, <code>RICS_DIGEST</code> B the Rust and Python lines that fix them”. The second row also gives the seed’s constant <code>RICS_DIGEST</code> , which cannot be recomputed at the pin, an owner.	R57, R106; finding 41, finding 48	forced by the source (the specification is incomplete; the blueprint follows the Rust, rightly)	MINOR
Table of artifacts, row Count blocks, :197	Cite <code>leaf.rs:606-622</code> (the count side), not <code>leaf.rs:664-668</code> (the signature of <code>prove_balance</code>).	R92; finding 18	error of the blueprint	MINOR
leanISA table, :220	<code>StateMsg</code> exists nowhere (the state channel’s message is <code>Regs</code>): drop it or name the type leanISA declares.	R92; finding 150; finding 84	error of the blueprint	MINOR
leanISA table, :221	Delete “(#13, taken)”: the blueprint records no state.	R89; finding 147	documentation	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
leanISA table, :222	The row consumes <code>constraintSoundness</code> , <code>WellFormedBytecode</code> , <code>constraintCompleteness</code> (leanISA Layer 10), not <code>HasFillBlocks</code> : both leanISA theorems take <code>WellFormedBytecode</code> prog. [decision]	R27; finding 137	error of the blueprint (against the leanISA roadmap)	MAJOR
ArkLib table, :232-251	Correct the names and places: <code>schwartz_zeipel_counting</code> is in the root namespace; <code>prob_eval_zero_le_div</code> is a PMF statement; <code>Sumcheck.Domain</code> is <code>SumcheckDomain</code> ; <code>Direction</code> is in <code>Prelude.lean</code> ; add the completeness theorem the spine uses (<code>OracleReduction.append_perfectCompleteness_of_guarded_verifiers</code>); say that <code>CheckClaim</code> 's oracle variant checks nothing at run time and that the sumcheck's perfect completeness is admitted too.	R96; finding 123	error of the blueprint	MINOR
ArkLib table, row round-by-round knowledge soundness, :239	Add: ArkLib's extractor type accepts a classical choice (its own <code>toRoundByRoundOfRel</code> is one), so a theorem in the existential forms <code>rbrKnowledgeSoundness</code> or <code>rbrKnowledgeSoundnessWorstCase</code> proves soundness only; every interface and theorem of the roadmap is stated in the <code>With</code> form with a compiled extractor, checked by a <code>def</code> without <code>noncomputable</code> at each phase's <code>Security</code> .	R1; finding 118	forced by an upstream library (ArkLib has no notion of running time)	CRITICAL (as a warning)
ArkLib table, the multilinear bridge, :246	Before a phase imports ArkLib's <code>ToCompPoly/Multilinear/Basic</code> , elaborate a file importing it and <code>CompPoly's Multilinear/Basic</code> (<code>eval_zero</code> is declared in both at the new pins); if it fails, cite the bridge with that caveat and report the clash upstream. Unverified.	R128; finding 135	forced by an upstream library (version skew)	NOTE
ArkLib table, row sampling, :251	"Layer 0 supplies the E instance (built from a K instance)".	R74; finding 126	documentation (avoidable audit surface)	MINOR
Ledger, heading, :257 (:261 at 144c5aa)	"ArkLib state at dca90385" becomes "ArkLib state at the pin", every row re-read at 7653a901 (the library review found no relevant admission lifted between the pins).	R84, R34; finding 141, finding 120	documentation (stale at 144c5aa)	MINOR
Ledger rows on the knowledge-soundness composition and on round-by-round to plain, :260-261	Composition (A2): the port is kept until ArkLib, on whatever framework, proves a guarded-first append for the worst-case named form, not "deleted when the pin moves past #615" (open and conflicting). Round-by-round to plain (A3): the plain corollary is stated from the transcript-level reading (probe <code>PlainReading</code> , 40 lines) and a local union bound, since upstream has no plan to prove the implication. A new row: ArkLib's typed Interaction framework has no knowledge soundness at 7653a901; the spine cannot migrate to it yet. [decision]	R34; finding 120	error of the blueprint's plan	MAJOR
Ledger row on context lifting, :262	"Not consumed: statements are shaped so that no lens is needed" is false for the bus phase, whose GKR runs on leaf tables derived from the stack; it becomes the reason the proposed GKR over a context with riders (Layer 5) needs no lens.	R10; finding 7	error of the blueprint; forced by an upstream library (lifting admitted at both pins)	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Ledger row on Fiat–Shamir and BCS, :263	No ArkLib theorem states Fiat–Shamir or BCS knowledge soundness at either pin (<code>fiatShamir_completeness</code> admitted and for a constant oracle, <code>BCSTransform</code> commented out, <code>Commitment.extractability</code> with body <code>False</code>): <code>FiatShamirSecurity</code> and <code>BcsSecurity</code> are assumed interfaces whose obligation is the literature. The upstream theorems, when they come, are about other constructions (one oracle on the full prefix or a duplex sponge; independent oracles per role) than leanVM’s one BLAKE2s map for the chain, the Merkle nodes and the grinding: a chain lemma and a role-separation lemma are owed here. #848 and #469 are single-salt and duplex-sponge Fiat–Shamir with no proof of work; #627 is a design issue.	R20, R22; finding 121; <code>literature.md</code> §E.6 (first finding); finding 54	error of the blueprint; forced by an upstream library	MAJOR
Ledger row on WHIR, Merkle, BLAKE2s, :265	Merkle trees are not absent upstream: VCVio at <code>f9dc47d9</code> has nineteen sorry-free Merkle-tree modules. Name VCVio’s library and VCVio issue 571 (ArkLib issue 4 is closed), with the decision whether Layer 11 builds on it (the fit with leanVM’s trees is unverified). [decision]	R77; finding 151	documentation	MINOR
Ledger row on ring switching, :267	ArkLib’s <code>RingSwitching/Packing</code> is Diamond–Posen’s interactive reduction (carrier message, batching vector, relocation sumcheck), 14 sorry at both pins; leanVM’s ring switching is rectangular, sends nothing and draws six challenges: new work of #3, at most the tensor algebra shared; no “profile” makes one the other.	R59; finding 43	error of the blueprint (of description)	MINOR
CompPoly table, :278	“the encoder of Annex B.3”: the pinned numbering is §B.5; cite Annex B by label.	R92; finding 63	error of the blueprint	MINOR
CompPoly paragraph, :281-284 (:285-288 at 144c5aa)	“no $GF(2^8) \hookrightarrow E$ embedding” becomes “... no embedding $GF(2^8) \hookrightarrow K$ (CompPoly now has the AES field $GF(2^8)$, #356, and its embedding into GHASH only)”. The sentence on the eager <code>Fintype BF64</code> is stale at 144c5aa, whose CompPoly pin meets its retirement condition; whether <code>lake test</code> can again be an executable is the owner’s check.	R84, R125; finding 129, finding 132	documentation (stale at 144c5aa; forced upstream, since lifted)	MINOR
Pinned conventions (:308-335), row by row				
Row <i>Sumcheck messages</i> , :315	The oracle protocol sends four coefficients and checks the round, the wire three and none. Either the wire’s message becomes the oracle protocol’s (three coefficients, the fourth derived, no round check), or the row stays and Layer 4 proves the transport lemma (a verifier composed with an injective decoding of wire messages onto the messages passing its round check keeps its round-by-round knowledge soundness), Fiat–Shamir then taking a challenge oracle on the encoded message. [decision]	R53; finding 25	deliberate deviation (it owes a lemma the blueprint does not state)	MINOR
Row <i>Statements and parameters</i> , :316	Add: the sizes are the prover’s; the non-interactive theorem is stated for the family, its error <code>niError Q</code> the maximum over admissible sizes (or the sum, if the interface is per instance), and <code>FiatShamirSecurity</code> is the family form, the challenge oracle taking the announced sizes with the statement. (A union over the admissible sizes costs about 2^{34} , a hand estimate.)	R6; finding 76	error of the blueprint (a missing decision)	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Row <i>The oracle</i> , :317	The stack’s oracle interface is the inner product: a query is a <code>Weight μ_stack</code> (specification Definition 3.13), the answer $\sum_w W(w) q(w)$; an evaluation is the query <code>eqWeight r. [decision]</code>	R18; finding 51 and section 8.7.1.5	error of the blueprint (caused by Layer 0’s evaluation interface)	MAJOR
Row <i>The oracle</i> , :317	Add: no phase before the opening queries the stack, and the opening queries it once (under the inner-product interface); the compilation depends on it (see the spine’s row on the oracle-freeness of the front). <code>[decision]</code>	R138; finding 2	error of the blueprint (an omission in the design of the seams)	MAJOR
Row <i>Errors</i> , :318	The spine fixes the closed form of every phase’s error as a function <code>piopError I</code> of the instance’s sizes, and <code>Phases.Security</code> demands each phase’s round-by-round bound at that error; <code>piop_rbrKnowledgeSoundness</code> is stated at <code>piopError I</code> and <code>piopError_le</code> bounds its sum at the caps. A phase may not declare its own error: with an unconstrained declaration the knowledge theorem holds of a protocol that checks nothing (declared error 1). <code>[decision]</code>	R5; finding 89	error of the blueprint (an omission the spine inherited)	MAJOR
Row <i>GKR</i> , :321 (the layer sumcheck)	“Layer sumcheck messages have degree 5 (eq $\times$ four multilinear)” becomes: the layer sumcheck is normalized; the round message is the cofactor of degree 4 (four coefficients on the wire, the constant one derived through the equality factor); the verifier checks $(1 + r_t) h(0) + r_t h(1)$ against the running claim; the new claim is $h(\chi_t)$; the layer’s final check has no equality factor. The table sumcheck is not normalized. (The register’s position R8 kept $5/ \mathbb{E} $ per round: a loose upper bound; $4/ \mathbb{E} $ is tight.)	R7 (R8); finding 5	error of the blueprint	MAJOR
Row <i>GKR</i> , :321 (the combiners)	“A fresh combiner λ per layer” becomes: a combiner is drawn after the roots and after every layer, the last layer included; the last is used by nothing and its error is 0 (after Fiat–Shamir it still changes every later challenge).	R9; finding 6	error of the blueprint	MAJOR
A new row <i>Leaf stacks</i> , after row <i>Stacks</i> (:322)	Per side, the boundary blocks of that side in the order of <code>I.boundary</code> (state, memory, bytecode), then each table’s flushes of that side in order; the count side one block per count column; stacked largest first at aligned offsets, ties by that index, no floor on the depth; the depth of the three trees is the push side’s. (The spine’s <code>tuples</code> lists the flushes first, which is immaterial to <code>Balanced</code> and a trap for the leaf layout.)	R50; finding 11	error of the blueprint (omission of a transcribed convention)	MINOR
Row <i>Table sumcheck</i> , :323	Prepend: the sumcheck ranges over the tables that have a constraint, a flush or a count column (for leanISA the six opcode tables, never the tables of the shared columns); $\tau_{\max}$ is the largest log-height among them and the final message has one value per column of each. The criterion is a decidable predicate of the spine (<code>SumcheckTable</code> in the table-phase dossier; <code>M3Instance.active</code> in the boundary dossier, which puts the rule in row <i>Statements and parameters</i> or a new row <i>Column groups</i>).	R11; finding 23, finding 20; finding 79	error of the blueprint (omission)	MAJOR
Row <i>Fiat–Shamir</i> , :325	Every step is BLAKE2s-256($cv \parallel block$) over 64 bytes, that is the RFC compression $F(paramIV, cv \parallel block, t = 64, f_0 = 1)$ with the chain state in the message, not the compression with cv as chaining value; leanISA’s <code>compress</code> is F . (The contract row :216 and <code>FsState.seed</code> , :1203-1204, are the dossier’s evidence for the same step.)	R66; finding 64	none given (Category B wording)	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Row <i>Verifier shape</i> , :326	Add: <code>verify</code> takes a typed <code>Program</code> and <code>PublicInput</code> , so the deployed rejection of a public word with a nonzero third limb and of a bytecode that is not $2^k \leq 2^{32}$ decodable instructions is the obligation of whoever builds those values from bytes (the fixture’s loader, T7’s <code>PublicInput.encode</code>); the differential fixture records a rejected public input as a parse failure, not as <code>verify = false</code> .	R49; finding 85	error of the blueprint (omission)	MINOR
Row <i>The wall</i> , :331	<code>LeanerVM/Protocol/Basic.lean</code> , an empty root module, imports the arithmetization: drop its import, or list it among the exceptions; the rule goes into <code>scripts/check-layers.sh</code> with an allow-list (code row).	R44; finding 111, finding 144	error of the blueprint (the witness text is false)	MINOR
Row <i>Holes</i> , :332 (the guard)	“ <code>X.Security</code> (perfect completeness, and round-by-round knowledge soundness ...)” becomes “ <code>X.Security</code> (the guard, and round-by-round knowledge soundness ...)”: a <code>Component.Guarded</code> (output purity and guard) extended by both <code>Complete</code> and <code>Security</code> , so that the knowledge theorem holds for a protocol whose completeness is unproved or imperfect. [decision]	R39; finding 39	deliberate deviation (of the spine from <code>ArkLib</code> ’s separate statements)	MINOR
Row <i>Holes</i> , :332 (the error)	“ <code>X.Def</code> (the reduction, its relations and its per-challenge error)”: the error leaves <code>Def</code> (into <code>Security</code> , or as the spine’s closed form of row <i>Errors</i>), or the reduction becomes a computable structure that <code>Def</code> extends; a <code>Def</code> carrying a real number makes every phase, <code>leanVmVerifier</code> and <code>piopExtractor</code> noncomputable. [decision]	R42 (R5); finding 122	none given (auditability)	MINOR
Row <i>Seams</i> , :333	“charged in the bus phase, coordinate by coordinate as ζ is drawn, $1/ \mathbb{E} $ per coordinate per constraint” becomes: carried by the bus phase’s state function as one more conjunct (every constraint extension, restricted to the coordinates of ζ drawn so far, is identically zero), which a coordinate makes true with probability at most $1/ \mathbb{E} $ whatever the number of constraints; <code>busError</code> has no term for it.	R47; finding 26, finding 10	error of the blueprint	MINOR
Row <i>Public input</i> , :335	Add: the choice of <code>sent</code> is pinned by two tests, Layer 3’s <code>#guard</code> on <code>leanIsaInstance.publicLines</code> and Layer 12’s differential fixture, which reads exactly two scalars.	R55; finding 101	documentation (avoidable audit surface until Layers 3 and 12 land)	MINOR
The spine (:337-615): the introduction, the relation ladder, the picture				
Introduction, :339-341	“(hole <code>S</code> ; its specification is a section of the hole comment on the dashboard <code>#12</code>)” goes: the spine is specified here.	R37; finding 138; appendix D.2	error of the blueprint	MAJOR
Introduction, :342-345	Name <code>GuardedVerdict</code> and <code>Keep0racles</code> among the spine’s generic modules under <code>ToArkLib</code> /: the pass-through and send components import them.	R91; finding 149	error of the blueprint	MINOR
The relation ladder, row <code>Ensemble.Statement</code> , :359	Over $\mathbb{K}$ the field sum accepts a tuple pushed twice and never pulled, and the side condition <code>length < ringChar K</code> admits at most one interaction per channel, so <code>Ensemble.Statement</code> holds of no ensemble with a real bus: Clean’s soundness is vacuous and its completeness false over $\mathbb{K}$ (<code>#16</code>); not “the side condition fails ... and a tuple pushed twice ... balances”.	R73; finding 125	error of the blueprint (imprecise wording)	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
The picture, :407	T4 = verify_knowledgeSound ◦ Refinement.knowledge_transport satisfiedBy_witnessOf ◦ constraintSoundness: the event-form transport, not the pointwise map_option_valid, whose polarity is the opposite of ArkLib’s bad event.	R72; finding 83	error of the blueprint (imprecision)	MINOR
The spine: what the spine fixes, the sketch, the prose (:410-586)				
Item 1, :412-414	Append: an instance is trusted data; the spine proves nothing about it beyond read_eval, and the master theorems hold of instances whose relation is empty; for leanISA the instance is Category B, its non-vacuity is m3Holds_stackOf (Layer 3), its faithfulness Layer 12’s fixture.	R102; finding 94	deliberate deviation (the wall), kept with the theorem it owes	NOTE
Item 6 :431-437; sketch :541, :545-547	piopError is the spine’s closed form of the instance’s sizes, piopError I, not the phases’ declarations (piopError (P) := P.toDef.err); Phases.Security demands each phase’s round-by-round bound at it. [decision]	R5; finding 89	error of the blueprint	MAJOR
Items 3 and 6, a new requirement	The front phases make no oracle query: a field of Phases (or a hypothesis of the compilation theorem) saying that each front phase’s verifier makes no query to TheOracle I, inhabited by the two built phases; or front phases typed as plain verifiers lifted by keepOracles, so that a query is a typing error. Without it a bundle may satisfy both master theorems and be uncompileable. [decision]	R138; finding 2	error of the blueprint (an omission in the design of the seams)	MAJOR
Sketch, Layout, :447-450	Layout with extend and one law (the extension at a cube point is in $\mathbb{K}$, or the equivalent on the eq weights); read a definition from extend.	R41; finding 95	none given (a design remark)	MINOR
Sketch, M3Instance, :455-466 (the sumcheck tables)	Add the decidable predicate of the tables the table sumcheck ranges over (a constraint, a flush or a count column), as in the row Table sumcheck above.	R11; finding 23; finding 79	error of the blueprint (omission)	MAJOR
Sketch, M3Instance, :455-466 (side conditions)	Two side conditions a bus phase of the deployed shape needs: one_le_d : $1 \leq d$ (a count column’s form has degree 1) and constrained_on_bus (a table with a constraint is on the bus, $\tau_j \leq \mu_{\text{bus}}$); either fields of M3Instance or hypotheses of the bus phase, stated in Layer 6. [decision]	R43; finding 14	error of the blueprint (the instance admits instances no deployed-shape bus phase serves)	MINOR
Sketch, the claims and Seam.bus, :477-489	BusOut takes the shape of the Rust’s BusVerify: one point, the forms per side and sumcheck table as data with the degree bound in the type, three totals, the column claims; and Seam.bus states the zerocheck at the point’s prefixes, the forms’ sums, the column claims, the public lines and aux. Smaller: keep the types and add the guards (at most $B + 3$ claims, one point per table, terms on sumcheck tables). Smallest, for the degree alone: VirtualTerm.poly a subtype within I.d, the degree conjunct deleted. [decision]	R2 (R3); finding 22, finding 92	error of the blueprint	MAJOR
Sketch, Component.Def, :501-504	err leaves Def (as in row Holes), so that the reduction, the composed verifier and the extractor compile. [decision]	R42 (R5); finding 122	none given (auditability)	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Sketch, Component.Complete, :505-507	Delete outputPure: every component over [] _o has a pure prover output (ArkLib’s Prover.instOutputIsPureEmpty); keep the hand-written guarded form, which is what an auditor reads.	R40; finding 93, finding 130	forced by an upstream library (retired when the spine imports ArkLib’s NoAmbient)	MINOR
Sketch, Component.Complete, :505-507 (an error)	If completeness is to carry an error: a field err in Component.Complete, completeness at that error, composition by ArkLib’s append_completeness_of_guarded_verifiers (errors add), piop_completeness at the sum and piop_perfectCompleteness its corollary. [decision]	R4 (R17); finding 119	error of the blueprint, conditional (disputed by R17)	MAJOR
Sketch, Component.Security, :508-515	Security extends a Component.Guarded, not Complete; Security.append gets a completeness-free path (it builds toComplete today). [decision]	R39; finding 39	deliberate deviation	MINOR
Sketch :551-553; prose :582-586	Name Refinement.knowledge_transport (the event form: accepting while the mapped extracted witness is invalid has probability at most the knowledge error; no prover conversion, the game’s witness type unchanged) as what T4 composes, and keep map_option_valid for the honest direction. The prose says the transport is proved and Layer 13 consumes it, and that what T4 waits on is ArkLib’s round-by-round-to-plain implication, whose existential conclusion forgets the named extractor.	R72, R119; finding 83; finding 96	error of the blueprint (a claim that it is not proved and has no consumer; imprecision)	MINOR
Prose, :556-560	“decision 8”, “decision 6” are defined nowhere in the blueprint: a section Decisions (row under How work is tracked) defines them.	R90; finding 148	documentation	MINOR
Prose, :560-574	“The seams carry claims, not challenges ... which is what lets the table sumcheck consume any list of linear claims without knowing the bus” becomes: the bus seam carries the point the bus phase ended at, since the table sumcheck runs at it (§5.5); α , β and the leaf layout stay inside the bus phase, which hands on the forms it built; the degree bound moves from a clause to a type. [decision]	R2; finding 22	error of the blueprint	MAJOR
Prose, :569-570 (:573-574 at 144c5aa)	“no LawfulBEq E at the pin”: re-check at the new pin (CompPoly 572f9973 has LawfulBEq (Ext P)); if found, drop the restriction to $\mathbb{K}$ -coefficient polynomials or keep it deliberately with a reason.	R84; finding 129	documentation (stale at 144c5aa)	MINOR
The spine: the holes (:588-615)				
Introduction :590-593; table :595-615	Replace by the draft: every hole named in words, in build order, one row per half; the columns “Existing work” and “Issue” leave; each layer section is its hole’s specification (not a section of the hole comment); “the generic holes carry the ledger letters” (false) and the history (“folded there on 2026-09-25”, “already in flight”, “done on the spine’s branch”) go.	R37, R85, R89; finding 138, finding 142, finding 147; appendix D.2	error of the blueprint (the delegation); documentation	MAJOR
Table, rows G4, P6, P7–P8 (:601, :609-610)	“Lemma 5.1” is the specification’s Lemma 5.2 (with Theorem 5.1); the spine’s Weight, WeightedClaim and FlockOut leave the products of the opening and Flock holes.	R92; finding 150, finding 18; appendix D.2	error of the blueprint	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Table, rows P6 and K3 (:609, :614)	Split the Flock hole like every other phase: its definition with its completeness (the schedule, the verifier, the circuit walk transcribed, the constants in Parameters, an honest run on a toy circuit) can be taken now and tested against the Python verifier’s walk; its security stays #3’s; the compiled-verifier hole (K3) says its fixture waits on it.	R60; finding 45	none given (planning)	MINOR
Table, row K3 (:614)	Its upstream work reads: #848 and #469 are single-salt and duplex-sponge Fiat-Shamir from state-restoration soundness, with an ideal per-round oracle and no proof of work; #627 is a design issue for the BCS transform whose plan excludes random-oracle soundness. (In the draft of the holes table this text moves to Layer 12’s upstream work.)	R20; finding 54	error of the blueprint	MAJOR
Table, every row (:595-615)	Twenty-one obligations of the composition have no hole: add one row each, or fold each into the nearest hole’s products: K3 gains the Merkle compilation, the chain lemma, the state-restoration step, the grinding model, the hash and program-hash terms, the family composition, the canonical-encoding checks; K1 the $\mathbb{K}$ -valuedness lemma and <code>encode over $\mathbb{E}$</code> ; a new hole “list-binding compilation”, between K1 and K3, the compilation theorem with its union-bound lemma and the oracle-freeness of the front; L1 the strided reader and the tie order; I1 the boundary-block bridge and the count columns; I2 the carrier of “R1CS $\Rightarrow$ compress”; P5 the deployed public-input phase; G1 the normalized variant and the dropped-coefficient transport; S the bound on <code>piopError</code> .	R137; finding 1 and section 6.4.2	error of the blueprint (the plan is incomplete, not wrong where it is stated)	MAJOR
Layer 0: the ArkLib dependency and the field instances (:625-650)				
:629-631 (:633-635 at 144c5aa)	“Add ArkLib at dca90385 ... the root pin 3468b38c wins” becomes “ArkLib is the ArkLib requirement; current pins in <code>upstreams.json</code> and <code>docs/dependencies.md</code> ” (or says what Layer 0 did then, and that the pins have moved).	R84; finding 129, finding 141	documentation (stale at 144c5aa)	MINOR
Sketch, the $\mathbb{K}$ sampler, :634	Its comment says it is only the building block of the $\mathbb{E}$ sampler: every leanVM challenge is in $\mathbb{E}$.	R74; finding 126	documentation (avoidable audit surface)	MINOR
Sketch, <code>evalOracle</code> and <code>evalOracle_answer</code> , :637-642	The instance becomes the inner product (query <code>Weight n</code> , answer <code>W.pair q</code>), with <code>Weight</code> and <code>Weight.pair</code> moved below <code>Field.lean</code> ; <code>evalOracle_answer</code> becomes <code>answer q (eqWeight p) = evalMle ... [decision]</code>	R18; table 8.18, finding 51	error of the blueprint	MAJOR
Sketch, :633-643 (the scalar interfaces)	Add <code>instOracleInterfaceE</code> and <code>instOracleInterfaceListE</code> (<code>instDefault</code> : a scalar or a coefficient list is read whole), with the sentence that ArkLib registers no default interface and that these go when it does.	R76; finding 128	error of the blueprint (omission)	MINOR
:645-647 (:649-650 at 144c5aa)	“At <code>CompPoly 3468b38c</code> a sampler through <code>Fintype</code> would enumerate the field (leanISA finding P3); since <code>CompPoly #331 Fintype K</code> is proof-only and the samplers go through explicit equivalences.”	R84; finding 129	documentation (stale at 144c5aa)	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Layer 1: hypercube tables, stacking, the index and bytecode columns (:652-755)				
Module list, :662	Drop BlockClaims.lean (“claims on aligned blocks”): it has no consumer (code row).	R81; finding 110	none given (avoidable audit surface)	MINOR
Sketch after :674; prose :734-746; tests :748-755	Add the low-index selection sliceLow and evalMle_boolVec_append (generic, in ToCompPoly/), Blocks.stridedLayout and Layout.piecewise (Stack.lean), and the prose: leanVM reads the eighteen BLAKE2S limb columns as strided slots of q_flock , the low eight coordinates frozen to the slot’s bits; Layer 3 transports the sum type to ColumnId. Tests against the Python’s Placement.stack_point.	R35; finding 104	error of the blueprint (omission)	MAJOR
Sketch, :704-731	Add evalMle_padHigh and the definitions of the two evaluators idxColumnEval and bytecodeColumnEval, which the verifier runs and listed statements mention.	R91; finding 109	error of the blueprint	MINOR
Layer 2: Clean components as polynomials (:757-787)				
Sketch, :762-771	A computable Expression.toCMvPolynomial (var i with $i < n$ to X_i , else 0, as Environment.fromArray reads a missing cell), with eval_toCMvPolynomial and a Mathlib-side bridge through which degreeBound bounds totalDegree; M3Table’s fields become CMvPolynomial; Direction becomes Side; Clean #466 stays the proof-side bridge.	R32; finding 78	forced by an upstream library (CompPoly’s conversion is noncomputable at both pins; retired if it becomes computable)	MAJOR
Sketch, Component.toM3, :773-774	Component.toM3 takes the lookup channels, and count is coordinate 2 of every pull on them, required to be a variable (count_is_var); a test that the six tables’ count columns are those of count_columns().	R70; finding 81	error of the blueprint (imprecision)	MINOR
Tests, :784-787	Add a test of the three block components (memory, bytecode, verifier) against their boundary blocks on the one-row witness.	R31; finding 72	error of the blueprint (omission)	MAJOR
Layer 3: the M3 instance of leanISA (:789-867)				
Sketch, :795-808	Sizes of the heights only (no witness determines the rate); Sizes.ofWitness takes prog; leanIsaBlocks, leanIsaμ and leanIsaBlocks_fits (the fit is a Blocks’ and is needed before the instance; Layout has no total); Sizes.Admissible with the caps and the stacking window $\mu_{stack} \leq 28$, the rate window $1 \leq \rho \leq 4$ inside it or as validRate (Layer 12); caps_of_admissible and sizes_of_satisfiedBy in place of admissible_iff_caps; tests reject $\mu_{stack} = 29$ and logInvRate = 5.	R33; finding 8, finding 105; finding 75	error of the blueprint (three statements cannot hold together)	MAJOR
Sketch, leanIsaInstance, :803-805	Component.toM3 of the six opcode tables; three column groups for the shared committed columns (tables with no constraint, flush or count); six boundary blocks written from memTable, bytecodeTable and leanIsaVerifier prog, with the index and program columns Coord.known and the final pc Coord.const; the deployed layout; three public lines; aux the Flock predicate of the region. Not “Ensemble.toM3 of the eight tables”.	R30; finding 71	error of the blueprint	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Sketch, :803	abbrev <code>leanIsaInstance ...</code> , reducible, so that <code>Decidable (M3Holds ...)</code> and the instances on <code>I.Stmt</code> are found by instance search, as for the toy.	R71; finding 82	error of the blueprint (omission of a constraint on the code)	MINOR
Sketch, the adaptor, :810-819 (the boundary blocks)	Add <code>boundary_tuples_eq</code> : under the three fixed-column conjuncts of <code>SatisfiedBy</code> , the multiset of the messages of the memory block, the bytecode block and the verifier row on each side equals the instance’s <code>boundaryTuples</code> of <code>stack0f</code> ; both adaptor theorems use it.	R31; finding 72	error of the blueprint (omission)	MAJOR
Sketch :811-819; prose :841-861 (the slot map, the lemma, the generator)	One instance-free structure, supplied by #3, carries the slot map (18 numbers, <code>hash_flock.rs:87-115</code>), the predicate on the packed column, the lemma “ <code>R1CS $\Rightarrow$ the limbs compress</code> ”, the generator and its lemma (<code>FlockSpec</code> ; or <code>blake2sLimbSlot</code> in <code>Parameters/</code> with the reader built from Layer 1’s lemma); the layout takes the slot map from it, not from <code>FlockInterface.limbColumns</code> ; <code>FlockWitnessGen</code> goes. [decision]	R15; finding 36; finding 77	error of the blueprint (cannot be stated as written)	MAJOR
Sketch, <code>witness0f</code> , :813	<code>witness0f</code> takes the public input, or the text says the lanes are read off cells 0 and 1 of <code>mem_0</code> , <code>mem_1</code> .	R70; finding 81	error of the blueprint (imprecision)	MINOR
Sketch :819; prose :822-824; tests :866-867	<code>witness0f_stack0f</code> is not an equality any <code>EnsembleWitness</code> satisfies: state the three equalities on the columns under <code>SatisfiedBy</code> and <code>Sizes.ofWitness</code> , test them with a <code>#guard</code> on the one-row witness, and say that neither T4 composition uses it.	R69; finding 80	error of the blueprint	MINOR
Prose, the auxiliary predicate, :850-851	“ <code>aux q</code> says that, for every compression, the bits packed into the <code>q_flock</code> region satisfy <code>Flock</code> ’s <code>R1CS</code> and hold 1 at the constant position 512”; without it the zero region satisfies <code>aux</code> and “ <code>aux $\Rightarrow$ the limbs compress</code> ” is false.	R56; finding 40	error of the blueprint (of wording)	MINOR
Prose, :850-861 (what Layer 3 supplies to Layer 9)	Layer 3 supplies a <code>FlockRegion I</code> : the column <code>q_flock</code> , <code>kBatch</code> , its height <code>8 + k_{batch}</code> , and <code>aux</code> equivalent to <code>Flock.Holds kBatch</code> of that column; the toy keeps <code>aux := True</code> and has none.	R16; finding 37	error of the blueprint (a design gap the spine left open)	MAJOR
Tests, :865-867 (the tie order)	Add the offsets of every committed column on the two configurations of the Layer 1 review’s <code>OffsetsProbe</code> , where a table’s height equals κ_{mem} , so that a tables-first order fails them.	R78; finding 106	error of the blueprint (a check deferred with no witness named)	MINOR
Layer 3, optional	Optionally, <code>Function.Injective</code> of the renaming for the instance’s layout: <code>Layout.comap</code> accepts a non-injective one, which aliases columns (the adaptor’s completeness catches it).	R131; finding 113	none given	NOTE
Layer 4: sumcheck for eq-weighted virtual polynomials (:869-912)				
Sketch, :878-907	Restate <code>sumcheck</code> and <code>batchClaims</code> as <code>Component.Defs</code> with their <code>Complete</code> and <code>Security</code> , their theorems in the <code>rbrKnowledgeSoundnessWorstCaseWith</code> form of row <i>Holes</i> (<code>Sumcheck.Def</code> and <code>Batch.Def</code> exist nowhere).	R36; finding 139	error of the blueprint	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
The eq-weighted variant, :898-901	Define the two sumcheck variants by name: the plain one (error $d/ F $, d the degree of the polynomial sent, the verifier evaluating the equality factors at the end: the table sumcheck) and the normalized one (error $d_c/ F $, d_c the cofactor’s degree: the GKR layers).	R7; finding 5	error of the blueprint	MAJOR
Layer 5: fingerprints, the grand product, and GKR (:914-947)				
Sketch, gkr, :930-937	gkr over a context: leaves (the leaf tables as functions of the statement and the oracles) and riders (tables that must vanish), a <code>Component.Def</code> whose output relation says the leaf claims hold at ζ and every rider’s extension vanishes at $\zeta_{<\tau}$, with errors $2/ \mathbb{E} $ per combiner (0 for the last), $4/ \mathbb{E} $ per round, $1/ \mathbb{E} $ per combination challenge. [decision]	R10 (R36); finding 7	error of the blueprint; forced by an upstream library (the first half)	MAJOR
gkrError, :940-942 (the rounds)	$4/ \mathbb{E} $ to each sumcheck round, not $5/ \mathbb{E} $ (the tracker’s hole for the generic GKR likewise).	R7 (R8); finding 5	error of the blueprint	MAJOR
gkrError, :941-942 (the binary layer)	$1/ \mathbb{E} $ to each combination challenge: two per radix-4 layer, one for the binary layer (odd μ).	R52; finding 16	error of the blueprint (omission)	MINOR
Layer 6: the bus phase (:949-979)				
Sketch :954-971; prose :975-977	Replace the code block by the slot’s types (<code>busPhase I : Phase.Def I I.Stmt (I.Stmt $\times$ BusOut I)</code> , <code>busComplete</code> , <code>busSecurity</code>) and the schedule <code>V_to_P (α, β); <code>P_to_V (R, R_c)</code>; the GKR’s; <code>P_to_V</code> the boundary evaluations. The invariant after (α, β) is “the two products at (α, β) differ”, not “the multisets differ”, and no boundary claim exists before the last message.</code>	R36; finding 139, finding 15	error of the blueprint	MAJOR
The bus phase’s composition (new sentence)	The bus phase is $[(\alpha, \beta); \text{roots}] \gg \text{gkr} \gg [\text{boundary evaluations}]$, with leaves the three leaf tables of the stack and riders the constraint tables. [decision]	R10; finding 7	error of the blueprint	MAJOR
busError, :973-975	<code>busError</code> is $4 \cdot 2^{\mu_{\text{bus}}}/ \mathbb{E} $ on (α, β) and <code>gkrError</code> on the GKR’s challenges; the zerocheck escape adds nothing (a conjunction escapes with the larger of the two probabilities).	R47; finding 10	error of the blueprint	MINOR
Four orders (new clauses)	The roots message is (R, R_c) , the bus root first; the boundary evaluations in the order the sides, blocks and coordinates first name a committed column, repeats skipped (<code>mem_0</code> , <code>mem_1</code> , <code>mem_2</code> , <code>cntfin_mem</code> , <code>cntfin_bc</code>); <code>BusOut</code> ’s linear claims table by table, then push, pull, count; $\zeta = (u_0, u_1, \chi_0, \dots)$ of the last layer.	R51; finding 12	error of the blueprint (omission of transcribed conventions)	MINOR
Tests, :978-979	Add a guard on sizes with $\tau_j = \kappa_{\text{mem}}$, where the order of the leaf stacks decides the offsets.	R50; finding 11	error of the blueprint	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Layer 7: the table sumcheck phase (:981-1004)				
Sketch, :986-995	Rewrite on the spine’s names: <code>tableSumcheck I</code> : <code>Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I)</code> , <code>tableSumcheckComplete</code> , <code>tableSumcheckSecurity</code> , <code>tableSummand I (s : BusOut I) (ξ : E)</code> ; delete the placeholder <code>tableSumcheck_relOut_implies_constraints</code> (what it meant is the bus phase’s zerocheck conjunct).	R36; finding 27, finding 139	error of the blueprint	MAJOR
Sketch, :987-989	$\tau_{\max}$ and $\sum_j width_j$ range over the sumcheck tables, not every table of the instance (at least 16 rounds and 110 values against leanVM’s $\tau_{\max}$ rounds and 104 values).	R11; finding 23; finding 79	error of the blueprint (omission)	MAJOR
Prose, :1000-1003	“it is charged to the (α, β) and GKR challenges of Layer 6 ... as an extra $\tau_{\max}/ E $ per constraint” becomes “it is the bus phase’s (Layer 6), and costs $\tau_{\max}/ E $ at most, once”.	R47; finding 26	error of the blueprint	MINOR
Layer 8: the public-input phase (:1006-1068)				
Prose, :1035-1037	For a line whose value was sent the pooled value is the value sent, as §8.2 and the pinned verifiers pool it; the check is what makes the pooled claims the statement’s: a verifier that pools the values it computes is knowledge sound with no check at all (the merged code does this; code row).	R13; finding 98	error of the design (introduced by an audit-surface compression)	MAJOR
Prose, :1053-1062	Layer 8 has a second phase for the deployed check: the equation on the two words, the values sent pooled, <code>Phase.Security at 1/ E </code> (key lemma <code>accepts_two_challenges</code> proved); Phases and Layer 12 take it; the specification’s per-limb phase is kept as the reference of §8.2, with the lemma that its acceptance implies the deployed one’s. “Layer 12 owes it” goes. [decision]	R12; finding 24, finding 99	deliberate deviation (decision 15) owing its theorem in the wrong place; a disagreement of the specification with its three implementations	MAJOR
Layer 9: the Flock and ring-switching boundary (:1070-1099)				
Sketch and prose, :1075-1092	Replace <code>FlockInterface</code> (the eighteen limb claims in, one weighted claim out: knowledge error 1 against leanVM’s verifier) by a <code>FlockPhase I R</code> at the spine’s seams, <code>Seam.pub</code> to <code>Seam.flock</code> , whose verifier reads no claim, hands the column claims on (the limb claims are opened as strided claims on <code>q_flock</code>) and adds the ring-switched weighted claim; write the schedule out ($k_{\text{batch}} + 1$ challenges; 64 values; one challenge; $8 + k_{\text{batch}}$ rounds of two values and a challenge; two values; one challenge; 8 rounds of two values and a challenge; 64 values; six challenges).	R14 (R36); finding 35, finding 28	error of the blueprint (the spine’s Flock slot is faithful; Layer 9 is not)	MAJOR
:1072, :1077 (over an abstract instance)	The Flock phase takes the <code>FlockRegion I</code> of Layer 3; <code>Flock.Holds</code> is the Flock roadmap’s predicate on a packed column and mentions no instance; the phase imports nothing from the arithmetization (the wall stands).	R16; finding 37	error of the blueprint (a design gap the spine left open)	MAJOR
:1080, :1091-1092 (limbColumns)	Delete <code>limbColumns</code> : the slot map is instance-free (Layer 3’s row).	R15; finding 36; finding 77	error of the blueprint	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
:1083 (completeness)	Either the Flock phase has a completeness error of its own, summed (R4), or it is perfectly complete for the specification’s honest prover, which sends the true coefficients (R17). [decision]	R4, R17; finding 119, finding 38	error of the blueprint (disputed between the two rows)	MAJOR
:1084 (knowledge soundness)	rbrKnowledgeSoundnessWorstCase becomes the With form, with a named extractor and knowledge state function (the existential form proves soundness only).	R1; finding 118	forced by an upstream library	CRITICAL (as a warning)
:1085 (the error)	Give flockError per challenge as a closed form next to the theorem (row Errors); its sum stays below $(4k_{\text{batch}} + 163)/ \mathbb{E} + 2^{32}/ \mathbb{E} $.	R58; finding 42	error of the blueprint	MINOR
:1088-1089 (the auxiliary predicate)	The strong auxiliary predicate includes the constant position 512, as in Layer 3 (the proposal applies wherever the phrase occurs).	R56; finding 40	error of the blueprint (of wording)	MINOR
Layer 10: the claim pool, the opening sumcheck, and the oracle protocol (:1101-1148)				
Title :1101; sketch :1109-1114	The opening phase is the batching challenge λ followed by one weighted query of the stack at W_λ , compared with C_λ : Phase.Def I (I.Stmt $\times$ FlockOut I) Unit, schedule V_to_P : E, error $(J - 1)/ \mathbb{E} $ on λ ; no sumcheck (§8.5: “there is no separate reduction sumcheck”); “ $2/ \mathbb{E} $ per round” leaves; the title loses “the opening sumcheck”. [decision]	R18; finding 51 and section 8.7.1.5	error of the blueprint (caused by Layer 0’s evaluation interface)	MAJOR
Sketch, :1109-1131	Delete Layer 10’s own Weight, WeightedClaim, Claim.toWeighted and its leanVmPiop block with master theorems over Set.univ and rbrKnowledgeSoundness: the spine’s are authoritative (this also removes an existential form, which R1 reads as a soundness statement only).	R36; finding 139	error of the blueprint	MAJOR
Sketch, :1122, :1130	piopError and piopError_le restated over I (the spine’s closed form), not over Sizes. [decision]	R5; finding 89	error of the blueprint	MAJOR
Sketch :1130; tests :1148	piopError_le’s comment states $\mu_{\text{bus}} \leq 28$, a lemma of Layer 3 from the window of Admissible: at the per-log caps alone the bound is false; with the window the phase errors sum to about 2^{-160} . “piopError_le by norm_num at the caps” becomes “at admissible sizes”.	R33; finding 8; finding 65	error of the blueprint	MAJOR
Prose, :1133-1135	“the commit phase’s, which reads the stack off that message” is true of its intermediate extractor at round 0; the extracted stack is piopExtractedStack.	R46; finding 91	error of the blueprint	MINOR
Prose, :1136-1144	The port is kept until an ArkLib framework proves a guarded-first append, not “deleted when the pin moves past #615”; “The port is done and closes hole C1” (history) goes. [decision]	R34, R89; finding 120, finding 147	error of the blueprint’s plan; documentation	MAJOR
Layer 11: WHIR over binary Reed–Solomon codes, and Merkle trees (:1150-1193)				
Sketch, ladder, :1164	structure Level with fold, rateExp, queries, oodSamples, grindingBits and the analysis parameter $\eta : \mathbb{Q}$; whirError a function of η ; the deployed η_i per (μ, ρ) recorded as Category B.	R65; finding 62	none given	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Sketch and prose, :1169, :1173, :1185	whirOpen’s level-0 batching challenge is the opening phase’s λ (the compilation replaces the whole opening phase by whirOpen), not a second batching; encode maps $\mathbb{E}$ -valued messages of every level, with the lemma that a $\mathbb{K}$ -valued message gives a $\mathbb{K}$ -valued word.	R19 (R18); findings 51 and 58	error of the blueprint	MAJOR
Sketch, :1170	def columnWeight (κ R) (x) : Weight κ with its closed-form evaluator and encode_columnWeight (x) (f) : encode κ R f x = (columnWeight κ R x).pair f (Lemma lem:colweight), in place of the existential encode_column_weight, which every linear map satisfies.	R62; finding 59	error of the blueprint (non-vacuity)	MINOR
Annex B numbers, :1169-1188	Cite Annex B by label: “Annex B.4” (:1172) is §B.3, “Lemma B.7” (:1170) is Lemma B.14, “Theorem B.2” (:1175) is Theorem B.7 (thm:rbr), “Protocol B.1” (:1185) is Protocol B.6, “Definition B.1” (:1188) is Definition B.4.	R92; finding 63	error of the blueprint	MINOR
McaJohnson, :1176-1177	Say that [BCHKS25] Theorem 4.6 is a preprint theorem with a sketched proof, and that McaJohnson is stated with the printed parameter (as ArkLib’s admission is).	R114 (R115); literature.md §C.4	documentation (a fact about the assumption)	NOTE
Merkle, :1179-1182	Name VCVio’s Merkle-tree library and VCVio issue 571 as the upstream, and decide whether Layer 11 builds on it. [decision]	R77; finding 151	documentation	MINOR
Prose :1185-1190, or the conventions	Record four transcript facts of WHIR (Category B): the level-0 lane relayout $f(u, x) = q(x, u)$ and the rotation of the terminal point; the out-of-domain claim’s intro round polynomial, sent before each level’s λ_i , and the query batch’s, sent after it (as the citation check corrected); the last round message omitted; the grinding nonce read before the positions at every level, 17 bits.	R64; finding 61	error of the blueprint (omission)	MINOR
Layer 12: compilation, the transcript, the proof, and the executable verifier (:1195-1240)				
Sketch, Proof, :1209-1211	Proof.merkle is the pruned wire form (rows at the distinct sorted positions, one “octopus” per phase) that the Rust prover emits; merkleVerify checks a phase, not a path; at level 0 the row holds the committed lanes and the verifier supplies the zero prefix.	R63; finding 60	error of the blueprint (omission)	MINOR
Sketch, :1214-1215	“WHIR in place of the evaluation oracle” becomes “WHIR in place of the opening phase, over the level-0 word as the commit message”. [decision]	R18; finding 51 and section 8.7.1.5	error of the blueprint	MAJOR
Sketch :1218-1221; prose :1230-1232	verify_iff_compiled is stated against a leanerVM bcsCompile (roots for oracle messages, authenticated rows for queries), the chain as a QueryImpl, and the grinding checks conjoined, not ArkLib’s Verifier.fiatShamir of the oracle verifier (which would hash whole oracle messages); “unconditional, about two definitions, and a check only because verify is written from §8.5 before the right-hand side is unfolded”.	R21; finding 121; finding 52	error of the blueprint; forced by an upstream library (no BCS transform in ArkLib)	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Sketch, the assumed interfaces, :1223-1227	Restated so that they are true and can be inhabited: <code>RbrToStateRestoration</code> , <code>BcsSecurity</code> (up to $3(Q^2 + 1)/2^{256}$, the straight-line extractor reading leaves off the query log), <code>ChainFiatShamirSecurity</code> (leanVM’s hash chain, with proof-of-work rounds), their witness obligation the literature; <code>verify_knowledgeSound</code> takes them with <code>McaJohnson</code> , the list-binding compilation and the Flock phase. An interface over all IOPPs with error $Q \cdot \max_i \varepsilon_i$ is false (hash term, oracle messages, grinding).	R20, R22; finding 54; finding 121; <code>literature.md</code> §E.6	error of the blueprint; forced by an upstream library	MAJOR
Sketch :1224; prose :1234-1235 (the error)	<code>niError Q</code> is $(Q + \text{rounds})$ times the largest per-challenge error of the compiled protocol, namely $L_0 \cdot \varepsilon_i$ at each oracle-protocol challenge, Theorem B.7’s terms at WHIR’s, $2^{-17}(1 - \gamma_i)^{t_i}$ at each ground query round, plus the hash term $cQ^2/2^{256}$; <code>FiatShamirSecurity</code> carries the grinding bits, the proof-of-work factor marked an assumption no published theorem covers.	R23; <code>literature.md</code> §A.4 (first three findings); finding 57	error of the blueprint	MAJOR
Sketch :1224; prose :1234-1235 (the data)	The compiled reduction carries a second function <code>grind : pSpec.ChallengeIdx</code> $\rightarrow \mathbb{N}$ (17 at the query rounds, 0 elsewhere), <code>FiatShamirSecurity</code> takes it, and <code>niError</code> is defined from <code>err</code> and <code>grind</code> .	R139; finding 3	error of the blueprint (an omission in the data of Layer 12)	MINOR
Sketch, :1226-1227	<code>niError s Q</code> becomes the family’s <code>niError Q</code> (the maximum over admissible sizes, or the sum).	R6; finding 76	error of the blueprint (a missing decision)	MAJOR
A new theorem, replacing Layer 11’s sentence :1190	State the list-binding compilation: an extractor (list-decode, select), its state function, its errors ($L_0 \cdot \varepsilon_i$ at the front’s challenges, Theorem B.7’s at WHIR’s) and two lemmas (every member of the list is k -valued and of the right size; the front’s verifier makes no oracle query); carry L_0 into <code>niError</code> .	R24 (R23, R138); finding 55; <code>literature.md</code> §A.4 (second finding)	error of the blueprint (omission)	MAJOR
The compilation theorem (a hypothesis)	If the oracle-freeness of the front is not a field of Phases, it is a hypothesis of the compilation theorem, stated as a property of each front phase’s <code>OracleVerifier</code> . [decision]	R138; finding 2	error of the blueprint	MAJOR
Tests, :1237-1240 (the proof of work)	The mutations also cover a wrong grinding nonce at each ground level, a truncated Merkle path, a nonzero third limb in a digest half and an announced size above its cap; and, in reverse, each transcript the specification rejects is run through the Rust and Python verifiers.	R25; <code>literature.md</code> §G.5 (first finding)	error of the blueprint	MAJOR
Tests, :1237-1240 (canonical encodings)	List the three canonical-encoding and consumption checks among <code>verify</code> ’s (zero upper limbs of the announced sizes, a zero third limb in each half of a root, a fully consumed stream) with a mutation each; add a trailing scalar, a surplus sibling, a level-0 row of the wrong width, a duplicated position.	R61; finding 13; finding 67	error of the blueprint (omission)	MINOR
Layer 13: T4 and the fixtures (:1242-1259)				
Sketch, :1247-1250 (the program)	Both base theorems take <code>hwf : WellFormedBytecode prog</code> , as <code>constraintSoundness</code> and <code>constraintCompleteness do</code> (a bytecode with a JUMP in the sentinel slot has balancing steps and no valid execution; the repository proves it), or <code>verify</code> decides the program condition and rejects. [decision]	R27; finding 137; finding 73	error of the blueprint (against the leanISA roadmap)	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Sketch, :1247-1248 (soundness)	A random-oracle statement: a Q -query adversary against the compiled verifier with the compression as the random oracle, a straight-line extractor (leaves off the query log, list decoding, selection), the event “accepts, and witness0f of the extracted stack fails SatisfiedBy” bounded by $\text{niError } Q$, the existential corollary, and the pointwise <code>execution_of_extracted</code> that recursion (T6) needs; the instantiation of the oracle by BLAKE2s named as the one non-formal step (and in <code>architecture.md</code>). The citation check adds that the Merkle leaves are iterated BLAKE2s, which a random oracle for the one 64-byte compression does not model (recorded in the box).	R28, R121; findings 73 and 86; finding 53	error of the blueprint (and of <code>architecture.md</code> ’s T4)	MAJOR
Sketch :1249-1250; prose :1255-1257 (resources)	<code>baseProver_complete</code> from a satisfying witness with admissible sizes and a valid rate (with the Flock generator), and <code>baseProver_complete_of_execution</code> with a fit hypothesis <code>hfit</code> ; the prose says <code>hfit</code> is the resource condition <code>architecture.md</code> asks for (a valid execution may need $\kappa = 32$, beyond the stack the verifier accepts). [decision]	R29; finding 74	error of the blueprint	MAJOR
Sketch :1249-1250; prose :1255-1257 (grinding)	<code>prove</code> takes fuel and returns <code>Option Proof</code> , the grinding being a search; <code>baseProver_complete</code> is <code>prove ... = some proof → verify ... proof = true</code> ; “perfect completeness survives the transform without any assumption” goes.	R26; finding 56	error of the blueprint	MAJOR
Prose, :1255	One sentence: <code>baseProver_complete</code> is existential in the witness (it composes <code>leanISA</code> ’s <code>constraintCompleteness</code>); the computable form through the witness generator (T2) is <code>architecture.md</code> ’s and out of scope here.	R120; finding 155	deliberate deviation (a scope choice not written down)	NOTE
Acceptance tests and nearby false statements (:1261-1358), in order				
Test 5, :1280	“distinguished by <code>leanIsaTables</code> flush polynomials” becomes “distinguished by the flush polynomials of the <code>leanISA</code> instance” (<code>leanIsaTables</code> exists nowhere).	R92; finding 150; finding 84	error of the blueprint	MINOR
Test 6, :1283	“The state function of Layer 7 charges the zerocheck to Layer 6’s challenges” becomes “The state function of Layer 6 carries the zerocheck”.	R47; finding 26	error of the blueprint	MINOR
Test 7, :1284-1286	<code>sumCube_padHigh</code> , not <code>sumCube_prodVars</code> ; copying multiplies by $2^{\tau_{\max} - \tau_j}$, which is 0 in characteristic two; witnesses: the test of <code>Padding.lean</code> and, once Layer 7 lands, <code>tableSummand_target</code> .	R82; finding 112	error of the blueprint	MINOR
Test 10, :1293-1295	Say that the phase and the spine cannot notice a missing third line (an instance with two lines has all their theorems); what fails is <code>satisfiedBy_witness0f</code> at <code>SatisfiedBy.word0_eq</code> , whose right side has a zero top limb by type. Witnesses: <code>threeLimbs</code> and, when Layer 3 lands, a <code>#guard</code> that the <code>leanISA</code> instance’s public lines list <code>mem_2</code> with cells $(0, 0)$ and <code>sent = false</code> .	R54; finding 100	documentation (an incomplete audit trail, not a divergence)	MINOR
Test 14, :1306-1307	The witness is <code>bytecodeColumn_answer_boolVec</code> with the oracle <code>#guards</code> , and a <code>#guard</code> of <code>bytecodeColumn</code> on a sixteen-instruction program with distinct operands against <code>leaf.rs:585-604</code> ; not <code>bytecodeColumn_slot</code> , which restates the definition.	R79; finding 107, finding 150	error of the blueprint (stale witness)	MINOR
Test 15, :1308-1310	Largest first is sufficient, not necessary, and says nothing of equal sizes; “fails for any other placement” goes; the witness’s 4, 2, 1 are heights.	R80; finding 108	error of the blueprint	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Test 17, :1314-1316	Add the message count of the bus phase (a combiner after every layer, the last included).	R9; finding 6	error of the blueprint	MAJOR
Test 20, :1323-1325	No exceptional-challenge clause in any phase; no verifier inverts a challenge-dependent value; the pinned Rust Flock prover's $(1+r)^{-1}$ at $r=1$ is a completeness defect of the implementation, recorded in docs/leanvm-target.md and no part of flockError; the Flock phase's test runs the round at $r=1$ (R17). Or: the Flock phase's completeness error is its own field and is summed (R4). [decision]	R17, R4 (R111); finding 38, finding 119	error of the blueprint	MAJOR
Test 21, :1326-1328	Add: the error of verify does not depend on the prover's choice of sizes.	R6; finding 76	error of the blueprint	MAJOR
Test 23, :1331-1332 (ring switching)	“plus Flock's $< 2^{-183}$ ” becomes “plus the Flock phase's, below $2^{-183} + 2^{-160}$, the second term being ring switching's”, which dominates.	R58; finding 42; finding 65	error of the blueprint	MINOR
Test 23, :1331-1332 (what the number is)	$\sum \text{piopError} < 2^{-150}$ is the interactive error at admissible sizes (below 2^{-159} with the window, false at the per-log caps alone); it is not the security level of verify, which the WHIR query rounds set at 128 bits including 17 bits of grinding.	R23, R33; findings 57 and 65	error of the blueprint	MAJOR
Test 24, :1333-1336 (the existential forms)	Add: ArkLib's extractor type accepts a classical choice, so the existential forms prove soundness only; every interface is in the With form with a compiled extractor.	R1; finding 118	forced by an upstream library	CRITICAL (as a warning)
Test 24, :1337-1339 (the witness)	The witness is piopExtractedStack_eq, for every bundle and every transcript; the composed extractOut is the last phase's and is not the stack; the rfl test covers zero-round phases only.	R46; finding 91	error of the blueprint	MINOR
Test 24, :1333-1340 (efficiency)	“computable”, and for the compiled extractor an efficient list decoder named as the intended implementation; knowledge soundness of verify is claimed only in the information-theoretic sense (Lean does not measure running time).	R48; literature.md §A.4 (fourth finding)	none given	MINOR
Test 24, :1339-1340 (Layer 3's guard)	“#guard witnessOf (stackOf w) = w” becomes a #guard of the three equalities on the columns.	R69; finding 80	error of the blueprint	MINOR
Test 25, :1341-1345	The grep also returns Protocol/Basic.lean: drop its import or list it; “a rule in scripts/check-layers.sh is still to be written” (history) goes once the rule is written.	R44, R89; finding 144, finding 111, finding 147	error of the blueprint (the witness text is false)	MINOR
Test 28, :1356-1358 (a side condition)	Name the side condition one_le_d.	R43; finding 14	error of the blueprint	MINOR
Test 28, :1356-1358 (the degree in the type)	Met by the type once the degree bound moves into the statement (the row polynomials of the proposed BusOut, or VirtualTerm.poly a subtype): the test cubicTerm becomes a typing failure. [decision]	R2; finding 92, finding 22	error of the blueprint	MAJOR
A new test (the constant position)	“The constant position is part of the predicate. The zero block satisfies the R1CS and is no compression. Witness: aux fails on the zero region.”	R56; finding 40	error of the blueprint	MINOR
A new test (the sumcheck's tables)	On an instance with a taller table of no constraint and no flush, the number of rounds of the table sumcheck is the smaller height.	R11; finding 23	error of the blueprint (omission)	MAJOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
Interfaces supplied to later work (:1360-1429)				
The list, :1362-1419	The list is the public boundary: add the unlisted load-bearing names (PublicInput.check, pooled, prover, verifier, pSpec, verifier_verify, publicInputComplete, publicInputSecurity; the generic declarations of GuardedVerdict and KeepOracles; idxColumnEval, bytecodeColumnEval, evalMle_padHigh, Blocks.total, Blocks.offset, Blocks.lowPoint, Blocks.selectorWeight; sliceLow and evalMle_boolVec_append if the strided reader is taken); Column moves to Layer 0's names, Weight and WeightedClaim out of "generic"; Ensemble.toM3 out of "Spine". 198 of 333 non-private declarations are unlisted today, against "Everything not listed is a proof, a helper, or a test" (:1419).	R91; finding 149, finding 109	error of the blueprint	MINOR
The list, :1369, :1398	Delete Ensemble.toM3 (or define it as the six-table part only).	R30; finding 71	error of the blueprint	MAJOR
The list, :1400	Add instSampleableTypeK, instOracleInterfaceE, instOracleInterfaceListE.	R76, R74; finding 128, finding 126	error of the blueprint (omission)	MINOR
The list, :1404	Drop BlockClaim.	R81; finding 110	none given (avoidable audit surface)	MINOR
The assumed interfaces, :1419-1423	FlockWitnessGen leaves the assumed interfaces: the instance-free Flock structure of Layer 3 carries the generator and its lemma.	R15; finding 77	error of the blueprint	MAJOR
The composition, :1427-1429	The port KnowledgeAppend.lean is deleted when an ArkLib framework proves the append, not "when the pin moves past #615". [decision]	R34; finding 120	error of the blueprint's plan	MAJOR
Boundaries (:1431-1453)				
With leanISA, :1433-1441	A further request: a constraintCompleteness that returns a witness of minimal heights (or T4 takes hfit as its hypothesis), and a resource hypothesis for constraintCompleteness itself (Caps bounds every table by 2^{32} rows, ValidExecution bounds only κ). [decision]	R29, R122; findings 74 and 88	error of the blueprint (R29); none given (R122, leanISA's)	MAJOR
With leanISA, :1439-1441 (the limbs)	"reconciled by FlockInterface.limbColumns" becomes "reconciled by the instance's layout".	R15; finding 36	error of the blueprint	MAJOR
With leanISA, :1440 (the reference)	"(finding F3 of the status file)" becomes "(the leanISA status's finding on the BLAKE2S value limbs)": the protocol status's F3 is another finding.	R85; finding 142	documentation	MINOR
Ordering and parallelism (:1455-1470)				
:1457-1462 (the new hole)	Place the new hole "list-binding compilation" between K1 and K3, and the obligations folded into existing holes with them (the holes-table row of R137).	R137; finding 1	error of the blueprint (the plan is incomplete)	MAJOR
:1459-1460	Delete "C1, the port of one ArkLib file, is done on the spine's branch" (history).	R89; finding 147	documentation	MINOR

Continued on the next page.

Table A.1, continued.

Passage	Change	Register row; full text	Classification	Severity
How work is tracked (:1472-1515), and two new sections				
The section, :1472-1515	Replace by the draft: a table of homes, one per kind of fact; the tracker changed by editing its body; each hole specified in its layer section, not in the hole comment (:1485-1487); claims by editing the checklist line or by an intention issue; discrepancies in the sources recorded under a name in <code>leanvm-target.md</code> (:1500-1501); the edits a review asks of the tracker made in the pull request that meets it.	R88, R37, R93, R135; finding 146, finding 138, finding 152, finding 154; appendix D.1	documentation; error of the blueprint (R37)	MAJOR
:1511-1513	Describe the review by what it does (three passes: the statements against the specification, fidelity to the pinned sources, hygiene), not as the <code>leanerVM-review</code> skill (none exists; it is <code>adversarial-review</code> , under the <code>git-ignored .claude/</code>); or track the skills.	R87; finding 145	documentation	MINOR
New sections <i>Decisions</i> and the index	A section <i>Decisions</i> , one row per decision (number, name, the choice in one line, the convention or test that records it), and the index of names that replaces the codes.	R90, R85; finding 148, finding 142; section 13.5.2.3	documentation	MINOR
References (:1517-1549)				
:1530-1537	The Fiat–Shamir paper is Canetti, Chen, Holmgren, Lombardi, Rothblum, Rothblum, Wichs (STOC 2019); Ligerito is Novakovic and Angeris; BCHKS25’s title is “On Proximity Gaps for Reed–Solomon Codes”; round-by-round knowledge soundness is [CMS19] Definition 8.5 (BibTeX in the literature dossier).	R98; <code>literature.md</code> §A.4 (fifth finding)	error of the blueprint	MINOR
:1538	Thaler (2022): Proposition 4.1 (sum-check), §4.6 (GKR), §6.6.2 (multiset equality by fingerprinting), not “chapter 4” for both; [Tha13] and [STW24] for GKR-based grand products.	R99; <code>literature.md</code> §D.3	error of the blueprint	MINOR

A.2 Changes to the code

The changes the review proposes to the Lean sources and tests already on `main`, with the file and lines at `b435631` (paths under `LeanerVM/Protocol/` unless another root is named). Code for layers that are not built (the adaptor, the phases of Layers 6, 7, 9 and 10, the compilation) is specified by the blueprint and appears in table A.1 only. Columns and classifications as in table A.1.

Table A.2: Proposed changes to the code (files and lines at b435631).

File and lines	Change	Register row; full text	Classification	Severity
The spine and its generic half				
Spine/Compose.lean:149; ToArkLib/Component.lean:66	The error becomes the spine’s closed form <code>piopError I</code> of the instance’s sizes: <code>err</code> leaves <code>Component.Def</code> , <code>Security</code> carries it with its <code>rbr</code> , and <code>Phases.Security</code> demands each phase’s <code>rbr</code> at the spine’s error (the alternative: a bound field in <code>Phases.Security</code>). [decision]	R5; finding 89	error of the blueprint (an omission the spine inherited)	MAJOR
ToArkLib/Component.lean:66; PublicInput.lean:374-381	<code>err</code> out of <code>Component.Def</code> (into <code>Security</code>), or the reduction in a computable structure that <code>Def</code> extends; then <code>publicInputPhase</code> , <code>leanVmVerifier</code> and <code>piopExtractor</code> compile. [decision]	R42; finding 122	none given (auditability)	MINOR
ToArkLib/Component.lean:79-80	Delete the field <code>outputPure</code> ; <code>Complete.append</code> takes <code>ArkLib</code> ’s <code>Prover.instOutputIsPureEmpty</code> ; keep the hand-written guarded.	R40; finding 93	forced by an upstream library (retired when the spine imports <code>NoAmbient</code>)	MINOR
ToArkLib/Component.lean:84-85	If completeness is to carry an error: <code>err</code> in <code>Complete</code> , composed by <code>ArkLib</code> ’s guarded completeness <code>append</code> (errors add), <code>piop_completeness</code> with <code>piop_perfectCompleteness</code> as its corollary. [decision]	R4 (R17); finding 119	error of the blueprint, conditional (disputed)	MAJOR
ToArkLib/Component.lean:90-93, 176, 182-184	A <code>Component.Guarded</code> (output purity and guard) extended by both <code>Complete</code> and <code>Security</code> ; <code>Security.append</code> gets a path that does not build to <code>Complete</code> . [decision]	R39; finding 39	deliberate deviation	MINOR
Spine/Compose.lean (new)	Define <code>piopExtractedStack P S</code> , the fold of <code>extractOut</code> and the <code>extractMids</code> down to round 0, and prove <code>piopExtractedStack P S s tr = tr {0, _}</code> for every bundle, statement and transcript; the tests’ <code>rfl</code> example becomes a corollary.	R46; finding 91	error of the blueprint	MINOR
Spine/Seams.lean:71-95, 130-134, 175-177	<code>BusOut</code> with a point, the forms per side and sumcheck table over row polynomials within the degree bound, three totals and the column claims, and <code>Seam.bus</code> accordingly; smallest, for the degree alone: <code>VirtualTerm.poly</code> a subtype within <code>I.d</code> , the degree conjunct deleted (the test <code>cubicTerm</code> becomes a typing failure). [decision]	R2 (R3); finding 22, finding 92	error of the blueprint	MAJOR
Spine/Instance.lean:73-81	Layout with <code>extend</code> and one law; <code>read</code> a definition; the toy’s <code>slice</code> becomes a theorem.	R41; finding 95	none given (a design remark)	MINOR
Spine/Instance.lean:119-148 (M3Instance)	The decidable predicate of the sumcheck tables (a constraint, a flush or a count column).	R11; finding 23; finding 79	error of the blueprint (omission)	MAJOR
Spine/Instance.lean:119-148 (M3Instance)	The fields <code>one_le_d</code> and <code>constrained_on_bus</code> , or the same as hypotheses of the bus phase. [decision]	R43; finding 14	error of the blueprint	MINOR
Spine/Compose.lean (Phases)	A field saying that each front phase’s verifier makes no query to <code>TheOracle I</code> , inhabited by the commit and public-input phases; or front phases typed as plain verifiers lifted by <code>keepOracles</code> . [decision]	R138; finding 2	error of the blueprint	MAJOR

Continued on the next page.

Table A.2, continued.

File and lines	Change	Register row; full text	Classification	Severity
ToArkLib/Refinement.lean:55-57	Add Refinement.knowledge_transport (the event form, proved in the probe P6Surface) as an ArkLib candidate; keep map_option_valid for the honest direction.	R119, R72; finding 96; finding 83	error of the blueprint	MINOR
ToArkLib/ PassThrough.lean:1-5 (header, as SendOracle.lean:1-7 does for SendSingleWitness)	Name ArkLib's ReduceClaim.oracleReduction, of which Component.passThrough is the named-form version.	R124; finding 131	none given	NOTE
ToVCVio/ UniformSample.lean:35	At the new VCVio pin a4232d08, replace probEvent_uniformSample_le_of_card_le by SampleableType.prEvent_uniformSample_le_div_iff; keep or upstream the subsingleton form.	R129; finding 136	none given (a duplicate of upstream after the pin moved)	NOTE
Basic.lean:3; LeanerVM.lean:19; scripts/check-layers.sh:30-41; scripts/test-policy-checks.py	Delete the empty LeanerVM/Protocol/Basic.lean (and its import), or drop its import of the arithmetization; write the wall's rule into check-layers.sh with an allow-list, and a planted violation into the policy tests.	R44; finding 111, finding 144	error of the blueprint (the witness text is false), and dead code	MINOR
The public-input phase				
PublicInput.lean:127-128, and verifier, prover.output, guarded.out, the state function (:350-366)	Pool the values sent (pooledFrom) in place of the values computed from the statement; with the per-limb check the state function's round 1 is unchanged in substance. The check becomes load-bearing: add as tests, next to threeLimbs, the refutations of its removal and of its weakening.	R13; finding 98	error of the design (an audit-surface compression)	MAJOR
PublicInput.lean (new)	Build the deployed phase: checkWords, deployedVerifier, deployedPhase, deployedSecurity at $1/ \mathbb{E} $ (key lemma accepts_two_challenges), and the lemma that the per-limb phase's acceptance implies the deployed one's. [decision]	R12; finding 99, finding 24	deliberate deviation owing its theorem	MAJOR
Layer 0 and Layer 1				
Field.lean:109	evalOracle_answer {n : ℕ}: n implicit, as the blueprint has it.	R75; finding 127	none given (a code/blueprint mismatch)	MINOR
Field.lean:80 (docstring)	Say that the $\mathbb{K}$ sampler is only the building block of the $\mathbb{E}$ sampler.	R74; finding 126	none given (avoidable audit surface)	MINOR
Field.lean:18, 29, 31, 37, 66 (docstring)	Rewrite without the roadmap, the layer numbers and the code (“the eager Fintype BF64 instance” in words); at 144c5aa, “of VCVio, a direct requirement”.	R130; finding 153	documentation	NOTE

Continued on the next page.

Table A.2, continued.

File and lines	Change	Register row; full text	Classification	Severity
tests/LeanerVMTests/ Protocol/Field.lean:25-26, 37-38 (at 144c5aa)	The comments name the column $[1, x, x + 1, x^2]$ (words 1, 2, 3, 4 as <code>K.ofBits</code>), not <code>[1, 2, 3, 4]</code> : <code>(2 : K)</code> is 0 at the new <code>CompPoly</code> pin.	R127; finding 134	none given (an upstream change adopted)	NOTE
ToCompPoly/ Multilinear.lean:50-51; Stack.lean	Add <code>sliceLow</code> , <code>evalMle_boolVec_append</code> , <code>Blocks.stridedLayout</code> , <code>Layout.piecewise</code> , with tests against the Python's <code>Placement.stack_point</code> .	R35; finding 104	error of the blueprint (omission)	MAJOR
FixedColumns.lean:84-87; tests/LeanerVMTests/ Protocol/ FixedColumns.lean:61-74	Delete <code>bytecodeColumn_slot</code> and its three examples; add the sixteen-instruction <code>#guard</code> against the Rust encoder (written with <code>K.ofBits</code> at the new pin).	R79; finding 107	error of the blueprint (stale witness), and avoidable audit surface	MINOR
BlockClaims.lean and its test	Delete (eight public declarations, no consumer).	R81; finding 110	none given (avoidable audit surface)	MINOR
Layer 1, unused or duplicated declarations	<code>prodVars</code> and its two lemmas, <code>unstack_eval</code> , <code>stack_eval_ambient_zero</code> , <code>stackColumn_eval</code> , <code>stackColumn_eval_ambient</code> , <code>bytecodeSlotColumn</code> , <code>slice_bytecodeColumn</code> , <code>evalMle_lagrangeBasis</code> (<code>CompPoly</code> has <code>eqTilde_eq_prod</code>): private or deleted.	R132; finding 114	none given (audit surface)	NOTE
lake test (no file named)	The owner checks whether <code>lake test</code> can again be an executable: the eager Fintype <code>BF64</code> that forbade it is gone at 144c5aa.	R125; finding 132	forced by an upstream library (retired)	NOTE

A.3 Changes that depend on a decision of the owner

The rows marked [decision] above, grouped by the choice they wait on. For each: the alternatives the register or the dossiers offer, one line each, and the review’s recommendation as the register or the dossier gives it (where only the options chapter, chapter 11, gives one, the entry says so). The other rows of the two tables do not depend on these choices, except in wording.

1. **The oracle interface of the stack** (R18, with R19; rows in Scope, the row *The oracle*, Layers 0, 10, 11 and 12).

- (i) Keep the evaluation oracle and the opening sumcheck, and owe a theorem relating that protocol to the deployed one.
- (ii) Give the stack the inner-product interface (a query is a weight, specification Definition 3.13); the opening phase is λ and one weighted query, which WHIR realizes.
- (iii) End the oracle protocol at the Flock seam and let the compilation append WHIR.

Recommendation: (ii). (i) is rejected, since no implication exists between two protocols with different transcripts; (iii) is acceptable, but changes the master theorems’ shape and leaves the stack’s interface unused (section 8.7.1.5).

2. **The shape of the bus seam** (R2, against the position R3; rows in the spine’s sketch and prose, test 28, the code table).

- (a) **BusOut** in the shape of the Rust’s **BusVerify**: one point, forms per side and sumcheck table with the degree bound in the type, three totals, the column claims.
- (b) Keep the types and add the guards to **Seam.bus** (at most $B + 3$ claims, one point per table, terms on sumcheck tables), checked by the table verifier and proved dead in the composition (Layer 12).
- (c) The smallest fix, for the degree only: **VirtualTerm.poly** a subtype within **I.d**.
- (d) No change beyond the bus phase’s two side conditions: the seam’s surplus burdens only the table phase (the position of **gt-bus.md** §E.1–E.4, R3).

Recommendation: (a), the table-phase dossier’s redesign, which the spine dossier endorses (register R2; finding 22). The second reading of the majors finds no tension between R2 and R3: the bus phase fills its slot, and the table phase cannot serve the seam (**verify-majors.md**, summary).

3. **Which public-input check the phase models** (R12; the pooling of R13 is needed either way).

- (a) Build the deployed phase (the combined equation on the two words, the values sent pooled) and let the master theorems and Layer 12 take it; keep the specification’s per-limb phase as the reference, with the lemma that its acceptance implies the deployed one’s; report the ambiguity of §8.2 to leanVM.
- (b) Keep only the specification’s phase: the theorem is then about a verifier none of the three executable verifiers runs, and the debt assigned to Layer 12 cannot be paid there.

Recommendation: (a) (findings 24 and 99).

4. **Whether completeness carries an error** (R4 against R17, with R39; rows *Holes*, the spine’s sketch, Layer 9, test 20).

- (a) A field **err** in **Component.Complete**, errors added by ArkLib’s guarded completeness append, **piop_completeness** with **piop_perfectCompleteness** as its corollary, and a completeness error for the Flock phase (finding 119).
- (b) Perfect completeness stays: the specification’s Flock is perfectly complete, the Lean honest prover sends the true coefficients, and the Rust prover’s defect at $r_{eq} = 1$ is recorded in **docs/leanvm-target.md** (findings 38 and 46).
- (c) Independently of (a) or (b): separate the guard from completeness (**Component.Guarded**), so that knowledge soundness no longer presupposes completeness (finding 39).

Recommendation: the dossiers disagree. The opening dossier (section 8.7.3.1) agrees with (b) on the facts and calls (a)’s field “the right insurance”. The options chapter (section 11.1, item 5) recommends (c) with (b): completeness left perfect.

5. **Whether the framework stays on ArkLib’s OracleReduction** (R34; the ledger rows, Layer 10, the interfaces).

- (a) Stay on the framework ArkLib calls legacy, keep the port of #615 until some ArkLib framework

proves a guarded-first append, prove the plain corollary locally, and record the decision with its retirement condition.

(b) Plan the migration to ArkLib’s typed Interaction framework, which at 7653a901 has no knowledge soundness, no round-by-round notion and no extractor.

Recommendation: (a) (finding 120; the options chapter, item 9: nothing upstream is ready to receive the spine).

6. **How the phases’ error is constrained** (R5; rows *Errors*, item 6, Layer 10, the code table).

(a) A closed-form bound as a field of `Phases.Security` (or a new `Phases.Error`), the phases still declaring their errors.

(b) The spine fixes each phase’s error as a function `piopError I` of the instance’s sizes and demands each phase’s round-by-round bound at it (the schedules then fixed by the spine too).

Recommendation: (b) (finding 89).

7. **Where the error lives in the hole interfaces** (R42; row *Holes*, the spine’s sketch, the code table).

(a) `err` moves from `Component.Def` into `Security`.

(b) The reduction becomes a computable structure that `Def` extends.

Recommendation: the dossier gives the reason for (a) (the error is a property of the soundness proof, not of the reduction) without ranking the two (finding 122); the design recommended in item 6 also takes `err` out of `Def`.

8. **The program hypothesis of the base theorems** (R27; the leanISA contract row, Layer 13).

(a) Both theorems take `hwf : WellFormedBytecode prog`.

(b) `verify` decides the program condition on the public program and rejects otherwise, which removes the hypothesis from the soundness theorem.

Recommendation: none; the register calls it a technical decision. The drafted texts of both dossiers take (a) (finding 137; finding 73).

9. **The resources of base completeness** (R29, R122; Layer 13, the boundary with leanISA).

(a) The completeness theorem from an execution takes a fit hypothesis `hfit`, which is then the resource condition `architecture.md` asks for.

(b) The leanISA roadmap supplies a `constraintCompleteness` that returns a witness of minimal heights (itself needing a resource hypothesis, R122).

Recommendation: none given (finding 74 offers both).

10. **The round message of the sumchecks** (R53; row *Sumcheck messages*).

(a) The wire’s message becomes the oracle protocol’s (three coefficients, no round check): the compilation is literal and one check leaves the audit.

(b) The oracle protocol keeps four coefficients and the round check, and Layer 4 proves the transport lemma.

Recommendation: the dossier does not rank them (finding 25); the options chapter (section 11.2) keeps the deviation, that is (b), with the lemma owed.

11. **The GKR inside the bus phase** (R10; the ledger row on context lifting, Layers 5 and 6).

(a) Layer 5’s GKR over a context, with the leaf tables derived from the stack and the zerocheck riding along; retired when ArkLib proves its lifting theorems and offers riders.

(b) ArkLib’s context lifting, whose completeness and knowledge-soundness theorems are admitted at both pins, and which alone would not carry the zerocheck clause.

(c) A monolithic bus phase (named by the options chapter, item 6, not by a dossier).

Recommendation: (a) (finding 7).

12. **Where the Flock slot map and the adaptor’s Flock obligations live** (R15, R16; Layers 3 and 9, the interfaces, the boundaries).

(a) `blake2sLimbSlot : Fin 18 → Fin 256` in `Parameters/`, the low-index selection generic, Layer 3 building the reader, `limbColumns` deleted (finding 36).

(b) One instance-free `FlockSpec`, supplied by #3, with the slot map, the predicate, the lemma “R1CS $\Rightarrow$ compress”, the generator and its lemma; `FlockWitnessGen` and `limbColumns` deleted; the phase

takes the `FlockRegion I` of R16 (finding 77).

Recommendation: none ranked; the register merges them into one change (R15), and (b) includes (a)'s slot map with the two obligations (a) leaves without a carrier.

13. **The bus phase's two side conditions** (R43; the spine's sketch, test 28). Fields `one_le_d` and `constrained_on_bus` of `M3Instance`, or the same as hypotheses of the bus phase in Layer 6. Recommendation: none given (finding 14).
14. **How the front's oracle-freeness is enforced** (R138; row *The oracle*, the spine, Layer 12, the code table).
 - (a) A field of `Phases` (or a hypothesis of the compilation theorem) that no front phase's verifier queries the stack, inhabited by the two built phases.
 - (b) Front phases typed as plain verifiers lifted by `keepOracles`, so that a query is a typing error.
 Recommendation: none ranked (finding 2).
15. **The error over the prover's announced sizes** (R6; row *Statements and parameters*, Layer 12, test 21). The maximum over admissible sizes, with `FiatShamirSecurity` in family form and the sizes in the challenge oracle's input; or the sum, if the interface is per instance (a union over about 2^{34} sizes, a hand estimate). Recommendation: the family form with the maximum (finding 76).
16. **The Merkle trees** (R77; the ledger, Layer 11). Build Layer 11 on `VCVio`'s Merkle-tree library (`VCVio` issue 571), or write the trees here as planned. Recommendation: none; whether `VCVio`'s trees fit leanVM's (BLAKE2s, the leaf encoding, pruned paths) is unverified (finding 151).
17. **The obligations no hole owns** (R137; the holes table, the ordering). One new hole per obligation, or each folded into the nearest hole with one new hole for the list-binding compilation. Recommendation: none ranked; finding 1 gives the folding in full.

Appendix B

The probes

Every Lean or Python probe a conclusion of this report rests on, with its source, the command that ran it, its recorded output and the revision it ran at. Probes ran against `main` at `b435631` with the old pins unless marked otherwise; those re-run after the upgrade are marked with their outcome at the new pins. A probe marked “written, not run” supports no conclusion beyond the paper argument it accompanies.

B.1 The spine, the public-input phase, Layer 1

B.1.1 The probes on the built code

The probes of the three code dossiers: what each establishes, its source, the command that ran it and its recorded output. All ran against `leanerVM main` at `b435631` with the old pins (ArkLib `dca90385`, CompPoly `3468b38c`, VCVio `f9dc47d9`), before the checkout moved to `144c5aa` on 2026-09-30; none can run at the new pins without a rebuild. A probe that was written and not run, or argued on paper, says so in its heading. “Accepted” means that the file elaborated with exit status 0, so that every `#guard` evaluated to `true` and every `example` and theorem was checked by the kernel.

B.1.1.1 The spine: non-vacuity by probe (D)

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-spine.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.1; B (audit surface) is section 10.5.2; C (the statements) is section 4.4; D (non-vacuity by probe), G (what was not verified) and the appendix (the probes) is ?? B.1.1.1; E (conformance) is section 8.9.1.1; F (findings) is section 8.9.2.1.

Ground moved during the review. The checkout now holds `main` at `144c5aa` (Lean 4.34.1, all pins moved). Everything here is about `b435631`; every Lean line number is `git show b435631:<path>`. All seven probes (P1 to P6, with P3 in three files) were run and recorded **before** the move, at `b435631` with the old pins; their outputs are in the appendix. Three probes (P2Relation, P3aSeams, P5PassThrough) use the numeral `(2 : K)`, which at the old pin is the polynomial `x` (nonzero, not Boolean) and at the new CompPoly pin is `0`; they must be re-run with `K.ofBits 2` after a rebuild, and the conclusions they support are marked accordingly. One probe was planned and not written (the read-everything phase, D.3 (d)); the sibling dossier reaches the same conclusion on paper.

All probes ran against `leanerVM main` at `b435631` (ArkLib `dca90385`, CompPoly `3468b38c`, VCVio `f9dc47d9`), each under the shared lock. Their full sources and outputs are in the appendix. “Accepted” below means the file elaborated with exit status 0, so every `#guard` evaluated to `true` and every `example` and theorem was checked by the kernel.

Library objects used in this section, introduced once:

- **OracleReduction** (ArkLib): a pair of an honest prover and a verifier for a fixed message schedule (`ProtocolSpec n`: for each of `n` rounds, who speaks and the type of what is sent). The verifier may query the input oracles and the prover’s messages.

- **Verifier.GuardedForm** (ArkLib Security/CoordinateWiseSpecialSoundness/Guarded.lean:112): the data of a Boolean check and a verdict function, with the proof that the verifier outputs the verdict when the check passes and rejects otherwise.
- **Extractor.RoundByRound** (ArkLib Security/RoundByRound.lean:77): not one function from transcripts to witnesses, but a family of intermediate witness types `WitMid m`, one per round boundary $m = 0 \dots n$, with `WitMid 0` the input witness type, a map `extractOut` from the output witness to `WitMid n`, and maps `extractMid m : WitMid (m+1) → WitMid m` that walk back one round at a time, each seeing the transcript so far.
- **Verifier.KnowledgeStateFunction** (ArkLib RoundByRound.lean:164): a predicate `toFun m stmt t ranscript witMid` that (`toFun_empty`) is the input relation on the empty transcript, (`toFun_next`) cannot be made true by a prover message (if it is true after the message on `w`, it was true before on `extractMid w`), and (`toFun_full`) is true on every full transcript on which the verifier can output a statement in the output relation.
- **rbrKnowledgeSoundnessWorstCaseWith** (ArkLib RoundByRound.lean:553): for one named extractor and one named knowledge state function, for every input statement, every challenge round and every fixed transcript prefix, the probability over the fresh challenge that the state goes from false to true is at most the error declared for that challenge.

```
-- .lake/packages/ArkLib/ArkLib/OracleReduction/Security/RoundByRound.lean:553-568 (ArkLib dca90385)
def rbrKnowledgeSoundnessWorstCaseWith
  (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (WitMid : Fin (n + 1) → Type)
  (extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid)
  (kSF : verifier.KnowledgeStateFunction init impl relIn relOut extractor)
  (rbrKnowledgeError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∀ stmtIn : StmtIn,
  ∀ i : pSpec.ChallengeIdx,
  ∀ transcript : Transcript i.l.castSucc pSpec,
  Pr[fun challenge =>
    ∃ witMid,
      ¬ kSF i.l.castSucc stmtIn transcript
        (extractor.extractMid i.l stmtIn (transcript.concat challenge) witMid) ∧
        kSF i.l.succ stmtIn (transcript.concat challenge) witMid
    | $t (pSpec.Challenge i)] ≤ rbrKnowledgeError i
```

Two facts about this definition drive the results of D.3. The error is any function `ChallengeIdx → ℝ≥0`: nothing bounds it by one. And the statement bounds a probability by it: with a declared error of `1` the inequality holds of every state function.

M3Holds on the toy instance (D.1) What the repository’s tests do (tests/LeanerVMTests/Protocol/Spine.lean:26-98, read in full). The honest stack is accepted at statement 1 (:26). Five mutated stacks or statements fail exactly one clause each, the three other checkable clauses being asserted to hold: `badConstraint` (:32-38), `badBalance` (:41-47), `badCount` (:50-56), the wrong statement (:59-61), `badLine` (:65-71). `badBalanceSum` (:76-85) is a stack whose pushed and pulled first coordinates have the same sum in K and are different multisets. The fifth clause, `aux`, is `True` on the toy and is tested by nothing. The claim of the test’s docstring and of acceptance test 27 (“each of the four checkable clauses failing alone”) is met.

What is missing, and added by probe P2Relation.lean (accepted).

1. *Multiplicity*. Every mutation of the repository changes a **value**; none changes a **count**. The toy cannot: its table pushes two tuples and its boundary pulls two, whatever the stack. The probe builds two variants of the instance.
 - **toyOnce**: the boundary is one block of height one pulling $(1, 0, \dots)$. On the honest stack the pushed tuples are $(1, 0, \dots)$, $(1, 0, \dots)$ and the pulled tuple is $(1, 0, \dots)$: the two sides are the same **set** (each tuple of one side occurs in the other, both guards pass) and different multisets. Only **Balanced** fails; **M3Holds** fails.
 - **toyNever**: no boundary. The tuple is pushed twice and never pulled. For each of the sixteen coordinates the sum over the pushed side equals the sum over the (empty) pulled side, $1 + 1 = 0$ in characteristic two: a balance summed in the field accepts. Only **Balanced** fails; **M3Holds**

fails. This is the case the module docstring of `Instance.lean` names (:30-31, “in characteristic two a tuple pushed twice and never pulled would sum to zero”) and that no test exercised.

2. *The side filter.* `toySwap` exchanges the two sides (the table’s flush pulls, the boundary pushes). `flushTuples` and `boundaryTuples` return the tuples on the declared side and none on the other (lengths 0, 2 and 2, 0), and the honest stack satisfies `M3Holds toySwap`. The filters `decide (f.1 = s)` and `decide (b.side = s)` are therefore not swapped.
3. *The row ranges.* One tuple per row of the table (2^{τ}) and one per row of a block (2^{κ} , also for $\kappa = 0$).
4. *Which cells `CountsNonzero` ranges over.* Every row of every column listed in `counts`, and no other cell: a stack with zeros in column 0, column 2 and the padding cells satisfies the clause; a zero in either cell of column 1 fails it.

Conclusion. `Balanced` is a multiset equality counted in $\mathbb{N}$ and the list construction is right on every case tried. No defect found in the five clauses as definitions.

The seams, one by one (D.2) Probe `P3aSeams.lean` (accepted). The repository gives the claims no decision procedure; the probe supplies them by `inferInstanceAs` on the unfolded propositions and states, by `Iff.rfl`, that the predicates it decides are the seams.

Seam	Inhabitant (on the honest stack, statement 1)	Near misses, each rejected
<code>Seam.commit</code>	the honest stack (repository test :26, and <code>Iff.rfl</code> at :166-167)	the repository’s five mutations
<code>Seam.bus</code>	the zerocheck claim of the toy’s constraint at the points (0) and (1) with value 0, and the column claim “column 0 at the point (x) is 1”	a false linear claim; a true claim whose term is cubic; a false column claim; the wrong statement
<code>Seam.table</code>	the column claim	a false column claim; the wrong statement
<code>Seam.pub</code>	the column claim	a false column claim. The wrong statement is inside the seam: the public seam no longer mentions the statement
<code>Seam.flock</code>	the column claim and the weighted claim “the weight selecting cell 0 pairs to 1”	a false weighted claim; a false column claim
<code>Seam.done</code>	everything	none: <code>Seam.done toy = Set.univ</code> (proved in the probe)

Two further facts the probe establishes, both intended by the design and both worth stating in the report because a reader of the seam table may assume the opposite.

- **A seam does not imply the relation before it.** On `badConstraint` (column 2 is [2, 0], so the constraint $X_2^2 - X_2$ is violated on row 0) the zerocheck claim at the point (1) with value 0 is true: the extension of the table [a, 0] is $(1 - \zeta) \cdot a$, zero at $\zeta = 1$. So $((2, ([zeroAt1], [col0One])), \text{badConstraint})$ is in `Seam.bus toy` and `badConstraint` is outside `M3Holds toy` 2. This is the zerocheck escape; the bus phase’s error must pay for it.
- **A seam does not say which claims are emitted.** $((2, ([], [])), \text{badConstraint})$ is in `Seam.bus toy`.

Phases.Complete and Phases.Security (D.3)

(a) **The repository’s pass-through phases** `trivPhases` and `trivComplete` (tests/.../Spine.lean:103-119) inhabit `Phases toy` and `Phases.Complete` with five zero-round phases that drop every claim. The test’s docstring says that `Phases.Security` “has no such inhabitant” and gives the reason (“dropping every claim is ... not knowledge sound”), but proves nothing. Probe `P5PassThrough.lean` (accepted; #print axioms `no_security` gives the three standard axioms) proves it for the bus phase, where it fails first:

```

/-- The reflection hypothesis of `Phase.passThroughSecurity` is false at the bus seam. -/
theorem not_reflects : ¬ ∀ (s : K) (o : ∀ i, TheOracle toy i),
  ((dropAll s, o), ()) ∈ Seam.bus toy → ((s, o), ()) ∈ Seam.commit toy

```

```

/-- No `Phase.Security` exists for the pass-through bus phase: not only the repository's
constructor fails, the type is empty. -/
theorem no_security (S : Phase.Security toy (Phase.passThrough toy dropAll) (Seam.commit toy)
  (Seam.bus toy)) : False

```

The proof takes the knowledge state function of S at the trivial shared-oracle state, the statement 2 with the oracle `badConstraint` (outside `M3Holds`, inside the bus seam with no claim, both by `decide +kernel`), and the empty transcript; the verifier accepts it with probability one, so `toFun_full` makes the state true at the end, which for a phase with no round is the beginning, where `toFun_empty` says the state is `M3Holds`. Exactly where the task expected it to fail: a pass-through bus phase cannot reflect `Seam.bus` back into `M3Holds`.

(b) A cheap inhabitant of `Phases.Security`: five phases that check nothing, at declared error

1 Probe `P4Junk.lean` (accepted; `#print axioms` gives `propext`, `Classical.choice`, `Quot.sound`).

The phase `wasted f ε` has one round: the verifier draws a challenge in E , ignores it, makes no query, and outputs f of its input statement. Its verifier accepts every transcript (`wPure`, a `Verifier.PureForm`).

- It is perfectly complete against any two seams that f carries one into the other (`wastedComplete`): the hypothesis of the repository's `Phase.passThroughComplete`.
- With declared error 1 it has a `Phase.Security` against **any** two seams, under that completeness hypothesis alone (`wastedSecurity`). The knowledge state function is the input seam before the challenge and `True` after it (`wState`, which takes no hypothesis on f or on the seams); the bound is $\Pr[\dots] \leq 1$ (`w_rbr`).

```

-- probes/code-spine/P4Junk.lean
def wState (f : StmtIn → StmtOut) :
  (wVerifier I f).toVerifier.KnowledgeStateFunction init impl relIn relOut
  (wExtractor I (StmtIn := StmtIn)) where
  toFun := fun m stmt _ => if m.val = 0 then (stmt, ()) ∈ relIn else True
  toFun_empty := fun _ _ => by simp
  toFun_next := fun m hm => by
    fin_cases m
    exact absurd hm (by decide)
  toFun_full := fun _ _ _ => by simp

noncomputable def junkSecurity : junkPhases.Security where
  bus := wastedSecurity toy _ fun _ _ h => (by simp, by simp, by simp, h.2.2.2.1, h.2.2.2.2)
  table := wastedSecurity toy _ fun _ _ h => (by simp, h.2.2.2.1, h.2.2.2.2)
  pub := wastedSecurity toy _ fun _ _ h => (by simp, h.2.2)
  flock := wastedSecurity toy _ fun _ _ _ h => (by simp, by simp)
  opening := wastedSecurity toy _ fun _ _ _ => trivial

example {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] σ (StateT σ ProbComp)) :
  (leanVmVerifier junkPhases).toVerifier.rbrKnowledgeSoundnessWorstCase init impl (M3Rel toy)
  (Seam.done toy) (piofError junkPhases) :=
  piof_rbrKnowledgeSoundness_exists junkPhases junkSecurity init impl

theorem piofError_junk (i : junkPhases.toDef.pSpec.ChallengeIdx) : piofError junkPhases i = 1

```

So `Phases.Security toy` is inhabited, both master theorems apply to `junkPhases`, and the protocol they are about commits to a stack and accepts it whatever it is. The knowledge theorem is true of it because its error is 1 at each of its five challenges (`piofError_junk`). Nothing in the construction uses the toy: the same five phases inhabit `Phases.Security I` for every instance I .

Reading. `piof_rbrKnowledgeSoundness` has content only together with a bound on `piofError P`. No such bound is stated in the spine, and the blueprint's `piofError_le` (Layer 10, :1130) is stated over ($s : \text{Sizes}$), a parameter the spine's `piofError P` does not have (finding *the declared error is unconstrained*).

(c) **An always-rejecting verifier** A verifier that always rejects is round-by-round knowledge sound at error zero (the state “the input relation holds” never has to become true). It is excluded from `Phases.Security` by the structure itself:

```
-- LeanerVM/Protocol/ToArkLib/Component.lean:90-93
structure Security (D : Def StmtIn OStmtIn WitIn StmtOut OStmtOut WitOut)
  (relIn : Set ((StmtIn × ∀ i, OStmtIn i) × WitIn))
  (relOut : Set ((OStmtOut × ∀ i, OStmtOut i) × WitOut))
  extends Complete D relIn relOut where
```

Complete.complete is perfect completeness on the input seam, so on any statement and stack in the input seam the verifier accepts the honest run with probability one. **Seam.commit toy** is inhabited (D.1). This is a reading of the definitions, not a probe: a machine-checked refutation would need the probability of an event under a failing **OptionT** computation, which adds nothing to the argument.

The exclusion is only as strong as the input seam is inhabited: on an instance whose relation is empty (D.4 exhibits one) perfect completeness is vacuous and an always-rejecting bundle is a **Phases.Security**.

(d) A phase that reads the whole stack (argued on paper; the probe was not written) In the ideal oracle model the verifier may query the stack’s extension at any point of E^μ , any number of times: neither ArkLib’s **OracleVerifier** nor the spine counts queries. At the 2^μ points of the cube the answers are the cells. **M3Holds** and every claim’s **Holds** are decidable. So for every seam there is a zero-round phase whose verifier reads the 2^μ cells, decides the **input** seam itself, rejects if it fails and otherwise outputs any statement in the output seam (no claim at all will do). It is perfectly complete, and knowledge sound at error zero: acceptance implies the input seam. Five of them inhabit **Phases.Security I** for every **I**, with **piopError** = 0 (there is no challenge).

This is **not machine-checked** here (the probe was not written: it needs an oracle-querying verifier and the unfolding of 2^μ simulated queries, and the machine’s memory limit was reached during this session). The sibling dossier **gt-table-pub.md** (section 6, E.4 item 6) reaches the same conclusion on paper; the two readings agree. What would verify it: a probe on an instance with $\mu = 0$ (one cell, one query), whose verifier is `do let v ← query; if decide (relIn s (#v[v.limb 0])) then pure (f s) else failure`.

What (b) and (d) mean together. The hypotheses of the master theorems are satisfiable, for every instance, by protocols that are not leanVM’s: one with no check and error 1, one with no interaction, error 0 and a verifier as expensive as the prover. This is expected of a composition theorem. What pins the phases to leanVM’s is outside the spine and, today, outside the repository:

1. a bound on **piopError P** (excludes (b));
2. the phases’ *definitions* being leanVM’s, which no theorem of the oracle protocol states; it is Layer 12’s **verify_iff_compiled**, together with the fixture “a proof of the pinned Rust prover is accepted by **verify**”, that ties the composed verifier to the deployed one (excludes (d), whose transcript is empty);
3. review of each phase’s **Def** against the specification.

What a layout can do (D.4) Probe **P2Relation.lean**, last section (accepted).

toyAlias is the toy with the layout that reads **every** column from cells 0, 1 (extend $c\ z = (z, 0, 0)$ for every c). The reading law **read_eval** holds of it (**alias_read_eval**, proved by the same **simp** call as the toy’s), so it is a **Layout** and **toyAlias** is an **M3Instance**. On it the three columns are one column, and **M3Holds** asks for incompatible things: balance forces cells 0, 1 to be 1, 1 and the public line on column 2 forces cell 1 to be 0. The honest stack fails; the exhaustive guard over cells 0, 1 and the statement in {0, 1, 2} finds no satisfying stack. That the relation is empty for **every** stack and statement is the two-line argument above, on paper; the guard is evidence, not a proof.

Consequences, for the report:

- The master theorems hold of **toyAlias** (they hold of every instance). Completeness is then vacuous and knowledge soundness says that every accepting prover has been lucky.
- A layout **cannot ignore the stack**. This is a consequence of the law, on paper: if **read q c** did not depend on q , the law at the all-zero stack and at the all-one stack would give $0 = 1$, since the extension of a constant table is that constant at every point. More precisely the law **determines read from extend**: on a cube point x of the column, **(read q c)[x]** is the stack’s extension at **extend c x**, and an extension at a cube point is the cell. So **read** is redundant data (proposal in section B), and the only freedom of an instance is **extend**.

- What a wrong `extend` costs is **completeness and faithfulness, never soundness** of the Lean chain. If the layout aliases or mixes columns, `M3Holds I` is a relation on the aliased columns; the adaptor’s soundness lemma `satisfiedBy_witnessOf (M3Holds → SatisfiedBy)` is then a theorem about that instance and, if it is proved, the chain to `ValidExecution` is intact. What breaks is `m3Holds_stackOf` and `witnessOf_stackOf` (no stack packs distinct columns into the same cells), and the agreement with the deployed layout (the Lean verifier rejects the Rust prover’s proofs).
- **Who rules such instances out.** Not the spine. For `leanISA: witnessOf_stackOf` and `m3Holds_stackOf` (blueprint Layer 3, not built) for non-vacuity, and the transcription of `witness.rs:67-101` with Layer 12’s fixture for faithfulness. Until they exist, no statement on `main` says that the relation of any instance other than the toy is inhabited.

The extractor (D.5) Is anything marked noncomputable? Probe `P1Axioms.lean`: `Lean.isNoncomputable` is false for `piopExtractor`, `commitExtractor`, `Component.sendExtractor`, `Phases.Security.toDef`, `Component.Security.append`, `ArkLib’s Extractor.RoundByRound.append`, `Component.passThroughExtractor`, `PublicInput.extractor` and `publicInputSecurity`. It is true for `publicInputPhase` only (its error is a real number), as the blueprint says (:1049).

Does #print axioms mean anything here? No. It reports `propext`, `Classical.choice`, `Quot.sound` for `piopExtractor`, for `commitExtractor` and for `ArkLib’s Extractor.RoundByRound.append`, all three computable definitions, because the proofs inside them (the casts between witness types) use classical lemmas. `Classical.choice` in the axiom list of a definition does not say that its **data** was chosen classically. The blueprint does not claim otherwise; the earlier review’s validation paragraph lists axioms and should not be read as evidence of computability.

Does the repository’s test prove that the extractor returns the committed stack? Only for phases with no round. The test (`tests/.../Spine.lean:157-159`) is

```
example (S : trivPhases.Security) :
  (piopExtractor trivPhases S).extractOut ((1 : K), fun i : Fin 0 → i.elim0)
    honestFullTranscript () = honest := rfl
```

It is true for the reason `ArkLib’s append` gives (`Append/StateFunction.lean:129-139`): when the second component has no round, the composed `extractOut` is the first component’s applied to the second’s. With five zero-round phases the composed `extractOut` is the commit phase’s, which returns the first message. As soon as one phase has a round, the composed `extractOut` is the **last** such phase’s, and its value lives in that phase’s last intermediate witness type. Probe `P3cExtractor.lean`, with the repository’s own `publicInputPhase` in the bundle and four pass-throughs:

```
-- accepted
example (realPubSecurity Sb St Sf So).toDef.witMid (Fin.last 3) = Unit := rfl
example : (realPubSecurity Sb St Sf So).toDef.witMid 0 = Column 3 := rfl
example (s : K) (tr : ...Transcript (Fin.succ 0, _)) (w : ...witMid (Fin.succ 0, _)) :
  (piopExtractor realPub (realPubSecurity Sb St Sf So)).extractMid 0, _
  (s, fun i : Fin 0 → i.elim0) tr w = (tr 0, _) : Column 3 := rfl
-- rejected, as expected: the test's statement with this bundle
example (tr : realPub.toDef.pSpec.FullTranscript) :
  (piopExtractor realPub (realPubSecurity Sb St Sf So)).extractOut
    ((1 : K), fun i : Fin 0 → i.elim0) tr () = honest := rfl
```

```
P3cExtractor.lean:58:49: error: Type mismatch
  honest
has type
  Column 3
but is expected to have type
  (realPubSecurity Sb St Sf So).toDef.witMid (Fin.last realPub.toDef.n)
```

So for `leanVM`’s phases “`piopExtractor ... extractOut` returns the stack” is not a false statement but an ill-typed one. The stack is what the composed extractor returns **at round 0**, by its `extractMid` at the commit round, and there it is the transcript’s first message whatever it is handed (third example). The spine defines no function “the stack extracted from a full transcript” (the composite of `extractOut` and the `extractMids` down to round 0); the statement “the extractor’s column `q` satisfies `M3Holds`” of

the blueprint’s headline is therefore not a statement about any definition of the repository (finding *the extracted stack is not a definition*).

Could a phase’s Security supply an extractor defined by Classical.choice without anything noticing? Yes as to the mechanism, and it would not matter as to the result.

- *Mechanism.* `Component.Security.extractor` is a field of type `Extractor.RoundByRound ...`. Lean’s types do not record computability. A phase author can write `noncomputable def fooSecurity : Phase.Security ...` with a classical `extractMid`; `Phases.Security.toDef`, `piopExtractor` and both theorems accept it, `piopExtractor` itself stays computable (it is a projection of its argument), `#print axioms` does not change, and the repository’s checks (`audit-lean.sh` forbids `axiom`, `sorry`, `admit`, `unsafe`, `native_decide`, per `AGENTS.md`) do not forbid `noncomputable`: `publicInputPhase` is already `noncomputable`, legitimately. ArkLib itself ships such an extractor, `Extractor.RoundByRoundOneShot.toRoundByRoundOfRel` (`RoundByRound.lean:118-123`, if $h : \exists v, (stmtIn, v) \in relIn$ then $h.choose$ else `witIn`). Acceptance test 24 is enforced by review only.
- *Result.* From the commit phase on, a phase’s input and output witness types are `Unit` (`Phase.Def`, `Phase.lean:37-38`), and `eqIn` forces its `WitMid 0` to be `Unit`. Whatever a phase’s extractor does, at the seam it hands `()` to the phase before it. The stack comes from the commit phase’s extractor alone, which is the spine’s, is computable, and reads the message. A classical phase extractor cannot produce a stack. The risk acceptance test 24 guards against (a witness chosen because one exists) is excluded by the **design** “the oracle is the witness”, not by the test.

The commit phase’s security (D.6) Probe `P3bCommit.lean` (accepted), every statement by `Iff.rfl` or `rfl`:

```
-- Before the message: M3Holds of the candidate witness.
example (s : K) (o : ∀ i, NoOracle i) (w : Column 3) :
  ((commitSecurity toy).kSF init impl).toFun 0 (s, o) default w ↔ M3Holds toy s w := Iff.rfl
-- After the message: M3Holds of the message, whatever the candidate witness.
example (s : K) (o : ∀ i, NoOracle i) (tr : (commitSpec toy).FullTranscript) (w : Column 3) :
  ((commitSecurity toy).kSF init impl).toFun (Fin.last 1) (s, o) tr w ↔
    M3Holds toy s (tr 0) := Iff.rfl
-- The extractor: both maps return the message.
example ... : (commitExtractor toy).extractOut (s, o) tr () = tr 0 := rfl
example ... : (commitExtractor toy).extractMid 0 (s, o) tr w = tr 0 := rfl
```

The state function says what the docstring says (`SendOracle.lean:143-144`). Error zero is right: the phase has no challenge (`sendOracle_rbr` is the elimination of an empty index type), so its error function is the empty function and the `0` written in `sendOracle (:65)` is never applied. The extractor is “read the message”.

Three facts behind sections B and C.7 (D.7) Probe `P6Surface.lean` (accepted; both theorems with the three standard axioms).

1. `outputPure_of_def`: for every `Component.Def` whatsoever, `D.red.prover.OutputIsPure` holds, because the shared oracle specification is empty (`[]o`) and a computation with no possible query is its pure result. ArkLib has this at the pin as the instance `Prover.instOutputIsPureEmpty` (`Composition/Sequential/NoAmbient.lean:39-42`), in a module the spine does not import. The field `Component.Complete.outputPure` is therefore redundant (section B).
2. `@Refinement : {Stmt : Type} → {W1 : Type u1} → {W2 : Type u2} → ...`: the two witness universes are independent, so a refinement from the stack (`Type`) to Clean’s `EnsembleWitness (Type 1)` can be formed, as the blueprint’s ladder needs (`:364-369`). Checked by `#check` and by an example from `ℕ` to a structure in `Type 1`.
3. `knowledge_transport`: if a verifier is knowledge sound for `R` with a named straight-line extractor at error ϵ (ArkLib’s `knowledgeSoundnessWith`), then, for the same provers and the same extractor, the probability that the verifier accepts and the extracted witness, mapped by a refinement $R \rightarrow S$, is invalid for `S` is at most ϵ . Three lines: the event is monotone (`probEvent_mono`) and `map_valid` is the pointwise step. No prover conversion and no `Nonempty` hypothesis: the game is not changed, the map is applied inside its event. This is the probabilistic transport the blueprint says is not proved (`:582-586`), in the form Layer 13 needs (section C.7).

What was not verified, and what would verify it (G)

Claim	Status	What would verify it
The relation of <code>toyAlias</code> is empty for every stack and statement (D.4)	evidence by an exhaustive <code>#guard</code> over cells <code>0</code> , <code>1</code> and statements in <code>{0, 1, 2}</code> at the old pin; the paper argument is two lines	a theorem $\forall \text{ input } q, \neg \text{M3Holds } \text{toyAlias } \text{input } q$, whose proof needs <code>CompPoly</code> 's evaluation of <code>X 0</code> on a row (no <code>eval_X</code> lemma at 3468b38c; the sibling dossiers may have one)
<code>Layout.read</code> is determined by <code>extend</code> (C.2)	on paper	a lemma <code>read_eq_of_extend</code> from <code>CompPoly</code> 's <code>eval_mle_eq_eval</code> (cited in <code>FileId.lean:50</code>)
An always-rejecting bundle has no <code>Phases.Security</code> on an instance with an inhabited relation (D.3 (c))	a reading of <code>Security</code> extends <code>Complete</code> and of <code>perfectCompleteness</code>	a probe with a rejecting zero-round verifier and <code>perfectCompleteness_eq_prob_one</code>
The read-everything phase inhabits <code>Phases.Security I</code> at error <code>0</code> for every <code>I</code> (D.3 (d))	on paper; agrees with <code>gt-table-pub.md</code> section 6	a probe on an instance with $\mu = 0$: one oracle query, if <code>decide (relIn s (#v[v.lim b 0]))</code> then <code>pure (f s)</code> else failure, with <code>Verifier.GuardedForm.of_probEvent_pos</code> for the state function
The probes that use <code>(2 : K)</code> mean the same at the new <code>CompPoly</code> pin	they do not (<code>(2 : K) = 0</code> there, brief §8); their conclusions are about b435631 with the old pins, where <code>2 : K</code> is the polynomial <code>x (tests/.../Spine.lean:30)</code>	re-run <code>P2Relation</code> , <code>P3aSeams</code> , <code>P5PassThrough</code> with <code>K.ofBits 2</code> after the rebuild
Line numbers of <code>Field.lean</code> and <code>KnowledgeAppend.lean</code>	at b435631 (the working tree had moved by the time the counting script first ran; the script was re-run on <code>git show b435631</code> : and the numbers in sections B and C are from that run)	—
The earlier review's validation paragraph (<code>docs/reviews/protocol-spine.md:19-28</code>)	not re-run (no build allowed); <code>#print axiom s</code> re-established by probe P1 for the declarations listed there and more	<code>./scripts/validate.sh</code> on b435631

B.1.1.2 The spine's probes in full

Every probe is a plain Lean file under `.claude/reports/blueprint-review/probes/code-spine/`, run from the repository root (leanerVM main at b435631, ArkLib dca90385) with

```
flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/<File>.lean
```

The output of each run is the file `<File>.out` beside it, reproduced below the source. An empty output with `exit=0` means every `#guard`, `example` and theorem of the file was accepted.

P1Axioms.lean: axioms and computability of the load-bearing declarations **What it establishes.** Every stated theorem of the spine is proved with the kernel's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`), with no `sorryAx`. `Lean.isNoncomputable` is false for `piopExtractor`, `commitExtractor`, `Component.sendExtractor`, `Phases.Security.toDef`, `Component.Security.append`, ArkLib's `Extractor.RoundByRound.append`, `Component.passThroughExtractor`, `PublicInput.extractor` and `publicInputSecurity`, and true for `publicInputPhase` only (its error is a real number). `#print axioms` reports `Classical.choice` for computable definitions too, because the proofs inside them (the casts between witness types) use classical lemmas: it is no evidence of computability (D.5).

Source. `.claude/reports/blueprint-review/probes/code-spine/P1Axioms.lean`

```
import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose
import LeanerVM.Protocol.ToArkLib.Refinement
import LeanerVM.Protocol.PublicInput

/-!
Probe P1 (code-spine): axioms and computability of the spine's load-bearing declarations.
```

```

-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp

#print axioms piop_perfectCompleteness
#print axioms piop_rbrKnowledgeSoundness
#print axioms piop_rbrKnowledgeSoundness_exists
#print axioms piopExtractor
#print axioms commitExtractor
#print axioms Component.sendExtractor
#print axioms commitSecurity
#print axioms commitComplete
#print axioms Component.Security.append
#print axioms Component.Complete.append
#print axioms Phases.Security.toDef
#print axioms M3Holds
#print axioms Component.passThroughSecurity
#print axioms publicInputSecurity
#print axioms Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first
#print axioms Verifier.KnowledgeStateFunction.appendGuarded
#print axioms Extractor.RoundByRound.append
#print axioms Refinement.map_option_valid
#print axioms Toy.read_eval

open Lean Elab Command in
run_cmd do
  let env ← getEnv
  for n in [``piopExtractor, ``commitExtractor, ``Component.sendExtractor,
    ``Phases.Security.toDef, ``Phases.Complete.toDef, ``Component.Security.append,
    ``Component.Complete.append, ``Extractor.RoundByRound.append, ``leanVmPiop,
    ``leanVmVerifier, ``leanVmProver, ``piopError, ``Phases.toDef, ``Component.Def.append,
    ``commitDef, ``Component.sendOracle, ``Component.sendProver, ``Component.sendVerifier,
    ``publicInputPhase, ``publicInputSecurity, ``publicInputComplete, ``commitSecurity,
    ``commitComplete, ``Component.passThrough, ``Component.passThroughSecurity,
    ``Component.passThroughExtractor, ``PublicInput.extractor, ``PublicInput.prover,
    ``PublicInput.verifier, ``Verifier.KnowledgeStateFunction.appendGuarded,
    ``Component.guardedAppend, ``M3Holds, ``Toy.toy, ``Toy.honest,
    ``Extractor.Straightline.map, ``OracleReduction.append] do
    logInfo m!"{n}: noncomputable={Lean.isNoncomputable env n}"

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/PIAxioms.lean

```

Recorded output.

```

'LeanerVM.Protocol.piop_perfectCompleteness' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.piop_rbrKnowledgeSoundness' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.piop_rbrKnowledgeSoundness_exists' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.piopExtractor' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.commitExtractor' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Component.sendExtractor' depends on axioms: [propext, Quot.sound]
'LeanerVM.Protocol.commitSecurity' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.commitComplete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Component.Security.append' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Component.Complete.append' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Phases.Security.toDef' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.M3Holds' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Component.passThroughSecurity' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.publicInputSecurity' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first' depends on axioms:
↪ [propext,
  Classical.choice,
  Quot.sound]
'LeanerVM.Protocol.Verifier.KnowledgeStateFunction.appendGuarded' depends on axioms: [propext,
  Classical.choice,
  Quot.sound]
'Extractor.RoundByRound.append' depends on axioms: [propext, Classical.choice, Quot.sound]

```

```

'LeanerVM.Protocol.Refinement.map_option_valid' depends on axioms: [propext, Quot.sound]
'LeanerVM.Protocol.Toy.read_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
LeanerVM.Protocol.piopExtractor: noncomputable=false
LeanerVM.Protocol.commitExtractor: noncomputable=false
LeanerVM.Protocol.Component.sendExtractor: noncomputable=false
LeanerVM.Protocol.Phases.Security.toDef: noncomputable=false
LeanerVM.Protocol.Phases.Complete.toDef: noncomputable=false
LeanerVM.Protocol.Component.Security.append: noncomputable=false
LeanerVM.Protocol.Component.Complete.append: noncomputable=false
Extractor.RoundByRound.append: noncomputable=false
LeanerVM.Protocol.leanVmPiop: noncomputable=false
LeanerVM.Protocol.leanVmVerifier: noncomputable=false
LeanerVM.Protocol.leanVmProver: noncomputable=false
LeanerVM.Protocol.piopError: noncomputable=false
LeanerVM.Protocol.Phases.toDef: noncomputable=false
LeanerVM.Protocol.Component.Def.append: noncomputable=false
LeanerVM.Protocol.commitDef: noncomputable=false
LeanerVM.Protocol.Component.sendOracle: noncomputable=false
LeanerVM.Protocol.Component.sendProver: noncomputable=false
LeanerVM.Protocol.Component.sendVerifier: noncomputable=false
LeanerVM.Protocol.publicInputPhase: noncomputable=true
LeanerVM.Protocol.publicInputSecurity: noncomputable=false
LeanerVM.Protocol.publicInputComplete: noncomputable=false
LeanerVM.Protocol.commitSecurity: noncomputable=false
LeanerVM.Protocol.commitComplete: noncomputable=false
LeanerVM.Protocol.Component.passThrough: noncomputable=false
LeanerVM.Protocol.Component.passThroughSecurity: noncomputable=false
LeanerVM.Protocol.Component.passThroughExtractor: noncomputable=false
LeanerVM.Protocol.PublicInput.extractor: noncomputable=false
LeanerVM.Protocol.PublicInput.prover: noncomputable=false
LeanerVM.Protocol.PublicInput.verifier: noncomputable=false
LeanerVM.Protocol.Verifier.KnowledgeStateFunction.appendGuarded: noncomputable=false
LeanerVM.Protocol.Component.guardedAppend: noncomputable=false
LeanerVM.Protocol.M3Holds: noncomputable=false
LeanerVM.Protocol.Toy.toy: noncomputable=false
LeanerVM.Protocol.Toy.honest: noncomputable=false
LeanerVM.Protocol.Extractor.Straightline.map: noncomputable=false
OracleReduction.append: noncomputable=false
exit=0

```

P2Relation.lean: M3Holds on variants of the toy instance **What it establishes.** What the repository's tests leave out (D.1, D.4): the two multiplicity cases (`toyOnce`, a tuple pushed twice and pulled once; `toyNever`, pushed twice and never pulled, where a balance summed in the field accepts), each failing `Balanced` alone; the side filter (`toySwap`); the row ranges; the cells `CountsNonzero` ranges over; and a layout that reads every column from the same two cells (`toyAlias`, a `Layout`, on which the exhaustive guard finds no satisfying stack). The probe uses the numeral $(2 : K)$, which at the old `CompPoly` pin is the polynomial x and at the new pin is 0 ; it must be re-run with `K.ofBits 2` after a rebuild.

Source. `.claude/reports/blueprint-review/probes/code-spine/P2Relation.lean`

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose

/-!
Probe P2 (code-spine): `M3Holds` on variants of the toy instance.

* multiplicity: a tuple pushed twice and pulled once, a tuple pushed twice and never pulled;
* the side filter: the same instance with the two sides exchanged;
* the layout's freedom: every column read from the same two cells.
-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp

namespace Probe

/-! ## Multiplicity -/

/-- A boundary block of height one ( $\kappa = 0$ ): pulls the single tuple  $(1, 0, \dots)$ . -/
def boundaryOnce : BoundaryBlock shape where
   $\kappa := 0$ 

```

```

side := .pull
coords := Vector.ofFn fun k ↦ if k.val = 0 then .const 1 else .const 0

/-- The toy whose boundary pulls `(1, 0, ...)` once, where the table pushes it twice. -/
abbrev toyOnce : M3Instance := { toy with boundary := [boundaryOnce] }

/-- The toy with no boundary: the table pushes `(1, 0, ...)` twice and nothing is pulled. -/
abbrev toyNever : M3Instance := { toy with boundary := [] }

-- Pushed twice, pulled once: the two sides are the same *set* and differ as multisets.
#guard (toyOnce.tuples honest .push).length = 2
#guard (toyOnce.tuples honest .pull).length = 1
#guard (toyOnce.tuples honest .push).map (fun t ↦ t.get 0) = [1, 1]
#guard (toyOnce.tuples honest .pull).map (fun t ↦ t.get 0) = [1]
#guard (toyOnce.tuples honest .push).all fun t ↦ (toyOnce.tuples honest .pull).any fun u ↦
  t.toList == u.toList
#guard (toyOnce.tuples honest .pull).all fun t ↦ (toyOnce.tuples honest .push).any fun u ↦
  t.toList == u.toList
-- Only the balance clause fails.
#guard toyOnce.ConstraintsVanish honest
#guard ¬ toyOnce.Balanced honest
#guard toyOnce.CountsNonzero honest
#guard toyOnce.PublicLinesHold (1 : K) honest
#guard ¬ M3Holds toyOnce (1 : K) honest

-- Pushed twice, never pulled: every coordinate sums to zero in `K` on both sides (the sum of
-- the empty side is zero), so a balance summed in the field accepts; the multiset balance
-- rejects, and only it fails.
#guard (toyNever.tuples honest .push).length = 2
#guard (toyNever.tuples honest .pull).length = 0
#guard (List.finRange 16).all fun k ↦
  ((toyNever.tuples honest .push).map fun t ↦ t.get k).sum =
  ((toyNever.tuples honest .pull).map fun t ↦ t.get k).sum
#guard toyNever.ConstraintsVanish honest
#guard ¬ toyNever.Balanced honest
#guard toyNever.CountsNonzero honest
#guard toyNever.PublicLinesHold (1 : K) honest
#guard ¬ M3Holds toyNever (1 : K) honest

/-! ## The side filter -/

/-- The toy's flush, pulled instead of pushed. -/
def flushPull : Side × Vector (CMvPolynomial 3 K) 16 := (.pull, Toy.flush.2)

/-- The toy's boundary block, pushed instead of pulled. -/
def boundaryPush : BoundaryBlock shape := { Toy.boundary with side := .push }

/-- The toy with the two sides exchanged. -/
abbrev toySwap : M3Instance :=
{ toy with
  flushes := fun _ ↦ [flushPull]
  flushes_degree := fun _ f h k ↦ by
    rw [List.mem_singleton.mp h]
    exact toy.flushes_degree 0 Toy.flush (List.mem_singleton_self _) k
  boundary := [boundaryPush] }

#guard (toySwap.tuples honest .push).length = 2
#guard (toySwap.tuples honest .pull).length = 2
#guard (toySwap.flushTuples honest .push).length = 0
#guard (toySwap.flushTuples honest .pull).length = 2
#guard (toySwap.boundaryTuples honest .push).length = 2
#guard (toySwap.boundaryTuples honest .pull).length = 0
#guard M3Holds toySwap (1 : K) honest
#guard ¬ toySwap.Balanced ((#v[1, 0, 1, 1, 1, 0, 0, 0]) : Column 3)

/-! ## The row ranges -/

-- One pushed tuple per row of the table (2 rows), one pulled tuple per row of the block
-- (2 rows); a block of height one contributes one.
#guard (toy.flushTuples honest .push).length = 2 ^ toy.τ 0
#guard (toy.boundaryTuples honest .pull).length = 2 ^ Toy.boundary.κ
#guard (toyOnce.boundaryTuples honest .pull).length = 2 ^ boundaryOnce.κ

```

```

-- The count clause ranges over every row of the count column and over nothing else: a zero
-- in the padding (cells 6, 7) or in another column's cell does not fail it.
#guard toy.CountsNonzero honest
#guard toy.CountsNonzero ({#v[0, 0, 1, 1, 0, 0, 0, 0]}) : Column 3)
#guard ~ toy.CountsNonzero ({#v[1, 1, 0, 1, 1, 0, 1, 1]}) : Column 3)
#guard ~ toy.CountsNonzero ({#v[1, 1, 1, 0, 1, 0, 1, 1]}) : Column 3)

/-! ## The layout's freedom: every column read from the same cells -/

/-- Every column is the slice at cells `0, 1`. -/
def aliasSlice (q : Column 3) (c : Col) : Column 1 :=
  ({#v[q.values.get (0, by decide), q.values.get (1, by decide)]})

/-- Every column's selector is `(0, 0)`. -/
def aliasExtend (c : Col) (z : Vector E 1) : Vector E 3 := #v[z.head, 0, 0]

/-- The reading law holds of the aliased layout. -/
theorem alias_read_eval (q : Column 3) (c : Col) (z : Vector E 1) :
  CMLPolynomialEval.evalMle (aliasSlice q c).values (algebraMap K E) z =
    CMLPolynomialEval.evalMle q.values (algebraMap K E) (aliasExtend c z) := by
  simp [CMLPolynomialEval.evalMle, CMLPolynomialEval.evalMle,
    CMLPolynomialEval.evalMleValues, CMLPolynomialEval.evalMleStep, CMLPolynomialEval.map,
    Vector.head, Vector.tail, aliasExtend, aliasSlice]

/-- The aliased layout: a `Layout`, since the law is a reading law. -/
def aliasLayout : Layout 3 Col (fun _ => 1) where
  read := aliasSlice
  extend := aliasExtend
  read_eval := alias_read_eval

/-- The toy with every column read from cells `0, 1`. -/
abbrev toyAlias : M3Instance := { toy with layout := aliasLayout }

#guard (toyAlias.column honest (0, 0)).values.toList = (toyAlias.column honest (0, 2)).values.toList
#guard ~ M3Holds toyAlias (1 : K) honest
-- Balance needs cells `0, 1` to be `1, 1`; the public line needs cell `1` to be `0`.
#guard toyAlias.Balanced honest
#guard ~ toyAlias.PublicLinesHold (1 : K) honest
#guard toyAlias.PublicLinesHold (1 : K) ({#v[1, 0, 1, 1, 1, 0, 0, 0]}) : Column 3)
#guard ~ toyAlias.Balanced ({#v[1, 0, 1, 1, 1, 0, 0, 0]}) : Column 3)

-- Evidence (not a proof) that the relation of the aliased toy is empty: no stack with cells
-- `0, 1` in `{0, 1, 2}` satisfies it at a statement in `{0, 1, 2}`. On paper: balance forces
-- cells `0, 1` to be `1, 1`, and the public line on column 2, now cells `0, 1`, forces cell 1
-- to be `0`.
#guard ([0, 1, 2] : List K).all fun a => ([0, 1, 2] : List K).all fun b =>
  ([0, 1, 2] : List K).all fun s =>
    decide (~ M3Holds toyAlias s ({#v[a, b, 1, 1, 1, 0, 0, 0]}) : Column 3))

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P2Relation.lean

```

Recorded output.

```
exit=0
```

P3aSeams.lean: each seam, an inhabitant and its near misses **What it establishes.** Each seam on the honest toy stack has an inhabitant and each of its conjuncts a near miss (the table of D.2): among them a true cubic claim rejected by the degree conjunct of **Seam.bus**, the wrong statement inside **Seam.pub**, and **Seam.done** toy = **Set.univ**. Two facts of the design follow: a seam does not imply the relation before it (the zerocheck escape on **badConstraint**), and a seam does not say which claims are emitted. The repository gives the claims no decision procedure; the probe supplies them by **inferInstanceAs** on the unfolded propositions. It uses $(2 : K)$ and must be re-run with **K.ofBits 2** after a rebuild.

Source. `.claude/reports/blueprint-review/probes/code-spine/P3aSeams.lean`

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose
import LeanerVM.Protocol.PublicInput

/-!
Probe P3a (code-spine): each seam on the toy instance, an inhabitant and near misses; the
zerocheck escape; the commit phase's state function and extractor; the type of the composed
extractor's output when a phase has rounds.
-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp

namespace Probe

/-! ## Decision procedures for the claims and the seams (the repository has none)

Spelled with `inferInstanceAs` on the unfolded proposition, as the repository's public-input
test does: the instance `by unfold ...; infer_instance` elaborates and then exhausts the memory
under `#guard` (observed: exit 137 on the first version of this probe). -/

instance {I : M3Instance} (q : Column I.μ) (c : ColumnClaim I) : Decidable (c.Holds q) :=
  inferInstanceAs (Decidable (CMLPolynomialEval.eval₂Mle (I.column q c.col).values
    (algebraMap K E) c.point = c.value))

instance {I : M3Instance} (q : Column I.μ) (c : LinearClaim I) : Decidable (c.Holds q) :=
  inferInstanceAs (Decidable ((c.terms.map fun t => t.eval q).sum = c.value))

instance {I : M3Instance} (q : Column I.μ) (c : WeightedClaim I) : Decidable (c.Holds q) :=
  inferInstanceAs (Decidable (c.weight.pair q = c.value))

/-- The predicate of the bus seam, on a statement and a stack. -/
def busPred (I : M3Instance) (s : I.Stmt × BusOut I) (q : Column I.μ) : Prop :=
  (∀ c ∈ s.2.linear, c.Holds q) ∧
  (∀ c ∈ s.2.linear, ∀ t ∈ c.terms, t.poly.totalDegree ≤ I.d) ∧
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q

/-- It is the bus seam, by definition. -/
example (I : M3Instance) (s : I.Stmt × BusOut I) (o : ∀ i, TheOracle I i) :
  ((s, o), ()) ∈ Seam.bus I ↔ busPred I s (o 0) := Iff.rfl

instance (I : M3Instance) (s : I.Stmt × BusOut I) (q : Column I.μ) :
  Decidable (busPred I s q) :=
  inferInstanceAs (Decidable ((∀ c ∈ s.2.linear, c.Holds q) ∧
    (∀ c ∈ s.2.linear, ∀ t ∈ c.terms, t.poly.totalDegree ≤ I.d) ∧
    (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q))

def tablePred (I : M3Instance) (s : I.Stmt × TableOut I) (q : Column I.μ) : Prop :=
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧ I.aux q

example (I : M3Instance) (s : I.Stmt × TableOut I) (o : ∀ i, TheOracle I i) :
  ((s, o), ()) ∈ Seam.table I ↔ tablePred I s (o 0) := Iff.rfl

instance (I : M3Instance) (s : I.Stmt × TableOut I) (q : Column I.μ) :
  Decidable (tablePred I s q) :=
  inferInstanceAs (Decidable ((∀ c ∈ s.2.columns, c.Holds q) ∧ I.PublicLinesHold s.1 q ∧
    I.aux q))

def pubPred (I : M3Instance) (s : I.Stmt × PubOut I) (q : Column I.μ) : Prop :=
  (∀ c ∈ s.2.columns, c.Holds q) ∧ I.aux q

example (I : M3Instance) (s : I.Stmt × PubOut I) (o : ∀ i, TheOracle I i) :
  ((s, o), ()) ∈ Seam.pub I ↔ pubPred I s (o 0) := Iff.rfl

instance (I : M3Instance) (s : I.Stmt × PubOut I) (q : Column I.μ) :
  Decidable (pubPred I s q) :=
  inferInstanceAs (Decidable ((∀ c ∈ s.2.columns, c.Holds q) ∧ I.aux q))

def flockPred (I : M3Instance) (s : I.Stmt × FlockOut I) (q : Column I.μ) : Prop :=
  (∀ c ∈ s.2.columns, c.Holds q) ∧ ∀ c ∈ s.2.weighted, c.Holds q

example (I : M3Instance) (s : I.Stmt × FlockOut I) (o : ∀ i, TheOracle I i) :

```



```

((s, o), ()) ∈ Seam.flock I ↔ flockPred I s (o 0) := Iff.rfl

instance (I : M3Instance) (s : I.Smt × FlockOut I) (q : Column I.μ) :
  Decidable (flockPred I s q) :=
  inferInstanceAs (Decidable ((∀ c ∈ s.2.columns, c.Holds q) ∧ ∀ c ∈ s.2.weighted, c.Holds q))

/-! ## The claims of the probe -/

def eZero : E := 0
def eOne : E := 1
/-- A point of `E` outside `{0, 1}`: the image of `2 : K` (the polynomial `x`). -/
def eTwo : E := ofK 2

/-- Column 0 of the honest stack is `[1, 1]`: its extension is `1` everywhere. -/
def col0One : ColumnClaim toy := ({0, 0}, #v[eTwo], eOne)
def col0Zero : ColumnClaim toy := ({0, 0}, #v[eTwo], eZero)

/-- The zerocheck claim of the toy's constraint at the point `{0}`: value `0`. -/
def zeroAt0 : LinearClaim toy :=
  ({(eOne, 0, CMvPolynomial.X 2 * CMvPolynomial.X 2 - CMvPolynomial.X 2, #v[eZero])}, eZero)
/-- The same at the point `{1}`. -/
def zeroAt1 : LinearClaim toy :=
  ({(eOne, 0, CMvPolynomial.X 2 * CMvPolynomial.X 2 - CMvPolynomial.X 2, #v[eOne])}, eZero)
/-- The same with the wrong value. -/
def zeroWrong : LinearClaim toy :=
  ({(eOne, 0, CMvPolynomial.X 2 * CMvPolynomial.X 2 - CMvPolynomial.X 2, #v[eZero])}, eOne)
/-- A cubic term, whose claim is true of the honest stack (column 2 is `[1, 0]`, so `X₂³` is
`[1, 0]` and its extension at `{0}` is `1`). -/
def cubicTrue : LinearClaim toy :=
  ({(eOne, 0, CMvPolynomial.X 2 * CMvPolynomial.X 2 * CMvPolynomial.X 2, #v[eZero])}, eOne)

/-- The stack of the repository's test `badConstraint`: column 2 is `[2, 0]`, not Boolean. -/
def badConstraint : Column 3 := (#v[1, 1, 1, 1, 2, 0, 0, 0])

/-! ## The bus seam -/

-- An inhabitant: the zerocheck claim and a column claim, on the honest stack at statement 1.
#guard busPred toy ((1 : K), ({zeroAt0, zeroAt1, [col0One]}) honest
-- Near misses, one per conjunct.
#guard ¬ busPred toy ((1 : K), ({zeroWrong, [col0One]}) honest -- a false linear claim
#guard ¬ busPred toy ((1 : K), ({cubicTrue, [col0One]}) honest -- a true cubic claim
#guard cubicTrue.Holds honest -- (it is true)
#guard ¬ busPred toy ((1 : K), ({zeroAt0, [col0Zero]}) honest -- a false column claim
#guard ¬ busPred toy ((0 : K), ({zeroAt0, [col0One]}) honest -- the wrong statement

-- The zerocheck escape: on `badConstraint` the constraint is violated on the cube (row 0), so
-- the claim at the point `{0}` is false, and the claim at the point `{1}` is true. The bus seam
-- holds of a stack outside `M3Holds`; the bus phase's error is what pays for it.
#guard ¬ toy.ConstraintsVanish badConstraint
#guard ¬ zeroAt0.Holds badConstraint
#guard zeroAt1.Holds badConstraint
#guard busPred toy ((2 : K), ({zeroAt1, [col0One]}) badConstraint
#guard ¬ M3Holds toy (2 : K) badConstraint

-- A bus statement with no claim holds of every stack whose public line holds: the seam does
-- not say which claims are emitted.
#guard busPred toy ((2 : K), ([], [])) badConstraint

/-! ## The table and public-input seams -/

#guard tablePred toy ((1 : K), ([col0One]) honest
#guard ¬ tablePred toy ((1 : K), ([col0Zero]) honest
#guard ¬ tablePred toy ((0 : K), ([col0One]) honest
#guard pubPred toy ((1 : K), ([col0One]) honest
#guard ¬ pubPred toy ((1 : K), ([col0Zero]) honest
-- The public seam no longer sees the statement: the wrong statement is inside it.
#guard pubPred toy ((0 : K), ([col0One]) honest

/-! ## The Flock seam -/

/-- The weight that selects cell 0 of the stack. -/
def cell0Weight : Weight 3 where
  onCube := Vector.ofFn fun i ↦ if i.val = 0 then 1 else 0

```

```

mle := fun r ↦ CMLPolynomialEval.evalMle (Vector.ofFn fun i ↦ if i.val = 0 then 1 else 0) r
mle_eq := fun _ ↦ rfl

def w0One : WeightedClaim toy := (cell0Weight, e0One)
def w0Zero : WeightedClaim toy := (cell0Weight, eZero)

#guard flockPred toy ((1 : K), ([col0One], [w0One])) honest
#guard ¬ flockPred toy ((1 : K), ([col0One], [w0Zero])) honest
#guard ¬ flockPred toy ((1 : K), ([col0Zero], [w0One])) honest

/-! ## The last seam has no near miss: it is the whole set -/

example : Seam.done toy = Set.univ := Set.eq_univ_of_forall fun _ ↦ trivial

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P3aSeams.lean

```

Recorded output.

```

| exit=0

```

P3bCommit.lean: the commit phase’s state function and extractor **What it establishes.** Every statement by `Iff.rfl` or `rfl`: the commit phase’s knowledge state function is `M3Holds` of the candidate witness before the message and `M3Holds` of the message after it, whatever the candidate witness; both maps of its extractor return the message (D.6).

Source. `.claude/reports/blueprint-review/probes/code-spine/P3bCommit.lean`

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose
import LeanerVM.Protocol.PublicInput

/-!
Probe P3b (code-spine): each seam on the toy instance, an inhabitant and near misses; the
zerocheck escape; the commit phase's state function and extractor; the type of the composed
extractor's output when a phase has rounds.
-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp

namespace Probe

/-! ## The commit phase: state function and extractor -/

section Commit

variable {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp))

-- Before the message: `M3Holds` of the candidate witness.
example (s : K) (o : ∀ i, NoOracle i) (w : Column 3) :
  ((commitSecurity toy).kSF init impl).toFun 0 (s, o) default w ↦ M3Holds toy s w := Iff.rfl

-- After the message: `M3Holds` of the message, whatever the candidate witness.
example (s : K) (o : ∀ i, NoOracle i) (tr : (commitSpec toy).FullTranscript) (w : Column 3) :
  ((commitSecurity toy).kSF init impl).toFun (Fin.last 1) (s, o) tr w ↦
    M3Holds toy s (tr 0) := Iff.rfl

-- The extractor: both maps return the message.
example (s : K) (o : ∀ i, NoOracle i) (tr : (commitSpec toy).FullTranscript) :
  (commitExtractor toy).extractOut (s, o) tr () = tr 0 := rfl
example (s : K) (o : ∀ i, NoOracle i) (tr : (commitSpec toy).FullTranscript) (w : Column 3) :
  (commitExtractor toy).extractMid 0 (s, o) tr w = tr 0 := rfl

```

```

-- The commit phase's security is the one the composition starts from.
example : (commitSecurity toy).extractor = commitExtractor toy := rfl
example : (commitDef toy).n = 1 := rfl

end Commit

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P3bCommit.lean

```

Recorded output.

```

| exit=0

```

P3cExtractor.lean: the composed extractor when a phase has rounds **What it establishes.**

With the repository's own `publicInputPhase` in the bundle and four pass-through phases, the composed extractor's `extractOut` lives in the last phase's intermediate witness type (`Unit`); the stack is what `extractMid` returns at round 0, and there it is the transcript's first message whatever it is handed; the repository test's `rfl` statement is ill-typed for this bundle, which is the expected error of the output (D.5).

Source. `.claude/reports/blueprint-review/probes/code-spine/P3cExtractor.lean`

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose
import LeanerVM.Protocol.PublicInput

/-!
Probe P3c (code-spine): the type and the value of the composed extractor when a phase has
rounds. The last declaration is expected to fail.
-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp

namespace Probe

/-! ## The composed extractor when a phase has rounds -/

/-- The toy's phases with the real public-input phase (two rounds) and four pass-throughs. -/
noncomputable def realPub : Phases toy where
  bus := Phase.passThrough toy fun s => (s, ([], []))
  table := Phase.passThrough toy fun p => (p.1, ([]))
  pub := publicInputPhase toy
  flock := Phase.passThrough toy fun p => (p.1, ([], []))
  opening := Phase.passThrough toy fun _ => ()

example : realPub.toDef.n = 3 := rfl

section

variable (Sb : Phase.Security toy realPub.bus (Seam.commit toy) (Seam.bus toy))
  (St : Phase.Security toy realPub.table (Seam.bus toy) (Seam.table toy))
  (Sf : Phase.Security toy realPub.flock (Seam.pub toy) (Seam.flock toy))
  (So : Phase.Security toy realPub.opening (Seam.flock toy) (Seam.done toy))

/-- A security bundle whose public-input field is the repository's. -/
def realPubSecurity : realPub.Security := (Sb, St, publicInputSecurity toy, Sf, So)

-- The output slot of the composed extractor is the public-input phase's last intermediate
-- witness, `Unit`: it is not the stack.
example : (realPubSecurity Sb St Sf So).toDef.witMid (Fin.last 3) = Unit := rfl

-- The stack is what the composed extractor returns at round 0, from the transcript's first

```

```

-- message.
example : (realPubSecurity Sb St Sf So).toDef.witMid 0 = Column 3 := rfl

-- At round 0 the composed extractor returns the transcript's first message, whatever the
-- phases' extractors and whatever intermediate witness it is handed.
example (s : K)
  (tr : realPub.toDef.pSpec.Transcript (Fin.succ ((0, Nat.zero_lt_succ 2) : Fin 3)))
  (w : (realPubSecurity Sb St Sf So).toDef.witMid (Fin.succ ((0, Nat.zero_lt_succ 2) : Fin 3))) :
  (piopExtractor realPub (realPubSecurity Sb St Sf So)).extractMid
    ((0, Nat.zero_lt_succ 2) : Fin 3) (s, fun i : Fin 0 → i.elim0) tr w =
    (tr (0, Nat.zero_lt_succ 0) : Column 3) := rfl

-- EXPECTED TO FAIL: the repository's test statement, with the real public-input phase in the
-- bundle. The output slot is `Unit`, so the statement "`extractOut` returns the stack" does not
-- typecheck.
example (tr : realPub.toDef.pSpec.FullTranscript) :
  (piopExtractor realPub (realPubSecurity Sb St Sf So)).extractOut
    ((1 : K), fun i : Fin 0 → i.elim0) tr () = honest := rfl

end

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P3cExtractor.lean

```

Recorded output.

```

.claude/reports/blueprint-review/probes/code-spine/P3cExtractor.lean:58:49: error: Type mismatch
  honest
has type
  Column 3
but is expected to have type
  (realPubSecurity Sb St Sf So).toDef.witMid (Fin.last realPub.toDef.n)
exit=1

```

P4Junk.lean: five phases that check nothing, at declared error 1 **What it establishes.** Phases. Security toy is inhabited by five phases that draw a challenge, check nothing and declare error 1 (junkSecurity); both master theorems apply to them; piopError junkPhases i = 1 at each of the five challenges (piopError_junk). Nothing in the construction uses the toy (D.3 (b)).

Source. .claude/reports/blueprint-review/probes/code-spine/P4Junk.lean

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose
import LeanerVM.Protocol.ToArkLib.GuardedVerdict
import LeanerVM.Protocol.ToArkLib.KeepOracles

/-!
Probe P4 (code-spine): `Phases.Security` of the toy instance is inhabited by five phases that
check nothing, each with one wasted challenge whose declared error is `1`.

A phase `wasted f ε`: the verifier draws one challenge in `E`, ignores it, and maps the
statement by `f`. It is complete whenever `f` carries the input seam into the output seam
(the pass-through's completeness). With declared error `ε = 1` it is round-by-round knowledge
sound for *any* two seams: the state is the input seam before the challenge and `True` after
it, and a probability is at most `1`.
-/

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp
  OracleSpec ProtocolSpec
open scoped NNReal ENNReal

namespace Probe

```

```

variable (I : M3Instance) {StmtIn StmtOut : Type}

/-- One round: a challenge in `E`. -/
@[reducible]
def wSpec : ProtocolSpec 1 := {!v[.V_to_P], !v[E]}

instance : ∀ i, OracleInterface (wSpec.Message i)
| {0, h} => nomatch h

instance : ∀ i, SampleableType (wSpec.Challenge i)
| {0, _} => (inferInstance : SampleableType E)

/-- The prover receives the challenge and outputs the mapped statement. -/
def wProver (f : StmtIn → StmtOut) :
  OracleProver [] σ StmtIn (TheOracle I) Unit StmtOut (TheOracle I) Unit wSpec where
  PrvState := fun _ => (StmtIn × ∀ i, TheOracle I i) × Unit
  input := _root_.id
  receiveChallenge
  | {0, _} => fun st => pure fun _ => st
  sendMessage
  | {0, h} => nomatch h
  output := fun st => pure ((f st.1.1, st.1.2), ())

/-- The verifier ignores the challenge, queries nothing and maps the statement. -/
def wVerifier (f : StmtIn → StmtOut) :
  OracleVerifier [] σ StmtIn (TheOracle I) StmtOut (TheOracle I) wSpec where
  verify := fun s _ => pure (f s)
  outputOracle := .inl (keepOracles (TheOracle I) wSpec)

theorem wVerifier_run (f : StmtIn → StmtOut) (s : StmtIn) (o : ∀ i, TheOracle I i)
  (tr : wSpec.FullTranscript) :
  (wVerifier I f).toVerifier.run (s, o) tr = pure (f s, o) := by
  simp only [Verifier.run, OracleVerifier.toVerifier]
  rw [OracleVerifier.materializeOutput_of_keepOracles _ rfl]
  rfl

/-- The verifier accepts every transcript. -/
def wPure (f : StmtIn → StmtOut) : (wVerifier I f).toVerifier.PureForm where
  verify := fun p _ => (f p.1, p.2)
  verify_eq := fun {s, o} tr => wVerifier_run I f s o tr

/-- The phase, with a declared error `ε` on its one challenge. -/
noncomputable def wasted (f : StmtIn → StmtOut) (ε : ℝ≥0) : Phase.Def I StmtIn StmtOut where
  n := 1
  pSpec := wSpec
  red := (wProver I f, wVerifier I f)
  err := fun _ => ε

theorem wProver_run_support (f : StmtIn → StmtOut) (s : StmtIn) (o : ∀ i, TheOracle I i)
  (pr : wSpec.FullTranscript × (StmtOut × ∀ i, TheOracle I i) × Unit)
  (hpr : pr ∈ support ((wProver I f).run (s, o) ())) : pr.2 = ((f s, o), ()) := by
  have h0 : wSpec.dir 0 = .V_to_P := rfl
  simp only [Prover.run, Prover.runToRound, Fin.induction_one,
    Prover.processRound_of_dir_eq_V_to_P 0 h0] at hpr
  simp only [ChallengeIdx, Challenge, wProver, id_eq, liftM_pure, bind_pure_comp, map_pure,
    pure_bind, Functor.map_map, support_map, Set.mem_image] at hpr
  obtain {r, -, rfl} := hpr
  rfl

variable {relIn : Set ((StmtIn × ∀ i, TheOracle I i) × Unit)}
  {relOut : Set ((StmtOut × ∀ i, TheOracle I i) × Unit)}

/-- Completeness, under the pass-through's hypothesis. -/
noncomputable def wastedComplete (f : StmtIn → StmtOut) (ε : ℝ≥0)
  (hc : ∀ s o, ((s, o), ()) ∈ relIn → ((f s, o), ()) ∈ relOut) :
  Phase.Complete I (wasted I f ε) relIn relOut where
  outputPure := {_, fun _ => rfl}
  guarded := (wPure I f).toGuardedForm
  complete := fun init impl => by
    apply Reduction.perfectCompleteness_of_run_support
    intro stmtIn witIn hIn x hx
    obtain {s, o} := stmtIn

```

```

obtain (pr, hpr, rfl) :=
  Reduction.mem_support_run_of_guarded _ (wPure I f).toGuardedForm (s, o) witIn hx
have hout := wProver_run_support I f s o pr hpr
have hc' : (wPure I f).toGuardedForm.check (s, o) pr.1 = true := rfl
rw [if_pos hc']
refine (_, rfl, ?_, congrArg Prod.fst hout)
have hw : pr.2.2 = () := rfl
exact hw ▸ hc s o hIn

/-- The extractor keeps the trivial witness. -/
def wExtractor : Extractor.RoundByRound (OracleSpec.emptySpec.{0, 0})
  (StmtIn × ∀ i, TheOracle I i) Unit Unit wSpec (fun _ ↦ Unit) where
  eqIn := rfl
  extractMid := fun _ _ _ ↦ ()
  extractOut := fun _ _ _ ↦ ()

variable {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] σ (StateT σ ProbComp))

/-- A knowledge state function for ANY two seams: the input seam before the challenge, `True`
after it. No hypothesis on `f`, `relIn`, `relOut`. -/
def wState (f : StmtIn → StmtOut) :
  (wVerifier I f).toVerifier.KnowledgeStateFunction init impl relIn relOut
  (wExtractor I (StmtIn := StmtIn)) where
  toFun := fun m stmt _ ↦ if m.val = 0 then (stmt, ()) ∈ relIn else True
  toFun_empty := fun _ _ ↦ by simp
  toFun_next := fun m hm ↦ by
    fin_cases m
    exact absurd hm (by decide)
  toFun_full := fun _ _ _ ↦ by simp

/-- Round-by-round knowledge soundness at declared error `1`: a probability is at most one. -/
theorem w_rbr (f : StmtIn → StmtOut) :
  (wVerifier I f).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl relIn relOut
  (fun _ ↦ Unit) (wExtractor I) (wState I init impl f) (fun _ ↦ 1) := by
  intro stmtIn i tr
  simp

/-- The security half at declared error `1`, under the completeness hypothesis alone. -/
noncomputable def wastedSecurity (f : StmtIn → StmtOut)
  (hc : ∀ s o, ((s, o), ()) ∈ relIn → ((f s, o), ()) ∈ relOut) :
  Phase.Security I (wasted I f 1) relIn relOut where
  toComplete := wastedComplete I f 1 hc
  witMid := fun _ ↦ Unit
  extractor := wExtractor I
  kSF := fun init impl ↦ wState I init impl f
  rbr := fun init impl ↦ w_rbr I init impl f

/-! ## The toy's five phases, checking nothing -/

/-- Five phases that draw a challenge and drop every claim. -/
noncomputable def junkPhases : Phases toy where
  bus := wasted toy (fun s ↦ (s, ([], []))) 1
  table := wasted toy (fun p ↦ (p.1, ([]))) 1
  pub := wasted toy (fun p ↦ (p.1, ([]))) 1
  flock := wasted toy (fun p ↦ (p.1, ([], []))) 1
  opening := wasted toy (fun _ ↦ ()) 1

/-- `Phases.Security` of the toy is inhabited by them. -/
noncomputable def junkSecurity : junkPhases.Security where
  bus := wastedSecurity toy _ fun _ _ h ↦ (by simp, by simp, by simp, h.2.2.2.1, h.2.2.2.2)
  table := wastedSecurity toy _ fun _ _ h ↦ (by simp, h.2.2.2.1, h.2.2.2.2)
  pub := wastedSecurity toy _ fun _ _ h ↦ (by simp, h.2.2)
  flock := wastedSecurity toy _ fun _ _ _ h ↦ (by simp, by simp)
  opening := wastedSecurity toy _ fun _ _ _ ↦ trivial

example : junkPhases.toDef.n = 6 := rfl

/-- Their completeness halves. -/
noncomputable def junkComplete : junkPhases.Complete :=
  (junkSecurity.bus.toComplete, junkSecurity.table.toComplete, junkSecurity.pub.toComplete,
   junkSecurity.flock.toComplete, junkSecurity.opening.toComplete)

/-- Both master theorems hold of the junk protocol. -/

```

```

example {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  (leanVmPiop junkPhases).perfectCompleteness init impl (M3Rel toy) (Seam.done toy) :=
  piop_perfectCompleteness junkPhases junkComplete init impl

example {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  (leanVmVerifier junkPhases).toVerifier.rbrKnowledgeSoundnessWorstCase init impl (M3Rel toy)
  (Seam.done toy) (piopError junkPhases) :=
  piop_rbrKnowledgeSoundness_exists junkPhases junkSecurity init impl

/-- Appending keeps a constant declared error. -/
theorem append_err_const {S1 S2 S3 W1 W2 W3 : Type} {ι1 ι2 ι3 : Type} {O1 : ι1 → Type}
  {O2 : ι2 → Type} {O3 : ι3 → Type} [V i, OracleInterface (O1 i)]
  [V i, OracleInterface (O2 i)] [V i, OracleInterface (O3 i)]
  (D1 : Component.Def S1 O1 W1 S2 O2 W2) (D2 : Component.Def S2 O2 W2 S3 O3 W3) (c : ℝ≥0)
  (h1 : ∀ i, D1.err i = c) (h2 : ∀ i, D2.err i = c) : ∀ i, (D1.append D2).err i = c := by
  intro i
  change Sum.elim D1.err D2.err (ChallengeIdx.sumEquiv.symm i) = c
  cases ChallengeIdx.sumEquiv.symm i with
  | inl a => exact h1 a
  | inr b => exact h2 b

/-- Every challenge of the junk protocol carries the declared error `1`. -/
theorem piopError_junk (i : junkPhases.toDef.pSpec.ChallengeIdx) : piopError junkPhases i = 1 := by
  unfold piopError Phases.toDef
  refine append_err_const _ _ 1 (append_err_const _ _ 1 (append_err_const _ _ 1
    (append_err_const _ _ 1 (append_err_const _ _ 1 ?_ ?_) ?_) ?_) ?_ i
  · rintro (j, hj)
    fin_cases j
    exact absurd hj (by decide)
  all_goals intro _; rfl

#print axioms junkSecurity
#print axioms piopError_junk

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P4Junk.lean

```

Recorded output.

```

'Probe.junkSecurity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Probe.piopError_junk' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0

```

P5PassThrough.lean: the pass-through bus phase has no security **What it establishes.** The reflection hypothesis of `Phase.passThroughSecurity` is false at the bus seam (`not_reflects`), and the type `Phase.Security` of the pass-through bus phase is empty (`no_security`), where the repository’s test docstring only asserts it (D.3 (a)). The probe uses `(2 : K)` and must be re-run with `K.ofBits 2` after a rebuild.

Source. `.claude/reports/blueprint-review/probes/code-spine/P5PassThrough.lean`

```

import LeanerVM.Protocol.Spine.Toy
import LeanerVM.Protocol.Spine.Compose

/-!
Probe P5 (code-spine): the pass-through bus phase of the repository's test (`trivPhases.bus`)
has no `Phase.Security` against the spine's seams, on the toy instance.

`badConstraint` at statement `2` is outside `M3Holds` and the pass-through's output on it (no
claim) is inside the bus seam. A knowledge state function must be true at the end of an
accepting transcript and, the phase having no round, the end is the beginning, where the state
is the input seam.
-/

```

```

open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy CompPoly CPoly OracleComp
  OracleSpec ProtocolSpec

namespace Probe

/-- The test's stack `badConstraint`: column 2 is `[2, 0]`. -/
def badConstraint : Column 3 := (#v[1, 1, 1, 1, 2, 0, 0, 0])

/-- The statement map of `trivPhases.bus`: no claim. -/
def dropAll (s : K) : K × BusOut toy := (s, ([], []))

/-- Outside the relation. -/
theorem bad_not_m3Holds : ¬ M3Holds toy (2 : K) badConstraint := by decide +kernel

/-- Inside the bus seam, with no claim. -/
theorem bad_mem_bus :
  ((dropAll 2, fun _ : Fin 1 ↦ badConstraint), ()) ∈ Seam.bus toy :=
  (by simp [dropAll], by simp [dropAll], by simp [dropAll], by decide +kernel, trivial)

/-- The reflection hypothesis of `Phase.passThroughSecurity` is false at the bus seam. -/
theorem not_reflects : ¬ ∀ (s : K) (o : ∀ i, TheOracle toy i),
  ((dropAll s, o), ()) ∈ Seam.bus toy → ((s, o), ()) ∈ Seam.commit toy := fun h ↦
  bad_not_m3Holds (h 2 (fun _ ↦ badConstraint) bad_mem_bus)

/-- The empty implementation of the empty shared oracle. -/
def noImpl : QueryImpl [] (StateT Unit ProbComp) := fun t ↦ PEmpty.elim t

/-- No `Phase.Security` exists for the pass-through bus phase: not only the repository's
constructor fails, the type is empty. -/
theorem no_security (S : Phase.Security toy (Phase.passThrough toy dropAll) (Seam.commit toy)
  (Seam.bus toy)) : False := by
  have ksf := S.kSF (pure ()) noImpl
  let stmt : K × ∀ i, TheOracle toy i := (2, fun _ ↦ badConstraint)
  let tr : (Phase.passThrough toy dropAll).pSpec.FullTranscript := fun i ↦ Fin.elim0 i
  have htr : tr = (default : (Phase.passThrough toy dropAll).pSpec.Transcript 0) :=
    funext fun i ↦ Fin.elim0 i
  have hpos : Pr[fun stmtOut ↦ (stmtOut, ()) ∈ Seam.bus toy | OptionT.mk do
    (simulateQ noImpl ((Phase.passThrough toy dropAll).red.verifier.toVerifier.run stmt
      tr)).run' (← (pure () : ProbComp Unit))] > 0 := by
    have hrunk : (Phase.passThrough toy dropAll).red.verifier.toVerifier.run stmt tr =
      pure (dropAll 2, fun _ ↦ badConstraint) :=
      Component.passThroughVerifier_toVerifier_run (TheOracle toy) dropAll 2
      (fun _ ↦ badConstraint) tr
    rw [hrunk]
    change Pr[_ | OptionT.mk (do let st ← (pure () : ProbComp Unit); (simulateQ noImpl
      (OptionT.run (pure (dropAll 2, fun _ : Fin 1 ↦ badConstraint))))).run' st] > 0
    rw [OptionT.run_pure, simulateQ_pure]
    rw [gt_iff_lt, probEvent_pos_iff]
    refine ((dropAll 2, fun _ ↦ badConstraint), ?_, bad_mem_bus)
    simp
  have hfull := ksf.toFun_full stmt tr () hpos
  rw [htr] at hfull
  exact bad_not_m3Holds ((ksf.toFun_empty stmt _).mpr hfull)

#print axioms no_security
end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P5PassThrough.lean

```

Recorded output.

```

'Probe.no_security' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0

```


P6Surface.lean: three facts behind the audit surface and the transport What it establishes.

The three facts of D.7: every `Component.Def` over the empty shared oracle has a pure prover output (`outputPure_of_def`), so the field `outputPure` is redundant; the two witness universes of `Refinement` are independent (`#check @Refinement`); and the probabilistic transport of knowledge along a refinement holds in the event form, in three lines (`knowledge_transport`).

Source. `.claude/reports/blueprint-review/probes/code-spine/P6Surface.lean`

```
import LeanerVM.Protocol.ToArkLib.Component
import LeanerVM.Protocol.ToArkLib.Refinement

/-!
Probe P6 (code-spine): three facts behind section B's proposals and section C.7.

1. The field `Component.Complete.outputPure` holds of every component: the shared oracle is
   empty, so the prover's output makes no query. (ArkLib has this at the pin as the instance
   `Prover.instOutputIsPureEmpty`, `Composition/Sequential/NoAmbient.lean:39-42`, in a module
   leanerVM does not import and whose `.olean` is not built; it is restated here.)
2. The universes of `Refinement`: whether its two witness types may live in different
   universes (`Column μ : Type`, Clean's `EnsembleWitness : Type 1`). The last `example` of that
   section is the one that decides it.
3. Knowledge transports along a refinement at the same error, with the map applied in the
   event and the game, the provers and the extractor left alone.
-/

open OracleComp OracleSpec ProtocolSpec LeanerVM.Protocol
open scoped NNReal

namespace Probe

/-! ## 1. `outputPure` is automatic -/

/-- The result of a computation that has no possible oracle query (ArkLib
`NoAmbient.lean:25-27` at `dca90385`, restated). -/
def runEmpty {α : Type} : OracleComp [] → α → α
| .pure a => a
| .liftBind t _ => isEmptyElim t

/-- Every computation over the empty specification is its pure result (`NoAmbient.lean:30-35`,
restated). -/
theorem eq_pure_runEmpty {α : Type} (oa : OracleComp [] → α) : oa = pure (runEmpty oa) := by
  cases oa with
  | pure a => rfl
  | queryBind t _ => exact isEmptyElim t

/-- The field `outputPure`, for every component, from its definition alone. -/
theorem outputPure_of_def {StmtIn : Type} {i1 : Type} {OStmtIn : i1 → Type} {WitIn : Type}
  {StmtOut : Type} {i2 : Type} {OStmtOut : i2 → Type} {WitOut : Type}
  [V i1, OracleInterface (OStmtIn i1)] [V i2, OracleInterface (OStmtOut i2)]
  (D : Component.Def StmtIn OStmtIn WitIn StmtOut OStmtOut WitOut) :
  D.red.prover.OutputIsPure :=
  (fun st => runEmpty (D.red.prover.output st),
   fun st => eq_pure_runEmpty (D.red.prover.output st))

#print axioms outputPure_of_def

/-! ## 3. Knowledge transports along a refinement, in the event -/

variable {i1 : Type} {oSpec : OracleSpec i1} {StmtIn WitIn StmtOut WitOut : Type} {n : ℕ}
  {pSpec : ProtocolSpec n} [V i1, SampleableType (pSpec.Challenge i1)]
  {σ : Type} (init : ProbComp σ) (impl : QueryImpl oSpec (StateT σ ProbComp))

/-- If `E` extracts a witness of `R` except with probability `ε`, then the same extractor
followed by the map of a refinement from `R` to `S` yields a witness of `S` except with
probability `ε`. -/
theorem knowledge_transport {R : Set (StmtIn × WitIn)} {relOut : Set (StmtOut × WitOut)}
  {W2 : Type} {S : Set (StmtIn × W2)} (f : Refinement R S)
  {verifier : Verifier oSpec StmtIn StmtOut pSpec}
  {E : Extractor.Straightline oSpec StmtIn WitIn WitOut pSpec} {ε : ℝ≥0}
  (h : verifier.knowledgeSoundnessWith init impl R relOut E ε)
  (stmtIn : StmtIn) (witIn : WitIn)
  (prover : Prover oSpec StmtIn WitIn StmtOut WitOut pSpec) :
```

```

let pImpl : QueryImpl (oSpec + [pSpec.Challenge]o) (StateT σ ProbComp) :=
  impl.addLift challengeQueryImpl
let exec := do
  let <{transcript, (_, witOut)}, stmtOut>, proveQueryLog, verifyQueryLog)
  <- (Reduction.mk prover verifier).runWithLog stmtIn witIn
  let extractedWitIn? <-
    liftM (E stmtIn witOut transcript proveQueryLog.fst verifyQueryLog).run
  return (stmtIn, extractedWitIn?, stmtOut, witOut)
Pr[fun (stmtIn, extractedWitIn?, stmtOut, witOut) =>
  (∀ w ∈ extractedWitIn?, (stmtIn, f.map stmtIn w) ∉ S) ∧ (stmtOut, witOut) ∈ relOut
  | OptionT.mk do (simulateQ pImpl exec.run) (← init)] ≤ ε :=
le_trans (probEvent_mono fun _ _ hx =>
  (fun w hw hR => hx.1 w hw (f.map_valid _ _ hR), hx.2)) (h stmtIn witIn prover)

#print axioms knowledge_transport

/-! ## 2. The universes of `Refinement` -/

#check @Refinement

/-- A witness type in `Type 1`, as Clean's `EnsembleWitness` is. -/
structure Big : Type 1 where
  carrier : Type

/-- Decides the question: a refinement from a relation whose witness is in `Type` to one whose
witness is in `Type 1`. Accepted if the two witness universes of `Refinement` are independent,
rejected if `{W₁ W₂ : Type _}` gave them one universe. -/
example : Refinement (Stmt := Unit) {p : Unit × ℕ | True} {p : Unit × Big | True} where
  map := fun _ n => (Fin n)
  map_valid := fun _ _ => trivial

end Probe

```

Command. run from the repository root, under the shared lock:

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/code-spine/P6Surface.lean

```

Recorded output.

```

'Probe.outputPure_of_def' depends on axioms: [propext, Classical.choice, Quot.sound]
'Probe.knowledge_transport' depends on axioms: [propext, Classical.choice, Quot.sound]
@Refinement : {Stmt : Type} → {W₁ : Type u₁} → {W₂ : Type u₂} → Set (Stmt × W₁) → Set (Stmt × W₂) → Type (max
↪ u₁ u₂)
(two linter warnings on unused binder names, omitted)
exit=0

```

B.1.1.3 The public-input phase: mutation probes (C)

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-pubinput.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.2; B (faithfulness), E (the flag `PublicLine.sent`) and F (statement checks) is section 8.9.1.2; C (mutation probes) and D (the top limb) is ?? B.1.1.3; G (findings) is section 8.9.2.2.

Every probe of this part is a copy of the phase’s module with one mutation (C.0); the dossier reproduces the mutation and the declarations appended to the copy, and the rest of each file is the module at `b435631`. Each was run with the command of C.0 on its own file, from the repository root, under the shared lock; the probes marked “written, not run” were not compiled, because the environment was lost (C.9).

Method (C.0) How a probe is built. The phase’s module has private imports and private lemmas (`lineClaim_holds_iff`, `line_challenge_unique`, `pooled_mem_pub`, `bad_challenge_unique`, `simulateQ_queryValues`, `prover_run_support`), which a plain file cannot reach. So each probe is a **copy of the whole module, kept a module file**, under the namespace `LeanerVM.Protocol.Probe<k>`, with one mutation applied by exact string replacement, and with new declarations appended after the copied text.

The script is `probes/code-pubinput/tools/mutate.py`; the replacements of mutation `k` are `probes/code-pubinput/tools/m<k>.py`; the shared text (the two refutation lemmas, the pool that reads the message) is `probes/code-pubinput/tools/lib.py`. The probe does not import the original `LeanerVM.Protocol.PublicInput`.

The baseline. `Probe0Baseline.lean` is the module with only the namespace changed (the `diff` against the source is the two namespace lines). It compiles:

```
$ flock .claude/reports/blueprint-review/logs/lean.lock lake env lean \
  -D autoImplicit=false -D relaxedAutoImplicit=false \
  .claude/reports/blueprint-review/probes/code-pubinput/Probe0Baseline.lean
'LeanerVM.Protocol.Probe0.publicInputSecurity' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe0.publicInputComplete' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0
```

Every probe below was run with the same command on its own file, from the repository root, under the shared lock.

What “caught” means. For each mutation three things are reported.

- *Unchanged proofs:* the verifier is changed, the verdict lemma `verifier_verify` is restated for the verifier as changed (it describes what the verifier does), and `complete`, `stateFunction`, `rbr` are left as they are, except where a one-line adaptation is forced by the restated verdict. The first declaration that stops compiling is reported with the compiler’s message.
- *Repair:* whether another state function (or another honest prover) proves the theorem for the mutated verifier at the same error. If it does, the mutation is **not caught**: a developer who makes the mutation can still produce `Phase.Security`, and with it the master theorems.
- *Refutation:* where no repair exists, a Lean theorem that the mutated verifier has **no** round-by-round knowledge soundness at any error below one (whatever the extractor, the intermediate witness types and the state function), or is not perfectly complete. Then the mutation is **caught**.

The objects, for a reader who does not know ArkLib (pin `dca90385`).

- A *verifier* (`Verifier`) maps a statement and a full transcript to an optional output statement. An *oracle verifier* (`OracleVerifier`) does the same but reads the prover’s messages and the input oracles through queries; `toVerifier` turns it into a verifier by answering the queries.
- *Perfect completeness* of a reduction (a prover and a verifier) against an input relation and an output relation (`Reduction.perfectCompleteness`, `ArkLib/OracleReduction/Security/Basic.lean`): on every input in the input relation, with probability one the verifier accepts, its output is in the output relation with the prover’s output witness, **and the prover’s output statement equals the verifier’s** (`perfectCompleteness_eq_prob_one`, :164-175).
- A *round-by-round extractor* (`Extractor.RoundByRound`, `ArkLib/OracleReduction/Security/RoundByRound.lean`:80-91) maps witnesses backwards, round by round, from the output witness to the input witness. Here all witnesses are `Unit`, since the stack is the oracle.
- A *knowledge state function* (`Verifier.KnowledgeStateFunction`, same file :164-186) is a predicate on (round, statement, partial transcript, intermediate witness) with three laws: at round 0 it is the input relation (`toFun_empty`); across a **prover** message, if it holds after the message it held before (`toFun_next`); if the verifier can output a statement in the output relation, it holds of the full transcript (`toFun_full`).
- *Round-by-round knowledge soundness for a given extractor and state function* (`Verifier.rbrKnowledgeSoundnessWorstCaseWith`, same file :553-568): for every statement, every challenge round and every transcript prefix, the probability over the fresh challenge that the state goes from false to true is at most the error of that challenge. The existential form `rbrKnowledgeSoundnessWorstCase` (:537-551) says that some witness types, some extractor and some state function exist.

```
-- ArkLib dca90385, ArkLib/OracleReduction/Security/RoundByRound.lean:553-568
def rbrKnowledgeSoundnessWorstCaseWith
  (relIn : Set (StmtIn × WitIn)) (relOut : Set (StmtOut × WitOut))
  (verifier : Verifier oSpec StmtIn StmtOut pSpec)
  (witMid : Fin (n + 1) → Type)
  (extractor : Extractor.RoundByRound oSpec StmtIn WitIn WitOut pSpec WitMid)
```

```

    (kSF : verifier.KnowledgeStateFunction init impl relIn relOut extractor)
    (rbrKnowledgeError : pSpec.ChallengeIdx → ℝ≥0) : Prop :=
  ∀ stmtIn : StmtIn,
  ∀ i : pSpec.ChallengeIdx,
  ∀ transcript : Transcript i.1.castSucc pSpec,
  Pr[fun challenge =>
    ∃ witMid,
    ¬ kSF i.1.castSucc stmtIn transcript
      (extractor.extractMid i.1 stmtIn (transcript.concat challenge) witMid) ∧
      kSF i.1.succ stmtIn (transcript.concat challenge) witMid
    | $t (pSpec.Challenge i)] ≤ rbrKnowledgeError i

```

The verifier under test, as it stands (LeanerVM/Protocol/PublicInput.lean:126-137 and :229-234):

```

/-- What the verifier pools: the claims it received, then one claim per public line. -/
def pooled (s : I.Stmt × TableOut I) (r : E) : I.Stmt × PubOut I :=
  (s.1, (s.2.columns ++ (I.publicLines s.1).map (lineClaim I r)))

/-- The values the verifier expects in the prover's message: the values at the challenge of the
lines whose value is sent, in order. -/
def expectedValues (input : I.Stmt) (r : E) : List E :=
  ((I.publicLines input).filter (·.sent)).map (lineValue I r)

/-- The verifier's check: the prover's message is the expected values. -/
def check (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool :=
  decide (cs = expectedValues I s.1 r)

def verifier : OracleVerifier [] (I.Stmt × TableOut I) (TheOracle I)
  (I.Stmt × PubOut I) (TheOracle I) pSpec where
  verify := fun s chals => do
    let cs ← liftM queryValues
    if check I s (chals (0, rfl)) cs then pure (pooled I s (chals (0, rfl))) else failure
  outputOracle := .inl (keepOracles (TheOracle I) pSpec)

```

The observation everything below follows from: **pooled does not take cs**. The message enters the verifier in `check` and nowhere else, and the proof of `rbr` (PublicInput.lean:350-366) never mentions `check`.

The two refutation lemmas (shared by the probes) (C.1) They are stated for the phase's schedule `pSpec` (a challenge in `E`, then a message `List E`) and proved in each probe that uses them; source in `tools/lib.py`, first developed in the plain file `ScratchRefute.lean` against the real `LeanerVM.Protocol.PublicInput` (exit 0).

```

/-- The transcript `(r, cs)`, built as a knowledge state function reads it. -/
def tr2 (r : E) (cs : List E) : pSpec.FullTranscript :=
  Transcript.concat (m := (1 : Fin 2)) cs
  (Transcript.concat (m := (0 : Fin 2)) r (default : Transcript 0 pSpec))

/-- A guarded verifier on the schedule `pSpec` that, on one statement outside the input
relation, has for every challenge a message it accepts into the output relation is not
round-by-round knowledge sound at an error below one: whatever the extractor and the state
function. -/
theorem not_rbr {StmtIn StmtOut : Type} {relIn : Set (StmtIn × Unit)}
  {relOut : Set (StmtOut × Unit)}
  (V : Verifier [] StmtIn StmtOut pSpec) (G : V.GuardedForm)
  (impl : QueryImpl [] (StateT Unit ProbComp))
  (stmt : StmtIn) (hin : (stmt, ()) ∉ relIn) (msg : E → List E)
  (hacc : ∀ r, G.check stmt (tr2 r (msg r)) = true ∧
    (G.out stmt (tr2 r (msg r)), ()) ∈ relOut)
  (ε : pSpec.ChallengeIdx → ℝ≥0) (hε : ε (0, rfl) < 1) :
  ¬ V.rbrKnowledgeSoundnessWorstCase (pure ()) impl relIn relOut ε := by
  rintro (WitMid, ext, kSF, h)
  have hbound := h stmt (0, rfl) (default : Transcript 0 pSpec)
  -- Every challenge is a bad challenge.
  have hall : ∀ r : E, ∃ witMid,
    ¬ kSF (Fin.castSucc 0) stmt (default : Transcript 0 pSpec)
      (ext.extractMid 0 stmt
        (Transcript.concat (m := (0 : Fin 2)) r
          (default : Transcript 0 pSpec)) witMid) ∧

```

```

      kSF (Fin.succ 0) stmt (Transcript.concat (m := (0 : Fin 2)) r
        (default : Transcript 0 pSpec)) witMid := by
    intro r
    obtain (hc, hout) := hacc r
    have hpos : Pr[fun stmtOut => (stmtOut, ()) ∈ relOut
      | OptionT.mk do
        (simulateQ impl (V.run stmt (tr2 r (msg r))))).run'
        (← (pure () : ProbComp Unit))] > 0 := by
    have hv : V.run stmt (tr2 r (msg r)) = pure (G.out stmt (tr2 r (msg r))) := by
      have := G.verify_eq stmt (tr2 r (msg r))
      rw [if_pos hc] at this
      exact this
    rw [hv]
    change Pr[_ | OptionT.mk (do let st ← (pure () : ProbComp Unit); (simulateQ impl
      (OptionT.run (pure (G.out stmt (tr2 r (msg r)))) :
        OptionT (OracleComp [].) StmtOut))).run' st)] > 0
    rw [OptionT.run_pure, simulateQ_pure]
    rw [gt_iff_lt, probEvent_pos_iff]
    refine (G.out stmt (tr2 r (msg r)), ?_, hout)
    simp
    have hfull := kSF.toFun_full stmt (tr2 r (msg r)) () hpos
    have hnext := kSF.toFun_next 1 rfl stmt
      (Transcript.concat (m := (0 : Fin 2)) r (default : Transcript 0 pSpec))
      (msg r) _ hfull
    refine (_, ?_, hnext)
    intro h0
    exact hin ((kSF.toFun_empty stmt _).mpr h0)
  -- So the bad event has probability one.
  have hone : Pr[fun challenge : E => ∃ witMid,
    ¬ kSF (Fin.castSucc 0) stmt (default : Transcript 0 pSpec)
      (ext.extractMid 0 stmt
        (Transcript.concat (m := (0 : Fin 2)) challenge
          (default : Transcript 0 pSpec)) witMid) ∧
      kSF (Fin.succ 0) stmt (Transcript.concat (m := (0 : Fin 2)) challenge
        (default : Transcript 0 pSpec)) witMid | $t E] = 1 := by
    rw [probEvent_eq_one_iff]
    exact (by simp, fun r _ => hall r)
  have hle : (1 : ℝ≥0∞) ≤ ((ε (0, rfl) : ℝ≥0) : ℝ≥0∞) := hone ▸ hbound
  exact absurd (ENNReal.coe_lt_one_iff.mpr hε) (not_lt.mpr hle)

/-- A reduction none of whose outcomes, on one input in the input relation, satisfies the
completeness event is not perfectly complete. -/
theorem not_perfectCompleteness {ι : Type} {oSpec : OracleSpec ι}
  {StmtIn WitIn StmtOut WitOut : Type} {n : ℕ} {pSpec : ProtocolSpec n}
  [V ι, SampleableType (pSpec.Challenge ι)]
  {σ : Type} (init : ProbComp σ) (impl : QueryImpl oSpec (StateT σ ProbComp))
  {relIn : Set (StmtIn × WitIn)} {relOut : Set (StmtOut × WitOut)}
  (reduction : Reduction oSpec StmtIn WitIn StmtOut WitOut pSpec)
  (stmtIn : StmtIn) (witIn : WitIn) (hin : (stmtIn, witIn) ∈ relIn)
  (h : ∀ x ∈ support (reduction.run stmtIn witIn).run, ∀ result, x = some result →
    ¬ ((result.2, result.1.2.2) ∈ relOut ∧ result.1.2.1 = result.2)) :
  ¬ reduction.perfectCompleteness init impl relIn relOut := by
  intro hc
  rw [Reduction.perfectCompleteness_eq_prob_one] at hc
  have h1 := hc stmtIn witIn hin
  dsimp only at h1
  have hpos := lt_of_lt_of_eq (zero_lt_one' ℝ≥0∞) h1.symm
  obtain (x, hx, hev) := probEvent_pos_iff.mp hpos
  rw [OptionT.mem_support_iff, OptionT.run_mk, mem_support_bind_iff] at hx
  obtain (s, _, hx) := hx
  exact h (some x) (support_simulateQ_run'_subset _ _ s hx) x rfl hev

```

The first lemma is refutation in the strong sense: it denies ArkLib's **existential** `rbrKnowledgeSoundness WorstCase`, so no choice of extractor, witness types or state function helps, and it holds for every error below one, not only $1/|E|$. It is stated for one choice of the shared-oracle initial state (`pure ()`) and implementation; the phase's `Phase.Security` owes its bound for every choice, so one choice refutes it.

Results so far (C.2)

#	Mutation	complete	stateFunction/rbr unchanged	Knowledge sound by another state function?	Verdict
1	no check; lines' values pooled	compiles	stateFunction.toF un_full fails	yes, same error (rbr', publicInput Security')	not caught
2	no check; prover's values pooled	compiles	toFun_full fails	no: refuted (not_knowledgeSound, toy)	caught by knowledge soundness
3a	first value checked; lines' values pooled	compiles	toFun_full fails	yes, same error (rbr')	not caught
3b	first value checked; prover's values pooled	compiles	toFun_full fails	no: refuted (two lines sent)	caught by knowledge soundness
4a	the pinned verifiers' equation; lines' values pooled	compiles	toFun_full fails	yes, same error (rbr')	not caught
4b	the pinned verifiers' equation; prover's values pooled (the deployed verifier)	by C.5 and C.6's arguments		sound at $1/ E $ by a new argument; key lemma proved (C.13)	a sound verifier, not a mutation to catch
5	lines not sent are not pooled	compiles	toFun_full fails	no: refuted (a line not sent)	caught by knowledge soundness
any	any message, any check the message passes; lines' values pooled	proved (completeG)		yes, same error (rbrG, securityG)	not caught , in general
6	claims pooled at a wrong point (paper, C.12)	fails at lineClaim_ holds_iff	fails	no: both refuted on a height-4 instance	caught by completeness (unverified)
7a	wrong check, honest prover unchanged (written, not run, C.10)	fails at decide_eq_ true hmsg	compiles	(completeness refuted)	caught by completeness against a fixed prover (unverified)
7b	wrong check, prover adapted, lines' values pooled (run, C.4 swappedCheck)	proved		yes, same error	not caught
8	extra check (written, not run, C.11)	fails	compiles	(completeness refuted, for every prover)	caught by completeness (unverified)

In every probe the unchanged `stateFunction` fails with the same message, at its field `toFun_full`, because its last round names the original check and the original `pooled`:

```
error: Type mismatch
  Verifier.GuardedForm.of_probEvent_pos (guarded I) init impl stmt tr (fun stmtOut => (stmtOut, x+) ∈ Seam.pub
    ↪ I) h
has type
  (guarded I).check stmt tr = true ∧ ((guarded I).out stmt tr, x+) ∈ Seam.pub I
but is expected to have type
  if h0 : ↑(Fin.last 2) = 0 then (stmt, ()) ∈ Seam.table I
  else
    if h1 : ↑(Fin.last 2) = 1 then ((pooled I stmt.1 (tr {0, ↯})), stmt.2), () ∈ Seam.pub I
    else check I stmt.1 (tr {0, ↯}) (tr {1, ↯}) = true ∧ ((pooled I stmt.1 (tr {0, ↯})), stmt.2), () ∈ Seam.pub
    ↪ I
```

That failure alone says little: the state function is part of the proof, not of the statement (`Phase.Security` carries it as a field, `kSF`), so whoever changes the verifier changes it too. The decisive columns are the last two.

(Details of each probe: sections C.3 to C.13.)

Mutation 1: the check removed, the lines' values pooled. NOT CAUGHT (C.3) File `probes/code-pubinput/Probe1NoCheck.lean` (replacements in `tools/ml.py`).

The mutation (the verifier reads the message and ignores it), with the verdict lemma and the guarded form restated for it:

```
verify := fun s chals ↪ do
  let _cs ← liftM queryValues
  pure (pooled I s (chals {0, rfl}))

theorem verifier_verify (s : I.Stmt × TableOut I) (o : ∀ i, TheOracle I i)
  (tr : pSpec.FullTranscript) :
  (verifier I).toVerifier.verify (s, o) tr = pure (pooled I s (tr 0), o) := by
  -- the original proof, ending `rw [simulateQ_optionT_bind_run, simulateQ_queryValues, pure_bind]; rfl`
def guarded : (verifier I).toVerifier.GuardedForm where
  check := fun _ ↪ true
  out := fun p tr ↪ (pooled I p.1 (tr 0), p.2)
  verify_eq := fun (s, o) tr ↪ (verifier_verify I s o tr).trans (if_pos rfl).symm
```

complete compiles with one line adapted (`have hc : (guarded I).check (s, o) pr.1 = true := rfl`). The unchanged `stateFunction` fails at `toFun_full` (message in C.2; line 341 of the probe).

The repair, appended to the probe, is the state function with the check dropped from its last round; `rbr'` has the original proof of `rbr`, word for word:

```
def stateFunction' :
  (verifier I).toVerifier.KnowledgeStateFunction init impl (Seam.table I) (Seam.pub I)
  (extractor I) where
  toFun := fun m stmt tr _ ↪
    if h0 : m.val = 0 then ((stmt, ()) ∈ Seam.table I)
    else ((pooled I stmt.1 (tr {0, by omega})), stmt.2), () ∈ Seam.pub I
  toFun_empty := fun _ _ ↪ Iff.rfl
  toFun_next := fun m hm stmt tr msg w h ↪ by
    have hml : m = 1 := by
      fin_cases m
      · exact absurd hm (by decide)
      · rfl
    subst hml
    exact h
  toFun_full := fun stmt tr _ h ↪
    (Verifier.GuardedForm.of_probEvent_pos (guarded I) init impl stmt tr _ h).2

theorem rbr' :
  (verifier I).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (Seam.table I)
  (Seam.pub I) (fun _ ↪ Unit) (extractor I) (stateFunction' I init impl)
  (fun _ ↪ error) := by
  -- the original proof of `rbr`
```

```
def publicInputSecurity' (I : M3Instance) :
  Phase.Security I (publicInputPhase I) (Seam.table I) (Seam.pub I) where
  toComplete := publicInputComplete I
  witMid := fun _ => Unit
  extractor := PublicInput.extractor I
  kSF := PublicInput.stateFunction' I
  rbr := PublicInput.rbr' I
```

Output (exit 1 because of the one expected error):

```
ProbeNoCheck.lean:341:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe1.PublicInput.verifier_verify' depends on axioms: [propext, Classical.choice,
  ↳ Quot.sound]
'LeanerVM.Protocol.Probe1.publicInputComplete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe1.publicInputSecurity' depends on axioms: [propext, sorryAx, Classical.choice,
  ↳ Quot.sound]
'LeanerVM.Protocol.Probe1.publicInputSecurity'' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Reading. `publicInputSecurity'` is a complete `Phase.Security` of the verifier that checks nothing, against the spine's two seams, at the error $1/|E|$, on the kernel's three axioms. It fills the field `pub` of `Phases.Security`, so `piop_rbrKnowledgeSoundness` holds of a protocol whose public-input verifier accepts every message. This is not a defect of the theorem: the mutated verifier **is** knowledge sound. The prover's message in this phase is a function of public data; a verifier that recomputes it has no need to read it.

The general form: any message, any check the message passes. NOT CAUGHT (C.4) File `p robes/code-pubinput/ProbeAnyCheck.lean` (`tools/anycheck.py`): the module unchanged, and appended to it a phase with two parameters, the prover's message `msg` and the verifier's check `chk`.

```
variable (I : M3Instance)
  (msg : I.Stmt → E → List E)
  (chk : (I.Stmt × TableOut I) → E → List E → Bool)

/-- The prover sends `msg`. -/
def proverG : OracleProver [] (I.Stmt × TableOut I) (TheOracle I) Unit
  (I.Stmt × PubOut I) (TheOracle I) Unit pSpec where
  PrvState
  | (0, _) => ((I.Stmt × TableOut I) × ∀ i, TheOracle I i) × Unit
  | _ => E × ((I.Stmt × TableOut I) × ∀ i, TheOracle I i) × Unit
  input := _root_.id
  receiveChallenge
  | (0, _) => fun st => pure fun r => (r, st)
  | (1, h) => nomatch h
  sendMessage
  | (0, h) => nomatch h
  | (1, _) => fun st => pure (msg st.2.1.1 st.1, st)
  output := fun st => pure ((pooled I st.2.1.1 st.1, st.2.1.2), ())

/-- The verifier checks `chk` and pools the lines' values. -/
def verifierG : OracleVerifier [] (I.Stmt × TableOut I) (TheOracle I)
  (I.Stmt × PubOut I) (TheOracle I) pSpec where
  verify := fun s chals => do
    let cs ← liftM queryValues
    if chk s (chals (0, rfl)) cs then pure (pooled I s (chals (0, rfl))) else failure
  outputOracle := .inl (keepOracles (TheOracle I) pSpec)

/-- Perfect completeness, whatever the message and the check, as long as the one passes the
other. -/
theorem completeG (hchk : ∀ s r, chk s r (msg s.1 r) = true) {σ : Type} (init : ProbComp σ)
  (impl : QueryImpl [] (StateT σ ProbComp)) :
  (OracleReduction.mk (proverG I msg) (verifierG I chk)).perfectCompleteness init impl
  (Seam.table I) (Seam.pub I)

/-- Round-by-round knowledge soundness at `1/|E|`, whatever the check: no hypothesis on
`chk`. -/
theorem rbrG :
  (verifierG I chk).toVerifier.rbrKnowledgeSoundnessWorstCaseWith init impl (Seam.table I)
  (Seam.pub I) (fun _ => Unit) (extractor I) (stateFunctionG I chk init impl)
```



```

    (fun _ => error)

/-- Both halves, for every message and every check the message passes. -/
def securityG (I : M3Instance) (msg : I.Stmt → E → List E)
  (chk : (I.Stmt × TableOut I) → E → List E → Bool)
  (hchk : ∀ s r, chk s r (msg s.l r) = true) :
  Phase.Security I (phaseG I msg chk) (Seam.table I) (Seam.pub I)

/-- No check at all, and a prover that sends nothing. -/
def noCheckNoMessage (I : M3Instance) := securityG I (fun _ _ => []) (fun _ _ _ => true)
  fun _ _ => rfl

/-- No check at all, and a prover that sends forty-two zeros. -/
def noCheckJunk (I : M3Instance) :=
  securityG I (fun _ _ => List.replicate 42 0) (fun _ _ _ => true) fun _ _ => rfl

/-- The values of the lines with their cells swapped. -/
def swappedValues (I : M3Instance) (input : I.Stmt) (r : E) : List E :=
  ((I.publicLines input).filter (·.sent)).map fun l => (1 + r) * ofK l.cell1 + r * ofK l.cell0

/-- A wrong check, the cells swapped, with a prover that sends what the wrong check expects. -/
def swappedCheck (I : M3Instance) :=
  securityG I (swappedValues I) (fun s r cs => decide (cs = swappedValues I s.l r))
  fun _ _ => decide_eq_true rfl

/-- The specification's phase is the instance with the expected values and their check. -/
def specification (I : M3Instance) :=
  securityG I (PublicInput.expectedValues I) (PublicInput.check I) fun _ _ => decide_eq_true rfl

```

(The proofs are the module's own with `check` replaced by `chk` and `expectedValues` by `msg`; full text in the probe file.) Output, exit 0:

```

'LeanerVM.Protocol.ProbeAny.securityG' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.ProbeAny.noCheckNoMessage' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.ProbeAny.noCheckJunk' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.ProbeAny.swappedCheck' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.ProbeAny.specification' depends on axioms: [propext, Classical.choice, Quot.sound]

```

Reading. The two theorems of the phase constrain the *pool* (which claims, at which point, with which value) and nothing else. They do not constrain the prover's message (its length, its content), nor the check on it. The phase as merged is the instance `specification` of a family every member of which has the same theorems. In particular:

- mutation 1 (no check), mutation 3 in the form “first value only” and mutation 4 in the form “the pinned verifiers' equation” are members (each also run on its own, C.5);
- mutation 7 (a wrong check) is a member **when the honest prover is changed with it** (`swappedCheck`). Completeness is a statement about the pair (prover, verifier), and nothing fixes the honest prover's message but the verifier's check. So a wrong check on this message is caught by completeness only if the prover is held fixed (C.10).

Mutations 3a and 4a: a weaker check, the lines' values pooled. NOT CAUGHT (C.5) Files `Probe3a.lean`, `Probe4a.lean` (`tools/m3a.py`, `tools/m4a.py`, shared `tools/weak.py`). The verifier's check is replaced by `checkWeak`; `check` stays defined, since the unchanged state function names it.

```

/-- The weakened check: the first value only. -/ -- mutation 3a
def checkWeak (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool :=
  decide (cs.head? = (expectedValues I s.l r).head?)

/-- The pinned verifiers' check: with two lines whose value is sent, one equation on the two
values, `c₀ + y·c₁ = e₀ + y·e₁`; with any other number, the check per value. -/ -- mutation 4a
def checkWeak (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool :=
  match (I.publicLines s.l).filter (·.sent), cs with
  | [l₀, l₁], [c₀, c₁] => decide (c₀ + y * c₁ = lineValue I r l₀ + y * lineValue I r l₁)
  | ls, cs => decide (cs = ls.map (lineValue I r))

```

In both, `complete` compiles after the honest message is shown to pass the weak check (one line in 3a, a case split in 4a), the unchanged `stateFunction` fails at `toFun_full` (line 354, line 365), and the state function with `checkWeak` in its last round gives `rbr'` by the original proof. Output of both (exit 1, the one expected error each):

```
Probe3a.lean:354:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe3a.PublicInput.verifier_verify' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe3a.PublicInput.complete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe3a.PublicInput.stateFunction' depends on axioms: [propext, sorryAx, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe3a.PublicInput.rbr' depends on axioms: [propext, sorryAx, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe3a.PublicInput.rbr'' depends on axioms: [propext, Classical.choice, Quot.sound]

Probe4a.lean:365:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe4a.PublicInput.verifier_verify' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe4a.PublicInput.complete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe4a.PublicInput.stateFunction' depends on axioms: [propext, sorryAx, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe4a.PublicInput.rbr' depends on axioms: [propext, sorryAx, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe4a.PublicInput.rbr'' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Mutation 2: no check, the prover's values pooled. CAUGHT by knowledge soundness (C.6)
File `Probe2TrustProver.lean` (`tools/m2.py`). This is the verifier that trusts the prover. The pool that reads the message (shared by probes 2, 3b and 4b, in `tools/lib.py`):

```
-- The claims on the lines, a line whose value is sent taking the next value of the message
(zero when the message is too short), the others the value the verifier computes. -/
def claimsFrom (r : E) : List (PublicLine I.toShape) → List E → List (ColumnClaim I)
| [], _ => []
| l :: ls, cs =>
  if l.sent then (l.col, linePoint l.pos r, cs.headD 0) :: claimsFrom r ls cs.tail
  else lineClaim I r l :: claimsFrom r ls cs

-- The pool, with the values the prover sent. -/
def pooledFrom (s : I.Smt × TableOut I) (r : E) (cs : List E) : I.Smt × PubOut I :=
  (s.1, (s.2.columns ++ claimsFrom I r (I.publicLines s.1) cs))

-- On the expected values, the claims are the lines' claims. -/
theorem claimsFrom_expected (r : E) (ls : List (PublicLine I.toShape)) :
  claimsFrom I r ls ((ls.filter (·.sent)).map (lineValue I r)) = ls.map (lineClaim I r)

-- On the expected values, the pool is the pool of the lines' values. -/
theorem pooledFrom_expected (s : I.Smt × TableOut I) (r : E) :
  pooledFrom I s r (expectedValues I s.1 r) = pooled I s r
```

The mutation:

```
verify := fun s chals => do
  let cs ← liftM queryValues
  pure (pooledFrom I s (chals (0, rfl)) cs)
```

`complete` compiles (the honest message makes the two pools equal, by `pooledFrom_expected`). The unchanged `stateFunction` fails at `toFun_full` (line 376). No repair exists: appended to the probe, on the toy instance, with the stack `badLine` whose column 2 is `[1, 1]` where the statement 1 says `(1, 0)`:

```
abbrev badLine : Column 3 := (#v[1, 1, 1, 1, 1, 1, 0, 0])

abbrev badSmt : (K × TableOut toy) × (∀ i, TheOracle toy i) := (((1 : K), ([])), fun _ => badLine)

-- The wrong stack is outside the table seam. -/
theorem badSmt_not_mem : (badSmt, ()) ∉ Seam.table toy := by
  rintro (-, hl, -)
  have h := (hl _ (List.mem_singleton_self _)).2
  exact one_ne_zero h
```

```

/-- What a truthful prover sends: the extension of its column 2 at `(r)`. -/
def trueValue (r : E) : E :=
  CMLPolynomialEval.eval:Mle (toy.column badLine (0, 2)).values (algebraMap K E)
  (linePoint (n := 1) (by decide) r)

/-- The mutated verifier accepts it at every challenge, with a pool that holds of the wrong
stack. -/
theorem accepts (r : E) :
  (guarded toy).check badStmt (tr2 r [trueValue r]) = true ∧
  ((guarded toy).out badStmt (tr2 r [trueValue r]), ()) ∈ Seam.pub toy := by
  refine (rfl, ?_, trivial)
  intro c hc
  change c ∈ [({(0, 2), linePoint (n := 1) (by decide) r, trueValue r) : ColumnClaim toy}] at hc
  rw [List.mem_singleton] at hc
  subst hc
  rfl

/-- No extractor, no state function and no error below one make the mutated verifier
round-by-round knowledge sound against the two seams. -/
theorem not_knowledgeSound (ε : pSpec.ChallengeIdx → ℝ≥0) (hε : ε {0, rfl} < 1) :
  ¬ (verifier toy).toVerifier.rbrKnowledgeSoundnessWorstCase (pure ()) noOracle
  (Seam.table toy) (Seam.pub toy) ε :=
  not_rbr _ (guarded toy) noOracle badStmt badStmt_not_mem (fun r ↦ [trueValue r]) accepts ε hε

```

Output (exit 1, the one expected error):

```

Probe2TrustProver.lean:376:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe2.PublicInput.verifier_verify' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe2.PublicInput.complete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe2.PublicInput.stateFunction' depends on axioms: [propext, sorryAx, Classical.choice,
↪ Quot.sound]
'LeanerVM.Protocol.Probe2.PublicInput.rbr' depends on axioms: [propext, sorryAx, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe2.PublicInput.not_knowledgeSound' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]

```

Reading. With the prover's values pooled, the check is what ties the pool to the statement, and its removal is caught: there is no `Phase.Security`. This is the design of the three pinned verifiers and of the build the earlier review read.

Mutation 3b: first value checked, the prover's values pooled. CAUGHT by knowledge soundness (C.7) File `Probe3b.lean` (`tools/m3b.py`). Verifier: if `checkWeak ...` then `pure (pooledFrom ...)` else `failure`, with `checkWeak` of mutation 3a. `complete` compiles; unchanged `stateFunction` fails at `toFun_full` (line 390). Refuted on an instance with two lines whose value is sent, by a prover that sends the right first value and the true extension of its wrong second column:

```

abbrev twoSent : M3Instance :=
  { toy with publicLines := fun v ↦
    [({(0, 0), v, 1, true, by decide}), ({(0, 1), 1, 1, true, by decide})] }

/-- Column 0 is `[1, 1]`, as the statement `1` says; column 1 is `[1, 0]`, not `[1, 1]`. -/
abbrev badSecond : Column 3 := {#v[1, 1, 1, 0, 1, 0, 0, 0]}

def cheat (r : E) : List E := [(1 + r) * ofK 1 + r * ofK 1, trueValue 1 r]

theorem accepts (r : E) :
  (guarded twoSent).check badStmt (tr2 r (cheat r)) = true ∧
  ((guarded twoSent).out badStmt (tr2 r (cheat r)), ()) ∈ Seam.pub twoSent

theorem not_knowledgeSound (ε : pSpec.ChallengeIdx → ℝ≥0) (hε : ε {0, rfl} < 1) :
  ¬ (verifier twoSent).toVerifier.rbrKnowledgeSoundnessWorstCase (pure ()) noOracle
  (Seam.table twoSent) (Seam.pub twoSent) ε

```

```

Probe3b.lean:390:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe3b.PublicInput.complete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe3b.PublicInput.not_knowledgeSound' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]

```

Mutation 5: the claim of a line whose value is not sent is not pooled. CAUGHT by knowledge soundness (C.8) File `Probe5.lean` (`tools/m5.py`). The pool keeps only the lines whose value is sent (for leanISA: the claim on `mem_2` is dropped); prover and verifier output the same pool; the check is the original.

```
-- The mutated pool: the claims received, then one claim per line whose value is sent. -/
def pooledSent (s : I.Stmt × TableOut I) (r : E) : I.Stmt × PubOut I :=
  (s.1, {s.2.columns ++ ((I.publicLines s.1).filter (·.sent)).map (lineClaim I r)})
```

`complete` compiles (a sub-list of true claims). Unchanged `stateFunction` fails at `toFun_full` (line 359). Refuted on the toy with its one line not sent: the honest empty message passes the check, the pool is empty, and the wrong stack is accepted at every challenge.

```
abbrev noneSent : M3Instance :=
  { toy with publicLines := fun v ↦ [({0, 2}, v, 0, false, by decide)] }

theorem accepts (r : E) :
  (guarded noneSent).check badStmt (tr2 r []) = true ∧
  ((guarded noneSent).out badStmt (tr2 r []), ()) ∈ Seam.pub noneSent

theorem not_knowledgeSound (ε : pSpec.ChallengeIdx → ℝ≥0) (hε : ε {0, rfl} < 1) :
  ¬ (verifier noneSent).toVerifier.rbrKnowledgeSoundnessWorstCase (pure ()) noOracle
  (Seam.table noneSent) (Seam.pub noneSent) ε
```

```
Probe5.lean:359:4: error: Type mismatch [the message of C.2]
'LeanerVM.Protocol.Probe5.PublicInput.complete' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Probe5.PublicInput.not_knowledgeSound' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
```

Reading. Given an instance that lists the line, dropping its claim is caught. Whether the instance lists it is another matter (section D).

The probe environment was lost at 09:18 on 2026-09-30 (read this before trusting “not run”) (C.9) Between the pause and the restart the review branch received the merge 8d3ea7d of main with 144c5aa chore(deps): upgrade to Lean 4.34.1 (#61), which changes lean-toolchain (v4.33.1 to v4.34.1) and lake-manifest.json (ArkLib dca90385 to 7653a901, VCVio f9dc47d9 to a4232d08, CompPoly 3468b38c to 572f9973, Clean 93c9d1ef to 42fe4b26, Mathlib 0df444a3 to d13f23b7, PolyFun c0c92369 to 41d3b21d at a new URL). The first `lake env lean` after that (mine, probe 7a) made Lake re-resolve the manifest: it checked every package out at its new revision and **deleted and re-cloned PolyFun**, whose oleans at the old revision are gone (`find / -name PolyFun.olean` finds nothing). The LeanerVM oleans under `.lake/build` were built by 4.33.1 and are refused by 4.34.1 (“incompatible header”); run with the 4.33.1 binary and a hand-made `LEAN_PATH` over the surviving package builds, they fail on unknown module prefix ‘PolyFun’. So from that moment **no probe can be compiled against the pins without a rebuild**, which the rules forbid. The coordinator’s restart message said “nothing else changed”; the merge contradicts it, and it invalidated `lake env` as the way to run probes for every agent (the brief’s section 8, added afterwards, records the change and asks for no Lean run). I edited no tracked file; the change to `.lake/packages` is Lake’s own reaction to the merged manifest. (The library sources under `.lake/packages/` are now at the new revisions too: every library citation in this dossier was read before 09:18, or is read with `git show <pin>:<path>` inside the package’s repository.)

Probes 1, 2, 3a, 3b, 4a, 5, `ProbeAnyCheck`, `ProbeWordsLemma` and `ScratchRefute` ran before the change, and their outputs above are as printed. Probes 7a and 8 are written (files `Probe7a.lean`, `Probe8.lean`, replacements `tools/m7a.py`, `tools/m8.py`) and **not run**; probe 6 and the deployed-verifier phase (4b) are argued on paper below and not written. Each is marked. Where a conclusion below rests on an unrun probe, the conclusion is stated as what the probe is written to show, with the reason it should compile, and is flagged **unverified**.

Mutation 7: a wrong check (the cells swapped). Caught by completeness ONLY IF the honest prover is held fixed; probe 7a written, not run (C.10) *With the prover changed with the check:* the probe `ProbeAnyCheck` (C.4, run) proves `swappedCheck I : Phase.Security I (phaseG I (swa`

ppedValues I) (fun s r cs ↦ decide (cs = swappedValues I s.1 r))) (Seam.table I) (Seam.pub I). Both halves hold. **Not caught.**

With the honest prover unchanged (file `Probe7a.lean`, **not run**): the verifier checks `checkWrong`, the prover sends `expectedValues`, the lines' values are pooled.

```

/-- The values of the lines with their cells swapped: `(1 + r)·cell1 + r·cell0`. -/
def swappedValues (input : I.Stmt) (r : E) : List E :=
  ((I.publicLines input).filter (·.sent)).map fun l ↦ (1 + r) * ofK l.cell1 + r * ofK l.cell0

/-- The wrong check: the message is the swapped values. -/
def checkWrong (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool :=
  decide (cs = swappedValues I s.1 r)

```

Expected: complete fails at `have hc : (guarded I).check (s, o) pr.1 = true := decide_eq_true hm sg (hmsg : pr.1 1 = expectedValues I s.1 (pr.1 0))`, but the check asks for `swappedValues`; `rbr` compiles (it never reads the check). The refutation written in the probe, on the toy at the statement 1 with the honest stack (`honest`, column 2 = `[1, 0]`): the honest message is `[(1 + r)·1 + r·0]`, the swapped value is `[(1 + r)·0 + r·1]`, and `1 + r = r` gives `1 = 0`; so the check rejects at every challenge, every outcome of the run is none, and `not_perfectCompleteness (C.1, proved)` concludes:

```

theorem honest_rejected (r : E) : checkWrong toy honestStmt.1 r (expectedValues toy 1 r) = false := by
  apply decide_eq_false
  intro h
  change [(1 + r) * ofK 1 + r * ofK 0] = [(1 + r) * ofK 0 + r * ofK 1] at h
  have h1 : ofK (1 : K) = 1 := map_one (algebraMap K E)
  have h0 : ofK (0 : K) = 0 := map_zero (algebraMap K E)
  rw [h0, h1, List.cons.injEq] at h
  have : (1 : E) = 0 := by linear_combination h.1
  exact one_ne_zero this

/-- Not perfectly complete, for every initial state and implementation of the shared oracle. -/
theorem not_complete {σ : Type} (init : ProbComp σ) (impl : QueryImpl [] (StateT σ ProbComp)) :
  ¬ (OracleReduction.mk (prover toy) (verifier toy)).perfectCompleteness init impl
  (Seam.table toy) (Seam.pub toy) := by
  refine not_perfectCompleteness init impl _ honestStmt () honestStmt_mem ?_
  intro x hx result hres
  obtain (pr, hpr, rfl) := Reduction.mem_support_run_of_guarded _ (guarded toy) honestStmt () hx
  obtain (hmsg, -) := prover_run_support toy honestStmt.1 honestStmt.2 pr hpr
  have hc : (guarded toy).check honestStmt pr.1 = false := by
    show checkWrong toy honestStmt.1 (pr.1 0) (pr.1 1) = false
    rw [hmsg]
    exact honest_rejected (pr.1 0)
  rw [hc] at hres
  exact absurd hres (by simp)

```

Reading. “A wrong check makes completeness unprovable” holds against a fixed honest prover. Nothing in the phase, the spine or the blueprint fixes the honest prover’s message except the verifier’s own check; the blueprint’s acceptance tests contain no transcript-level fixture for this phase (the dumped-proof fixture is Layer 12’s). So a wrong check together with a matching prover passes both theorems; only a differential test against the Rust prover’s transcript (Layer 12’s fixture) would reject it.

With the prover’s values pooled (the deployed design, not probed): a wrong check is caught whatever the prover, since the check is then what makes the pooled claims true of an honest stack: with the honest prover the check rejects; with the prover adapted to the wrong check the pooled claims carry the swapped values and are false of the honest stack (whenever `cell0 ≠ cell1`), so completeness fails either way.

Mutation 8: an extra check leanVM does not make. Caught by completeness; written, not run (C.11) File `Probe8.lean` (**not run**). The verifier requires, on top of the check, that the message’s first value be nonzero:

```

/-- The check with an extra condition: the first value of the message is nonzero. -/
def checkExtra (s : I.Stmt × TableOut I) (r : E) (cs : List E) : Bool :=
  check I s r cs && decide (cs.headD 0 ≠ 0)

```

Expected: `complete` fails at the same line as in 7a (the honest message passes `check` but `decide_eq_true hmsg` does not prove `checkExtra`); `rbr` compiles. The refutation written in the probe: the toy at the statement `0`, whose line has cells `(0, 0)`, with the honest stack `honest0 := (#v[1, 1, 1, 1, 0, 0, 0, 0])`; the line's value is $(1 + r) \cdot 0 + r \cdot 0 = 0$ at every challenge, so the extra condition fails on the honest message, and (since `check` pins the message to that value) on every message: no prover passes, honest or adapted. So this mutation is caught **regardless of the prover**, unlike mutation 7.

```
theorem honest_rejected (r : E) :
  checkExtra toy honestStmt.1 r (expectedValues toy 0 r) = false := by
  unfold checkExtra
  rw [Bool.and_eq_false_iff]
  right
  apply decide_eq_false
  intro h
  apply h
  change (1 + r) * ofK 0 + r * ofK 0 = 0
  have h0 : ofK (0 : K) = 0 := map_zero (algebraMap K E)
  rw [h0, mul_zero, mul_zero, add_zero]
```

Mutation 6: the claims pooled at a wrong point. Caught by completeness (and by knowledge soundness); argued on paper, not written (C.12) Argued on paper; not written. Change `lineClaim` to pool at `(r, 1, 0, ..., 0)` (or any point other than `(r, 0, ..., 0)`), keeping `lineValue`.

Where the proofs stop. `lineClaim_holds_iff` (PublicInput.lean:141-148) rewrites with `evalMle_linePoint l.pos`, whose left side is the extension **at linePoint**; with another point the rewrite finds no instance and the lemma is unprovable (its statement is false: the extension at `(r, 1, 0, ...)` is the line through cells 2 and 3, by the selection identity `evalMle_append_boolVec` at index 1, ToCompPoly/Multilinear.lean:342-348). Everything downstream fails: `pooled_mem_pub` and so `complete`; `bad_challenge_unique` and so `rbr`.

Completeness is refuted, on an instance with a column of height 4 (log-height 2, so that a second coordinate exists; the toy's columns have height 2, on which every point `(r)` is the line point, so the toy cannot witness this mutation): one line with cells `(0, 0)`, the honest stack `[0, 0, 1, 1]`. It is in the table seam. The pooled claim has value `lineValue = 0` and the extension at `(r, 1)` is $(1 - r) \cdot 1 + r \cdot 1 = 1 \neq 0$ at every challenge; so the verifier's output is outside `Seam.pub` on every outcome and `not_perfectCompleteness` applies. *Knowledge soundness is refuted too*: the stack `[1, 1, 0, 0]` violates the line and its claims at `(r, 1)` hold with value `0` at every challenge (`not_rbr`). Both refutations need the evaluation of a concrete two-variable table at `(r, 1)` with `r` symbolic, which the toy's `read_eval` (Spine/Toy.lean:52-57) shows `simp` can do on tables of this size. **Unverified** in Lean.

Mutation 4b: the deployed verifier (the words' equation, the prover's values pooled). Sound at $1/|E|$, by a different argument; the key lemma run, the phase not written (C.13) The deployed verifier is the mutation "4a plus 2": the check `checkWeak` of C.5 (mutation 4a) and the pool `pooledFrom` of C.6. Its completeness holds by the argument of C.6 (the honest message makes `pooledFrom` equal to `pooled`, and it passes the words' equation, as 4a's `complete` shows). Its knowledge soundness at $1/|E|$ is not a corollary of the phase's: the bad event is now "some message passes the words' equation and makes all pooled claims true of a stack outside the table seam", and the message is free. The key lemma is proved (probe `ProbeWordsLemma.lean`, a plain file, **run before the environment change**, exit 0):

```
-- What an accepting run of the pinned verifiers says at the challenge `r`, when the three
pooled claims are true of a stack whose memory limbs have cells `a e` at index 0 and `b e` at
index 1: the scalars are the extensions of limbs 0 and 1, the extension of limb 2 is zero, and
the equation on the words holds. -/
structure Accepts (a b : Fin 3 → K) (w0 w1 r c0 c1 : E) : Prop where
  claim0 : (1 - r) * ofK (a 0) + r * ofK (b 0) = c0
  claim1 : (1 - r) * ofK (a 1) + r * ofK (b 1) = c1
  claim2 : (1 - r) * ofK (a 2) + r * ofK (b 2) = 0
  words : c0 + y * c1 = w0 + r * (w0 + w1)

-- Accepting at two distinct challenges forces the cells: the public words are the two-limb
words of the cells, and the top cells are zero. -/
theorem accepts_two_challenges {a b : Fin 3 → K} {w0 w1 r1 r2 c0 c1 c0' c1' : E}
```

```

      (hr : r1 ≠ r2) (h1 : Accepts a b w0 w1 r1 c0 c1) (h2 : Accepts a b w0 w1 r2 c0' c1') :
      w0 = E.ofLimbs (a 0) (a 1) 0 ∧ w1 = E.ofLimbs (b 0) (b 1) 0 ∧ a 2 = 0 ∧ b 2 = 0

/-- For public words with a zero top limb, as leanISA's are by type: the six cells are the four
lanes and two zeros. -/
theorem cells_eq_lanes {a b : Fin 3 → K} {p : Fin 4 → K} {r1 r2 c0 c1 c0' c1' : E}
  (hr : r1 ≠ r2)
  (h1 : Accepts a b (E.ofLimbs (p 0) (p 1) 0) (E.ofLimbs (p 2) (p 3) 0) r1 c0 c1)
  (h2 : Accepts a b (E.ofLimbs (p 0) (p 1) 0) (E.ofLimbs (p 2) (p 3) 0) r2 c0' c1') :
  (a 0 = p 0 ∧ a 1 = p 1 ∧ a 2 = 0) ∧ (b 0 = p 2 ∧ b 1 = p 3 ∧ b 2 = 0)

/-- A public word with a nonzero top limb is accepted at one challenge at most, whatever the
stack: the binding itself excludes it, without the separate rejection of `read_public`. -/
theorem top_limb_zero_of_two_challenges {a b : Fin 3 → K} {w0 w1 r1 r2 c0 c1 c0' c1' : E}
  (hr : r1 ≠ r2) (h1 : Accepts a b w0 w1 r1 c0 c1) (h2 : Accepts a b w0 w1 r2 c0' c1') :
  w0.limb 2 = 0 ∧ w1.limb 2 = 0

'WordsLemma.accepts_two_challenges' depends on axioms: [propext, Classical.choice, Quot.sound]
'WordsLemma.cells_eq_lanes' depends on axioms: [propext, Classical.choice, Quot.sound]
'WordsLemma.top_limb_zero_of_two_challenges' depends on axioms: [propext, Classical.choice, Quot.sound]

```

The proof is eleven lines: substituting the claims into the words' equation gives $(1 - r) \cdot A + r \cdot B = (1 + r) \cdot w_0 + r \cdot w_1$ with $A = a_0 + y \cdot a_1$, $B = b_0 + y \cdot b_1$ in E ; two roots force $A = w_0$, $B = w_1$ (the phase's own degree-one argument, `line_unique`, over E instead of K); $A = \text{ofLimbs } a_0 \ a_1 \ 0$ by `ofLimbs_eq`; the third claim is a line through zero. No independence of $1, y$ over K is used until the limbs are read off (`cells_eq_lanes`).

The phase, if built. Over an abstract instance the words' equation needs a shape: it combines the sent values with the powers $1, y, y^2, \dots$, and beyond three sent lines those are dependent over K , so the argument fails. The check of C.5 (mutation 4a) handles this by falling back to the per-value check when the number of sent lines is not two; on that verifier, with `pooledFrom`, the knowledge state function is

- round 1: $\exists cs, \text{checkWeak } s \ r \ cs = \text{true} \wedge ((\text{pooledFrom } s \ r \ cs, o), ()) \in \text{Seam.pub } I,$
- round 2: the same for the message sent,

and the bad-challenge uniqueness splits into: two sent lines l_0, l_1 (then the pooled claims force cs to be the true extensions, and `accepts_two_challenges` with the third claim replaced by the unsent lines' claims), or not (then $cs = \text{expectedValues}$, `pooledFrom` = `pooled`, and the existing `bad_challenge_unique`). The `List` plumbing (that `claimsFrom` holding of the stack forces cs to be the true extensions of the sent lines and the unsent lines' claims) is an induction on the lines. Estimated at 120 lines; **not written**, since the environment was lost. Error: $1/|E|$ on the one challenge, as for the specification's verifier.

B.1.1.4 The public-input phase: the top limb, along the chain (D)

The user's example: "If the verifier does not check, through some reduction or claim, that the top lane of both public inputs is 0 (the column `mem_2` at cells 0 and 1), will the knowledge-soundness and completeness theorems catch this?"

In the phase and the spine: no (D.1) The third claim exists because the instance lists a third line. The phase pools one claim per element of `I.publicLines s.1 (pooled, C.0)`; `publicLines : Stmt → List (PublicLine toShape)` is a field of `M3Instance (Spine/Instance.lean:141-142 at b435631)`:

```

/-- The columns whose first two cells the statement fixes. -/
publicLines : Stmt → List (PublicLine toShape)

```

and the relation's clause is over the same list (`Spine/Instance.lean:207-211`):

```

/-- Cells 0 and 1 of every column the statement fixes hold their values (§8.2). -/
def PublicLinesHold (input : I.Stmt) (q : Column I.μ) : Prop :=
  ∀ l ∈ I.publicLines input,
    (I.column q l.col).values.get (0, Nat.two_pow_pos _) = l.cell0 ∧
    (I.column q l.col).values.get (1, Nat.one_lt_two_pow l.pos.ne') = l.cell1

```


Every theorem of the phase and of the spine is stated for an arbitrary $I : \text{M3Instance}$: `publicInputSecurity : (I : M3Instance) → Phase.Security I (publicInputPhase I) (Seam.table I) (Seam.pub I) (PublicInput.lean:391-397)`, `piop_perfectCompleteness (P : Phases I) (C : P.Complete) ...`, `piop_rbrKnowledgeSoundness (P : Phases I) (S : P.Security) ...` (`Spine/Compose.lean:162-177`). An instance whose `publicLines` lists `mem_0` and `mem_1` only is an `M3Instance` like any other; on it `M3Holds` says nothing of `mem_2`, the phase pools two claims, and every theorem instantiates by `publicInputSecurity I'`, `piop_rbrKnowledgeSoundness P S`. No probe is needed to see this (the quantification is in the types quoted); the probe that was planned (an instance `twoLines` with `example := publicInputSecurity twoLines` and a `#guard` that a stack with a nonzero `mem_2` cell 0 satisfies `M3Holds twoLines`) was not run (C.9) and would only have displayed it. Mutation 5 (C.8) is the same fact from the other side: given a listed line, dropping its claim is caught; whether it is listed is not the phase's business.

So the oracle protocol's theorems do not catch a missing third line. What does is the adaptor, not built (blueprint Layer 3, `protocol-blueprint.md:813-816` at `b435631`):

```
def witnessOf (prog) (s) (q : Column (leanIsaInstance prog s).μ) :
  EnsembleWitness (leanIsaEnsemble prog)                -- total and computable
theorem satisfiedBy_witnessOf (hs : s.Admissible prog)    -- the caps: a hypothesis
  (h : M3Holds (leanIsaInstance prog s) input q) : SatisfiedBy prog input (witnessOf prog s q)
```

and behind it `leanISA`'s `constraintSoundness (leanISA blueprint :1163-1164, not built)`, `SatisfiedBy → ∃ t, AssignmentRepresents w t ∧ ValidExecution prog input t`, and `T4's baseVerifier_extracts Execution (protocol-blueprint.md:1247-1248)`. The links, read in the Lean sources at `b435631`:

1. *The type of the public words.* `PublicInput` is four lanes of K , and its words are built with a zero top limb (`LeanerVM/Semantics/Memory.lean:100-110`):

```
/-- The 256-bit public input as four `K` lanes (specification §2). -/
structure PublicInput where
  /-- `input₀, ..., input₃`. -/
  lanes : Fin 4 → K
  deriving DecidableEq

/-- The first memory word, `input₀ + input₁·y`. -/
def PublicInput.word0 (p : PublicInput) : E := E.ofLimbs (p.lanes 0) (p.lanes 1) 0

/-- The second memory word, `input₂ + input₃·y`. -/
def PublicInput.word1 (p : PublicInput) : E := E.ofLimbs (p.lanes 2) (p.lanes 3) 0
```

with `E.ofLimbs c0 c1 c2 = c0 + c1·y + c2·y²` (`Parameters/Field.lean:92, :145-146`). **A public word with a nonzero top limb is not representable** as a statement. This matches the Rust, whose `read_public` rejects it before the protocol starts (`cpu/mod.rs:139-143`: “The transcript binds a public input as two 128-bit halves, so a third limb would be dropped and two statements would share a transcript. `if public_input.iter().any(|half| half.c2 != 0) { return Err(CpuError::PublicInput); }`”), and the Python, whose `Digest.halves` builds `E(w0, w1)`, `E(w2, w3)` with no third limb (`verifier.py:216-219`).

2. *What `SatisfiedBy` says of cells 0 and 1.* Two conjuncts, on the image the prover data names (`LeanerVM/Arithmetization/Statement.lean:337-340`):

```
/-- The first word of the image is `input₀ + input₁·y` (§8.2). -/
word0_eq : (imageOf w.data).2.read (gpow 0) = some input.word0
/-- The second word of the image is `input₂ + input₃·y` (§8.2). -/
word1_eq : (imageOf w.data).2.read (gpow 1) = some input.word1
```

The image is `MemImage κ = Fin (2 ^ κ) → E` (`Memory.lean:77`): the word at index 0 is one element of E , **three limbs**, and it must equal `input.word0`, whose limb 2 is 0. The memory block's rows are tied to that image limb by limb by `SeedRowsAreTheImage (Statement.lean:279-283`: the row of index i is `(idx i, cntFin i, [limb 0, limb 1, limb 2])` of `(imageOf w.data).2 i`). So `SatisfiedBy` requires, of the committed columns `MEM_LO`, `MEM_HI`, `MEM_TOP` at row 0, the values `(input₀, input₁, 0)`, and at row 1 `(input₂, input₃, 0)`. **The link holds**: the top cells are required to be zero, as three-limb values.

3. *What `satisfiedBy_witnessOf` must derive it from.* `witnessOf` “rebuilds ... the image from the memory columns” (`protocol-blueprint.md:863-864`), so `(imageOf (witnessOf q).data).2 0` is the word `ofLimbs (mem_0[0]) (mem_1[0]) (mem_2[0])` read off `q`. `word0_eq` then needs `mem_0[0] = input₀`,

`mem_1[0] = input1` and `mem_2[0] = 0`. `M3Holds (leanIsaInstance prog s) input q` supplies exactly `PublicLinesHold`, i.e. the cells of the listed lines. With three lines the three equalities are there. With two lines the third is not, and `satisfiedBy_witnessOf` is **false**: the stack that is honest except for `mem_2[0] = 1` satisfies `M3Holds` of the two-line instance (nothing else reads that cell as a constraint; the bus carries it consistently from the seed row through every read to the finalize row), and its witness has `(imageOf ...).2.read (gpow 0) = some (ofLimbs input0 input1 1) ≠ some input.word0 (limb_ofLimbs, Field.lean:120)`. A `witnessOf` that zeroed the top limb itself would break `mem_balanced` or `SeedRowsAreTheImage` on any stack where a table reads cell 0, so no definition of `witnessOf` rescues the theorem for all stacks.

4. *What the semantics says.* `ValidExecution prog input t` (`Semantics/Execution.lean:110-113`) contains `HasPublicBoundary: t.image.read (gpow 0) = some input.word0`, the same three-limb equality on the trace’s image; `constraintSoundness` transports `word0_eq` to it through `AssignmentRepresents.image_eq` (`Statement.lean:360`). So a stack with `mem_2[0] = 1` accepted by a two-line protocol would be extracted to a witness with no valid execution on `input` (a program that reads cell 0 and branches on its top limb has an accepting stack and no valid execution on any `PublicInput`), and `baseVerifier_extractsExecution` would be false.

Anchor. If the third line were dropped from `leanIsaInstance`, the theorem left without a proof is `satisfiedBy_witnessOf` (its conjunct `word0_eq`; and `word1_eq` for cell 1), and through it T4’s `baseVerifier_extractsExecution`. Nothing in `LeanerVM/Protocol/` would change: the phase, the spine and both master theorems would still hold, on the two-line instance, verbatim. The check on the top limb is therefore **not** a verifier check that the proof-system theorems can catch; it is a fact about which lines the `leanISA` instance lists, caught one layer down, at the wall between the proof system and the arithmetization, by a theorem that is not yet built. This is the right place for it (the instance is where `leanISA` enters), and it means the audit of “does the verifier check the top limb” is an audit of `leanIsaInstance.publicLines` against `SatisfiedBy.word0_eq`, i.e. of a definition, plus the existence of `satisfiedBy_witnessOf`.

In the tests: acceptance test 10’s witness exists today (D.2) Acceptance test 10 (`protocol-blueprint.md:1293-1295` at `b435631`): “The claim `mem_2(rm, 0...) = 0` is pooled although no scalar is sent for it; omitting it lets a non-canonical public word pass the line. Witness: the Layer 8 mutation with a nonzero `mem_2` at cell 0 and the third claim dropped.” The test file at `b435631` (`tests/LeanerVMTests/Protocol/PublicInput.lean:151-180`) has the literal three-line configuration (the earlier review’s item “exercised nowhere” was met):

```

/-- Three lines shaped like the memory limbs: two with their value sent, and a third with cells
` (0, 0) ` and no value sent. -/
abbrev threeLimbs : M3Instance :=
  { toy with publicLines := fun v ↦
    [({(0, 0), v, 1, true, by decide}), ({0, 1}, 1, 1, true, by decide),
      ({0, 2}, 0, 0, false, by decide)] }
...
/-- The top column's cell 0 changed to `1`. -/
def badTopLimb : Column 3 := {#v[1, 1, 1, 1, 1, 0, 0, 0]}
...
-- The wrong top cell passes the check, which sees the two sent values only, and fails the third
-- claim; the pool without the third claim accepts the stack.
#guard check threeLimbs stmtLimbs y (trueValues threeLimbs badTopLimb 1 y)
#guard ~ ∀ c ∈ (pooled threeLimbs stmtLimbs y).2.columns, c.Holds badTopLimb
#guard ∀ c ∈ ((pooled threeLimbs stmtLimbs y).2.columns.take 2), c.Holds badTopLimb

```

It rejects what the acceptance test says: the check accepts the stack with the wrong top cell (no scalar sees it), the third claim fails of it, and the pool without the third claim (the “mutation with the third claim dropped”, written as `.take 2`) accepts it. Two limits:

- it is a `#guard` on the *pool*, not on the theorems: it shows the third claim is what rejects the stack, not that a phase without it has no `Phase.Security` (that is the refutation of C.8, on `noneSent`);
- it is on a toy-shaped instance built in the test, not on `leanIsaInstance` (not built). The acceptance test’s real content, that `leanISA`’s instance lists `mem_2` with cells `(0, 0)`, is checkable only when Layer 3 lands; the blueprint’s prose says it (:834-836).

In the deployed verifiers: the top limb rides no scalar and is pooled at 0 (D.3) Rust prover (cpu/mod.rs:606-616): “The top limb of both public words is zero, so its evaluation is zero at every `r_pi` and rides no scalar”; `pi_limbs = [interp_k(...c0...), interp_k(...c1...), F192::ZERO]`, only `pi_limbs[..2]` are added to the stream. Rust verifier (cpu/mod.rs:745-756): two scalars read into `pi_limbs[..2]`, `pi_limbs[2]` stays `F192::ZERO`, the combined check, then `finish_claims`, whose `bind_pi_claim (:674-682)` pools `[MEM_LO, MEM_HI, MEM_TOP]` at `(r, 0, ...)` with the three `pi_limbs`. Python (verifier.py:1398-1402): `public_limbs = (*transcript.next_scalars(2), ZERO)`, the combined check `poly_eval(public_limbs, Y) == multilinear_eval(public_input.halves(), [public_challenge])`, then three claims on `MEMORY_0, MEMORY_1, MEMORY_2`. Recursion guest (crates/rec_aggregation/guests/aggregate.py:1680-1689): `assert mem == mem_lo + mem_hi * Y_TOWER` and three pool entries, the third 0. The statement itself cannot have a nonzero top limb (`read_public`, D.1 item 1; the Python’s Digest is 256 bits, verifier.py:204-219). The Lean phase pools the third claim at `lineValue r l₂ = (1 + r)·0 + r·0 = 0` (`test limbValues`), the same claim.

So the deployed verifiers do check the top limb, through the third pooled claim; the specification says so too (`08-end-to-end-protocol.tex:83-84`: “pools them, with \$0\$ for the top limb, as claims on `$\mem_0, \mem_1, \mem_2$`”). The Lean matches. What no document says in one place is that the check lives in the instance’s list of lines and is enforced by the adaptor’s theorem, not by the phase’s; D.1 is that sentence.

B.1.1.5 Layer 1’s probes (I)

Cross-references. A reference such as “(D.5)” or “section B” in this part is to a section of the dossier `dossiers/code-layer1.md`, whose codes the headings keep in parentheses: A (catalogue) is section 4.3; E (audit surface) is section 10.5.3; F (conformance) is section 8.9.1.3; G (findings) is section 8.9.2.3; I (probes) is ?? B.1.1.5; B (faithfulness), C (statements), D (the wall and the To-folders) and H (the upgrade) is not reproduced in these fragments; they are in the dossier.

All under `.claude/reports/blueprint-review/probes/code-layer1/`, written by the interrupted first agent of this task and run once each by this one on 2026-09-30, each output saved next to its probe (`*.out`). **How they ran**, given that the owner moved the checkout at 09:18 (reflog: `reset 09:18:19, checkout main 09:18:20, pull to 144c5aa 09:18:26, back to the review branch 09:18:33, merge main 09:18:36`; the dependency sources under `.lake/packages/` were switched to the new pins at 09:18:47-09:19:05, their `.olean` files were not rebuilt):

- `offsets.py, values.py`: `python3 -B <file>` at 09:16, exit 0. They import the pinned Python verifier read-only from `/home/scaraven/Documents/leanEthereum/leanVM/python-verifier` (at `a386121f`).
- `OffsetsProbe.lean` (finished 09:17:02) and `ValuesProbe.lean` (finished 09:18:17.99), with the brief’s command `flock .claude/reports/blueprint-review/logs/lean.lock lake env lean <file>`, both before the checkout moved: sources at `b435631`, libraries at the old pins, exit 0.
- `DuplicatesProbe.lean`: the first attempt with `lake env lean` (09:18:22) found the file missing (the owner’s checkout of `main` had removed the tracked probe files for thirteen seconds) and did nothing else. Both it and `StridedProbe.lean` were then run at 09:20 **not through lake** but with the old pin’s Lean binary directly on the `b435631` build artefacts, which the move had not touched (no `.olean` under `.lake/` was newer than 2026-09-30 00:00, checked just before; `.lake/build` dates from the orchestrator’s build of 2026-09-29 17:18, the packages’ builds from 2026-09-28/29):

```
LP="$(pwd)/.lake/build/lib/lean"; for d in .lake/packages/*/.lake/build/lib/lean; do LP="$LP:$(pwd)/$d";
done
LEAN_PATH="$LP" timeout 590 flock .claude/reports/blueprint-review/logs/lean.lock \
/home/scaraven/.elan/toolchains/leanprover--lean4--v4.33.1/bin/lean <probe>.lean
```

Lean loads compiled `.olean` files only and never reads the (switched) library sources, so these runs elaborate against `b435631` and the old pins exactly as `lake env lean` did before the move; both exit 0. They were made before the coordinator’s instruction to stop running Lean arrived; no Lean process was started after it. The orchestrator may prefer to treat these two as “written, not run through the brief’s command”; the conclusions resting on them are marked in the text (`DuplicatesProbe`: D.2 and G.5; `StridedProbe`: B.8 and G.1) and each is also a short argument a reader can check by hand.

offsets.py (I.1) What it establishes. Runs the pinned Python verifier's own `stack_offsets` and `build_layout` (imported read-only from the leanVM checkout at `a386121f`) and a line-by-line transcription of the Rust `stack_offsets` (`crates/lean_vm/src/witness.rs:67-79`), and prints the offsets and selectors of two configurations: the dossier's small configuration, and the pinned verifier's real layout for one admissible announcement (92 committed blocks, `stack_log = 22`). Its numbers are the expected values of `OffsetsProbe.lean`.

Command. `python3 -B .claude/reports/blueprint-review/probes/code-layer1/offsets.py`

```

"""Probe for task code-layer1, deliverable B.2.

Runs the pinned Python verifier's own `stack_offsets` and `build_layout` (imported read-only
from the leanVM checkout at a386121f; no bytecode cache is written) and a line-by-line
transcription of the Rust `stack_offsets` (crates/lean_vm/src/witness.rs:67-79), and prints
the offsets and selectors of two configurations.

Run:  python3 -B .claude/reports/blueprint-review/probes/code-layer1/offsets.py
"""

import sys

sys.dont_write_bytecode = True
sys.path.insert(0, "/home/scaraven/Documents/leanEthereum/leanVM/python-verifier")
import verifier as V  # noqa: E402

def rust_stack_offsets(kappas):
    """witness.rs:67-79, transcribed: `order.sort_by(|a,b| kappas[b].cmp(kappas[a]).then(a.cmp(b)))`."""
    n = len(kappas)
    order = [i for i in range(n) if kappas[i] is not None]
    order.sort(key=lambda i: (-kappas[i], i))
    offsets = [0] * n
    off = 0
    for i in order:
        offsets[i] = off
        off += 1 << kappas[i]
    return offsets, off, order

def report(name, kappas, names=None):
    print(f"== {name}")
    print("sizes in column-index order:", kappas)
    r_off, r_total, order = rust_stack_offsets(kappas)
    p_off, p_log = V.stack_offsets(kappas)
    print("Rust transcription offsets :", r_off, "total", r_total)
    print("Python verifier offsets    :", p_off, "log2_ceil(total)", p_log)
    assert r_off == p_off
    print("stacking order (column indices, first block first):", order)
    print("sorted sizes (the Blocks.size sequence)           :", [kappas[i] for i in order])
    print("offset per block, in stacking order                 :", [r_off[i] for i in order])
    print("selector per block (offset >> size)                  :", [r_off[i] >> kappas[i] for i in order])
    if names:
        for i in order:
            print(f"    block of column {i:3d} {names[i]:<14s} size {kappas[i]:2d} offset {r_off[i]:8d} selector
                  ↪ {r_off[i] >> kappas[i]}")
    print()

# 1. The small configuration of the dossier: six shared columns and two table columns.
names = ["mem_0", "mem_1", "mem_2", "cntfin_mem", "cntfin_bc", "q_flock", "table col a", "table col b"]
report("small configuration", [4, 4, 4, 4, 2, 5, 4, 4], names)

# 2. The pinned verifier's real layout for one admissible announcement:
#   log_memory 16, table log-heights (xor, mul, set, deref, jump, blake2s) = (3, 16, 0, 5, 16, 3),
#   a bytecode of 2^4 instructions (the table has 2^(4+4) words; its content is irrelevant here).
bytecode = [V.K(0)] * (1 << 8)
layout = V.build_layout(bytecode, 16, (3, 16, 0, 5, 16, 3))
cols = []
for column, p in enumerate(layout.placements):
    cols.append((column, p.variables, p.index, p.low))
committed = [(c, k, off) for (c, k, off, low) in cols if low == 0]
virtual = [(c, k, off, low) for (c, k, off, low) in cols if low != 0]

```

```
print("== the pinned Python verifier's build_layout(log_memory=16, taus=(3,16,0,5,16,3), kbc=4)")
print("columns:", len(cols), "committed blocks:", len(committed), "strided (BLAKE2S limb) columns:",
      ↪ len(virtual))
print("stack_log:", layout.stack_log)
order = sorted(committed, key=lambda t: t[2])
print("stacking order, as (column index, size, offset):")
print(order)
print("sorted sizes:", [k for (_, k, _) in order])
print("offsets      :", [off for (_, _, off) in order])
print("column index of each block:", [c for (c, _, _) in order])
# the same sizes through the Rust transcription, virtual columns as None
kappas = [None if low != 0 else k for (_, k, _, low) in cols]
r_off, r_total, r_order = rust_stack_offsets(kappas)
assert [r_off[c] for (c, _, _) in committed] == [off for (_, _, off) in committed]
assert r_order == [c for (c, _, _) in order]
print("Rust transcription agrees on every committed column; total", r_total, "log2_ceil", V.log2_ceil(r_total))
print("global column bases of the six tables:", V.GLOBAL_COLUMN_BASES, "widths", V.TABLE_WIDTHS)
```

Recorded output.

[illegible]

Rust transcription agrees on every committed column; total 2165512 log2_ceil 22
 global column bases of the six tables: (6, 21, 36, 44, 59, 73) widths (15, 15, 8, 15, 14, 37)

OffsetsProbe.lean (I.2) What it establishes. Blocks.offset and Blocks.selector on sorted sizes reproduce leanVM's offsets once the columns are mapped by leanVM's order, and the tables-first order gives the same Blocks and other offsets (B.2).

Command. flock .claude/reports/blueprint-review/logs/lean.lock lake env lean .claude/reports/blueprint-review/probes/code-layer1/OffsetsProbe.lean

```

/-
  Probe for task code-layer1, deliverable B.2.

  leanVM's `stack_offsets` (`crates/lean_vm/src/witness.rs:67-79`, `python-verifier/verifier.py:
  305-311`) transcribed as a list function, and Layer 1's `Blocks.offset` / `Blocks.selector`
  fed with the order it produces. The expected numbers are the output of `offsets.py`, which
  runs the pinned Python verifier.

  Run from the repository root:
    flock .claude/reports/blueprint-review/logs/lean.lock \
      lake env lean .claude/reports/blueprint-review/probes/code-layer1/OffsetsProbe.lean
-/
import LeanerVM.Protocol.ToCompPoly.Stacking

open LeanerVM.Protocol

namespace Probe

/-! ## `stack_offsets`, transcribed -/

/-- The stacking order: column indices sorted by size, largest first, ties by index. -/
def stackOrder (kappas : List ℕ) : List ℕ :=
  (List.range kappas.length).mergeSort fun a b =>
    decide (kappas[b]! < kappas[a]!) || (kappas[a]! == kappas[b]! && decide (a ≤ b))

/-- Prefix sums of the heights along a list of sizes. -/
def prefixOffsets : List ℕ → ℕ → List ℕ
| [], _ => []
| k :: ks, off => off :: prefixOffsets ks (off + 2 ^ k)

/-- The offset of every column, in column-index order (`offsets[i] = off` for `i` in order). -/
def stackOffsets (kappas : List ℕ) : List ℕ :=
  let order := stackOrder kappas
  let offs := prefixOffsets (order.map (kappas[·]!)) 0
  (List.range kappas.length).map fun i => (offs[order.idxOf i]?).getD 0

/-- A `Blocks` from a list of sizes that is already largest first. -/
def blocksOfList (l : List ℕ) (h : l.Pairwise (· ≥ ·)) : Blocks where
  n := l.length
  size := fun i => l[i]
  descending := by
    intro a b hab
    rcases eq_or_lt_of_le hab with rfl | hlt
    · exact le_rfl
    · exact (List.pairwise_iff_getElem.mp h) a.val b.val a.isLt b.isLt hlt

/-- Every offset of a `Blocks`, first block first. -/
def offsetsOf (B : Blocks) : List ℕ := (List.finRange B.n).map B.offset

/-- Every selector of a `Blocks`, first block first. -/
def selectorsOf (B : Blocks) {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) : List ℕ :=
  (List.finRange B.n).map fun b => (B.selector hμ b).val

/-! ## The small configuration

Column-index order: `mem_0, mem_1, mem_2, cntfin_mem, cntfin_bc, q_flock, table a, table b`. -/

def small : List ℕ := [4, 4, 4, 4, 2, 5, 4, 4]

-- The transcription gives what the pinned Python verifier and the Rust give (offsets.py).
#guard stackOrder small = [5, 0, 1, 2, 3, 6, 7, 4]

```

```

#guard stackOffsets small = [32, 48, 64, 80, 128, 0, 96, 112]

/-- The sorted sizes, as a `Blocks`. -/
def smallBlocks : Blocks := blocksOfList [5, 4, 4, 4, 4, 4, 4, 2] (by decide)

#guard (stackOrder small).map (small[·]!) = [5, 4, 4, 4, 4, 4, 4, 2]

theorem smallBlocks_total_le : smallBlocks.total ≤ 2 ^ 8 := by decide

example : smallBlocks.total = 132 := by decide

-- `Blocks.offset` and `Blocks.selector`, block by block, against the pinned verifiers' values.
#guard offsetsOf smallBlocks = [0, 32, 48, 64, 80, 96, 112, 128]
#guard selectorsOf smallBlocks smallBlocks_total_le = [0, 2, 3, 4, 5, 6, 7, 32]

-- Column `i` sits on block `(stackOrder small).idxOf i`; through that map the offsets are
-- leanVM's, column by column.
#guard (List.range 8).map (fun i ↦ (offsetsOf smallBlocks)[(stackOrder small).idxOf i]!) =
  stackOffsets small

/-- The order the blueprint had before finding F17: ties broken with the tables' columns
first. It sorts to the same sizes, so it is the same `Blocks`. -/
def tablesFirst : List ℕ := [5, 6, 7, 0, 1, 2, 3, 4]

#guard tablesFirst.map (small[·]!) = (stackOrder small).map (small[·]!)

-- Near miss: through that map `mem_0` sits at offset 64, not 32, and nothing about the
-- `Blocks` differs: the order of equal sizes lives in the map from columns to blocks only.
#guard (List.range 8).map (fun i ↦ (offsetsOf smallBlocks)[tablesFirst.idxOf i]!) =
  [64, 80, 96, 112, 128, 0, 32, 48]
#guard (List.range 8).map (fun i ↦ (offsetsOf smallBlocks)[tablesFirst.idxOf i]!) ≠
  stackOffsets small

-- The column-index order itself is not largest first, so it is no `Blocks`.
example : ¬ small.Pairwise (· ≥ ·) := by decide

/-! ## The pinned verifier's layout for one admissible announcement

`build_layout(log_memory = 16, taus = (3, 16, 0, 5, 16, 3), kbc = 4)`: 110 columns, 92 of
them committed. The three lists are copied from the output of `offsets.py`. -/

/-- The sizes of the 92 committed columns, in column-index order (the eighteen BLAKE2S limb
columns, global indices 82 to 99, left out). -/
def committedSizes : List ℕ :=
  [16, 16, 16, 16, 4, 11] ++ List.replicate 15 3 ++ List.replicate 15 16 ++
  List.replicate 8 0 ++ List.replicate 15 5 ++ List.replicate 14 16 ++
  List.replicate 9 3 ++ List.replicate 10 3

/-- The global column index of each committed column, in the same order. -/
def committedIndex : List ℕ := List.range 82 ++ (List.range 10).map (· + 100)

/-- Python: "column index of each block". -/
def pyOrder : List ℕ :=
  [0, 1, 2, 3, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 59, 60, 61, 62, 63,
  64, 65, 66, 67, 68, 69, 70, 71, 72, 5, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56,
  57, 58, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 73, 74, 75, 76, 77, 78,
  79, 80, 81, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 36, 37, 38, 39, 40, 41, 42, 43]

/-- Python: "offsets", first block first. -/
def pyOffsets : List ℕ :=
  [0, 65536, 131072, 196608, 262144, 327680, 393216, 458752, 524288, 589824, 655360, 720896,
  786432, 851968, 917504, 983040, 1048576, 1114112, 1179648, 1245184, 1310720, 1376256,
  1441792, 1507328, 1572864, 1638400, 1703936, 1769472, 1835008, 1900544, 1966080, 2031616,
  2097152, 2162688, 2164736, 2164768, 2164800, 2164832, 2164864, 2164896, 2164928, 2164960,
  2164992, 2165024, 2165056, 2165088, 2165120, 2165152, 2165184, 2165216, 2165232, 2165240,
  2165248, 2165256, 2165264, 2165272, 2165280, 2165288, 2165296, 2165304, 2165312, 2165320,
  2165328, 2165336, 2165344, 2165352, 2165360, 2165368, 2165376, 2165384, 2165392, 2165400,
  2165408, 2165416, 2165424, 2165432, 2165440, 2165448, 2165456, 2165464, 2165472, 2165480,
  2165488, 2165496, 2165504, 2165505, 2165506, 2165507, 2165508, 2165509, 2165510, 2165511]

/-- The sorted sizes. -/
def sortedSizes : List ℕ :=
  List.replicate 33 16 ++ [11] ++ List.replicate 15 5 ++ [4] ++ List.replicate 34 3 ++

```

```

List.replicate 8 0

#guard committedSizes.length = 92
#guard (stackOrder committedSizes).map (committedIndex[·]!) = pyOrder
#guard (stackOrder committedSizes).map (committedSizes[·]!) = sortedSizes

/-- The sorted sizes as a `Blocks`. -/
def realBlocks : Blocks := blocksOfList sortedSizes (by decide +kernel)

#guard offsetsOf realBlocks = pyOffsets
#guard realBlocks.total = 2165512
example : realBlocks.total ≤ 2 ^ 22 := by decide +kernel
example : ¬ realBlocks.total ≤ 2 ^ 21 := by decide +kernel

end Probe

```

Recorded output.

```

(no output)
exit=0

```

values.py (I.3) What it establishes. Reference numbers from the pinned leanVM, to be compared with Layer 1's Lean definitions in `ValuesProbe.lean`. Every evaluation is made by a function of the pinned Python verifier (`multilinear_eval`, `index_mle`, `eq_eval`, `stack_offsets`, `Placement`), imported read-only; the bytecode table is built by a line-by-line transcription of the Rust `bytecode_columns` and `stacked_bytecode_table`, since the Python verifier takes that table as an input file and has no encoder of its own.

Command. `python3 -B .claude/reports/blueprint-review/probes/code-layer1/values.py`

```

"""Probe for task code-layer1, deliverables B.3, B.5, B.6, B.7 and C.

Reference numbers from the pinned leanVM (a386121f), to be compared with Layer 1's Lean
definitions in `ValuesProbe.lean`. Every evaluation below is made by a function of the pinned
Python verifier (`multilinear_eval`, `index_mle`, `eq_eval`, `stack_offsets`, `Placement`),
imported read-only. The bytecode table is built by a line-by-line transcription of the Rust
`bytecode_columns` (crates/lean_vm/src/cpu/layout.rs:229-309) and `stacked_bytecode_table`
(crates/lean_vm/src/leaf.rs:585-604): the Python verifier takes that table as an input file and
has no encoder of its own.

Run:  python3 -B .claude/reports/blueprint-review/probes/code-layer1/values.py
"""

import sys

sys.dont_write_bytecode = True
sys.path.insert(0, "/home/scaraven/Documents/leanEthereum/leanVM/python-verifier")
import verifier as V # noqa: E402

K, E = V.K, V.E

def limbs(x):
    return (x.c0.value, x.c1.value, x.c2.value)

def gk(i):
    """g^i in K (primitives/field/mod.rs:82, g_pow)."""
    r = K(1)
    for _ in range(i):
        r = r * K(2)
    return r

# ---- the bytecode table -----
OP = {"xor": gk(0), "mul": gk(1), "set": gk(2), "deref": gk(3), "jump": gk(4), "blake2s": gk(5)} #
↳ tables.rs:95-100

def bytecode_columns(prog):
    """layout.rs:229-309: (opcode, o1, o2, o3, fpc, ffp, extra0, extra1) per instruction."""

```



```

cols = [[] for _ in range(8)]
for op in prog:
    kind = op[0]
    z = K(0)
    if kind in ("xor", "mul"):
        _, a, b, c = op
        row = [OP[kind], gk(a), gk(b), gk(c), z, z, z, z]
    elif kind == "set":
        _, o, k = op # k = (c0, c1, c2)
        row = [OP[kind], gk(o), K(k[0]), K(k[1]), K(k[2]), z, z, z] # :257, :270
    elif kind == "deref":
        _, o1, o2, o3, mode = op
        f_pc = K(1) if mode == "pc" else z # isa.rs:69-74
        f_fp = K(1) if mode == "fp" else z
        row = [OP[kind], gk(o1), gk(o2), gk(o3), f_pc, f_fp, z, z]
    elif kind == "jump":
        _, oc, od, of = op
        row = [OP[kind], gk(oc), gk(od), gk(of), z, z, z, z]
    elif kind == "blake2s":
        _, ins, cv, out, md = op
        row = [OP[kind], gk(ins[0]), gk(ins[1]), gk(ins[2]), gk(ins[3]), gk(cv), gk(out), gk(md)]
    for c in range(8):
        cols[c].append(row[c])
return cols

def stacked_bytecode_table(cols, kbc):
    """leaf.rs:585-604: column i at slot 3 + i, `table[(slot << kbc)..((slot+1) << kbc)]`. """
    table = [K(0)] * (1 << (4 + kbc))
    for i, vals in enumerate(cols):
        slot = 3 + i
        assert len(vals) == 1 << kbc
        table[(slot << kbc):((slot + 1) << kbc)] = vals
    return table

PAD = ("set", 0, (0, 0, 0)) # lean_compiler/src/lib.rs:162
prog = [
    ("xor", 1, 2, 3),
    ("mul", 4, 5, 6),
    ("set", 7, (11, 12, 13)),
    ("deref", 8, 9, 10, "pc"),
    ("deref", 21, 22, 23, "fp"),
    ("deref", 24, 25, 26, "cell"),
    ("jump", 27, 28, 29),
    ("blake2s", (14, 15, 16, 17), 18, 19, 20),
] + [PAD] * 8
kbc = 4
table = stacked_bytecode_table(bytecode_columns(prog), kbc)
print("== bytecode table, 16 instructions, 256 cells (cell i + 16*s holds slot s of instruction i)")
print([w.value for w in table])

zeta = (E(3, 1, 0), E(5, 0, 7), E(2, 2, 2), E(0, 1, 0))
alpha = (E(9, 0, 1), E(0, 4, 0), E(6, 6, 0), E(1, 2, 3))
print("== bytecode multilinear at (zeta, alpha), as verifier.py:566 evaluates it")
print(limbs(V.multilinear_eval(table, (*zeta, *alpha))))
print("== with alpha reversed")
print(limbs(V.multilinear_eval(table, (*zeta, *reversed(alpha)))))

# ---- the index column -----
print("== index_mle at zeta (4 variables) and at its first 2 coordinates")
print(limbs(V.index_mle(zeta)), limbs(V.index_mle(zeta[:2])))
print("== the MLE of [g^0 .. g^15] at zeta, by multilinear_eval")
print(limbs(V.multilinear_eval([gk(i) for i in range(16)], zeta)))

# ---- stacking: selectors, the zero-padded and the one-padded stack -----
sizes = [2, 1, 0]
tables = [[1, 2, 3, 4], [5, 6], [7]]
offsets, mu = V.stack_offsets(sizes)
print("== blocks of sizes 2, 1, 0: offsets", offsets, "mu", mu)
z3 = zeta[:3]
placements = [V.Placement(k, off) for k, off in zip(sizes, offsets)]
sel = [p.eq_above(z3) for p in placements]

```


Recorded output.

ValuesProbe.lean (I.4) **What it establishes.** The index column, the bytecode column (256 cells and the (ζ, α) evaluation), the selector weights, both paddings, the column-claim weight and back-loaded padding

agree with the pinned Python's numbers, with their near misses; `bytecodeColumn_slot`'s proof also proves the opposite layout's analogue; `Layout.comap` aliases; and the axioms of seventeen Layer 1 declarations and of the probe's `bytecodeColumn_answer_slots` (B.3-B.7, C.3, C.4).

Command. flock .claude/reports/blueprint-review/logs/lean.lock lake env lean .claude/reports/blueprint-review/probes/code-layer1/ValuesProbe.lean

[illegible]

```

-- All 256 cells: the encoding, the opcode values, the slot order and the cell order.
#guard (bytecodeColumn prog16).values.toList.map (·.toNat) = pyTable

/-- The oracle's answer at `(ζ, α)`. -/
def bcAnswer : E := answer (bytecodeColumn prog16) (zeta ++ alpha)

-- The evaluation the pinned verifier makes (`verifier.py:566`), and the native evaluator.
#guard bcAnswer = E.ofLimbs 1219889995492 3571792677380 1279835512890
#guard bytecodeColumnEval prog16 zeta alpha = E.ofLimbs 1219889995492 3571792677380 1279835512890
-- Near miss: with `α` reversed the pinned verifier gets another value, and so does the oracle.
#guard answer (bytecodeColumn prog16) (zeta ++ alphaRev) =
  E.ofLimbs 1332010270628 4168742050444 1982034182034
#guard bcAnswer ≠ E.ofLimbs 1332010270628 4168742050444 1982034182034

/-- The opposite layout: slot bits low, instruction bits high. -/
def wrongColumn (prog : Program) : Column (4 + prog.logSize) :=
  (Vector.ofFn fun i ↦
    let p := (cubeSplit 4 prog.logSize).symm i
    (encodeSlots (prog.code p.2))[p.1])

/-- The analogue of `bytecodeColumn_slot` for the opposite layout, by the same proof script:
the *shape* of that theorem holds of either layout. -/
theorem wrongColumn_slot (prog : Program) (i : Fin (2 ^ prog.logSize)) (s : Fin 16) :
  (wrongColumn prog).values[cubeIndex (k := 4) s i] = (encodeSlots (prog.code i))[s] := by
  simp [wrongColumn, ← cubeSplit_apply]

-- The statement of `bytecodeColumn_slot` itself, word for word, is false of the opposite
-- layout: cell `0 + 16 * 3` is instruction 0's opcode there, and slot 0 of instruction 3 here.
#guard (bytecodeColumn prog16).values.toList[48]! = Opcode.xor.code
#guard (wrongColumn prog16).values.toList[48]! = 0
#guard (wrongColumn prog16).values.toList.map (·.toNat) ≠ pyTable

/-- What the bus phase needs and Layer 1 does not state: the program's share of a bytecode
block is one evaluation of the bytecode column, the slots weighted by `eq(w, ·)`. -/
theorem bytecodeColumn_answer_slots (prog : Program) (z : Vector E prog.logSize)
  (w : Vector E 4) :
  eval₂Mle (bytecodeColumn prog).values (algebraMap K E) (z ++ w) =
    ∑ s : Fin 16, (lagrangeBasis w)[s] *
      eval₂Mle (bytecodeSlotColumn prog s).values (algebraMap K E) z := by
  rw [eval₂Mle, evalMle_split]
  refine Finset.sum_congr rfl fun s _ => ?_
  congr 1
  rw [eval₂Mle, ← slice_bytecodeColumn]
  congr 1
  apply Vector.ext
  intro i hi
  simp [slice, CMLPolynomialEval.map]

/-! ## B.5 The index column -/

#guard answer (idxColumn 4) zeta = E.ofLimbs 950617 874814 877803
#guard idxColumnEval zeta = E.ofLimbs 950617 874814 877803
#guard idxColumnEval (#v[E.ofLimbs 3 1 0, E.ofLimbs 5 0 7] : Vector E 2) = E.ofLimbs 109 29 108
#guard (idxColumn 4).values.toList.map (·.toNat) = (List.range 16).map (2 ^ ·)

/-! ## B.2, B.3 Selectors and the two paddings -/

/-- Blocks on 2, 1 and 0 variables. -/
def blocks : Blocks where
  n := 3
  size := ![2, 1, 0]
  descending := by
    show ∀ a b : Fin 3, a ≤ b → ![2, 1, 0] b ≤ ![2, 1, 0] a
    decide

theorem blocks_total_le : blocks.total ≤ 2 ^ 3 := by decide

/-- The tables `[1, 2, 3, 4]`, `[5, 6]`, `[7]`. -/
def tables : blocks.Tables K :=
  show (b : Fin 3) → CMLPolynomialEval K (![2, 1, 0] b) from fun b ↦ match b with
  | 0 => (#v[1, 2, 3, 4] : CMLPolynomialEval K 2)
  | 1 => (#v[5, 6] : CMLPolynomialEval K 1)
  | 2 => (#v[7] : CMLPolynomialEval K 0)

```

```

/-- The tables over `E`. -/
def tablesE : blocks.Tables E := fun b ↦ CMLPolynomialEval.map (algebraMap K E) (tables b)

/-- The first three coordinates of `ζ`. -/
def z3 : Vector E 3 := #v[E.ofLimbs 3 1 0, E.ofLimbs 5 0 7, E.ofLimbs 2 2 2]

/-- The selector weights, block by block. -/
def weights : List E := (List.finRange 3).map fun b ↦ blocks.selectorWeight blocks_total_le b z3

/-- The blocks at the low coordinates, block by block. -/
def blocksAt : List E := (List.finRange 3).map fun b ↦
  evalMle (tables b) (algebraMap K E) (blocks.lowPoint blocks_total_le b z3)

-- `eq(sel_b, ζ_hi)` as `Placement.eq_above` computes it (`verifier.py:299-302`).
#guard weights = [E.ofLimbs 3 2 2, E.ofLimbs 6 8 8, E.ofLimbs 2 26 30]
#guard blocksAt = [E.ofLimbs 46 11 42, E.ofLimbs 0 3 0, E.ofLimbs 7 0 0]
-- The witness stack (pad 0) and a leaf stack (pad 1) at `ζ`.
#guard evalMle (blocks.stackColumn tables 3).values (algebraMap K E) z3 = E.ofLimbs 38 3 34
#guard evalMle (blocks.stackAt tablesE 3 1) z3 = E.ofLimbs 32 19 54
-- The padding term of equation (2), `1 + Σ_b eq(sel_b, ζ_hi)` (`verifier.py:589`).
#guard 1 + weights.sum = E.ofLimbs 6 16 20
#guard (List.zipWith (· * ·) weights blocksAt).sum + (1 + weights.sum) = E.ofLimbs 32 19 54
-- Near misses: the wrong pad, and the padding term read as the constant 1.
#guard evalMle (blocks.stackAt tablesE 3 0) z3 ≠ E.ofLimbs 32 19 54
#guard (List.zipWith (· * ·) weights blocksAt).sum + 1 ≠ E.ofLimbs 32 19 54

/-! ## B.7 A column claim as a weight -/

/-- The claim's point on block 1, lifted. -/
def lifted : Vector E 3 := blocks.extendPoint blocks_total_le (1 : Fin 3) #v[E.ofLimbs 3 1 0]

/-- A stack no honest prover commits. -/
def arbitrary : Column 3 := #v[9, 8, 7, 6, 5, 4, 3, 2]

#guard lifted = #v[E.ofLimbs 3 1 0, 0, 1]
#guard (eqWeight lifted).pair arbitrary = E.ofLimbs 6 1 0
#guard (eqWeight lifted).mle #v[E.ofLimbs 1 1 0, E.ofLimbs 0 2 0, E.ofLimbs 7 0 0] =
  E.ofLimbs 9 18 0
-- Near miss: the selector bits reversed, `(1, 0)`, weigh cells 2 and 3.
#guard (eqWeight (#v[E.ofLimbs 3 1 0, 1, 0] : Vector E 3)).pair arbitrary ≠ E.ofLimbs 6 1 0

/-! ## B.4 Back-loaded padding -/

/-- The table `[3, 5]` on one variable. -/
def short : CMLPolynomialEval K 1 := #v[3, 5]

#guard evalMle (padHigh short 2) ((#v[7] : Vector K 1) ++ (#v[11, 13] : Vector K 2)) = 1935
#guard (padHigh short 2).toList = [0, 0, 0, 0, 0, 0, 3, 5]
-- Near miss: padding on the low variables (front-loaded) is another table.
#guard (padHigh short 2).toList ≠ [0, 0, 0, 3, 0, 0, 0, 5]

/-! ## C A layout need not separate its columns -/

/-- Three columns all read off block 0: `Layout.comap` asks nothing of the renaming. -/
def aliased : Layout 3 (Fin 3) (fun _ ↦ 2) :=
  (blocks.layout blocks_total_le).comap (fun _ ↦ (0 : Fin 3)) (fun _ ↦ rfl)

#guard (List.finRange 3).all fun c ↦ (aliased.read arbitrary c).values.toList = [9, 8, 7, 6]

/-! ## Axioms -/

#print axioms Blocks.unstack_eval
#print axioms Blocks.unstack_getElem
#print axioms Blocks.readColumn_eval
#print axioms Blocks.layout
#print axioms Layout.comap
#print axioms Blocks.stack_eval_ambient
#print axioms Blocks.stack_eval_ambient_one
#print axioms Blocks.stackColumn_eval_ambient
#print axioms sumCube_padHigh
#print axioms evalMle_padHigh
#print axioms ColumnClaim.holds_iff_weighted

```

```
#print axioms eqWeight
#print axioms idxColumn_eval
#print axioms idxColumnEval_eq
#print axioms bytecodeColumn_answer_boolVec
#print axioms bytecodeColumn_eval
#print axioms BlockClaim.isValid_iff_pairing
#print axioms bytecodeColumn_answer_slots

end Probe
```

Recorded output.

```
'LeanerVM.Protocol.Blocks.unstack_eval2' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.unstack_getElem' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.readColumn_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.layout' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Layout.comap' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.stack_eval_ambient' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.stack_eval_ambient_one' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.Blocks.stackColumn_eval_ambient' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.sumCube_padHigh' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.evalMle_padHigh' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.ColumnClaim.holds_iff_weighted' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.eqWeight' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.idxColumn_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.idxColumnEval_eq' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.bytecodeColumn_answer_boolVec' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.bytecodeColumn_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
'LeanerVM.Protocol.BlockClaim.isValid_iff_pairing' depends on axioms: [propext, Classical.choice, Quot.sound]
'Probe.bytecodeColumn_answer_slots' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0
```

DuplicatesProbe.lean, run with the old pin's Lean binary, not through Lake (I.5) What it establishes. `evalMle_lagrangeBasis` is `CompPoly`'s `eqTilde_eq_prod`, three in-layer duplicates, and largest-first is not necessary for alignment (D.2, G.5). Run with the old pin's Lean binary on the untouched `b435631` artefacts, not with lake `env lean` (see the introduction of this part).

Command. `LEAN_PATH=... flock ... leanprover--lean4---v4.33.1/bin/lean .claude/reports/blueprint-review/probes/code-layer1/DuplicatesProbe.lean`

```
/-
  Probe for task code-layer1, deliverables D and E: which Layer 1 statements are already in
  CompPoly at the pin (3468b38c), and which are instances of one another.

  Run from the repository root:
    flock .claude/reports/blueprint-review/logs/lean.lock \
      lake env lean .claude/reports/blueprint-review/probes/code-layer1/DuplicatesProbe.lean
-/
import LeanerVM.Protocol.ToCompPoly.BitProductTable
import LeanerVM.Protocol.ToCompPoly.AmbientStacking

open LeanerVM.Protocol CompPoly CMLPolynomialEval

namespace Probe

variable {R : Type*} [CommRing R]

/-- `evalMle_lagrangeBasis` is CompPoly's `eqTilde_eq_prod` (`Multilinear/Basic.lean:600`),
with the factors written in the other order. -/
example {n : ℕ} (w x : Vector R n) :
  evalMle (lagrangeBasis w) x = ∏ k : Fin n, ((1 - x[k]) * (1 - w[k]) + x[k] * w[k]) := by
  have h := eqTilde_eq_prod w x
  rw [eqTilde, ← eval_mle_eq_eval] at h
  rw [h]
  exact Finset.prod_congr rfl fun i _ => by ring

/-- `unstack_eval` is `unstack_eval2` at the identity map. -/
example (B : Blocks) {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (q : CMLPolynomialEval R μ) (b : Fin B.n)
  (z : Vector R (B.size b)) :
  evalMle q (B.extendPoint hμ b z) = evalMle (B.unstack hμ q b) z := by
```

```

have h := B.unstack_eval₂ (RingHom.id R) hμ q b z
simp [eval₂Mle, CMLPolynomialEval.map] using h

/-- `stack_eval` is `unstack_eval` on a stack. -/
example (B : Blocks) (t : B.Tables R) {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (pad : R) (b : Fin B.n)
  (z : Vector R (B.size b)) :
  evalMle (B.stackAt t μ pad) (B.extendPoint hμ b z) = evalMle (t b) z := by
  rw [B.unstack_eval hμ, B.unstack_stackAt]

/-- `stack_eval_ambient_zero` is `stack_eval_ambient` at pad zero. -/
example (B : Blocks) (t : B.Tables R) {μ : ℕ} (hμ : B.total ≤ 2 ^ μ) (z : Vector R μ) :
  evalMle (B.stackAt t μ 0) z =
    ∑ b : Fin B.n, B.selectorWeight hμ b z * evalMle (t b) (B.lowPoint hμ b z) := by
  simp using B.stack_eval_ambient t hμ 0 z

/-- The largest-first order is sufficient for alignment, not necessary: sizes 1, 1, 2 are
aligned at offsets 0, 2, 4 and are no `Blocks`. -/
example : ¬ Antitone (![1, 1, 2] : Fin 3 → ℕ) := fun h =>
  absurd (h (show (0 : Fin 3) ≤ 2 by decide)) (by decide)

#guard (2 ^ 1 | 0) ∧ (2 ^ 1 | 2) ∧ (2 ^ 2 | 4)

end Probe

```

Recorded output.

```

(no output)
exit=0

```

StridedProbe.lean, run with the old pin's Lean binary, not through Lake (I.6) What it establishes.

The low-index selection identity holds and is twenty lines from Layer 1's lemmas (B.8, G.1). Run with the old pin's Lean binary on the untouched b435631 artefacts, not with lake env lean (see the introduction of this part).

Command. LEAN_PATH=... flock ... leanprover--lean4---v4.33.1/bin/lean .claude/reports/blueprint-review/probes/code-layer1/StridedProbe.lean

```

/-
Probe for task code-layer1, deliverable C and the findings: the selection identity Layer 1
does not have. leanVM reads the eighteen BLAKE2S limb columns off `q_flock` by freezing the
LOW coordinates to a slot's bits (`crates/pcs/src/stack_open.rs:84-97`, `StackClaim::Strided`:
"Equivalent to a `Point` with `low_point = slot_bits ++ point`"). Layer 1 slices on the high
index only. This file states and proves the mirrored identity, to show what is missing and
that it is a generic fact of the same size as `evalMle_append_boolVec`.

Run from the repository root:
  flock .claude/reports/blueprint-review/logs/lean.lock \
    lake env lean .claude/reports/blueprint-review/probes/code-layer1/StridedProbe.lean
-/
import LeanerVM.Protocol.ToCompPoly.Multilinear
import LeanerVM.Parameters.Field

open LeanerVM.Protocol LeanerVM.Parameters CompPoly CMLPolynomialEval

namespace Probe

variable {R : Type*} [CommRing R]

/-- The strided slice of a table at the low index `i`: the entries whose low `k` bits are `i`. -/
def sliceLow {k m : ℕ} (t : CMLPolynomialEval R (k + m)) (i : Fin (2 ^ k)) :
  CMLPolynomialEval R m :=
  Vector.ofFn fun j => t[cubeIndex i j]

/-- A Boolean low coordinate selects the strided slice at that index. -/
theorem evalMle_boolVec_append {k m : ℕ} (t : CMLPolynomialEval R (k + m)) (i : Fin (2 ^ k))
  (s : Vector R m) :
  evalMle t ((boolVec i : Vector R k) ++ s) = evalMle (sliceLow t i) s := by
  rw [evalMle_eq_sum, sum_cube_split, evalMle_eq_sum]
  refine Finset.sum_congr rfl fun j _ => ?_
  rw [Finset.sum_eq_single i]

```

```

· rw [lagrangeBasis_cubeIndex, lowVec_append, highVec_append]
  have h := lagrangeBasis_boolVec (R := R) i i
  simp only [if_true] at h
  simp only [Fin.getElem_fin] at h
  rw [h, one_mul]
  simp [sliceLow]
· intro i' _ hi
  rw [lagrangeBasis_cubeIndex, lowVec_append, highVec_append]
  have h := lagrangeBasis_boolVec (R := R) i i'
  simp only [if_neg hi] at h
  simp only [Fin.getElem_fin] at h
  rw [h, zero_mul, mul_zero]
· intro h
  exact absurd (Finset.mem_univ _) h

/-- Slot 1 of four (two low bits) of an eight-cell table: cells 1 and 5. -/
def packed : CMLPolynomialEval K 3 := #v[10, 11, 12, 13, 20, 21, 22, 23]

#guard (sliceLow (k := 2) (m := 1) packed (1 : Fin 4)).toList = [11, 21]
-- The point `(slot bits 1, 0 | z)` reads the strided slice at `z`.
#guard evalMle packed (#v[1, 0, 7] : Vector K 3) =
  evalMle (#v[11, 21] : CMLPolynomialEval K 1) #v[7]
-- Near miss: the aligned slice at high index 1 is cells 2 and 3, another column.
#guard (slice (k := 1) (m := 2) packed (1 : Fin 4)).toList = [12, 13]

#print axioms evalMle_boolVec_append

end Probe

```

Recorded output.

```

'Probe.evalMle_boolVec_append' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0

```

The counting script of section E (count2.py, scratch, not a probe) (I.7) What it establishes. The counting script of the audit surface of Layer 1 (section E); scratch, not a probe.

Command. `python3 -B count2.py`

```

import re, subprocess, json
files = ["LeanerVM/Protocol/ToCompPoly/Multilinear.lean", "LeanerVM/Protocol/ToCompPoly/BitProductTable.lean",
  "LeanerVM/Protocol/ToCompPoly/Stacking.lean", "LeanerVM/Protocol/ToCompPoly/AmbientStacking.lean",
  "LeanerVM/Protocol/Stack.lean", "LeanerVM/Protocol/Padding.lean", "LeanerVM/Protocol/ClaimWeights.lean",
  "LeanerVM/Protocol/BlockClaims.lean", "LeanerVM/Protocol/FixedColumns.lean"]
decl = re.compile(r'^(?:@[^\[\]]*\s*)?(private\s+)?(def|theorem|abbrev|structure|lemma)\s+(\S+)')
tot = {}
for f in files:
  src = subprocess.run(["git", "show", "b435631:" + f], capture_output=True, text=True, cwd="/home/scaraven/Document_1
    ↪ s/Verified-zkEVM/leanerVM").stdout.split("\n")
  rows=[]
  for i,l in enumerate(src):
    m = decl.match(l)
    if not m or m.group(1): continue
    kind=m.group(2); name=m.group(3)
    j=i; n=0
    if kind in ("def", "abbrev", "structure"):
      while j < len(src) and src[j].strip()!="":
        n+=1; j+=1
    else:
      while True:
        n+=1
        if ':' in src[j]: break
        j+=1
    k=i-1; doc=""
    while k>=0 and src[k].startswith('omit'): k-=1
    if k>=0 and src[k].rstrip().endswith('-/'):
      while k>=0:
        doc+=1
        if src[k].rstrip().startswith('/-'): break
        k-=1

```

```

    rows.append((i+1,kind,name,n,doc))
    tot[f]=rows
    print(f, len(rows), sum(r[3] for r in rows), sum(r[3]+r[4] for r in rows))
    json.dump(tot, open("decls2.json","w"))

```

Recorded output.

```

LeanerVM/Protocol/ToCompPoly/Multilinear.lean 43 98 137
LeanerVM/Protocol/ToCompPoly/BitProductTable.lean 13 30 39
LeanerVM/Protocol/ToCompPoly/Stacking.lean 38 108 147
LeanerVM/Protocol/ToCompPoly/AmbientStacking.lean 10 35 48
LeanerVM/Protocol/Stack.lean 9 34 46
LeanerVM/Protocol/Padding.lean 6 12 20
LeanerVM/Protocol/ClaimWeights.lean 4 12 18
LeanerVM/Protocol/BlockClaims.lean 8 30 40
LeanerVM/Protocol/FixedColumns.lean 12 29 44

```

(columns: public declarations, statement lines, statement lines with docstrings; the split by class uses the lists of section A.)

Not done, unverified, and contradictions with the brief (J)

- **Unverified: the Rust encoder was not executed.** The bytecode table `ValuesProbe` compares against is built by `values.py`'s transcription of `bytecode_columns` and `stacked_bytecode_table`, checked by reading against `cpu/layout.rs:229-309` and `leaf.rs:585-604` (B.6). Running the pinned Rust `bytecode_table` on the same program and comparing would verify it.
- **Unverified: the proposed strided reader and layout combinator** (G.1) are sketched, not built; only the selection lemma is proved (`StridedProbe`).
- **Not run at the new pin.** No probe was run against `144c5aa`; `ValuesProbe` and `StridedProbe` need their K numerals rewritten first (H). The conclusions are about `b435631` and the old pins, which is the review's object (brief §8).
- **How two Lean probes ran** (I): `DuplicatesProbe` and `StridedProbe` were run with the old pin's Lean binary and `LEAN_PATH` on the untouched `b435631` artefacts, not with `lake env lean`, because the checkout had moved; this was before the instruction to stop running Lean reached me. If the orchestrator discounts them, D.2's duplicate claims and B.8's lemma stand as short proofs on paper (a three-line rewrite with `eqTilde_eq_prod`; the mirror of `evalMle_append_boolVec`).
- **Contradiction with the brief's §2** (not of my making): the `leanerVM` checkout was moved during this task (reflog in I), and `.lake/packages/` switched to the new pins; the coordinator's resume message ("nothing else changed") predates it. I moved no checkout, ran no build, and wrote only under `.claude/reports/blueprint-review/` (the dossier and the six `*.out` files).
- **Facts of sibling dossiers used, and checked where I read the same lines:** the tie order and the six shared columns first (`gt-bus.md` A.5, G7; I read `witness.rs:63-102` and `cpu/layout.rs:9-49`); the leaf stacks' pad 1 and no floor (`gt-bus.md` A.5; not re-read); the limb columns as strided slots of `q_flock` (`gt-flock-ring.md` 2.4, 5.4; I read `stack_open.rs:73-98`, `hash_flock.rs:82-115`, `py:878-895`); the table `sumcheck` binds the highest variable first and pads by the product of the high variables (`gt-table-pub.md`; I read `constraints.rs:250-290`, `py:598-626`). All agree with what I read. Nothing in the brief's §7 is contradicted by this task's sources.
- **Not examined:** the rest of the blueprint's Layer 3 (beyond the lines that touch Layer 1), the leaf stacks' block order in code (not built), the pad cells of the committed stack past the last lane (`witness.rs:41-60`; no Layer 1 statement concerns them).

B.2 The opening and the compilation

B.2.1 The opening and the compilation: probes

All files are under `.claude/reports/blueprint-review/probes/gt-opening-compile/`. The two Lean probes were run from the repository root with `flock .claude/reports/blueprint-review/logs/lean.lock lake env lean <file>`, first on 2026-09-29 against the old pins (`leanerVM b435631`, `ArkLib dca90385`,

CompPoly 3468b38c, VCVio f9dc47d9; both `exit=0`), then again on 2026-09-30 after the rebuild, against the new pins (ArkLib 7653a901, CompPoly 572f9973, VCVio a4232d08, Lean 4.34.1; both `exit=0`, log `rerun-newpin.log`). They import only modules the build of main compiles; ArkLib's `Commitments/Functional/Basic.lean` and `FiatShamir/Basic.lean` are **not** compiled by that build (no leanerVM module imports them: 47 ArkLib `.oleans` under `.lake/packages/Arklib/.lake/build` on 2026-09-29, none under `Commitments/`, `FiatShamir/`, `Data/CodingTheory/`), so the two definitions the probes need from them are copied verbatim, with their source lines named, and the copies are what elaborated. The two Python scripts are scratch computations; they run no Lean.

The citation check (`dossiers/verify-gt-opening.md`, section H) found both Lean sources below identical to the files in the probe directory, and `rerun-newpin.log` recording both files at the new pins with `exit=0` and the outputs as quoted; no log of the 2026-09-29 run at the old pins exists in the probe directory or in `logs/`, so that run could not be checked. It re-ran `whir_params.py`: the output is identical to the one below, `True` included, and `verifier_pinned.py` is byte-identical to `git show a386121f:python-verifier/verifier.py`. Its own scratch script, running this port over all 56 supported sizes, corrected three ranges the dossier read off the six printed sizes (section 8.7.3.2): L_0 runs from 110 to 648, the fold terms from 128.19 to 148.68 bits, the list-unioned algebraic terms from 141.0 to 154.2 bits; every term still clears 128.

B.2.1.1 InnerProductOracle.lean: the inner-product interface typechecks, and the opening phase is one query

What it establishes. That the inner-product oracle interface recommended in section 8.7.1.5 is expressible at the pins: (1) ArkLib's `OracleInterface` accepts `Weight n` (leanerVM's structure, in `Type`) as a query type; (2) the evaluation query is the special case of the equality weight, by the built lemma `eqWeight_pair`; (3) ArkLib's `Commitment.Scheme` over that interface has as opening statement (commitment, weight, claimed value): an inner-product commitment scheme; (4) the opening phase “one challenge λ , then one weighted query” is an ArkLib `OracleVerifier` over a one-round schedule. A wrapper `Stack` is used so that the probe's instance does not overlap the built `evalOracle`.

Source.

```

/-
Probe (task gt-opening-compile): is alternative (ii) expressible at the pins?

(ii) gives the stack an inner-product oracle interface: a query is a weight `W` on the cube, the
answer is ` $\sum_w W(w) \cdot q(w)$ ` (specification Definition 3.13). The probe checks that
1. ArkLib's `OracleInterface` accepts `Weight n` (leanerVM's structure, in `Type`) as a query type;
2. the evaluation query is the special case of the equality weight, by the built lemma
   `eqWeight_pair`;
3. ArkLib's `Commitment.Scheme` over that interface has as opening statement
   (commitment, weight, claimed value): an inner-product commitment scheme;
4. the opening phase "one challenge  $\lambda$ , then one weighted query" is an ArkLib `OracleVerifier`
   over a one-round schedule.
A wrapper `Stack` is used so that the probe's instance does not overlap the built `evalOracle`.
-/
import LeanerVM.Protocol.ClaimWeights
import LeanerVM.Protocol.ToArkLib.KeepOracles
import ArkLib.OracleReduction.Security.Basic

open LeanerVM.Protocol LeanerVM.Parameters CompPoly OracleComp OracleSpec ProtocolSpec

namespace Probe

structure Stack (n : ℕ) where
  col : Column n

/-- (ii): a query is a weight, the answer is the pairing. -/
instance innerProductOracle (n : ℕ) : OracleInterface (Stack n) where
  Query := Weight n
  toOC :=
    { spec := (Weight n) → E
      impl := fun W ↦ do return W.pair (← read).col }

theorem answer_eq (n : ℕ) (q : Stack n) (W : Weight n) :
```

```

OracleInterface.answer q W = W.pair q.col := rfl

/-- The evaluation oracle of Layer 0 is the special case `W = eq(p, ·)`. -/
theorem answer_eqWeight (n : ℕ) (q : Stack n) (p : Vector E n) :
  OracleInterface.answer q (eqWeight p)
    = CMLPolynomialEval.evalMle q.col.values (algebraMap K E) p :=
  eqWeight_pair p q.col

/-- Verbatim copy of `Commitment.Opening` (ArkLib `dca90385`,
`ArkLib/Commitments/Functional/Basic.lean:59-64`), whose module the build of `main` did not
compile (no leanerVM module imports it). -/
structure OpeningCopy {ι : Type} (oSpec : OracleSpec ι) (Data Commitment Decommitment ComKey VerifKey : Type)
  [0 : OracleInterface Data] {n : ℕ} (pSpec : ProtocolSpec n) where
  opening : (ComKey × VerifKey) →
    Proof oSpec (Commitment × (q : 0.Query) × 0.Response q) (Data × Decommitment) pSpec

/-- Over the inner-product interface the statement of the opening is a commitment, a weight and
a claimed value: ArkLib's functional commitment scheme is then an inner-product commitment
scheme. -/
example (n : ℕ) {ι : Type} (oSpec : OracleSpec ι) (Cm Dec CK VK : Type) {m : ℕ}
  (pSpec : ProtocolSpec m)
  (S : OpeningCopy oSpec (Stack n) Cm Dec CK VK pSpec) (ck : CK) (vk : VK) :
  Proof oSpec (Cm × (W : Weight n) × E) (Stack n × Dec) pSpec :=
  S.opening (ck, vk)

/-! ## The opening phase as "λ, then one weighted query" -/

/-- One verifier message, the batching challenge. -/
@[reducible]
def openSpec : ProtocolSpec 1 := (!v[V.to_P], !v[E])

instance : ∀ i, OracleInterface (openSpec.Message i)
  | (0, h) => nomatch h

instance : ∀ i, SampleableType (openSpec.Challenge i)
  | (0, _) => (inferInstance : SampleableType E)

abbrev OneStack (n : ℕ) : Fin 1 → Type := fun _ ↦ Stack n

/-- A pooled claim. -/
structure Claim (n : ℕ) where
  weight : Weight n
  value : E

/-- `W_λ = Σ_j λ^j W_j` on the cube. (The probe takes the extension as the evaluator; the closed
form the compiled verifier runs is `Σ_j λ^j W_j.mle`, equal to it by linearity.) -/
def batchWeight {n : ℕ} (lam : E) (cs : List (Claim n)) : Weight n where
  onCube := Vector.ofFn fun i ↦ (cs.zipIdx.map fun c ↦ lam ^ c.2 * c.1.weight.onCube.get i).sum
  mle := fun r ↦ CMLPolynomialEval.evalMle _ r
  mle_eq := fun _ ↦ rfl

/-- `C_λ = Σ_j λ^j c_j`. -/
def batchValue {n : ℕ} (lam : E) (cs : List (Claim n)) : E :=
  (cs.zipIdx.map fun c ↦ lam ^ c.2 * c.1.value).sum

/-- The one query of the opening phase. -/
def queryStack {n : ℕ} (W : Weight n) : OracleComp [OneStack n] E :=
  liftM <| OracleSpec.query (show [OneStack n].Domain from (0, W))

/-- The opening verifier of (ii): read λ, query the stack at `W_λ`, compare with `C_λ`. -/
def openVerifier (n : ℕ) :
  OracleVerifier [] (List (Claim n)) (OneStack n) Unit (OneStack n) openSpec where
  verify := fun cs chals ↦ do
    let lam : E := chals (0, rfl)
    let a ← liftM (queryStack (batchWeight lam cs))
    if a = batchValue lam cs then pure () else failure
  outputOracle := .inl (keepOracles (OneStack n) openSpec)

#check @openVerifier
#print axioms openVerifier
#print axioms answer_eqWeight

end Probe

```

.claude/reports/blueprint-review/probes/gt-opening-compile/InnerProductOracle.lean; OpeningCopy is a verbatim copy of ArkLib dca90385, ArkLib/Commitments/Functional/Basic.lean:59-64

Command.

```
flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/gt-opening-compile/InnerProductOracle.lean
```

Output on 2026-09-29 (old pins: leanerVM b435631, ArkLib dca90385) and on 2026-09-30 (new pins: ArkLib 7653a901, Lean 4.34.1), identical up to the date:

```
InnerProductOracle.lean:56:23: warning: Variable name `W` is not explicitly referenced. [...]
openVerifier : (n : ℕ) → OracleVerifier []o (List (Claim n)) (OneStack n) Unit (OneStack n) openSpec
'Probe.openVerifier' depends on axioms: [propext, Classical.choice, Quot.sound]
'Probe.answer_eqWeight' depends on axioms: [propext, Classical.choice, Quot.sound]
exit=0
```

The warning is the unused binder `W` in the copied `OpeningCopy`; harmless. The two declarations the conclusion rests on use only the three standard axioms.

B.2.1.2 FiatShamirChain.lean: what the Fiat–Shamir transform takes, and a chain plugged in

What it establishes. What ArkLib’s Fiat–Shamir transform takes as the challenge oracle, and whether a concrete hash chain can be plugged in (used in section 8.7.3.1). `Verifier.fiatShamir` is copied verbatim from ArkLib dca90385, ArkLib/OracleReduction/FiatShamir/Basic.lean:129-136, with the section variables of :68-70 made explicit; everything it uses (`NonInteractiveVerifier`, `fsChallengeOracle`, `Messages.deriveTranscriptFS`) is in compiled modules.

Source.

```
/-
Probe (task gt-opening-compile): what does ArkLib's Fiat–Shamir transform take as the challenge
oracle, and can a concrete hash chain be plugged in?

`ArkLib/OracleReduction/FiatShamir/Basic.lean` is not compiled by the build of `main` (no leanerVM
module imports it), so `Verifier.fiatShamir` is copied verbatim from ArkLib `dca90385`,
`FiatShamir/Basic.lean:129-136`, with the section variables of `:68-70` made explicit. Everything
it uses (`NonInteractiveVerifier`, `fsChallengeOracle`, `Messages.deriveTranscriptFS`) is in
compiled modules.
-/
import ArkLib.OracleReduction.Security.Basic

open ProtocolSpec OracleComp OracleSpec

namespace Probe

variable {n : ℕ} {pSpec : ProtocolSpec n} {ι : Type} {oSpec : OracleSpec ι}
{StmtIn StmtOut : Type}
[VCVCompatible StmtIn] [V ι, VCVCompatible (pSpec.Challenge ι)]

/-- Verbatim: the (slow) Fiat-Shamir transformation for the verifier. -/
def fiatShamirCopy (V : Verifier oSpec StmtIn StmtOut pSpec) :
  NonInteractiveVerifier (V ι, pSpec.Message ι) (oSpec + fsChallengeOracle StmtIn pSpec)
  StmtIn StmtOut where
  verify := fun stmtIn proof => do
    let messages : pSpec.Messages := proof 0
    let transcript ← (messages.deriveTranscriptFS (oSpec := oSpec) stmtIn)
    Option.getM (← (V.verify stmtIn transcript).run)

/-- The index of a challenge-oracle query: a round that is a challenge, with the statement and
the messages sent before it. -/
example : (fsChallengeOracle StmtIn pSpec).Domain
```

```

    = ((i : pSpec.ChallengeIdx) × (StmtIn × pSpec.MessagesUpTo i.l.castSucc)) := rfl

/-- A "hash chain" in the only form the transform can see: a function from the statement and the
messages so far to the challenge of each round. -/
abbrev Chain (StmtIn : Type) (pSpec : ProtocolSpec n) :=
  (i : pSpec.ChallengeIdx) → StmtIn × pSpec.MessagesUpTo i.l.castSucc → pSpec.Challenge i

/-- The chain as a deterministic implementation of the challenge oracle (no shared oracle). -/
def chainImpl (chain : Chain StmtIn pSpec) :
  QueryImpl ([]₀ + fsChallengeOracle StmtIn pSpec) Id
  | .inl q => PEmpty.elim q
  | .inr (i, t) => chain i t

/-- The compiled verifier run under a concrete chain: a deterministic function of the statement
and of ALL the prover's messages. -/
def runWithChain (V : Verifier []₀ StmtIn StmtOut pSpec) (chain : Chain StmtIn pSpec)
  (stmt : StmtIn) (messages : ∀ i, pSpec.Message i) : Option StmtOut :=
  (simulateQ (chainImpl chain) ((fiatShamirCopy V).verify stmt (fun | 0 => messages))).run

#check @runWithChain
#print axioms runWithChain

-- An oracle verifier is not a `Verifier`: it must be converted first, and the conversion's
-- statement and messages are the oracles in the clear.
#check @OracleVerifier.toVerifier

end Probe

```

.claude/reports/blueprint-review/probes/gt-opening-compile/FiatShamirChain.lean; fiatShamirCopy is a verbatim copy of ArkLib dca90385, ArkLib/OracleReduction/FiatShamir/Basic.lean:129-136

Command.

```

flock .claude/reports/blueprint-review/logs/lean.lock lake env lean
↪ .claude/reports/blueprint-review/probes/gt-opening-compile/FiatShamirChain.lean

```

Output (both runs, 2026-09-29 at the old pins and 2026-09-30 at the new pins):

```

@runWithChain : {n : ℕ} → {pSpec : ProtocolSpec n} → {StmtIn StmtOut : Type} →
  Verifier []₀ StmtIn StmtOut pSpec → Chain StmtIn pSpec → StmtIn →
  ((i : pSpec.MessageIdx) → pSpec.Message i) → Option StmtOut
'Probe.runWithChain' depends on axioms: [propext, Quot.sound]
@OracleVerifier.toVerifier : ... → OracleVerifier oSpec StmtIn OStmtIn OStmtOut pSpec →
  Verifier oSpec (StmtIn × ((i : ιₛᵢ) → OStmtIn i)) (StmtOut × ((i : ιₛₒ) → OStmtOut i)) pSpec
exit=0

```

Reading. The copied `Verifier.fiatShamir` elaborates without the `VCVCompatible` instances of its file's variable line (they are unused by the definition); the challenge oracle's query is the pair (statement, messages so far) and any function of that shape is an implementation; the compiled verifier under a chain is a deterministic `Option StmtOut`; an oracle verifier's plain form carries the oracle statements and messages in the clear.

B.2.1.3 `whir_params.py` and `bus_depth.py`: the parameters and the bus depth (scratch, Python 3; no Lean)

What `whir_params.py` establishes. It ports `whir_config.rs:260-311, 419-707` (crates/pcs/src/whir_config.rs at a386121f: the ladder and the per-level search) to Python floats and compares with the pinned `python-verifier/verifier.py` (a copy extracted with `git show a386121f:python-verifier/verifier.py` as `verifier_pinned.py`, since the working tree moved). It reproduces the query table of `verifier.py:910` for every (ρ, μ) and reads off η , m , the Johnson list bound L and the per-term security bits (table 8.26).

Command. In the probe directory, run on 2026-09-29 against the then-pinned working tree and on 2026-09-30 against the extracted copy; same output:

```
PYTHONDONTWRITEBYTECODE=1 python3 whirl_params.py
```

Source.

```
# Scratch port of crates/pcs/src/whirl_config.rs (pin a386121f) parameter derivation,
# to reproduce the query table of python-verifier/verifier.py:910 and to read off
# eta, m, the Johnson list bound L and the per-term security bits.
import math, sys
sys.dont_write_bytecode = True
sys.path.insert(0, ".")

SECURITY_BITS = 128
QUERY_GRINDING_BITS = 17
INITIAL_FOLDING_FACTOR = 6
SUBSEQUENT_FOLDING_FACTOR = 4
RS_INIT_RED = 3
RS_SUB_RED = 1
RESIDUAL_MAX_LOG = 5
LOG_Q = 192.0
RING_DEG = (1 << 31) + (1 << 15) + (1 << 7) + (1 << 3) + (1 << 1) + 1
M_MAX = 4096

def reduced_rate(lir, cols):
    dim = 2.0 ** cols
    return (dim - 1.0) / 2.0 ** (cols + lir)

def m_param(lir, cols, eta):
    s = math.sqrt(reduced_rate(lir, cols))
    return float(max(math.ceil(s / eta), 3))

def thm_log_a(lir, eta, cols):
    rho = reduced_rate(lir, cols)
    s = math.sqrt(rho)
    gamma = 1.0 - s - eta
    m = m_param(lir, cols, eta)
    half = m + 0.5
    num = 2.0 * half ** 5 + 3.0 * half * gamma * rho
    den = 3.0 * rho ** 1.5
    n = 2.0 ** (cols + lir)
    a = (num / den) * n + half / s
    return math.log2(a)

def log_a(lir, eta, cols, ilv):
    return thm_log_a(lir, eta, cols) + max(ilv - 1.0, 0.0)

def per_q(lir, cols, eta):
    rho = reduced_rate(lir, cols)
    gamma = 1.0 - math.sqrt(rho) - eta
    return math.log2(1.0 / (1.0 - gamma))

def list_log2(lir, cols, eta):
    rho = reduced_rate(lir, cols)
    return math.log2(1.0 / (2.0 * eta * math.sqrt(rho)))

def alg_bits(lir, cols, eta, prevq, ood):
    deg = max(RING_DEG, prevq + ood, 2)
    return LOG_Q - math.log2(deg) - list_log2(lir, cols, eta)

def ood_bits(lir, cols, eta, mu, s):
    l = list_log2(lir, cols, eta)
    lm = math.log2(mu)
    if s == 0:
        return LOG_Q - l - lm
    return s * (LOG_Q - lm) - (2.0 * l - 1.0)

def eta_for_m(lir, cols, m):
    s = math.sqrt(reduced_rate(lir, cols))
    eta = s / m
```

```

import struct
while m_param(lir, cols, eta) > m:
    eta = math.nextafter(eta, math.inf)
return eta

def optimize(level, lir, cols, ilv, prevq):
    target = float(SEcurity_BITS)
    qtargt = float(max(SEcurity_BITS - QUERY_GRINDING_BITS, 1))
    mu = cols + ilv
    block = 1 << (cols + lir)
    best = None
    for m in range(3, M_MAX + 1):
        eta = eta_for_m(lir, cols, m)
        max_eta = 1.0 - math.sqrt(reduced_rate(lir, cols))
        if eta >= max_eta:
            continue
        pg = LOG_Q - log_a(lir, eta, cols, ilv)
        if pg + 1e-12 < target:
            break
        pq = per_q(lir, cols, eta)
        if not math.isfinite(pq) or pq <= 0:
            continue
        q = math.ceil(qtargt / pq)
        if q > block:
            continue
        if level == 0:
            s = 0
        else:
            s = next((s for s in range(1, 9) if ood_bits(lir, cols, eta, mu, s) + 1e-12 >= target), None)
            if s is None:
                continue
        if ood_bits(lir, cols, eta, mu, s) + 1e-12 < target or alg_bits(lir, cols, eta, prevq, s) + 1e-12 <
            ⇨ target:
            continue
        if best is None or q < best["queries"]:
            best = dict(eta=eta, queries=q, ood=s, m=m, pg=pg, pq=pq,
                        L=2.0 ** list_log2(lir, cols, eta),
                        oodbits=ood_bits(lir, cols, eta, mu, s),
                        alg=alg_bits(lir, cols, eta, prevq, s),
                        qbits=q * pq, rho=reduced_rate(lir, cols),
                        gamma=1.0 - math.sqrt(reduced_rate(lir, cols)) - eta)
    return best

def ladder(log_n, lir):
    rates, cols, ilv, ks = [lir], [log_n - INITIAL_FOLDING_FACTOR], [INITIAL_FOLDING_FACTOR],
    ⇨ [INITIAL_FOLDING_FACTOR]
    n_run, r_run, f_run, red = log_n - INITIAL_FOLDING_FACTOR, lir, INITIAL_FOLDING_FACTOR, RS_INIT_RED
    while n_run > RESIDUAL_MAX_LOG:
        k = min(SUBSEQUENT_FOLDING_FACTOR, n_run)
        nxt = n_run - k
        rate = r_run + (f_run - red)
        red = RS_SUB_RED
        rates.append(rate); cols.append(nxt); ilv.append(k); ks.append(k)
        n_run -= k; r_run = rate; f_run = k
    return rates, cols, ilv, ks, n_run

def derive(log_n, lir):
    rates, cols, ilv, ks, yr = ladder(log_n, lir)
    levels, prevq = [], 0
    for i in range(len(rates)):
        b = optimize(i, rates[i], cols[i], ilv[i], prevq)
        assert b is not None, (log_n, lir, i)
        b.update(rate=rates[i], cols=cols[i], ilv=ilv[i], k=ks[i])
        levels.append(b)
        prevq = b["queries"]
    return levels, yr

if __name__ == "__main__":
    import verifier_pinned as V
    ok = True
    for lir in range(1, 5):
        for mu in range(15, 29):
            levels, yr = derive(mu, lir)

```

```

mine = tuple(l["queries"] for l in levels)
tab = V.WHIR_QUERIES[lir - 1][mu - 15]
cfg = V.derive_config(mu, lir)
same = (mine == tab) and tuple(l["rate"] for l in levels) == cfg.log_inv_rates and tuple(l["k"] for
    ↪ l in levels) == cfg.folds
ok &= same
if not same:
    print("MISMATCH", lir, mu, mine, tab)
print("query table of verifier.py:910 reproduced from the whir_config.rs derivation:", ok)
for (mu, lir) in [(15, 1), (18, 1), (22, 1), (28, 1), (28, 2), (28, 4)]:
    levels, yr = derive(mu, lir)
    print(f"\nmu={mu} log_inv_rate={lir} residual={yr}")
    for i, l in enumerate(levels):
        print(f"  L{i}: fold k={l['k']} rate=2^{l['rate']} msg_cols=2^{l['cols']} rho_red={l['rho']:.6f} "
            f"m={l['m']} eta={l['eta']:.5f} gamma={l['gamma']:.5f} L<= {l['L']:.2f} "
            f"queries={l['queries']} ood={l['ood']} | bits: mca-fold={l['pg']:.2f} "
            ↪ query={l['qbits']:.2f}(+17 grind) "
            f"ood={l['oodbits']:.2f} algebraic(list-unioned)={l['alg']:.2f}")

```

.claude/reports/blueprint-review/probes/gt-opening-compile/whir_params.py

Output (2026-09-30):

```

query table of verifier.py:910 reproduced from the whir_config.rs derivation: True

mu=15 log_inv_rate=1 residual=5
  L0: fold k=6 rate=2^-1 msg_cols=2^9 rho_red=0.499023 m=395 eta=0.00179 gamma=0.29180 L<= 395.77 queries=223
    ↪ ood=0 | bits: mca-fold=132.94 query=111.00(+17 grind) ood=179.46 algebraic(list-unioned)=152.72
  L1: fold k=4 rate=2^-4 msg_cols=2^5 rho_red=0.060547 m=306 eta=0.00080 gamma=0.75313 L<= 2526.97 queries=55
    ↪ ood=1 | bits: mca-fold=133.22 query=111.00(+17 grind) ood=167.22 algebraic(list-unioned)=149.70

mu=18 log_inv_rate=1 residual=4
  L0: fold k=6 rate=2^-1 msg_cols=2^12 rho_red=0.499878 m=311 eta=0.00227 gamma=0.29071 L<= 311.08 queries=224
    ↪ ood=0 | bits: mca-fold=131.67 query=111.00(+17 grind) ood=179.55 algebraic(list-unioned)=152.72
  L1: fold k=4 rate=2^-4 msg_cols=2^8 rho_red=0.062256 m=70 eta=0.00356 gamma=0.74692 L<= 562.20 queries=56
    ↪ ood=1 | bits: mca-fold=140.88 query=111.01(+17 grind) ood=171.15 algebraic(list-unioned)=151.87
  L2: fold k=4 rate=2^-7 msg_cols=2^4 rho_red=0.007324 m=19 eta=0.00450 gamma=0.90991 L<= 1297.07 queries=32
    ↪ ood=1 | bits: mca-fold=146.52 query=111.12(+17 grind) ood=169.32 algebraic(list-unioned)=150.66

mu=22 log_inv_rate=1 residual=4
  L0: fold k=6 rate=2^-1 msg_cols=2^16 rho_red=0.499992 m=216 eta=0.00327 gamma=0.28962 L<= 216.00 queries=225
    ↪ ood=0 | bits: mca-fold=130.29 query=111.00(+17 grind) ood=179.79 algebraic(list-unioned)=153.25
  L1: fold k=4 rate=2^-4 msg_cols=2^12 rho_red=0.062485 m=80 eta=0.00312 gamma=0.74691 L<= 640.16 queries=56
    ↪ ood=1 | bits: mca-fold=135.93 query=111.01(+17 grind) ood=170.36 algebraic(list-unioned)=151.68
  L2: fold k=4 rate=2^-7 msg_cols=2^8 rho_red=0.007782 m=42 eta=0.00210 gamma=0.90968 L<= 2698.54 queries=32
    ↪ ood=1 | bits: mca-fold=137.03 query=111.00(+17 grind) ood=166.62 algebraic(list-unioned)=149.60
  L3: fold k=4 rate=2^-10 msg_cols=2^4 rho_red=0.000916 m=7 eta=0.00432 gamma=0.96542 L<= 3822.93 queries=23
    ↪ ood=1 | bits: mca-fold=145.91 query=111.64(+17 grind) ood=166.20 algebraic(list-unioned)=149.10

mu=28 log_inv_rate=1 residual=2
  L0: fold k=6 rate=2^-1 msg_cols=2^22 rho_red=0.500000 m=110 eta=0.00643 gamma=0.28647 L<= 110.00 queries=228
    ↪ ood=0 | bits: mca-fold=129.15 query=111.02(+17 grind) ood=180.41 algebraic(list-unioned)=154.22
  L1: fold k=4 rate=2^-4 msg_cols=2^18 rho_red=0.062500 m=81 eta=0.00309 gamma=0.74691 L<= 648.00 queries=56
    ↪ ood=1 | bits: mca-fold=129.84 query=111.01(+17 grind) ood=169.86 algebraic(list-unioned)=151.66
  L2: fold k=4 rate=2^-8 msg_cols=2^14 rho_red=0.007812 m=46 eta=0.00192 gamma=0.90969 L<= 2944.18 queries=32
    ↪ ood=1 | bits: mca-fold=130.39 query=111.01(+17 grind) ood=165.78 algebraic(list-unioned)=149.48
  L3: fold k=4 rate=2^-10 msg_cols=2^10 rho_red=0.000976 m=8 eta=0.00390 gamma=0.96486 L<= 4100.00 queries=23
    ↪ ood=1 | bits: mca-fold=139.15 query=111.11(+17 grind) ood=165.19 algebraic(list-unioned)=149.00
  L4: fold k=4 rate=2^-13 msg_cols=2^6 rho_red=0.000120 m=4 eta=0.00274 gamma=0.98630 L<= 16644.06 queries=18
    ↪ ood=1 | bits: mca-fold=140.20 query=111.41(+17 grind) ood=161.63 algebraic(list-unioned)=146.98
  L5: fold k=4 rate=2^-16 msg_cols=2^2 rho_red=0.000011 m=5 eta=0.00068 gamma=0.99594 L<= 218453.33 queries=14
    ↪ ood=1 | bits: mca-fold=134.67 query=111.22(+17 grind) ood=154.94 algebraic(list-unioned)=143.26

mu=28 log_inv_rate=2 residual=2
  L0: fold k=6 rate=2^-2 msg_cols=2^22 rho_red=0.250000 m=82 eta=0.00610 gamma=0.49390 L<= 164.00 queries=113
    ↪ ood=0 | bits: mca-fold=128.75 query=111.02(+17 grind) ood=179.84 algebraic(list-unioned)=153.64
  L1: fold k=4 rate=2^-5 msg_cols=2^18 rho_red=0.031250 m=43 eta=0.00411 gamma=0.81911 L<= 688.00 queries=45
    ↪ ood=1 | bits: mca-fold=131.87 query=111.01(+17 grind) ood=169.69 algebraic(list-unioned)=151.57
  L2: fold k=4 rate=2^-8 msg_cols=2^14 rho_red=0.003906 m=40 eta=0.00156 gamma=0.93594 L<= 5120.31 queries=28
    ↪ ood=1 | bits: mca-fold=128.89 query=111.00(+17 grind) ood=164.19 algebraic(list-unioned)=148.68
  L3: fold k=4 rate=2^-11 msg_cols=2^10 rho_red=0.000488 m=7 eta=0.00316 gamma=0.97476 L<= 7175.01 queries=21
    ↪ ood=1 | bits: mca-fold=137.55 query=111.47(+17 grind) ood=163.58 algebraic(list-unioned)=148.19

```

```

L4: fold k=4 rate=2^-14 msg_cols=2^6 rho_red=0.000060 m=3 eta=0.00258 gamma=0.98967 L<= 24966.10 queries=17
  ↳ ood=1 | bits: mca-fold=139.51 query=112.14(+17 grind) ood=160.46 algebraic(list-unioned)=146.39
L5: fold k=4 rate=2^-17 msg_cols=2^2 rho_red=0.000006 m=9 eta=0.00027 gamma=0.99734 L<= 786432.00 queries=13
  ↳ ood=1 | bits: mca-fold=128.22 query=111.22(+17 grind) ood=151.25 algebraic(list-unioned)=141.42

mu=28 log_inv_rate=4 residual=2
L0: fold k=6 rate=2^-4 msg_cols=2^22 rho_red=0.062500 m=27 eta=0.00926 gamma=0.74074 L<= 216.00 queries=57
  ↳ ood=0 | bits: mca-fold=131.68 query=111.01(+17 grind) ood=179.44 algebraic(list-unioned)=153.25
L1: fold k=4 rate=2^-7 msg_cols=2^18 rho_red=0.007812 m=11 eta=0.00804 gamma=0.90358 L<= 704.00 queries=33
  ↳ ood=1 | bits: mca-fold=136.47 query=111.36(+17 grind) ood=169.62 algebraic(list-unioned)=151.54
L2: fold k=4 rate=2^-10 msg_cols=2^14 rho_red=0.000977 m=8 eta=0.00391 gamma=0.96484 L<= 4096.25 queries=23
  ↳ ood=1 | bits: mca-fold=135.15 query=111.09(+17 grind) ood=164.83 algebraic(list-unioned)=149.00
L3: fold k=4 rate=2^-13 msg_cols=2^10 rho_red=0.000122 m=4 eta=0.00276 gamma=0.98620 L<= 16400.02 queries=18
  ↳ ood=1 | bits: mca-fold=136.23 query=111.22(+17 grind) ood=161.19 algebraic(list-unioned)=147.00
L4: fold k=4 rate=2^-16 msg_cols=2^6 rho_red=0.000015 m=3 eta=0.00129 gamma=0.99483 L<= 99864.38 queries=15
  ↳ ood=1 | bits: mca-fold=134.51 query=113.94(+17 grind) ood=156.46 algebraic(list-unioned)=144.39
L5: fold k=4 rate=2^-19 msg_cols=2^2 rho_red=0.000001 m=3 eta=0.00040 gamma=0.99841 L<= 1048576.00 queries=12
  ↳ ood=1 | bits: mca-fold=130.43 query=111.51(+17 grind) ood=150.42 algebraic(list-unioned)=141.00

```

A caveat on floating point: the port uses Python's `math` where the Rust uses `f64` methods; the reproduction of every query count of the table for all 56 (ρ, μ) pairs is the evidence that the two agree where it matters. The m , η and bit values are the port's; any last-ulp difference would not move them visibly.

What `bus_depth.py` establishes. It runs the pinned verifier's own `build_layout` and `bus_layout` over a covering grid of sizes to find the maximal bus depth under the stacking window, and at the per-log caps (section 8.7.3.2). The dossier records its output, not a separate command line; it imports the same extracted copy `verifier_pinned.py`.

Source.

```

# Scratch: bus depth and blueprint error sums at admissible sizes, from the pinned Python
# verifier's own layout code (python-verifier/verifier.py at a386121f).
import sys, math, itertools
sys.dont_write_bytecode = True
sys.path.insert(0, ".")
import verifier_pinned as V

print("tables:", [(t.opcode, t.width, len(t.flushes.push), len(t.flushes.pull), len(t.count_columns),
  ↳ t.n_constraints) for t in V.TABLES])

def depths(log_mem, taus, kbc):
    lay = V.build_layout(range(16 * 2**kbc), log_mem, taus)
    fr = (0, lay.log_memory, lay.log_bytecode)
    push = V.bus_layout(fr, lay.push)
    pull = V.bus_layout(fr, lay.pull)
    cnt = V.bus_layout((), lay.count)
    return lay.stack_log, push.depth, pull.depth, cnt.depth

best = None
# exhaustive over a coarse but covering grid: all taus equal to t or at the floor, memory, bytecode
for log_mem in range(16, 27):
    for kbc in range(0, 29):
        for t in range(0, 27):
            for pattern in itertools.product([0, 1], repeat=6):
                taus = [t if b else 0 for b in pattern]
                taus[5] = max(taus[5], 3)
                try:
                    mu, dp, dl, dc = depths(log_mem, taus, kbc)
                except V.VerificationError:
                    continue
                if not (15 <= mu <= 28):
                    continue
                if best is None or dp > best[0]:
                    best = (dp, mu, log_mem, tuple(taus), kbc, dl, dc)
print("max bus depth found under the window mu<=28:", best)
# at the per-log caps alone (ignoring the stack window)
mu, dp, dl, dc = depths(32, [32]*6, 28)
print("per-log caps (mem 32, tau 32, kbc 28), ignoring the window: stack_log", mu, "bus depth", dp, dl, dc)

```


.claude/reports/blueprint-review/probes/gt-opening-compile/bus_depth.py

Output (2026-09-30, same as 2026-09-29):

```
tables: [(0, 15, 5, 5, 4, 0), (1, 15, 5, 5, 4, 0), (2, 8, 3, 3, 2, 0), (3, 15, 5, 5, 4, 0), (4, 14, 5, 5, 4,
↪ 2), (5, 37, 11, 11, 10, 0)]
max bus depth found under the window mu<=28: (28, 28, 16, (0, 0, 23, 0, 23, 3), 26, 28, 26)
per-log caps (mem 32, tau 32, kbc 28), ignoring the window: stack_log 41 bus depth 38 38 37
```

Per table: opcode, width, push blocks, pull blocks, count columns, constraints. The grid is not exhaustive over all 33^6 height vectors, so “28” is a lower bound on the maximum that the bus dossier’s counting argument (gt-bus.md, section D.4) confirms as the maximum.

B.3 The boundary with leanISA

The eight probes of the boundary dossier. Every file is under .claude/reports/blueprint-review/probes/boundary-adaptor/ and was run from the repository root under the shared lock; the recorded output follows each source, with the exit status on its last line. An empty output with `exit=0` means every `#guard` passed and every `example` elaborated. `Shapes`, `KnownColumn`, `Transport`, `UniverseFail`, `DefInstance` and `PolyBridge` ran on 2026-09-29 against `main` at `b435631` with the old pins (ArkLib `dca90385`, CompPoly `3468b38c`, Clean `93c9d1ef`). Two of them use numerals of `K` other than `0` and `1` (`KnownColumn`: `2`; `PolyBridge`: `3, 5, 7`), which at the new CompPoly pin mean `0, 1, 1, 1`; their conclusions stand at the old pin, and both were re-run at the new pins, with `K.ofBits n` in place of the numerals, as `KnownColumnNew` and `PolyBridgeNew` on 2026-09-30. The other four were not re-run; nothing they rest on changed between the two revisions of the proof-system sources, which differ in proofs and imports only. Line numbers are shown beside sources and outputs.

B.3.1 Shapes

What it establishes. The two relations, their universes and the public input. It prints the types of `EnsembleWitness`, `Air.Flat.Table` and `Component`, all in `Type 1` (output lines 1–4), of `M3Rel` and `Column`, in `Type`, of `Refinement`, whose two witness universes are independent (output lines 10–13), and of `Extractor`. `Straightline.map`, whose `WitIn'` is in `Type`; the statements of `SatisfiedBy` (output lines 29–55), `Caps`, `PublicInput` and its two words (69–78), `ValidExecution`, `HasPublicBoundary`, `Program`, `M3Holds` and `M3Instance` (100–134), `PublicLine`; and what `w.Constraints` unfolds to, Clean’s `ConstraintsHold` with constraints and lookups only, no bus guarantees (154–159). Its four `examples` prove that the top limb of both public words is zero for every input (source lines 57–61), that a word of the image whose top limb is nonzero is not `p.word0`, whatever the input (63–69), and that the three public lines give `word0_eq` (71–76); the dossier cites these source lines as 45–63, twelve lines too low. The axioms of every arithmetization declaration the chain will use, of the two master theorems and of `bytecodeColumn_eval`, `idxColumn_eval` are the kernel’s three; `Refinement.map_option_valid` depends on two (output lines 168–187). Used in sections 8.8.1.1 and 8.8.2.5.

Revision. Run on 2026-09-29 against leanerVM `main` at `b435631` with the old pins (ArkLib `dca90385`, CompPoly `3468b38c`, Clean `93c9d1ef`). Exit 0. Not re-run at the new pins; nothing it rests on changed between the two revisions of the proof-system sources (proofs and imports only).

Source.

```
1  /-
2  Probe (boundary-adaptor): the two relations, their universes, and the public input.
3  Scratch work for the blueprint review; not part of the repository.
4  Run from the repository root:
5      flock .claude/reports/blueprint-review/logs/lean.lock \
6          lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/Shapes.lean
7  -/
```

```

8   import LeanerVM
9
10  open LeanerVM LeanerVM.Parameters LeanerVM.Semantics LeanerVM.Arithmetization
11  open LeanerVM.Protocol
12  open Air.Flat (Ensemble EnsembleWitness Component)
13
14  set_option pp.universes true
15
16  /-! ## 1. Universes: the witness of `SatisfiedBy` against the witness of `M3Holds` -/
17
18  #check @EnsembleWitness
19  #check @Air.Flat.Table
20  #check @Air.Flat.Component
21  #check @leanIsaEnsemble
22  #check @SatisfiedBy
23  #check @M3Holds
24  #check @M3Rel
25  #check @Column
26  #check @Refinement
27  #check @Refinement.map_option_valid
28  #check @Extractor.Straightline.map
29
30  set_option pp.universes false
31
32  /-! ## 2. The statement types as written -/
33
34  #print SatisfiedBy
35  #print Caps
36  #print PublicInput
37  #print PublicInput.word0
38  #print PublicInput.word1
39  #print PublicIO
40  #print ValidExecution
41  #print HasPublicBoundary
42  #print Program
43  #print M3Holds
44  #print M3Instance
45  #print PublicLine
46
47  /-! ## 3. Which `ConstraintsHold` does `w.Constraints` unfold to? -/
48
49  #print Air.Flat.EnsembleWitness.Constraints
50  #print Air.Flat.Table.Constraints
51  #print Operations.ConstraintsHold
52  #print Air.Flat.EnsembleWitness.interactions
53  #print ProverData
54
55  /-! ## 4. The top limb of a public word is zero by construction -/
56
57  example (p : PublicInput) : p.word0.limb 2 = 0 := by
58    simp [PublicInput.word0]
59
60  example (p : PublicInput) : p.word1.limb 2 = 0 := by
61    simp [PublicInput.word1]
62
63  /- A word of the image whose top limb is nonzero is not a public word, whatever the input: the
64  fact `word0_eq` needs from the third public line. -/
65  example (p : PublicInput) (a b c : K) (hc : c ≠ 0) : E.ofLimbs a b c ≠ p.word0 := by
66    intro h
67    have := congrArg (fun z : E ↦ z.limb 2) h
68    simp [PublicInput.word0] at this
69    exact hc this
70
71  /- With the top limb zero and the two low limbs those of the statement, the word is the public
72  word: the three public lines give `word0_eq`. -/
73  example (p : PublicInput) (a b c : K) (ha : a = p.word0.limb 0) (hb : b = p.word0.limb 1)
74    (hc : c = 0) : E.ofLimbs a b c = p.word0 := by
75    subst ha hb hc
76    simp [PublicInput.word0]
77
78  /-! ## 5. Axioms of what the chain will use on the arithmetization side, and of the master
79  theorems -/
80

```

```

81 #print axioms assumptions_of_blake2sRowsValid
82 #print axioms memRowOf_bindings
83 #print axioms bytecodeRowOf_bindings
84 #print axioms bytecodeRowOf_decodes
85 #print axioms verifier_pull_eval
86 #print axioms verifier_push_eval
87 #print axioms rowAt_toElements
88 #print axioms decode_eq_some_iff
89 #print axioms assignmentRepresents_image
90 #print axioms xorTable
91 #print axioms jumpTable
92 #print axioms blake2sTable
93 #print axioms memTable
94 #print axioms bytecodeTable
95 #print axioms leanIsaVerifier
96 #print axioms piop_rbrKnowledgeSoundness
97 #print axioms piop_perfectCompleteness
98 #print axioms Refinement.map_option_valid
99 #print axioms bytecodeColumn_eval
100 #print axioms idxColumn_eval

```

.claude/reports/blueprint-review/probes/boundary-adaptor/Shapes.lean

Command (from the repository root).

```

flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/Shapes.lean

```

Recorded output (Shapes.out).

```

1  @EnsembleWitness : {F : Type} →
2    [inst : FiniteField F] → {PublicIO : TypeMap} → [inst_1 : ProvableType PublicIO] → Ensemble F PublicIO → Type
3    ↪ 1
4  Air.Flat.Table : (F : Type) → [FiniteField F] → Type 1
5  Component : (F : Type) → [FiniteField F] → Type 1
6  leanIsaEnsemble : Program → Ensemble K PublicIO
7  SatisfiedBy : (prog : Program) → PublicInput → EnsembleWitness (leanIsaEnsemble prog) → Prop
8  M3Holds : (I : M3Instance) → I.Stmt → Column I.μ → Prop
9  M3Rel : (I : M3Instance) → Set.{0} (Prod.{0, 0} (Prod.{0, 0} I.Stmt ((i : Fin 0) → NoOracle i)) (Column I.μ))
10 Column : ℕ → Type
11 @Refinement.{u_1,
12   u_2} : {Stmt : Type} →
13   {W1 : Type u1} →
14   {W2 : Type u2} → Set.{u1} (Prod.{0, u1} Stmt W1) → Set.{u2} (Prod.{0, u2} Stmt W2) → Type (max u1 u2)
15 @Refinement.map_option_valid.{u_1,
16   u_2} : ∀ {Stmt : Type} {W1 : Type u1} {W2 : Type u2} {R : Set.{u1} (Prod.{0, u1} Stmt W1)}
17   {S : Set.{u2} (Prod.{0, u2} Stmt W2)} (f : Refinement.{u1, u2} R S) (x : Stmt) (w? : Option.{u1} W1),
18   (∀ (w : W1), Membership.mem.{u1, u1} w? w → Membership.mem.{u1, u1} R (Prod.mk.{0, u1} x w)) →
19   ∀ (w' : W2),
20     Membership.mem.{u2, u2} (Option.map.{u1, u2} (Refinement.map.{u1, u2} f x) w?) w' →
21     Membership.mem.{u2, u2} S (Prod.mk.{0, u2} x w')
22 @Extractor.Straightline.map.{u_1} : {ι : Type} →
23   {oSpec : OracleSpec.{0, u1} ι} →
24   {stmtIn WitIn WitOut : Type} →
25   {n : ℕ} →
26   {pSpec : ProtocolSpec n} →
27   (StmtIn → WitIn → WitOut) →
28   Extractor.Straightline.{u1} oSpec StmtIn WitIn WitOut pSpec →
29   Extractor.Straightline.{u1} oSpec StmtIn WitIn' WitOut pSpec
30 structure LeanerVM.Arithmetization.SatisfiedBy (prog : Program) (input : PublicInput)
31   (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop
32 number of parameters: 3
33 fields:
34   LeanerVM.Arithmetization.SatisfiedBy.public_input_eq : w.publicInput = PublicIO.ofInput input
35   LeanerVM.Arithmetization.SatisfiedBy.constraints : w.Constraints
36   LeanerVM.Arithmetization.SatisfiedBy.state_balanced : BalancedPair w StatePull.toRaw StatePush.toRaw
37   LeanerVM.Arithmetization.SatisfiedBy.mem_balanced : BalancedPair w MemPull.toRaw MemPush.toRaw
38   LeanerVM.Arithmetization.SatisfiedBy.bytecode_balanced : BalancedPair w BytecodePull.toRaw BytecodePush.toRaw
39   LeanerVM.Arithmetization.SatisfiedBy.counts_nonzero : CountsNonzero w

```

```

39   LeanerVM.Arithmetization.SatisfiedBy.caps : Caps w
40   LeanerVM.Arithmetization.SatisfiedBy.index_columns : IndexColumnsAreRowIndex w
41   LeanerVM.Arithmetization.SatisfiedBy.seed_rows : SeedRowsAreTheImage w
42   LeanerVM.Arithmetization.SatisfiedBy.bytecode_rows : BytecodeRowsAreTheProgram prog w
43   LeanerVM.Arithmetization.SatisfiedBy.blake2s_valid : Blake2sRowsValid w
44   LeanerVM.Arithmetization.SatisfiedBy.word0_eq : (imageOf w.data).snd.read (gpow 0) = some input.word0
45   LeanerVM.Arithmetization.SatisfiedBy.word1_eq : (imageOf w.data).snd.read (gpow 1) = some input.word1
46   constructor:
47     LeanerVM.Arithmetization.SatisfiedBy.mk {prog : Program} {input : PublicInput}
48       {w : EnsembleWitness (leanIsaEnsemble prog)} (public_input_eq : w.publicInput = PublicIO.ofInput input)
49       (constraints : w.Constraints) (state_balanced : BalancedPair w StatePull.toRaw StatePush.toRaw)
50       (mem_balanced : BalancedPair w MemPull.toRaw MemPush.toRaw)
51       (bytecode_balanced : BalancedPair w BytecodePull.toRaw BytecodePush.toRaw) (counts_nonzero : CountsNonzero
52         ↪ w)
53       (caps : Caps w) (index_columns : IndexColumnsAreRowIndex w) (seed_rows : SeedRowsAreTheImage w)
54       (bytecode_rows : BytecodeRowsAreTheProgram prog w) (blake2s_valid : Blake2sRowsValid w)
55       (word0_eq : (imageOf w.data).snd.read (gpow 0) = some input.word0)
56       (word1_eq : (imageOf w.data).snd.read (gpow 1) = some input.word1) : SatisfiedBy prog input w
57   structure LeanerVM.Arithmetization.Caps {prog : Program} (w : EnsembleWitness (leanIsaEnsemble prog)) : Prop
58   number of parameters: 2
59   fields:
60     LeanerVM.Arithmetization.Caps.minLogMem_le : minLogMem ≤ (imageOf w.data).fst
61     LeanerVM.Arithmetization.Caps.le_maxLogMem : (imageOf w.data).fst ≤ maxLogMem
62     LeanerVM.Arithmetization.Caps.heights : ∀ t ∈ w.tables, ∃ τ ≤ maxLogRows, t.table.length = 2 ^ τ
63     LeanerVM.Arithmetization.Caps.blake2s_height : 2 ^ minLogRowsBlake2s ≤ (blake2sRows w).length
64     LeanerVM.Arithmetization.Caps.well_shaped : WellShapedData w.data
65   constructor:
66     LeanerVM.Arithmetization.Caps.mk {prog : Program} {w : EnsembleWitness (leanIsaEnsemble prog)}
67       (minLogMem_le : minLogMem ≤ (imageOf w.data).fst) (le_maxLogMem : (imageOf w.data).fst ≤ maxLogMem)
68       (heights : ∀ t ∈ w.tables, ∃ τ ≤ maxLogRows, t.table.length = 2 ^ τ)
69       (blake2s_height : 2 ^ minLogRowsBlake2s ≤ (blake2sRows w).length) (well_shaped : WellShapedData w.data) :
70       ↪ Caps w
71   structure LeanerVM.Semantics.PublicInput : Type
72   number of parameters: 0
73   fields:
74     LeanerVM.Semantics.PublicInput.lanes : Fin 4 → K
75   constructor:
76     LeanerVM.Semantics.PublicInput.mk (lanes : Fin 4 → K) : PublicInput
77   def LeanerVM.Semantics.PublicInput.word0 : PublicInput → E :=
78   fun p => E.ofLimbs (p.lanes 0) (p.lanes 1) 0
79   def LeanerVM.Semantics.PublicInput.word1 : PublicInput → E :=
80   fun p => E.ofLimbs (p.lanes 2) (p.lanes 3) 0
81   structure LeanerVM.Arithmetization.PublicIO (F : Type) : Type
82   number of parameters: 1
83   fields:
84     LeanerVM.Arithmetization.PublicIO.lanes : Vector F 4
85   constructor:
86     LeanerVM.Arithmetization.PublicIO.mk {F : Type} (lanes : Vector F 4) : PublicIO F
87   def LeanerVM.Semantics.ValidExecution : (prog : Program) → PublicInput → Trace prog → Prop :=
88   fun prog input t => HasPublicBoundary input t ∧ run prog t.image t.steps Regs.initial = some (Regs.final prog)
89   def LeanerVM.Semantics.HasPublicBoundary : {prog : Program} → PublicInput → Trace prog → Prop :=
90   fun {prog} input t =>
91     minLogMem ≤ t.k ∧
92     t.k ≤ maxLogMem ∧ t.image.read (gpow 0) = some input.word0 ∧ t.image.read (gpow 1) = some input.word1
93   structure LeanerVM.Semantics.Program : Type
94   number of parameters: 0
95   fields:
96     LeanerVM.Semantics.Program.logSize : ℕ
97     LeanerVM.Semantics.Program.logSize_le : self.logSize ≤ maxLogBytecode
98     LeanerVM.Semantics.Program.code : Fin (2 ^ self.logSize) → Instr
99   constructor:
100     LeanerVM.Semantics.Program.mk (logSize : ℕ) (logSize_le : logSize ≤ maxLogBytecode)
101     (code : Fin (2 ^ logSize) → Instr) : Program
102   def LeanerVM.Protocol.M3Holds : (I : M3Instance) → I.Stmt → Column I.μ → Prop :=
103   fun I input q => I.ConstraintsVanish q ∧ I.Balanced q ∧ I.CountsNonzero q ∧ I.PublicLinesHold input q ∧ I.aux q
104   structure LeanerVM.Protocol.M3Instance : Type 1
105   number of parameters: 0
106   parents:
107     LeanerVM.Protocol.M3Instance.toShape : Shape
108   fields:
109     LeanerVM.Protocol.Shape.ntab : ℕ
110     LeanerVM.Protocol.Shape.τ : Fin self.ntab → ℕ
111     LeanerVM.Protocol.Shape.width : Fin self.ntab → ℕ

```

```

110   LeanerVM.Protocol.M3Instance.Smt : Type
111   LeanerVM.Protocol.M3Instance.constraints : (j : Fin self.ntab) → List (CPoly.CMvPolynomial (self.width j) K)
112   LeanerVM.Protocol.M3Instance.flushes : (j : Fin self.ntab) →
113     List (Side × Vector (CPoly.CMvPolynomial (self.width j) K) 16)
114   LeanerVM.Protocol.M3Instance.d : ℕ
115   LeanerVM.Protocol.M3Instance.constraints_degree : ∀ (j : Fin self.ntab),
116     ∀ C ∈ self.constraints j, C.totalDegree ≤ self.d
117   LeanerVM.Protocol.M3Instance.flushes_degree : ∀ (j : Fin self.ntab),
118     ∀ f ∈ self.flushes j, ∀ (k : Fin 16), (f.2.get k).totalDegree ≤ self.d
119   LeanerVM.Protocol.M3Instance.counts : (j : Fin self.ntab) → List (Fin (self.width j))
120   LeanerVM.Protocol.M3Instance.boundary : List (BoundaryBlock self.toShape)
121   LeanerVM.Protocol.M3Instance.μ : ℕ
122   LeanerVM.Protocol.M3Instance.layout : Layout self.μ self.ColumnId fun c => self.τ c.fst
123   LeanerVM.Protocol.M3Instance.publicLines : self.Smt → List (PublicLine self.toShape)
124   LeanerVM.Protocol.M3Instance.aux : Column self.μ → Prop
125   LeanerVM.Protocol.M3Instance.decAux : DecidablePred self.aux
126 constructor:
127   LeanerVM.Protocol.M3Instance.mk (toShape : Shape) (Smt : Type)
128     (constraints : (j : Fin toShape.ntab) → List (CPoly.CMvPolynomial (toShape.width j) K))
129     (flushes : (j : Fin toShape.ntab) → List (Side × Vector (CPoly.CMvPolynomial (toShape.width j) K) 16)) (d :
130       ↪ ℕ)
131     (constraints_degree : ∀ (j : Fin toShape.ntab), ∀ C ∈ constraints j, C.totalDegree ≤ d)
132     (flushes_degree : ∀ (j : Fin toShape.ntab), ∀ f ∈ flushes j, ∀ (k : Fin 16), (f.2.get k).totalDegree ≤ d)
133     (counts : (j : Fin toShape.ntab) → List (Fin (toShape.width j))) (boundary : List (BoundaryBlock toShape))
134     (μ : ℕ)
135     (layout : Layout μ toShape.ColumnId fun c => toShape.τ c.fst) (publicLines : Smt → List (PublicLine
136       ↪ toShape))
137     (aux : Column μ → Prop) (decAux : DecidablePred aux) : M3Instance
138 field notation resolution order:
139   LeanerVM.Protocol.M3Instance, LeanerVM.Protocol.Shape
140 structure LeanerVM.Protocol.PublicLine (S : Shape) : Type
141 number of parameters: 1
142 fields:
143   LeanerVM.Protocol.PublicLine.col : S.ColumnId
144   LeanerVM.Protocol.PublicLine.cell0 : K
145   LeanerVM.Protocol.PublicLine.cell1 : K
146   LeanerVM.Protocol.PublicLine.sent : Bool
147   LeanerVM.Protocol.PublicLine.pos : 0 < S.τ self.col.fst
148 constructor:
149   LeanerVM.Protocol.PublicLine.mk {S : Shape} (col : S.ColumnId) (cell0 cell1 : K) (sent : Bool)
150     (pos : 0 < S.τ col.fst) : PublicLine S
151 def Air.Flat.EnsembleWitness.Constraints : {F : Type} →
152   [inst : FiniteField F] →
153   {PublicIO : TypeMap} →
154   [inst_1 : ProvableType PublicIO] → {ens : Ensemble F PublicIO} → EnsembleWitness ens → Prop :=
155   fun {F} [FiniteField F] {PublicIO} [ProvableType PublicIO] {ens} witness =>
156     ∀ table ∈ witness.allTables, table.Constraints
157 def Air.Flat.Table.Constraints : {F : Type} → [inst : FiniteField F] → Air.Flat.Table F → Prop :=
158   fun {F} [FiniteField F] table =>
159     ∀ row ∈ table.table, Operations.ConstraintsHold (table.environment row) table.component.operations
160 def Operations.ConstraintsHold : {F : Type} → [inst : FiniteField F] → Environment F → Operations F → Prop :=
161   fun {F} [FiniteField F] env ops =>
162     (∀ e ∈ ops.constraints, (fun x => Expression.eval env x) e = 0) ∧ ∀ l ∈ ops.lookups, l.Contains env
163 def Air.Flat.EnsembleWitness.interactions : {F : Type} →
164   [inst : FiniteField F] →
165   {PublicIO : TypeMap} →
166   [inst_1 : ProvableType PublicIO] → {ens : Ensemble F PublicIO} → EnsembleWitness ens → List (Interaction
167     ↪ F) :=
168   fun {F} [FiniteField F] {PublicIO} [ProvableType PublicIO] {ens} witness =>
169     List.flatMap (fun table => table.interactions) witness.allTables
170 def ProverData : Type → Type :=
171   fun F => String → (n : ℕ) → Array (Vector F n)
172 'LeanerVM.Arithmetization.assumptions_of_blake2sRowsValid' depends on axioms: [propext, Classical.choice,
173   ↪ Quot.sound]
174 'LeanerVM.Arithmetization.memRowOf_bindings' depends on axioms: [propext, Classical.choice, Quot.sound]
175 'LeanerVM.Arithmetization.bytecodeRowOf_bindings' depends on axioms: [propext, Classical.choice, Quot.sound]
176 'LeanerVM.Arithmetization.bytecodeRowOf_decodes' depends on axioms: [propext, Classical.choice, Quot.sound]
177 'LeanerVM.Arithmetization.verifier_pull_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
178 'LeanerVM.Arithmetization.verifier_push_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
179 'LeanerVM.Arithmetization.rowAt_toElements' depends on axioms: [propext, Classical.choice, Quot.sound]
180 'LeanerVM.Arithmetization.decode_eq_some_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
181 'LeanerVM.Arithmetization.assignmentRepresents_image' depends on axioms: [propext, Classical.choice, Quot.sound]
182 'LeanerVM.Arithmetization.xorTable' depends on axioms: [propext, Classical.choice, Quot.sound]

```

```

178 'LeanerVM.Arithmetization.jumpTable' depends on axioms: [propext, Classical.choice, Quot.sound]
179 'LeanerVM.Arithmetization.blake2sTable' depends on axioms: [propext, Classical.choice, Quot.sound]
180 'LeanerVM.Arithmetization.memTable' depends on axioms: [propext, Classical.choice, Quot.sound]
181 'LeanerVM.Arithmetization.bytecodeTable' depends on axioms: [propext, Classical.choice, Quot.sound]
182 'LeanerVM.Arithmetization.leanIsaVerifier' depends on axioms: [propext, Classical.choice, Quot.sound]
183 'LeanerVM.Protocol.piop_rbrKnowledgeSoundness' depends on axioms: [propext, Classical.choice, Quot.sound]
184 'LeanerVM.Protocol.piop_perfectCompleteness' depends on axioms: [propext, Classical.choice, Quot.sound]
185 'LeanerVM.Protocol.Refinement.map_option_valid' depends on axioms: [propext, Quot.sound]
186 'LeanerVM.Protocol.bytecodeColumn_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
187 'LeanerVM.Protocol.idxColumn_eval' depends on axioms: [propext, Classical.choice, Quot.sound]
188 exit=0

```

B.3.2 KnownColumn

What it establishes. The public column of a boundary block is part of the relation, and an instance that treats it as committed while the adaptor rebuilds it falls into the trap of section 8.8.1.6. A model on the spine's toy instance: `toyOf p` has one boundary block whose coordinate 0 is `Coord.known p`, the column `p` standing for the program; `toyCommitted` reads the same coordinate off column 1 of the table, as if the program were committed. The honest stack satisfies `M3Holds (toyOf prog)` and fails `Balanced` for another program, so the relation is about the program in the instance. The stack `pushesTwo` satisfies `M3Holds toyCommitted` while `(toyOf prog).Balanced` fails on it: the witness an adaptor would rebuild from it, the block built from the program, does not balance. The numeral `2 : K` is nonzero at the old `CompPoly` pin, as the spine's own test `badConstraint` uses it; at the new pin it is `0`, hence the variant `KnownColumnNew`.

Revision. Run on 2026-09-29 against `leanerVM` main at `b435631` with the old pins (ArkLib `dca90385`, `CompPoly` `3468b38c`, Clean `93c9d1ef`). Exit 0.

Source.

```

1  /-
2  Probe (boundary-adaptor): the public column of a boundary block is part of the relation, and
3  what goes wrong when an instance treats it as committed while the adaptor rebuilds it.
4  A model on the spine's toy instance; scratch work for the blueprint review.
5  Run from the repository root:
6      flock .claude/reports/blueprint-review/logs/lean.lock \
7          lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumn.lean
8  -/
9  import LeanerVM.Protocol.Spine.Toy
10
11  open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy
12
13  namespace Probe
14
15  /-- The toy instance with the boundary block's public column a parameter: the model of
16  `leanIsaInstance prog s`, the column `p` standing for the program. -/
17  abbrev toyOf (p : Column 1) : M3Instance :=
18    { toy with
19      boundary := [{ κ := 1, side := .pull,
20        coords := Vector.ofFn fun k => if k.val = 0 then .known p else .const 0 } ] }
21
22  /-- The same instance built wrongly: the block's column is read from the stack (column 1 of the
23  table), as if the program were committed. -/
24  abbrev toyCommitted : M3Instance :=
25    { toy with
26      boundary := [{ κ := 1, side := .pull,
27        coords := Vector.ofFn fun k =>
28          if k.val = 0 then .committed (0, 1) rfl else .const 0 } ] }
29
30  /-- The public program of the model. -/
31  def prog : Column 1 := (#v[1, 1])
32
33  /-- Another program. -/
34  def prog' : Column 1 := (#v[1, 2])
35
36  -- 1. The relation is about the program in the instance: the honest stack of `prog` satisfies the

```

```

37 -- relation of `prog`, and not the relation of `prog`.
38 #guard M3Holds (toyOf prog) (1 : K) honest
39 #guard ¬ (toyOf prog).Balanced honest
40 #guard ¬ M3Holds (toyOf prog) (1 : K) honest
41
42 /-- A stack that pushes `[2, 2]` and whose column 1 is `[2, 2]`. -/
43 def pushesTwo : Column 3 := {#v[2, 2, 2, 2, 1, 0, 0, 0]}
44
45 -- 2. The trap. Against the instance that reads the block's column off the stack, the relation
46 -- holds of a stack that has nothing to do with the public program ...
47 #guard M3Holds toyCommitted (1 : K) pushesTwo
48 -- ... while the witness an adaptor would rebuild from it, the table's columns read from the
49 -- stack and the block's column built from the public program, does not balance.
50 #guard ¬ (toyOf prog).Balanced pushesTwo
51 -- The pulled tuples of the two instances on the same stack: the first from the stack, the second
52 -- from the program.
53 #guard (toyCommitted.tuples pushesTwo .pull).map (fun t ↦ t.get 0) = [2, 2]
54 #guard ((toyOf prog).tuples pushesTwo .pull).map (fun t ↦ t.get 0) = [1, 1]
55
56 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumn.lean

Command (from the repository root).

```
flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumn.lean
```

Recorded output (KnownColumn.out).

```
1 | exit=0
```

B.3.3 KnownColumnNew

What it establishes. The same as KnownColumn (appendix B.3.2) at the new pins, with K.ofBits 2 in place of the numeral 2; every guard passes.

Revision. Run on 2026-09-30, after the rebuild, against the checkout docs/protocol-blueprint-review at b022e4f, whose Lean sources are main at 144c5aa, with the new pins (CompPoly 572f9973, Clean 42fe4b26, ArkLib 7653a901). Exit 0.

Source.

```

1 /-
2   Probe (boundary-adaptor), NEW-PIN VARIANT (CompPoly 572f9973, K.ofBits in place of numerals): the public
3   ⇨ column of a boundary block is part of the relation, and
4   what goes wrong when an instance treats it as committed while the adaptor rebuilds it.
5   A model on the spine's toy instance; scratch work for the blueprint review.
6   Run from the repository root:
7     flock .claude/reports/blueprint-review/logs/lean.lock \
8       lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumnNew.lean
9   -/
10 import LeanerVM.Protocol.Spine.Toy
11
12 open LeanerVM.Parameters LeanerVM.Protocol LeanerVM.Protocol.Toy
13
14 namespace Probe
15
16 /-- The toy instance with the boundary block's public column a parameter: the model of
17 `leanIsaInstance prog s`, the column `p` standing for the program. -/
18 abbrev toyOf (p : Column 1) : M3Instance :=
19   { toy with
20     boundary := [{ κ := 1, side := .pull,

```



```

20         coords := Vector.ofFn fun k ↦ if k.val = 0 then .known p else .const 0 }} }
21
22 /-- The same instance built wrongly: the block's column is read from the stack (column 1 of the
23 table), as if the program were committed. -/
24 abbrev toyCommitted : M3Instance :=
25   { toy with
26     boundary := [{ κ := 1, side := .pull,
27       coords := Vector.ofFn fun k ↦
28         if k.val = 0 then .committed (0, 1) rfl else .const 0 }} ] }
29
30 /-- The public program of the model. -/
31 def prog : Column 1 := {#v[1, 1]}
32
33 /-- Another program. -/
34 def prog' : Column 1 := {#v[1, K.ofBits 2]}
35
36 -- 1. The relation is about the program in the instance: the honest stack of `prog` satisfies the
37 -- relation of `prog`, and not the relation of `prog'`.
38 #guard M3Holds (toyOf prog) (1 : K) honest
39 #guard ¬ (toyOf prog').Balanced honest
40 #guard ¬ M3Holds (toyOf prog') (1 : K) honest
41
42 /-- A stack that pushes `[2, 2]` and whose column 1 is `[2, 2]`. -/
43 def pushesTwo : Column 3 := {#v[K.ofBits 2, K.ofBits 2, K.ofBits 2, K.ofBits 2, 1, 0, 0, 0]}
44
45 -- 2. The trap. Against the instance that reads the block's column off the stack, the relation
46 -- holds of a stack that has nothing to do with the public program ...
47 #guard M3Holds toyCommitted (1 : K) pushesTwo
48 -- ... while the witness an adaptor would rebuild from it, the table's columns read from the
49 -- stack and the block's column built from the public program, does not balance.
50 #guard ¬ (toyOf prog).Balanced pushesTwo
51 -- The pulled tuples of the two instances on the same stack: the first from the stack, the second
52 -- from the program.
53 #guard (toyCommitted.tuples pushesTwo .pull).map (fun t ↦ t.get 0) = [K.ofBits 2, K.ofBits 2]
54 #guard ((toyOf prog).tuples pushesTwo .pull).map (fun t ↦ t.get 0) = [1, 1]
55
56 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumnNew.lean

Command (from the repository root).

```
flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/KnownColumnNew.lean
```

Recorded output (KnownColumnNew.out).

```
1 | exit=0
```

B.3.4 Transport

What it establishes. Whether the chain “knowledge of `M3Holds` $\rightarrow$ `SatisfiedBy` $\rightarrow$ `ValidExecution`” typechecks with the adaptor’s target witness in **Type 1**. The instance, the witness map and the two theorems not yet built are hypotheses of each **example**; nothing is an axiom. Example 1: the adaptor is a **Refinement**, which accepts a target witness in **Type 1**. `bad_of_bad`: the transport of the bad event of ArkLib’s game (“no witness in the slot is valid”), which follows from `map_valid` in one line, where `Refinement.map_option_valid` has the other polarity. Examples 3 and 4: the pointwise composition; when `prog` has no valid execution on `input`, whatever the extractor returns is outside `M3Rel`, so the event of ArkLib’s game is “the verifier accepts”, and the conclusion of the chain is a soundness statement for the language $\{(\text{prog}, \text{input}) \mid \exists t, \text{ValidExecution prog input } t\}$; with the hypothesis `SentinelSafe prog` of constraint soundness, the hypothesis reaches the conclusion. Used in section 8.8.3.6.

Revision. Run on 2026-09-29 against leanerVM main at b435631 with the old pins (ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef). Exit 0. Not re-run at the new pins.

Source.

```

1  /-
2  Probe (boundary-adaptor): does the chain
3    knowledge of `M3Holds` → `SatisfiedBy` → `ValidExecution`
4    typecheck, with the adaptor's target witness in `Type 1`?
5    The instance, the witness map and the two theorems not yet built are hypotheses of each
6    `example`; nothing here is an axiom. Scratch work for the blueprint review.
7    Run from the repository root:
8      flock .claude/reports/blueprint-review/logs/lean.lock \
9      lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/Transport.lean
10  -/
11  import LeanerVM
12
13  open LeanerVM LeanerVM.Parameters LeanerVM.Semantics LeanerVM.Arithmetization
14  open LeanerVM.Protocol
15  open Air.Flat (Ensemble EnsembleWitness Component)
16
17  namespace Probe
18
19  /-- A stand-in for `leanIsaInstance prog s`: any instance whose statement type is
20  `PublicInput`. (The toy's tables, no public line.) -/
21  abbrev stub : M3Instance :=
22    { Toy.toy with Stmt := PublicInput, publicLines := fun _ => [] }
23
24  /-- The relation of leanISA in ArkLib's shape, over the statement type of `M3Rel`. Its witness
25  type is in `Type 1`. -/
26  def SatRel (prog : Program) :
27    Set ((PublicInput × (∀ i, NoOracle i)) × EnsembleWitness (leanIsaEnsemble prog)) :=
28    {p | SatisfiedBy prog p.1.1 p.2}
29
30  /-- 1. The adaptor is a `Refinement`: the structure accepts a target witness in `Type 1`. -/
31  example (prog : Program) (witnessOf : Column stub.μ → EnsembleWitness (leanIsaEnsemble prog))
32    (satisfiedBy_witnessOf :
33      ∀ input q, M3Holds stub input q → SatisfiedBy prog input (witnessOf q)) :
34    Refinement (M3Rel stub) (SatRel prog) where
35      map := fun _ q => witnessOf q
36      map_valid := fun x q hx => satisfiedBy_witnessOf x.1 q hx
37
38  /-- 2. The polarity ArkLib's game needs. Its bad event is "no witness in the slot is valid"
39  (an empty slot is bad); `Refinement.map_option_valid` is stated for "every witness in the slot
40  is valid" (an empty slot is good). The transport of the bad event is this lemma, which follows
41  from `map_valid` directly. -/
42  theorem bad_of_bad {Stmt : Type} {W1 : Type} {W2 : Type 1} {R : Set (Stmt × W1)}
43    {S : Set (Stmt × W2)} (f : Refinement R S) (x : Stmt) (w? : Option W1)
44    (h : ∀ w' ∈ w?.map (f.map x), (x, w') ∉ S) : ∀ w ∈ w?, (x, w) ∉ R := by
45    intro w hw hR
46    exact h (f.map x w) (Option.mem_map_of_mem _ hw) (f.map_valid x w hR)
47
48  /-- 3. The composition of the chain, pointwise, in the form the probability bound consumes:
49  when `prog` has no valid execution on `input`, whatever the extractor returns is outside
50  `M3Rel`, so the event of ArkLib's knowledge-soundness game is the event "the verifier accepts".
51  The conclusion of the chain is therefore a soundness statement for the language
52  `{(prog, input) | ∃ t, ValidExecution prog input t}`. -/
53  example (prog : Program) (input : PublicInput)
54    (witnessOf : Column stub.μ → EnsembleWitness (leanIsaEnsemble prog))
55    (satisfiedBy_witnessOf :
56      ∀ q, M3Holds stub input q → SatisfiedBy prog input (witnessOf q))
57    (constraintSoundness :
58      ∀ w, SatisfiedBy prog input w → ∃ t, ValidExecution prog input t)
59    (hno : ¬ ∃ t, ValidExecution prog input t) (o : ∀ i, NoOracle i)
60    (q? : Option (Column stub.μ)) :
61    ∀ q ∈ q?, ((input, o), q) ∉ M3Rel stub := by
62    intro q _ hq
63    exact hno (constraintSoundness _ (satisfiedBy_witnessOf q hq))
64
65  /-- 4. The same with the hypothesis of `constraintSoundness` the leanISA roadmap states
66  (`WellFormedBytecode prog`, of which soundness uses the sentinel field, built as
67  `SentinelSafe prog`): the hypothesis reaches the conclusion of the chain. -/
68  example (prog : Program) (input : PublicInput)
69    (witnessOf : Column stub.μ → EnsembleWitness (leanIsaEnsemble prog))
70    (satisfiedBy_witnessOf :
71      ∀ q, M3Holds stub input q → SatisfiedBy prog input (witnessOf q))
72    (constraintSoundness :

```

```

73     SentinelSafe prog → ∀ w, SatisfiedBy prog input w → ∃ t, ValidExecution prog input t)
74     (hwf : SentinelSafe prog)
75     (q : Column stub.μ) (h : M3Holds stub input q) : ∃ t, ValidExecution prog input t :=
76     constraintSoundness hwf _ (satisfiedBy_witnessOf q h)
77
78 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/Transport.lean

Command (from the repository root).

```

flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/Transport.lean

```

Recorded output (Transport.out).

```

1 | exit=0

```

B.3.5 UniverseFail

What it establishes. Expected to fail. ArkLib's straight-line extractor cannot be stated with the witness of `SatisfiedBy`, which lives in `Type 1`: both an extractor type whose witness is an `EnsembleWitness` and the post-composition `Extractor.Straightline.map witnessOf E` are rejected with a universe mismatch. So `Extractor.Straightline.map` cannot serve the adaptor (section 8.8.3.6).

Revision. Run on 2026-09-29 against leanerVM main at b435631 with the old pins (ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef). Exit 1, with the two recorded errors, as expected. Not re-run at the new pins.

Source.

```

1  /-
2  Probe (boundary-adaptor), EXPECTED TO FAIL: ArkLib's knowledge-soundness game and its
3  straight-line extractor cannot be stated with the witness of `SatisfiedBy`, which lives in
4  `Type 1`. Scratch work for the blueprint review.
5  Run from the repository root:
6      flock .claude/reports/blueprint-review/logs/lean.lock \
7          lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/UniverseFail.lean
8  -/
9  import LeanerVM
10
11  open LeanerVM LeanerVM.Parameters LeanerVM.Semantics LeanerVM.Arithmetization
12  open LeanerVM.Protocol OracleComp OracleSpec ProtocolSpec
13  open Air.Flat (Ensemble EnsembleWitness Component)
14
15  namespace Probe
16
17  /-- Expected failure 1: a straight-line extractor whose output is an `EnsembleWitness`. -/
18  example (prog : Program) {n : ℕ} (pSpec : ProtocolSpec n) : Type :=
19      Extractor.Straightline []. PublicInput (EnsembleWitness (leanIsaEnsemble prog)) Unit pSpec
20
21  /-- Expected failure 2: post-composing an extractor of stacks with a map to `EnsembleWitness`. -/
22  example (prog : Program) {n : ℕ} (pSpec : ProtocolSpec n) (μ : ℕ)
23      (witnessOf : PublicInput → Column μ → EnsembleWitness (leanIsaEnsemble prog))
24      (E : Extractor.Straightline []. PublicInput (Column μ) Unit pSpec) :=
25      Extractor.Straightline.map witnessOf E
26
27  end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/UniverseFail.lean

Command (from the repository root).

```
flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/UniverseFail.lean
```

Recorded output (UniverseFail.out).

```
1 .claude/reports/blueprint-review/probes/boundary-adaptor/UniverseFail.lean:19:41: error: Application type
  ↳ mismatch: The argument
2   EnsembleWitness (leanIsaEnsemble prog)
3 has type
4   Type 1
5 of sort `Type 2` but is expected to have type
6   Type
7 of sort `Type 1` in the application
8   Extractor.Straightline []₀ PublicInput (EnsembleWitness (leanIsaEnsemble prog))
9 .claude/reports/blueprint-review/probes/boundary-adaptor/UniverseFail.lean:25:29: error: Application type
  ↳ mismatch: The argument
10  witnessOf
11 has type
12  PublicInput → Column μ → EnsembleWitness (leanIsaEnsemble prog)
13 of sort `Type 1` but is expected to have type
14  PublicInput → Column μ → ?m.27 prog pSpec μ witnessOf E
15 of sort `Type` in the application
16  Extractor.Straightline.map witnessOf
17 exit=1
```

B.3.6 DefInstance

What it establishes. Expected to fail in part. With a stand-in for `leanIsaInstance prog s` written as a `def` with parameters, the statement of `satisfiedBy_witnessOf`’s hypothesis elaborates (the `example` at lines 33–34: the elaborator unfolds the definition to see the statement type), but instance search does not see through it: the two `#guards` on `inst` fail with “Type mismatch ... expected to have type `Bool`”, no `Decidable` instance being found, while the same guards on the `abbrev instA` pass. Used in section 8.8.3.1 and finding 82.

Revision. Run on 2026-09-29 against `leanerVM` main at `b435631` with the old pins (ArkLib `dca90385`, CompPoly `3468b38c`, Clean `93c9d1ef`). Exit 1, with the recorded errors on the two guards of the `def`, as expected. Not re-run at the new pins.

Source.

```
1  /-
2   Probe (boundary-adaptor): does the adaptor's statement typecheck when the instance is a `def`
3   (not an `abbrev`), and is its relation still decided by evaluation?
4   The two `#guard`s on the `def` instance are EXPECTED TO FAIL (no `Decidable` instance is
5   found); the same guards on the `abbrev` instance pass.
6   Scratch work for the blueprint review.
7   Run from the repository root:
8       flock .claude/reports/blueprint-review/logs/lean.lock \
9       lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean
10  -/
11  import LeanerVM.Protocol.Spine.Toy
12  import LeanerVM.Semantics.Instruction
13
14  open LeanerVM.Parameters LeanerVM.Semantics LeanerVM.Protocol LeanerVM.Protocol.Toy
15
16  namespace Probe
17
18  /-- A stand-in for `leanIsaInstance prog s`, as a `def` with parameters: the statement type is
19  `PublicInput`, and the one public line reads lane 0 of the input. -/
20  def inst (_prog : Program) (_s : ℕ) : M3Instance :=
21  { toy with
22    Stmt := PublicInput
```

```

23     publicLines := fun input ↦ [({0, 2}, input.lanes 0, 0, true, by decide)] }
24
25 /-- The same, as an `abbrev`. -/
26 abbrev instA (_prog : Program) (_s : ℕ) : M3Instance :=
27   { toy with
28     Stmt := PublicInput
29     publicLines := fun input ↦ [({0, 2}, input.lanes 0, 0, true, by decide)] }
30
31 /-- The statement of `satisfiedBy_witnessOf`'s hypothesis elaborates with `input : PublicInput`
32 on the `def`: the elaborator unfolds it to see the statement type. -/
33 example (prog : Program) (s : ℕ) (input : PublicInput) (q : Column (inst prog s).μ) : Prop :=
34   M3Holds (inst prog s) input q
35
36 /-- A program and an input for the evaluation below. -/
37 def prog0 : Program := (0, by decide, fun _ ↦ .xor 1 1 1)
38 def input1 : PublicInput := (fun _ ↦ 1)
39 def input0 : PublicInput := (fun _ ↦ 0)
40
41 -- On the `abbrev`, the relation is decided by evaluation, as the toy's is.
42 #guard M3Holds (instA prog0 0) input1 honest
43 #guard ¬ M3Holds (instA prog0 0) input0 honest
44
45 -- On the `def`, instance search does not see through the definition (EXPECTED TO FAIL).
46 #guard M3Holds (inst prog0 0) input1 honest
47 #guard ¬ M3Holds (inst prog0 0) input0 honest
48
49 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean

Command (from the repository root).

```

flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean

```

Recorded output (DefInstance.out).

```

1 .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean:46:7: error: Type mismatch
2   M3Holds (inst prog0 0) input1 honest
3 has type
4   Prop
5 but is expected to have type
6   Bool
7 .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean:46:0: error: cannot evaluate code
8   ↳ because 'sorryAx' uses 'sorry' and/or contains errors
9 .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean:47:7: error: Type mismatch
10  ¬M3Holds (inst prog0 0) input0 honest
11 has type
12   Prop
13 but is expected to have type
14   Bool
15 .claude/reports/blueprint-review/probes/boundary-adaptor/DefInstance.lean:47:0: error: cannot evaluate code
16   ↳ because 'sorryAx' uses 'sorry' and/or contains errors
17 exit=1

```

B.3.7 PolyBridge

What it establishes. From a Clean expression to the polynomial an `M3Instance` holds. It prints `CompPoly`'s two conversions at the pin, then defines `exprToCMv`, a direct translation of a Clean expression over `K` to a computable `CMvPolynomial` in twelve lines, a variable past the width reading `0` as `Environment`. `fromArray` reads a missing cell; and checks by `#guard` that it agrees with Clean's `Expression.eval` on a row for the first JUMP residual `b + v_cond · w`, that the polynomial's `totalDegree` is 2, and that a variable past the width reads `0` on both sides (section 8.8.3.5). That `toCMvPolynomial` is noncomputable rests on its source (`Multivariate/MvPolyEquiv/Core.lean:41`). At the new `CompPoly` pin the row's numerals 3, 5, 7 read as 1, 1, 1, hence the variant `PolyBridgeNew`.

Revision. Run on 2026-09-29 against leanerVM main at b435631 with the old pins (ArkLib dca90385, CompPoly 3468b38c, Clean 93c9d1ef). Exit 0.

Source.

```

1  /-
2  Probe (boundary-adaptor): from a Clean expression to the polynomial an `M3Instance` holds.
3  The blueprint's Layer 2 produces Mathlib's `MvPolynomial`; the instance holds CompPoly's
4  `CMvPolynomial`. CompPoly's conversion from the first to the second is noncomputable at the
5  pin; a direct, computable translation exists. Scratch work for the blueprint review.
6  Run from the repository root:
7      flock .claude/reports/blueprint-review/logs/lean.lock \
8          lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridge.lean
9  -/
10 import LeanerVM
11 import CompPoly.Multivariate.MvPolyEquiv.Eval
12 import CompPoly.Multivariate.Operations
13
14 open LeanerVM.Parameters CompPoly CPoly
15
16 namespace Probe
17
18 -- The two directions of CompPoly's bridge at the pin.
19 #print CPoly.toCMvPolynomial
20 #print CPoly.fromCMvPolynomial
21
22 /-- A direct translation of a Clean expression over `K` to a computable polynomial in the first
23 `n` row variables; a variable past the width reads `0`, as Clean's `Environment.fromArray`
24 reads a missing cell. -/
25 def exprToCMv (n : ℕ) : Expression K → CMvPolynomial n K
26   | .var v => if h : v.index < n then CMvPolynomial.X (v.index, h) else 0
27   | .const c => CMvPolynomial.C c
28   | .add a b => exprToCMv n a + exprToCMv n b
29   | .mul a b => exprToCMv n a * exprToCMv n b
30
31 /-- The first `JUMP` residual, `b + v_cond · w`, on a row `(v_cond, w, b)`. -/
32 def residual : Expression K := .add (.var (2)) (.mul (.var (0)) (.var (1)))
33
34 /-- A row. -/
35 def row : Array K := #[3, 5, 7]
36
37 /-- The row as the assignment of the three variables. -/
38 def rowFn : Fin 3 → K := fun i => row[i.val]?.getD 0
39
40 /-- Clean's value of the residual on the row. -/
41 def cleanValue : K := residual.eval (Environment.fromArray row fun _ => #[])
42
43 /-- The polynomial's value on the row. -/
44 def polyValue : K := (exprToCMv 3 residual).eval rowFn
45
46 -- The translation runs, agrees with Clean's evaluation on the row, and has the degree the
47 -- expression shows.
48 #guard polyValue = cleanValue
49 #guard (exprToCMv 3 residual).totalDegree = 2
50
51 -- A variable past the width is read as zero on both sides.
52 def wide : Expression K := .add (.var (5)) (.var (0))
53 def wideClean : K := wide.eval (Environment.fromArray row fun _ => #[])
54 def widePoly : K := (exprToCMv 3 wide).eval rowFn
55 #guard widePoly = wideClean
56
57 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridge.lean

Command (from the repository root).

```

flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridge.lean

```

Recorded output (PolyBridge.out).

```

1 def CPoly.toCMvPolynomial.{u_1} : {n : ℕ} →
2   {R : Type u_1} → [inst : CommSemiring R] → MvPolynomial (Fin n) R → CMvPolynomial n R :=
3   fun {n} {R} [CommSemiring R] p =>
4     match p with
5     | AddMonoidAlgebra.ofCoeff { support := s, toFun := f, mem_support_toFun := mem_support_toFun } =>
6       let unlawful := Unlawful.ofList (List.map (fun m => (CMvMonomial.ofFinsupp m, f m)) s.toList);
7       (unlawful, ...)
8 def CPoly.fromCMvPolynomial.{u_1} : {n : ℕ} →
9   {R : Type u_1} → [inst : CommSemiring R] → CMvPolynomial n R → MvPolynomial (Fin n) R :=
10  fun {n} {R} [CommSemiring R] p =>
11    let support := List.map CMvMonomial.toFinsupp (Lawful.mononials p);
12    let toFun := fun f => p[CMvMonomial.ofFinsupp f]?.getD 0;
13    have mem_support_fun := ...;
14    AddMonoidAlgebra.ofCoeff { support := support.toFinset, toFun := toFun, mem_support_toFun := ... }
15  exit=0

```

B.3.8 PolyBridgeNew

What it establishes. The same as PolyBridge (appendix B.3.7) at the new pins, with `K.ofBits n` for the row's numerals; every guard passes. `toCMvPolynomial` is still a noncomputable def at the new CompPoly pin 572f9973 (MvPolyEquiv/Core.lean:41, read with `git show`).

Revision. Run on 2026-09-30, after the rebuild, against the checkout `docs/protocol-blueprint-review` at `b022e4f`, whose Lean sources are main at `144c5aa`, with the new pins (CompPoly 572f9973, Clean 42fe4b26, ArkLib 7653a901). Exit 0.

Source.

```

1  /-
2  Probe (boundary-adaptor), NEW-PIN VARIANT (CompPoly 572f9973, K.ofBits in place of numerals): from a Clean
3  ↪ expression to the polynomial an `M3Instance` holds.
4  The blueprint's Layer 2 produces Mathlib's `MvPolynomial`; the instance holds CompPoly's
5  `CMvPolynomial`. CompPoly's conversion from the first to the second is noncomputable at the
6  pin; a direct, computable translation exists. Scratch work for the blueprint review.
7  Run from the repository root:
8  flock .claude/reports/blueprint-review/logs/lean.lock \
9  lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridgeNew.lean
10 -/
11 import LeanerVM
12 import CompPoly.Multivariate.MvPolyEquiv.Eval
13 import CompPoly.Multivariate.Operations
14
15 open LeanerVM.Parameters CompPoly CPoly
16
17 namespace Probe
18
19 -- The two directions of CompPoly's bridge at the pin.
20 #print CPoly.toCMvPolynomial
21 #print CPoly.fromCMvPolynomial
22
23 /- A direct translation of a Clean expression over `K` to a computable polynomial in the first
24 `n` row variables; a variable past the width reads `0`, as Clean's `Environment.fromArray`
25 reads a missing cell. -/
26 def exprToCMv (n : ℕ) : Expression K → CMvPolynomial n K
27 | .var v => if h : v.index < n then CMvPolynomial.X (v.index, h) else 0
28 | .const c => CMvPolynomial.C c
29 | .add a b => exprToCMv n a + exprToCMv n b
30 | .mul a b => exprToCMv n a * exprToCMv n b
31
32 /- The first `JUMP` residual, `b + v_cond · w`, on a row `(v_cond, w, b)`. -/
33 def residual : Expression K := .add (.var {2}) (.mul (.var {0}) (.var {1}))
34
35 /- A row. -/
36 def row : Array K := #[K.ofBits 3, K.ofBits 5, K.ofBits 7]
37
38 /- The row as the assignment of the three variables. -/

```

```

38 def rowFn : Fin 3 → K := fun i ↦ row[i.val]?.getD 0
39
40 /-- Clean's value of the residual on the row. -/
41 def cleanValue : K := residual.eval (Environment.fromArray row fun _ _ ↦ #[])
42
43 /-- The polynomial's value on the row. -/
44 def polyValue : K := (exprToCMv 3 residual).eval rowFn
45
46 -- The translation runs, agrees with Clean's evaluation on the row, and has the degree the
47 -- expression shows.
48 #guard polyValue = cleanValue
49 #guard (exprToCMv 3 residual).totalDegree = 2
50
51 -- A variable past the width is read as zero on both sides.
52 def wide : Expression K := .add (.var {5}) (.var {0})
53 def wideClean : K := wide.eval (Environment.fromArray row fun _ _ ↦ #[])
54 def widePoly : K := (exprToCMv 3 wide).eval rowFn
55 #guard widePoly = wideClean
56
57 end Probe

```

.claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridgeNew.lean

Command (from the repository root).

```
flock .claude/reports/blueprint-review/logs/lean.lock \
lake env lean .claude/reports/blueprint-review/probes/boundary-adaptor/PolyBridgeNew.lean
```

Recorded output (PolyBridgeNew.out).

```

1 def CPoly.toCMvPolynomial.{u_1} : {n : ℕ} →
2   {R : Type u_1} → [inst : CommSemiring R] → MvPolynomial (Fin n) R → CMvPolynomial n R :=
3   fun {n} {R} [CommSemiring R] p =>
4     match p with
5     | AddMonoidAlgebra.ofCoeff { support := s, toFun := f, mem_support_toFun := mem_support_toFun } =>
6       let unlawful := Unlawful.ofList (List.map (fun m => (CMvMonomial.ofFinsupp m, f m)) s.toList);
7       (unlawful, ...)
8   def CPoly.fromCMvPolynomial.{u_1} : {n : ℕ} →
9     {R : Type u_1} → [inst : CommSemiring R] → CMvPolynomial n R → MvPolynomial (Fin n) R :=
10     fun {n} {R} [CommSemiring R] p =>
11       let support := List.map CMvMonomial.toFinsupp (Lawful.mononials p);
12       let toFun := fun f => p[CMvMonomial.ofFinsupp f]?.getD 0;
13       have mem_support_fun := ...;
14       AddMonoidAlgebra.ofCoeff { support := support.toFinset, toFun := toFun, mem_support_toFun := ... }
15   exit=0

```

B.4 Re-runs at the new pins

The probes below were re-run on 2026-09-30 at the upgraded checkout (144c5aa: Lean 4.34.1, ArkLib 7653a901, CompPoly 572f9973, VCVio a4232d08), each under the shared lock, after adapting what the upgrade changed: numerals in $\mathbb{K}$ rewritten as $K.ofBits$, the probability notation $\Pr\{\text{let } x \leftarrow c\}[P\ x]$, and the imports. The full adaptation diffs, commands, outputs and timings are in the dossier dossiers/probes-rerun.md on the review branch; no run took more than a minute.

Probe	Adaptation	Exit	Outcome at the new pins
public input, baseline (control)	regenerated from the current module	0	agrees

Probe	Adaptation	Exit	Outcome at the new pins
public input, mutation 7a (wrong check, honest prover fixed)	regenerated; refutation lemmas on the new notation	1	completeness refuted as predicted (<code>not_complete</code> proved); the unchanged <code>rbr</code> fails at its state function, as in every other mutation
public input, mutation 8 (extra check)	same	1	same pattern
public input, the words lemma (<code>ProbeWordsLemma</code>)	none	0	agrees
public input, mutations 1, 2, <code>ProbeAnyCheck</code> (controls)	regenerated	1 / 1 / 0	agree (the same expected failures)
public input, mutations 4b and 6	—	—	never written; on paper only
spine <code>P2Relation</code> , <code>P3aSeams</code>	<code>2 : K → K.ofBits 2</code>	0 / 0	agree
spine <code>P5PassThrough</code>	<code>K.ofBits 2</code> , new notation	0	agrees (<code>no_security</code> on the three axioms)
spine <code>P4Junk</code> (control, unchanged)	none	0	agrees
Layer 1 <code>ValuesProbe</code>	<code>K.ofBits</code> everywhere	0	agrees: every check against the Python verifier's numbers holds
Layer 1 <code>StridedProbe</code> , <code>OffsetsProbe</code> , <code>DuplicatesProbe</code>	<code>K.ofBits</code> / none	0	agree
<code>SeamBusShape</code> , <code>SeamBusMember</code> , <code>NoCheckFlock</code>	none	0	agree
<code>FieldFidelity</code>	every word $\rightarrow$ <code>K.ofBits</code>	1 (one <code>\#synth</code>)	the eleven Rust reference products still agree; the instance lines differ
<code>SamplerDiamond2</code>	none	1	the sampler diamond on $\mathbb{K}$ no longer exists (no <code>FinEnum K</code>); nothing to compare
Layer0	none	0	agrees; instance names print differently
<code>NonVacuity</code> , <code>PlainReading</code>	new notation	0 / 0	agree
<code>Extractors</code>	none	0	agrees
<code>AxiomsLeanerVM</code>	none	0	byte-identical: every master theorem, phase, composition and extractor on <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> ; <code>Component.sendExtractor</code> on <code>propext</code> , <code>Quot.sound</code> ; no <code>sorryAx</code>

Three library facts recorded at the old pin changed with the new one: `LawfulBEq E` now exists; $\mathbb{K}$'s field operations no longer leak onto `BitVec 64`; and the sampler diamond on $\mathbb{K}$ is gone. None affects a finding.

Appendix C

Index of the letter codes

Every code met in the proof system’s documents, with its family, its meaning in one sentence, where it is defined, and the name the report proposes in its place, or “kept”.

The codes are grouped by family, and sorted by code within a family. “Replaced by” gives the name the proposal of section 13.5.2 puts in its place, or says that the code is kept (with the name beside it).

The column “Uses” counts occurrences of the code string, whatever it means there; section 13.4.2 splits the codes that mean several things. For the spine’s code the count is by hand (the letter **S** alone is also a Lean variable and a class letter): 22 uses, BP 10, ST 4, TB 3, HC 3, OC 2.

The codes of the three reviews: each defined in its handoff’s disposition table; “uses” are in the handoff and, in parentheses, in the status.

“BP”, “ST”, “TB”, “HC”, “OC”, “RS”, “RL”, “RP” abbreviate the blueprint, the status, the tracker body, the hole comment, the other comments, and the handoffs of the reviews of the spine, of Layer 1 and of the public-input phase.

Table C.1: Index of the codes.

Code	Meaning	Defined at	Uses	Replaced by
Units of work (the documents’ “holes”)				
S	the spine: instance, relation, seams, phase interfaces, composition, toy instance	blueprint:597; hole comment:5	22 (see above)	the spine; short form spine
0	the ArkLib dependency and the field instances (Layer 0); a layer number used as a hole code	tracker body:20; status:89; not in the blueprint’s table	2	field instances (Layer 0); short form field-instances
C1	knowledge-soundness composition, ported from ArkLib pull request 615	blueprint:611; hole comment:129	41: BP 5, ST 9, TB 5, HC 4, OC 3, RS 6, RL 4, RP 5	knowledge-soundness composition (the port of ArkLib pull request 615); short form knowledge-append
G1	generic sumcheck, definition and completeness (Layer 4)	blueprint:598; no section in the hole comment	26: BP 7, ST 8, TB 7, HC 3, OC 1	sumcheck: definition and completeness (Layer 4); short form sumcheck-def
G2	generic sumcheck, round-by-round knowledge soundness (Layer 4)	blueprint:599	12: BP 3, ST 4, TB 4, HC 1	sumcheck: knowledge soundness (Layer 4); short form sumcheck-security
G3	batching of claims by powers of one challenge (Layer 4)	blueprint:600	15: BP 2, ST 7, TB 5, HC 1	batching by powers (Layer 4); short form batching
G4	fingerprint, product-determines-multiset lemma, collision bound (Layer 5)	blueprint:601	12: BP 2, ST 5, TB 4, HC 1	fingerprint and collision bound (Layer 5); short form fingerprint
G5	grand-product GKR, definition and completeness (Layer 5)	blueprint:602; hole comment:28	14: BP 2, ST 4, TB 4, HC 4	grand-product GKR: definition and completeness (Layer 5); short form gkr-def
G6	grand-product GKR, knowledge soundness (Layer 5)	blueprint:602; hole comment:28	14: BP 5, ST 2, TB 2, HC 4, OC 1	grand-product GKR: knowledge soundness (Layer 5); short form gkr-security

Code	Meaning	Defined at	Uses	Replaced by
I1	Clean expressions as polynomials (Layer 2)	blueprint:604; no section in the hole comment	26: BP 3, ST 5, TB 6, HC 1, OC 2, RS 9	Clean expressions as polynomials (Layer 2); short form <code>clean-polynomials</code>
I2	the adaptor (Layer 3): the leanISA instance, <code>stack0f</code> , <code>witness0f</code> and their theorems	blueprint:605; hole comment:44	49: BP 5, ST 8, TB 11, HC 6, OC 3, RS 15, RL 1	the adaptor (Layer 3); short form <code>adaptor</code>
K1	WHIR opening over binary Reed-Solomon codes (Layer 11)	blueprint:612; hole comment:144	25: BP 4, ST 8, TB 9, HC 3, OC 1	WHIR opening (Layer 11); short form <code>whir</code>
K2	Merkle trees, BLAKE2s byte hasher, WHIR parameters (Layer 11)	blueprint:613; hole comment:158	17: BP 4, ST 4, TB 5, HC 3, OC 1	Merkle trees, byte hasher, WHIR parameters (Layer 11); short form <code>merkle-parameters</code>
K3	transcript, proof object, compiled verifier <code>verify</code> and its refinement theorem (Layer 12)	blueprint:614; hole comment:172	29: BP 4, ST 7, TB 7, HC 6, RS 5	transcript, proof object, compiled verifier (Layer 12); short form <code>compiled-verifier</code>
K4	the two base theorems of target T4 (Layer 13)	blueprint:615; hole comment:186	10: BP 2, ST 1, TB 2, HC 3, RS 2	the base theorems of T4 (Layer 13); short form <code>base-theorems</code>
L1	tables and stacking (Layer 1): hypercube tables, stacking, the fixed columns	blueprint:603; no section in the hole comment	21: BP 6, ST 6, TB 7, RL 2	tables and stacking (Layer 1); short form <code>tables-stacking</code>
P1	bus phase, definition and completeness (Layer 6)	blueprint:606; hole comment:61	21: BP 4, ST 5, TB 4, HC 3, OC 1, RS 2, RP 2	bus phase: definition and completeness (Layer 6); short form <code>bus-def</code>
P2	bus phase, knowledge soundness (Layer 6)	blueprint:606; hole comment:61	13: BP 4, ST 2, TB 3, HC 4	bus phase: knowledge soundness (Layer 6); short form <code>bus-security</code>
P3	table sumcheck phase, definition and completeness (Layer 7)	blueprint:607; hole comment:75	19: BP 4, ST 6, TB 3, HC 3, OC 1, RP 2	table sumcheck phase: definition and completeness (Layer 7); short form <code>table-def</code>
P4	table sumcheck phase, knowledge soundness (Layer 7)	blueprint:607; hole comment:75	12: BP 4, ST 2, TB 2, HC 4	table sumcheck phase: knowledge soundness (Layer 7); short form <code>table-security</code>
P5	public-input phase, both halves (Layer 8)	blueprint:608; hole comment:89	36: BP 2, ST 15, TB 3, HC 3, OC 3, RS 7, RP 3	public-input phase (Layer 8); short form <code>public-input</code>
P6	Flock phase as an interface supplied by issue 3 (Layer 9)	blueprint:609; hole comment:102	19: BP 2, ST 4, TB 4, HC 3, OC 2, RS 4	Flock phase, the interface to the Flock roadmap (Layer 9); short form <code>flock-interface</code>
P7	opening phase and claim pool, definition and completeness (Layer 10)	blueprint:610; hole comment:115	20: BP 2, ST 6, TB 5, HC 4, RS 1, RP 2	opening phase: definition and completeness (Layer 10); short form <code>opening-def</code>
P8	opening phase, knowledge soundness (Layer 10)	blueprint:610; hole comment:115	16: BP 5, ST 2, TB 3, HC 4, RS 2	opening phase: knowledge soundness (Layer 10); short form <code>opening-security</code>
Layers				
Layer 0 to Layer 13	the blueprint's section number	blueprint :625-1259	"layer" 172 times in the blueprint	kept, with the name: "the bus phase (Layer 6)"
Upstream ledger entries				
A1	ArkLib admits the sumcheck's single-round knowledge soundness	blueprint:259; status:173; tracker body:63	32: BP 3, ST 9, TB 2, OC 1, RS 9, RL 3, RP 5	admitted upstream: sumcheck round knowledge soundness
A2	ArkLib admits the knowledge-soundness composition of appended reductions	blueprint:260; status:174; tracker body:64	39: BP 7, ST 9, TB 2, HC 3, OC 2, RS 8, RL 3, RP 5	admitted upstream: composition of knowledge soundness
A3	ArkLib admits "round-by-round implies plain" knowledge soundness	blueprint:261; status:175; tracker body:65	24: BP 1, ST 6, TB 1, RS 11, RL 2, RP 3	admitted upstream: round-by-round implies plain
A4	ArkLib admits every context-lifting security theorem (not consumed)	blueprint:262 only; absent from the status's and the tracker's tables	16: BP 1, ST 4, RS 6, RL 3, RP 2	admitted upstream: context lifting (not consumed)

Code	Meaning	Defined at	Uses	Replaced by
A5	ArkLib admits or lacks Fiat-Shamir and BCS security	blueprint:263; status:176; tracker body:66	17: BP 4, ST 2, TB 1, HC 1, RS 7, RL 2	admitted or absent upstream: Fiat-Shamir and BCS security
A6	ArkLib has no grand product, GKR, batching or stacking	blueprint:264; status:177; tracker body:67	16: BP 2, ST 3, TB 1, HC 2, RS 5, RL 3	absent upstream: grand product, GKR, batching, stacking
A7	ArkLib has no WHIR, Merkle tree or BLAKE2s	blueprint:265; status:178; tracker body:68	6: BP 2, ST 2, TB 1, HC 1	absent upstream: WHIR, Merkle trees, BLAKE2s
A8	ArkLib admits mutual correlated agreement up to the Johnson bound	blueprint:266; status:179; tracker body:69	7: BP 4, ST 1, TB 1, HC 1	admitted upstream: correlated agreement up to Johnson
A9	ArkLib admits the ring-switching packing lemmas	blueprint:267; status:180; tracker body:70	5: BP 1, ST 2, TB 1, HC 1	admitted upstream: ring-switching packing
C1	Clean has no polynomial reading of expressions and no degree bound	status:181; tracker body:71; not in the blueprint's table	2 of the 41 above	absent in Clean: expressions as polynomials
C2	request to leanISA: power-of-two heights in Caps, bus data per channel (done)	status:182; tracker body:72; not in the blueprint's table	9: ST 2, TB 1, RS 2, RL 2, RP 2	asked of leanISA: power-of-two heights, bus data per channel (done)
C3	Clean lacks a balance counted in the naturals with direction tags	status:183; tracker body:73; not in the blueprint's table	5: ST 1, TB 1, RL 1, RP 2	absent in Clean: balance counted in the naturals
C4	Clean lacks fixed columns and sound prover data	status:184; tracker body:74; not in the blueprint's table	5: ST 1, TB 1, RL 1, RP 2	absent in Clean: fixed columns and sound prover data
Findings about ArkLib beyond the ledger				
A10	ArkLib relations are sets of pairs; the documented refactor has not happened	status:332	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A11	ArkLib's averaged round-by-round notion is weaker than the worst-case one the layers prove	status:334	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A12	ArkLib's extractability for commitments is False-valued; not cited	status:336	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A13	two ArkLib statements contain sorry in their types; never cited	status:337	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A14	ArkLib's typed-interaction replacement has no security definitions yet	status:338	2: ST 2	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A15	ArkLib's toy problem is sorry-free; its code-generation probes are the pattern for verify	status:340	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A16	ArkLib pins CompPoly at a tag; leanerVM's pin wins the resolution	status:341	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A17	ArkLib's position-query oracle instance on Vector is global, so Column must be a structure	status:342	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A18	ArkLib main had moved 246 commits past the pin on 2026-09-24	status:345	4: ST 3, TB 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
A19	ArkLib pull request 615's composition file compiles unchanged against the pin	status:346	3: ST 3	its description, as a name in the register of docs/dependencies.md, the code kept in one column there

Code	Meaning	Defined at	Uses	Replaced by
Findings internal to the specification				
S6	specification section 8.4 (Fiat-Shamir) is T0D0 (inherited from leanISA)	status:285	1: ST 1	Fiat-Shamir section is T0D0 (in docs/leanvm-target.md)
S9	the proof of specification Lemma 5.2 is T0D0	status:286	2: BP 1, ST 1	proof of the product lemma is T0D0 (in docs/leanvm-target.md)
S10	specification section 5.3 does not state the degree of a radix-4 round polynomial	status:287	1: ST 1	degree of the radix-4 round polynomial not stated (in docs/leanvm-target.md)
S11	specification section 8.5 does not say the push and pull roots are one scalar	status:289	1: ST 1	one bus root not stated (in docs/leanvm-target.md)
S12	Annex B omits the 17 grinding bits per level that the Rust applies	status:291	1: ST 1	grinding per level not in Annex B (in docs/leanvm-target.md)
S13	retired: a bound of 5 instead of 4, read off a PDF that is not the pinned text	status:293	8: ST 2, TB 1, RS 5	retired (in docs/leanvm-target.md)
S14	the PDF leanVM-b-2.pdf is not the pinned text and must not be cited	status:297	4: ST 2, TB 1, RS 1	the PDF is not the pinned text (in docs/leanvm-target.md)
S15	the specification gives no rule for stacking blocks of equal size	status:303	3: ST 2, RL 1	no rule for equal-size blocks (in docs/leanvm-target.md)
S16	the specification's equations are numbered (1) to (4); "equation (5.4)" does not exist	status:303	3: ST 2, RL 1	equations are (1) to (4) (in docs/leanvm-target.md)
Findings, Rust and Python against the specification				
F1	no domain-separation labels: four numeric tags in lane 3	status:305	9: BP 4, ST 3, TB 1, HC 1	numeric tags, no labels (in docs/leanvm-target.md)
F2	Flock's fixed coordinate is a hard-coded constant without provenance	status:307	2: BP 1, ST 1	Flock's fixed coordinate has no provenance (in docs/leanvm-target.md)
F3	one root for the push and pull products	status:309	8: BP 4, ST 2, TB 1, HC 1	one root for push and pull (in docs/leanvm-target.md)
F4	the table sumcheck's target is derived by the verifier, never sent	status:310	4: BP 1, ST 2, HC 1	sumcheck target derived, not sent (in docs/leanvm-target.md)
F5	the three bus forms share the last three powers of the batching challenge	status:311	6: BP 3, ST 1, TB 1, HC 1	bus forms share the last three powers (in docs/leanvm-target.md)
F6	the table round polynomial is a cubic; three scalars travel	status:312	9: BP 4, ST 3, TB 2	cubic round polynomial, three scalars (in docs/leanvm-target.md)
F7	one coefficient of every round polynomial is never transmitted	status:313	1: ST 1	one coefficient never sent (in docs/leanvm-target.md)
F8	ring-switched claims take the low powers in the opening batch	status:315	5: BP 2, ST 1, TB 1, HC 1	ring-switched claims take the low powers (in docs/leanvm-target.md)
F9	the Python verifier omits four caps that the Rust verifier checks	status:316	3: ST 2, TB 1	Python verifier omits four caps (in docs/leanvm-target.md)
F10	the Rust verifier's rejection set; structure checks are <code>assert!</code> s	status:319	2: BP 1, ST 1	rejection set of the Rust verifier (in docs/leanvm-target.md)
F11	the count tree holds the tables' count columns only	status:321	2: ST 1, TB 1	count tree holds the tables' counts only (in docs/leanvm-target.md)
F12	fill blocks make announced heights exact	status:322	1: ST 1	fill blocks make heights exact (in docs/leanvm-target.md)
F13	grinding binds the nonce even when the check fails	status:324	1: ST 1	grinding binds the nonce on failure (in docs/leanvm-target.md)
F14	the seed hashes one constant naming the circuit, not the matrices	status:325	2: ST 1, HC 1	seed hashes a constant naming the circuit (in docs/leanvm-target.md)

Code	Meaning	Defined at	Uses	Replaced by
F15	the bound on the stack height is checked separately from the caps	status:327	1: ST 1	stack height bound checked separately (in docs/leanvm-target.md)
F16	128 security bits round by round; the bus needs no grinding	status:328	1: ST 1	security level and no grinding for the bus (in docs/leanvm-target.md)
F17	blocks of equal size are stacked in the order of the column index	status:330	7: ST 4, RL 1, RP 2	equal-size blocks in column-index order (in docs/leanvm-target.md)
F18	the pinned verifiers check one equation on the public words, the specification one per limb	status:330	7: ST 5, RP 2	one equation on the words, not one per limb (in docs/leanvm-target.md)
Findings about Clean				
C5	Clean has no degree bound on expressions (inherited from leanISA)	status:355 (by reference)	3: BP 1, ST 1, RL 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
C6	Clean has no prover-chosen heights (inherited from leanISA)	status:355 (by reference)	3: BP 1, ST 1, RL 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
C10	Clean's EnsembleWitness has no generator	status:355	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
C11	Clean's Ensemble.Statement assumes the non-overflow side condition	status:356	1: ST 1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
Findings about CompPoly				
P1	CompPoly's extension arithmetic is not kernel-reducible from a module (inherited from leanISA)	status:360 (by reference)	1 of the 21 of P1	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
P3	CompPoly's Fintype BF64 instance is evaluated at the start of an executable (inherited)	status:360 (by reference)	4 of the 19 of P3	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
P4	CompPoly has no sum over the cube and no pointwise product of tables	status:360	1 of the 12 of P4	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
P5	CompPoly's additive NTT is instantiated only at GF(2^8)	status:361	1 of the 35 of P5	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
P6	CompPoly has no Frobenius on extensions	status:362	1 of the 19 of P6	its description, as a name in the register of docs/dependencies.md, the code kept in one column there
Findings about the Lean environment				
E6	a Lake race on ArkLib's lint plugin at first build	status:365	2: ST 2	its description, as a name in the register of tests/README.md, the code kept in one column there
E7	no LawfulBEq E at the CompPoly pin, so no polynomial with coefficients in E	status:368	3: ST 1, TB 1, RS 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E8	no OracleInterface E at the ArkLib pin	status:375	3: ST 1, TB 1, RS 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E9	decide cannot unfold CompPoly's X and * inside a module	status:380	4: ST 1, TB 1, RS 2	its description, as a name in the register of tests/README.md, the code kept in one column there

Code	Meaning	Defined at	Uses	Replaced by
E10	a phase with a nonzero error is a noncomputable bundle	status:383	2: ST 1, RP 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E11	a binder over <code>Finset.univ</code> : <code>Finset</code> E makes a linter enumerate the field	status:388	1: ST 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E12	<code>simp</code> does not rewrite inside instance arguments of <code>Component.Def.red</code>	status:391	3: ST 1, RP 2	its description, as a name in the register of tests/README.md, the code kept in one column there
E13	a <code>Decidable</code> instance written by <code>unfold</code> never evaluates under <code>#guard</code>	status:396	3: ST 1, RP 2	its description, as a name in the register of tests/README.md, the code kept in one column there
E14	Mathlib's overlapping-instances linter rejects <code>[Zero R]</code> under <code>[CommRing R]</code>	status:400	4: ST 3, RP 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E15	<code>#guard</code> finds no <code>Decidable</code> instance for vectors of computed length	status:405	2: ST 1, RP 1	its description, as a name in the register of tests/README.md, the code kept in one column there
E16	a generic module's destination is read off its objects and imports (a rule, not a fact about Lean)	status:407	6: ST 5, RP 1	deleted: already the blueprint's convention <i>Generic code</i> (protocol-blueprint.md:329)
E17	being stated over any ring does not make a declaration generic (a rule, not a fact about Lean)	status:415	6: ST 3, RL 1, RP 2	deleted: already the blueprint's convention <i>Generic code</i> (protocol-blueprint.md:329)
E18	a <code>#guard</code> with a numeral of E next to an operation finds no <code>Decidable</code> instance	status:429	2: ST 1, RP 1	its description, as a name in the register of tests/README.md, the code kept in one column there
The one leanISA finding cited				
R21	leanISA finding: the repository holds two BLAKE2s compressions	leanisa-status.md; cited at blueprint:216, hole comment:162	2: BP 1, HC 1	not covered by the proposal
Findings of the review of the spine				
A1	the bus seam admits linear claims of any degree	protocol-spine.md: 32-47, the disposition table	9 (2)	its description, used inside the handoff only
A2	the Flock predicate's strength is undetermined	protocol-spine.md: 32-47, the disposition table	7 (2)	its description, used inside the handoff only
A3	the master knowledge theorem does not name its extractor	protocol-spine.md: 32-47, the disposition table	11 (1)	its description, used inside the handoff only
A4	a list of public cells cannot be served by the public-input phase	protocol-spine.md: 32-47, the disposition table	6 (2)	its description, used inside the handoff only
A5	the adaptor's soundness theorem needs the caps as a hypothesis	protocol-spine.md: 32-47, the disposition table	7 (1)	its description, used inside the handoff only
A6	no inhabitant of a phase's security; the pass-through has no knowledge half	protocol-spine.md: 32-47, the disposition table	5 (0)	its description, used inside the handoff only
B1	citation drift, and the PDF handed to the review is not the pinned text	protocol-spine.md: 32-47, the disposition table	2 (0)	its description, used inside the handoff only
B2	a status finding was stated against a text that is not the pin	protocol-spine.md: 32-47, the disposition table	2 (0)	its description, used inside the handoff only

Code	Meaning	Defined at	Uses	Replaced by
C1	the pull request's description is stale	protocol-spine.md: 32-47, the disposition table	3 (0)	its description, used inside the handoff only
C2	module docstrings do not state the target contribution	protocol-spine.md: 32-47, the disposition table	2 (0)	its description, used inside the handoff only
H1-H6	six reductions of the audit surface: a <code>Shape</code> record, derived <code>Decidable</code> instances, <code>Seam.of</code> , a one-line <code>read_eval</code> , a shorter <code>guardedAppend</code> , the pass-through's knowledge half	protocol-spine.md: 32-47, the disposition table	27 (2)	its description, used inside the handoff only
I1	polynomials with coefficients in <code>E</code> have no ring operations at the pin	protocol-spine.md: 32-47, the disposition table	5 (0)	its description, used inside the handoff only
I2	scalar prover messages have no oracle interface	protocol-spine.md: 32-47, the disposition table	3 (0)	its description, used inside the handoff only
I3	the layout plan covers aligned blocks only	protocol-spine.md: 32-47, the disposition table	3 (0)	its description, used inside the handoff only
Findings of the review of Layer 1				
A1	the block-claim pairing cannot be used by the opening phase	protocol-layer1.md: 26-43, the disposition table	3 (2)	its description, used inside the handoff only
A2	<code>Blocks.layout</code> has not the type of an instance's layout	protocol-layer1.md: 26-43, the disposition table	3 (1)	its description, used inside the handoff only
A3	<code>bytecodeColumn_slot</code> restates the definition	protocol-layer1.md: 26-43, the disposition table	2 (1)	its description, used inside the handoff only
A4-A6	three weak tests	protocol-layer1.md: 26-43, the disposition table	8 (1)	its description, used inside the handoff only
B0, B6	items of the review skill's checklist, "clean"	nowhere in the handoff	1 each; defined nowhere in the handoff	its description, used inside the handoff only
B1	the order of equal-size blocks is stated backwards in the blueprint	protocol-layer1.md: 26-43, the disposition table	4 (1)	its description, used inside the handoff only
B2	the claims docstrings describe one weight construction where the source has two	protocol-layer1.md: 26-43, the disposition table	5 (1)	its description, used inside the handoff only
B3	"equation (5.4)" does not exist	protocol-layer1.md: 26-43, the disposition table	2 (1)	its description, used inside the handoff only
B4	a citation that is a comment cut mid-sentence	protocol-layer1.md: 26-43, the disposition table	3 (1)	its description, used inside the handoff only
C1	per-file copyright and author headers in two modules	protocol-layer1.md: 26-43, the disposition table	4 (1)	its description, used inside the handoff only
C2-C9	eight hygiene items	protocol-layer1.md: 26-43, the disposition table	11 (0)	its description, used inside the handoff only
D1	the import rule is not enforced by the script the blueprint names	protocol-layer1.md: 26-43, the disposition table	3 (2)	its description, used inside the handoff only
D2-D17	sixteen mismatches between the documents and the branch, described together (protocol-layer1.md:182-192); D3 and D6 are never named singly	protocol-layer1.md: 26-43, the disposition table	20 (0)	its description, used inside the handoff only
Findings of the review of the public-input phase				
A1	the output drops a claim on a short message	public-input-phase.md:66-79, the disposition table	5 (1)	its description, used inside the handoff only

Code	Meaning	Defined at	Uses	Replaced by
A2	the flag <code>sent</code> is fixed by no statement and no test	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	5 (1)	its description, used inside the handoff only
A3	several tests restate definitions	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	3 (0)	its description, used inside the handoff only
A4	the message type is wider than two scalars	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (0)	its description, used inside the handoff only
B1	the specification's check and the deployed check are not equivalent at a fixed challenge	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	5 (1)	its description, used inside the handoff only
B2	Schwartz-Zippel cited as Lemma 5.2; it is Lemma 3.8	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (0)	its description, used inside the handoff only
B3	six slips in the changed documents	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (0)	its description, used inside the handoff only
C1	fragile proofs declared the template for three more phases	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	5 (1)	its description, used inside the handoff only
C2	ten unprefixed public names	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (1)	its description, used inside the handoff only
C3	the module <code>docstring</code> has no pin and no provenance	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (0)	its description, used inside the handoff only
C4	generic lemmas kept in the phase module	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	2 (0)	its description, used inside the handoff only
H1-H6	six reductions of the audit surface: one output, one list of sent values, five lemmas <code>private</code> , <code>pubWitMid</code> inlined, a shared lemma for guarded verdicts, a shared embedding	<code>public-input-phase</code> . <code>md:66-79</code> , the disposition table	52 (2)	its description, used inside the handoff only
The Flock roadmap's milestones (13 occurrences in all)				
F1	the Flock roadmap's milestone "executable algebra"	the body of issue 3, :92-101	-	the name issue 3 gives it
F3	the Flock roadmap's milestone "baseline reduction"	the body of issue 3, :92-101	-	the name issue 3 gives it
F4	a milestone of the Flock roadmap	the body of issue 3, :92-101	-	the name issue 3 gives it
F5	the Flock roadmap's milestone "packed openings"	the body of issue 3, :92-101	-	the name issue 3 gives it
F6	the Flock roadmap's milestone "concrete PCS"	the body of issue 3, :92-101	-	the name issue 3 gives it: "the concrete PCS (#3, F6)"
F8	the Flock roadmap's milestone "concrete hash instances"	the body of issue 3, :92-101	-	the name issue 3 gives it
Decisions (27 references in all)				
decision 1	leanISA's <code>Caps</code> requires it, so the two roadmaps state one relation (decision, taken)	status :212-277	-	kept, with its name: "heights are powers of two"
decision 2	the public input is the statement; the instance indexes the protocol family (decision, taken)	status :212-277	-	kept, with its name: "statement and parameters"
decision 3	in the bus phase, where the point is drawn; no separate phase (decision, taken)	status :212-277	-	kept, with its name: "where the zerocheck is charged"
decision 4	the <code>To<Library></code> folders, or an <code>ArkLib</code> branch (decision, open)	status :212-277	-	kept, with its name: "where generic code lives"
decision 5	depends on the cost of the encoder (decision, open)	status :212-277	-	kept, with its name: "an end-to-end run of the honest prover"
decision 6	the protocol's witness is the committed column, in <code>Type 0</code> (decision, taken)	status :212-277	-	kept, with its name: "the witness is the stack"

Code	Meaning	Defined at	Uses	Replaced by
decision 7	every phase is tested on the toy instance (decision, taken)	status :212-277	-	kept, with its name: “phases over an abstract instance”
decision 8	the caps are a hypothesis of the adaptor’s soundness theorem (decision, taken)	status :212-277	-	kept, with its name: “five clauses, caps outside”
decision 9	the pushed and pulled tuple lists are permutations of each other (decision, taken)	status :212-277	-	kept, with its name: “balance is a permutation”
decision 10	per component, composed by the ported theorem (decision, taken)	status :212-277	-	kept, with its name: “worst-case round-by-round knowledge soundness”
decision 11	the protocol’s extractor is <code>piopExtractor</code> (decision, taken)	status :212-277	-	kept, with its name: “extractors are computable definitions”
decision 12	the predicate is Flock’s R1CS on the committed region (decision, taken)	status :212-277	-	kept, with its name: “the strong Flock predicate”
decision 13	the statement fixes cells 0 and 1 of listed columns (decision, taken)	status :212-277	-	kept, with its name: “public lines, not cells”
decision 14	the bus seam bounds every term by it (decision, taken)	status :212-277	-	kept, with its name: “the degree bound belongs to the instance”
decision 15	the prover sends the values; a line says whether its value is sent (decision, taken)	status :212-277	-	kept, with its name: “the public-input transcript”
Acceptance tests (30 references in all)				
acceptance test 1	the error carries a factor 4	blueprint :1266-1358	-	kept, with its name: “fingerprint degree”
acceptance test 2	a zero pad zeroes every product	blueprint :1266-1358	-	kept, with its name: “padding leaves are 1”
acceptance test 3	a read with count zero balances	blueprint :1266-1358	-	kept, with its name: “the nonzero count root is load-bearing”
acceptance test 4	the push and pull roots are one scalar	blueprint :1266-1358	-	kept, with its name: “one root, not two”
acceptance test 5	coordinate 0 separates the three buses	blueprint :1266-1358	-	kept, with its name: “domain separators”
acceptance test 6	the point is drawn after the commitment	blueprint :1266-1358	-	kept, with its name: “zerocheck point recycling”
acceptance test 7	the lift keeps the table’s sum	blueprint :1266-1358	-	kept, with its name: “back-loaded padding”
acceptance test 8	the wire carries three coefficients	blueprint :1266-1358	-	kept, with its name: “degree three, three scalars”
acceptance test 9	the three bus forms share the last three powers	blueprint :1266-1358	-	kept, with its name: “shared bus powers”
acceptance test 10	the third claim is pooled though nothing is sent for it	blueprint :1266-1358	-	kept, with its name: “top limb of the public input”
acceptance test 11	one member of the list satisfies all claims	blueprint :1266-1358	-	kept, with its name: “joint list binding”
acceptance test 12	sizes and root are observed before the first challenge	blueprint :1266-1358	-	kept, with its name: “absorb before squeeze”
acceptance test 13	bit k is coordinate k	blueprint :1266-1358	-	kept, with its name: “index column bit order”
acceptance test 14	the opcode is slot 3, low bit first; witness <code>bytecodeColumn_answer_bool Vec</code>	blueprint :1266-1358	-	kept, with its name: “bytecode slot bits”
acceptance test 15	blocks sit at multiples of their size, largest first	blueprint :1266-1358	-	kept, with its name: “selector alignment”
acceptance test 16	the highest variable is bound first	blueprint :1266-1358	-	kept, with its name: “variable order”
acceptance test 17	one radix-2 layer when the height is odd	blueprint :1266-1358	-	kept, with its name: “radix and parity”
acceptance test 18	the tables’ count columns only	blueprint :1266-1358	-	kept, with its name: “counts in the count tree”
acceptance test 19	the witness is read off the committed column	blueprint :1266-1358	-	kept, with its name: “the extractor reads the stack”

Code	Meaning	Defined at	Uses	Replaced by
acceptance test 20	perfect completeness has no side condition	blueprint :1266-1358	-	kept, with its name: “no exceptional challenge”
acceptance test 21	the protocol is a family indexed by the sizes	blueprint :1266-1358	-	kept, with its name: “sizes are parameters”
acceptance test 22	four numeric tags	blueprint :1266-1358	-	kept, with its name: “tags, not labels”
acceptance test 23	the bound at the caps	blueprint :1266-1358	-	kept, with its name: “numeric error”
acceptance test 24	an extractor chosen classically proves soundness, not knowledge	blueprint :1266-1358	-	kept, with its name: “the extractor computes”
acceptance test 25	no phase imports the arithmetization	blueprint :1266-1358	-	kept, with its name: “the import rule holds”
acceptance test 26	a phase’s output relation is the next one’s input relation by definition	blueprint :1266-1358	-	kept, with its name: “seams are the contract”
acceptance test 27	each clause fails alone	blueprint :1266-1358	-	kept, with its name: “the toy instance is honest”
acceptance test 28	a cubic term is rejected	blueprint :1266-1358	-	kept, with its name: “the bus seam bounds the degree”
Target theorems				
T1	arithmetization and ISA equivalence, both directions	architecture.md:203	7: BP 4, ST 1, TB 2	kept, owned by architecture.md: “arithmetization and ISA equivalence”
T2	witness-generator correctness	architecture.md:230	3: BP 2, ST 1	kept, owned by architecture.md: “witness-generator correctness”
T4	base proof extraction and completeness, the proof system’s target	architecture.md:261	21: BP 11, ST 1, TB 2, HC 3, RS 4	kept, owned by architecture.md: “base proof extraction and completeness”
T5	recursive-verifier correctness	architecture.md:277	6: BP 5, HC 1	kept, owned by architecture.md: “recursive-verifier correctness”
T6	recursion and correct-trace extractability (open)	architecture.md:284	4: BP 4	kept, owned by architecture.md: “recursion extraction”
Categories (22 occurrences in all)				
Category A	a kind of source	blueprint :178-184	-	“written from the specification”
Category B	a kind of source	blueprint :178-184	-	“transcribed from its source”
leanth’s audit codes				
FW-1, and in leanth-reuse.md FW-4, DR-101, DR-110, DR-112, DR-113, PS-2, PS-3, F-1, F-3, F-6, F-8	audit codes of the private repository leanth (“catalog audit FW-1”)	the private repository	1 in the blueprint (:1337)	not covered by the proposal
A contributor’s private codes				
P05	a private code for the algebra slice (issue 31, batching by powers): “P05 depends only on #18’s multilinear API”	nowhere	1 (the body of issue 31)	“the algebra”
P17	“no P17 replacement was implemented”	nowhere	1 (issue-12-comment-5749874958.md:5)	not covered by the proposal

Appendix D

Proposed texts for the documents and the tracker

The texts below are drafts, ready to apply once the owner has reviewed this report: the blueprint's section on how work is tracked, its table of holes with names in place of codes, the head of the status file for `main` as it now stands, the new body of the tracking issue, and one paragraph per related issue and pull request. Nothing has been posted or edited. Where a text must change again because of a technical finding of this review, the place is marked in bold; appendix A lists those changes.

Drafts, ready to apply once the owner agrees; nothing was edited and nothing was posted. Each passage is quoted as it stands at `b435631` (the status and the “How work is tracked” section are unchanged at `144c5aa`; the blueprint's lines after 50 are 4 higher there) and then given in full as proposed. Where a passage has to change again because of the review's technical findings, the change follows it in a short paragraph that ends with the register rows it comes from (appendix F). In the proposed texts, links are written as the blueprint writes them (relative paths from `docs/roadmap/`, GitHub anchors computed from the headings at `144c5aa`).

D.1 The blueprint's section “How work is tracked”

As it stands (protocol-blueprint.md:1472-1515):

```
## How work is tracked

- **This document** says what is wanted. It is edited by pull request and does not record history, status, or who is doing what.
- **[protocol-status.md](protocol-status.md)** says where things stand: coverage per layer, the frontier, the upstream ledger with issue numbers, open findings against the sources, and the decisions still pending. It is hand-maintained, headed by the commit it describes, and rewritten whole when a layer lands.
- **Issue [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12)** is the dashboard: the layer checklist with one of *open / claimed / in review / landed* per layer, links to the intention issue and pull request of each, the upstream ledger, and the frontier. It links to this document and to the status file at `main`, holds nothing that is not in them, and is updated when the status file is. Where the issue and the files disagree, the files win.
- **Holes.** Each unit of [The holes](#the-holes) is a section of the hole comment on the dashboard #12, stating what it consumes from the spine, what it produces, its tests and its upstream-watch rows. To claim a hole, or a slice of one, comment on #12 naming the hole and the exact declarations taken; the checklist line moves to *claimed*. No `[Hole]` or `[Intention]` issue is opened for it (the tracker stays small and readable whole); an issue is opened only when a slice needs its own discussion thread, titled `[Intention]: protocol - <hole>: ...`, and it is closed by the pull request that lands the slice. A hole's `Def` with its `Complete`, and its `Security`, are separate slices and separate pull requests. Whoever takes a hole reads its upstream-watch rows first.
- **Upstream watch.** The status file keeps a table of external pull requests and issues that would replace or feed a hole: repository and number, the hole, the state, the condition under which it is adopted (usually a pin bump), and the date last checked. Whoever bumps a pin rewrites the table; whoever opens a hole's pull request cites the rows it consumed.
- **To report a problem with this roadmap** – a wrong or unclear target, a source discrepancy, a
```

- missing prerequisite – open an issue titled `[Roadmap]: protocol – ...` naming the layer and the acceptance test or convention it touches. Durable source discrepancies are also recorded in [leanvm-target.md](../leanvm-target.md).
- ****To change this document****, open a pull request that edits it, titled `docs(protocol): ...`, and reference the dashboard in the description; say which layer, acceptance test, ledger entry or convention it touches. A pull request that lands a layer rewrites protocol-status.md whole in the same change and ticks the layer on the dashboard. A decision taken on a pending item is written here, as the convention or acceptance test it settles, and removed from the status file and the dashboard.
- ****Upstream work**** is tracked in the ledger of the status file: each entry carries the ArkLib (or Clean, CompPoly) issue or pull request number once opened, and the leanerVM layer that deletes its local copy when the pin moves.
- Pull requests carry `awaiting-review` when the author is done and `awaiting-author` after a review that asks for changes; the reviewer runs the `leanerVM-review` skill's three passes (specification, fidelity, hygiene) and reads the changed modules in full. A phase or component pull request imports spine names only, never `LeanerVM.Arithmetization`, and its tests run on the toy instance.

As proposed. The earlier draft's version, tightened: the table of homes stays; the tracker is changed by editing its body; a maintainer sets the labels (contributors cannot, section 13.2.7, item 4); the review skill is described by what it does, since the skill's files are under the git-ignored `.claude/` and its name is wrong (section 13.2.5, item 7); "hole" is kept and defined.

How work is tracked

A fact has one home, and every other place links to it and restates nothing.

Kind of fact	Home
What is wanted and what was decided: the targets, the conventions, the specification of every hole, the acceptance tests, the decisions, the upstream ledger	this document
What is built on `main`, what can start now, the external work to watch	protocol-status.md
Who is taking which hole	the checklist in the body of issue #12 (https://github.com/Verified-zkEVM/leanerVM/issues/12)
Discrepancies between the leanVM specification, its Rust and its Python verifier at the pin	[leanvm-target.md](../leanvm-target.md)
Limits of the pinned libraries	[dependencies.md](../dependencies.md)
Lean pitfalls met while building	[tests/README.md](../tests/README.md)
What a review found at one commit	its handoff under [docs/reviews/](../reviews/), an archive

- ****This document**** is the specification. It changes only by a pull request titled `docs(protocol): ...` that names the hole, acceptance test, decision or convention it touches. It records no state, no date of landing, no number of a pull request in flight and no person.
- ****The status**** says what is built and nothing about what is wanted. Its coverage table has one row per hole of [The holes](#the-holes); a hole is built when the declarations its row there marks in bold exist under `LeanerVM/`. The pull request that lands a hole updates that row and the status's head in the same change and writes nothing else there by hand.
- ****Issue #12**** is the tracker: a link to this document and to the status, and one checklist line per hole with its name, its state (*open*, *claimed*, *in review*, *on `main`*), the claimant's intention issue or pull request, and a link to its layer section here. It holds no specification, no ledger, no decision and no list of pull requests. It is changed by editing its body; no comment is added and no issue opened to change it. Where it and the files disagree, the files win.
- ****Holes.**** A hole is a unit of work the spine leaves open: a phase, a generic component, the adaptor, or a piece of the compilation. Its specification is its layer section here: what it produces, what it needs, its sources at the pin, its tests, and the upstream work to read first. A hole in two halves (the definition with its completeness; the knowledge soundness) is two checklist lines and two pull requests.
- ****To take a hole****, or part of one: with write access, edit its checklist line; without, open an issue titled `[Intention]: protocol - <name of the hole>: ...` whose body names the declarations taken, and a maintainer links it from the line. An intention issue is closed by the pull request that lands it. Claims are not made in comments.
- ****Names.**** Holes, ledger entries, decisions and findings are called by the names the index at the end of this document gives them. Layers, acceptance tests, decisions and the target theorems of [architecture.md](../architecture.md) keep their numbers, always with the name beside the number: "the extractor computes (acceptance test 24)". A layer, test or decision of the leanISA roadmap is always called "leanISA ...". No letter-number code is introduced.
- ****Decisions.**** A decision taken is written here as the convention, signature or acceptance test it settles, and as one row, with its name, of [Decisions](#decisions); an open decision is a row of the same table. Neither is copied anywhere else.

- **Upstream.** The ledger in [Dependencies and exact contracts](#dependencies-and-exact-contracts) says, for each result this work needs that is admitted or absent upstream at the pin, what replaces it here, the upstream issue or pull request, and when the local copy is deleted. External work in flight that may feed or replace a hole is the status's upstream watch. Whoever bumps a pin updates the ledger, the pins this document states, and the watch.
- **Discrepancies in the sources.** A difference between the specification, the Rust and the Python verifier at the pin is recorded under a name in [leanvm-target.md](../leanvm-target.md); where it binds a definition, the convention or layer section here cites it by that name.
- **Reviews.** A review's handoff records one commit. What is accepted from it becomes text of this document, and any edit it asks of the tracker is made, in the pull request that meets it; the handoff's disposition cites that text. Nothing cites a handoff as a specification.
- **To report a problem with this document**, open an issue titled `[Roadmap]: protocol - ...` naming the hole and the acceptance test or convention it touches.
- **Pull requests** are titled `feat(protocol): ...` and land with `./scripts/validate.sh` green. The description names the hole, the sources and the pin, whether each new definition is written from the specification or transcribed from its source, the ledger entries it touches and the target theorem it feeds. A maintainer labels it `awaiting-review` when the author is done and `awaiting-author` after a review that asks for changes. The reviewer reads the changed modules in full, in three passes: the statements against the specification, fidelity to the pinned sources, and hygiene. A phase or a generic component imports spine names only, never `LeanerVM.Arithmetization`, and is tested on the toy instance.

Two choices are left to the owner. (1) If the check proposed in section 13.5.1.4 is adopted, the second bullet gains one sentence:

```
`scripts/check-protocol-status.py`, run by `./scripts/validate.sh`, fails when the coverage table and the repository disagree.
```

(2) The link Decisions presumes the section “Decisions” of section 13.5.1.3; until it exists, the bullet reads

```
... as one row, with its name, of the status's decision list
```

and the status keeps that list.

D.2 The blueprint’s table “The holes”, its heading “The build”, and the two sentences that send the reader to the hole comment

As it stands (protocol-blueprint.md:588-615):

```
### The holes
```

Every hole consumes spine names only. Its specification is a section of the hole comment on the dashboard #12 (the `[Hole]` issues were folded there on 2026-09-25); `L1` groups the leaves of Layer 1 already in flight; the generic holes carry the ledger letters of the ArkLib work they will become.

```
| Hole | Unit | Produces | Consumes | Existing work | Issue |
| --- | --- | --- | --- | --- | --- |
| S | the spine | everything under [What the spine fixes](#what-the-spine-fixes); built under
↳ `LeanerVM/Protocol/Spine/` | Layer 0 only (Layer 1's `Blocks.layout` inhabits `Layout` for aligned blocks)
↳ | leanth's explore branch |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| G1 | virtual sumcheck, `Sumcheck.Def` and completeness (Layer 4) | `Virtual`, `sumcheck`,
↳ `sumcheck_perfectCompleteness` | nothing | #42, ArkLib #1128, ArkLib `main`'s `Sumcheck/Interaction/` |
↳ [#37](https://github.com/Verified-zkEVM/leanerVM/issues/37) |
| G2 | sumcheck rbr knowledge, `Sumcheck.Security` (Layer 4, A1) | `sumcheck_rbrKnowledgeSoundness`, `d/\|F\|`
↳ per round | G1 | ArkLib #1129, `Interaction/Soundness.lean` |
↳ [#37](https://github.com/Verified-zkEVM/leanerVM/issues/37) |
| G3 | batching by powers, `Batch.Def` and `Security` (Layer 4) | `batchClaims`, `(k - 1)\|F\|` | nothing |
↳ #43, ArkLib #615's `gammaPowers` | [#31](https://github.com/Verified-zkEVM/leanerVM/issues/31) |
| G4 | fingerprint, Lemma 5.1, the collision bound (Layer 5) | `fingerprint`, `sideProduct`,
↳ `sideProduct_poly_eq_iff`, `sideProduct_collision` | nothing | #39, ArkLib #901 |
↳ [#33](https://github.com/Verified-zkEVM/leanerVM/issues/33) |
```

```

| G5, G6 | GKR: `Gkr.Def` and completeness; `Gkr.Security` (Layer 5) | `gkr`, `gkrError`, its two theorems | G1
↳ (G6 also G2) | ArkLib #818 as a pattern |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| L1 | Layer 1: tables, stacking, the fixed columns | `Blocks`, `stack_eval`, `unstack_eval2`,
↳ `stack_eval_ambient`, `bitProductTable`, `powersTable`, `placeSlice`; `Blocks.layout`, `Layout.comap`,
↳ `ColumnClaim.holds_iff_weighted`, `padHigh`, `BlockClaim`, `idxColumn_eval`,
↳ `bytecodeColumn_answer_boolVec`, `bytecodeColumn_eval` | Layer 0; the spine's `Layout` and claims | leanth,
↳ per the catalog [leanth-reuse.md](leanth-reuse.md) | #27, #32, #35, #36 |
| I1 | Clean components as polynomials (Layer 2) | `Expression.toMvPolynomial`, `degreeBound`,
↳ `Component.toM3`, `Ensemble.toM3`, the two bridge theorems | Clean | Clean #466 |
↳ [#28](https://github.com/Verified-zkEVM/leanerVM/issues/28) |
| I2 | the adaptor (Layer 3) | `leanISAInstance`, `stackOf`, `witnessOf`, `satisfiedBy_witnessOf`,
↳ `m3Holds_stackOf`, `witnessOf_stackOf` | S, I1, leanISA Layers 5–8, L1; #3 (the Flock witness generator,
↳ "the RICS holds ⇒ the limb slots compress") | |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| P1, P2 | the bus phase (Layer 6) | `busPhase`, `leaf_decomposition`, `busError`; its `Security` | S, G5 (P2
↳ also G6, G4), L1 | | [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| P3, P4 | the table sumcheck phase (Layer 7) | `tableSummand`, `tableSummand_target`, `tableSumcheck`; its
↳ `Security` | S, G1 (P4 also G2) | #42 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| P5 | the public-input phase (Layer 8) | `publicInputPhase`, both halves | S, L1 | |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| P6 | the Flock phase at the flock seam (Layer 9) | `FlockOut`, `limbColumns`, `flockError_le`; the inhabitant
↳ is #3's | S | #3, ArkLib #383, #893 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| P7, P8 | the claim pool and the opening phase (Layer 10) | `Weight`, `WeightedClaim`, `openingPhase`; its
↳ `Security` | S, G3, G1, L1, #43 | |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| C1 | the knowledge-soundness append (ledger A2) | `Verifier.KnowledgeStateFunction.appendGuarded`,
↳ `Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first` in
↳ `LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean`, the port of ArkLib #615's `Append/Knowledge.lean`: done
↳ on the spine's branch, as the port; deleted at the pin bump | ArkLib only | ArkLib #615, #676 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| K1 | WHIR over binary Reed–Solomon codes (Layer 11) | `encode`, `whirOpen`, `whirOpen_rbrSoundness`,
↳ `McaJohnson` | `WeightedClaim`, Layers 0, 1 | #3 F6, ArkLib #383, #992 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| K2 | Merkle, BLAKE2s bytes, the WHIR parameters (Layer 11) | `merkleRoot`, `merkleVerify`, `blake2sBytes`,
↳ `ladder` | leanISA Layer 1 | ArkLib #4 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| K3 | transcript, `Proof`, `verify`, `verify_iff_compiled`, the FS and BCS interfaces (Layer 12) | as Layer 12
↳ | S (schedules, phase `Def`s), K1, K2 | ArkLib #848, #469, #627 |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |
| K4 | T4 (Layer 13) | `baseVerifier_extractExecution`, `baseProver_complete` | K3, I2, leanISA Layer 10 | |
↳ [#12](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972) |

```

As proposed. Every row is named in words; the rows are in build order; a hole in two halves has one row per half, matching the checklist; the field instances (Layer 0), a line of today's checklist, get a row; the columns "Existing work" and "Issue" leave (their content goes, without state, to each layer section's "upstream work to read first", and to the tracker); "Consumes" becomes "Needs" and names holes, not codes or pull requests. In "Produces", names the spine owns are removed (**Weight**, **WeightedClaim** from the opening phase, **FlockOut** from the Flock phase: section 13.2.5, item 6), "Lemma 5.1" becomes the specification's Lemma 5.2 and Theorem 5.1 (section 13.2.5, item 8), and the generic shapes **Sumcheck**, **Def**, **Gkr.Def**, **Batch.Def**, which the spine replaced by **Component.Def** (section 13.2.4, item 7), are gone. A name in bold is one this hole and no other declares, so that the status's coverage can be read off the repository (section 13.5.1.4); names not yet built follow the pattern of the public-input phase (**xPhase**, **xComplete**, **xSecurity**) and are proposals.

The holes

A hole is a unit of work the spine leaves open: a phase, a generic component, the adaptor, or a piece of the compilation. Every hole consumes spine names only, and its layer section below is its specification: what it produces, what it needs, its sources, its tests and the upstream work to read first. The table lists every hole in build order. A name in bold is declared by this hole and no other; the hole is built when the names in bold of its row are declared.

```

| Hole | Specified in | Produces | Needs |
| --- | --- | --- | --- |
| field instances | [Layer 0]{#layer-0-the-arklib-dependency-and-the-field-instances} | `Column`,
↳ `instSampleableTypeK`, **`instSampleableTypeE`**, **`card_E`**, **`evalOracle`**, `evalOracle_answer`,
↳ `instOracleInterfaceE`, `instOracleInterfaceListE` | nothing |

```

```

| the spine | [The spine](#the-spine) | everything under [What the spine fixes](#what-the-spine-fixes), under
↳ `LeanerVM/Protocol/Spine/` and the generic modules of `LeanerVM/Protocol/ToArkLib/` it imports; marked by
↳ **M3Holds**, **Phases**, **piop_perfectCompleteness**, **piop_rbrKnowledgeSoundness** | field
↳ instances |
| the knowledge-soundness composition | [What the spine fixes](#what-the-spine-fixes), item 5 |
↳ **Verifier.KnowledgeStateFunction.appendGuarded**,
↳ **Verifier.append_rbrKnowledgeSoundnessWorstCaseWith_of_guarded_first** (`ToArkLib/KnowledgeAppend.lean`,
↳ the port of ArkLib pull request 615, deleted when the pin passes it) | ArkLib only |
| tables and stacking | [Layer 1](#layer-1-hypercube-tables-stacking-the-index-and-bytecode-columns) |
↳ `Blocks`, **Blocks.stack_eval**, **Blocks.unstack_eval**, **Blocks.stack_eval_ambient**,
↳ `bitProductTable`, `powersTable`, `placeSlice`, **Blocks.layout**, `Layout.comap`,
↳ **ColumnClaim.holds_iff_weighted**, `padHigh`, `BlockClaim`, **idxColumn_eval**,
↳ **bytecodeColumn_answer_boolVec**, `bytecodeColumn_eval` | field instances; the spine's `Layout` and
↳ claims |
| Clean expressions as polynomials | [Layer 2](#layer-2-clean-components-as-polynomials) |
↳ **Expression.toMvPolynomial**, `Expression.degreeBound`, `Component.toM3`, **Ensemble.toM3**,
↳ **toM3_constraints_iff**, **toM3_flushes_eq** | Clean |
| the adaptor | [Layer 3](#layer-3-the-m3-instance-of-leanisa) | **leanIsaInstance**, `stackOf`, `witnessOf`,
↳ **satisfiedBy_witnessOf**, **m3Holds_stackOf**, `witnessOf_stackOf` | the spine; Clean expressions as
↳ polynomials; tables and stacking; leanISA Layers 5 to 8; from the Flock roadmap (#3), its witness generator
↳ and the lemma "the RICS holds  $\Rightarrow$  the limb slots compress" |
| sumcheck: definition and completeness | [Layer 4](#layer-4-sumcheck-for-eq-weighted-virtual-polynomials) |
↳ `Virtual`, **sumcheck**, **sumcheck_perfectCompleteness** | nothing |
| sumcheck: knowledge soundness | [Layer 4](#layer-4-sumcheck-for-eq-weighted-virtual-polynomials) |
↳ **sumcheck_rbrKnowledgeSoundness**, error `d/\|F\|` per round | sumcheck: definition and completeness |
| batching by powers | [Layer 4](#layer-4-sumcheck-for-eq-weighted-virtual-polynomials) | **batchClaims**
↳ with both halves, error `(k - 1)/\|F\|` | nothing |
| fingerprint and collision bound | [Layer 5](#layer-5-fingerprints-the-grand-product-and-gkr) | `fingerprint`,
↳ `sideProduct`, **sideProduct_poly_eq_iff** (specification Lemma 5.2), **sideProduct_collision**
↳ (specification Theorem 5.1) | nothing |
| grand-product GKR: definition and completeness | [Layer 5](#layer-5-fingerprints-the-grand-product-and-gkr) |
↳ `ProductTree`, **gkr**, `gkrError`, **gkr_perfectCompleteness** | sumcheck: definition and completeness
↳ |
| grand-product GKR: knowledge soundness | [Layer 5](#layer-5-fingerprints-the-grand-product-and-gkr) |
↳ **gkr_rbrKnowledgeSoundness** | grand-product GKR: definition and completeness; sumcheck: knowledge
↳ soundness |
| bus phase: definition and completeness | [Layer 6](#layer-6-the-bus-phase) | `pushLeaves`, `pullLeaves`,
↳ `countLeaves`, `leaf_decomposition`, `busError`, **busPhase**, **busComplete** | the spine;
↳ grand-product GKR: definition and completeness; tables and stacking (`Blocks.stack_eval_ambient_one`,
↳ `idxColumn_eval`, `bytecodeColumn_eval`) |
| bus phase: knowledge soundness | [Layer 6](#layer-6-the-bus-phase) | **busSecurity** | bus phase:
↳ definition and completeness; grand-product GKR: knowledge soundness; fingerprint and collision bound |
| table sumcheck phase: definition and completeness | [Layer 7](#layer-7-the-table-sumcheck-phase) |
↳ `tableSummand`, `tableSummand_target`, **tableSumcheck**, **tableSumcheckComplete** | the spine;
↳ sumcheck: definition and completeness |
| table sumcheck phase: knowledge soundness | [Layer 7](#layer-7-the-table-sumcheck-phase) |
↳ **tableSumcheckSecurity** | table sumcheck phase: definition and completeness; sumcheck: knowledge
↳ soundness |
| public-input phase | [Layer 8](#layer-8-the-public-input-phase) | **publicInputPhase**,
↳ **publicInputComplete**, **publicInputSecurity**; the computable `PublicInput.pSpec`, `check`,
↳ `pooled`, `prover`, `verifier` that the compiled verifier reads | the spine; tables and stacking
↳ (`evalMLe_append_boolVec`) |
| Flock phase | [Layer 9](#layer-9-the-flock-and-ring-switching-boundary) | **FlockInterface**, with
↳ `limbColumns` and `flockError_le`; its inhabitant is the Flock roadmap's (#3) | the spine |
| opening phase: definition and completeness | [Layer
↳ 10](#layer-10-the-claim-pool-the-opening-sumcheck-and-the-oracle-protocol) | **openingPhase**,
↳ **openingComplete** | the spine; batching by powers; sumcheck: definition and completeness; tables and
↳ stacking (`ColumnClaim.holds_iff_weighted`) |
| opening phase: knowledge soundness | [Layer
↳ 10](#layer-10-the-claim-pool-the-opening-sumcheck-and-the-oracle-protocol) | **openingSecurity** |
↳ opening phase: definition and completeness; batching by powers; sumcheck: knowledge soundness |
| WHIR opening | [Layer 11](#layer-11-whir-over-binary-reedsolomon-codes-and-merkle-trees) | `encode`,
↳ `encode_column_weight`, **whirOpen**, **whirOpen_rbrSoundness**, `McaJohnson` | the spine's
↳ `WeightedClaim`; field instances; tables and stacking |
| Merkle trees, the byte hasher and the WHIR parameters | [Layer
↳ 11](#layer-11-whir-over-binary-reedsolomon-codes-and-merkle-trees) | **merkleRoot**, **merkleVerify**,
↳ **blake2sBytes**, **ladder** | leanISA Layer 1 (`compress`) |
| transcript, proof object and compiled verifier | [Layer
↳ 12](#layer-12-compilation-the-transcript-the-proof-and-the-executable-verifier) | `FsState`, `Proof`,
↳ `RoundPoly.decode`, `leanVmIopp`, **verify**, **verify_iff_compiled**, `FiatShamirSecurity`,
↳ `BcsSecurity`, **verify_knowledgeSound** | the definitions of the five phases (not their knowledge
↳ soundness); WHIR opening; Merkle trees, the byte hasher and the WHIR parameters |

```



```
| the base theorems (target T4) | [Layer 13](#layer-13-t4-and-the-fixtures) |
↪ **baseVerifier_extractsExecution**, **baseProver_complete** | transcript, proof object and compiled
↪ verifier; the adaptor; leanISA Layer 10 |
```

With the review’s technical findings, the generic rows of Layers 4 and 5 produce **Component.Defs** with their **Complete** and **Security** in the named form, the sumcheck’s in both variants the verifiers run (the plain one, and the normalized one of the GKR layers, whose round message is a degree-4 cofactor); the bus phase’s row produces a **BusOut I** in the shape of the Rust’s **BusVerify** (one point, forms per side and sumcheck table with the degree bound in the type, three totals, the column claims), which the table sumcheck’s rows need; and the Flock phase’s row produces a phase from **Seam.pub** to **Seam.flock** that hands the limb claims on and adds the ring-switched claim, needing a **FlockRegion I** from the adaptor in place of **FlockInterface** and **limbColumns**. The opening phase’s rows produce the batching challenge λ and one weighted query of the stack and need neither sumcheck row, the WHIR row taking λ as its level-0 batching; the base theorems’ row states both theorems with **WellFormedBytecode prog**. The Merkle row stays as drafted until the owner decides whether Layer 11 builds on VCVio’s Merkle trees, whose fit with leanVM’s trees is unverified (from R36, R7, R2, R14, R16, R18, R19, R27, R77).

The heading and first paragraph of the build, as they stand (protocol-blueprint.md:617-623):

```
## The build: the spine, then fourteen layers

Every layer names what to define and what to prove, intrinsically. Each layer's tests are part
of the layer. The spine (hole S) is built after Layer 0; the layers
keep their numbers and are read as the holes of the table above. Layers 3 and 6 to 10 are written
over `I : M3Instance`; where a signature below says `(prog) (s)`, read `leanIsaInstance prog s`.
A layer lands only fully proved; a hole's `Def` and `Security` are separate landings.
```

As proposed. The heading says the spine comes first, which is false (Layer 0 precedes it, :620); “hole S” and “read as the holes” go. No link in the repository points at the old anchor **#the-build-the-spine-then-fourteen-layers** (grep over the tracked Markdown).

```
## The layers

Fourteen layers, numbered 0 to 13, are built in order, with the spine after Layer 0. Each layer
section specifies the holes of [The holes](#the-holes) that name it: what to define and what to
prove, its sources, its tests, which are part of it, and the upstream work to read first. The
phases (Layers 6 to 10) are written over an abstract `I : M3Instance` with the statement types
of `Phases`; Layer 3 builds the leanISA instance and the adaptor. A hole lands only fully
proved, and its definition with its completeness lands apart from its knowledge soundness.
```

The review recommends restating the sketches of Layers 4 to 7, 9 and 10 on the spine’s types (section 13.2.4), after which no signature below takes **(prog) (s)** and today’s sentence “where a signature below says **(prog) (s)**, read **leanIsaInstance prog s**” has nothing left to explain; if the restatement lands in a later pull request than this text, the sentence stays until it does (from R36).

The two sentences that send the reader to the hole comment. As they stand (protocol-blueprint.md:339-341):

```
The spine is the one pull request (hole S; its specification is a section of the [hole
↪ comment](...#issuecomment-5833669972) on the dashboard #12) that fixes everything two neighbouring pieces of
↪ work would otherwise have to agree on, and proves the composition once.
```

And :590-593, quoted in the table above. As proposed: the first becomes

```
The spine fixes everything two neighbouring pieces of work would otherwise have to agree on, and proves the
↪ composition once.
```

the second is replaced by the new first paragraph of “The holes” above.

D.3 The status file’s head and “At a glance”, for main at 144c5aa

As it stands (protocol-status.md:1-27, identical at b435631 and 144c5aa):


```
# Status: the leanVM proof system on ArkLib
```

```
This file records where the [protocol roadmap](protocol-blueprint.md) stands as of `main` at
`f4d858c` (Layer 1, PR #59, merged on 2026-09-29), checked on 2026-09-29, together with the open
pull requests and the public-input phase (hole P5) as built on the branch
`feat/protocol-public-input` (pull request #60); this snapshot accompanies that branch. It is a
hand-maintained snapshot, rewritten whole when a layer or hole lands or a decision is taken; the
roadmap is the authority on what is wanted, and the tracking issue
[#12](https://github.com/Verified-zkEVM/leanerVM/issues/12) mirrors the hole checklist below.
Open pull requests and prerequisite-branch adoption are not changes landed on `main`. The
dependency pins are unchanged.
```

```
## Where this roadmap stands
```

```
**At a glance.** Layer 0 is on `main` (#15, 2026-09-11), and so is the spine (hole S, #58,
`5cb7da6`, 2026-09-28) with the knowledge-soundness append ArkLib admits (hole C1, ledger A2),
ported from ArkLib #615 to `LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean`: `#print axioms` on
`piop_rbrKnowledgeSoundness` gives the kernel's three axioms. The spine is generic over
`I : M3Instance` and imports nothing from `LeanerVM/Arithmetization/` (the wall holds). Concrete
in it: the instance and the relation `M3Holds` (decidable; the toy instance decides it by
evaluation), the claims and the six seams, the hole interfaces
`Component.Def`/`Complete`/`Security` with their binary composition, the pass-through phase and
the commit phase with both halves proved, the bundle `Phases`, the protocol's extractor
`piopExtractor` and the two master theorems `piop_perfectCompleteness` and
`piop_rbrKnowledgeSoundness`, each conditional on the phases only, and the refinement of
relations. It was revised before merging after the adversarial review
[protocol-spine.md](../reviews/protocol-spine.md).
```

What is wrong with it today: it describes `f4d858c` and calls the public-input phase a draft on a branch (section 13.2.1, items 1 to 5); it says the pins are unchanged, and pull request 61 has moved four of them (section 13.2.11, item 7); it names holes by code. The “At a glance” paragraph itself is still true of the spine; what it omits is everything after it.

As proposed. The head names the revision it describes (the status cannot name the commit that contains it; it names the last merge it accounts for, as today), states the pins once, and says what the file is. “At a glance” becomes a table of what is on `main` and three short paragraphs. Names replace codes throughout; a hole’s name is the one of appendix D.2.

```
# Status: the leanVM proof system on ArkLib
```

```
Where the [protocol blueprint](protocol-blueprint.md) stands on `main` at `144c5aa`
(2026-09-29), checked on 2026-09-30. This file says what is built and nothing about what is
wanted: the blueprint is the specification, and issue
[#12](https://github.com/Verified-zkEVM/leanerVM/issues/12) says who is taking which hole.
Open pull requests are not part of `main`.
```

```
The pins are those of `upstreams.json` at `144c5aa`: leanVM `a386121f` (unchanged), ArkLib
`7653a901`, CompPoly `572f9973`, Clean `42fe4b26`, VCVio `a4232d08`, PolyFun `41d3b21d`, Lean
and Mathlib `v4.34.1`. The upgrade to them, pull request #61, is described in
[dependencies.md](../dependencies.md#lean-434-port-review).
```

```
## Where this stands
```

```
**At a glance.** Five holes are on `main`, and the pins moved once since the last of them:
```

```
| On `main` | Pull request | Commit | Merged |
| --- | --- | --- | --- |
| field instances (Layer 0) | #15 | `51021d9` | 2026-09-11 |
| the spine | #58 | `5cb7da6` | 2026-09-28 |
| the knowledge-soundness composition, ported from ArkLib pull request 615 | #58 | `5cb7da6` | 2026-09-28 |
| tables and stacking (Layer 1) | #59 | `f4d858c` | 2026-09-29 |
| the public-input phase (Layer 8), both halves, error `1/\|E\|` on its one challenge | #60 | `b435631` |
↪ 2026-09-29 |
| no hole: the upgrade to Lean 4.34.1 and the new pins | #61 | `144c5aa` | 2026-09-29 |
```

```
The two master theorems, `piop_perfectCompleteness` and `piop_rbrKnowledgeSoundness`, are
proved over an abstract instance `I : M3Instance` and are conditional on the five phases after
the commit (`Phases.Complete`, `Phases.Security`); of those five, the public-input phase is
built. The upgrade changed proofs and imports under `LeanerVM/Protocol/` and no statement; the
two probability bridges (`ToArkLib/GuardedVerdict.lean`, `ToVCVio/UniformSample.lean`) were
restated in VCVio's new probability notation with the same relations and bounds.
```

The import rule of the proof system (no module above the adaptor imports ``LeanerVM/Arithmetization/``) is checked by review, not by ``scripts/check-layers.sh``. Two modules import the arithmetization: ``FixedColumns.lean``, whose bytecode column names the program, an exception the blueprint lists; and ``Basic.lean``, an empty placeholder that nothing imports and the blueprint does not list.

Not built: the bus, table sumcheck, Flock and opening phases; the generic components (sumcheck, batching by powers, the fingerprint and collision bound, grand-product GKR); Clean expressions as polynomials; the adaptor; the WHIR opening; Merkle trees, the byte hasher and the WHIR parameters; the transcript, the proof object and the compiled verifier; the two base theorems (target T4). Three pull requests are open, each part of a hole: #39 (the fingerprint polynomial) and #43 (power batching), both based on the branch of the closed #18, and #42 (the honest round polynomials of a sumcheck), based on ``main``; all three still put their modules under ``LeanerVM/Protocol/Generic/``, which the convention `*Generic code*` replaced.

Settled since the draft. The review’s axiom probe, re-run at 144c5aa with the new pins, gives Lean’s three axioms and no `sorryAx` for the two master theorems, the existential form, both halves of the commit and public-input phases, the composition lemmas and the extractors, exactly as at the old pins (appendix B); Layer 1’s theorems and the audit of 3298 declarations were not re-run by the review, so the status keeps its line on them only after the owner’s own audit at 144c5aa. The composition ArkLib admits is still admitted at 7653a901 (pull request 615 open), so the port stays and the table’s third row stands as drafted; and the new CompPoly pin changed what a numeral means in $\mathbb{K}$, which pull request 61 carried through the fixtures but not through one test comment (`tests/LeanerVMTTests/Protocol/Field.lean:25-26, 37-38`, to be fixed in the same pull request), so “At a glance” gains the paragraph below (from R34, R127).

``#print axioms`` at ``144c5aa`` gives the kernel’s three axioms, and no ``sorryAx``, for the two master theorems and both halves of the commit and public-input phases. The knowledge-soundness composition is still admitted upstream at ArkLib ``7653a901`` (pull request 615 is open), so the port stays. Since CompPoly ``572f9973`` a numeral in ``K`` has its characteristic-two value ``(2 : K) = 0``: fixtures and tests write an encoded word as ``K.ofBits n``.

The rest of the status, at the same revision. The head and “At a glance” above replace :1-27. The following lines must also change, or the file contradicts its own head:

- :55-56 “(review finding D1)”: name the finding (“the Layer 1 review’s finding that the import rule has no script”) and add `Basic.lean`, as above.
- :58-60: “The public-input phase (hole P5) is built, on the branch `feat/protocol-public-input`, pull request #60, a draft, ...” becomes “The public-input phase is on `main` (#60, b435631, 2026-09-29).”; the rest of the paragraph stands.
- :78-80 “Three pull requests are open beside #60”: “Three pull requests are open”.
- :90-109, the coverage table: row “P5” (:101) “built, draft #60 ... lands when #60 merges” becomes “on `main` (#60, b435631)”; row “G4” (:95) “Lemma 5.1” becomes “Lemma 5.2 and Theorem 5.1”; every hole code becomes the name of appendix D.2.
- :113-119: the sentence “#60 carries the public-input phase” and the table row “#60 (draft)” go.
- :146-148: “The public-input phase (P5) is the draft #60 ... #60 ticks P5 on the dashboard when it merges” becomes “The public-input phase is on `main` (#60)”.
- :159-160: the holes that can start now, by name.
- :161-163: “The pins have not moved. ArkLib `main` is 246 commits past dca90385 (finding A18); the next bump is one planned change ...” becomes “The pins moved with pull request 61 (see the head).”
- :177 “`ToCompPoly/{Stacking,AmbientStacking,Claims}.lean`”: `Claims.lean` does not exist; the claim record is `BlockClaims.lean`.
- :178 “ArkLib #4 (Merkle)”: closed on 2026-09-27 in favour of VCVio issue 571 (section 13.2.9).
- :179 “the #907 slices landing on `main`”: ArkLib issue 907 closed on 2026-09-26.
- :190-206, the upstream watch: its states are dated 2026-09-24 and several are wrong (section 13.2.1, items 8 to 11).
- :212 heading “Decisions pending” over decisions mostly taken: “Decisions”.
- :332, :355, :360: the findings about ArkLib, Clean and CompPoly are headed by the former pins; they say so (“at the former pin dca90385”).

The upgrade settled three rows and left the others standing. ArkLib's typed executor and one-round bound (:201, "adopt when: the pin bump") are in the pin, but the typed framework still has no knowledge soundness, no round-by-round notion and no extractor, so the row becomes the statement that the spine stays on OracleReduction with the port, still needed while pull request 615 is open; CompPoly 572f9973 contains #331, so "a CompPoly containing #331" (:162) is met and the eager-Fintype row is retired; Clean 42fe4b26, Clean pull request 474 merged over Clean's main, ports proofs only and leaves balanceOf and BalancedInteractions unchanged, so the row on Clean's balance stands. Clean pull request 464 is not in the new pin (it has no DirectedChannel), and 466 and 446 were open when the review read them, 446 as a draft; whether any of 446 is inside 42fe4b26 is unverified, so their rows keep their state until someone checks, and the ledger's other rows stand at the new pins (table 3.8) (from R34, R73, R84, R125).

D.4 The body of issue 12, and where the hole comment goes

As it stands. The body (issue-12-body.md:1-97, unchanged since 2026-09-28 14:28 UTC) has nine parts: pointers and pins (:1-4), a claim to hold nothing of its own (:6), related issues (:8), "What this roadmap builds (revision 2)" (:10-14, a 1,700-word restatement of the spine), the hole checklist (:16-40), "Ordering" (:42), "Open pull requests (2026-09-28)" (:44-57), "Upstream ledger" (:59-74), "Upstream watch" (:76-78), "Frontier" (:80-85), "Decisions pending" (:87-89) and "How to work on this" (:91-96). Its checklist, quoted whole because it is what the new body replaces:

Hole checklist

```
Status per hole: **open** / **claimed** (by a comment on this issue naming the declarations taken) / **in
  ↳ review** (the PR carrying `awaiting-review`) / **landed** (the merged PR). Each hole's specification
  ↳ (produces, consumes, tests, upstream watch) is a section of the [hole
  ↳ comment](https://github.com/Verified-zkEVM/leanerVM/issues/12#issuecomment-5833669972). Tick a hole only
  ↳ when its pull request has merged with `./scripts/validate.sh` green and its tests in `tests/`. A hole with
  ↳ two halves ticks twice.

- [x] **0 – ArkLib dependency and field instances (Layer 0).** – *landed* (#15, 2026-09-11)
- [ ] **L1 – Layer 1: generic tables and stacking, and the leaves.** Generic half in draft #18 (rebase onto
  ↳ `main` pending; #25, #26 merged into it); leaves in review: #40 (`stack_eval_ambient`, #36), #38
  ↳ (`unstack`, `BlockClaim`, #35), #41 (`idxColumn_eval`, `bytecodeColumn_slot`, #32), #26 (coefficient
  ↳ transport, #27). – *in review*; nothing lands before #18 does
- [x] **5 – the spine.** – *landed* (#58, `5cb7da6`, 2026-09-28; reviewed the same day, [docs/reviews/protocol]
  ↳ -spine.md](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/reviews/protocol-spine.md))
- [ ] **G1 – virtual sumcheck, `Sumcheck.Def` and completeness (Layer 4).** #37; the honest round algebra and
  ↳ the ArkLib bridge in #42 – *claimed*
- [ ] **G2 – sumcheck round-by-round knowledge, `Sumcheck.Security` (Layer 4, ledger A1).** #37; a one-round
  ↳ leaf prepared, unpublished – *claimed*
- [ ] **G3 – batching by powers, `Batch.Def` and `Security` (Layer 4).** #31; the algebra and the `(J - 1)/|F|`
  ↳ count in #43 – *claimed*
- [ ] **G4 – fingerprint, Lemma 5.1, the collision bound (Layer 5).** #33; the fingerprint in #39 – *claimed*
- [ ] **G5, G6 – GKR: a `Component.Def` for the grand product and its two halves (Layer 5).** – *open*
- [ ] **I1 – Clean components as polynomials (Layer 2).** #28; Clean #466 approved, unmerged – *claimed*
- [ ] **I2 – the adaptor (Layer 3): `leanIsaInstance` (an `M3Instance` with `d := 2`, three public lines on
  ↳ `mem_0`, `mem_1`, `mem_2`, and a layout with two readers: #18's `Blocks` for the aligned blocks, a strided
  ↳ reader for the eighteen BLAKE2S limb slots of `q_flock`, `stackOf`, `witnessOf`, `satisfiedBy_witnessOf
  ↳ (hs : s.Admissible prog)`, `m3Holds_stackOf`, `witnessOf_stackOf`.** – *open; needs I1, #18, and #3's
  ↳ witness generator and compression lemma*
- [ ] **P1, P2 – the bus phase (Layer 6): `Phase.Def I I.Stmt (I.Stmt × BusOut I)` against `Seam.commit`,
  ↳ `Seam.bus`.** – *open; P1 needs G5*
- [ ] **P3, P4 – the table sumcheck phase (Layer 7): `Phase.Def I (I.Stmt × BusOut I) (I.Stmt × TableOut I)`
  ↳ against `Seam.bus`, `Seam.table`.** – *open; needs G1*
- [ ] **P5 – the public-input phase (Layer 8): `Phase.Def I (I.Stmt × TableOut I) (I.Stmt × PubOut I)` against
  ↳ `Seam.table`, `Seam.pub`, over `I.publicLines`: one challenge, one pooled column claim per line, the prover
  ↳ sends nothing.** – *open; the smallest phase, a good first hole*
- [ ] **P6 – the Flock phase (Layer 9): `Phase.Def I (I.Stmt × PubOut I) (I.Stmt × FlockOut I)` against
  ↳ `Seam.pub`, `Seam.flock`; its input predicate is the strong `aux`, Flock's R1CS on `q_flock` (decision 12),
  ↳ the eighteen column claims over the strided reader `limbColumns`; the inhabitant is #3's.** – *open; needs
  ↳ #3*
- [ ] **P7, P8 – the opening phase (Layer 10): `Phase.Def I (I.Stmt × FlockOut I) Unit` against `Seam.flock`,
  ↳ `Seam.done`.** – *open; needs G1, G3*
- [x] **C1 – the knowledge-soundness append (ledger A2).** – *landed* with the spine (#58):
  ↳ `LeanerVM/Protocol/ToArkLib/KnowledgeAppend.lean`, the port of ArkLib #615's `Append/Knowledge.lean` at
  ↳ `ca7a2577`, with its attribution; `Component.Security.append` and the master knowledge theorem take no
  ↳ assumption. The file is deleted, and its two names replaced by ArkLib's, when the pin moves past #615
```

```
- [ ] **K1 – WHIR over binary Reed–Solomon codes (Layer 11).** Shared with #3 F6 – *open*
- [ ] **K2 – Merkle trees, BLAKE2s bytes, the WHIR parameters (Layer 11).** – *open*
- [ ] **K3 – transcript, `Proof`, `verify`, `verify_iff_compiled`, the FS and BCS interfaces (Layer 12).** –
  ↳ *open; needs the phase `Def`s, K1, K2*
- [ ] **K4 – T4: `baseVerifier_extractsExecution`, `baseProver_complete` (Layer 13).** – *open; needs K3, I2,
  ↳ leanISA Layer 10*
- [x] **VCVio controls for K3 and P7.** #29, #30 – *landed upstream* (VCVio #784)
```

Ordering: P1, P3, P5 (`Def`s), G1 to G6, I1, K1 and K2 can start now on `main`; I2 after #18 and I1 land; P2,
 ↳ P4, P8 also need the `Security` of the generic component they use. K3 needs the phase `Def`s, K1 and K2. K4
 ↳ last.

Against the repository at b435631/144c5aa and GitHub on 2026-09-30, its state is wrong in five lines: Layer 1 is on main (#59), not “in review” behind #18 (:21); the public-input phase is on main (#60) and its line specifies the opposite transcript, “the prover sends nothing” (:32); pull requests 18, 38, 40, 41 are closed (:21, :50-53); the VCVio controls (:40) are not a hole; and the pins of :4 and :14 are no longer the pins (section 13.2.11).

As proposed, ready to paste as the whole body. It restates nothing of the blueprint: the specification, the ledger, the decisions, the ordering and the list of pull requests are gone (they are in the blueprint, on GitHub, or generated in the status). The state of each line is derived from main at 144c5aa (declarations present), from the merged pull requests (15, 58, 59, 60), and from the open intention issues and pull requests (28, 31, 33, 37; 39, 42, 43), not from the old body. Links to sections use the anchors of the headings at 144c5aa; those of appendix D.2 (“The holes”, “The layers”) are the proposed ones.

```
**Blueprint:** [ `docs/roadmap/protocol-blueprint.md` ](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/
  ↳ /roadmap/protocol-blueprint.md): what is wanted; the specification of every hole, in its layer section; the
  ↳ decisions; the upstream ledger.
**Status:** [ `docs/roadmap/protocol-status.md` ](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadm
  ↳ ap/protocol-status.md): what is built on `main`, what can start now, the external work to
  ↳ watch.
**Pins:** [ `upstreams.json` ](https://github.com/Verified-zkEVM/leanerVM/blob/main/upstreams.json).
```

This issue is the checklist below and nothing else. It changes by an edit of this body, never by a comment;
 ↳ where it and the files disagree, the files win. The rules are in the blueprint's [How work is tracked](http
 ↳ s://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#how-work-is-tracked).

Related: #4 (leanISA: the relation `SatisfiedBy` the adaptor targets), #3 (Flock: the Flock phase, and the
 ↳ witness generator and compression lemma the adaptor takes), #16 and #20 (Clean's bus balance over binary
 ↳ fields), #23 (fixed columns after Clean pull request 446).

Holes

State: ***open***; ***claimed*** (the claimant's intention issue, and any pull request carrying part of the hole);
 ↳ ***in review*** (the hole's pull request, labelled `awaiting-review`); ***on `main`*** (the merged pull
 ↳ request). A line is ticked when its pull request has merged.

```
- [x] **Field instances** (Layer 0): on `main`, #15. [Specification](https://github.com/Verified-zkEVM/leanerVM/
  ↳ /blob/main/docs/roadmap/protocol-blueprint.md#layer-0-the-arklib-dependency-and-the-field-instances)
- [x] **The spine** on `main`, #58. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/
  ↳ roadmap/protocol-blueprint.md#the-spine)
- [x] **The knowledge-soundness composition** (the port of ArkLib pull request 615): on `main`, #58.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#wha
  ↳ t-the-spine-fixes)
- [x] **Tables and stacking** (Layer 1): on `main`, #59.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#lay
  ↳ er-1-hypercube-tables-stacking-the-index-and-bytecode-columns)
- [ ] **Clean expressions as polynomials** (Layer 2): claimed, #28; upstream as Clean pull request 466.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#lay
  ↳ er-2-clean-components-as-polynomials)
- [ ] **The adaptor** (Layer 3): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/doc
  ↳ s/roadmap/protocol-blueprint.md#layer-3-the-m3-instance-of-leanisa)
- [ ] **Sumcheck: definition and completeness** (Layer 4): claimed, #37; the honest round polynomials in #42.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#lay
  ↳ er-4-sumcheck-for-eq-weighted-virtual-polynomials)
- [ ] **Sumcheck: knowledge soundness** (Layer 4): claimed, #37.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#lay
  ↳ er-4-sumcheck-for-eq-weighted-virtual-polynomials)
```

```

- [ ] **Batching by powers** (Layer 4): claimed, #31; the batching algebra in #43.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-4-sumcheck-for-eq-weighted-virtual-polynomials)
- [ ] **Fingerprint and collision bound** (Layer 5): claimed, #33; the fingerprint polynomial in #39.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-5-fingerprints-the-grand-product-and-gkr)
- [ ] **Grand-product GKR: definition and completeness** (Layer 5): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-5-fingerprints-the-grand-product-and-gkr)
- [ ] **Grand-product GKR: knowledge soundness** (Layer 5): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-5-fingerprints-the-grand-product-and-gkr)
- [ ] **Bus phase: definition and completeness** (Layer 6): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-6-the-bus-phase)
- [ ] **Bus phase: knowledge soundness** (Layer 6): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-6-the-bus-phase)
- [ ] **Table sumcheck phase: definition and completeness** (Layer 7): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-7-the-table-sumcheck-phase)
- [ ] **Table sumcheck phase: knowledge soundness** (Layer 7): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-7-the-table-sumcheck-phase)
- [x] **Public-input phase** (Layer 8), both halves: on `main`, #60. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-8-the-public-input-phase)
- [ ] **Flock phase** (Layer 9): open; its inhabitant is #3's. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-9-the-flock-and-ring-switching-boundary)
- [ ] **Opening phase: definition and completeness** (Layer 10): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-10-the-claim-pool-the-opening-sumcheck-and-the-oracle-protocol)
- [ ] **Opening phase: knowledge soundness** (Layer 10): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-10-the-claim-pool-the-opening-sumcheck-and-the-oracle-protocol)
- [ ] **WHIR opening** (Layer 11): open; shared with #3.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-11-whir-over-binary-reedsolomon-codes-and-merkle-trees)
- [ ] **Merkle trees, the byte hasher and the WHIR parameters** (Layer 11): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-11-whir-over-binary-reedsolomon-codes-and-merkle-trees)
- [ ] **Transcript, proof object and compiled verifier** (Layer 12): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-12-compilation-the-transcript-the-proof-and-the-executable-verifier)
- [ ] **The base theorems** (Layer 13, target T4 of `docs/architecture.md`): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-13-t4-and-the-fixtures)

## To take a hole

With write access, edit its line above. Without, open an issue titled `[Intention]: protocol - <name of the hole>: ...` whose body names the declarations taken; a maintainer links it here. To report a problem with the blueprint, open `[Roadmap]: protocol - ...`.

```

The review's findings change the checklist in three places, each mirrored by a row of the table of appendix D.2 and following the assignment of the obligations no hole owns (finding 1): the one Flock line becomes two, its definition with completeness (which can start now and be tested against the Python verifier's walk) and its knowledge soundness (which stays the Flock roadmap's); the one public-input line becomes two, so that the ticked line stays true while the hole gains the deployed verifiers' combined check; and a line for the list-binding compilation goes between the Merkle line and the compiled verifier's. The opening lines keep their names while the specification they point to loses its sumcheck, and the other unowned obligations are folded into the specifications of existing lines and add none (from R60, R12, R137, R24, R18).

```

- [x] **Public-input phase: the specification's per-limb check** (Layer 8), both halves: on `main`, #60.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-8-the-public-input-phase)
- [ ] **Public-input phase: the deployed combined check** (Layer 8): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-8-the-public-input-phase)
- [ ] **Flock phase: definition and completeness** (Layer 9): open. [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-9-the-flock-and-ring-switching-boundary)
- [ ] **Flock phase: knowledge soundness** (Layer 9): open; its inhabitant is #3's.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-9-the-flock-and-ring-switching-boundary)

```

```
- [ ] List-binding compilation (Layer 12): open.
  ↳ [Specification](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#layer-12-compilation-the-transcript-the-proof-and-the-executable-verifier)
```

Two notes on the proposed body. The VCVio controls line of today’s checklist (:40) is not a hole; the controls landed upstream (VCVio pull request 784) and their two intention issues should close (appendix D.5). The line on Clean expressions as polynomials says “claimed” and not “in review”: its implementation is a Clean pull request, not a leanerVM one, and no leanerVM pull request carries the `awaiting-review` label today.

Where each section of the hole comment goes. The comment (`issue-12-comment-5833669972.md`, thirteen sections, last edited 2026-09-28) is the specification the blueprint points to (section 13.2.3). Each section goes to one layer section of the blueprint; most of its content is already there, some is only in the comment and must move, and some is obsolete and must not move.

Table D.1: Where each section of the hole comment goes.

Section of the comment	Goes to	Already in the blueprint	Only in the comment: move	Obsolete: do not move
the spine (:5-26)	The spine	everything, as built (:410-554); acceptance tests 24 to 27	nothing	the pre-build signatures (Pool, Seam.pool, busSpec... openSpec, Sumcheck.Def, Gkr.Def, Batch.Def, KnowledgeAppend as an assumption, knowledgeSound_of_refinement), which its own note at :9 disowns; “settle them on this issue” (:24)
grand-product GKR, both halves (:28-42)	Layer 5	ProductTree, gkr, gkrError with its three terms, the schedule (convention <i>GKR</i> , :321; :940-942), the tests gkr 1 2, gkr 3 3 and the changed leaf	the test “the message count for odd μ (acceptance test 17)” (:38); “upstream as Proof System/GKR/GrandProduct” (:40)	LeanerVM/Protocol/Generic/Gkr.lean (:32; Layer 5 names ToArkLib/GrandProduct.lean); gkr ... : Gkr.Def and Gkr.Security.complete, .rbr
the adaptor (:44-59)	Layer 3	all of it: $d := 2$, the three public lines, the two readers, satisfiedBy_witness0f (hs ...), the strong auxiliary predicate, the witness generator (:833-861)	nothing	“consumes ... #38, #40” (closed pull requests, now Layer 1 on main)
bus phase, both halves (:61-73)	Layer 6	the leaf tables, busError, leaf_decomposition, one root and $R_c \neq 0$ (:955-977); the zerocheck charge (convention <i>Seams</i> , :333); the test on two blocks	the Layer 1 lemmas the leaf decomposition needs, stack_eval_ambient_one, idxColumn_eval, bytecodeColumn_eval (:66); the tests “the whole phase run on the toy instance’s honest q” (the blueprint says “on the Layer 3 one-row witness”, which a phase over an abstract instance cannot use before the adaptor exists), the zero-count mutation (acceptance test 3) and the pad-0 mutation (test 2) (:71)	busPhase I (G : Gkr.Def) : Phase.Def I Unit (BusOut I) (busSpec I) (M3Holds I) (Seam.bus I) (:66); “specification (5.4)” (it is §5.4, equation (2)); “#40”, “#41”
table sumcheck phase, both halves (:75-87)	Layer 7	tableSummand, tableSummand_target, the errors, the output relation (:987-1003); the summand’s powers, padding and variable order (convention <i>Table sumcheck</i> , :323)	the test “a row violating a constraint” (:85) in place of the blueprint’s “violating a JUMP identity” (:1004), which the toy instance cannot express; the test “the reversed variable order mismatches eq (test 16)”	tableSumcheck I (S : Sumcheck.Def) : Phase.Def I (BusOut I) (TableOut I) (tableSpec I) ... (:80); “#42’s honest round algebra” (a pull request as a dependency)
public-input phase (:89-100)	Layer 8	all of it, as built and as decided (decision 15)	nothing	the whole section: “the prover sends nothing” (:93-94) is the design the public-input phase replaced
Flock phase (:102-113)	Layer 9	the strong predicate, limbColumns, the strided reader (:1088-1099); ArkLib pull request 383 and issue 893 (ledger row on ring-switching packing)	nothing	nothing beyond the claim line
opening phase, both halves (:115-127)	Layer 10	Weight, WeightedClaim (the spine’s), the pool order (convention <i>Claim pool order</i> , :324), the errors ($J - 1/ E $ and $2/ E $ per round	the tests “the phase run on the toy instance; a claim value changed after λ is drawn rejected; the empty pool” (:125; Layer 10’s tests are about the whole protocol only); that the ($J - 1/ E $ term is to be stated in ArkLib’s probability form (:121)	Claim.toWeighted over “a BlockClaim of #38” (:120; the Layer 1 review showed a phase cannot form one, and the step is ColumnClaim.holds_iff_weighted); openingPhase I (B : Batch.Def) (S : Sumcheck.Def) : Phase.Def I (FlockOut I) Unit (openSpec I) ...; Pool (:123)

Section of the comment	Goes to	Already in the blueprint	Only in the comment: move	Obsolete: do not move
knowledge-soundness composition (:129-142)	The spine, item 5	all of it (:423-430, :1136-1144)	“its two witness lemmas reuse the pinned ArkLib’s proofs, and the wrappers into the existential and averaged forms are left out” (:138-139): a fact about the module, for its docstring, not the blueprint	“The hole closed with the merge of #58” (history)
WHIR opening (:144-156)	Layer 11	novelBasis, encode, encode_column_weight, whirOpen, its two theorems, McaJohnson, the joint list binding (acceptance test 11), the tests at $\kappa = 3$ and at toy parameters	the test “every pinned coding parameter with an achievability witness (... a list-decoding pin can be false at the production rate)” (:152)	LeanerVM/Protocol/Generic/Whir.lean (Layer 11 names ToArkLib/Whir.lean); the upstream watch “ArkLib #4 (Merkle umbrella)”, closed, and “ArkLib #907 slices ... landing daily”, closed (:154)
Merkle trees, byte hasher, WHIR parameters (:158-170)	Layer 11	the six parameters, ladder, ladder_queries_eq, blake2sBytes, merkleRoot, merkleVerify, the tests on the four-leaf tree and against verifier.py:910	the test “blake2sBytes on the RFC 7693 test vector” (:166); that the Merkle module is generic (the blueprint names no module for merkleRoot, merkleVerify)	LeanerVM/Protocol/Generic/Merkle.lean; “ArkLib #4 (Merkle trees, definition and security)” (:168), closed in favour of VCVio issue 571
transcript, proof, compiled verifier (:172-184)	Layer 12	FsState and its four operations, the seed (convention <i>Fiat-Shamir</i> , :325), Proof, RoundPoly, decode, leanVmIopp, verify, settleFixedClaims, verify_iff_compiled, the two interfaces, verify_knowledgeSound, the tests on the dumped proof and six mutations, the code-generation probe (:1230-1231)	“leanVM’s chain is Merkle–Damgård, not a duplex sponge, so the transfer theorem, not the sponge, is what applies” (:182), which fixes what FiatShamirSecurity is an instance of	“the spine’s six ProtocolSpecs” (:178; the spine fixes one)
the base theorems (:186-196)	Layer 13	both statements, with the same program hypotheses (section 13.2.6); the test on the fixture	nothing (pioError_le at the caps, :194, is already Layer 10’s test, :1148)	“verify_knowledgeSound ◦ knowledgeSound_of_refinement” (:190; the spine has Refinement.map_option_valid)

Twelve of the thirteen sections end with a “Claim” line, in one of three forms (section 13.2.3, item 9); none moves: the way to take a hole is the blueprint’s “How work is tracked” (appendix D.1). “Upstream watch” rows that are still open move without their state into each layer section’s “upstream work to read first” (section 13.5.1.2).

The layer sections receive the statement types the review fixes: the bus phase from `Seam.commit` to `Seam.bus`, with a `BusOut I` in the shape of the Rust’s `BusVerify`; the table sumcheck from `Seam.bus` to `Seam.table`; the Flock phase from `Seam.pub` to `Seam.flock`, handing the limb claims on and adding the ring-switched claim; the opening phase from `Seam.flock` to `Seam.done`, whose schedule is the batching challenge λ and one weighted query, with no sumcheck. The base theorems’ hypotheses change: both take `WellFormedBytecode prog`; soundness becomes a statement about a Q -query adversary against the random-oracle verifier, with a straight-line extractor and the error `niError Q`; completeness starts from a satisfying witness with admissible sizes and a valid rate, and its form from an execution takes a fit hypothesis. The adaptor’s theorems change their hypotheses in two places: admissibility gains the rate window and the stacking window, and the Flock witness generator and the lemma “the R1CS holds $\Rightarrow$ the limb slots compress” come from one instance-free structure supplied by the Flock roadmap (from R2, R14, R18, R27, R28, R29, R33, R15).

The comment itself. Once its content is in the blueprint, its author edits it, in place, to one paragraph, so that the links to it still land somewhere (an edit, not a new comment):

```
The specifications of the holes are in the blueprint, each in its layer section: see
[The holes](https://github.com/Verified-zkEVM/leanerVM/blob/main/docs/roadmap/protocol-blueprint.md#the-holes).
This comment held them from 2026-09-25 until that move; its earlier text is in the edit history.
```

The four other comments on issue 12 (a contributor’s three of 2026-09-20 to 2026-09-24 and the owner’s of 2026-09-28) are a record and need nothing: their live content (four claims) is in the bodies of intention issues 28, 31, 33 and 37.

D.5 The other issues and pull requests, as far as the proof system’s documentation goes

The owner’s rule applies to each: a change is an edit of the existing body (or description), not a comment and not a new issue. The states are those of 2026-09-30.

Issue 3 (the Flock roadmap). Its body does not mention the proof system. What the proof system asks of Flock (the strong auxiliary predicate, the strided reader of the limb slots, the witness generator, the compression lemma) is in the owner’s comment of 2026-09-28 (issuecomment-5872015596) and already in the blueprint, Layers 3 and 9 (protocol-blueprint.md:850-861, 1088-1099). Proposed: one line added to the body’s “Related trackers”:

```
leanerVM #12 (the leanVM proof system): it consumes this roadmap as its Flock phase and takes the witness
↪ generator and the compression lemma in its adaptor; the interface is specified in
↪ `docs/roadmap/protocol-blueprint.md`, Layers 3 and 9.
```

Title: nothing. On the proof system’s side, the blueprint cites this issue’s milestones by their codes (“#3’s F3–F5 and F8”, :1097; “#3 F6”, :612; “F1 there”, status :363), which collide with the status’s findings of the same letter; the blueprint should use the names issue 3 already gives them (“the concrete PCS (#3, F6”).

Issue 4 (the leanISA tracker). Its “Related” line says “#12 (the proof-system roadmap, which proves `SatisfiedBy`; its requests #13 are settled in the roadmap)”. The proof system proves knowledge of its own relation on the committed stack, and reaches `SatisfiedBy` only through the adaptor, which is not built. Proposed, in the body:

```
#12 (the proof-system roadmap: its adaptor, not yet built, turns a stack satisfying the proof system's relation
↪ into a witness satisfying `SatisfiedBy`; its two requests, #13, are settled)
```

The same line’s “decision 15” should read “leanISA decision 15”, the proof system having a decision 15 of its own. Title: nothing.

Issue 16 (Clean’s bus argument is vacuous over K). Nothing: its body is about leanISA and Clean, and the blueprint cites it correctly (:359).

Issue 20 (Clean bus balance over binary fields). Nothing: its line on #12, “the proof-system roadmap reads the M3 bus off the same channels”, is right.

Issue 23 (fold three hypotheses into fixed columns after Clean pull request 446). Two edits of the body. Its “Blocked on” says the Clean bump “also has to survive P3, the eager Fintype BF64, see the status file”: a bare code that is the table sumcheck’s definition on issue 12; it should read “leanISA status finding P3 (the eager Fintype BF64)”, or name it only. Its “Where it is recorded” should add the proof system: the upstream ledger’s entry on fixed columns and sound prover data (today the status’s row : 184), consumed by the adaptor, which states the three facts as known coordinates of the leanISA instance (Coord.known). Title: nothing.

The eager-Fintype clause of “Blocked on” is deleted: the new CompPoly pin 572f9973 contains CompPoly pull request 331, which retired it. Whether the new Clean pin 42fe4b26, Clean pull request 474 merged over Clean’s main, contains Clean pull request 446 is unverified: 446 was an open draft when the review read it, so “Blocked on” keeps it until someone checks the pin’s history (from R125; the state of 446 from the library review, section 3.3.6).

Issue 28 (intention: Layer 2). Its first line, “Tracks #12, Layer 2/C1”, uses the code of the ledger entry on Clean, which is also the code of the knowledge-soundness composition on issue 12’s checklist. Proposed:

| Tracks #12: the hole ‘Clean expressions as polynomials’ (Layer 2), whose upstream is Clean pull request 466.

Title: nothing (it could carry the hole’s name: “[Intention]: protocol - Clean expressions as polynomials: ...”).

Issues 29 and 30 (intention: the VCVio controls). Their bodies already record that the controls landed through VCVio pull request 784; the blueprint’s rule is that an intention issue is closed by the pull request that lands it (:1490-1491). Proposed: close both, with no comment; remove their line from issue 12 (appendix D.4). Titles: nothing.

Issue 31 (intention: batching by powers). Three phrases of the body are stale or coded: “P05 depends only on #18’s multilinear API. The game depends on P05” (a private code for the algebra slice, and a closed pull request): “The algebra depends only on LeanerVM/Protocol/ToCompPoly/Multilinear.lean on main. The game depends on the algebra”; “tracked by [ArkLib #3]”, which is closed: “ArkLib #1”; “ledger A6”: “the ledger entry ‘grand product, GKR, batching and stacking are absent upstream’”. Title: nothing.

Issues 27, 32, 35, 36 (intentions: Layer 1). Closed with pull request 59. Nothing.

Issue 33 (intention: fingerprint and collision bound). “Base: PR #18 at 41b79b3c...” and “Depends on the corrected fingerprint slice over PR #18”: pull request 18 is closed and its content is on main (pull request 59). Proposed: “Base: main, using LeanerVM/Protocol/ToCompPoly/Multilinear.lean” and “over main”. Title: nothing.

Issue 34. A merged pull request on the semantics, not an intention issue. Nothing.

Issue 37 (intention: sumcheck). Nothing: it names ArkLib #1 (and says it supersedes the closed ArkLib #3) and “Tracking: #12”.

Pull request 39 (symbolic fingerprints), for the fingerprint and collision bound. Base feat/leanth-reuse, the branch of the closed #18; the module is LeanerVM/Protocol/Generic/Fingerprint.lean. The description says “Built on #18 for #33”. The owner’s comment of 2026-09-28 says where it meets the spine (the fingerprint enters the weights of VirtualTerms; the collision bound is the bus phase’s knowledge half). Proposed: the base becomes main on rebase and the module moves to the folder the convention *Generic code* assigns (its objects are CompPoly tables and Mathlib polynomials); the description is edited to name the hole (“fingerprint and collision bound”, Layer 5), the intention issue (#33), ArkLib issue 901, the specification section (§5.2) at the pin, that it is written from the specification, the ledger entry (the grand product absent upstream), and to absorb the owner’s comment, which then needs no successor. Title: nothing.

Pull request 42 (honest sumcheck polynomials), for the sumcheck’s definition and completeness. Base main; modules LeanerVM/Protocol/Generic/HonestSumcheck.lean and HonestSumcheck

Upstream.lean. Proposed: the modules move to the destination folder; the description, which names #37 and ArkLib #1, gains the hole’s name (“sumcheck: definition and completeness”, Layer 4) and the owner’s comment of 2026-09-28 (the spine has no `Sumcheck.Def`; a generic component is a `Component.Def` with its `Complete` and `Security`). Title: nothing.

Pull request 43 (power batching), for batching by powers. Base feat/leanth-reuse; module `LeanerVM/Protocol/Generic/PowerBatching.lean`; description “Built on #18”. Proposed: rebase onto `main`, move the module, and edit the description to name the hole (“batching by powers”, Layer 4), #31, and the owner’s comment of 2026-09-28: `pairing_batchWeight` is stated over the same inner product as the spine’s `Weight.pair`, and one of the two is to be restated through the other so that two pairings do not coexist. Title: nothing.

Both fit, so both descriptions promise a consumer that exists. The fingerprint of pull request 39 enters the weights of the bus forms that the fixed `BusOut I` carries per side and sumcheck table, in the shape of the Rust’s `BusVerify`, and its collision bound stays the bus phase’s knowledge half; the power batching of pull request 43 is the opening phase’s batching by λ , which the review keeps when it removes the opening’s sumcheck, so its consumer is the opening phase, whose one weighted query WHIR realizes (from R2, R18).

Appendix E

The brief, reviewed

The brief (`.claude/prompts/blueprint-spine-review.md`) asked, as its first step, for a critical reading of itself. This appendix records that reading, the assumptions the review adopted where the brief left a choice open, what happened during the session that the brief did not foresee, and what a future run of the same brief would gain from a few changes.

E.1 What the brief settles well

The brief fixes the object (the blueprint and the code built from it), the three criteria in priority order, the revisions of every source, the rule that the pinned tex is the specification and the PDF is not to be cited, and the limits of the session (no edit to a tracked file, nothing posted, no checkout moved). It separates the pin of a dependency from its upstream `main`, and it names the phases, the master theorems and the vocabulary the report should use. All of this was followed as written.

E.2 Choices the brief leaves open, and the choice made

Which source is “deployed”. The brief treats the specification, the Rust and the Python verifier together as the ground truth, asks that a disagreement between them be recorded, and asks of every theorem whether it describes “the deployed protocol or a repaired one”. It does not say which source *is* the deployed protocol. The review takes the Rust verifier at the pin to be the deployed protocol and the tex to be the specification (section 1.3). The distinction mattered at once: the public-input check differs between them, and the phase built on `main` follows the specification.

Which source the Lean should follow when they disagree. The brief does not say. The repository’s convention (“Category A” content is written from the specification and then diffed against the code) and the owner’s instruction that built the public-input phase (“as the specification writes it”) favour the specification; the review’s finding is that this produced a theorem about a verifier nobody runs. The report records each such disagreement, and recommends the rule of section 11.2: model the deployed verifier’s checks, take the arguments from the specification, report the difference upstream.

VCVio. The brief lists local checkouts of ArkLib, CompPoly and Clean beside the repository; none was listed for VCVio (one exists, at the new pin). Its pinned sources under `.lake/packages/VCVio/` were read; its upstream `main` from a shallow clone made under the review’s own directory.

The build. The brief expects probes that build. The main checkout’s build was stale (it predated Layer 1 and the public-input phase), so the review rebuilt `main` once at the start (3280 jobs, exit 0). The owner later asked that no further expensive build be run on this machine, which is memory limited; when the upgrade of `main` then made every probe impossible, the owner authorised one more build (3452 jobs, exit 0), and the twenty-two probes that carry a conclusion were re-run at the new pins.

Where the report lives. The brief says anywhere convenient, never committed. Mid-session the owner asked for the work to be placed on a separate branch and, on being asked, for the report to be committed there. The review branch is `docs/protocol-blueprint-review`, created from `main` at `b435631`; no tracked file of `main` is changed on it, and it is not pushed.

Secondary repositories. The comparable verification efforts were consulted through the web only (chapter 11), since nothing in them needed to be built or run.

Models. The owner asked, after the first wave, that simpler tasks run on a cheaper model. The ground truth, the spine and public-input audits, the boundary, ArkLib’s adequacy, the obligations tree and the hostile re-derivation ran on the review’s own model; the Layer 1 audit, the other libraries with Layer 0, the citation checks, the LaTeX transcriptions, the register, the probe re-runs, the documentation inventory and the literature survey ran on Opus.

E.3 What happened that the brief did not foresee

- A usage limit stopped eleven of twelve agents an hour into the first wave. The three that had written their dossiers incrementally lost nothing; the others lost their conclusions and were resumed from their transcripts. The rule “write the dossier incrementally” was added to the shared brief and held from then on.
- The owner upgraded `main` to Lean 4.34.1 with new library pins and merged it into the review branch while eleven agents were reading. The proof-system statements did not change; the library sources under `.lake/packages/` did, so every library citation now names its revision and old-pin files are read from git; a numeral in $\mathbb{K}$ changed meaning, and the probes that carry a conclusion were re-run.
- The leanVM checkout moved off the pin during the same upgrade (a pull and a branch checkout by the owner). The pin is an ancestor of the new head, so every leanVM file was read at the pin through git from then on, and the citation checks re-read the earlier dossiers’ lines that way.
- Two agents forked helpers of their own; the owner stopped two of those, and the literature agent was told to do its survey itself.

E.4 Scope the brief does not name, which the review covered

- **The setup.** The announced sizes, their canonical encoding, the caps and the stacking window are checks of the deployed verifier that precede every phase; the brief’s list of phases omits them, and the check-by-check enumeration of chapter 14 includes them.
- **Ring switching,** a step of the Flock phase with its own challenges and error.
- **The generic components** (the sumcheck, the batching, the GKR), which three phases are built from and where two of the major findings sit.
- **The error accounting,** where the review found the spine’s error to be the phases’ own declaration, and **the shape of the non-interactive theorems,** which the brief does not ask about and which the review found not well formed.

E.5 What a future run of the brief would gain

1. Say which source is the deployed protocol and which source the Lean follows when the sources disagree, and what is then owed.
2. Say whether probes may copy repository modules; the mutation probes of the public-input phase could only be built as copies of the whole module, its lemmas being private.
3. Name the setup and the generic components among the things to review.
4. Give the expected form of the catalogue: a catalogue of every declaration with its snippet is several hundred pages, and the review kept full snippets for the load-bearing and interface declarations and one line for the rest.
5. State the budget: how many agents may run at once and on which model. The first wave exhausted the session’s allowance in under an hour.
6. Freeze the checkouts for the session’s duration, or say what to do when they move.

Appendix F

The register of findings

Every finding of the review’s dossiers, merged where two dossiers state the same one, numbered and ordered by severity, with where each is stated in full, its classification and the status of its evidence; then the negative results (what was checked and found right) and what was not verified. The register was assembled from the dossiers by a script and is reproduced from `dossiers/register.md` on the review branch.

Corrections after assembly. Four rows were weakened or corrected after the register was assembled and are reproduced here as assembled: **R7** (the blueprint’s GKR round error is a valid, loose bound; the wrong parts are the wire, the layer check and the missing normalized variant), **R35** (the strided reader is assigned to Layer 3, not to no layer), **R23** (the level-0 list size runs to 648 over all 56 sizes, not to 396) and **R28** (the random oracle must also stand for the multi-block leaf hash, not only for the compression); see section 8.2 and section 8.7.

The register holds 138 findings (1 critical, 37 major, 63 minor, 37 notes) and 2 positions, 115 negative results and 58 unverified items. It is an index and a summary, never the only record: every row points to the dossier section that states it in full, and the dossier’s text governs where the two differ.

Numbering. The rows are numbered by severity and, within a severity, by subject. **R1** to **R136** were fixed when the register was first published. Rows added later (**R137** onward, from the dossier on the obligations) are numbered after them and shown at the end of their severity group, so that no earlier number changes; within a group, the order by subject therefore holds for the earlier rows only.

Revisions. The object of the review is leanerVM main at `b435631`; leanVM at the pin `a386121f`. Library citations name their revision: old pins ArkLib `dca90385`, CompPoly `3468b38c`, Clean `93c9d1ef`, VCVio `f9dc47d9` (those of `b435631`); new pins after `144c5aa` ArkLib `7653a901`, CompPoly `572f9973`, Clean `42fe4b26`, VCVio `a4232d08`. Blueprint line numbers are at `b435631` (they are +4 at HEAD after line 51). Lean probe outputs are at the old pins unless a row says otherwise.

Status column. *cited-checked*: a citation check (the dossiers `verify-gt-*`) covered the row’s citations; corrections it made are applied and marked “corrected by the check”. *probe-run*: a Lean or Python probe that ran supports it (at the old pins unless stated). *paper*: the argument is a reading or a proof on paper only. *unverified*: the dossier says a part is unverified. Several tags may apply; the text in parentheses says which part.

Merging. A finding stated by two or more dossiers is one row that cites every dossier and shows each dossier’s severity; the row sits at the highest. Findings that are merely related are separate rows. Where dossiers disagree, each position is a row (appendix F.4.1). Two rows are not findings but positions of one dossier that another dossier’s finding contradicts; they carry no severity and sit beside the row they dispute.

F.1 The register of findings

Columns: the row's number; its short name; its severity as the dossiers give it (merged rows show each dossier's); its subject; where it is stated in full (dossier and section or finding heading, with the finding box of this report where one states that dossier section); its classification as the dossier gives it (— for a finding about the code, the status, a source, or where the dossier gives none); the status of its evidence. The claim, the decisive evidence and the proposed change of every critical and major row follow the first table (appendix F.1.2); for the minor rows and the notes they are in the dossier section named. Evidence is at the pins named: Lean and blueprint lines at **b435631**, leanVM at **a386121f**.

F.1.1 The critical and major rows

Table F.1: The critical and major rows of the register, in the register’s order.

R#	Short name	Severity	Subject	Where stated	Classification	Status
Critical (1 finding)						
R1	The existential forms of round-by-round knowledge soundness carry no knowledge content	critical (as a warning about ArkLib’s definition; not a defect of a leanerVM theorem)	master theorems and error	lib-arklib §G.1 (finding 118); §D.4, §D.6	a deviation forced by an upstream library (ArkLib has no notion of running time)	probe-run (old pin)
Major (37 findings, 2 positions)						
R2	The bus seam admits statements the deployed table sumcheck cannot serve	major (gt-table-pub, code-spine)	relation and seams	gt-table-pub §8 (first finding) (finding 22), §6 E.1; code-spine §F “The degree conjunct of Seam.bus” (finding 92), §C.3	an error of the blueprint	cited-checked; probe-run
R3	The real bus phase can fill the spine’s bus slot; the seam’s surplus generality burdens only the table phase	— (a position of gt-bus, not a finding; placed beside the row it disputes)	relation and seams	gt-bus Summary item 2, §E.1, §E.2, §E.4	—	cited-checked; paper
R4	The spine’s completeness cannot carry an error, and Flock may need one	major	relation and seams	lib-arklib §G.2 (finding 119); §E.3	an error of the blueprint, conditional: “if the honest Flock prover fails at $r_{eq} = 1 \dots$ otherwise a note”	paper
R5	The declared error is unconstrained: the knowledge theorem has content only with a bound on <code>piopError</code>	major	master theorems and error	code-spine §F “The declared error is unconstrained” (finding 89); §C.5, §D.3 (b)	an error of the blueprint (an omission the spine inherited)	probe-run
R6	The composition over the prover’s announced sizes is unspecified	major	phase: setup	boundary-adaptor §G.6 (finding 76); §B.3	an error of the blueprint (a missing decision)	paper; unverified (the count 2^{34} is a hand estimate)
R7	The GKR layer sumcheck is the normalized variant; the blueprint gives it the wrong degree, the wrong error and no definition	major	phase: bus	gt-bus G1 (finding 5)	an error of the blueprint	cited-checked; probe-run (Python)
R8	The GKR last-layer rounds cost $5/ E $ each, as the blueprint says	— (a premise of gt-table-pub, not a finding; placed beside the row it disputes)	phase: bus	gt-table-pub §5 D.3 (“On the rounds of the last layer $\epsilon_G = 5/ E $ ”); verify-gt-table-pub §4 item 5	—	cited-checked; paper

continued on the next page

Table F.1, continued

R#	Short name	Severity	Subject	Where stated	Classification	Status
R9	One combiner more than GKR layers: the last is drawn and never used	major (by the brief's definition; the fix is one sentence)	phase: bus	gt-bus G2 (finding 6)	an error of the blueprint	cited-checked; probe-run (Python)
R10	Layer 5's generic GKR cannot be appended in the bus phase and cannot carry the zerocheck clause	major	phase: bus	gt-bus G3 (finding 7)	an error of the blueprint; the first half also a deviation forced by an upstream library	cited-checked; paper
R11	The table sumcheck's tables are not distinguished from the instance's other tables	major (gt-table-pub, boundary-adaptor); note (gt-bus)	phase: table sumcheck	gt-table-pub §8 (second finding) (finding 23), §4 C.1 item 4; boundary-adaptor §G.9 (finding 79), §E.4; gt-bus G16 (finding 20)	an error of the blueprint (an omission; boundary-adaptor: a consequence of the modelling choice, unstated)	cited-checked (gt-table-pub, gt-bus parts); unverified (a reading of Layer 3's text: <code>leanIsaInstance</code> is not built)
R12	The public-input theorems are about a verifier none of the three executable verifiers runs; the debt is assigned to Layer 12, which cannot pay it	major (gt-table-pub, code-pubinput)	phase: public input	gt-table-pub §8 (third finding) (finding 24), §7; code-pubinput §G.2 (finding 99), §B.1, §C.13	gt-table-pub: a deliberate deviation (decision 15; status finding F18). code-pubinput: a disagreement between the specification and its three implementations (not an unsoundness); the blueprint follows the specification deliberately and owes the deployed verifier's theorem in the wrong place and shape	cited-checked; probe-run (key lemma); unverified (the deployed phase's full <code>Phase.Security</code> , on paper)
R13	The check on the public-input message is not load-bearing for any theorem of the phase	major	phase: public input	code-pubinput §G.1 (finding 98); §C.3, §C.4, §C.5	an error of the design, introduced by an audit-surface compression (the blueprint's Layer 8 text describes a check tying the pool to the statement; the code's does not)	probe-run
R14	Layer 9's Flock interface consumes the eighteen limb claims and emits one claim; leanVM's Flock reads no claim	major (gt-flock-ring); minor (gt-table-pub)	phase: Flock and ring switching	gt-flock-ring §8.1 (finding 35), §2.4, §5.1; gt-table-pub §8 "Layer 9's sketch" (finding 28), §6 E.2	an error of the blueprint (the spine's Flock slot is faithful; Layer 9 and the tracker's Flock section are not)	cited-checked; independent check agrees

continued on the next page

Table F.1, continued

R#	Short name	Severity	Subject	Where stated	Classification	Status
R15	The instance depends on a Flock interface that depends on the instance (the limb slot map, aux, the compression lemma)	major (gt-flock-ring, boundary-adaptor; “cannot be stated as written”)	phase: Flock and ring switching	gt-flock-ring §8.2 (finding 36); boundary-adaptor §G.7 (finding 77)	an error of the blueprint	cited-checked (gt-flock-ring part); paper
R16	The Flock phase cannot be written over an abstract instance, which gives it only an opaque predicate	major (cannot be stated as written)	phase: Flock and ring switching	gt-flock-ring §8.3 (finding 37); §1 item 7; probe 9.1	an error of the blueprint (a design gap the spine left open)	cited-checked; probe-run (not re-run by the check)
R17	Acceptance test 20 files a completeness failure under a soundness error, and attributes to the protocol an inverse only the Rust prover takes	major	phase: Flock and ring switching	gt-flock-ring §8.4 (finding 38); §4	an error of the blueprint	cited-checked; probe-run (Python)
R18	The opening phase runs a sumcheck leanVM does not run	major	phase: opening and compilation	gt-opening-compile §I.1 (finding 51); §A	an error of the blueprint, caused by the evaluation interface of Layer 0	probe-run (the proposed interface typechecks); paper (the transcript comparison)
R19	Layer 11’s <code>whirOpen</code> batches a second time, and its <code>encode</code> serves level 0 only	major	phase: opening and compilation	gt-opening-compile §I.8 (finding 58); §B.6, §A.4	an error of the blueprint	paper
R20	No ArkLib theorem can inhabit the planned Fiat–Shamir and BCS interfaces, and the interface as commented is false	major (lib-arklib, gt-opening-compile)	phase: opening and compilation	lib-arklib §G.4 (first half; finding 121), §F.1; gt-opening-compile §I.4 (finding 54), §E.2	an error of the blueprint; the missing upstream theorems are a deviation forced by ArkLib, whose workaround is to name the literature and the construction	paper (that the constant-oracle statement is false in general is lib-arklib’s inference, not machine-checked)
R21	<code>verify_iff_compiled</code> names definitions that do not describe the compiled verifier	major (lib-arklib, gt-opening-compile)	phase: opening and compilation	lib-arklib §G.4 (second half; finding 121); gt-opening-compile §I.2 (finding 52), §E.4	an error of the blueprint; the absence of a BCS transform in ArkLib is a deviation forced upstream	probe-run; paper
R22	Upstream Fiat–Shamir and BCS theorems will be about other constructions than leanVM’s one-map BLAKE2s chain	major (the interface cannot be discharged by “the upstream theorem”)	phase: opening and compilation	literature §E.6 (first finding)	a deviation forced by an upstream library, to be recorded with its workaround	paper; unverified (whether a cross-role input is exploitable; whether a later ArkLib provides a chain-based transform)

continued on the next page

Table F.1, continued

R#	Short name	Severity	Subject	Where stated	Classification	Status
R23	The non-interactive error omits grinding, the list-size factor and the hash term, so it cannot carry leanVM's 128-bit claim	major (literature: grinding, list factor; gt-opening-compile); minor (literature: hash term); note (gt-flock-ring)	phase: opening and compilation	literature §A.4 (first, second and third findings), §A.3; gt-opening-compile §I.7 (finding 57), §F.2-F.3; gt-flock-ring §8.13 (finding 47)	an error of the blueprint (and, for the list factor, a gap in the specification, whose §8.4 is T000)	cited-checked (gt-flock-ring part); paper; probe-run (Python scratch of the parameters)
R24	The list-binding compilation is a theorem the blueprint names and does not state	major	phase: opening and compilation	gt-opening-compile §I.5 (finding 55); §E.3, §G	an error of the blueprint (omission)	paper
R25	The proof of work is the one verifier check no planned theorem or mutation makes load-bearing	major	phase: opening and compilation	literature §G.5 (first finding); §G.3, §G.4	an error of the blueprint	paper
R26	Perfect completeness does not survive the Fiat–Shamir transform: the honest prover's grinding is a search	major	phase: opening and compilation	gt-opening-compile §I.6 (finding 56); §E.5	an error of the blueprint	paper
R27	The base soundness (and completeness) theorems lack <code>Well FormedBytecode</code>	major (docs-debt, boundary-adaptor; a statement cannot be proved as written)	adaptor and leanISA boundary	docs-debt §H.1 (finding 137), §B.6; boundary-adaptor §G.3 (first part; finding 73), §C	an error of the blueprint, against the leanISA roadmap (the program condition is faithful to leanVM, whose compiler guarantees it)	paper (the sentinel counterexample is a proved repository test)
R28	The base soundness theorem is not a probability statement about <code>verify</code> , and concludes no extracted witness	major (boundary-adaptor, gt-opening-compile)	adaptor and leanISA boundary	boundary-adaptor §G.3 (second part; finding 73), §C, §E.6; gt-opening-compile §I.3 (finding 53), §E.1	an error of the blueprint (and of docs/architecture. <code>md</code> 's T4, which it transcribes)	paper; probe-run (Transport)
R29	The base completeness theorem is false without resource hypotheses and needs a witness generator	major	adaptor and leanISA boundary	boundary-adaptor §G.4 (finding 74); §C	an error of the blueprint	paper
R30	The instance is not <code>Ensemble.toM3</code> of the eight tables	major	adaptor and leanISA boundary	boundary-adaptor §G.1 (finding 71); §A.5	an error of the blueprint	probe-run
R31	No bridge lemma relates the boundary blocks to the Clean memory, bytecode and verifier rows	major	adaptor and leanISA boundary	boundary-adaptor §G.2 (finding 72); §A.2, §A.5	an error of the blueprint (omission)	paper
R32	The polynomial bridge of Layer 2 is noncomputable at the pin	major	adaptor and leanISA boundary	boundary-adaptor §G.8 (finding 78); §E.5	a deviation forced by an upstream library, with a workaround; retired if <code>CompPoly</code> makes <code>toCMvPolynomial</code> computable	probe-run

continued on the next page

Table F.1, continued

R#	Short name	Severity	Subject	Where stated	Classification	Status
R33	Layer 3's sizes and admissibility statements cannot be stated or hold as written (<code>Sizes</code> , <code>Sizes.ofWitness</code> , <code>Sizes.Admissible</code> , <code>admissible_iff_caps</code> , <code>leanIsaInstance_fits</code>)	major (gt-bus, boundary-adaptor, code-layer1; code-layer1: “by the scale’s letter”, the fix is one line); minor (gt-opening-compile §I.15)	adaptor and leanISA boundary	gt-bus G4 (finding 8); boundary-adaptor §G.5 (finding 75); code-layer1 §G.2 (finding 105); gt-opening-compile §I.15 (first half; finding 65), §F.1	an error of the blueprint	cited-checked (gt-bus part); paper
R34	The blueprint plans on a framework ArkLib calls legacy, and on a pull request that may not merge	major	libraries	lib-arklib §G.3 (finding 120); §F.2, §C.3	an error of the blueprint’s plan	paper; probe-run (<code>PlainReading</code> , old pin)
R35	The strided reader of the eighteen BLAKE2s limb columns exists nowhere and is assigned to no layer	major	Layer 1	code-layer1 §G.1 (finding 104); §B.8	an error of the blueprint (omission)	probe-run (the lemma; run outside lake, old binary); unverified (the reader and combinator are sketched)
R36	The per-layer sketches (Layers 4 to 7, 9, 10) and the tracker use types the spine does not have	major (docs-debt); minor (gt-table-pub, gt-bus, code-spine)	documentation	docs-debt §H.3 (finding 139), §B.4; gt-table-pub §8 “Layer 7’s sketch” (finding 27), §4 C.2; gt-bus G11 (finding 15); code-spine §E.2	an error of the blueprint	cited-checked (gt-table-pub, gt-bus parts); paper
R37	The blueprint hands the holes’ specification to a tracker comment that contradicts it; its public-input section says “the prover sends nothing”	major (docs-debt); minor (gt-table-pub)	documentation	docs-debt §H.2 (finding 138), §B.3; gt-table-pub §8 “The tracker’s text for the public-input phase is stale” (finding 30)	an error of the blueprint (its delegated specification of the public-input phase differs from §8.2; the built phase is faithful)	cited-checked (gt-table-pub part); paper
R38	The status’s finding that the Python verifier omits the caps is false at the pin (F9)	major , of the status (gt-bus); minor (gt-table-pub, gt-flock-ring, gt-opening-compile)	documentation	gt-bus G5 (finding 9), §F; gt-table-pub §8 “Two status findings are wrong” (finding 29); gt-flock-ring §8.10 (finding 44); gt-opening-compile §I.16 (finding 66)	an error of the status	cited-checked
R137	Twenty-one obligations of the composition have no hole, no layer and no owner	major	master theorems and error	obligations §5.1; §4.2, §2	an error of the blueprint (the plan is incomplete, not wrong where it is stated)	paper (the tree rests on the sibling dossiers’ recorded probes and on reading)
R138	The compilation needs every phase before the opening to make no oracle query, and nothing states or enforces it	major	phase: opening and compilation	obligations §5.2; node 2.1.2.3	an error of the blueprint (an omission in the design of the seams)	paper

F.1.2 Claim, evidence and proposed change of the critical and major rows

For each critical and major row, in the register’s order: the claim, the decisive evidence and the proposed change, as the register gives them. The full text, with the passages as they stand and as proposed, is in the dossier section the row names (table F.1).

R1. The existential forms of round-by-round knowledge soundness carry no knowledge content (critical)

Claim. ArkLib’s round-by-round extractor is any function, so an extractor that picks a witness by classical choice makes the verifier deciding any relation’s language knowledge sound at error 0. Theorems in the existential forms (`piop_rbrKnowledgeSoundness_exists`, the sketches of Layers 9 and 10) are soundness statements only; the spine’s named form is adequate only with a compiled extractor.

Evidence. ArkLib Security/RoundByRound.lean:77-86, 118-123 at dca90385; probe Extractors (chooser_sound, exit 0, old pin)

Proposed change. state every interface in the named (With) form with a compiled extractor checked by a def without noncomputable; change :239, :1084, :1333 (lib-arklib.md §G.1)

R2. The bus seam admits statements the deployed table sumcheck cannot serve (major)

Claim. Seam.bus accepts any number of linear claims, terms of one table at unrelated points, terms on any table, and a degree bound written as a clause on the statement. Round-by-round knowledge soundness quantifies over every input statement, so the deployed table-sumcheck verifier cannot be proved complete and sound against it without guards (degree, claim count, one point per table) that leanVM does not have; over the seam the error on ξ is unbounded.

Evidence. Spine/Seams.lean:130-134, 175-181 at b435631; probes SeamBusShape.lean, SeamBusMember.lean (exit 0 per the dossier)

Proposed change. give BusOut the shape of the Rust’s BusVerify (leaf.rs:849-860): one point, forms per side and sumcheck table with the degree in the type, three totals, column claims; smallest fix: make VirtualTerm.poly a subtype within the degree bound (gt-table-pub.md §8; code-spine.md §F)

R3. The real bus phase can fill the spine’s bus slot; the seam’s surplus generality burdens only the table phase (position)

Claim. The deployed bus phase’s output is expressible as a BusOut I (five linear and five column claims for leanVM), it is perfectly complete (given the side conditions of R43), and a knowledge state function exists. Seam.bus admits statements the bus phase never emits, but gt-bus calls that “a burden on the table phase’s completeness”, met by “a table phase written over the terms’ own points”, and finds nothing required by the spine that matters.

Evidence. gt-bus.md §E.1-E.4 (paper)

Proposed change. none proposed beyond the side conditions and the order of the linear claims

R4. The spine’s completeness cannot carry an error, and Flock may need one (major)

Claim. Component.Complete is perfect completeness, while ArkLib composes completeness with additive errors (proved at both pins); acceptance test 20 charges the Flock prover’s failure at `r_eq = 1` to flockError, a soundness error.

Evidence. ToArkLib/Component.lean:84-85; blueprint :1083, :1323-1325; crates/flock/src/zerocheck.rs:117; ArkLib GuardedCompleteness.lean:161-173

Proposed change. add err to Component.Complete, compose by append_completeness_of_guarded_verifiers, state piop_completeness with the sum and piop_perfectCompleteness as its corollary (lib-arklib.md §G.2)

R5. The declared error is unconstrained: the knowledge theorem has content only with a bound on `piopError` (major)

Claim. `piopError` is whatever each phase declares; five phases that draw a challenge, check nothing and declare error 1 inhabit `Phases.Security` for the toy, and both master theorems hold of them. The blueprint’s only bound, `piopError_le`, takes an argument (`s`) that the spine’s `piopError` does not.

Evidence. `Spine/Compose.lean:149`; `ToArkLib/Component.lean:66, 148`; blueprint :318, :1130; probe `P4Junk.lean` (`piopError_junk`, old pins)

Proposed change. the spine fixes each phase’s error as a function `piopError I` of the instance’s sizes and demands each phase’s bound at it (recommended), or a bound field in `Phases.Security` (code-spine.md §F)

R6. The composition over the prover’s announced sizes is unspecified (major)

Claim. The sizes are the prover’s and admissibility is the verifier’s check, but the per-instance theorems bound one size vector each, `verify_iff_compiled` quantifies $\exists s$, and `ArkLib`’s fixed `ProtocolSpec` cannot express a schedule that depends on a first message. A union over the admissible sizes costs about 2^{34} ; the $Q \cdot \max \epsilon$ Fiat–Shamir bound needs the sizes inside the hashed statement of one protocol.

Evidence. blueprint :316, :1218-1221, :1226-1227, :1326-1328; `cpu/mod.rs:130-178`; `verifier.py:1372-1379`

Proposed change. state the non-interactive theorem for the family (`niError Q` the maximum, or the sum, over admissible sizes) and `FiatShamirSecurity` in family form with the sizes in the challenge oracle’s input (boundary-adaptor.md §G.6)

R7. The GKR layer sumcheck is the normalized variant; the blueprint gives it the wrong degree, the wrong error and no definition (major)

Claim. Each deployed GKR round sends a degree-4 cofactor (four elements on the wire, the constant coefficient derived through the equality factor); the running claim is the cofactor’s value and the layer’s final check has no equality factor. The blueprint says degree 5 and error $5/|E|$, and Layer 4 defines only the plain variant, so a verifier built as written rejects every Rust proof or is a different verifier.

Evidence. `gkr.rs:399-404, 410-413`; `fiat_shamir/src/transcript.rs:297-302`; `verifier.py:411-413, 445, 449`; blueprint :321, :898-901, :941-942; Python probe H.1 (honest prover accepted by the pinned Python verifier; re-run by the check)

Proposed change. define both sumcheck variants in Layer 4 by name; $4/|E|$ per GKR round in `gkrError` and the tracker (gt-bus.md G1)

R8. The GKR last-layer rounds cost $5/|E|$ each, as the blueprint says (position)

Claim. `gt-table-pub`’s argument that the zerocheck escape adds nothing uses $\epsilon_G = 5/|E|$ per round of the last GKR layer; its check reads `next_round_poly(5, claim, Some(equality_point))` as five coefficients of a degree-4 cofactor with the equality factor applied by the verifier, “so the full round polynomial has degree 5, consistent with the dossier’s $5/|E|$ and with the blueprint’s row GKR”.

Evidence. `gkr.rs:399-401`

Proposed change. —

R9. One combiner more than GKR layers: the last is drawn and never used (major)

Claim. Both verifiers draw a combiner after the roots and after every layer, the last included; the blueprint says one per layer. Harmless in the oracle model, but after Fiat–Shamir the extra squeeze changes every later challenge.

Evidence. `gkr.rs:369, 391, 423`; `verifier.py:435, 453`; blueprint :321, :940-941; Python probe H.1 ($\mu^2/4 + \mu + 1$ challenges for even μ)

Proposed change. “a combiner after the roots and after every layer, the last included, error 0”; add the count to acceptance test 17 (gt-bus.md G2)

R10. Layer 5’s generic GKR cannot be appended in the bus phase and cannot carry the zerocheck clause (major)

Claim. The GKR’s oracles are the leaf tables, but a phase’s one oracle is the stack; carrying the GKR along that map needs ArkLib’s context lifting, whose completeness and knowledge soundness are admitted at the pin. And the appended state function cannot carry a clause (the zerocheck) that changes with the GKR’s own challenges.

Evidence. blueprint :262, :606, :930-937; Spine/Phase.lean:37-38; ToArkLib/Component.lean:143-149; ArkLib LiftContext/Reduction.lean:355-365, 542-560 (sorry, dca90385)

Proposed change. restate Layer 5’s GKR over a context with leaves and riders; the bus phase is $[(\alpha, \beta); \text{roots}] \gg \text{gkr} \gg [\text{boundary evaluations}]$ (gt-bus.md G3)

R11. The table sumcheck’s tables are not distinguished from the instance’s other tables (major)

Claim. Layer 3 makes the six shared columns tables of the instance, and Layer 7 takes $\tau_{\max}$ and the final message’s width over every table: at least 16 rounds and 110 values, where leanVM runs $\tau_{\max}$ rounds over the six opcode tables (as few as 3) and sends 104 values; `verify_iff_compiled` would then fail.

Evidence. blueprint :836-839, :987-989; constraints.rs:250; verifier.py:607; cpu/mod.rs:344-351; 08-end-to-end-protocol.tex:76-77

Proposed change. a decidable criterion (`I.SumcheckTable` in `gt-table-pub`, `M3Instance.active` in `boundary-adaptor`: a constraint, a flush or a count column) and Layer 7 ranges over it; a test with a taller table without constraints (gt-table-pub.md §8; boundary-adaptor.md §G.9)

R12. The public-input theorems are about a verifier none of the three executable verifiers runs; the debt is assigned to Layer 12, which cannot pay it (major)

Claim. The specification checks each limb; the Rust verifier, the Python verifier and the recursion guest check one combined equation $c_0 + y \cdot c_1 = (1+r) \cdot w_0 + r \cdot w_1$ and pool the scalars sent. That is a different oracle verifier, so Layer 12’s compilation theorem cannot hold with a Rust-faithful `verify` and cannot detect the difference with a per-limb one (its fixture is an honest proof).

Evidence. 08-end-to-end-protocol.tex:30-31; cpu/mod.rs:752-755; verifier.py:1400; aggregate.py:1681-1684; blueprint :1053-1062, :1218-1221; probe ProbeWordsLemma.lean (accepts_two_challenges, old pins)

Proposed change. Layer 8 builds a second phase for the deployed check (pooling the values sent, `Phase.Security` at $1/|E|$), consumed by the master theorems and Layer 12; keep the specification’s phase as reference; report the ambiguity of §8.2 to leanVM (gt-table-pub.md §8; code-pubinput.md §G.2)

R13. The check on the public-input message is not load-bearing for any theorem of the phase (major)

Claim. The verifier pools the values it computes from the statement, not the values received, so a verifier that ignores the message is perfectly complete and round-by-round knowledge sound at the same seams, extractor and error; proved for every message function and every check the message passes. The earlier review’s “each of five wrong verifiers breaks a stated theorem” is false of the merged code, and the status repeats it.

Evidence. PublicInput.lean:127-128, 350-366; docs/reviews/public-input-phase.md:104-114, 552-573; protocol-status.md:440; probes mutation 1 and ProbeAnyCheck (securityG, no sorryAx, old pins)

Proposed change. pool the values sent (`pooledFrom`), which makes the check load-bearing (its removal and weakening are then refuted, §C.6, §C.7); reword blueprint :1035-1037; strike status :440; add the refutations as tests (code-pubinput.md §G.1)

R14. Layer 9’s Flock interface consumes the eighteen limb claims and emits one claim; leanVM’s Flock reads no claim (major)

Claim. The deployed Flock verifier reads no pooled claim; the limb claims stay in the pool and are opened as strided claims on `q_flock`. A `FlockInterface` whose input is the eighteen claims and whose output is one weighted claim drops them, so with leanVM’s verifier its knowledge-soundness error is 1.

Evidence. `cpu/mod.rs:799-805` (the dossier cites `:790-814`; quotation abbreviated, corrected by the check);
blueprint `:1075-1092`

Proposed change. a `FlockPhase I R` at the spine’s seams (public-input seam to Flock seam) that hands the claims on and adds the ring-switched claim; the schedule written out (`gt-flock-ring.md §8.1`)

R15. The instance depends on a Flock interface that depends on the instance (the limb slot map, aux, the compression lemma) (major)

Claim. The instance’s layout takes its slot map from `FlockInterface.limbColumns`, but `FlockInterface` takes the instance as its parameter; the slot map is eighteen numbers and a generic identity. boundary-adaptor adds that the instance’s `aux` must be Flock’s R1CS, which only issue #3 defines, and that the lemma “R1CS implies the limbs compress” that `satisfiedBy_witnessOf` consumes has no carrier in any signature.

Evidence. blueprint `:846-848, :855-857, :1077`; `hash_flock.rs:87-115`

Proposed change. an instance-free structure supplied by issue #3 (`blake2sLimbSlot` in `Parameters/` in `gt-flock-ring`; `FlockSpec` with `slot`, `Holds`, `compress_of_holds`, `gen`, `holds_gen` in boundary-adaptor); delete `limbColumns` and `FlockWitnessGen` (`gt-flock-ring.md §8.2`; boundary-adaptor.md §G.7)

R16. The Flock phase cannot be written over an abstract instance, which gives it only an opaque predicate (major)

Claim. Layer 9 is to be written over an abstract instance behind the wall, but the instance’s auxiliary predicate names no column, no compression count and no relation (the sketch reaches for a leanISA size). On the toy, whose predicate is `True`, a Flock phase that checks nothing is knowledge sound at error zero: today a reader of the master theorems trusts everything about Flock.

Evidence. `Spine/Instance.lean:143-148`; blueprint `:331, :1072`; probe `NoCheckFlock.lean` (exit 0 per the dossier, old pins)

Proposed change. a `FlockRegion I` (the column, `kBatch`, its height, `aux ↔ Flock.Holds`) supplied by Layer 3 and taken by Layer 9 (`gt-flock-ring.md §8.3`)

R17. Acceptance test 20 files a completeness failure under a soundness error, and attributes to the protocol an inverse only the Rust prover takes (major)

Claim. No verifier inverts a challenge-dependent value. The pinned Rust Flock prover derives a coefficient with $(1 + r)^{-1}$ and at $r = 1$ emits a proof its verifier rejects: an implementation completeness defect (probability at most $(k_batch + 1)/|E|$), not part of `flockError`; the spine has no place for a completeness error.

Evidence. `crates/flock/src/zerocheck.rs:116-118`; `fiat_shamir/src/transcript.rs:296-302`;
blueprint `:1323-1325`; Python probe 9.2 (re-run by the check)

Proposed change. rewrite test 20 (“no exceptional-challenge clause, in any phase”; the Rust prover’s defect recorded in `docs/leanvm-target.md`); the same sentence drafted for issue #3 (`gt-flock-ring.md §8.4`)

R18. The opening phase runs a sumcheck leanVM does not run (major)

Claim. After the one batching challenge λ the deployed prover’s next message is the first round polynomial of WHIR’s own sumcheck on $\sum_x q(x) \cdot W_\lambda(x) = C_\lambda$, interleaved with WHIR’s commitments and closing on one equation; there is no reduction sumcheck and no evaluation of $\tilde{q}$ (specification §8.5: “there is no separate reduction sumcheck”). The blueprint’s Layer 10 (a μ -round sumcheck on $W_\lambda \cdot q$ and a final evaluation query) followed by WHIR in place of the evaluation oracle has μ extra rounds and an extra scalar, so `verify_iff_compiled` cannot hold for Rust proofs.

Evidence. `08-end-to-end-protocol.tex:94-101`; `pcs/src/stack_open.rs:452-461`; `pcs/src/whir.rs:1324-1334`; `verifier.py:1012, 1362`; blueprint `:1112-1114, :1214`; probe `InnerProductOracle.lean` (the inner-product oracle interface typechecks against `ArkLib`; old and new pins)

Proposed change. give the stack the inner-product oracle interface (a query is a `Weight  $\mu$` , the answer $\sum_w W(w) \cdot q(w)$, specification Definition 3.13); the opening phase is λ then one weighted query, and WHIR realizes that query (`gt-opening-compile.md §I.1, §A.5`)

R19. Layer 11’s `whirOpen` batches a second time, and its `encode` serves level 0 only (major)

Claim. WHIR’s level-0 batch is the opening phase’s λ , not a second batching; and `encode` must map E-valued messages of every level, with a lemma that a K-valued message gives a K-valued word (level 0 being that case).

Evidence. `b-polynomial-commitment-scheme.tex:108-110, 306`; `pcs/src/whir.rs:419-485`; blueprint :1169, :1173, :1185

Proposed change. `whirOpen`’s level-0 batch is λ ; `encode` $(\kappa R) : (\text{Fin } (2^\kappa) \rightarrow E) \rightarrow (\text{Fin } (2^{(\kappa+R)}) \rightarrow E)$ with the K-valued lemma (`gt-opening-compile.md §I.8`)

R20. No ArkLib theorem can inhabit the planned Fiat–Shamir and BCS interfaces, and the interface as commented is false (major)

Claim. At both ArkLib pins `fiatShamir_completeness` is admitted and stated for a constant challenge oracle, Fiat–Shamir soundness is a `TODD`, round-by-round to state-restoration is commented out, `BCSTransform` is commented out, and `Commitment.extractability` has body `False`; the blueprint’s “each an ArkLib theorem to come” is unfounded. `gt-opening-compile` adds that an interface quantified over all IOPPs with error $Q \cdot \max_i \epsilon_i$ is false (hash term, oracle messages, grinding), which would make the theorem vacuous, and that #627 is a design issue, not a pull request.

Evidence. ArkLib `FiatShamir/Basic.lean:163-175`, `Implications.lean:230-254, 305-313`, `BCS/Basic.lean`, `Commitments/Functional/Basic.lean:250-255` (both pins); ArkLib PR #848 `SingleSalt.lean:973-993`; blueprint :263, :614, :1223-1225

Proposed change. `state FiatShamirSecurity/BcsSecurity` as assumed interfaces whose obligation is the literature; `gt-opening-compile` proposes three: `RbrToStateRestoration`, `BcsSecurity` (with $3(Q^2+1)/2^{256}$), `ChainFiatShamirSecurity` (with proof-of-work rounds) (`lib-arklib.md §G.4`; `gt-opening-compile.md §I.4`)

R21. `verify_iff_compiled` names definitions that do not describe the compiled verifier (major)

Claim. The sketch applies ArkLib’s `Verifier.fiatShamir` to an oracle verifier whose messages are the stack and the codewords in the clear: it would hash whole oracle messages, not Merkle roots; no `decode : Proof → messages` exists; the Merkle compilation, the grinding checks and the prover-chosen sizes are omitted.

Evidence. ArkLib `Basic.lean:490-496`, `FiatShamir/Basic.lean:129-136`, `BCS/Basic.lean:59-63` (`dca90385`); `fiat_shamir/src/transcript.rs:9-12, 225-228`; `fiat_shamir/src/lib.rs:165-174`; blueprint :1218-1221, :1231-1232; probe `FiatShamirChain.lean` (what `Verifier.fiatShamir` takes; old and new pins)

Proposed change. a leanerVM `bcsCompile` (roots for oracle messages, authenticated rows for queries), the chain as a `QueryImpl`, `grindingChecks` conjoined (`gt-opening-compile.md §I.2, §E.4`)

R22. Upstream Fiat–Shamir and BCS theorems will be about other constructions than leanVM’s one-map BLAKE2s chain (major)

Claim. ArkLib’s transforms use one random oracle on the full prefix or a duplex sponge, and the textbook BCS analysis uses independent oracles per role; leanVM uses one 64-byte BLAKE2s map for the chain, the untagged Merkle nodes and grinding. A chain lemma and a role-separation lemma are owed here.

Evidence. blueprint :263, :1218-1221; `fiat_shamir/src/merkle.rs:44-51`; ArkLib README at 7653a901; [CY24] Construction 25.1.1; [CO25] §2.3

Proposed change. rewrite ledger row A5: obligations the upstream theorems do not discharge as stated; name the chain lemma and the role-separation lemma (`literature.md §E.6`)

R23. The non-interactive error omits grinding, the list-size factor and the hash term, so it cannot carry leanVM’s 128-bit claim (major)

Claim. leanVM’s query rounds reach 128 bits only with 17 bits of grinding (the query rounds alone give about 111 bits), but `FiatShamirSecurity`’s error $Q \cdot \max_i \epsilon_i$ and `niError` have no proof-of-work factor,

and no published theorem covers Fiat–Shamir with proof of work for multi-round IOPs; the commitment is list-binding only, so each oracle-protocol challenge costs $L_0 \cdot \epsilon_i$ after compilation (gt-opening-compile computes $L_0 \leq 110$ to 396 at level 0, up to 2^{20} at the deepest levels); and the hash-collision term ($c \cdot Q^2 / 2^{256}$) is missing. $\sum \text{piopError} < 2^{-150}$ says nothing about the deployed security level, which the WHIR query rounds set.

Evidence. blueprint :1224, :1234-1235, test 23; whir_config.rs:25-27, 36-38, 57-60, 537-568, 593-599; fiat_shamir/src/lib.rs:30-51, 126-178; leaf.rs:108-118; scratch whir_params.py (reproduces the Python query table); [CY24] Theorems 25.2.1, 31.3.1

Proposed change. $\text{niError } Q = (Q + \text{rounds}) \times$ the largest per-challenge error of the compiled protocol ($L_0 \cdot \epsilon_i$ at oracle-protocol challenges, Theorem B.7’s terms, $2^{-17} \cdot (1 - \gamma_i)^{t_i}$ at ground query rounds) + the hash term; FiatShamirSecurity with grinding bits, the proof-of-work factor marked an assumption (literature.md §A.4; gt-opening-compile.md §I.7)

R24. The list-binding compilation is a theorem the blueprint names and does not state (major)

Claim. The master theorems are about one committed q ; the deployed commitment binds a K -valued interleaved word that may be close to up to L_0 codewords over E . The compilation must prove that some list member satisfies every pooled claim (every member is K -valued and of the right size: a two-line argument not in the specification) with the oracle protocol’s errors multiplied by L_0 ; none of FiatShamir Security, BcsSecurity, McaJohnson mentions the list.

Evidence. b-polynomial-commitment-scheme.tex:4, 70-72, 133-137; whir_config.rs:593-599; blueprint :1190, :1296-1299

Proposed change. in Layer 12, the theorem of §E.3 (“List-binding compilation”) with its extractor (list-decode, select), state function, errors and two lemmas (gt-opening-compile.md §I.5)

R25. The proof of work is the one verifier check no planned theorem or mutation makes load-bearing (major)

Claim. leanVM’s 128-bit claim depends on grinding, yet the planned interfaces and Layer 12’s six mutations leave it outside every theorem and test, together with truncated Merkle paths, digest canonicity and sizes above their caps.

Evidence. blueprint :1224, :1238-1240; fiat_shamir/src/merkle.rs:22-37

Proposed change. restate the interface with grinding (R23); extend Layer 12’s mutations (a wrong grinding nonce at each level, a truncated path, a nonzero third limb, a size above its cap) and run transcripts the specification rejects through the Rust and Python verifiers (literature.md §G.5)

R26. Perfect completeness does not survive the Fiat–Shamir transform: the honest prover’s grinding is a search (major)

Claim. The blueprint says perfect completeness survives the transform “without any assumption”; the honest prover’s grind loops over nonces with no bound, and no theorem says a nonce exists for every reachable chain state, so the Lean `prove` must be partial or fuelled. On the Flock exceptional point, §E.5 agrees with gt-flock-ring that the protocol is perfectly complete and the exceptional point is the Rust prover’s.

Evidence. fiat_shamir/src/lib.rs:126-155; Spine/Compose.lean:94-104; blueprint :1255-1257

Proposed change. `prove` takes fuel and returns `Option Proof`; `baseProver_complete` is `prove ... = some proof → verify ... proof = true` (gt-opening-compile.md §I.6)

R27. The base soundness (and completeness) theorems lack WellFormedBytecode (major)

Claim. Layer 13’s two base theorems are said to compose `constraintSoundness` and `constraintCompleteness`, which require `WellFormedBytecode prog`; a bytecode with a JUMP in the sentinel slot has steps that balance and no valid execution, so the soundness theorem is false without it.

Evidence. blueprint :222, :1247-1255; leanisa-blueprint.md:1156-1166; architecture.md:224-228; leanvm-target.md:57-62; the proved test `tests/LeanerVMTests/Semantics/Execution.lean:415-461`

Proposed change. add (hwf : WellFormedBytecode prog) to both, or have verify decide the condition and reject (a technical decision for the orchestrator) (docs-debt.md §H.1; boundary-adaptor.md §G.3)

R28. The base soundness theorem is not a probability statement about verify, and concludes no extracted witness (major)

Claim. “Except with probability niError” is attached to verify prog input proof = true, a decidable hypothesis about fixed values: there is no randomness in it. The theorem must be about a Q-query adversary against the random-oracle verifier with a straight-line extractor, the instantiation of the oracle by BLAKE2s being the one non-formal step, named; and its conclusion $\exists t$, ValidExecution is soundness for a language, dropping the extracted witness.

Evidence. blueprint :326, :1247-1248; architecture.md:264-273; probe Transport (examples 3-4, old pins)

Proposed change. the random-oracle statement with an adversary, its query log and the extractor, bounded by niError Q, plus the existential corollary and the pointwise execution_of_extracted (boundary-adaptor.md §G.3; gt-opening-compile.md §I.3, §E.1); the same repair in architecture.md

R29. The base completeness theorem is false without resource hypotheses and needs a witness generator (major)

Claim. A valid execution may use $\kappa = 32$, whose four memory columns exceed the 2^{28} stack the verifier accepts, and AssignmentRepresents fixes κ , so no accepted proof of that witness exists; “witness of t” needs the witness generator (T2), which the blueprint excludes; constraintCompleteness takes Well FormedBytecode, not HasFillBlocks; the rate is not chosen by the execution.

Evidence. blueprint :147-148, :1249-1250, :1255-1257; Semantics/Execution.lean:108-110; Arithmetization/Statement.lean:358; cpu/mod.rs:174-176; pcs.rs:51; verifier.py:1379

Proposed change. baseProver_complete from a satisfying witness with admissible sizes and a valid rate, and baseProver_complete_of_execution with a fit hypothesis hfit; ask leanISA for a minimal-height constraintCompleteness (boundary-adaptor.md §G.4)

R30. The instance is not Ensemble.toM3 of the eight tables (major)

Claim. The memory and bytecode seed/finalize blocks and the state boundary must be boundary blocks whose index and program coordinates are Coord.known data of the instance (decision 8); with those blocks as tables of the instance the program would be committed, M3Holds would not imply the mixed witness’s balance, and no adaptor theorem could be proved. No Ensemble.toM3 : Ensemble → M3Instance can exist (an instance needs μ , layout, public lines, aux, Stmt).

Evidence. blueprint :21, :77, :803-805, :1369, :1398; Spine/Instance.lean:86-98, 136; cpu/layout.rs:352-395; leaf.rs:453; probe KnownColumn (exit 0, old pins; KnownColumnNew re-run at the new pins, passes)

Proposed change. leanIsaInstance as Component.toM3 of the six opcode tables, three column groups, six boundary blocks with Coord.known program columns, the deployed layout, three public lines, the Flock aux; delete Ensemble.toM3 (boundary-adaptor.md §G.1)

R31. No bridge lemma relates the boundary blocks to the Clean memory, bytecode and verifier rows (major)

Claim. SatisfiedBy’s balances range over all interactions, including the memory block’s, the bytecode block’s and the verifier’s rows, which in the instance are boundary tuples; Layer 2’s two bridges concern tables only, and nothing states that those rows’ messages are the boundary tuples. Without it neither direction of the adaptor can be proved.

Evidence. blueprint :775-777, :810-819; Clean FlatEnsemble.lean:225-226 (93c9d1ef); Arithmetization/Statement.lean:207-209

Proposed change. add boundary_tuples_eq to Layer 3 and a Layer 2 test on the one-row witness (boundary-adaptor.md §G.2)

R32. The polynomial bridge of Layer 2 is noncomputable at the pin (major)

Claim. Layer 2 produces Mathlib polynomials while the instance holds `CompPoly`'s; `CompPoly`'s conversion is noncomputable at both pins, which would make `M3Holds` of the `leanISA` instance undecidable by evaluation and `verify` noncomputable. A direct computable translation from Clean expressions takes twelve lines.

Evidence. blueprint :762-771, :866-867, :1230-1231; Spine/Instance.lean:123-125; `CompPoly MvPoly Equiv/Core.lean`:41 (3468b38c and 572f9973); probe `PolyBridge` (exit 0; `PolyBridgeNew` re-run at the new pins, passes)

Proposed change. `Expression.toCMvPolynomial` with `eval_toCMvPolynomial` and a Mathlib-side bridge for the degree (boundary-adaptor.md §G.8)

R33. Layer 3's sizes and admissibility statements cannot be stated or hold as written (Sizes, Sizes.ofWitness, Sizes.Admissible, admissible_iff_caps, leanISAInstance_fits) (major)

Claim. Both verifiers reject a stack outside [15, 28] and a rate outside [1, 4], but `leanISA`'s `Caps` has neither window, so either `verify_iff_compiled` and `pioError_le` are false or `admissible_iff_caps` is (it also has a free `w`); no witness determines the announced rate, so `Sizes.ofWitness` is ill-defined and lacks `prog`; `leanISAInstance_fits` uses a `total` that `Layout` has not, while the fit is an argument `Blocks.layout` needs before the instance exists.

Evidence. cpu/mod.rs:157-176; verifier.py:1377, 1379; blueprint :795-808, :1130, :1218-1220; `Arithmetization/Statement.lean`:79-81, 250-263; Spine/Instance.lean:73-81; `Stack.lean`:97

Proposed change. Sizes of the heights only; `leanISABlocks`, `leanISAμ`, `leanISABlocks_fits`; `Sizes.Admissible` with both windows; `validRate`; `caps_of_admissible`, `sizes_of_satisfiedBy`; tests for `μ_stack = 29`, `logInvRate = 5` (gt-bus.md G4; boundary-adaptor.md §G.5; code-layer1.md §G.2)

R34. The blueprint plans on a framework ArkLib calls legacy, and on a pull request that may not merge (major)

Claim. `ArkLib` calls `OracleReduction` legacy; its typed interaction framework (grown to 7653a901) has no knowledge soundness, no round-by-round notion and no extractor; #615 is open and conflicting; no relevant admission was lifted between the pins. The blueprint expects #615 to land and upstream to prove the round-by-round to plain implication.

Evidence. `ArkLib AGENTS.md`, docs/design/05-roadmap.md (7653a901); issue #676, PR #615 (read with gh); blueprint :257-267, :1143, :1429; status :201 (accurate)

Proposed change. keep the port until any framework proves a guarded-first append; state the plain corollary locally (probe `PlainReading`, 40 lines, plus a local union bound); a ledger row for the typed framework (lib-arklib.md §G.3)

R35. The strided reader of the eighteen BLAKE2s limb columns exists nowhere and is assigned to no layer (major)

Claim. `leanVM` reads the limb columns as strided slots of `q_flock` (the low eight coordinates frozen to the slot's bits). Layer 1 has only high-index selection, `Layout.comap` cannot place a limb column, and no layer sketches the selection lemma, the reader or the combinator; `leanISAInstance` cannot be built without them.

Evidence. pcs/src/stack_open.rs:84-97; verifier.py:884-894; hash_flock.rs:87-115; `ToCompPoly/Multilinear.lean`:50-51; blueprint :841-848; probe `StridedProbe.lean` (generic lemma, twenty lines)

Proposed change. add `sliceLow`, `evalMle_boolVec_append`, `Blocks.stridedLayout`, `Layout.piecewise` to Layer 1, with tests against the Python's `Placement.stack_point` (code-layer1.md §G.1)

R36. The per-layer sketches (Layers 4 to 7, 9, 10) and the tracker use types the spine does not have (major)

Claim. Layer 6's `BusOut` with challenges and totals, Layer 7's `BusOut × ColumnClaims`, Layer 9's `FlockInterface`, Layer 10's own `Weight`, `WeightedClaim`, `leanVmPiop` and master theorems over `Set.univ`

and `rbrKnowledgeSoundness`, and `Sumcheck.Def`, `Gkr.Def`, `Batch.Def` plus seven undefined names; the text beside them says the spine is authoritative. `gt-bus` adds that Layer 6’s invariant after (α, β) must be “the two products differ”, not “the multisets differ”; `gt-table-pub` that `tableSumcheck_relOut_implies_constraints` has no statement.

Evidence. `blueprint` :952, :954-979, :984, :988, :991-992, :1077-1085, :1104, :1109-1130 against `Spine/Seams.lean`:130-135, `Spine/Compose.lean`:162-177

Proposed change. restate Layers 6, 7, 9 and 10 on Layer 8’s pattern over the spine’s seams and Layers 4, 5 as `Component.Defs` in the named form; delete the pre-spine blocks (`docs-debt.md` §H.3; `code-spine.md` §E.2)

R37. The blueprint hands the holes’ specification to a tracker comment that contradicts it; its public-input section says “the prover sends nothing” (major)

Claim. The blueprint points to the hole comment three times; eight of its thirteen sections carry pre-spine signatures that do not typecheck against `Phase.Def`, and its public-input section says the prover sends nothing and the scalars are Layer 12’s encoding, where §8.2, the Rust and the code have the prover send two scalars.

Evidence. `blueprint` :339-340, :590-591, :1485-1487; hole comment :3, :93-94 (re-read with `gh`); `08-end-to-end-protocol.tex`:29; `cpu/mod.rs`:746-749; `PublicInput.lean`:99

Proposed change. move each section of the hole comment into the blueprint (`docs-debt.md` §G.4 map); the comment becomes a pointer (drafts only)

R38. The status’s finding that the Python verifier omits the caps is false at the pin (F9) (major)

Claim. The Python verifier’s `build_layout` checks $16 \leq \log_memory \leq 32$, every height ≤ 32 , $\tau_BLAKE2S \geq 3$ and a power-of-two bytecode; the Rust itself says the other two verifiers reject there. Nothing is to be reported upstream.

Evidence. `verifier.py`:856-864, called at :1378; `log2_strict` at :252-254; `cpu/mod.rs`:164-165; `status` :164, :316-318

Proposed change. delete F9 and `st`:164; correct the earlier reviews that cite it (`gt-bus.md` G5)

R137. Twenty-one obligations of the composition have no hole, no layer and no owner (major)

Claim. Twenty-one items that some arrow of the chain from the phases to `baseVerifier_extracts` `Execution` needs appear in no hole’s “Produces” column and no layer’s sketch (among them the Merkle compilation, the chain lemma, the state-restoration step, the grinding model, the hash and program-hash terms, the composition over the announced sizes, the list-binding compilation, the strided reader, the boundary-block bridge, the deployed public-input phase, the bound on `pioPError`). Filling the twenty-two planned units yields the two master theorems and a `verify` with no theorem tying them together.

Evidence. `blueprint` holes table :594-617 and layers :619-1259 against the tree of §2 (the closing list of §4.2)

Proposed change. add one hole per item or fold each into the nearest hole’s “Produces” column (K3, K1, a new list-binding hole, L1, I1, I2, P5, G1, S), as §5.1 assigns (`obligations.md` §5.1)

R138. The compilation needs every phase before the opening to make no oracle query, and nothing states or enforces it (major)

Claim. The list-binding compilation replaces the commit message by a codeword and keeps the front phases’ verifiers unchanged, which is possible only if they never query the stack. The two built phases do not, but `Phase.Def` lets a verifier query the stack at will (neither `ArkLib` nor the spine counts queries), so a bundle may satisfy both master theorems and be uncompileable; no sentence of the blueprint, the tracker or the spine states the requirement.

Evidence. `PublicInput.lean`:222-231; `ArkLib OracleVerifier.numQueries` (sorry); `code-spine` §D.3 (d); `gt-table-pub` §6 E.4 item 6

Proposed change. a field of `Phases` (or a hypothesis of the compilation theorem) that each front phase’s verifier makes no query, inhabited by the two built phases; or front phases typed as plain verifiers lifted by `keepOracles`; a sentence in the conventions row “The oracle” (`obligations.md` §5.2)

F.1.3 The minor rows and the notes

Table F.2: The minor rows and the notes of the register, in the register’s order.
Claim, evidence and proposed change are in the dossier section named.

R#	Short name	Severity	Subject	Where stated	Status
Minor (63 findings)					
R39	Knowledge soundness is bundled with perfect completeness	minor	relation and seams	gt-flock-ring §8.5 (finding 39)	cited-checked; paper
R40	The <code>outputPure</code> field is redundant, and <code>guarded</code> nearly so	minor (code-spine); note (lib-arklib)	relation and seams	code-spine §F “The <code>outputPure</code> field is redundant” (finding 93), §D.7; lib-arklib §G.7 (finding 130)	probe-run
R41	<code>Layout.read</code> is determined by <code>extend</code>	minor	relation and seams	code-spine §F “read is determined by <code>extend</code> ” (finding 95); §C.2	paper (the lemma <code>read_eq_of_extend</code> was not run)
R42	The error bundled in <code>Component.Def</code> makes every phase, the composed verifier and the extractor noncomputable	minor (auditability)	relation and seams	lib-arklib §G.5 (finding 122)	probe-run
R43	Two side conditions a bus phase over an abstract instance needs	minor	relation and seams	gt-bus G10 (finding 14); probe H.2	cited-checked; probe-run (not re-run by the check)
R44	<code>Protocol/Basic.lean</code> imports the arithmetization and is not a listed exception of the wall	minor (code-spine, code-layer1, docs-debt)	relation and seams	code-spine §E.3 row 25; code-layer1 §G.8 (finding 111), §D.1; docs-debt §H.8 (finding 144), §B.5 item 12	paper (git grep)
R45	The blueprint’s headline describes master theorems that do not exist on <code>main</code>	minor	master theorems and error	code-spine §F “The headline describes theorems that do not exist” (finding 90); §C.4	paper (reading)
R46	The extracted stack is not a definition; acceptance test 24’s <code>rfl</code> witness covers zero-round phases only	minor	master theorems and error	code-spine §F “The extracted stack is not a definition” (finding 91); §D.5	probe-run
R47	The zerocheck escape is over-charged and charged three or four different ways	minor (gt-table-pub, gt-bus)	master theorems and error	gt-table-pub §8 “The zerocheck escape” (finding 26), §5 D.1, D.3; gt-bus G6 (finding 10), §D.3	cited-checked; paper
R48	“Computable” is not “efficient” for the compiled extractor	minor (for T4 as stated) / note	master theorems and error	literature §A.4 (fourth finding)	paper
R49	Two deployed checks are discharged by type in Lean, and the blueprint does not say who parses	minor	phase: setup	boundary-adaptor §G.15 (finding 85); §B.2 item 2	paper
R50	The order of the leaf stacks is nowhere stated, and the spine’s <code>tuples</code> lists them in the opposite order	minor	phase: bus	gt-bus G7 (finding 11)	cited-checked
R51	Orders the blueprint leaves open: the two roots, the boundary evaluations, the linear claims, the point	minor	phase: bus	gt-bus G8 (finding 12)	cited-checked
R52	The binary GKR layer’s combination challenge has no error assigned	minor	phase: bus	gt-bus G12 (finding 16)	cited-checked

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R53	Sumcheck round messages: four coefficients and a check in the oracle protocol, three and none on the wire; the owed lemma is unstated	minor	phase: table sumcheck	gt-table-pub §8 “The round message” (finding 25)	cited-checked; paper
R54	The top limb is enforced only through the instance’s third line and the unbuilt adaptor theorem; acceptance test 10 names the wrong witness	minor (code-pubinput); boundary-adaptor agrees in its §D (a conclusion, no severity)	phase: public input	code-pubinput §G.3 (finding 100), §D.1, §D.2; boundary-adaptor §D, Summary item 3	paper; probe-run (the load-bearing inequality); unverified (the two-line display and the counterexample’s stack were not built)
R55	PublicLine.sent is pinned by prose only	minor	phase: public input	code-pubinput §G.4 (finding 101); §E	paper
R56	The auxiliary predicate must include the constant-one position	minor (imprecise wording on a load-bearing definition)	phase: Flock and ring switching	gt-flock-ring §8.6 (finding 40)	cited-checked; independent check agrees
R57	Flock is listed as written from the specification; its circuit, layout and constants exist only in the Rust and the Python	minor	phase: Flock and ring switching	gt-flock-ring §8.7 (finding 41)	cited-checked
R58	The Flock error quoted in acceptance test 23 leaves out ring switching	minor (gt-flock-ring, gt-opening-compile)	phase: Flock and ring switching	gt-flock-ring §8.8 (finding 42); gt-opening-compile §I.15 (second half; finding 65), §F.1	cited-checked (gt-flock-ring part); paper
R59	The ledger row on ring switching suggests a missing profile; ArkLib’s construction is another protocol	minor	phase: Flock and ring switching	gt-flock-ring §8.9 (finding 43); §6	cited-checked; independent check agrees
R60	The executable verifier needs the Flock phase’s definition, which no roadmap has started	minor (planning)	phase: Flock and ring switching	gt-flock-ring §8.11 (finding 45)	cited-checked
R61	The canonical-encoding and stream-consumption checks are absent from the blueprint, and Layer 12’s mutations exercise too little	minor (gt-bus, gt-opening-compile)	phase: opening and compilation	gt-bus G9 (finding 13); gt-opening-compile §I.17 (finding 67), §C.4, §D	cited-checked (gt-bus part; its search wording was overstated, the substance holds); paper
R62	encode_column_weight is an existence statement every linear map satisfies	minor (non-vacuity)	phase: opening and compilation	gt-opening-compile §I.9 (finding 59); §B.6	paper
R63	The proof object’s Merkle format is unspecified between the pruned wire form and the raw form	minor	phase: opening and compilation	gt-opening-compile §I.10 (finding 60); §C.2, §B.4, §D.4	paper
R64	Four transcript facts of WHIR are not in the blueprint	minor (Category B)	phase: opening and compilation	gt-opening-compile §I.11 (finding 61); §B.1	paper
R65	Level lacks WHIR’s analysis parameter; the OOD count and grinding bits are parameters	minor	phase: opening and compilation	gt-opening-compile §I.12 (finding 62); §B.3	paper
R66	The chain’s step is the 64-byte hash, not the compression with the chain state as chaining value	minor (Category B)	phase: opening and compilation	gt-opening-compile §I.14 (finding 64); §C.1	paper

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R67	The blueprint does not state the scope of its random-oracle assumption	minor	phase: opening and compilation	literature §B.4	paper
R68	The Lean theorem is classical; the quantum random-oracle model is not mentioned	minor	phase: opening and compilation	literature §G.5 (second finding); §G.1	paper
R69	witnessOf_stackOf has no Lean statement	minor	adaptor and leanISA boundary	boundary-adaptor §G.10 (finding 80); §E.2	paper
R70	witnessOf has no public input, and the count columns have no derivation	minor (imprecision)	adaptor and leanISA boundary	boundary-adaptor §G.11 (finding 81); §A.4 rows 1, 4	paper
R71	leanISAInstance must be reducible	minor	adaptor and leanISA boundary	boundary-adaptor §G.12 (finding 82)	probe-run
R72	Refinement.map_option_valid has the wrong polarity for ArkLib's knowledge game	minor (harmless)	adaptor and leanISA boundary	boundary-adaptor §G.13 (finding 83); §E.6	probe-run
R73	The relation ladder conflates Clean's field sum with Clean's balance relation	minor	adaptor and leanISA boundary	lib-others §G.1 (finding 125); §C.5	probe-run
R74	SampleableType K is unused audit surface, and the blueprint disagrees with itself about it (Layer 0)	minor	libraries	lib-others §G.2 (finding 126); §E.1	probe-run
R75	evalOracle_answer takes n explicitly, unlike the blueprint (Layer 0)	minor (cosmetic)	libraries	lib-others §G.3 (finding 127)	paper (reading)
R76	The scalar oracle interfaces are trusted surface the blueprint does not list (Layer 0)	minor	libraries	lib-others §G.4 (finding 128)	paper (reading)
R77	The upstream ledger misses the Merkle trees of the pinned VCvio and cites a closed issue	minor (possibly avoidable audit surface)	libraries	docs-debt §H.15 (finding 151); §B.9	paper; unverified (fit with leanVM's Merkle trees; state at a4232d08)
R78	The order of equal-size blocks in the witness stack is pinned by no statement or test before the compiled verifier	minor	Layer 1	code-layer1 §G.3 (finding 106)	probe-run
R79	Acceptance test 14 names a tautology as its witness; no test compares the bytecode column with leanVM's encoder	minor (code-layer1, docs-debt)	Layer 1	code-layer1 §G.4 (finding 107), §C.3; docs-debt §H.14 (one item; finding 150)	probe-run (old pins; the Rust encoder was transcribed, not executed)
R80	Acceptance test 15 overclaims that stack_eval fails for any other placement	minor	Layer 1	code-layer1 §G.5 (finding 108)	probe-run (outside lake, old binary)
R81	BlockClaims.lean has no consumer	minor (audit surface)	Layer 1	code-layer1 §G.7 (finding 110); §A.8, §E.1	paper (git grep)
R82	Acceptance test 7 names the wrong lemma and a witness that does not exist	minor	Layer 1	code-layer1 §G.9 (finding 112); §F.2	paper
R83	The status and the tracker describe a repository two merges old	minor (docs-debt, code-pubinput)	documentation	docs-debt §H.4 (finding 140); §B.1, §B.2, §B.11; code-pubinput §G.6 (finding 103)	paper (reading; gh read)

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R84	After the upgrade the blueprint states superseded pins as current, and Layer 0's prose is stale	minor (docs-debt; lib-others, at 144c5aa)	documentation	docs-debt §H.5 (finding 141), §B.11; lib-others §G.5 (finding 129)	paper (reading)
R85	Letter codes collide, and two references point at the wrong finding	minor	documentation	docs-debt §H.6 (finding 142); §D, §F	paper (count script)
R86	The same fact is written in three to six places	minor	documentation	docs-debt §H.7 (finding 143); §A	paper
R87	The review tooling the blueprint prescribes is not in the repository	minor	documentation	docs-debt §H.9 (finding 145)	paper
R88	“How work is tracked” is contradicted by practice	minor	documentation	docs-debt §H.10 (finding 146); §B.7	paper
R89	The blueprint records state and history, against its own rule	minor	documentation	docs-debt §H.11 (finding 147); §B.4 item 11	paper
R90	Decisions are numbered, but most of their content is gone and the numbers collide	minor	documentation	docs-debt §H.12 (finding 148); §B.8	paper
R91	The list of public names is not the public boundary: most public declarations are unlisted, some load-bearing, three mislabelled	minor (docs-debt, code-layer1); no severity given (code-spine §E.1)	documentation	docs-debt §H.13 (finding 149); code-layer1 §G.6 (finding 109), §F.3; code-spine §E.1 (last paragraph)	probe-run (Python)
R92	Broken names and citations in the blueprint, status and tracker	minor (gt-bus, docs-debt, boundary-adaptor, gt-opening-compile)	documentation	gt-bus G14 (finding 18); docs-debt §H.14 (finding 150), §B.5, §B.10; boundary-adaptor §G.14 (finding 84); gt-opening-compile §I.13 (finding 63)	cited-checked (gt-bus part); paper
R93	Source discrepancies are not recorded where both status files say they are	minor	documentation	docs-debt §H.16 (finding 152); §B.7 item 6	paper
R94	The status's round-message finding says the cubic is sent “at four nodes”; the wire carries coefficients	minor	documentation	gt-table-pub §8 “Two status findings are wrong” (first bullet; finding 29)	cited-checked
R95	The status's finding on the one bus root misreads the specification (S11)	minor	documentation	gt-bus G13 (finding 17)	cited-checked
R96	Names and line references in the blueprint's ArkLib table are wrong in places	minor	documentation	lib-arklib §G.6 (finding 123); §B.2	probe-run
R97	The status's dates for ArkLib #615 and #818 are last-update dates, not opening dates	minor	documentation	lib-arklib §F.5 (last paragraph; finding 124)	paper (gh read)
R98	Misattributed and mistitled references in the blueprint	minor	documentation	literature §A.4 (fifth finding)	paper (sources downloaded)
R99	The blueprint's citation for grand products points to the wrong Thaler chapter	minor	documentation	literature §D.3	paper
R100	The obligation outline behind architecture.md's labels is not attributed	minor	documentation	literature §E.6 (second finding)	paper

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R139	The per-challenge error has no carrier for the grinding bits	minor	phase: opening and compilation	obligations §5.3; node 2.1.2.1.3	paper
Note (37 findings)					
R101	The spine’s slot does not pin the protocol: a phase that reads the whole oracle fills it	note (gt-table-pub); a conclusion (code-spine)	relation and seams	gt-table-pub §8 Notes (finding 33), §6 E.4 item 6; code-spine §0 item 2, §D.3 (d)	cited-checked (gt-table-pub part); paper; unverified (the probe of code-spine §D.3 (d) was not run)
R102	The instance is data the theorems trust; only the toy’s relation is known inhabited	note	relation and seams	code-spine §F “The instance is data the theorems trust” (finding 94); §C.1, §C.2, §D.4	probe-run (emptiness: guard plus paper)
R103	The protocol’s knowledge soundness is non-adaptive in the statement	note	master theorems and error	boundary-adaptor §G.17 (finding 87); §B.3	paper
R104	The upper per-log caps are implied by the stacking window	note	phase: setup	gt-opening-compile §I Notes (first bullet) (finding 68); §D.1; verify-gt-bus §4 item 2	paper
R105	Seven disagreements between specification, Rust and Python in the setup, commitment and bus; none between the two verifiers	note	phase: setup	gt-bus G15 (finding 19); §A.6	cited-checked
R106	The Fiat–Shamir seed binds a constant that cannot be recomputed at the pin	note	phase: setup	gt-flock-ring §8.14 (finding 48)	cited-checked
R107	The specification gives no round-by-round analysis of the bus phase	note	phase: bus	gt-bus G17 (finding 21); §D.1	cited-checked; paper
R108	The specification’s batching error is loose by one, gives no error for the recycled point, and does not say which coefficient is dropped	note	phase: table sumcheck	gt-table-pub §8 Notes (finding 31); §5 D.1, D.2	cited-checked; paper
R109	Stale comments in the Rust on the round message	note	phase: table sumcheck	gt-table-pub §8 Notes (finding 32), §2 A.4; verify-gt-table-pub §4 item 3; verify-gt-bus §4 item 5	cited-checked
R110	The public-input message type is wider than the transcript	note	phase: public input	code-pubinput §G.5 (finding 102)	paper
R111	The pinned Rust prover of Flock is not perfectly complete	note	phase: Flock and ring switching	gt-flock-ring §8.12 (finding 46); §4.1	cited-checked; probe-run (Python)
R112	Stale comments in the Rust on what binds the counter and the flags	note	phase: Flock and ring switching	gt-flock-ring §8.15 (finding 49); verify-gt-flock-ring §4 item 8	cited-checked
R113	A weighted claim carries the 2^μ cube values of its weight	note	phase: opening and compilation	gt-flock-ring §8.16 (finding 50)	cited-checked
R114	The one coding-theory input is a preprint theorem with a sketched proof	note	phase: opening and compilation	literature §C.4 (first finding)	paper

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R115	Three readings of the constant in Theorem 4.6; leanVM and ArkLib use the printed one	note	phase: opening and compilation	literature §C.4 (second finding); §C.3 item 4	paper
R116	A stale comment in the pinned Rust on level-0 grinding	note	phase: opening and compilation	literature §G.1 (bullet “Grinding”)	paper
R117	The Rust’s own security accounting unions only the algebraic checks over the Johnson list	note (for docs/leanvm-target.md)	phase: opening and compilation	gt-opening-compile §I Notes (second bullet) (finding 69); §F.2	paper
R118	A stale comment in the Rust on the size of a WHIR Merkle leaf	note (for docs/leanvm-target.md)	phase: opening and compilation	gt-opening-compile §I Notes (third bullet) (finding 70)	paper
R119	Knowledge transport along the adaptor is three lines in the event form	note	adaptor and leanISA boundary	code-spine §F “Knowledge transport” (finding 96); §D.7	probe-run
R120	The completeness base theorem composes leanISA’s existence theorem, where <code>architecture.md</code> says witness generation	note (docs-debt); part of a conclusion (boundary-adaptor §F)	adaptor and leanISA boundary	docs-debt §H.19 (finding 155); boundary-adaptor §F, Summary item 6	paper
R121	The base extraction theorem (T4) drops the extracted witness that recursion (T6) needs	note	adaptor and leanISA boundary	boundary-adaptor §G.16 (finding 86); §F	paper
R122	<code>constraintCompleteness</code> needs a resource hypothesis of its own	note (leanISA’s, out of scope)	adaptor and leanISA boundary	boundary-adaptor §G.18 (finding 88); §C	paper (the distinct-states argument is not a Lean proof)
R123	The zkVM verification literature is almost entirely constraint-level	note	adaptor and leanISA boundary	literature §F.3	paper
R124	Locally re-implemented objects that ArkLib has (<code>passThrough</code> , <code>sendOracle</code>)	note	libraries	lib-arklib §G.8 (finding 131)	paper
R125	The eager <code>Fintype BF64</code> blocked every executable at the old pins; retired at 144c5aa	note (retired)	libraries	lib-others §G.6 (finding 132); §B.3, §F.3	paper
R126	<code>simp</code> misread <code>K</code> arithmetic at the old pins; the kernel rejected the result	note (retired)	libraries	lib-others §G.7 (finding 133); §H.7	probe-run (old pins)
R127	Numerals in <code>K</code> changed meaning at the new pins	note	libraries	lib-others §G.8 (finding 134); §F.3	paper (scan)
R128	ArkLib’s multilinear bridge is likely unimportable at the new pins	note (unverified)	libraries	lib-others §G.9 (finding 135); §F.2	unverified
R129	The local counting bound is redundant with <code>VCVio</code> at the new pin	note	libraries	lib-others §G.10 (finding 136); §F.1	paper (reading)
R130	The Layer 0 module’s comments cite the roadmap, layer numbers and a letter code	note	libraries	docs-debt §H.17 (finding 153); §B.5 item 13	paper
R131	<code>Layout.comap</code> accepts a non-injective renaming, so a layout can alias columns	note	Layer 1	code-layer1 §G.10 (first bullet) (finding 113); §C.4	probe-run
R132	Unused or duplicated Layer 1 declarations	note	Layer 1	code-layer1 §G.10 (second bullet) (finding 114); §E proposals 2-5	probe-run (outside lake)

continued on the next page

Table F.2, continued

R#	Short name	Severity	Subject	Where stated	Status
R133	Disagreements between leanVM's sources on the witness stack	note	Layer 1	code-layer1 §G.10 (fourth bullet) (finding 116); §B.2, §B.6	paper
R134	Two dispositions of the earlier spine review are met only in part	note	documentation	code-spine §F “Two statements the earlier review's disposition” (finding 97); §0 item 3, §D.3 (a)	probe-run
R135	A review's tracker edits were never applied, while its handoff says “met”	note	documentation	docs-debt §H.18 (finding 154)	paper
R136	Blueprint line numbers shift by four at HEAD; the upgrade changed more than the pins table	note (an observation of the citation check)	documentation	verify-gt-table-pub §4 item 1	cited-checked
R140	The smallest set of unbuilt nodes that makes the master theorems about leanVM	note (a reading aid)	master theorems and error	obligations §5.4; §2.2, §2.3, §2.6	paper

F.2 Negative results

What a dossier says it checked and found right, with how and any caveat, grouped by dossier in the order the dossiers were processed. The absence of a finding on these points is itself evidence.

Table F.3: Negative results of the review’s dossiers.

#	Dossier	What was checked and found right	How	Caveat
N1	gt-table-pub §0 item 1, §2	The table sumcheck’s transcript agrees across specification, Rust and Python: one challenge ξ , $\tau_{\max}$ rounds of three field elements, a final message of 104 values, one final check, a derived target.	reading; cited-checked	—
N2	gt-table-pub §4 C.1	The blueprint’s table-sumcheck conventions match the ground truth: variable order, joining round, padding (<code>padHigh</code> , <code>evalMle_padHigh</code> give the Rust’s factor), degree, powers of ξ , derived target, claim-pool order, the third public claim with value 0, the limbs as strided claims.	reading; cited-checked	the final message matches only for the six opcode tables (R11)
N3	gt-table-pub §0 item 3, §5 D.2	The blueprint’s errors $(B+2)/ E $ on ξ and $3/ E $ per round are correct, tight and on the right challenges.	paper	—
N4	gt-table-pub §2 A.4	Checked and agreeing across the three sources: the moment of every challenge; the power assignment; the three sides and their order; the order of the 104 values (<code>tables.rs:436-863</code> against <code>verifier.py:823-834</code>); the slot of each BLAKE2s limb; the pool order.	reading; cited-checked	—
N5	gt-table-pub §2 A.4	The Rust-only guard <code>zeta.len() < n</code> is unreachable ($\mu_{\text{bus}} \geq \tau_{\max} + 1$); nothing to model.	reading	—
N6	gt-table-pub §5 D.4	The public input’s per-limb error $1/ E $ is correct, tight and on the one challenge (attained by the test <code>PublicInput.lean:106-110</code>).	paper; repository test	—
N7	gt-table-pub §8 Notes	Perfect completeness needs no exceptional-challenge clause in the table sumcheck and the public input (the only inverse in the prover is of the constant $g + g^2$).	paper	—
N8	verify-gt-table-pub §1	301 citations checked; no quotation misquoted; the load-bearing quotations (the Rust verifier loop, the public-input checks in Rust, Python and the guest, <code>BusVerify</code> , the tex, <code>ArkLib RoundByRound.lean</code> , the spine’s blocks) are verbatim.	citation check	four numbering corrections and two wrong lines (Dossiers processed)
N9	gt-bus Summary item 1, §A.6	Specification, Rust and Python agree on the transcript of the setup, the commitment and the bus phase, up to the specification’s omissions; the Rust and Python verifiers accept the same transcripts in these phases.	reading line by line (not a differential test)	—
N10	gt-bus §E.5	The spine is right on the bus: <code>tuples</code> takes every row of every flush and boundary block of a side; <code>Balanced</code> is a permutation of 16-tuples, separator included; <code>CountsNonzero</code> covers every cell of every listed column; <code>Seam.commit</code> is <code>M3Holds</code> of the oracle (<code>Iff.rfl</code>); <code>Seam.bus</code> carries the public lines and <code>aux</code> ; the toy’s tests make each clause fail alone.	reading; cited-checked	—
N11	gt-bus §E.3	Every framework block of leanVM (state, memory, bytecode; seed and finalization) is expressible as a <code>BoundaryBlock</code> ; the boundary depends on the program and the sizes, never on the public input.	reading; cited-checked	the instance is then not <code>Ensemble.toM3</code> of the ensemble (R30)

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N12	gt-bus §F	Status findings confirmed at the pin: one scalar root for push and pull (F3, but an agreement, not a divergence), the count tree holds only the tables' count columns (F11), ties put the shared columns first (F17), the caps and stacking bound (F15), padding leaves 1 and stack padding 0.	reading; cited-checked	—
N13	gt-bus §C	The blueprint matches: the seed; one (α, β) message; radix and parity; $R_c \neq 0$; one root; the derived remainders as linear claims; the count tree; the padding; the fingerprint; the layer check; <code>busError</code> on (α, β) ; acceptance tests 1, 2, 3, 4, 15, 17.	reading; cited-checked	test 18's clause is loose; test 28 is incomplete (R43)
N14	gt-bus §C	Deliberate deviations to keep: the sizes index the instance instead of being a message (owes absorption before the root and a uniform bound); the commitment is sent as the oracle (Layers 11, 12 owe the compilation); the verifier's assertions become layout lemmas (owes four lemmas about <code>leanIsaInstance</code> , none listed in Layer 3); the round identity is checked in the oracle protocol (owes <code>RoundPoly.decode</code>).	reading	the four lemmas are not listed in Layer 3
N15	gt-bus §E.2	The multiset lemma (Lemma 5.2) holds as used; <code>CountsNonzero</code> against $R_c \neq 0$ is exact.	paper	—
N16	gt-bus §H.1	Python probe: an honest GKR prover written from the specification is accepted by the pinned Python verifier; ζ never contains the last combiner.	probe run; re-run by the check with identical output	—
N17	verify-gt-bus §1, §4 item 7	314 citation rows checked (about 400 references); no quotation misquoted; no finding weakened by the corrections.	citation check	—
N18	gt-flock-ring §2.6, §10	The Flock transcript: fifteen steps, three sources, no disagreement; one equality check (the lincheck terminal identity); Flock commits nothing of its own and the verifier queries nothing during the phase.	reading; independent check agrees (stream count $162 + 2 \cdot k_batch$, $2 \cdot k_batch + 25$ challenges)	—
N19	gt-flock-ring §10	The blueprint's strided limb claims (:841-846) match <code>stack_open.rs:84-97</code> , <code>305-323</code> and <code>verifier.py:892</code> , <code>295-302</code> .	reading; cited-checked	—
N20	gt-flock-ring §10	The claim pool order (:324: ring-switched claim first, then bus, table, public-input claims) matches <code>cpu/mod.rs:656-667</code> , <code>stack_open.rs</code> , <code>verifier.py:1413</code> , <code>08:98</code> .	reading; cited-checked	—
N21	gt-flock-ring §10	The Fiat–Shamir seed convention (:325) matches <code>cpu/mod.rs:82-93</code> , <code>verifier.py:1366-1368</code> .	reading; cited-checked	—
N22	gt-flock-ring §10	The strong reading of <code>aux</code> (status decision 12) is right; a predicate on the limb slots alone has no honest prover; the spine's docstrings carry it.	reading	the constant position must be added (R56)
N23	gt-flock-ring §10	The phase order <code>pub » flock » opening</code> matches <code>cpu/mod.rs:745-768</code> , <code>verifier.py:1397-1413</code> .	reading	—
N24	gt-flock-ring §1 item 4, §5.2	<code>flockError_le</code> is right for a fixed committed polynomial: $(4 \cdot k_batch + 163)/ E $ is Annex C.7's sum and matches the Rust's parameters term by term; $2^{32}/ E $ bounds the ring-switching degree.	paper	list size after compilation (R23)

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N25	gt-flock-ring §1 item 4	The partially fixed zerocheck’s hypothesis holds for the pinned constants: the 128 equality weights of the seven fixed coordinates are independent over F_2 (rank 128).	Python probe; re-run by the check	—
N26	gt-flock-ring §10	<code>Blake2sRowsValid</code> is what the adaptor must reach; blocks and rows correspond one to one (no padding block).	reading	—
N27	gt-flock-ring §1 item 6, §6	Ring switching serves the bit witness only; K-valued columns need none; in leanVM it sends nothing, checks nothing and draws six challenges.	reading; independent check agrees	—
N28	gt-flock-ring §1 item 3, §4	Perfect completeness of the specification’s Flock protocol is attainable (a prover sending the true coefficients).	paper; Python probe 9.2	the Rust prover is not perfectly complete (R111)
N29	verify-gt-flock-ring §1	About 381 citations checked; all eight independent checks agree with the dossier; both Python probes reproduce.	citation check	—
N30	code-spine §0 item 1; probe P1	No theorem of the spine is wrong for what it states; every stated theorem uses only the kernel’s three standard axioms.	probe-run (old pins)	—
N31	code-spine §C.1	<code>M3Holds</code> says what the specification’s accept list says, clause by clause (05:10-16, 101, 111; 06:74-78; 08:29-33, 100).	reading	—
N32	code-spine §D.1; probe P2	<code>Balanced</code> handles the side filter, the row ranges and multiplicity (a tuple pushed twice and pulled once; pushed twice and never pulled, which a field-summed balance accepts).	probe-run	—
N33	code-spine §D.2; probe P3a	Each seam is inhabited and refuted conjunct by conjunct.	probe-run	—
N34	code-spine §D.5, §D.6; probes P3b, P1	The commit phase’s state function and extractor are as documented; the extractor chain is computable.	probe-run	—
N35	code-spine §E.1	The blueprint’s spine sketch (:442-554) matches the code name for name and signature for signature; the differences listed change no statement.	reading	the interface list omits load-bearing names (R91)
N36	code-spine §E.3 row 25	The spine’s five modules and <code>ToArkLib/</code> import nothing from the arithmetization.	git grep	<code>Protocol/Basic.lean</code> does (R44)
N37	code-spine §E.2	<code>Seam.done</code> equals <code>Set.univ</code> : same set, different spelling.	probe-run	—
N38	code-spine §0 item 9, §C.7	<code>Refinement</code> ’s two witness universes are independent.	reading	—
N39	code-spine §E.3	Acceptance tests 26 (seams are the contract), 27 (the toy is honest) and 28 (degree at the seam) are met.	reading	26 says nothing about content; 28 is met at the cost of a guard (R2)
N40	code-spine §C.6	The quantification over the oracle state σ , <code>init</code> , <code>impl</code> is without content (one <code>impl</code> , never called); it is forced by <code>ArkLib</code> .	reading	it costs three binders in eleven load-bearing declarations
N41	code-spine §B	The audit surface was measured: 70 declarations and 217 code lines for completeness; 76 and 248 for knowledge soundness in its existential form; 100 and 354 in the named form.	count script at b435631	—
N42	code-pubinput §G Negative results, §B	The public-input phase’s schedule, challenge, number of scalars, point $(r, \theta, \dots, \theta)$, honest values, three claims and their order, rejection and error agree between the specification, the three implementations and the Lean.	reading line by line	the check itself differs (R12)
N43	code-pubinput §G Negative results	The bit order of <code>linePoint</code> is proved (<code>evalMle_linePoint</code>) and separated from the other order by a two-variable test.	reading	—

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N44	code-pubinput §G Negative results	$1 + r$ against $1 - r$ is joined by <code>CharTwo.sub_eq_add</code> , correct in characteristic two.	reading	—
N45	code-pubinput §F	The error is charged to the right challenge and is tight.	reading	—
N46	code-pubinput §F	No admitted <code>ArkLib</code> theorem is in the closure of the phase’s two bundles.	on a verbatim copy	<code>#print axioms</code> on the original file was not run
N47	code-pubinput §F, §C.3	The knowledge state function’s three rounds are right and none is trivial.	probe-run	—
N48	code-pubinput §E	<code>sent</code> is no part of the relation (<code>M3Holds</code> does not read it).	reading	—
N49	code-pubinput §C.6-C.8	What the theorems do catch: pooling the prover’s values unchecked, checking the first value only and pooling the prover’s values, not pooling an unsent line’s claim; each refuted in Lean on a concrete instance.	probe-run	—
N50	code-layer1 §B.1	The cube’s bit order and the order of points agree with leanVM.	reading	—
N51	code-layer1 §B.2	Stacking offsets and selectors agree: on the pinned Python verifier’s real layout (92 blocks, <code>stack_log = 22</code>) <code>Blocks.offset</code> reproduces all 92 offsets.	Python and Lean probes (<code>offsets.py</code> , <code>OffsetsProbe</code>)	the tie order is supplied by nobody (R78)
N52	code-layer1 §B.3, §B.4	Padding values, the one-padded leaf identity (equation (2) of §5.4) and back-loaded padding agree.	reading and probes	—
N53	code-layer1 §B.5	The index column agrees (generator $g = x$; coordinate k carries $g^{(2^k)}$).	reading	—
N54	code-layer1 §B.6	<code>bytecodeColumn</code> of a sixteen-instruction program covering every opcode and <code>DEREF</code> mode equals, cell for cell (256 cells), the table of a transcription of the Rust encoder, and its extension at (ζ, α) equals <code>verifier.py:566</code> .	probes <code>ValuesProbe</code> , <code>values.py</code>	the Rust encoder was transcribed, not executed
N55	code-layer1 §B.7	A column claim as a weighted claim on the stack agrees.	reading and probe	—
N56	code-layer1 §C, §0 item 2	No Layer 1 theorem is false or vacuous; the hypotheses $B.\text{total} \leq 2^\mu$, <code>Antitone size</code> have inhabitants; seventeen declarations depend only on the three standard axioms.	reading; <code>#print axioms</code> probe	—
N57	code-layer1 §D.1	No file of <code>LeanerVM/Protocol</code> imports <code>Clean</code> or <code>LeanerVM.Semantics</code> directly; no phase imports <code>FixedColumns</code> .	git grep	<code>Basic.lean</code> imports the arithmetization (R44)
N58	code-layer1 §D	The four <code>ToCompPoly</code> modules are generic; <code>CompPoly</code> at both pins shares only <code>eqTilde_eq_prod</code> and <code>eqTilde_append</code> with them; <code>ArkLib</code> at both pins has none of them.	reading; <code>DuplicatesProbe</code>	—
N59	code-layer1 §F	The Layer 1 sketch matches the code declaration by declaration.	reading	—
N60	code-layer1 §E	Today an auditor of the master theorems reads no Layer 1 declaration (17 definitions, 61 lines, become trusted with the adaptor; about 8 more with the executable verifier).	count	—
N61	code-layer1 §G.10, §H	<code>idxColumnEval</code> is efficient (binary exponentiation in <code>CompPoly</code> ’s K); the upgrade to <code>144c5aa</code> changed only proofs in Layer 1, and the tests’ numerals were rewritten as <code>K.ofBits n</code> , the same words, so every test keeps its meaning.	reading; git diff	—

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N62	lib-arklib §G.9	ArkLib’s KnowledgeStateFunction, rbrKnowledgeSoundnessWorstCaseWith, perfect Completeness, GuardedForm, OutputIsPure read in full at dca90385: no way to satisfy the named worst-case form at error 0 for a verifier accepting a false statement except through the extractor or an empty output-witness type (visible in the statement).	reading	—
N63	lib-arklib §C.1, §G.9	The port KnowledgeAppend.lean differs from ArkLib PR #615’s file only in headers, the module system, docstrings and dropped wrappers.	line-by-line diff	—
N64	lib-arklib §G.9	The composed error is per challenge with no additive term, and the seam round is handled (backward_right, extractMid_seam).	reading in full	—
N65	lib-arklib §B.1, §G.9	Every #print axioms on leanerVM’s declarations (master theorems, both phases, composition, port) reports the three standard axioms only.	probe AxiomsLeanerVM (old pin)	—
N66	lib-arklib §D.2	The named form matches the literature’s round-by-round knowledge soundness (Block et al. 2023, Definition 3.13) in its worst-case-per-prefix form and per-round errors.	reading	the extractor may be any function (R1)
N67	lib-arklib §D.4	Non-vacuity of the notion: an accept-everything verifier has no knowledge state function and is not knowledge sound at error 0 (it is at error 1, as every verifier is).	probe NonVacuity	—
N68	lib-arklib §D.1	The transcript-level half of the plain reading of the master theorem is proved.	probe PlainReading	—
N69	lib-arklib §D.6	The spine’s named extractor (“return the first message”) compiles.	probe Extractors	—
N70	lib-arklib §F.1-F.4	At 7653a901 the spine’s vocabulary exists with the same shapes; the new probability notation is notation only; the two theorems of KnowledgeAppend.lean differ from b435631 only in the spelling of three probabilities; the admitted theorems (sumcheck, composition, round-by-round to plain, context lifting, Fiat–Shamir completeness) are exactly as admitted as at dca90385; the port is still needed.	reading; baseline comparison by name; git diff	—
N71	lib-arklib §B.2	Most rows of the blueprint’s ArkLib table name existing, proved objects at the stated place.	reading and probes	the exceptions are R96
N72	lib-others §B.4; probe FieldFidelity	K, E, y, ofK, g agree with the specification, Rust and Python bit for bit (moduli, bit order, limb order, embedding, generator, product formula); the compiled arithmetic reproduces eleven Rust reference products.	probe-run (old pins)	a re-run at the new pins needs K.ofBits literals
N73	lib-others §B; probe Layer0	Irreducibility, field structure and $ E = 2^{192}$ are proved in CompPoly without native_decide (a Rabin certificate checked by rfl), on the standard axioms.	probe-run	—
N74	lib-others §E.1	Both samplers are uniform by proof (uniformity is a class law of SampleableType); $\Pr[x \mid \mathcal{S}^t E] = 2^{-192}$; the K diamond is definitional at the old pins and disappears at the new.	probes Layer0, SamplerDiamond2	—
N75	lib-others §E.2, §E.3	The column oracle answers the specification’s evaluation $\hat{q}(r) = \sum \text{ofK}(q_i) \cdot \text{eq}(r, i)$ in little-endian cube order; card_E; NoOracle, OneOracle, noOracle_eq are correct.	reading and probe	—
N76	lib-others §E.4	The counting bounds are correct, tight, with load-bearing hypotheses, and the same statement before and after the probability-API port.	probe CountingBounds	—

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N77	lib-others §D.3	The trusted base is the kernel and the three standard axioms, enforced lexically and transitively with a negative control; #guard tests are compiled-code evidence, not proof.	reading	—
N78	lib-others §C.5-C.7	Clean’s balance is vacuous over K (unsatisfiable, not merely unsound); leanISA’s <code>BalancedPair</code> replaces it; unchanged at Clean 42fe4b26.	probe <code>CleanBalance</code>	—
N79	docs-debt §B.10	Every local Markdown link and anchor of the 24 tracked Markdown files resolves (150 links).	probe <code>check_anchors.py</code> (re-run)	—
N80	docs-debt §B.10	Every one of 81 citations into the pinned leanVM sources resolves to an existing range once its crate is known.	citation probe	22 are ambiguous as written (<code>transcript.rs</code> , <code>witness.rs</code> , <code>lib.rs</code> , <code>filler.rs</code> , drafts)
N81	docs-debt §B.10	40 of 43 citations into the pinned libraries are exact (old pins).	reading	three drifted (R92)
N82	docs-debt §B.10	All 30 references to numbered acceptance tests point at the intended test.	reading	—
N83	docs-debt §B.10, §H.3	The spine’s sketch (:442-554) agrees with the code; the Layer 8 sketch agrees with <code>PublicInput.lean</code> .	reading; <code>check_interface_names.py</code>	—
N84	docs-debt (second pass)	The body of issue 12 and the hole comment are unchanged since 2026-09-28 14:28 UTC.	gh read; diff against the saved copies	—
N85	literature Summary item 1, §A.3	“After Fiat–Shamir the largest bound is what a random-oracle query buys” is right at its core ([CY24] Theorems 31.2.1, 31.3.1; [BGKTTZ23] Theorem 3.15).	literature	the hash term, the list factor and grinding are missing (R23)
N86	literature §A.3	The master theorems are round-by-round (as [BCS16] Theorem 1.5 requires), with per-challenge errors ([CY24] Definition 31.1.2) and a straight-line extractor that reads the oracle ([CMS19] Definition 8.5).	literature	—
N87	literature Summary item 2, §C.4	leanVM’s PCS regime (Johnson with slack, [BCHKS25] Theorem 4.6) is the right, provable one; Annex B states Theorem 4.6 exactly as printed (term by term at $M = 1$).	term-by-term comparison	a preprint with a sketched proof; the Rust’s arithmetic and η search not checked
N88	literature §B.4	The known diagonalization attacks ([KRS25], [Fen26]) do not apply: the statement is bound and the trace committed before the first challenge at the pin.	reading <code>cpu/mod.rs:78-93</code> , <code>108-124</code> , <code>fiat_shamir/src/lib.rs</code> , <code>08:55-73</code>	—
N89	literature §D.3	The blueprint’s arithmetization references match the specification’s <code>refs.bib</code> ; no missed source.	literature search	—
N90	literature §E.6	ArkLib’s round-by-round definitions, the <code>KnowledgeStateFunction</code> docstring, the basic Fiat–Shamir query and the admitted MCA theorem are textually unchanged between <code>dca90385</code> and <code>7653a901</code> .	git show	—
N91	literature Summary item 4, §F.3	The blueprint’s cut (a relation on one committed column and an adaptor to the trace relation) matches the only published plan (SP1) and [KSHC26]’s obligations; characteristic 2 forces combinatorial balance, which the blueprint has.	literature	—
N92	literature §A (search)	No published theorem states round-by-round soundness with per-round proof of work as a Fiat–Shamir compilation result for multi-round IOPs.	negative result of a search, not proof of absence	—

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N93	boundary-adaptor §H, §A.4	M3Holds’s five clauses against SatisfiedBy’s thirteen conjuncts: every conjunct has a source once the caps hypothesis, the boundary lemma, the count derivation and the Flock consequence lemma are in place.	reading	the boundary lemma and Flock lemma are missing (R31, R15)
N94	boundary-adaptor §H	w.Constraints is asserts and lookups only; leanISA emits no lookup and only JUMP asserts, so toM3_constraints_iff is provable in principle.	reading; probe Shapes	—
N95	boundary-adaptor §H	The degree bound $d = 2$ holds: every flush coordinate of the six tables has degree at most two.	reading Tables/*.lean	—
N96	boundary-adaptor §H, §E.4	The state boundary is constants only, as layout.rs:354-358; the memory and bytecode blocks’ committed coordinates are columns of tables whose height equals the block’s by construction.	reading	—
N97	boundary-adaptor §H, §A.4, §A.6	Caps is derivable from admissibility plus the construction of witness0f; Sizes.of Witness w = some s follows from SatisfiedBy.	reading	—
N98	boundary-adaptor §H, §B.2	The chain names one program and one input throughout; no theorem lets the prover choose either.	reading	—
N99	boundary-adaptor §H, §D	The top limb is anchored (agreeing with code-pubininput §D).	reading; probe Shapes	—
N100	boundary-adaptor §H	Refinement accepts a Type 1 target witness; the pointwise composition of T4 typechecks; Extractor.Straightline.map cannot serve.	probes Transport, UniverseFail	—
N101	boundary-adaptor §H	The arithmetization declarations the chain will use, the two master theorems, bytecodeColumn_eval, idxColumn_eval depend on the kernel’s three axioms only (map_option_valid: two).	probe Shapes	—
N102	gt-opening-compile §I Negative results, §A.2	The moment and the powers of λ agree across the three sources; the blueprint’s claim-pool order row agrees.	reading	—
N103	gt-opening-compile §I Negative results, §B.1	The Python’s verify_whir matches the Rust’s succinct verifier message by message.	reading	—
N104	gt-opening-compile §I Negative results, §H.3	The Python’s derive_config and the Rust’s configs_for_rate give the same ladder for all 56 (ρ , μ); the query counts of the Python table are exactly the Rust’s floating-point search.	Python scratch port (whir_params.py)	—
N105	gt-opening-compile §I Negative results, §B.2	The column weight’s closed form agrees with Lemma lem:colweight.	reading	—
N106	gt-opening-compile §I Negative results, §C.1, §C.3	The digest encoding, the tags, the seed, the squeeze and the grinding predicate agree between Rust and Python; the round-polynomial encoding agrees; the blueprint’s Fiat–Shamir row has the right tags, seed and squeeze.	reading	the chain’s step is the 64-byte hash (R66)
N107	gt-opening-compile §I Negative results, §B.3	The blueprint’s six WHIR constants and its ladder_queries_eq pin agree with the deployed parameters.	reading; Python scratch	—
N108	gt-opening-compile §I Negative results, §A.6, §E.2	WeightedClaim.Holds agrees with Definition 3.13; McaJohnson agrees with ArkLib’s admitted statement and Annex B.	reading	—

continued on the next page

Table F.3, continued

#	Dossier	What was checked and found right	How	Caveat
N109	gt-opening-compile §D.7	Query sampling is exactly uniform (d-bit chunks of a uniform 192-bit string, $d \leq 28$); both verifiers derive the lanes, block lengths and leaf widths from the announced sizes; both floor μ at 15.	reading	—
N110	gt-opening-compile §0 item 3, §F.1	With the stacking window the blueprint’s own phase errors sum to about 2^{-160} (ring switching dominates; the (α, β) term is at most 2^{-162}).	paper; Python scratch	false at the per-log caps without the window (R33)
N111	gt-opening-compile §0 item 4, §G	Every codeword within radius $\gamma < 1 - \rho$ of a K-valued word is K-valued, so the list consists of K-valued multilinear of the right size.	paper (a two-line argument, not in the specification)	—
N112	obligations §5.5	Every node of the blueprint’s Layers 0 to 13 and of its holes table has a place in the tree; no blueprint obligation is redundant (three declarations serve nothing on the chain: <code>witnessOf_stackOf</code> , <code>Extractor.Straightline.map</code> , <code>piop_rbr KnowledgeSoundness_exists</code> , which are non-vacuity tests or upstream candidates).	reading (the tree of §2)	—
N113	obligations §5.5	The per-challenge errors of the built phase ($1/ E $) and of the two sketched phases the dossiers worked (table <code>sumcheck</code> $(B+2)/ E $ and $3/ E $; opening $(J-1)/ E $ on λ) are correct, tight and on the right challenges; the bus phase’s (α, β) term is correct within a factor 2.	paper (from gt-table-pub, code-pubinput, gt-opening-compile)	—
N114	obligations §5.5	The composition of errors through the spine is per challenge with no additive term; the seams add nothing.	reading (lib-arklib §G.9)	—
N115	obligations §5.5	The hypotheses table reproduces boundary-adaptor §C’s rows (none found wrong) and adds six: the grinding check, the canonical encodings, <code>R1CS_DIGEST</code> , the <code>F_2</code> -independence of the fixed weights, the oracle-freeness of the front, the bound on <code>piopError</code> .	reading	—

F.3 What was not verified

What a dossier marks unverified, written and not run, out of task, or not checkable after the upgrade, with what would verify it.

Table F.4: Claims the dossiers mark unverified, and what would verify them.

#	Claim	Dossier	What would verify it
U1	The Rust and Python verifiers behave as read.	gt-table-pub §10	run both on a dumped proof and on the mutations of §3 B.1, B.2
U2	The errors of §5 D.2, D.3 and §6 E.3.	gt-table-pub §10	Layer 4's sumcheck and the bus phase's Security with the zerocheck conjunct
U3	The finding on the sumcheck's tables.	gt-table-pub §10	leanISAInstance, when built
U4	The combined public-input check has error $1/ E $.	gt-table-pub §10	the second phase proposed (code-pubinput proved the key lemma)
U5	leanISA's tables have the Rust's column order.	gt-table-pub §10	compare Arithmetization/Tables/*.lean with tables.rs: 436-863
U6	leant's ZerocheckClaim pattern, which the blueprint's row Seams cites.	gt-table-pub §10	—
U7	The probes SeamBusShape and SeamBusMember still elaborate at the new pins.	verify-gt-table-pub §1 item 7	re-run after the rebuild
U8	The Rust verifier and prover (read only); the equality of the Rust and Python verdicts is by reading.	gt-bus §1	a differential test (the repository's own tests compare them)
U9	The attack without the $\tau_{\text{BLAKE2S}} \geq 3$ floor.	gt-bus §1, §B	a verifier without the cap at $\tau_5 = 2$
U10	The public-input phase rejects a public word with a nonzero third limb (defence in depth).	gt-bus §1, §B	the public-input dossiers (code-pubinput, boundary-adaptor §D)
U11	Not examined: Fiat-Shamir beyond absorption order; the opening; the table sumcheck beyond the seam; the recursion guest.	gt-bus §1	gt-opening-compile, gt-table-pub
U12	The Lean probe BusSeam.lean's output, the control copy's failure, #print axioms Probe.x0_degree.	verify-gt-bus §1 item 12	re-run after the rebuild
U13	The Rust prover's behaviour at $r_{\text{eq}} = 1$.	gt-flock-ring §11	a Rust unit test on send_round and next_round_poly
U14	R1CS_DIGEST is the digest of the matrices.	gt-flock-ring §11	the commit the Rust's comment names
U15	The Rust circuit computes BLAKE2s.	gt-flock-ring §11	the theorem of §5.3
U16	lincheck.rs beyond 1400, zerocheck/multilinear.rs beyond 330, univariate_skip*.rs beyond 200.	gt-flock-ring §11	—
U17	The WHIR part of the opening; the recursive verifier's hint M_lc.	gt-flock-ring §11	gt-opening-compile
U18	The sketches FlockPhase, FlockRegion, Component.Guarded.	gt-flock-ring §11	write them
U19	What in the lincheck prover needs $n_{\text{outer}} \geq 8$ (the floor's reason).	verify-gt-flock-ring §4 item 5	read the lincheck prover
U20	The Lean probe NoCheckFlock.lean.	verify-gt-flock-ring	re-run after the rebuild
U21	The relation of toyAlias is empty for every stack and statement.	code-spine §G	a theorem $\forall \text{input } q, \neg \text{M3Holds toyAlias input } q$
U22	Layout.read is determined by extend.	code-spine §G	a lemma read_eq_of_extend
U23	An always-rejecting bundle has no Phases.Security on an instance with an inhabited relation.	code-spine §G	a probe with a rejecting zero-round verifier

continued on the next page

Table F.4, continued

#	Claim	Dossier	What would verify it
U24	The read-everything phase inhabits <code>Phases.Security I</code> at error 0 for every <code>I</code> .	code-spine §G	a probe on an instance with $\mu = 0$
U25	The probes that use $(2 : K)$ at the new <code>CompPoly</code> pin.	code-spine §G	re-run <code>P2Relation</code> , <code>P3aSeams</code> , <code>P5PassThrough</code> with <code>K.ofBits 2</code>
U26	The earlier spine review’s validation paragraph.	code-spine §G	<code>./scripts/validate.sh</code> on <code>b435631</code>
U27	Mutation 6 (claims pooled at a wrong point).	code-pubinput §0, §C.9	write and run after the rebuild
U28	The deployed phase’s full <code>Phase.Security</code> (mutation 4b).	code-pubinput §0, §C.13	write <code>deployedSecurity</code>
U29	The two-line-instance display for §D.1; <code>#print axioms</code> on the original <code>PublicInput.lean</code> .	code-pubinput §0	run after the rebuild
U30	Probes 7a (a wrong check caught by completeness against a fixed prover) and 8 (an extra check caught by completeness).	code-pubinput §C.10, §C.11	run <code>Probe7a.lean</code> , <code>Probe8.lean</code> after the rebuild
U31	The Rust bytecode encoder was not executed (compared through a transcription).	code-layer1 §J	run the pinned Rust <code>bytecode_table</code> on the same program
U32	The proposed strided reader and layout combinator.	code-layer1 §J	build them
U33	No probe was run at the new pin; <code>ValuesProbe</code> , <code>StridedProbe</code> need <code>K.ofBits</code> rewrites.	code-layer1 §J	re-run after the rebuild
U34	<code>DuplicatesProbe</code> and <code>StridedProbe</code> ran outside lake (the old Lean binary on the <code>b435631</code> artefacts).	code-layer1 §J, §I	re-run with <code>lake env lean</code> after the rebuild (else §D.2 and §B.8 stand as short paper proofs)
U35	Not examined: the rest of Layer 3; the leaf stacks’ block order in code; the pad cells past the last lane.	code-layer1 §J	—
U36	The probes at the new pin (written in the old probability notation).	lib-arklib §F.3	a mechanical rewrite and a run after the rebuild
U37	<code>fiatShamir_completeness</code> is false in general for a protocol whose honest prover fails at the default challenge.	lib-arklib §G.4	a counterexample probe
U38	Whether the honest Flock prover fails at <code>r_eq = 1</code> (the condition of R4).	lib-arklib §G.2	answered by <code>gt-flock-ring §4</code> and §8.12: the Rust prover does, the specification’s does not
U39	<code>FieldFidelity</code> at the new pin with <code>K.ofBits</code> literals.	lib-others §F.3	re-run after the rebuild
U40	<code>instSampleableTypeK</code> is the only <code>K</code> sampler at the new pins.	lib-others §E.1	a probe at the new pins
U41	The <code>ArkLib/CompPoly eval_zero</code> clash.	lib-others §G.9	elaborate a file importing both
U42	Numerals inside vectors and other shapes at <code>144c5aa</code> .	lib-others §F.3	a wider scan
U43	All seven probes at the new pins.	lib-others §H	re-run after the rebuild
U44	Whether <code>VCVio</code> ’s Merkle-tree library fits leanVM’s Merkle trees (BLAKE2s, the leaf encoding, pruned paths), and its state at <code>a4232d08</code> .	docs-debt §H.15	read <code>VCVio</code> ’s library against <code>fiat_shamir/src/merkle.rs: 14-67</code> at both pins
U45	The counts of letter codes are lower bounds (the pattern skips codes in backticks).	docs-debt §D	a pattern that includes backticks
U46	About 22 references read as abstracts only, eight not opened, four through fetch summaries; the SP1, Pico and advisory items from a stopped helper’s notes not re-checked.	literature (what I could not open)	open and read them
U47	Whether a refereed or Lean proof of [BCHKS25] Theorem 4.6 exists (ArkLib’s is admitted at both pins), and whether Haböck’s update of [Hab25] exists.	literature (what I could not open)	search again
U48	The list size <code>L_0</code> at leanVM’s production parameters (estimated 2^7 to 2^{10}).	literature §A.4	the Rust’s <code>validate()</code> output

continued on the next page

Table F.4, continued

#	Claim	Dossier	What would verify it
U49	Whether a cross-role input of the shared BLAKE2s map (a Merkle node against a chain block) is exploitable.	literature §E.6	an analysis of role separation
U50	Whether an ArkLib revision after 7653a901 provides a chain-based transform.	literature §E.6	watch upstream
U51	The injectivity of <code>prog ↦ (prog.logSize, bytecodeColumn prog)</code> .	boundary-adaptor §H	a probe proving it
U52	The counterexample program of §D (<code>SET_CONSTANT [g^0, y^1]</code>): its two-line stack.	boundary-adaptor §H, §D	build the stack
U53	The count of admissible size vectors (about 2^{34}).	boundary-adaptor §H, §B.3	Layer 3's μ
U54	Shapes, Transport, UniverseFail, DefInstance at the new pins.	boundary-adaptor §H	re-run after the rebuild
U55	A halting run visits distinct states.	boundary-adaptor §H, §G.18	a Lean proof
U56	The two Lean probes (<code>InnerProductOracle.lean</code> , <code>FiatShamirChain.lean</code>).	gt-opening-compile §0, §H	—
U57	That <code>fiatShamir_completeness</code> for a constant challenge oracle is false in general.	gt-opening-compile §E.2	a counterexample probe
U58	That a grinding nonce exists for every reachable chain state.	gt-opening-compile §E.5	none by proof; the Lean <code>prove</code> is fuelled instead

F.4 Counts, disagreements and merged rows

A row’s place is the highest severity any dossier gives it. Table F.5 counts the rows by that severity, table F.6 by subject and severity; the two positions are counted in neither as findings.

Severity	Rows
critical	1
major	37
minor	63
note	37
positions (not findings, no severity)	2
total findings	138

Table F.5: The register’s rows by severity.

Subject	critical	major	minor	note	total
relation and seams	0	2	6	2	10
master theorems and error	1	2	4	2	9
phase: setup	0	1	1	3	5
phase: commit	0	0	0	0	0
phase: bus	0	3	3	1	7
phase: table sumcheck	0	1	1	2	4
phase: public input	0	2	2	1	5
phase: Flock and ring switching	0	4	5	2	11
phase: opening and compilation	0	10	9	6	25
adaptor and leanISA boundary	0	7	5	5	17
libraries	0	1	4	7	12
Layer 1	0	1	5	3	9
documentation	0	3	18	3	24
all	1	37	63	37	138

Table F.6: The register’s findings by subject and severity (positions excluded).

F.4.1 Disagreements between dossiers

Where two dossiers disagree, each position is a row; the row’s note names the other.

- **R2** (The bus seam admits statements the deployed table sumcheck cannot serve): DISAGREEMENT with R3: gt-bus holds that the surplus generality costs nothing that matters. code-spine endorses gt-table-pub’s redesign.
- **R3** (The real bus phase can fill the spine’s bus slot; the seam’s surplus generality burdens only the table phase): DISAGREEMENT with R2: gt-table-pub and code-spine hold that the table phase which meets the seam must run checks leanVM does not run, so it is not the deployed verifier.
- **R4** (The spine’s completeness cannot carry an error, and Flock may need one): DISAGREEMENT with R17 (gt-flock-ring): there the specification’s Flock is perfectly complete, only the Rust prover fails, so a Lean honest prover sending the true coefficients needs no completeness error. lib-arklib’s own condition is answered by gt-flock-ring §4 and §8.12. The check of gt-flock-ring adds that ArkLib’s completeness composition has side hypotheses (**hSeam**, completeness from every shared state). gt-opening-compile §E.5 agrees with gt-flock-ring that the exceptional point is the Rust prover’s, and calls lib-arklib’s error field “the right insurance”.
- **R7** (The GKR layer sumcheck is the normalized variant; the blueprint gives it the wrong degree, the wrong error and no definition): DISAGREEMENT with R8 on the GKR round error (4 against 5). The check corrected G1’s wording on the table sumcheck: “three of the cubic’s four coefficients are sent”, not “the whole cubic” (verify-gt-bus.md §4 item 1).
- **R8** (The GKR last-layer rounds cost $5/|E|$ each, as the blueprint says): DISAGREEMENT with R7: gt-bus reads the check as made on the cofactor (normalized sumcheck), so the error is $4/|E|$

and the blueprint’s $5/|E|$ is not tight. Both are upper bounds; neither dossier’s conclusion on the zerocheck changes (the escape is a maximum with $\epsilon_G \geq 1/|E|$ either way). Both agree on the combination challenges ($1/|E|$ each) and on a zero error for the unused last combiner only in gt-bus.

- **R17** (Acceptance test 20 files a completeness failure under a soundness error, and attributes to the protocol an inverse only the Rust prover takes): DISAGREEMENT with R4 (lib-arklib §G.2), which would give the spine a completeness error for Flock. The check notes that “no verifier takes an inverse” is literally false (constants are inverted); read “no inverse of a challenge-dependent value”. gt-opening-compile §E.5 agrees that the exceptional point belongs to the pinned Rust prover.

F.4.2 Merged rows

Rows that state one finding of two or more dossiers, with the dossiers each cites.

- **R2** (The bus seam admits statements the deployed table sumcheck cannot serve): gt-table-pub, code-spine
- **R11** (The table sumcheck’s tables are not distinguished from the instance’s other tables): gt-table-pub, gt-bus, boundary-adaptor
- **R12** (The public-input theorems are about a verifier none of the three executable verifiers runs; the debt is assigned to Layer 12, which cannot pay it): gt-table-pub, code-pubinput
- **R14** (Layer 9’s Flock interface consumes the eighteen limb claims and emits one claim; leanVM’s Flock reads no claim): gt-table-pub, gt-flock-ring
- **R15** (The instance depends on a Flock interface that depends on the instance (the limb slot map, aux, the compression lemma)): gt-flock-ring, boundary-adaptor
- **R20** (No ArkLib theorem can inhabit the planned Fiat–Shamir and BCS interfaces, and the interface as commented is false): lib-arklib, gt-opening-compile
- **R21** (verify_iff_compiled names definitions that do not describe the compiled verifier): lib-arklib, gt-opening-compile
- **R23** (The non-interactive error omits grinding, the list-size factor and the hash term, so it cannot carry leanVM’s 128-bit claim): gt-flock-ring, literature, gt-opening-compile
- **R27** (The base soundness (and completeness) theorems lack WellFormedBytecode): docs-debt, boundary-adaptor
- **R28** (The base soundness theorem is not a probability statement about verify, and concludes no extracted witness): boundary-adaptor, gt-opening-compile
- **R33** (Layer 3’s sizes and admissibility statements cannot be stated or hold as written (Sizes, Sizes.ofWitness, Sizes.Admissible, admissible_iff_caps, leanIsaInstance_fits)): gt-bus, code-layer1, boundary-adaptor, gt-opening-compile
- **R36** (The per-layer sketches (Layers 4 to 7, 9, 10) and the tracker use types the spine does not have): gt-table-pub, gt-bus, code-spine, docs-debt
- **R37** (The blueprint hands the holes’ specification to a tracker comment that contradicts it; its public-input section says “the prover sends nothing”): gt-table-pub, docs-debt
- **R38** (The status’s finding that the Python verifier omits the caps is false at the pin (F9)): gt-table-pub, gt-bus, gt-flock-ring, gt-opening-compile
- **R40** (The outputPure field is redundant, and guarded nearly so): code-spine, lib-arklib
- **R44** (Protocol/Basic.lean imports the arithmetization and is not a listed exception of the wall): code-spine, code-layer1, docs-debt
- **R47** (The zerocheck escape is over-charged and charged three or four different ways): gt-table-pub, gt-bus
- **R54** (The top limb is enforced only through the instance’s third line and the unbuilt adaptor theorem; acceptance test 10 names the wrong witness): code-pubinput, boundary-adaptor
- **R58** (The Flock error quoted in acceptance test 23 leaves out ring switching): gt-flock-ring, gt-opening-compile
- **R61** (The canonical-encoding and stream-consumption checks are absent from the blueprint, and Layer 12’s mutations exercise too little): gt-bus, gt-opening-compile
- **R79** (Acceptance test 14 names a tautology as its witness; no test compares the bytecode column with leanVM’s encoder): code-layer1, docs-debt

- **R83** (The status and the tracker describe a repository two merges old): code-pubinput, docs-debt
- **R84** (After the upgrade the blueprint states superseded pins as current, and Layer 0's prose is stale): lib-others, docs-debt
- **R91** (The list of public names is not the public boundary: most public declarations are unlisted, some load-bearing, three mislabelled): code-spine, code-layer1, docs-debt
- **R92** (Broken names and citations in the blueprint, status and tracker): gt-bus, docs-debt, boundary-adaptor, gt-opening-compile
- **R101** (The spine's slot does not pin the protocol: a phase that reads the whole oracle fills it): gt-table-pub, code-spine
- **R120** (The completeness base theorem composes leanISA's existence theorem, where `architecture.md` says witness generation): docs-debt, boundary-adaptor

References

The works the review consulted, as the literature dossier records them; bracketed labels in the text are the dossier's own tags for these works.

- [ABF26] Gal Arnon, Dan Boneh, and Giacomo Fenzi. Open problems in list decoding and correlated agreement. Cryptology ePrint Archive, Paper 2026/680, 2026. Version of 2026-07-06; Definitions A.3–A.5, Theorem 4.12, Table 1.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. In *Advances in Cryptology – EUROCRYPT 2025*, 2025. Full version: Cryptology ePrint Archive, Paper 2024/1586 (revised 2024-11-21); Definitions 4.5, 4.9, Conjecture 4.12, Theorem 5.2.
- [AGG⁺26] Jeremy Avigad, Anat Ganor, Lior Goldberg, David Levit, Ohad Nir, Yoav Seginer, and Alon Titelman. Formal verification of the S-two AIR, 2026. Sections 2–5.
- [AGL⁺21] Jeremy Avigad, Lior Goldberg, David Levit, Yoav Seginer, and Alon Titelman. A verified algebraic representation of Cairo program execution, 2021. CPP 2022 venue unverified; not opened.
- [AY25] Gal Arnon and Eylon Yogev. Towards a white-box secure Fiat-Shamir transformation. Cryptology ePrint Archive, Paper 2025/329, 2025. CRYPTO 2025 (major revision); abstract read only.
- [BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. Linear-time accumulation schemes. Cryptology ePrint Archive, Paper 2025/753, 2025. TCC 2025; not opened (unverified content).
- [BCGM25] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. All polynomial generators preserve distance with mutual correlated agreement. Cryptology ePrint Archive, Paper 2025/2051, 2025. CCC 2026; abstract read only.
- [BEG⁺94] Manuel Blum, William S. Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. *Algorithmica*, 12(2–3):225–244, 1994. Reference data from leanVM refs.bib; not opened.
- [BGK⁺23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat-Shamir security of FRI and related SNARKs. In *Advances in Cryptology – ASIACRYPT 2023*, 2023. Full version: Cryptology ePrint Archive, Paper 2023/1071 (revised 2024-03-05); Definition 3.13, Theorem 3.15.
- [BRW26] Benedikt Bünz, Ron D. Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026. Revised 2026-09-19 (after the leanVM pin); Section 5, Appendix C.3, Theorem 8, Lemma 9, Remark 11.
- [BSCH⁺25] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On proximity gaps for Reed–Solomon codes. Cryptology ePrint Archive, Paper 2025/2055, 2025. Single version, 2025-11-06; Theorems 1.3, 1.5, 1.6, 4.6, Corollaries 1.4, 1.7.
- [BSCI⁺20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS 2020)*, pages 900–909, 2020. Reference data from ePrint 2026/680; not opened.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography (TCC 2016-B)*, 2016. Full version: Cryptology ePrint Archive, Paper 2016/116; Theorems 1.2 and 1.5.

- [CCH⁺18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. Fiat-shamir from simpler assumptions. Cryptology ePrint Archive, Paper 2018/1004, 2018. Definition 5.3, Proposition 5.4, Theorem 5.8.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-shamir: From practice to theory. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*, pages 1082–1090, 2019. Publisher PDF not opened (HTTP 403); content read in ePrint 2018/1004.
- [CDHZ25] Alessandro Chiesa, Zijing Di, Zihan Hu, and Yuxi Zheng. How to prove post-quantum security for succinct non-interactive reductions. Cryptology ePrint Archive, Paper 2025/2166, 2025. EUROCRYPT 2026 (major revision); abstract read only.
- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In *Theory of Cryptography (TCC 2019)*, 2019. Full version: Cryptology ePrint Archive, Paper 2019/834; Definitions 8.3–8.5, Theorem 8.6.
- [CO25] Alessandro Chiesa and Michele Orrù. A Fiat–Shamir transformation from duplex sponges. In *Theory of Cryptography (TCC 2025)*, volume 16268 of *LNCS*, pages 452–474, 2025. Full version: Cryptology ePrint Archive, Paper 2025/536 (PDF of 2026-03-27); Theorems 6.1, 6.2; LNCS data from arXiv:2607.23752, reference 73.
- [CS25] Elizabeth Crites and Alistair Stewart. On Reed–Solomon proximity gaps conjectures. Cryptology ePrint Archive, Paper 2025/2046, 2025. Abstract read only.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. Version compiled 2026-03-25; Definitions 31.1.2, 31.1.6; Theorems 25.2.1, 26.1.1, 31.2.1, 31.3.1; Section 28.3.2.
- [Del25] G. Dell’Imagine. Introducing Clean, a formal verification DSL for ZK circuits in Lean4. zkSecurity blog, <https://blog.zksecurity.xyz/posts/clean/>, 2025. Reference data from arXiv:2607.23752, reference 57; not opened.
- [DKT26] Quang Dao, Scott Duke Kominers, and Justin Thaler. Reed-Solomon codes beyond Johnson: Efficient decoding and smaller cryptographic proofs. Cryptology ePrint Archive, Paper 2026/2056, 2026. Abstract read only.
- [DMWG23] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. Weak Fiat-Shamir attacks on modern proof systems. In *IEEE Symposium on Security and Privacy (S&P) 2023*, 2023. Cryptology ePrint Archive, Paper 2023/691; read by a helper of this task, not re-read.
- [DP26] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. In *Advances in Cryptology – EUROCRYPT 2026*, 2026. Cryptology ePrint Archive, Paper 2024/504 (last revised 2026-05-14); abstract read only.
- [DT24] Quang Dao and Justin Thaler. Constraint-packing and the sum-check protocol over binary tower fields. Cryptology ePrint Archive, Paper 2024/1038, 2024. Abstract read only.
- [eth26a] eth-act. zisk-fv: Lean 4 formal verification of RISC-V compliance of ZisK. <https://github.com/eth-act/zisk-fv>, 2026. Pushed 2026-09-23.
- [Eth26b] Ethereum Foundation. The proximity prize. <https://proximityprize.org/>, 2026. Read 2026-09-29.
- [Eth26c] Ethereum Foundation. soundcalc: a universal soundness calculator across hash-based zkEVMs and security regimes. <https://github.com/ethereum/soundcalc>, 2026. README read 2026-09-30 (repository pushed 2026-09-21).
- [Fen26] Giacomo Fenzi. How to prove more false statements: Fiat–Shamir limitations on (generated) R1CS. Cryptology ePrint Archive, Paper 2026/1838, 2026. Version of 2026-09-09; Definition 2, Theorem 2.
- [GG25] Rohan Goyal and Venkatesan Guruswami. Optimal proximity gaps for subspace-design codes and (random) Reed-Solomon codes. Cryptology ePrint Archive, Paper 2025/2054, 2025. Abstract read only.

- [GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. A simplified round-by-round soundness proof of FRI. Cryptology ePrint Archive, Paper 2025/1993, 2025. TCC 2026 (minor revision); abstract and changelog read only.
- [Gru24] Angus Gruen. Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108, 2024. Sections 3.1, 3.2.
- [Gun26] Cody Gunton. On formal verification and a bug in SP1 hypercube. Ethereum Foundation zkEVM blog, <https://zkevm.ethereum.foundation/blog/sp1-fv>, 2026. 2026-05-20; read by a helper of this task, not re-read.
- [Hab25] Ulrich Haböck. A note on mutual correlated agreement for Reed–Solomon codes. Cryptology ePrint Archive, Paper 2025/2110, 2025. Version of 2025-11-17; Theorem 2.
- [HJR⁺25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. Jagged polynomial commitments (or: How to stack multilinear). Cryptology ePrint Archive, Paper 2025/917, 2025. EUROCRYPT 2026 (minor revision); abstract read only.
- [Hol19] Justin Holmgren. On round-by-round soundness and state restoration attacks. Cryptology ePrint Archive, Paper 2019/1261, 2019.
- [Irr] Irreducible. Multi-multiset matching (M3). Binius documentation, <https://www.binius.xyz/basics/binius-v0/m3/definition>. Undated web page, read 2026-09-30.
- [Jo26] Sunghyeon Jo. Reed–Solomon mutual correlated agreement beyond the Johnson radius. Cryptology ePrint Archive, Paper 2026/1432, 2026. Abstract read only.
- [Kad25] George Kadianakis. Shipping an L1 zkEVM #2: The security foundations. Ethereum Foundation blog, <https://blog.ethereum.org/2025/12/18/zkevm-security-foundations>, 2025. 2025-12-18; read via a fetch summary.
- [Kam26] Antonio Kambiré. Proximity gaps conjecture fails near capacity over prime fields. arXiv:2604.09724, 2026. Reference data from ePrint 2026/680 only; unverified.
- [KDT25] C. Kwan, Q. Dao, and J. Thaler. Verifying Jolt zkVM lookup semantics. In *Financial Cryptography and Data Security (FC 2025)*, volume 15751 of *LNCS*, 2025. Reference data from arXiv:2607.23752, reference 24; not opened.
- [KKH26] Dmitry Krachun, Stepan Kazanin, and Ulrich Haböck. Failure of proximity gaps close to capacity. Cryptology ePrint Archive, Paper 2026/782, 2026. Abstract read only.
- [Kob26] Nadim Kobeissi. Verification theatre: False assurance in formally verified cryptographic libraries. Cryptology ePrint Archive, Paper 2026/192, 2026. Abstract read only.
- [KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to prove false statements: Practical attacks on Fiat-Shamir. In *Advances in Cryptology – CRYPTO 2025*, 2025. Full version: Cryptology ePrint Archive, Paper 2025/118 (revised 2025-12-11); Theorems 1, 2, Remark 3, Section 5; venue as cited by ePrint 2026/1838.
- [KSHC26] Arman Kolozyan, Tom Sorger, Alexander Hicks, and Stefanos Chaliasos. ZKP security tools and verification: Coverage, effectiveness, adoption, and challenges, 2026. Version 1, 2026-07-26; Section VI.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to Reed-Solomon erasure codes. In *Proceedings of the 55th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2014)*, pages 316–325, 2014. arXiv:1404.3458; bibliographic data from dblp and ACM listings; not opened.
- [Mar26] Diego Marmosoler. Isabelle/STARK: A formalization of zk-STARK in Isabelle/HOL, 2026. Version of 2026-09-22; abstract read only.
- [NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. Cryptology ePrint Archive, Paper 2025/1187, 2025. Abstract read only.

- [Net25] Nethermind. We verified the verifier: a first for zero-knowledge proof systems. <https://www.nethermind.io/blog/we-verified-the-verifier-a-first-for-zero-knowledge-proof-systems>, 2025. 2025-09-02; read via a fetch summary.
- [Net26a] Nethermind. Formal verification of the OpenVM RISC-V zkVM — technical report. Technical report, Nethermind, 2026. Dated 2026-02-01; REPORT.pdf in <https://github.com/openvm-org/openvm-fv>.
- [Net26b] Nethermind. pico-fv: Formal verification of the Brevis Pico zkVM. <https://github.com/NethermindEth/pico-fv>, 2026. Pushed 2026-06-11.
- [PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023. Abstract read only.
- [Riv26] Matthieu Rivain. AES-based grinding for MPC-in-the-head signatures. Cryptology ePrint Archive, Paper 2026/1625, 2026. Abstract read only.
- [SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Paper 2020/1275, 2020. Abstract read only.
- [STW23] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216, 2023. EUROCRYPT 2024; abstract read only.
- [Suc26] Succinct Labs. sp1-lean (SP1Clean). <https://github.com/succinctlabs/sp1-lean>, 2026. Branch dtumad/v1.0-release at eb6f44b9 (2026-08-12).
- [TDW⁺26] Devon Tuma, Quang Dao, James Waters, Alexander Hicks, and Nicholas Hopper. VCVio: Verified cryptography in Lean via oracle effects and handlers. Cryptology ePrint Archive, Paper 2026/899, 2026. Abstract read only.
- [Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology – CRYPTO 2013*, 2013. Cited by ePrint 2025/118, Remark 3; not opened.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*, volume 4 of *Foundations and Trends in Privacy and Security*. 2022. Author PDF of 2023-07-18; Proposition 4.1, Sections 4.6, 5.2, 6.6.2.
- [Vera] Verified-zkEVM. ArkLib: Formally verified arguments of knowledge in Lean. <https://github.com/Verified-zkEVM/ArkLib>. Revisions dca90385 and 7653a901.
- [Verb] Verified-zkEVM. Clean: Lean circuit DSL. <https://github.com/Verified-zkEVM/clean>. Revision 42fe4b26.
- [Verc] Verified-zkEVM. CompPoly. <https://github.com/Verified-zkEVM/CompPoly>. Revision 572f9973.
- [Ver25] Veridise. RISC Zero’s zkVM security: How Veridise enabled provable & continuous ZK security. <https://veridise.com/blog/audit-insights/risc-zeros-zk-vm-security-how-veridise-enabled-risc-zero-to-achieve-provable-continuous-zk-sec> 2025. 2025-03-27; read via a fetch summary.
- [Ver26] Verified-zkEVM. riscv-zkvm: Lean extraction of the Sail RISC-V specification for verified zkVM projects. <https://github.com/Verified-zkEVM/riscv-zkvm>, 2026.